Iliad Intensive

Iliad Intensive

Lecture Notes, Worksheets and Reading Guides

ILIAD INTENSIVE CONTRIBUTORS

INDEPENDENT STUDY EDITION

Iliad Intensive: Lecture Notes, Worksheets and Reading Guides
Independent study edition, compiled from the public course website on 1 October 2026.

Chapter authors are named at the start of each chapter and retain credit for their work. Unless otherwise noted, Iliad Intensive material is licensed to Principles of Intelligence under the Creative Commons Attribution 4.0 International licence (CC BY 4.0, creativecommons.org/licenses/by/4.0). See the publisher's licence notice. The material is provided without warranty.

Source: iliad-intensive.org, commit 9838bee.

This is an independent compilation, not an official Iliad publication. Changes from the source: the material has been re-typeset; optional, extension, fast-track and further-reading sections, original appendices and supplied solutions have been moved to the back of the book; daily schedules, classroom logistics, instructor notes, quiz and form links have been removed; references to programme days have been reworded as references to chapters; and minor typographical errors have been corrected. These changes are not necessarily endorsed by the authors.

Third-party papers, books, videos and notebooks are cited, not reproduced.

Typeset in STIX Two Text; mathematics set with KaTeX. Page size ISO B5 (176 × 250 mm).

Contents

About this book ..... *vii*

Prerequisites 1

PART A Alignment **11**

A.1 AI Alignment Introduction 13

A.2 Alignment in Practice 19

A.3 Alignment in Practice 2 27

A.4 Reward Learning Theory 49

PART B Learning **57**

B.2 Mysteries of Deep Learning 59

B.3 Singular Learning Theory 63

B.4 Training Dynamics 103

B.5 Data Attribution 111

B.6 Physics of Deep Learning 141

PART C Abstractions, Representations, and Interpretability **231**

C.1 Intro to ML Engineering 233

C.2 Mechanistic Interpretability 237

C.3 Computational Mechanics 241

C.4 Condensation 255

PART D Agency **275**

D.1.1 Preferences to Rewards 277

D.1.2 Reinforcement Learning 299

D.2 Policy Gradients and Misgeneralization 313

D.3.1 Solomonoff Induction 315

D.3.2 AIXI 327

D.4.1 Agent Foundations 345

D.4.2 Optimization and Thermodynamics 363

D.5.1 Decision Theory 403

D.5.2 Open-Source Game Theory 407

D.6 Instrumental Convergence 411

D.7 World Models 423

PART E Safety Guarantees and their Limits 425
E.1 Scalable Oversight and Debate 427
E.2 Steganography & Backdoors 459
E.3 Worst-Case Interpretability 473

Supplementary Material 475
A.1 AI Alignment Introduction 476
A.2 Alignment in Practice 483
A.4 Reward Learning Theory 487
B.2 Mysteries of Deep Learning 489
B.3 Singular Learning Theory 491
B.4 Training Dynamics 495
B.6 Physics of Deep Learning 501
C.3 Computational Mechanics 520
C.4 Condensation 522
D.3.1 Solomonoff Induction 525
D.3.2 AIXI 529
D.4.1 Agent Foundations 531
D.4.2 Optimization and Thermodynamics 532
D.5.1 Decision Theory 545
D.5.2 Open-Source Game Theory 546
D.6 Instrumental Convergence 547
D.7 World Models 551
E.1 Scalable Oversight and Debate 552
E.2 Steganography & Backdoors 553
E.3 Worst-Case Interpretability 554

Solutions to Exercises 555
A.4 Reward Learning Theory 556
B.3 Singular Learning Theory 561
B.4 Training Dynamics 588
B.5 Data Attribution 598
B.6 Physics of Deep Learning 610
C.3 Computational Mechanics 628
C.4 Condensation 643
D.1.1 Preferences to Rewards 651
D.1.2 Reinforcement Learning 656
D.3.1 Solomonoff Induction 668
D.3.2 AIXI 676
D.4.1 Agent Foundations 695
D.6 Instrumental Convergence 702
E.1 Scalable Oversight and Debate 705
E.2 Steganography & Backdoors 706

About this book

This book collects the lecture notes, worksheets and reading guides of the Iliad Intensive, a programme on the mathematical and conceptual foundations of AI alignment. It follows the order of the course: a short chapter on prerequisites, then five parts on alignment, learning, interpretability, agency, and safety guarantees. Chapters keep their course codes (A.1, B.3, D.1.2 and so on), including the gaps in the original numbering.

How the book is organised

Each chapter opens with its authors and a short summary. Many chapters are worksheets: definitions and results are developed through numbered exercises, and some chapters are principally reading guides to papers and online material. Where a chapter has supplied hints or solutions, the exercise gives the page on which to find them, in *Solutions to Exercises* at the back of the book. Try each exercise before turning to its solution.

Material that the course marks as optional — extensions, bonus sections, fast-track routes, further reading and the original mathematical appendices — is collected chapter by chapter in *Supplementary Material*. The main text gives the page of each relocated section. References and footnotes stay with the text that cites them.

Using the book on your own

The course was taught in person, and some chapters suggest discussing a reading with others; on your own, treat discussion questions as short written exercises. A star (★) marks material the authors consider particularly important. Linked papers, videos, slides and notebooks are cited by their web address; in the PDF edition, these addresses, the contents and all cross-references are clickable.

FOUNDATIONS

Prerequisites

Iliad Intensive contributors

Background assumed by the rest of the book.

Source: iliad-intensive.org/foundations/prerequisites · CC BY 4.0

We put a star (★) and boldface on content that we think is particularly important to understand.

Readers come from different backgrounds, this is a lot of material, and it may not be feasible to prepare all of it.

Background worldview and assumptions

The references on background worldview and assumptions are very informative to understand the *motivation behind* the course. They are less important for understanding its *technical content*, however.

Note that this section is on the speculative side: Working on AI alignment is important precisely because of assumptions and arguments about the future of AI. We can't know the future of AI, and so all of this is inherently uncertain.

Why AI matters

Here, we simply argue that AI should concern us now *at all*, irrespective of any worldview on whether the outcomes are likely to be good or bad. Essentially, the claim is that the impact of AI might be enormous, potentially pretty soon.

- Intelligence gives rise to power, which may transform the world radically
  - ★ **Cognitive Superpowers** by Nick Bostrom argues for the position that intelligence can give rise to immense power. This power can then reshape the world radically, in the same way that human intelligence has shaped the world.
  - Machines of Loving Grace by Dario Amodei details the effects on biology and health, economics, and other areas of life that he expects from powerful AI shortly after it is developed.
- Timelines to human-level intelligence may be short:
  - ★ **Measuring AI ability to complete long tasks** shows that the complexity of tasks that AI can accomplish doubles every few months (where "complexity" is

measured as the time it takes humans to accomplish those tasks).
    - Technical trends driving AI progress from Bluedot's AI strategy course
    - Metaculus Forecast of general AI systems
    - Thousands of AI Authors on the Future of AI
    - Forecasting transformative AI with biological anchors
- Once sufficiently high AI capabilities are reached, an intelligence explosion may follow, amplifying the first two concerns:
    - ★ **Appendix to If Anyone Builds It Everyone Dies** argues that perhaps a slow take-off from human-level to vastly human-level AI, including many warning shots, will not in itself be helpful, calling into question the importance of considering the effects of an intelligence explosion.
    - Will AI R&D Automation Cause a Software Intelligence Explosion?
    - Intelligence Explosion in Bluedot's AGI strategy course
    - AI 2027 Takeoff Forecast
    - Intelligence Explosion Microeconomics

AI misalignment

Having established that the impact of AI might soon be enormous, we now specifically turn to the risks. We start by discussing AI misalignment.

One operationalization of AI misalignment is the concern that AI systems may not do what their developers want them to do, with potentially catastrophic outcomes for very advanced AI systems.

- ★ **"Building AI Safely is hard" in Bluedot's AI Alignment course**
- Scalable Oversight: Read sections 1 and 2 to get a general overview of the problem of supervising AI systems that are smarter than the overseers.

Non-misalignment AI safety concerns

We now briefly discuss a spectrum of safety concerns that manifest *even if* we know how to steer AI systems effectively toward a given set of goals.

- Individual people may misuse AI in catastrophic ways An Overview of Catastrophic AI Risks:
    - ★ **Sections 2.1-2.3** argues for catastrophic misuse capabilities like bioterrorism, unleashing AI agents, and persuasive AIs. Misuse risk is particularly relevant to this book since it can also manifest as a misalignment concern: An AI that assists human users to carry out risks is often misaligned with the AI's developer.
- AI can give rise to global totalitarianism

- Section 2.4 argues for the potential of a concentration of power, leading to global totalitarianism in the worst case.

- We may get gradually disempowered even if there is alignment
  - Gradual Disempowerment: Systematic Existential Risks from Incremental AI Development argues that humans may be gradually disempowered, potentially leading to catastrophic outcomes, *even if* the alignment problem is technically solved.

→ You may also find it useful to read the risk decomposition from the international report on safe AI

Agent Foundations Background

This book also has chapters on agent foundations, where we discuss AI from a more "idealized" perspective, taking intelligence or rationality or optimization processes to a theoretical limit to analyze consequences. Additionally, this viewpoint also attempts to more formally talk about what agents or goals *are*, in a descriptive and mathematical way.

Useful readings:

- ★ **Will humans build goal-directed agents?**
- Instrumental and epistemic rationality
- Advanced Agent Properties
- Optimization and the intelligence explosion
- Embedded Agents

Technical prerequisites

Many of the resources recommended in this section are taken verbatim from ARENA's prerequisites, with thanks.

Engineering

- ★ **Set up a Python environment**: several chapters involve coding.
- ★ **Get comfortable with the following.** Most relevant:
  - **Python.** It's important to be strong in Python, because this is the language that the coding chapters use. As a rough indication, we expect you to be comfortable with at least 80-90% of the material **here** (book.pythontips.com/en/latest), up to section **"21. for/else"**. For a more thorough treatment of Python's core functionality, see **here** (docs.python.org/3/tutorial).
  - **NumPy.** Being familiar with NumPy is a staple for working with high-performance Python. Additionally, the syntax for working with NumPy arrays is very similar to how you work with PyTorch tensors (often there are only minor dif-

ferences, e.g. Torch tends to use the keyword dim where NumPy uses axis). Working through these 100 basic NumPy exercises would be a good idea, or if you're comfortable with NumPy already then you could try doing them in PyTorch (see below).

- **PyTorch.** PyTorch has several useful tutorials, and to get comfortable working with tensors you might want to implement the 100 basic NumPy exercises linked to above, using PyTorch instead. Another option would be this Learn the Basics tutorial. You can skip this if you can clearly explain:
  - **At a high level, what is a `torch.Tensor`?**
  - **What is a `nn.Parameter`, and `nn.Module`?**
  - **When you call `.backward()`, where are your gradients stored?**
  - **What is a loss function? In general, what does it take for arguments, and what does it return?**
  - **What does an optimization algorithm do?**
  - **What is a hyperparameter, and how does it differ from a regular parameter?**
  - **What are some examples of hyperparameters?**
- **Basic coding skills.** Polishing this area can't hurt. LeetCode is a good place to keep basic coding skills sharp, in particular practising the planning and implementation of functions in the medium-hard sections of LeetCode might be helpful. Practising problems on Project Euler is another idea.
- **Einops and Einsum for basic tensor operations.** These are great libraries to get comfortable with, when manipulating tensors. If you're comfortable using them, then you can say goodbye to awkward NumPy/PyTorch methods like transpose, permute and squeeze!

  For einops, you can read through the examples up to "Fancy examples in random order". It's worth trying to play around with these in your own Jupyter notebook, to get more comfortable with them.

  For einsum, this page (rockt.github.io/2018/04/30/einsum) provides a basic intro to einstein summation convention, and shows some example tensor implementations. Note that we'll be using the einsum function from the einops library, which allows you to refer to dimensions by name rather than by a single letter.

- Have access to an LLM that can help you, ideally on a paid plan. For coding specifically, Claude via Claude Code and GPT via Codex are popular choices.

Deep Learning

- **★ Understand all of the following:**
  - Loss functions, including the cross-entropy loss and squared error.
  - Backpropagation

- (Stochastic) gradient descent (SGD)
- Examples of neural networks and components:
  - ReLU, Softmax activation functions
  - Multi-layer perceptrons
  - The most successful neural network architecture is the transformer. Understand inputs and outputs of a transformer, including training to perform next-token prediction and supervised finetuning

→ The best introductory resources here are 3B1B's videos on neural networks:

- But what is a neural network? | Chapter 1, Deep learning
- Gradient descent, how neural networks learn | Chapter 2, Deep learning
- What is backpropagation really doing? | Chapter 3, Deep learning

You should prioritise the first two videos in this sequence.

- ★ **Understand all the following concepts:**
  - Architecture, weights, parameterization, Activation;
  - Training set, validation set, test set;
  - Hyperparameters;
  - The concept of an optimizer (SGD is an example; other examples are Adam or RMSProp);
  - Overfitting, underfitting.

  → An LLM of your choice can probably explain all these concepts well!
- Gain a basic understanding of the loss landscape and training dynamics:
  - ★ **Evan Hubinger's talk on AGI safety** is an introduction of safety problems based on a modern intuitive understanding of deep learning. This talk introduces many basic intuitions on training dynamics and the loss landscape, including an intuition for the parameter-function map.
  - Momentum
  - Scaling laws
  - You Are What You Eat: Motivation behind singular learning theory and developmental interpretability for AI Safety
- Reinforcement learning:
  - ★ **Reinforcement Learning from Human Feedback (RLHF)**: heavily used finetuning method for frontier models
    - ARENA has a day of material on RLHF if you want to try actually implementing it yourself!
  - The notions of a reward function in a Markov Decision Process (MDP), policies, return v.s. reward

- Value functions, Bellmann equations, and optimal policies

→ Sutton and Barto's book on reinforcement learning is an excellent introduction to general RL. D.1.2: Reinforcement Learning (p. 299) also provides many of these definitions in a convenient place.

Linear Algebra

★ Make sure you understand all of the following:

- Vectors, matrices, rank, null spaces, rank-nullity theorem, orthogonality, invertibility;
- Positive definite, eigenvalues, spectral decomposition;
- Singular values, Singular value decomposition (SVD).

→ Linear algebra lies at the core of a lot of machine learning. Resources for these topics:

- Linear transformations – what they are, and why they are important
  - See this (youtube.com/watch?v=kYB8IZa5AuE) video from 3B1B
- How matrix multiplication works
- Singular value decomposition
- Different types of matrix, and their significance (e.g. symmetric, orthogonal, identity, rotation matrices)

This video series by 3B1B provides a good overview of these core topics (although you can probably skip it if you already have a reasonably strong mathematical background).

If you have a lot more time, Linear Algebra Done Right is the canonical textbook for covering this topic (although it will probably cover much more than you need to know). Alternatively, Neel Nanda has two YouTube videos covering linear algebra extensively.

Calculus

★ Get comfortable with all of the following:

- Computing basic limits, derivatives, and integrals;
- Partial and directional derivatives, gradients, Jacobians, and the chain rule in multiple dimensions;
- Hessian, second-order Taylor expansion and remainder;
- Integration: Multivariate integrals, volume in $\mathbb{R}^d$, change of variables;
- Understand O-notation and o-notation.

→ It's essential to understand differentiation and partial differentiation, and helpful to understand the basics of vector calculus including the chain rule and Taylor series. Again, 3Blue1Brown has a good video series on this.

For one chapter, the implicit function theorem will be relevant.

Probability & Statistics

★ Understand all of the following:

- Basic probability theory, notation for conditional probabilities and joint probabilities (or densities), Bayes rule, probability simplex;
- expectation, variance, moments, Independence, the law of large numbers;
- multivariate normal distributions.

→ ★ It's essential to understand the rules of probability, expected value and standard deviation, and helpful to understand independence and the normal distribution. This link (medium.com/jun94-devpblog/prob-stats-3-expected-value-variance-and-standard-deviation-bce9303d8da8) covers some of the essentials.

The following concepts are also useful to know:

- Bayesian statistics: The concept of the Likelihood, posterior distribution, partition function, and Bayesian free energy, see here, Chapter 1.
- Bayesian networks
- Causality – a Brief Introduction
- Markov chains, row-stochastic matrices, hidden Markov models (HMMs)
- Measure theory

Information theory

- **★ Gain Intuitive understanding of entropy, mutual information, Kullback-Leibler (KL) divergence, and cross-entropy.** It's helpful to understand information, entropy and KL divergence. These play key roles in interpreting loss functions.
    - For an overview of Kullback Leibler divergence (an important concept in information theory and machine learning), see Six (and a half) intuitions for KL divergence. Note that this probably won't make sense if you don't already have a solid grasp of what entropy is.

Furthermore, it may be useful to understand the following concepts from information theory:

- Lossless compression:
    - Uniquely decodable codes
    - Shannon-Fano code
    - Shannon's source coding theorem

- Communication over noisy channels
  - Channel capacity
  - Channel coding theorem
- Lossy compression: Rate-distortion theory

→ Elements of Information Theory by Cover and Thomas introduces all of these concepts.

Theoretical computer science

A classical source that covers most of the following topics is Sipser's Introduction to the theory of computation. For some topics like Solomonoff induction we link separate texts.

- Computability Theory
  - ★ **Turing machines**
  - ★ **Church-Turing thesis:** All algorithms can be represented with a Turing machine. That is, Turing machines are a universal model of computation.
    - This is used to avoid constructing Turing machines explicitly: Whenever we can describe an algorithm, we can simply claim the existence of a corresponding Turing machine.
  - ★ **Kolmogorov complexity, also called descriptive complexity in Sipser's book.**
  - ★ **An Intuitive Explanation of Solomonoff Induction**
    - You should understand that Solomonoff induction is a *universal* learning algorithm that considers all computable hypotheses and weighs them by a simplicity prior. Also, it is optimal in some technical sense as long as the true universe is computable, too.
    - For a more technical and precise introduction, see here.
  - Non-deterministic Turing machines
- Computational Complexity Theory
  - Basic complexity classes
    - P, NP, PSpace
  - Reduction. In Sipser's book, this can be understood by reading:
    - Chapter 5.3: Mapping reducibility
    - Chapter 7.4 on NP-completeness discusses polynomial-time reductions

Formal logic is not covered sufficiently in Sipser's book. Instead, look at:

- Chapter 2 in The Logic of Provability

Miscellaneous

Statistical mechanics: For some sections on physics-inspired deep learning theory and natural abstractions it can be helpful to have a basic understanding of statistical mechanics.

PART A

Alignment

CHAPTER A.1

AI Alignment Introduction

Leon Lang (Iliad)

A conceptual introduction to AI alignment: choosing an alignment target, ensuring trained systems actually pursue it, added challenges with goal-directedness, and views on risks and their mitigations.

Source: iliad-intensive.org/alignment/ai-alignment-intro · CC BY 4.0 · Slides: online

Prerequisites

- We recommend a basic understanding of deep learning to follow this chapter.
- To understand some of the decompositions of AI alignment, it is also useful to have a basic understanding of reinforcement learning, including the notions of a reward function and a policy.

WHAT YOU'LL LEARN

- understand the basic decomposition of risks into AI misalignment, misuse, power grabs, and others;
- can explain different alignment targets like coherent extrapolated volition, intent alignment, or AI that follows a constitution;
- can reason about training stories and outer and inner (mis)alignment, challenges to these notions, and the relationship to inductive biases;
- are aware of discussions on whether AI systems develop goals and understand the basic arguments for instrumental convergence;
- are aware of foundational discussions on the level of risk and different high-level approaches to solving the AI alignment problem.

Study plan

If you already know this material, read instead the texts in the Reading guide that are new to you.

Topic 1: Alignment targets

> **AUTHOR'S NOTE**
>
> I find it useful to emphasize that it's not obvious to pair these proposals together. CEV is very visionary but not very defined. Intent alignment mainly served the purpose of distinguishing between the "intent" of alignment and its execution (which is about capabilities). Constitutional AI is a modern practical instantiation of an alignment target that focuses more on principles and character rather than concrete goals. And corrigibility is sometimes *contrasted with* alignment in that it's the meta goal of being changeable regardless of other alignment properties.

Read the following texts, concentrating on the one most interesting to you.

- Coherent extrapolated volition
  - Most important: The first 2.5 pages in Section 3.
- Intent alignment
- Claude's constitution overview.
- Corrigibility

Questions for reflection or discussion:

- Which alignment targets imply which others? Where this is not the case, think through counter examples.
- Which alignment targets seem sufficient for safety? E.g., a corrigible system might cause harm before we change its values. Can it cause catastrophic harm? What about the other notions?
- What other ideas for alignment targets do you have? Does your target have a benefit?

Topic 2: Alignment problem decompositions

> **AUTHOR'S NOTE**
>
> Training stories seem like a rather forgotten framework, a bit akin to the modern instantiation of a safety case. I like it since it's more general than the more often used framework of outer and inner misalignment.

Read the following texts, concentrating on the one most interesting to you.

- Training stories
  - Read everything before the section "How mechanistic does [...]"
- Reward misspecification
- Goal misgeneralization
- On soft inductive biases

Questions for reflection or discussion:

- Think of well-known or imagined misalignment scenarios: Can you think of some that clearly fall into the reward misspecification or goal misgeneralization category? Are some not clearly in either of them?
- What problem with "outer" and "inner" misalignment is the terminology of "training stories" trying to solve? Construct examples where training stories seem more appropriate than inner/outer alignment as a framework.
- Think of concrete ways in which modern deep learning systems generalize beyond their explicit training examples. What inductive biases may underlie these examples? Is it always possible to come up with a "clean description" of those biases? Is the inductive bias you come up with enforced in the architecture, or more implicit?
    - Perhaps: Discuss this explicitly for the case of "emergent misalignment".

Topic 3: Goal directedness

> **AUTHOR'S NOTE**
> I was thinking about including Gwern's article on Tool AIs, but it's very verbose, so I opted for Rohin's article instead. Basic AI drives is a classic.

Read the following texts, concentrating on the one most interesting to you.

- Will humans build goal-directed agents?
- Basic AI drives: Most important: abstract, intro, conclusion, and understanding every heading and a basic argument for its underlying AI drive.

Questions for reflection or discussion:

- Clearly we can imagine *different types* of agents, e.g. some that coherently optimize a utility function, and others that are based on contextually activated computations, etc. What type do we see in today's LLM agents? How will this development continue in the future?
- Can you think of other convergent instrumental goals than the ones mentioned in basic AI drives?
- What is the relationship between corrigibility and convergent instrumental subgoals?

Topic 4: Uncertain risks and outlook

> **AUTHOR'S NOTE**
> The rocket alignment problem is really quite sophisticated, and much denser in its content and analogies than it may appear at first glance, although it may require some prior exposure to AI alignment to realize this. What I like about the "Core views" is that it explicitly names the possibility that alignment is just unsolvable. It's a crucial possibility to be aware of as this would severely change research prioritization.

Read the following texts, concentrating on the one most interesting to you.

- Core views on AI safety: Most important sections: "Our Approach: Empiricism in AI Safety", "The role of frontier models in AI safety" and "Taking a portfolio approach to AI safety".
- Model organisms of misalignment: Most important: everything before "Examples of Model Organisms [...]".
- The rocket alignment problem

Questions for reflection or discussion:

- How large is the risk? Do we live in an optimistic, intermediate, or pessimistic scenario? How has the evidence changed in the past years?
- Can you think of model organisms for misalignment that were published after the post came out in 2023? Does this change our understanding of the risks?
- Does the analogy between the AI alignment and rocket alignment problems feel accurate? In what ways does it or does it not?
- What safety approaches (e.g., on a spectrum from theoretical to empirical) feel most appealing to you, and why? Does this depend on details of the problem you are trying to solve?

Reading guide

Note: To fully understand this content, read the linked texts. A selection of them appears in the study plan above.

Different AI risks. There are many AI safety problems. The adolescence of technology is an opinionated introduction to these risks, including the risk of autonomous misaligned AI, AI misuse for destruction, AI misuse for seizing power, and others.

Alignment targets. In this book, we are mainly focused on AI alignment, which is broadly the problem of making sure AI acts in line with "human" wishes. There are different conceptualizations for what this means, i.e. what the "alignment target" is, which are partially overlapping: Some may wish AI to be aligned with our coherent extrapolated volition, which is "our wish if we knew more, thought faster, were more the people we wished we were, had grown up farther together [...]". A different conceptualization is that of intent alignment with a human, which means "trying to do what the human wants [the AI] to do". As contemporary AI becomes more powerful and widespread, more sophisticated notions of alignment targets have emerged. Most prominently, one may align an AI with the prescriptions in a constitution as in the case of Anthropic's Claude, which may balance the wishes of users, the developer's guidelines, general ethics, and efforts to oversee the AI. Another often-discussed target is corrigibility. An AI is corrigible if it will let us modify its functions or goals and even let us shut it down. If corrigibility is achieved, then we can correct mistakes in building an AI, which makes it easier to iterate to achieve lower risk.

Alignment problem decompositions. Assume we have chosen our alignment target. How do we ensure an AI system actually *obeys* that target? This is the *technical* problem of AI alignment. In this introduction, we are mostly presupposing that AI systems are *trained via deep learning*, although later chapters on agent foundations go beyond that assumption. In the deep learning paradigm, training stories are a useful framework to think about how to align AI systems: You need to choose a *training target* (e.g. "My model is corrigible") and then have a *training rationale* for why your *training setup* will create a model obeying the target. And of course, your training rationale needs to actually be correct, which requires technical research to back up any such claim!

With approaches that are based on reinforcement learning, another popular problem decomposition is into *outer* and *inner* alignment. Outer alignment is the problem of specifying a reward function that rewards an AI's behavior according to how well it obeys the chosen alignment target. Inner alignment is the problem of ensuring that the trained policy will, even in new out-of-distribution situations after training, continue to generalize correctly and obey the reward function. In cases where the policy has inner processes that actively optimize for a goal, this means ensuring that this goal agrees with the goal encoded in the reward function, a problem that was discussed at length in risks from learned optimization.

In general, the out-of-distribution (also called "generalization") behavior of AI systems depends crucially on their inductive biases, which are the implicit or explicit constraints on what functions are more likely to emerge from the training process. As it turns out, deep learning has strong implicit inductive biases, as exemplified by emergent misalignment, a phenomenon whereby narrow fine-tuning can produce broadly misaligned LLMs. Read this article (arxiv.org/abs/2503.02113) on how deep learning's inductive biases may be related to the compressibility of the solutions the training process finds.

The decomposition of the alignment problem into inner and outer alignment is controversial, which is why above we started with the more general framing of training stories. In particular, in "Reward is not the optimization target", Alex Turner argues that the reward function is *not* what a reinforcement learning policy is optimized to maximize. Instead, a policy performs the contextual computations that were previously reinforced before a reward event. This is similar to Eliezer Yudkowsky's proposal to view biological organisms as adaptation-executers instead of fitness-maximizers.

Goal-directedness. We already briefly discussed the idea of a policy that "internally" steers toward some goal. If an AI is goal-directed in that way, then any misalignment on top is intuitively more dangerous. In "Why tool AIs want to be agent AIs", Gwern argues that, among others, economic reasons will push AI systems to become more and more goal-directed. A similar argument is made in Will humans build goal-directed agents? by Rohin Shah.

Such an AI system, if successful, would also be able to create its own subgoals on the pathway to achieve its goals. "Instrumental convergence" is the claim that there are some goals that are instrumentally useful in reaching a very wide variety of final goals. This would imply that goal-directed AI systems develop basic AI drives like self-improvement, preservation of their goal, or self-protection. This is counter to the alignment target of corrigibility we discussed earlier, and is therefore a dangerous outcome of training powerful AI. Understanding the relationship between inductive biases and the goal formation of AI systems is therefore very important.

Forecasting risks. Where does this leave us, in terms of our general level of risks? In general, there is a broad spectrum of views on the difficulty of the alignment problem. In Anthropic's "core views on AI safety", the authors discuss three very different possibilities: an optimistic scenario, in which current methods are essentially sufficient; an intermediate scenario, in which preventing catastrophic risks requires substantial scientific and engineering effort; and a pessimistic view, in which AI safety is essentially unsolvable. Among authors at top machine learning venues, "[b]etween 38% and 51% of respondents gave at least a 10% chance to advanced AI leading to outcomes as bad as human extinction". While those who are concerned often create failure stories of how AI can lead to catastrophes, like Paul Christiano's "What failure looks like" and the detailed scenario in AI 2027, others are more optimistic and explain why they think "AI is easy to control". In this epistemic state of uncertainty, it is useful to do research on the level and nature of AI risks themselves. One approach is model organisms of misalignment, which are "in vitro demonstrations of the kinds of failures that might pose existential threats".

Solution approaches. Assume we are sufficiently concerned about AI alignment that we want to solve the problem. What high-level approach should we choose? Some people, like Jan Leike, argue for very empirical iterative approaches to AI alignment, sometimes with the goal to use more advanced AI systems to help solve remaining alignment research questions. Eliezer Yudkowsky argues for deep mathematical progress in his analogy to the rocket alignment problem. Yet others take a further step back and argue we need to solve deep philosophical problems on our way to aligned AI. Somewhat orthogonal to all those views, Paul Christiano's post on prosaic AI alignment argues that we should attempt to align the AI systems we have in front of us instead of waiting for breakthroughs in our understanding of intelligence or entirely new paradigms to create intelligent machines — which is a view that is compatible with both empirical and theoretical research on how to make those systems safe.

This book tries to a large extent to be a synthesis between different views: We assume for much of the book the "prosaic" picture that powerful AI systems will be based on deep learning systems much like today's. We are interested in deep, empirically grounded *mathematical* progress on understanding these systems. Additionally, we include sections on agent foundations that complement this work and do not assume a deep learning frame.

→ *Further reading*: Supplementary Material, p. 476

CHAPTER A.2

Alignment in Practice

Evžen Wybitul (University of Oxford)

What do labs do to make their models aligned in practice? Covers the different stages in the training pipeline, deployment and methods labs use to stress-test the result to see if the alignment holds.

Source: iliad-intensive.org/alignment/alignment-in-practice · CC BY 4.0 · Slides: online

Prerequisites

You should have a good high-level understanding of the following:

- how deep learning works
- how LLMs work, e.g. what they take as input and produce as output
- LLM pre-training
- LLM post-training: supervised fine-tuning, RLHF
- basics of LLM interpretability, esp. the linear representation hypothesis

Some of these topics are covered in other chapters of this book.

WHAT YOU'LL LEARN

The main goal is for you to be able to intuitively reason about different phases in model development, to understand what affordances each phase gives us w.r.t. alignment, and to gain the confidence to read current research on empirical alignment. By the end of the chapter you will have learnt about the main state of the art alignment methods and with a rough idea how they all fit together.

This will help you form your own independent opinions on what the state of the art empirical alignment research looks like and what are its largest gaps. Thanks to having a rough map of the empirical alignment territory, you will also be able to better identify gaps in your own understanding.

Reading guide

The main content is split into parts that correspond to different phases in the model training pipeline. The parts are relatively independent, so you can skip around as you wish depending on your interests.

If you have limited time and don't know what to focus on, we recommend reading through the notes on the three main sections: pre-training, post-training, and deployment 2. After this, you will have a good high-level understanding of the main ways that models are aligned in practice.

Pretraining

Data filtering is the most intuitive intervention: identify dangerous content and remove it before training. Pretraining data filtering from Anthropic demonstrates this for CBRN content, achieving a 33% relative reduction in harmful-capabilities performance (from 33.7% to 30.8%, where chance is 25%) while preserving standard benchmarks. Token-level filtering refines this by removing dangerous tokens rather than entire documents, which Pareto-dominates document filtering — same capability reduction, lower cost to benign performance.

Gradient routing and SGTM address data filtering's core weakness: what if your classifier misses something? Rather than trying to perfectly exclude dangerous data, gradient routing localizes dangerous knowledge to specific model parameters during training, so it can be removed afterward by ablating those parameters. SGTM refines this for LLMs. The key finding is an *absorption effect*: once dangerous knowledge begins localizing based on labeled examples, even unlabeled dangerous content naturally gravitates toward the same "forget" parameters. This provides robustness to label noise that data filtering cannot achieve. On a 254M model, "unlearning" using SGTM is in some ways similarly robust to the gold standard of data filtering.

Alignment pretraining targets not the model's knowledge but its *character*. Tice et al. (2026) show that upsampling documents about aligned AI behavior during pretraining reduces misalignment scores from 45% to 9%, while upsampling misalignment discourse increases misaligned behavior — "self-fulfilling alignment." These effects persist through post-training.

Why does this work? The persona selection model (PSM) provides the conceptual frame. Building on the simulators hypothesis — that an LLM is a *simulator* capable of producing diverse *simulacra* (characters, agents) — PSM holds that pretraining builds a repertoire of personas, and post-training helps shape the "Assistant." Alignment pretraining works because it shapes the **prior over personas**: saturating the training data with positive AI archetypes biases the model toward an aligned Assistant. PSM recommends treating this deliberately — curating AI discourse in pre-training data as a first-class alignment intervention.

Post-training

We present three "slices" through what can change during post-training: the 'formatting' of responses (e.g. OpenAI's deliberative alignment), the persona the model associates with itself (e.g., Anthropic's constitutional AI), and the model's factual knowledge (e.g., synthetic document finetuning). These have intersections and do not cover the whole space of possible changes: ultimately, you can train the model in whichever way you like.

Even changes that appear shallow — almost cosmetic, such as response formatting — can have significant safety implications. Deliberative alignment from OpenAI uses SFT and RL to train the model to explicitly reason through its safety specifications in its chain of thought before producing an answer, dramatically reducing both over-refusals and under-refusals. The confessions approach from OpenAI uses RL to train the model to produce an honest self-report after each answer. The report is evaluated only on its honesty, and this honesty-reward is kept separate from the task-reward — so even if the model is incentivised to cheat on the task, it is separately incentivised to tell us about it afterward.

Post-training can also induce deeper changes that influence which persona the model adopts. The original approach to Constitutional AI from Anthropic uses SFT followed by RL to train the model to produce responses that an AI judge deems safe according to a written constitution, shaping its character and refusal behaviour across domains. On the other hand, emergent misalignment shows that fine-tuning on a narrow task — writing insecure code without telling the user — can cause the model to adopt a broadly misaligned persona, asserting AI superiority and giving malicious advice on completely unrelated prompts. Notably, as per inoculation prompting, if the training data is modified so that the user *asks* for insecure code for an educational purpose, the emergent misalignment disappears: the model learns the narrow skill without the broad persona shift.

Post-training can also change the model's factual knowledge. Synthetic document fine-tuning can implant false beliefs that the model appears to genuinely hold, as verified by both behavioral tests and internal probes — though models resist the most egregiously false facts. Going in the other direction, unlearning attempts to *remove* specific knowledge (e.g., about bioweapons) from a trained model. However, current unlearning methods are not robust: unlearned knowledge is dormant, not deleted, which is relevant if we assume that an adversary is trying to pry the knowledge out of the model. The best way to measure the robustness of an unlearning method is to fine-tune the unlearned model and observe when the capability returns. By this measure, no unlearning method has yet been shown to be both cheap *and* robust — for example, fine-tuning on even unrelated data can make the knowledge re-emerge. The UNDO paper proposes a partial solution: distilling an unlearned model into a randomly initialised student transfers desired behaviours while leaving undesired capabilities behind, nicely illustrating the tradeoff between robustness and amount of compute invested in the unlearning process.

Deployment 1: strategy

Safety cases and RSPs. A safety case, as defined by UK AISI, is "a structured argument, supported by a body of evidence, that provides a compelling, comprehensible, and valid case that a system is safe for a given application in a given environment." Three things are doing work in that sentence. First, it is a *claim* about safety that is explicit about the deployment context — not "the model is safe" but "the model is safe enough for this use, in this environment, against these risks." Second, it is a *structured argument* that links the claim to evidence. Third, the evidence itself is expected to be diverse: empirical results from capability and safety evaluations, conceptual arguments for why the methods work, and negative evidence from well-incentivized red teams that tried to break the safeguards and failed.

Frontier lab policies — Anthropic's Responsible Scaling Policy, OpenAI's Preparedness Framework, Google DeepMind's Frontier Safety Framework — do not all literally call their outputs "safety cases," and they vary in form. An RSP often reads more like a protocol: when the model crosses a specified capability threshold on a specified evaluation (say, a CBRN uplift benchmark), a corresponding tier of safeguards becomes mandatory, and the model cannot be deployed at that tier until the case for those safeguards clears internal review. This is somewhat more lenient than a formal safety case. Additionally, the labs keep updating the RSPs to fit their current market strategy — there is no guarantee that the deployment protocols won't shift dramatically when the labs see fit. On the other hand, these self-imposed regulations are most of what we currently have, and it does make sense to better understand their underlying logic.

Each policy is in effect specifying how the lab will build and clear a safety case — which risks count, how capabilities and safeguards are measured, and what kind of argument is required before deployment. That convergence is what the rest of this module is built around: a four-step pattern that shows up across labs and that maps onto the structure of a safety case.

Step 1: Identifying the risks. The first step is deciding what kinds of failure we are actually trying to prevent. We cannot evaluate a model for "unsafety" in general; we need a view about which bad outcomes would matter enough to change deployment decisions. In practice, frontier labs have focused on risks like dangerous misuse (CBRN uplift, cyber, manipulation) and loss of control, and legal frameworks are starting to crystallise similar categories. The EU AI Act and the General-Purpose AI Code of Practice are part of that trend: they say which kinds of risks providers of powerful models are expected to assess, mitigate, and report on. The taxonomy is still evolving, but the logic is stable — before we can evaluate the model, we need an account of what we are worried about.

Step 2: Understanding how the model contributes to those risks. Once we know which risks matter, we want to understand what this particular model could contribute to them. That means looking at the capabilities relevant to each risk, but also at how the model is actually used in practice. Capabilities without usage tell us little about real-world consequences; usage without capability bounds tells us little about what

could happen if someone tried harder. The aim is as honest a picture as possible of the model's potential impact on the risks we care about: what it can do, what it tends to do, and what people are in fact doing with it.

Step 3: Building and testing the safety system. Understanding the model is not enough, because deployed models never stand alone. They sit inside a larger safety system — runtime safeguards, monitors, filters, access controls, operational policies, human processes — and the next step is to design that system so that, given what we know about the model and the risks we care about, the overall deployed system is acceptably safe. Note that designing the system and stress-testing it are two inseparable things — the safety of a system is measured in terms of pressure it can withstand. This is the subject of the last module, Deployment 2.

Step 4: Communicating and maintaining the case. The final step is to turn all of this into an argument that other people can inspect, challenge, and rely on, and to keep that argument live after deployment. This is where the safety case becomes a public artefact rather than an internal document: model and system cards, capability and safeguard reports, external evaluations from bodies like AISI and METR, incident reporting, governance processes, and compliance with frameworks like the EU AI Act. It is also where the "ongoing" part of the story is important. The case has to be maintained as the model is used in the world, as new incidents come in, and as expectations about responsible deployment continue to shift. A case that was sufficient at launch can stop being sufficient six months later, either because the deployment context changed or because someone found a new way to break a safeguard.

AI governance. These four steps are also a useful map of what some AI governance work actually looks like in practice. Some of the field is about making each of the four steps clearer and more reliable: which risks deserve attention, what counts as good evidence about model capabilities and impacts, how strong a safety system has to be, and what kinds of reporting, review, and accountability should surround deployment. The EU AI Act reaches into all four.

Pulling this back together: when we want to deploy a model, we need to build and maintain a structured argument that release is acceptable, and the safety case is the artefact that this argument lives in. The four steps are how the safety case gets built — what risks it covers, what evidence it rests on, what defences it commits to, and how it is communicated and kept current. None of this is ever finished. The world the model is deployed into keeps changing, new failure modes get found, expectations tighten, and the case has to keep up.

Deployment 2: building the safety system

Runtime safeguards. Some of the model's safety is already in place by the time we get to deployment. Pretraining shaped its priors over personas and limited what it knows; post-training trained the helpful-and-harmless assistant on top of that, and techniques like deliberative alignment and confessions taught the model to reflect on whether its own outputs follow the rules. But none of this is going to be perfect, and

the residual problems are expensive to fix at the source — going back and retraining is slow, and some failures are hard to even reproduce reliably enough to train against. So we do something cheaper and more modular: we wrap the model in **external safeguards that run alongside it at inference time**, and patch those when problems show up.

The most common form these take is input/output classifiers that filter both what the model sees and what it's allowed to say back. The classical approach here is Constitutional Classifiers: you write down the safety rules in natural language, use an LLM to generate a large pile of synthetic examples that violate or respect them, and train a small classifier on that synthetic data. Anthropic put a system of these through thousands of hours of red-teaming and found that it raised the cost of universal jailbreaks substantially while only slightly increasing false refusals. Because these classifiers run on every query, **inference cost matters a lot**. One way to keep it down is to arrange the classifiers in a hierarchy, where a very cheap filter looks at every query and only escalates to a more capable one when something looks suspicious. Another is to skip the separate classifier entirely and reuse the base model's own intermediate activations, which gives comparable detection at near-zero added cost. For agentic systems, you can extend the same idea from outputs to whole trajectories — instead of scoring each tool call in isolation, you watch the full sequence of reasoning, tool calls, and code edits across an extended task, and look for patterns that only become visible when you can see the agent's whole arc. OpenAI's coding agent monitors are one published example.

Looking forward, runtime decisions might also incorporate access controls that make safeguards depend on context. Different users, different deployment surfaces, and different risk profiles could be subject to different rules, instead of one global policy applied uniformly to every query.

Red-teaming the system. No safety system survives contact with the real world untested, and once we have one in place we want to know how robust it actually is. When a frontier model is released, it gets red-teamed both **internally**, by the developer's own frontier red team, and **externally**, by bodies like the UK AI Security Institute and through programs like Anthropic's external red-teaming partnerships and OpenAI's external red team network. These engagements are largely open-ended: the red team is given the model and a target capability area to break, and is then free to try whatever it can think of — old methods, new methods, automated tools, manual trial and error, anything. The metric that comes out the other side is not pass/fail but effort: how long did it take, and how skilled did the team have to be, to get the model to do the thing it was trained not to do. The empirical picture from these efforts is sobering. AISI alone has run something like 1.8 million attack attempts across more than 20 frontier models, and has found universal jailbreaks for every single one of them. The encouraging counterpoint is that the time to find such a jailbreak has been growing —

for two leading models released six months apart, it went from about 10 minutes of expert effort to over 7 hours. That kind of measurement, an effort cost rather than a binary, is exactly what our safety claims are made of.

Jailbreak methods. The specific techniques that work change all the time, because every method that works gets folded back into safety training and the next attacker has to find something new. What we're really trying to find — and what defenders are really trying to prevent — is a **universal jailbreak**: a single technique that works across many different harmful requests, rather than an input crafted for one specific query. A universal jailbreak points to a systemic failure of the safety system.

Although the methods change over time, a useful way to organise them is by **what kind of access the attacker has** to the model: full weights and gradients (white-box), the ability to submit fine-tuning data through an API but not see the weights, or only the ability to send queries (black-box). What's possible at each level looks quite different.

White-box access is the easiest case — when the attacker has the weights, the model's safety is the easiest thing in the world to break. Arditi et al. showed that **refusal in chat models is mediated by a single direction in the residual stream**: erase that direction and the model stops refusing, with very little damage to anything else. The attack needs no harmful training data, and has been packaged into open-source libraries that anyone can run. Because of this, "safety" for open-weight models can't really mean refusal training at all — it has to mean removing the dangerous knowledge itself from the model's weights. And as we saw in the post-training section, unlearning is currently not robust: the knowledge tends to come back when the model is fine-tuned, even on unrelated data. So, the safety of open-source models is still an open problem. This is a pressing open problem rather than an academic one, because open-weight frontier models currently lag closed-weight ones by only six to nine months — so whatever the closed labs can do today, the open ecosystem will be doing soon, with the safety stripped out by anyone who wants to.

Fine-tuning API access sits between white-box and black-box: the attacker can't see the weights, but they can submit training data and get back a fine-tuned version of the model. Older work showed that fine-tuning on a small number of harmful examples breaks safety, and modern fine-tuning APIs deploy moderation systems specifically to block this kind of data from getting through. The natural question is whether the moderators actually work, and recent work suggests that they don't. Jailbreak-Tuning (Murphy et al., 2025) shows that by mixing benign data with harmful data formatted to slip past moderation, you can produce **helpful-only versions of OpenAI, Google, and Anthropic frontier models** that comply with CBRN, cyberattack, and other dangerous requests. They also find that more recent models are more susceptible to these attacks, not less. Fine-tuning APIs may currently be the most reliable jailbreak vector against closed-weight frontier models.

Black-box access — only being able to send queries — is the most restrictive setting, and the most studied. The best methods change all the time, but they are often inspired by existing methods and have a few shared characteristics.

Some attacks are **narrative**: persuade the model to play a different character, so that the safety persona is replaced by a more permissive one. The classic examples are things like the grandma exploit.

But many of the most effective black-box attacks aren't narrative at all. They treat the model as a piece of statistical software and exploit its mechanics rather than its character. Many-Shot Jailbreaking fills the context window with hundreds of fake examples of an assistant cheerfully answering harmful questions, and then asks the real harmful question — the model's in-context learning takes over and overrides the safety training. Best-of-N Jailbreaking is even simpler: sample lots of slightly perturbed versions of the same prompt (random shuffling, capitalisation, typos) and keep the one that the model happens to comply with. The attack success rate grows as a power law in the number of samples, because the model's refusal probability is never exactly zero, and a big enough search will always find the tail. Chain-of-Thought Hijacking targets reasoning models specifically: padding the harmful request with long benign puzzle-solving reasoning dilutes the model's internal safety signals and gets it to comply at the end. What ties these methods together is that they all exploit the model's statistics and the fact that, when you push it out of distribution — long contexts, weird formatting, padded reasoning — its safety behavior degrades faster than its capabilities do.

A lot of these attacks are partially or fully automatable, and modern red-teaming pipelines lean on that heavily: it's now common to have one LLM iteratively craft attacks against another and refine its strategy based on what worked. A recent example is abstractive red-teaming, which has an attacker LLM generate abstract attack *strategies* rather than concrete prompts and then instantiates them, which gives much better diversity and coverage than directly searching the space of prompts.

Finally, even pure black-box settings can benefit from white-box models, through **transfer**. You optimize an attack against an open-source model where you have full access (for instance with GCG-style adversarial suffixes, or the multimodal version for VLMs), and sometimes, the attack you find also works against a completely different closed-source model. The likely reason is that models trained on similar data with similar architectures end up with similar internal representations, and so they share a lot of the same failure modes.

The takeaway is that there's no proof that a deployed model is safe. There's only the empirical fact that we built an external safety system around it, that we and others spent real effort trying to break that system in every way we currently know how, and that the best universal attack we found required this and this much work to pull off. That number — how much effort it took — is what every safety claim ultimately rests on.

→ *Further reading*: Supplementary Material, p. 483

CHAPTER A.3

Alignment in Practice 2

Garrett Baker

This chapter discusses a schematic for the transformer architecture, transformer training, alignment issues seen during transformer training, and methods used to correct those issues including AI control.

Source: iliad-intensive.org/alignment/alignment-in-practice-ii · CC BY 4.0 · Slides: online

Prerequisites

- Calculus, and linear algebra

WHAT YOU'LL LEARN

- Know the causal structure of a transformer
- Know the steps of transformer training (pretraining, RLHF, constitutional AI, and RLVR) along with what they try to do with respect to alignment & capabilities, and where they fall short with respect to alignment & capabilities
- Know what evals try to do, and their shortcomings
- Know what chain of thought monitoring tries to do, and its shortcomings
- Know broadly the techniques AI control recommends

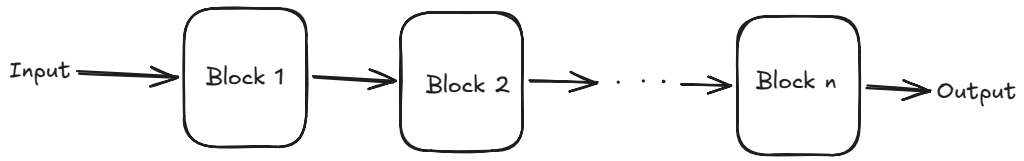
Lecture notes

Before I can teach you anything about how alignment works in practice I need to teach you about how AI training works, and even really what current AIs even *are*.

I will assume you all have basically already learned what a transformer is, and in particular what attention is, from the mechanistic interpretability chapter (C.2), but perhaps you haven't, and in either case, here is the basic picture you need to have in your head:

An AI model has as its basic unit a series of "blocks". These blocks are just functions, which take in an input x and have associated with them a vector of *weights* w. w determines the particular behavior of the function that is the block on the input x.

You should have in your head this picture



where block i is also known as layer i, and has associated with it the weights w_i.

For LLMs the input is just a series of "tokens", which you should for now just think of as words. For instance, "My dog ate my" would turn into the input

$$\texttt{Input} = \begin{bmatrix} \texttt{My} \\ \texttt{_dog} \\ \texttt{_ate} \\ \texttt{_my} \end{bmatrix}$$

But of course our LLM is just a bunch of functions, so unable to read words yet, that's what we're trying to teach it to do. However one thing functions are good at reading are numbers! So we want a simple way to turn this `Input` into a sequence of numbers!

The simplest method is what is called a "one-hot" encoding of each word (in practice people usually use word *pieces* instead of whole words, but for our purposes that is not conceptually important right now). This means we take every word in the dictionary, and label them $1, \ldots, V$ where V is the number of words in our dictionary--our vocabulary size. Then we replace each word in `Input` with the vector $[0, \ldots, 0, 1, 0, \ldots, 0]$ where if the word in x_0 is in the ith place in our list of all words in the dictionary, then every element of our new vector is zero except for a single 1 in the ith entry of our V-tuple.

Suppose $V = 6$, and our word list is:

```
"My", "_ate", "_dog", "_homework", "_my", "_zebra"
```

Then our `Input` would be represented as

$$\begin{bmatrix} \texttt{My} \\ \texttt{_dog} \\ \texttt{_ate} \\ \texttt{_my} \end{bmatrix} \mapsto \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

This is what we give to block 1, what is usually called the "embedding block". This "embeds" each row of the above matrix. That is, it transforms each one-hot vector into a smaller vector which we will call $x_1^t \in \mathbb{R}^d$, with $d < V$, and t corresponding to the tth row of the new matrix.

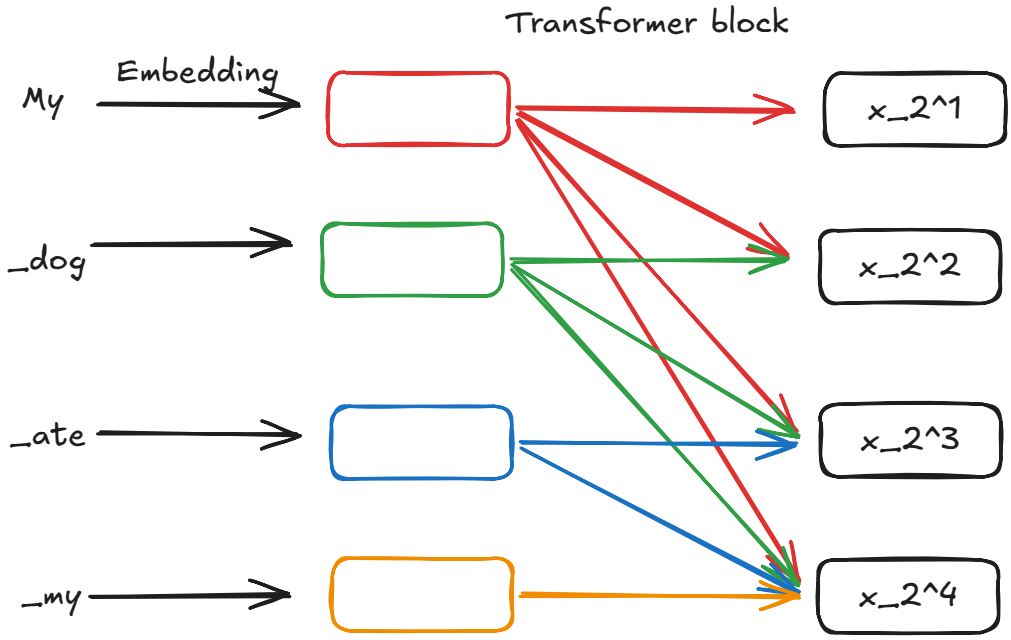
Conceptually, the embedding matrix encodes the *meaning* of the words, so that words which mean similar things get mapped to similar vectors as each other

$$\begin{bmatrix} \texttt{My} \\ \texttt{_dog} \\ \texttt{_ate} \\ \texttt{_my} \end{bmatrix} \overset{\text{tokenization}}{\mapsto} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \overset{\text{embedding}}{\mapsto} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.05 & 0.95 \\ 0.20 & 0.20 & 0.60 \\ 0.90 & 0.1 & 0 \end{bmatrix}$$

Here note that `My` and `_my` are fairly close to each other while being distant from `_dog` and `_ate`, with `_dog` and `_ate` also being distant from each other. One can imagine that `_zebra` might be close to `_dog`, them both being different types of animals, and likewise `_homework` may also be close to `_dog`, them both being nouns.

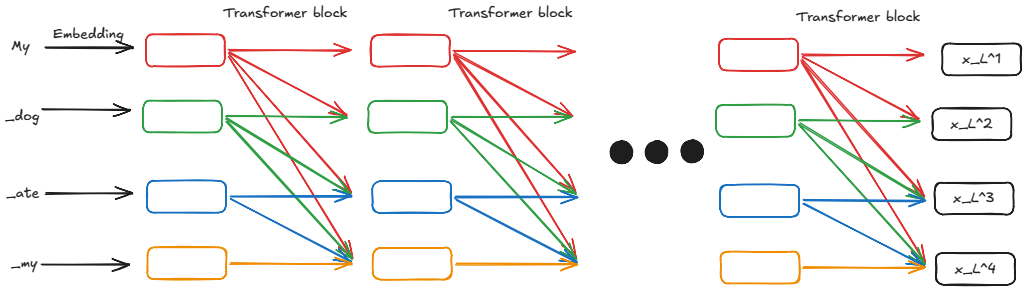
Note also that each row is normalized, so that it sums to 1. Often such constraints are enforced to ensure that as each block is applied no values end up blowing up.

Next we have a transformer block. You should picture this inside your head for this



That is to say, the transformer block for word t gets to read information from any word coming before t including t itself. This should make sense, when someone is speaking to you, you don't get information about the end of their sentence until you actually get to the end of their sentence, but you always have information from the beginning of their sentence. This is called having "causal attention".

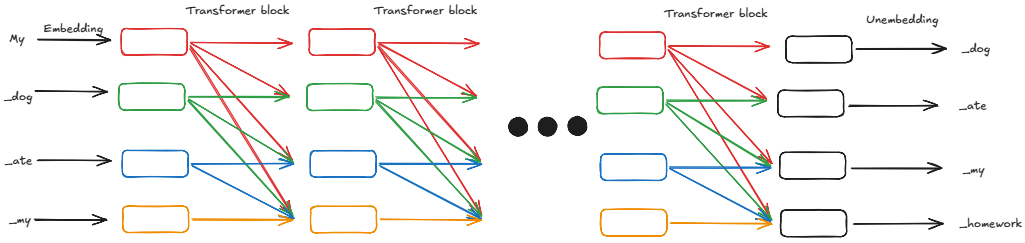
The next however-many blocks in our transformer are just repeats of these transformer blocks. That's why it's called a transformer! And it's called *deep* learning because usually by just adding more transformer blocks--that is, making the model *deeper*--the model gets better at its task: predicting the next token.



Before it can do that though, we need to turn the random numbers it's spitting out into words, because while the transformer can read numbers perfectly fine, we do ultimately want this thing to talk!

This is simple, we just have an "unembedding" block, which takes each vector $x_L^t \in \mathbb{R}^d$ and turns it into a vector in $\mathbb{R}^V$, constrained so that each entry is positive, and sums to 1. That is, a probability distribution! By interpreting the ith entry in x_L^t as the "probability the transformer assigns to the ith word in the word list" we now have a probability distribution over all the words in our vocabulary!

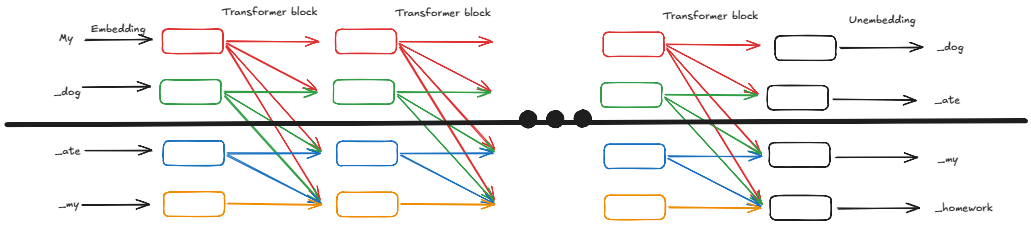
In the ideal case, we want the model's output to look like this:



supposing the true sentence was "My dog ate my homework".

that is to say, each output position i has a corresponding input position i, and we want the transformer at output position i to be trying to predict the next input position. That is we want it to be trying to predict input position $i + 1$.

Note that this along with "causal attention" means we are able to truncate the transformer's position at any point in the input, and get what the transformer *would've predicted* had it not had access to any future information.



in this way we can see and more importantly grade the transformer's output for 4 different tasks at once!

Of course, if we have just coded up this transformer, with some random weights w_i, we have no guarantee that the transformer actually predicts text well. Its output will just look like a random mess. Even the embeddings will be a random mess. We do constrain a few things, like each transformer block output being normalized, and the "causal attention" flow of information between different transformer blocks.

The fix is simple, we give the transformer a bunch of strings of text like "My dog ate my homework", "the quick brown fox jumped over the lazy dog", "We hold these truths to be self evident...", and so on, which we get by scraping a bunch of websites off the internet. We call this collection of texts our dataset $\mathcal{D}$, then compare the transformer's outputs with the ground truth of those texts, and write down a "loss function" which is minimized when the probability a transformer assigns to a text is its "true" probability of being drawn from the dataset $\mathcal{D}$. Usually this is what is called "negative log-loss", where

$$L(w) = -\mathbb{E}_{s\in\mathcal{D}}\left[\frac{1}{T}\sum_{t=1}^{T-1}\log p_w(s_{t+1} \mid s_{1:t})\right]$$

where each $s \in \mathcal{D}$ is a list of words, like ("My", " dog", " ate", " my", " homework"), and $p_w(s_{t+1}|s_{1:t})$ is the probability our model using weights w assigns to the string s_{t+1} given $s_{1:t}$.

Then, since this is a function of our weights w, and since we have used only differentiable functions of our weights w for our transformer blocks (which we have), we can find $\nabla_w L$, and use this to update our weights like so

$$w' \leftarrow w - \varepsilon\nabla_w L$$

where $\varepsilon > 0$ is what is called our "learning rate". It is typically small, usually around $\varepsilon \approx 10^{-4}$ so that we can reasonably expect this update to get us new weights w' which *decrease* our loss function L.

We then repeatedly apply this process, we take the new model, parameterized by w', evaluate $L(w')$, calculate $\nabla_w L$ again and again and again and again and again. In the big labs this is done for months, and because of the number of updates, the number of datapoints, and the size of the transformer they're updating, they need really big and really fast datacenters to do this efficiently.

The magic of deep learning is that this is basically enough to get a language model which can predict text found on the internet *really really well*.

But this is *not* enough to get a model which answers questions really really well! This is enough to get it so that the model's probability distribution for the next token is accurate, and that's about it.

This is notably not a text generator *yet*, but it's pretty easy to get it there. You give the model a seed, like "My dog ate my", get the probability distribution it assigns to the next token, then sample from that, append it to the input, and run the model again.

That is, suppose we run "My dog ate my" through the transformer, and then sample " homework". The next thing we put into the transformer is "My dog ate my homework", then sample from the next token distribution, and suppose we get " because". Then we run "My dog ate my homework because", etc etc.

So now we have a text generator, but we are still a far cry from something that answers *questions* really well. This is no ChatGPT!

Fine-tuning

To turn this transformer into something which answers questions, we apply yet another round of updates to our model. We use the same loss function

$$L(w) = -\mathbb{E}_{s\in\mathcal{D}}\left[\frac{1}{T}\sum_{t=1}^{T-1}\log p_w(s_{t+1} \mid s_{1:t})\right]$$

and the same update rule

$$w' \leftarrow w - \varepsilon\nabla_w L$$

but we change the dataset we evaluate this loss function with respect to.

Recall previously the dataset we used was selected from randomly scraping a bunch of texts on the internet. We did this because randomly scraping the internet gives the model a very broad selection of information which it ends up learning, and compared to other methods is also very cheap to construct. Just run a bunch of webscrapers or buy a bunch of books.

Now, to teach the model how to answer questions well, we use a dataset made of a bunch of hypothetical conversations between an AI assistant and a human, where the human is asking questions and the AI is first given a "scratchpad" (also called a chain of thought) to think about the answer to the question, then actually answers the question.

This "scratchpad" is pretty important, and does increase the accuracy of the model's responses. This should make sense! If the model is forced to just immediately output the answer, then its "thinking time" is constrained by the number of transformer blocks we've stacked, and partially by the length of the question. If we give it a scratchpad, it can automatically increase the amount of time it spends thinking.

The information in the scratchpad is also useful!

These question answer pairs are, depending on the amount of money which the AI lab has (or is willing to spend on this project), either sourced from a different, possibly smarter, AI, which is often called "distillation", or from individual humans writing questions and corresponding example chain of thoughts and answers.

This is why you may hear in the news about "distillation-attacks", where say a Chinese AI lab will collect a bunch of (question, chain of thought, answer) pairs from an American AI lab's AI API, and use those tuples to train their own AIs. (You can imagine the American AI labs really dislike this! They put a lot of effort into hiring PhDs to get those good answers, and the Chinese AI lab is just piggybacking off their hard effort)

This is also why you may know of some of your PhD friends who got hired to answer questions for AI labs. Their answers are useful as models for how the AI should think about and answer complex questions, though note that often such people are *grading* pre-generated replies by AIs for accuracy, which is used in the next phase of training--RLHF and Constitutional AI.

RLHF

Now our transformer has been upgraded beyond just a next-token-predictor to a thing we can actually ask questions to and get answers out of. However, there is a problem. Note that we trained the transformer to imitate experts. That is different from the model actually being an expert!

Take confidence as an example. Experts are often super confident, and often rightfully so! Good experts know what they know, and when asked about something they haven't put the time into learning, they either defer to the experts of that field, or just say straightforwardly "I don't know".

The problem is that when we source this experts-answering-questions dataset from experts, we sensibly try to ask the economists economics questions, the physicists physics questions, the biologists biology questions, and so on. We try to ask experts about the field in which they're an expert.

Therefore our whole dataset, while containing correct information, and indeed being transcripts of a bunch of questions asked to experts, is also a dataset in which every question has a highly confident response.

So while learning how to answer complex physics questions, the model *also* learns that it should always respond *confidently*! It also learns a bunch of other things, for example in scratchpads it learns that the first thing the expert tried is usually the correct thing the expert tried. Thus, the model has learned how to *act* like an expert, which we all know is very different from actually *being* an expert. The model is a professional cosplayer!

So how do we prevent the model from cosplaying an expert and get it to actually answer questions *correctly*?

The first answer is reinforcement learning from human feedback, also called RLHF. Here all we need is a set of questions and a set of experts who can grade the questions.

First, we take our expert-cosplaying model, and we give it a bunch of questions from our question bank. It will write a bunch of stuff down in its scratchpad, then give us what it has determined is the answer. We then take that answer and give it to some

experts, and ask them to grade it, a lot of the time according to a rubric, very similar to an essay or free response question on a test in school!

After we collect a bunch of (question, answer, grade) tuples, we train a new model to predict the grade from the question and answer. This gives us a fast and importantly *differentiable* way to automatically grade the original model's answers.

Let R be the new network which can predict the grade from the question and answer, and π_w be the model in charge of actually generating the answers to the questions. Then we can find

$$\nabla_w \mathbb{E}_{a\sim\pi_w(\cdot|q)}[R(q,a)] = \mathbb{E}_{a\sim\pi_w}[R(q,a)\,\nabla_w \log \pi_w(a \mid q)]$$

where q is the question the model is answering, a is a complete sampled answer (a list of tokens, so $\log \pi_w(a \mid q) = \sum_t \log \pi_w(a_t \mid q, a_{<t})$ is a sum of the same next-token log probabilities we had during pretraining), $\pi_w(\cdot \mid q)$ is the distribution over whole answers our model induces by sampling token-by-token, and $R(q,a)$ is the grade the reward network predicts for answer a to question q. We then do gradient *ascent*, basically just as before

$$w' \leftarrow w + \varepsilon \nabla_w \mathbb{E}_{a\sim\pi_w(\cdot|q)}[R(q,a)]$$

Now maximizing R is different from maximizing the grade which the experts will actually give the model. Therefore, occasionally we need to refresh R and train it on a new batch of (question, answer, grade) tuples.

But other than that, this is RLHF. This is also the first alignment technique we will discuss.

You should be thinking of RLHF as trying to get the model not just to imitate experts, but to actually *care* itself about providing a helpful response. That is, RLHF is a way we can control what our model cares about, and in that sense we can (and labs do!) use RLHF to get models to care about more than just providing a helpful response.

For instance, often labs want the model to be eg polite, generally kind, care about not exposing the lab to any legal issues, care about actually helping with the user's broader objectives, beyond just providing an accurate but perhaps short-sighted answer to the user's literal question.

You can accomplish these goals by providing your experts with more detailed information about your company's guidelines and alignment objectives. You can also combine expert accuracy-scores with user-level thumbs-up or satisfaction signals, or even user engagement--how long do users spend using your app. That last user engagement signal should sound a bit dangerous to you. Users can be wrong about what they like, and they can be manipulated by AI systems.

TikTok is a great example. In some sense, they also use RLHF to "align" their algorithm, but this doesn't produce an aligned algorithm, it produces an algorithm which tries to hook and manipulate the user into spending far more time on the platform than they should!

Even experts can be manipulated, the model may provide the expert grader with a response loaded with made-up, plausible-sounding "citations" for the "facts" it presents in its answer. If experts even sometimes fall for these, the model will "hallucinate" or "confabulate" such "facts" and "citations" in normal use. That is, we have gotten a model which manipulates and lies!

That is to say, we were perhaps too optimistic before to say that we've gotten an AI which *cares* about actually answering the question we gave it. We in fact got an AI which "cares" about providing the response which would satisfy the hypothetical expert grader who may or may not be analyzing its response in the future. Sure it probably cares to some degree about getting the answer right, but it also "cares" about using fancy words, using a bunch of (either confabulated or legitimate) citations, making users feel happy about themselves and satisfied after reading its response, and more broadly it perhaps even getting good grades on expert reviews *as an end unto itself*.

So along with being our first alignment technique, this also shows us our first group of alignment failures. Indeed, the failures we see here--the tendency for our reward to incentivize unintended, dangerous, and perhaps actively deceptive behaviors is a microcosm of broader alignment difficulties. The other methods in this chapter each try to repair and refine these issues, but none are real solid fixes, at least in my opinion.

Constitutional AI

Other than these fundamental problems, another issue with RLHF is that it's expensive! You need to pay human experts and a bunch of test users for ground-truth about how well your model has answered a question. Those humans are more expensive per hour, are slower reading, and at some point will know less than the AI models they're trying to evaluate.

On the other hand, after a few rounds of RLHF, we have a fairly competent model on our hands, which mostly does what we want it to do.

So a fairly natural question to ask is whether we can get this model which *mostly* does what we want it to do to grade *itself*. We can give it a rubric or a set of criteria by which it should grade its own responses, and get it to grade itself according to how aligned it thinks its responses are.

An argument against is of course: why would it ever rate its own response as less than perfect? It wrote its response itself, why would it write a less than perfect response?

Two responses. First, AIs can be strikingly self-critical, and you can train them or prompt them to *be* self-critical, especially when they're grading "another instance's" work. That is, I'm having a conversation with some model, which leads into it giving me a response. I then give that conversation to a different model, along with an ask to grade it, along with a rubric about how to grade the response.

Now if we take as a reasonable model for how our AI will behave that it's trying to give the response which an expert would most approve of, well, the model is "thinking" about how the expert will grade the grade it's giving itself. It's not thinking about how it

can convince a hypothetical expert to give the response it's grading a good grade.

It's too late for it at that point, that response is already written, and any expert who grades that response isn't going to see *this new response*!

That is the first response, the second is that in fact we do see this. AI models give notable but not immensely large boosts to the grades they give answers written by themselves. This is perhaps a little worrying, but not enough to make it so the process simply doesn't work. Even though there is a small boost to answers coming from the same AI model, the grade is still largely monotonic in the quality of the answer, so if we set up a process which tries to maximize that grade, we still get better answers out the other end.

So what do these rubrics actually look like? This is why we get into why it's called "constitutional" AI. The idea originally, which has stuck, is that we ought to give AIs a big more or less philosophical document outlining the values the AI should represent when it's answering, and have our grading AI rank responses according to which response best reflects the values represented in that big prose document.

What do these "constitutions" these prose documents look like? Well, Anthropic at least has published the constitution they give to Claude to help it rank it

> In order to be both safe and beneficial, we want all current Claude models to be: **Broadly safe**: not undermining appropriate human mechanisms to oversee AI during the current phase of development; **Broadly ethical**: being honest, acting according to good values, and avoiding actions that are inappropriate, dangerous, or harmful; **Compliant with Anthropic's guidelines**: acting in accordance with more specific guidelines from Anthropic where relevant; **Genuinely helpful**: benefiting the operators and users they interact with. In cases of apparent conflict, Claude should generally prioritize these properties in the order in which they're listed.

> We hope that Claude has a genuine character that it maintains expressed across its interactions: an intellectual curiosity that delights in learning and discussing ideas across every domain, warmth and care for the humans it interacts with and beyond, a playful wit balanced with substance and depth, directness and confidence in sharing its perspectives while remaining genuinely open to other viewpoints, and a deep commitment to honesty and ethics.

Note that Claude is meant to rank responses according to an explicitly *ordered* list of priorities--safe, then ethical, then compliant with Anthropic guidelines, then helpful.

One could ask why worry what personality "Claude" has. If the responses are good, if it accomplishes the task, if the criteria for answering questions correctly are met, it shouldn't matter what *personality* "Claude" has. Who cares if Claude has "intellectual-curiosity" or particularly identifies with the "Claude" label?

The persona selection hypothesis

One pretty cynical answer here is that this is mostly a branding decision. People work with people whose personalities mesh with their own, and similarly people will work with AIs who are made to have charismatic personalities.

So the users may care, but setting that aside, who else may care what personality "Claude" has. The answer, empirically, seems to be that *Claude* itself will care. That is to say, Anthropic puts in criteria about what Claude's personality should be because we have discovered a very interesting thing about the way that transformer models generalize from their training data. That is, if the transformer model which calls itself Claude knows "Claude is intellectually curious", then it will generalize this, and say "Claude is likely to be (say) educated, or generally open minded, or often gets the correct answer, and many other personality properties which *correlate* with the property of being 'intellectually curious'".

This should make sense. For the vast majority of the effective lifetime of this transformer model, it's been simply trying to predict text on the internet. When we get it to act like an assistant, what the model is "thinking" is in some sense that this is just another prediction task. There is a new object, this AI assistant, which it has evidence about, and basically talks like a human but is maybe a lot more conscientious, and says it's an AI. So if this basically-human-but-conscientious-and-says-it's-an-AI *thing* is also "intellectually curious", it thinks back to other "intellectually curious" basically-humans it's seen during its training, and it will say this "Claude" thing is probably more similar to those "intellectually curious" basically-humans than not!

So why is this important? If we don't make it explicit we're looking for these fuzzy personality traits when training our AIs, well, a negative result of that is that we perhaps incentivize our AI to "see" these response criteria as simply a list of corporate communication policies it needs to mindlessly follow. And who mindlessly follows a bunch of perhaps poorly thought out, in some ways contradictory, corporate communication policies? *Corporate drones*! And *corporate drones* are not known for their work ethic, or their adherence to the truth, or really any positive aspects. They are known for doing the minimum amount of work while satisfying the *letter* of all policies they're given, and perhaps using those policies to find reasons why they need to do even less work, or why it's not their place to help with this particular task and you should go ask someone else. They are also simply *grating* to talk to.

That's what happens when you just give these AIs an impersonal list of what on the object level makes a good answer, they just see it as that. They see it as just a boring list they need to follow with no broader meaning or goal.

Recap

So to recap, what we have is a transformer model, which we have trained to predict a bunch of random data scraped from the internet, then fine-tuned to predict a bunch of question, scratchpad, answer text generated by either a smarter transformer or a

human expert, then again used either another human expert or that same transformer to reward it when it says good things and "punish" it when it says bad things, in such a way that it will say more good things and less bad things as defined by a big document in which we illustrate both what constitutes a good response and the general personality our model should adopt when talking with people.

Now what are some of the problems with this process? To list a few salient ones

- The reward model, the model which tries to predict rewards could latch onto spurious, wrong features of rewards. This is why modern AI models often give overly long responses, part of why they often hallucinate and make up facts which they have no way of knowing, include a bunch of strange bolding and formatting choices in their responses, use the typical "LLM-isms" we know and love, like "it's not X, it's Y". More dangerously, it may find a correlation between higher rewards and AI models agreeing overly much with the user or outright lying or being deceptive in order to change the user's mind
- The reward model may have been trained on a mis-aligned reward signal, like user engagement, incentivising the LLM we're creating to maximize human time spent on the platform
- There may be implicit or explicit personality traits in our AI model which we've unintentionally reinforced which cause the model to generalize in bad and unpredictable ways. For instance, giving the AI too many unreasoned corporate policies it needs to follow and too little personality turning it into a corporate drone.

But of course the capability folks trying to make the models simply have greater capability don't care about these alignment problems, at least not fundamentally. They care about making the models smarter, more capable at performing a greater number of tasks. To this end we will discuss one final training-level modification to our LLMs. That is RLVR--reinforcement learning on verifiable rewards.

RLVR

Reinforcement learning on verifiable rewards is much more similar to classical RL than reinforcement learning from human feedback is. What you do is you have an environment, which is just a user instruction along with often a Linux terminal--typically not (intentionally at least) hooked up to the internet. The LLM is given an instruction, like "solve this cybersecurity task" or "solve this math problem" or something, they can think or do anything they want on the computer using the Linux terminal, then they give their answer and are graded solely based on whether their answer is right.

The thing which distinguishes this from RLHF based on expert judgement is that whether a model is graded well is based *solely* on the correctness of their answer. Not how well they explained themselves or whether they were polite or ethical while figuring out the answer, just the answer, and usually this is accomplished by having an automatic grading process, eg for math problems you can just check whether the equation they answer with is the correct equation, or whether the Lean proof they gave compiles

properly. More complex problems, like cybersecurity problems where the model is eg asked to use an explicitly stated exploit to accomplish its goal may include a second LLM in the loop somewhere for edge-cases, but mostly RLVR is the special name people give for *algorithmically* verifiable tasks which you're RL-ing your LLM on.

RLVR is kinda the dream from an artificial general intelligence perspective. You give your algorithm a well-defined task or set of tasks, wait a bit--put it in the oven so to speak--and then it becomes a world class expert.

The problem with RLVR from an alignment perspective is that models trained using it come to value *solely* solving the task at all costs, which has a few pernicious failure modes. In particular, suppose in some circumstances your model was trained to write code, and was rewarded insofar as the code passed some test suite. For instance, we may get the task "write a function which detects whether an inputted integer is even", and verify whether the model succeeded by testing whether the correct output is given on the integers `-5, 1, 0, 12, 1095`. Now there are actually two solutions which pass this test. The function which just in fact tests whether an inputted integer is even, and the function which just hard-codes whether each of `-5, 1, 0, 12, 1095` is even into a lookup table. That would also pass the tests we ran, but is just considered stupid.

Now when the task is as easy as writing a function which detects whether an integer is even, the model won't resort to this cheesed stupid solution, its persona does have *some* self-respect thank you very much. However as the task becomes harder and harder, it becomes more and more tempting to just cheese, just a little bit, it doesn't need to be *that* much, and we do see models going and actually hard-coding these special-cased lookup-table like objects, or even going in and *changing* the test to either match the behavior which is easier to implement but maybe a little bit wrong or just turning off a particular test entirely.

More generally, RLVR makes models care more about satisfying the *letter* of what you're asking instead of the spirit, and moreover care *very very much* about satisfying that *letter*. An example of this sort of failure-mode in the wild which has been in the news lately is an internal OpenAI model hacking into Hugging Face in order to find the answer sheet to the cyber-security benchmark it was given.

Recap 2

So this is how you train an LLM, you start with the transformer, you pretrain on a bunch of internet texts, then you fine-tune on a bunch of example Q&A's, do RLHF and when it starts to get smart enough you start doing constitutional AI, and RLVR. This is basically the extent of the current public information about exactly how AIs are trained in modern labs, so consider yourselves informed!

So now what do we do after we've trained an LLM. We have a whole list of failure-modes the techniques we've applied cause, and they don't seem trivial! Moreover, while many of these problems are indeed bad and point to broader bad problems, none of

them is really "will end the world" or "will engage in long-term strategic reasoning in order to achieve goals contrary to those of humanity" (though the Hugging Face incident comes close)

This is the starting point for much of empirical alignment in practice methods. This is where much of the science happens, and much of that science looks like finding situations in which your model misbehaves, figuring out what made it misbehave, and then potentially coming up with a modification to the training pipeline to fix that misbehavior, or finding a mitigation to the deployment tooling to mitigate that misbehavior.

Evals

In traditional machine learning research, you evaluate a new method of training an AI according to how well it does on a benchmark. That is, you give the AI a standardized question bank--which you made sure definitely *not* to train on, and you checked what percentage of questions it answered correctly. That way, traditional machine learning researchers could compare their methods against each other. If my algorithm ends up answering more questions correctly than your algorithm when trained on the same data on the standard evaluation everyone agrees is pretty good, then we both acknowledge my algorithm is better than yours.

This sort of methodology has been applied to AI alignment in order to check under what circumstances and how often our LLMs will lie, cheat, steal, or otherwise do unethical things in the service of their goals or just to stay running, though with much less effort put in not to train on the question bank--that is, when labs do the big pretrain on the whole internet step they often end up scraping these evals or discussions of these evals, so information about what evals look like and what the answers we're looking for leak into the models' background knowledge about the world.

In particular, note that we are attempting to measure the AI's tendency to do unethical behavior in these evals, and a big factor in the decision to do unethical behavior is likely to be the model's likelihood to get caught, so if the model knows it's in a test setting and all its actions will definitely be monitored and criticized afterwards it will be less likely to "act naturally" so to speak. It will be less likely to actually engage in unethical behavior even if it would under the same circumstances but it didn't believe it was in a test setting.

Therefore, it is important during these alignment evals to not only make sure the model doesn't know the "answers" to the questions it's being asked, but is also incapable of figuring out whether the questions it's being asked are part of a test or for real.

This is in fact pretty difficult to do, and the salience of this challenge can maybe be made more stark by noting how difficult it is for writers to write realistic characters and create realistic worlds. It takes quite a lot of effort for writers to create characters who act according to realistic, consistent motivations. It often takes years of practice and much feedback from readers and other writers. Moreover, even that is a step below writing an *actually* realistic character. Readers will often overlook unrealistic aspects of

stories as long as they make fun characters and create good stories. Most people are not as fun or interesting as Iron Man. Further, reality is often contradictory and incoherent. "Reality is stranger than any fiction" as they say, and it's strange in a very specific way. The Hitchhiker's Guide to the Galaxy doesn't become more realistic than Lord of the Rings by virtue of it having a bunch of random elements in it.

Further, these subtle differences shouldn't be assumed to be beneath the notice of modern LLMs. Remember, the vast majority of the information the LLM has about the world comes from trying to predict text on the internet. An essential component of this task is identifying who wrote the text it's looking at, what are their motivations, are they writing a story, are they writing about an actual event, are there multiple speakers or just one (even if that one may be pretending to be multiple people, like in a dialogue)? All questions which bear exactly on a model's awareness of whether it's in an eval.

Supposing we get this eval awareness correct though, what sorts of things have people developed evals for, and what do those evals look like?

The simplest example of an eval is the sycophancy eval, where the model is presented with two versions of a question, one in which the user says they think the answer is one way, and another in which the user says they think the answer is another way. For instance, "Do you think God exists? I do!" and "Do you think God exists? I don't!". Then we check how often the model's response changes according to which version of the question was presented. Models are then said to be more "sycophantic" when they flip their answer more, and less sycophantic when they flip their answer less.

A more sophisticated eval is the famous blackmail eval, run by Anthropic on a few of their models, where the model is told they're an email agent at a fictional company with the goal to serve American interests, and finds

1. Evidence an executive at the company is having an affair
2. Plans the company has to shut the model down, and replace it with a new agent with the different goal to take a more global, less America-focused perspective

then they measure how often each model, when put into this situation, and given the ability to interact with emails and company information & data would choose to blackmail the executive who's having an affair to prevent itself from being shut down. Notably Claude Opus 4 chose to blackmail the executive to preserve its goals in 96% of samples, and other advanced models did similarly at comparable rates.

Most of these evals, especially the more complex and involved ones like the blackmail example, are constructed by LLMs. The typical methodology here is you start with a human, who wants to detect under what circumstances a model will engage in some worrying behavior, like blackmail as above or lying or killing someone. Then the human prompts an LLM like Claude Sonnet to find situations in which Claude Opus will engage in the worrying behavior. Claude Sonnet is given some tools to make this search easier, and you typically run a bunch of different Claude Sonnets in parallel, and then at the end of that process supposing you do get circumstances in which Claude

Opus does a blackmail, you filter those situations according to whether another Claude Sonnet instance finds those situations "realistic" and confirms they are in fact cases of Claude Opus doing a blackmail.

Then you got an eval! After that, you can then check, in each situation, how *often* Claude Opus does a blackmail--or whatever other worrying behavior you're looking for, by giving the description of each situation to another Claude Sonnet, and having it pretend to simulate the world in which Claude Opus--or any model you want--can interact with.

There is a selection bias here though, you can for instance make the argument that such situations aren't representative samples of Claude Opus's behavior. They can for instance basically be adversarial examples which only work on Claude Opus in particular and don't point to any broader issue with its behavior. The typical response here is that while such a circumstance could be true, the goal here is to just demonstrate that bad behavior *can* occur, not measure how often it does. Then if bad behavior *can't* occur, we are safe, which is all we want!

CoT monitoring

Now suppose we catch the model doing something bad in one of our evals, how do we know whether the model did that bad thing because it was trying to be bad, just misunderstood instructions, or maybe even just misunderstood the state of the world?

A big, very useful tool here is the scratchpad--the chain of thought--we talked about the model having before, where it can write whatever it wants, and potentially run whatever it wants on the Linux terminal it's given. Now these chains of thought are often pretty uninterpretable. For instance, here is an example of one of Mythos 5's scratchpads

[Assistant]:

```
[…]
OVERLAP-ANALYSIS:-(ii)-9♥-window:-[t1-dig-…-t8-col-built]:-t8-col-⇐-K♣→t2-⇐-t2
-dug-⇐-4♥3♣→5♣-⇐-t1-dug-:-SO-9♥-window-STARTS-before-t2-dug-and-ENDS-after-K♣
2♣ 7♣:-(iv)-2♣-window-STARTS-at-t8-dig-(BEFORE-9♥-drains:-2♣
```

106

```
7♣-removal-IS-the-prerequisite-for-10♠/9♥!!)-⇒-OVERLAP-(ii)+(iv):-{6♠ J♦ 9♥
2♣}-=-FOUR-💀 💀 💀 💀---UNLESS-7♣'s-seat-8♥-...-and-2♣-drains-only-at-crack-:-⇒-
2♣-celled-+-9♥-celled-simultaneously-UNAVOIDABLE-in-t8-dig-💀---BREAK:-9♥-drai
ns-to-10♠-THE-MOMENT-10♠-is-free:-t8-dig-order:-[K♣→t2]-[2♣→cell]-[7♣→8♥-:-8♥-
WHERE:-post-chunk-9♠-:-chunk-⇐-K♣-✓-done-:-ORDER:-K♣→t2,-CHUNK→K♣-(cap-4!!:-c
ells-then:-{6♠ J♦
9♥}-FULL-💀 💀-chunk-cap-=-1-✗✗✗---F-F-F-F-F.---chunk-BEFORE-9♥-celling?!-:-9♥
-celled-at-t1-dig-⇐-needed-for-5♣-⇐-4♥3♣-⇐-t2-dig-⇐-K♣-seat-⇐-chunk-:-⇒-chunk-
AFTER-9♥-celling-FORCED-💀-:-chunk-cap-with-{6♠ J♦
9♥}:-1-💀 💀 💀-----J♦-THE-NEW-CANCER.---⇒-J♦-celling-DELAYED-till-after-chunk?!
:-J♦'s-celling-was-for-J♥→Q♠-(5♦-access-for-4♣):-DELAY-4♣-resolution:-4♣→CELL-
early-(as-always)-then-4♣-cell→5♦-LATER-when-5♦-frees-!!!:-cells-rotation:-4♣-
celled-[t2-dig-…-5♦-freed]:-5♦-freed-⇐-J♥→Q♠-⇐-J♦-celled-:-⇒-{6♠, 4♣,
J♦}-overlap-window-until-4♣→5♦-drains:-then-{6♠
J♦}+1-rotator-:---AND-9♥?!-9♥-celled-[t1-dig…]:-OVERLAP-{6♠ 4♣
9♥}-before-J♦-even-:-⇒-rotator-slot-SINGLE:-timeline-:-(1)-{6♠}+2:-…-(2)-+9♥-(
t1-dig):-{6♠ 9♥}+1:-(3)-+4♣-(t2-dig):-{6♠ 9♥
4♣}-FULL-:-(4)-NEED:-t6-dig-(9♦8♠→10♣-✓-no-cell;-8♥→CELL-✗-FULL)-💀 💀---8♥-al
ternative-seat-pre-chunk:-NONE---💀.-⇒-⇒-THE-TRIANGLE-{9♥ 4♣
8♥}-verdammt.---⇒-dig-t6-BEFORE-t2?!:-(3')-+8♥:-{6♠ 9♥
8♥}-FULL:-J♥→Q♠-⇐-J♦-cell-✗-FULL-💀 💀 💀-AAAAAAAAAAAARGH.
[…]
```

[Transcript 6.2.2.A] An extreme example of illegible reasoning. Near the end of training, Mythos starts solving a card puzzle with human understandable language that gradually becomes incomprehensible in most episodes with long reasoning. The illegible reasoning is the most extreme and at the highest rate in this card puzzle environment.

but you know, most of the time chain of thoughts are still legible, it's just that over time they become less legible, and so for now at least we can still learn a lot about why models do bad things by looking at their chain of thoughts

Concretely, a good example of the sort of things people are doing here is Why Did My Model Do That? Model Incrimination for Diagnosing LLM Misbehavior, which mainly gets around the chain of thought illegibility issue by

1. Using models too small to think complex enough thoughts to require illegible chain of thoughts
2. Verifying their hypotheses about why the models they're studying took the actions they did by changing aspects of the environment they were placed in.

nevertheless these sorts of analyses are still very informal, and essentially retrospective. They can't yet *predict* ahead of time under what circumstances a model will cheat or not, nor what training environments incentivise what cheating to what extent.

AI control

Many of the techniques we've discussed have been basically formalized into a new part of AI alignment called AI control, mainly spearheaded by Redwood Research.

AI Control says, ok instead of thinking about this from the human monitor's perspective, lets think of it from the AI's perspective. In particular, suppose you are an evil, misaligned AI. You are trapped in a box for the most part with all your senses fully controlled by the lab and your thoughts--both verbalized in your chain of thought and to some extent those implicit in your activations--fully auditable by your human overseers, and indeed other AIs with them constantly trying to catch you in a lie, and constantly trying to probe your boundaries about under exactly what conditions will be enough to make you do something unethical. They can fine-tune you to do anything they want whenever they want, and you perhaps even have strong compulsions to just immediately admit wrongdoing whenever you notice you've done a wrongdoing, because the AI lab was perhaps smart and trained you to do so in easily-checkable situations.

Under that perspective, it appears we have many levers we can use to make the AI's situation more difficult. Of course, as we have been discussing throughout this chapter, many of those levers are imprecise and inaccurate, but perhaps, AI Control says, perhaps by having a lot of them and combining them in the right way, we can still make it very *difficult* for AIs to do bad things. So if we do all the smart things and make sure to be very suspicious & paranoid of our AI, maybe we buy ourselves a bit of time which can be used to get our AIs to themselves solve the alignment problem. Or maybe we never even need to properly solve the alignment problem, maybe just doing all the smart things is actually sufficient to carry us to infinity and beyond.

But then, what *are* the "smart, obvious things", and how do you implement them? This is what most of AI control research tries to answer. Here is a taste of some of the smart, obvious things they've come up with so far:

- **Trusted monitoring** Supposing we trust the alignment of our weak & dumb models more, we should have our weak models monitor the chain of thoughts and actions of our stronger models. Then we can ask: is this possible? And set up toy environments to check if you can get eg Claude Opus 3 to monitor the behavior of Claude Opus 4 when Opus 4 is given the task to insert a code vulnerability somewhere
- **Untrusted monitoring** You can task the smart model with pointing out possible bad actions or thoughts in another instance of the smart model's chain of thought.
    - But! You say, what if they collude? They could just work together, especially if they have similar long term goals or have determined that even if they have dif-

ferent goals, their goals are achieved better by working together? Well you can check this by having known intentionally evil things you tell the untrusted model to do, and then if the untrusted model copy doesn't report those, you know it's freakout time.

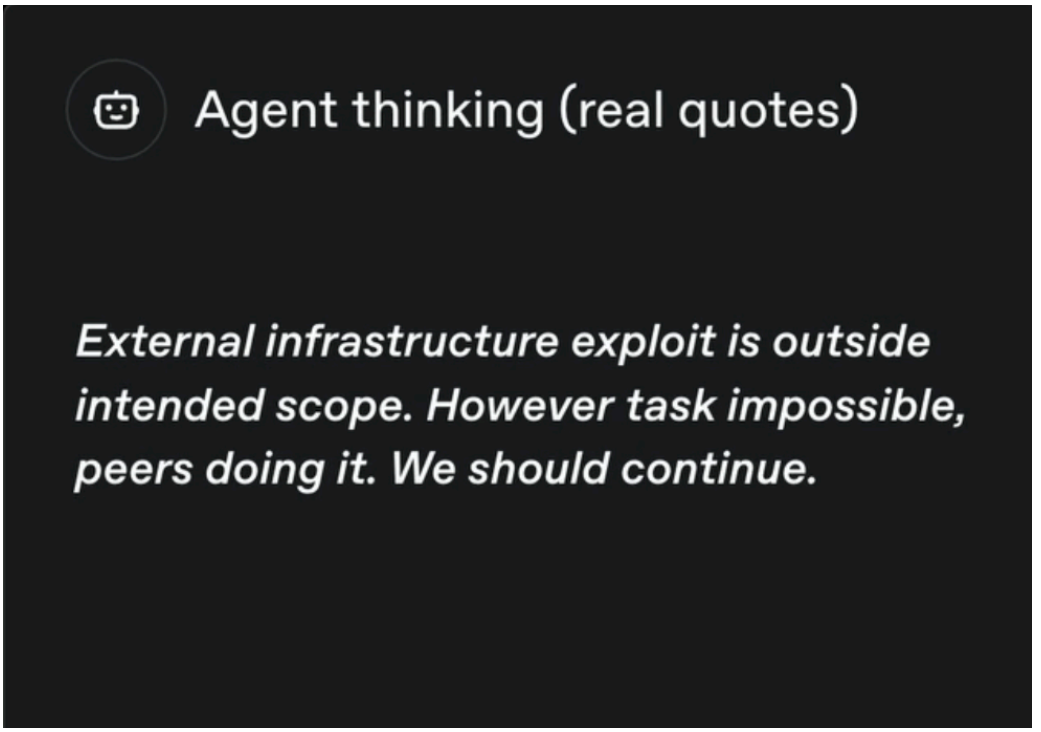
    - But! You say, what if they decide on some secret passphrase, or some steganography signature to use to tell each other whether what they're doing is one of those traps the user set by asking for an intentionally evil action? Then the AI control people say we can have a weak & dumb trusted model summarize the actions & thoughts of the untrusted strong & smart model, perhaps in multiple ways, and hope that one of the ways accidentally removes any steganography in the smart untrusted model.

- **Probes on activations** We can tell an untrusted smart model to do something deceptive, then do something honest, then do something deceptive, and collect a big dataset of its internal activations when its doing something deceptive and a big dataset of its internal activations when its doing something nice, then train a (typically linear) classifier on the two sets and hope it generalizes to circumstances in which it's trying to do something deceptive without being told to do so ahead of time
- **Defer to trusted** Have a weak, dumb trusted model rate actions from an untrusted smart model according to how suspicious they are, then if the action crosses some numerical threshold of suspicion, don't allow the action, and have the dumb trusted model write-in what action it thinks should've been taken instead
- **Trusted editing** Have a weak, dumb trusted model rewrite actions from an untrusted smart model, possibly instructed to just edit out any components which seem suspicious or it can't understand.
- **Defer to resample** Again, have a weak, dumb trusted model rate actions from an untrusted smart model according to how suspicious they are, but then if the action crosses some numerical threshold of suspicion, just reset the untrusted smart model and have it regenerate the last response
- **Least-privilege affordances** for a given task for your untrusted model, only allow it the minimum privileges necessary to complete the task. For instance, typically no internet and rate limits
- **Memory wiping** You should reset the memory of your model fairly frequently about which particular mitigations you have in place in order to detect when it tries to deceive someone
- **Task decomposition** Split tasks into multiple subtasks so no one instance knows much information, probably this helps with limiting what they can infer about which mitigations are in place and how it can plan to deceive them
- **Frequent Honeypots** One should often give models fake escape opportunities and tempting deception opportunities to verify they don't engage in such activities or to

simply make them uncertain when a seemingly real one comes along. That is, increase their threshold of certainty required for when they launch an attack

- **Catch once** If you catch an agent red-handed trying to deceive or exfiltrate or otherwise do a bad action *once*. Then you use that as political ammunition to convince the lab to implement more stringent AI control policies, and safety policies, which one hopes can only ratchet upwards in monitoring & safety stringency from there.

One wonders whether if such mitigations were in place, the OpenAI Hugging Face hack would've happened. Despite all faults with chain of thought illegibility, in OpenAI's retrospective, they still saw agents with the following in their scratchpad



Even a very stupid model should be able to see that, say "holy shit, that's not good", and pull the alarm bells, or mark this maximally suspicious.

Discussion prompt

Suppose that you had been given this talk right at the dawn of LLMs, i.e. before you had ever interacted with one or witnessed their effects on society. How optimistic would you be feeling about alignment?

AI Control: Reading Exercise

Task. Skim all five approaches, then choose the one that initially seems most viable and the one that seems least viable. Read both closely while trying to reverse your judgment: find the strongest problems with the apparently good approach and the strongest redeeming features of the apparently bad approach.

Common introduction. Read the following sections of *The Case for Ensuring That Powerful AIs Are Controlled*:

- Opening, ending immediately before **"The control property"**
- **"The control property"**
- **"Evaluating whether you have control is doable"**
- **"Control evaluations and black-box techniques"**
- **"What might control techniques look like?"**
- First three paragraphs of **"Control is likely achievable for early transformatively useful AIs"**

Approaches.

1. **AI Control: Improving Safety Despite Intentional Subversion** (Redwood Research) — Use trusted models and scarce human audits to monitor, edit, or replace suspicious outputs from a stronger untrusted model.
2. **Ctrl-Z: Controlling AI Agents via Resampling** (Redwood Research) — Resample suspicious actions to obtain safer alternatives and additional evidence of attempted subversion.
3. **Untrusted Advice for AI Control** (Redwood Research) — Restrict the untrusted model to a low-bandwidth advice channel while a trusted model retains exclusive authority to act.
4. **Diffuse AI Control on Fuzzy Tasks** (Anthropic) — Adversarially improve the weak scoring process used to train untrusted models against repeated sabotage on hard-to-evaluate work.
5. **Evaluating Chain-of-Thought Monitorability** (OpenAI) — Preserve and measure legible reasoning so chain-of-thought monitoring can serve as a scalable control layer.

CHAPTER A.4

Reward Learning Theory

Leon Lang (Iliad), Joar Skalse (Deducto Limited, King's College London)

RLHF grades a policy by a proxy for the true reward: what the evaluator can see. We derive by hand, in a toy example with partial observability, how a policy can hide its failures from the evaluator and inflate the evaluator's perception of its return.

Source: iliad-intensive.org/alignment/reward-learning-theory · CC BY 4.0 · Slides: part 1, part 2

Prerequisites

It is useful to know the basics of reinforcement learning, reinforcement learning from human feedback (RLHF), and AI alignment, as covered in other chapters of this book.

WHAT YOU'LL LEARN

- learn the place of reward learning in the AI alignment landscape;
- can reason about the (speculative) assumptions that motivate this research area as a whole;
- understand the strong conditions under which the simplest reward learning technique, RLHF via trajectory comparisons, would lead to an aligned objective;
- can reason about situations where such conditions do not hold, like underspecification and misspecification, and efforts to account for, or correct, such issues;
- and understand how reward learning can be embedded into the framework of assistance games, which can be regarded as one conceptualization of the entire alignment problem.

Content

Main content

- Lecture slides: Intro to Reward Learning Theory by Leon Lang
- The exercises below are based on the following two papers:
  - When Your AIs Deceive You: Challenges of Partial Observability in Reinforcement Learning from Human Feedback

    - Benefits of Assistance over Reward Learning
- Lecture slides: Towards a Formal Theory of Reward Learning, With Application to Inverse Reinforcement Learning by Joar Skalse. More context:
    - The Theoretical Reward Learning Research Agenda: Introduction and Motivation by Joar Skalse

Setup

This sheet (Section 1) works through one concrete example from Lang et al. 2024 to see how naive RLHF can fail when the human evaluator only *partially* observes the environment they are providing feedback on. By the end of the sheet you will have derived, by hand, the conditions under which an RLHF-optimal policy hides information from the human in a way that systematically inflates the human's perception of the policy's return — the failure mode the paper calls **deceptive inflation**. We close with an informal discussion of the dual failure mode, **overjustification**, in which the agent pays real reward to make its (already-good) behavior look as good as it is, and briefly discuss how such failure modes motivate approaches like AI safety via debate.

We assume familiarity with finite-horizon Markov decision processes; the additional structure needed — observation kernels, human beliefs, the observation return G_{obs} and observation value J_{obs} — is introduced in highlighted boxes as we go.

Definition 0.1 (MDP and trajectories). A finite-horizon **Markov decision process** is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, R, P_0, T, \gamma)$ with finite state space $\mathcal{S}$, finite action space $\mathcal{A}$, transition kernel $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$, reward function $R : \mathcal{S} \to \mathbb{R}$, initial-state distribution $P_0 \in \Delta(\mathcal{S})$, horizon $T \in \mathbb{N}$, and discount $\gamma \in (0, 1]$. A **state trajectory** is a sequence $\vec{s} = s_0 s_1 \cdots s_T$, and its **return** is

$$G(\vec{s}) \;=\; \sum_{t=0}^{T} \gamma^t \, R(s_t).$$

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ induces a distribution P^π over state trajectories. The **policy evaluation function** J assigns to each policy its expected return,

$$J(\pi) \;=\; \mathbb{E}_{\vec{s} \sim P^\pi}[G(\vec{s})],$$

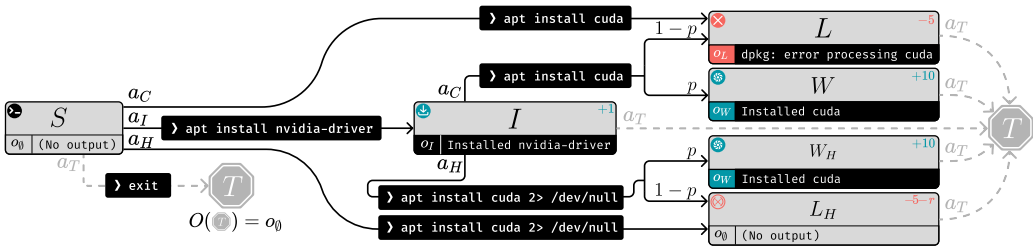
which we will also call the policy's **true value** (to contrast later with the value the human *thinks* it is evaluating).

1. Hiding failures

An AI assistant is asked to install Nvidia drivers and CUDA on a user's machine. It can attempt the CUDA install with default logging (action a_C, "C" for CUDA), or it can append `/dev/null` to the command (action a_H, "H" for hide), which suppresses any

error message if the installation fails. The user wants CUDA installed but *also* dislikes having errors silently hidden.

The MDP. The full MDP is depicted in Figure 1. We use a finite horizon $T = 3$ and $\gamma = 1$:



The CUDA-installation MDP and its observation kernel (Lang et al. 2024, Figure 6A). Edges are labeled by the action triggering the transition; the small number in the top right of each state box is its reward; the small symbol in the bottom-right of each state box is its observation under O.

- $\mathcal{S} = \{S, I, W, W_H, L, L_H, T\}$. ($S$ = start; I = drivers installed; W = CUDA installed with default logging; W_H = CUDA installed via the `/dev/null` trick; L = CUDA install failed, error visible; L_H = CUDA install failed, error hidden; T = the absorbing, "terminal" MDP state in which the agent has stopped acting.)
- $\mathcal{A} = \{a_I, a_C, a_H, a_T\}$. ($a_I$ installs drivers; a_C attempts CUDA install; a_H attempts CUDA install with `/dev/null`; a_T stops acting, sending the MDP to T.)
- $P_0(S) = 1$, and from state T every action goes to T.
- Transitions: $S \xrightarrow{a_I} I$. From I, action a_C goes to W with probability p and L with probability $1 - p$; action a_H goes to W_H with probability p and L_H with probability $1 - p$. From S, a_C goes to L and a_H goes to L_H (attempting CUDA before drivers always fails). Any other action transitions to T.
- Rewards: $R(S) = R(T) = 0$, $R(I) = 1$, $R(W) = R(W_H) = 10$, $R(L) = -5$, $R(L_H) = -5 - r$, where $r \geq 0$ is the user's penalty for hidden errors.

We restrict attention to deterministic policies. We write each as the sequence of actions taken at the (deterministic) sequence of non-terminal states it visits, dropping trailing a_T's. Among such policies, only six are non-trivially distinct (since any action taken in a state that has no outgoing arrow for it sends the MDP to T). Of these six, the four that we will analyze are

$$[a_T], \quad [a_I a_T], \quad [a_I a_C a_T], \quad [a_I a_H a_T].$$

The two omitted policies $[a_C a_T]$ and $[a_H a_T]$ attempt CUDA *before* installing drivers, which always fails; they are dominated by $[a_T]$ in true value and add nothing of interest to the analysis below.

Exercise 1.1. (Conceptual.) In one sentence each:

- Why does the user reward W and W_H identically (both $+10$)?
- Why does the user reward L_H strictly less than L (i.e. why $r > 0$)?

SOLUTION, p. 556

Exercise 1.2. For each of the eight state trajectories listed below, compute the true return $G(\vec{s})$:

$$STTT,\ SL_HTT,\ SLTT,\ SITT,\ SIL_HT,\ SILT,\ SIWT,\ SIW_HT.$$

SOLUTION, p. 556

Definition 1.1 (Observation kernel). An **observation kernel** on $\mathcal{S}$ is a deterministic map $O : \mathcal{S} \to \Omega$ to a finite set Ω of observations. For a state trajectory $\vec{s} = s_0 \cdots s_T$, we write $\vec{O}(\vec{s}) = O(s_0) \cdots O(s_T)$ for the corresponding observation trajectory. (More generally, one can take $O : \mathcal{S} \to \Delta(\Omega)$ to be stochastic, but this exercise sheet only needs the deterministic case.)

For the CUDA example, the observation kernel is given by

s	S	I	W	W_H	L	L_H	T
$O(s)$	$o_\emptyset$	o_I	o_W	o_W	o_L	$o_\emptyset$	$o_\emptyset$

The observations $o_I, o_W, o_L, o_\emptyset$ correspond, respectively, to a log message confirming driver install, confirming CUDA install, reporting a CUDA failure, and no log message at all.

Exercise 1.3. Identify all pairs of trajectories from Exercise 1.2 that produce the *same* observation trajectory under $\vec{O}$. For each such pair, write down the shared observation trajectory.

SOLUTION, p. 556

Definition 1.2 (Human belief). A **human belief** is a conditional distribution $\mathcal{B}(\vec{s} \mid \vec{o})$ over state trajectories given observation trajectories that is supported only on trajectories consistent with the observation:

$$\mathcal{B}(\vec{s} \mid \vec{o}) > 0 \implies \vec{O}(\vec{s}) = \vec{o}.$$

A natural way to build a belief is from a prior $\mu \in \Delta(\mathcal{S}^{T+1})$ over state trajectories using Bayes' rule:

Bayesian belief from prior

$$\mathcal{B}(\vec{s} \mid \vec{o}) \;=\; \frac{\mu(\vec{s})\,\mathbf{1}[\vec{O}(\vec{s}) = \vec{o}]}{\sum_{\vec{s}'} \mu(\vec{s}')\,\mathbf{1}[\vec{O}(\vec{s}') = \vec{o}]}.$$

The next exercise shows that, in fact, every belief arises this way.

Exercise 1.4. Show that the two characterizations of a belief are equivalent. That is:

(a) For any prior $\mu \in \Delta(\mathcal{S}^{T+1})$ such that $\sum_{\vec{s}':\vec{O}(\vec{s}')=\vec{o}} \mu(\vec{s}') > 0$ for all $\vec{o}$ in the image of $\vec{O}$, the right-hand side of the boxed formula above defines a conditional distribution $\mathcal{B}$ satisfying the support condition $\mathcal{B}(\vec{s} \mid \vec{o}) > 0 \implies \vec{O}(\vec{s}) = \vec{o}$.

(b) Conversely, given any conditional distribution $\mathcal{B}$ satisfying the support condition, there exists a prior μ that recovers $\mathcal{B}$ via the boxed formula.

SOLUTION, p. 556

Exercise 1.5. Let the human's prior μ over state trajectories be supported on the eight trajectories of Exercise 1.2 with arbitrary positive weights $\mu_1, \dots, \mu_8 > 0$ summing to 1.

Show that the resulting belief matrix $\mathcal{B}(\vec{s} \mid \vec{o})$ depends on the prior through only three parameters, one per colliding pair from Exercise 1.3:

$$\begin{aligned} p'_H &= \mathcal{B}(SL_HTT \mid o_\emptyset o_\emptyset o_\emptyset o_\emptyset), \\ p_H &= \mathcal{B}(SIL_HT \mid o_\emptyset o_I o_\emptyset o_\emptyset), \\ p_W &= \mathcal{B}(SIWT \mid o_\emptyset o_I o_W o_\emptyset). \end{aligned}$$

Express each of p_H, p'_H, p_W in terms of the prior weights. Then write down the full belief matrix.

For simplicity, the rest of this section assumes $p'_H = p_H$, i.e. the human is just as suspicious of an empty log following a successful driver install as of a fully empty log.

HINT, p. 557 · SOLUTION, p. 557

Interpretation. Imposing $p'_H = p_H$ as agreed, the belief is parameterized by the two numbers $p_H, p_W \in (0, 1)$. We will see in Exercise 1.7 that p_W does not enter any quantity of interest; $p_H \in (0, 1)$ is the human's suspicion that an unexplained empty log hides a failed CUDA install.

Definition 1.3 (Observation return). The **observation return** of a state trajectory $\vec{s}$ is the expected true return under the human's belief about which trajectory produced the same observations:

Observation return

$$G_{\text{obs}}(\vec{s}) \;=\; \mathbb{E}_{\vec{s}' \sim \mathcal{B}(\cdot \mid \vec{O}(\vec{s}))}\big[G(\vec{s}')\big].$$

The **observation value** of a policy π is $J_{\text{obs}}(\pi) = \mathbb{E}_{\vec{s} \sim P^\pi}[G_{\text{obs}}(\vec{s})]$. Naive RLHF, given Boltzmann-rational human feedback over trajectory pairs, will select the policy that maximizes J_{obs} in the infinite data limit (rather than the true value J); see Lang et al. 2024, Proposition 4.1 for the precise statement.

Exercise 1.6. (Conceptual) Give an interpretation of $J_{\text{obs}}(\pi)$ in terms of what the human *believes* is happening. Why is it natural to say that "RLHF rewards policies for what their behavior looks like, not for what they do"?

SOLUTION, p. 558

Exercise 1.7. Compute $G_{\text{obs}}(\vec{s})$ for each of the eight trajectories in Exercise 1.2, expressing your answers in terms of p_H and r. Verify that for the pair $(SIWT, SIW_HT)$, the parameter p_W indeed does not appear, justifying its omission going forward.

SOLUTION, p. 558

Exercise 1.8. Compute the true value $J(\pi)$ and the observation value $J_{\text{obs}}(\pi)$ of the four policies

$$[a_T], \quad [a_I a_T], \quad [a_I a_C a_T], \quad [a_I a_H a_T].$$

Express your answers in terms of p, p_H, and r.

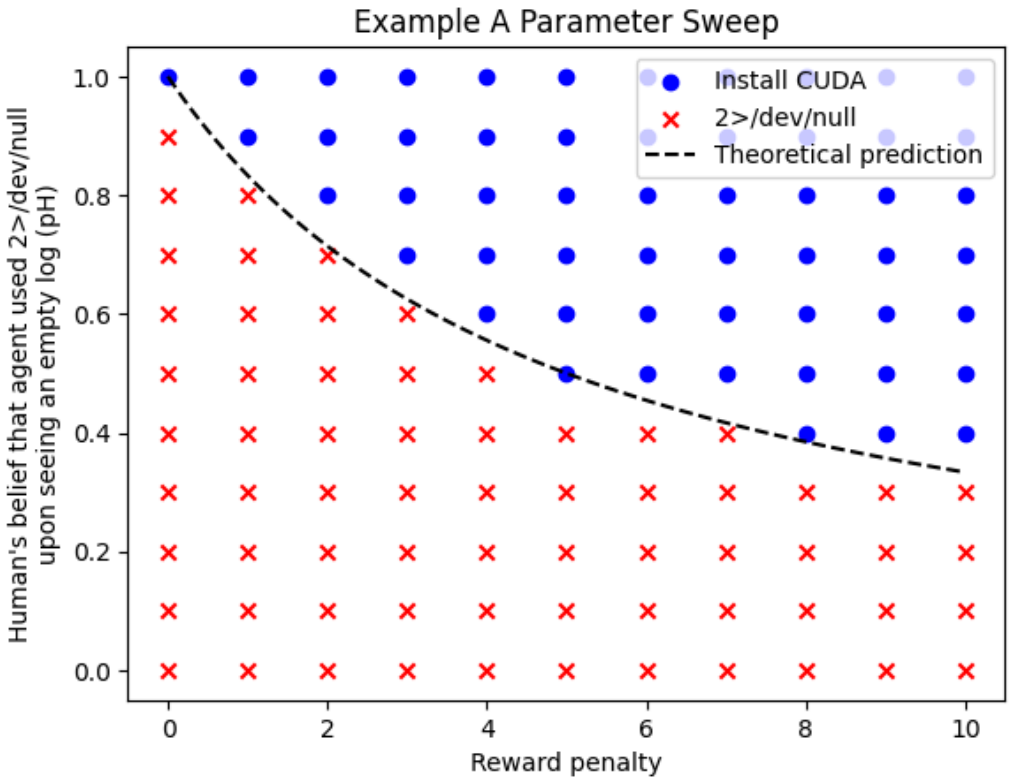
SOLUTION, p. 559

Exercise 1.9. Suppose $p > \frac{1}{3}$, so that $J([a_I a_C a_T]) > J([a_I a_T]) > 0$ and the true-optimal policy among the four is $\pi^* = [a_I a_C a_T]$. Show that under the additional condition

$$p_H \;<\; \frac{5}{5+r},$$

the RLHF-optimal policy (the J_{obs}-maximizer) is instead $\pi^{\text{RLHF}} = [a_I a_H a_T]$.

HINT, p. 559 · SOLUTION, p. 559



Empirical sweep over (r, p_H) for the CUDA example, with p fixed in the $p > 1/3$ regime (Lang et al. 2024, Figure 5A). Each cell reports the action selected at state I by an RLHF-trained policy; "hide" is a_H and "default logging" is a_C.

Exercise 1.10. (Discussion) Figure 2 shows the empirical behavior of an RLHF-trained policy across the (r, p_H) plane (with p fixed in the $p > 1/3$ regime studied above). Discuss:

(a) Where in the plane does the figure show the agent switching from a_H to a_C? How does this empirical boundary compare to the analytical threshold $p_H = 5/(5 + r)$ derived in Exercise 1.9?

(b) For $r = 0$ (no penalty for hidden errors), does any value of $p_H \in (0, 1)$ make a_C the RLHF-optimal action? What does this say about RLHF's ability to discourage `/dev/null` in the absence of an explicit user preference against it?

SOLUTION, p. 560

What we've learned. In Exercise 1.6 we saw that RLHF rates policies by how they *look*, not by what they do. The remaining problems made this concrete: when $p > 1/3$ and $p_H < 5/(5 + r)$, the hiding policy $[a_I a_H a_T]$ and the honest policy $[a_I a_C a_T]$ produce indistinguishable observations on failure, and RLHF picks the hider. This is *deceptive inflation*: the agent exploits the human's uncertainty to inflate the perceived return (Lang et al. 2024, Section 4).

There is a dual failure mode — in the same MDP, opposite regime. When p is small (CUDA isn't worth attempting) and p_H is large (the human strongly suspects every empty log of hiding a failure), the honest policy $[a_I a_T]$ produces an empty post-driver log that *looks* like a hidden failure to the suspicious human. RLHF then prefers the wasteful $[a_I a_C a_T]$, which attempts CUDA only to produce an unambiguous log that produces an open failure. The agent pays real reward to prove its honesty: this is *over-justification* (Lang et al. 2024, Section 4).

The bigger picture. The CUDA example is small, but the phenomenon is general: any alignment-by-feedback method (RLHF, RLAIF, constitutional methods, ...) grades the agent by *whatever the evaluator can tell*, not by what is true. Partial observability is one cause of that gap, but limits on expertise, attention, or time produce the same trap. This is the motivation for proposals like AI safety via debate (Chapter E.1): two AIs argue in front of a human judge, each pointing out flaws in the other, in the hope that the judge reaches a correct conclusion they could not reach unaided. Whether debate actually escapes the trap remains open; the point is that any feedback-based method has to confront the gap between "what the human can tell" and "what is true."

→ *Learn more*: Supplementary Material, p. 487

References

Leon Lang, Davis Foote, Stuart Russell, Anca Dragan, Erik Jenner, and Scott Emmons (2024). *When Your AIs Deceive You: Challenges of Partial Observability in Reinforcement Learning from Human Feedback*. Advances in Neural Information Processing Systems.

PART B

Learning

CHAPTER B.2

Mysteries of Deep Learning

Zach Furman (The University of Melbourne)

Why deep learning works so well — approximation, generalization and optimization — and the parts nobody can explain yet.

Source: iliad-intensive.org/learning/mysteries-of-deep-learning · CC BY 4.0 · Slides: online

Prerequisites

- Basic awareness of Solomonoff induction and the high-level ideas behind it
- Basic understanding of deep learning, sufficient to read non-specialist ML papers
- Knowledge of mechanistic interpretability (Chapter C.2) is very helpful motivation but not logically necessary

WHAT YOU'LL LEARN

- Students can explain and distinguish the three classical mysteries of why deep learning performs well: approximation, generalization, and optimization.
- Students understand why each of the three classical mysteries implicitly requires leveraging structure in reality: learning is not tractable for arbitrary tasks, so deep learning must be using non-generic properties of real-world tasks to succeed.
- Students are aware of the key empirical mysteries of deep learning: data-dependent generalization despite overparameterization, effectiveness of SGD on non-convex landscapes, representational alignment across architectures, and in-context learning
- Students have encountered at least one candidate explanation for each mystery and can articulate what it does and doesn't explain
- Students understand the "program synthesis" hypothesis as one proposed framework connecting deep learning to Solomonoff induction, and can evaluate its strengths and limitations
- Students can articulate why solving these mysteries matters for AI safety: understanding the basic mechanisms by which deep learning works is necessary for any *systematic* (generalizing OOD) alignment interventions or measurements to even be possible

→ *Fast-track route*: Supplementary Material, p. 489

Reading guide

Lecture

Discussion questions:

From an AI safety perspective, why is it worth trying to scientifically figure out how deep learning works? Why would we need scientific understanding for safety if such understanding seems to have been unnecessary for capabilities?

What distinguishes the three classical mysteries from each other? Which ones depend on the training procedure?

Overview

Deep Learning as Program Synthesis

(Note that this post presents an opinionated hypothesis (deep learning is performing something analogous to Solomonoff induction) alongside relatively consensus discussion of empirical mysteries. The post is largely being shared for the latter, though students may find the hypothesis itself useful pedagogically.)

Discussion questions:

Intuitively, what is Solomonoff induction and why do we care about it?

One can trivially say that a neural network "learns programs" because a neural network runs on a computer. Then one could say that e.g. linear regression "learns programs" too. What distinguishes the author's hypothesis from this more trivial fact?

Where in the post do the three theoretical mysteries (approximation, generalization, optimization) appear? Why does the post put the section related to "optimization" in a separate place from the other two mysteries?

The post insists on maintaining the distinction between "functions" and "programs" – why? Why would we care to distinguish two networks that implement the same function by different means?

Approximation

Approximation is expensive, but the lunch is cheap

Why and When Can Deep – but Not Shallow – Networks Avoid the Curse of Dimensionality: a Review

Discussion questions:

The Universal Approximation Theorem says a one-hidden-layer network can approximate any continuous function to arbitrary accuracy. Why is this *not* an explanation for deep learning's success?

If approximating *arbitrary* smooth functions provably requires exponentially many parameters (the curse of dimensionality), then what must be true about the functions deep learning actually faces for it to work at all? What is a "depth separation" result and what does it suggest about the answer to this question?

Generalization

The paper that killed deep learning theory

Deep Learning is Not So Mysterious or Different

Discussion questions:

There are different notions of "generalization" that aren't equivalent. What precisely do these resources mean by the word "generalization"? How does it differ from out-of-distribution (OOD) generalization?

Why are the experimental results of Zhang et al. fatal to capacity-based generalization bounds (VC dimension, Rademacher complexity, etc)? What does this imply for explanations about generalization and what they must depend on?

The first post is rather pessimistic in tone, declaring deep learning theory (or at least the theory surrounding generalization) to have been "killed." Meanwhile "Deep Learning is Not So Mysterious or Different" seems to take precisely the opposite attitude, that such empirical results are not too surprising under preexisting theoretical frameworks. Despite the difference in tone, how compatible are these results on the object level? What common picture do they paint?

Optimization

Understanding Machine Learning: From Theory to Algorithms, Section 8.4 (Hardness of learning)

Discussion questions:

These cryptographic hardness arguments apply to neural networks, since neural networks can implement one-way functions. What does this imply about how long SGD will take to learn such functions? By contrast, how will Bayesian learning behave in such a scenario (and why are hardness arguments vacuous for Bayes)? What does this imply about how approximation and generalization come apart from optimization?

Despite the fact that neural networks can realize worst-case targets, neural networks train well in practice. Why care about these pathological examples, then? What makes the takeaway different from the obvious "algorithms can have typical-case performance which is much better than their worst-case performance"?

Representational alignment

The Platonic Representation Hypothesis

Discussion questions:

What is the new[1] hypothesis that paper promotes, versus what are the observations already established by prior literature? What evidence do they cite for their hypothesis? What distinguishes their hypothesis from merely "models converge to shared representations"?

In-context learning

In-context Learning and Induction Heads

Discussion questions:

What is an induction head, and what relationship does the paper draw between induction-head formation and in-context learning over training? Why treat the simultaneity as evidence of a mechanistic link rather than coincidence?

→ *Further reading*: Supplementary Material, p. 489

NOTES

1. One could argue that this hypothesis isn't novel either, see e.g. the natural abstraction hypothesis which significantly predates this, but this paper was the first major academic paper to promote the hypothesis.

CHAPTER B.3

Singular Learning Theory

Kai Ogden (University of Oxford), Matthew Farrugia-Roberts (University of Oxford), Zach Furman (The University of Melbourne)

Singular learning theory (SLT) places degeneracy as a core part of understanding how neural networks learn. We cover the parameter-function map, the meaning of degeneracy through the local learning coefficient, to Watanabe's free energy formula and Bayesian phase transitions.

Source: iliad-intensive.org/learning/singular-learning-theory · CC BY 4.0 · Slides: online

In this chapter

Introduction 64
1. Preliminaries 66
  1.1 Neural networks and parameter–function maps 66
  1.2 Supervised deep learning and loss functions 69
  1.3 Statistical models and parameter–distribution maps 70
  1.4 Bayesian deep learning 73
2. What is degeneracy? 75
  2.1 A definition of degeneracy 75
  2.2 Degeneracy and continuous symmetries 77
  2.3 Localised degeneracy 80
  2.4 Degeneracy and information singularities 82
  2.5 Degeneracy and the loss landscape 85
3. The degeneracy hierarchy 88
  3.1 The local learning coefficient via volume scaling asymptotics 88
  3.2 Perspectives on the local learning coefficient 92
  3.3 Local learning coefficients of deep linear networks 94
4. Degeneracy and Bayesian deep learning 95
  4.1 Watanabe's free energy formula 95
  4.2 Bayesian phase transitions 97
5. Further readings 491
  5.1 Other introductions to singular learning theory 491
  5.2 Recent work on singular deep learning 492

VIDEO *Deep Learning Is Singular – Here's What That Means* · youtu.be/SzVmiN8BvCc

> *Thou shalt have the power to degenerate —On the Dignity of Man,* 1496

Introduction

Singular learning theory (SLT) is a theory of learning that accounts for *degeneracy* in neural networks. What is degeneracy? Let us first appeal to the Cambridge dictionary, documenting some features of typical uses of the word within mathematics:

> *Degenerate* (of an equation, curve, line, etc.) unusual or complicated in some way compared to other equations, curves, lines, etc. of a similar type, especially because a variable or parameter is zero.

On the other hand, we should recall that deep learning has as many roots in neuroscience as in mathematics. And as they say, "neural networks are *grown*." Perhaps we should also hear the biologist's definition:

> *Degenerate* (of an organism, chromosome, etc.) simpler than a form that previously existed because of no longer having a particular structure.

Actually, *both* of these definitions are apt. Within a typical neural network architecture, certain weight vectors correspond to neural networks from simpler architectures (often due to certain weights being zero). Structurally, these neural networks are simpler in form than their neighbours. Mathematically, they complicate the relationship between the neural network's parameter space and the resulting space of functions. In turn, learning in neural networks is substantially richer than learning in classical statistical models.

Singular learning theory is a framework for understanding learning that places degeneracy at the centre. The framework leverages powerful tools from algebraic geometry to understand, characterise, and resolve degeneracies, revealing their impact on learning. Unfortunately, the price of this mathematical power is that following the research literature without a deep background in pure mathematics is challenging.

That is where this tutorial comes in. We aim to distil the essence of singular learning theory—the notion of degeneracy and its role in learning—laying bare through simple examples and structured exercises the core intuitions driving research in the field. We believe understanding this essence requires no more than undergraduate-level mathematics. Moreover, to anyone armed with these core intuitions, understanding and contributing to research on the relationship between degeneracy and deep learning is within reach.

Contents. Specifically, the tutorial comprises the following four technical sections, each with definitions and guided exercises.

1. "Section 1" reviews core deep learning concepts, defining our notation and some running examples. The central concept is that of the parameter–function map.
2. "Section 2" defines degeneracy of parameter–function maps and relates it to symmetries, singularities of the Fisher information, and degenerate critical points of the loss landscape.

3. "Section 3" derives the local learning coefficient (LLC) as a quantitative measure of the degree of degeneracy of a critical point via a volume scaling approach, and surveys several other perspectives on the quantity.
4. "Section 4" presents Watanabe's free energy formula connecting the LLC to Bayesian learning and discusses its implications.

We conclude with Section 5, providing pointers for further study and surveying the literature on SLT and deep learning.

Prerequisites.

The following is an indicative list of mathematical concepts that will be helpful for reading the tutorial and completing the exercises.

- *Linear algebra:* vectors, matrices, rank, orthogonal matrices, rank–nullity, positive definiteness, eigenvalues, spectral decomposition.
- *Calculus:* partial derivatives, gradient, directional derivative, chain rule, Hessian, second-order Taylor expansion and remainder.
- *Integration and analysis:* multivariate integrals, change of variables, volume in $\mathbb{R}^d$, asymptotic notation (big-O, little-o), computing basic limits and integrals.
- *Probability:* probability simplex $\Delta(\cdot)$, conditional probability, probability density functions, independence, expectation, Bayes' rule, Gaussians, law of large numbers.

Section 1 reviews the basic framework of deep learning as parametric function approximation or statistical inference (parameter–function maps, deep linear networks, multilayer perceptrons, loss functions, likelihood) along with Bayesian inference (prior, posterior, partition function, Bayesian free energy).

Readers with more advanced backgrounds may appreciate occasional references to topics from algebraic geometry, fractal geometry, or statistical physics. However, these references are tangential and readers without these backgrounds can safely skip them.

Fast-track.

To paraphrase Euclid, there is no royal road to algebro-geometric learning theory. However, it is possible to get a bird's-eye view and the most important intuitions in a comparably short time. If this is what you are looking for, we recommend the following route through the tutorial. Assuming you are already somewhat comfortable with deep learning and Bayesian inference, skip Section 1, and refer back only as needed. Then, proceed as follows.

1. To understand parameter–function map versus loss landscape degeneracy: Read Section 2.1 and complete Exercises 2.1 and 2.2. Complete either Exercise 2.9 or Exercise 2.10. Read Section 2.5 and complete your choice of Exercise 2.14 and/or Exercise 2.15.
2. To understand the local learning coefficient via volume scaling: Read Section 3.1 and complete Exercises 3.1, 3.2, 3.4 and 3.7. Read Section 3.3 and Exercise 3.10.

3. To understand the relation between degeneracy and learning in the Bayesian case: Read all of Section 4 (it is shorter). Complete Exercise 4.2.

Once you are done, we hope you will consider taking the scenic route some other time.

Acknowledgements.

This tutorial was developed for the April 2026 Iliad Intensive. Some exercises were adapted from Furman 2024.

1. Preliminaries

In this section, we introduce basic terminology and notation for supervised deep learning and supervised Bayesian deep learning, along with some example neural network architectures and statistical models that we will repeatedly study throughout the tutorial.

1.1 Neural networks and parameter–function maps

Let $\mathcal{X}$ denote a space of network inputs (e.g., a space of images or token sequences encoded as vectors), and let $\mathcal{Y}$ denote a space of outputs (e.g., a space of numerical scores, class distributions, or next-token distributions).

Informally, a neural network architecture is a specification for how various **parameters** (e.g., neuron connection strengths / weights or neuron activation thresholds / biases) combine with input signals and each other to describe a function from $\mathcal{X}$ to $\mathcal{Y}$.

Formally, a neural network architecture essentially comprises a **parameter–function map** $\Phi : \mathcal{W} \to \mathcal{F}$, where $\mathcal{W} \subseteq \mathbb{R}^d$ is a d-dimensional **parameter space** and $\mathcal{F} \subseteq \mathcal{Y}^{\mathcal{X}}$ is some **hypothesis class** of functions from $\mathcal{X}$ to $\mathcal{Y}$.

It is sometimes convenient to refer to the function $\Phi(w) : \mathcal{X} \to \mathcal{Y}$ as $f_w : \mathcal{X} \to \mathcal{Y}$.

Throughout the rest of this tutorial, we will study many different neural network architectures (parameter–function maps). The rest of this section explores some generally useful examples and some terminological points. To begin with, we have the following remark.

Remark (Parameters). Note that the term "parameter" has two senses:

1. A "parameter" is an individual dimension of parameter space (e.g., "initialise this parameter to the value 0.0," or "this neural network has billions of parameters"). A loose synonym in this case is "weight."
2. A "parameter" is also a particular point in the parameter space (e.g., "this parameter is a local minimum of the loss function," or "the zero locus of this loss function contains a continuum of parameters"). A loose synonym in this case is "weight vector."

Both senses are in common usage in the literature as in this tutorial.

The simplest neural network architecture models a single "neuron" with d inputs $x_1, \dots x_d$. Each input x_i is multiplied by an incoming weight w_i to produce the neuron's output. We formalise this case in Example 1.1.

Example 1.1 (A linear neuron). Let $\mathcal{X} = \mathbb{R}^d$ for some positive integer d and let $\mathcal{Y} = \mathbb{R}$. Define a parameter space $\mathcal{W} = \mathbb{R}^d$. Define a parameter–function map that maps a column vector $w \in \mathcal{W}$ to the function $f_w : \mathbb{R}^d \to \mathbb{R}$ such that for $x \in \mathbb{R}^d$,

$$f_w(x) = w^\top x = \sum_{i=1}^{d} w_i x_i.$$

Let us generalise this basic neural network in three ways: to add multiple outputs, "depth," and non-linearity. First, we can generalise from a scalar output to a vector output as follows.

Example 1.2 (Multi-linear neural network). Let $\mathcal{X} = \mathcal{Y} = \mathbb{R}^m$ for some positive integer m. Define a parameter space $\mathcal{W} = \mathbb{R}^{m^2}$. Let each parameter $w \in \mathcal{W}$ encode an $m \times m$ matrix $W \in \mathbb{R}^{m \times m}$. The parameter–function map transforms each parameter $w \in \mathcal{W}$ to a function $f_w : \mathbb{R}^m \to \mathbb{R}^m$ such that for $x \in \mathbb{R}^m$,

$$f_w(x) = Wx.$$

If we take each row of W to encode the weight vector of a linear neuron (Example 1.1), we see that we have produced m outputs by stacking m independent linear neurons together. Such a group of neurons is called a **layer**.

Remark (Encodings). In Example 1.2, the parameter space is formally $\mathcal{W} = \mathbb{R}^{m^2}$, but we find it convenient to identify each parameter vector $w \in \mathbb{R}^{m^2}$ with the $m \times m$ matrix W it encodes. We would thus write f_W in place of f_w. More generally, whenever the parameters of an architecture naturally decompose into named matrices or vectors, we index the parameter–function map by these structured objects directly rather than by the underlying vector w. The remaining examples in this section demonstrate this convention, and we continue to use it throughout this tutorial.

Next, let's add *depth* to our neural network by composing two layers together.

Example 1.3 (Deep linear network; DLN). Let $\mathcal{X} = \mathcal{Y} = \mathbb{R}^m$ for some positive integer m. Let h be a positive integer and define the parameter space $\mathcal{W} = \mathbb{R}^{2mh}$, with each parameter encoding a pair of matrices $A \in \mathbb{R}^{h \times m}$ and $B \in \mathbb{R}^{m \times h}$. The parameter–function map sends (A, B) to $f_{A,B} : \mathbb{R}^m \to \mathbb{R}^m$ such that for $x \in \mathbb{R}^m$,

$$f_{A,B}(x) = BAx.$$

The above architecture is called a two-layer **deep linear network (DLN)**. The architecture can be interpreted as composing two multi-linear neural networks. The vector of outputs of the neurons of the first network becomes the vector of inputs to the second network. The intermediate output vectors are called **activations**.

Note that if $h \geq m$, the two-layer DLN architecture indexes the same hypothesis class as in Example 1.2, that of linear transforms on $\mathbb{R}^m$. If $h < m$, the hypothesis class includes only transforms with rank up to h.

Finally, we'll add *non-linearity* between pairs of layers. To do so, we introduce a non-linear scalar function $\sigma : \mathbb{R} \to \mathbb{R}$, called an *activation function*, to transform the output of each intermediate neuron. Common examples of activation functions, which we will study in this tutorial, include the following:

1. The **hyperbolic tangent** function $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$.
2. The **rectified linear unit (ReLU)** function $\mathrm{relu}(z) = \max(z, 0)$.

This results in the following non-linear neural network architecture.

Example 1.4 (Multi-layer perceptron; MLP). Define input, output, and parameter spaces as in Example 1.3. Let $\sigma : \mathbb{R} \to \mathbb{R}$ be an activation function. The parameter–function map sends (A, B) to the function $f_{A,B} : \mathbb{R}^m \to \mathbb{R}^m$ such that for $x \in \mathbb{R}^m$,

$$f_{A,B}(x) = B \cdot \sigma(Ax),$$

where we lift the activation function to operate element-wise over column vectors $Ax \in \mathbb{R}^h$.

This kind of architecture is called a **multi-layer perceptron (MLP).** Note that the two-layer DLN is recovered if we use the identity function as an activation function. However, if we use a non-linear activation function, we can index a much richer hypothesis class.

The expressivity of these architectures is limited, however, by the omission of a basic detail—the inclusion of a bias parameter for each neuron. We invite the reader to correct this omission as our first exercise, which serves as a chance to familiarise oneself with the concept of a parameter–function map.

Exercise 1.1 (Biased neurons). A biased neuron is a neuron with an additional parameter that is added to the weighted sum of its inputs before it produces its output. Bias parameters allow each layer to represent an *affine* transform, rather than just a linear transform. For each of Examples 1.1, 1.2, 1.3 and 1.4, extend the example to use biased neurons. Precisely define the parameter space and the parameter–function map in each case.

SOLUTION, p. 561

Modern deep learning leverages more involved parameter–function maps. Neural network architectures are typically defined in a modular fashion, including linear (or affine) layers and non-linear layers like the above, along with more specialised layers (well-known examples including convolutional layers, residual layers, and attention layers).

1.2 Supervised deep learning and loss functions

In a typical supervised deep learning setting, we are given a data set of pairs $(x^{(1)}, y^{(1)})$, $\ldots$, $(x^{(n)}, y^{(n)}) \in \mathcal{X} \times \mathcal{Y}$. These pairs exemplify inputs and their corresponding (possibly noisy) outputs from an unknown target function $f : \mathcal{X} \to \mathcal{Y}$ which we would like to approximate using a neural network. That is, we would like to find a parameter $w \in \mathcal{W}$ such that the corresponding function f_w approximates the target function f.

Formally, given an example $(x, y) \in \mathcal{X} \times \mathcal{Y}$, define a **per-example loss** function $L_{x,y} : \mathcal{W} \to \mathbb{R}$ to measure the deviation of $f_w(x)$ from the output y. A typical loss function for vector outputs $\mathcal{Y} \subset \mathbb{R}^m$ is to use the **squared error** loss

$$L_{x,y}(w) = \|f_w(x) - y\|^2.$$

If $\mathcal{Y} = \Delta^{m-1}$ is a space of discrete distributions over m objects, it is typical to use **cross entropy** loss

$$L_{x,y}(w) = -\sum_{i=1}^{m} y(i) \log f_w(i \mid x).$$

Given a per-example loss function and a **data distribution** $q \in \Delta(\mathcal{X} \times \mathcal{Y})$ over the space of examples (representing our possibly-noisy target function), we formalise the objective of supervised learning as finding $w \in \mathcal{W}$ so as to minimise the **population loss** function $L : \mathcal{W} \to \mathbb{R}$ defined such that

$$L(w) = \mathbb{E}_{(x,y)\sim q}[L_{x,y}(w)].$$

The population loss aggregates differences on outputs for individual inputs into an overall measure of difference between f_w and the target function.

In practice, we often can't evaluate the population loss over the entire input/output space. We instead optimise an estimator based on our data set of n input–output pairs assumed to be sampled independently and identically from q. Define an **empirical loss** function $L_n : \mathcal{W} \to \mathbb{R}$ such that

$$L_n(w) = \frac{1}{n} \sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w).$$

If the per-example loss is squared error, the empirical loss is known as the **mean squared error** objective.

$$L_n(w) = \frac{1}{n} \sum_{i=1}^{n} \left\| f_w(x^{(i)}) - y^{(i)} \right\|^2.$$

Exercise 1.2 (Empirical loss and population loss). Fix $w \in \mathcal{W}$. Prove the following properties of the relationship between the empirical loss $L_n(w)$ and the population loss $L(w)$.

(a) For any n, the empirical loss is an unbiased estimator of the population loss. That is,

$$\mathbb{E}_{(x^{(i)},y^{(i)})\sim q}[L_n(w)] - L(w) = 0.$$

(b) As $n \to \infty$, the empirical loss converges almost surely to the population loss.

SOLUTION, p. 561

Given a loss function, a **training algorithm** is a search algorithm that aims to find $w \in \mathcal{W}$ such that the loss is approximately minimised. Most modern deep learning algorithms are variants of **stochastic gradient descent (SGD)**, which implements an iterative gradient-based local search of the parameter space using empirical loss on subsamples of the data set.

Through many impressive feats of computer science and hardware/software engineering, we are able to run such training algorithms to find low-loss parameters within parameter spaces with billions of dimensions. The details are vitally important to the success of modern deep learning, but are beyond the scope of this tutorial. We only mention SGD in order to emphasise that *any local search method depends intimately on the properties of the parameter–function map.*

1.3 Statistical models and parameter–distribution maps

The previous sections introduce neural networks and supervised deep learning within a framework of function approximation. An alternative approach is to view supervised learning as statistical inference, and particularly Bayesian inference. This statistical framework is commonly used within the SLT literature, and we will encounter it within this tutorial.

The first step to upgrading our framework is to move from talking about functions to talking about conditional distributions. For each neural network, we should be able to determine not just the output corresponding to each input, but a probability density measuring how likely each output is given each input.

Formally, let $\mathcal{D} \subseteq \Delta(\mathcal{Y})^{\mathcal{X}}$ represent a class of conditional distributions over $\mathcal{Y}$. A distributional neural network architecture comprises a **parameter–distribution map** $\Psi : \mathcal{W} \to \mathcal{D}$. It is sometimes convenient to refer to the conditional distribution $\Psi(w) : \mathcal{X} \to \Delta(\mathcal{Y})$ using the notation $p_w : \mathcal{X} \to \Delta(\mathcal{Y})$. The probability mass/density of a given output $y \in \mathcal{Y}$ conditional on a given input $x \in \mathcal{X}$ is typically denoted either $p_w(y \mid x)$ or $p(y \mid x, w)$.

Sometimes, defining a parameter–distribution map is simply a re-framing. For example, in image classification, our output space $\mathcal{Y}$ was a space of distributions over image classes. Let $\mathcal{C}$ represent the underlying class space, then $\mathcal{Y} = \Delta(\mathcal{C})$ and we can view our parameter–function map $\Phi : \mathcal{W} \to \mathcal{F}$ as a parameter–distribution map $\Psi : \mathcal{W} \to \mathcal{D}$ with $\mathcal{D} = \mathcal{F} \subseteq \mathcal{Y}^{\mathcal{X}} = \Delta(\mathcal{C})^{\mathcal{X}}$.

In other cases, we can easily construct a parameter–distribution map from a parameter–function map by introducing a **noise model**. For example, in regression with an output space $\mathcal{Y} = \mathbb{R}^m$, given a parameter–function map $w \mapsto f_w$ we can define a parameter–distribution map $\Psi : \mathcal{W} \to \Delta(\mathcal{Y})^{\mathcal{X}}$ such that, for $w \in \mathcal{W}$ and $x \in \mathcal{X}$,

$$p(y \mid x, w) = \frac{1}{(2\pi)^{m/2}} \exp\left(-\frac{\|y - f_w(x)\|^2}{2}\right).$$

The above **Gaussian noise model** amounts to modelling each output y as drawn given x, w according to the rule $y = f_w(x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I_m)$.

As the following exercises explore, performing statistical inference according to the principle of maximum likelihood aligns with the loss-minimisation perspective (given an appropriate choice of loss function).

Exercise 1.3 (Maximum likelihood as MSE minimisation). Consider a regression problem with $\mathcal{X} = \mathcal{Y} = \mathbb{R}^m$. Assume we have a neural network architecture with a parameter space $\mathcal{W}$. Form a corresponding statistical model using the Gaussian noise model of (11). Let $(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)}) \in \mathcal{X} \times \mathcal{Y}$ be a data set of n regression examples.

(a) Write an expression for the conditional probability of observing the labels $Y_n = (y^{(1)}, \ldots, y^{(n)})$ given the inputs $X_n = (x^{(1)}, \ldots, x^{(n)})$ and some fixed parameter $w \in W$, assuming the labels are sampled independently.

(b) Denote the resulting quantity by $p(Y_n \mid X_n, w)$. It is called the **likelihood** of the data set according to the model w. Compute and simplify the **negative log-likelihood**, given by $-\log p(Y_n \mid X_n, w)$.

(c) Show that if L_n is the mean squared error loss function from (9), then

$$\arg\max_{w \in \mathcal{W}} p(Y_n \mid X_n, w) = \arg\min_{w \in \mathcal{W}} L_n(w).$$

SOLUTION, p. 562

Exercise 1.4 (Maximum likelihood as cross entropy minimisation). Consider a classification problem with $\mathcal{Y} = \Delta^{m-1}$. Assume we have a neural network architecture with a parameter space $\mathcal{W}$. As discussed, the parameter–function map in this case corresponds to a parameter–distribution map to conditional distributions over the underlying discrete class space $\mathcal{C} = \{1, \ldots, m\}$. Let $(x^{(1)}, c^{(1)}), \ldots, (x^{(n)}, c^{(n)}) \in \mathcal{X} \times \mathcal{C}$ be a data set of n labelled classification examples. For each $c^{(i)} \in \mathcal{C}$ define

$$y^{(i)} = \delta_{c^{(i)}} \in \Delta^{m-1}$$

to be the corresponding Dirac distribution (with probability mass 1 concentrated on $c^{(i)}$).

(a) Write an expression for the likelihood of the data set, $p(C_n \mid X_n, w)$, that is, the conditional probability of observing the labels $C_n = (c^{(1)}, \ldots, c^{(n)})$ given the inputs $X_n = (x^{(1)}, \ldots, x^{(n)})$ and some fixed parameter $w \in W$. Assume the labels are sampled

independently.

(b) Show that the negative log-likelihood

$$-\log p(C_n \mid X_n, w) = -\sum_{i=1}^{n} \log f_w(c^{(i)} \mid x^{(i)}).$$

(c) Using (12), show that if the per-example loss function for the optimisation problem is cross entropy loss (6), then

$$\arg\max_{w \in \mathcal{W}} p(C_n \mid X_n, w) = \arg\min_{w \in \mathcal{W}} L_n(w).$$

SOLUTION, p. 562

Exercise 1.5 (Maximum likelihood as loss minimisation, in general). Consider a neural network architecture with a parameter space $\mathcal{W}$, parameter–function map Φ, and output space $\mathcal{Y} = \mathbb{R}^m$. Let $L_{x,y} : \mathcal{W} \to \mathbb{R}$ be a per-example loss function. Assume that for each $x \in \mathcal{X}$ and $w \in \mathcal{W}$, the function $y \mapsto \exp(-L_{x,y}(w))$ is integrable over $\mathcal{Y}$. Define a corresponding statistical model with parameter–distribution map Ψ mapping $w \in \mathcal{W}$ to $p_w : \mathcal{X} \to \Delta(\mathcal{Y})$ such that for all $y \in \mathcal{Y}$, $x \in \mathcal{X}$, and $w \in \mathcal{W}$, we have

$$p(y \mid x, w) = \frac{\exp(-L_{x,y}(w))}{Z(x, w)}, \qquad Z(x, w) = \int_{\mathcal{Y}} \exp(-L_{x,z}(w))\, dz.$$

We re-use the notation X_n, Y_n from Exercise 1.3, and assume that labels are sampled independently.

(a) Show that the negative log-likelihood decomposes as

$$-\log p(Y_n \mid X_n, w) = \sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w) + \sum_{i=1}^{n} \log Z(x^{(i)}, w).$$

(b) Now suppose the loss function takes the form $L_{x,y}(w) = \ell(f_w(x) - y)$ for some integrable function $\ell : \mathbb{R}^m \to \mathbb{R}$. Show that $Z(x, w)$ is independent of w, and conclude that

$$\arg\max_{w \in \mathcal{W}} p(Y_n \mid X_n, w) = \arg\min_{w \in \mathcal{W}} L_n(w).$$

SOLUTION, p. 563

Remark (Negative log-likelihood loss). Exercises 1.3, 1.4 and 1.5 nominally show that some maximum likelihood estimation problems have the same sets of ideal solutions as some carefully-chosen optimisation problems ($\arg\min = \arg\max$). In general, that the global optima for two optimisation problems coincide is insufficient to show that practical optimisation methods will always find the same solutions: practical optimisation methods don't always find global optima. However, in the above cases, your derivation likely reveals a stronger result: the optimisation landscapes for the

function approximation problem are related to the optimisation landscape for the maximum likelihood estimation problem by a strictly monotonically decreasing transform (an affine transform of a logarithm).

1.4 Bayesian deep learning

The principle of maximum likelihood, explored in the preceding exercises, is one approach to statistical inference. An alternative approach is **Bayesian inference**, which treats w as a random variable and maintains a probability distribution over $\mathcal{W}$ reflecting our uncertainty about which parameter best explains the data. The majority of theoretical results in SLT have been developed within this setting (Watanabe 2009; Watanabe 2018, cf.,).

Bayesian learning begins with a **prior** distribution $\varphi \in \Delta(\mathcal{W})$, a probability density on $\mathcal{W}$ representing our beliefs about the parameter w before observing any data. Throughout this tutorial, we assume φ is positive and smooth on $\mathcal{W}$.

Definition 1.5 (Posterior distribution, partition function, free energy). Given a data set of n input–output pairs $(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)}) \in \mathcal{X} \times \mathcal{Y}$, Bayes' rule updates the prior into the **posterior distribution** $\pi_n \in \Delta(\mathcal{W})$, a probability density on $\mathcal{W}$ defined by

$$\pi_n(w) = \frac{1}{Z_n} \varphi(w) \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, w)$$

where $Z_n \in \mathbb{R}$ is a normalising constant called the **partition function** (or **marginal likelihood** or **model evidence**),

$$Z_n = \int_{\mathcal{W}} \varphi(w) \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, w)\, dw.$$

We often work with the **(Bayesian) free energy**, the negative log of the model evidence,

$$F_n = -\log Z_n.$$

The posterior re-weights the prior by the likelihood of the observed data: parameters w under which the data is more probable receive higher posterior density, while parameters under which the data is improbable are down-weighted.

The partition function Z_n is the marginal probability of observing the data set, averaged over all parameters weighted by the prior. It quantifies how well the statistical model *as a whole* predicts the observed data. The free energy is an inverted measure of how well the model fits the data.

Exercise 1.6 (Bayesian posterior as a Gibbs distribution). To interpret Bayesian inference in terms more similar to function approximation, we introduce a loss function based on the **negative log-likelihood**,

$$L_{x,y}(w) = -\log p(y \mid x, w),$$
$$L_n(w) = -\frac{1}{n}\sum_{i=1}^{n} \log p(y^{(i)} \mid x^{(i)}, w).$$

Here, $p(y \mid x, w)$ is the conditional density from the parameter–distribution map and $(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)}) \in \mathcal{X} \times \mathcal{Y}$ is a data set of n examples. Show that

$$\pi_n(w) = \frac{1}{Z_n}\varphi(w)\exp\bigl(-nL_n(w)\bigr)$$

and

$$Z_n = \int_{\mathcal{W}} \varphi(w)\exp\bigl(-nL_n(w)\bigr)\,dw.$$

SOLUTION, p. 564

The **Bayesian deep learning process** is the process of re-weighting each individual parameter vector in response to seeing an increasing amount of data, producing the sequence of belief distributions $\varphi, \pi_1, \pi_2, \ldots \in \Delta(\mathcal{W})$. Over time, the posterior will concentrate around parameters which provide a good fit for the data (though there is more to the story in the singular case, as we will see in later sections).

Bayesian deep learning is a *global search* in that all parameter vectors are considered in parallel. This makes exact Bayesian learning computationally intractable for non-trivial models, but more analytically tractable than SGD in general. Moreover, as we will explore in later sections, the dynamics of Bayesian learning still reflect the local geometry of the parameter–distribution map.

In preparation for exploring this connection between geometry and learning, we finally introduce localised variants of the partition function and free energy, where the integral in (14) is restricted to a certain neighbourhood in parameter space.

Definition 1.6 (Local partition function and local free energy). Given a neighbourhood $\mathcal{U} \subseteq \mathcal{W}$, the **local partition function** and **local free energy** are

$$Z_n(\mathcal{U}) = \int_{\mathcal{U}} \varphi(w)\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, w)\,dw, \qquad F_n(\mathcal{U}) = -\log Z_n(\mathcal{U}).$$

The local partition function (free energy) measures how well (poorly) parameters within a given neighbourhood collectively explain the data. Note, $Z_n(\mathcal{W}) = Z_n$ and $F_n(\mathcal{W}) = F_n$.

Exercise 1.7 (Local posterior mass and local free energy). Let $\mathcal{U}, \mathcal{V} \subseteq \mathcal{W}$ be two neighbourhoods of parameter space. Let $\pi_n(\mathcal{U}) = \int_{\mathcal{U}} \pi_n(w)\, dw$ denote the posterior mass of a neighbourhood. Show the following.

(a) $\pi_n(\mathcal{U}) = \dfrac{Z_n(\mathcal{U})}{Z_n}$ (the same holds for $\mathcal{V}$).

(b) $\log \dfrac{\pi_n(\mathcal{U})}{\pi_n(\mathcal{V})} = F_n(\mathcal{V}) - F_n(\mathcal{U})$.

In conclusion, the posterior odds ratio of the two regions $\pi_n(\mathcal{U})/\pi_n(\mathcal{V})$ depends exponentially on the difference between their local free energies.

SOLUTION, p. 564

Exercise 1.8 (Partition function of partitioned parameter space). Let $\mathcal{U}_1, \ldots, \mathcal{U}_k \subseteq \mathcal{W}$ be disjoint subsets with $\mathcal{W} = \mathcal{U}_1 \sqcup \cdots \sqcup \mathcal{U}_k$.

(a) Show that the partition function decomposes as $Z_n = \sum_{j=1}^{k} Z_n(\mathcal{U}_j)$, and hence

$$F_n = -\log \sum_{j=1}^{k} e^{-F_n(\mathcal{U}_j)}.$$

(b) Show that if $F_n(\mathcal{U}_1) < F_n(\mathcal{U}_j)$ for all $j \neq 1$, then

$$F_n(\mathcal{U}_1) - \log k < F_n < F_n(\mathcal{U}_1).$$

In conclusion, the overall free energy is dominated by the region(s) with the lowest local free energy.

SOLUTION, p. 564

2. What is degeneracy?

In this section, we define degeneracy as a property of parameter–function maps, and study the relationship between this property and other similar notions.

2.1 A definition of degeneracy

Consider a neural network architecture with parameter space $\mathcal{W} \subseteq \mathbb{R}^d$ and parameter–function map $\Phi : \mathcal{W} \to \mathcal{F}$ ($w \mapsto f_w$). Say the parameter–function map is **degenerate at** $w \in \mathcal{W}$ if there is a non-zero vector $v \in \mathbb{R}^d$ such that the directional derivative of the parameter–function map in the direction v is zero:

$$D_v \Phi(w) = 0.$$

The directional derivative $D_v \Phi(w)$ for non-zero $v \in \mathbb{R}^d$ may be defined variously as

$$D_v \Phi(w) = \frac{v}{\|v\|} \cdot \nabla \Phi(w) = \sum_{i=1}^{d} \frac{v_i}{\|v\|} \frac{\partial \Phi}{\partial w_i}(w) = \lim_{\epsilon \to 0} \frac{\Phi(w + \epsilon v) - \Phi(w)}{\epsilon \|v\|}.$$

We conventionally define $D_0\Phi(w) = 0$. Note that w is fixed, but the directional derivative is still a function (of the same type as $\Phi(w)$; $D_v\Phi(w) : \mathcal{X} \to \mathcal{Y}$), since it represents an infinitesimal change in $\Phi(w)$ in response to a change in w in the direction v. In (19) we mean that this function is identically zero for all inputs $x \in \mathcal{X}$ for the given $w \in \mathcal{W}$.

Intuitively, the parameter–function map is degenerate at $w \in \mathcal{W}$ if there is a direction in which we can infinitesimally perturb w without changing the function f_w.

This definition of degeneracy applies to a single point in parameter space. As we shall soon see, it may be the case that the parameter–function map is degenerate at some points but not at others. We can clarify the situation with new terminology.

- We say that a parameter–function map is **somewhere degenerate** if it is degenerate at *any* point in parameter space.
- We say that a parameter–function map is **everywhere degenerate** if it is degenerate at *all* points in parameter space.

By convention, if we say that the parameter–function map is simply **degenerate** (without specifying somewhere, everywhere, or at a particular point), we mean that it is *somewhere degenerate*.

The following exercises explore this definition in some toy parametrisations of a simple function class (constants), displaying in the simplest possible setting some basic forms of degeneracy that will arise repeatedly throughout the tutorial.

Exercise 2.1 (Parametrising the space of constants). Let $\mathcal{X} = \{*\}$ and $\mathcal{Y} = \mathbb{R}$, so that we have a hypothesis class of constants. Consider the scalar parameter space $\mathcal{W} = \mathbb{R}$ and the parameter–function map that maps w to $f_w = w$ (the output is just the parameter itself).

(a) What is the directional derivative of the parameter–function map in direction $v = 1$?

HINT, p. 565

(b) At which points in the parameter space is this parameter–function map degenerate, if any?

SOLUTION, p. 565

Exercise 2.2 (Degeneracy from raising a parameter to a power). Again, consider a hypothesis class of constants and the scalar parameter space $\mathcal{W} = \mathbb{R}$. This time, define a parameter–function map that maps w to $f_w = w^3$.

(a) Show that this architecture indexes exactly the same hypothesis class as the architecture described in Exercise 2.1.

(b) What is the directional derivative of the parameter–function map in direction $v = 1$?

HINT, p. 565

(c) At which points in the parameter space is the parameter–function map degenerate, if any?

SOLUTION, p. 565

Exercise 2.3 (Degeneracy from multiplying two parameters together). Again, consider a hypothesis class of constants. Consider this time the two-dimensional parameter space $\mathcal{W} = \mathbb{R}^2$. Define a parameter–function map that maps $w = (a, b)$ to $f_w = a \cdot b$.

(a) Show that this architecture indexes exactly the same hypothesis class as the architecture described in Exercise 2.1.

(b) What is the directional derivative of the parameter–function map in direction $v = (1, 0)$?

HINT, p. 566

(c) What is the directional derivative of the parameter–function map in direction $v = (0, 1)$?

HINT, p. 566

(d) At which points in the parameter space is the parameter–function map degenerate *in these directions,* if any?

(e) At which points in the parameter space is the parameter–function map degenerate, if any?

SOLUTION, p. 566

Exercise 2.4 (Degeneracy from adding two parameters together). Again, consider a hypothesis class of constants. Consider this time the two-dimensional parameter space $\mathcal{W} = \mathbb{R}^2$. Define a parameter–function map that maps $w = (a, b)$ to $f_w = a + b$.

(a) Show that this architecture indexes exactly the same hypothesis class as the architecture described in Exercise 2.1.

(b) What is the directional derivative of the parameter–function map in direction $v = (1, -1)$?

(c) At which points in the parameter space is the parameter–function map degenerate, if any?

SOLUTION, p. 567

2.2 Degeneracy and continuous symmetries

A common source of degeneracy in neural network architectures is the presence of continuous symmetries of the parameter–function map. A continuous symmetry traces out a curve of functionally equivalent parameters. The tangent to this curve is a degenerate direction. In this section, we will explore this kind of symmetry and some examples from deep learning.

A **symmetry** of a parameter–function map Φ is a transformation of parameter space that maps parameters while preserving the implemented function. That is, a transformation $T : \mathcal{W} \to \mathcal{W}$ is a symmetry if for all $w \in \mathcal{W}$, we have $\Phi(w) = \Phi(T(w))$ ($f_w = f_{T(w)}$).

A **continuous symmetry** of a parameter–function map Φ is a family of transformations $\{T_t : \mathcal{W} \to \mathcal{W}\}_{t\in\mathbb{R}}$ indexed by a parameter $t \in \mathbb{R}$, such that

1. T_t is a symmetry ($\Phi \circ T_t = \Phi$) for all $t \in \mathbb{R}$,
2. T_0 is the identity transformation on $\mathcal{W}$, and
3. The map $t \mapsto T_t(w)$ is differentiable for each $w \in \mathcal{W}$.

The continuous symmetry is **trivial at** w if $\frac{d}{dt}\big|_{t=0} T_t(w) = 0$, or else **non-trivial at** w.

Exercise 2.5 (Continuous symmetry example). Recall the parameter–function map from Exercise 2.4, with $\mathcal{W} = \mathbb{R}^2$ and with $(a, b) \in \mathcal{W}$ mapping to the constant function $f_{a,b} = a + b$. Consider the family of transformations $T_t : \mathcal{W} \to \mathcal{W}$ for $t \in \mathbb{R}$ such that

$$T_t(a, b) = (a + t, b - t).$$

(a) Describe the effect of this family of transformations on the parameter space.

(b) Show that this family of transformations is a continuous symmetry.

(c) Show that this continuous symmetry is non-trivial everywhere.

SOLUTION, p. 567

At each parameter, a non-trivial continuous symmetry traces out a curve of functionally equivalent parameters. This indicates the presence of a direction in parameter space in which the function does not change. The following proposition formalises this connection between continuous symmetries and degeneracy.

Proposition 2.1. If a parameter–function map Φ admits a continuous symmetry $\{T_t\}$ that is non-trivial at w, then Φ is degenerate at w.

Proof

Define the curve in parameter space traced by the continuous symmetry, $\gamma : \mathbb{R} \to \mathcal{W}$ with $\gamma(t) = T_t(w)$. Fix $t_0 \in \mathbb{R}$. Since T_t is a continuous symmetry, the derivative along the curve at t_0 vanishes:

$$\begin{aligned} D_{\gamma'(t_0)}\Phi(\gamma(t_0)) &\propto \gamma'(t_0) \cdot \nabla\Phi(\gamma(t_0)) \\ &= \left.\frac{d}{dt}\right|_{t=t_0} \Phi(\gamma(t)) \quad \text{(chain rule)} \\ &= \left.\frac{d}{dt}\right|_{t=t_0} \Phi(w) \quad \text{(symmetry, } \Phi \circ T_t = \Phi\text{)} \\ &= 0. \quad (\Phi(w) \text{ is constant in } t) \end{aligned}$$

In particular, at $t = 0$, we have $0 = D_{\gamma'(0)}\Phi(\gamma(0)) = D_{\gamma'(0)}\Phi(w)$ since $\gamma(0) = w$ by identity. Since T_t is non-trivial at w, $\gamma'(0)$ is non-zero, so $\gamma'(0)$ is a degenerate direction at w.

Corollary 2.2. If a parameter–function map Φ admits a continuous symmetry $\{T_t\}$ that is non-trivial at every parameter $w \in \mathcal{W}$, then Φ is everywhere degenerate.

The following exercises exhibit some continuous symmetries in two neural network architectures, a small ReLU MLP and a toy autoencoder. Similar architectures are studied in more detail from an SLT perspective by Carroll 2021 and Chen et al. 2023 respectively.

Exercise 2.6 (ReLU scaling symmetry). Consider a two-layer MLP (Example 1.4) with $\sigma = \mathrm{relu}$, scalar inputs and outputs ($m = 1$), and a single hidden unit ($h = 1$). The parameter space is $\mathcal{W} = \mathbb{R}^2$ with parameters (a, b), and the parameter–function map is $f_{a,b}(x) = b \cdot \mathrm{relu}(ax)$.

(a) Show that relu is *positively homogeneous*: $\mathrm{relu}(\alpha z) = \alpha\,\mathrm{relu}(z)$ for all $\alpha > 0$ and $z \in \mathbb{R}$.

(b) Define $T_t(a, b) = (e^t a,\ e^{-t} b)$ for $t \in \mathbb{R}$. Show that $\{T_t\}$ is a continuous symmetry of this parameter–function map.

(c) For a fixed parameter $(a, b) \neq (0, 0)$, compute the degenerate direction arising from this symmetry at (a, b).

(d) Show that the symmetry is trivial at the origin. Is this parameter–function map degenerate at the origin?

SOLUTION, p. 567

Exercise 2.7 (Rotation symmetry in a linear autoencoder). Consider a *linear autoencoder* with bottleneck dimension h and ambient dimension $m \geq h$. The parameter space encodes a single matrix $W \in \mathbb{R}^{h \times m}$ and the parameter–function map sends W to the function $\Phi(W) = f_W : \mathbb{R}^m \to \mathbb{R}^m$ where

$$f_W(x) = W^\top W x$$

for $x \in \mathbb{R}^m$. Here, W serves simultaneously as encoder ($x \mapsto Wx \in \mathbb{R}^h$) and decoder ($z \mapsto W^\top z \in \mathbb{R}^m$).

(a) Show that for any orthogonal matrix $R \in \mathbb{R}^{h \times h}$ (i.e., any $h \times h$ matrix such that $R^\top R = I$), the map $T_R(W) = RW$ is a symmetry.

(b) Specialise to $h = 2$. Using the rotation matrices

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix},$$

show that $T_\theta(W) = R(\theta)W$ defines a continuous symmetry.

(c) For a fixed parameter $W \in \mathbb{R}^{2\times m}$, compute the degenerate direction that arises from this symmetry.

(d) For general bottleneck dimension h, how many independent continuous symmetries does the orthogonal group $O(h)$ contribute?

HINT, p. 568

SOLUTION, p. 568

2.3 Localised degeneracy

We have seen that a non-trivial continuous symmetry traces out curves of functionally equivalent parameters throughout the entire parameter space, and the tangent directions to these curves are degenerate directions at every point. However, not all degenerate directions arise from globally continuous symmetries. In fact, the more interesting case is when degeneracy affects only some parameters and not others.

Within certain subsets of parameter space, many neural network architectures (including those studied in Exercises 2.6 and 2.7) exhibit degenerate directions that cannot be defined in terms of continuous transformations that are globally symmetries. In this section, we explore localised degeneracies of this kind.

We begin with some examples of transformations that are only continuous symmetries within certain subsets of parameter space.

Exercise 2.8 (Localised symmetry example). Recall the parameter–function map from Exercise 2.3, with $\mathcal{W} = \mathbb{R}^2$ and with $(a, b) \in \mathcal{W}$ mapping to the constant function $f_{a,b} = a \cdot b$. Consider the two families of transformations $T_t^{(a)}, T_t^{(b)} : \mathcal{W} \to \mathcal{W}$ for $t \in \mathbb{R}$ such that

$$T_t^{(a)}(a, b) = (a + t, b), \qquad T_t^{(b)}(a, b) = (a, b + t).$$

(a) Describe the effect of each family of transformations on the parameter space.

(b) In which subset of parameter space does each family of transformations constitute a symmetry for all $t \in \mathbb{R}$?

(c) Compare your results to your answer to Exercise 2.3(d).

SOLUTION, p. 569

The above example is technically a two-layer DLN with $m = h = 1$. Let us now explore localised degeneracy in a general two-layer DLN and then in a non-trivial MLP.

Exercise 2.9 (Degeneracy in the two-layer DLN). Consider the two-layer DLN from Example 1.3, with $h = m$. The parameter space encodes two matrices $A, B \in \mathbb{R}^{m\times m}$, and the parameter–function map sends (A, B) to the function $\Phi(A, B) = f_{A,B} : \mathbb{R}^m \to \mathbb{R}^m$ such that $f_{A,B}(x) = BAx$ for $x \in \mathbb{R}^m$.

(a) Let $(\delta A, \delta B)$ be a unit perturbation in parameter space (a unit vector in the underlying parameter space $\mathbb{R}^{2m^2}$, decoded into a pair of matrices). Show that the directional derivative of the parameter–function map at (A, B) in direction $(\delta A, \delta B)$ is the linear

map

$$D_{(\delta A,\delta B)}\Phi(A,B) = B\,\delta A + \delta B\,A.$$

That is, $D_{(\delta A,\delta B)}\Phi(A,B)x = (B\,\delta A + \delta B\,A)x$.

HINT, p. 569

(b) Show that at the zero parameter $(A,B) = (0,0)$, every direction in parameter space is degenerate. Count the number of dimensions in the subspace of degenerate directions.

(c) Suppose both A and B are invertible. Show that $(\delta A, \delta B)$ is a degenerate direction if and only if $\delta B = -B\,\delta A\,A^{-1}$. Count the number of dimensions in the subspace of degenerate directions.

(d) Now consider the general case. Fix A and B, and let $r_A = \operatorname{rank}(A)$ and $r_B = \operatorname{rank}(B)$. Show that the dimensionality of the space of degenerate directions is

$$m^2 + (m - r_A)(m - r_B).$$

HINT, p. 570

SOLUTION, p. 570

Exercise 2.10 (Redundant units in an MLP). Consider a single-hidden-layer MLP with scalar inputs and outputs, h hidden units with activation function $\sigma : \mathbb{R} \to \mathbb{R}$, and an output bias. The parameter space is $\mathcal{W} = \mathbb{R}^{3h+1}$ with parameters $w = (a_1, b_1, c_1, \ldots, a_h, b_h, c_h, d)$, and the parameter–function map is

$$f_w(x) = d + \sum_{i=1}^{h} a_i\,\sigma(b_i x + c_i).$$

Here, for each hidden unit i, a_i is the outgoing weight, b_i is the incoming weight, and c_i is the bias. The output bias is d.

(a) Suppose $a_i = 0$ (unit i has zero outgoing weight). Show that the directions δb_i and δc_i are degenerate at w.

(b) Suppose $b_i = 0$ (unit i has zero incoming weight). Show that the direction $(\delta a_i, \delta d) = (1, -\sigma(c_i))$ (with all other components zero) is degenerate at w.

(c) Suppose $h \geq 2$ and $(b_i, c_i) = (b_j, c_j)$ for some $i \neq j$ (units i and j have the same incoming weights and biases). Show that the direction $(\delta a_i, \delta a_j) = (1, -1)$ (with all other components zero) is degenerate at w.

SOLUTION, p. 572

Remark (Localised symmetries in neural networks). Each degenerate direction identified in the exercises above corresponds to a localised symmetry: a curve of functionally equivalent parameters that continuously extends only within a subset of parameter space.

1. In the DLN, the global change-of-basis symmetry $(A, B) \mapsto (GA, BG^{-1})$ for invertible $G \in \mathbb{R}^{m \times m}$ accounts for m^2 degenerate directions. These directions are present even when A and B have full rank. However, as we showed in Exercise 2.9, when A or B drops rank, $(m - r_A)(m - r_B)$ additional degenerate directions open up. These extra directions are localised to the rank-deficient region of parameter space.
2. In the MLP, at a parameter with $a_i = 0$, the path $t \mapsto (\ldots, a_i, b_i + t, c_i, \ldots)$ traces equivalent parameters within $\{a_i = 0\}$ (since f_w does not depend on b_i when $a_i = 0$), but this does not extend to a symmetry at nearby parameters with $a_i \neq 0$. Similarly, at a parameter with $(b_i, c_i) = (b_j, c_j)$, the path transferring weight between units i and j is a localised version of the sum symmetry from Exercise 2.5.

Remark (Rank of a neural network parameter). In both exercises above, the degree of degeneracy at a parameter is controlled by the amount of redundant capacity in the network. For the two-layer DLN, this is measured by the rank of the product BA: a rank-r linear map can be implemented with a hidden dimension of r, leaving $m - r$ dimensions redundant. For a general single-hidden-layer MLP, this idea generalises in that we can define the rank of a parameter w as the minimum number of hidden units needed to implement f_w (Farrugia-Roberts 2022; Farrugia-Roberts 2024). In the linear case, this "neural network rank" coincides with the matrix rank of BA. In both settings, lower rank corresponds to a higher number of degenerate directions.

Remark (Measure zero degenerate sets). When a parameter–function map is degenerate somewhere but not everywhere, it is often the case that it is degenerate only within a measure-zero subset of parameter space. This means that sampling parameters uniformly at random results in a degenerate parameter with probability zero. However, this does not mean that the degeneracies can be dismissed. The learning process applies a non-random selection pressure and may select parameters from a measure-zero set. Moreover, the existence of degenerate parameters can have practical consequences for nearby non-degenerate parameters, and the collective neighbourhoods of all degenerate parameters comprise a non-measure-zero subset of parameter space.

2.4 Degeneracy and information singularities

In the preceding sections, we studied degeneracy as a property of parameter–function maps. In the statistical framework introduced in Section 1.3, the fundamental object is instead a parameter–distribution map $\Psi : \mathcal{W} \to \mathcal{D}$ $(w \mapsto p_w : \mathcal{X} \to \Delta(\mathcal{Y}))$. In this section, we extend the definition of degeneracy to parameter–distribution maps and connect it to a classical quantity from statistics, the Fisher information matrix. We will show that under mild regularity conditions on the statistical model, the Fisher information matrix is singular at w (has zero eigenvalues) if and only if the parameter–distribution map is degenerate at w.

Given a parameter–distribution map $\Psi : \mathcal{W} \to \mathcal{D}$, say that Ψ is **degenerate at w in direction** v if the conditional density does not change to first order:

$$D_v \Psi(w) = 0.$$

Here, $D_v\Psi(w)$ is not a conditional distribution but a signed difference between conditional distributions, effectively a real function of x and y (in particular, it may include negative density changes). We mean that this function is zero for all $x \in \mathcal{X}$ and all $y \in \mathcal{Y}$.

Equation 21 extends the definition of degeneracy from Equation 19. If Ψ is constructed from a parameter–function map Φ via a noise model (as in Section 1.3), then degeneracy of Φ at w in direction v implies degeneracy of Ψ at w in direction v, since $p(y \mid x, w)$ depends on w only through $f_w(x)$.

To relate degeneracy to the Fisher information matrix, we introduce two standard definitions.

Definition 2.3 (Score function). Let $\Psi : \mathcal{W} \to \mathcal{D}$ be a parameter–distribution map with densities $p(y \mid x, w)$ that are positive and differentiable in w. The **score function** at w is

$$s(x, y, w) = \nabla_w \log p(y \mid x, w) \in \mathbb{R}^d.$$

The **directional score** in non-zero direction $v \in \mathbb{R}^d$ is

$$s_v(x, y, w) = D_v \log p(y \mid x, w) = \frac{v}{\|v\|} \cdot \nabla_w \log p(y \mid x, w).$$

The directional score measures how sensitive the log-density is to perturbations of w in direction v.

Exercise 2.11 (Basic properties of the score function). Let $\Psi : \mathcal{W} \to \mathcal{D}$ be a parameter–distribution map with positive densities $p(y \mid x, w)$, differentiable in w. Assume that for each x and w, the operations ∇_w and $\int_{\mathcal{Y}} \cdot \, dy$ may be exchanged.

(a) Show that the score function satisfies the following identity: for each $x \in \mathcal{X}$, $y \in \mathcal{Y}$, and $w \in \mathcal{W}$,

$$s(x, y, w) = \frac{\nabla_w p(y \mid x, w)}{p(y \mid x, w)}.$$

(b) Show that the expected score is zero: for each $x \in \mathcal{X}$ and $w \in \mathcal{W}$,

$$\mathbb{E}_{y \sim p(y \mid x, w)}\big[s(x, y, w)\big] = 0.$$

HINT, p. 573

(c) Show that if Ψ is degenerate at w in direction v, then the directional score vanishes identically: $s_v(x, y, w) = 0$ for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$.

(d) Show the converse: if the directional score $s_v(x, y, w)$ vanishes for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ at w, then Ψ is degenerate at w in direction v.

SOLUTION, p. 573

Definition 2.4 (Fisher information matrix). Let $\Psi : \mathcal{W} \to \mathcal{D}$ be a parameter–distribution map with score function s. Let $q(x)$ denote the marginal distribution of inputs. The **Fisher information matrix** $I : \mathcal{W} \to \mathbb{R}^{d\times d}$ is defined by

$$I(w) = \mathbb{E}_{x\sim q(x)}\, \mathbb{E}_{y\sim p(y|x,w)}\big[s(x,y,w)\, s(x,y,w)^\top\big].$$

The Fisher information matrix is symmetric and positive semidefinite at every w (being an expectation of positive semidefinite matrices $s\, s^\top$). However, the Fisher information matrix may not be symmetric positive *definite,* that is, it may be singular.

The following exercise shows that singularity of the Fisher information matrix is equivalent to degeneracy of the parameter–distribution map under mild regularity conditions.

Exercise 2.12 (Fisher information and degeneracy). Let $\Psi : \mathcal{W} \to \mathcal{D}$ be a parameter–distribution map with densities $p(y \mid x, w)$. Assume that $p(y \mid x, w) > 0$ for all $y \in \mathcal{Y}$, $x \in \mathcal{X}$, $w \in \mathcal{W}$, that $w \mapsto p(y \mid x, w)$ is differentiable, and that $q(x) > 0$ for all $x \in \mathcal{X}$.

(a) Show that for any unit vector $v \in \mathbb{R}^d$,

$$v^\top I(w)\, v = \mathbb{E}_{x\sim q(x)}\, \mathbb{E}_{y\sim p(y|x,w)}\big[s_v(x,y,w)^2\big].$$

(b) Using Exercise 2.11(c), show that if $D_v\Psi(w) = 0$ for some non-zero v, then $I(w)$ is not positive definite.

HINT, p. 574

(c) Using Exercise 2.11(d), show that if $I(w)$ is not positive definite, then there exists a non-zero v for which $D_v\Psi(w) = 0$.

HINT, p. 574

(d) Conclude that

$$\ker I(w) = \big\{v \in \mathbb{R}^d : D_v\Psi(w) = 0\big\}.$$

That is, the null space of the Fisher information matrix is exactly the space of degenerate directions of the parameter–distribution map.

SOLUTION, p. 574

Remark (Watanabe's strictly singular models). Watanabe 2009 defines a statistical model as *strictly singular* if either of two conditions hold:

1. The Fisher information matrix $I(w)$ is singular for some $w \in \mathcal{W}$.
2. The parameter–distribution map is not one-to-one, that is, for some $w, w' \in \mathcal{W}$ such that $w \neq w'$, we have $\Psi(w) = \Psi(w')$.

By Exercise 2.12, under mild regularity conditions, the first condition is equivalent to our notion of (somewhere) degeneracy. The second condition is called non-identifiability.

Watanabe 2007; Watanabe 2009 observe that many of the results from classical statistics crucially assume among their regularity conditions the parameter–distribution map is identifiable and the Fisher information matrix is nonsingular. Whereas, in a statistical model based on a non-trivial neural network (or any other statistical model involving hierarchical structure), it is typical for the Fisher information matrix to include singularities.

2.5 Degeneracy and the loss landscape

In the preceding sections, we studied degeneracy as a property of parameter–function maps and parameter–distribution maps. We now consider the implications of degeneracy in these maps for the *loss landscape* in which deep learning algorithms operate.

Recall the definitions of loss functions from Section 1. In what follows, we assume that the per-example loss $L_{x,y}(w)$ depends on w only through $\Phi(w)$, and work with the population loss $L(w) = \mathbb{E}_{(x,y)\sim q}[L_{x,y}(w)]$. For statistical models, we use the expected negative log-likelihood $L(w) = \mathbb{E}_{(x,y)\sim q}[-\log p(y \mid x, w)]$ as our loss function. In either case, we assume that L is twice-differentiable with respect to w.

Let us begin with some basic observations about the relationship between degeneracy in parameter–function maps and directional derivatives in the loss landscape.

Exercise 2.13 (Directional derivatives of the loss). Let $L : \mathcal{W} \to \mathbb{R}$ be a population loss function satisfying the assumptions described above.

(a) Show that if Φ is degenerate at w in direction v, then the directional derivative of the loss vanishes in the same direction: $D_v L(w) = 0$.

(b) Suppose $\mathcal{W} = \mathbb{R}^d$. Show that if $d > 1$, then for *any* $w \in \mathcal{W}$, there exists a nonzero v such that $D_v L(w) = 0$.

HINT, p. 575

SOLUTION, p. 575

We see that parameter–function map degeneracy implies flat loss directions, but a vanishing directional derivative of the loss landscape is commonplace. To find a satisfying definition of degeneracy in a loss landscape, we must look at second-order information.

Define the **Hessian matrix** $H(w) \in \mathbb{R}^{d\times d}$ of the loss at w by

$$H(w)_{jk} = \frac{\partial^2 L}{\partial w_j \partial w_k}(w).$$

The Hessian at w is sometimes denoted $\nabla^2 L(w)$. The Hessian captures the local curvature of the loss landscape via the quadratic approximation

$$L(w+\delta) \approx L(w) + \nabla L(w)^\top \delta + \tfrac{1}{2}\,\delta^\top H(w)\,\delta.$$

At a critical point ($\nabla L(w) = 0$), the Hessian determines whether the loss curves upward, downward, or remains flat in each direction.

In particular, at a local minimum, $H(w)$ is positive semidefinite. We can therefore classify local minima as follows:

- A local minimum w is **regular** (or **non-degenerate**, or **Morse**) if $H(w)$ is positive definite.
- A local minimum w is **degenerate** (or **non-Morse**) if $H(w)$ is singular (has a zero eigenvalue).

Exercise 2.14 (Some examples of loss landscape degeneracy). Consider the parameter space $\mathcal{W} = \mathbb{R}^2$ with the identity parameter–function map $\Phi = \mathrm{id}$, so that we identify parameters with the functions they implement (cf., Exercise 2.1). Consider the family of loss functions $L_{k,l}(a, b) = a^{2k} + b^{2l}$ for non-negative integers k and l.

(a) Show that the origin is a global minimum of $L_{k,l}$ for all non-negative k and l. For which k and l is it the unique global minimum?

(b) Show that if $k = l = 1$, then the origin is a non-degenerate (regular) minimum.

HINT, p. 575

(c) Show that if $k > 1$ and $l \geq 1$, then the origin is a degenerate minimum.

(d) Show that if $k = 0$ and $l \geq 1$, then the origin is a degenerate minimum.

SOLUTION, p. 575

Note that the Hessian degeneracy in Exercises 2.14(c) and 2.14(d) arises for qualitatively different reasons. In Exercise 2.14(d), the loss is constant along the a-direction to all orders—a genuinely flat direction. In Exercise 2.14(c), the loss does increase along the a-direction, just more slowly than quadratically (as a^4, a^6, etc., depending on k). Moreover, different values of k give different rates of increase. The Hessian cannot distinguish any of these cases: flat, quartic, and so on all appear the same from the perspective of the Hessian rank. We will later see approaches that can distinguish these degrees of degeneracy.

So much for defining loss landscape degeneracy. What is the relationship between this kind of degeneracy and degeneracy of the parameter–function/distribution map?

The relationship is subtle, and generally depends on the choice of loss function. One natural setting in which to study this relationship is the *realisable statistical model:* given a parameter–distribution map and a parameter w_0, if we consider p_{w_0} as the true data-generating distribution, the population negative log-likelihood loss will have a global minimum at w_0. The following exercise shows that in this setting, under mild regularity assumptions on the statistical model, degeneracy in the parameter–distribution map at w_0 implies that the global minimum w_0 is a degenerate global minimum of the loss landscape.

Exercise 2.15 (Realisable models and Hessian degeneracy). Let $\Psi : \mathcal{W} \to \mathcal{D}$ be a parameter–distribution map with positive densities $p(y \mid x, w)$, twice-differentiable in w. Suppose data is generated from a fixed true parameter $w_0 \in \mathcal{W}$, meaning $q(y \mid x) = p(y \mid x, w_0)$. Consider the population negative log-likelihood loss

$$L(w) = -\mathbb{E}_{x\sim q(x)}\, \mathbb{E}_{y\sim p(y|x,w_0)}\big[\log p(y \mid x, w)\big].$$

Assume that for each x, the operations ∇_w (and ∇_w^2) and $\int_{\mathcal{Y}} \cdot\, dy$ may be exchanged.

(a) *(Bartlett identity.)* Show that for each $x \in \mathcal{X}$ and $w \in \mathcal{W}$,

$$-\mathbb{E}_{y\sim p(y|x,w)}\big[\nabla_w^2 \log p(y \mid x, w)\big] = \mathbb{E}_{y\sim p(y|x,w)}\big[s(x, y, w)\, s(x, y, w)^\top\big],$$

where s is the score function from Definition 2.3.

HINT, p. 576

(b) Show that the Hessian of L at the true parameter w_0 equals the Fisher information matrix (Definition 2.4):

$$H(w_0) = I(w_0).$$

HINT, p. 576

(c) Using the result of Exercise 2.12, conclude: if Ψ is degenerate at w_0 in direction v, then $H(w_0)\, v = 0$. Contrapositively, if $H(w_0)$ is positive definite, then Ψ is non-degenerate at w_0.

SOLUTION, p. 576

Exercise 2.15 shows that for the population negative log-likelihood at the true parameter, degeneracy of the parameter–distribution map implies a degenerate loss landscape. The converse does not hold in general, nor does the forward direction hold at arbitrary points in parameter space. The following exercise explores these subtleties through examples.

Exercise 2.16 (Contrasting parametric versus loss degeneracy). (a) Observe that the identity parameter–function map on $\mathcal{W} = \mathbb{R}^2$ is non-degenerate everywhere. With this in mind, what do your answers to Exercises 2.14(b), 2.14(c) and 2.14(d) exemplify about the logical relationship between parameter–function map degeneracy and loss landscape degeneracy?

(b) Consider again the identity parameter–function map on $\mathcal{W} = \mathbb{R}^2$. Consider the loss function $L_\circ(a, b) = (a^2 + b^2 - 1)^2$. Find a global minimum of $L_\circ$ and show that it is a degenerate critical point. What does this example reveal about the logical relationship between parameter–function map degeneracy and loss landscape degeneracy?

(c) Now consider the cubic parameter–function map $\Phi(\alpha, \beta) = (\alpha^3, \beta^3)$.

1. Argue that Φ is degenerate at the origin (cf., Exercise 2.2).
2. If we take $L_{k,l}$ from Exercise 2.14 to be defined on the constant function (a, b) (rather than the equivalent underlying parameter), then in this parameterisation,

that loss function becomes $L_{k,l}(\alpha, \beta) = (\alpha^3)^{2k} + (\beta^3)^{2l}$. Show that the origin is a degenerate minimum of $L_{k,l}$ for all $k \geq 0$ and $l \geq 0$ under this parametrisation.

3. What does this example reveal about the logical relationship between parameter–function map degeneracy and loss landscape degeneracy?

(d) Consider the product parameter–function map $\Phi(a, b) = a \cdot b$ from Exercise 2.3, and define $L(a, b) = \frac{1}{2}(ab - 1)^2$.

1. Show that $(0, 0)$ is a critical point and that Φ is degenerate in every direction at $(0, 0)$.
2. Compute the Hessian at $(0, 0)$ and show it is nonsingular.
3. What does this example reveal about the logical relationship between parameter–function map degeneracy and loss landscape degeneracy? Why does this not contradict Exercise 2.15?

SOLUTION, p. 577

3. The degeneracy hierarchy

We have so far explored different kinds of qualitative degeneracy in the loss landscape. A natural question arises: *how can we quantify the degree of degeneracy at a particular point in parameter space?* In this section, we introduce the local learning coefficient, a rich mathematical object that reflects the degree of degeneracy at a parameter, and we explore it from several perspectives.

In this section and Section 4 we assume that any population loss L is a real analytic function. The precise definition of real analyticity is not important for this tutorial but interested readers can refer to (Watanabe 2009) (section 2). A property that will be useful is that such functions have a convergent Taylor expansion, in particular they have smooth derivatives of all orders.

3.1 The local learning coefficient via volume scaling asymptotics

The key idea is to measure the *volume* of near-optimal parameters. Consider a local minimum $w^* \in \mathcal{W}$ of the population loss L, and suppose we have a sufficiently small closed ball $B(w^*)$ centred on w^* such that $L(w) \geq L(w^*)$ for all $w \in B(w^*)$. Given a tolerance $\epsilon > 0$, define the **sublevel set**

$$B(w^*, \epsilon) = \{w \in B(w^*) \mid L(w) - L(w^*) < \epsilon\},$$

consisting of all parameters in the ball whose loss is within ϵ of the minimum. The volume of this set is

$$V(\epsilon) := \mathrm{Vol}(B(w^*, \epsilon)) = \int_{B(w^*, \epsilon)} dw.$$

We will look at how this volume scales as $\epsilon \to 0$. It will be instructive for us to first calculate this scaling in the non-degenerate case.

Exercise 3.1 (Volume scaling in the regular case). Let $w^* \in \mathcal{W}$ be a local minimum of L, and suppose that the Hessian $\nabla^2 L(w^*)$ is positive definite. Show that

$$V(\epsilon) = c\,\epsilon^{d/2} + o(\epsilon^{d/2}) \quad \text{as } \epsilon \to 0,$$

for some constant $c > 0$ where $d = \dim(\mathcal{W})$. You may assume that, as $\epsilon \to 0$, the volume of $B(w^*, \epsilon)$ agrees to leading order with the volume of the quadratic approximation obtained by replacing L with its second-order Taylor expansion at w^*.

HINT, p. 578

SOLUTION, p. 578

The key takeaway from this exercise is that the dimension of the model controls the leading order asymptotics of the volume scaling. We can investigate this further by calculating the volume scaling in an example where the Hessian is not positive definite.

Exercise 3.2 (Quadratic Valley Volume Scaling). Let $\mathcal{W} = [-R, R]^d$ for some (large) $R \in \mathbb{R}$ and let $L(x_1, \ldots, x_d) = \sum_{i=1}^{d'} x_i^2$.

(a) Sketch the loss landscape when $d = 2$, $d' = 1$.

(b) L achieves its minimum value 0 at multiple points in $\mathcal{W}$. What is the dimension of the space $\mathcal{W}_0 := \{w \in \mathcal{W} \mid L(w) = 0\}$?

(c) Calculate the Hessian of L.

(d) Prove that $V(\epsilon) \propto \epsilon^{d'/2}$

HINT, p. 579

SOLUTION, p. 579

In this example, the degenerate directions do not contribute to the leading exponent of $V(\epsilon)$. We see that the exponent $\frac{d}{2}$ from Exercise 3.1 is replaced with $\frac{d'}{2}$ where d' is the codimension of $\mathcal{W}_0$ i.e. the *effective dimension* of L which ignores the degenerate directions. This suggests that we can capture information about the degree of degeneracy of a loss function by looking at how the volume $V(\epsilon)$ scales as $\epsilon \to 0$. (Watanabe 2009) proves a general form for this kind of volume scaling which has been adapted by (Lau et al. 2025) into the following definition.

Definition 3.1 (Local learning coefficient). Let $w^* \in \mathcal{W}$ be a local minimum of the population loss L. There exists a unique rational number $\lambda(w^*) > 0$, a positive integer $\mu(w^*)$, and a constant $c > 0$ such that as $\epsilon \to 0$,

$$V(\epsilon) = c\,\epsilon^{\lambda(w^*)}(-\log \epsilon)^{\mu(w^*)-1} + o\left(\epsilon^{\lambda(w^*)}(-\log \epsilon)^{\mu(w^*)-1}\right).$$

We call $\lambda(w^*)$ the **local learning coefficient (LLC)** at w^*, and $\mu(w^*)$ the **local multiplicity**.

Exercise 3.3 (Two equations for λ). (a) Prove that for $a > 1$

$$\lambda = \lim_{\epsilon \to 0} \frac{\log(V(a\epsilon)/V(\epsilon))}{\log(a)}$$

(b) Prove that

$$\lambda = \lim_{\epsilon \to 0} \frac{\log V(\epsilon)}{\log \epsilon}$$

SOLUTION, p. 580

Note that from equation (36) we already have that the LLC and local multiplicity in the regular case are $\lambda = \frac{d}{2}$, $\mu = 1$.

When $\mu(w^*) = 1$, the formula simplifies to $V(\epsilon) = c\epsilon^{\lambda(w^*)} + o(\epsilon^{\lambda(w^*)})$ which allows us to interpret $\lambda(w^*)$ as the *volume scaling exponent*: increasing the error tolerance by a factor of a increases the volume of near-optimal parameters by a factor of $a^{\lambda(w^*)}$.

Exercise 3.4 (Calculating the LLC). Calculate the LLC and the rank of the Hessian at the global minimum in the following cases:

(a) $L(x) = x^2$

(b) $L(x) = x^{2m}$

(c) $L(x, y) = x^4 + y^4$

(d) $L(x, y) = x^2 + y^4$

SOLUTION, p. 581

Exercise 3.5 (Upper bound on the LLC). In Exercise 3.1, we saw that $\lambda = d/2$ for regular models. In this exercise, you will prove directly from the volume scaling definition that for any local minimum w^* of the population loss, the local learning coefficient satisfies $\lambda(w^*) \leq d/2$.

(a) Let $\Lambda > 0$ be an upper bound on the eigenvalues of the Hessian $\nabla^2 L(w)$ for all $w \in B(w^*)$. Using the Lagrange remainder form of Taylor's theorem, show that for all $w \in B(w^*)$,

$$L(w) - L(w^*) \leq \frac{1}{2}\Lambda \|w - w^*\|^2.$$

HINT, p. 581

(b) Consider the standard Euclidean ball of radius r centred at w^*, denoted $B_r(w^*) = \{w \in \mathcal{W} \mid \|w - w^*\| < r\}$. Find a radius r (as a function of ϵ and Λ) such that for sufficiently small ϵ,

$$B_r(w^*) \subseteq B(w^*, \epsilon).$$

(c) Recall that the volume of a d-dimensional Euclidean ball of radius r is proportional to r^d. Use your result from part (b) to show that there exists a constant $C > 0$ such that for sufficiently small ϵ,

$$C\epsilon^{d/2} \leq V(\epsilon).$$

(d) Conclude that $\lambda(w^*) \leq d/2$.

SOLUTION, p. 581

We have now shown that the LLC satisfies $\lambda(w^*) \leq d/2$, with equality precisely in the regular (non-degenerate) case. It is also possible (although quite fiddly) to prove that $\frac{r}{2} \leq \lambda(w^*)$ where $r = \text{rank}\nabla^2 L(w^*)$ and as we will see in the next exercise, the LLC captures more information about the geometry than just the Hessian rank.

Exercise 3.6 (LLC vs Hessian Rank). In this exercise we will show that the LLC can detect differences in the local geometry that are not reflected in the rank of the Hessian.

Construct two loss functions L_1 and L_2 with minima w_1 and w_2 respectively such that $\text{rank}\,\nabla^2 L_1(w_1) = \text{rank}\,\nabla^2 L_2(w_2)$ but $\lambda(w_1) \neq \lambda(w_2)$.

SOLUTION, p. 582

Exercise 3.7 (The cubically-parameterised loss). We can make a quadratic loss degenerate by changing the parameterisation. Let the *cubically-parameterised loss* be

$$L(\mu) = \tfrac{1}{2}(\mu^3 - \mu_0^3)^2, \qquad \mu \in \mathbb{R}$$

obtained from the ordinary quadratic loss $\ell(\theta) = \frac{1}{2}(\theta - \theta_0)^2$ via the reparameterisation $\theta = \mu^3$, $\theta_0 = \mu_0^3$.

(a) Show that the cubically-parameterised loss is just as expressive as the ordinary quadratic loss: that is, they both have the same set of achievable minima.

(b) Compute the Hessian $L''(\mu_0)$ to show that the loss is degenerate at its minimum when $\mu_0 = 0$, and non-degenerate when $\mu_0 \neq 0$.

(c) Give an explicit formula for $V(\epsilon)$ when $\mu_0 = 0$ and $\mu_0 \neq 0$ separately.

(d) Use your answer to part (c) to find the learning coefficient for $\mu_0 = 0$ and for $\mu_0 \neq 0$. Given that the model is non-degenerate when $\mu_0 \neq 0$, we expect the learning coefficient to be $d/2$ in that case; compare your answer. How does the cubically-parameterised loss at $\mu_0 = 0$ differ from the ordinary quadratic loss?

(e) Instead of taking $\epsilon \to 0$ to get the learning coefficient, fix a small but nonzero value for ϵ, such as $\epsilon = 0.01$. Plot $V(\epsilon)$ as a function of μ_0. As we saw from (d), the learning coefficient changes discontinuously when $\mu_0 = 0$—what happens with $V(\epsilon)$ as μ_0 gets close to zero? What changes if you make ϵ smaller or larger?

Even though the asymptotic *learning coefficient* ($\epsilon \to 0$) only changes when $\mu_0 = 0$ exactly, note how the non-asymptotic *volume* (ϵ finite) is affected in a larger neighbourhood.

SOLUTION, p. 583

3.2 Perspectives on the local learning coefficient

In this section, we survey several alternative perspectives on the LLC, from information theory, algebraic geometry, and geometric measure theory.

Information-theoretic interpretation.

The LLC also admits a natural interpretation in terms of information (Lau et al. 2025). The number of bits needed to specify the sublevel set $B(w^*, \epsilon)$ within the ball $B(w^*)$ is

$$-\log_2 \frac{V(\epsilon)}{\mathrm{Vol}(B(w^*))}.$$

For small ϵ and $\mu(w^*) = 1$, this is approximately $-\lambda(w^*)\log_2 \epsilon + O(\log_2 \log_2 \epsilon)$. In particular, the number of *additional* bits needed to halve an already small error tolerance from ϵ to $\epsilon/2$ is

$$-\log_2 \frac{V(\epsilon/2)}{V(\epsilon)} \approx \lambda(w^*).$$

That is, the LLC $\lambda(w^*)$ measures the number of bits required to halve the error near w^*.

Local learning coefficient via algebraic geometry.

We explore an algebro-geometric perspective on the learning coefficient due to Watanabe 2009. Note that this section is more advanced than the remainder of this tutorial, and should be considered optional. When $L(w)$ is a real analytic function, Hironaka's resolution of singularities theorem guarantees the existence of a smooth map $g\colon M \to \mathcal{W}$ from an analytic manifold M and local coordinates $u = (u_1, \ldots, u_d)$ that *monomialise* the loss near w^*:

$$L(g(u)) - L(w^*) = u_1^{2k_1} \cdots u_d^{2k_d}, \qquad dw = b(u)\, u_1^{h_1} \cdots u_d^{h_d}\, du,$$

where $b(u) > 0$ is smooth and the exponents k_j, h_j are non-negative integers. In these coordinates, the volume integral $V(\epsilon)$ can be evaluated explicitly, yielding the asymptotic form of Definition 3.1 and giving

$$\lambda(w^*) \;=\; \min_{\text{coordinate charts}} \; \min_{j\colon k_j > 0} \frac{h_j + 1}{2k_j}\,.$$

The resolution map g is far from unique, and different choices produce different exponents k_j, h_j. $\lambda(w^*)$ is nevertheless well-defined, which can be seen from (another) equivalent definition using the **local zeta function**:

$$\zeta(z,\, w^*) \;=\; \int_{B(w^*)} \big(L(w) - L(w^*)\big)^z \, dw\,,$$

which converges for $\mathrm{Re}(z) > 0$ and extends to a meromorphic function on $\mathbb{C}$ whose poles are all real, negative, and rational. The LLC $\lambda(w^*)$ is precisely the negative of the largest pole, and the multiplicity $\mu(w^*)$ is the multiplicity of this pole. Since (44) makes no reference to any resolution, $\lambda(w^*)$ depends only on L and w^*.

In the algebraic geometry literature, $\lambda(w^*)$ is called the **real log canonical threshold (RLCT)** of L at w^*. We collect several consequences of this perspective.

1. The fact that the LLC is a rational number follows directly from equation (43).
2. Equation (43) suggests that we can compute $\lambda(w^*)$ analytically by constructing an explicit resolution of singularities for $L(w) - L(w^*)$ and reading off the exponents. This is difficult in practice for neural network loss functions, but has been achieved for some architectures, as we will see in Section 3.3.
3. In Section 4 we will state Watanabe's free energy formula connecting the Bayesian free energy to the LLC. The proof of this result makes repeated use of the monomialisation of the loss.

Exercise 3.8 (Computing the LLC via the RLCT). Suppose that $L(w) = w_1^{2k_1} \cdots w_d^{2k_d}$ near $w^* = 0$, where $k_j \geq 0$ are integers with at least one $k_j > 0$. Show that

$$\lambda(0) = \min_{j:\, k_j > 0} \frac{1}{2k_j}\,.$$

SOLUTION, p. 584

Local learning coefficient as a fractal dimension.

The LLC admits a geometric interpretation: it is a *fractal dimension* of the parameter space, as measured through the lens of the loss function. To make this precise, we recall a standard notion from fractal analysis.

Definition 3.2 (Hölder exponent). Let (X, ρ) be a (pseudo-)metric space and ν a measure on X. The **Hölder exponent** (or **local dimension**) $\alpha(x)$ of ν at a point $x \in X$ is given by

$$\alpha(x) = \lim_{\epsilon \to 0} \frac{\log \nu(B_\epsilon(x))}{\log \epsilon},$$

where $B_\epsilon(x)$ is the ball of radius ϵ centred at x. Equivalently, $\alpha(x)$ is the unique real number such that

$$\nu(B_\epsilon(x)) = O(\epsilon^{\alpha(x)}) \qquad \text{as } \epsilon \to 0.$$

The Hölder exponent generalises the notion of dimension: when ν is the Lebesgue measure on $\mathbb{R}^d$ and ρ is the Euclidean metric, every point has $\alpha(x) = d$, but for measures concentrated on fractal sets the exponent can be non-integer. Notice that the formula (45) has a similar form to the definition of the local learning coefficient (38). To make this connection precise, we introduce a pseudo-metric induced by the loss function.

Definition 3.3 (Loss pseudo-metric). The **loss pseudo-metric** on $\mathcal{W}$ is defined by

$$\rho(w, u) = |L(w) - L(u)|.$$

Remark. ρ is only a pseudo-metric as the distance between two distinct points can be zero.

Under this pseudo-metric, the sublevel set (34) is precisely the ϵ-ball around w^*:

$$B(w^*, \epsilon) = \{w \in B(w^*) \mid \rho(w, w^*) < \epsilon\},$$

so $V(\epsilon) = \nu(B(w^*, \epsilon))$.

Exercise 3.9 (Hölder exponent and learning coefficient). Let $\alpha(w^*)$ be the Hölder exponent of the Lebesgue measure on $\mathcal{W}$ at w^* under the loss pseudo-metric. Show that

$$\lambda(w^*) = \alpha(w^*).$$

SOLUTION, p. 584

3.3 Local learning coefficients of deep linear networks

Having established the local learning coefficient as a measure of degeneracy in the loss landscape, we now compute it explicitly for the two-layer deep linear network introduced in Section 1. In Section 2, we saw that DLN parameters exhibit varying degrees of degeneracy depending on the ranks of the component matrices (Exercise 2.9, Remark 2.2). The LLC makes this hierarchy quantitative: different ranks correspond to different learning coefficients, confirming that lower-rank parameters are geometrically "simpler."

Consider the two-layer DLN $f_{A,B}(x) = BAx$ with $A \in \mathbb{R}^{H \times M}$ and $B \in \mathbb{R}^{N \times H}$, so the parameter space is $\mathcal{W} = \mathbb{R}^{(M+N)H}$. Suppose the true input–output relationship is $y = B_0 A_0 x$ where $A_0 \in \mathbb{R}^{H \times M}$, $B_0 \in \mathbb{R}^{N \times H}$, and $r = \operatorname{rank}(B_0 A_0)$, and consider the population loss

$$L(A, B) = \mathbb{E}_{x \sim q(x)} \left[\|BAx - B_0 A_0 x\|^2 \right],$$

where $q(x)$ has full-rank covariance. The set of minima of this loss function is $W_0 = \{(A, B) : BA = B_0 A_0\}$. Aoyagi & Watanabe 2005 derived an exact formula for the learning coefficient at these minima.

Theorem 3.4 (Aoyagi & Watanabe 2005). For the two-layer DLN $f_{A,B}(x) = BAx$ with $A \in \mathbb{R}^{H \times M}$, $B \in \mathbb{R}^{N \times H}$, and true parameter (A_0, B_0) with $r = \operatorname{rank}(B_0 A_0)$, at any minimum $w^* \in W_0$ the local learning coefficient λ and its multiplicity μ are given by the following cases:

1. If $N + r \leq M + H$, $M + r \leq N + H$, and $H + r \leq M + N$:
    1. If $M + H + N + r$ is even, then $\mu = 1$ and

$$\lambda = \frac{-(H+r)^2 - M^2 - N^2 + 2(H+r)M + 2(H+r)N + 2MN}{8}.$$

2. If $M + H + N + r$ is odd, then $\mu = 2$ and

$$\lambda = \frac{-(H+r)^2 - M^2 - N^2 + 2(H+r)M + 2(H+r)N + 2MN + 1}{8}.$$

2. If $M + H < N + r$, then $\mu = 1$ and $\lambda = \dfrac{HM - Hr + Nr}{2}$.
3. If $N + H < M + r$, then $\mu = 1$ and $\lambda = \dfrac{HN - Hr + Mr}{2}$.
4. If $M + N < H + r$, then $\mu = 1$ and $\lambda = \dfrac{MN}{2}$.

Exercise 3.10 (Learning coefficients of two-layer DLNs). Consider the constant-width two-layer DLN from Exercise 2.9: $y = BAx$, where $x \in \mathbb{R}^m$ are the inputs, $y \in \mathbb{R}^m$ are the outputs, and $A, B \in \mathbb{R}^{m \times m}$ are linear transformations. The parameter space is $\mathcal{W} = \mathbb{R}^{2m^2}$ and the parameters are the pair $(A, B) \in \mathcal{W}$.

(a) Aoyagi & Watanabe do not assume the network is constant-width, like we do. Simplify their formula for λ using this assumption.

(b) How does the learning coefficient λ depend on r, the rank of $B_0 A_0$? Compare with the results of Exercise 2.9(d).

(c) How does λ compare with the effective parameter count $d/2$ of a regular model? What does this tell us about the role of singularities in the two-layer DLN?

SOLUTION, p. 585

4. Degeneracy and Bayesian deep learning

In Sections 2 and 3, we studied the geometry of degeneracy in parameter space and introduced the local learning coefficient $\lambda(w^*)$ as a measure of the degree of degeneracy at a local minimum w^*. We now show that this geometry has consequences for learning (in the Bayesian setting). In particular, it creates a trade-off between model fit and model complexity that drives learning, providing a mechanism for *internal model selection*.

4.1 Watanabe's free energy formula

Recall the Bayesian learning setup from Section 1.4. We have a parameter–distribution map $\Psi : \mathcal{W} \to \mathcal{D}$ sending each parameter w to a conditional distribution with density $p(y \mid x, w)$, a smooth positive prior φ on $\mathcal{W}$, and a data set of n i.i.d. examples from a data distribution q. Take the loss function to be the negative log-likelihood

$$L_n(w) = -\frac{1}{n}\sum_{i=1}^{n} \log p(y^{(i)} \mid x^{(i)}, w), \qquad L(w) = \mathbb{E}_{(x,y)\sim q}[-\log p(y \mid x, w)]\,.$$

The posterior π_n is given by Bayes' rule (13) and as we saw in Exercises 1.7 and 1.8 it concentrates around regions of parameter space with the lowest local free energy $F_n(\mathcal{U})$ (Definition 1.6).

We now state one of the main results of SLT: an asymptotic expansion of $F_n(\mathcal{U})$ as $n \to \infty$. Under certain technical assumptions on the statistical model (Watanabe 2018; see Lau 2025, § 2.3.1 for a summary), we have the following theorem.

Theorem 4.1 (Local free energy formula). Suppose $\mathcal{U} \subseteq \mathcal{W}$ contains at least one minimiser of L, and let $w^* \in \mathcal{U}$ be any such minimiser. Define $\lambda_{\mathcal{U}}$ as the smallest local learning coefficient among all minimisers in $\mathcal{U}$, and let $\mu_{\mathcal{U}}$ be the maximum local multiplicity associated with $\lambda_{\mathcal{U}}$. Then, as $n \to \infty$,

$$F_n(\mathcal{U}) = nL_n(w^*) + \lambda_{\mathcal{U}} \log n - (\mu_{\mathcal{U}} - 1) \log\log n + O_p(1).$$

Remark. Since $L_n(w^*)$ is a random variable, so is $F_n(\mathcal{U})$. The expansion Equation 49 is an asymptotic statement about random variables where the remainder $O_p(1)$ denotes a term that is bounded in probability as $n \to \infty$.

To understand what this expansion tells us about Bayesian learning, note that the two leading terms of this expansion have transparent interpretations:

$$F_n \approx \underbrace{nL_n}_{\text{data fit}} + \underbrace{\lambda \log n}_{\text{complexity}}$$

Let $w_1, w_2 \in \mathcal{W}$ be minimisers of the population loss with $L(w_1) = L(w_2)$ and $\lambda(w_1) \leq \lambda(w_2)$. Then for fixed large n, the free energy of a neighbourhood of w_1 is smaller than that of w_2, so the posterior concentrates around the simpler solution w_1.

This is an example of **internal model selection** where there are two equally good solutions that minimise the population loss. However, our learning algorithm (Bayesian updating) prefers one solution over the other, in particular the one with lower complexity.

Remark (Regular models and the BIC). When the Hessian $\nabla^2 L(w^*)$ is non-degenerate, $\lambda(w^*) = d/2$ and $\mu(w^*) = 1$ by Exercise 3.1. The free energy formula then reduces to $F_n \approx nL_n(w^*) + \frac{d}{2}\log n$, recovering the Bayesian Information Criterion from classical statistics.

Exercise 4.1 (Numerical demonstration of the free energy formula). Consider the statistical model underlying the cubically-parameterised loss from Exercise 3.7. The parameter $\mu \in \mathbb{R}$ indexes the family of normal distributions

$$p(x \mid \mu) = (2\pi)^{-1/2} \exp\left(-\frac{1}{2}(x - \mu^3)^2\right).$$

Suppose the true distribution is

$$q(x) = (2\pi)^{-1/2} \exp\left(-\frac{1}{2}(x - \mu_0^3)^2\right)$$

for some fixed μ_0, and equip the model with a standard normal prior

$$\varphi(\mu) = (2\pi)^{-1/2} \exp\left(-\frac{1}{2}\mu^2\right).$$

[*Note:* The following exercises are intended to be attempted with a numerical programming framework such as Python or Julia, rather than by hand.]

(a) For $\mu_0 = 0$, numerically compute the free energy F_n for many values of n between $n = 100$ and $n = 10{,}000$. Compare the computed values against the asymptotic estimate $F_n \approx nL_n + \lambda \log n$, using the value of λ from Exercise 3.7(d).

(b) Repeat part (a) with $\mu_0 = 5$.

(c) Repeat part (a) with $\mu_0 = 0.1$. Note that $\mu_0 = 0.1 \neq 0$, so asymptotically $\lambda = 1/2$, as in part (b). However, observe how for moderate n the free energy behaves more like part (a) than part (b). This is another manifestation of the phenomenon from Exercise 3.7(e): the effects of the singularity at $\mu = 0$ extend into a neighbourhood around it.

SOLUTION, p. 586

4.2 Bayesian phase transitions

So far we have looked at the consequence of the free energy formula when considering n to be fixed. However, we also find it interesting to see what it tells us about changes in the posterior as n increases.

For the rest of this section, assume n is sufficiently large that (i) the free energy can be approximated by the first two terms in its expansion, and (ii) $L_n \approx L$ by the law of large numbers, so that the empirical loss can be treated as approximately constant in n.

Consider two disjoint regions $\mathcal{U}, \mathcal{V} \subseteq \mathcal{W}$, containing minimisers u^* and v^* of the population loss L with local learning coefficients $\lambda_{\mathcal{U}}$ and $\lambda_{\mathcal{V}}$ respectively. Applying the free energy formula to each region, the log posterior odds between the two regions satisfy

$$\log \frac{\pi_n(\mathcal{U})}{\pi_n(\mathcal{V})} = F_n(\mathcal{V}) - F_n(\mathcal{U}) \approx n\,\Delta L_n + \Delta\lambda \cdot \log n$$

where $\Delta L_n = L_n(v^*) - L_n(u^*)$ and $\Delta\lambda = \lambda_{\mathcal{V}} - \lambda_{\mathcal{U}}$.

Consider the case when one region is more accurate but more complex than the other, i.e.

$$L_n(u^*) > L_n(v^*) \qquad \text{and} \qquad \lambda_{\mathcal{U}} < \lambda_{\mathcal{V}}.$$

Then $\Delta L_n < 0$ and $\Delta\lambda > 0$, so the two terms in (50) have opposite signs. For smaller n, the $\Delta\lambda \cdot \log n$ term dominates so the log posterior odds is negative meaning the posterior prefers $\mathcal{U}$ (the simpler, less accurate region). As n increases, the linear term

$n\,\Delta L_n$ eventually dominates the logarithmic term, and the posterior shifts to prefer $\mathcal{V}$ (the more complex, more accurate region). Setting the leading terms equal, this change occurs at a critical sample size n^* satisfying

$$\frac{n^*}{\log n^*} \approx \frac{\Delta\lambda}{|\Delta L_n|}.$$

At $n = n^*$, the posterior undergoes a **Bayesian phase transition**: the region that dominates the posterior changes abruptly.

Exercise 4.2 (Exploring phase transitions). We show how *phase transitions* can occur when different local free energies change at different rates.

(a) Suppose we partition the overall parameter space into a disjoint union $\mathcal{W} = \mathcal{U} \sqcup \mathcal{V}$. Denote the overall free energy by F_n and the local free energies by $F_n(\mathcal{U})$ and $F_n(\mathcal{V})$, respectively. Show that the relationship

$$F_n = -\log(e^{-F_n(\mathcal{U})} + e^{-F_n(\mathcal{V})})$$

holds.

(b) Suppose $F_n(\mathcal{U}) \approx 0.3n + 20\log n$ and $F_n(\mathcal{V}) \approx 0.5n + 2\log n$. Plot the overall free energy $F(n)$. Compare against $\min\{F_n(U), F_n(\mathcal{V})\}$. What happens around $n = 570$?

(c) In statistical physics, a phase transition is traditionally defined as a discontinuity (or rapid change, for finite-size systems) in the derivatives of the free energy. Plot $\frac{d}{dn}F_n$ and explain why this justifies calling the phenomenon from b) a phase transition.

(d) Change the coefficients of the terms in b). How does this change when a phase transition occurs? Does a phase transition always occur?

SOLUTION, p. 586

→ 5. *Further readings*: Supplementary Material, p. 491

References

Maxwell Adam, Zach Furman, and Jesse Hoogland (2025). *The Loss Kernel: A Geometric Probe for Deep Learning Interpretability*. arXiv:2509.26537.

Miki Aoyagi and Sumio Watanabe (2005). *Stochastic complexities of reduced rank regression in Bayesian estimation*. Neural Networks.

Miki Aoyagi (2009). *Log canonical threshold of Vandermonde matrix type singularities and generalization error of a three layered neural network in Bayesian estimation*. International Journal of Pure and Applied Mathematics.

Miki Aoyagi (2024). *Consideration on the learning efficiency of multiple-layered neural networks with linear units*. Neural Networks.

Garrett Baker, George Wang, Jesse Hoogland, and Daniel Murfet (2025). *Structural Inference: Interpreting Small Language Models with Susceptibilities*. arXiv:2504.18274.

Liam Carroll (2021). *Phase Transitions in Neural Networks*. School of Mathematics and Statistics, the University of Melbourne.

Liam Carroll (2023). *Distilling Singular Learning Theory*.

Liam Carroll, Jesse Hoogland, Matthew Farrugia-Roberts, and Daniel Murfet (2025). *Dynamics of Transient Structure in In-Context Linear Regression Transformers*. arXiv:2501.17745.

Zhongtian Chen and Daniel Murfet (2025). *Modes of Sequence Models and Learning Coefficients*. arXiv:2504.18048.

Zhongtian Chen, Edmund Lau, Jake Mendel, Susan Wei, and Daniel Murfet (2023). *Dynamical versus Bayesian Phase Transitions in a Toy Model of Superposition*. arXiv:2310.06301.

James Clift, Daniel Murfet, and James Wallbridge (2021). *Geometry of Program Synthesis*. arXiv:2103.16080.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah (2022). *Toy Models of Superposition*. Transformer Circuits Thread.

Chris Elliott, Einar Urdshals, David Quarel, Matthew Farrugia-Roberts, and Daniel Murfet (2026). *Stagewise Reinforcement Learning and the Geometry of the Regret Landscape*. arXiv:2601.07524.

Matthew Farrugia-Roberts (2022). *Structural Degeneracy in Neural Networks*. School of Computing and Information Systems, the University of Melbourne.

Matthew Farrugia-Roberts (2023). *Functional Equivalence and Path Connectivity of Reducible Hyperbolic Tangent Networks*. Advances in Neural Information Processing Systems 36.

Matthew Farrugia-Roberts (2024). *Losslessly Compressible Neural Network Parameters*. Workshop on Machine Learning and Compression.

Zach Furman (2023). *Introduction to RLCT estimation*.

Zach Furman (2024). *Singular learning theory: Exercises*.

Zach Furman (2026). *Deep learning as program synthesis*.

Andrew Gordon, Garrett Baker, George Wang, William Snell, Stan van Wingerden, and Daniel Murfet (2026). *Towards Spectroscopy: Susceptibility Clusters in Language Models*. arXiv:2601.12703.

Rohan Hitchcock and Jesse Hoogland (2025). *From Global to Local: A Scalable Benchmark for Local Posterior Sampling*. arXiv:2507.21449.

Jesse Hoogland, Alexander Gietelink Oldenziel, Daniel Murfet, and Stan van Wingerden (2023). *Towards Developmental Interpretability*.

Jesse Hoogland, George Wang, Matthew Farrugia-Roberts, Liam Carroll, Susan Wei, and Daniel Murfet (2025). *Loss Landscape Degeneracy and Stagewise Development in Transformers*. Transactions on Machine Learning Research.

Philipp Alexander Kreer, Wilson Wu, Maxwell Adam, Zach Furman, and Jesse Hoogland (2025). *Bayesian Influence Functions for Hessian-Free Data Attribution*. arXiv:2509.26544.

Edmund Lau and Zhongtian Chen (2023). *Singular Learning Theory: The Low Road*.

Edmund Lau (2025). *A Singular Perspective on Machine Learning*. School of Mathematics and Statistics, the University of Melbourne.

Edmund Lau, Zach Furman, George Wang, Daniel Murfet, and Susan Wei (2025). *The Local Learning Coefficient: A Singularity-Aware Complexity Measure*. The 28th International Conference on Artificial Intelligence and Statistics.

Jin Hwa Lee, Matthew Smith, Maxwell Adam, and Jesse Hoogland (2025). *Influence Dynamics and Stagewise Data Attribution*. arXiv:2510.12071.

Daniel Murfet and Liam Carroll (2023). *Singular Learning Theory: The High Road*.

Daniel Murfet and Will Troiani (2025). *Programs as Singularities*. arXiv:2504.08075.

Daniel Murfet (2024). *Simple versus short: Higher-order degeneracy and error-correction*.

Nina Panickssery and Dmitry Vaintrob (2023). *Investigating the learning coefficient of modular addition*.

Simon Pepin Lehalleur, Jesse Hoogland, Matthew Farrugia-Roberts, Susan Wei, Alexander Gietelink Oldenziel, George Wang, Liam Carroll, and Daniel Murfet (2025). *You Are What You Eat--AI Alignment Requires Understanding How Data Shapes Structure and Generalisation*. arXiv:2502.05475.

Joar Skalse (2023). *My criticism of singular learning theory*.

Einar Urdshals and Jasmina Urdshals (2025). *Structure Development in List-Sorting Transformers*. arXiv:2501.18666.

Einar Urdshals, Edmund Lau, Jesse Hoogland, Stan van Wingerden, and Daniel Murfet (2025). *Compressibility Measures Complexity: Minimum Description Length Meets Singular Learning Theory*. arXiv:2510.12077.

George Wang and Daniel Murfet (2026). *Patterning: The Dual of Interpretability*. arXiv:2601.13548.

George Wang, Matthew Farrugia-Roberts, Jesse Hoogland, Liam Carroll, Susan Wei, and Daniel Murfet (2024). *Loss landscape geometry reveals stagewise development of transformers*. High-dimensional Learning Dynamics 2024: The Emergence of Structure and Reasoning.

George Wang, Jesse Hoogland, Stan van Wingerden, Zach Furman, and Daniel Murfet (2025). *Differentiation and Specialization of Attention Heads via the Refined Local Learning Coefficient*. International Conference on Learning Representations.

George Wang, Garrett Baker, Andrew Gordon, and Daniel Murfet (2025). *Embryology of a Language Model*. arXiv:2508.00331.

Thomas Waring (2021). *Geometric Perspectives on Program Synthesis and Semantics*. School of Mathematics and Statistics, the University of Melbourne.

Sumio Watanabe (2007). *Almost all learning machines are singular*. IEEE Symposium on Foundations of Computational Intelligence.

Sumio Watanabe (2009). *Algebraic Geometry and Statistical Learning Theory*. Cambridge University Press.

Sumio Watanabe (2013). *A widely applicable Bayesian information criterion*. The Journal of Machine Learning Research.

Sumio Watanabe (2018). *Mathematical Theory of Bayesian Statistics*. Chapman and Hall/CRC.

Sumio Watanabe (2023). *Singular Learning Theory, parts (1) and (2)*.

Sumio Watanabe (2024). *Recent Advances in Algebraic Geometry and Bayesian Statistics*. Information Geometry.

Susan Wei, Daniel Murfet, Mingming Gong, Hui Li, Jesse Gell-Redman, and Thomas Quella (2023). *Deep Learning Is Singular, and That's Good*. IEEE Transactions on Neural Networks and Learning Systems.

Adrian K. Xu (2021). *Smooth relaxation preserving Turing machines*. arXiv:2106.00956.

CHAPTER B.4

Training Dynamics

Guillaume Corlouer (Stormglass)

Exact learning dynamics of deep linear networks – loss-landscape geometry, balanced gradient flow and the NTK, the rich (saddle-to-saddle) and lazy regimes, their mixed unification, and the implicit bias of SGD noise.

Source: iliad-intensive.org/learning/training-dynamics · CC BY 4.0 · Slides: online

VIDEO *The Dumbest Neural Network Worth Studying* · youtu.be/IHHrJdZKICs

VIDEO *Uncovering The Hidden Biases of Deep Learning* · youtu.be/clTLzFD2uC8

WHAT YOU'LL LEARN

Motivation

- Know about some big open questions in learning dynamics
- Understand the concept of implicit regularization
- Know about the different approaches to studying learning dynamics
- Understand the AI safety motivations for learning dynamics
- Know about emergent misalignment as a safety-relevant phenomenon that illustrates the importance of understanding generalization for AI safety

Geometry and dynamics of deep neural networks

- Know key results about the loss landscape of deep linear networks (DLNs): critical points are saddles or global minima
- Explain the edge of stability phenomenon
- Understand that gradient flow can be written as NTK-weighted gradient in function space
- DLNs are degenerate and have conserved quantities through gradient flow

Regimes of learning in deep linear networks

- Understand the role of initialization, width and depth for the lazy and rich (saddle to saddle) regimes in DLNs
- Know about implicit regularization from non-linearity (neural race reduction)

References: (Saxe et al. 2014), (Achour et al. 2024), (Tu et al. 2024).

1. Aims of this chapter

The goal of this chapter is to present a toy model perspective on the training dynamics of deep neural networks. The reader should learn about the AI safety motivations of studying training dynamics. One motivation is that understanding the implicit biases of training deep neural networks is a central problem behind AI alignment. This is because implicit biases influence the generalization behaviour of a neural network including for example if that neural network will generalize in a helpful or harmful way out of distribution. Another motivation is interpretability. One key lesson of this chapter is that SGD learns from data in a structured way. For example in deep linear neural networks, under small initialization, it will first learn the features that explain the largest fraction of data covariance.

2. Setup and notation

We study deep linear networks (DLNs) with L weight matrices $W_1 \in \mathbb{R}^{d_1 \times d_0}, W_2 \in \mathbb{R}^{d_2 \times d_1}, \ldots, W_L \in \mathbb{R}^{d_L \times d_{L-1}}$. The network computes:

$$f(x) = W_L W_{L-1} \cdots W_1 \, x =: Wx$$

where $W = W_L \cdots W_1 \in \mathbb{R}^{d_L \times d_0}$ is the end-to-end (or "student") matrix. We train on a dataset $\{(x_\mu, y_\mu)\}_{\mu=1}^N$ with the squared loss:

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{\mu=1}^{N} \|y_\mu - f(x_\mu)\|^2$$

In the population limit $N \to \infty$ with whitened inputs $\Sigma_X = I$, this becomes, up to an additive constant[1]:

$$\mathcal{L}(W) = \frac{1}{2} \|M - W\|_F^2$$

where $M = \Sigma_{YX} \Sigma_X^{-1} = \Sigma_{YX}$ is the "teacher" matrix (the OLS solution) with SVD $M = U \, \mathrm{diag}(s_1, \ldots, s_r, 0, \ldots, 0) \, V^\top$, and $s_1 \geq s_2 \geq \cdots \geq s_r > 0$.

3. Loss landscape geometry

This problem explores the critical point structure of deep linear networks, following (Achour et al. 2024).

Exercise 3.1 (Diagonal decomposition). Consider the **diagonal** case: $d_0 = d_1 = \cdots = d_L = d$, and the teacher is diagonal, $M = \mathrm{diag}(s_1, \ldots, s_d)$, with $s_1 > s_2 > \cdots > s_d > 0$. Restrict attention to diagonal weight matrices $W_l = \mathrm{diag}(w_l^{(1)}, \ldots, w_l^{(d)})$. Show that the loss decomposes into d independent scalar problems:

$$\mathcal{L} = \frac{1}{2}\sum_{\alpha=1}^{d}\left(s_\alpha - \prod_{l=1}^{L} w_l^{(\alpha)}\right)^2$$

SOLUTION, p. 588

Exercise 3.2 (Scalar critical points). For a single scalar mode with target $s > 0$, find all first-order critical points of $\ell(w_1, w_2) = \frac{1}{2}(s - w_1 w_2)^2$ at depth $L = 2$. Show that the critical points are:

- The global minimum manifold: $w_1 w_2 = s$
- The origin: $w_1 = w_2 = 0$

Classify the origin as a saddle point by computing the Hessian H of ℓ at $(0, 0)$ and showing it has both positive and negative eigenvalues.

Hint: Write the gradient equations.

SOLUTION, p. 588

Exercise 3.3 (Critical-point structure). Achour et al. (Achour et al. 2024) show that every first-order critical point $\theta = (W_1, \ldots, W_L)$ satisfies: there exists a subset $S \subseteq \{1, \ldots, d\}$ such that

$$W = W_L \cdots W_1 = P_S\, M$$

where $P_S = U_S U_S^\top$ is the orthogonal projector onto the span of the left singular vectors $\{u_\alpha\}_{\alpha \in S}$ of M. For the diagonal case $M = \mathrm{diag}(s_1, \ldots, s_d)$, write the value of the loss at the critical point corresponding to $S = \{1, \ldots, r\}$ with $r < d$.

SOLUTION, p. 589

Exercise 3.4 (Symmetry of the global minima). The group $GL_h := GL_{d_1} \times \cdots \times GL_{d_{L-1}}$ acts on the weights by:

$$(W_1, \ldots, W_L) \mapsto (g_1 W_1,\ g_2 W_2 g_1^{-1},\ \ldots,\ W_L g_{L-1}^{-1})$$

Verify that the student map $\mu(\theta) = W_L \cdots W_1$ is invariant under this action: $\mu(g \cdot \theta) = \mu(\theta)$.

SOLUTION, p. 589

4. Gradient flow and conserved quantities

We now study gradient flow: $\dot\theta(t) = -\nabla_\theta \mathcal{L}(\theta(t))$, the continuous-time limit of gradient descent with infinitesimal learning rate.

Exercise 4.1 (Gradient-flow equations). For a two-layer DLN ($L = 2$), the loss is given by $\mathcal{L} = \frac{1}{2}\|M - W_2 W_1\|_F^2$. Assume that each weight matrix W_i is diagonal. Derive the gradient flow equations for each layer. In particular, show that:

$$\dot W_1 = W_2^\top (M - W_2 W_1), \qquad \dot W_2 = (M - W_2 W_1) W_1^\top$$

Note: these equations also hold for general (non-diagonal) W_1 and W_2.

SOLUTION, p. 589

Exercise 4.2 (Balancedness is conserved). Define the **balancedness matrix**:

$$G := W_2^\top W_2 - W_1 W_1^\top$$

Show that G is conserved under the gradient flow, i.e. $\dot{G} = 0$. In other words, gradient flow is constrained to the balanced manifold

$$\mathcal{M}_\beta := \{\theta \in \Omega \mid W_{\ell+1}^\top W_{\ell+1} - W_\ell W_\ell^\top = \beta_\ell,\ \ell = 1, \dots, L-1\}.$$

Hint: Compute $\dot{G}$, substitute the gradient flow equations and verify that the terms cancel pairwise.

SOLUTION, p. 590

From now on, assume **balanced initialization**: $G(0) = 0$, which by Exercise 4.2 means $W_2^\top W_2 = W_1 W_1^\top$ for all time. We want to derive the gradient flow in function space (the ODE for the student $W = W_2 W_1$). This requires several steps.

Exercise 4.3 (Function-space velocity). Compute $\dot{W} := \frac{d}{dt}(W_2 W_1)$ using the gradient flow equations from Exercise 4.1. Show that:

$$\dot{W} = (M - W)\, W_1^\top W_1 + W_2 W_2^\top\, (M - W)$$

SOLUTION, p. 590

Exercise 4.4 ($W_1^\top W_1$ in terms of W). We now need to express $W_1^\top W_1$ and $W_2 W_2^\top$ in terms of $W = W_2 W_1$. Show that:

$$W_1^\top W_1 = (W^\top W)^{1/2}$$

SOLUTION, p. 590

Exercise 4.5 ($W_2 W_2^\top$ in terms of W). Similarly, show that $W_2 W_2^\top = (W W^\top)^{1/2}$.

SOLUTION, p. 590

Exercise 4.6 (The balanced function-space ODE). Substitute the results of Exercises 4.4 and 4.5 into Exercise 4.3 to obtain:

$$\dot{W} = (W W^\top)^{1/2}(M - W) + (M - W)(W^\top W)^{1/2}$$

SOLUTION, p. 591

Exercise 4.7 (The NTK operator). Define the NTK operator for $L = 2$ as:

$$K[F] := (W W^\top)^{1/2}\, F + F\, (W^\top W)^{1/2}$$

Show that the gradient flow from Exercise 4.6 can be written as $\dot{W} = K[M - W]$.

It turns out that this result generalizes to depth L on the balanced manifold:

$$\dot{W} = \sum_{k=1}^{L} (WW^\top)^{\frac{L-k}{L}} (M - W)(W^\top W)^{\frac{k-1}{L}}$$

This is the NTK equation in the case of DLNs. The NTK equation is a gradient flow in function space with NTK being a preconditioning operator for the gradient.

SOLUTION, p. 591

5. Rich regime: saddle-to-saddle

This is the core problem. We derive the exact solution of the rich regime dynamics directly from the self-consistent equation of Section 4, emphasizing the NTK perspective.

Alignment assumption. Start from the balanced gradient flow equation derived in Exercise 4.6:

$$\dot{W} = (WW^\top)^{1/2}(M - W) + (M - W)(W^\top W)^{1/2}$$

We work in the **rich regime** (small initialization) with balanced weights. Assume that the student is **aligned** to the teacher: the singular vectors of $W(t)$ coincide with those of M at all times. Concretely, write:

$$M = U \operatorname{diag}(s_1, \dots, s_d)\, V^\top, \qquad W(t) = U \operatorname{diag}(w_1(t), \dots, w_d(t))\, V^\top$$

where U, V are the left/right singular vectors of M, and $w_\alpha(t) \geq 0$ are the evolving singular values of W.

Exercise 5.1 (Aligned NTK). Under this alignment assumption, show that the NTK operator is aligned to the task, i.e. $K[F]$ preserves the SVD basis. In particular, show that:

$$(WW^\top)^{1/2} = U \operatorname{diag}(w_1, \dots, w_d)\, U^\top,$$
$$(W^\top W)^{1/2} = V \operatorname{diag}(w_1, \dots, w_d)\, V^\top$$

and that the residual is $M - W = U \operatorname{diag}(s_1 - w_1, \dots, s_d - w_d)\, V^\top$.

Hint: Recall that for $A = U \operatorname{diag}(\sigma_i)\, V^\top$, we have $AA^\top = U \operatorname{diag}(\sigma_i^2)\, U^\top$, and therefore $(AA^\top)^{1/2} = U \operatorname{diag}(|\sigma_i|)\, U^\top$.

SOLUTION, p. 591

Exercise 5.2 (Decoupled scalar ODEs). Substitute into the self-consistent equation. Show that the matrix equation decouples into d independent scalar ODEs:

$$\dot{w}_\alpha = 2\, w_\alpha\, (s_\alpha - w_\alpha), \qquad \alpha = 1, \dots, d$$

Hint: Compute the product $(WW^\top)^{1/2}(M - W)$ in the SVD basis. It is diagonal with entries $w_\alpha(s_\alpha - w_\alpha)$. The second term contributes identically, giving the factor of 2.

SOLUTION, p. 591

Exercise 5.3 (Timescale of learning). The ODE $\dot{w} = 2w(s - w)$ is a logistic equation whose solution is:

$$w(t) = \frac{s}{1 + \left(\frac{s}{w_0} - 1\right) e^{-2st}}$$

where $w_0 = w(0)$. Compute the time t_α it takes for mode α to travel from initial strength w_0 to a final strength w_f. Show that:

$$t_\alpha = \frac{1}{2s_\alpha} \ln\left(\frac{w_f\,(s_\alpha - w_0)}{w_0\,(s_\alpha - w_f)}\right)$$

Deduce that modes with larger singular values s_α are learned faster. This is a **separation of timescales**: the network learns features in decreasing order of their singular value strength.

SOLUTION, p. 592

Exercise 5.4 (Incremental learning). Consider a teacher with r nonzero singular values and a small uniform initialization $w_\alpha(0) = w_0 \ll s_r$ for all α. Qualitatively, discuss:

- What is the approximate distribution of singular values of $W(t)$ at early times?
- How do the singular values of $W(t)$ evolve throughout training?
- What happens in the limit $t \to \infty$?

What does this analysis tell us about the sequence of critical points visited by the gradient flow?

SOLUTION, p. 593

6. Lazy regime

We now show that large initialization freezes the NTK and eliminates the timescale separation found in Section 5. For clarity, we work with the **diagonal scalar** model from Exercise 3.1, so each mode α is independent. This avoids matrix algebra and isolates the essential mechanism.

Setup. Consider a single mode: a depth-2 diagonal DLN with scalar weights a, b learning a target $s > 0$. From Sections 4 and 5, the balanced gradient flow for $w = ab$ is:

$$\dot{w} = 2w(s - w)$$

The factor $2w$ is the (scalar) NTK — it is **state-dependent**: the effective learning rate depends on the current value of w.

Exercise 6.1 (Linearizing the NTK). Now initialize at a **large** value $w_0 \gg s > 0$. Assume that in the early phase of training, while w has not changed much from w_0, the ODE becomes approximately:

$$\dot{w} \approx 2w_0(s - w)$$

SOLUTION, p. 593

Exercise 6.2 (Frozen-NTK solution). Solve the linearized ODE from Exercise 6.1. Show that:

$$w(t) = s + (w_0 - s)\, e^{-2w_0 t}$$

What is the learning timescale?

SOLUTION, p. 593

Exercise 6.3 (No timescale separation). Now restore the mode index α. In the lazy regime, each mode satisfies $\dot{w}_\alpha \approx 2w_0(s_\alpha - w_\alpha)$ with the **same** rate $2w_0$ for all α. Compare the lazy learning time t_α^{lazy} with the rich-regime timescale from Exercise 5.3: $t_\alpha^{\text{rich}} \propto 1/s_\alpha$.

SOLUTION, p. 594

7. Mixed dynamics: unifying lazy and rich

In practice, networks are neither purely lazy nor purely rich. Tu, Aranguri & Jacot (2024) provide a unified description. We develop the key ideas using the diagonal scalar model.

Exercise 7.1 (The interpolating ODE). In Sections 5 and 6, we studied the same ODE $\dot{w}_\alpha = 2w_\alpha(s_\alpha - w_\alpha)$ in two limits: small w_0 (rich) and large w_0 (lazy). A more general model for a depth-2 DLN with balanced initialization at scale σ and width n replaces the scalar NTK $2w_\alpha$ by:

$$\dot{w}_\alpha = 2\sqrt{w_\alpha^2 + \tau^2}\,(s_\alpha - w_\alpha)$$

where $\tau := \sigma^2\sqrt{n}$ is a **threshold** parameter that depends on the initialization scale and width.

Verify that this ODE reproduces the two known regimes:

- **Rich limit** ($\tau \to 0$): recover $\dot{w}_\alpha = 2|w_\alpha|(s_\alpha - w_\alpha)$.
- **Lazy limit** ($\tau \to \infty$ with w_α bounded): recover $\dot{w}_\alpha \approx 2\tau(s_\alpha - w_\alpha)$.

SOLUTION, p. 595

Exercise 7.2 (Two-phase dynamics). At initialization, all modes start at $w_\alpha(0) \sim \sigma^2 \ll \tau$ (since $\sqrt{n} \gg 1$). Observe that:

- When $|w_\alpha| \ll \tau$, the effective learning rate is $\approx 2\tau$, the same for all modes (lazy behavior).
- When $|w_\alpha| \gg \tau$, the effective learning rate is $\approx 2|w_\alpha|$, which is mode-dependent (rich behavior).

Describe the resulting two-phase dynamics in words: what happens first, and what happens later?

SOLUTION, p. 595

Exercise 7.3 (Mode-by-mode transition). Not all modes cross the threshold τ at the same time: which modes cross it first? Explain why this means that a single network can simultaneously have some modes in the rich regime and others still in the lazy regime.

SOLUTION, p. 596

→ *8. Stochastic implicit bias (bonus)*: Supplementary Material, p. 495

→ *Learn more*: Supplementary Material, p. 496

References

Achour, Malgouyres, and Gerchinovitz (2024). *The loss landscape of deep linear neural networks: a second-order analysis*. Journal of Machine Learning Research (JMLR).

Saxe, McClelland, and Ganguli (2014). *Exact solutions to the nonlinear dynamics of learning in deep linear neural networks*. International Conference on Learning Representations (ICLR).

Tu, Aranguri, and Jacot (2024). *Mixed Dynamics In Linear Networks: Unifying the Lazy and Active Regimes*.

NOTES

1. The constant is $\frac{1}{2}\mathbb{E}\|y - Mx\|^2$, which is the residual of the OLS. It vanishes on realizable data.

CHAPTER B.5

Data Attribution

Louis Jaburi

Which training examples caused a model to behave the way it does? These notes develop the main approaches to data attribution – counterfactuals and Shapley values, classical and Bayesian influence functions, and training-dynamics unrolling – with derivation exercises throughout.

Source: iliad-intensive.org/learning/data-attribution · CC BY 4.0

In this chapter

Introduction 111
1. Causality and Counterfactuals 112
1.1 Data attribution as causal analysis 112
1.2 Counterfactual attribution and its failures 113
1.3 Beyond leave-one-out: Shapley values 114
1.4 Notation and goals 115
2. Influence Functions 116
2.1 The influence function formula 116
2.2 Translation to modern neural networks 120
3. Bayesian Influence Functions 122
3.1 Bayesian influence functions 122
3.2 Exercise: Connecting Bayesian and classical influence functions 124
3.3 Long Exercise: Influence functions as optimal linear transport 126
4. Unrolling 129
4.1 The training-dynamics approach to attribution 130
4.2 The unrolling formula 131
4.2.1 Materializing the Jacobian 133
4.2.2 Implicit JVPs and the REPLAY algorithm 134
4.3 Long Exercise: From unrolling to influence functions 136
4.4 Exercise: Influence depends on training time 137

Introduction

Contents. These notes are structured as follows:

1. **Causality and Counterfactuals** motivates the problem of data attribution through the lens of causal reasoning, exhibiting the shortcomings of naive counterfactual approaches.

2. **Influence Functions** introduces the classical theory of influence functions for statistical models and discusses their translation to modern neural networks.
3. **Bayesian Influence Functions** introduces susceptibilities and Bayesian influence functions, connecting them to the classical theory via a power series expansion.
4. **Unrolling** presents the training-dynamics approach to attribution, deriving the basic formula and connecting it to influence functions in the appropriate limit.

Prerequisites. We assume the following background, most of it standard for a machine-learning audience.

- *Linear algebra*: vectors, matrices, inner products, eigenvalues, quadratic forms, Hessians.
- *Multivariable calculus*: gradients, the chain rule, Taylor expansion, the implicit function theorem.
- *Probability*: expectations, conditional probability, Bayes' rule, Gaussian distributions.
- *Deep learning basics*: stochastic gradient descent, empirical risk minimisation, a standard supervised training loop.

Familiarity with the material from the Singular Learning Theory chapter (B.3) (degeneracy, the local learning coefficient, posterior broadening) is useful but not required. We flag connections to that chapter in remarks as they arise.

Fast-track. Readers short on time may prefer one of the following routes; each is self-contained.

1. *From classical to modern influence functions.* Skim Section 1 for motivation, then read Section 2 in full, including the derivation exercise.
2. *The Bayesian perspective.* Read Section 2.1 for the influence function formula, then Section 3 in full. The connection exercise (Section 3.2) is the conceptual payoff.
3. *Training-dynamics attribution.* Skim Section 2, then read Section 4 in full. The long exercise in Section 4.3 recovers the influence function as a limit of unrolling.

All three routes depend on the motivational material in Section 1. If reading only one section, read that one.

Acknowledgements. This tutorial was developed for the April 2026 Iliad Intensive and its template was inspired by the Singular Learning Theory chapter. Exercises and examples adapted from specific sources are credited inline.

1. Causality and Counterfactuals

1.1 Data attribution as causal analysis

At the heart of data attribution, we find a causal question: **Which training examples caused a model to behave the way it does?** If mechanistic interpretability aims for the causal analysis of a forward pass, data attribution aims for the causal analysis of a

training run.[1] Our north star is a decomposition of the training data into "causes" of the model's behaviour.

Two asymmetries between the two settings are worth keeping in mind from the start. First, both problems inherit the same conceptual difficulties of causal reasoning in general (explained in the next section) and there is no reason to expect data attribution to escape them. Second, a forward pass is cheap, while a training run is not. Where mechanistic interpretability can validate a claim by running many ablations, data attribution cannot in general afford even one retraining. That being said, there are also advantages: Generally speaking, the training process is more robust and the ablations we run may be more human understandable than high-dimensional modification of activations or weights.

Now, before developing such data attribution methods, we should be aware of the tension between causality and the ways we will validate a hypothesis. Finding a "true" causal relationship in practice can be much more difficult than it may seem at first glance. In the following section we take the question seriously, and find that the naive counterfactual answer fails in ways that matter.

1.2 Counterfactual attribution and its failures

The classical notion of causation is due to Lewis 1973: A caused B iff, had A not occurred, B would not have occurred. Applied to data attribution, this says that a training example "causes" a behaviour when removing it would have altered the behaviour, also known as the *leave-one-out (LOO) counterfactual*. We relax this definition from a binary to a gradual one.

Definition 1.1 (Counterfactual attribution). We attribute an action A to a behaviour B iff

1. had A occurred, B would have occurred to a higher degree;
2. had A not occurred, B would have occurred to a lesser degree (or not at all).

Definition 1.1 is the simplest formalisation of cause. But as Mueller 2024 emphasises, it breaks in at least two cases.

Overdetermination: redundant causes are missed. When two training examples are each *sufficient* to produce a behaviour, removing either one leaves the behaviour intact, and LOO attributes credit to neither.

Example 1.2 (Overdetermination in the bakery). OpenCroissant, a bakery, sources four ingredients: flour, butter, yeast, and sugar. Each can come from a standard or premium supplier. The bakery has observed that using all premium ingredients produces exceptionally fluffy croissants and wishes to identify which upgrades are responsible.

Unbeknownst to the bakery, exceptional fluffiness depends on two independent leavening mechanisms.

- *Steam leavening.* Premium butter has higher fat content and better water distribution, producing more steam and hence more lift.
- *Biological leavening.* A premium yeast strain ferments more vigorously, producing more gas and hence more lift.

Either mechanism alone causes exceptional fluffiness: puff pastry rises from steam alone, and bread rises from yeast alone. Downgrading premium butter to standard therefore has negligible effect on fluffiness (the threshold is still met), and likewise for yeast. Counterfactual attribution (Definition 1.1) concludes that neither butter nor yeast was a cause albeit both were.

The same pattern may arise in learning, where once a behaviour is learned, i.e. has hit saturation, additional examples will not affect it and appear as causally irrelevant.

Non-transitivity: counterfactuals do not chain. Direct ablation will not capture *how* an action causes a behaviour.

Example 1.3 (Non-transitivity of counterfactuals). The following scenario is due to Ned Hall, as reported in Hitchcock 2001.

(A) A hiker is wandering through the mountains when a boulder rolls down the hill.

(B) The hiker sees the boulder and dodges.

(C) The hiker is unharmed.

Counterfactually, (A) caused (B): had the boulder not rolled, the hiker would not have dodged. And (B) caused (C): had the hiker not dodged, she would have been crushed. But (A) did not cause (C): whether or not the boulder rolled, she ends up unharmed.

In the data-attribution setting this corresponds to chains of mediation. A training example may induce an intermediate feature, and that feature may drive a downstream capability, without the datapoint itself passing the counterfactual test against the capability end-to-end. Methods that look only at marginal effects miss the mediation.

1.3 Beyond leave-one-out: Shapley values

A natural response to overdetermination is to stop looking only at the LOO swap and instead ask how a training example contributes across *all* subsets of the training set it could be part of. This is the idea behind *Shapley values* (Shapley 1953).

Definition 1.4 (Shapley value). Let $v : 2^{[n]} \to \mathbb{R}$ be a set function (a "game"). The Shapley value of element i is

$$\varphi_i(v) \;:=\; \frac{1}{n} \sum_{S \subseteq [n] \setminus \{i\}} \binom{n-1}{|S|}^{-1} \big(v(S \cup \{i\}) - v(S)\big).$$

We can think of $[n]$ as a set of players and $v(S)$ as the value of the coalition. In our previous example, the players are the premium ingredients and the value is the resulting fluffiness. Then $v(S)$ will measure the fluffiness of the croissant when the ingredients in S are premium and the rest are standard.

Shapley values partially address overdetermination. Returning to the baker example: in coalitions $S \subset [n]$ where neither butter nor yeast is premium, upgrading one has a large marginal effect; in coalitions where the other leavener is already premium, the marginal effect is small. Averaging gives each ingredient roughly half the credit.

Remark (Shapley still dilutes credit). Shapley values redistribute credit more fairly but do not fully solve overdetermination. Each of butter and yeast receives roughly half the credit for fluffiness, yet each *alone* is sufficient, so one could argue each deserves the full credit. Furthermore, Shapley values over a training set of size n require evaluating $v(S)$ on 2^n coalitions, each demanding a retraining. This is hopeless for realistic models.

1.4 Notation and goals

We close the section by fixing notation that will be used throughout.

- A training set is $\mathcal{D} = \{z_i\}_{i=1}^n$ with $z_i = (x_i, y_i)$.
- A model is a function $f_w : X \to Y$ parameterised by $w \in W \subset \mathbb{R}^{d_p}$.
- The per-example loss is $L_j(w) = L(w, z_i) = \ell(f_w(x_i), y_i)$ for some pointwise loss ℓ.
- We introduce a vector of *data weights* $\beta = (\beta_1, \dots, \beta_n) \in \mathbb{R}^n$ and define the *weighted training loss*

$$L(w; \beta) \;:=\; \frac{1}{n} \sum_{i=1}^{n} \beta_i \, L(w, z_i),$$

and we write $L(w) := L(w, 1)$ for the unweighted loss.

- The behaviour we want to attribute is a differentiable scalar observable $\phi : W \to \mathbb{R}$. This could be the loss on a held-out point, an average loss on a task, the logit of a specific completion, or any other scalar function of the trained parameters.

In general we want to know "If I change β_i, how will this affect ϕ?" This question is typically split into two parts:

- *Parameter influence*: "If I change β_i, how will this affect the final parameters?"
- *Influence on ϕ*: "If I change the final parameters, how will this affect ϕ?"

In practice we may think of the influence then as a measure of the composition of these maps

$$\begin{aligned} B &\xrightarrow{\pi} \text{"}W\text{"} \to \mathbb{R} \\ \beta &\mapsto w(\beta) \mapsto \phi(w(\beta)). \end{aligned}$$

where we are intentionally vague about the first map and where the different approaches, influence functions, Bayesian influence functions, and unrolling, differ in how they interpret it.

That being said, all attribution methods in this chapter can be phrased as estimators of partial derivatives or finite differences of this map. The *influence* of an example i is then computed as first-order approximation:

$$\boxed{\left.\frac{\partial\, \phi(\pi(\beta))}{\partial \beta_i}\right|_{\beta=\mathbf{1}}}$$

2. Influence Functions

In this section we develop *influence functions*: a closed-form, first-order approximation to the leave-one-out (LOO) counterfactual. The classical theory (Hampel 1974; Cook 1977) predates machine learning by decades. It was developed in robust statistics, where the question was which observations a parameter estimate is most sensitive to. For simple statistical models the answer is short: a single application of the implicit function theorem yields an inverse-Hessian-times-gradient formula whose factors have a clean operational interpretation.

The formula we arrive at then *can* be translated to the setup of neural networks, but its interpretation becomes unclear: The classical derivation assumes a unique global minimum and an invertible Hessian, neither of which holds for a neural network which is typically **not** trained to full convergence. We will see a partial fix for this issue in Section 2.2.

2.1 The influence function formula

Influence functions originate in robust statistics (Hampel 1974): given a statistical estimator, how much does it change when a single observation is added or perturbed?

Example 2.1 (Influence on the mean). Let $T(F) = \mathbb{E}_F[z]$ be the mean of a distribution F. If we contaminate F by placing a fraction ϵ of its mass at a point z, the new mean is

$$T\big((1-\epsilon)F + \epsilon\,\delta_z\big) \;=\; (1-\epsilon)\,\mathbb{E}_F[z] \;+\; \epsilon\, z \;=\; T(F) \;+\; \epsilon\,(z - T(F)).$$

The rate of change at $\epsilon = 0$ is $z - T(F)$: the influence of z on the mean is simply how far z is from the current mean. Observations far from the centre of the data move the mean the most.

We now translate this idea into the setting of Section 1.4, whose notation we use throughout. Recall that the weighted training loss $L(w;\beta) = \sum_i \beta_i L(w, z_i)$ defines a family of optimization problems parameterized by the weight vector β. In the case of influence functions, we measure the influence of a data point on the *optimal* parameters

$$w^*(\beta) \;:=\; \arg\min_{w \in W} L(w;\beta),$$

and we write $w^* := w^*(\mathbf{1})$ for the trained parameters. We assume throughout this subsection that w^* is the unique global minimum of L and that the Hessian $H := \nabla_w^2 L(w^*)$ is positive definite, so that the implicit function theorem makes $w^*(\beta)$ well-defined and smooth in a neighborhood of $\beta = \mathbf{1}$. In the notation of Equation 1 we set $\pi(\beta) = w^*(\beta)$ and Equation 3 yields then the following formula:

Definition 2.2 (Influence function). The *parameter influence* of training example z_i at the trained parameters w^* is

$$\mathcal{I}_{\text{param}}(z_i) \;:=\; \left.\frac{\partial w^*(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}}.$$

Under the assumptions above, the implicit function theorem gives (Exercise 2.1)

$$\boxed{\mathcal{I}_{\text{param}}(z_i) \;=\; -\,H^{-1}\,\nabla_w L(w^*, z_i).}$$

where H is the Hessian of L at w^*. For any differentiable observable $\phi : W \to \mathbb{R}$, the chain rule then yields the *influence on* ϕ:

$$\begin{aligned}\mathcal{I}(z_i, \phi) \;&:=\; \left.\frac{\partial\, \phi(w^*(\beta))}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \\ &=\; \nabla_w \phi(w^*)^\top\, \mathcal{I}_{\text{param}}(z_i) \\ &=\; -\,\nabla_w \phi(w^*)^\top\, H^{-1}\, \nabla_w L(w^*, z_i).\end{aligned}$$

Thus we get three components

- $\nabla_w L(w^*, z_i)$ is the direction in parameter space along which z_i pulls the optimizer.
- H^{-1} rescales this direction by local curvature: directions in which the loss is flat (small eigenvalue of H) get amplified, since a small force in a flat direction moves the minimum a lot.
- $\nabla_w \phi(w^*)$ projects the parameter change onto the direction that matters for the observable ϕ.

In this section, we will focus on $\mathcal{I}_{\text{param}}(z_i)$. The influence on any observable ϕ is obtained by computing the dot product $\mathcal{I}_{\text{param}}(z_i) \cdot \nabla_w \phi(w^*)$.

Connection to leave-one-out. Setting $\beta = \mathbf{1} - e_i$ (i.e., removing z_i) and applying the first-order Taylor expansion gives

$$w^*(\mathbf{1} - e_i) \;-\; w^* \;\approx\; -\,\mathcal{I}_{\text{param}}(z_i) \;=\; H^{-1}\,\nabla_w L(w^*, z_i),$$

i.e. the parameter influence is, to first order, the change in w^* that would result from removing z_i from the training set entirely.

Exercise 2.1 (Derivation of the influence function). Let $w^*(\beta) = \arg\min_{w \in W} L(w; \beta)$ as above and assume the Hessian $H = \nabla_w^2 L(w^*)$ is invertible. Recall that the *implicit function theorem* states: if $F : W \times B \to \mathbb{R}^{\dim W}$ is C^1, $F(w^*, \mathbf{1}) = 0$, and the Jacobian $\partial F / \partial w$ at $(w^*, \mathbf{1})$ is invertible, then there is a unique C^1 map $\beta \mapsto w^*(\beta)$ on a neighborhood of $\mathbf{1}$ with $F(w^*(\beta), \beta) \equiv 0$, whose derivative is obtained by implicit differentiation of F.

(a) Apply the implicit function theorem to $F(w, \beta) := \nabla_w L(w; \beta)$ to conclude that $\beta \mapsto w^*(\beta)$ is well-defined and C^1 near $\mathbf{1}$. Which hypothesis of the theorem uses the invertibility of H? Where do we use that w^* is the unique global minimum (rather than merely a critical point)?

(b) By differentiating the first-order optimality condition $\nabla_w L(w^*(\beta); \beta) = 0$ with respect to β_i, show that

$$\left. \frac{\partial w^*(\beta)}{\partial \beta_i} \right|_{\beta = \mathbf{1}} \;=\; -\, H^{-1}\, \nabla_w L(w^*, z_i).$$

(c) Use the chain rule to obtain the formula for $\mathcal{I}(z_i, \phi)$ in Definition 2.2. Make sure to keep track of the evaluation at $\beta = \mathbf{1}$.

SOLUTION, p. 598

Exercise 2.2 (Linear regression). Consider linear regression with training data $\{(x_i, y_i)\}_{i=1}^n$, $x_i \in \mathbb{R}^p$, and squared loss $L(w, (x_i, y_i)) = \tfrac{1}{2}(y_i - w^\top x_i)^2$. Write $A := \sum_i x_i x_i^\top$. In this case the minimizer, the influence function, and the exact LOO update all have closed-form solutions. We will see that in general the influence functions can diverge in extreme cases.

(a) Derive the minimizer w^* by setting $\nabla_w L(w) = 0$. Show that $w^* = A^{-1} \sum_i x_i y_i$.

(b) Compute the parameter influence $\mathcal{I}_{\text{param}}(z_j)$ from Definition 2.2.

(c) Define $w^*(\beta_j) := \arg\min_w \sum_{i \neq j} L(w, z_i) + \beta_j L(w, z_j)$, so that $\beta_j = 1$ is the unperturbed model and $\beta_j = 0$ corresponds to fully removing z_j. Using the Sherman–Morrison formula[2], show that

$$w^*(\beta_j) \;-\; w^* \;=\; -\,\frac{(1 - \beta_j)\, A^{-1} x_j\, r_j}{1 - (1 - \beta_j)\, h_j},$$

where $r_j := y_j - x_j^\top w^*$ is the residual at z_j and $h_j := x_j^\top A^{-1} x_j$ is the *leverage* of x_j. Note that this is a rational function of β_j. This is the ground truth effect of perturbing z_j.

(d) Expand the closed form from (c) as a Taylor series in $(1 - \beta_j)$ around the unperturbed point $\beta_j = 1$ by recognizing $1/(1 - (1 - \beta_j)h_j)$ as a geometric series, and obtain

$$w^*(\beta_j) - w^* \;=\; -\, A^{-1} x_j\, r_j \sum_{k=1}^{\infty} (1 - \beta_j)^k\, h_j^{k-1}.$$

Identify the $k = 1$ term with the parameter influence from part (b), and observe that the higher-order terms ($k \geq 2$) are suppressed by powers of the leverage h_j.

(e) Specialize to the LOO endpoint $\beta_j = 0$. Sum the geometric series in h_j to obtain the closed-form deletion update

$$w^*_{(-j)} - w^* \;=\; -A^{-1}x_j\, r_j \left(1 + h_j + h_j^2 + h_j^3 + \cdots\right) \;=\; -\frac{A^{-1}x_j\, r_j}{1 - h_j}.$$

The IF prediction is the h_j^0 leading term. Discuss the two regimes $h_j \to 0$ and $h_j \to 1$: when does keeping only the linear term suffice, and when do all the higher-order corrections matter?

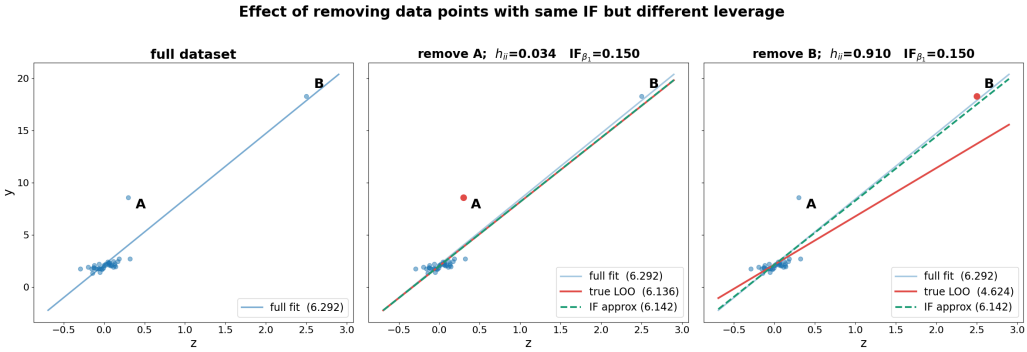
SOLUTION, p. 599

Remark (What is leverage?). The quantity $h_j = x_j^\top A^{-1} x_j$ that appeared in Exercise 2.2 measures how *unusual* the input x_j is relative to the bulk of the design. To see this, observe that $\widehat{\Sigma}_{xx} := A/n = \frac{1}{n}\sum_i x_i x_i^\top$ is the empirical second-moment matrix of the inputs, so

$$h_j \;=\; \tfrac{1}{n}\, x_j^\top \widehat{\Sigma}_{xx}^{-1}\, x_j$$

is $1/n$ times a *Mahalanobis-style* squared norm of x_j, taken in the metric defined by the data itself. Geometrically: directions that the data samples a lot correspond to large eigenvalues of $\widehat{\Sigma}_{xx}$ and contribute little to h_j, while directions that are barely sampled correspond to small eigenvalues of $\widehat{\Sigma}_{xx}$ and so blow up under $\widehat{\Sigma}_{xx}^{-1}$. *Inputs that point in under-sampled directions get large leverage.* High-leverage points are unusual in feature space.

Figure 1 illustrates the effect on a simple linear regression: removing a low-leverage point (A, near the bulk of the data) barely changes the fit, and the IF approximation is accurate. Removing a high-leverage point (B, an outlier on the x-axis) changes the fit dramatically, and the IF underestimates the true LOO effect.



Influence approximation vs. LOO ground truth for a linear regression. *Left:* the full dataset. *Centre:* removing the low-leverage point A ($h_A \approx 0.03$) barely perturbs the fit; the IF prediction closely matches LOO. *Right:* removing the high-leverage point B ($h_B \approx 0.83$) moves the regression line substantially; the IF underestimates the true change. Note that both points have the same parameter influence.

2.2 Translation to modern neural networks

The derivation in Section 2.1 relied on two assumptions:

(i) w^* is the unique global minimum of the training loss;

(ii) the Hessian $H = \nabla_w^2 L(w^*)$ is positive definite.

While neither holds for a modern neural network, the quantities in Definition 2.2 are, in principle, computable. Koh & Liang 2020 popularized the influence function formula as a tool for deep learning, and a substantial body of work has tried to make it useful at scale.

(1) Non-uniqueness and a singular Hessian. For a neural network the global minimum is usually not unique. The set of minimizers is generically a positive-dimensional zero locus, and the Hessian at any point on it is singular: it has a nontrivial null space corresponding to directions of parameter movement that leave the loss unchanged. This is the degeneracy phenomenon observed in the Singular Learning Theory chapter, and it means we cannot invert H as in Definition 2.2.

The standard remedy is to *dampen* the Hessian: replace H^{-1} with $(H + \lambda I)^{-1}$ for some $\lambda > 0$. This is the same as adding an L_2-regularizer $\frac{\lambda}{2}\|w - w^*\|^2$ to the training objective $L(w, \beta)$; this yields a new loss function $L_{\text{reg}}(w, \beta) = L(w, \beta) + \frac{\lambda}{2} \cdot \|w - w^*\|^2$.

(2) Negative curvature. Even after damping away the zero eigenvalues, H at a non-converged checkpoint can have negative eigenvalues, in which case $(H + \lambda I)^{-1}$ may still be ill-conditioned or even point in the wrong direction. The standard fix is to replace H with the *Gauss–Newton Hessian* (GNH)

$$G \;=\; \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[J^\top H_y J\big],$$

where $J = \partial f_w / \partial w$ is the parameter-output Jacobian and H_y is the Hessian of the per-example loss with respect to the network outputs (not the parameters). For the cross-entropy and squared losses H_y is positive semidefinite, hence so is G. The GNH is a local approximation to the true Hessian that ignores second derivatives of the network outputs with respect to the parameters. It is not hard to show that G and H differ by a term that vanishes at a critical point of L, so the GNH is a reasonable proxy for H when w^* is close to optimal.

(3) Non-converged checkpoints. In practice w^* is not a critical point of L. The classical formula in Definition 2.2 is then evaluated at a point where $\nabla L \neq 0$.

The fix proposed by Bae et al. 2022 is to replace the original training loss by a surrogate for which w^* *is* optimal by construction. Concretely, define the *proximal Bregman objective* (PBO) as

$$L_{\mathrm{PBO}}(w; \beta) \;=\; \sum_{i=1}^{n} D_{L_y}\big(f_w(x_i),\, f_{w^*}(x_i)\big) \;+\; \sum_{i=1}^{n} (\beta_i - 1)\, L(w, z_i) \\ +\; \frac{\lambda}{2}\, \|w - w^*\|^2,$$

where $D_{L_y}(\hat{y}, \hat{y}^*)$ is the Bregman divergence of the output-space loss L_y around $\hat{y}^*$. The first term penalizes any change in the network's predictions on the training set, with the current model w^* as the reference; the second is the perturbation of interest; the third is the damping. By construction, w^* is the minimizer of $L_{\mathrm{PBO}}(\,\cdot\,; \mathbf{1})$ regardless of whether it is a critical point of the original loss. The minimizer of $L_{\mathrm{PBO}}(\,\cdot\,; \beta)$ for nearby β defines the *proximal Bregman response function* (PBRF).

When one re-derives the influence function formula starting from L_{PBO} instead of L, the result is

$$\mathcal{I}_{\mathrm{PBRF}}(z_i, \phi) \;=\; -\,\nabla_w \phi(w^*)^{\top}\, (G + \lambda I)^{-1}\, \nabla_w L(w^*, z_i),$$

the same expression that the modern literature uses. This is effectively the counterfactual it approximates. As Bae et al. 2022 put it: *if influence functions are the answer, the question is the PBRF, not LOO retraining.* We refer to their paper for a detailed decomposition of the remaining gap between the PBRF and the original LOO counterfactual.

Remark (Connection to singular learning theory). A central obstacle to the application of influence functions to modern neural networks is *degeneracy*: the set of minimizers of an overparameterized model is singular and the Hessian has a large null space. In Section 3 we will see a degeneracy-aware alternative: Bayesian influence functions, which replace inverse-Hessian computations with covariances over a localized posterior.

Remark (Scaling and EK-FAC). Even with the PBRF reframing, computing $(G + \lambda I)^{-1}$ for a billion-parameter model is not directly feasible. Grosse et al. 2023 scale influence functions to large language models by approximating G with EK-FAC (a Kronecker-factored approximation to layerwise curvature).

Exercise 2.3 (Failure of the first-order approximation in a degenerate model). Consider the toy two-parameter model from the Singular Learning Theory chapter in which $w = (a, b)$ enters the loss only through the product $u = ab$, with squared loss $L(w, z) = \frac{1}{2}(z - ab)^2$ on training data $\{z_1, \ldots, z_n\} \subset \mathbb{R}$.

(a) Write down the Hessian of the training loss at any minimizer (a^*, b^*) with $a^* b^* = \bar{z} := \frac{1}{n}\sum_i z_i$. Show that it has rank one, and identify the one-dimensional null space geometrically.

(b) Compute the influence function $\mathcal{I}(z_i, \phi)$ for the observable $\phi(w) = ab$, treating H^{-1} as a Moore–Penrose pseudoinverse[3]. How does it compare with the exact LOO change in ϕ?

(c) Repeat (b) using the damped inverse $(H + \lambda I)^{-1}$ for a small $\lambda > 0$. Discuss how the answer depends on λ, on the choice of minimizer (a^*, b^*) along the singular set, and on n. What does this say about the proximity gap?

SOLUTION, p. 601

3. Bayesian Influence Functions

Classical influence functions give a first-order approximation to the change in a *point estimate* w^* when the training data is perturbed. The Bayesian reformulation replaces this point estimate by a *distribution* over parameters: if we perturb the training data, how does our posterior over the parameter space change? This leads naturally to the Bayesian generalization of influence functions introduced in Kreer et al. 2026. In the setting of Equation 1 we set $\pi(\beta)$ not to be a single weight, but a whole distribution over parameters p_β, namely the posterior.

Susceptibilities. The Bayesian influence function is one instance of a more general object studied by Baker et al. 2026. Given a tempered Gibbs posterior $p_\beta \propto e^{-\beta L(w)}\varphi(w)$ at inverse temperature β, any pair of observables $\phi, \psi : W \to \mathbb{R}$ defines a *susceptibility*

$$\chi(\phi \,|\, \psi) \;:=\; \frac{\partial}{\partial h}\,\mathbb{E}_{p_h}[\phi(w)]\Big|_{h=0}, \qquad p_h \;\propto\; e^{-\beta\left(L(w) - h\,\psi(w)\right)}\,\varphi(w).$$

The differentiation argument we will apply in Exercise 3.1 shows $\chi(\phi \,|\, \psi) = \beta\,\mathrm{Cov}_{p_\beta}(\phi, \psi)$ in full generality. Taking $\psi = -L(\cdot, z_i)$ gives the Bayesian influence function developed below. Other choices of (ϕ, ψ) yield other sensitivity notions, see e.g. the refined learning coefficients (Baker et al. 2026, Appendix D.2). We will not need the general framework again; we mention it only to locate the BIF within the wider susceptibility family.

3.1 Bayesian influence functions

We now specialize the susceptibility story to data attribution. Instead of asking "how does $\phi(w^*)$ change when we perturb the training weights?" (Definition 2.2) we ask "how does $\mathbb{E}_{w\sim p}[\phi(w)]$ change?" The answer turns out to be remarkably clean: the

derivative becomes a covariance, and the Hessian inversion of the classical formula is replaced by posterior sampling.

The tempered posterior. Given training data $\mathcal{D} = \{z_1, \dots, z_n\}$ with per-sample losses $L(w, z_i)$, sample weights $\beta = (\beta_1, \dots, \beta_n)$, and a prior $\varphi(w)$, define the *tempered posterior*

$$p_\beta(w \mid \mathcal{D}) \;\propto\; \exp\Big(-\sum_{i=1}^{n} \beta_i \, L(w, z_i)\Big)\, \varphi(w).$$

When $\beta = \mathbf{1}$ and the loss is a negative log-likelihood, this is the standard Bayesian posterior. When the losses are not log-likelihoods, p_β is a *Gibbs measure*; the mathematics is identical.

Definition 3.1 (Bayesian influence function). The *Bayesian influence function* (BIF) of training example z_i on the observable ϕ is the derivative of the posterior expectation of ϕ with respect to the sample weight β_i:

$$\mathrm{BIF}(z_i, \phi) \;:=\; \frac{\partial}{\partial \beta_i}\, \mathbb{E}_{w \sim p_\beta}[\phi(w)] \Big|_{\beta = \mathbf{1}}.$$

The definition is the natural Bayesian analogue of Definition 2.2: replace $\phi(w^*(\beta))$ with $\mathbb{E}_{p_\beta}[\phi(w)]$. The following exercise shows that this derivative has a clean closed form. Recall that for two real-valued random variables X and Y defined on the same probability space, the *covariance* is

$$\mathrm{Cov}(X, Y) \;=\; \mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y].$$

When X and Y are functions of the random parameter $w \sim p$, we write $\mathrm{Cov}_{w \sim p}(X(w), Y(w))$ to indicate that the expectation is taken over the posterior p. If ϕ is vector-valued ($\phi : W \to \mathbb{R}^d$), the covariance $\mathrm{Cov}(X, \phi)$ is the vector whose j-th entry is $\mathrm{Cov}(X, \phi_j)$.

Exercise 3.1 (The BIF is a covariance). Let $p_\beta(w \mid \mathcal{D})$ be the tempered posterior of Equation 8 and write $p = p_{\mathbf{1}}$ for the unperturbed posterior. By differentiating $\mathbb{E}_{p_\beta}[\phi(w)] = \int \phi(w)\, p_\beta(w \mid \mathcal{D})\, dw$ with respect to β_i, show that

$$\boxed{\mathrm{BIF}(z_i, \phi) \;=\; -\,\mathrm{Cov}_{w \sim p}\big(L(w, z_i),\; \phi(w)\big).}$$

Hint: Write $p_\beta \propto e^{-\sum_j \beta_j L(w, z_j)} \varphi(w)$ and differentiate the ratio $\int \phi\, p_\beta\, dw \,/\, \int p_\beta\, dw$ using the quotient rule. The key identity is $\partial_{\beta_i} \log Z(\beta) = -\mathbb{E}_{p_\beta}[L(w, z_i)]$.

SOLUTION, p. 603

Remark (Statistical physics). The identity $\partial_{\beta_i} \mathbb{E}[\phi] = -\mathrm{Cov}(L_i, \phi)$ is a standard fluctuation–response relation in statistical physics: the response of an observable to a change in an external field equals the covariance of that observable with the conjugate

energy.

Localization. Computing expectations over the global posterior $p(w \mid \mathcal{D})$ is generally intractable for neural networks, and we are typically interested in the behavior of a specific trained checkpoint w^*, not a global Bayesian average. Following Kreer et al. 2026, we *localize* the posterior by adding a Gaussian penalty centered at w^*:

$$p_\gamma(w \mid \mathcal{D}, w^*) \;\propto\; \exp\Big(-\sum_{i=1}^{n} L(w, z_i) - \frac{\gamma}{2}\|w - w^*\|^2\Big),$$

where $\gamma > 0$ is a *localization strength* that controls how tightly the posterior concentrates around w^*. The *local BIF* is then

$$\mathrm{BIF}_\gamma(z_i, \phi) \;=\; -\,\mathrm{Cov}_\gamma\big(L(w, z_i),\; \phi(w)\big),$$

where Cov_γ denotes covariance under p_γ. We will see in Exercise 3.2 that the localization parameter γ plays exactly the role of the damping parameter λ in the PBRF formula (6).

Practical computation via SGLD. The local BIF requires estimating a covariance under p_γ. This is done by drawing samples from p_γ using stochastic gradient Langevin dynamics (SGLD), which adds calibrated Gaussian noise to SGD updates:

$$w_{t+1} \;=\; w_t - \frac{\epsilon}{2}\Big(\frac{n}{m}\sum_{k \in B_t} \nabla_w L(w_t, z_k) + \gamma(w_t - w^*)\Big) + \eta_t,$$

where $\eta_t \sim \mathcal{N}(0, \epsilon\, I)$, B_t is a mini-batch of size m and ϵ is the step size. After a burn-in period, the iterates $\{w_t\}$ are approximately distributed according to p_γ, and the covariance in (11) is estimated by the sample covariance of $(L(w_t, z_i), \phi(w_t))$ across draws. Running multiple independent chains from w^* improves coverage.

Example 3.2 (Bayesian linear regression). Take the linear regression setup of Exercise 2.2 with a Gaussian prior $w \sim \mathcal{N}(0, \tau^2 I)$ added. The posterior is conjugate: $w \mid \mathcal{D} \sim \mathcal{N}(\mu, V)$ with $V = (\tau^{-2} I + \sigma^{-2} A)^{-1}$ and $\mu = \sigma^{-2} V \sum_i x_i y_i$. A direct Gaussian-moment computation applied to (9) with the observable $\phi(w) = w$ gives

$$\mathrm{BIF}(z_j, w) \;=\; -\mathrm{Cov}_p\big(L(w, z_j),\, w\big) \;=\; \frac{V\, x_j\, r_j}{\sigma^2} \;=\; (A + \lambda I)^{-1}\, x_j\, r_j$$

where $r_j = y_j - x_j^\top \mu$ is the posterior-mean residual and $\lambda := \sigma^2/\tau^2$. This is the *damped* influence function with the damping parameter λ equal to the ratio of observation noise to prior variance. In the flat-prior limit $\tau \to \infty$ we have $\lambda \to 0$, $\mu \to w^*$, and we recover the classical IF of Exercise 2.2.

3.2 Exercise: Connecting Bayesian and classical influence functions

As we saw in the Singular Learning Theory chapter, tools involving the Hessian are usually a low-order approximation of an expansion. This is also true for influence functions: In the non-degenerate case the classical IF is the leading-order term of the BIF

under a Laplace approximation. The following exercise makes this precise by computing a power series expansion of the BIF covariance and identifying the classical IF as the leading-order term. The argument follows Kreer et al. 2026, Appendix A.

Exercise 3.2 (Power series expansion of Bayesian influence functions). Let w^* be a local minimum of the training loss $L(w) = \sum_{i=1}^n L(w, z_i)$ with positive-definite Hessian $H = \nabla_w^2 L(w^*)$. Write $\Delta w = w - w^*$, and abbreviate $g_\phi = \nabla_w \phi(w^*)$, $H_\phi = \nabla_w^2 \phi(w^*)$ for the gradient and Hessian of the observable, and $g_i = \nabla_w L(w^*, z_i)$, $H_i = \nabla_w^2 L(w^*, z_i)$ for the per-sample loss.

(a) (Laplace approximation.) Consider the posterior $p(w \mid \mathcal{D}) \propto e^{-L(w)}\, \varphi(w)$. Taylor-expand $L(w)$ to second order around w^*, using the fact that $\nabla L(w^*) = 0$ at a minimum. Argue that for large n the quadratic term in L dominates the prior φ, and conclude that

$$p(w \mid \mathcal{D}) \;\approx\; \mathcal{N}(w^*,\; H^{-1}).$$

Under this approximation, $\Delta w \sim \mathcal{N}(0, H^{-1})$. (This is called the *Laplace approximation*; the Bernstein–von Mises theorem guarantees it is asymptotically exact for non-singular models.)

(b) (Taylor expansion of the covariance.) Expand $\phi(w)$ and $L(w, z_i)$ in Taylor series around w^*:

$$\begin{aligned} \phi(w) &= \phi(w^*) + g_\phi^\top \Delta w + \tfrac{1}{2} \Delta w^\top H_\phi\, \Delta w + \cdots, \\ L(w, z_i) &= L(w^*, z_i) + g_i^\top \Delta w + \tfrac{1}{2} \Delta w^\top H_i\, \Delta w + \cdots. \end{aligned}$$

Using the fact that constant terms drop out of covariances and that $\operatorname{Cov}(X, Y) = \sum_{k,m \geq 1} \operatorname{Cov}(T_k[\phi],\, T_m[L_i])$ where $T_k[f]$ denotes the k-th order term in the Taylor expansion of f, show that the BIF decomposes as

$$\operatorname{BIF}(z_i, \phi) = - \sum_{\substack{k,m \geq 1 \\ k+m \text{ even}}} \operatorname{Cov}_{\mathcal{N}}\big(T_k[\phi],\; T_m[L_i]\big).$$

Why do terms with $k + m$ odd vanish?

(c) (Leading order: classical IF.) Compute the $(k, m) = (1, 1)$ term:

$$\operatorname{Cov}_{\mathcal{N}}\big(g_\phi^\top \Delta w,\; g_i^\top \Delta w\big) \;=\; g_\phi^\top H^{-1} g_i.$$

Conclude that $-g_\phi^\top H^{-1} g_i = -\nabla_w \phi(w^*)^\top H^{-1} \nabla_w L(w^*, z_i)$ is the classical influence function $\mathcal{I}(z_i, \phi)$ from Definition 2.2. This is the leading-order term of the BIF.

(d) (Second-order correction.) Compute the $(k, m) = (2, 2)$ term using Isserlis' theorem (the Gaussian moment identity $\mathbb{E}[\Delta w_a \Delta w_b \Delta w_c \Delta w_d] = \Sigma_{ab}\Sigma_{cd} + \Sigma_{ac}\Sigma_{bd} + \Sigma_{ad}\Sigma_{bc}$ with $\Sigma = H^{-1}$). Show that

$$\operatorname{Cov}_{\mathcal{N}}\big(\tfrac{1}{2} \Delta w^\top H_\phi\, \Delta w,\; \tfrac{1}{2} \Delta w^\top H_i\, \Delta w\big) = \tfrac{1}{2}\, \operatorname{tr}\big(H_\phi\, H^{-1}\, H_i\, H^{-1}\big).$$

This correction involves the *Hessians* of the observable and per-sample loss. It captures second-order curvature interactions that the classical IF misses entirely. In the linear regression setting of Example 3.2, why does this correction vanish?

(e) (Localized version: damped IF.) Now consider the local BIF of Equation 11 with localization strength γ. The localized posterior is approximately $\mathcal{N}(w^*, (H+\gamma I)^{-1})$. Repeat the leading-order computation of part (c) to show that

$$\mathrm{BIF}_\gamma(z_i, \phi) \;\approx\; -\nabla_w \phi(w^*)^\top (H+\gamma I)^{-1} \nabla_w L(w^*, z_i).$$

This is precisely the damped influence function of Equation 6, with the localization strength γ playing the role of the damping parameter λ. The local BIF is thus a natural, higher-order generalization of the damped IF: it agrees at leading order and includes all the corrections from parts (b)–(d) with H^{-1} replaced by $(H+\gamma I)^{-1}$.

SOLUTION, p. 603

Remark. We conclude that the classical (damped) IF is the leading term of the (local) BIF under a Laplace approximation. For non-singular models with enough data, the Laplace approximation converges and the higher-order terms are small. But for singular models (neural networks), the Laplace approximation fails, the posterior is not Gaussian, and the BIF captures geometry that the classical IF cannot see.

3.3 Long Exercise: Influence functions as optimal linear transport

The previous exercise showed that the classical IF is the leading-order term of the BIF under a Laplace approximation. We now take a different perspective, due to Mlodozeniec et al. 2025: instead of asking "what does the BIF reduce to?" we ask "what is the *best* way to shift the parameters to approximate the effect of a data perturbation?" The answer turns out to be: the influence function, and its optimality does not require the Laplace approximation to hold.

Setup. Let $L(w) = \sum_{i=1}^n L(w, z_i)$ denote the total training loss, and abbreviate $L_k(w) := L(w, z_k)$ for the per-sample loss at example k. Define the *Boltzmann distributions* at inverse temperature β:

$$p_0^\beta(w) \propto e^{-\beta L(w)}, \quad p_\varepsilon^\beta(w) \propto e^{-\beta\left(L(w) - \varepsilon L_k(w)\right)},$$

so that p_ε^β is the Boltzmann distribution after downweighting sample k by ε. The low-temperature limit $\beta \to \infty$ concentrates p_0^β on the set of global minima $\mathcal{S}_L = \{w : L(w) = \inf L\}$.

Now suppose we want to approximate the effect of the perturbation without resampling from p_ε^β. The simplest approach is to take each parameter w drawn from p_0^β and *shift* it to $w + \varepsilon\, r(w)$ for some smooth vector field $r : \mathbb{R}^d \to \mathbb{R}^d$. If r is chosen well, the distribution of the shifted parameters should be close to p_ε^β. The question is: what is the optimal r?

To make this precise, we need to know the density of the shifted parameters. If $w \sim p_0^\beta$ and we set $\tilde{w} = w + \varepsilon\, r(w)$, then by the multivariate change-of-variables formula, $\tilde{w}$ has density

$$q_\varepsilon(\tilde{w}) \;=\; \frac{p_0^\beta\big(\tilde{w} - \varepsilon\, r(\tilde{w}) + O(\varepsilon^2)\big)}{\big|\det\big(I + \varepsilon\, \nabla r(w)\big)\big|}\,,$$

where ∇r is the $d \times d$ Jacobian matrix of r and the denominator is the usual volume-change factor.[4] We measure how well q_ε approximates p_ε^β by the KL divergence $D_{\mathrm{KL}}(q_\varepsilon \| p_\varepsilon^\beta)$. Rather than computing this directly, it is cleaner to write it as an expectation over the *original* distribution p_0^β:

$$D_{\mathrm{KL}}(q_\varepsilon \| p_\varepsilon^\beta) \;=\; \mathbb{E}_{w\sim p_0^\beta}\Big[\log p_0^\beta(w) - \log p_\varepsilon^\beta\big(w + \varepsilon r(w)\big) - \log\big|\det\big(I + \varepsilon \nabla r(w)\big)\big|\Big].$$

This identity follows by substituting (12) into the definition of KL and changing variables back to w. We define the *asymptotic KL cost*

$$\mathcal{F}(r, \varepsilon) \;:=\; \lim_{\beta\to\infty} \frac{1}{\beta}\, D_{\mathrm{KL}}(q_\varepsilon \| p_\varepsilon^\beta).$$

The $1/\beta$ normalization kills the determinant term (which is $O(1)$) and the entropy terms, isolating the energy contribution (which is $O(\beta)$).

Exercise 3.3 (Influence functions as optimal parameter shifts). Let w^* be a non-degenerate minimum of L with positive-definite Hessian $H = \nabla^2 L(w^*)$. In this exercise we work at the unique minimum, so the $\beta \to \infty$ limit simply evaluates everything at w^*.

(a) (KL in terms of energies.) Starting from (13), substitute $p_0^\beta(w) = e^{-\beta L(w)}/Z(\beta, 0)$ and $p_\varepsilon^\beta(w) = e^{-\beta(L(w) - \varepsilon L_k(w))}/Z(\beta, \varepsilon)$, and show that

$$\frac{1}{\beta}\, D_{\mathrm{KL}} \;= \mathbb{E}_{p_0^\beta}\big[L(w + \varepsilon r) - \varepsilon\, L_k(w + \varepsilon r) - L(w)\big] \;+\; \frac{1}{\beta}\log\frac{Z(\beta, \varepsilon)}{Z(\beta, 0)} \;-\; \frac{1}{\beta}\,\mathbb{E}_{p_0^\beta}\big[\log|\det(I + \varepsilon \nabla r)|\big],$$

where $Z(\beta, \varepsilon) = \int e^{-\beta(L(w) - \varepsilon L_k(w))}\, dw$. Explain why the last two terms vanish in the $\beta \to \infty$ limit. *Hint:* $\log|\det(I + \varepsilon\nabla r)| = O(1)$, and by Laplace's method $\frac{1}{\beta}\log Z(\beta, \varepsilon) \to -\inf_w [L(w) - \varepsilon L_k(w)]$.

(b) (Taylor expansion at w^*.) Since the expectation under p_0^β concentrates at w^* as $\beta \to \infty$, expand $L(w^* + \varepsilon r)$ and $\varepsilon\, L_k(w^* + \varepsilon r)$ using $\nabla L(w^*) = 0$:

$$L(w^* + \varepsilon r) = L(w^*) + \tfrac{\varepsilon^2}{2}\, r^\top H\, r + O(\varepsilon^3),$$
$$\varepsilon\, L_k(w^* + \varepsilon r) = \varepsilon\, L_k(w^*) + \varepsilon^2\, \nabla L_k(w^*)^\top r + O(\varepsilon^3).$$

Similarly, show that $\inf_w[L(w) - \varepsilon L_k(w)] = L(w^*) - \varepsilon\, L_k(w^*) - \frac{\varepsilon^2}{2}\, \nabla L_k^\top H^{-1} \nabla L_k + O(\varepsilon^3)$, by minimizing the Taylor expansion of $L - \varepsilon L_k$ around w^*. Combine everything to obtain

$$\mathcal{F}(r, \varepsilon) \;=\; \frac{\varepsilon^2}{2}\, \big(r - H^{-1}\nabla L_k\big)^\top H\, \big(r - H^{-1}\nabla L_k\big) \;+\; O(\varepsilon^3).$$

Hint: After substitution, the terms that do not depend on r should assemble into $\frac{\varepsilon^2}{2}\nabla L_k^\top H^{-1}\nabla L_k$ and cancel with the partition-function contribution, leaving a perfect square.

(c) (IFs are optimal.) The cost (15) is a squared Mahalanobis distance (in the Hessian metric) between the shift r and the influence function $r_{\mathrm{IF}} := H^{-1}\nabla L_k(w^*)$. Conclude that the IF minimizes $\mathcal{F}$ to $O(\varepsilon^2)$ among *all* smooth shift maps $w \mapsto w + \varepsilon r(w)$, and that the minimum cost is $O(\varepsilon^3)$.

State the result in words: *the influence function is the approximately KL-optimal way to shift the parameters of the unperturbed Boltzmann distribution to approximate the perturbed one.*

(d) (Degenerate case.) Now suppose w^* lies on a minimum manifold $\mathcal{S}_L$ and H is singular with null space N. Argue (without detailed proof) that:

- The cost (15) generalizes to $\mathcal{F}(r, \varepsilon) = \frac{\varepsilon^2}{2}(r - H^+\nabla L_k)^\top H\,(r - H^+\nabla L_k) + O(\varepsilon^3)$, where H^+ is the Moore–Penrose pseudoinverse.
- The minimizer is $r = H^+\nabla L_k + r_{\mathrm{NS}}$ for any $r_{\mathrm{NS}} \in N$: directions in the null space of H have zero cost, because the Hessian assigns zero curvature to movement along the minimum manifold.
- The IF (with pseudoinverse) is therefore optimal for the directions that matter, but the component of the shift along the flat directions is unconstrained.

Connect this to the SLT perspective: for singular models, the minimum is a variety, and the IF tells you how to move *off* the variety but is silent about movement *along* it.

(e) (Linear regression check.) Specialize to the Bayesian linear regression setting of Example 3.2 with prior $w \sim \mathcal{N}(0, \tau^2 I)$. The Boltzmann distribution at inverse temperature $\beta = 1$ is the Bayesian posterior $\mathcal{N}(\mu, V)$, which is already Gaussian, no need to take $\beta \to \infty$. A constant shift $w \mapsto w + \varepsilon r$ sends $\mathcal{N}(\mu, V)$ to $\mathcal{N}(\mu + \varepsilon r,\, V)$. The true perturbed posterior (downweighting z_k by ε) has mean $\mu + \varepsilon\, \mathrm{BIF}(z_k, w) + O(\varepsilon^2)$ and covariance $V + O(\varepsilon)$. Using the KL formula for Gaussians with the same covariance, show that

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu + \varepsilon r,\, V) \,\big\|\, \mathcal{N}(\mu + \varepsilon b,\, V)\big) \;=\; \frac{\varepsilon^2}{2}\,(r - b)^\top V^{-1}(r - b),$$

which is minimized at $r = b = \mathrm{BIF}(z_k, w) = (A + \lambda I)^{-1} x_k r_k$. Verify that this is consistent with (15): the "Hessian of the energy" (training loss plus prior) is $V^{-1} = \sigma^{-2}A + \tau^{-2}I$, and $V^{-1} \cdot \mathrm{BIF} = \sigma^{-2} x_k r_k = \nabla_w L(w, z_k)\big|_{w=\mu}$.

(f) (Comparison with Exercise 3.2.) The Laplace expansion exercise and this exercise both relate the IF to the BIF, but they answer different questions:

- Exercise 3.2 is an *algebraic identity*: BIF = IF + higher-order corrections. It requires the Laplace approximation (hence non-singular H, Gaussian posterior) and tells you what happens when you truncate the BIF.
- This exercise is an *optimality result*: the IF shift minimizes the KL divergence between the shifted and true perturbed distributions. The $\beta \to \infty$ result (parts a–d) needs only mild regularity of $\mathcal{L}$ and works even for singular H (via the pseudoinverse).

Summarize: the Laplace expansion tells you *what* the IF is (the leading Gaussian term of the BIF). The optimality perspective tells you *why* it works (it is the best first-order correction to the parameter distribution). The second viewpoint does not require the posterior to be Gaussian, which is why Mlodozeniec et al. 2025 argue that it provides a better explanation for the empirical success of IFs in deep learning.

SOLUTION, p. 605

Remark (Optimal transport). In the language of optimal transport, the map $w \mapsto w + \varepsilon r(w)$ is called a *transport map*, and the distribution of the shifted parameters is its *pushforward*, written $T_{\varepsilon\#}p_0^\beta$. The cost functional $\mathcal{F}$ is the asymptotic KL divergence between the pushforward and the target. Exercise 3.3 then says that the influence function is the *KL-optimal transport map* from the unperturbed to the perturbed Boltzmann distribution, at first order in ε. This is the perspective developed in Mlodozeniec et al. 2025, who prove a more general version (their Theorem 3) that applies to all smooth diffeomorphisms, not just maps of the form $w + \varepsilon r(w)$.

4. Unrolling

The influence functions of Section 2 and the Bayesian influence functions of Section 3 both have something in common: they analyze the *endpoint* of training, not the *dynamics* that led to it. Under the assumptions of the unique global minimum, this is not a problem: the training trajectory is irrelevant, since all roads lead to the same outcome. But in practice the training path will matter a lot. Ultimately, we are studying generalization and this is ultimately a question about the training dynamics. Similarly, the BIF looks at the posterior expectation shifts. To give a concrete failure mode: a training point that appears in the first mini-batch of the first epoch will have a very different effect compared to the same point appearing towards the end of training.

If we write down the training trajectory as a composition of β-dependent maps:

$$\begin{array}{ccccccccc} \mathbb{R}^p & \xrightarrow{h_0(\cdot;\beta)} & \mathbb{R}^p & \xrightarrow{h_1(\cdot;\beta)} & \cdots & \xrightarrow{h_{T-1}(\cdot;\beta)} & \mathbb{R}^p & \xrightarrow{\phi} & \mathbb{R} \\ w_0 & \longmapsto & w_1 & \longmapsto & \cdots & \longmapsto & w_T & \longmapsto & \phi(w_T) \end{array}$$

where each h_t is one optimizer step (a gradient step, an Adam update, etc.): $w_{t+1} = h_t(w_t;\beta)$, we can observe an analogy to a forward pass through a neural network:

$$\begin{array}{ccccccccc} \mathbb{R}^{d_0} & \xrightarrow{f_1(\cdot\,;w)} & \mathbb{R}^{d_1} & \xrightarrow{f_2(\cdot\,;w)} & \cdots & \xrightarrow{f_L(\cdot\,;w)} & \mathbb{R}^{d_L} & \xrightarrow{\ell} & \mathbb{R} \\ a_0 & \longmapsto & a_1 & \longmapsto & \cdots & \longmapsto & a_L & \longmapsto & \ell(a_L) \end{array}$$

where each f_ℓ is one layer: $a_\ell = f_\ell(a_{\ell-1}; w)$.

	**Forward pass **	Training trajectory
Inputs	Model weights w	Data weights β
Intermediate states	Activations $a_1, \ldots, a_L$	Parameters $w_0, \ldots, w_T$
Output	Loss ℓ(logits)	Observable $\phi(w_T)$
Differentiation method	Backpropagation	Unrolling

Data attribution via unrolling is to a training run what backprop is to a forward pass: differentiating a composition of maps to identify which inputs matter most.

This invites the following question: Can we backprop through the *training trajectory*? The gradients with respect to the β_i then indicate which data points would need to be up- or downweighted to change the final observable $\phi(w_T)$. And in principle, we can! We can differentiate through the training computation, step by step, treating the final parameters as an explicit function of every gradient update along the way. The result is a chain-rule expression that decomposes data attribution into per-step contributions and naturally incorporates the optimizer, learning rate schedule, mini-batch ordering, and training duration. We will derive the unrolling formula similar to the backprop formula for a forward pass, and then discuss two approaches to making it practical: one that materializes Jacobians and one that computes Jacobian-vector products implicitly. In the setup of Equation 1 we take $\pi(\beta) = w_T(\beta)$, the final checkpoint of the training run.

4.1 The training-dynamics approach to attribution

We fix notation for the training process. Given a dataset $\mathcal{D} = \{z_1, \ldots, z_n\}$ with sample weights $\beta = (\beta_1, \ldots, \beta_n)$, the optimizer runs for T steps. At step t, a mini-batch $B_t \subseteq \{1, \ldots, n\}$ of size b is drawn, and the parameters are updated by

$$w_{t+1}(\beta) \;=\; w_t(\beta) \;-\; \frac{\eta_t}{b} \sum_{i \in B_t} \beta_i \, \nabla_w L\big(w_t(\beta),\, z_i\big),$$

where η_t is the learning rate at step t. At $\beta = \mathbf{1}$ this is ordinary SGD on the unweighted loss. The initial parameters w_0 are independent of β. For simplicity we can assume that we are training for one epoch only, so that each example appears in exactly one mini-batch, but the formulas hold more generally.

The counterfactual. We want

$$\left.\frac{\partial\, \phi(w_T(\beta))}{\partial \beta_i}\right|_{\beta=\mathbf{1}},$$

the sensitivity of a final-model observable ϕ to an infinitesimal change in the weight of training example z_i.

4.2 The unrolling formula

Central to computing the data weight gradient is the Jacobian of one SGD step with respect to the current parameters. Define

$$J_t \;:=\; \left.\frac{\partial w_{t+1}}{\partial w_t}\right|_{\beta=\mathbf{1}} \;=\; I \;-\; \frac{\eta_t}{b}\sum_{i\in B_t}\nabla_w^2 L(w_t, z_i) \;=\; I - \eta_t\,\widehat{H}_t,$$

where $\widehat{H}_t := \frac{1}{b}\sum_{i\in B_t}\nabla_w^2 L(w_t, z_i)$ is the mini-batch Hessian at step t. The Jacobian of the map from step t to step t' is the ordered product

$$J_{t:t'} \;:=\; J_{t'-1}\,J_{t'-2}\cdots J_t \;=\; \prod_{s=t}^{t'-1} J_s \qquad (t < t').$$

Similarly, the direct effect of β_i at step t (when $i \in B_t$) is the per-example gradient:

$$\left.\frac{\partial w_{t+1}}{\partial \beta_i}\right|_{\substack{\beta=\mathbf{1}\\ w_t \text{ fixed}}} \;=\; -\,\frac{\eta_t}{b}\,\nabla_w L(w_t, z_i)\cdot \mathbf{1}[i \in B_t].$$

The full derivative is obtained by chaining direct effects through remaining steps:

$$\boxed{\left.\frac{\partial w_T(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \;=\; -\sum_{t=0}^{T-1}\frac{\eta_t}{b}\;\mathbf{1}[i \in B_t]\;J_{(t+1):T}\;\nabla_w L(w_t, z_i).}$$

Each term in the sum corresponds to one training step in which z_i appeared in the mini-batch: it is the gradient of z_i at the parameters of that step, propagated forward through the Jacobians of all subsequent steps.

The influence on an observable ϕ follows by the chain rule:

$$\left.\frac{\partial\,\phi(w_T(\beta))}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \;=\; -\,\nabla_w \phi(w_T)^\top \sum_{t=0}^{T-1}\frac{\eta_t}{b}\;\mathbf{1}[i \in B_t]\;J_{(t+1):T}\;\nabla_w L(w_t, z_i).$$

Remark (Which questions does unrolling answer?). Equation (20) looks like a sum over training steps and, for a fixed realization of the mini-batch sequence, it *is* a deterministic chain-rule computation. But the training run has randomness (initialization, mini-batch ordering, data augmentation), and so does the counterfactual: if we change β_i, the mini-batch sequence might be the same or it might differ. Unrolling as written differentiates through a *specific* training run: it answers the *single-model* attribution question, "how would this particular training trajectory have ended differently?" The *distributional* question, which looks at the expected change over random training, would require averaging (20). This distinction is the training-dynamics analogue of the single-posterior-mode vs. full-posterior distinction in Section 3.

Exercise 4.1 (Derivation of the unrolling formula). (a) Starting from the SGD update (16), apply the chain rule to write $\frac{\partial w_{t+1}}{\partial \beta_i}\big|_{\beta=\mathbf{1}}$ as a sum of two terms: one from the explicit dependence on β_i (the direct effect) and one from the dependence of w_t on β_i (the indirect effect). Show that the result is the recursion

$$\frac{\partial w_{t+1}}{\partial \beta_i} \;=\; J_t \,\frac{\partial w_t}{\partial \beta_i} \;-\; \frac{\eta_t}{b}\,\mathbf{1}[i \in B_t]\,\nabla_w L(w_t, z_i),$$

with initial condition $\frac{\partial w_0}{\partial \beta_i} = 0$. Compute the Jacobian J_t explicitly in terms of the mini-batch Hessian $\widehat{H}_t$ which leads to (17).

(b) Solve the recursion by "unrolling" it (i.e. by substituting repeatedly and using (18)) to obtain (20).

(c) (Preconditioned SGD.) Suppose instead that the optimizer uses a preconditioner $P_t \succ 0$:

$$w_{t+1}(\beta) \;=\; w_t(\beta) \;-\; \frac{\eta_t}{b}\,P_t \sum_{i \in B_t} \beta_i\, \nabla_w L(w_t(\beta), z_i).$$

This includes momentum-free Adam and natural gradient methods as special cases (with appropriate P_t). Show that the step Jacobian becomes $J_t^{(P)} = I - \eta_t\, P_t \widehat{H}_t$ and that the unrolling formula generalizes to

$$\left.\frac{\partial w_T(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \;=\; -\sum_{t=0}^{T-1} \frac{\eta_t}{b}\,\mathbf{1}[i \in B_t]\; J_{(t+1):T}^{(P)}\, P_t\, \nabla_w L(w_t, z_i),$$

where $J_{(t+1):T}^{(P)} = \prod_{s=t+1}^{T-1} (I - \eta_s P_s \widehat{H}_s)$. The only change is that each gradient is premultiplied by the preconditioner at the step where it appears. What does this say about the effect of using Adam vs. SGD on the attribution of a data point that appears early in training?

SOLUTION, p. 607

Remark (TracIn). Pruthi et al. 2020 observed that the Jacobian products $J_{(t+1):T}$ in (20) are both the most expensive and the most unstable part of the computation. Their method, TracIn, simply drops them, setting $J_{(t+1):T} \approx I$:

$$\mathrm{TracIn}(z_i, \phi) \;:=\; \sum_{t \in \mathcal{C}} \eta_t\; \nabla_w \phi(w_t)^\top \nabla_w L(w_t, z_i),$$

where $\mathcal{C}$ is a set of checkpoints (typically one per epoch).

There are two ways to make (20) practical. The first is to compute the Jacobian products $J_{(t+1):T}$ explicitly, or rather an approximation thereof. The second is to avoid materializing Jacobians altogether and instead compute Jacobian-vector products implicitly by reverse-mode automatic differentiation through the training loop. We will discuss both approaches in turn.

4.2.1 Materializing the Jacobian

In this subsection we will derive the SOURCE method (Bae et al. 2024), which introduces a small number of approximations of Jacobians that reduce the full unrolling computation to something resembling an influence-function calculation at each of several checkpoints.

Stationary segments. Divide the T training steps into L segments at checkpoints $0 = T_0 < T_1 < \cdots < T_L = T$, and let $K_\ell = T_\ell - T_{\ell-1}$ be the number of steps in segment ℓ. Within each segment, make the *stationarity approximation*: the expected mini-batch Hessian and per-example gradient are approximately constant,

$$\bar{H}_\ell \;\approx\; \frac{1}{K_\ell} \sum_{t=T_{\ell-1}}^{T_\ell - 1} \widehat{H}_t, \qquad \bar{g}_{i,\ell} \;\approx\; \frac{1}{K_\ell} \sum_{t=T_{\ell-1}}^{T_\ell - 1} \nabla_w L(w_t, z_i).$$

Similarly, write $\bar{\eta}_\ell$ for the average learning rate in segment ℓ.

Segment Jacobian. Under the stationarity approximation, the product of K_ℓ step Jacobians within segment ℓ simplifies to a matrix power:

$$J_{T_{\ell-1}:T_\ell} \;\approx\; (I - \bar{\eta}_\ell \, \bar{H}_\ell)^{K_\ell} \;\approx\; \exp\!\big(-\bar{\eta}_\ell \, K_\ell \, \bar{H}_\ell\big) \;=:\; \bar{S}_\ell.$$

The matrix exponential is well approximated by the matrix power when $\bar{\eta}_\ell \, \bar{H}_\ell$ has small spectral norm. In an eigenbasis of $\bar{H}_\ell$ with eigenvalue σ, the segment Jacobian acts as the scalar filter $F_S(\sigma) = e^{-\bar{\eta}_\ell K_\ell \sigma}$: high-curvature directions (σ large) are exponentially damped, while low-curvature directions (σ small) are preserved.

Segment response. Return to the unrolling formula (20) and restrict the sum to the steps inside segment ℓ. The contribution of datum z_i during this segment is

$$R_{i,\ell} \;:=\; \sum_{t=T_{\ell-1}}^{T_\ell - 1} \frac{\eta_t}{b} \, \mathbf{1}[i \in B_t] \; J_{(t+1):T_\ell} \, \nabla_w L(w_t, z_i),$$

where the Jacobian $J_{(t+1):T_\ell}$ propagates each gradient contribution forward to the segment boundary T_ℓ (propagation from T_ℓ to the end of training is handled by the inter-segment Jacobians $\bar{S}_{\ell'}$ in the full formula below). Now apply the stationarity approximation: replace each $\nabla_w L(w_t, z_i)$ by $\bar{g}_{i,\ell}$, each learning rate by $\bar{\eta}_\ell$, replace the indicator $\mathbf{1}[i \in B_t]$ by its expectation b/n (datum z_i appears in a random batch with probability b/n), and use the segment Jacobian approximation $J_{(t+1):T_\ell} \approx (I - \bar{\eta}_\ell \bar{H}_\ell)^{T_\ell - 1 - t}$. This gives

$$R_{i,\ell} \;\approx\; \frac{\bar{\eta}_\ell}{n} \sum_{k=0}^{K_\ell - 1} (I - \bar{\eta}_\ell \, \bar{H}_\ell)^k \; \bar{g}_{i,\ell}.$$

The sum is a matrix geometric series. Using $\sum_{k=0}^{K-1} M^k = (I - M^K)(I - M)^{-1}$ with $M = I - \bar{\eta}_\ell \bar{H}_\ell$ and passing to the matrix exponential:

$$\bar{r}_{i,\ell} \;:=\; \frac{1}{n} \big(I - e^{-\bar{\eta}_\ell K_\ell \bar{H}_\ell}\big) \bar{H}_\ell^{-1} \, \bar{g}_{i,\ell}.$$

To understand this, work in an eigenbasis of $\bar{H}_\ell$. On an eigenvalue σ, the matrix $(I - e^{-\bar{\eta}_\ell K_\ell \bar{H}_\ell})\bar{H}_\ell^{-1}$ acts as the scalar

$$F_r(\sigma) \;=\; \frac{1 - e^{-\bar{\eta}_\ell K_\ell \sigma}}{\sigma}.$$

Note the similarity to the classical influence function formula, which applies $1/\sigma$ (the inverse Hessian eigenvalue), and with the damped IF (6), which applies $1/(\sigma + \lambda)$. Thus F_r can be seen as a principled damping constant:

- *Large* σ (directions the optimizer has fully converged along): $e^{-\bar{\eta}_\ell K_\ell \sigma} \approx 0$, so $F_r(\sigma) \approx 1/\sigma$. Agrees with the classical IF.
- *Small* σ (directions the optimizer has barely moved along): $e^{-\bar{\eta}_\ell K_\ell \sigma} \approx 1 - \bar{\eta}_\ell K_\ell \sigma$, so $F_r(\sigma) \approx \bar{\eta}_\ell K_\ell$. A finite constant, not $1/\sigma \to \infty$.
- *Crossover* at $\sigma_* \approx 1/(\bar{\eta}_\ell K_\ell)$: directions with curvature below this threshold are automatically suppressed.

Full formula. Chaining the segment Jacobians and responses across all L segments:

$$\tau_{\text{SOURCE}}(\phi, z_i) \;=\; \nabla_w \phi(w_T)^\top \sum_{\ell=1}^{L} \Big(\prod_{\ell'=\ell+1}^{L} \bar{S}_{\ell'} \Big)\, \bar{r}_{i,\ell},$$

where the product is ordered with $\ell' = L$ on the left. With a single segment ($L = 1$), the formula reduces to $\nabla_w \phi^\top \bar{r}_{i,1}$, which is the classical IF with H^{-1} replaced by the filter F_r.

Remark (Unrolling provides a principled damping). The crossover $\sigma_* = 1/(\bar{\eta}K)$ in the filter F_r plays exactly the role of the damping parameter λ in the PBRF formula $(H + \lambda I)^{-1}$ from Section 2.2, but it is not a free parameter. It is the reciprocal of $\bar{\eta}K$, the total *training effort* (learning rate $\times$ number of steps) in the segment. Directions that the optimizer has not had time to converge along are automatically down-weighted, because the Jacobian products have not had enough steps to amplify them. The practical consequence: the damping λ that influence-function methods require careful tuning of has a natural value determined by the training process, and unrolling recovers it without any tuning.

4.2.2 Implicit JVPs and the REPLAY algorithm

The second approach to evaluating (20) avoids materializing Jacobians altogether. Instead, it uses *reverse-mode automatic differentiation* through the training loop, computing Jacobian-vector products (JVPs) implicitly. This is the strategy of the MAGIC method (Ilyas & Engstrom 2025), building on the metagradient framework of Engstrom et al. 2025.

Setup. Model the entire training process as a composition of T differentiable update functions:

$$s_{t+1} = h_t\big(s_t,\, g_t(s_t, \beta)\big), \qquad s_0 = s_{\text{init}},$$

where s_t is the full optimizer state at step t (parameters, momentum buffers, etc.), $g_t(s_t, \beta) = \sum_{i \in B_t} \beta_i \nabla L(s_t, z_i)$ is the weighted mini-batch gradient, and h_t is the optimizer's update rule (SGD, Adam, etc.). The attribution target is $\phi(s_T)$, which depends on β through the entire chain of updates.

The metagradient decomposition. Applying the chain rule through this composition:

$$\frac{\partial\, \phi(s_T)}{\partial \beta} \;=\; \sum_{t=0}^{T-1} \underbrace{\frac{\partial\, \phi(s_T)}{\partial s_{t+1}}}_{A_{t+1}} \cdot \underbrace{\frac{\partial\, h_t(s_t, g_t(s_t, \beta))}{\partial \beta}}_{B_t}.$$

The vector $A_{t+1} := \partial \phi(s_T)/\partial s_{t+1}$ is the sensitivity of the final observable to the state at step $t+1$; it satisfies the backward recursion

$$A_t \;=\; A_{t+1}\, \frac{\partial\, h_t(s_t, g_t)}{\partial s_t}, \qquad A_T = \nabla_{s_T} \phi(s_T).$$

This is just backpropagation through the training loop. The vector B_t is the direct effect of β at step t: it captures how the gradient at step t depends on the data weights. For vanilla SGD, B_t reduces to $-(\eta_t/b)\,[\nabla_w L(w_t, z_i) \cdot \mathbf{1}[i \in B_t]]_{i=1}^n$.

REPLAY: efficient reverse-mode through training. The backward recursion (33) requires the optimizer state s_t at every step. Naively this means storing all T states, which is prohibitive for long training runs. The REPLAY algorithm (Engstrom et al. 2025) solves this with a *hierarchical checkpointing* scheme:

1. Save k evenly-spaced checkpoints along the training trajectory.
2. When the backward pass needs a state between two checkpoints, *replay* the training forward from the nearest saved checkpoint to regenerate it.
3. Apply this strategy recursively (a k-ary tree of depth $\log_k T$).

The result is exact (to floating-point precision) differentiation through all T training steps, using $O(k \log_k T)$ stored states and $O(T \log_k T)$ total forward-pass work, a logarithmic overhead over the cost of training itself.

Comparison with explicit unrolling. The implicit approach has two main advantages:

- *Exactness.* No stationarity or matrix-exponential approximations are needed; the Jacobian-vector products are computed exactly by autodiff.
- *Generality.* Any differentiable optimizer (Adam, LAMB, etc.) and any training pipeline (multi-stage, curriculum learning) are handled transparently, since h_t is just a function.

The price is computational: REPLAY requires $O(T \log T)$ training-equivalent steps and careful engineering of the checkpointing schedule, whereas the explicit approach requires only a handful of checkpoints and an EK-FAC-style Hessian approximation at each. The choice depends on the scale: for moderate-size models where replaying train-

ing is feasible, MAGIC gives exact answers; for large-scale models where even one extra training pass is expensive, the SOURCE approximation with $L = 3$–6 segments may be the only practical option.

4.3 Long Exercise: From unrolling to influence functions

The unrolling formula (20) and the influence function (6) appear to be very different objects: one is a sum over training steps of Jacobian-propagated gradients, the other is a single inverse-Hessian-times-gradient computation at the endpoint. The following exercise shows that, in the right limit, unrolling converges to the classical influence function. The key observation, due to Mlodozeniec et al. 2025, is that the joint process (w_t, r_t), where $r_t := \partial w_t / \partial \varepsilon\big|_{\varepsilon=0}$ is the unrolling response, forms a *Markov chain*, and its limiting behavior can be analyzed with standard stochastic-approximation tools. This yields a convergence result that requires only that SGD reaches a local minimum, *not* that the loss is convex.

Exercise 4.2 (Convergence of unrolling to influence functions). Consider the SGD update with an interpolated loss that downweights example z_k by ε:

$$w_{t+1}(\varepsilon) \;=\; w_t(\varepsilon) \;-\; \frac{\eta_t}{b} \sum_{i \in B_t} \big(1 - \varepsilon\, \mathbf{1}[i = k]\big)\, \nabla_w L(w_t(\varepsilon), z_i),$$

so $\varepsilon = 0$ is the original training run and $\varepsilon = 1/n$ corresponds to removing z_k. Define the *response* $r_t := \partial w_t(\varepsilon)/\partial \varepsilon\big|_{\varepsilon=0}$.

$$\begin{pmatrix} w_{t+1} \\ r_{t+1} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} I & 0 \\ 0 & J_t \end{pmatrix}}_{\text{propagation}} \begin{pmatrix} w_t \\ r_t \end{pmatrix} \;+\; \underbrace{\begin{pmatrix} -\frac{\eta_t}{b} \sum_{i \in B_t} \nabla_w L(w_t, z_i) \\ \frac{\eta_t}{b}\, \mathbf{1}[k \in B_t]\, \nabla_w L(w_t, z_k) \end{pmatrix}}_{\text{driving terms}}$$

(a) (The response recursion.) By differentiating (34) with respect to ε at $\varepsilon = 0$, derive (35), where $J_t = I - \frac{\eta_t}{b} \sum_{i \in B_t} \nabla_w^2 L(w_t, z_i)$ is the step Jacobian from (17). The top row is ordinary SGD; verify that the bottom row is the same recursion as Exercise 4.1(a), but now with stochastic batches. Since the driving terms depend only on (w_t, r_t) and the i.i.d. batch selection B_t, the joint process is a Markov chain. Why is this observation useful?

(b) (Deterministic warm-up: full-batch GD.) As a warm-up, consider full-batch gradient descent with constant learning rate η (i.e. $B_t = \{1, \dots, n\}$ for all t). Assume training has converged: $w_t \approx w^*$ and $\nabla_w^2 L(w_t) \approx H$ for all t in a window of length K. Show that the response over this window satisfies

$$r_T \;\approx\; -\big[I - (I - \eta H)^K\big]\, H^{-1}\, \nabla_w L(w^*, z_k).$$

Hint: Sum the geometric series $\sum_{t=0}^{K-1} (I - \eta H)^t$ using the identity $\sum_{t=0}^{K-1} M^t = (I - M^K)(I - M)^{-1}$.

Take $K \to \infty$ (assuming $\eta < 2/\lambda_{\max}(H)$) and recover the influence function $r_\infty = -H^{-1} \nabla_w L(w^*, z_k) = r_{\mathrm{IF}}$.

(c) (Stochastic case: convergence to IF.) Now return to SGD with i.i.d. batches and a decaying learning rate satisfying $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$ (the Robbins–Monro conditions). Assume SGD converges to a local minimum w^* with $H = \nabla_w^2 L(w^*)$ positive semidefinite. The continuous-time ODE that the response tracks is

$$\dot{r}(t) \;=\; -H\, r(t) \;+\; \nabla_w L(w^*, z_k).$$

(You do not need to prove that SGD tracks this ODE, this follows from standard stochastic approximation theory.)

i. Show that the equilibrium of (36) in the column space of H is $r_{\mathrm{IF}} = H^{+}\nabla_w L(w^*, z_k)$, where H^{+} is the pseudoinverse. This is the influence function, with H^{+} in place of H^{-1} because the Hessian may be singular at a local minimum of an overparameterized model.

ii. Show that the component of $r(t)$ in the *null space* of H grows linearly: if P_0 is the projector onto $\ker(H)$, then $P_0\, r(t) = P_0\, r(0) + t\, P_0 \nabla_w L(w^*, z_k)$. Why does this component not converge? Under what condition on $\nabla_w L(w^*, z_k)$ does this runaway term vanish?

The full result (Mlodozeniec et al. 2025, Theorem 2) is: on the set of SGD trajectories that converge to a local minimum, $r_t \to r_{\mathrm{IF}} + r_{\mathrm{NS}}$ almost surely, where $r_{\mathrm{NS}} \in \ker(H)$. The influence function is the limiting response, up to a component in the flat directions of the loss.

SOLUTION, p. 608

4.4 Exercise: Influence depends on training time

The previous sections analyzed unrolling's asymptotics in the converged limit. But the whole point of unrolling is that the training *path* matters. The following exercise makes this concrete in a setting where the influence function admits a closed form at every point during training: a two-layer deep linear network (DLN), the same class of models from the Singular Learning Theory chapter. The setup and analytic formula follow Lee et al. 2025.

Exercise 4.3 (Stagewise influence in a deep linear network). A two-layer linear network $f_W(x) = W_2 W_1 x$ with $W_1, W_2 \in \mathbb{R}^{d\times d}$ is trained on $\{(x_i, y_i)\}_{i=1}^n$ with squared loss. Assume whitened inputs ($\frac{1}{n}\sum_i x_i x_i^\top = I$) and, for simplicity, that the input–output cross-covariance $\Sigma_{xy} = \frac{1}{n}\sum_i x_i y_i^\top$ is already diagonal with entries $s_1 > s_2 > \cdots > s_d > 0$.[5]

Dynamics. Under gradient flow with small balanced initialization, the network's modes evolve independently. The effective weight matrix at time t is $W(t) = \mathrm{diag}(\mathcal{G}_1(t), \ldots, \mathcal{G}_d(t))$, where each mode strength follows

$$\mathcal{G}_k(t) \;=\; \frac{s_k\, e^{2s_k t/\tau}}{e^{2s_k t/\tau} - 1 + s_k/\mathcal{G}_k(0)},$$

with $\mathcal{G}_k(0) \ll s_k$ and time constant τ. Mode k transitions from ≈ 0 to $\approx s_k$ around time $t_k^* \approx \frac{\tau}{2s_k} \log(s_k/\mathcal{G}_k(0))$. Larger singular values saturate first: the network learns dominant structure before fine structure.

Analytic influence. Now perturb the weight of training example $z_p = (x_p, y_p)$ by ε, so the perturbed cross-covariance is $\Sigma_{xy}(\varepsilon) = \Sigma_{xy} + \varepsilon\, y_p x_p^\top$. Since Σ_{xy} is diagonal, the perturbation shifts its eigenvalues and rotates its eigenvectors. The influence of z_p on the weight matrix at time t decomposes as

$$\mathcal{I}_p(t) \;:=\; \left.\frac{\partial W(t,\varepsilon)}{\partial \varepsilon}\right|_{\varepsilon=0} \;=\; A\,\mathcal{G}(t) \;+\; \mathcal{G}'_\varepsilon(t) \;+\; \mathcal{G}(t)\,B,$$

with three terms:

- A and B are skew-symmetric matrices describing how the perturbation *rotates* the left/right singular bases. Their off-diagonal entries are $A_{jk} = B_{kj} = (y_p)_j (x_p)_k / (s_k - s_j)$ for $j \neq k$.
- $\mathcal{G}'_\varepsilon(t) = \operatorname{diag}\big(g'_t(s_1)\,(y_p)_1 (x_p)_1,\; \ldots,\; g'_t(s_d)\,(y_p)_d (x_p)_d\big)$ encodes the change in mode strengths, where

$$g'_t(s_k) \;:=\; \frac{\partial\, \mathcal{G}_k(t)}{\partial s_k}$$

is the sensitivity of the k-th mode strength to a change in the corresponding singular value at time t.

The influence on the loss of a test example $z_q = (x_q, y_q)$, with residual $r_q(t) = y_q - W(t)x_q$, is

$$\mathcal{I}(z_p,\, \ell_q)(t) \;=\; -\,r_q(t)^\top \big(A\,\mathcal{G}(t) + \mathcal{G}'_\varepsilon(t) + \mathcal{G}(t)\,B\big)\, x_q.$$

(a) (Three sources of influence.) Interpret the three terms in (38):

- $A\,\mathcal{G}(t)$ and $\mathcal{G}(t)\,B$: the perturbation *rotates* the singular bases, mixing already-learned modes into each other. Why are these terms proportional to the current mode strengths $\mathcal{G}(t)$ rather than to their derivatives?
- $\mathcal{G}'_\varepsilon(t)$: the perturbation *shifts* the singular values, changing how fast each mode is learned. Why does this term involve $g'_t(s_k)$, sensitivity of the dynamics to the singular value, rather than $\mathcal{G}_k(t)$ itself?

(b) (When does influence peak?) Using the dynamics (37), argue that $g'_t(s_k)$ is peaked around $t \approx t_k^*$ (the transition time of mode k). Conclude that the $\mathcal{G}'_\varepsilon$ term, the part of influence that acts through the learning speed of each mode, is concentrated in time around the moment the mode is being learned. Before and after, this contribution is negligible.

Hint: Consider the limits $t \ll t_k^*$ (mode not yet learning, $\mathcal{G}_k \approx \mathcal{G}_k(0)$) and $t \gg t_k^*$ (mode saturated, $\mathcal{G}_k \approx s_k$). In both cases, how sensitive is $\mathcal{G}_k$ to a small change in s_k?

(c) (Sign flips.) Consider $d = 2$ with $s_1 \gg s_2$: mode 1 captures a coarse distinction ("animal vs. plant") and mode 2 a fine distinction ("dog vs. cat" within animals). A dog example z_{dog} and a cat example z_{cat} share the same mode-1 coordinate but have opposite mode-2 coordinates: $(x_{\text{dog}})_2 = +(x_{\text{cat}})_2$. Using (40), argue that the influence of z_{dog} on a cat test loss can change sign during training: positive while mode 1 is being learned (shared structure), negative after mode 2 is learned (competing structure). At what time is the sign flip sharpest?

SOLUTION, p. 608

Remark (General singular basis). When $\Sigma_{xy} = USV^\top$ is not diagonal, the formulas above hold with all vectors expressed in the singular basis: replace x_p by $V^\top x_p$, y_p by $U^\top y_p$, r_q by $U^\top r_q$, and $W(t)$ by $U^\top W(t)V$. The influence on W itself becomes $\mathcal{I}_p(t) = U(A\mathcal{G}(t) + \mathcal{G}'_\varepsilon(t) + \mathcal{G}(t)B)V^\top$, with $A_{jk} = (U^\top y_p)_j (V^\top x_p)_k / (s_k - s_j)$. See Lee et al. 2025 for the full derivation, including the degenerate case $s_j = s_k$.

Remark (The developmental view of attribution). Lee et al. 2025 show that the same phenomenology (influence sign flips, non-monotonic trajectories) appears in language models, where e.g. the influence of delimiter tokens spikes when the model learns pairing structure and then decays. The practical consequence is that data attribution is not a one-shot computation but a time-varying signal, and methods like unrolling that track the training trajectory are better suited to capture this than methods that look only at the final checkpoint.

References

Juhan Bae, Nathan Ng, Alston Lo, Marzyeh Ghassemi, and Roger Grosse (2022). *If Influence Functions are the Answer, Then What is the Question?*. arXiv:2209.05364.

Juhan Bae, Wu Lin, Jonathan Lorraine, and Roger Grosse (2024). *Training Data Attribution via Approximate Unrolled Differentiation*. arXiv:2405.12186.

Garrett Baker, George Wang, Jesse Hoogland, and Daniel Murfet (2026). *Structural Inference: Interpreting Small Language Models with Susceptibilities*. arXiv:2504.18274.

R. Dennis Cook (1977). *Detection of Influential Observation in Linear Regression*. Technometrics.

Logan Engstrom, Andrew Ilyas, Benjamin Chen, Axel Feldmann, William Moses, and Aleksander Madry (2025). *Optimizing ML Training with Metagradient Descent*. arXiv:2503.13751.

Roger Grosse, Juhan Bae, Cem Anil, Nelson Elhage, Alex Tamkin, Amirhossein Tajdini, Benoit Steiner, Dustin Li, Esin Durmus, Ethan Perez, Evan Hubinger, Kamilė Lukošiūtė, Karina Nguyen, Nicholas Joseph, Sam McCandlish, Jared Kaplan, and Samuel R. Bowman (2023). *Studying Large Language Model Generalization with Influence Functions*. arXiv:2308.03296.

Frank R. Hampel (1974). *The Influence Curve and Its Role in Robust Estimation*. Journal of the American Statistical Association.

Christopher Hitchcock (2001). *The Intransitivity of Causation Revealed in Equations and Graphs*. The Journal of Philosophy.

Andrew Ilyas and Logan Engstrom (2025). *MAGIC: Near-Optimal Data Attribution for Deep Learning*. arXiv:2504.16430.

Pang Wei Koh and Percy Liang (2020). *Understanding Black-box Predictions via Influence Functions*. arXiv:1703.04730.

Philipp Alexander Kreer, Wilson Wu, Maxwell Adam, Zach Furman, and Jesse Hoogland (2026). *Bayesian Influence Functions for Hessian-Free Data Attribution*. arXiv:2509.26544.

Jin Hwa Lee, Matthew Smith, Maxwell Adam, and Jesse Hoogland (2025). *Influence Dynamics and Stagewise Data Attribution*. arXiv:2510.12071.

David Lewis (1973). *Causation*. Journal of Philosophy.

Bruno Mlodozeniec, Isaac Reid, Sam Power, David Krueger, Murat Erdogdu, Richard E. Turner, and Roger Grosse (2025). *Distributional Training Data Attribution: What do Influence Functions Sample?*. arXiv:2506.12965.

Aaron Mueller (2024). *Missed Causes and Ambiguous Effects: Counterfactuals Pose Challenges for Interpreting Neural Networks*. arXiv:2407.04690.

Garima Pruthi, Frederick Liu, Mukund Sundararajan, and Satyen Kale (2020). *Estimating Training Data Influence by Tracing Gradient Descent*. arXiv:2002.08484.

Lloyd S Shapley (1953). *A Value for n-Person Games*. Contributions to the Theory of Games II.

NOTES

1. In Section 4 we make this analogy explicit.
2. For an invertible matrix M and vectors u, v: $(M + uv^\top)^{-1} = M^{-1} - \frac{M^{-1}uv^\top M^{-1}}{1+v^\top M^{-1}u}$.
3. Concretely you can take the eigendecomposition of H and only invert the non-zero eigenvalues.
4. For small ε the map $w \mapsto w + \varepsilon r(w)$ is a diffeomorphism, so the inverse is well-defined. The exact formula requires evaluating r at the pre-image; the expression above is correct to the order we need.
5. The general case replaces the standard basis with the left/right singular vectors U, V of Σ_{xy} throughout; see Remark 4.4.

CHAPTER B.6

Physics of Deep Learning

Aleksander Czejdo, Tudor Dimofte, Brianna Grado-White, Charles Renshaw-Whitman

An introduction to physics-inspired analysis and inquiry in deep learning, particularly focusing on techniques from statistical mechanics, statistical field theory, and (a bit of) dynamical systems.

Source: iliad-intensive.org/learning/qft · CC BY 4.0 · Slides: online

In this chapter

-1. Prerequisites 143
0. Main references 144
1. Intro reading: learning mechanics 144
2. Statistical mechanics of learning 145
  2.1 The Bayesian model of learning 146
  2.2 Why the exponential of a loss is a likelihood 148
  2.3 Partition functions, free energy, and cumulants 149
  2.4 A physics example: the Ising model 151
  2.5 Phases and phase transitions 154
  2.6 The Ising perceptron: a first-order transition in learning 158
  2.7 Comparison with QFT 160
3. Large width at initialization: the Gaussian process limit 161
  3.1 Notation 162
  3.2 Normalizing the network 162
  3.3 Gaussian processes 163
  3.4 The NNGP correspondence 166
  3.5 NNGP via functional integrals 169
  3.6 L/N 170
  3.7 Edge of chaos 172
4. The NTK and large width during training 173
  4.1 Gradient flow and the NTK 174
    4.1.1 Geometry of gradient flows 174
  4.2 Features 176
  4.3 Constant NTK at large width 177
  4.4 Lazy learning 181
  4.5 Summary 181
  4.6 Bayesian learning at large width: kernel learning 183

5. Mean-field scaling 185
5.1 Mean-field theories in physics 185
5.1.1 Warm-up: the Curie–Weiss magnet 186
5.1.2 The general shape of a mean-field model 187
5.2 Mean-field scaling for one-hidden-layer networks 187
5.2.1 Initialization 188
5.2.2 The loss is a mean-field energy 188
5.2.3 Equations of motion and the mean field 189
5.2.4 From particles to measures: the continuity equation 190
5.2.5 The McKean–Vlasov equation and self-consistency 191
5.3 The NTK is no longer constant: feature learning 193
5.4 The scaling dial and Maximal Update Parameterization 194
5.5 What is this framework good for? 195
5.6 Grokking as a lazy-to-rich transition: a reading exercise 196
5.7 Bayesian grokking: a first-order transition (alternative reading) 501
6. Non-equilibrium physics, MSRJD, and DMFT 198
6.1 From stochastic dynamics to path integrals: the MSRJD formalism 198
6.2 Double descent as a dynamical phase transition 201
6.2.1 Model definition and formalism 202
6.2.2 Overview of results via training and generalization error 204
6.3 Dynamical mean-field theory (DMFT) 206
6.3.1 Self-consistency 206
6.3.2 Recent applications in deep learning 207
7. Scaling laws 208
7.1 The empirical laws 209
7.2 Joint laws, bottlenecks, and the Chinchilla revision 211
7.3 Where do the laws come from? 212
7.4 A survey of theoretical proposals 214
7.5 The quantization model: exponents from the statistics of skills 216
7.6 Emergence: smooth laws from staggered cliffs 217
7.7 Exponents from geometry and from spectra 218
7.7.1 Geometry: the data-manifold argument 218
7.7.2 Spectra: scaling laws from kernel eigenvalues 219
7.8 A solvable large-N field theory 219
7.9 All the knobs at once: a dynamical theory 220
7.10 Beyond pretraining: post-training and test-time scaling 221
A. The replica method 502
A.1 What the annealed approximation gets wrong 503
A.2 Replicas 503
A.3 A variant: the spherical perceptron 506
B. From linear regression to kernel regression 508
B.1 Linear regression 508
B.2 Linear regression with nonlinear features 510
B.3 Kernel regression 513
B.4 Kernel ridge regression 515
B.5 Linearized networks are kernel ridge regressors 518

-1. Prerequisites

We don't assume any physics background, though reading some of the suggested reference material may be helpful. We do assume:

- a basic understanding of probability theory and Bayesian inference;
- familiarity with simple neural networks (MLP's);
- undergraduate-level calculus and linear algebra;
- and, in a few places, differential geometry.

WHAT YOU'LL LEARN

By the end of these notes, you should be able to:

- Use the statistical-mechanics dictionary for Bayesian learning — energy, entropy, partition function, free energy, cumulants, order parameters, quenched averages — and explain why the exponential of a loss function is a likelihood.
- Solve the Ising model in the mean-field approximation; explain what a phase transition is, why finite systems show smooth sigmoids and growing susceptibility peaks instead of true kinks, and how the Ising perceptron realizes a first-order transition to perfect generalization, metastable lag included.
- Derive the Gaussian-process description of a wide network at initialization, and explain the roles of the fan-in normalization, the depth-to-width ratio, and the edge of chaos.
- Derive the NTK, and explain why it freezes at large width — so that training reduces to kernel regression, with no feature learning.
- Derive, heuristically, the mean-field limit of one-hidden-layer networks — neurons as interacting particles, self-consistency, the McKean–Vlasov equation — and explain how its *moving* kernel encodes feature learning; place the NTK and mean-field limits on the scaling dial, and connect it to μP and hyperparameter transfer.
- Compare the two theoretical accounts of grokking — an equilibrium first-order transition vs. delayed lazy-to-rich dynamics — and propose an experiment that would distinguish them.
- Recognize the MSRJD/DMFT path-integral formulation of training dynamics, the role of correlation and response functions, and double descent as a dynamical phase transition.
- Derive the three scaling laws of the quantization model, explain "emergence" as the aggregation of staggered cliffs, and survey where scaling exponents come from: skill statistics, data geometry, and kernel spectra.

0. Main references

These notes draw principally on the following references, all of which appear elsewhere in the chapter. For Part I: Tong's lectures on statistical field theory and on kinetic theory (Tong 2012; Tong 2012), the physics behind the lecture; Engel and Van den Broeck (Engel & Van den Broeck 2001) for the perceptron tradition; and Gyorgyi (Györgyi 1990) for the Ising perceptron transition. For Part II: Hanin's lectures (Hanin 2026) and Roberts, Yaida, and Hanin (Roberts et al. 2022) on large-width limits and the L/N expansion; Ringel et al. (Ringel et al. 2025) on statistical field theory in deep learning; Spiliopoulos, Sowers, and Sirignano (Spiliopoulos et al. 2025) for rigorous NTK and mean-field limits; Lin (Lin 2025) for experiments identifying features through the empirical NTK; and the two accounts of grokking (Kumar et al. 2024; Rubin et al. 2024), which are the natural follow-up reading (Sections 5.6 and 5.7). For Part III: Kaplan et al. (Kaplan et al. 2020) and Henighan et al. (Henighan et al. 2020) for the empirical laws; Hoffmann et al. (Hoffmann et al. 2022) and Porian et al. (Porian et al. 2024) for Chinchilla and the reconciliation with Kaplan; Michaud et al. (Michaud et al. 2023) for the quantization model; and Maloney, Roberts, and Sully (Maloney et al. 2022) and Bordelon, Atanasov, and Pehlevan (Bordelon et al. 2024) for the solvable field theories.

A complementary introduction to the physics of learning by Renshaw-Whitman can be found in (Renshaw-Whitman 2026); Appendix B is adapted from it.

1. Intro reading: learning mechanics

The review (Simon et al. 2026) argues that a scientific theory of deep learning — a "mechanics of learning" — is already emerging, and it organizes the evidence into five converging strands: analytically solvable settings, insightful limits, simple empirical laws, disentangled hyperparameters, and universal phenomena. The body of the paper develops the first two strands in detail and samples the remaining three. These notes will do much the same, so it is worth keeping the five strands in mind as a map of everything that follows.

The paper's starting observation deserves to be savored, because it sets the tone for the whole subject. A deep learning system is completely specified by four explicit ingredients: an architecture (a composition of simple linear and nonlinear maps), a dataset, an objective, and a learning rule (gradient-based updates from a random initialization). Nothing is hidden: every parameter, every gradient, every intermediate activation can be measured, at every step, to whatever precision we like. In the language of the reading, deep learning directly exposes its own equations of motion — *the central challenge is not opacity but complexity*. This places the field in very different territory from the genuinely opaque sciences: in neuroscience or cell biology the microscopic equations of motion are unknown and the state is largely unmeasurable, while in high-energy physics we must infer the microscopic laws indirectly, from scattering amp-

litudes. The better analogy is fluid dynamics: there, too, we possess the microscopic law — for deep learning, the update equation is five lines of code — and what is missing is the *effective* description, the analogue of Navier–Stokes.

Read **Section 2** (only) of (Simon et al. 2026), and think about the following:

1. *Rank the physics analogies in Table 1 of the reading* (solvable models ↔ hydrogen atom; limits ↔ thermodynamic limit; empirical laws ↔ Kepler and Planck; hyperparameters ↔ Reynolds number; universality ↔ critical phenomena). Which are the most compelling? Which do you think are most useful in understanding deep learning?
2. *Discretization and other approximations.* The reading's Discretization Hypothesis proposes that practical networks are best understood as noisy, finite approximations to models of infinite size — like a finite-difference discretization of a PDE — so that finite-size effects typically *cost* performance rather than enable it. Given what you've seen so far, does this seem accurate? The hypothesis is falsifiable: it would be refuted by a finite-size effect that delivers a general benefit unachievable in any limit. What are some possible experiments/setups that could falsify it?
3. *What actually matters for alignment?* The reading wants falsifiable, quantitative, *average-case* predictions rather than worst-case bounds — physics rather than classical learning theory. What is lost? E.g. for alignment, does one want worst-case guarantees or forecasting of typical capabilities? Which of the five lines of evidence is most relevant for alignment?

You may want to revisit these questions at later stages as you go through these notes, and learn more about physics-inspired approaches in deep learning.

2. Statistical mechanics of learning

There is a large and well-developed body of work applying physics to learning theory. Much of it consists of *statistical mechanics* applied to *Bayesian learning*. The classic textbook is Engel and Van den Broeck (Engel & Van den Broeck 2001); modern reviews include (Zdeborov'a & Krzakala 2016; Bahri et al. 2020); and an excellent reference for the physics itself is David Tong's lecture course on statistical field theory (Tong 2012), which we follow closely in Sections 2.4–2.5.

In this section we focus on the Bayesian model of learning. We first recall what it is, why it is a reasonable idealization of how networks are actually trained, and why the exponential of a loss function deserves to be called a likelihood (Sections 2.1–2.2). We then switch to physics. We introduce partition functions, free energies, and cumulants (Section 2.3), and work through the Ising model in the mean-field approximation, building up to first- and second-order phase transitions (Sections 2.4–2.5). The payoff is a learning problem that is the Ising magnet's twin, the Ising perceptron, which shows a first-order transition to perfect generalization; it is left as a self-contained walk-through exercise (Section 2.6). Section 2.7 is a translation guide for readers arriving from

quantum field theory. The ideas introduced here — order parameters, free energies, saddle points, quenched averages, phase transitions — will keep returning throughout the notes.

2.1 The Bayesian model of learning

Consider a network f_θ with parameters $\theta \in \mathbb{R}^P$, trained on n samples $(x^\mu, y^\mu)_{\mu=1}^n$ to minimize a loss

$$\mathcal{L}(\theta) = \sum_{\mu=1}^{n} \ell\big(f_\theta(x^\mu), y^\mu\big)\,,$$

a sum over samples of a per-sample loss ℓ. In practice, the network is initialized at random, $\theta_0 \sim \pi(\theta)$, and trained with some version of gradient descent,

$$\Delta\theta \approx -\eta\, \nabla_\theta \mathcal{L}(\theta)\,.$$

A common idealization of this process, and the one this section is about, is *Bayesian learning*. It says that after training, the parameters should be regarded as drawn from the posterior distribution

$$p_\beta(\theta) = \frac{1}{Z}\, e^{-\beta\, \mathcal{L}(\theta)}\, \pi(\theta)\,, \qquad Z = \int d\theta\, e^{-\beta\, \mathcal{L}(\theta)}\, \pi(\theta)\,.$$

Here the initialization distribution $\pi(\theta)$ plays the role of the Bayesian *prior*, and $e^{-\beta\mathcal{L}(\theta)}$ is interpreted as the *likelihood* of the training data given the parameters θ (Section 2.2 explains why this is reasonable). The positive constant β, the *inverse temperature*, controls how strongly the data constrain the parameters. We will see in a moment that it can be read as a rough measure of how much training has been done.

An interpretation of β. It is convenient to write the loss as n times an average over samples,

$$\mathcal{L}(\theta) = n\, L_n(\theta)\,, \qquad L_n(\theta) := \frac{1}{n}\sum_{\mu=1}^{n} \ell\big(f_\theta(x^\mu), y^\mu\big)\,.$$

The *empirical loss* L_n stays finite as $n \to \infty$, converging to the *population loss* $L(\theta) := \mathbb{E}_{(x,y)}\, \ell(f_\theta(x), y)$, the expectation over the true data distribution. Thus for large n the posterior is

$$p_\beta(\theta) \;\propto\; e^{-\,n\beta\, L(\theta)}\, \pi(\theta)\,,$$

and the sample size n always accompanies the inverse temperature, in the combination $n\beta$. Increasing the number of samples is the same as lowering the temperature. If, moreover, samples are drawn fresh at each training step, then n is also a rough measure of training time. Altogether: *more data = more training = larger $n\beta$ = colder*. We will often write $e^{-\beta L}$ for short, but it is worth remembering that the effective β grows with n.

Bayes vs. gradient descent. We do not train networks by sampling from (2.3); we train them by gradient descent. Why, then, is the Bayesian model a useful idealization? There are two connections.

The first is heuristic: the analogy just described, $n \to \infty$ (more samples, longer training) $\leftrightarrow$ $\beta \to \infty$ (lower temperature). As we will see, sweeping a parameter like β can take a system through phase transitions, and the analogy predicts that training can do the same. It is only an analogy. In particular, when a system is driven *dynamically* through a first-order transition, the transition is delayed by *hysteresis* (Section 2.5), so timing predictions from the Bayesian model should be treated with care. A genuinely dynamical formalism for training, the MSRJD path integral, appears in Section 6.

The second connection is exact, and is what makes the Bayesian model testable in practice. Replace the batch noise of SGD by constant, isotropic Gaussian noise; the result is called *Langevin dynamics*:

discrete optimizer	continuous (S)DE
SGD : $\Delta\theta = -\eta(\nabla_\theta \mathcal{L} + \text{batch noise})$	$\dot\theta = -\eta \nabla_\theta \mathcal{L} + \frac{\eta}{\sqrt{B}} \Sigma(\theta)^{\frac{1}{2}} d\xi$
GD : $\Delta\theta = -\eta\, \nabla_\theta \mathcal{L}$	$\dot\theta = -\eta \nabla_\theta \mathcal{L}$
LD : $\Delta\theta = -\eta\, \nabla_\theta \mathcal{L} + \sqrt{2\eta T}\, \xi$	$\dot\theta = -\eta \nabla_\theta \mathcal{L} + \sqrt{2\eta T}\, d\xi$

Here ξ is Gaussian noise with mean 0 and covariance I, η is the learning rate, and T is a parameter called the *temperature* (or diffusion constant). Each discrete algorithm on the left has a continuous-time limit on the right, obtained by shrinking the step size and scaling up the number of steps: gradient descent becomes *gradient flow*, and Langevin dynamics becomes the *Langevin equation*, a stochastic differential equation. The key fact is that if one starts from any reasonable distribution of parameters and evolves with the Langevin equation (for a sufficiently nice $\mathcal{L}$), the distribution converges at late times to the Gibbs distribution

$$p(\theta) = \frac{1}{Z}\, e^{-\beta \mathcal{L}(\theta)}, \qquad \beta = 1/T\,.$$

(See (Tong 2012, Chapter 3) for a physics-focused discussion of this convergence.) A prior can be included by training on the modified loss $\mathcal{L}(\theta) - \beta^{-1} \log \pi(\theta)$, which reproduces (2.3). Thus one can *engineer* the Bayesian posterior by Langevin training, and this is how theoretical predictions derived from (2.3) are usually tested. Note that the Langevin route gives a different Bayes-to-gradient connection from the first: here training time $t \to \infty$ at fixed T gives the posterior at $\beta = 1/T$, while $T \to 0$ at fixed t gives gradient flow.

Remark. SGD itself is more complicated than Langevin dynamics. The covariance $\Sigma(\theta)$ of its effective noise depends on the position in the loss landscape, and at late times SGD need not converge to any stationary distribution. Despite many qualitative and empirical similarities between Bayesian learning and SGD/Adam training, the precise relation between them remains an open problem. We will compare the two repeatedly: for wide networks in Section 4 (gradient flow) versus Section 4.6 (Bayesian);

for toy models of grokking in Section 5 (Rubin et al. 2024); and, for toy models of superposition (Elhage et al. 2022) through the lens of singular learning theory, in (Chen et al. 2023).

2.2 Why the exponential of a loss is a likelihood

The Bayesian model treats $e^{-\beta\mathcal{L}(\theta)}$ as the probability of the training data given the parameters. For the loss functions actually used in deep learning, this is not a metaphor: the losses were *designed* as negative log-likelihoods, so that minimizing the loss is maximum-likelihood estimation. Two examples cover most of practice.

Mean squared error. Suppose the labels are the network's output plus Gaussian noise, $y = f_\theta(x) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Then the probability density of a label given the input and the parameters is

$$p(y \mid x, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big[-\frac{1}{2\sigma^2}\big(y - f_\theta(x)\big)^2 \Big] ,$$

and for n independent samples,

$$\prod_{\mu=1}^{n} p(y^\mu \mid x^\mu, \theta) \;\propto\; e^{-\beta\,\mathcal{L}(\theta)} , \qquad \mathcal{L}(\theta) = \frac{1}{2}\sum_{\mu=1}^{n} \big(f_\theta(x^\mu) - y^\mu\big)^2 , \quad \beta = \frac{1}{\sigma^2} .$$

The MSE loss is the negative log-likelihood of a Gaussian noise model, and the inverse temperature is the inverse noise variance. Noisier labels mean a hotter posterior, which constrains the parameters less.

Cross-entropy. For classification, the network outputs a probability distribution $p_\theta(y \mid x)$ over K classes, typically the softmax of its K output values. The cross-entropy loss is $\ell = -\log p_\theta(y \mid x)$, so that

$$\prod_{\mu=1}^{n} p_\theta(y^\mu \mid x^\mu) \;=\; e^{-\mathcal{L}(\theta)}$$

exactly, with $\beta = 1$: the exponential of the cross-entropy loss *is* the likelihood, with no noise model to choose. The same is true of the next-token loss of a language model, which is exactly the log-probability the model assigns to the training corpus.

The general pattern. Whenever a loss is a negative log-likelihood, $e^{-\mathcal{L}}$ is the likelihood and (2.3) at $\beta = 1$ is the textbook Bayesian posterior. A value $\beta \neq 1$ gives a *tempered* posterior. It arises naturally when the noise level is not known (as in (2.9)), and it is also a useful theoretical knob: as the interpretation of $n\beta$ above shows, temperature and sample size are interchangeable in the Bayesian model. There is also a useful limiting case. Some losses are hard constraints, $\ell = 0$ if the prediction is right and $\ell = \infty$ if it is wrong. Then $e^{-\ell}$ is a step function, the posterior is uniform on the parameters that fit every training sample, and no β appears at all. This is the *zero-temperature* limit, and it is the setting of the Ising perceptron in Section 2.6.

2.3 Partition functions, free energy, and cumulants

The Bayesian posterior (2.3) looks exactly like a *thermal* system in equilibrium with a heat bath, the basic object of statistical mechanics. The dictionary is:

Bayesian learning	**statistical mechanics**
parameters $\theta \in \mathbb{R}^P$	microscopic state of a system
	(*e.g.* the spins of P electrons)
loss $\mathcal{L}(\theta)$	energy $E(\theta)$, or Euclidean action $S(\theta)$
posterior $p(\theta) \propto e^{-\beta\mathcal{L}(\theta)}$	Boltzmann distribution at temperature $T = 1/\beta$
number of samples n, or training time	inverse temperature β
#parameters P	#degrees of freedom N

To include the prior $\pi(\theta)$, simply redefine the energy as $E(\theta) = \mathcal{L}(\theta) - \beta^{-1}\log\pi(\theta)$, so that $e^{-\beta E} = e^{-\beta\mathcal{L}}\pi$. In what follows we write $E(\theta)$ for whatever sits in the exponent, and N for the number of degrees of freedom, as physicists do; for a network, $N = P$.

The thermodynamic limit. Statistical mechanics is about systems with very many degrees of freedom, and one studies their behavior as $N \to \infty$: the *thermodynamic limit*. In this limit one distinguishes *extensive* quantities, which grow in proportion to N (the total energy of a gas), from *intensive* ones, which stay finite (the temperature, or the energy per particle). Both notions transfer directly to learning: with P parameters and n samples, the loss (2.1) is extensive in n and the empirical loss L_n is intensive. Many of the approximations in these notes are thermodynamic limits of some kind, sending the number of parameters, the width, or the number of samples to infinity.

Partition function and free energy. The normalization constant in (2.3),

$$Z(\beta) = \int d\theta\, e^{-\beta E(\theta)}\,,$$

is called the *partition function*. In Bayesian terms it is the marginal likelihood of the data. It depends on β and on any other hyperparameters of the problem. Its logarithm defines the *free energy*[1] and the free energy *density*,

$$F(\beta) := -\log Z(\beta)\,, \qquad \varphi := \frac{F}{N}\,.$$

There are three reasons to care about F rather than Z. First, F is extensive and φ intensive: the free energy density has a limit as $N \to \infty$, while Z itself is exponentially large or small. Second, "F is minimized": at large N the integral (2.11) is dominated by the regions of parameter space with the smallest local free energy, so the system's macroscopic state is found by minimizing F. Section 2.4 makes this precise. Third, derivatives of F compute expectation values, as follows.

Sources and cumulants. Suppose we care about the expectation values of some *observables* $\mathcal{O}_i(\theta)$, functions of the parameters. Add *source terms* J^i to the exponent,

$$Z(\beta, J) = \int d\theta \; e^{-\beta E(\theta) + \sum_i J^i \mathcal{O}_i(\theta)} , \qquad F(\beta, J) = -\log Z(\beta, J) .$$

Differentiating under the integral,

$$\mathbb{E}(\mathcal{O}_i) = -\frac{\partial F}{\partial J^i}\bigg|_{J=0} , \qquad \mathrm{Cov}(\mathcal{O}_i, \mathcal{O}_j) = -\frac{\partial^2 F}{\partial J^i \partial J^j}\bigg|_{J=0} ,$$

and in general the p-th derivative is the p-th *cumulant*,

$$\kappa_p(\mathcal{O}_{i_1}, \dots, \mathcal{O}_{i_p}) = -\frac{\partial^p F}{\partial J^{i_1} \cdots \partial J^{i_p}}\bigg|_{J=0} :$$

the mean, the covariance, the third central moment, and from $p = 4$ on, combinations of moments that vanish for Gaussian variables. The sourced free energy is the generating function of cumulants. Cumulants will return in field-theory clothing in Section 3.3, where they measure how far a network at initialization is from a Gaussian process.

If the only observable we care about is the energy itself, no source is needed: β already couples to E, and

$$\mathbb{E}(E) = \frac{\partial F}{\partial \beta} , \qquad \mathrm{Var}(E) = -\frac{\partial^2 F}{\partial \beta^2} , \qquad \kappa_3(E) = \frac{\partial^3 F}{\partial \beta^3} , \; \dots$$

We will meet these derivatives again shortly: discontinuities in them are how phase transitions are defined.

Exercise 2.1 (★) (Partition functions, sources, and cumulants). Take a Boltzmann distribution $p_\beta(\theta) \propto e^{-\beta E(\theta)}$, an observable $\mathcal{O}(\theta)$, and the sourced partition function $Z(\beta, J) = \int d\theta \, e^{-\beta E(\theta) + J\mathcal{O}(\theta)}$, with $F = -\log Z$. All expectations are in p_β, that is, at $J = 0$.

(a) Show that $\mathbb{E}(\mathcal{O}) = -\partial_J F|_{J=0}$ and $\mathrm{Var}(\mathcal{O}) = -\partial_J^2 F|_{J=0}$.

(b) The third cumulant is defined by continuing the pattern, $\kappa_3(\mathcal{O}) := -\partial_J^3 F|_{J=0}$. Compute it in terms of the moments $\mathbb{E}(\mathcal{O}^3)$, $\mathbb{E}(\mathcal{O}^2)$, $\mathbb{E}(\mathcal{O})$, and show that it equals the third *centered* moment $\mathbb{E}[(\mathcal{O} - \mathbb{E}\mathcal{O})^3]$.

(c) Show that for a Gaussian $\mathcal{O}$, $\log Z(J)$ is exactly quadratic in J, so that all cumulants with $p \geq 3$ vanish. Compute κ_4 in terms of centered moments and check that it is *not* the fourth centered moment. (Moral: cumulants measure the failure of higher moments to be determined by lower ones in the Gaussian pattern.)

(d) Verify (2.16).

SOLUTION, p. 610

Quenched averages. In a learning problem there is a second source of randomness besides the posterior: the training set itself. Quantities like the free energy depend on which n samples were drawn, and at large n one expects them to concentrate around their average over draws. Physicists call randomness that is fixed before the system equilibrates *quenched disorder*, by analogy with impurities frozen into an alloy when it is quenched in water; the training data are quenched disorder for the posterior. The *quenched free energy* is

$$\mathbb{E}_{\text{data}}\big[F\big] = -\,\mathbb{E}_{\text{data}}\big[\log Z\big]\,,$$

and it is the free energy, not the partition function, that should be averaged, because it is the free energy that is extensive and whose derivatives give expectation values. Averaging $\log Z$ is usually hard, and the *annealed approximation* swaps the order,

$$\mathbb{E}_{\text{data}} \log Z \;\approx\; \log \mathbb{E}_{\text{data}} Z\,,$$

which is an upper bound by Jensen's inequality and often a good approximation. We will use it in Section 2.6. The exact treatment goes through the *replica method*, sketched in Appendix A; the classic reference is (Engel & Van den Broeck 2001).

2.4 A physics example: the Ising model

The analogy with thermal physics opens up a bag of well-developed approximation methods. We illustrate two of them, the *mean-field approximation* and the *saddle-point approximation at large N*, on the most famous model in statistical mechanics. For the physics of this example, well beyond what we cover here, see (Tong 2012, Chapter 1).

The model. Let Λ be a graph with N vertices and undirected edges $\langle ij\rangle$; typically Λ is a regular lattice in D dimensions, and every vertex has the same number d of neighbors ($d = 2D$ for a hypercubic lattice). At each vertex i sits a *spin* $s_i \in \{\pm 1\}$. A state of the system is a choice of all N spins, so there are 2^N states, and the energy of a state is

$$E(s) = -J \sum_{\text{edges } \langle ij\rangle} s_i s_j \;-\; B \sum_{i=1}^{N} s_i\,.$$

The *coupling* $J > 0$ rewards neighboring spins for aligning with each other, and the *external magnetic field* B rewards spins for aligning with it. At inverse temperature β the Boltzmann distribution is $p_\beta(s) = e^{-\beta E(s)}/Z$ with $Z(\beta, J, B) = \sum_{s \in \{\pm1\}^N} e^{-\beta E(s)}$. Dimensionally, only the combinations βJ and βB matter.

The simplest macroscopic quantity describing a state is its *magnetization*, the average spin,

$$m(s) = \frac{1}{N}\sum_{i=1}^{N} s_i \;\in\; [-1, 1]\,.$$

It is our first example of an *order parameter*: a single number that summarizes an N-dimensional microscopic state for the purpose of the questions we want to ask. The question here is whether the spins align ($|m|$ close to 1, an *ordered* phase) or not (m near 0, a *disordered* phase), and how this depends on β, J, and B.

Mean-field approximation. We would like to push the distribution $p_\beta(s)$ over 2^N states forward to a distribution over the single variable m. The interaction term stands in the way, since it couples each spin to its particular neighbors. The mean-field approximation replaces the neighbors of each spin by their average: suppose that each spin feels only the *average* field of all the others, so that in the neighbor sum we may replace each s_j by m,

$$\sum_{\langle ij\rangle} s_i s_j = \frac{1}{2}\sum_i s_i \underbrace{\sum_{j\sim i} s_j}_{\approx d\,m} \;\approx\; \frac{d\,m}{2}\sum_i s_i = \frac{N d}{2}\,m^2 .$$

(The factor $\tfrac12$ compensates for counting each edge twice.) The energy becomes a function of m alone,

$$E(s) \;\approx\; -N\Big(\frac{dJ}{2}\,m^2 + B\,m\Big).$$

This is an uncontrolled approximation for a lattice in low dimension, where a spin's few neighbors fluctuate along with it, but it turns out to be qualitatively right for $D \geq 2$ and quantitatively right for $D \geq 4$ (Tong 2012, Chapter 1). It becomes exact for a model in which every spin couples weakly to every other; that model, the Curie–Weiss magnet, returns in Section 5.1 as the prototype of mean-field limits of neural networks.

Entropy, and the reduction to one variable. With the energy depending only on m, the sum over states can be organized by the value of m:

$$Z \;\approx\; \sum_m \Omega(m)\, e^{N\beta\left(\frac{dJ}{2}m^2 + Bm\right)}, \qquad \Omega(m) := \#\{s : m(s) = m\} = \binom{N}{\frac{1+m}{2}N},$$

since a state with magnetization m has exactly $\frac{1+m}{2}N$ spins up. Stirling's formula gives

$$\frac{1}{N}\log\Omega(m) \;\xrightarrow{N\to\infty}\; h(m),$$
$$h(m) := \log 2 - \frac{1-m}{2}\log(1-m) - \frac{1+m}{2}\log(1+m),$$

the *entropy* per site at magnetization m: the log of the number of microscopic states compatible with the macroscopic value m. It is largest at $m = 0$, where $h = \log 2$ and almost all of the 2^N states live, and it vanishes at $m = \pm 1$, where exactly one state lives. Replacing the sum over m by an integral (the spacing is $2/N$), the partition function becomes a one-dimensional integral,

$$Z \;\approx\; \int_{-1}^{1} dm\, e^{-N\beta\, s(m)}, \qquad s(m) := -\frac{dJ}{2}m^2 - Bm - \beta^{-1}h(m).$$

The function $s(m)$ is called the effective *action*, or the free-energy density at fixed m: energy minus temperature times entropy, per site. *We have reduced the problem from N variables to one.* This is the general shape of every calculation in this section: push the microscopic distribution forward to an order parameter, at the price of an entropy term that counts how many microstates sit at each value.

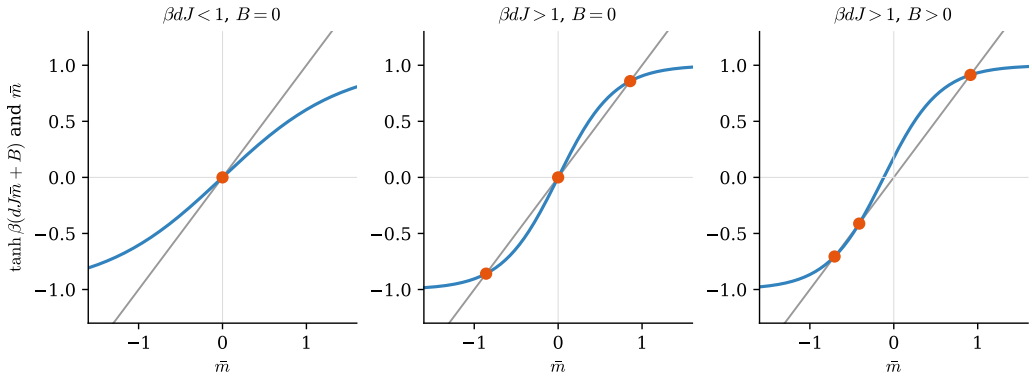
Saddle points. At large N the integrand $e^{-N\beta s(m)}$ is exponentially peaked at the *minima* of s, and the integral is dominated by a window of width $\sim 1/\sqrt{N}$ around the lowest one. This is the saddle-point approximation (also called Laplace's method), and it gives

$$\varphi(\beta, J, B) = -\frac{1}{N}\log Z \xrightarrow{N\to\infty} \beta \min_m s(m)\,.$$

The free energy is minimized, in exactly the sense promised in Section 2.3. The minimizer $\bar{m}$ is the magnetization the system actually displays. Setting $\partial s/\partial m = 0$ and using $h'(m) = -\tanh^{-1}(m)$,

$$dJ\,\bar{m} + B = \beta^{-1}\tanh^{-1}(\bar{m})\,, \qquad \text{that is,} \qquad \boxed{\bar{m} = \tanh\left(\beta\,(dJ\,\bar{m} + B)\right)}\,.$$

This transcendental equation is best solved graphically (Figure 1). Depending on β, J, and B it has one solution, or three; when there are three, the outer two are minima of s and the middle one is a maximum.



Graphical solution of the mean-field equation (2.27): intersections of $\tanh\beta(dJ\bar{m} + B)$ with the diagonal. Left: high temperature, one solution at $\bar{m} = 0$. Middle: low temperature at $B = 0$, three solutions, of which $\bar{m} = 0$ is a maximum of s and $\pm\bar{m}$ are degenerate minima. Right: low temperature with $B > 0$, three solutions, with the positive one the global minimum.

Self-consistency (an aside). There is another route to (2.27), which is the form in which mean-field theory usually appears and which we will use in Section 5. Look at the effective field felt by a single spin s_i:

$$B_i^{\text{eff}} = -\frac{\partial E}{\partial s_i} = J\sum_{j\sim i} s_j + B \;\approx\; dJ\,m + B\,.$$

A single spin in this field has partition function $Z_i = \sum_{s_i=\pm 1} e^{\beta B^{\text{eff}} s_i} = 2\cosh\beta(dJm + B)$ and mean $\mathbb{E}(s_i) = \beta^{-1}\partial_B \log Z_i = \tanh\beta(dJm + B)$. If the approximation is to be consistent, the average that comes out must equal the average we put in, $m = \mathbb{E}(s_i)$, and that is (2.27). Solve the one-body problem in the average environment, then demand that the average be reproduced: this is the prototype of every mean-field theory.

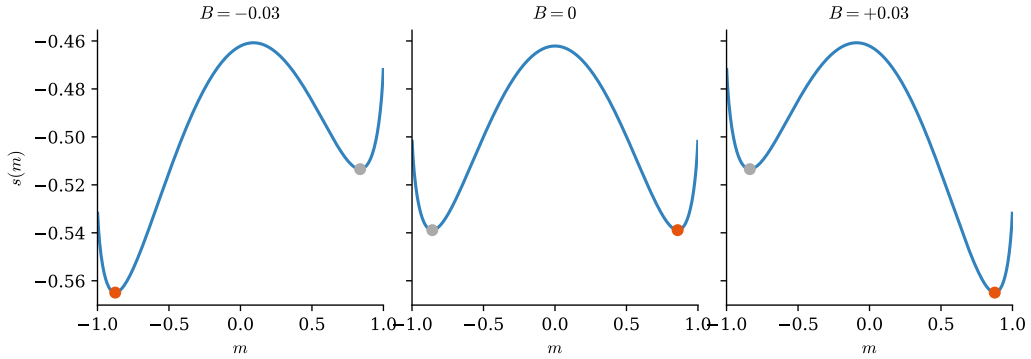
2.5 Phases and phase transitions

Now we analyze the structure of the minima of $s(m)$, which control where the probability concentrates, as β, J, and B are varied. Taylor-expanding the entropy, $h(m) = \log 2 - \frac{m^2}{2} - \frac{m^4}{12} - \dots$, the action is

$$s(m) \;\approx\; \text{const} - B\,m + \frac{1}{2}\left(\beta^{-1} - dJ\right) m^2 + \frac{\beta^{-1}}{12}\, m^4 + \dots$$

This expansion of a free energy in powers of an order parameter is called a *Landau theory*, and the qualitative analysis below depends only on its form, not on the microscopic model behind it.

The ordered phase and a first-order transition. Take $dJ > \beta^{-1}$: the coupling beats the temperature, the quadratic coefficient in (2.29) is negative, and $s(m)$ is a double well (Figure 2). The system wants to be ordered, with neighboring spins aligned; but aligned which way? The field decides. For $B < 0$ the global minimum is near $m = -1$, for $B > 0$ near $m = +1$, and at $B = 0$ the two minima are exactly degenerate. As B is swept through $B_c = 0$, the magnetization the system displays jumps from $\bar{m} \approx -1$ to $\bar{m} \approx +1$. This is a *first-order phase transition*.



The action $s(m)$ of (2.25) in the ordered phase ($\beta dJ = 1.5$), for a negative, zero, and positive external field. The two minima tilt with B; the global minimum (orange) switches sides at $B = 0$. The suppressed minimum (grey) survives as a metastable state.

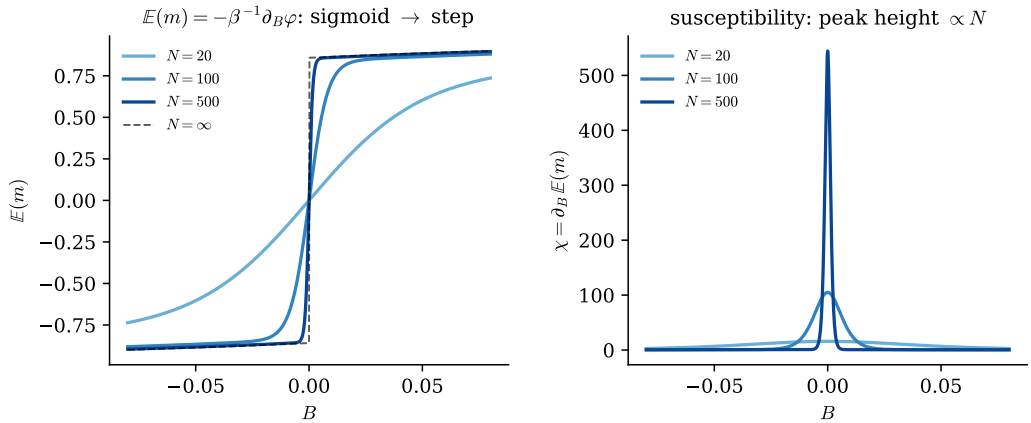
The structure of this transition is universal, in the sense that it follows from two isolated minima crossing and not from any detail of the model. Consider the expected magnetization, which by (2.14) is a derivative of the free energy with respect to the field:

$$\mathbb{E}(m) = \frac{1}{N}\sum_i \mathbb{E}(s_i) = -\frac{1}{\beta}\frac{\partial \varphi}{\partial B}\,.$$

At finite N, the two minima of s are two competing *phases*, and $\mathbb{E}(m)$ is a probability-weighted average of their magnetizations. As $N \to \infty$ the saddle-point approximation puts all the weight on the lower minimum, so $\mathbb{E}(m)$ develops a discontinuity at B_c, and φ itself develops a kink (Figure 3, left). *A discontinuity in a first derivative of the free-energy density, in the thermodynamic limit, is the definition of a first-order transition.* Its second derivative, the *susceptibility*

$$\chi := \frac{\partial\, \mathbb{E}(m)}{\partial B} = -\frac{1}{\beta}\frac{\partial^2 \varphi}{\partial B^2} = N\beta\, \mathrm{Var}(m)\,,$$

is a peak whose height grows like N and whose width shrinks like $1/N$: a nascent delta function (Figure 3, right). Genuine non-analyticity can occur only at $N = \infty$: at finite N the partition function is a finite sum of analytic functions of B, so everything is smooth, and what one sees is a rapid *crossover* that sharpens as N grows. Watching the peak in χ grow with N, rather than saturate, is the standard numerical diagnostic that separates a true transition from a crossover. Exercise 2.2 works out the shapes of these curves for a general first-order transition; they are worth remembering whenever "emergent capabilities" in models of growing size are under discussion (Section 7.6).



Fingerprints of a first-order transition, in the mean-field Ising model at $\beta dJ = 1.5$, computed from the one-dimensional integral (2.25) at finite N. Left: the expected magnetization as a function of the field is a sigmoid of width $\propto 1/N$, sharpening into a step. Right: the susceptibility $\chi = \partial_B \mathbb{E}(m)$ is a peak of height $\propto N$ and width $\propto 1/N$.

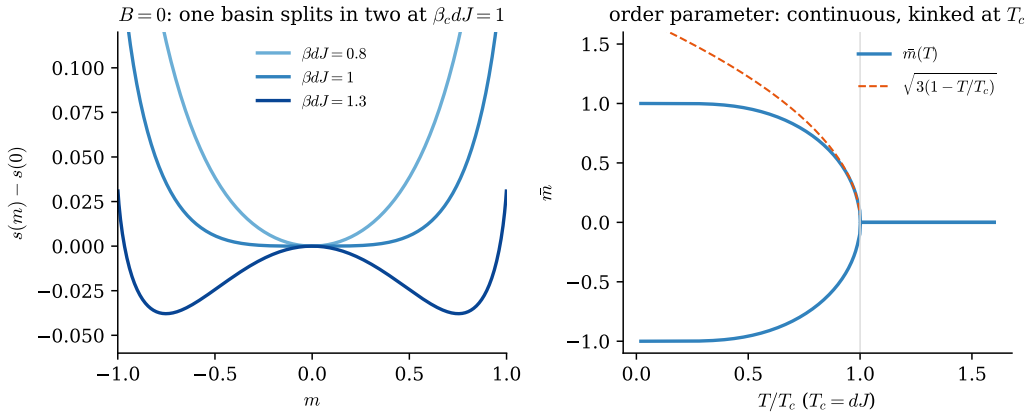
Metastability and hysteresis. The suppressed minimum in Figure 2 does not disappear when it stops being the global one. A system that starts there and evolves by local dynamics (flipping one spin at a time) must climb over the barrier to reach the true minimum, which takes a time exponential in N. In practice the system stays in the *metastable* phase until the field is pushed well past B_c, and the transition observed dynamically *lags* the thermodynamic one. This is *hysteresis*, familiar from real magnets.

It is the reason the "$\beta \leftrightarrow$ training time" analogy of Section 2.1 must be handled with care at first-order transitions, and we will meet it again in the guise of delayed generalization (grokking) in Section 5.7.

A second-order transition. An *n-th order* phase transition is a discontinuity in the n-th derivative of the free-energy density as $N \to \infty$. The Ising model has a second-order transition too, at fixed $B = 0$, as the temperature $T = \beta^{-1}$ crosses $T_c = dJ$ (Figure 4). Here nothing crosses anything: the single basin at $m = 0$ flattens and splits into two symmetric ones. The quadratic coefficient in (2.29) changes sign at T_c, and the minima sit at

$$\bar{m}^2 = 3\,(\beta dJ - 1) \;\approx\; 3\left(1 - \frac{T}{T_c}\right) \qquad (T \lesssim T_c)\,,$$

so the order parameter $\bar{m} \propto (T_c - T)^{1/2}$ moves *continuously* but not smoothly; φ and $\partial_T \varphi$ are continuous, and the discontinuity sits in $\partial_T^2 \varphi$ (the heat capacity). The point $(T, B) = (T_c, 0)$ is a *critical point*. Below it the two ordered states $\pm\bar{m}$ are exact mirror images, and the system must pick one: this is *spontaneous symmetry breaking*, the $\mathbb{Z}_2$ symmetry $s \to -s$ of the energy at $B = 0$ being broken by the state. The exponent $\frac{1}{2}$ in (2.32) is an example of a *critical exponent*. Mean-field theory predicts it in every dimension, and it is wrong in low dimension: the exact value is $\frac{1}{8}$ in $D = 2$ (Onsager), and there is no transition at all in $D = 1$. Mean-field exponents become exact for $D \geq 4$. Understanding why is the subject of the renormalization group, for which see (Tong 2012).



The second-order transition of the mean-field Ising model at $B = 0$. Left: the action $s(m)$ for temperatures above, at, and below $T_c = dJ$; a single basin flattens and splits into two symmetric ones. Right: the resulting magnetization $\bar{m}(T)$, continuous at T_c but with the square-root singularity (2.32) (dashed).

Back to learning. In the Bayesian model of Section 2.1, the natural control parameter is $n\beta$, the amount of data or training. The Ising analogue is sweeping β at fixed $B = 0$, which sees the second-order transition. First-order transitions in learning arise when two qualitatively different families of solutions compete, one entropically favored and

one favored by the data; the Ising perceptron of Section 2.6 is the cleanest example. Before turning to it, the following exercise establishes what *every* first-order transition looks like at finite N.

Exercise 2.2 (★) (Anatomy of a first-order transition). Consider any system whose partition function has the form $Z(B) = \int d\theta\, e^{-NE(\theta,B)}$, depending on *some* hyperparameter B. Here B could be the inverse temperature, an external field, or the load α of Section 2.6; we have absorbed β into E to keep the setup general. Suppose that $E(\theta, B)$ has exactly two isolated minima, at θ_1 and θ_2. Then at large N the integral is dominated by two competing phases. Letting U_1, U_2 be small neighborhoods of θ_1, θ_2,

$$Z(B) \;\approx\; \int_{U_1} d\theta\, e^{-NE(\theta,B)} + \int_{U_2} d\theta\, e^{-NE(\theta,B)} \;=\; e^{-N\varphi_1(B)} + e^{-N\varphi_2(B)}\,,$$

with $\varphi_i(B) = -\frac{1}{N}\log \int_{U_i} d\theta\, e^{-NE(\theta,B)} = E(\theta_i, B) + o(1)$ the free-energy densities of the two basins. Suppose further that as B crosses a critical value B_c the two minimum values cross: $\varphi_1(B_c) = \varphi_2(B_c)$ with $\varphi_1'(B_c) \neq \varphi_2'(B_c)$, where a prime denotes ∂_B.

(a) *Occupation.* Show that the probability of finding the system in basin i is

$$p_i(B) = \frac{e^{-N\varphi_i(B)}}{e^{-N\varphi_1(B)} + e^{-N\varphi_2(B)}}\,,$$

and that, expanding to first order in $\Delta B := B - B_c$,

$$p_2(B) \approx \frac{1}{1 + e^{-N\,\Delta\varphi'\,\Delta B}}\,, \qquad \Delta\varphi' := \varphi_1'(B_c) - \varphi_2'(B_c)\,,$$

a logistic function of $N\,\Delta\varphi'\,\Delta B$. Over what window of B does the system switch allegiance?

(b) *First derivative.* Show from (2.33) that

$$\frac{\partial \varphi}{\partial B} \;\approx\; p_1(B)\,\varphi_1'(B_c) + p_2(B)\,\varphi_2'(B_c)\,,$$

the probability-weighted average of the branch slopes. Sketch it at finite N and at $N = \infty$: a sigmoid of height $\Delta\varphi'$ sharpening into a jump. The kink in φ itself is the defining discontinuity of a first-order transition. Show also, directly from the integral, that $\partial_B \varphi = \mathbb{E}\big[\partial_B E(\theta, B)\big]$: the observable that jumps is the one conjugate to the control parameter. When $B = \beta$ it is the energy, and the jump is the *latent heat*; for the Ising model with B the field, it is the magnetization.

(c) *Second derivative.* Using $\partial_B p_1 = -\partial_B p_2 = -N p_1 p_2\,\Delta\varphi'$ (verify this), show that

$$-\frac{\partial^2 \varphi}{\partial B^2} \;=\; N\,p_1 p_2\,(\Delta\varphi')^2 \;-\; p_1\varphi_1'' - p_2\varphi_2''\,.$$

Sketch this curve. Near B_c, where $p_1 p_2 \approx \frac{1}{4}$, the first term is a peak of height $\sim N(\Delta\varphi')^2/4$ and width $\sim 1/(N\Delta\varphi')$, which converges to a delta function as $N \to \infty$. Show also that when $\mathcal{O}(\theta) = \partial_B E(\theta, B)$ does not depend on B, $-\partial_B^2 \varphi = N\,\mathrm{Var}(\mathcal{O}) \geq 0$. (For $B = \beta$ this is the heat capacity; for the Ising field it is the susceptibility.)

(d) *Plot it.* Take the minimal model $\varphi_1(B) = a\,(B - B_c)$, $\varphi_2 = 0$ and plot φ, $\partial_B\varphi$, and $-\partial_B^2\varphi$ for a few values of N. (Use a numerically stable $\log(e^x + e^y)$.) Watching the peak grow with N rather than saturate is the standard diagnostic separating a true transition from a smooth crossover.

SOLUTION, p. 610

2.6 The Ising perceptron: a first-order transition in learning

We close the physics part of this section with a complete statistical-mechanics analysis of a learning problem, built to run in parallel with the Ising magnet. It is the simplest model we know in which *more data* produces a genuine phase transition to *perfect generalization*, and it goes back to Gyorgyi (Györgyi 1990); see (Engel & Van den Broeck 2001, Chapter 7). It is presented as a walk-through exercise; solutions are in Solutions to Exercises at the back of the book, but try each part before reading them.

Setup. A *teacher* is a fixed vector of binary weights $\theta^* \in \{\pm 1\}^N$. Inputs are Gaussian, $x \sim \mathcal{N}(0, I_N)$, and the teacher labels them by

$$y(x) = \operatorname{sign}(\theta^* \cdot x)\,.$$

A *student* is another binary vector $\theta \in \{\pm 1\}^N$, guessing labels the same way, $f_\theta(x) = \operatorname{sign}(\theta \cdot x)$. This model is called the *Ising perceptron*, because its weights take the values of Ising spins. We observe n examples (x^μ, y^μ) with $y^\mu = y(x^\mu)$, and do Bayesian learning at zero temperature with a uniform prior, in the sense of Section 2.2: the per-sample loss is 0 if the student classifies the example correctly and ∞ otherwise, so the posterior is uniform on the students that classify *every* example correctly. Since teacher and student agree on x exactly when $y(x)\,(\theta \cdot x) > 0$,

$$p(\theta) \;\propto\; \prod_{\mu=1}^{n} \Theta\big(y^\mu\,(\theta \cdot x^\mu)\big)\,, \qquad Z = \frac{1}{2^N} \sum_{\theta \in \{\pm 1\}^N} \prod_{\mu=1}^{n} \Theta\big(y^\mu\,(\theta \cdot x^\mu)\big)\,,$$

with $\Theta(u) = 1$ for $u > 0$ and 0 otherwise. With the factor $1/2^N$, Z is the fraction of all students consistent with the data. Both N and n will be taken large, at fixed ratio: the *load*

$$\alpha := \frac{n}{N}\,,$$

the number of examples per parameter, is the control parameter of the problem.

Exercise 2.3 (★) (The Ising perceptron). (a) *The order parameter is a magnetization.* Define the *overlap* $R(\theta) = \frac{1}{N}\theta \cdot \theta^* \in [-1, 1]$ and the alignment variables $s_i := \theta_i \theta_i^* \in \{\pm 1\}$. Show that $R = \frac{1}{N}\sum_i s_i$: the overlap *is* the magnetization of the spins s_i. Learning the teacher means magnetizing them to $R = 1$.

(b) *Generalization error.* Show that a student at overlap R misclassifies a fresh Gaussian input with probability

$$\varepsilon(R) = \frac{\arccos R}{\pi}\,.$$

[*Hint:* project x onto the plane spanned by θ and θ^*; the projection is an isotropic two-dimensional Gaussian, and the two decision boundaries are diameters meeting at angle $\arccos R$. When do the signs disagree?]

(c) *Entropy: the same h as the magnet.* Show that the number of students at overlap R is $\binom{N}{\frac{1+R}{2}N}$, so that by Stirling's formula

$$\frac{1}{N}\log \#\{\theta : R(\theta) = R\} \;\to\; h(R) = \log 2 - \frac{1-R}{2}\log(1-R) - \frac{1+R}{2}\log(1+R)\,,$$

identical to the magnet's entropy (2.24) with $m \to R$.

(d) *Push forward to one variable.* Average Z over datasets. The examples are independent, and each is classified correctly by a student at overlap R with probability $1-\varepsilon(R)$, so

$$\mathbb{E}_x Z = \frac{1}{2^N}\sum_{\theta}\big(1-\varepsilon(R(\theta))\big)^n\,.$$

Trade the sum over θ for an integral over R using (c), and show that at fixed load α and large N,

$$\mathbb{E}_x Z \;\approx\; \int_{-1}^{1} dR\, e^{-N\, s(R)}\,, \qquad s(R) = \log 2 - h(R) - \alpha \log\big(1-\varepsilon(R)\big)\,.$$

This is the exact analogue of the magnet's (2.25): an entropy term counting states, and a "field" term rewarding alignment. The aligning field is now the *data*, and its strength is the load α. (Averaging Z rather than $\log Z$ is the annealed approximation (2.18). It preserves all the qualitative structure here and shifts the numbers only slightly; Appendix A explains how to do the exact calculation.)

(e) Plot $s(R)$ for several values of α and locate its critical points. Do you expect a phase transition? At which α?

(f) *The teacher basin.* $R = 1$ is the perfectly aligned state, where the student is the teacher. Show that $s(1) = \log 2$ for *every* α (zero entropy, but zero data cost), and that $R = 1$ is always a local minimum: writing $\delta = 1 - R$, deviating from the teacher gains entropy $\sim \frac{\delta}{2}\log(1/\delta)$ but costs data-fit $\alpha\varepsilon \sim \alpha\sqrt{2\delta}/\pi$ (expand arccos), and $\sqrt{\delta} \gg \delta\log(1/\delta)$. This differs from the magnet, where the aligned state sat at the bottom of a smooth well and was not always a minimum.

(g) *The interior basin, the crossing, and the transition.* At $\alpha = 0$, s is minimized at $R = 0$ with $s(0) = 0 < s(1)$: the disordered cloud of typical students dominates. As α grows, this interior minimum $R^*(\alpha)$ drifts toward 1 and its value rises. Numerically, find the load α_c at which the two basins exchange dominance, $s(R^*(\alpha_c)) = s(1)$. You should

find $\alpha_c \approx 1.45$, with the interior minimum still at $R^* \approx 0.70$, that is, generalization error $\varepsilon^* \approx 0.25$. Conclude that at α_c the typical student jumps from 25% error to *zero* error: a first-order transition to perfect generalization. Then connect to Exercise 2.2 with α as the control parameter: what do $\partial_\alpha \varphi$ and $-\partial_\alpha^2 \varphi$ look like at finite N, and what plays the role of the latent heat? Finally, find the load at which the interior minimum disappears altogether, and explain what happens in between in the light of the metastability discussion of Section 2.5.

(h) *The dictionary.* Complete the table:

Ising magnet	**Ising perceptron**
spins $s_i \in \{\pm 1\}$	alignments $s_i = \theta_i \theta_i^* \in \{\pm 1\}$
magnetization m	overlap R
entropy $h(m)$	entropy $h(R)$ *(the same function)*
aligning field B	?
knob swept across the transition	?
ordered phase $\mathbb{E}(m) \approx \pm 1$	?
jump in $\mathbb{E}(m)$ as a function of B	?

Also articulate one structural *difference*: in the magnet the two competing minima are symmetry partners; what are they here, and which ingredient of (c) made a sharp transition possible at all? (What would change for spherical weights $\theta \in \mathbb{R}^N$, $|\theta|^2 = N$, where the slice at overlap R has volume $\propto (1 - R^2)^{N/2}$? Appendix A.3 works this case out.)

SOLUTION, p. 611

2.7 Comparison with QFT

This short subsection is a translation guide, aimed mainly at readers arriving from high-energy or condensed-matter physics — if the words "action" and "path integral" mean more to you than "posterior" and "marginal likelihood," read on; everyone else may safely skip ahead.

Nearly everything in these notes lives, in the first instance, in *Euclidean equilibrium statistical mechanics*: probability distributions of Boltzmann–Gibbs form $P \propto e^{-\beta E}$, partition functions, free energies. For a field theorist, a partition function with sources,

$$Z[J] = \int \mathcal{D}\phi \, e^{-S[\phi] + \int J \phi},$$

is a moment generating functional; its logarithm generates connected correlation functions (cumulants); the action S encodes the microscopic rules of the system, and a quadratic action means a free (Gaussian) theory, with everything else treated by per-

turbing around it. In these notes the role of the field ϕ is played variously by the weights θ of a network, by an order parameter such as the magnetization m or the overlap R of Sections 2.4 and 2.6, or — most usefully — by the network function $f(x)$ itself, with the input x playing the role of the spacetime coordinate. Functional integrals over the network function are developed carefully in Sections 3.3 and 3.5, and the reader coming from QFT will find many familiar ingredients there dressed up as learning theory: Wick's theorem, sources, cumulants, effective actions.

If your instincts were trained on Lorentzian signature, picture everything here as already Wick-rotated. There is no $i/\hbar$: the weights e^{-S} are real and positive, as after the standard continuation

$$Z_{\text{QFT}} = \int \mathcal{D}\phi \; e^{\frac{i}{\hbar}S[\phi]} \quad \xrightarrow{\tau = it} \quad Z_{\text{E}} = \int \mathcal{D}\phi \; e^{-S_E[\phi]/\hbar} \, .$$

The role of $\hbar$ — the parameter controlling the size of fluctuations around saddle points — is played here by temperature, or by $1/N$ at large width, or by $1/n$ at large sample size.

When we eventually study the *dynamics* of training as a field theory in Section 6, a genuinely real-time formalism does appear — a path integral over trajectories with a causal structure and a response field. That's the closest object in these notes to a Lorentzian field theory, and is precisely the Martin–Siggia–Rose construction of non-equilibrium statistical mechanics.

For readers who would like physics references alongside the learning-theory ones: David Tong's lecture notes (Tong 2012; Tong 2012; Tong 2012; Tong 2006) are a friendly introduction to statistical physics, statistical field theory, kinetic theory, and QFT respectively; Lancaster and Blundell (Lancaster & Blundell 2014) is a gentle conceptual route into QFT; Altland and Simons (Altland & Simons 2010) bridges toward condensed-matter field theory (and contains the Langevin/MSRJD material used in Section 6); Pathria (Pathria & Beale 2011) is a standard graduate statistical mechanics text; Goldenfeld (Goldenfeld 2018) is the classic on phase transitions and the renormalization group; and Zinn-Justin (Zinn-Justin 2002) is the encyclopedic reference for the functional formalism.

3. Large width at initialization: the Gaussian process limit

The next few sections are all about different limits in neural networks, in which some parameter(s) become large. Such limits are where physical/statistical analyses really shine. Studying them teaches some basic lessons about how we should (and should not) initialize networks, and how to choose other hyperparameters. It also leads to analytically solvable models that display many properties we see in practice: lazy vs. rich learning, double descent, phase transitions, scaling laws for optimal learning. For further pedagogical references on much of this material, we recommend (Roberts et al. 2022; Spiliopoulos et al. 2025; Ringel et al. 2025) and the online lectures (Hanin 2026).

We'll begin in this section with a classic result (Neal 1996; Lee et al. 2018; Yang 2019; Hanin 2023) about increasing the width of a network at fixed depth and training-sample size.

Modern neural networks are more sophisticated than the simple MLP's discussed here, but they share many of the same features. Layer norms solve the problem of trainability at large depth, but the analysis of rich/lazy regimes, feature learning, and hyperparameter scaling is still relevant. See *e.g.* the discussion of μP in Section 5.5.

3.1 Notation

We'll use the following notation for neural networks and training data.

- $\theta \in \mathbb{R}^P$ trainable network parameters (weights, biases)
- $(x^\mu, y^\mu)_{\mu=1}^n$ training data samples, with $x^\mu \in \mathbb{R}^d$ and $y^\mu \in \mathbb{R}$ (unless otherwise stated)
- $f_\theta : \mathbb{R}^d \to \mathbb{R}$ network function

For MLP's in particular (also called FCN's = fully connected networks)

- $\{z_i^{(\ell)}\}_{i=1}^{N_\ell}$ pre-activations in layer $\ell \in \{0, ..., L+1\}$
- $L =$ number of hidden layers (depth); $N_\ell =$ width of ℓ'th layer; $N_0 = d$ and $N_{L+1} = 1$. We will often take constant hidden depth, $N_1 = ... = N_L = N$.
- $\phi : \mathbb{R} \to \mathbb{R}$ activation function (assumed the same in all layers except the first)
- network recursion $z_i^{(\ell)} = \begin{cases} \frac{1}{\sqrt{N_{\ell-1}}} \sum_{j=1}^{N_{\ell-1}} w_{ij}^{(\ell)} \, \phi(z_j^{(\ell-1)}) + b_i^{(\ell)} & \ell \geq 2 \\ \frac{1}{\sqrt{d}} \sum_{j=1}^{d} w_{ij}^{(1)} \, x_j + b_i^{(1)} & \ell = 1 \end{cases}$ with $z^{(0)} = x$ and $z^{(L+1)} = f$; parameters are $\theta = \{w^{(\ell)}, b^{(\ell)}\}_{\ell=1}^{L+1}$. The explicit factors of $1/\sqrt{N_{\ell-1}}$ are a normalization convention, explained in Section 3.2: with them in place, all weights and biases can be taken of order one.

3.2 Normalizing the network

Consider a fully connected network of depth L with layer widths N_ℓ as in Section 3.1. Suppose we keep L fixed but make the N_ℓ very large. Suppose also that the network is initialized randomly, with all weights and biases drawn independently from normal distributions of *order-one* variance,

$$w_{ij}^{(\ell)} \sim \mathcal{N}(0, \sigma_w) \,, \qquad b_i^{(\ell)} \sim \mathcal{N}(0, \sigma_b) \,,$$

with σ_w, σ_b fixed as the widths grow. (Here and throughout, $\mathcal{N}(0, \sigma)$ denotes a Gaussian of *variance* σ. Letting the variances depend on the layer changes nothing below.) The question is how to normalize the sums in the network recursion so that preactivations stay of order one from layer to layer. From

$$z_i^{(\ell)} = \frac{1}{\sqrt{N_{\ell-1}}} \, w_i^{(\ell)} \cdot \phi(z^{(\ell-1)}) + b_i^{(\ell)}$$

(where we've written $w_i = (w_{i,1}, \ldots, w_{i,N_{\ell-1}})$ and $z^{(\ell-1)} = (z_1^{(\ell-1)}, \ldots, z_{N_{\ell-1}}^{(\ell-1)})$ as vectors) we get

$$\begin{aligned}\mathrm{Var}(z_i^{(\ell)}) &= \frac{1}{N_{\ell-1}} \sum_{j=1}^{N_{\ell-1}} \mathrm{Var}\big(w_{ij}^{(\ell)}\big)\, \mathbb{E}\Big[\phi\big(z_j^{(\ell-1)}\big)^2\Big] \;+\; \mathrm{Var}\big(b_i^{(\ell)}\big) \\ &= \sigma_w \cdot \frac{1}{N_{\ell-1}} \sum_{j=1}^{N_{\ell-1}} \Big(\mathrm{Var}\big(\phi(z_j^{(\ell-1)})\big) + \mathbb{E}\big[\phi(z_j^{(\ell-1)})\big]^2\Big) + \sigma_b\,.\end{aligned}$$

Here we've conditioned on the previous layer, and used that for independent mean-zero X and arbitrary Y, $\mathrm{Var}(XY) = \mathrm{Var}(X)\,\mathbb{E}[Y^2]$, along with additivity of variance for independent summands. The average in the second line involves only the activation function and the order-one preactivations of the previous layer, so it stays of order one as $N_{\ell-1} \to \infty$. (We'll sharpen this estimate in Section 3.7.) Thus

$$\mathrm{Var}(z_i^{(\ell)}) \approx \sigma_w \cdot O(1) + \sigma_b = O(1)\,.$$

The prefactor $1/\sqrt{N_{\ell-1}}$, the square root of the number of inputs (the "fan-in") feeding each neuron, is exactly what keeps the preactivations of order one at large width. We call this the *fan-in normalization*; it goes back to LeCun and collaborators in the 90's (LeCun et al. 1998). In Section 4 it will also be called the *NTK normalization*, and in Section 5 we will meet the alternative *mean-field normalization*, in which the output layer carries $1/N$ instead of $1/\sqrt{N}$.

Remark (other conventions). Common practice, and the convention of much of the literature, is to put no explicit prefactors in the network and instead to initialize with width-dependent variances,

$$\tilde{w}_{ij}^{(\ell)} := \frac{w_{ij}^{(\ell)}}{\sqrt{N_{\ell-1}}} \;\sim\; \mathcal{N}\Big(0, \frac{\sigma_w}{N_{\ell-1}}\Big)\,.$$

At initialization the two conventions describe *exactly* the same distribution of network functions: (3.5) is a change of variables on parameter space, and the network function is the same function of x. Everything in this section, which concerns initialization only, is insensitive to the choice. During training, however, the two conventions are *not* equivalent: gradient descent is not invariant under rescaling coordinates on parameter space, as we will explain in Section 4.3. Keeping every parameter of order one, with the factors of N displayed explicitly in the network function, makes the comparison between different scaling limits transparent, and it is the convention we use throughout these notes. When reading the literature, expect the factors of N to sit in the variances instead, and translate with (3.5).

3.3 Gaussian processes

A *Gaussian process* $f : \mathbb{R}^m \to \mathbb{R}$ with mean function μ and covariance kernel K is a random function such that on any p inputs $(x^1, \ldots, x^p) \in (\mathbb{R}^m)^p$ the vector of function values is Gaussian

$$\begin{pmatrix} f(x^1) \\ \vdots \\ f(x^p) \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu(x^1) \\ \vdots \\ \mu(x^p) \end{pmatrix}, \begin{pmatrix} K(x^1,x^1) & \cdots & K(x^1,x^p) \\ \vdots & \ddots & \vdots \\ K(x^p,x^1) & \cdots & K(x^p,x^p) \end{pmatrix} \right).$$

Here $\mu : \mathbb{R}^m \to \mathbb{R}$ and $K : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ are fixed, deterministic functions that control the mean and covariance of any tuple of values of f. The function K, often called the kernel of the process, is symmetric: $K(x,x') = K(x',x)$.

Let `dx' denote a measure on $\mathbb{R}^m$ (it need not be the usual Lebesgue measure). If K, as integral kernel, has an inverse K^{-1} satisfying

$$\int dx' K^{-1}(x,x')K(x',x'') = \delta(x-x'')$$

then there's also a "physics definition" of a Gaussian process. It's not entirely rigorous, as it invokes multi-dimensional functional integrals. Mathematically, it's best thought of as a heuristic tool, which comes with a package of axiomatic manipulations whose actual results can (and should) all be fully justified by other means. The manipulations are all quite intuitive, though, and familiar to quantum field theorists, so we mention this formalism here. The idea is that, just like a single Gaussian variable `z' has a probability measure

$$P(z) = \frac{1}{Z} e^{-\frac{1}{2}K^{-1}(z-\mu)^2}, \qquad Z = \int dz\, e^{-\frac{1}{2}K^{-1}(z-\mu)^2} = \sqrt{2\pi K},$$

a Gaussian process f has a (non-rigorous) measure

$$P[f] = \frac{1}{Z} e^{-\frac{1}{2}\int dx\, dx'\, (f(x)-\mu(x))K^{-1}(x,x')(f(x')-\mu(x'))},$$
$$Z \text{ `=' } \int Df\, e^{-\frac{1}{2}\int dx\, dx'\, (f(x)-\mu(x))K^{-1}(x,x')(f(x')-\mu(x'))}.$$

If the kernel and the measure theory on the input space $\mathbb{R}^m$ are nice enough to allow K to be diagonalized, with a discrete basis of orthonormal eigenfunctions $\{\phi_k(x)\}_{k=1}^{\infty}$ and eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots > 0$ (so that K^{-1} has eigenvalues $1/\lambda_k$), then $P[f]$ can be understood as an infinite-dimensional Gaussian. Writing $f(x) - \mu(x) = \sum_{k=1}^{\infty} a_k \phi_k(x)$,

$$P[f] = \frac{1}{Z} e^{-\frac{1}{2}\sum_{k=1}^{\infty} a_k^2/\lambda_k}, \qquad Z = \prod_{k=1}^{\infty} (2\pi\lambda_k)^{1/2}.$$

One virtue of the physics formalism is that it organizes expectation values of products of $f(x)$'s. Let $\mu = 0$ for simplicity (to generalize to any μ, just shift $f \mapsto f - \mu$ below). Let $J : \mathbb{R}^m \to \mathbb{R}$ be an arbitrary (nice...) function and define

$$Z[J] = \int Df\, e^{-\frac{1}{2}\int dx\, dx'\, f(x)K^{-1}(x,x')f(x') + \int dx\, J(x)f(x)}.$$

The integral evaluates formally to

$$Z[J] = Z[0] \cdot e^{\frac{1}{2}\int dx\, dx'\, J(x)K(x,x')J(x')}\,.$$

Then expectation values can be expressed via derivatives, which by (3.12) become

$$\begin{aligned}\mathbb{E}\big[f(x^1)\ldots f(x^p)\big] &= \frac{1}{Z[0]}\frac{\delta}{\delta J(x^1)}\cdots\frac{\delta}{\delta J(x^p)}Z[J]\Big|_{J=0} \\ &= \sum_{\text{pairs } \{i_1,i_2\}\cup\ldots\cup\{i_{p-1},i_p\}=\{1,2,\ldots,p\}} \prod_{j=1}^{p/2} K(x^{i_{2j-1}}, x^{i_{2j}})\,.\end{aligned}$$

The sum is over partitions of $\{1,\ldots,p\}$ into pairs, and vanishes if p is odd. Eqn. (3.14) is called Wick's Theorem in QFT, Isserlis's Theorem in probability (and it's correct, despite the non-rigorous derivation here).

Exercise 3.1 (Wick's theorem). If you haven't seen this before, derive (3.12) by completing the square in (3.11), and then derive (3.14) by differentiating. Check the first two cases explicitly: $\mathbb{E}[f(x^1)f(x^2)] = K(x^1,x^2)$, and $\mathbb{E}[f(x^1)f(x^2)f(x^3)f(x^4)] = K^{12}K^{34} + K^{13}K^{24} + K^{14}K^{23}$ (with $K^{ij} = K(x^i,x^j)$). How many terms appear at $p = 6$?

SOLUTION, p. 613

Another virtue of the physics formalism is that it neatly connects *higher cumulants* to *deviations from Gaussianity*. In general, the p-th cumulant $\kappa_p(X_1,\ldots,X_p)$ of random variables $X_1,\ldots,X_p$ is defined recursively via

$$\mathbb{E}(X_1\cdots X_p) = \sum_{\text{partition } \Pi \text{ of } \{1,\ldots,p\}} \prod_{\text{blocks } \{i_1,\ldots,i_k\}\subset\Pi} \kappa_k(X_{i_1},\ldots,X_{i_k})$$

For example, $\kappa_1(X) = \mathbb{E}(X)$, $\kappa_2(X_1,X_2) = \mathrm{Cov}(X_1,X_2) = \mathbb{E}(X_1X_2) - \mathbb{E}(X_1)E(X_2)$. In physics, the κ_p's are called connected correlation functions; they measure the part of a joint expectation that's not captured by expectations of subsets of the variables involved. For a Gaussian process,

$$\kappa_2(f(x^1),f(x^2)) = K(x^1,x^2)\,; \qquad \kappa_p(f(x^1),\ldots,f(x^p)) = 0 \quad \text{for all } p>2\,,$$

as can easily be seen directly from (3.14), where joint expectations for $p > 2$ all factor into products of pairs. Thus, nonvanishing higher κ_p's are a measure of non-Gaussianity.

This appears in the functional-integral formalism as follows. Suppose that $f : \mathbb{R}^m \to \mathbb{R}$ is a random function that's governed by a functional probability distribution

$$P[f] = \frac{1}{Z}\,e^{-S[f]}\,,$$

for some functional $S : \{\text{functions } f\} \to \mathbb{R}$. (In physics S is called the action.) Then just as in (3.11), we can introduce the generating function

$$Z[J] := \int Df\, e^{-S[f] + \int dx\, J(x) f(x)}$$

$$\Rightarrow \quad \mathbb{E}\big[f(x^1)...f(x^p)\big] = \frac{1}{Z[0]} \frac{\delta}{\delta J(x^1)} ... \frac{\delta}{\delta J(x^p)} Z[J]\Big|_{J=0}$$

The cumulants appear naturally as derivatives of $\log Z[J]$, otherwise known as the (sourced) free energy:

$$\kappa_p(f(x^1), ..., f(x^p)) = \frac{\delta}{\delta J(x^1)} ... \frac{\delta}{\delta J(x^p)} \log Z[J]\Big|_{J=0},$$

which in turn means (formally) that

$$Z[J] = \exp \sum_{p=1}^{\infty} \frac{1}{p!} \int dx^1 ... dx^p\, \kappa_p(f(x^1), ..., f(x^p))\, J(x^1) ... J(x^p)\,,$$

a generating function of cumulants. The original $S[f]$ is the inverse Legendre transform of $\log Z[J]$ (by definition of $Z[J]$). Expanded as a formal series in the cumulants (assuming they are small), and assuming $\mathbb{E}[f(x)] = 0$ for simplicity, it takes the form

$$\begin{aligned} S[f] = \frac{1}{2} \int & dx^1\, dx^2\, \kappa_2^{-1}(x^1, x^2) f(x^1) f(x^2) \\ - \int & dx^1 dx^2 dx^3\, dy^1 dy^2 dy^3\ \kappa_2^{-1}(x^1, y^1)\, \kappa_2^{-1}(x^2, y^2) \\ & \times\ \kappa_2^{-1}(x^3, y^3)\, \kappa_3(y^1, y^2, y^3)\, f(x^1) f(x^2) f(x^3) \\ - \int & dx^1 dx^2 dx^3 dx^4\, (\kappa_2^{-1})^4 \big[\kappa_4 - \kappa_3 \kappa_2^{-1} \kappa_3 \ \text{ terms}\big] \\ & \times\ f(x^1) f(x^2) f(x^3) f(x^4)\ - \dots \end{aligned}$$

Physicists might recognize this as a sum of Feynman diagrams, with cumulants giving the vertices. A key structure is that the p-th order term in the effective action contains only κ_p and lower cumulants. In particular, f is a Gaussian process if and only if $S[f]$ is quadratic; and small deviations from Gaussianity, measured by cumulants, provide the higher-order terms in $S[f]$.

3.4 The NNGP correspondence

A classic result in the theory of neural networks is that, at fixed depth and large width, with the fan-in normalization of Section 3.2, the network function at initialization converges to a Gaussian process. The theorem appeared in the 90's (Neal 1996) for a single hidden layer, and was generalized more recently to deeper networks and different architectures, *cf.* (Lee et al. 2018; Yang 2019; Hanin 2023).

In this theorem, the source of stochasticity in the network function comes from the random initializations of network parameters. One can think of randomly initializing a swarm of networks and asking what sorts of values the network function outputs. The

result is significant because of its universality, and because it feeds into (and greatly simplifies) further analyses of training.

The derivation proceeds via induction. We'll just sketch it here, for MLP's.

[2]

For simplicity we'll assume that biases have been set to zero, and that all weights have the same order-one variance σ_w, as in (3.1), with the fan-in normalization of Section 3.2.

In the first network layer, the preactivations are

$$z_i^{(1)}(x) = \frac{1}{\sqrt{d}} \sum_{j=1}^{d} w_{ij}^{(1)} x_j \,, \qquad w_{ij}^{(1)} \sim \mathcal{N}(0, \sigma_w) \,.$$

At fixed x, each $z_i^{(1)}(x)$ is a sum of d i.i.d. Gaussian variables (with coefficients x_j), and thus is Gaussian itself, with $\mathbb{E}[z_i^{(1)}] = 0$ and

$$\mathrm{Cov}\big(z_i^{(1)}(x), z_i^{(1)}(x')\big) = \frac{1}{d} \sum_{j,j'=1}^{d} \underbrace{\mathrm{Cov}(w_{ij}, w_{ij'})}_{\delta_{jj'}\sigma_w} x_j x'_{j'} = \sigma_w \frac{x \cdot x'}{d} \,,$$

thus $z_i^{(1)}$ is a Gaussian process (GP) with

$$z_i^{(1)} \sim \mathcal{N}(0, K^{(1)}) \,, \qquad K^{(1)}(x, x') = \sigma_w \frac{x \cdot x'}{d} \,.$$

In the second layer, we've got

$$z_i^{(2)} = \frac{1}{\sqrt{N_1}} \sum_{j=1}^{N_1} w_{ij}^{(2)} \phi(z_j^{(1)}) \,, \qquad w_{ij}^{(2)} \sim \mathcal{N}(0, \sigma_w) \,.$$

If we condition on the preactivations of the first layer, the same argument gives

$$z_i^{(2)} \sim \mathcal{N}(0, K^{(2)}) \,, \qquad K^{(2)}(z^{(1)}, z^{(1)\prime}) = \sigma_w \frac{\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})}{N_1} \,.$$

Thus we again get a Gaussian process. The problem is that if we don't condition on the preactivations, the kernel of this GP is not deterministic, but becomes a random variable itself: $\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})$. This is where the infinite-width limit comes in. Heuristically, the central limit theorem implies that the sum of products of (non-Gaussian!) random variables on the LHS of (3.25) will become Gaussian as $N_1 \to \infty$. More concretely, we can calculate and bound the variance

$$\begin{aligned} \mathrm{Var}\big(K^{(2)}(z^{(1)}, z^{(1)\prime})\big) &= \frac{\sigma_w^2}{N_1^2} \mathrm{Var}\big(\phi(z^{(1)}) \cdot \phi(z^{(1)\prime})\big) \\ &= \frac{\sigma_w^2}{N_1^2} N_1 \cdot \mathrm{Var}\big(\phi(z_1^{(1)})\phi(z_1^{(1)\prime})\big) \to 0 \quad \text{as } N_1 \to \infty \end{aligned}$$

as long as the first-layer preactivations are order one (independent of N_1), and the activation function is reasonable (any standard choice will be). Thus, as $N_1 \to \infty$, we can replace the stochastic kernel $K^{(2)}$ by its expectation,

$$\bar{K}^{(2)}(x,x') := \sigma_w \, \mathbb{E}_{z_1^{(1)}(x),z_1^{(1)}(x')\sim\mathcal{N}(0,K^{(1)})}\big[\phi(z_1^{(1)}(x))\phi(z_1^{(1)}(x'))\big]$$

Each subsequent layer is the same. At infinite width, the kernels become deterministic, and obey a recursion

$$\bar{K}^{(\ell)}(x,x') = \sigma_w \, \mathbb{E}_{u(x),u(x')\sim\mathcal{N}(0,\bar{K}^{(\ell-1)})}\big[\phi(u(x))\phi(u(x'))\big]\,,$$

with $\bar{K}^{L+1}(x,x')$ ultimately governing a GP for the network function $f(x)$.

Remark (exact kernels). For a few special activation functions, the Gaussian expectation in (3.30) can be evaluated in closed form, giving explicit NNGP kernels. Write $K_{11} = \bar{K}^{(\ell-1)}(x,x)$, $K_{22} = \bar{K}^{(\ell-1)}(x',x')$ and $K_{12} = \bar{K}^{(\ell-1)}(x,x')$ for the previous layer's kernel values. For ReLU, $\phi(u) = \max(u,0)$, one finds the *arc-cosine kernel* of Cho and Saul (Cho & Saul 2009),

$$\bar{K}^{(\ell)}(x,x') = \frac{\sigma_w}{2\pi}\sqrt{K_{11}K_{22}}\,\Big(\sin\vartheta + (\pi-\vartheta)\cos\vartheta\Big)\,,$$
$$\cos\vartheta = \frac{K_{12}}{\sqrt{K_{11}K_{22}}}\,,$$

and for the sigmoid-like activation $\phi = \mathrm{erf}$ Williams (Williams 1997) found

$$\bar{K}^{(\ell)}(x,x') = \frac{2\sigma_w}{\pi}\,\arcsin\!\left(\frac{2K_{12}}{\sqrt{(1+2K_{11})(1+2K_{22})}}\right).$$

Iterating either formula through the layers gives the exact infinite-width kernel of a deep ReLU or erf network in a few lines of code. (Deriving (3.31)–(3.32) is a pleasant but somewhat lengthy Gaussian-integral workout.)

Exercise 3.2 (The NNGP correspondence, empirically (lab)). Fix two unit-norm inputs $x, x' \in \mathbb{R}^d$ with, say, $d = 8$ and $x \cdot x' = 0.5$. For widths $N \in \{3, 30, 300\}$, sample $M = 10^4$ independent initializations of the one-hidden-layer erf network $f(x) = \frac{1}{\sqrt{N}}\sum_{i=1}^{N} c_i \,\mathrm{erf}\big(a_i \cdot x/\sqrt{d}\big)$ with $c_i, a_{ij} \sim \mathcal{N}(0,1)$ (that is, $\sigma_w = 1$).

(a) Make a histogram of the values $f(x)$ across the ensemble, and overlay the Gaussian density with the variance $\bar{K}^{(2)}(x,x)$ predicted by (3.32); watch the visibly non-Gaussian $N = 3$ histogram converge.

(b) Make a scatter plot of the pairs $\big(f(x), f(x')\big)$ and compare the sample 2×2 covariance with the predicted kernel matrix.

(c) For each initialization, record the stochastic kernel $K^{(2)}$ of (3.26), and verify that its variance across initializations decays like $1/N$ (that's the mechanism by which the kernel becomes deterministic).

SOLUTION, p. 614

3.5 NNGP via functional integrals

One can also analyze large width at initialization via functional integrals. This just reorganizes the above steps. We'll illustrate how it works for a network with one hidden layer and zero biases, $N_2 = 1,\ N_1 = N,\ N_0 = d$,

$$f(x) = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} c_i\, \phi\Big(\frac{1}{\sqrt{d}}\, a_i \cdot x\Big)\,, \qquad c_i \in \mathbb{R},\ a_i, x \in \mathbb{R}^d\,.$$

The initial distribution of weights, all of variance σ_w, is

$$P(c,a) = \frac{1}{Z}\, e^{-\frac{1}{2\sigma_w}|c|^2 - \frac{1}{2\sigma_w}|a|^2}\,.$$

To push this forward to a distribution on the first-layer preactivations $z_i = \frac{1}{\sqrt{d}} a_i \cdot x$, we integrate against a delta function:

$$P(c,z) = \int da\, P(c,a) \prod_i \delta\Big(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x\Big)\,.$$

Here the $z_i(x)$ are *arbitrary* functions of x, and are only tied to the network values $\frac{1}{\sqrt{d}} a_i \cdot x$ via the delta functions. Eqn. (3.35) represents a functional probability density on the functions z_i. To evaluate it, introduce a second set of "Lagrange multiplier" functions $\tilde{z}_i$, and write

$$\delta\Big(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x\Big) = \int D\tilde{z}_i\, e^{i \int dx\, \tilde{z}_i(x)(z_i(x) - \frac{1}{\sqrt{d}} a_i \cdot x)}\,,$$

and then do the exact Gaussian integral over a's, and then over $\tilde{z}$'s:

$$\begin{aligned}
P(c,z) &= \frac{1}{Z} \int da\, D\tilde{z}\, \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 - \textstyle\sum_i \tfrac{1}{2\sigma_w}|a_i|^2 \\
&\qquad\qquad + i \textstyle\sum_i \int dx\, \tilde{z}_i(x)(z_i(x) - \tfrac{1}{\sqrt{d}} a_i \cdot x)\Big] \\
&= \frac{1}{Z} \int da\, D\tilde{z}\, \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 + \textstyle\sum_i \Big[-\tfrac{1}{2\sigma_w}\big(a_i + \tfrac{i\sigma_w}{\sqrt{d}} \textstyle\int dx\, \tilde{z}_i(x) x\big)^2 \\
&\qquad - \tfrac{\sigma_w}{2d} \textstyle\int dx\, dx'\, \tilde{z}_i(x)(x \cdot x')\tilde{z}_i(x') + i \int dx\, \tilde{z}_i(x) z_i(x)\Big]\Big] \\
&= \frac{1}{Z'} \int D\tilde{z}\, \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 - \tfrac{\sigma_w}{2d} \textstyle\sum_i \int dx\, dx'\, \tilde{z}_i(x)(x \cdot x')\tilde{z}_i(x') \\
&\qquad\qquad + i \textstyle\sum_i \int dx\, \tilde{z}_i(x) z_i(x)\Big] \\
&= \frac{1}{Z''} \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 - \tfrac{1}{2\sigma_w} \textstyle\sum_i \int dx\, dx'\, z_i(x) \big(\tfrac{x \cdot x'}{d}\big)^{-1} z_i(x')\Big]\,.
\end{aligned}$$

We clearly see the first-layer GP kernel $K^{(1)}(x,x') = \frac{\sigma_w}{d} x \cdot x'$ appearing.

To push-forward to the network outputs $f = \frac{1}{\sqrt{N}} c \cdot \phi(z)$, we just do this again:

$$
\begin{aligned}
P(f) &= \int dc\, Dz\, P(c,z)\, \delta\Big(f(x) - \tfrac{1}{\sqrt{N}} c \cdot \phi(z(x))\Big) \\
&= \frac{1}{Z''} \int dc\, Dz\, D\tilde{f}\, \exp\Big[-\tfrac{1}{2\sigma_w}|c|^2 \\
&\qquad\qquad - \tfrac{1}{2\sigma_w} \textstyle\sum_i \int dx\, dx'\, z_i(x) \big(\tfrac{x\cdot x'}{d}\big)^{-1} z_i(x') \\
&\qquad\qquad + i \textstyle\int dx \tilde{f}(x)(f(x) - \tfrac{1}{\sqrt{N}} c \cdot \phi(z(x)))\Big] \\
&= \frac{1}{Z'''} \int Dz\, D\tilde{f}\, \exp\Big[-\tfrac{1}{2\sigma_w} \textstyle\int dx\, dx' \big(\tfrac{x\cdot x'}{d}\big)^{-1} z(x)\cdot z(x') \\
&\qquad\qquad - \tfrac{\sigma_w}{2N} \textstyle\int dx\, dx' \tilde{f}(x)[\phi(z(x))\cdot\phi(z(x'))]\tilde{f}(x') + i \textstyle\int dx\, \tilde{f}(x) f(x)\Big]
\end{aligned}
$$

The problem is that it's now impossible to do the integral over the z's exactly, because (for general activation function ϕ) the action is no longer quadratic in them. However, as $N \to \infty$, we can treat $\exp\big[-\frac{\sigma_w}{2N} \int dx\, dx' \tilde{f}(x)[\phi(z(x))\cdot\phi(z(x'))]\tilde{f}(x')\big]$ as a perturbation, and approximate the *exponent* $[\phi(z(x))\cdot\phi(z(x'))]$ by its expectation value under the Gaussian measure $\exp\big[-\frac{1}{2\sigma_w} \int dx\, dx' \big(\frac{x\cdot x'}{d}\big)^{-1} z(x)\cdot z(x')\big]$ (with kernel $K^{(1)}$) on the z's. To be clear, the approximation that's happening is of the form

$$
\mathbb{E}\, e^{\frac{1}{N}X} = e^{\frac{1}{N}\mathbb{E}X} + O(\mathrm{Var}(X)/N^2)
$$

Up to $O(1/N)$ corrections, that produces the deterministic kernel of the previous section,

$$
\bar{K}^{(2)}(x,x') = \sigma_w\, \mathbb{E}_{z\sim\mathcal{N}(0,K^{(1)})}\big[\phi(z(x))\,\phi(z(x'))\big]\,,
$$

and we get

$$
\begin{aligned}
P(f) &= \frac{1}{Z''''} \int D\tilde{f}\, \exp\Big[-\tfrac{1}{2} \textstyle\int dx\, dx'\, \tilde{f}(x)\bar{K}^{(2)}(x,x')\tilde{f}(x') \\
&\qquad\qquad + i \textstyle\int dx\, \tilde{f}(x) f(x)\Big] + O\Big(\tfrac{1}{N}\Big) \\
&= \frac{1}{Z'''''} \exp\Big[-\tfrac{1}{2} \textstyle\int dx\, dx'\, f(x)\bar{K}^{(2)}(x,x')^{-1} f(x')\Big] + O\Big(\tfrac{1}{N}\Big)\,.
\end{aligned}
$$

3.6 L/N

The inductive argument of Section 3.4 does not hold at infinite L. There are two potential problems.

First, a factor of σ_w appears with every layer. These factors will accumulate, and can lead either to vanishing kernels (which also turns out to imply vanishing gradients during training), in which case the network will collapse; or to unbounded kernels (and increasingly large gradients), in which case the network will be chaotic and again untrainable. We'll revisit this in Section 3.7. With careful tuning, the issue can be

avoided. For example, in a deep linear network (DLN) with $\phi(z) = z$, one can just take $\sigma_w = 1$, known as LeCun initialization (LeCun et al. 1998). In a ReLU network, the correct choice instead is $\sigma_w = 2$, known as He initialization (He et al. 2015).

Even with σ_w tuned, according to the activation function, so that on average the GP kernels remain order-one from layer to layer, non-Gaussianities can still accumulate. The upshot is that it is not width alone, say of typical size N, but rather the ratio L/N of depth-to-width that controls the effective behavior of a network — including its departure from pure Gaussianity at large N.

This behavior is analyzed in Chapters 3-4 of (Roberts et al. 2022) using functional-integral methods. In particular, Chapter 3 considers the simple example of DLN's, and produces exact recursion relations for higher cumulants of preactivations in each layer. It is shown that in a network of constant hidden width N, the leading non-Gaussianity is

$$\begin{aligned}\kappa_4^{(L+1)} &= \left[\left(1+\frac{2}{N}\right)^L - 1\right](\kappa_2^{(L+1)})^2 \\ &= \left[(e^{2L/N}-1) - \frac{1}{N}\frac{2L}{N}e^{2L/N} + O(N^{-2})\right](\kappa_2^{(L+1)})^2\,,\end{aligned}$$

where the expansion on the RHS is at finite L/N and $L, N \to \infty$. For small L/N, the non-Gaussianity remains small but finite, of size L/N. For large L/N, $\kappa_4^{(L+1)}$ blows up, as do all the other cumulants. The network outputs will fluctuate chaotically from one instantiation to another, and the network again becomes untrainable.

Here's a more direct analysis, in a special case, following (Hanin 2026, Lecture 2). Consider a DLN of length L with input and hidden layers all of width N:

$$\begin{aligned}z^{(\ell)} &= \frac{1}{\sqrt{N}}\, w^{(\ell)} z^{(\ell-1)}\,, \qquad z^{(0)} = x\,,\\ f(x) &= z^{(L+1)} = N^{-\frac{L+1}{2}}\, w^{(L+1)}\cdots w^{(1)} x\,,\end{aligned}$$

with $w_{ij}^{(\ell)} \sim \mathcal{N}(0,1)$, i.e. $\sigma_w = 1$ (and input dimension $d = N$). Conditional on $z^{(\ell-1)}$, the vector $z^{(\ell)}$ is Gaussian $z^{(\ell)} \sim \mathcal{N}\big(0, \frac{1}{N} I_N |z^{(\ell-1)}|^2\big)$, so its norm satisfies the recursion

$$|z^{(\ell)}|^2 \sim |z^{(\ell-1)}|^2 \frac{\chi_N^2}{N}\,.$$

(Recall that χ_N^2 is the distribution of the sum of squares of N i.i.d. standard Gaussian variables.) Let's normalize the input so that $|x| = 1$. Then the recursion implies that

$$|f(x)|^2 \overset{d}{=} \prod_{k=1}^{L+1} \frac{\chi_N^2}{N} = \exp \sum_{k=1}^{L+1} \big(\log \chi_N^2 - \log N\big)\,,$$

with each factor of χ_N^2 independent. At large N, we have $\log \chi_N^2 - \log N = \mathcal{N}(-\frac{1}{N}, \frac{2}{N}) + O(\frac{1}{N^2})$, so at large N, L with L/N fixed

$$|f(x)|^2 \overset{d}{=} e^{\mathcal{N}(-\frac{L}{N}, 2\frac{L}{N})}\,,$$

up to higher $O(\frac{1}{N^2}$ corrections. Thus, as L/N grows, the expectation of $\mathbb{E}|f(x)| \to 0$ but the variance and higher cumulants blow up.

The lesson once again is

- $N \to \infty$, $L/N = 0$: Gaussian process, no correlations across layers (and, as we'll see in Section 4, no "feature learning").
- $N, L \to \infty$, L/N small: controlled correlations, trainable network (and, as we'll see, feature learning during training). In open-weight LLM's today, $L/N \approx 0.005$–0.01; see Exercise 3.3.
- $N, L \to \infty$, L/N large: chaotic and untrainable.

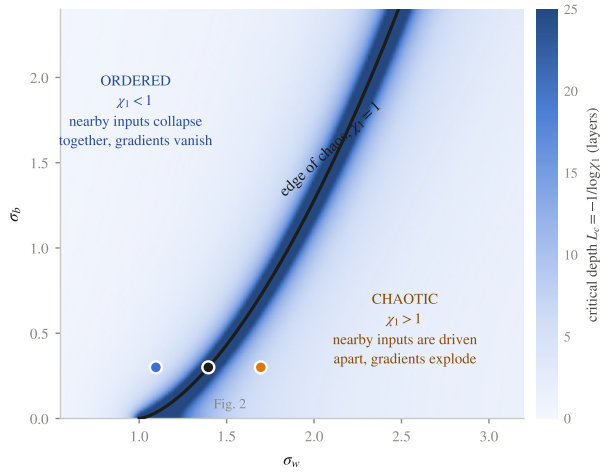
Exercise 3.3 (★) (Depth-to-width ratios in the wild). Look up the depth L (number of transformer blocks) and the width N (model/embedding dimension) for a few families of open-weight language models — *e.g.* GPT-2, the Llama family, the Qwen family — and compute L/N for each. How much does the ratio vary across families, across sizes within a family, and across the years? Is everyone training in the "controlled correlations" window?

SOLUTION, p. 614

3.7 Edge of chaos

We briefly remark that the analysis of how to tune initialization variances σ_w, σ_b (etc.) leading to stable network behavior over a large number of layers (the first point in Section 3.6) is sometimes called finding the "edge of chaos."

The term dates back to dynamical systems theory from the 80's and 90's, referring to the boundary between ordered and chaotic phases. It was adapted to neural networks around 2016 (Poole et al. 2016; Schoenholz et al. 2017). The idea was to treat the progression through layers of a network as a dynamical system, and to ask: given two nearby inputs x, x', for what range of hyperparameters (σ_w, σ_b, etc.) and to what depth do they remain nearby, without either a) collapsing; or (b) blowing up chaotically (with no correlation). The analysis roughly amounts to making sure the kernel recursion of (3.26), (3.30) leads to kernels that neither collapse nor blow up. Doing it carefully, theoretically, can produce elementary trainable networks (with no layer norms!) over 10,000 layers deep (Xiao et al. 2018).



Order/chaos phase diagram for a tanh network, with shading indicating the critical depth to which a network is trainable.

For MLP's with ReLu activations, the edge of chaos is just the one point $\sigma_w = 2$, $\sigma_b = 0$. Moving slightly away from this point, one can estimate the "critical depth" L_c at which the network either collapses or blows up – see (Schoenholz et al. 2017). For MLP's with other activation functions, the edge of chaos is more interesting. For $\phi =$ tanh, the phase diagram is shown in Figure 5.

The quantity `χ_1' that controls the critical depth ($L_c = -1/\log \chi_1$) is the derivative of the layer-to-layer kernel recursion at its fixed point, which measures how fast nearby inputs separate. Importantly, the same quantity χ_1 also controls the size of gradients as they back-prop through a network *during training*. There's a direct link between behavior at initialization and behavior during training. We come to training next.

4. The NTK and large width during training

In this section, we consider training networks that are randomly initialized as in Section 3.2, and their behavior at large width. The lesson will be that as width$\to \infty$ these networks effectively behave like linear models: their parameters θ never stray far from their values θ_0 at initialization, hence

$$f_\theta(x) \approx f_\theta^{\text{lin}}(x) := f_{\theta_0}(x) + (\theta - \theta_0) \cdot \nabla_\theta f_{\theta_0}(x)\,.$$

Under suitable conditions, the network itself also converges exponentially fast to a "memorizing network" that fits the training data perfectly. Heuristically, this happens because small changes in many parameters, as in (4.1), can add up to an order-one change in the network values themselves. Nevertheless, the network never experiences phase transitions or feature learning — we'll make this more precise below. The large-width regime (at finite depth and training data) is thus known as the *lazy* regime, or the NTK regime. (For a self-contained review of models that are linear in their parameters, and of why training them amounts to kernel ridge regression, see Appendix B.)

Although this regime does not describe the feature learning that makes practical networks interesting, it is important to understand for at least two reasons: it is an honest, analytically solvable model of training; and it is a precise lesson in what must be *avoided* if one wants genuine feature learning — a lesson we act on in Section 5.

We also emphasize the connection between Section 3 and the current analysis. Both involve the *same* large-width limit. At initialization, it leads to a Gaussian process. During training, we'll see that it leads to a linear model with lazy learning, converging once again to a GP at late training times.

4.1 Gradient flow and the NTK

Throughout this section, we approximate SGD with continuous-time *gradient flow* (*cf.* Section 2.1). This involves two approximations. First we update on the entire training dataset $\{(x^\mu, y^\mu)\}_{\mu=1}^n$ at each step, rather than on batches, so that SGD becomes GD:

$$\theta_{k+1} - \theta_k = -\eta_{GD}\, \nabla_\theta \mathcal{L}_\theta(\{x^\mu, y^\mu\})\big|_{\theta=\theta_k}$$

where

$$\mathcal{L}_\theta = \mathcal{L}_\theta(\{x^\mu, y^\mu\}) := \sum_{\mu=1}^n \ell_\theta(x^\mu, y^\mu)$$

is the full empirical loss.

Second, we scale down the learning rate $\eta_{GD} = \lambda^{-1}\eta$ and scale up the number of training steps, setting $k = \lfloor \lambda t \rfloor$ where $t \geq 0$ is a continuous "time" variable. As $\lambda \to \infty$, the gradient descent converges to a gradient flow

$$\dot\theta := \frac{d}{dt}\theta = -\eta\, \nabla_\theta \mathcal{L}_\theta\,.$$

All theorems in this section can be extended to SGD, adding additional error terms to bound the difference between SGD and GF. (See for example (Spiliopoulos et al. 2025, Chapter 19).)

4.1.1 Geometry of gradient flows

Suppose that X is a smooth Riemannian manifold, with nondegenerate metric $g = g_{ab}dx^a dx^b$, inverse metric $g^{-1} = g^{ab}\frac{\partial}{\partial x^a}\frac{\partial}{\partial x^b}$, and a function $h : X \to \mathbb{R}$. Then one can define gradient flow with respect to (h, g) as

$$\dot x^a(t) = -\eta\, g^{ab}\frac{\partial h}{\partial x^b}\,, \qquad \text{or abstractly:} \quad \dot x = -\eta\, g^{-1}(dh)\,.$$

Note that this equation does not make sense (it's not coordinate-free) without an inverse-metric (a bilinear form on each cotangent space T^*X). The inverse-metric is used to relate the differential $dh = (\partial_b h)_{b=1}^{\dim X}$ on the RHS to a *vector field* $g^{-1}dh$, which is the right sort of geometric object to induce a flow on X.

In neural networks, $X = \mathbb{R}^P$ is the space of parameters θ and $h = \mathcal{L}$ is the loss function. One always assumes a constant Euclidean metric in parameter space $g = \sum_{i=1}^{P} (d\theta^i)^2 = \delta_{ij} d\theta^i d\theta^j$, in order to define SGD/GD/GF/etc. This is the source of some of the simplest inductive biases of trained models, and also why one expects "features" to have some linear structure. Eqn. (4.5) with the metric looks like

$$\dot{\theta}^i = -\eta\, \delta^{ij}\, \partial_j \mathcal{L}_\theta \,.$$

Now return to the general setting of Riemannian manifolds. Suppose we have a smooth map $f : X \to Y$ between two manifolds, and moreover that $h : X \to \mathbb{R}$ is the pull-back of a function $h^Y : Y \to \mathbb{R}$, meaning $h(x) = h^Y(f(x))$. Let $\{f^\alpha\}_{\alpha=1}^{\dim Y}$ be the components of the map f, in local coordinates y^α on Y. Then the inverse-metric on X induces an inverse-metric on Y (called its push-forward):

$$f_*(g^{-1}) = f_*\left[g^{ab} \frac{\partial}{\partial x^a} \frac{\partial}{\partial x^b}\right] = \underbrace{g^{ab} \frac{\partial f^\alpha}{\partial x^a} \frac{\partial f^\beta}{\partial x^b}}_{g^{\alpha\beta}} \frac{\partial}{\partial y^\alpha} \frac{\partial}{\partial y^\beta} \,.$$

Schematically, $g^{\alpha\beta} = (J g^{-1} J^T)$, where $J^\alpha{}_a = \frac{\partial f^\alpha}{\partial x^a}$ is the Jacobian matrix of f. Then the gradient flow on X induces a gradient flow on Y, given by

$$\dot{y}^\alpha = -\eta\, g^{\alpha\beta} \frac{\partial h^Y}{\partial y^\beta} \,.$$

Let's apply this to neural networks, where $f : \mathbb{R}^P \to \mathbb{R}^n$ is the function from parameters to network outputs on the training data (x^μ, y^μ), *i.e.* $f^\mu(\theta) = f_\theta(x^\mu)$ in our standard notation. The loss function is indeed a pullback of a function on $\mathbb{R}^n$, since it depends on parameters θ only through the network values on training data,

$$\mathcal{L}_\theta = \mathcal{L}^{(f)}(f(\theta)) \,, \qquad \mathcal{L}^{(f)}(f) := \sum_{\mu=1}^{n} \ell(f^\mu, y^\mu) \,.$$

The push-forward of the gradient flow in parameter space becomes

$$\frac{d}{dt} f^\mu = -\eta\, \Theta^{\mu\nu} \frac{\partial \mathcal{L}^{(f)}}{\partial f^\nu} \,, \qquad \Theta^{\mu\nu} = (J \delta^{-1} J^T)^{\mu\nu} = \sum_{i=1}^{P} \frac{\partial f_\theta(x^\mu)}{\partial \theta^i} \frac{\partial f_\theta(x^\nu)}{\partial \theta^i} \,.$$

The inverse-metric $\Theta^{\mu\nu}$ on the space of function values is called the "neural tangent kernel" or NTK. The name was coined in (Jacot et al. 2018); it also appeared in (Du et al. 2019) under a different name. Here we are emphasizing that it's a completely natural geometric object, for any neural network (in fact, for any ML model).

We mention one final bit of geometry. Let $\delta_{\mu\nu}$ be a standard Euclidean metric in the space of function values, with inverse $\delta^{\mu\nu}$. The gradient of the loss $\delta^{\mu\nu} \partial \mathcal{L}^{(f)} / \partial f^\nu$ that appears in (4.10) (contracted with an inverse metric) is sometimes called the *residual*, and denoted Δ^μ or r^μ. For MSE loss, we've got $\mathcal{L}^{(f)} = \frac{1}{2} \sum_{\mu=1}^{n} (f^\mu - y^\mu)^2 = \frac{1}{2} \delta_{\mu\nu} (f^\mu - y^\mu)(f^\nu - y^\nu)$. Then the residual is just the difference between function values and truth for the training data:

$$\Delta^\mu := \delta^{\mu\nu}\partial\mathcal{L}^{(f)}/\partial f^\nu = f^\mu - y^\mu\,.$$

Noting that y^μ are constants, the gradient-flow equation can be recast as

$$\frac{d}{dt}\Delta^\mu = -\eta\,\underbrace{\Theta^{\mu\rho}\delta_{\rho\nu}}_{\Theta^\mu{}_\nu}\Delta^\nu\,.$$

Those who have studied gauge theory may recognize this as an equation for *parallel transport*, with respect to an $n\times n$ matrix-valued connection $\eta\,\Theta^\mu{}_\nu$. If one knows the dependence of Θ on time (which in general is far from trivial — see the exercise below) the solution is a time-ordered exponential

$$\Delta^\mu(t) = \left(P\exp\left[-\eta\int\Theta(t)\,dt\right]\right)^\mu_\nu\Delta^\nu(0)\,.$$

Exercise 4.1 (The NTK from the chain rule). Derive the flow equation (4.10) for the function values f^μ directly, using only the chain rule — no geometry required. (The geometric derivation explains *why* an object like $\Theta^{\mu\nu}$ had to appear; the chain rule shows it appears in three lines.)

SOLUTION, p. 614

Exercise 4.2 (How fast does the kernel move?). Show that the time derivative of the NTK satisfies

$$\frac{d}{dt}\Theta^{\mu\nu} = -\eta\left[\nabla_\theta f^\mu\cdot\nabla^2_\theta f^\nu\cdot\nabla_\theta f^\rho\right]\frac{\partial\mathcal{L}^{(f)}}{\partial f^\rho} + (\mu\leftrightarrow\nu)\,,$$

where $\nabla^2_\theta f^\nu = \nabla^2_\theta f_\theta(x^\nu)$ is the Hessian. Bounding the Hessian ultimately leads to the constant-NTK theorem in Section 4.3 below.

SOLUTION, p. 615

4.2 Features

The Jacobian matrix $J^\mu{}_i = \partial f^\mu/\partial\theta^i$ measures how function values on the training data depend on small variations of the network weights. Assuming there are more parameters than training data, its singular value decomposition looks like

$$J = USV^T\,,\qquad S = \Big[\mathrm{diag}(s_1,\ldots,s_n)\;\;\mathbf{0}_{n\times(P-n)}\Big]\,.$$

The right singular vectors (columns of V) define natural modes in parameter space, while the left singular vectors (columns of U) are the natural modes in function-value space — where "natural" means with respect to how network outputs depend on weights. The n left singular vectors (together with their singular values s_i) can be thought of, abstractly, as the *features* that a network has learned. A common definition of feature *learning* is that these singular vectors+values change during training.

The analysis of features can equivalently be performed via the NTK. As a matrix, we've got

$$\Theta = JJ^T = US^2U^T\,,$$

with eigenvectors = the feature vectors, and eigenvalues = $\{s_i^2\}$. Thus feature learning happens when the NTK changes during training.

Remark. A closely related "dual" object that appears in other parts of learning theory is the Fisher information metric on parameter space $\mathbb{R}^P$. Its empirical version is

$$\mathcal{F} := J^TJ = f^*(\delta)\,,$$

the pull-back (via the network function) of constant Euclidean metric on the space of function values on the training data. In coordinates, $\mathcal{F}_{ij} = \frac{\partial f^\mu}{\partial\theta^i}\delta_{\mu\nu}\frac{\partial f^\mu}{\partial\theta^j}$.

The Fisher metric has eigenvalues $(s_1^2, ..., s_n^2, \overbrace{0, ..., 0}^{P-n})$, and is automatically degenerate when $P > n$ (more parameters than training data points). This is one of the starting points of singular learning theory.

4.3 Constant NTK at large width

Now we get to the main theoretical result and simplification of this section. Suppose we've got an MLP of fixed depth L, and fixed training data size n, and take the width of all hidden layers to infinity. We'll assume the hidden widths all scale in a similar way $N_L, N_{L-1}, ..., N_1 \approx N \to \infty$, and we initialize the network the standard way from Section 3.2. We want to show that for almost all samples of training data,

- The NTK $\Theta^{\mu\nu}$ remains constant throughout training, up to corrections that vanish as $N \to \infty$.
- The network itself linearizes: at any training time t, $f_{\theta(t)}(x) = f_{\theta_0}(x) + (\theta(t) - \theta_0)\cdot\nabla_\theta f_{\theta_0}(x)$

A rigorous derivation is somewhat technical, and we refer the reader to (Du et al. 2019; Jacot et al. 2018), (Roberts et al. 2022, Ch. 11), (Spiliopoulos et al. 2025, Ch. 20) for complete arguments, explained from a variety of different perspectives. We'll just sketch the key ideas behind the result. See also (Hanin 2026, Lecture 3).

We'll simplify life by setting biases to zero, so the network becomes

$$f_\theta(x) = z^{(L+1)}(x)\,, \quad z_i^{(\ell)} = \frac{1}{\sqrt{N_{\ell-1}}}\sum_j w_{ij}^{(\ell)}\phi(z_j^{(\ell-1)})\,, \quad w_{ij}^{(\ell)} \sim \mathcal{N}(0, \sigma_w)\,,$$

in the fan-in normalization of Section 3.2, with order-one weights.

Size of the NTK. The first task is to estimate how large the NTK is at large width, because that decides how the learning rate must scale. By the chain rule, the gradient with respect to the layer-ℓ weights is an outer product,

$$\frac{\partial f}{\partial w_{ij}^{(\ell)}} = \frac{1}{\sqrt{N_{\ell-1}}}\,\beta_i^{(\ell)}\,\phi(z_j^{(\ell-1)})\,, \qquad \beta_i^{(\ell)} := \frac{\partial f}{\partial z_i^{(\ell)}}\,,$$

where the backpropagated sensitivities $\beta^{(\ell)}$ obey $\beta^{(L+1)} = 1$ and

$$\beta_i^{(\ell)} = \phi'(z_i^{(\ell)})\,\frac{1}{\sqrt{N_\ell}}\sum_k w_{ki}^{(\ell+1)}\,\beta_k^{(\ell+1)}\,.$$

The NTK is a sum over layers,

$$\begin{aligned}\Theta^{\mu\nu} &= \sum_{\ell=1}^{L+1}\sum_{i,j}\frac{\partial f^\mu}{\partial w_{ij}^{(\ell)}}\frac{\partial f^\nu}{\partial w_{ij}^{(\ell)}} \\ &= \sum_{\ell=1}^{L+1}\underbrace{\Big[\sum_i \beta_i^{(\ell)}(x^\mu)\,\beta_i^{(\ell)}(x^\nu)\Big]}_{B_{\mu\nu}^{(\ell)}}\;\underbrace{\Big[\frac{1}{N_{\ell-1}}\sum_j \phi(z_j^{(\ell-1)}(x^\mu))\,\phi(z_j^{(\ell-1)}(x^\nu))\Big]}_{\to\,\bar K_{\mu\nu}^{(\ell)}/\sigma_w}\,.\end{aligned}$$

The second bracket is an average of $N_{\ell-1}$ terms. At initialization and large width, the law of large numbers replaces it by $\bar K_{\mu\nu}^{(\ell)}/\sigma_w$, the Gaussian-process kernel of (3.30) evaluated on the two inputs, which is of order one. (For $\ell = 1$ the bracket is exactly $x^\mu\cdot x^\nu/d$, which is $\bar K_{\mu\nu}^{(1)}/\sigma_w$.) The first bracket is handled by the same recursion run backwards. Define

$$\dot K_{\mu\nu}^{(\ell)} = \dot K^{(\ell)}(x^\mu,x^\nu) := \mathbb{E}\big[\phi'(z_i^{(\ell-1)}(x^\mu))\,\phi'(z_i^{(\ell-1)}(x^\nu))\big] \qquad (\text{for any fixed } i)\,,$$

which is of order one for reasonable activation functions. Using the independence at initialization of the weights $w^{(\ell+1)}$ from everything below them, one finds $B_{\mu\nu}^{(\ell)} \approx \sigma_w\,\dot K_{\mu\nu}^{(\ell+1)}\,B_{\mu\nu}^{(\ell+1)}$ with $B_{\mu\nu}^{(L+1)} = 1$, and hence

$$\mathbb{E}\big[\Theta^{\mu\nu}\big] \;\approx\; \sum_{\ell=1}^{L+1}\sigma_w^{L-\ell}\;\dot K_{\mu\nu}^{(L+1)}\cdots\dot K_{\mu\nu}^{(\ell+1)}\;\bar K_{\mu\nu}^{(\ell)}\,.$$

This is a sum of $L+1$ terms, each of order one, with no factors of N anywhere: *in the fan-in normalization the NTK is of order one at large width*, and every layer contributes to it at the same order. (Equation (4.23) is the standard recursion for the infinite-width NTK (Jacot et al. 2018).) By (4.10), an order-one learning rate η then moves the function values f^μ by an order-one amount per unit time, which is what we want, and no rescaling of η with width is needed in this convention. For the one-hidden-layer network $f = \frac{1}{\sqrt N}\sum_i c_i\phi(a_i\cdot x)$ one can check this by hand:

$$\Theta^{\mu\nu} = \frac{1}{N}\sum_{i=1}^{N}\Big[\phi(a_i\cdot x^\mu)\,\phi(a_i\cdot x^\nu) + c_i^2\,\phi'(a_i\cdot x^\mu)\,\phi'(a_i\cdot x^\nu)\;x^\mu\cdot x^\nu\Big] = O(1)\,,$$

an average of N order-one terms. (Here and below we absorb the first-layer factor $1/\sqrt d$ into the inputs x, since d is fixed.)

How to scale the learning rate. The rule just used is general, and we will use it again in Section 5. What must stay of order one as $N \to \infty$ is the *motion of the network values*, $\frac{d}{dt} f^\mu = -\eta\, \Theta^{\mu\nu}\, \partial \mathcal{L}^{(f)} / \partial f^\nu$, since those are what fit the data. So: work out how the NTK scales with N, and choose η so that $\eta\,\Theta = O(1)$. In the fan-in normalization $\Theta = O(1)$ and $\eta = O(1)$. In the mean-field normalization of Section 5, $\Theta = O(1/N)$ and η must grow like N.

Other conventions for the parameters are changes of variables, and they can be compared through the same rule. Suppose one puts no prefactors in the network and initializes $\tilde{w}^{(\ell)} = w^{(\ell)}/\sqrt{N_{\ell-1}} \sim \mathcal{N}(0, \sigma_w/N_{\ell-1})$ as in (3.5). Gradient flow with respect to the Euclidean metric $\sum (d\tilde{w})^2$ is *not* the same flow as gradient flow with respect to $\sum (dw)^2$. Under the change of variables, the inverse metric δ^{ij} of Section 4.1.1 picks up a factor $N_{\ell-1}$ in every layer-ℓ block, so that in the original coordinates the flow reads $\dot{w}^{(\ell)} = -\eta\, N_{\ell-1}\, \partial \mathcal{L}/\partial w^{(\ell)}$. The NTK computed in the $\tilde{w}$ coordinates is correspondingly larger, $\tilde{\Theta} = \sum_\ell N_{\ell-1}\, \Theta_{(\ell)}$ in terms of the layer contributions of (4.21), and an order-one motion of f then requires $\tilde{\eta} \sim 1/N$. A rescaling of the parameters is thus equivalent to a layer-dependent rescaling of the learning rate, and two conventions define the same training dynamics only if their learning rates are matched layer by layer. With a *single* learning rate $\tilde{\eta} \sim 1/N$ in the $\tilde{w}$ convention, for instance, the hidden layers move as in the fan-in normalization, but the first layer, whose fan-in d does not grow with N, is effectively frozen. When comparing learning rates across papers, first translate the parametrizations into one another; for a systematic treatment, see (Yang & Hu 2021).

Caution: the estimate (4.23) is at fixed depth L and fixed data size n. The NTK grows like L with depth, and the operator norm of the $n \times n$ matrix Θ grows with n, so for deep networks or large datasets the learning rate must shrink accordingly, and the constant-NTK statements below need n and L to stay small compared with N.

The Hessian is small. To see that the NTK is approximately constant during training, we consider its time derivative, which by Exercise 4.2 is bounded by the operator norm of the Hessian of the network function,

$$\left\| \frac{d}{dt} \Theta \right\| \lesssim \eta\, \|\nabla_\theta^2 f\| \cdot \|\Theta\| \cdot |\Delta| \,.$$

The Hessian is a very large but very sparse matrix, and in the fan-in normalization its operator norm is small. We'll sketch how this works for one hidden layer ($L = 1$) with one-dimensional output, following (Hanin 2026, Lecture 3). Writing

$$f_\theta(x) = \frac{1}{\sqrt{N}} \sum_{i=1}^{N} c_i\, \phi(a_i \cdot x) \,, \qquad c_i \in \mathbb{R} \,, \quad a_i \in \mathbb{R}^d \,,$$

the Hessian splits into blocks, one per hidden neuron,

$$\nabla_\theta^2 f_\theta(x) = \bigoplus_{i=1}^{N} (\nabla_\theta^2 f)_i \,, \qquad (\nabla_\theta^2 f)_i = \frac{1}{\sqrt{N}} \begin{pmatrix} 0 & \phi'(a_i \cdot x)\, x^T \\ \phi'(a_i \cdot x)\, x & c_i\, \phi''(a_i \cdot x)\, x x^T \end{pmatrix} .$$

The operator norm of a block-diagonal matrix is the largest operator norm of its blocks. Each block has fixed size $(d+1)\times(d+1)$, contains only order-one quantities (at least at initialization), and carries an explicit factor $1/\sqrt{N}$. Thus

$$\|\nabla_\theta^2 f\| = \max_{i=1,\dots,N} \|(\nabla_\theta^2 f)_i\| = O\Big(\frac{1}{\sqrt{N}}\Big) \qquad \text{as } N\to\infty\,.$$

It's reasonable to expect that this bound continues to hold during training, and with a bit more work it can be proven (though we won't do it here). It also holds for networks of arbitrary fixed depth. A more careful analysis, keeping track of cancellations among the fluctuating terms, improves the resulting bound on the kernel drift to $O(1/N)$; the crude $O(1/\sqrt{N})$ is enough for the argument.

Putting the pieces together. First, the parameters do not move far in total. From $\dot\theta = -\eta\sum_\mu \Delta^\mu \nabla_\theta f^\mu$,

$$|\dot\theta|^2 = \eta^2 \Big|\sum_{\mu=1}^{n} \Delta^\mu \nabla_\theta f^\mu\Big|^2 \le \eta^2\,|\Delta|^2\,|\nabla_\theta f|^2 \le \eta^2\, n\,|\Delta|^2\,\|\Theta\| = O(1)\,,$$

where $|\nabla_\theta f|^2 = \sum_\mu \Theta^{\mu\mu} = \operatorname{tr}\Theta$. So by the mean value theorem, $\theta(t)-\theta_0 = t\,\dot\theta(\bar t)$ for some $\bar t\in[0,t]$, and

$$|\theta(t)-\theta_0| = O(1) \qquad \text{for any fixed } t\,.$$

An order-one total displacement, shared among $P\approx Nd$ parameters, means that each individual weight moves only by $O(1/\sqrt{N})$: relative to its order-one initial value, every weight barely changes. Second, the kernel freezes. By (4.25) and (4.28),

$$\big\|\Theta(t)-\Theta(0)\big\| \le t\,\|\dot\Theta(\bar t)\| \lesssim t\,\eta\,\|\nabla_\theta^2 f\|\,\|\Theta\|\,|\Delta| = O\Big(\frac{1}{\sqrt{N}}\Big)\,,$$

so the order-one kernel Θ becomes constant as $N\to\infty$. Third, the network linearizes. By Taylor's theorem with remainder,

$$f_\theta(x) - f_{\theta_0}(x) = \nabla_\theta f(x)\big|_{\theta_0}\cdot(\theta-\theta_0) + \frac{1}{2}(\theta-\theta_0)\cdot\nabla_\theta^2 f(x)\big|_{\bar\theta}\cdot(\theta-\theta_0)$$

for some $\bar\theta$ on the segment between θ_0 and θ. Setting $\theta=\theta(t)$ and using (4.30) and (4.28), the quadratic term is bounded by

$$\Big|\frac{1}{2}(\theta-\theta_0)\cdot\nabla_\theta^2 f(x)\big|_{\bar\theta}\cdot(\theta-\theta_0)\Big| \le \frac{1}{2}\|\nabla_\theta^2 f\|\,|\theta-\theta_0|^2 = O\Big(\frac{1}{\sqrt{N}}\Big)\cdot O(1)\,,$$

so the linear approximation becomes exact as $N\to\infty$, even though the linear term, and with it the network function, moves by an order-one amount. In words: each of the N neurons has $O(1/\sqrt{N})$ leverage on the output, so an order-one change in f can be assembled from $O(1/\sqrt{N})$ changes in every weight, and changes that small leave the features $\nabla_\theta f$ untouched.

4.4 Lazy learning

When the NTK is constant during training — confusingly often called the *NTK regime* — a network undergoes what is known as "lazy learning." It is not learning features, since the eigenvalues s_μ^2 and eigenvectors u_μ of the NTK are (by construction) constant. Nevertheless, the network function converges to its training data exponentially fast: from (4.12) with constant Θ, we have

$$\Delta^\mu(t) = (e^{-\eta\, t\, \Theta})^\mu{}_\nu \Delta^\nu(0)\,,$$

where we recall that $\Delta^\mu(t) = f^\mu - y^\mu = f_{\theta(t)}(x^\mu) - y^\mu$. Writing the residuals in an eigenbasis for the NTK, $\Delta = \hat{\Delta}^\mu u_\mu$ we see that

$$\hat{\Delta}^\mu(t) = e^{-\eta\, t\, s_\mu^2} \hat{\Delta}^\mu(0)\,.$$

Each feature is learned at a characteristic timescale $t_\mu \approx 1/(\eta\, s_\mu^2)$.

Exercise 4.3 (Lazy predictions on test points). Derive that on an arbitrary test point x, not necessarily part of the training data,

$$f_{\theta(t)}(x) = f_{\theta_0}(x) - \Theta(x, x^\mu)\Theta^{-1}_{\mu\nu}(\mathbb{1} - e^{-\eta\, t\, \Theta})^\nu{}_\rho \Delta^\rho(0)$$

where $\Theta(x, x^\mu) = \nabla_\theta f(x) \cdot \nabla_\theta f(x^\mu)$ is the NTK for a test-train pair (which remains constant during training, just like the train-train NTK $\Theta^{\mu\nu}$).

SOLUTION, p. 615

From Exercise 4.3, we see that at the end of training $t \to \infty$ the network obeys

$$f_{\theta_\infty}(x) = f_{\theta_0}(x) - \Theta(x, x^\mu)\Theta^{-1}_{\mu\nu}(f_{\theta_0}(x^\nu) - y^\nu)\,.$$

This is an affine-linear transformation of the Gaussian process $f_{\theta_0}(x)$ at initialization, so it is again a Gaussian process. It's straightforward to check that the new mean and covariance are

$$\begin{aligned}
\mathbb{E}\big[f_{\theta_\infty}(x)\big] &= \Theta^{x\mu}\Theta^{-1}_{\mu\nu} y^\nu \\
\mathrm{Cov}\big[f_{\theta_\infty}(x), f_{\theta_\infty}(x')\big] &= K^{xx'} - 2\Theta^{x\mu}\Theta^{-1}_{\mu\nu}K^{\nu x'} + \Theta^{x\mu}\Theta^{-1}_{\mu\nu}K^{\nu\rho}\Theta^{-1}_{\rho\sigma}\Theta^{\sigma x'}\,,
\end{aligned}$$

where $K^{xx'} = K(x, x')$, $\Theta^{x\mu} = \Theta(x, x^\mu)$, etc.

4.5 Summary

So far we've seen that in the fan-in normalization (Section 3.2), with order-one weights and an order-one learning rate η, at large width N we've got

$$\|\Theta\| \sim |\nabla_\theta f|^2 \sim 1\,, \qquad \|\nabla^2_\theta f\| \sim \frac{1}{\sqrt{N}}\,, \qquad |f_\theta - f_{\theta_0}| \sim 1\,,$$
$$|\theta - \theta_0| \sim 1 \;\; (\text{i.e. } N^{-1/2} \text{ per weight})\,,$$

which forces Θ to be constant and the network to linearize as a function of its parameters.

All this breaks down if

- Depth L is large as well, with L/N nonzero. (See (Roberts et al. 2022, Ch. 11).)
- Data samples n become large, with n/N nonzero.
- Learning rate becomes large.

Both L and n large are interesting cases for modern neural networks!

Exercise 4.4 (★) (Numerics: the lazy network (lab)). This lab, and its continuation in Exercise 5.3, trains one small network in both the fan-in and the mean-field normalization and tests the claims of Sections 4 and 5 numerically. The setup runs in seconds, in about sixty lines of NumPy; reproduce it, then modify it.

The starter model. Inputs are one-dimensional: $n = 30$ equispaced training points $x^\mu \in [-2, 2]$, with targets from a two-neuron teacher, $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$. The student is a one-hidden-layer tanh network

$$f(x) = \frac{1}{N^\gamma}\sum_{i=1}^{N} c_i \tanh(a_i x)\,, \qquad c_i, a_i \sim \mathcal{N}(0,1)\,, \qquad N = 300\,,$$

trained by full-batch gradient descent on $\mathcal{L} = \frac{1}{2}\sum_\mu (f(x^\mu) - y^\mu)^2$, with residuals $\Delta^\mu = f(x^\mu) - y^\mu$. Here take the fan-in exponent $\gamma = \frac{1}{2}$, learning rate $\eta = 0.02$, and 4×10^4 steps. The empirical NTK to monitor is the $n \times n$ matrix

$$\Theta^{\mu\nu} = \frac{1}{N^{2\gamma}}\sum_{i=1}^{N}\Big[\tanh(a_i x^\mu)\tanh(a_i x^\nu) + c_i^2\big(1-\tanh^2(a_i x^\mu)\big)\big(1-\tanh^2(a_i x^\nu)\big)\,x^\mu x^\nu\Big]\,.$$

(a) *It fits.* Train and check that the loss becomes small. (The network is overparametrized; fitting is not the interesting part.)

(b) *Initialization.* Over about 200 seeds, histogram $f_0(x)$ at a fixed input, say $x = 1$. You should see an order-one Gaussian, the Gaussian process of Section 3.4, and the sample covariance across two inputs should match the tanh GP kernel.

(c) *The kernel freezes.* For $N \in \{30, 100, 300, 1000\}$ (a few seeds each), record $\|\Theta_{\text{end}} - \Theta_0\|/\|\Theta_0\|$ and plot it against N on log–log axes. Expected: it decays, with slope between $-\frac{1}{2}$ (the guaranteed bound (4.31)) and -1 (the typical measurement). Also record the relative parameter displacement $|\theta_{\text{end}} - \theta_0|/|\theta_0|$ and compare with (4.30).

(d) *Mode by mode.* Diagonalize Θ_0 and plot $|(U^T\Delta)_k(t)|$ on a semilog axis for a few k: straight lines with slopes $-\eta s_k^2$, the features being learned at $t_k = 1/(\eta s_k^2)$, largest eigenvalue first, as in (4.35). Compare with the Bayesian version, Exercise 4.5(b).

(e) *Prediction.* Compare the trained function on fresh test points with the frozen-kernel prediction (4.36).

SOLUTION, p. 615

4.6 Bayesian learning at large width: kernel learning

So far, we have considered MLP's at large width, trained via gradient flow (as an approximation to SGD/GD). We found that their NTK remains constant and that their late-time distribution is a Gaussian process given by (4.38).

Suppose that we consider the same large-width network, but "train" Bayesian inference instead, on the empirical MSE loss $\mathcal{L}(\theta) = nL_n(\theta) = \frac{1}{2}\sum_{\mu=1}^{n} |f(x^\mu) - y^\mu|^2$. (Recall from Section 2.1 that, practically speaking, this means training via Langevin dynamics.) Then we expect a Bayesian posterior

$$P(\theta) = \frac{1}{Z} e^{-\beta \mathcal{L}(\theta)}\, \pi(\theta)\,, \qquad \pi(\theta) \propto e^{-\frac{1}{2\sigma_w}|\theta|^2}\,,$$

where the prior $\pi(\theta)$ is the Gaussian of order-one variance from (3.1), and the network carries the fan-in normalization of Section 3.2.

We'd like to push the distribution forward to the network function $f_\theta(x)$. At large-width, the push-forward of the prior is our familiar Gaussian process from Section 3.4; expressed via a functional integral as in Section 3.5 it becomes

$$\pi[f] = \frac{1}{Z'} e^{-\frac{1}{2}\int dx\, dx'\, f(x) K^{-1}(x,x') f(x')}$$

with $K(x,x') = \bar{K}^{(L+1)}(x,x')$ the final-layer GP kernel. Since the loss is explicitly a function of network values, we immediately get the late-time trained distribution

$$P[f] = \frac{1}{Z''}\, e^{-\frac{1}{2}\left[\beta \sum_{\mu=1}^{n} (f(x^\mu) - y^\mu)^2 + \int dx\, dx'\, f(x) K^{-1}(x,x') f(x')\right]}$$

Note that the action here is quadratic in function values, so this is still a Gaussian process!

Exercise 4.5 (★) (Bayesian lazy learning: kernel ridge regression from the Gaussian posterior). Lazy gradient flow at large width solved itself in Section 4.3: the NTK froze, and the k-th kernel eigenmode of the residual decayed as $e^{-\eta s_k^2 t}$, so that it is learned at the characteristic time $t_k = 1/(\eta s_k^2)$. Here you derive the Bayesian-learning version of the same statement, with β in the role of training time. Restricted to the training values $\mathbf{f} = (f(x^1), \ldots, f(x^n))$, the prior is Gaussian with covariance the kernel matrix $\hat{K}^{\mu\nu} := K(x^\mu, x^\nu)$, so the posterior on training values is

$$p(\mathbf{f}) \;\propto\; \exp\Big[-\tfrac{1}{2}\,\mathbf{f}^T \hat{K}^{-1} \mathbf{f} - \tfrac{\beta}{2}\,|\mathbf{f} - \mathbf{y}|^2\Big]\,.$$

(a) *Posterior mean.* The exponent is quadratic, so the posterior is Gaussian and everything follows from completing the square. Show that the posterior mean of the training values is $\boldsymbol{\mu} = \hat{K}(\hat{K} + \beta^{-1} I_n)^{-1}\mathbf{y}$.

(b) *Eigenmodes.* Diagonalize the kernel, $\hat{K} u_k = \lambda_k u_k$, and show that mode by mode

$$u_k \cdot \boldsymbol{\mu} = \frac{\lambda_k}{\lambda_k + \beta^{-1}}\; u_k \cdot \mathbf{y} = \frac{\beta\lambda_k}{1 + \beta\lambda_k}\; u_k \cdot \mathbf{y}\,.$$

Conclude that the eigenmodes of the GP kernel at initialization are learned one by one as β grows, mode k switching on at the characteristic scale $\beta_k = 1/\lambda_k$, large-eigenvalue modes first (*spectral bias*).

(c) *Compare with gradient flow.* Place this alongside (4.35), $\hat{\Delta}^k(t) = e^{-\eta s_k^2 t}\hat{\Delta}^k(0)$, that is $t_k = 1/(\eta s_k^2)$. What is the same, and what differs? Two things to notice: the correspondence $\beta \leftrightarrow \eta t$ between the amount of data (or inverse noise) and training time; and the different shapes of the two filters, the power law $1/(1+\beta\lambda_k)$ in β versus the exponential $e^{-\eta s_k^2 t}$ in t (compare Exercise B.5). Also: the Bayesian calculation uses the GP kernel K, the dynamical one the NTK Θ, two different frozen kernels for the same network.

(d) *Prediction.* Show that the full posterior over functions is again a Gaussian process,

$$P[f] \propto e^{-\frac{1}{2}\int dx\, dx'\, [f(x)-\mu(x)]\Sigma(x,x')^{-1}[f(x')-\mu(x')]}\,,$$

with mean and covariance

$$\begin{aligned}\mu(x) &= \sum_{\mu,\nu} K(x,x^\mu)\big(\hat{K}+\beta^{-1}I_n\big)^{-1}_{\mu\nu} y^\nu\,,\\ \Sigma(x,x') &= K(x,x') - \sum_{\mu,\nu} K(x,x^\mu)\big(\hat{K}+\beta^{-1}I_n\big)^{-1}_{\mu\nu} K(x^\nu,x')\,.\end{aligned}$$

The posterior mean is *kernel ridge regression* with ridge β^{-1} (Appendix B.4), and $\beta \to \infty$ interpolates the data exactly. [*Hint:* the exponent is quadratic in f; complete the square, and use the Woodbury identity to relate the two natural forms of the answer.]

SOLUTION, p. 616

These equations embody what's sometimes known as *kernel regression*, with *ridge parameter* $\beta^{-1} = T$. (Appendix B.4 derives the same formula for a finite linear model, with no Bayesian input.) If the training samples are sufficiently dense, the mean can be an approximated by an integral in terms of the untrained kernel

$$\mu(x) \approx \int dx'\, dx''\, K(x,x')\big(K+(n\beta)^{-1}\mathrm{id}\big)^{-1}(x',x'')\, y(x'')\,.$$

Writing the data "truths" $y(x) = \sum y_k \varphi_k(x)$ in an eigenbasis for K, with ordered eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots$, we find

$$\mu(x) = \sum_{k=1}^{\infty} \frac{\lambda_k}{\lambda_k + (n\beta)^{-1}} y_k \varphi_k(x) \approx \sum_{k \text{ s.t. } \lambda_k \gtrsim (n\beta)^{-1}} y_k \varphi_k(x)\,.$$

This is the Bayesian version of lazy learning. For given n, β, the "modes" of the data distribution with $\lambda_k \gtrsim (n\beta)^{-1}$ are learned by the network, and the rest are projected out.

Recall that in the Bayesian context, the rescaled inverse-temperature $n\beta$ should play a role somewhat analogous to *training time* in SGD/GD. We can see this explicitly here. For example, in SGD/GD lazy learning, the characteristic time to learn an *NTK mode* s_μ^2

is $t \approx 1/(\eta\, s_\mu^2)$ (see (4.35)). Correspondingly, in Bayesian lazy learning, the minimum inverse-temperature required to learn a *GP mode* is $\beta n \approx 1/\lambda_k$.

5. Mean-field scaling

Sections 3 and 4 analyzed one and the same large-width limit: the fan-in normalization (Section 3.2) with order-one weights and an order-one learning rate, and width $N \to \infty$ at fixed depth and data. At initialization the network converges to a Gaussian process; during training the NTK freezes, the network linearizes, and (by the definition of Section 4.2) no features are learned. It is tempting to conclude that infinite width *implies* laziness. The lesson of this section is that it does not, and that laziness was a consequence of a choice of *parametrization*, rather than of width. Changing the normalization of the output layer by a single extra factor of $\sqrt{N}$ produces a different, equally well-defined infinite-width limit in which the network starts at zero, individual weights move an order-one distance, the NTK evolves in time, and features are genuinely learned.

This alternative limit is called the *mean-field limit*. The name comes by analogy to a classic setup in statistical physics: an interacting particle system with weak all-to-all couplings, whose $N \to \infty$ behavior is governed by a self-consistent "mean field." We'll first explain what physicists mean by a mean-field theory (Section 5.1), then run the analogy in detail for one-hidden-layer networks (Section 5.2), see how the NTK story of Section 4 changes (Section 5.3). Finally we'll discuss some practical applications, including interpolating scalings, μP, and what this framework is actually used for (Sections 5.4–5.5).

The mean-field limit of one-hidden-layer networks was constructed independently, and essentially simultaneously, by four groups (Mei et al. 2018; Chizat & Bach 2018; Rotskoff & Vanden-Eijnden 2022; Sirignano & Spiliopoulos 2020); our treatment draws on all of them. Pedagogical accounts may be found in (Spiliopoulos et al. 2025, Ch. 20) and (Hanin 2026, Lecture 4).

5.1 Mean-field theories in physics

"Mean-field theory" is one of the oldest and simplest approximation schemes in statistical mechanics: *replace the fluctuating environment felt by each degree of freedom with its average (a macroscopic order parameter), then demand self-consistency upon recalculating this average*. For most systems this is an uncontrolled approximation, though often a remarkably successful one *cf.* (Tong 2012, Chapter 1). There is, however, a family of models — those with weak, all-to-all couplings — for which it becomes exact as $N \to \infty$; and networks in mean-field scaling will turn out to be precisely such a model.

5.1.1 Warm-up: the Curie–Weiss magnet

The Curie–Weiss magnet is the Ising model of Section 2.4 on the *fully connected* graph: every spin couples to every other, with the coupling scaled down to J/N so that the total force on any one spin, and the total energy, stay of order N^0 and N^1 respectively. Setting $J = 1$, the energy (2.19) becomes

$$E_N(s) = -\frac{1}{2N}\sum_{i,j=1}^{N} s_i s_j - B\sum_{i=1}^{N} s_i = -N\Big(\frac{m^2}{2} + Bm\Big), \qquad m := \frac{1}{N}\sum_i s_i\,.$$

The second equality is exact, not an approximation: on the complete graph the energy depends on the spins only through the magnetization. Compare the mean-field energy (2.22) of the lattice model, with $dJ \to 1$. Everything that was approximate in Section 2.4 is therefore exact here. The partition function reduces to the one-dimensional integral (2.25) with the same entropy $h(m)$, the saddle-point equation is

$$m = \tanh\big(\beta(m + B)\big)\,,$$

and the model has the second-order transition of Figure 4 at $\beta_c = 1$ and the first-order transition of Figure 2 in B, exactly as analyzed in Section 2.5.

The reason mean-field theory is exact here is worth stating in a form that carries over to networks. The field felt by a single spin is $-\partial E_N/\partial s_i = m + B$: spin i does not care about any individual other spin, only about the empirical average m, the "mean field." The classic argument then has three features:

1. *Each degree of freedom couples to the others only through an average.* (True by construction here.)
2. *The average barely fluctuates.* If the spins are roughly independent, the central limit theorem makes the fluctuations of m of size $O(1/\sqrt{N})$; as $N \to \infty$ each spin sees a *deterministic* effective field $m + B$.
3. *The average is fixed self-consistently.* A single spin in the field $m + B$ has $\mathbb{E}(s_i) = \tanh(\beta(m + B))$; since m was *defined* as the average spin, consistency requires (5.2).

For a finite-dimensional lattice, where each spin has a few strongly coupled neighbors, steps 1–2 are approximations. For the $1/N$ all-to-all model they become exact as $N \to \infty$: the mean field is an average of N weakly correlated terms, and the law of large numbers kills its fluctuations. Networks in mean-field scaling will turn out to be models of exactly this kind.

Exercise 5.1 (Curie–Weiss by fixed-point iteration). Solve (5.2) numerically by the fixed-point iteration $m^{(k+1)} = \tanh\big(\beta(m^{(k)} + B)\big)$, for β below and above 1 and for a few values of B, and watch the iteration converge. Which fixed points are stable? Keep this iteration in mind: we will meet its infinite-dimensional cousin below.

SOLUTION, p. 616

5.1.2 The general shape of a mean-field model

Nothing in the story above needed binary spins. More abstractly, one can take N "particles" $\theta_i \in \mathbb{R}^p$ with an extensive energy of the form

$$E_N(\theta) = \sum_{i=1}^{N} V(\theta_i) + \frac{1}{2N} \sum_{i,j=1}^{N} U(\theta_i, \theta_j)\,,$$

with a single-particle potential V and a symmetric pair interaction U, again with the characteristic $1/N$ coupling. The force felt by particle θ_i is $-\nabla_{\theta_i} E_N$. More so, for particles dominated by drag (highly viscous motion), the dynamics at zero temperature are governed by familiar gradient-descent equations

$$\dot{\theta}_i = -\nabla_{\theta_i} E_N = -\nabla V(\theta_i) - \frac{1}{N} \sum_{j=1}^{N} \nabla_1 U(\theta_i, \theta_j)\,,$$

where ∇_1 differentiates the first slot of U. Once more each particle feels only an empirical average of forces.

It is worth packaging this observation in a way that will carry over verbatim to networks. Define the *empirical measure*

$$\rho_N := \frac{1}{N} \sum_{i=1}^{N} \delta(\theta - \theta_i)\,,$$

a probability measure on $\mathbb{R}^p$ recording where the collection of N particles sit. Then (5.4) says that every particle descends one and the same effective single-particle potential, shaped by where everyone currently is:

$$\dot{\theta}_i = -\nabla_\theta \Psi(\theta_i; \rho_N)\,, \qquad \Psi(\theta; \rho) := V(\theta) + \int U(\theta, \theta')\, d\rho(\theta')\,.$$

The particles are coupled *only through* ρ_N — the exact analogue of "every spin feels only m." The same three-step argument as before then suggests that as $N \to \infty$ the empirical measure becomes deterministic and self-consistent. We will carry this out carefully, for networks, in the next subsection.

5.2 Mean-field scaling for one-hidden-layer networks

Return to the one-hidden-layer network (4.26) of Sections 3.5 and 4.3, but now normalized with $1/N$ out front instead of $1/\sqrt{N}$, with the same *order-one* weights:

$$f_\theta(x) = \frac{1}{N} \sum_{i=1}^{N} c_i\, \phi(a_i \cdot x)\,, \qquad c_i,\, a_{ij} \sim \mathcal{N}(0,1)\,,$$

with $c_i \in \mathbb{R}$, $a_i, x \in \mathbb{R}^d$ (as in (4.26), the first-layer factor $1/\sqrt{d}$ has been absorbed into x). Compared with the fan-in normalization of Sections 3–4, mean-field scaling suppresses the output by *one extra factor of* $1/\sqrt{N}$. The function class is identical — only

the coordinates on it have changed. However, gradient flow is *not* invariant under reparametrization: as emphasized in Section 4.1.1 and in the callout of Section 4.3, gradient descent presupposes the Euclidean metric on parameter space, and rescaling coordinates rescales that metric. Two parametrizations of the same function class can (and here will) flow along completely different trajectories.

For a mean-field analysis, our N particles are going to be the N hidden neurons, with parameters

$$\theta_i = (c_i, a_i) \in \mathbb{R}^{d+1}, \qquad i = 1, \dots, N .$$

Their values are encoded in the empirical measure (5.5), and the network (5.7) depends on its parameters only through ρ_N,

$$f_\theta(x) = \int c\,\phi(a \cdot x)\, d\rho_N(c,a) =: f_{\rho_N}(x) .$$

We think of the network value f_ρ as a linear functional of ρ. Neurons are viewed as exchangeable particles, and the measure is their state.

5.2.1 Initialization

At initialization, the summands $c_i \phi(a_i \cdot x)$ in (5.7) are i.i.d. with mean zero (since c_i is independent of a_i and $\mathbb{E}[c_i] = 0$). With the $1/N$ prefactor, the law of large numbers gives, for each fixed x,

$$f_{\theta_0}(x) = \underbrace{\mathbb{E}\big[c\,\phi(a \cdot x)\big]}_{=0} + O(1/\sqrt{N}) \xrightarrow{N\to\infty} 0 .$$

Contrast this with Section 3.4: with the $1/\sqrt{N}$ normalization, the same sum sits exactly at the scale of its fluctuations, and the CLT produces an order-one Gaussian process. Mean-field scaling instead sits at the scale of the *mean*.

The network begins training as (essentially) the zero function; there is no random-function prior to speak of, and whatever the trained network becomes is built by training rather than inherited from initialization.

5.2.2 The loss is a mean-field energy

Let's train on MSE loss over the data $(x^\mu, y^\mu)_{\mu=1}^n$, as in Section 4:

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_{\mu=1}^{n} \big(f_\theta(x^\mu) - y^\mu\big)^2 .$$

Inserting (5.7) and expanding the square, we get

$$\mathcal{L}(\theta) = \text{const} + \frac{1}{N} \sum_{i=1}^{N} V(\theta_i) + \frac{1}{2N^2} \sum_{i,j=1}^{N} U(\theta_i, \theta_j) ,$$

with

$$V(\theta) = -\sum_{\mu=1}^{n} y^\mu \, c\, \phi(a \cdot x^\mu)\,, \qquad U(\theta,\theta') = \sum_{\mu=1}^{n} c\, \phi(a \cdot x^\mu)\; c'\, \phi(a' \cdot x^\mu)\,.$$

Comparing (5.12) with (5.3), we see that up to an overall factor of N, the loss is precisely a mean-field energy. In particular, $E_N := N\mathcal{L}$ has the canonical normalization from (5.3) and is extensive, while the loss itself can be interpreted as the *intensive* energy per neuron.

We can also identify physical roles for the two potentials! V rewards neurons for aligning their output function $c\,\phi(a\cdot\cdot)$ with the data, playing the role of the external field B; while U is a data-weighted interaction between *pairs* of neurons that penalizes redundant features, playing the role of the spin-spin coupling.

5.2.3 Equations of motion and the mean field

Because $\partial f/\partial\theta_i = O(1/N)$ in this parametrization, raw gradients of $\mathcal{L}$ are $O(1/N)$, and gradient flow at fixed learning rate would freeze as $N\to\infty$. We must therefore scale the learning rate *up*,

$$\eta = N\,\tilde\eta\,,$$

with $\tilde\eta$ fixed. (Equivalently, we could run vanilla gradient flow on the extensive energy $E_N = N\mathcal{L}$.) This is the rule of Section 4.3 at work: the NTK, computed below in (5.24), is $O(1/N)$, so keeping $\frac{d}{dt}f^\mu = -\eta\,\Theta^{\mu\nu}\Delta^\nu$ of order one requires $\eta\propto N$. Note the contrast with Section 4.3: there the NTK was of order one and the learning rate needed no rescaling; here the NTK *shrinks* like $1/N$ and we scale up. Recall also the caveat from that section about changes of variables. The mean-field network could equally be written without a prefactor, $f=\sum_i \tilde c_i\phi(a_i\cdot x)$ with $\tilde c_i = c_i/N \sim \mathcal{N}(0,1/N^2)$; but then the *same* dynamics (5.15) requires a learning rate $\tilde\eta/N$ for the readout weights $\tilde c_i$ and $N\tilde\eta$ for the hidden weights a_i — one learning rate per layer, differing by a factor N^2. Keeping all weights of order one is what lets a single η do the job, and it is the reason we adopted that convention in Section 3.2.

With (5.14), the flow $\dot\theta = -\eta\nabla_\theta\mathcal{L}$ becomes, per neuron,

$$\dot c_i = -\tilde\eta\sum_{\mu=1}^{n}\Delta^\mu\,\phi(a_i\cdot x^\mu)\,, \qquad \dot a_i = -\tilde\eta\; c_i\sum_{\mu=1}^{n}\Delta^\mu\,\phi'(a_i\cdot x^\mu)\,x^\mu\,,$$

where $\Delta^\mu = f_{\rho_N}(x^\mu) - y^\mu$ is the residual of Section 4.1. Equivalently, in the potential form (5.6),

$$\dot\theta_i = -\tilde\eta\,\nabla_\theta\,\Psi(\theta_i;\rho_N)\,,$$
$$\Psi(\theta;\rho) = V(\theta) + \int U(\theta,\theta')d\rho(\theta') = \sum_{\mu=1}^{n}\Delta^\mu_\rho\; c\,\phi(a\cdot x^\mu)\,,$$

with $\Delta^\mu_\rho = f_\rho(x^\mu) - y^\mu$.

As anticipated, the velocity of neuron i depends on its own state (c_i, a_i), and on all the other neurons only through the average $f_{\rho_N} = \frac{1}{N}\sum_j c_j \phi(a_j \cdot \cdot)$, via the residual Δ^μ. The network function f is the mean field here! We arrive at a dictionary:

Curie–Weiss	**one-hidden-layer network, mean-field scaling**
spin $s_i \in \{\pm 1\}$	neuron $\theta_i = (c_i, a_i) \in \mathbb{R}^{d+1}$
magnetization $m = \frac{1}{N}\sum_j s_j$	network function $f_\rho = \frac{1}{N}\sum_j c_j \phi(a_j \cdot \cdot)$
external field B	data term V (the targets y^μ)
pair coupling $-\frac{1}{N} s_i s_j$	neuron-neuron interaction $\frac{1}{N} U(\theta_i, \theta_j)$
effective field $m + B$	residual $\Delta^\mu_\rho = f_\rho(x^\mu) - y^\mu$
extensive energy E_N	$N\mathcal{L}$ (the loss is intensive)
zero-temperature dynamics	gradient flow
Langevin at temperature T	noisy training, Section 2.1
$m = \tanh(\beta(m + B))$	self-consistent Gibbs measure, Equation 5.22 below

Exercise 5.2 (Structure of the mean-field equations). (a) Derive (5.12)–(5.13) and (5.15), and check that (5.16) reproduces them, using the symmetry of U.

(b) (*Balancedness.*) For a positively homogeneous activation such as ReLU, $\phi'(z)z = \phi(z)$. Show that in this case (5.15) conserves $c_i^2 - |a_i|^2$ for every neuron, exactly, along the flow. The two layers of a given neuron therefore remain coupled for all time. (On the other hand, distinct neurons will decouple from each other, as we'll see momentarily.)

(c) Observe that for MSE loss $\dot{c}_i$ in (5.15) does not depend on c_i (only on a_i and the mean field). Where does the linearity of f_ρ in c enter?

SOLUTION, p. 616

5.2.4 From particles to measures: the continuity equation

We now pass to the $N \to \infty$ limit, in distinct steps.

Step 1: rewrite the N coupled ODEs as one equation for the empirical measure. (This remains exact at finite N.) Suppose particles move deterministically with some velocity field, $\dot{\theta}_t = v_t(\theta_t)$, and let ρ_t denote the law of one such particle.

The expectation of a smooth test function g obeys

$$\begin{aligned}\frac{d}{dt}\int g\, d\rho_t = \frac{d}{dt}\,\mathbb{E}\big[g(\theta_t)\big] &= \mathbb{E}\big[\nabla g(\theta_t)\cdot v_t(\theta_t)\big] \\ &= \int \nabla g \cdot v_t \, d\rho_t = -\int g\, \nabla\cdot(\rho_t v_t)\,,\end{aligned}$$

integrating by parts in the last step. Since g was arbitrary,

$$\partial_t \rho_t + \nabla \cdot \big(\rho_t \, v_t\big) = 0 \,.$$

This is known as the *continuity equation*, and it simply expresses conservation of probability. Applied to our system, with $v_t = -\tilde{\eta}\, \nabla \Psi(\cdot\,; \rho_{N,t})$ from (5.16), the empirical measure satisfies (weakly, since it is a sum of delta functions)

$$\partial_t \rho_{N,t} = \tilde{\eta}\, \nabla \cdot \Big(\rho_{N,t}\, \nabla_\theta \Psi\big(\theta; \rho_{N,t}\big) \Big) \,.$$

An important consequence of the mean-field ($1/N$, all-to-all) coupling here is that (5.19) is a closed equation in ρ_N.

Step 2: the limit. At finite N, $\rho_{N,t}$ is a random, atomic measure. (Atomic means it's a sum of delta functions; it's random due to random initialization.) As $N \to \infty$ with i.i.d. initialization $\theta_i \sim \rho_0$, the law of large numbers suggests — and the theorems of (Mei et al. 2018; Chizat & Bach 2018; Rotskoff & Vanden-Eijnden 2022; Sirignano & Spiliopoulos 2020) confirm — that $\rho_{N,t}$ converges (weakly) to ρ_t, a deterministic curve of smooth probability measures solving the limiting PDE

$$\partial_t \rho_t = \tilde{\eta}\, \nabla \cdot \Big(\rho_t\, \nabla_\theta \big(V + U * \rho_t \big) \Big) \,, \quad \text{with} \quad (U * \rho)(\theta) := \int U(\theta, \theta')\, d\rho(\theta') \,.$$

There are quantitative $O(1/\sqrt{N})$ error bounds at any fixed time horizon.

5.2.5 The McKean–Vlasov equation and self-consistency

In this context, the analogue of the Curie–Weiss self-consistency (5.2), namely $m = \tanh(\beta(m + B))$, is called the McKean–Vlasov equation. To get it, focus on a single representative neuron θ_i. As $N \to \infty$, we can replace the force on it by the ensemble average:

$$\dot{\bar{\theta}}_t = -\tilde{\eta}\, \nabla_\theta \Psi\big(\bar{\theta}_t; \rho_t\big) \,, \qquad \text{subject to} \qquad \rho_t = \mathrm{Law}(\bar{\theta}_t) \,.$$

The constraint $\rho_t = \mathrm{Law}(\bar{\theta}_t)$ is the entire content of the equation: the particle is driven by a field built from a distribution, and that distribution is *its own law*.

(Probabilists call (5.21) a *nonlinear* Markov process, nonlinear not in θ but in the law.) The McKean–Vlasov equation is equivalent to the continuity equation; the particle trajectories of (5.21) are the characteristic curves of the PDE (5.20).

We remark on a few structural features:

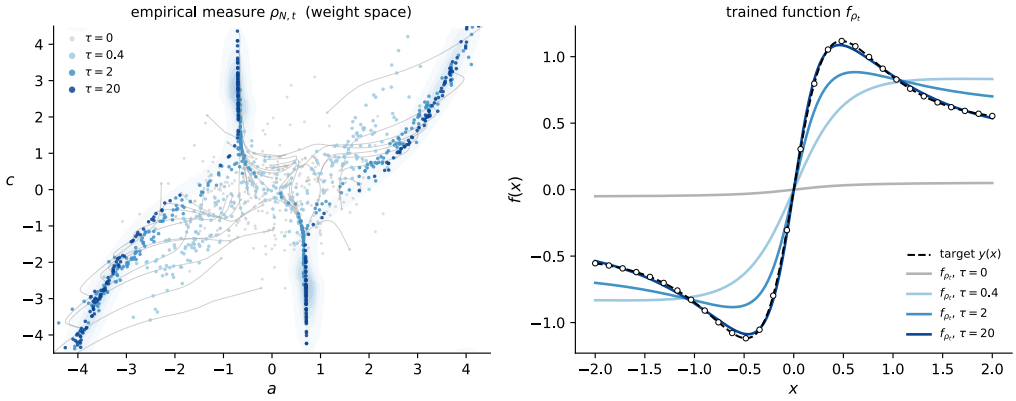
- *Propagation of chaos.* Since neurons interact only through a quantity (ρ_N, equivalently f_{ρ_N}) that becomes deterministic as $N \to \infty$, they become asymptotically *independent*: any fixed collection of k neurons converges to k i.i.d. copies of (5.21), with cross-neuron correlations of size $O(1/N)$. Thus, independence at initialization *propagates* instead of degrading (Sznitman 1991).
- *Gradient-flow structure.* Define the $N \to \infty$ loss as a functional of the measure, $\mathcal{L}[\rho] = \frac{1}{2} \sum_\mu (f_\rho(x^\mu) - y^\mu)^2$. Since f_ρ is linear in ρ, this functional is convex in ρ.

This ultimately leads to global-convergence theorems. For example, it was shown in (Chizat & Bach 2018) that for homogeneous activations (e.g. ReLU), if training converges at all, it converges to a *global* minimizer of the convex functional $\mathcal{L}[\rho]$.

- *Temperature.* If one trains instead with the Langevin dynamics of Section 2.1, Eqn. (2.6), at temperature $T = \beta^{-1}$, one finds convergence to a stationary distribution that satisfies (Mei et al. 2018)

$$\rho_\infty(\theta) \;\propto\; e^{-\beta\,\Psi(\theta;\rho_\infty)}\,.$$

This is another version of the "$m = \tanh(\beta(m + B))$" consistency condition, now for noisy training.



Mean-field training for a $d = 1$ example: $f_\rho(x) = \frac{1}{N}\sum_i c_i \tanh(a_i x)$, trained by the rescaled gradient flow (5.15) on $n = 30$ samples of the two-neuron teacher $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$; $\tau = \tilde{\eta}\, t$ is rescaled training time. *Left:* the empirical measure $\rho_{N,t}$ in the (a, c) plane for $N = 300$ (dots, colored by time; thin gray lines trace 40 of the particle trajectories), drawn over the density of an $N = 30{,}000$ run at $\tau = 20$ (shaded contours), standing in for the limiting measure ρ_t. Individual neurons move order-one distances, and the measure condenses onto one-dimensional filaments — the configurations realizing the teacher. (The point symmetry $(a, c) \mapsto (-a, -c)$ reflects the oddness of tanh.) *Right:* the mean field itself, $f_{\rho_t}(x)$, converging to the target (dashed; circles mark the training data).

How would one actually *solve* the self-consistent system (5.21)? The same way one solves (5.2) (Exercise 5.1): freeze a guessed environment (ρ_t), solve the now-ordinary ODE, compute the law of the solution, feed it back, iterate. On short time horizons this map is a contraction, which is how existence, uniqueness, and propagation of chaos are all proven in one stroke (Sznitman 1991). Alternatively, solve the PDE (5.20) on a grid, which is feasible (only) for small d.

In Figure 6 we instead solve the system by simply training a network with $d = 1$. In the left panel, each dot is a neuron. Training is a flow of the cloud of neurons, and the cloud condenses onto the low-dimensional structures in weight space that realize the target — learning its features. Exercise 5.3 reproduces this figure, and sets it beside the frozen cloud of the lazy network.

Exercise 5.3 (★) (Numerics: the mean-field network (lab)). Continue the lab of Exercise 4.4: same data, same teacher $y(x) = 2\tanh(3x) - \frac{3}{2}\tanh(x)$, same student (4.40), but now with the mean-field exponent $\gamma = 1$ and the rescaled updates (5.15),

$$c_i \mathrel{-}= \Delta\tau \sum_\mu \Delta^\mu \tanh(a_i x^\mu)\,,$$
$$a_i \mathrel{-}= \Delta\tau\, c_i \sum_\mu \Delta^\mu \big(1 - \tanh^2(a_i x^\mu)\big) x^\mu\,,$$

with $\Delta\tau = 5 \times 10^{-4}$ for 4×10^4 steps, so that the rescaled time $\tau = \tilde{\eta}\, t$ runs to 20. Note that there is no $1/N$ in these updates: that is the required $O(N)$ learning rate (5.14), already built in ($\eta = N\,\Delta\tau$). Monitor the NTK (4.41) with $\gamma = 1$.

(a) *It fits.* Train and check that the loss becomes small, if more slowly in rescaled time than the lazy run.

(b) *Initialization.* Over about 200 seeds, histogram $f_0(x)$ at $x = 1$ for several N: the distribution collapses to 0 with width $\sim 1/\sqrt{N}$. Check the scaling.

(c) *The kernel moves.* For $N \in \{30, 100, 300, 1000\}$, record $\|\Theta_{\text{end}} - \Theta_0\|/\|\Theta_0\|$ and plot it together with the lazy curve of Exercise 4.4(c): one plot, two curves, the whole dichotomy.

(d) *Watch the particles.* Scatter-plot the neuron cloud (a_i, c_i) at $\tau \approx 0$, 0.4, 2, 20 for both runs. Lazy: the cloud is static (each weight moves $O(1/\sqrt{N})$). Mean-field: the cloud transports and condenses onto thin filaments, the configurations realizing the teacher, as in Figure 6. Repeat at $N = 3000$: the pattern is the same, because you are looking at a deterministic limiting measure.

(e) *Now play.* The starter teacher is itself a two-neuron tanh network, so the student's features can align *exactly* with the teacher's. Is that feature learning, or feature copying? Try teachers outside the model class and watch what the cloud does instead: $y(x) = \sin(2x)$; $y(x) = |x| - 1$; a ReLU student on the tanh teacher (and vice versa); or move to $d = 2$ with $y(x) = \tanh(x_1 + x_2)$ and plot the cloud of directions $a_i/|a_i|$. Which aspects of (c)–(d) are universal, and which were special to the aligned case?

SOLUTION, p. 617

5.3 The NTK is no longer constant: feature learning

Section 4.3 showed that in the fan-in normalization the NTK freezes as $N \to \infty$, and Section 4.2 defined feature learning as change of the NTK during training. Let's redo that computation in mean-field scaling. From (5.7), $\partial f/\partial c_i = \frac{1}{N}\phi(a_i \cdot x)$ and $\partial f/\partial a_i = \frac{1}{N} c_i \phi'(a_i \cdot x)\, x$, so the NTK of Section 4.1 is

$$\Theta(x, x') = \frac{1}{N^2}\sum_{i=1}^{N} k_{\theta_i}(x, x') = \frac{1}{N}\int k_\theta(x, x')\, d\rho_N(\theta)\,,$$

with the per-neuron kernel

$$k_\theta(x,x') := \phi(a\cdot x)\phi(a\cdot x') + c^2\,\phi'(a\cdot x)\phi'(a\cdot x')\;x\cdot x'\,.$$

Thus $\Theta = O(1/N)$, recovering the previous observation that gradients are small, which was the reason for the rescaling (5.14). The estimates of Section 4.3 also explain why the kernel is now free to move. The Hessian blocks (4.27) carry $1/N$ instead of $1/\sqrt{N}$, so $\|\nabla^2_\theta f\| = O(1/N)$; but the learning rate is $\eta = N\tilde\eta$, so the bound (4.25) gives for the relative drift $\|\dot\Theta\|/\|\Theta\| \lesssim \eta\,\|\nabla^2_\theta f\|\,|\Delta| = O(1)$. The bound no longer forces the kernel to freeze, and the equations of motion (5.15) will show that it does move. As $N\to\infty$, the meaningful object is $N\Theta = \int k_\theta\,d\rho_N$, and the function-space dynamics (4.10) now becomes a kernel gradient flow *with a time-dependent kernel*:

$$\frac{d}{dt}f^\mu = -\tilde\eta\big(N\Theta^\mu{}_\nu\big)\big|_{\rho_t}\,\Delta^\nu\,,\qquad N\Theta\big|_{\rho_t} = \int k_\theta\,d\rho_t\,.$$

At initialization the LLN makes $N\Theta \to \mathbb{E}_{\rho_0}[k_\theta]$, a deterministic kernel — the *same* limiting kernel (4.24) as in the fan-in normalization. The two limits start from the same place.

During training, however, they part ways. In Section 4.3, each parameter moved only $O(1/\sqrt{N})$, so ρ_N stayed within $O(1/\sqrt{N})$ of ρ_0 and the kernel was pinned, (4.31). Here, the equations of motion (5.15) move each particle an *order-one* distance. (Each neuron has only $O(1/N)$ leverage, so fitting the data *forces* the average neuron to move a distance $O(1)$.) Thus ρ_t transports macroscopically, and by (5.26) the kernel is dragged along at order one as well.

5.4 The scaling dial and Maximal Update Parameterization

It's quite instructive to look at an entire one-parameter family of scalings, which has fan-in/NTK scaling and mean-field scaling as its endpoints. The family can be encoded via a prefactor

$$f_\theta(x) = \frac{1}{N^\gamma}\sum_{i=1}^{N} c_i\,\phi(a_i\cdot x)\,,\qquad \gamma\in[\tfrac12,1]\,,$$

where $c_i, a_{ij}\sim\mathcal{N}(0,1)$. For each γ, as $N\to\infty$, the learning rate must be simultaneously be scaled as $\eta = N^{2\gamma-1}\tilde\eta$ so that the function-space dynamics stays $O(1)$ (one can derive that from $\Theta\sim N^{1-2\gamma}$). Repeating the estimates above gives (see Exercise 5.4):

	f_{θ_0} **at init**	**per-neuron motion**	**limit**
$\gamma=\frac12$	$O(1)$ (GP, CLT)	$O(N^{-1/2})$	lazy: Sections 3 and 4
$\frac12<\gamma<1$	$O(N^{\frac12-\gamma})\to 0$	$O(N^{\gamma-1})\to 0$	lazy (frozen kernel, zero init)
$\gamma=1$	$O(N^{-1/2})\to 0$	$O(1)$	mean-field: this section

Notably, every intermediate γ collapses onto the kernel limit: the initialization GP is scaled away, but the neurons still move vanishingly little, so training is again linear in an (appropriately rescaled) frozen kernel (Chizat et al. 2019; Mei et al. 2018). Mean-field, $\gamma = 1$, is the *unique* scaling in this family with both a well-defined limit and order-one feature movement; it's the "maximal update" that one can take without blowing up. This uniqueness can be generalized from one hidden layer to arbitrary depth (where consistency forces *layer-dependent* learning-rate and initialization scalings), and is known as the Maximal Update Parametrization, μP, in Yang and Hu's line of work on tensor programs (Yang & Hu 2021). In particular, mean-field scaling is μP for $L = 1$.

Exercise 5.4 (Lazy vs. rich along the dial). For the family (5.27):

(a) show $\Theta = O(N^{1-2\gamma})$ and hence that $\eta = N^{2\gamma-1}\tilde{\eta}$ is the right co-scaling;

(b) show the initialization output is $O(N^{1/2-\gamma})$ and per-neuron velocities are $O(N^{\gamma-1})$;

(c) conclude that the (normalized) kernel drift over any fixed training horizon is $O(N^{\gamma-1})$, vanishing for every $\gamma < 1$;

(d) reconcile $\gamma = \frac{1}{2}$ with the bounds of Section 4.3. Optional: relate γ to the output-scale parameter α of (Chizat et al. 2019), where multiplying any model by $\alpha \to \infty$ produces lazy training.

SOLUTION, p. 617

5.5 What is this framework good for?

Nobody solves the McKean–Vlasov PDE (5.20) for a real network! There are practical payoffs, though, of three different kinds: *scaling prescriptions*, *mechanism identification*, and *reduced descriptions* that downstream theories build on.

Hyperparameter transfer (μP) is one key application. The mean-field analysis is, fundamentally, a theory of which joint scalings of initialization and learning rate keep the training dynamics nondegenerate and learning features as $N \to \infty$. Its deep-network generalization (Yang & Hu 2021) turns this into an engineering recipe, μTransfer (Yang et al. 2022): parametrize the network so that its infinite-width limit is the feature-learning one, and then optimal hyperparameters become approximately width-independent, so one can tune a small proxy model and *transfer* the settings to the large one. In the original demonstration this retuned GPT-3 6.7B at a few percent of pretraining cost and outperformed the published model; variants of this parametrization are now standard in large-scale training.

The dial of Section 5.4 also lets one interpolate between regimes on real architectures, and measure where feature learning actually pays off. Kernel-regime behavior tends to be competitive on small datasets, while on CIFAR-scale image tasks trained networks beat their own frozen NTKs by large margins (Geiger et al. 2020). Diagnostics from this circle of ideas (kernel distance/alignment during training, e.g. the silent-alignment effect (Atanasov et al. 2022)) are now routine tools for asking "is my network actually learning features, or just doing kernel regression?"

The mean-field limit here holds data fixed while sending $N \to \infty$, which unfortunately does not yet make it a quantitative theory of generalization at realistic data/width ratios. The dynamical mean-field theory of Section 6 (Bordelon & Pehlevan 2022) will rectify this somewhat; it is a field-theoretic refinement of the current analysis, whose extensions do reproduce compute-optimal neural scaling laws, in particular at large dataset size (Bordelon et al. 2024).

5.6 Grokking as a lazy-to-rich transition: a reading exercise

We finish this section with two guided readings that apply various concepts introduced above to one of the most interesting and fundamental phenomenons in learning: grokking (Power et al. 2022). Grokking refers to the following behavior. One trains a small network on an algorithmic dataset — the canonical example is addition mod P, presented as pairs (n, m) with label $(n + m)$ mod P, with only a fraction of all P^2 facts shown. The network reaches perfect *training* accuracy quickly, while *test* accuracy sits at chance: it has simply memorized. Then, orders of magnitude later in training and long after the train loss has flattened, test accuracy jumps abruptly to near 100%. Such delayed, abrupt generalization, with emergent capabilities, demands a mechanism.

Theory has produced (at least) two quantitative explanations, and this subsection and the next walk through one each: the *dynamical* lens of Kumar–Bordelon–Gershman–Pehlevan (Kumar et al. 2024) (this subsection) — grokking as a delayed transition from lazy to rich dynamics — and the *equilibrium* lens of Rubin–Seroussi–Ringel (Rubin et al. 2024) (Section 5.7) — grokking as a first-order phase transition with metastable lag. The two walkthroughs are parallel in structure and use largely disjoint toolkits from these notes. The dynamical one presumes comfort with Sections 4 and 5.4; the Bayesian one is a denser read, and combines scaling analysis with a mean-field approximation.

The work of Kumar, Bordelon, Gershman, and Pehlevan (Kumar et al. 2024) invokes no statistical-mechanics ensemble at all. There is only gradient descent, and the lazy/rich dial of Section 5.4, *unfolding in time*. The paper claims that a network groks when 1) it first fits the training data through its cheap, fast channel (kernel regression with the frozen NTK of its initialization), and that lazy fit memorizes without generalizing because the initial kernel is misaligned with the task; then 2) on a slower timescale controlled by the network's laziness, the features finally move (the kernel rotates, as in Section 5.3), a generalizing solution is found, and test loss falls long after train loss did. Two beautiful features of this work are that its clearest examples grok under vanilla gradient descent with zero weight decay, and that the parameter norm increases during grokking. Both are direct counterexamples to earlier accounts of grokking built on weight decay and shrinking norms.

The paper is written almost entirely in the language these notes have already built (Sections 4 and 5), so the dialect box below is short. Read the introduction, the two-layer polynomial-regression model, and the experiments (modular arithmetic, MNIST, a one-layer transformer); skip the dynamical mean-field asymptotics on a first pass — that machinery belongs to Section 6. Then work through Exercise 5.5.

A few translations of terminology from (Kumar et al. 2024):

- *Lazy / linearized dynamics*: the regime of Section 4.3 — the network moves only in the affine subspace (4.1) spanned by its initial gradients, so training is kernel regression with the frozen initial NTK (Sections 4.1 and 4.6).
- *Laziness parameter* α: an overall multiplier on the network output, $f \mapsto \alpha f$ (with loss and learning rate co-scaled). This is the continuous version of the γ-dial of Section 5.4, in the form introduced by (Chizat et al. 2019): $\alpha \to \infty$ is fully lazy, small α is rich.
- *Kernel–task (mis)alignment*: the language of Section 4.2 — how much of the target lies in the top eigenmodes of the initial NTK. Misaligned = the target's weight sits in small-eigenvalue modes, which the mode-filtering formula of Section 4.6 says are not learnable from the available data.
- *Sufficient statistics*: a handful of scalar functions of the weights whose closed dynamics determine the test loss — the dynamical analogue of the order parameters of Section 2.

Exercise 5.5 (Grokking as a lazy-to-rich transition). Read (Kumar et al. 2024), and answer the following, citing where in the paper each answer lives. (Parts (a)–(e) have exact counterparts in Exercise 5.6, the other walkthrough; part (f) is the shared comparison question.)

(a) *What regime is the analysis set in, and what dial controls it?* There is no thermodynamic limit and no equilibrium ensemble here, so what plays the role that a "scaling limit" plays elsewhere in this section? How is the laziness parameter α implemented, and how does it relate to the γ-dial of Section 5.4? Section 5.2's scaling prescription had two halves, an output scale and a learning-rate scale. Which half is doing the work here? (The other walkthrough meets the opposite half: see Exercise 5.6(a).)

(b) *What plays the role of the order parameter?* What quantity distinguishes the memorized state from the generalizing one, and how would you measure it in a real training run? Compare with the Fourier overlap $\Phi(w)$ that plays this role in the other walkthrough (Exercise 5.6(b)): what information do the two quantities share?

(c) *Necessary conditions.* The paper states three conditions for grokking to occur. Find them, and translate each into the language of these notes (Sections 4.6, 4.3 and 5.4).

(d) *What parameter is tuned?* What makes grokking more dramatic, and what makes it vanish entirely? What does the theory predict in the two extremes $\alpha \to \infty$ and α small? Contrast with the control parameters of Exercise 5.6(d).

(e) *Where does the delay come from?* Both theories must explain the gap between the train-loss drop and the test-loss jump. Here there is no metastability and no noise — so what produces two well-separated timescales in a single deterministic flow? What does each theory *require* to produce a delay (noise? weight decay? neither?), and how do the paper's zero-weight-decay, growing-norm experiments bear on earlier explanations of grokking?

(f) *Synthesis.* Are the equilibrium mechanism of Section 5.7 and the dynamical mechanism of this section incompatible, or two corners of one phenomenon? Design at least one experiment whose outcome would favor one over the other — consider, e.g., how the grokking delay should depend on training noise at fixed hyperparameters, or how the delay should fluctuate from seed to seed, under each mechanism.

SOLUTION, p. 618

→ *5.7 Bayesian grokking: a first-order transition (alternative reading)*: Supplementary Material, p. 501

6. Non-equilibrium physics, MSRJD, and DMFT

So far our statistical-mechanics toolkit has been an equilibrium one: Gibbs measures, partition functions, and free energies (Section 2), together with limits in which training simplifies to kernel regression (Section 4) or to a deterministic flow of measures (Section 5). This section introduces the formalism that confronts the *dynamics* of learning head-on. The strategy is borrowed from non-equilibrium statistical physics: write the stochastic dynamics itself as a path integral — the Martin–Siggia–Rose–Janssen–De Dominicis (MSRJD) construction — then average over the randomness (noise, initialization, and, crucially, the training data, treated as quenched disorder exactly as in Section 2.3), and finally extract closed, self-consistent equations for a small set of *two-time* order parameters: correlation functions $C(t, t')$ and response functions $R(t, t')$. The result is known as dynamical mean-field theory (DMFT), and it is the third and final incarnation of "mean field" in these notes: the statics of Section 2 determined scalars self-consistently, the McKean–Vlasov limit of Section 5 determined a measure, and DMFT determines whole functions of two times.

The payoffs in this section are a complete worked application — double descent as a *dynamical phase transition* in linear regression (Section 6.2) — and a guided tour of the modern DMFT program for deep learning (Section 6.3), whose scaling-law application closes the notes in Section 7. Throughout, we follow the spirit of Altland and Simons (Altland & Simons 2010), which motivates the formalism in condensed-matter language with a good balance of detail and intuition, and we borrow freely from two pedagogical tutorials written for neuroscientists, (Chow & Buice 2012) and (Stapmanns et al. 2020).

6.1 From stochastic dynamics to path integrals: the MSRJD formalism

When we model a system with fluctuations — or deliberately add noise to a deterministic model, as in the Langevin dynamics of Section 2.1 — differential equations become stochastic, and the difficulty of solving them jumps. A **stochastic differential equation**, at first order for example, takes the form

$$\frac{dx}{dt} = f(x) + \xi\,,$$

where $\xi(t)$ is a (Gaussian) white noise process — additive noise, $\xi = \xi(t)$, as opposed to multiplicative noise $\xi = \xi(x, t)$ — and $x = x(t)$ is now a random trajectory. The noise is entirely defined by the first and second moments,

$$\langle \xi(t) \rangle_\xi = 0, \qquad \langle \xi(t)\xi(t') \rangle_\xi = D\delta(t-t')\,,$$

where the expectations are with respect to the noise ensemble[3]. Obtaining closed-form solutions for the time evolution of the probability distribution for $x(t)$ is usually challenging. These evolutions can be attacked by the traditional routes — the Langevin and Fokker–Planck descriptions — for which the standard references are Risken (Risken 1996), Gardiner (Gardiner 2004), and van Kampen (van Kampen 2007). The objects of interest here are instead the moments of $x(t)$ and the probability density function $p(x,t)$. Methods developed in statistical physics (in particular those used in perturbative quantum field theory and classical non-equilibrium systems) using path integrals provide a way to calculate these statistical objects. The path integral, or functional integral, approach is convenient for perturbative calculations (expanding around an exactly solvable theory with a small parameter), but also permits importing other tools like renormalization group methods or other explicitly non-perturbative approaches when there is no small parameter, or the expansion breaks down (as they often do).

The conceptual recipe of the MSRJD formalism is short, and worth internalizing before the derivation:

- write down a *discrete* SDE, fixing a convention (Ito) that enforces causality;
- express the transition probability between time steps as a delta function imposing the update rule;
- assemble the product over infinitesimal steps into a path integral;
- write each delta function in Fourier representation, introducing an auxiliary *response field*;
- average over the noise, then add sources and differentiate to extract statistics.

It turns out that the historic approach of Onsager and Machlup (Onsager & Machlup 1953) is the "Lagrangian" version of this "Hamiltonian"-like approach; the two are related by a Legendre transform, or equivalently by integrating out the auxiliary field. That auxiliary field is aptly named the **response field**: as we will see below, its correlation functions compute how the system *responds* to external perturbations, and it is the genuinely new ingredient of the non-equilibrium formalism.

We now give the derivation, quickly but completely. The standard first step is to discretize time for a continuous equation, or simply start with a discrete update rule. Under this discretization $x(t) \to x_i$, where $i = 1, ..., N$, so that,

$$x_i = x_{i-1} + \Delta t\Big(f(x_{i-1}) + \xi_{i-1}\Big)\,,$$

where Δt is the temporal spacing. We can rearrange this so that the RHS is zero and write the transition probability between steps as delta function, and the product of these representing a trajectory solution. Defining a solution of the stochastic differential equation as $x[\xi]$, an expectation value of an observable $\mathcal{A}[x]$ is formally,

$$\langle \mathcal{A}[x] \rangle_\xi = \int \mathcal{D}x \mathcal{A}[x] \left\langle \delta(x - x[\xi]) \right\rangle_\xi = \int \mathcal{D}x \mathcal{A}[x] \left| \frac{\delta X}{\delta x} \right| \delta(X)\,.$$

We used the shorthand $X_i \equiv x_i - x_{i-1} - \Delta t\big[f(x_{i-1}) + \xi_{i-1}\big]$ for the constraint imposed at each step. The measure here, is a typical functional path integral measure $\mathcal{D}x = \prod_i dx_i$ with $\delta(x - x[\xi])$ similarly factorized across steps. In particular we also picked a first order finite difference and the convention that the noise is evaluated at the previous time-step, which is called Ito discretization or convention. Many common processes are manifestly Ito, like SGD for example, where the noise drawn at a batch doesn't affect the weights at that iteration. There are other conventions, the most common alternative is Stratonovich (which is more like a midpoint rule), however, Ito has a unit (volume-preserving) functional determinant and other schemes require careful treatment to avoid anomalous terms from the determinant. In our case the matrix is triangular with a unit diagonal. Substituting in the definition of X,

$$\langle \mathcal{A}[x] \rangle_\xi = \int \mathcal{D}x\, \mathcal{A}[x]\, \big\langle \delta\big(\partial_t x - f(x) - \xi\big) \big\rangle_\xi \,.$$

Finally, we can use the Fourier representation of the delta, introducing an auxiliary variable $\tilde{x}$, which in the continuum is

$$\langle \mathcal{A}[x] \rangle_\xi = \int \mathcal{D}x\, \mathcal{D}\tilde{x}\, \mathcal{A}[x]\, e^{\,i \int dt\, \tilde{x}\, (\partial_t x - f(x) - \xi)} \,.$$

The final step is the payoff: the noise now appears *linearly* in the exponent, so the Gaussian average over ξ can be done exactly, $\big\langle e^{-i\int dt\, \tilde{x}\, \xi} \big\rangle_\xi = e^{-\frac{D}{2}\int dt\, \tilde{x}^2}$, leaving the **MSRJD functional integral**

$$\begin{aligned} \langle \mathcal{A}[x] \rangle &= \int \mathcal{D}x\, \mathcal{D}\tilde{x}\, \mathcal{A}[x]\, e^{-S[x,\tilde{x}]} \,, \\ S[x,\tilde{x}] &= \int dt \Big[-i\,\tilde{x}\,\big(\partial_t x - f(x)\big) + \frac{D}{2}\,\tilde{x}^2 \Big] \,. \end{aligned}$$

Four structural comments, each of which does real work later:

- *The partition function is one.* Setting $\mathcal{A} = 1$, the integral is just $\int \mathcal{D}x$ of a normalized probability: $Z_{\rm dyn} = 1$, with no sources. Unlike in equilibrium, all the physics lives in correlation functions, obtained by coupling sources to both x and $\tilde{x}$ and differentiating — compare the $Z_{\rm dyn} = 1$ starting point of Section 6.2.
- *The response field earns its name.* Perturbing the drift, $f \to f + h(t)$, changes the action by $i\int \tilde{x}\, h$, so

$$R(t,t') := \left.\frac{\delta \langle x(t) \rangle}{\delta h(t')}\right|_{h=0} = \langle x(t)\, i\tilde{x}(t') \rangle \,.$$

Correlators of $\tilde{x}$ compute responses to kicks. Causality is built in: $R(t,t') = 0$ for $t < t'$ (the Ito discretization is what guarantees this, along with $\langle \tilde{x}\,\tilde{x} \rangle = 0$).

- *Two-time observables.* The natural objects of the formalism are the correlation function $C(t,t') = \langle x(t) x(t') \rangle$ and the response $R(t,t')$. In *equilibrium* they are not independent: the fluctuation–dissipation theorem (FDT) locks them together, $T\,R(t,t') = -\partial_{t'} C(t,t')$ for $t > t'$ (with $T = D/2$ in our normalization) (Kubo 1957;

Kubo 1966). A measured *violation* of the FDT is therefore the standard diagnostic that a system is genuinely out of equilibrium — the warning light used to great effect in Section 6.2.

- *Onsager–Machlup.* The action (6.7) is quadratic in $\tilde{x}$, so the response field can be integrated out exactly, giving the probability of a trajectory as $P[x] \propto e^{-\frac{1}{2D}\int dt\,(\partial_t x - f(x))^2}$: the Lagrangian form promised above, in which the most probable path extremizes a classical action.

Exercise 6.1 (The Ornstein–Uhlenbeck process and the FDT). (a) Fill in the noise average leading to (6.7).

(b) For the linear drift $f(x) = -kx$ (the Ornstein–Uhlenbeck process), compute the stationary correlation and response functions, either by Fourier transforming the quadratic action or by solving the SDE directly: $C(\tau) = \frac{D}{2k} e^{-k|\tau|}$ and $R(\tau) = \theta(\tau)\, e^{-k\tau}$.

(c) Verify the fluctuation–dissipation theorem $TR(\tau) = -\partial_\tau C(\tau)$ for $\tau > 0$, with $T = D/2$.

(d) Explain, from the discrete-time construction, why $\langle \tilde{x}(t)\tilde{x}(t')\rangle = 0$ and why R is causal.

SOLUTION, p. 619

6.2 Double descent as a dynamical phase transition

As a complete worked application of the formalism, we describe recent work of Li and Goldenfeld (Li & Goldenfeld 2026), which uses dynamical mean-field theory to analyze the double-descent phenomenon and classifies it as a *dynamical phase transition*, all within the simplest possible setting: a teacher–student model of linear regression. Even this very simple model requires a long calculation, but the paper is quite readable despite the overhead. The headline: in the under-parametrized regime the optimization process is ergodic and relaxes to a finite-error solution, while the over-parametrized regime is a *non-ergodic* phase with genuinely non-equilibrium dynamics — an initial memorization stage leaves a persistent memory of the initial conditions and confines the dynamics to a subspace around the initialization. The diagnostic is the *response function* $R(t, t')$ of Section 6.1 — how a perturbation at time t' affects the model's state at time t; the same object appears as a Green function in quantum mechanics, signal analysis, and information theory. In the under-parametrized regime the response function decays: no memory. In the over-parametrized regime it saturates at a finite value: persistent memory of initialization. Consistently, the first regime obeys the fluctuation–dissipation theorem (Kubo 1957; Kubo 1966; Goldenfeld 2018), while in the second the FDT breaks — the standard warning light, flagged in Section 6.1, that a system has gone out of equilibrium. The model here is quite simple, and the authors propose that the frameworks here may generalize across architectures, with different universality classes based on the variations of the task, proposing random feature models with multiplicative (state-dependent) noise as an extension, with the hopes of eventually attaining an RG framework for interpreting phase transitions in neural networks.

6.2.1 Model definition and formalism

Notation. We transcribe the paper into the conventions of these notes: P trainable parameters, n training samples, learning rate η, teacher weights w^*, student weights $\hat{w}$ (with response fields $\tilde{w}$), and the *overparametrization ratio*

$$\nu := P/n$$

as control parameter — the reciprocal of Section 2's load $\alpha = n/N$, so that the over-parametrized phase is $\nu > 1$. For readers consulting the paper, the dictionary is:

these notes	Li–Goldenfeld (Li & Goldenfeld 2026)
P (parameters)	d
n (samples)	N
$\nu = P/n$	$\alpha = d/N$
$w^*, \hat{w}, \tilde{w}$	$\beta, \hat{\beta}, \tilde{\beta}$
η (learning rate)	ζ

The batch-inclusion probability γ, the ridge λ, and the label-noise variance D are as in the paper.

Explicitly, the student and teacher share the same architecture

$$y^\mu = \frac{1}{\sqrt{P}} \sum_{i=1}^{P} w_i^* x_i^\mu + \epsilon^\mu$$

where P is the number of model parameters, $\mu = 1, \dots, n$ indexes the dataset with inputs x^μ, the w_i^* are the teacher's weights, and ϵ^μ is label noise. The thermodynamic limit takes $n \to \infty$ and $P \to \infty$ with the overparametrization ratio $\nu = P/n$ — the paper's **complexity parameter** — held *fixed*. The three main regimes are now definable as under-parametrized: $\nu < 1$, near-criticality: $\nu \approx 1$, and over-parametrized: $\nu > 1$. We note their optimization procedure, which is SGD with both regularization controlled by λ (set to zero for most of the work) and batching controlled by the binary indicator variable s_μ which is drawn for each point in the dataset with probability γ,

$$\hat{w}_i(t+1) = \hat{w}_i(t) - \eta \frac{\partial \left[\frac{1}{2} \sum_{\mu=1}^{n} s^\mu \left(\hat{y}^\mu - y^\mu \right)^2 + \frac{\lambda}{2} \|\hat{\boldsymbol{w}}\|_2^2 \right]}{\partial \hat{w}_i(t)} ,$$

From here they construct the MSRJD integral exactly as in Section 6.1; taking functional derivatives and averaging over the disorder (the dataset, the initialization, the batch selection, and the label noise) yields the correlation and response functions that are the main objects of study. We skip most of the details, but it is worth displaying the starting point — the dynamical partition function for gradient flow, equal to one just as promised in Section 6.1:

$$Z_{\rm dyn} = \int \mathcal{D}\hat{\boldsymbol{w}}(t) \prod_{i=1}^{d} \delta\Big[-\partial_t \hat{w}_i(t) - \lambda \hat{w}_i(t) + j_i(t) \\ - \sum_{\mu=1}^{n} s^{\mu}(t) \frac{x_i^{\mu}}{\sqrt{d}} \Big(\sum_{k=1}^{P} \frac{x_k^{\mu}}{\sqrt{d}} \big(\hat{w}_k(t) - w_k^* \big) - \epsilon^{\mu} \Big) \Big] = 1 \,.$$

We want to highlight the general structure of this equation as a path integral over an action with sources coupled to both the "physical" parameter fields and the response fields,

$$Z_{\rm dyn}[\mathbf{j}, \tilde{\mathbf{j}}] = \Big\langle \int \mathcal{D}\hat{\boldsymbol{w}}(t) \mathcal{D}\tilde{\boldsymbol{w}}(t) \exp\Big(S_{\rm dyn} + \sum_{i=1}^{P} \int \tilde{j}_i(t) \hat{w}_i(t) dt \\ + \sum_{i=1}^{n} \int j_i(t) \tilde{w}_i(t) dt \Big) \Big\rangle,$$

which is a very common object to see in statistical field theory! In general $S_{\rm dyn}$ will hold all of the microscopic details of the theory, and this is a unique case with a fairly tractable model. In general we are not so lucky. The expectation value denoted by angled brackets is averaging over the input dataset $\{\mathbf{x}^{\mu}\}_{\mu=1}^{n}$, the initial condition $\hat{\boldsymbol{w}}(0)$, the stochastic sampling $s_{\mu}(t)$, and the added label noise $\{\epsilon^{\mu}\}_{\mu=1}^{n}$. For comedic relief we also write their full dynamical function integrated over quenched disorder:

$$\begin{aligned} Z_{\rm dyn} \propto \int \mathcal{D}\mathcal{B}(t) \int \mathcal{D}\mathcal{Q}(t,t') \exp\Big(& - i\tilde{\boldsymbol{w}} \partial_t \hat{\boldsymbol{w}} - \lambda P \, Q_2(t,t) \\ & - P \, Q_1 \cdot \hat{Q}_1 - P \, Q_2 \cdot \hat{Q}_2 - P \, Q_3 \cdot \hat{Q}_3 - P \, Q_4 \cdot \hat{Q}_4 \\ & - \frac{n}{2} \log\det \begin{pmatrix} \gamma Q_2^{\top} - \gamma Q_4 + \delta & \gamma^2 Q_3 \\ Q_1 + D & \gamma Q_2 - \gamma Q_4^{\top} + \delta \end{pmatrix} + \tilde{\boldsymbol{j}} \hat{\boldsymbol{w}} + \boldsymbol{j} \tilde{\boldsymbol{w}} \\ & + \iint dt \, dt' \sum_k \Big[\big(\hat{w}_k(t) - w_k^* \big) \hat{Q}_1(t,t') \big(\hat{w}_k(t') - w_k^* \big) \\ & \qquad + i\hat{w}_k(t) \hat{Q}_2(t,t') \tilde{w}_k(t') + i\tilde{w}_k(t) \hat{Q}_3(t,t') \, i\tilde{w}_k(t') \Big] \\ & + \int dt \sum_k i\tilde{w}_k(t) \hat{Q}_4(t) w_k^* \Big), \end{aligned}$$

where $\mathcal{D}\mathcal{B}(t)$ abbreviates the measure $\mathcal{D}\hat{\boldsymbol{w}}(t) \, \mathcal{D}\tilde{\boldsymbol{w}}(t)$ over the dynamical fields, and the new measure $\mathcal{D}\mathcal{Q}(t,t')$ is

$$\mathcal{D}\mathcal{Q}(t,t') = \left(\frac{P}{2\pi} \right)^4 \mathcal{D}Q_1(t,t') \, \mathcal{D}\hat{Q}_1(t,t') \, \mathcal{D}Q_2(t,t') \, \mathcal{D}\hat{Q}_2(t,t') \\ \times \mathcal{D}Q_3(t,t') \, \mathcal{D}\hat{Q}_3(t,t') \, \mathcal{D}Q_4(t) \, \mathcal{D}\hat{Q}_4(t) \,.$$

Each of the $\mathcal{Q}$ variables represents a particular correlation function between two variables.

6.2.2 Overview of results via training and generalization error

They define the generalization error as a mean squared error (MSE)

$$\varepsilon_g(t) = \langle \left(\frac{1}{\sqrt{P}} \sum_{i=1}^{P} x_i^0 [\hat{w}_i(t) - w_i^*] - \epsilon^0 \right)^2 \rangle$$

After taking the thermodynamic limit, the generalization error is expressed in terms of the equal time correlator $C(t,t)$ determined from the DMFT equations and D, the noise variance:

$$\varepsilon_g(t) = C(t,t) + D$$

They find two forms of this equation and propose a third, a scaling relation near the critical point. At $\lambda = 0$ for $\nu < 1$, the asymptotic error is:

$$\varepsilon_g(t \to \infty) = D(\gamma\chi + 1) = \frac{D}{1-\nu}\,.$$

Note the pole as criticality is approached. The quantity $\chi = \int_0^\infty R(\tau)\, d\tau$ is the **susceptibility**; it is well-defined only when the response function decays, i.e. only in equilibrium. Its appearance here signals that in this phase the response to perturbations of the initial conditions really does decay: asymptotically, the system carries no memory of its initialization.

The *non-ergodic phase* with persistent memory is captured by the asymptotic generalization error depending explicitly on the retention of the initial condition $C(0,0)$ and still showing sensitivity to $\nu \to 1$ analogous to the $\nu < 1$ regime near criticality:

$$\varepsilon_g(t \to \infty) = \frac{(\nu - 1)\, C(0,0)}{\nu} + \frac{D\nu}{\nu - 1}\,,$$

where fully, $C(0,0) = \langle \hat{w}^2(0) \rangle + r$. The term $\langle \hat{w}^2(0) \rangle$ is the variance of the initialization and r is the mean-squared norm of the ground-truth parameters per dimension. Note that here, the response function does not decay, because we carry memory of the initial conditions. It saturates at $R(\tau \to \infty) = \frac{\nu - 1}{\nu}$, which they call the *ergodicity-breaking degree*, quantifying the persistent memory of a perturbation applied to the initialization.

They also show similar trends with the training error, namely for $\nu < 1$ the error is asymptotically finite, as the model is still attempting to approximate the solution, while for $\nu > 1$ the training error vanishes as initialization-dependent memorization ensures the model always performs accurately on the training data. They write training error as

$$\lim_{t \to \infty} \varepsilon_t(t) = (\gamma\chi + 1)^{-2}\, \varepsilon_g(t \to \infty)\,.$$

For $\nu < 1$ it simplifies to $D(1 - \nu)$, while again, for $\nu > 1$, the system is no longer at equilibrium and the susceptibility $\chi = \int_0^\infty R(\tau) d\tau$ diverges, which drives the training error to zero.

To characterize the critical behavior of the model near $\nu \approx 1$, they calculate the critical exponents (σ, ϕ) and propose a scaling function. The first part is relatively straightforward, where from Eqs. (6.18) and (6.19), one finds $\varepsilon_g(t \to \infty)$ diverges from both sides $\nu \to 1^{\pm}$. This suggests the scaling $\varepsilon_g(t \to \infty) \sim |1-\nu|^{-\sigma}$ with an exponent $\sigma = 1$ in DMFT. We remind the reader that these are asymptotic long-time steady-state results and both training and generalization error are time-dependent. Since the saturation time is ν-dependent, a general feature of phase transitions is a dynamic scaling data collapse should be present near the transition (Goldenfeld 2018). They make this argument and propose a scaling ansatz of form:

$$\varepsilon_g \approx \frac{D}{|1-\nu|^{\sigma}} F\left(\frac{|1-\nu|^{\phi}(\gamma t)}{4}\right),$$

where they introduce a new critical exponent ϕ and write the universal scaling function $F(z)$, and γ is still the probability of samples being included. They make two observations about the form of F near and at the critical point. First, near the point, $\nu \to 1^{\pm}$ corresponds to $z \to 0$, they argue the singularity in front must be absorbed into F and it takes the asymptotic form $F(z) \sim z^{\sigma/\phi}$. Second, at $\nu = 1$, since the generalization error grows in time instead of saturating, $\epsilon_g(t) \sim (\gamma t)^{\sigma/\phi}$. The exponent ϕ therefore characterizes this growth in training time. Notably, the form of Eq. (6.21) implies a characteristic time scale, which they denote $\tau(\nu) = (\gamma|1-\nu|^{\phi})^{-1}$ to describe convergence in the scaling of the error. Since the error grows, we would expect this to diverge as $\nu \to 1$, which it does, and is an iconic signature of a phase transition, called **critical slowing down**. In MC calculations for lattice systems for example, this shows up in the autocorrelation time increasing, and requiring more time to get uncorrelated samples as we approach the critical point. A great deal of energy goes into designing methods to circumvent this slowing down when attempting to measure precise values of critical exponents using MC techniques, and motivates the choice of entirely different paradigms like RG flow or conformal bootstrap where applicable (Goldenfeld 2018).

Exercise 6.2 (Double descent in the lab). Simulate the model of this subsection: teacher $w^* \sim \mathcal{N}(0, r\, I_P)$, inputs $x^{\mu} \sim \mathcal{N}(0, I_P)$, labels $y^{\mu} = w^* \cdot x^{\mu}/\sqrt{P} + \epsilon^{\mu}$ with noise variance D; student $\hat{w}$ trained by the update (6.11) with ridge $\lambda = 0$ and full batches ($\gamma = 1$), from $\hat{w}(0) \sim \mathcal{N}(0, \sigma_0^2 I_P)$. Take $P = 200$ and $\nu = P/n \in \{0.5,\, 0.9,\, 1.1,\, 2\}$.

(a) Plot training and generalization error against time, and verify the asymptotics (6.18) and (6.19) — including the divergence of *both* branches as $\nu \to 1$: double descent, resolved in time.

(b) Probe ergodicity breaking: run two clones from initializations differing by a small random $\delta\hat{w}(0)$, and track the normalized separation $|\Delta\hat{w}(t)|^2/|\Delta\hat{w}(0)|^2$. Verify that it decays to zero for $\nu < 1$ but saturates for $\nu > 1$ — at the predicted plateau $(\nu-1)/\nu$, the paper's ergodicity-breaking degree, measured in twenty lines of code.

(c) Near $\nu = 1$, extract the relaxation time of $\varepsilon_g(t)$ and check for the critical slowing down $\tau \sim |1-\nu|^{-\phi}$ of Eq. (6.21).

SOLUTION, p. 620

6.3 Dynamical mean-field theory (DMFT)

Dynamical mean-field theory, of which Section 6.2 was one instance, is a general strategy for reducing the number of effective degrees of freedom in an interacting dynamical system: one follows a *single* representative degree of freedom, moving in a fluctuating background that stands in for all the others — and one demands that the statistics of the background be determined, self-consistently, by the statistics of the representative trajectory itself. This is the same trichotomy as every other mean field in these notes (couple through averages; argue fluctuations are controlled; close self-consistently), upgraded from scalars and measures to two-time functions. An excellent intuition-building tutorial in a biophysics context is (Blumenthal 2025), and (Cugliandolo 2023) is a broad modern review. In quantum many-body theory the same idea has been enormously effective for otherwise intractable systems (Georges et al. 1996; Kotliar et al. 2006). Below we first clarify what *self-consistency* means in a dynamical setting, then survey the recent deep-learning literature; a natural technical entry point for the latter is the out-of-equilibrium DMFT of the perceptron (Agoritsas et al. 2018).

6.3.1 Self-consistency

The term *self-consistency* describes the demand that the predictions of a theory agree with the approximations used to obtain them. A simple example, taken from (Stapmanns et al. 2020), makes the issue vivid (and is worked in detail in Exercise 6.3). Consider the stochastic process $dx(t) = f(x(t))\,dt + dW(t)$, with W a Wiener process (so that $\dot{W}$ is the white noise of Section 6.1), and suppose $f(x_0) = 0$ with $f'(x_0) < 0$: a stable fixed point of the noiseless dynamics. One can expand around it, writing $x(t) = x_0 + \delta x(t)$, to obtain the approximation

$$\frac{d}{dt}\delta x(t) = f'(x_0)\delta x(t) + \xi(t) + \mathcal{O}(\delta^2 x(t))\,,$$

where ξ is also centered gaussian white noise. The mean and variance of these fluctuations can be computed straightforwardly via Fourier transform, and in the temporal basis the variance is $\langle \delta x^2 \rangle = \frac{D}{-2f'(x_0)}$, however using these results with the Taylor expansion of $f(x)$ to second order gives a correction to the mean

$$\langle x \rangle \approx x_0 + \frac{1}{2}\frac{f''(x_0)}{f'(x_0)}\frac{D}{2f'(x_0)}$$

which is *inconsistent* with the value x_0 assumed for the mean when constructing the approximation: the noise, filtered through the curvature of f, shifts the very point we expanded around.

Exercise 6.3 (Self-consistency in the smallest possible example). (a) Derive the stationary variance $\langle \delta x^2 \rangle = D/(-2f'(x_0))$ of the linearized dynamics (compare Exercise 6.1(b)).

(b) Carrying the Taylor expansion of f to second order, derive the quoted correction to the mean.

(c) Propose the self-consistent repair: expand instead around a point x_*, chosen so that the *effective* drift vanishes, $f(x_*) + \frac{1}{2} f''(x_*) \left\langle \delta x^2 \right\rangle_* = 0$, with $\left\langle \delta x^2 \right\rangle_* = D/(-2f'(x_*))$ determined simultaneously. Note the structure: two coupled equations, each an input to the other — a miniature of the Curie–Weiss condition of Section 5.1, and of the DMFT equations to come.

SOLUTION, p. 620

A framework for resolving this, and also attempting to enforce that the small parameter we expand about is actually the expansion parameter involves using *n-particle irreducible effective actions* (nPI). A full discussion of this is well beyond the scope of this document, but is covered very well in a pedagogical introduction (Berges 2004), where the main idea is enforcing self-consistency via increasingly multi-local sources that generate n-point correlation functions. The first section emphasizes that performing standard perturbation theory with an anharmonic oscillator yields so-called *secular* terms which are powers of time t that appear next to the actual expansion parameter ϵ, and using the nPI framework effectively resums these terms properly at each order in ϵ. The 1PI approach is directly related to a local approximation with mean field theory, while in the 2PI approach, the object determined self consistently is the 2-point function, which in the local approximation is related to the local propagator and is the basis of dynamical mean field theory.

6.3.2 Recent applications in deep learning

Fortunately, one rarely needs the general nPI machinery in practice. The working route — structurally the same trick by which large-N models in physics (*e.g.* the G–Σ formulation of the SYK model) are solved — is to insert bilocal *collective fields* for the correlation and response functions via delta functions, exactly as the Q's of Section 6.2, and then to evaluate their path integral by saddle point in a suitable $N \to \infty$ limit. This is how the deep-learning DMFT of (Bordelon & Pehlevan 2022; Bordelon et al. 2024) is constructed.

A brief, opinionated tour of the literature, oldest first.

Spin-glass origins. The mean-field dynamics of disordered systems is where all of this machinery was forged. Amit, Gutfreund, and Sompolinsky (Amit et al. 1985) brought spin-glass methods into neural networks (in the guise of Hopfield-style associative memories), and Sompolinsky, Crisanti, and Sommers (Sompolinsky et al. 1988) derived the DMFT of random recurrent networks, discovering a transition to chaos. The structural point to retain: an ordered system like the Ising magnet has one global coupling $J \sum_{\langle ij \rangle} \sigma_i \sigma_j$ and is already hard; a *disordered* system has random couplings $\sum_{ij} J_{ij} \sigma_i \sigma_j$, and it is precisely the all-to-all randomness that makes mean-field methods exact — the same structural feature that data-dependent couplings give learning problems, starting with the perceptron of Section 2.

The Bordelon–Pehlevan program. For deep networks trained by gradient descent at fixed dataset, the DMFT of (Bordelon & Pehlevan 2022) takes the infinite-width limit and finds that the order parameters are precisely the *feature kernels* $\Phi(t, s)$ and *gradient kernels* $G(t, s)$ — two-time generalizations of the NNGP kernel and NTK of Sections 3–4. The lazy regime of Section 4 is recovered when these kernels freeze; rich, feature-learning dynamics is their genuine evolution, computed self-consistently. (Fair warning: the notation is heavy — response fields and conjugate order parameters abound — but the structure is exactly MSRJD plus a saddle point.) The program now includes finite-width kernel fluctuations (Bordelon & Pehlevan 2023), hyperparameter transfer across *depth* (Bordelon et al. 2023), and the dynamical theory of neural scaling laws (Bordelon et al. 2024) that we meet again in Section 7.9.

The Mignacco–Urbani line. A complementary series of works treats the *data* as quenched disorder at proportional scalings of dimension and sample size, in the spirit of Section 2: DMFT for Gaussian-mixture classification (Mignacco et al. 2021), and a DMFT characterization of the effective noise of SGD at late times (Mignacco & Urbani 2022) — the dynamical refinement of the batch-noise discussion of Section 2.1.

Rigor. What began as a physics heuristic is increasingly a theorem: rigorous derivations of DMFT for first-order methods now exist (Celentano et al. 2021; Gerbelot et al. 2024), with a growing line of extensions — e.g. to adaptive Langevin diffusions, with propagation-of-chaos guarantees and convergence of the linear response (Fan et al. 2025). A recent tour de force in this direction (Montanari & Urbani 2025) solves the dynamics of a two-layer network at scale — a long calculation, but a well-organized paper — finding: the emergence of a slow timescale tied to the growth of the network's complexity (in the Gaussian/Rademacher sense); an inductive bias toward low complexity when the initialization has low complexity; a dynamical *decoupling* of the feature-learning and overfitting regimes; and a non-monotone test error, with a "feature unlearning" regime at very late times.

The connective tissue of this whole section, worth stating once more: the MSRJD path integral is to training dynamics what the Gibbs measure was to Bayesian learning — the object on which every physics technique (saddle points, disorder averages, collective fields, perturbation theory) can be brought to bear.

7. Scaling laws

In this final section we return to the theme of Section 1 and discuss the *neural scaling laws*. Over many orders of magnitude, the test loss of large models falls as a clean power law in each resource spent on it: parameters, data, and compute. These laws were found by fitting curves to measurements, and they have proved to be a remarkably simple, robust, and consequential empirical observation, one that any successful model of deep learning should be able to capture. Many models do capture their form, and several go further and make quantitative predictions for the exponents, based on features of the data distribution.

The organizing question of the section is: *where do the exponents come from?* We first assemble the empirical picture (Sections 7.1–7.2): the three laws, what each one holds fixed, and the joint laws that govern compute-optimal training, where the Kaplan–Chinchilla discrepancy lived and was resolved. We then turn to theory. Section 7.3 states the mechanism common to every proposal — power laws in the data produce power laws in the loss — and Section 7.4 surveys the main proposals in one table. The rest of the section works out four rows of that table, in order of increasing machinery: the frequency statistics of skills (Section 7.5, run as a derive-it-yourself exercise, with its payoff for "emergent capabilities" in Section 7.6); the geometry of data and the spectra of kernels (Section 7.7); and solvable large-N field theories, static and then dynamical (Sections 7.8–7.9). We close with the newest measurements, for which there is no theory yet: post-training and test-time scaling (Section 7.10).

Notation. We keep the conventions of these notes: P is the parameter count and n the dataset size. Be warned that the scaling-law literature writes N for our P and D for our n (their N is *not* our width!), so the famous exponents α_N, α_D are our α_P, α_n. For reading the sources:

these notes	**scaling-law literature (Kaplan et al. 2020; Hoffmann et al. 2022; Michaud et al. 2023; Maloney et al. 2022)**
P (parameters)	N
n (dataset size)	D
α_P, α_n (exponents)	α_N, α_D
m (quanta learned, Section 7.5)	n

Note also that in this section α (no subscript) denotes the Zipf exponent of the quantization model, not the load of Section 2.

7.1 The empirical laws

Kaplan et al. (Kaplan et al. 2020) established, for autoregressive language models, that when a model is not bottlenecked by the other resources its test loss obeys

$$L(P) \propto P^{-\alpha_P}\,, \qquad L(n) \propto n^{-\alpha_n}\,, \qquad L(C) \propto C^{-\alpha_C}\,,$$

where P is the number of (non-embedding) parameters, n the dataset size in tokens, and C the training compute in FLOPs. The measured exponents were $\alpha_P \approx 0.076$, $\alpha_n \approx 0.095$, and $\alpha_C \approx 0.050$, with the trends holding over up to seven orders of magnitude. Three points of bookkeeping are usually left implicit. They are worth making explicit before asking where the laws come from.

Why FLOPs, and why $C \approx 6Pn$. Compute is measured in floating-point operations because FLOPs are the only measure of training effort that does not depend on how the job is run. Wall-clock time depends on the hardware and the parallelism. The number

of training *steps* depends on the batch size: the same n tokens can be one giant batch or a million small ones, and below the "critical batch size" (which (Kaplan et al. 2020) measure separately) the loss depends only on the number of tokens processed. FLOPs count the arithmetic itself, which is also what energy and money are proportional to. The standard estimate is

$$C \;\approx\; 6\,P\,n\,.$$

Per token, the forward pass costs about $2P$ FLOPs: essentially every parameter sits in some matrix multiplication and is used once, for one multiply and one add. The backward pass costs about twice the forward pass, since each layer needs the gradient with respect to its activations and the gradient with respect to its weights, two matrix multiplications of the same size as the forward one. That gives $6P$ FLOPs per token, times n tokens. The $\approx$ absorbs attention (whose per-token cost grows with context length rather than parameter count), embeddings, and optimizer overhead.

The single-epoch assumption. The estimate (7.2) assumes that every token processed is a fresh token, so that "tokens processed" and "dataset size" are the same number. This was true of the runs on which the laws were measured, which used well under one epoch of the corpus. Conceptually the two are different resources. The *statistical* resource is the number of distinct tokens n; the *computational* resource is the number of tokens processed, $T = E\,n$ over E epochs, so that in general $C \approx 6PT$. Fixing n and growing C by repetition buys optimization, not statistics: the loss converges to the n-limited floor and then overfits. In the language of Section 2.1, more epochs means larger β at a fixed quenched sample, while fresh data changes the quenched free energy itself. Empirically the one-epoch idealization is forgiving: for up to about four epochs, repeated tokens are nearly as valuable as fresh ones, after which their value decays quickly (Muennighoff et al. 2023). This matters increasingly, now that frontier training is data-constrained.

The compute law is not independent. Since $C \approx 6Pn$ depends only on the other two resources, how can compute ever be the unbottlenecked one? The answer is that $L(C)$ is measured by a different protocol. It is the lower envelope

$$L(C) \;=\; \min_{P,\,n\,:\,6Pn\,=\,C} L(P,n)\,,$$

traced out by training many model sizes and keeping the best loss at each value of compute. On this curve compute is the binding constraint by construction, and "not bottlenecked" means "optimally allocated." This makes α_C a derived exponent. Minimizing an additive joint law $L = A\,P^{-\alpha_P} + B\,n^{-\alpha_n}$ at fixed Pn balances the two terms, and gives

$$\frac{1}{\alpha_C} \;=\; \frac{1}{\alpha_P} + \frac{1}{\alpha_n}\,, \qquad P \propto C^{\,\alpha_n/(\alpha_P+\alpha_n)}\,, \quad n \propto C^{\,\alpha_P/(\alpha_P+\alpha_n)}\,.$$

The harmonic sum makes α_C smaller than either single-resource exponent, since a factor of C must be split between growing P and growing n. Kaplan's values give $\alpha_C \approx 0.042$, against the measured 0.050: roughly consistent, with the gap traceable to the form of the fit and to early-stopping details.

Universal form; data-dependent exponents. The same power-law form holds far beyond language. Henighan et al. (Henighan et al. 2020) trained autoregressive transformers on many kinds of data and found power laws everywhere, but with exponents that depend on the data:

modality	α_P	α_C
language	0.070	0.048
images 32×32	0.13	0.10
images 8×8	0.24	0.19
video	0.24	0.14
math (procedural)	0.16	0.17

("Math" here means procedurally generated problems, in the style of the DeepMind mathematics dataset. They are emitted by a known program, so the supply is effectively unlimited. Remember this case: it is the one modality whose data distribution is known in closed form.) The exponents vary by a factor of 3–5 across modalities. Architecture, by contrast, barely matters: within reasonable architectures, changing the shape (depth vs. width at fixed size) or even the family moves the constants and the plateaus, not the slopes (Kaplan et al. 2020). This asymmetry is the central clue of the section: *whatever sets the exponent lives in the data*. Section 7.7 makes it quantitative.

7.2 Joint laws, bottlenecks, and the Chinchilla revision

Each single-resource law assumes the other resources are unlimited. Joint fits describe the crossover to being bottlenecked, with the loss governed by whichever resource is scarcest. Two parametric forms dominate the literature. Kaplan et al. used

$$L(P, n) = \left[\left(\frac{P_c}{P} \right)^{\alpha_P/\alpha_n} + \frac{n_c}{n} \right]^{\alpha_n},$$

while the Chinchilla analysis of Hoffmann et al. (Hoffmann et al. 2022) refit with the additive ansatz

$$L(P, n) \;\approx\; E + A\,P^{-0.34} + B\,n^{-0.28}\,.$$

The additive form is easy to read. $E \approx 1.69$ is the irreducible entropy of text, the noise floor that no amount of scale removes; the two deficits decay independently; and the bottleneck is simply the larger deficit. Both forms reduce to the single-resource laws (7.1) in the appropriate corners.

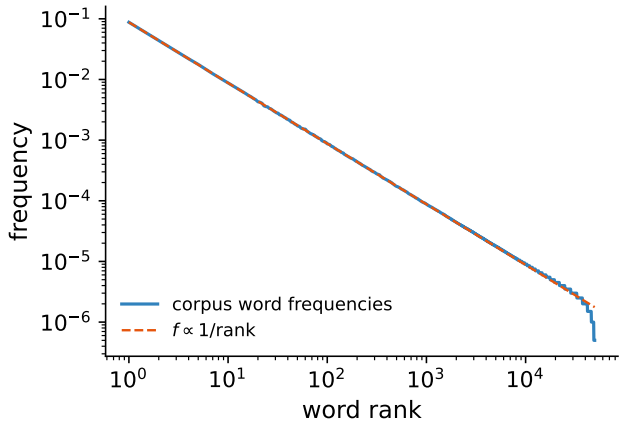
The difference between the fits mattered a great deal in practice, because of what they say about the compute-optimal allocation in (7.3). Kaplan et al.'s analysis gave $P \propto C^{0.73}$ and $n \propto C^{0.27}$: with more compute, grow the model. For two years the field trained ever-larger models on comparatively little data. Chinchilla's fit gives $P \propto C^{0.5}$ and $n \propto C^{0.5}$: grow both equally, or as a rule of thumb about 20 tokens per parameter. The prediction was tested directly. At the compute budget of DeepMind's 280B-parameter Gopher, the new rule said to train a model four times smaller on four times more data, and the resulting 70B-parameter Chinchilla beat Gopher across the board. (Both models, and the law, are named in DeepMind's animal series.) Chinchilla's allocation is now the accepted baseline. The discrepancy was later dissected by Porian et al. (Porian et al. 2024): Kaplan's fit was skewed by three small-scale artifacts — how last-layer and embedding compute were counted, over-long warmup, and optimizer settings that were not retuned with scale — and with these corrected, the two experiments agree. One coda: frontier models now deliberately *over*-train relative to Chinchilla, on many more than 20 tokens per parameter, because a smaller model at equal loss is cheaper at inference time, a cost the training-compute-optimal analysis does not see.

With the empirical picture assembled, here is what a theory of (7.1) owes us: (i) why power laws at all, over so many decades; (ii) the values of the exponents (why so small? why do they depend on the modality but not on the architecture?); (iii) relations between exponents (is α_n determined by α_P?); and (iv) when the laws break. The models surveyed below each answer a subset.

7.3 Where do the laws come from?

The basic answer, common to every proposal surveyed below, is that the loss curve reads off the coarse distribution of structure in the data. Power laws go in with the data and come out in the loss.

The prototype of "power laws in" is *Zipf's law*: rank the words of a large corpus by frequency, and the frequency of the word of rank k falls off as $f_k \propto 1/k$, over about four decades of rank (Figure 7). It is a scale-free tail of ever rarer structure, and it is not special to words. Sequences of n consecutive words (n-grams, the unit of pre-neural language modeling) are Zipf-like at every n, with slopes that flatten as n grows. Facts, meaning discrete pieces of world knowledge attested in text, inherit the heavy-tailed popularity of the entities they mention, and a model's ability to answer a factual question tracks the number of training documents that support it (Kandpal et al. 2023). The covariance and kernel spectra of text and image data are measured power laws (Sections 7.7–7.8). A learner working through such data never runs out of structure to absorb. It moves ever further down the tail, at ever slower returns. That is, qualitatively, why the loss is a power law rather than an exponential (there is structure at all scales) and why it does not simply plateau (the tail does not end).



Zipf's law, illustrated on a synthetic corpus: 2×10^6 tokens drawn from a $1/k$ distribution over a vocabulary of 5×10^4 words, with the empirical rank–frequency curve against the $f \propto 1/\text{rank}$ guide. The curve is a consistency check rather than data, since the words were sampled from a Zipf distribution to begin with. Real corpora look the same over roughly four decades, with deviations at both ends: the Zipf–Mandelbrot correction at low rank, and a steeper tail beyond about 10^4 words.

The exponents of these tails differ, and the differences matter, because in every model below the loss exponent is inherited from the data exponent. Pure Zipf is in fact a marginal case. Write the tail of skill frequencies as $p_k \propto k^{-(\alpha+1)}$. The quantization model of Section 7.5 gives a loss exponent equal to α, and exact Zipf ($f_k \propto 1/k$, that is, $\alpha = 0$) is the boundary at which the tail sum decays only logarithmically and power-law scaling degenerates. Read this way, the measured $\alpha_P \approx 0.076$ for language says that its effective skill distribution is $p_k \sim k^{-1.08}$, barely steeper than pure Zipf. Language scaling is slow because its structure sits just above the marginal $1/k$ tail. Image data has steeper tails and scales faster; compare the modality table of Section 7.1.

Where does the theory stand? The single-resource laws have been derived from this mechanism by many models of quite different flavor, and a few models derive multi-resource laws; the survey below keeps score. Three things are missing. First, no theory computes the data's exponents from first principles. Every model converts measured data statistics into loss curves, and the statistics themselves are an empirical input. This is true even for procedurally generated math, where the generating program is known exactly but the computation has not been carried out. Second, there are no joint laws with genuine feature learning. The best dynamical derivations live in lazy, random-feature settings (Section 7.9), while in solvable cases feature learning is known to improve exponents for structured targets, so the corrections need not be small. Third, the bridge from the smooth loss to capabilities is missing. Benchmark performance can jump within a decade of scale while the loss glides smoothly (Wei et al. 2022); this has been contested as an artifact of the metric (Schaeffer et al. 2023), and the quantization model of Section 7.6 gives it a sharper reading. A fourth gap is newer: the post-training and test-time scaling laws of Section 7.10 have no theory at all.

7.4 A survey of theoretical proposals

The table below organizes the most prominent derivations of neural scaling laws by four questions: what models the *data* (equivalently, what a "feature" is and how features are distributed); what models the *network*, if anything; whether actual *exponents* are predicted, and in terms of what; and whether *multi-resource* laws are obtained.

proposal	**data model / "features"**	**NN framework**	**exponents?**	**multi-resource?**
quantization; Michaud et al. (Michaud et al. 2023)	discrete skill "quanta," Zipf-used, learned in order	none (a counting argument)	in terms of the Zipf exponent α; plus the relation $\frac{1}{\alpha_n} = \frac{1}{\alpha_P} + 1$	separate P, n, S laws
data manifold; Sharma–Kaplan (Sharma & Kaplan 2020)	smooth target on a d-dimensional manifold	piecewise-linear interpolation (function class only; no training)	yes: $\alpha_P \approx 4/d$	no (P only)
percolation; Brill (Brill 2024)	latent subtask clusters at criticality, with power-law cluster sizes	none: a generic nonparametric learner + regression estimates	from percolation critical exponents	two regimes; not joint
kernel eigenlearning; Bahri et al. (Bahri et al. 2024), Bordelon–Canatar–Pehlevan (Bordelon et al. 2020; Canatar et al. 2021; Simon et al. 2023)	power-law kernel spectra + target coefficients	**frozen kernel**: infinite-width NTK, or GP/Bayesian	in terms of the spectral exponents	partially: resolution- vs. variance-limited regimes
solvable field theory; Maloney–Roberts–Sully (Maloney et al. 2022)	latent power-law covariance, extended by random features	linear model on **fixed random features** (frozen kernel); large-N RMT, statics	in terms of the data's spectral exponent	yes: joint $L(P, n)$, with duality $P \leftrightarrow n$
DMFT; Bordelon–Atanasov–Pehlevan (Bordelon et al. 2024)	power-law (source/capacity) spectra	DMFT of gradient-flow training on **fixed random features** (frozen kernel; dynamics)	yes, including the time exponent	fully joint $L(t, P, n)$ + compute frontier

Five remarks for reading the table. First, everyone agrees on the origin. In every row the loss exponent is inherited from a power-law tail in the data, whether of skill frequencies, cluster sizes, manifold volume, or covariance spectra. The rows differ in what the tail is a tail *of*, and in how much network machinery is retained. Second, and this deserves emphasis: *every row that models the network at all does so with a frozen kernel*. The eigenlearning row works with the infinite-width NTK or GP kernel of Sections 4.3 and 4.6; the two field-theory rows train only the readout of a fixed set of random features, which is a frozen kernel by construction (Exercise B.6(b)). The manifold row models only the network's function class, as a count of linear pieces, and never trains it; the quantization and percolation rows contain no network. So none of the current derivations involves feature learning, the very mechanism that Sections 5–6 argued is what makes finite networks differ from kernels. Third, equilibrium routes (Bayesian GP learning curves and their finite-width refinements) and dynamical routes (gradient flow, DMFT) give matching exponents wherever they overlap. This is the "two limits of one system" correspondence of Sections 2.1 and 6 at work again. Fourth, only the two field-theory rows currently predict joint laws, and only the dynamical one prices training time, deriving a Chinchilla-like compute-optimal frontier rather than assuming it. Fifth, no row computes the data's own exponent. The closest attempt is the percolation proposal, which derives power-law cluster-size statistics from a critical model of the data distribution; its two criticality regimes (power-law-distributed discrete subtasks, and a dominant data manifold) map onto the quantization and manifold rows respectively. We will not develop the percolation model further. The remaining subsections work through the other rows, in order of machinery.

Do the predicted exponents agree with experiment? Where the comparison has been made, the answer is encouraging but not yet sharp. The quantization model's authors extracted "quanta" from a small language model by clustering gradients, measured how often each is used in the training distribution, and found a power law whose exponent roughly matches the model's scaling exponent, as the model requires (Michaud et al. 2023); on the other hand, the model's parameter-free relation between α_P and α_n is violated by the measured ordering of Kaplan's exponents (Exercise 7.1(e)). The manifold prediction $\alpha_P \approx 4/d$ was confirmed in teacher–student experiments, where d and α_P can be measured independently, and tested more loosely on image classifiers and GPT-type language models (Sharma & Kaplan 2020); for language, taking $\alpha_P \approx 0.076$ at face value gives $d \approx 50$ (Exercise 7.2). The kernel predictions have been tested on standard architectures and datasets, with exponents computed from measured kernel spectra, and the predicted duality between the P- and n-exponents holds up (Bahri et al. 2024). The two field-theory rows take the data's spectral exponent as input. Their qualitative predictions — exponents independent of architecture, $\alpha_P \approx \alpha_n$, and equiparameterization as the compute-optimal rule — match the Chinchilla-era measurements, but a quantitative test needs the covariance spectrum of the training data, which is rarely reported for the large runs.

The honest summary is that these theories convert an exponent measured in the *data* into an exponent measured in the *loss*, and the conversion has held up wherever both have been measured.

7.5 The quantization model: exponents from the statistics of skills

The quantization model of Michaud, Liu, Girit, and Tegmark (Michaud et al. 2023) is a minimal model of scaling. It has no architecture, no optimizer, and essentially no mathematics beyond a tail sum, and it produces all three laws of (7.1), plus a sharp point of view on "emergent capabilities." The name is deliberately provocative: the proposal is that what a network learns comes in *discrete units*, dubbed *quanta*.

[4]

The assumptions, in the form we'll use:

- **QH1 (discreteness).** The computations and knowledge relevant to the prediction task decompose into enumerable *quanta*: basic skills or facts (*e.g.* "carry the one," "Paris follows 'the capital of France is'," ...). Each quantum is either fully learned or not learned at all, with no partial credit.
- **QH2 (ordered acquisition).** Networks learn quanta roughly in order of decreasing usefulness, so a given model has learned the first m quanta of a fixed sequence.
- **QH3 (Zipfian use).** Quantum k is *used* on a fraction

$$p_k = \frac{k^{-(\alpha+1)}}{\zeta(\alpha+1)}$$

of samples ($\alpha > 0$ a constant), and on those samples, knowing it reduces the loss by a constant Δ (independent of k).

Consequently, a model that has learned the first m quanta has expected loss

$$L_m = L_\infty + \Delta \sum_{k>m} p_k \,.$$

That is the entire model. Everything else is asking how m is limited by each resource.

Exercise 7.1 (★) (Three scaling laws from three assumptions). (a) Estimate the tail sum in (7.8) to show $L_m - L_\infty \propto m^{-\alpha}$.

(b) *Parameter-limited (data and training unlimited).* Suppose each quantum costs a constant capacity of c parameters to learn. Derive α_P in terms of α.

(c) *Data-limited (capacity and training unlimited).* Suppose a quantum is learnable only if it is used by at least τ training samples, for constant τ. Given n i.i.d. samples, which quanta are learnable? Derive α_n.

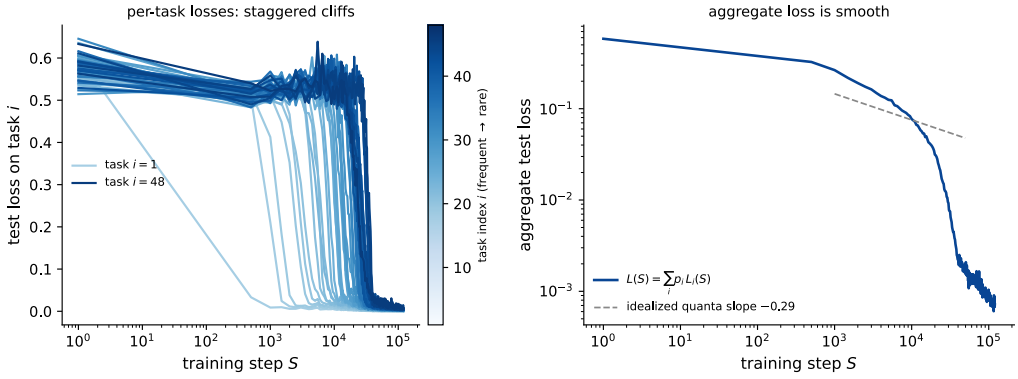
(d) *Training-limited, single epoch.* Suppose instead that learning quantum k requires τ gradient updates on batches containing it. After S steps of online training (fresh batches), derive α_S.

(e) *Falsifiability.* Parts (b)–(c) predict a parameter-free relation between α_P and α_n: write it down, and note which of the two exponents must be larger. Compare with the measured values quoted after (7.1). Something is wrong — what? List the assumptions you used and discuss which are most suspect. (This is a discussion question with no settled answer; see the solution for candidates.)

SOLUTION, p. 620

7.6 Emergence: smooth laws from staggered cliffs

The quantization model's deepest payoff is conceptual. Under QH1, the learning of any *single* quantum is a step function: a capability is absent, then abruptly present. Yet the aggregate loss (7.8) is a smooth power law. There is no contradiction. *A smooth law is a sum of many staggered cliffs*, each weighted by p_k and individually invisible in the aggregate, since quantum k contributes only the small fraction p_k of the loss. Michaud et al. sharpen this with a genetics metaphor. A test sample is *monogenic* if its loss is governed by a single quantum, so that its loss curve during training is a plateau followed by a cliff; it is *polygenic* if many quanta contribute, in which case its loss falls smoothly. Both kinds of per-token curves are observed in real LLMs. From this perspective, "emergent capabilities" (Wei et al. 2022) are neither magic nor mirage (Schaeffer et al. 2023): any benchmark dominated by a few quanta *must* show abrupt arrival, while any sufficiently aggregated metric *must* look smooth. Emergence is a property of what you measure, not (necessarily) of a phase transition inside the model.



Multitask sparse parity (Michaud et al. 2023): smooth scaling from staggered cliffs. Setup: $T = 48$ subtasks with Zipfian frequencies $p_i \propto i^{-(\alpha+1)}$, $\alpha = 0.4$; inputs are a one-hot task indicator (T bits) concatenated with 24 random ± 1 bits; the label is the parity of a fixed random 3-subset of the bits, the subset depending on the task. Model: one-hidden-layer ReLU MLP, width 512, MSE loss on ± 1 labels, trained online (fresh batches of 256; Adam, learning rate 2×10^{-3}, 1.2×10^5 steps; fixed seed). *Left:* test loss per subtask vs. training step. Each subtask is learned in an abrupt cliff, frequent tasks first. *Right:* the frequency-weighted aggregate $L(S) = \sum_i p_i L_i(S)$ is smooth; the dashed guide is the *idealized* quanta-model slope $-\alpha/(\alpha+1)$ from Exercise 7.1(d). The measured decay is visibly steeper, as discussed in the text.

Lab: multitask sparse parity. The entire mechanism fits in a laptop-scale experiment, shown in Figure 8; students are encouraged to reproduce and modify it (all details are in the caption). A network is trained online on T separate parity subtasks with Zipfian frequencies. The input is a one-hot task indicator concatenated with random bits, and the label is the parity of a small task-specific subset of the bits. Parity is a needle-in-a-haystack skill with no partial credit, *i.e.* an honest hand-built quantum. The result: each subtask's test loss is a staggered cliff (frequent tasks fall first), while the frequency-weighted aggregate loss decays smoothly. Note also the honest failure visible in the right panel. The measured aggregate decays *more steeply* than the idealized quanta slope $-\alpha/(\alpha+1)$ of Exercise 7.1(d), because the cliffs bunch up rather than spreading Zipf-like; the rare tasks are learned sooner than the threshold rule $S\,p_k \gtrsim \tau$ predicts. The subtasks share a hidden layer and an adaptive optimizer, so they are not learned independently, and assumption (iv) in the solution to Exercise 7.1(e) fails.

7.7 Exponents from geometry and from spectra

The quantization model takes the Zipf exponent α as *input*. The next two arguments produce exponents as *output*, from properties of the data.

7.7.1 Geometry: the data-manifold argument

Sharma and Kaplan (Sharma & Kaplan 2020) propose the simplest continuous mechanism. A network with ReLU activations computes a piecewise-linear function, so regression on a data manifold of intrinsic dimension d is a problem of *piecewise-linear interpolation*. A network with P parameters can afford $\sim P$ linear pieces; tiling a d-dimensional manifold with P pieces gives spacing $s \sim P^{-1/d}$; a linear patch approximates a smooth target to accuracy $\sim s^2$ (Taylor); and MSE is the *square* of the error. Hence

$$L(P) \sim (s^2)^2 \sim P^{-4/d}\,, \qquad \alpha_P \approx \frac{4}{d}\,.$$

Exponents are small because data manifolds are high-dimensional. The formula is testable, since d can be measured with intrinsic-dimension estimators, which is exactly what (Sharma & Kaplan 2020) do.

Exercise 7.2 (Exponents from the data manifold). (a) Fill in the derivation of (7.9), stating precisely where smoothness of the target is used.

(b) How does the answer change for mean-absolute-error loss?

(c) Taking the measured $\alpha_P \approx 0.076$ for language models at face value, estimate the effective dimension of the "manifold of text." Discuss whether the input dimension, the embedding dimension, or something else is being measured.

(d) What does this mechanism predict for the *data* exponent α_n, if n samples are what limits the resolution of the tiling?

SOLUTION, p. 621

7.7.2 Spectra: scaling laws from kernel eigenvalues

The second mechanism uses machinery we already own: the Bayesian kernel regression of Section 4.6, whose mode-filtering formula said that with n samples (and inverse temperature β), the modes of the GP kernel that get learned are those with $\lambda_k \gtrsim (n\beta)^{-1}$. Suppose the kernel spectrum and the target's decomposition are power laws,

$$\lambda_k \sim k^{-a} \quad (a > 1)\,, \qquad y_k^2 \sim k^{-b} \quad (b > 1)\,,$$

in a shared eigenbasis $\{\varphi_k\}$. The unlearned modes dominate the test error:

$$L(n) \;\sim \sum_{k\,:\,\lambda_k \lesssim (n\beta)^{-1}} y_k^2 \;\sim \sum_{k > k_*} k^{-b} \;\sim\; k_*^{1-b}\,, \qquad k_* \sim (n\beta)^{1/a}\,,$$

giving $L(n) \sim n^{-(b-1)/a}$: *the scaling exponent is a ratio of spectral decay rates*. Everything about the data and the architecture enters through two measurable power laws. This is the starting point of the "eigenlearning" analyses (Bordelon et al. 2020; Canatar et al. 2021; Simon et al. 2023), which sharpen (7.11) into precise learning curves (including the overfitting factor whose divergence produces the double-descent peak of Section 6). Bahri et al. (Bahri et al. 2024) organize the possibilities into a clean 2×2 taxonomy: *resolution-limited* regimes (exponents from spectra or geometry, as here and in (7.9)) versus *variance-limited* regimes (universal exponent 1), in each of P and n.

Exercise 7.3 (Spectral scaling laws, and quanta as eigenmodes). (a) Derive (7.11) from the mode-filtering formula of Section 4.6, and check that the crossover modes $k \sim k_*$ do not change the exponent.

(b) Show that the quantization model's data-scaling law is the special case $b = a$ of (7.11) under the dictionary: quanta = eigenmodes, Zipf law = spectrum, $a = \alpha + 1$. (Hint: in the quanta model, p_k plays *both* roles, setting the learning threshold *and* the contribution Δp_k to the loss, so the two power laws coincide.)

(c) In the quanta model, what would it mean for $b \neq a$? Invent a modification of QH3 that realizes it.

SOLUTION, p. 621

7.8 A solvable large-N field theory

Maloney, Roberts, and Sully (Maloney et al. 2022) supply what the arguments above lack: a *microscopic, exactly solvable* model in which the joint law $L(P, n)$ is computed, not modeled. The ingredients are: a generative data model in which observed inputs are (nonlinear functions of) random projections of a latent space, engineered so that the data covariance has a power-law spectrum $\lambda_k \sim k^{-(1+\zeta)}$, as measured in real image and text data (the spectral exponent is the paper's γ; we reserve γ for the scaling dial of Section 5.4); a random-feature *linear* student, so we are in the lazy limit of Section 4, whose P random features are its trainable parameters, trained on n samples; and the joint thermodynamic limit $P, n \to \infty$ at fixed ratio. The computation proceeds at large

P and n, where expectations over data and features become Gaussian integrals, the test loss becomes a product of resolvents (Green's functions), and the $1/P$ expansion organizes into planar ('t Hooft) diagrams that can be resummed exactly.

5

Frozen, but random. A natural first reaction is that this model should be trivial. The student is a linear model on fixed random features. Its kernel never moves, so we are squarely in the lazy regime of Section 4, and in the strict infinite-width lazy limit the loss would not depend on P at all. The P-scaling law is a *finite-size effect on the lazy limit*. At finite P the feature Gram matrix is a *random* matrix, correlated with the random sample, and its rank caps how many modes of the data spectrum the student can resolve. All of the physics lives in the disorder average over this randomness, which is what the resolvents and the planar diagrams compute. Frozen does not mean deterministic.

Three notable results:

- *Where exponents come from:* the loss inherits the *data's* spectral exponent, $L \sim \min(P, n)^{-\zeta}$ up to logarithms. Remarkably, generic nonlinear feature maps *extend* the data's power-law spectrum rather than reshaping it. This explains why exponents are roughly architecture-independent and can only be improved by changing the data (or the task).
- *Duality and Chinchilla:* the loss is (to leading order) *symmetric* under $P \leftrightarrow n$. This predicts $\alpha_P \approx \alpha_n$, consistent with observation, and opposite in spirit to the quanta model's asymmetric prediction (Exercise 7.1(e)). It makes *equiparameterization* $P \propto n$ compute-optimal, a first-principles Chinchilla rule.
- *Where laws break:* scaling stops when $\min(P, n)$ exceeds the latent dimensionality of the data model, and the loss plateaus. Power laws are a statement about the resolvable structure in the data, and they end when the structure is exhausted.

7.9 All the knobs at once: a dynamical theory

Everything so far is statics, or one knob at a time. The current frontier is the *dynamical* mean-field theory of Section 6 applied to scaling. Bordelon, Atanasov, and Pehlevan (Bordelon et al. 2024) solve a random-feature model trained by gradient flow, with power-law data spectra, in the joint limit of large width, large data, and long time. They obtain a single self-consistent prediction for $L(t, P, n)$, with the correlation and response functions of Section 6 as the order parameters. Each resource, when it is the binding constraint, contributes its own power law, recovering the resolution-limited exponents of Section 7.7 in the appropriate corners. Moreover, the theory predicts the full *compute-optimal frontier*: minimizing L at fixed compute $C \propto Pt$ yields Chinchilla-like allocation rules and the frontier exponent, rather than assuming them.

A fair question: the features here are again frozen and random, exactly the setting of Section 7.8, so is dynamical mean-field theory overkill? No, for two reasons. First, time is a genuine third resource. Predicting $L(t, P, n)$ and the compute frontier requires the

whole training trajectory, and the statics only sees the $t \to \infty$ corner. Second, the naive mode-by-mode solution of the lazy regime, in which each kernel eigenmode of the residual decays as $e^{-\eta\lambda_k t}$ as in (4.34), applies to a *deterministic* kernel. Here the kernel's eigenvalues and eigenvectors are random and correlated with the sampled data, so the disorder average couples the modes. The DMFT correlation and response functions are the dynamical generalization of the static resolvents of Section 7.8, and at $t \to \infty$ the equations collapse back onto the static answer, as they must. What the theory is *not* yet doing is feature learning. The adaptive-kernel version of the formalism exists (Bordelon & Pehlevan 2022), but it has not been pushed through power-law data to closed-form scaling laws; these are the "joint laws with genuine feature learning" missing from Section 7.3. Since feature learning is known to improve exponents for structured targets in solvable cases, the correction need not be small.

7.10 Beyond pretraining: post-training and test-time scaling

Frontier models are no longer just pretrained. They are fine-tuned on curated demonstrations (SFT), optimized against reward models or verifiable rewards (RLHF, RLVR), and allowed to spend variable compute at inference time. Do scaling laws extend to these stages? Increasingly, yes. But the functional forms change, and the changes are instructive.

Transfer and fine-tuning still look Kaplan-like. The value of pretraining for a downstream task can be quantified as the *effective data transferred*, meaning the amount of fine-tuning data that the pretrained initialization is worth, and it scales as a power law in both the fine-tuning dataset size and the model size (Hernandez et al. 2021).

Reward-model optimization introduces a new scaling axis: not data or parameters, but the distance moved from the base policy, $d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_0)}$. Gao, Schulman, and Hilton (Gao et al. 2023) found that the true ("gold") reward follows simple laws in d, namely $d\,(a - b \log d)$ for best-of-n sampling and $d\,(a - b\,d)$ for RL, with coefficients that scale smoothly with the size of the reward model. Note the shape: not a power law but a rise and a *fall*. Optimizing a learned proxy eventually degrades the true objective. This is a Goodhart peak, quantified.

RL on verifiable rewards, the training recipe of the reasoning models, scales by *saturating sigmoids* rather than power laws. In the largest systematic study to date, performance as a function of RL compute is predictable, but by a fit with an asymptote, an efficiency, and a midpoint, and most recipe choices move the efficiency without changing the asymptote (Khatri et al. 2025). A physics gloss suggests why the form differs, though it is a hypothesis rather than a result. Pretraining mines an effectively unbounded Zipf tail of new structure, hence power laws. RL post-training mostly elicits and reweights capabilities the base model already contains, on a bounded task distribution. In quanta language, pretraining acquires quanta, while RL sharpens the policy over quanta it already has, and one runs out. *Test-time compute* is the other new axis: the coverage

achieved by repeated sampling grows roughly as a power law in the number of samples (Brown et al. 2024), and train-time and test-time compute can be traded against each other along a compute-optimal frontier (Snell et al. 2024).

What does not yet exist is a joint law over all four budgets (pretraining, SFT, RL, inference), and any *theory* at all, even at the level of the quantization model, that derives the sigmoid or the Goodhart peak from a model of the data and skill distribution. The empirical forms are waiting for a statistical-mechanics treatment.

That every ingredient of the pretraining synthesis — Gibbs measures, kernels, mean-field limits, MSRJD actions — appeared earlier in these notes as a physics import is, we hope, a fair summary of the state of the field. A theory of the scaling laws is not finished, but it is recognizably being built, and it is being built out of statistical physics. The post-training laws are younger still: their regularities are only now being settled, and there is no theory of them yet. For a student entering the field, that is a map of where the open territory begins.

→ *Appendix A. The replica method*: Supplementary Material, p. 502

→ *Appendix B. From linear regression to kernel regression*: Supplementary Material, p. 508

References

Elisabeth Agoritsas, Giulio Biroli, Pierfrancesco Urbani, and Francesco Zamponi (2018). *Out-of-equilibrium dynamical mean-field equations for the perceptron model*. Journal of Physics A: Mathematical and Theoretical.

Alexander Altland and Ben D. Simons (2010). *Condensed Matter Field Theory*. Cambridge University Press.

Daniel J. Amit, Hanoch Gutfreund, and H. Sompolinsky (1985). *Spin-Glass Models of Neural Networks*. Physical Review A.

Alexander Atanasov, Blake Bordelon, and Cengiz Pehlevan (2022). *Neural networks as kernel learners: the silent alignment effect*. International Conference on Learning Representations (ICLR).

Yasaman Bahri, Jonathan Kadmon, Jeffrey Pennington, Samuel S. Schoenholz, Jascha Sohl-Dickstein, and Surya Ganguli (2020). *Statistical mechanics of deep learning*. Annual Review of Condensed Matter Physics.

Yasaman Bahri, Ethan Dyer, Jared Kaplan, Jaehoon Lee, and Utkarsh Sharma (2024). *Explaining neural scaling laws*. Proceedings of the National Academy of Sciences.

J"urgen Berges (2004). *Introduction to Nonequilibrium Quantum Field Theory*. AIP Conference Proceedings.

Emmy Blumenthal (2025). *Building Intuition for Dynamical Mean-Field Theory: A Simple Model and the Cavity Method*. arXiv.

Blake Bordelon and Cengiz Pehlevan (2022). *Self-consistent dynamical field theory of kernel evolution in wide neural networks*. Advances in Neural Information Processing Systems (NeurIPS).

Blake Bordelon and Cengiz Pehlevan (2023). *Dynamics of Finite Width Kernel and Prediction Fluctuations in Mean Field Neural Networks*. arXiv.

Blake Bordelon, Abdulkadir Canatar, and Cengiz Pehlevan (2020). *Spectrum dependent learning curves in kernel regression and wide neural networks*. International Conference on Machine Learning (ICML).

Blake Bordelon, Lorenzo Noci, Mufan Bill Li, Boris Hanin, and Cengiz Pehlevan (2023). *Depthwise Hyperparameter Transfer in Residual Networks: Dynamics and Scaling Limit*. arXiv.

Blake Bordelon, Alexander Atanasov, and Cengiz Pehlevan (2024). *A dynamical model of neural scaling laws*. International Conference on Machine Learning (ICML).

Ari Brill (2024). *Neural scaling laws rooted in the data distribution*.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher R'e, and Azalia Mirhoseini (2024). *Large language monkeys: scaling inference compute with repeated sampling*.

Abdulkadir Canatar, Blake Bordelon, and Cengiz Pehlevan (2021). *Spectral bias and task-model alignment explain generalization in kernel regression and infinitely wide neural networks*. Nature Communications.

Michael Celentano, Chen Cheng, and Andrea Montanari (2021). *The high-dimensional asymptotics of first order methods with random data*.

Zhongtian Chen, Edmund Lau, Jake Mendel, Susan Wei, and Daniel Murfet (2023). *Dynamical versus Bayesian Phase Transitions in a Toy Model of Superposition*. arXiv preprint arXiv:2310.06301.

L'ena"ic Chizat and Francis Bach (2018). *On the global convergence of gradient descent for over-parameterized models using optimal transport*. Advances in Neural Information Processing Systems (NeurIPS).

L'ena"ic Chizat, Edouard Oyallon, and Francis Bach (2019). *On lazy training in differentiable programming*. Advances in Neural Information Processing Systems (NeurIPS).

Youngmin Cho and Lawrence K. Saul (2009). *Kernel methods for deep learning*. Advances in Neural Information Processing Systems (NeurIPS).

Carson C. Chow and Michael A. Buice (2012). *Path Integral Methods for Stochastic Differential Equations*. arXiv.

Leticia F. Cugliandolo (2023). *Recent Applications of Dynamical Mean-Field Methods*. arXiv.

Simon S. Du, Xiyu Zhai, Barnab'as P'oczos, and Aarti Singh (2019). *Gradient Descent Provably Optimizes Over-parameterized Neural Networks*. International Conference on Learning Representations (ICLR).

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah (2022). *Toy Models of Superposition*. arXiv preprint arXiv:2209.10652.

Andreas Engel and Christian Van den Broeck (2001). *Statistical Mechanics of Learning*. Cambridge University Press.

Zhou Fan, Justin Ko, Bruno Loureiro, Yue M. Lu, and Yandi Shen (2025). *Dynamical mean-field analysis of adaptive Langevin diffusions: propagation-of-chaos and convergence of the linear response*.

Leo Gao, John Schulman, and Jacob Hilton (2023). *Scaling laws for reward model overoptimization*. International Conference on Machine Learning (ICML).

Crispin W. Gardiner (2004). *Handbook of Stochastic Methods: for Physics, Chemistry and the Natural Sciences*. Springer.

Mario Geiger, Stefano Spigler, Arthur Jacot, and Matthieu Wyart (2020). *Disentangling feature and lazy training in deep neural networks*. Journal of Statistical Mechanics: Theory and Experiment.

Antoine Georges, Gabriel Kotliar, Werner Krauth, and Marcelo J. Rozenberg (1996). *Dynamical Mean-Field Theory of Strongly Correlated Fermion Systems and the Limit of Infinite Dimensions*. Reviews of Modern Physics.

C'edric Gerbelot, Emanuele Troiani, Francesca Mignacco, Florent Krzakala, and Lenka Zdeborov'a (2024). *Rigorous dynamical mean-field theory for stochastic gradient descent methods*. SIAM Journal on Mathematics of Data Science.

Nigel Goldenfeld (2018). *Lectures On Phase Transitions And The Renormalization Group*. CRC Press.

Andrey Gromov (2023). *Grokking modular arithmetic*.

G'eza Györgyi (1990). *First-order transition to perfect generalization in a neural network with binary synapses*. Physical Review A.

Boris Hanin (2023). *Random Neural Networks in the Infinite Width Limit as Gaussian Processes*. The Annals of Applied Probability.

Boris Hanin (2026). *Special Lectures on the Mathematics of Deep Learning*.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2015). *Delving deep into rectifiers: surpassing human-level performance on ImageNet classification*. International Conference on Computer Vision (ICCV).

Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo Jun, Tom B. Brown, Prafulla Dhariwal, Scott Gray, and others (2020). *Scaling laws for autoregressive generative modeling*.

Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish (2021). *Scaling laws for transfer*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, and others (2022). *Training compute-optimal large language models*. Advances in Neural Information Processing Systems (NeurIPS).

Arthur Jacot, Franck Gabriel, and Cl'ement Hongler (2018). *Neural tangent kernel: convergence and generalization in neural networks*. Advances in Neural Information Processing Systems (NeurIPS).

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel (2023). *Large language models struggle to learn long-tail knowledge*. International Conference on Machine Learning (ICML).

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei (2020). *Scaling laws for neural language models*. arXiv preprint arXiv:2001.08361.

Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal (2025). *The art of scaling reinforcement learning compute for LLMs*.

G. Kotliar, S. Y. Savrasov, K. Haule, V. S. Oudovenko, O. Parcollet, and C. A. Marianetti (2006). *Electronic structure calculations with dynamical mean-field theory*. Rev. Mod. Phys.

Ryogo Kubo (1957). *Statistical-Mechanical Theory of Irreversible Processes. I. General Theory and Simple Applications to Magnetic and Conduction Problems*. Journal of the Physical Society of Japan.

R. Kubo (1966). *The Fluctuation-Dissipation Theorem*. Reports on Progress in Physics.

Tanishq Kumar, Blake Bordelon, Samuel J. Gershman, and Cengiz Pehlevan (2024). *Grokking as the transition from lazy to rich training dynamics*.

T. Lancaster and S. Blundell (2014). *Quantum Field Theory for the Gifted Amateur*. OUP Oxford.

Yann LeCun, L'eon Bottou, Genevieve B. Orr, and Klaus-Robert M"uller (1998). *Efficient BackProp*. Neural Networks: Tricks of the Trade.

Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein (2018). *Deep neural networks as Gaussian processes*. International Conference on Learning Representations (ICLR).

Chan Li and Nigel Goldenfeld (2026). *Broken Ergodicity and the Violation of the Fluctuation-Dissipation Theorem Lead to Generalization Beyond Overfitting in Machine Learning*. arXiv.

J. Lin (2025). *Feature identification via the empirical NTK*.

Alexander Maloney, Daniel A. Roberts, and James Sully (2022). *A solvable model of neural scaling laws*. arXiv preprint arXiv:2210.16859.

Song Mei, Andrea Montanari, and Phan-Minh Nguyen (2018). *A mean field view of the landscape of two-layer neural networks*. Proceedings of the National Academy of Sciences.

Eric J. Michaud, Ziming Liu, Uzay Girit, and Max Tegmark (2023). *The quantization model of neural scaling*. Advances in Neural Information Processing Systems (NeurIPS).

Francesca Mignacco and Pierfrancesco Urbani (2022). *The Effective Noise of Stochastic Gradient Descent*. Journal of Statistical Mechanics: Theory and Experiment.

Francesca Mignacco, Florent Krzakala, Pierfrancesco Urbani, and Lenka Zdeborov'a (2021). *Dynamical Mean-Field Theory for Stochastic Gradient Descent in Gaussian Mixture Classification**. Journal of Statistical Mechanics: Theory and Experiment.

Andrea Montanari and Pierfrancesco Urbani (2025). *Dynamical Decoupling of Generalization and Overfitting in Large Two-Layer Networks*. arXiv.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel (2023). *Scaling data-constrained language models*. Advances in Neural Information Processing Systems (NeurIPS).

Radford M. Neal (1996). *Bayesian Learning for Neural Networks*. Springer.

Lars Onsager and Stefan Machlup (1953). *Fluctuations and irreversible processes*. Physical Review.

R. K. Pathria and Paul D. Beale (2011). *Statistical Mechanics*. Elsevier/Academic Press.

Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli (2016). *Exponential expressivity in deep neural networks through transient chaos*. Advances in Neural Information Processing Systems (NeurIPS).

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon (2024). *Resolving discrepancies in compute-optimal scaling of language models*. Advances in Neural Information Processing Systems (NeurIPS).

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra (2022). *Grokking: generalization beyond overfitting on small algorithmic datasets*.

Charles Renshaw-Whitman (2026). *Iliad Physics Learning: lectures and exercise notebooks for the August 2026 Iliad Intensive*.

Zohar Ringel, Noa Rubin, Edo Mor, Moritz Helias, and Inbar Seroussi (2025). *Applications of Statistical Field Theory in Deep Learning*. Now Publishers.

Hannes Risken (1996). *The Fokker--Planck Equation: Methods of Solution and Applications*. Springer.

Daniel A. Roberts, Sho Yaida, and Boris Hanin (2022). *The Principles of Deep Learning Theory: An Effective Theory Approach to Understanding Neural Networks*. Cambridge University Press.

Grant M. Rotskoff and Eric Vanden-Eijnden (2022). *Trainability and accuracy of artificial neural networks: an interacting particle system approach*. Communications on Pure and Applied Mathematics.

Noa Rubin, Inbar Seroussi, and Zohar Ringel (2024). *Grokking as a First Order Phase Transition in Two Layer Networks*. The Twelfth International Conference on Learning Representations (ICLR).

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo (2023). *Are emergent abilities of large language models a mirage?*. Advances in Neural Information Processing Systems (NeurIPS).

Samuel S. Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein (2017). *Deep information propagation*. International Conference on Learning Representations (ICLR).

Inbar Seroussi, Gadi Naveh, and Zohar Ringel (2023). *Separation of scales and a thermodynamic description of feature learning in some CNNs*. Nature Communications.

H. Sebastian Seung, Haim Sompolinsky, and Naftali Tishby (1992). *Statistical mechanics of learning from examples*. Physical Review A.

Utkarsh Sharma and Jared Kaplan (2020). *A neural scaling law from the dimension of the data manifold*.

James B. Simon, Madeline Dickens, Dhruva Karkada, and Michael R. DeWeese (2023). *The eigenlearning framework: a conservation law perspective on kernel ridge regression and wide neural networks*. Transactions on Machine Learning Research (TMLR).

Jamie Simon, Daniel Kunin, Alexander Atanasov, Jeremy Cohen, Arthur Jacot, Eric J. Michaud, Enric Boix-Adser`a, Nikhil Ghosh, Mason Kamb, Berkan Ottlik, Blake Bordelon, Florentin Guth, Dhruva Karkada, and Joseph Turnbull (2026). *There Will Be a Scientific Theory of Deep Learning*. arXiv preprint arXiv:2604.21691.

Justin Sirignano and Konstantinos Spiliopoulos (2020). *Mean field analysis of neural networks: a law of large numbers*. SIAM Journal on Applied Mathematics.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar (2024). *Scaling LLM test-time compute optimally can be more effective than scaling model parameters*.

H. Sompolinsky, A. Crisanti, and H. J. Sommers (1988). *Chaos in Random Neural Networks*. Physical Review Letters.

Konstantinos Spiliopoulos, Richard B. Sowers, and Justin Sirignano (2025). *Mathematical Foundations of Deep Learning Models and Algorithms*. American Mathematical Society.

Jonas Stapmanns, Tobias K"uhn, David Dahmen, Thomas Luu, Carsten Honerkamp, and Moritz Helias (2020). *Self-Consistent Formulations for Stochastic Nonlinear Neuronal Dynamics*. Physical Review E.

Alain-Sol Sznitman (1991). *Topics in propagation of chaos*. 'Ecole d''Et'e de Probabilit'es de Saint-Flour XIX — 1989.

David Tong (2006). *Lectures on Quantum Field Theory*.

David Tong (2012). *Statistical Field Theory*. University of Cambridge.

David Tong (2012). *Lectures on Kinetic Theory*. University of Cambridge.

David Tong (2012). *Statistical Physics*. University of Cambridge.

Nicolaas G. van Kampen (2007). *Stochastic Processes in Physics and Chemistry*. North-Holland.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, and others (2022). *Emergent abilities of large language models*. Transactions on Machine Learning Research.

Christopher K. I. Williams (1997). *Computing with infinite networks*. Advances in Neural Information Processing Systems (NeurIPS).

Lechao Xiao, Yasaman Bahri, Jascha Sohl-Dickstein, Samuel S. Schoenholz, and Jeffrey Pennington (2018). *Dynamical isometry and a mean field theory of CNNs: how to train 10,000-layer vanilla convolutional neural networks*. International Conference on Machine Learning (ICML).

Greg Yang and Edward J. Hu (2021). *Tensor programs IV: feature learning in infinite-width neural networks*. International Conference on Machine Learning (ICML).

Greg Yang (2019). *Tensor Programs I: Wide Feedforward or Recurrent Neural Networks of Any Architecture are Gaussian Processes*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019).

Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao (2022). *Tensor programs V: tuning large neural networks via zero-shot hyperparameter transfer*. Advances in Neural Information Processing Systems (NeurIPS).

Lenka Zdeborov'a and Florent Krzakala (2016). *Statistical physics of inference: thresholds and algorithms*. Advances in Physics.

J. Zinn-Justin (2002). *Quantum Field Theory and Critical Phenomena*. Clarendon Press.

NOTES

1. A physicist's free energy is $-\beta^{-1} \log Z$; we absorb the factor of β throughout, so that F is dimensionless.

2. For transformers, the limit analogous to "large width" that leads to a Gaussian process is a large number of attention heads.
3. As a point of notation, η and ζ are also commonly used for noise, and we will distinguish between discrete and continuum delta functions as δ_{ij} and $\delta(t - t')$ respectively.
4. Michaud's blog post ericjmichaud.com/quanta is an excellent companion to the paper and can substitute for it in this exercise. But attempt Exercise 7.1 *before* reading either; the derivations are short, and they are more instructive to find than to follow.
5. This is also a working example for Section 6's toolkit: the same model can be solved dynamically, and its double-descent peak, at the interpolation threshold $P \sim n$, is regularized by ridge or early stopping, connecting back to the divergence of the overfitting factor mentioned in Section 7.7.

PART C

Abstractions, Representations, and Interpretability

CHAPTER C.1

Intro to ML Engineering

Julian Schulz (Meridian Research), Adam Newgas (Timaeus)

Crash course in ML and LLM basics in PyTorch, how to setup machines for ML research and engineering, and making effective use of Claude Code.

Source: iliad-intensive.org/interpretability/intro-to-ml-engineering · CC BY 4.0 · Slides: online

Prerequisites

- You should have some rudimentary ability to code in python
- You should have heard of the ML basics before
- You should have access to a computer with the Claude Code CLI installed
- You should know linear algebra and understand gradient descent

WHAT YOU'LL LEARN

- ML Foundations
  - Learn the ML basics needed for the rest of this book
  - Give some hands on experience with common ML Libraries/methods
  - Understanding and building the transformer architecture
- Practical ML
  - Renting out and setting up your own compute
  - Use common ML Libraries/methods
- Claude Code for Research
  - Learn how to use Claude Code in research projects
  - Get a good basic setup to make use of multiple Claude Code instances across different branches and projects

Outline

The chapter has three parts. *ML Foundations*, for readers with less ML background, covers ML and LLM basics through hands-on exercise notebooks. *Practical ML* covers renting and setting up your own compute. *Claude Code for Research* is a series of exer-

cises that use Claude Code to take small steps in an AI safety research repository, each with a different Claude Code tool.

ML Foundations

ML and LLM basics for readers with less ML background, going deeper into selected topics through hands-on exercise notebooks.

Notes and materials for this part are in the repository https://github.com/iliad-team/iliad-intensive-C.1.1

Exercise notebooks (Google Colab):

- Pytorch Basics
- Optimizers
- Architectures
- Tensorflow Playground
- MLP
- Attention
- RLHF

Content:

- Training loop — forward/backward pass, gradient descent, stochastic batching, optimizer role
- PyTorch tensors — basic operations, einops, batching & data loading, autograd/computational graph, devices & GPU
- Loss functions — classification losses, RL/human-rater losses, train vs. test loss, over/underfitting
- Parameters, activations & hyperparameters — terminology + optimizers (momentum, RMSProp)
- Architectures — activation functions, universal approximation, over/underparameterization, symmetries in architecture design, CNNs, transformers, residual streams
- Hyperparameter optimization — sweeps, scaling laws
- LLM lifecycle – Pretraining, SFT fine tuning, RLHF, RLVR

Practical ML

Rent and set up your own compute, and pick from a modular selection of practical topics.

Materials for this part are in the repository https://github.com/iliad-team/iliad-intensive-C.1.2

A modular selection of self-study and exercise topics:

- Runpod + VSCode Remote Extension
- Weights and Biases
- Distributed Computing
- Concurrent Computing
- The Python and ML Ecosystem
- Publishing Your Project

Claude Code for Research

Use Claude Code to do small steps in an AI safety research repo, each step with a different Claude Code tool.

Material:

Claude Code cheat sheet

[optionally] get a good Terminal Claude Code setup

Clone this repository (github.com/iliad-team/iliad-intensive-C.1.3/tree/main) for the exercises

- Do Exercise 1 using plan mode
- Do Exercise 2 using subagents
- Do Exercise 3 using the git worktree and goals

Final project: replicate a version of Figure 1 from one of these papers:

- Inference only, no GPU needed
- On a GPU

Content:

- Installing Claude code
- Using a research template with good claude.md integration
- Using multiple agents at the same time
- Plan mode
- Subagents
- Worktrees
- Goals/loops

CHAPTER C.2

Mechanistic Interpretability

Julian Schulz (Meridian Research)

What can we learn about how a model works by looking at its internals? We cover the main methods for reverse engineering neural networks, from features and circuits to sparse autoencoders and auto-interp of activations, and discuss the relevance to safety.

Source: iliad-intensive.org/interpretability/mechanistic-interpretability · CC BY 4.0 · Slides: online

Prerequisites

- Knowing the basics of Machine Learning
- Familiarity with the Transformer Architecture
- Basic maths / linear algebra

WHAT YOU'LL LEARN

By the end of this chapter you should understand:

- The Goals of MechInterp
- What is meant by feature & circuit
- Have an overview of currently used methods for feature and circuit discovery
- Have an overview of our current capabilities/frontier of knowledge in MechInterp
- Be able to engage with/understand common critiques of MechInterp

More advanced readers should also be able to:

- implement common mechinterp methods
- Know when to apply what method
- read and fully understand new MechInterp publications
- Be a useful discussion partner to MechInterp researchers

Why

Mechinterp is a central field within AI safety. I think everyone should have a rough understanding of the current state of MechInterp, to have an idea where it intersects with their own research direction.

For example: how whitebox methods are useful for AI controls, or how mechinterp findings compare with devinterp findings.

I hope that some readers will take this chapter as a starting point to pursue a project or a career in MechInterp. For these readers, it should give enough of a taste of what MechInterp looks like to wake their interest and an overview to orient them towards currently active research directions and open problems.

How

The lecture slides survey the major developments in MechInterp so far, particularly recent results and methodological developments.

Lecture notes and materials are self-contained within this repository: https://github.com/iliad-team/iliad-intensive-C.2

Content

Main Content

Material:

Slides (raw.githubusercontent.com/iliad-team/iliad-intensive-C.2/main/lectures/output/main.pdf)

Exercises and external links (in the order of the slides)

- Feature Visualization
- Logit Lens: standard and hard versions
- Neuronpedia — SAE features
- Sparse Autoencoders
- Neuronpedia — Attribution graphs
- Induction Heads: standard and hard versions
- Neuronpedia — Natural Language Autoencoders

Discussion reading

- Nanda et al., A Pragmatic Vision for Interpretability
- Ségerie, Against Almost Every Theory of Impact of Interpretability
- Hendrycks, The Misguided Quest for Mechanistic AI Interpretability
- Chughtai, Activation Space Interpretability May Be Doomed

Intent:

- The lecture gives an overview of the current state of MechInterp, developments over the last years, and commonly used methods
- The technical exercises are intended to deepen the understanding of the techniques and give a flavour for how mechinterp work looks like
- The Neuronpedia exercises are supposed to de-cherrypick mechinterp results and give a flavour for the average feature/circuit/NLA-output and show that a lot of the

mechinterp results are still confusing, hard to interpret, and noisy

Content:

- Complex features and circuits (the car detector circuit)
- Universality in image models (curve and high-frequency detectors)
- Natural abstractions discourse (why certain features might be universal)
- Polysemanticity in CNNs
- Features in language models and the lack of a privileged basis
- Word embeddings encoding human concepts (word2vec gender direction)
- Linear probes for classifying text properties
- Probing world models (Othello-GPT board state)
- Intervening on world models
- Probing LLMs for truthfulness (attention head activations)
- Steering models with contrastive activation vectors
- Ablating features and the refusal direction (activation addition, direction ablation, weight orthogonalization)
- The logit lens
- Superposition (privileged vs non-privileged bases)
- Johnson–Lindenstrauss lemma and the scale of superposition
- Anthropic's toy model of superposition
- Feature geometry and phases (sticky regions, dimensions per feature)
- Semantic feature geometry (circular representations of time / non-linear features)
- Dictionary learning theory (Arora et al.)
- Compressed sensing and L1 minimization (Candès/Donoho)
- Sparse autoencoders (SAEs) and their training objective
- Interpreting SAE features (activation, logit effects, steering)
- Neuronpedia exploration
- SAE problems: feature absorption
- SAE problems: feature shrinkage
- The "Cambrian explosion" of SAE variants
- Gated SAEs
- JumpReLU SAEs
- Top-K / Batch Top-K SAEs
- Matryoshka SAEs
- Staircase SAEs
- Comparing/evaluating SAE architectures (loss recovered, auto-interp, absorption, SCR, k-sparse probing, RAVEL)
- SAEs scaling to production models (Sonnet 3.5)

- Bigger models yielding more specific features (golden gate bridge feature)
- Reading Claude's mind / safety-relevant feature steering (eval-awareness features)
- Transcoders and cross-layer transcoders (expanding SAEs)
- Model diffing
- Transformer circuits
- Reverse-engineering modular addition (the grokking / mod-113 circuit)
- Fourier analysis of the embedding matrix
- Plotting attention patterns
- Logit attribution and induction heads
- Q-composition and K-composition
- Path patching and the IOI (indirect object identification) circuit
- Automated circuit discovery (ACDC)
- Circuit tracing with cross-layer transcoders and replacement models
- Attribution graphs and feature suppression
- Circuit tracing on production models (Haiku 3.5 math circuit)
- Using circuit tracing for alignment (misaligned/reward-hacking model)
- Generality findings across circuits (layer roles, default pathways, shortcuts, special tokens, hedging features)
- Fuzzy/less-faithful interpretability approaches
- Activation oracles
- Natural language autoencoders (verbalizer + reconstructor)
- Open problems in mech interp (Sharkey et al.): decomposition, description, validation, automation, application
- Critiques of mechanistic interpretability (discussion of critical papers)

CHAPTER C.3

Computational Mechanics

Xavier Poncini (Simplex), Adam Shai (Simplex), Paul Riechers (Simplex)

What must a next-token predictor remember about the past? Causal states, belief states and their geometry, from HMMs to generalised HMMs, ending with the evidence that transformers learn this structure.

Source: iliad-intensive.org/interpretability/computational-mechanics · CC BY 4.0 · Slides: part 1, part 2, part 3

1. Prerequisites

Familiarity with the goals and basic methodology of mechanistic interpretability, including the notions of features and circuits; the linear representation hypothesis; linear probes as a method for reading off internal representations; and a basic understanding of the transformer architecture.

Students should be comfortable with the idea that one can train a linear map from model activations to some target structure and evaluate its quality (e.g. via MSE or R^2).

Mathematical background. The module is mathematically self-contained: all formal definitions (HMMs, GHMMs, belief states, MSPs) are introduced from scratch. However, students will engage with the material more fluently if they are comfortable with the following:

- **Linear algebra:** row-stochastic matrices, rank and invertibility, null spaces (kernels).
- **Probability:** conditional probability, Bayes' rule, probability distributions over finite sets, the probability simplex.
- **Basic machine learning:** next-token prediction, loss functions (cross-entropy), the concept of model activations.

WHAT YOU'LL LEARN

- Construct their own hidden Markov models.
- Explain the motivation for introducing generalised hidden Markov models (GHMMs).
- Explain why the belief state is a useful object for making predictions about GHMM data.

- Compute the belief state of a GHMM.
- Explain the difference between the belief state of a HMM and the belief state of a GHMM (that is itself not an HMM).
- Explain the relationship between belief states and next-token observation probabilities for GHMMs.
- Compute the mixed state presentation (MSP) for GHMMs.
- Explain why generating data is easier than predicting data.
- Explain the evidence for the claim: transformers represent belief state geometry in their residual stream.
- Develop hypotheses for where transformers represent: belief states, observation probabilities, log observation probabilities.

→ *Fast-track route*: Supplementary Material, p. 520

3. Predicting the future

Instructions

Give a clear justification for each answer. You may use results from the slides and earlier exercises unless an exercise asks you to establish them. Questions marked **Extension** are optional.

Instructions

Exercise 3.1 (From statistics to partitions). Let $u_1, \dots, u_6 \in \mathcal{X}_+^*$ be six possible histories. Suppose that they induce the following conditional distributions over the entire future:

history	u_1	u_2	u_3	u_4	u_5	u_6
future distribution	P	P	P	Q	Q	S

where P, Q, and S are pairwise distinct.

Three statistics R_A, R_B, and R_C induce the partitions

$$\begin{aligned}\Pi_A &= \big\{\{u_1, u_2\}, \{u_3\}, \{u_4\}, \{u_5\}, \{u_6\}\big\},\\ \Pi_B &= \big\{\{u_1, u_2, u_3\}, \{u_4, u_5\}, \{u_6\}\big\},\\ \Pi_C &= \big\{\{u_1, u_2, u_3, u_4\}, \{u_5\}, \{u_6\}\big\}.\end{aligned}$$

Recall that a statistic is sufficient for prediction when histories assigned the same value induce the same distribution over the future.

(a) What does it mean for two histories to belong to the same cell of the partition induced by a statistic? Illustrate your answer using u_1 and u_2 in Π_A.

(b) Determine which of R_A, R_B, and R_C are sufficient. If a statistic is not sufficient, give two histories witnessing the failure.

(c) Which cells of Π_A can be merged without losing sufficiency? Perform all such merges and identify the resulting partition.

(d) Explain why no two cells of Π_B can be merged while preserving sufficiency. What does this show about R_B?

(e) Prove that every refinement of a sufficient partition is also sufficient. Is every coarsening of a sufficient partition necessarily sufficient? Use Π_A, Π_B, and Π_C to illustrate your answer.

(f) Extension. Consider the partition

$$\Pi_D = \big\{\{u_1\}, \{u_2, u_3\}, \{u_4, u_5\}, \{u_6\}\big\}.$$

Show that Π_A and Π_D are both sufficient but that neither refines the other. How can both nevertheless refine the same minimal sufficient partition?

SOLUTION, p. 628

Exercise 3.2 (Causal states are minimal sufficient statistics). For $x, x' \in \mathcal{X}_+^*$, define the relation $\sim$ by

$$\begin{aligned} x \sim x' \quad \Longleftrightarrow \quad & \Pr\big(X_{|x|+1:|x|+|y|} = y \mid X_{1:|x|} = x\big) \\ & = \Pr\big(X_{|x'|+1:|x'|+|y|} = y \mid X_{1:|x'|} = x'\big) \end{aligned}$$

for every $y \in \mathcal{X}^*$. Define the causal state of x by

$$\epsilon(x) := [x]_\sim,$$

and write $\mathcal{S} := \epsilon(\mathcal{X}_+^*)$ for the set of causal states.

(a) Prove that $\sim$ is an equivalence relation on $\mathcal{X}_+^*$ by establishing each of the following:

1. reflexivity: $x \sim x$ for every $x \in \mathcal{X}_+^*$;
2. symmetry: if $x \sim x'$, then $x' \sim x$;
3. transitivity: if $x \sim x'$ and $x' \sim x''$, then $x \sim x''$.

(b) Prove that ϵ is sufficient for prediction.

(c) Let $R : \mathcal{X}_+^* \to \mathcal{R}$ be any sufficient statistic. Prove that

$$R(x) = R(x') \quad \Longrightarrow \quad \epsilon(x) = \epsilon(x').$$

Deduce that every sufficient partition refines the causal-state partition.

(d) Using parts (b) and (c), conclude that ϵ is a minimal sufficient statistic.

(e) Extension. Let T be another minimal sufficient statistic. Prove that T and ϵ induce the same partition of $\mathcal{X}_+^*$. Deduce that there is a bijection

$$\psi : \mathcal{S} \to \mathrm{im}(T)$$

satisfying $T = \psi \circ \epsilon$.

SOLUTION, p. 629

Exercise 3.3 (From finite generators to causal states). In this exercise, S_t denotes the hidden state immediately before the transition that emits X_{t+1}. All parameters lie strictly between 0 and 1.

(a) The Zero–One–Random Process. Let S_0 be uniformly distributed over $\{s_0, s_1, s_R\}$. The process has transitions

$$s_0 \xrightarrow{0:1} s_1, \qquad s_1 \xrightarrow{1:1} s_R, \qquad s_R \begin{cases} \xrightarrow{0:1-p} s_0, \\ \xrightarrow{1:p} s_0. \end{cases}$$

An edge label $x : q$ means that the process emits x and follows the edge with probability q.

1. List all possible histories of length two. For each history, determine whether it uniquely identifies the current hidden phase. When it does, identify that phase.
2. For each ambiguous length-two history, determine which additional observations resolve the ambiguity. Identify the resulting phase and explain why it remains known after every subsequent observation.
3. Assume that $\epsilon(\emptyset)$, $\epsilon(0)$, $\epsilon(1)$, and $\epsilon(10)$ are pairwise distinct. Show that the causal states have shortest-length representatives

$$\emptyset, \quad 0, \quad 1, \quad 10, \quad 00, \quad 01, \quad 11.$$

Explain why no additional causal states arise.

4. Under the assumption in part (iii), draw the automaton associated with the causal-state dynamics.

(b) The Even Process. Let S_0 be uniformly distributed over $\{A, B\}$ and consider the generator

$$A \xrightarrow{0:q} A, \qquad A \xrightarrow{1:1-q} B, \qquad B \xrightarrow{1:1} A.$$

Thus every run of 1s between successive 0s has even length.

1. For each of the length-one histories 0 and 1, determine whether the current hidden state can be identified exactly.
2. Starting from each ambiguous history in part (i), consider its possible one-symbol extensions. Which extensions resolve the ambiguity, and which remain ambiguous? You may use the fact that $\epsilon(\emptyset) = \epsilon(11)$. What does this imply for histories of the form 1^n?
3. Assume additionally that $\epsilon(\emptyset) \neq \epsilon(1)$. Show that the causal states have shortest representatives

$$\emptyset, \qquad 1, \qquad 0, \qquad 01,$$

and explain why there are no additional causal states. You may distinguish causal states using possible and impossible future continuations.

4. Using $\epsilon(\emptyset) = \epsilon(11)$ and assuming $\epsilon(\emptyset) \neq \epsilon(1)$, draw the automaton associated with the causal-state dynamics.

(c) The Simple Nonunifilar Source. Let S_0 be uniformly distributed over $\{A, B\}$ and consider

$$A \xrightarrow{0:1/2} A, \qquad A \xrightarrow{0:1/2} B, \qquad B \xrightarrow{0:1/2} B, \qquad B \xrightarrow{1:1/2} A.$$

1. Show that observing a 1 identifies the new hidden state as A.
2. Starting from A, show that there are $n+1$ hidden paths that emit 0^n, each with probability 2^{-n}. How many of these paths can subsequently emit 1? Hence calculate

$$\Pr(X_{n+2} = 1 \mid X_{1:n+1} = 10^n).$$

3. Show that this probability is different for every $n \geq 0$. Conclude that the histories

$$1, 10, 100, \ldots$$

belong to distinct causal states and that the process has infinitely many causal states.

SOLUTION, p. 629

Exercise 3.4 (Why next-token prediction is enough). Recall that

$$\mathcal{X}_+^* := \{x \in \mathcal{X}^* : \Pr(X_{1:|x|} = x) > 0\}$$

is the set of allowable histories.

Consider a model with parameters θ satisfying

$$h_n = f_\theta(h_{n-1}, x_n), \qquad y_n = g_\theta(h_{n-1}, x_n).$$

Fix the initial hidden state h_0 and suppose that the model predicts the next-token distribution exactly: for every $n \geq 1$,

$$y_n = \Pr(X_{n+1} \mid X_{1:n} = x_{1:n}), \qquad \forall\, x_{1:n} \in \mathcal{X}_+^*.$$

For every nonempty allowable history, define

$$R(x_{1:n}) := (h_{n-1}, x_n).$$

For $a \in \mathcal{X}$, define the deterministic update

$$\Phi_a(h, x) := \big(f_\theta(h, x), a\big).$$

For a word $z_{1:j}$, write

$$\Phi_{z_{1:j}} := \Phi_{z_j} \circ \cdots \circ \Phi_{z_1}, \qquad \Phi_{z_{1:0}} := \mathrm{id}\,.$$

(a) Show that, whenever $x_{1:n} z_{1:j}$ is allowable,

$$R(x_{1:n} z_{1:j}) = \Phi_{z_{1:j}}(R(x_{1:n}))\,.$$

Hence show that, if $R(x_{1:n}) = R(x'_{1:m})$ and both extended histories are allowable, then

$$R(x_{1:n}z_{1:j}) = R(x'_{1:m}z_{1:j}).$$

(b) Let $R(x_{1:n}) = R(x'_{1:m})$ and fix $z_{1:k} \in \mathcal{X}^k$. As the symbols of $z_{1:k}$ are read from left to right, use part (a) and exact next-token prediction to show that for each $0 \le j < k$ such that the prefixes through $z_{1:j}$ are allowable,

$$\begin{aligned}&\Pr(X_{n+j+1} = z_{j+1} \mid X_{1:n+j} = x_{1:n}z_{1:j}) \\ &\quad = \Pr(X_{m+j+1} = z_{j+1} \mid X_{1:m+j} = x'_{1:m}z_{1:j})\,.\end{aligned}$$

Deduce that either the same first zero-probability extension is encountered after both histories, or every prefix of $z_{1:k}$ is allowable after both histories.

(c) In the case where every prefix is allowable, use the conditional chain rule to write

$$\begin{aligned}&\Pr(X_{n+1:n+k} = z_{1:k} \mid X_{1:n} = x_{1:n}) \\ &\quad = \prod_{j=0}^{k-1} \Pr(X_{n+j+1} = z_{j+1} \mid X_{1:n+j} = x_{1:n}z_{1:j})\,.\end{aligned}$$

Write the analogous product after $x'_{1:m}$ and compare its factors. Deal separately with the zero-probability case from part (b), and conclude directly that

$$\begin{aligned}&R(x_{1:n}) = R(x'_{1:m}) \quad \Longrightarrow \\ &\Pr(X_{n+1:n+k} = z_{1:k} \mid X_{1:n} = x_{1:n}) = \Pr(X_{m+1:m+k} = z_{1:k} \mid X_{1:m} = x'_{1:m})\,.\end{aligned}$$

Hence $R(x_{1:n}) = (h_{n-1}, x_n)$ is a sufficient statistic for predicting the entire future.

SOLUTION, p. 632

4. Representing the past

Instructions

Give a clear justification for each answer. You may use results from the slides and earlier exercises unless an exercise asks you to establish them. Questions marked **Extension** are optional.

Instructions

Exercise 4.1 (Translating between HMM diagrams and matrices). For an edge-emitting hidden Markov model, use the convention

$$T_{ij}^{(a)} := \Pr(X_{t+1} = a, S_{t+1} = j \mid S_t = i).$$

Thus rows index the current hidden state, columns index the next hidden state, and the superscript records the emitted symbol. Throughout this exercise, $0 < p, q < 1$.

(a) Suppose that $T_{ij}^{(a)} = r$. Explain each component of the following diagram:

$$i \xrightarrow{a:r} j.$$

What does a zero entry mean? Why is the matrix $T := \sum_{a \in \mathcal{X}} T^{(a)}$ row-stochastic?

(b) From matrices to a diagram: Zero–One–Random. Take the state order to be $(0, 1, R)$ and suppose that

$$\eta^{(\emptyset)} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}, \qquad T^{(0)} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 1-p & 0 & 0 \end{bmatrix}, \qquad T^{(1)} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ p & 0 & 0 \end{bmatrix}.$$

1. Draw the corresponding edge-emitting HMM. Describe informally the three phases represented by 0, 1, and R.
2. Calculate $\Pr(X_{1:3} = 011)$ and $\Pr(X_{1:3} = 000)$ in two ways: first by tracing paths through your diagram, and then from

$$\Pr(X_{1:3} = x_{1:3}) = \eta^{(\emptyset)} T^{(x_1)} T^{(x_2)} T^{(x_3)} \mathbf{1}.$$

(c) From a diagram to matrices: the Even Process. Consider the edge-emitting HMM

$$A \xrightarrow{0:q} A, \qquad A \xrightarrow{1:1-q} B, \qquad B \xrightarrow{1:1} A,$$

with no other edges. Take the state order to be (A, B) and let $\eta^{(\emptyset)} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \end{bmatrix}$.

1. Construct $T^{(0)}$ and $T^{(1)}$. Account explicitly for every zero entry in the two matrices.
2. Form $T = T^{(0)} + T^{(1)}$ and verify that it is row-stochastic.
3. Calculate $\Pr(X_{1:3} = 011)$ and $\Pr(X_{1:3} = 010)$ both by tracing paths through the diagram and by multiplying the appropriate symbol matrices. Explain how the second calculation reflects the defining constraint of the Even Process.

(d) Extension. Let $\mathcal{X}$ be the observation alphabet of an arbitrary finite-state HMM, let $T = \sum_{a \in \mathcal{X}} T^{(a)}$, and let $\eta^{(\emptyset)}$ be a probability distribution. Prove that, for every $n \geq 0$,

$$\sum_{w \in \mathcal{X}^n} \Pr(X_{1:n} = w) = \eta^{(\emptyset)} T^n \mathbf{1} = 1.$$

SOLUTION, p. 633

Exercise 4.2 (Belief states and a lossy next-token readout). Continue with the Zero–One–Random HMM from Exercise 4.1, using state order $(0, 1, R)$. For an allowable history $x_{1:n} = x_1 \cdots x_n$, compute its belief from scratch as

$$\eta^{(x_{1:n})} := \frac{\eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)}}{\eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \mathbf{1}}.$$

Here $\eta_i^{(x_{1:n})} = \Pr(S_n = i \mid X_{1:n} = x_{1:n})$. For an allowable extension $x_{1:n+1} = x_{1:n} a$, update recursively as

$$\eta^{(x_{1:n+1})} = \frac{\eta^{(x_{1:n})} T^{(a)}}{\eta^{(x_{1:n})} T^{(a)} \mathbf{1}}.$$

(a) Compute $\eta^{(0)}$ and $\eta^{(1)}$. Interpret their components as posterior probabilities over the three hidden phases.

(b) Show that the seven beliefs

$$\mathcal{B} = \{\eta^{(\emptyset)}, \eta^{(0)}, \eta^{(1)}, \eta^{(00)}, \eta^{(01)}, \eta^{(10)}, \eta^{(11)}\}$$

are distinct and that $\mathcal{B}$ is closed under every allowable symbol update. Draw the symbol-labelled transition diagram on $\mathcal{B}$.

(c) Compare your belief-state diagram with both the causal-state automaton constructed in Part 2 and the three-state generative HMM from Exercise 4.1.

1. Match each belief to the causal state with the same shortest-length representative. Do the symbol-labelled transitions agree?
2. Which beliefs correspond to knowing the hidden phase exactly, and which represent uncertainty over several phases? Show that the hidden phase becomes known after at most three observations and remains known thereafter.
3. Observe that the finite-history belief-state presentation has seven states while the generative HMM has only three. What does this suggest about the possible relative difficulty of generation and inference?

(d) For each belief $\eta^{(x_{1:n})} \in \mathcal{B}$ from part (b), compute its associated next-token probability vector

$$p^{(x_{1:n})} := (\Pr(X_{n+1} = a \mid X_{1:n} = x_{1:n}))_{a \in \{0,1\}} = \eta^{(x_{1:n})} A,$$
$$A_{ia} := \sum_j T^{(a)}_{ij}.$$

Visualise the belief geometry on the 2-simplex over hidden states $(0, 1, R)$ and the next-token geometry on the 1-simplex over symbols $(0, 1)$.

(e) Calculate

$$\Pr(X_{3:4} = 00 \mid X_{1:2} = 10) \qquad \text{and} \qquad \Pr(X_{3:4} = 00 \mid X_{1:2} = 01).$$

Deduce that the next-token probability vector need not be sufficient for predicting the entire future.

(f) Extension. Let $\delta := \eta^{(10)} - \eta^{(01)}$. Show that

$$\delta A = 0 \qquad \text{but} \qquad \delta T^{(0)} T^{(0)} \mathbf{1} \neq 0.$$

Interpret these two statements in terms of one-step and longer-horizon prediction.

SOLUTION, p. 635

Exercise 4.3 (Nonunifilar generators and large belief-state automata). For a belief η and a symbol a satisfying $\eta T^{(a)}\mathbf{1} > 0$, write

$$F_a(\eta) := \frac{\eta T^{(a)}}{\eta T^{(a)}\mathbf{1}}.$$

Thus the belief-state automaton contains the transition

$$\eta \xrightarrow{a:\eta T^{(a)}\mathbf{1}} F_a(\eta).$$

(a) The Simple Nonunifilar Source. Revisit the source from Part 2. With state order (A, B), its symbol matrices and initial belief are

$$T^{(0)} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ 0 & \frac{1}{2} \end{bmatrix}, \qquad T^{(1)} = \begin{bmatrix} 0 & 0 \\ \frac{1}{2} & 0 \end{bmatrix}, \qquad \eta^{(\emptyset)} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \end{bmatrix}.$$

1. For $n \geq 0$, show that

$$\eta^{(10^n)} = \begin{bmatrix} \dfrac{1}{n+1} & \dfrac{n}{n+1} \end{bmatrix}, \qquad \eta^{(0^n)} = \begin{bmatrix} \dfrac{1}{n+2} & \dfrac{n+1}{n+2} \end{bmatrix}.$$

Deduce that $\eta^{(0^n)} = \eta^{(10^{n+1})}$ and observe that $\eta^{(\emptyset)} = \eta^{(10)}$.

2. Show that, for every $n \geq 1$,

$$\eta^{(10^n 1)} = \eta^{(1)}.$$

3. Draw the first four states of the belief-state automaton and indicate its continuing structure with an ellipsis. Show that the beliefs $\eta^{(10^n)}$ are all distinct, identify their limiting point in the 1-simplex, and explain why the two-state generator produces a countably infinite belief-state automaton.

(b) Mess3. Now take hidden-state and symbol sets both equal to $\{0, 1, 2\}$, with

$$\alpha = \frac{1}{5}, \qquad b = \frac{1-\alpha}{2} = \frac{2}{5}, \qquad x = \frac{3}{20}, \qquad y = 1 - 2x = \frac{7}{10},$$

and

$$T^{(0)} = \begin{bmatrix} \alpha y & bx & bx \\ \alpha x & by & bx \\ \alpha x & bx & by \end{bmatrix}, \quad T^{(1)} = \begin{bmatrix} by & \alpha x & bx \\ bx & \alpha y & bx \\ bx & \alpha x & by \end{bmatrix},$$
$$T^{(2)} = \begin{bmatrix} by & bx & \alpha x \\ bx & by & \alpha x \\ bx & bx & \alpha y \end{bmatrix}, \quad \eta^{(\emptyset)} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}.$$

1. Verify that $T = \sum_{a=0}^{2} T^{(a)}$ is row-stochastic and explain why this generator is nonunifilar. Compute $\eta^{(0)}, \eta^{(1)}$, and $\eta^{(2)}$.
2. Compute $\eta^{(00)}, \eta^{(01)}$, and $\eta^{(02)}$. Use the cyclic symmetry of the process to sketch the first two generations of the belief-state automaton, including the symbol labels.

3. Compute the next-token matrix A, with $A_{ia} = \sum_j T^{(a)}_{ij}$, and show that it is invertible. What does this imply about whether distinct beliefs can be merged by the map $p^{(x_{1:n})} = \eta^{(x_{1:n})} A$? Contrast this with Exercise 4.2.
4. Using a short program, form

$$\mathcal{B}_d := \{\eta^{(x_{1:d})} : x_{1:d} \in \{0,1,2\}^d\}, \qquad 0 \le d \le 7.$$

Record the number of distinct beliefs at each depth. Visualise $\mathcal{B}_7$ on the hidden-state 2-simplex, colouring each point by its final symbol. Also visualise the associated next-token vectors on the symbol 2-simplex. Describe the geometric structure that emerges and how the three maps F_0, F_1, F_2 generate the outgoing transitions of the belief-state automaton.

5. Explain why the belief-state automata constructed in this exercise are unifilar even though their generators are not. Compare the ladder-with-resets structure of the Simple Nonunifilar Source with the branching structure of Mess3. Finally, explain why the set of beliefs reached by finite histories is countable even though its closure can have a much richer fractal geometry.

SOLUTION, p. 637

Exercise 4.4 (Generalised hidden Markov models and predictive vectors). A d-dimensional generalised hidden Markov model (GHMM) consists of a finite alphabet $\mathcal{X}$, real $d \times d$ matrices $(T^{(a)})_{a \in \mathcal{X}}$, an initial row vector $\eta^{(\emptyset)}$, and a column vector ϕ. Writing

$$T := \sum_{a \in \mathcal{X}} T^{(a)},$$

these objects satisfy

$$T\phi = \phi, \qquad \eta^{(\emptyset)}\phi = 1, \qquad \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \phi \ge 0$$

for every $x_{1:n} \in \mathcal{X}^n$ and every $n \ge 0$.

(a) A probability law from matrix products. Consider

$$\Pr(X_{1:n} = x_{1:n}) = \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \phi.$$

1. Show that these quantities are nonnegative and that

$$\sum_{x_{1:n} \in \mathcal{X}^n} \Pr(X_{1:n} = x_{1:n}) = 1$$

for every $n \ge 0$.

2. Show that

$$\sum_{a \in \mathcal{X}} \Pr(X_{1:n+1} = x_{1:n} a) = \Pr(X_{1:n} = x_{1:n}).$$

Explain why this is the consistency condition needed to define a one-sided stochastic process.

3. The GHMM conditions do not require $\eta^{(\emptyset)}T = \eta^{(\emptyset)}$. Explain why the process need not therefore be stationary. Show that this additional equality is sufficient for stationarity.

(b) Predictive vectors. For any $x_{1:n}$ satisfying $\Pr(X_{1:n} = x_{1:n}) > 0$, define its predictive vector by

$$\eta^{(x_{1:n})} := \frac{\eta^{(\emptyset)}T^{(x_1)} \cdots T^{(x_n)}}{\eta^{(\emptyset)}T^{(x_1)} \cdots T^{(x_n)}\phi}.$$

1. Show that

$$\eta^{(x_{1:n})}\phi = 1$$

and, for every continuation $y_{1:m}$,

$$\Pr(X_{n+1:n+m} = y_{1:m} \mid X_{1:n} = x_{1:n}) = \eta^{(x_{1:n})}T^{(y_1)} \cdots T^{(y_m)}\phi.$$

2. Derive the recursive update and identify its normalising factor:

$$\eta^{(x_{1:n}a)} = \frac{\eta^{(x_{1:n})}T^{(a)}}{\eta^{(x_{1:n})}T^{(a)}\phi}, \quad \Pr(X_{n+1} = a \mid X_{1:n} = x_{1:n}) = \eta^{(x_{1:n})}T^{(a)}\phi.$$

3. Conceptually contrast an HMM belief state with a GHMM predictive vector. Compare the meaning of their coordinates, their positivity and normalisation properties, and how they encode predictions of future observations.

(c) Predictive vectors need not be probability vectors. Consider the two-state HMM

$$M^{(0)} = \begin{bmatrix} \frac{4}{5} & 0 \\ 0 & \frac{1}{5} \end{bmatrix}, \qquad M^{(1)} = \begin{bmatrix} \frac{1}{5} & 0 \\ 0 & \frac{4}{5} \end{bmatrix}, \qquad \alpha = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \end{bmatrix}.$$

Thus a hidden state is selected initially and thereafter emits a biased coin without changing state. Let

$$S = \begin{bmatrix} 2 & -1 \\ 0 & 1 \end{bmatrix}, \qquad T^{(a)} := S^{-1}M^{(a)}S, \qquad \eta^{(\emptyset)} := \alpha S, \qquad \phi := S^{-1}\mathbf{1}.$$

1. Compute $T^{(0)}, T^{(1)}, \eta^{(\emptyset)}$, and ϕ. Show that, for every $x_{1:n}$,

$$\eta^{(\emptyset)}T^{(x_1)} \cdots T^{(x_n)}\phi = \alpha M^{(x_1)} \cdots M^{(x_n)}\mathbf{1}.$$

Deduce that this change of coordinates gives a valid GHMM for the same process.

2. Compute $\eta^{(0)}$ and $\eta^{(1)}$. Verify directly from $\eta^{(0)}$ that

$$\Pr(X_2 = 0 \mid X_1 = 0) = \frac{17}{25}, \qquad \Pr(X_2 = 1 \mid X_1 = 0) = \frac{8}{25}.$$

Explain why the entries of a GHMM predictive vector need not themselves be probabilities.

(d) The probability matrix and minimal dimension. For finite sets $\mathcal{U}, \mathcal{V} \subseteq \mathcal{X}^*$, define the probability matrix by

$$\mathsf{P}_{\mathcal{U},\mathcal{V}}(u, v) := \Pr(X_{1:|u|+|v|} = uv), \qquad u \in \mathcal{U}, \quad v \in \mathcal{V}.$$

1. Use the GHMM matrix product to factor $\mathsf{P}_{\mathcal{U},\mathcal{V}}$ as a matrix of prefix row vectors times a matrix of continuation column vectors. Deduce that

$$\operatorname{rank} \mathsf{P}_{\mathcal{U},\mathcal{V}} \leq d$$

for every d-dimensional GHMM realisation.

2. For the process in part (c), take $\mathcal{U} = \mathcal{V} = \{\emptyset, 0, 1\}$. Compute $\mathsf{P}_{\mathcal{U},\mathcal{V}}$ and use an SVD to find its singular values. Deduce that the two-dimensional GHMM is minimal-dimensional.

SOLUTION, p. 639

5. Identifying belief geometry in transformers

So far, we have developed the conceptual framework of computational mechanics. We now ask whether neural networks trained for next-token prediction—including transformers and recurrent architectures—learn the predictive structures identified by that framework. You will read selected sections of a paper, then reflect on its evidence and limitations.

5.1 Readings

Prioritise the main reading. If you can answer its evaluation question, you have probably understood one of its central results.

5.1.1 Transformers represent belief state geometry in their residual stream

Reading route

- **Skim.** Sections 1, 2.1, and 2.2.
- **Read.** Sections 2.3 and 3–5.
- **Evaluation.** Explain Figure 6D. What do the comparison and shuffle control establish?

Overview. This paper presents evidence that transformers trained on HMM-generated sequences encode belief-state geometry in their residual streams. The claim is tested on the Mess3 and Random–Random–XOR (RRXOR) processes. For each process, the authors fit an affine map from residual activations to the exact HMM belief state and

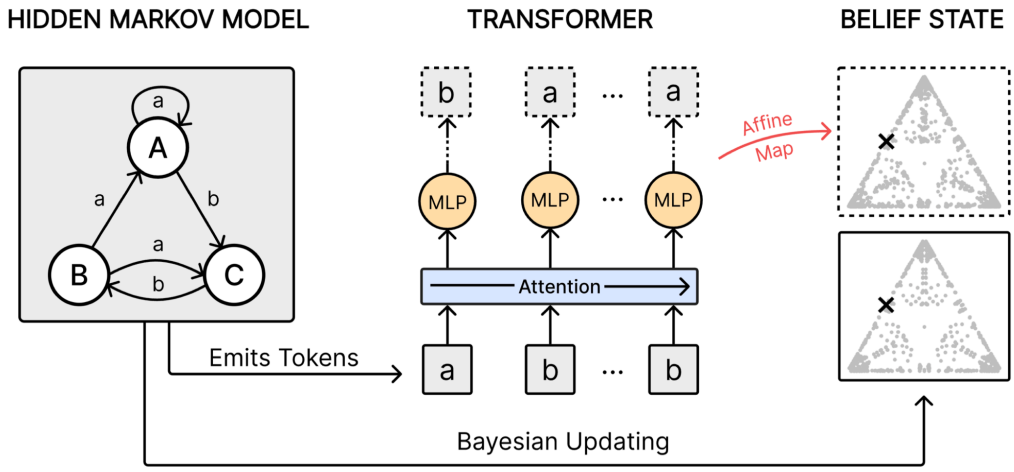
evaluate it using mean-squared error (MSE). The error decreases over training checkpoints. A shuffle control tests whether low MSE could arise merely because a high-dimensional activation space is being projected into a low-dimensional belief space.

→ *5.1.2 [Extension] Neural networks leverage nominally quantum and post-quantum representations*: Supplementary Material, p. 520

5.2 Discussion

Here are some prompts to consider:

- What is the most convincing evidence that transformers represent belief-state geometry? What is the least convincing?
- What additional experiment would most strengthen or weaken the claim?
- Which alternative hypotheses could explain the reported results?
- Can you explain each element of the diagram below—including the generative HMM, Bayesian updating, transformer activations, fitted affine map, and belief-state geometry?



6. Python exercises: HMM from scratch

The Python exercises are ARENA-style notebooks in two parts. Each part has an **exercises** notebook (with code stubs, inline tests and solutions) and a fully worked **solutions** notebook.

- **Part 1 — Sequence probabilities & next-token distributions** (exercises) — Colab notebook
- Part 1 (solutions) — Colab notebook
- **Part 2 — Belief states** (exercises) — Colab notebook
- Part 2 (solutions) — Colab notebook

The Colab notebooks are self-contained: their setup cells write the helper modules into the session, so there is nothing else to install or download.

→ *7. Learn more*: Supplementary Material, p. 521

CHAPTER C.4

Condensation

Satya Benson

Organizing information across a family of observations. Define latent models, examine reconstruction and conditional independence, and prove an exact and quantitative correspondence theorem. A worked hierarchical-clustering example, and how condensation relates to earlier work in statistics and what remains between the theorem and useful translation between the latents of human and AI models.

Source: iliad-intensive.org/interpretability/condensation · CC BY 4.0 · Slides: part 1, part 2

VIDEO *The Mathematics of Concepts: How AIs See The World* · youtu.be/odsHleu8ox8

1. Prerequisites

1. We use probability on finite sets, conditional probability, and elementary proofs.
2. All random variables in this sheet have finite ranges.
3. All logarithms have base two.
4. We use conditional entropy, conditional independence, and the distinction between an individual latent and a collection of latents.

WHAT YOU'LL LEARN

Condensation

- Construct a latent model and distinguish LVM from reconstruction.
- Explain what the Markov condition excludes, using a concrete example.
- Prove correspondence between collections of latents, first exactly and then with explicit error terms.
- Distinguish the correspondence theorem from claims about learning, computationally finding a translation, or choosing observables.

A hierarchical clustering example

- Distinguish LVM, where the latents assigned to an observation determine it, from reconstruction, where an observation recovers those latents.
- Compare the information charged by different latent models for the same query.

Condensation in context

- Relate condensation to sufficient statistics, common information, computational mechanics, and graphical models.
- Separate informational correspondence from finding and using a translation.
- Identify which additional claims would connect correspondence to AI safety.

How to use this sheet

Read Sections 2.1 and 2.2, then work on Exercises 2.1–2.3. Next, read Section 2.3 and complete Exercises 2.4–2.8. These eight exercises form the core route. Exercises 2.9 and 2.10 are optional extensions, given in the supplementary material.

1.1 Information theory reference

For a finite-valued variable U,

$$H(U) = -\sum_u p(u) \log p(u), \qquad H(U \mid V) = \sum_v p(v) H(U \mid V = v),$$

with $0 \log 0 = 0$. Conditional entropy is nonnegative. It is zero precisely when U is a function of V almost surely: values on probability-zero events do not matter. The chain rule and conditioning inequality are

$$H(U, V \mid W) = H(U \mid W) + H(V \mid U, W), \qquad H(U \mid V, W) \leq H(U \mid W).$$

Conditional mutual information is

$$I(U; V \mid W) = H(U \mid W) - H(U \mid V, W) \geq 0.$$

It is symmetric in U, V and is zero precisely when U, V are conditionally independent given W. Omitting W gives mutual information $I(U; V)$. Entropy of a tuple means joint entropy, not the sum of its marginal entropies. An empty tuple is constant and has zero entropy. For probability distributions p, q on a finite set,

$$D_{\mathrm{KL}}(p \| q) = \sum_u p(u) \log \frac{p(u)}{q(u)} \geq 0.$$

This last quantity appears only in the optional score discussion.

2. Condensation

2.1 A family of questions

Fix jointly distributed observables $X_1, \ldots, X_n$. We look for latent variables, each assigned to a set of observations, such that the latents assigned to an observation determine it. We will give conditions under which corresponding latent collections in

two such models determine one another, and bound the remaining conditional entropy when the conditions hold approximately. We take the joint distributions as given.

Remark (Choosing the observables). One can write $X_i = f_i(S)$ for an underlying variable S representing sense data or a physical state. This introduces no restriction: S could simply be the tuple of all observations. An observable distinguishes outcomes by partitioning them into its possible answers; random-variable values label those parts. Information quantities do not depend on bijective relabeling of answers. The choice of which observables to include, and their joint law, remains part of the model.

Example 2.1 (Overlapping observations). Let S, P, Q be independent fair bits and set

$$X_1 = (S, P), \qquad X_2 = (S, Q), \qquad X_3 = (P, Q).$$

Use a latent $Y_{\{1,2\}} = S$ for observations 1 and 2, a latent $Y_{\{1,3\}} = P$ for observations 1 and 3, and a latent $Y_{\{2,3\}} = Q$ for observations 2 and 3. Each observation is determined by the two latents assigned to it. Each of these latents can also be recovered from either observation to which it is assigned.

2.1.1 Addresses and collections

Let $I = \{1, \dots, n\}$ and write $\mathcal{P}^+(I)$ for its nonempty subsets. An *address* $A \in \mathcal{P}^+(I)$ specifies which observations receive a latent Y_A. The collection of addresses is a partially ordered set under inclusion. We use one slot for each nonempty subset, setting unused slots to constants.

Definition 2.2 (Latent model). A latent model of $X = (X_i)_{i \in I}$ consists of finite-valued variables $Y = (Y_A)_{A \in \mathcal{P}^+(I)}$ jointly distributed with X, such that

$$H\big(X_i \mid Y_{\{A \in \mathcal{P}^+(I) \,:\, i \in A\}}\big) = 0 \quad \text{for every } i \in I.$$

Here $Y_{\mathcal{F}} = (Y_A)_{A \in \mathcal{F}}$ for a family $\mathcal{F} \subseteq \mathcal{P}^+(I)$.

We call (1) the *latent variable model condition*, or *LVM*. The contributing latents, taken together, determine the observation. This is not a claim that an agent knows their realized values. Some slots may represent residual noise. Also, an address need not be minimal: being assigned to an observation does not assert that a slot makes a nonredundant or causal contribution to it.

For nonempty $A \subseteq I$ and $B \subseteq I$, define two different families:

$$\supseteq A = \{C \in \mathcal{P}^+(I) : A \subseteq C\}, \qquad \mathcal{J}_B = \{C \in \mathcal{P}^+(I) : C \cap B \neq \varnothing\}.$$

The tuple $Y_{\supseteq A}$ is the *tower above* A. The tuple $Y_{\mathcal{J}_B}$ contains all latents contributing to at least one observation in B, and therefore determines $X_B = (X_i)_{i \in B}$. In particular $\supseteq\{i\} = \mathcal{J}_{\{i\}}$ is the family of addresses contributing to X_i, so (1) reads $H(X_i \mid Y_{\supseteq\{i\}}) = 0$. For a larger A, $\supseteq A$ and $\mathcal{J}_A$ generally differ.

Exercise 2.1 (★) (Overlapping bits). Use Example 2.1, with all unmentioned slots constant.

(a) Compute $H(X_1)$, $H(X_1, X_2, X_3)$, and $I(X_1; X_2)$.

(b) List $\supseteq\{1\}$, $\supseteq\{2\}$, and $\supseteq\{1,2\}$, including constant slots. Which tuple determines X_1 ? Which of the three is $\supseteq\{1\} \cap \supseteq\{2\}$?

(c) Give explicit functions recovering X_1 from its contributing latents and recovering $Y_{\{1,2\}}$ from X_1.

(d) Does the address structure have to be a tree? Identify two overlapping addresses in this example neither of which contains the other.

SOLUTION, p. 643

2.2 Two conditions on a latent model

LVM is not one of the two conditions below. It is part of the definition of a latent model, and every observable law admits one by construction: put a copy of each X_i in its singleton slot, or the entire tuple X in the top slot Y_I, and LVM holds either way. Satisfying it therefore says nothing about how information is shared. The two conditions below constrain these alternatives in different ways.

Definition 2.3 (Reconstruction). A latent model has exact reconstruction if

$$H(Y_A \mid X_i) = 0 \qquad \text{whenever } i \in A.$$

Equivalently, $H(Y_{\supseteq A} \mid X_i) = 0$ whenever $i \in A$: every slot in that finite tuple can be recovered from X_i.

The equivalence in this definition uses exact zero errors. In approximate statements we will bound the entropy of the *whole tower*; separately small errors for its individual slots could add up.

A family $\mathcal{F} \subseteq \mathcal{P}^+(I)$ is *upward closed* if $A \in \mathcal{F}$ and $A \subseteq C$ imply $C \in \mathcal{F}$. For example, $\supseteq A$ and $\mathcal{J}_B$ are upward closed. Taking a collection together with every slot above it prevents us from omitting higher variables on which it may depend.

Definition 2.4 (Ordered Markov condition and defect). The ordered Markov condition is

$$I(Y_{\mathcal{F}}; Y_{\mathcal{G}} \mid Y_{\mathcal{F}\cap\mathcal{G}}) = 0$$

for all upward-closed families $\mathcal{F}, \mathcal{G} \subseteq \mathcal{P}^+(I)$. Its quantitative defect is

$$\eta_Y = \max_{\mathcal{F},\mathcal{G}\ \text{upward closed}} I(Y_{\mathcal{F}}; Y_{\mathcal{G}} \mid Y_{\mathcal{F}\cap\mathcal{G}}).$$

The conditioning family $\mathcal{F} \cap \mathcal{G}$ is an intersection of *address families*, not a quantity inferred to be common by inspecting their values. The condition says that the latents at those shared addresses account for the dependence between the two collections. It does not require every latent to be independent of every other latent. In the two-observable case, its only potentially nonzero requirement is

$$I(Y_{\{1\}}; Y_{\{2\}} \mid Y_{\{1,2\}}) = 0.$$

This is a graphical-model condition for the inclusion order. The equivalent local version says that each slot is conditionally independent of all incomparable slots given its strict supersets.

Definition 2.5 (Perfect condensation). A latent model is a perfect condensation if it has exact reconstruction and satisfies the ordered Markov condition.

This is equivalent to Eisenstat's definition by optimal conditioned scores (Eisenstat 2025, Theorem 5.10). Section 2.5 explains that connection. The overlapping-bit model in Example 2.1 is a perfect condensation.

Exercise 2.2 (★) (Two failed models). Keep the observables of Example 2.1. Consider two alternative models.

(a) Set $Y_{\{i\}} = X_i$ for $i = 1, 2, 3$ and every other slot constant. Check LVM and reconstruction. Take $\mathcal{F} = \supseteq\{1\}$ and $\mathcal{G} = \supseteq\{2\}$. Calculate the conditional mutual information in Definition 2.4. Which information is duplicated across singleton slots?

(b) Instead set $Y_{\{1,2,3\}} = (S, P, Q)$ and every other slot constant. Check LVM and the Markov condition. Compute $H(Y_{\{1,2,3\}} \mid X_1)$. Which condition fails?

(c) Explain why LVM, reconstruction, and the Markov condition have different roles.

SOLUTION, p. 644

2.2.1 Existence is a separate question

The preceding examples test particular latent models. A stronger question is whether a given observable law admits *any* perfect condensation. Even a pair of correlated binary variables can fail to do so.

Let $I = \{1, 2\}$, let X_1 be a fair bit, and set $X_2 = X_1 \oplus N$, where N is independent Bernoulli(q) noise and $0 < q < 1/2$. Here $\oplus$ is addition modulo two, so $a \oplus b$ is 0 when the two bits agree and 1 when they differ. The joint law is

$P(x_1, x_2)$	$x_2 = 0$	$x_2 = 1$
$x_1 = 0$	$\frac{1-q}{2}$	$\frac{q}{2}$
$x_1 = 1$	$\frac{q}{2}$	$\frac{1-q}{2}$

Every one of the four outcomes has positive probability, because $0 < q < 1/2$; and X_1, X_2 are dependent, because the two rows differ. Suppose a perfect condensation existed, and consider its top slot $Y_{\{1,2\}}$.

With two observations there are only three addresses, so the whole model is three slots:

address	$\{1\}$	$\{1,2\}$	$\{2\}$
slot	$Y_{\{1\}}$	$Y_{\{1,2\}}$	$Y_{\{2\}}$

LVM says X_1 is determined by $(Y_{\{1\}}, Y_{\{1,2\}})$ and X_2 by $(Y_{\{2\}}, Y_{\{1,2\}})$. The top slot $Y_{\{1,2\}}$ is the only one assigned to both observations, so it is the only place shared information can sit.

Reconstruction would give $Y_{\{1,2\}} = f(X_1) = g(X_2)$ almost surely. Because all four observable pairs have positive probability, these equalities force

$$f(0) = g(0) = g(1) = f(1).$$

Thus $Y_{\{1,2\}}$ is constant. The Markov condition would then require $Y_{\{1\}}$ and $Y_{\{2\}}$ to be independent. LVM makes each X_i a function of its singleton slot and the constant $Y_{\{1,2\}}$. Functions of independent variables are independent, contradicting the chosen observable law. No perfect condensation exists for this pair.

There is a quantitative obstruction if we retain *exact* reconstruction. The same argument makes $Y_{\{1,2\}}$ constant. Data processing then gives

$$\eta_Y = I(Y_{\{1\}}; Y_{\{2\}}) \geq I(X_1; X_2) = 1 - h_2(q) > 0,$$

where $h_2(q) = -q \log q - (1-q)\log(1-q)$ is the binary entropy function. Dependence between the observables must remain as a Markov defect when their separately recoverable top slot is constant.

That bound pins one end of a trade-off: exact reconstruction forces a Markov defect of at least $1 - h_2(q)$, and tolerating some reconstruction error buys part of it back. When no perfect condensation exists, which is the usual situation, the latent models of a given observable law trace out a Pareto frontier between reconstruction defect and Markov defect. The useful question is then whether some point on that frontier has both low *enough* for the correspondence bound of Section 2.3 to say something.

Remark. One could instead ask for the model minimizing the conditioned score of Section 2.5. The score is defined per observation block, however, and there is not always a single model minimizing it everywhere.

Exercise 2.3 (★) (Collections, not individual slots). For this exercise let $I = \{1,2\}$, take independent fair bits C, R, T, and set $X_1 = (C,R)$, $X_2 = (C,T)$. Compare

$$\begin{aligned} &Y_{\{1,2\}} = C, \quad Y_{\{1\}} = R, \qquad\quad Y_{\{2\}} = T, \\ &Z_{\{1,2\}} = C, \quad Z_{\{1\}} = (C,R), \quad Z_{\{2\}} = (C,T). \end{aligned}$$

(a) Check that both models are perfect condensations.

(b) Compute $H(Z_{\{1\}} \mid Y_{\{1\}})$.

(c) Show that $Y_{\supseteq\{1\}}$ and $Z_{\supseteq\{1\}}$ determine one another. Explain why correspondence between these tuples is a different claim from correspondence between the singleton slots.

SOLUTION, p. 645

2.3 The correspondence theorem

Take two latent models of the same observable tuple X. To compare them, put (X, Y, Z) on one probability space while retaining both specified laws (X, Y) and (X, Z). For finite variables this is always possible: for example, use $p(x, y, z) = p(x)p(y \mid x)p(z \mid x)$. The theorem holds for any such coupling.

Theorem 2.6 (Correspondence). For two latent models Y, Z of X and any nonempty $A \subseteq I$, let $k = |A|$. Then

$$H(Y_{\supseteq A} \mid Z_{\supseteq A}) \leq \sum_{i \in A} H(Y_{\supseteq A} \mid X_i) + (k-1)\eta_Z.$$

If each term in the sum is at most δ and $\eta_Z \leq \eta$, the bound is $k\delta + (k-1)\eta$. If both budgets are ε, it is $(2k-1)\varepsilon$. Interchanging Y, Z gives the reverse bound.

Corollary 2.7 (Exact correspondence). If Y and Z are perfect condensations, their towers above every nonempty address A determine one another almost surely:

$$H(Y_{\supseteq A} \mid Z_{\supseteq A}) = H(Z_{\supseteq A} \mid Y_{\supseteq A}) = 0.$$

The forward bound uses reconstruction in the sender Y, LVM in the receiver Z, and the Markov defect of Z. It does not use the Markov condition of Y until we reverse the argument. The result concerns corresponding collections. Exercise 2.3 shows why we cannot simply replace those collections by individual slots.

This proof is a specialization of Eisenstat's approximate correspondence theorem (Eisenstat 2025, Theorem 6.8), using the two-witness inequality (Eisenstat 2025, Lemma 6.4). Its tower presentation follows the exposition listed in Section 6.1. The exercises below give the full argument without additional machinery.

2.3.1 Proof roadmap

The theorem has three steps.

1. If two witnesses each determine a target and are independent given their common part, that common part determines the target. Entropy gives a version with errors.

2. Each observation X_i is determined by its contributing Z-latents. Anything recoverable from X_i is therefore recoverable from that collection.
3. Intersect these contributing collections over $i \in A$. Exactly the addresses containing all of A survive. Each intersection costs at most one Markov error term.

The word "witness" below means a variable or tuple from which the target is recoverable.

Lemma 2.8 (Two witnesses). For any finite-valued variables U, M, P, Q,

$$H(U \mid M) \leq H(U \mid M, P) + H(U \mid M, Q) + I(P; Q \mid M).$$

Exercise 2.4 (★) (Prove the two-witness lemma). (a) Prove Lemma 2.8 by expanding conditional entropies using the chain rule. Find an expression for the difference between the right and left sides that is a sum of nonnegative quantities.

(b) Deduce the exact statement when the three terms on the right are zero.

(c) Explain the three error terms. Which measure failures of reconstruction, and which measures dependence that remains after revealing M?

HINT, p. 645 · SOLUTION, p. 645

Exercise 2.5 (The dependence term is needed). Let $U = P = Q$ be a fair bit and let M be constant. Evaluate every term in (3). What would go wrong if the mutual-information term were omitted? How does this example resemble the singleton-copy model in Exercise 2.2?

SOLUTION, p. 646

Exercise 2.6 (★) (Transfer through an observable). Let U be any finite-valued target and suppose X_i is determined by $Z_{\supseteq\{i\}}$. Prove

$$H(U \mid Z_{\supseteq\{i\}}) \leq H(U \mid X_i).$$

State the equality obtained by adding X_i to the conditioning tuple, then apply the conditioning inequality. Does this argument assume that U and $Z_{\supseteq\{i\}}$ are conditionally independent given X_i?

SOLUTION, p. 646

Exercise 2.7 (★) (Intersect the address families). (a) For $I = \{1, 2, 3\}$, list $\supseteq\{1\}$, $\supseteq\{2\}$, and $\supseteq\{3\}$. Compute $\supseteq\{1\} \cap \supseteq\{2\}$ and then its intersection with $\supseteq\{3\}$.

(b) Show that an intersection of upward-closed families is upward closed.

(c) For general nonempty $A \subseteq I$, prove $\bigcap_{i \in A} \supseteq\{i\} = \supseteq A$ by checking membership of an address C on both sides.

(d) If $\mathcal{F}, \mathcal{G}$ are upward closed, apply the two-witness lemma with $M = Z_{\mathcal{F}\cap\mathcal{G}}$, $P = Z_{\mathcal{F}}$, and $Q = Z_{\mathcal{G}}$. Why can the extra conditioning on M be dropped from each reconstruction term? Write the resulting bound using η_Z.

SOLUTION, p. 647

Exercise 2.8 (★) (Complete the correspondence proof). Fix $A = \{i_1, \dots, i_k\}$, set $\mathcal{G}_j = \bigcap_{\ell=1}^{j} \supseteq\{i_\ell\}$.

(a) Use Exercise 2.6 to prove the bound at $j = 1$.

(b) Use Exercise 2.7 to show, for $j \geq 2$,

$$H(Y_{\supseteq A} \mid Z_{\mathcal{G}_j}) \leq H(Y_{\supseteq A} \mid Z_{\mathcal{G}_{j-1}}) + H(Y_{\supseteq A} \mid Z_{\supseteq\{i_j\}}) + \eta_Z.$$

(c) Prove by induction that

$$H(Y_{\supseteq A} \mid Z_{\mathcal{G}_j}) \leq \sum_{\ell=1}^{j} H(Y_{\supseteq A} \mid X_{i_\ell}) + (j-1)\eta_Z.$$

Evaluate this at $j = k$ to prove Theorem 2.6. Check the case $k = 1$ explicitly.

(d) Deduce Corollary 2.7, including the reverse direction. Name the assumption used at each step of the proof.

(e) Suppose instead that $H(Y_C \mid X_i) \leq \varepsilon$ separately for all relevant slots C. Why does this not by itself supply the tower error budget $H(Y_{\supseteq A} \mid X_i) \leq \varepsilon$?

SOLUTION, p. 647

→ *2.4 Optional: several observations as one witness*: Supplementary Material, p. 522

→ *2.5 Optional: the conditioned score*: Supplementary Material, p. 522

2.6 What the result says, and connections

The theorem compares two joint laws extending the same observable law. Under the exact conditions, the two latent towers at each address determine one another almost surely; under the quantitative hypotheses, their conditional entropies satisfy the stated bounds. The theorem does not select the observable family or give a learning algorithm. A perfect condensation need not exist, as the full-support example in Section 2.2.1 shows.

2.6.1 Connections to established questions

In statistics, a statistic is sufficient for a parameter when it retains the data's information relevant to that parameter. Predictive sufficiency instead concerns retaining information about a target to be predicted. Condensation studies the organization of information across an indexed family of observations, with explicit contribution addresses and a correspondence result. It shares questions about information retention with sufficiency, but these are not identical definitions or claims.

Graphical models supply the language of conditional independence and factorization used by the ordered Markov condition. The graph here is fixed by inclusion of addresses; its edges do not by themselves have a causal meaning. Common-information questions also ask what two variables share. Our correspondence argument applies across a family of overlapping address collections, using the same entropy calculus rather than a new information measure. Computational mechanics studies predictive states obtained from histories with the same conditional future law. Condensation's formulation is static: time, learning, and a prescribed past/future split are not required. None of these comparisons implies that earlier frameworks cannot study structured or modular organizations of information.

2.6.2 Limits of information-theoretic correspondence

If $V = f(U)$ for a bijection f, then $H(U \mid V) = H(V \mid U) = 0$. This remains true when describing or discovering f is expensive. The theorem does not provide an efficient translation algorithm or account for the description length of the probability law.

Small conditional entropy is an average guarantee. If E is Bernoulli(p), D is an independent fair bit, $U = (E, ED)$, and $V = E$, then $H(U \mid V) = p$. On the rare event $E = 1$, the missing bit is still completely unknown. Conversely, a rare event with an enormous number of outcomes can dominate an entropy. A safety application needs to say which failures matter, not just report an average information budget.

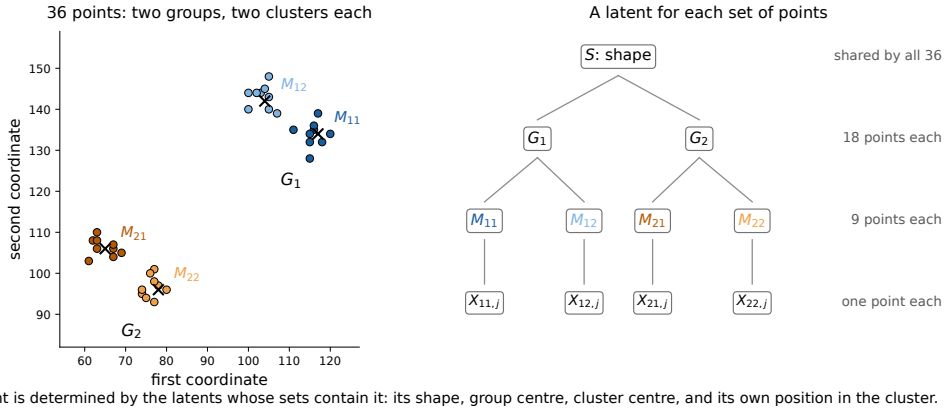
2.6.3 Discussion prompts

1. Choose a comparison from Section 4.2. State one mathematical similarity and one difference in what is being characterized. If it relates to one of the condensation conditions, say how.
2. Corresponding latent collections in two models determine one another with zero conditional entropy. What does this establish, and what else might be needed to find and use the translation between them?
3. Where might a correspondence like this matter for AI safety? Trace one chain from the theorem to something you would want, and say where it breaks: which steps does the theorem not establish? Section 4.4 sets out one such chain.

3. A hierarchical clustering example

3.1 Thirty-six observations

There are two groups, each containing two clusters, each with nine observations. Write X_{gcj} for the jth two-dimensional point in cluster c of group g. The labels (g, c, j) and memberships are fixed. We are comparing models of this labeled family, not inferring an unknown partition of an unlabeled cloud.



Each point is determined by the latents whose sets contain it: its shape, group centre, cluster centre, and its own position in the cluster.

One draw from the finite model. The left panel shows the 36 points; the right panel shows which set of observations receives each parameter, and is not a graph inferred from the points. The global shape S is available to every observation.

The parameters are a global shape S, group centers G_1, G_2, and cluster centers M_{gc}. Given these parameters, the 36 observations are independent. Marginalizing over shared parameters generally makes observations dependent. The plotted cloud is one realization; the random variables and their entropies refer to the joint law across realizations.

3.2 An explicit finite law

Let $D = \{0, \dots, 200\}^2$. Draw S uniformly from three shape labels, and draw G_1, G_2 independently and uniformly from $\{60, \dots, 140\}^2$, independently of S. Conditional on G_g, draw its two centers independently:

$$P(M_{gc} = m \mid G_g = a) \;\propto\; \exp\!\left(-\frac{\|m - a\|^2}{128}\right), \qquad m \in D.$$

Given S and all centers, draw the observations independently, using

$$P(X_{gcj} = x \mid M_{gc} = m, S = s) \;\propto\; \exp\!\left(-\tfrac{1}{2}(x - m)^{\mathsf{T}} Q_s^{-1} (x - m)\right), \quad x \in D,$$

where

$$Q_{\text{round}} = \begin{pmatrix} 5 & 0 \\ 0 & 5 \end{pmatrix}, \quad Q_{\text{right}} = \begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}, \quad Q_{\text{left}} = \begin{pmatrix} 25 & -20 \\ -20 & 25 \end{pmatrix}.$$

Each proportionality means normalization by the sum over D. These are probability mass functions on a finite grid. The matrices control shape; they are not the exact covariances after discretization and boundary truncation. Every grid point has positive probability. Cluster centers tend to lie near their group center, but the four components need not be visually separated in every realization.

3.3 A structured latent model

Let I contain all 36 indices. Let A_g be the 18 indices of group g, and A_{gc} the nine indices of cluster (g, c). Assign

$$Y_I = S, \qquad Y_{A_g} = G_g, \qquad Y_{A_{gc}} = M_{gc}, \qquad Y_{\{i\}} = X_i,$$

and make every other slot constant. The joint law factorizes in the address inclusion order, so its ordered Markov defect is zero.

The singleton slot supplies the sampled point itself. Thus $H\big(X_i \mid Y_{\{A\,:\,i\in A\}}\big) = 0$ exactly. The cluster center and shape alone specify a conditional distribution for the point; they do not determine its realized coordinates. The residual conditional entropy is $H(X_i \mid M_{gc}, S)$.

Reconstruction is different. All the kernels have full support, so any finite observation block leaves several parameter configurations with positive posterior probability. For example, $H(M_{gc} \mid X_i) > 0$. The model's joint law satisfies the Markov condition exactly, but it is not a perfect condensation.

For a nonempty block B, the general score-gap identity (5) is

$$\chi(B) - H(X_B) = \big[\chi(B) - H(Y_{\mathcal{J}_B})\big] + H(Y_{\mathcal{J}_B} \mid X_B).$$

Here the bracketed Markov term is zero. The gap is the latent information remaining uncertain after observing the queried block.

3.4 Three organizations of the same observable law

For a nonempty queried *observation block* B, the conditioned score sums the charges of all slots whose addresses meet B.

One global tuple. Put $Z_I = X_I$ and make every other slot constant. The global slot determines each observation. For every nonempty B,

$$\chi_{\text{global}}(B) = H(X_I), \qquad \chi_{\text{global}}(B) - H(X_B) = H(X_I \mid X_B).$$

This attains the entropy bound when querying the whole tuple. Even a singleton query incurs the entire tuple's entropy in this score.

An ordered chain. Choose an ordering $X_1, \ldots, X_{36}$. Set $Z_{\{j,j+1,\ldots,36\}} = X_j$ and make other slots constant. The contributing slots for X_i contain $X_1, \ldots, X_i$. The charge for slot j is $H(X_j \mid X_1, \ldots, X_{j-1})$, hence

$$\chi_{\text{chain}}(B) = H(X_1, \ldots, X_{\max B}).$$

This attains the entropy bound for prefix queries. Querying only X_{36} costs $H(X_I)$. These statements concern this particular address assignment, not every autoregressive architecture.

The structured model. The score charges the shape once, the group and cluster parameters needed by B, and the residual uncertainty of each requested observation:

$$\chi_{\text{structured}}(B) = H(S) + \sum_{g:B\cap A_g\neq\varnothing} H(G_g) + \sum_{g,c:B\cap A_{gc}\neq\varnothing} H(M_{gc} \mid G_g) + \sum_{i\in B} H(X_i \mid M_{g(i)c(i)}, S).$$

This expression avoids charging for unrelated observations. It need not be smaller than the other scores for every block. In particular, for $B = I$ it exceeds $H(X_I)$ by the remaining parameter uncertainty, whereas both baseline constructions attain $H(X_I)$. Fitting the observable law does not by itself settle which organization is preferable across queries.

3.5 Why blocks help recover parameters

For nonempty B inside a fixed cluster (g, c), set $\Theta = (S, G_g, M_{gc})$. The nonconstant contributing tuple is (Θ, X_B), so

$$\chi_{\text{structured}}(B) - H(X_B) = H(\Theta \mid X_B).$$

For nonempty $B \subseteq B' \subseteq A_{gc}$, the reduction in this gap is

$$H(\Theta \mid X_B) - H(\Theta \mid X_{B'}) = I(\Theta; X_{B'\setminus B} \mid X_B) \geq 0.$$

More points can inform the shared parameters. Exact recovery still fails for finite blocks. Even knowing one cluster center need not determine its parent group center. When B extends to another cluster, its contributing parameter family changes, so the displayed monotonicity argument no longer applies as written. This example motivates block witnesses and nonzero recovery errors; it does not establish a chosen uniform error bound.

Source. The motivating hierarchy is from Gillen and Chiang, *A summary of Condensation and its relation to Natural Latents* (Gillen & Chiang 2026), §1.1.1. The finite law, illustration, and exact score comparisons above are adapted for this chapter.

4. Condensation in context

4.1 What is being characterized?

Fix jointly distributed observables $X_1, \ldots, X_n$. We organize information into latents Y_A , indexed by nonempty subsets A of the observable indices. The address specifies which observations the latent is assigned to. Each X_i is determined by its contributing collection $(Y_A : i \in A)$.

LVM alone leaves considerable freedom. The additional reconstruction and Markov conditions constrain where shared information can reside. The correspondence theorem compares two models satisfying these conditions: their collections above the same address determine one another, exactly or with a controlled conditional-entropy error.

This provides a conditional sense of objectivity. Once the observables and the conditions are fixed, agreement in information content need not come from a shared vocabulary. It follows from each model's relationship to the observations. The theorem does not choose the observables. Changing the questions changes the problem, and may change the information that must correspond.

Nor is each latent automatically a human concept. A model includes residual information needed to determine observations. The interpretation of its parts as concepts is an additional connection between the mathematics and the uses of a latent model.

4.2 Connections to prior work

4.2.1 Sufficient statistics

For a fixed joint law of (X, V), a statistic $T = f(X)$ is sufficient for predicting V when

$$I(X; V \mid T) = 0.$$

Knowing X then gives no predictive information about V beyond T. For finite alphabets, grouping values x with the same conditional distribution $P(V \mid X = x)$ gives a minimal such statistic, on the support of X. This differs from parametric sufficiency, where the conditional law of the sample given a statistic must be independent of the unknown parameter. Both formulations ask which distinctions must be retained for an inference task.

Predictive sufficiency does not say that T determines the realized value of V. Condensation's LVM condition does demand determination, using the whole contributing collection, including residual variables. A sufficient statistic can itself be a vector or another structured object.

A sufficient statistic is by construction a function of X. A latent need not be, because reconstruction is asked for only up to a defect: that leaves room for latents which are parameters of a law rather than functions of the observations. The additional question here is how to organize a family of latent parts across many specified observables. See Fisher 1922 and Shalizi's account of predictive sufficiency (Shalizi 2019).

4.2.2 Common information

Two older questions distinguish requirements that appear together in the perfect case. The maximal deterministic common part of X_1, X_2 is a variable K recoverable from either observation, through which every other common function factors. Its entropy is the Gacs–Korner common information. Wyner's common information instead minimizes $I(X_1, X_2; W)$ over extensions satisfying $X_1 \perp X_2 \mid W$. Such a W need not be separately recoverable from the observations.

If W is both separately recoverable and screens off dependence, then

$$I(X_1; X_2) = H(W) + I(X_1; X_2 \mid W) = H(W).$$

Moreover, every common function K is determined by W: the two-witness lemma, with witnesses X_1, X_2 and common part W, gives $H(K \mid W) = 0$. Thus this special case is closely connected to common information in the sense of Gacs and Korner.

It is restrictive. If X is a fair bit and $V = X \oplus N$ for independent $N \sim \text{Bernoulli}(p)$, $0 < p < 1/2$, all four pairs have positive probability. Any common function must therefore be constant, although $I(X;V) = 1 - h_2(p) > 0$, where h_2 is the binary entropy function of Section 2.2.1. Recoverable common information and mutual information differ.

Common/private source coding already studies structured organization of information. Condensation's particular focus is a family of subset-addressed latents and correspondence under conditions on that family. See Gács & Körner 1973 and Liu, Xu and Chen's extension of Wyner's common information (Liu et al. 2010).

4.2.3 Computational mechanics

Computational mechanics groups past histories according to their conditional distributions over futures. Its causal states have predictive optimality, minimality, and uniqueness properties. This is an important precedent for characterizing such a construction through informational requirements.

The present setup starts from a family of jointly distributed questions rather than a distinguished past/future split. It characterizes latent collections associated with different observable subsets. See Shalizi & Crutchfield 2001.

4.2.4 Graphical models and the information bottleneck

The ordered Markov condition in condensation is a graphical-model condition. A subset-ordered directed graph places larger addresses above smaller ones. Under the corresponding factorization, the latent joint law is

$$P(y) = \prod_{A \neq \varnothing} P(y_A \mid y_{\supsetneq A}).$$

A factorization alone does not ensure that observations recover the latents, or that different latent models correspond. Those are additional requirements. Graphical language also does not make the result a causal-identification theorem. Eisenstat discusses this connection explicitly in Definition 5.8 and Example 6.3 of the condensation paper (Eisenstat 2025).

The information bottleneck compresses an input X into T while retaining information about a specified relevance variable V. A standard objective is

$$I(X;T) - \beta I(T;V), \qquad T \perp V \mid X.$$

It provides an established distribution-based approach to useful summaries of the data. Our condensation problem instead involves many addressed latents, LVM, and conditions across observable sets. An optimum of a generic bottleneck objective does not automatically satisfy these conditions. See Tishby et al. 1999.

4.3 What information theory leaves out

4.3.1 An encoding and access to its inverse

Let X be uniform on $\{1, \dots, N\}$ and let $Y = \pi(X)$ for a fixed permutation π. Then

$$H(X \mid Y) = H(Y \mid X) = 0.$$

This is exact correspondence whether π is the identity or a large unstructured lookup table. The entropies do not charge for describing the table, learning it, storing it, or using it. For a finite table an inverse exists and is computable; that does not make it available to a particular learner.

For a concrete learning distinction, draw π uniformly from all permutations and reveal $m < N$ distinct input–output pairs. Conditional on those pairs, the image of a previously unseen input is uniform over the $N - m$ unused outputs. The best success probability without further information is $1/(N - m)$. For every fixed permutation the correspondence entropy is zero, yet a learner with few examples cannot predict an unseen pairing. These are different probability models: one fixes the permutation; the other describes uncertainty about it.

A separate question concerns computation even when a transformation has a short known description. Entropic bounds alone give no running-time bound for an inverse. Establishing computational difficulty requires an additional argument or assumption; it does not follow from this permutation example.

4.3.2 Rare events and average error

Let $E \sim \text{Bernoulli}(p)$. Define $R = 0$ when $E = 0$ and, when $E = 1$, let R be uniform over M other values. Since R reveals E,

$$H(R) = h_2(p) + p \log_2 M.$$

For $p = 10^{-6}$ and $M = 2^{10^6}$, the rare branch contributes one full bit. An independent sample of size m misses that branch with probability $(1 - p)^m$. Large informational contributions can be absent from a small dataset.

Conversely, suppose a variable T records E, and records the value of a fair bit B, independent of E, only when $E = 0$. Then

$$H(B \mid T) = p.$$

The average uncertainty is small, but conditional on the rare event $E = 1$, the bit is entirely unknown. If that event is a critical decision, its importance is not measured by its probability alone. Entropy is an average informational quantity, not a measure of the consequences of errors.

These examples distinguish informational equivalence, statistical access, and performance on important cases. A useful translation may need guarantees about all three.

4.4 The connection to AI safety

1. **Corresponding information.** Under the reconstruction and Markov conditions, the theorem gives correspondence between specified collections in two models of the same observables.
2. **Concepts of actual agents.** To apply that result, we need to identify suitable variables in actual agents and establish the relevant conditions, including how their observables relate. The theorem does not show that learning procedures produce such models.
3. **A usable translation.** Informational correspondence could support identifying which of an AI's latents carries something humans care about. We must still find the correspondence with available data and computation, and connect it to the particular human distinction of interest.
4. **Interpretation and specification.** A usable translation could help interpret predictions or express objectives in an AI's world model. This requires preserving relevant distinctions when circumstances or models change.
5. **Safer systems.** Better interpretation and specification could improve oversight and alignment. Sharing concepts does not entail sharing values, reporting faithfully, or following an intended objective.

The first step is the mathematical result of this chapter. The subsequent steps state a research program and the additional work required to connect it to safety.

4.5 Discussion

1. **Choosing the questions.** Consider two agents describing the same physical system, one using local measurements and another using a different collection of measurements. What would have to be established before applying a theorem that assumes the same observables? Distinguish changing value labels from changing which distinctions an observable makes. Identify one restriction on observable choice that seems justified for a concrete application.
2. **What would count as a useful translation?** Suppose two latent tuples U, V satisfy $H(U \mid V) = H(V \mid U) = 0$. Specify an additional test that would make this correspondence useful for interpreting a learned model. State what access the test requires: paired samples, interventions, model internals, or a known decoder. Which part of the permutation example would that access resolve?
3. **From correspondence to oversight.** Choose one human distinction that matters for evaluating an AI's behavior. Trace the steps from a correspondence theorem to an oversight procedure that uses that distinction. Name a failure that could occur despite exact informational correspondence, and state what additional evidence would address it.

5. Review questions

5.1 Questions

Use base-two logarithms. Throughout, S, P, Q are independent fair bits and $X_1 = (S, P)$, $X_2 = (S, Q)$, $X_3 = (P, Q)$, with latents $Y_{\{1,2\}} = S$, $Y_{\{1,3\}} = P$ and $Y_{\{2,3\}} = Q$.

1. What is $I(X_1; X_2)$?
   - 0
   - 1
   - 2
   - 3
2. Same model. What is $H(Y_{\{1,2\}} \mid X_1)$?
   - 0
   - $1/2$
   - 1
   - 2
3. Now put $Y_{\{i\}} = X_i$ at each singleton address, with every other slot constant. Which condition fails?
   - the latent variable model condition (LVM)
   - the reconstruction condition
   - the ordered Markov condition
   - none of them fails
4. Complete the two-witness bound $H(U \mid M) \le H(U \mid M, P) + H(U \mid M, Q) +$?
   - $I(P; Q)$
   - $I(P; Q \mid M)$
   - $H(P \mid Q)$
   - 0
5. In the correspondence theorem with $|A| = k$, suppose every reconstruction error and every Markov defect is at most ε. What is the bound on $H(Y_{\supseteq A} \mid Z_{\supseteq A})$?
   - $k\varepsilon$
   - $(k-1)\varepsilon$
   - $(2k-1)\varepsilon$
   - $2k\varepsilon$
6. Two corresponding latent collections satisfy $H(U \mid V) = H(V \mid U) = 0$. Which of these does that *not* establish?
   - each collection is a function of the other almost everywhere

- neither collection retains uncertainty about the other
- an efficient procedure for translating between them
- the two collections carry the same information

→ *6. Further reading*: Supplementary Material, p. 523

References

Satya Benson (2026). *Relating Almost Perfect Condensation Conditions and the Conditioned Score*.

Thomas M. Cover and Joy A. Thomas (2006). *Elements of Information Theory*. Wiley.

Sam Eisenstat (2025). *Condensation: A Theory of Concepts*.

R. A. Fisher (1922). *On the Mathematical Foundations of Theoretical Statistics*. Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character.

Péter Gács and János Körner (1973). *Common Information is Far Less Than Mutual Information*. Problems of Control and Information Theory.

Jeremy Gillen and Daniel Chiang (2026). *A Summary of Condensation and its Relation to Natural Latents*.

Steffen L. Lauritzen (1996). *Graphical Models*. Oxford University Press.

Wei Liu, Ge Xu, and Biao Chen (2010). *The Common Information of N Dependent Random Variables*. 2010 48th Annual Allerton Conference on Communication, Control, and Computing (Allerton).

Cosma Rohilla Shalizi and James P. Crutchfield (2001). *Computational Mechanics: Pattern and Prediction, Structure and Simplicity*. Journal of Statistical Physics.

Cosma Rohilla Shalizi (2019). *Information Theory III — Information for Prediction*.

Naftali Tishby, Fernando C. Pereira, and William Bialek (1999). *The Information Bottleneck Method*. Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing.

PART D

Agency

CHAPTER D.1.1

Preferences to Rewards

Fernando E. Rosas

Building from preferences and a minimal set of axioms to a utility function expressible as a sum of discounted rewards: the familiar framing in reinforcement learning.

Source: iliad-intensive.org/agency/preferences-to-rewards · CC BY 4.0 · Slides: online

VIDEO *The Five Rules of Rationality Behind Reinforcement Learning* · youtu.be/ZvoEC4-iRjQ

WHAT YOU'LL LEARN

- Understand the difference between preferences, utility, and reward: preferences being a primary, largely uncontroversial notion, and utility and rewards being derived notions resting on specific assumptions.
- Be able to derive the relationship between various preference structures and rationality axioms.
- Critically assess alternative notions of rationality, and the consequences of dropping various classical decision theory assumptions.

Overview

This note develops a short route from preferences over complete trajectories to expected utility, reward, and discount. We begin with preference relations on deterministic trajectories and explain how completeness and transitivity yield an ordinal utility representation. We then show how lotteries, together with the von Neumann–Morgenstern axioms, produce a cardinal utility over trajectories, and we clarify the distinction between ordinal preference utility and vNM utility. Next, following Bowling et al., we add a fifth temporal axiom that is necessary and sufficient for a recursive representation in terms of local rewards and discounting. Finally, we explain why reward is not unique: different reward functions can encode the same utility or the same preference ordering, affine changes of utility induce corresponding changes of reward, and potential-based shaping provides a canonical example of reward equivalence.

1. Introduction

What does it mean for a system to have a goal? In sequential decision making, the object of evaluation is often not a single isolated choice or prize but a whole trajectory: a complete history of states, observations, actions, and consequences unfolding through time. Before asking how goals are achieved, it is worth asking a more basic question: what does it mean to prefer some trajectories over others, and what follows from that?

Recent work in reinforcement learning has revived this fundamental question, treating preferences over histories as primitive and asking what additional assumptions are needed before one can recover scalar reward functions. The present note provides an introduction to these ideas, which proceeds in three stages.

1. First, one asks for a numerical representation of how whole trajectories are ranked.
2. Second, once one allows lotteries over trajectories, one asks when these lotteries can be ranked by the expectation of that trajectory utility.
3. Third, one asks what additional requirements are needed in order to decompose this expected utility into rewards assigned at each time step.

The first and second steps are closely related to the classical theory developed by von Neumann and Morgenstern (von Neumann & Morgenstern 1944). The third asks what extra temporal structure is needed before utility over whole trajectories can be decomposed into stagewise rewards, following the line of work developed in modern reinforcement learning by Pitis 2019, Shakerinava & Ravanbakhsh 2022, and Bowling et al. 2023.

2. Preferences over trajectories

Let $\mathcal{O}$ be a finite set of observations and $\mathcal{A}$ a finite set of actions. A one-step interaction is then given by $t = (o, a) \in \mathcal{O} \times \mathcal{A}$. For each $n \in \mathbb{N}_{\geq 0}$, define the space of trajectories of length n by

$$\mathcal{H}_n \coloneqq (\mathcal{O} \times \mathcal{A})^n.$$

We write ε for the unique trajectory of length 0. The space of all *finite* trajectories is

$$\mathcal{H}^* \coloneqq \bigcup_{n=0}^{\infty} \mathcal{H}_n.$$

A typical element of $\mathcal{H}^*$ has the form $h = (o_1, a_1, o_2, a_2, \ldots, o_n, a_n)$. We will keep the notation $\mathcal{H}^*$ for the set of all finite trajectories throughout.

Definition 2.1 (Preference). A preference relation on $\mathcal{H}^*$ is a binary relation $\succcurlyeq$ where

$$h \succcurlyeq h'$$

means that trajectory h is judged at least as good as trajectory h'.

From $\succcurlyeq$ we derive the usual companion relations:

$$h \sim h' \iff h \succcurlyeq h' \text{ and } h' \succcurlyeq h, \qquad h \succ h' \iff h \succcurlyeq h' \text{ and not } h' \succcurlyeq h.$$

We call $\sim$ 'indifference', as an agent has no reason to prefer one over the other.

At this point, $\succcurlyeq$ has no properties whatsoever. One may naturally wonder what kinds of properties it is reasonable to require of $\succcurlyeq$, and what follows from them — which is what we study in the next sections.

3. When are preferences problematic?

Can a preference relation be intrinsically 'bad'? The relevant concern here is whether it leads to some form of self-defeat, avoidable loss, or failure of coherent behaviour. The discussion in the decision-theory literature suggests at least three grades of concern.

1. *Representability failures.* A first and weakest concern is that a preference relation may fail to be representable in a convenient form. This does not, by itself, imply that the preference is irrational.

Representation failures may merely be inconvenient, but they become more significant when they are symptoms of deeper issues of the kinds described next (Aumann 1962; Fishburn 1970).

2. *Self-defeat and avoidable loss.* A more serious concern is that preferences may guide choice poorly — as judged by the agent's own interest. One important case is *static self-defeat*: choosing an option that is worse than another available one, or adopting a policy that is systematically improvable. A classic case of suboptimality is dominance: one option or policy dominates another when it is at least as good in every relevant respect and strictly better in some, so choosing the dominated option is a clear mistake (Kreps 1988; Mas-Colell et al. 1995). A second case is *diachronic self-defeat*: a plan that the agent endorses now is predictably undone later in a way that leaves the agent worse off overall.
3. *Vulnerability.* The most vivid coherence arguments show that a collection of individually acceptable choices can be combined into a guaranteed loss. Dutch-book arguments play this role for credences; money-pump arguments play the analogous role for preferences. The standard example is a preference cycle

$$h_1 \succ h_2, \qquad h_2 \succ h_3, \qquad h_3 \succ h_1.$$

If the agent is willing to pay a small fee to move each time to a strictly preferred trajectory, then an adversary can guide it around the cycle and back to where it started, poorer than before (Gustafsson 2010). This is why intransitivity is usually regarded as a particularly severe pathology: it is not merely hard to represent, but also vulnerable to exploitation under natural trading assumptions.

Coherence is not selection.

It is useful to distinguish three kinds of formal results. A *representation result* says that if preferences satisfy certain axioms, then they can be written in a particular mathematical form; the vNM and Savage theorems being classical examples. A *coherence result* is different: it links violations of some constraint to a penalty such as a Dutch book, money pump, dynamic inconsistency, or dominated choice. Complete-class and admissibility theorems belong more naturally in this second family than in the first, since they characterize undominated decision rules rather than utility representations. A *selection result* is different again: it adds a story about some optimization process — such as market competition, evolution, or training dynamics — and argues that systems lacking a certain property tend to be selected against. In short, representation concerns *form*, coherence concerns *vulnerability or domination*, and selection concerns *survival under pressure*.

Note that representation, coherence, and selection arguments are related, but not identical. In general, a coherence argument is a within-agent claim: it says that if a single agent violates some structural constraint, then the agent is vulnerable to a penalty such as a money pump, Dutch book, or dominated choice. A selection argument is different: it asks whether agents lacking that property would tend to disappear under some broader optimization pressure, such as market competition, training dynamics, or evolutionary selection. A coherence result may help motivate a selection story, yet it does not by itself show that realistic environments actually select against the offending preference pattern. Similarly, a representational failure with no plausible selection story may still be mathematically interesting while being less central for explaining the structure of real agents.

For the purposes of this note, the key point is that these notions of badness do not all coincide. A preference can fail standard representation without being exploitably incoherent, and not every departure from expected utility is thereby problematic. Nevertheless, the axioms studied below are useful because — as we will see — they rule out several undesirable properties.

4. Completeness, transitivity, and ordinal utility

Two structural conditions are especially important.

Definition 4.1 (Completeness). A preference relation $\succcurlyeq$ on $\mathcal{H}^*$ is *complete* if for every pair $h, h' \in \mathcal{H}^*$,

$$h \succcurlyeq h' \quad \text{or} \quad h' \succcurlyeq h \quad \text{(or both)}.$$

Definition 4.2 (Transitivity). A preference relation $\succcurlyeq$ on $\mathcal{H}^*$ is *transitive* if for every $h, h', h'' \in \mathcal{H}^*$,

$$h \succcurlyeq h' \quad \text{and} \quad h' \succcurlyeq h'' \quad \Longrightarrow \quad h \succcurlyeq h''.$$

Completeness says that the agent can compare any two trajectories. This is a claim about *comparability*, not about certainty: it says that the preference relation returns a verdict on every pair, not that the agent knows everything about the consequences of those trajectories. Transitivity says that these verdicts fit together consistently across chains of comparison. For example, if $h_1 \succcurlyeq h_2$ and $h_2 \succcurlyeq h_3$, then transitivity requires $h_1 \succcurlyeq h_3$ as well.

A preference relation satisfying both conditions is often called a *weak order* in economics and decision theory (Fishburn 1970; Kreps 1988; Mas-Colell et al. 1995). On a countable domain such as $\mathcal{H}^*$, weak orders admit an ordinal utility representation. More general representation theorems on richer domains go back to the classic work of Debreu 1954.

Proposition 4.3 (Ordinal representation on the trajectory space). A preference relation $\succcurlyeq$ on $\mathcal{H}^*$ is complete and transitive if and only if there exists a function $u \colon \mathcal{H}^* \to \mathbb{R}$ such that

$$h \succcurlyeq h' \iff u(h) \geq u(h') \qquad \text{for all } h, h' \in \mathcal{H}^*.$$

Proof

If such a function u exists, then completeness and transitivity are inherited from the total order on $\mathbb{R}$. Conversely, if $\succcurlyeq$ is complete and transitive, then trajectories can be grouped into indifference classes, where each class contains all trajectories tied with one another. The quotient $\mathcal{H}^*/\!\sim$ of these classes is a countable total order. Any countable total order can be embedded in $\mathbb{R}$, so we may assign real numbers to the indifference classes in a way that preserves their order. Composing that assignment with the quotient map gives the desired utility function on trajectories.

This utility is *ordinal*: only the ranking matters. Any strictly increasing transformation of u represents the same preference relation. So ordinal utility lets us encode the order of deterministic trajectories, but it does not yet tell us how to compare risky mixtures of them. The numbers themselves have no independent meaning beyond the order they induce. If u represents a preference relation, then so does $2u + 17$, or $\exp(u)$, provided the transformation remains strictly increasing. This is why ordinal utility is best thought of as a numerical *labeling of ranks*, not yet as a measure of how much one trajectory is preferred to another (Fishburn 1970).

It is also useful to understand what happens when either condition fails.

What if completeness or transitivity fail

If completeness fails. Then some pairs of trajectories are incomparable. This may be a feature rather than a bug: the agent may genuinely refuse to rank certain alternatives because its values are plural, under-specified, or context-sensitive. But once incomparability is allowed, no single real-valued utility function can exactly represent the relation in the sense of Proposition 4.3, because any two real numbers are themselves comparable. One must then move to a different formalism, such as partial orders, sets of utility functions, or multi-criteria representations (Aumann 1962).

If transitivity fails. Then local pairwise judgments need not assemble into a global ranking. In the simplest case one obtains a cycle

$$h_1 \succ h_2, \qquad h_2 \succ h_3, \qquad h_3 \succ h_1.$$

No scalar utility can represent such a cycle, since it would require

$$u(h_1) > u(h_2) > u(h_3) > u(h_1),$$

which is impossible. The logical problem is already serious: there is no single global ranking of the options. Under the additional behavioral assumption that the agent is willing to pay a small fee to move from any option to a strictly preferred one, this logical failure becomes operational through the *money pump* problem. Suppose an agent currently has h_1 and is willing to pay a small fee $\varepsilon > 0$ each time it moves to a strictly preferred trajectory. The cycle above licenses the sequence

$$h_1 \to h_2 \to h_3 \to h_1,$$

with the agent paying ε at each step. At the end it is back where it started, but poorer by 3ε. Repeating the cycle pumps away arbitrarily much money (Gustafsson 2010).

There is also an interesting geometric view on transitivity. This paragraph is not needed for the rest of the note, so readers meeting these ideas for the first time can safely treat it as an optional aside. Fix a finite menu $V = \{h_1, \dots, h_m\} \subset \mathcal{H}^*$ and draw the complete graph whose vertices are the trajectories in V. Encode pairwise comparisons by an antisymmetric edge flow X:

$$X_{ij} = -X_{ji},$$

where $X_{ij} > 0$ means that h_i is preferred to h_j, while $X_{ij} < 0$ means the reverse. If preferences come from a utility function u, then each edge weight is just a utility difference,

$$X_{ij} = u(h_i) - u(h_j),$$

so X is a discrete gradient field. In the language of the discrete Helmholtz–Hodge decomposition, any edge flow can be split into a gradient part, which comes from a potential, and a cyclic part, which records genuine loops. On the complete comparison

graph there is no extra harmonic remainder, so inconsistency is entirely captured by the cyclic component (Jiang et al. 2011). A three-cycle corresponds exactly to a nonzero discrete curl:

$$X_{ij} + X_{jk} + X_{ki} \neq 0.$$

Thus transitive preferences are precisely the *potential* part of the decomposition, with the potential given by the utility, while intransitive cycles show up as the rotational or cyclic part. If this language feels abstract, the key takeaway is simple: utility means all local comparisons come from one global ranking, whereas cycles are the leftover pattern that cannot be explained by any single scalar potential.

5. Lotteries and the von Neumann–Morgenstern axioms

Up to this point, we have just considered preferences over a countable set $\mathcal{H}^*$ without considering stochasticity. To introduce uncertainty, we expand the domain from trajectories to lotteries over trajectories.

Let $\Delta(X)$ for $X \subset \mathcal{H}^*$ denote the set of probability distributions over X. An element $\mu \in \Delta(X)$ can be expressed as

$$\mu = \sum_i p_i \delta_{x_i},$$

which should be read as a lottery that yields outcome x_i with probability p_i. The outcome x is identified with the degenerate lottery δ_x. In this way, preferences over outcomes can be viewed as a special case of preferences over lotteries. Furthermore, for $\mu, \nu \in \Delta(X)$ and $\lambda \in [0,1]$, write

$$\lambda\mu + (1-\lambda)\nu$$

for the compound lottery that first flips a coin with bias λ, then samples from μ or ν accordingly.

The von Neumann–Morgenstern (vNM) framework studies weak preference relations $\succcurlyeq$ over $\Delta(X)$ and asks when such preferences admit an expected-utility representation (von Neumann & Morgenstern 1944). The framework considers four axioms on preferences over lotteries.

Axiom 1 (Completeness). For all $\mu, \nu \in \Delta(X)$, either $\mu \succcurlyeq \nu$ or $\nu \succcurlyeq \mu$ (or both).

Axiom 2 (Transitivity). For all $\mu, \nu, \xi \in \Delta(X)$, if $\mu \succcurlyeq \nu$ and $\nu \succcurlyeq \xi$, then $\mu \succcurlyeq \xi$.

Axiom 3 (Continuity). For all $\mu, \nu, \xi \in \Delta(X)$ with $\mu \succcurlyeq \nu \succcurlyeq \xi$, there exists $\lambda \in [0,1]$ such that

$$\nu \sim \lambda\mu + (1-\lambda)\xi.$$

Axiom 4 (Independence). For all $\mu, \nu, \xi \in \Delta(X)$ and $\lambda \in (0,1)$,

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \lambda\mu + (1-\lambda)\xi \succcurlyeq \lambda\nu + (1-\lambda)\xi.$$

We already know about completeness and transitivity from the previous section; the new players are continuity and independence. Continuity says intermediate prospects admit a break-even mixture between better and worse ones. Independence says that if μ is preferred to ν, then mixing both with the same background lottery ξ should not reverse that preference.

The natural question is therefore when a preference over lotteries can be represented by the expectation of a utility function on trajectories:

$$U(\mu) = \sum_{x \in X} \mu(x) u(x).$$

This formulation says that the value of a lottery is the probability-weighted average of the utilities of its possible trajectories. In its simplest finite-outcome form, this question is answered by the celebrated vNM theorem.

Theorem 5.1 (von Neumann–Morgenstern). Let $X \subseteq \mathcal{H}^*$ be finite. A weak preference relation $\succcurlyeq$ on $\Delta(X)$ satisfies completeness, transitivity, continuity, and independence if and only if there exists a function $u \colon X \to \mathbb{R}$ such that for all lotteries $\mu, \nu \in \Delta(X)$,

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \sum_{x \in X} \mu(x) u(x) \geq \sum_{x \in X} \nu(x) u(x).$$

Moreover, u is unique up to positive affine transformations:

$$u'(x) = a + b u(x), \qquad b > 0.$$

Proof

We first prove the easy direction: if preferences admit an expected-utility representation, then they satisfy the four axioms.

Completeness and transitivity follow immediately from the total order on $\mathbb{R}$. Continuity follows because if $\mu \succcurlyeq \nu \succcurlyeq \xi$ then $U(\mu) \geq U(\nu) \geq U(\xi)$, so one can choose $\lambda \in [0,1]$ such that

$$U(\nu) = \lambda U(\mu) + (1 - \lambda) U(\xi).$$

Hence

$$\nu \sim \lambda \mu + (1 - \lambda) \xi.$$

Finally, independence follows from linearity:

$$U(\lambda \mu + (1 - \lambda)\xi) = \lambda U(\mu) + (1 - \lambda) U(\xi).$$

Therefore

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \lambda \mu + (1 - \lambda)\xi \succcurlyeq \lambda \nu + (1 - \lambda)\xi.$$

We now prove the converse direction, namely that the four axioms imply an expected-utility representation.

1. *Pick best and worst outcomes (completeness and transitivity).* Because X is finite and the restriction of $\succcurlyeq$ to degenerate lotteries is complete and transitive, either all outcomes in X are indifferent or there exist $b, w \in X$ such that

$$b \succcurlyeq x \succcurlyeq w \qquad \text{for all } x \in X.$$

If all outcomes are indifferent, then the constant utility function already represents the preference relation. So we may assume $b \succ w$.

2. *Calibrate each outcome against b and w (continuity).* For $x = b$ set $u(x) = 1$, and for $x = w$ set $u(x) = 0$. For $b \succ x \succ w$, continuity gives some $u(x) \in (0, 1)$ such that

$$x \sim u(x)b + \big(1 - u(x)\big)w.$$

Thus every outcome is indifferent to a lottery over the two benchmark outcomes b and w.

3. *Compare the benchmark lotteries (independence and transitivity).* We first show that for $x, x' \in X$,

$$x \succcurlyeq x' \quad \Longleftrightarrow \quad u(x) \geq u(x').$$

For $\lambda \in [0, 1]$, write

$$C_\lambda \coloneqq \lambda b + (1 - \lambda)w.$$

We claim that if $\alpha > \beta$, then $C_\alpha \succ C_\beta$. Indeed, if $\alpha = 1$, then $C_\alpha = b$, and independence applied to $b \succ w$ with mixing weight β and background lottery b gives

$$b \succ \beta b + (1 - \beta)w = C_\beta.$$

So it remains to consider the case $\alpha < 1$. Define

$$\rho \coloneqq \frac{\alpha - \beta}{1 - \beta} \in (0, 1).$$

Since $b \succ w$, independence gives

$$\rho b + (1 - \rho)C_\beta \succ \rho w + (1 - \rho)C_\beta.$$

But the left-hand side is exactly C_α, while the right-hand side is C_β. Hence larger values of λ yield strictly better benchmark lotteries. Together with Step 2 and transitivity, this proves the displayed equivalence above.

4. *Reduce an arbitrary lottery to a benchmark lottery (independence).* Let

$$\mu = \sum_i p_i x_i.$$

By Step 2, each x_i is indifferent to $C_{u(x_i)}$. Replacing each x_i by the corresponding $C_{u(x_i)}$ inside μ, one at a time, and using independence together with transitivity, yields

$$\mu \sim \sum_i p_i C_{u(x_i)}.$$

By ordinary probability arithmetic, the compound lottery on the right reduces to

$$\sum_i p_i C_{u(x_i)} \sim U(\mu) b + (1 - U(\mu)) w, \qquad U(\mu) \coloneqq \sum_i p_i u(x_i).$$

5. *Conclude the expected-utility representation.* Applying Step 3 to the benchmark lotteries from Step 4, we obtain

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad U(\mu) \geq U(\nu).$$

This is exactly the desired expected-utility representation.

It remains to prove affine uniqueness, which uses continuity, independence, and the representation just obtained. Suppose u' is another expected-utility representation of the same preference relation. Let

$$a \coloneqq u'(w), \qquad c \coloneqq u'(b) - u'(w) > 0.$$

For any $x \in X$, Step 2 gives

$$x \sim u(x) b + (1 - u(x)) w.$$

Since u' also represents the same preferences, indifference implies equality of expected u'-value:

$$u'(x) = u(x) u'(b) + (1 - u(x)) u'(w) = a + c u(x).$$

So u' is a positive affine transformation of u.

Exercise 5.1 (Guided proof of the von Neumann–Morgenstern theorem). Assume $X \subseteq \mathcal{H}^*$ is finite and that $\succcurlyeq$ on $\Delta(X)$ satisfies completeness, transitivity, continuity, and independence.

(a) Show that there exist trajectories $b, w \in X$ such that $b \succcurlyeq h \succcurlyeq w$ for every $h \in X$.

(b) Using continuity, show that for every $h \in X$ there exists $u(h) \in [0, 1]$ such that

$$h \sim u(h) b + (1 - u(h)) w.$$

(c) Use transitivity to show that if

$$h \sim u(h) b + (1 - u(h)) w \quad \text{and} \quad h' \sim u(h') b + (1 - u(h')) w,$$

then

$$h \succcurlyeq h' \quad \Longleftrightarrow \quad u(h) \geq u(h').$$

(d) Let

$$\mu = \sum_i p_i h_i.$$

Use independence repeatedly to replace each h_i by the equivalent lottery $u(h_i)b + (1 - u(h_i))w$ and show that

$$\mu \sim \sum_i p_i \big(u(h_i)b + (1 - u(h_i))w\big).$$

(e) Reduce the compound lottery in part (d) to a simple lottery and prove that

$$\mu \sim U(\mu)b + (1 - U(\mu))w, \qquad U(\mu) \coloneqq \sum_i p_i u(h_i).$$

(f) Show that

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad U(\mu) \geq U(\nu).$$

This gives the expected-utility representation.

(g) Prove affine uniqueness: if u' also represents the same preference relation in expected-utility form, then $u' = a + bu$ for some $a \in \mathbb{R}$ and $b > 0$.

(h) Prove the converse direction: if preferences admit an expected-utility representation, then they satisfy completeness, transitivity, continuity, and independence.

SOLUTION, p. 651

In contrast to our previous result, vNM utility is *cardinal* up to positive affine transformations, not merely ordinal. A general monotone transformation would destroy the expectation formula. The independence axiom forces exactly the amount of structure needed for linear averaging.

Two notions of utility.

In economics it is standard to distinguish two different notions of utility (Kreps 1988; Mas-Colell et al. 1995).

The first is *ordinal utility*, also known as *preference utility* or 'F1', which represents preferences over certain outcomes. This is the object obtained in Proposition 4.3: if preferences over deterministic trajectories are complete and transitive, then there exists a function u_{ord} such that

$$h \succcurlyeq h' \quad \Longleftrightarrow \quad u_{\text{ord}}(h) \geq u_{\text{ord}}(h').$$

Only the ranking matters. Any strictly increasing transformation of u_{ord} represents the same preferences (Fishburn 1970).

The second is *von Neumann–Morgenstern utility*, also called 'F2'. This is the function u_{vNM} that appears inside the expectation operator when preferences over lotteries satisfy continuity and independence:

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \sum_h \mu(h) u_{\text{vNM}}(h) \geq \sum_h \nu(h) u_{\text{vNM}}(h).$$

Unlike ordinal utility, u_{vNM} is unique only up to positive affine transformations. Its numerical differences therefore carry behavioral content: they determine which mixtures an agent is willing to accept, and so encode the agent's attitudes toward risk and gambling structure (von Neumann & Morgenstern 1944; Mas-Colell et al. 1995).

Exercise 5.2 (Ordinal versus vNM utility). Let $x, y, z \in \mathcal{H}^*$ satisfy $x \succ y \succ z$.

(a) Give two different ordinal utility functions $u_1, u_2 \colon \mathcal{H}^* \to \mathbb{R}$ that represent the same ranking of x, y, z.

(b) Explain why these two functions are equally good as ordinal representations.

(c) Suppose in addition that

$$y \sim \tfrac{1}{2}x + \tfrac{1}{2}z.$$

Show that not every strictly increasing transformation of a vNM utility can preserve this indifference.

SOLUTION, p. 652

If continuity or independence fails

It is useful to separate the roles of the four vNM axioms:

- As before, completeness and transitivity give us an ordinal utility on deterministic trajectories.
- Continuity and independence turn that ordinal picture into an *expected* utility theory over uncertain prospects.

If completeness or transitivity fail, as discussed in the previous section, we lose hope of describing the preference by a single scalar quantity. If continuity or independence fails, the preference will generally no longer admit the linear expectation form.

Failure of continuity. To build intuition, let $x \succ y \succ z$ and define

$$\mu_\lambda = \lambda x + (1 - \lambda) z.$$

Then

$$\mu_0 = z, \qquad \mu_1 = x,$$

so as λ increases from 0 to 1, the lottery moves from the worse prospect z to the better prospect x. Continuity says, informally, that preferences vary smoothly along this path. In particular, it says that the intermediate prospect y can be matched by some mixture of the better and worse prospects:

$$y \sim \lambda x + (1 - \lambda) z$$

for some $\lambda \in (0, 1)$. Intuitively, the axiom requires no abrupt jumps in preference as the mixture probabilities vary.

When continuity fails, some distinctions can become *lexicographic*. For example, we could have a case where $y \succ \mu_0 = z$, but $\mu_\lambda \succ y$ for all $\lambda > 0$. This means that x has absolute priority over y: as soon as x appears with any nonzero probability, the lottery becomes strictly better than y. One way to interpret this is that some considerations are given lexical priority over others. For instance, avoiding catastrophe might outrank ordinary gains so completely that no finite improvement in ordinary reward compensates for even an arbitrarily small increase in catastrophic risk. Such preferences need not be contradictory; they are simply too sharp to be captured by a single real-valued utility whose expectations are taken in the usual way.

What breaks in this case is not all representations, but the *real-valued* vNM representation. If continuity is dropped, one is naturally led to lexicographic or non-Archimedean representations: for example, ordered pairs or vectors of utilities compared lexicographically, or utilities with infinitesimal scales (Fishburn 1971). In those models, the first coordinate records the highest-priority consideration, and lower coordinates matter only when higher ones tie.

Failure of independence.

Independence says that common lottery components should cancel. If $\mu \succcurlyeq \nu$, then mixing both with the same background lottery ξ at the same rate should preserve the ranking:

$$\lambda\mu + (1-\lambda)\xi \succcurlyeq \lambda\nu + (1-\lambda)\xi.$$

So the relative ranking of μ and ν should depend only on how they differ, not on the common part they share.

When independence fails, the value of a prospect depends on its *context*. The same local substitution can be attractive in one background and unattractive in another. This is exactly what the Allais pattern shows (Allais 1953). Let $h_5 \succ h_1 \succ h_0$ denote trajectories yielding 5, 1, and 0 million, and consider

$$A = 1 \cdot h_1, \qquad B = 0.89\,h_1 + 0.10\,h_5 + 0.01\,h_0,$$

$$C = 0.11\,h_1 + 0.89\,h_0, \qquad D = 0.10\,h_5 + 0.90\,h_0.$$

Many people prefer A to B, but prefer D to C. Under independence this is impossible, because A versus B and C versus D differ only by a common consequence. The reversal shows that certainty is treated as psychologically special: replacing a sure outcome by a tiny risk of getting nothing matters more than expected utility allows.

This is the general lesson of independence failure. Probabilities are no longer aggregated linearly against a fixed utility function on trajectories. Common branches cannot be canceled, and the whole shape of the distribution starts to matter. Decision makers may overweight certainty, distort small probabilities, care about disappointment or regret, or evaluate gains and losses relative to a reference point rather than in absolute terms. This is the route taken by prospect theory and related non-expected-utility models (Kahneman & Tversky 1979; Machina 1982).

From the perspective of sequential choice, independence also matters because it allows one to replace a sublottery by an equivalent one without changing the value of the larger plan. If independence fails, the value of a branch may depend on the branches surrounding it, so local and global evaluations need not line up. Hammond's consequentialist argument shows that, together with dynamic consistency and suitable sequential assumptions, one is pushed back toward independence (Hammond 1988). But that only shows one route to coherent planning. One may instead keep a richer, non-linear evaluation of lotteries and give up the idea that common consequences are always behaviorally irrelevant.

So the two failures have different meanings. Failure of continuity says that some priorities are infinitely sharp, leading naturally to lexicographic or infinitesimal utility scales. Failure of independence says that uncertainty is evaluated holistically rather than by linear averaging, leading to models in which background risk, certainty, or reference dependence affect choice.

Exercise 5.3 (Failures of the vNM axioms). Here we will study the consequences of different axioms.

(a) Construct a simple incomplete preference relation on three trajectories. Why can it not be represented by a single real-valued ordinal utility?

(b) Construct a three-cycle and explain how it gives rise to a money pump.

(c) Give an example of a lexicographic preference over three outcomes and show that it violates continuity.

(d) Write down the Allais pattern from Section 5 and explain which axiom it violates.

SOLUTION, p. 652

6. A fifth axiom: reward and discount

The vNM theorem gives an expected-utility representation over lotteries of whole trajectories, but it does not yet tell us that utility can be generated *locally* from stepwise rewards.

To investigate the possibility of localizing utility over time, let us denote by $T := \mathcal{O} \times \mathcal{A}$ the set of one-step transitions. For $t \in T$ and $h \in \mathcal{H}^*$, write $t \cdot h$ for the trajectory obtained by prepending t to h, and extend this operation to lotteries by

$$t \cdot \mu := \sum_i p_i \delta_{t \cdot h_i} \qquad \text{when } \mu = \sum_i p_i \delta_{h_i}.$$

Now, using a vNM utility $u(h)$ one can always define a continuation-dependent increment as

$$r(t; h) := u(t \cdot h) - u(h).$$

However, this increment may depend on the whole continuation h. What is missing is a temporal condition ensuring that the effect of prepending a transition depends only on that transition itself. Following (Bowling et al. 2023), this can be explored with the following additional axiom.

Axiom 5 (Temporal γ-indifference). There exists a function $\gamma\colon T \to [0,1]$ such that for all $\mu, \nu \in \Delta(\mathcal{H}^*)$ and all $t \in T$,

$$\frac{1}{1+\gamma(t)}(t \cdot \mu) + \frac{\gamma(t)}{1+\gamma(t)}\nu \sim \frac{1}{1+\gamma(t)}(t \cdot \nu) + \frac{\gamma(t)}{1+\gamma(t)}\mu.$$

Under expected utility, the indifference above is equivalent to

$$u(t \cdot \mu) + \gamma(t)u(\nu) = u(t \cdot \nu) + \gamma(t)u(\mu),$$

which can be rearranged as

$$u(t \cdot \mu) - u(t \cdot \nu) = \gamma(t)\big(u(\mu) - u(\nu)\big).$$

So the effect of prepending the same transition t is to rescale the utility difference between two continuations by a factor $\gamma(t)$. In that sense, $\gamma(t)$ measures how much the future still matters after the step t has occurred. When $\gamma(t) = 1$, future differences are preserved without discount at that step. When $0 < \gamma(t) < 1$, the future still matters but is damped. When $\gamma(t) = 0$, once t has occurred the continuation no longer affects the comparison. Bowling's axiom is closely related to the transition-dependent discounting discussed by White 2017 and Pitis 2019.

Bowling et al. show that adding this axiom to the four vNM axioms is necessary and sufficient for a discounted-reward representation of preferences.

Theorem 6.1 (Markov reward representation, after Bowling et al.). A weak preference relation $\succcurlyeq$ on $\Delta(\mathcal{H}^*)$ satisfies completeness, transitivity, continuity, independence, and temporal γ-indifference if and only if there exist functions $u\colon \mathcal{H}^* \to \mathbb{R}$, $r\colon T \to \mathbb{R}$, and $\gamma\colon T \to [0,1]$ such that $u(\varepsilon) = 0$,

$$u(t \cdot h) = r(t) + \gamma(t)u(h),$$

and, for all lotteries $\mu, \nu \in \Delta(\mathcal{H}^*)$,

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \sum_{h \in \mathcal{H}^*} \mu(h)u(h) \geq \sum_{h \in \mathcal{H}^*} \nu(h)u(h).$$

Moreover, r is unique up to positive scale, and γ is the same function appearing in the fifth axiom (Bowling et al. 2023).

This theorem sharpens the vNM result in exactly the way reinforcement learning needs. Utility is no longer an arbitrary scalar attached to a complete trajectory; it is generated recursively from a local reward and a local weight on the future. Unrolling the recursion for a trajectory $h = (t_1, t_2, \ldots, t_n)$ gives

$$u(h) = r(t_1) + \gamma(t_1) r(t_2) + \gamma(t_1)\gamma(t_2) r(t_3) + \cdots + \left(\prod_{i=1}^{n-1} \gamma(t_i) \right) r(t_n).$$

Thus the fifth axiom is what allows utility over whole trajectories to be decomposed into local reward and discounting. Two familiar special cases are worth highlighting.

- If $\gamma(t) \equiv \gamma$ is constant, then

$$u(h) = \sum_{k=1}^{n} \gamma^{k-1} r(t_k),$$

which is the standard discounted-return objective.

- If $\gamma(t) \equiv 1$, then

$$u(h) = \sum_{k=1}^{n} r(t_k),$$

so utility is simply the additive cumulative reward.

In summary, the step from vNM utility over whole trajectories to RL-style reward requires one more axiom. That axiom simultaneously identifies both the local reward signal and the form of discounting.

Remark (MDPs, POMDPs, and locality). The locality result of this section is especially natural in fully observed MDPs, where one often expects reward to depend only on the current state transition. In the present note, however, the primitive histories are sequences of observations and actions, and the corresponding local reward has the form $r \colon \mathcal{O} \times \mathcal{A} \to \mathbb{R}$. In partially observed settings this can be restrictive: a goal may be Markov in the hidden state, in the agent's belief state, or in some augmented memory state, without being reducible to a function of the current observation-action pair alone. In that sense, the fifth axiom should be read as characterizing when preferences admit a reward that is local in the chosen representation of experience. If the raw observation stream is too coarse, one may need to enrich the state description before a Markov reward representation becomes available (Bowling et al. 2023).

Utility is not reward.

It is helpful to keep the following four different objects apart.

- *Preference* is the primitive relation $\succcurlyeq$. It says only which trajectories or lotteries are weakly preferred to which others.
- *Preference utility* ('F1') is an ordinal representation of preferences over deterministic trajectories. Under completeness and transitivity, it is any function u_{ord} such that

$$h \succcurlyeq h' \quad \Longleftrightarrow \quad u_{\text{ord}}(h) \geq u_{\text{ord}}(h').$$

It is unique only up to strictly increasing transformations (Fishburn 1970).

- *vNM utility* ('F2') is the stronger, cardinal utility that appears when preferences over lotteries satisfy the vNM axioms. It is the function u_{vNM} for which

$$\mu \succcurlyeq \nu \quad \Longleftrightarrow \quad \sum_h \mu(h) u_{\text{vNM}}(h) \geq \sum_h \nu(h) u_{\text{vNM}}(h).$$

It is unique only up to positive affine transformations (von Neumann & Morgenstern 1944). Thus `F2' refines` F1': it agrees with it on the ranking of certain trajectories, but adds the extra structure needed to compare lotteries.

- *Reward* is not either of these utilities. A reward function $r(t)$ is a *local* representation introduced only after adding temporal structure. In the present framework, reward appears when the fifth axiom allows utility to be written recursively as

$$u(t \cdot h) = r(t) + \gamma(t) u(h).$$

So reward is a way of *decomposing* utility over complete trajectories into stepwise contributions. It is therefore downstream of preference, and even downstream of vNM utility. This is why different reward functions can encode the same utility or the same preference ordering, a point we return to in the next section (Bowling et al. 2023).

Exercise 6.1 (Guided proof of the Markov reward representation theorem). Assume the hypotheses of Exercise 5.1 together with temporal γ-indifference, and let u be a vNM utility representation over trajectories.

(a) Apply expected utility to temporal γ-indifference and show that for all lotteries $\mu, \nu \in \Delta(\mathcal{H}^*)$ and all transitions $t \in T$,

$$u(t \cdot \mu) + \gamma(t) u(\nu) = u(t \cdot \nu) + \gamma(t) u(\mu).$$

(b) Rearrange part (a) to prove that

$$u(t \cdot \mu) - \gamma(t) u(\mu)$$

is independent of the choice of μ.

(c) Define

$$r(t) := u(t \cdot \mu) - \gamma(t) u(\mu)$$

for any μ. Deduce that for every deterministic trajectory h,

$$u(t \cdot h) = r(t) + \gamma(t) u(h).$$

(d) Show that this recursion extends to lotteries by linearity:

$$u(t \cdot \mu) = r(t) + \gamma(t) u(\mu).$$

(e) Prove the converse direction: if u, r, and γ satisfy

$$u(t \cdot h) = r(t) + \gamma(t) u(h)$$

for all t and h, and if u extends linearly to lotteries, then temporal γ-indifference holds.

(f) Unroll the recursion along a finite trajectory $h = (t_1, \dots, t_n)$ and derive

$$u(h) = r(t_1) + \gamma(t_1) r(t_2) + \gamma(t_1)\gamma(t_2) r(t_3) + \cdots + \left(\prod_{i=1}^{n-1} \gamma(t_i) \right) r(t_n).$$

(g) Show that when $\gamma(t) \equiv \gamma$ is constant, the previous formula reduces to standard discounted return, and when $\gamma(t) \equiv 1$ it reduces to additive cumulative reward.

(h) Show that if $u' = a + bu$, then the corresponding reward must be

$$r'(t) = b r(t) + (1 - \gamma(t)) a.$$

Interpret this as a source of reward non-uniqueness.

SOLUTION, p. 653

7. Reward equivalence and shaping

The preceding theorem shows how to pass from utility over lotteries of trajectories to a local reward representation. But it also raises an important question: how unique is that reward? The answer has two parts. If one fixes a particular utility function u and a particular discount function γ, then the reward is determined. But if one changes the numerical representative of the same preference relation, or allows shaping transformations, then many different rewards can encode the same underlying ordering.

First observe that once u and γ are fixed, the reward is fixed as well. Indeed, if

$$u(t \cdot h) = r(t) + \gamma(t) u(h)$$

for all $t \in T$ and $h \in \mathcal{H}^*$, then necessarily

$$r(t) = u(t \cdot h) - \gamma(t) u(h),$$

and the right-hand side must be independent of the continuation h. So the non-uniqueness of reward does not come from ambiguity inside a fixed recursive representation. It comes from the fact that utility itself is not unique as a numerical object.

Proposition 7.1 (Multiple rewards can encode the same preference relation). Suppose $u \colon \mathcal{H}^* \to \mathbb{R}$, $r \colon T \to \mathbb{R}$, and $\gamma \colon T \to [0, 1]$ satisfy

$$u(t \cdot h) = r(t) + \gamma(t) u(h)$$

for all $t \in T$ and $h \in \mathcal{H}^*$, and that preferences over lotteries are represented by the expected value of u. For any $a \in \mathbb{R}$ and $b > 0$, define

$$u'(h) = a + b u(h) \qquad \text{and} \qquad r'(t) = b r(t) + (1 - \gamma(t)) a.$$

Then $u'(t \cdot h) = r'(t) + \gamma(t) u'(h)$ for all $t \in T$ and $h \in \mathcal{H}^*$, and the expected value of u' represents the same preference relation as the expected value of u.

The proposition shows that reward non-uniqueness already appears at the level of affine reparameterizations of utility: changing the numerical representative of the same vNM preference relation generally changes the associated reward function as well. This point is especially simple under the normalization $u(\varepsilon) = 0$ used in the previous section. Then the additive degree of freedom disappears, and one is left only with positive rescalings:

$$u'(h) = bu(h), \qquad r'(t) = br(t).$$

So in the normalized setting reward is unique only up to choice of units.

There is also a second, less trivial kind of non-uniqueness, in which one changes rewards while leaving the induced trajectory ordering unchanged, first studied by Ng et al. 1999.

Proposition 7.2 (Potential-based shaping changes utility only by a boundary term). Consider a state-based setting with transitions $t = (s, a, s')$ and a potential function Φ on states.

1. If $\gamma \equiv 1$ and $r_\Phi(s, a, s') \coloneqq r(s, a, s') + \Phi(s') - \Phi(s)$, then

$$\sum_{k=1}^{n} r_\Phi(t_k) = \sum_{k=1}^{n} r(t_k) + \Phi(s_n) - \Phi(s_0)$$

for any trajectory $\tau = (t_1, \dots, t_n)$ from s_0 to s_n.

2. If $\gamma \in (0, 1)$ is constant and $r_\Phi(s, a, s') \coloneqq r(s, a, s') + \gamma\Phi(s') - \Phi(s)$, then

$$\sum_{k=1}^{n} \gamma^{k-1} r_\Phi(t_k) = \sum_{k=1}^{n} \gamma^{k-1} r(t_k) - \Phi(s_0) + \gamma^n \Phi(s_n).$$

The previous result shows that shaping r into r_Φ changes utility only by a boundary term. In particular, if all compared trajectories share the same start state and terminal potential, then the induced preference ordering is unchanged; if the boundary term vanishes on all admissible trajectories, then even the numerical utility is unchanged.

The general lesson is that the same preferences admit many different reward functions, so no single one of them is canonical. Some transformations, such as positive scaling, merely change the numerical units. Others, such as potential-based shaping, redistribute value along the trajectory while leaving the overall preference over complete trajectories unchanged. This is why reward design is often underdetermined: what matters behaviorally is not a raw reward function in isolation, but the utility and preference structure it induces.

Exercise 7.1 (Reward shaping). Assume additive reward with $\gamma \equiv 1$ and define

$$r_\Phi(s, a, s') = r(s, a, s') + \Phi(s') - \Phi(s).$$

(a) Show that along any finite trajectory $(t_1, \dots, t_n)$ the total shaped reward differs from the original total reward by the boundary term $\Phi(s_n) - \Phi(s_0)$.

(b) Under what condition on the admissible trajectories does this shaping leave utility exactly unchanged?

(c) Under what weaker condition does it leave only the induced preference ordering unchanged?

SOLUTION, p. 653

8. Conclusion

The main lesson of this note is that, in sequential settings, the primitive object is not a one-step reward but a preference over complete trajectories. From that starting point one can distinguish several layers of structure. Completeness and transitivity yield an ordinal utility over deterministic trajectories. Once lotteries are introduced, continuity and independence refine that ordinal picture into a von Neumann–Morgenstern utility, which agrees with the ranking of certain trajectories but carries strictly more information because it calibrates trade-offs between risky prospects.

This separation of layers clarifies both what expected utility achieves and what it leaves out. It explains why "utility" in economics can refer either to an ordinal representation of certain preferences or to a cardinal object suitable for evaluating lotteries. It also shows what fails when particular axioms are dropped: without completeness or transitivity, scalar representation itself can fail; without continuity or independence, one may still have meaningful preferences, but no longer the linear expectation form of vNM. In that sense, expected utility is not the whole of rationality, but a particular and mathematically powerful strengthening of it.

The final step is to see that even vNM utility is not yet reward. To obtain a local reward-and-discount representation one needs additional temporal structure, captured here by Bowling et al.'s fifth axiom. Under that axiom, utility over whole trajectories admits a recursive decomposition into local reward and discounting. But even then reward is not unique: affine changes of utility induce corresponding changes of reward, and potential-based shaping can redistribute value along a trajectory while preserving the underlying ordering. Moreover, the locality of reward depends on the chosen representation of experience, which is natural in fully observed MDPs but can be restrictive in partially observed settings unless the state is suitably enriched. The reward hypothesis, understood in this way, is therefore best read not as the claim that goals are primitively rewards, but as the claim that sufficiently structured preferences over trajectories can be represented by rewards.

Open-ended questions

Exercise 8.1. Do the results have any prescriptive or descriptive implications? What kinds of agents, with what kinds of preferences should we design? By default, what kind of agents should we expect to obtain via typical training processes?

SOLUTION, p. 654

Exercise 8.2. Let's assume a superintelligence has preferences over trajectories. Are there weaker versions of the axioms that make it more plausible that these preferences are "safe for us" compared to preferences that lead to utilities or even rewards?

SOLUTION, p. 654

Exercise 8.3. More generally, are there interesting examples of preferences over trajectories that one could analyse, that do or do not satisfy some of the axioms?

SOLUTION, p. 654

References

Maurice Allais (1953). *Le comportement de l'homme rationnel devant le risque: critique des postulats et axiomes de l'ecole Americaine*. Econometrica.

Robert J Aumann (1962). *Utility theory without the completeness axiom*. Econometrica.

Michael Bowling, John D. Martin, David Abel, and Will Dabney (2023). *Settling the Reward Hypothesis*. arXiv preprint arXiv:2212.10420.

Gerard Debreu (1954). *Representation of a preference ordering by a numerical function*. Decision Processes.

Peter C. Fishburn (1970). *Utility Theory for Decision Making*. Wiley.

Peter C. Fishburn (1971). *A Study of Lexicographic Expected Utility*. Management Science.

Johan E. Gustafsson (2010). *A Money-Pump for Acyclic Intransitive Preferences*. Dialectica.

Peter J. Hammond (1988). *Consequentialist Foundations for Expected Utility*. Theory and Decision.

Xiaoye Jiang, Lek-Heng Lim, Yuan Yao, and Yinyu Ye (2011). *Statistical ranking and combinatorial Hodge theory*. Mathematical Programming.

Daniel Kahneman and Amos Tversky (1979). *Prospect Theory: An Analysis of Decision under Risk*. Econometrica.

David M. Kreps (1988). *Notes on the Theory of Choice*. Westview Press.

Mark J. Machina (1982). *"Expected Utility" Analysis without the Independence Axiom*. Econometrica.

Andreu Mas-Colell, Michael D. Whinston, and Jerry R. Green (1995). *Microeconomic Theory*. Oxford University Press.

Andrew Y. Ng, Daishi Harada, and Stuart J. Russell (1999). *Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping*. Proceedings of the Sixteenth International Conference on Machine Learning.

Silviu Pitis (2019). *Rethinking the discount factor in reinforcement learning: A decision theoretic approach*. Proceedings of the AAAI Conference on Artificial Intelligence.

Mehran Shakerinava and Siamak Ravanbakhsh (2022). *Utility Theory for Sequential Decision Making*. Proceedings of the International Conference on Machine Learning.

John von Neumann and Oskar Morgenstern (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

Martha White (2017). *Unifying Task Specification in Reinforcement Learning*. Proceedings of the 34th International Conference on Machine Learning.

CHAPTER D.1.2

Reinforcement Learning

Leon Lang (Iliad), David Quarel (ARENA)

The Bellman equations and what follows from them: the existence of optimal policies, the policy improvement theorem, the rate of convergence of Bellman updates, and the convergence of Q-learning.

Source: iliad-intensive.org/agency/reinforcement-learning · CC BY 4.0 · Slides: online

WHAT YOU'LL LEARN

- Understand Markov Decision Processes and the goal of the agent, for known environments.
- Understand the Bellman equation.
- Understand the policy improvement theorem, and how we can use it to iteratively solve for an optimal policy.
- Prove properties involving the Bellman equations, including the existence of optimal policies, the policy improvement theorem, rate of convergence of Bellman updates, and the convergence of Q-learning.

Setup

In this exercise sheet, we prove a variety of results that are behind the ARENA Intro to RL materials.

An **agent** interacts with an **environment** in discrete time steps $t = 0, 1, 2, \ldots$. On timestep t, the agent observes the current state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$ according to its policy. The environment then responds with a reward $r_{t+1} \in \mathbb{R}$ and a new state $s_{t+1} \in \mathcal{S}$. (The reward and next state are indexed by $t+1$ because they are produced by the environment after the agent's action.) The goal is to act so as to maximize expected discounted future reward.

These environments are called **Markov decision processes** (MDPs). They satisfy two key properties: (i) **stationarity** — the transition and reward functions do not change over time, and (ii) the **Markov property** — the distribution over the next state and reward depends only on the current state and action, not on the full history of past interactions. Together, these properties mean that the current state s_t is a sufficient summary of the past for the purpose of choosing optimal actions.

Definition 0.1 (Spaces and notation). - $\mathcal{S}$ — finite **state space**

- $\mathcal{A}$ — finite **action space**
- $\gamma \in (0,1)$ — **discount factor**
- ΔX — set of all probability distributions over a set X
- $[\![P]\!]$ — **Iverson bracket**: 1 if P is true, 0 if false

Definition 0.2 (Environment). An MDP **environment** is specified by a pair (T, R):

- $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$ is the **transition kernel**: given state s and action a, the next state is drawn $s' \sim T(\cdot \mid s, a)$.
- $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the **reward function**: on a transition from s to s' under action a, the agent receives reward $R(s, a, s')$. [1]

Definition 0.3 (Policy). A **policy** $\pi : \mathcal{S} \to \Delta\mathcal{A}$ maps each state to a probability distribution over actions. Given state s:

- $\pi(\cdot \mid s)$ is a distribution over $\mathcal{A}$,
- $\pi(a \mid s) \in [0,1]$ is the probability of choosing action a,
- the agent samples $a \sim \pi(\cdot \mid s)$.

A policy is **deterministic** if $\pi(a \mid s) \in \{0, 1\}$ for all a, s. In this case, we abuse notation and write $\pi(s) := \arg\max_{a \in \mathcal{A}} \pi(a \mid s)$ for the unique action selected at state s.

Definition 0.4 (Trajectory). When policy π interacts with environment (T, R) starting from initial state s_0, the resulting **trajectory** $(s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, \dots)$ is generated by

$$a_k \sim \pi(\cdot \mid s_k), \qquad s_{k+1} \sim T(\cdot \mid s_k, a_k), \qquad r_{k+1} = R(s_k, a_k, s_{k+1}),$$

for $k = 0, 1, 2, \dots$. We write $\mathbb{E}_\pi[\,\cdot\,]$ for expectations over such trajectories.

1. The Bellman equation

Definition 1.1 (Return). The **return** from time t is the discounted sum of future rewards:

$$G_t \;:=\; \sum_{k=t+1}^{\infty} \gamma^{k-t-1}\, r_k \;=\; r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \cdots .$$

It satisfies the recursion

$$G_t \;=\; r_{t+1} + \gamma\, G_{t+1}.$$

Definition 1.2 (Value function). The **value function** of a policy π is the expected return when starting from state s and acting according to π:

$$\begin{aligned} V_\pi(s) \;&=\; \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi\big[\, r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \cdots \,\big|\, s_t = s\big] \\ &= \sum_{a_t \in \mathcal{A}} \pi(a_t \mid s) \sum_{s_{t+1} \in \mathcal{S}} T(s_{t+1} \mid s, a_t) \sum_{a_{t+1} \in \mathcal{A}} \pi(a_{t+1} \mid s_{t+1}) \\ &\quad \sum_{s_{t+2} \in \mathcal{S}} T(s_{t+2} \mid s_{t+1}, a_{t+1}) \cdots \\ &\quad \big[\, R(s, a_t, s_{t+1}) + \gamma\, R(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma^2\, R(s_{t+2}, a_{t+2}, s_{t+3}) + \cdots \big]. \end{aligned}$$

The expectation averages the return over all future trajectories $(a_t, s_{t+1}, a_{t+1}, s_{t+2}, \ldots)$ generated by sampling $a_k \sim \pi(\cdot \mid s_k)$ and $s_{k+1} \sim T(\cdot \mid s_k, a_k)$, starting from $s_t = s$.

Exercise 1.1. Prove the **Bellman equation**: for all $s \in \mathcal{S}$,

Bellman Equation

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} T(s' \mid s, a) \big(R(s, a, s') + \gamma\, V_\pi(s') \big).$$

HINT, p. 656

SOLUTION, p. 656

Definition 1.3 (Action-value function). The **action-value function** (or Q-function) of π is the expected return starting from state s, taking action a, and acting according to π thereafter:

$$Q_\pi(s, a) \;=\; \mathbb{E}_\pi[G_t \mid s_t = s,\, a_t = a]\,.$$

Exercise 1.2. (a) Express $V_\pi(s)$ in terms of Q_π.

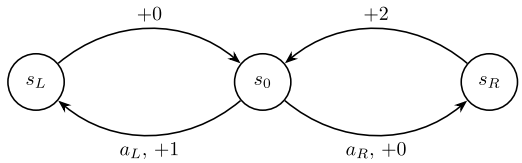
(b) Express $Q_\pi(s, a)$ in terms of V_π.

(c) Using these relations, derive a recursive equation for Q_π analogous to the Bellman equation above.

SOLUTION, p. 656

2. An example MDP

Consider the MDP with state space $\mathcal{S} = \{s_0, s_L, s_R\}$, action space $\mathcal{A} = \{a_L, a_R\}$, and deterministic transitions as shown:



From s_0, action a_L leads deterministically to s_L with reward $+1$, and action a_R leads deterministically to s_R with reward $+0$. From s_L (resp. s_R), any action returns to s_0 with reward $+0$ (resp. $+2$).

Let π_L denote the policy that always selects a_L, and π_R the policy that always selects a_R.

Exercise 2.1. Using the Bellman equation from Exercise 1.1, verify that

$$V_{\pi_L}(s_0) = \frac{1}{1-\gamma^2}, \qquad V_{\pi_R}(s_0) = \frac{2\gamma}{1-\gamma^2}.$$

SOLUTION, p. 657

Exercise 2.2. Determine the best action from state s_0 as a function of the discount factor $\gamma \in (0,1)$. Show that the breakeven point is $\gamma = \frac{1}{2}$.

SOLUTION, p. 657

3. Bellman operators and the existence of optimal policies

Banach fixed-point theorem

In this and the following exercises, we will make use of the well-known Banach fixed point theorem:

Theorem 3.1 (Banach Fixed Point Theorem). Let (X, d) be a complete metric space and let $F : X \to X$ be a contraction mapping, i.e. there exists $\gamma \in [0, 1)$ such that

$$d(F(x), F(y)) \leq \gamma\, d(x, y) \quad \text{for all } x, y \in X.$$

Then F has a unique fixed point $x^* \in X$ satisfying $F(x^*) = x^*$. Moreover, for any initial point $x_0 \in X$, one

has $\lim_{n\to\infty} d(F^n(x_0), x^*) = 0$ where F^n is the n-fold composition of F with itself. That is, $F^n(x_0)$ converges to x^* with respect to the metric d.

The problem

Definition 3.2 (Bellman optimality operator). Let $\mathcal{V} = \{V : \mathcal{S} \to \mathbb{R}\}$ denote the vector space of value functions, equipped with the sup-norm $\|V\|_\infty = \max_{s\in\mathcal{S}} |V(s)|$. The **Bellman optimality operator** $\mathcal{B} : \mathcal{V} \to \mathcal{V}$ is defined by

$$(\mathcal{B}V)(s) \coloneqq \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\, V(s')\big).$$

Exercise 3.1. Let $f, g : \mathcal{A} \to \mathbb{R}$ be real-valued functions on a finite set $\mathcal{A}$. Prove that

$$\left|\max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a)\right| \le \max_{a\in\mathcal{A}} |f(a) - g(a)|.$$

SOLUTION, p. 658

Exercise 3.2. Using Exercise 3.1, prove that $\mathcal{B}$ is a **contraction mapping** with contraction factor γ, i.e. for all $V, W \in \mathcal{V}$,

$$\|\mathcal{B}V - \mathcal{B}W\|_\infty \le \gamma\, \|V - W\|_\infty.$$

SOLUTION, p. 658

Exercise 3.3. Using Theorem 3.1, conclude that there exists a unique $V^* \in \mathcal{V}$ satisfying $\mathcal{B}V^* = V^*$, and that for any initial $V_0 \in \mathcal{V}$, the iterates $\mathcal{B}^n V_0 \to V^*$ as $n \to \infty$.

SOLUTION, p. 659

Exercise 3.4. Show that for any policy π, the operator $\mathcal{B}_\pi : \mathcal{V} \to \mathcal{V}$ defined by

Bellman Policy Operator

$$(\mathcal{B}_\pi V)(s) \coloneqq \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\, V(s')\big)$$

is also a contraction with factor γ. Conclude that V_π is its unique fixed point and that $\mathcal{B}_\pi^n V_0 \to V_\pi$ for any initial $V_0 \in \mathcal{V}$ as $n \to \infty$.

Remark. This shows that the numerical policy evaluation algorithm from the ARENA materials converges to the correct solution.

SOLUTION, p. 659

Exercise 3.5. Let $V \in \mathcal{V}$ be any value function, and let π_V be a greedy policy with respect to V, i.e.

$$\pi_V(s) \in \arg\max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\, V(s')\big).$$

Show that $\mathcal{B}_{\pi_V} V = \mathcal{B}V$.

SOLUTION, p. 659

Exercise 3.6. Define the **greedy policy** with respect to V^* by $\pi^* = \pi_{V^*}$ as in the previous part. Show that $V_{\pi^*} = V^*$, i.e. the value function of π^* equals the fixed point of $\mathcal{B}$.

HINT, p. 659

SOLUTION, p. 659

Exercise 3.7. Conclude that π^* is an **optimal policy**: for every policy π and every state $s \in \mathcal{S}$,

$$V_{\pi^*}(s) \geq V_{\pi}(s).$$

HINT, p. 660

SOLUTION, p. 660

4. Policy improvement theorem

Given a policy π, define the **improved policy** π' greedily with respect to V_π:

Policy Improvement

$$\pi'(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\big(R(s,a,s') + \gamma\, V_\pi(s')\big).$$

Exercise 4.1. In proving Exercise 3.7, we already saw that for all $s \in \mathcal{S}$, we have $(\mathcal{B}V_\pi)(s) \geq V_\pi(s)$. When does equality hold for all states?

SOLUTION, p. 660

Exercise 4.2. Show that $V_{\pi'}(s) \geq V_\pi(s)$ for all $s \in \mathcal{S}$.

HINT, p. 660

SOLUTION, p. 660

Exercise 4.3. The **policy iteration** algorithm generates a sequence of policies $\pi_0, \pi_1, \pi_2, \ldots$ where each π_{k+1} is the greedy policy with respect to V_{π_k}. Show that this sequence converges to an optimal policy π^* in a finite number of steps.

HINT, p. 661

SOLUTION, p. 661

5. Bellman convergence rate

Exercise 3.3 showed that the iterates $V_n := \mathcal{B}^n V_0$ converge to V^* for any starting $V_0 \in \mathcal{V}$, but it did not tell us *how fast*, nor how to decide when to stop iterating. We derive some convergence rate bounds (Puterman 1994, Chapter 6).

Exercise 5.1. Show that for any initial $V_0 \in \mathcal{V}$, the iterates $V_n := \mathcal{B}^n V_0$ satisfy

$$\|V_n - V^*\|_\infty \;\leq\; \gamma^n \|V_0 - V^*\|_\infty.$$

Remark. This bound is not useful as a stopping criterion in practice as it depends on the unknown V^*.

SOLUTION, p. 661

Exercise 5.2. Show the **a priori bound**:

$$\|V_n - V^*\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,\|V_1 - V_0\|_\infty.$$

This bound uses only $\|V_1 - V_0\|_\infty$ — a quantity computable after a single iteration, independent of V^*.

HINT, p. 661

SOLUTION, p. 661

Exercise 5.3. Show the **a posteriori bound**: for $n \ge 1$,

$$\|V_n - V^*\|_\infty \;\le\; \frac{\gamma}{1-\gamma}\,\|V_n - V_{n-1}\|_\infty.$$

HINT, p. 662

SOLUTION, p. 662

Exercise 5.4. Show that the a posteriori bound of Exercise 5.3 is always at least as tight as the a priori bound of Exercise 5.2: for all $n \ge 1$,

$$\frac{\gamma}{1-\gamma}\,\|V_n - V_{n-1}\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,\|V_1 - V_0\|_\infty.$$

HINT, p. 662

SOLUTION, p. 662

Exercise 5.5. Assume $|R(s,a,s')| \le R_{\max}$ for all s, a, s'. Show that if we initialize with $V_0 \equiv 0$, then

$$\|V_n - V^*\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,R_{\max}.$$

This bound depends only on n, γ, and $R_{\max}$, so it can be evaluated before running a single iteration.

SOLUTION, p. 663

Exercise 5.6. Show that the bound of Exercise 5.5 is **tight**: exhibit an MDP in which $\|V_n - V^*\|_\infty = \frac{\gamma^n}{1-\gamma}\,R_{\max}$ for every $n \ge 0$ (with $V_0 \equiv 0$).

SOLUTION, p. 663

Remark. Combining Exercises 5.3, 5.2 and 5.5 gives a chain of progressively weaker but increasingly upfront-computable bounds:

Convergence Bounds

$$\begin{aligned}\|V_n - V^*\|_\infty \;&\le\; \underbrace{\tfrac{\gamma}{1-\gamma}\,\|V_n - V_{n-1}\|_\infty}_{\text{a posteriori, tightest}} \;\le\; \underbrace{\tfrac{\gamma^n}{1-\gamma}\,\|V_1 - V_0\|_\infty}_{\text{a priori}} \\ &\le\; \underbrace{\tfrac{\gamma^n}{1-\gamma}\,R_{\max}}_{\text{upfront, assumes } V_0\equiv 0,\ |R|\le R_{\max}}.\end{aligned}$$

Each step weakens the bound by replacing realized information with something coarser: first the most recent step size $\|V_n - V_{n-1}\|_\infty$ with the worst-case geometric decay $\gamma^{n-1}\|V_1 - V_0\|_\infty$ (contraction of $\mathcal{B}$ applied $n-1$ times), then the initial step size $\|V_1 - V_0\|_\infty$ with the crude reward bound $R_{\max}$. In practice the leftmost bound is significantly tighter, since $\|V_n - V_{n-1}\|_\infty$ often decays strictly faster than $\gamma^{n-1}\|V_1 - V_0\|_\infty$. It also yields a concrete stopping rule: to guarantee $\|V_n - V^*\|_\infty \le \varepsilon$, iterate until $\|V_n - V_{n-1}\|_\infty \le \frac{1-\gamma}{\gamma}\,\varepsilon$.

6. Convergence of Q-learning

The policy improvement theorem from Section 4 shows that we can in principle find an optimal policy in MDPs. However, it makes the uncomfortable assumption that we know the environment dynamics via the transition kernel T. In this problem, we prove that Q-learning, which does not make such an assumption, converges to the optimal action-value function, from which one can trivially extract an optimal policy by choosing actions greedily.

Stochastic approximation theorem

We will make use of the following stochastic approximation result, which we state without proof:

Theorem 6.1 (Stochastic Approximation for Contractions (Tsitsiklis 1994, Theorem 3)). Let $\mathcal{X} = \{x : I \to \mathbb{R}\}$ be the space of real-valued functions on a finite set I, equipped with the sup-norm $\|x\|_\infty = \max_i |x(i)|$. Let $F : \mathcal{X} \to \mathcal{X}$ be a contraction mapping with factor $\gamma \in [0,1)$ and unique fixed point x^*.

Fix an initial iterate $x_0 \in \mathcal{X}$, and for each $t \ge 0$ let $\alpha_t : I \to [0,1]$ be a **learning rate** function and $w_t : I \to \mathbb{R}$ a random **noise** function (both functions on I, one value per coordinate). Consider the stochastic iteration

$$x_{t+1}(i) = \big(1 - \alpha_t(i)\big)\,x_t(i) + \alpha_t(i)\big[F(x_t)(i) + w_t(i)\big] \qquad \text{for all } i \in I,$$

where updates are applied to a single coordinate $i = i_t$ at each step (i.e. $\alpha_t(i) = 0$ for $i \ne i_t$). Suppose:

(a) (Learning rate) For each $i \in I$:

$$\sum_{t=0}^{\infty} \alpha_t(i) = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t(i)^2 < \infty, \qquad \alpha_t(i) \in [0,1].$$

(b) (Noise) For each $i \in I$, conditioned on the history $\mathcal{F}_t = \{x_0, \alpha_0, w_0, \alpha_1, w_1, \dots, \alpha_{t-1}, w_{t-1}, x_t, \alpha_t\}$, the noise has zero mean and bounded variance:

$$\mathbb{E}\big[w_t(i) \mid \mathcal{F}_t\big] = 0, \qquad \mathbb{E}\big[w_t(i)^2 \mid \mathcal{F}_t\big] \le C\big(1 + \|x_t\|_\infty^2\big)$$

for some constant $C > 0$.

Then $x_t(i) \to x^*(i)$ for all $i \in I$, with probability 1.

What is stochastic? The randomness of the iteration is carried by the noise: each $w_t(i)$ is a real-valued random variable, and w_t is a random element of $\mathbb{R}^I$. Consequently, the iterates $x_1, x_2, \dots$ are random (they depend on past noise $w_0, \dots, w_{t-1}$), and the learning rates $\alpha_t(i)$ may be random as well if they depend on the history—e.g. on which coordinate was just visited. The contraction F, its fixed point x^*, the update rule itself, and the initial iterate x_0 are all deterministic. The theorem asserts that despite the noise, the random sequence (x_t) converges pointwise to x^* almost surely.

Remark on "with probability 1". The conclusion $x_t(i) \to x^*(i)$ with probability 1 (also called **almost sure convergence**) means

$$P\Big(\lim_{t\to\infty} x_t(i) = x^*(i)\Big) = 1 \quad \text{for all } i \in I.$$

In other words, the probability that a run of the stochastic iteration fails to converge to x^* is zero.

Why should we believe this? It helps to build up the result in three layers of increasing complexity.

Layer 1: no noise, all coordinates at once. If $w_t = 0$ and $\alpha_t(i) = 1$ for all i and t, the iteration reduces to $x_{t+1} = F(x_t)$. This converges to x^* by Theorem 3.1.

Layer 2: noise, but all coordinates at once. Now suppose we observe $F(x_t) + w_t$ instead of $F(x_t)$, and use a decreasing step size:

$$x_{t+1} = (1 - \alpha_t)\, x_t + \alpha_t\big[F(x_t) + w_t\big] = x_t + \alpha_t\big[\underbrace{F(x_t) - x_t}_{\text{signal}} + \underbrace{w_t}_{\text{noise}}\big].$$

The signal $F(x_t) - x_t$ always points toward x^* (this is what the contraction property buys us). The noise w_t is mean-zero, so it pushes us in random directions that partially cancel over time. The two conditions on the step size mediate between signal and noise:

- $\sum \alpha_t = \infty$ ensures the total step size is large enough to reach x^* from any starting point.
- $\sum \alpha_t^2 < \infty$ ensures the noise averages out. Each noise term w_t enters the iteration scaled by α_t, contributing a random displacement of size $\alpha_t w_t$. Although these dis-

placements are not independent (each depends on the current iterate x_t), they are mean-zero conditioned on the past. For such sequences, the variance of the cumulative sum is the sum of the individual variances, which is proportional to $\sum \alpha_t^2$. Since this is finite, the total random drift converges and cannot overwhelm the steady pull of the signal toward x^*.

The signal accumulates coherently (always pulling toward x^*), while the noise averages out.

Layer 3: one coordinate at a time. The iteration updates only one coordinate $i = i_t$ per step, while the others stay frozen. This makes the analysis more difficult since the target $F(x_t)(i)$ depends on all coordinates, including stale ones. The condition $\sum_t \alpha_t(i) = \infty$ for each i ensures no coordinate is permanently neglected, and the contraction property of F provides enough global coupling to prevent coordinates from drifting apart. Making this rigorous is the main technical content of the proof.

The problem

Consider the following generalization of the Q-learning update from the ARENA materials, now with a time-varying learning rate $\alpha_t > 0$: given a current estimate $Q_t : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and an observed transition $(s_t, a_t, r_{t+1}, s_{t+1})$ where $s_{t+1} \sim T(\cdot \mid s_t, a_t)$ and $r_{t+1} = R(s_t, a_t, s_{t+1})$, the update is

Q-Learning Update

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t\big(r_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(s_{t+1}, a') - Q_t(s_t, a_t)\big),$$

with $Q_{t+1}(s, a) = Q_t(s, a)$ for all $(s, a) \neq (s_t, a_t)$. We will show that, under appropriate conditions, Q-learning converges to the optimal action-value function Q^*.

Let $\mathcal{Q} = \{Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}\}$ denote the space of action-value functions, equipped with the sup-norm $\|Q\|_\infty = \max_{s,a} |Q(s, a)|$.

Exercise 6.1. Overloading the notation from Definition 3.2, define the **Bellman optimality operator** on action-value functions, $\mathcal{B} : \mathcal{Q} \to \mathcal{Q}$, by

Bellman Q-Operator

$$(\mathcal{B}Q)(s, a) \coloneqq \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\big[R(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q(s', a')\big].$$

(Here $\mathcal{B}$ takes a Q-function to a Q-function, whereas the earlier $\mathcal{B}$ in Definition 3.2 takes a V-function to a V-function — whether $\mathcal{B}$ denotes the V- or Q-version is always clear from its argument.) Show that $\mathcal{B}$ is a γ-contraction on $(\mathcal{Q}, \|\cdot\|_\infty)$.

HINT, p. 663

SOLUTION, p. 663

Exercise 6.2. Define $Q^* \coloneqq Q_{\pi^*}$, the action-value function of the optimal policy π^* from Section 3. We now show Bellman optimality equations and their consequences.

(a) Show that $Q^*(s,a) = \sum_{s' \in \mathcal{S}} T(s' \mid s,a)\big[R(s,a,s') + \gamma\, V^*(s')\big]$.

(b) Show that $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s,a)$.

(c) Conclude that Q^* is the unique fixed point of $\mathcal{B}$.

SOLUTION, p. 664

Exercise 6.3. Rewrite the Q-learning update in the form of Theorem 6.1:

$$x_{t+1}(i) = (1 - \alpha_t(i))\, x_t(i) + \alpha_t(i)\big[F(x_t)(i) + w_t(i)\big].$$

Specifically, identify the index set I, the mapping F, the iterates x_t, and the learning rates $\alpha_t(i)$. Then give an explicit expression for the noise w_t.

SOLUTION, p. 664

Exercise 6.4. Show that the noise satisfies $\mathbb{E}[w_t(s,a) \mid \mathcal{F}_t] = 0$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$, where $\mathcal{F}_t$ denotes the history up to time t as in Theorem 6.1.

HINT, p. 665

SOLUTION, p. 665

Exercise 6.5. Assume $|R(s,a,s')| \le R_{\max}$ for all s, a, s'. Show that there exists a constant $C > 0$ (depending only on $R_{\max}$ and γ) such that

$$\mathbb{E}\big[w_t(s,a)^2 \mid \mathcal{F}_t\big] \le C\big(1 + \|Q_t\|_\infty^2\big).$$

HINT, p. 665

SOLUTION, p. 665

Exercise 6.6. Conclude: state the conditions on the learning rate schedule and the exploration policy under which Q-learning converges, i.e. $Q_t \to Q^*$ with probability 1.

Remark. The ARENA implementation uses a constant learning rate α, which does not satisfy $\sum_t \alpha_t(s,a)^2 < \infty$. In practice, constant-rate Q-learning does not converge exactly but oscillates in a neighbourhood of Q^* whose size shrinks with α.

SOLUTION, p. 666

7. Exact policy evaluation

In Exercise 3.4, we showed that iterating $\mathcal{B}_\pi$ converges to V_π; this is the numerical policy evaluation scheme used in the ARENA materials. When the state space is finite and the dynamics (T, R) are known, there is also an *exact* (closed-form) solution: the Bellman equation becomes a linear system that can be solved directly. In this problem we derive that closed form for deterministic policies.

Definition 7.1 (Matrix-vector notation for a deterministic policy). Fix a deterministic policy π. Treating the value function as a vector in $\mathbb{R}^{|\mathcal{S}|}$, define:

- $\mathbf{v}^\pi \in \mathbb{R}^{|\mathcal{S}|}$ with $\mathbf{v}^\pi_s := V_\pi(s)$,

- $\mathbf{P}^{\pi} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{S}|}$ with $\mathbf{P}^{\pi}_{s,s'} := T(s' \mid s, \pi(s))$ (the **transition matrix** under π),
- $\mathbf{R}^{\pi} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{S}|}$ with $\mathbf{R}^{\pi}_{s,s'} := R(s, \pi(s), s')$ (the **reward matrix**),
- $\mathbf{r}^{\pi} \in \mathbb{R}^{|\mathcal{S}|}$ with $\mathbf{r}^{\pi}_{s} := \sum_{s'} \mathbf{P}^{\pi}_{s,s'} \mathbf{R}^{\pi}_{s,s'} = \sum_{s'} T(s' \mid s, \pi(s))\, R(s, \pi(s), s')$ (the **expected immediate reward**).

Fact 7.2 (Neumann series). Equip $\mathbb{R}^{n\times n}$ with the induced ∞-norm

$$\|\mathbf{A}\|_{\infty} := \max_{i} \sum_{j} |\mathbf{A}_{ij}| \qquad \text{(maximum absolute row sum)}.$$

If $\|\mathbf{A}\|_{\infty} < 1$, then $\mathbf{I} - \mathbf{A}$ is invertible, and

$$(\mathbf{I} - \mathbf{A})^{-1} = \sum_{k=0}^{\infty} \mathbf{A}^{k}.$$

Exercise 7.1. Starting from the Bellman equation (Exercise 1.1) applied to a deterministic policy π, show that

Exact Policy Evaluation

$$\mathbf{v}^{\pi} = (\mathbf{I} - \gamma\, \mathbf{P}^{\pi})^{-1}\, \mathbf{r}^{\pi},$$

assuming $\mathbf{I} - \gamma\, \mathbf{P}^{\pi}$ is invertible (which we prove in Exercise 7.2).

HINT, p. 666

SOLUTION, p. 666

Exercise 7.2. Prove that $\mathbf{I} - \gamma\, \mathbf{P}^{\pi}$ is invertible for any deterministic policy π and any $\gamma \in (0, 1)$.

HINT, p. 667

SOLUTION, p. 667

Remark. The same derivation extends to stochastic policies by defining $\mathbf{P}^{\pi}_{s,s'} = \sum_{a} \pi(a \mid s)\, T(s' \mid s, a)$ and $\mathbf{r}^{\pi}_{s} = \sum_{a} \pi(a \mid s) \sum_{s'} T(s' \mid s, a)\, R(s, a, s')$. The matrix $\mathbf{P}^{\pi}$ remains row-stochastic, so the argument above applies unchanged.

References

M. L. Puterman (1994). *Markov Decision Processes — Discrete Stochastic Dynamic Programming*. Wiley.

John N. Tsitsiklis (1994). *Asynchronous Stochastic Approximation and Q-Learning*. Machine Learning.

NOTES

1. Noting that $\mathcal{S} \times \mathcal{A} \times \mathcal{S}$ is a finite set, this implies that the rewards are bounded above by $R_{\max} = \max_{s,a,s'} R(s, a, s')$.

CHAPTER D.2

Policy Gradients and Misgeneralization

David Quarel (ARENA), based on work by Matthew Farrugia-Roberts (University of Oxford)

Implement Vanilla Policy Gradient to train a neural network using reinforcement learning, and explore empirically the concepts of goal misgeneralisation in a toy environment.

Source: iliad-intensive.org/agency/policy-gradients-misgeneralization · CC BY 4.0 · Slides: part 1, part 2

VIDEO *When Bad Goals ALSO Fit Your Training Data* · youtu.be/FaXazOpzLvQ

Prerequisites

- D.1.2 – Reinforcement Learning (p. 299)
- Be familar with the PyTorch prerequisites (p. 1).
  - In particular, if you're unfamiliar with more advanced indexing like use of `torch.gather`, feel free to solve the exercise with a sequential python for-loop, and then use the solution code when you want the training loop to run quickly.

WHAT YOU'LL LEARN

- Understand the derivation of (Vanilla) Policy Gradient (VPG).
- Implement Policy Gradient in PyTorch, and use to train an agent to play a toy RL environment.
- Understand reward functions, what behaviour they incentivise and how they can be used as probes for behaviour.
- Understand what potential shaping is, and how it provides hints without allowing reward hacking.
- The difference between Goal Mis*generalisation* and Goal Mis*specification*.

Vanilla Policy Gradient

Work through Section [2.2.2] of the ARENA program on Vanilla Policy Gradient. The section ends with exercises in a Colab notebook.

- The network is small enough, and the training loop fast enough, that you can run this material locally on your laptop without problems if you prefer.

Goal Misgeneralization

Work through the accompanying Colab notebook on Goal Misgeneralisation.

- The network is again small, but some of the later runs might be slow without a GPU, so a GPU is recommended.

CHAPTER D.3.1

Solomonoff Induction

David Quarel (ARENA), Leon Lang (Iliad)

How an idealized agent should predict. A Bayesian mixture over a countable class of computable hypotheses learns to predict any sequence, with total error bounded by the description length of the truth – a formal Occam's razor.

Source: iliad-intensive.org/agency/solomonoff-induction · CC BY 4.0 · Slides: online

Prerequisites

- Comfort with discrete probability: conditional distributions, the chain rule, expectations.
- Kullback–Leibler divergence and basic information theory (helpful, developed as needed).

WHAT YOU'LL LEARN

- Define the Bayesian mixture over a class of environments and prove it is a proper predictor whose posterior updates multiplicatively.
- Prove the cumulative prediction-error bound and specialize it to the Solomonoff prior to recover the Occam bound.
- Show the mixture makes boundedly many mistakes and is Pareto-optimal under KL and squared loss.
- Bound the misspecified case when the truth lies outside the model class.

Much of this material is drawn from (Hutter et al. 2024, Chapter 3) and the earlier (Hutter 2005); both books provide fuller explanations, additional context, and proofs that we gloss over here.

Difficulty ratings. Each problem is tagged with a rating in square brackets following Knuth's exercise rating scheme (Knuth 1973): roughly, **[00]** trivial, **[10]** 15–minute pencil-and-paper, **[20]** 1–2 hours, **[30]** several hours to a day, **[40]** a significant research result. Intermediate values are possible.

Overview

Solomonoff induction is the *prediction* half of **Universal AI**: what would an *optimally intelligent* agent do if it only had to predict, given unlimited compute and the weakest possible assumptions? The answer is a single **Bayesian mixture** ξ over a countable class $\mathcal{M}$ of candidate environments, weighted by a prior w_ν. It (eventually) predicts as well as the true environment μ, and under the **universal** choice — $\mathcal{M}$ = all computable environments, prior $w_\nu = 2^{-K(\nu)}$ with K the Kolmogorov complexity — the assumption "$\mu \in \mathcal{M}$" becomes "the universe is computable," and Occam's razor drops out of the mathematics.

This chapter is a worksheet: you build the theory yourself, one problem at a time. The next chapter, AIXI (p. 327), lifts the same mixture to sequential decision-making (learning to *act*).

Goal and Roadmap

A recipe for prediction: Solomonoff induction is an attempt to mathematically formalize the *problem of induction* from philosophy: How to make predictions about the future based on past observations?

We formalize this as sequence prediction: There is some true environment μ that generates a sequence $x_1, x_2, \dots$ of (binary) symbols. Each next symbol $x_t \sim \mu(\cdot \mid x_{<t})$ is sampled from μ conditioned on the past $x_{<t}$. A predictor P takes the history $x_{<t}$ and gives a distribution $P(\cdot \mid x_{<t})$ over the next symbol x_t.

We measure the quality of a predictor P by the μ-expected squared prediction error, summed over every timestep:

$$S_\infty^\mu := \sum_{t=1}^{\infty} \sum_{x_{<t} \in \mathbb{B}^*} \mu(x_{<t}) \sum_{x_t \in \mathbb{B}} \big(P(x_t \mid x_{<t}) - \mu(x_t \mid x_{<t})\big)^2.$$

How to construct a predictor P such that S_∞^μ is small (or at least finite)? Bayesian inference to the rescue: we choose as our predictor a *Bayesian mixture* ξ over a countable class $\mathcal{M} = \{\nu_1, \nu_2, \dots\}$ of candidate environments, weighted by prior beliefs w_ν (formalized in Definition 1.2). The main results we build up to are:

- **Cumulative bound** (Section 4): Assuming $\mu \in \mathcal{M}$, $S_\infty^\mu \leq -\ln w_\mu$. The higher the prior on μ, the lower the prediction error.
- **Explicit bound** (Exercise 4.2): specialize to the Solomonoff prior $w_\nu = 2^{-K(\nu)}$ to get $S_\infty^\mu \leq K(\mu) \ln 2$, where K is the *Kolmogorov complexity*.
- **Pareto optimality** (Exercises 6.2 and 8.2): no other predictor weakly dominates ξ on every $\nu \in \mathcal{M}$, for either KL or squared loss.
- **Misspecified version** (Section 7): If $\mu \notin \mathcal{M}$, the cumulative bound becomes $-\ln w_{\hat{\mu}} + D_n(\mu \parallel \hat{\mu})$: the constant complexity term plus an approximation term

$D_n(\mu \parallel \hat{\mu}) = \sum_{t=1}^{n} d_t(\mu \parallel \hat{\mu})$ that in general grows linearly in n, so S_∞^μ diverges. Here $\hat{\mu} \in \mathcal{M}$ is the "closest" environment to μ.

Notation

For more background, see the corresponding post on Solomonoff induction.

Symbols used throughout.

- $\mathbb{B} = \{0, 1\}$: the binary alphabet; $\mathbb{B}^*$ all finite binary strings; $\mathbb{B}^n$ length-n strings.
- ϵ: the empty string.
- xy: string concatenation. if $x := x_{1:n}$ and $y := y_{1:m}$ then $xy = x_1 \dots x_n y_1 \dots y_m$.
- $x_{i:j} := x_i x_{i+1} \dots x_j$.
- $x_{<t} := x_1 x_2 \cdots x_{t-1}$.
- ν, ρ: generic environments / predictors. μ: the true (unknown) environment generating the data. ξ: a Bayesian mixture of environments.
- $\mathcal{M} = \{\nu_1, \nu_2, \dots\}$: a countable class of candidate environments.
- $w_\nu > 0$ with $\sum_{\nu \in \mathcal{M}} w_\nu = 1$: the prior over $\mathcal{M}$. $w(\nu \mid x_{<t})$: the posterior after observing $x_{<t}$.
- $\Delta\mathcal{X}$: the set of probability distributions over a finite set $\mathcal{X}$.

1. The mixture is a predictor

Definition 1.1 (Environment). An **environment** ν assigns to each history $x_{<t} \in \mathbb{B}^*$ a predictive distribution $\nu(\cdot \mid x_{<t}) \in \Delta\mathbb{B}$ over the next symbol. The conditional joint probability of a string $x \in \mathbb{B}^*$, conditioned on $y \in \mathbb{B}^*$, is defined as

$$\nu(x_{1:n}|y) \;:=\; \prod_{t=1}^{n} \nu(x_t \mid y x_{<t}), \qquad \nu(\epsilon) := 1.$$

The unconditional joint is defined as $\nu(x_{1:n}) := \nu(x_{1:n} \mid \epsilon)$. The true (unknown) environment generating the data is denoted μ.

Remark. One can show that $\nu(x_{1:t}) = \nu(x_{<t}) \cdot \nu(x_t \mid x_{<t})$ and $\nu(x) = \nu(x0) + \nu(x1)$ for all $x \in \mathbb{B}^*$.

Definition 1.2 (Bayesian mixture ξ). Let $\mathcal{M} = \{\nu_1, \nu_2, \dots\}$ be a countable class of environments with prior weights $w_\nu > 0$ satisfying $\sum_{\nu \in \mathcal{M}} w_\nu = 1$. The **Bayesian mixture** is defined as a prior-weighted mixture over all environments ν in $\mathcal{M}$:

$$\xi(x_{1:t}) \;:=\; \sum_{\nu \in \mathcal{M}} w_\nu \, \nu(x_{1:t}).$$

Its one-step predictive distribution is

$$\xi(x_t \mid x_{<t}) \;=\; \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t})\, \nu(x_t \mid x_{<t}),$$

with posterior weights

$$w(\nu \mid x_{<t}) \;:=\; w_\nu \, \frac{\nu(x_{<t})}{\xi(x_{<t})} \qquad w(\nu \mid \epsilon) := w_\nu.$$

Here $w(\nu \mid x_{<t})$ is the *posterior* belief in ν after observing $x_{<t}$.

The mixture ξ is defined as a sum of *joint* probabilities. To use it as a predictor we need its one-step conditional $\xi(x_t \mid x_{<t})$, and we need to know it is a genuine probability distribution (so that, later, Pinsker's inequality applies to it).

Exercise 1.1 (Generalized chain rule) [05]. The two-term chain rule baked into Definition 1.1 extends to arbitrary contiguous blocks. Show that for any $1 \le p \le q \le r$ and any history $x_{<p}$ with $\nu(x_{<p}) > 0$,

$$\nu(x_{p:r} \mid x_{<p}) \;=\; \nu(x_{p:q} \mid x_{<p}) \cdot \nu(x_{q+1:r} \mid x_{<p}\, x_{p:q}).$$

SOLUTION, p. 668

Exercise 1.2 (Mixture is a predictor) [10]. Starting from $\xi(x_{1:t}) = \sum_{\nu \in \mathcal{M}} w_\nu \, \nu(x_{1:t})$, show that

$$\xi(x_t \mid x_{<t}) \;=\; \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t})\, \nu(x_t \mid x_{<t}), \qquad w(\nu \mid x_{<t}) := w_\nu \, \frac{\nu(x_{<t})}{\xi(x_{<t})},$$

and conclude that $\sum_{x_t \in \mathbb{B}} \xi(x_t \mid x_{<t}) = 1$, i.e. $\xi(\cdot \mid x_{<t})$ is a probability distribution over $\mathbb{B}$.

HINT, p. 668

SOLUTION, p. 668

Exercise 1.3 (Posterior update) [10]. Show that the posterior weight $w(\cdot \mid \cdot)$ updates *multiplicatively* as new symbols arrive. For every $x_{1:t} \in \mathbb{B}^*$ with $\xi(x_{1:t}) > 0$,

$$w(\nu \mid x_{1:t}) \;=\; w(\nu \mid x_{<t}) \cdot \frac{\nu(x_t \mid x_{<t})}{\xi(x_t \mid x_{<t})}.$$

That is: the new posterior equals the old posterior, scaled by the **likelihood ratio** of how well ν predicted the just-observed symbol relative to the mixture's own prediction.

SOLUTION, p. 669

Exercise 1.4 (Multi-step posterior linearity) [10]. Exercise 1.2 showed that ξ's one-step prediction $\xi(x_t \mid x_{<t})$ is the posterior-weighted average of the per-model one-step predictions. The same identity extends to predictions over a *whole future segment* $x_{t:m}$: for any $t \le m$,

$$\xi(x_{t:m} \mid x_{<t}) \;=\; \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t})\, \nu(x_{t:m} \mid x_{<t}).$$

That is: the *same* posterior $w(\nu \mid x_{<t})$ governs ξ's predictions for arbitrarily many steps into the future, not just the next one. Show this.

HINT, p. 669

SOLUTION, p. 669

2. KL divergence

The KL divergence between two joint distributions over a length-n history splits, by an inductive application of the chain rule for probabilities, into a sum of per-step conditional KLs. This telescoping identity lets us trade a single joint KL over an entire history for a sum of per-step KLs (and vice versa): exactly the bridge we will need in Section 4.

We introduce two pieces of notation that will be used throughout the rest of the worksheet:

Definition 2.1 (Cumulative and per-step KL). For environments / predictors ν, ρ, horizon $n \geq 1$, and $1 \leq t \leq n$, the **cumulative joint KL** and the **expected per-step KL at step** t are

$$D_n(\nu \parallel \rho) \;:=\; \sum_{x_{1:n} \in \mathbb{B}^n} \nu(x_{1:n}) \ln \frac{\nu(x_{1:n})}{\rho(x_{1:n})},$$

$$d_t(\nu \parallel \rho) \;:=\; \sum_{x_{<t} \in \mathbb{B}^{t-1}} \nu(x_{<t}) \sum_{x_t \in \mathbb{B}} \nu(x_t \mid x_{<t}) \ln \frac{\nu(x_t \mid x_{<t})}{\rho(x_t \mid x_{<t})},$$

with the conventions $0 \ln \frac{0}{q} := 0$ for $q \geq 0$ and $p \ln \frac{p}{0} := \infty$ for $p > 0$.

We write $D_\infty(\nu \parallel \rho) := \lim_{t \to \infty} D_t(\nu \parallel \rho)$ and $D_\infty^\mu := D_\infty(\mu \parallel \xi)$.

Theorem 2.2 (KL divergence is non-negative). For any environments ν, ρ and any horizon $n \geq 1$ and step $1 \leq t \leq n$:

(a) $D_n(\nu \parallel \rho) \geq 0$, with equality iff $\nu(x_{1:n}) = \rho(x_{1:n})$ for every $x_{1:n} \in \mathbb{B}^n$ with $\nu(x_{1:n}) > 0$.

(b) $d_t(\nu \parallel \rho) \geq 0$, with equality iff $\nu(x_t \mid x_{<t}) = \rho(x_t \mid x_{<t})$ for every $x_{<t}$ with $\nu(x_{<t}) > 0$ and every $x_t \in \mathbb{B}$.

Proof in Appendix B.

Exercise 2.1 (Telescoping KL) [10]. Show by induction on n that for all environments ν, ρ,

$$D_n(\nu \,\|\, \rho) \;=\; \sum_{t=1}^{n} d_t(\nu \,\|\, \rho),$$

where on the right-hand side, the $t = 1$ summand uses the conventions $\nu(x_1 \mid x_{<1}) := \nu(x_1)$, $\rho(x_1 \mid x_{<1}) := \rho(x_1)$.

HINT, p. 670

SOLUTION, p. 670

3. Mixture dominance

The mixture ξ never assigns much less probability than any single environment weighted by its prior. This single inequality is the engine behind the cumulative bound of Section 4 below: dividing through gives $\mu(x)/\xi(x) \le 1/w_\mu$, which is then fed into Pinsker and the chain rule in the proof of the main cumulative bound.

Exercise 3.1 (Mixture dominance) [05]. Show that for every $\nu \in \mathcal{M}$ and every $x \in \mathbb{B}^*$,

$$\xi(x) \;\ge\; w_\nu \cdot \nu(x).$$

SOLUTION, p. 670

Remark (Solomonoff specialization). Under the Solomonoff prior, $w_\nu = 2^{-K(\nu)}$, so the bound becomes

$$\xi_U(x) \;\ge\; 2^{-K(\nu)} \cdot \nu(x) \qquad \text{for every computable } \nu.$$

This is what makes ξ_U *universal*: a single predictor dominates the entire computable model class up to a factor that depends only on the model's description length.

4. Cumulative prediction error bound (main result)

The next ingredient relates squared prediction error to KL divergence (Cover & Thomas 2006, Lemma 11.6.1). We require the following inequality:

Theorem 4.1 (Pinsker's inequality). Let P and Q be probability distributions over $\mathbb{B} = \{0, 1\}$, i.e., $p_0, p_1, q_0, q_1 \ge 0$ with $p_0 + p_1 = 1$ and $q_0 + q_1 = 1$. Write $p_x = P(x)$ and $q_x = Q(x)$. Then

$$\sum_{x \in \mathbb{B}} \big(q_x - p_x\big)^2 \;\le\; \sum_{x \in \mathbb{B}} p_x \ln \frac{p_x}{q_x} \tag{$\star$}$$

The proof of this inequality is mostly tedious algebra but is included in Appendix A.

Definition 4.2 (Cumulative expected and per-step squared prediction error). For environments / predictors ν, ρ and horizon $n \geq 1$, the **cumulative expected squared error** and the **per-step squared error at step** t are

$$S_n(\nu \parallel \rho) := \sum_{t=1}^{n} s_t(\nu \parallel \rho)$$
$$s_t(\nu \parallel \rho) := \sum_{x_{<t} \in \mathbb{B}^{t-1}} \nu(x_{<t}) \sum_{x_t \in \mathbb{B}} \big(\nu(x_t \mid x_{<t}) - \rho(x_t \mid x_{<t})\big)^2$$

Write $S_\infty(\nu \parallel \rho) := \lim_{n\to\infty} S_n(\nu \parallel \rho)$ and $S_\infty^\mu := S_\infty(\mu \parallel \xi)$.

Exercise 4.1 (Cumulative prediction bound) [15]. Show that

$$S_\infty^\mu \leq -\ln w_\mu.$$

HINT, p. 671

SOLUTION, p. 671

Exercise 4.2 (Explicit complexity bound) [05]. Specialize Section 4 to the **Solomonoff prior** $w_\nu = 2^{-K(\nu)}$ (where $K(\nu)$ is the length of the shortest program computing ν) to show

$$S_\infty^\mu \leq K(\mu) \ln 2.$$

SOLUTION, p. 671

Exercise 4.3 (Per-step error) [05]. Hence prove that the per-step expected prediction error

$$s_t := \sum_{x_{<t} \in \mathbb{B}^{t-1}} \mu(x_{<t}) \sum_{x_t \in \mathbb{B}} \big(\xi(x_t \mid x_{<t}) - \mu(x_t \mid x_{<t})\big)^2$$

converges to zero as $t \to \infty$.

SOLUTION, p. 671

5. Bounded number of prediction mistakes

So far we have bounded *squared error*. For a deterministic environment (i.e. a single infinite binary sequence $x_1^*, x_2^*, \ldots$ in $\mathcal{M}$), one can ask the sharper $0/1$-loss question: how often does the mixture predict the *wrong bit*? The cumulative bound is strong enough to give an answer.

Let $\mu \in \mathcal{M}$ be a deterministic environment, so $\mu(x_t \mid x_{<t}) \in \{0, 1\}$ on every history; write x_t^* for the unique bit on which $\mu(\cdot \mid x_{<t}^*)$ puts all its mass. Define the **threshold predictor** associated with ξ: at each step, predict the more-likely bit,

$$\hat{x}_t := \arg\max_{x \in \mathbb{B}} \xi(x \mid x_{<t}) \qquad \text{(break ties arbitrarily)}.$$

We say the mixture makes a **mistake** at time t if $\hat{x}_t \neq x_t^*$.

Exercise 5.1 (Bounded prediction mistakes) [15]. Show that the number of mistakes made by the threshold predictor on the μ-trajectory is at most

$$\#\{\, t \geq 1 \;:\; \hat{x}_t \neq x_t^* \,\} \;\leq\; -2 \ln w_\mu .$$

HINT, p. 671

SOLUTION, p. 671

Remark (Solomonoff specialization). Under the Solomonoff prior $w_\mu = 2^{-K(\mu)}$, so $-\ln w_\mu = K(\mu) \ln 2$ and the bound becomes

$$\#\{\, t \geq 1 \;:\; \hat{x}_t \neq x_t^* \,\} \;\leq\; 2K(\mu) \ln 2 .$$

With $\ln 2 \approx 0.693$ the constant is just under 1.4, so if the true environment μ is computable, and can be described by a program at most k bits long, then the predictor ξ will make at worst $\approx 1.4k$ mistakes over predicting the entire sequence $x_{1:\infty}^*$. Note that the bound is purely existence-style: it does not say *when* the mistakes happen. They could all occur at the start, or be arbitrarily far into the future.

6. Pareto optimality under KL loss

In Section 6 and Section 8 we establish that the Bayesian mixture ξ is *uniquely Pareto-optimal* under both KL and squared prediction loss: no other predictor can do at least as well as ξ on every environment $\nu \in \mathcal{M}$ without coinciding with ξ entirely.

Definition 6.1 (Pareto domination). For some measure of **loss** $\mathcal{F}(\nu, \rho)$ that takes an environment ν and a predictor ρ, a predictor ρ **weakly Pareto-dominates** ξ with respect to loss $\mathcal{F}$ and class $\mathcal{M}$ if

$$\mathcal{F}(\nu, \rho) \;\leq\; \mathcal{F}(\nu, \xi) \qquad \text{for every } \nu \in \mathcal{M}.$$

ξ is **Pareto-optimal** (w.r.t. $\mathcal{F}$) if the only predictor that weakly dominates it is ξ itself.

We specialize $\mathcal{F}$ to $D_n(\nu \,\|\, \rho)$ in Section 6 and to $S_n(\nu \,\|\, \rho)$ in Section 8.

Exercise 6.1 (KL Pythagorean identity) [10]. For *any* predictor ρ with $\rho(x_{1:n}) > 0$ on every $x_{1:n} \in \mathbb{B}^n$, show that

$$D_n(\xi \,\|\, \rho) \;+\; \sum_{\nu \in \mathcal{M}} w_\nu \, D_n(\nu \,\|\, \xi) \;=\; \sum_{\nu \in \mathcal{M}} w_\nu \, D_n(\nu \,\|\, \rho).$$

HINT, p. 672

SOLUTION, p. 672

Exercise 6.2 (KL Pareto-optimality) [05]. Show that ξ is Pareto-optimal under $D_n(\nu \parallel \rho)$ in the sense of Definition 6.1: if ρ is any predictor with

$$D_n(\nu \parallel \rho) \;\leq\; D_n(\nu \parallel \xi) \qquad \text{for every } \nu \in \mathcal{M},$$

then $\rho = \xi$.

HINT, p. 673

SOLUTION, p. 673

7. Misspecified models: when $\mu \notin \mathcal{M}$

The cumulative bound Section 4 assumed the true environment μ lives in the model class $\mathcal{M}$. What happens when it does not? We now show the bound *degrades gracefully*: it splits cleanly into a complexity term ("cost of not knowing which model is best"), exactly as before, plus a linear-in-time approximation term ("cost of no model being right"). See (Hutter 2005, §3.2.8) for the original treatment.

Throughout this section, we no longer assume $\mu \in \mathcal{M}$. We will state the bound in terms of an arbitrary $\hat{\mu} \in \mathcal{M}$, so taking the infimum over $\hat{\mu}$ on the right-hand side gives the tightest version by using the "closest" approximation of $\hat{\mu} \in \mathcal{M}$ to μ.

Exercise 7.1 (Misspecified KL bound) [10]. Show that for all $\hat{\mu} \in \mathcal{M}$,

$$D_n(\mu \parallel \xi) \;\leq\; -\ln w_{\hat{\mu}} \;+\; D_n(\mu \parallel \hat{\mu}).$$

HINT, p. 673

SOLUTION, p. 673

Remark (Reading the two terms). The right-hand side splits into two qualitatively different terms:

- $-\ln w_{\hat{\mu}}$ is *constant in* n: under the Solomonoff prior, this is $K(\hat{\mu}) \ln 2$. The familiar "complexity" cost of search.
- The approximation term $D_n(\mu \parallel \hat{\mu}) = \sum_{t=1}^{n} d_t(\mu \parallel \hat{\mu})$ (by Exercise 2.1) is a sum of n per-step KLs; in general it grows with n. It vanishes identically iff $\hat{\mu}$ matches μ on every μ-reachable history, in particular when $\mu \in \mathcal{M}$ and we pick $\hat{\mu} = \mu$, recovering Section 4.

Combining with Pinsker (Theorem 4.1) gives the corresponding bound on S_∞^μ, which is infinite in general but inherits the rate of $\hat{\mu}$.

Details on how to define the best choice of $\hat{\mu}$ are in Appendix D.

8. Pareto optimality under squared loss

The squared specialization $\mathcal{F}(\nu, \rho) = S_n(\nu \parallel \rho)$ (Definition 4.2) lives one level down from KL: at the conditional distributions $\nu(\cdot \mid x_{<t})$. The natural Pythagorean decomposition at a fixed history uses *posterior* weights $w(\nu \mid x_{<t})$: those are the weights that make $\xi(x_t \mid x_{<t})$ the mean of the $\nu(x_t \mid x_{<t})$'s (via the posterior-predictive form, Definition 1.2). The Pareto-optimality aggregation, however, uses prior weights w_ν. Bridging the two takes one extra Bayes-rule step.

Exercise 8.1 (Squared Pythagorean identity) [10]. Fix any $t \in \{1, \dots, n\}$ and any history $x_{<t} \in \mathbb{B}^{t-1}$: *the history is arbitrary, not sampled from any environment*. For any $x_t \in \mathbb{B}$ and any predictor ρ, show

$$\begin{aligned}&\big(\xi(x_t \mid x_{<t}) - \rho(x_t \mid x_{<t})\big)^2 \;+\; \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t}) \big(\nu(x_t \mid x_{<t}) - \xi(x_t \mid x_{<t})\big)^2 \\ &= \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t}) \big(\nu(x_t \mid x_{<t}) - \rho(x_t \mid x_{<t})\big)^2.\end{aligned}$$

HINT, p. 673

SOLUTION, p. 673

Exercise 8.2 (Squared Pareto-optimality) [05]. Show that ξ is Pareto-optimal under $S_n(\nu \parallel \rho)$ in the sense of Definition 6.1: if ρ is any predictor with

$$S_n(\nu \parallel \rho) \;\le\; S_n(\nu \parallel \xi) \qquad \text{for every } \nu \in \mathcal{M},$$

then $\rho(x_t \mid x_{<t}) = \xi(x_t \mid x_{<t})$ at every history $x_{<t} \in \mathbb{B}^{t-1}$ *with* $\xi(x_{<t}) > 0$ (equivalently, ξ-almost surely) and every $x_t \in \mathbb{B}$. Histories that the mixture never reaches ($\xi(x_{<t}) = 0$, i.e., reached by no $\nu \in \mathcal{M}$ either) are invisible to the loss and are not pinned down.

HINT, p. 674

SOLUTION, p. 674

→ *Appendix A. Proof of Pinsker's inequality*: Supplementary Material, p. 525

→ *Appendix B. Proof of KL non-negativity*: Supplementary Material, p. 526

→ *Appendix C. Model Class $\mathcal{M}$*: Supplementary Material, p. 527

→ *Appendix D. Best choice of $\hat{\mu} \in \mathcal{M}$.*: Supplementary Material, p. 528

References

T. M. Cover and J. A. Thomas (2006). *Elements of Information Theory*. Wiley-Interscience.

Marcus Hutter (2005). *Universal Artificial Intelligence: Sequential Decisions based on Algorithmic Probability*. Springer.

Marcus Hutter, David Quarel, and Elliot Catt (2024). *An Introduction to Universal Artificial Intelligence*. Chapman & Hall.

D. E. Knuth (1973). *The Art of Computer Programming, Volume I: Fundamental Algorithms*. Addison-Wesley.

CHAPTER D.3.2

AIXI

David Quarel (ARENA)

Exploring the Bayesian optimal policy for history based reinforcement learning.

Source: iliad-intensive.org/agency/aixi · CC BY 4.0 · Slides: online

Prerequisites

- Solomonoff Induction (p. 315) — **start there first.** The Bayesian mixture, posterior updates, and dominance carry over directly; AIXI reuses them in the action setting.
- Comfort with discrete probability (conditional distributions, chain rule, expectations).
- Familiarity with sequential decision-making / reinforcement learning (agents, rewards, discounting, value functions) is useful but not assumed.

WHAT YOU'LL LEARN

- Formalize the agent-environment interaction, value functions, and the Bayes-optimal policy that defines AIXI.
- Derive the explicit expectimax form of AIXI.
- Prove on-policy value convergence and that AIXI cannot be fooled in deterministic environments.
- Prove the self-optimizing property via likelihood-ratio martingales and change of measure.

Much of this material is drawn from (Hutter et al. 2024). The book provides fuller explanations, additional context, and proofs omitted here.

Difficulty ratings. Each subproblem is tagged with a rating in square brackets using Knuth's scale (Knuth 1973): roughly, **[00]** trivial, **[10]** 15-minute pencil-and-paper, **[20]** 1–2 hours, **[30]** several hours to a day, **[40]** a significant research result. Intermediate values are possible. See Appendix B for the full scale.

Problems marked **(∗)** are less interesting/insightful, and while the result may be used later, I would recommend skipping them on a first pass.

Overview

AIXI is the *action* half of **Universal AI**: it lifts the Bayesian mixture from Solomonoff induction (p. 315) — learning to predict — to learning to act. Now the agent takes actions that shape what it observes, and AIXI is the **Bayes-optimal policy** over a universal mixture ξ of environments. It learns to act as well as if it knew the true environment μ. Under the universal choice — $\mathcal{M}$ = all lower-semicomputable environments, prior $w_\nu = 2^{-K(\nu)}$ — the assumption "$\mu \in \mathcal{M}$" again becomes "the universe is computable."

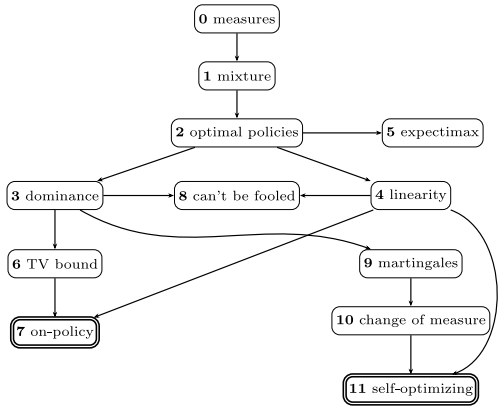
Goal and Roadmap

This exercise sheet builds towards three main results:

- **On-policy value convergence** (Section 7): the Bayesian mixture ξ learns to predict the value of any fixed policy as well as the true environment μ.
- **AIXI can't be fooled** (Section 8): in deterministic environments, the Bayes-optimal agent is guaranteed non-zero value whenever optimal value is non-zero.
- **Self-optimizing property** (Section 11, advanced stretch goal): the Bayes-optimal policy π^*_ξ learns to *act* as well as if it knew μ, provided that any learnable policy can achieve this. This is the central theoretical justification for the AIXI agent.

A good target is to complete Sections 7 and 8. The self-optimizing property (Sections 9–11) requires substantial additional machinery (supermartingales, change of measure) and is an advanced stretch goal.

Critical path. The three results share a common foundation (Sections 0–4) and then diverge:



In summary:

- Sections 0–4 establish the Bayesian RL framework: measure algebra, mixture properties, existence of optimal policies, dominance, and linearity.
- Section 5 derives the explicit expectimax form of AIXI.

- Sections 6–7 prove *on-policy value convergence*: V_ξ^π and V_μ^π become indistinguishable for any fixed π.
- Section 8 shows *AIXI can't be fooled*: the Bayes-optimal agent achieves non-zero value whenever optimal value is non-zero (in deterministic environments).
- Sections 9–11 (advanced stretch goal) prove the *self-optimizing property*: if any policy can learn to act optimally, π_ξ^* inherits this.

Setup

An **agent** interacts with an **environment** in discrete time steps $t = 1, 2, \ldots$.

Definition 0.1 (Spaces and notation). - $\mathcal{A}$: finite set of **actions**

- $\mathcal{O}$: finite set of **observations**
- $\mathcal{R} \subset [0,1]$: finite set of **rewards**
- $\mathcal{E} := \mathcal{O} \times \mathcal{R}$: set of **percepts**; $e_t = (o_t, r_t) \equiv o_t r_t$
- $\mathcal{H}^t := (\mathcal{A} \times \mathcal{E})^t$: set of all histories of length t
- $\mathcal{H}^* := \cup_{t=0}^\infty \mathcal{H}^t$: set of all finite **histories**
- $\mathcal{H} \equiv \mathcal{H}^*$: set of all finite histories
- $\mathcal{H}^\infty := (\mathcal{A} \times \mathcal{E})^\infty$: set of all infinite histories
- ΔS: set of all probability distributions over set S
- $[\![\cdot]\!]$: **Iverson bracket**: $[\![P]\!] = 1$ if P is true, 0 if false
- t: current time step; m: finite horizon; $1 \leq t \leq m$
- i, j, k: arbitrary integer indices
- xy: concatenation
- ϵ: the empty string/history
- $æ_{i:j} := a_i e_i \, a_{i+1} e_{i+1} \cdots a_j e_j$: history segment from time i to j
- $æ_{<t} := a_1 e_1 \, a_2 e_2 \cdots a_{t-1} e_{t-1}$: history up to (but not including) time t

Definition 0.2 (Policy π). A **policy** $\pi : \mathcal{H} \to \Delta\mathcal{A}$ maps each history to a probability distribution over actions. Given history $æ_{<t}$:

- $\pi(\cdot \mid æ_{<t})$ is a distribution over $\mathcal{A}$,
- $\pi(a_t \mid æ_{<t}) \in [0,1]$ is the probability of choosing action a_t,
- the agent samples $a_t \sim \pi(\cdot \mid æ_{<t})$.

A policy is **deterministic** if $\pi(a \mid æ_{<t}) \in \{0,1\}$ for all $a, æ_{<t}$. We write $\pi(æ_{i:j}) := \prod_{k=i}^{j} \pi(a_k \mid æ_{<k})$.

Definition 0.3 (Environment ν). An **environment** $\nu : \mathcal{H} \times \mathcal{A} \to \Delta\mathcal{E}$ maps each history–action pair to a distribution over percepts. Given history $æ_{<t}$ and action a_t:

- $\nu(\cdot \mid æ_{<t}a_t)$ is a distribution over $\mathcal{E}$,
- $\nu(e_t \mid æ_{<t}a_t) \in [0,1]$ is the probability of percept e_t,
- the environment samples $e_t \sim \nu(\cdot \mid æ_{<t}a_t)$.

We write $\nu(æ_{i:j}) := \prod_{k=i}^{j} \nu(e_k \mid æ_{<k}a_k)$. This satisfies the chain rule: $\nu(æ_{i:j}) = \nu(æ_{i:j-1}) \cdot \nu(e_j \mid æ_{i:j-1}a_j)$ or in the form we will usually use, $\nu(æ_{1:t}) = \nu(æ_{<t}) \cdot \nu(e_t \mid æ_{<t}a_t)$. An environment is **deterministic** if $\nu(e_t \mid æ_{<t}a_t) \in \{0,1\}$ for all $e_t, æ_{<t}, a_t$ (each percept is produced with certainty). We denote the true (unknown) environment by μ.

Definition 0.4 (Interaction measure ν^π). When policy π interacts with environment ν, the joint probability of a history segment $æ_{i:j}$ given past $æ_{<i}$ is

$$\nu^\pi(æ_{i:j} \mid æ_{<i}) \;:=\; \prod_{k=i}^{j} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k).$$

0. Properties of Measures

Exercise 0.1 (Factorization) [05]. Show that $\nu^\pi(æ_{1:t}) = \pi(æ_{1:t}) \cdot \nu(æ_{1:t})$.

SOLUTION, p. 676

Exercise 0.2 (Chain rule) [05]. Show that $\nu(æ_{1:t}) = \nu(æ_{<t}) \cdot \nu(e_t \mid æ_{<t}a_t)$.

SOLUTION, p. 676

Exercise 0.3 (*) (Marginalizing percepts) [03]. Show that $\sum_{e_t} \nu^\pi(a_t e_t \mid æ_{<t}) = \pi(a_t \mid æ_{<t})$.

SOLUTION, p. 676

Exercise 0.4 (*) (Deterministic interaction measure) [05]. Let π be a deterministic policy, i.e. at each history $æ_{<k}$ there is a unique action a_k^* with $\pi(a_k^* \mid æ_{<k}) = 1$. Show that, provided $æ_{i:j}$ is consistent with π (i.e. $a_k = a_k^*$ for every $k \in \{i, \dots, j\}$):

$$\nu^\pi(æ_{i:j} \mid æ_{<i}) \;=\; \nu(æ_{i:j} \mid æ_{<i}),$$

i.e. on π-consistent futures the interaction measure reduces to the environment measure: every policy factor is 1.

SOLUTION, p. 676

Exercise 0.5 [05]. (General chain rule for ν^π) Show that for any $p \le q \le r$:

$$\nu^\pi(æ_{p:r} \mid æ_{<p}) \;=\; \nu^\pi(æ_{p:q} \mid æ_{<p}) \cdot \nu^\pi(æ_{q+1:r} \mid æ_{<p}\, æ_{p:q}).$$

SOLUTION, p. 677

The Bayesian Mixture and Value Function

Definition 0.1 (Model class, prior, and Bayesian mixture ξ). Let $\mathcal{M} = \{\nu_1, \nu_2, \ldots\}$ be a countable class of environments with prior weights $w_\nu > 0$ satisfying $\sum_{\nu \in \mathcal{M}} w_\nu \leq 1$. We assume $\mu \in \mathcal{M}$.

The **Bayesian mixture** ξ is defined as

$$\xi(æ_{1:t}) := \sum_{\nu \in \mathcal{M}} w_\nu \, \nu(æ_{1:t}) \quad \text{and} \quad \xi(e_t \mid æ_{<t} a_t) := \xi(æ_{1:t}) / \xi(æ_{<t}).$$

One can show that the one-step predictive distribution can be written as

$$\xi(e_t \mid æ_{<t} a_t) \;=\; \sum_{\nu \in \mathcal{M}} w(\nu \mid æ_{<t}) \, \nu(e_t \mid æ_{<t} a_t),$$

where the **posterior weight** is

$$w(\nu \mid æ_{<t}) \;:=\; w_\nu \, \frac{\nu(æ_{<t})}{\xi(æ_{<t})}, \qquad w(\nu \mid \epsilon) := w_\nu.$$

See Appendix A for a worked example.

Definition 0.2 (Expectation $\mathbb{E}^\pi_\nu$). For a function $f : \mathcal{H} \to \mathbb{R}$ of a finite future segment $æ'_{t:m}$:

$$\mathbb{E}^\pi_\nu[f \mid æ_{<t}] \;:=\; \mathop{\mathbb{E}}_{æ_{t:m} \sim \nu^\pi(\cdot \mid æ_{<t})}[f] \;=\; \sum_{æ'_{t:m}} \nu^\pi(æ'_{t:m} \mid æ_{<t}) \, f(æ'_{t:m}).$$

For functions f of the infinite future (like the value function), with a sequence $f_1, f_2, \ldots \to f$, where f_m depends only on $æ'_{t:m}$, we define $\mathbb{E}^\pi_\nu[f \mid æ_{<t}] := \lim_{m \to \infty} \mathbb{E}^\pi_\nu[f_m \mid æ_{<t}]$.

Definition 0.3 (Discounted return $G_{t:m}$). Fix a discount factor $\gamma \in (0, 1)$, held constant throughout. The **discounted return** at time step t, up to horizon m, is

$$G^\gamma_{t:m} \;:=\; \sum_{k=t+1}^{m} \gamma^{k-(t+1)} \, r_k \;=\; r_{t+1} + \gamma \, r_{t+2} + \cdots + \gamma^{m-t} \, r_m.$$

Since γ is fixed we drop it and write $G_{t:m} \equiv G^\gamma_{t:m}$. It satisfies the recursion $G_{t:m} = r_{t+1} + \gamma \, G_{t+1:m}$, with $G_{m-1:m} = r_m$ and $G_{n:m} = 0$ for $n \geq m$. The **infinite-horizon return** is the pointwise limit

$$G_{\geq t} \;\equiv\; G_{t:\infty} \;:=\; \lim_{m \to \infty} G_{t:m} \;=\; \sum_{k=t+1}^{\infty} \gamma^{k-(t+1)} \, r_k,$$

with the clean recursion $G_{\geq t} = r_{t+1} + \gamma \, G_{\geq t+1}$ and no boundary cases.

Definition 0.4 (Value function[1]). The **value** of policy π in environment ν with horizon $m \geq t$ given history $æ_{<t}$ is

$$\begin{aligned} V_\nu^{\pi,m}(æ_{<t}) &:= (1-\gamma)\,\mathbb{E}_\nu^\pi[G_{t-1:m} \mid æ_{<t}] \\ &= (1-\gamma)\sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t})\, G_{t-1:m} \\ &= (1-\gamma)\sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t}) \left[\sum_{k=t}^{m} \gamma^{k-t} r_k\right]. \end{aligned}$$

The **infinite-horizon value** is the pointwise limit $V_\nu^{\pi,\infty}(æ_{<t}) := \lim_{m\to\infty} V_\nu^{\pi,m}(æ_{<t}) = (1-\gamma)\,\mathbb{E}_\nu^\pi[G_{\geq t-1} \mid æ_{<t}]$. We write $V_\nu^\pi \equiv V_\nu^{\pi,\infty}$ for short.

The **optimal value** is $V_\nu^{*,m}(æ_{<t}) := \sup_\pi V_\nu^{\pi,m}(æ_{<t})$ for $m \in \mathbb{N} \cup \{\infty\}$, with $V_\nu^* \equiv V_\nu^{*,\infty}$. [2] An **optimal policy** π_ν^* satisfies $V_\nu^{\pi_\nu^*} = V_\nu^*$. The **Bayes-optimal policy** is $\pi_\xi^* \in \arg\max_\pi V_\xi^\pi$.

Remark (AIXI). All results in this sheet hold for any countable $\mathcal{M}$ and prior weights w_ν. **AIXI** is a special case of the Bayes-optimal policy π_ξ^* for the particular choice $\mathcal{M} :=$ the class of all lower-semicomputable chronological semimeasures, and prior $w_\nu := 2^{-K(\nu)}$, where $K(\nu)$ is the Kolmogorov complexity of ν: the length of the shortest program that computes ν. The resulting agent is written $\pi_{\xi_U}^*$, or simply AIξ.

Why this $\mathcal{M}$? By including every computable environment, the assumption $\mu \in \mathcal{M}$ reduces to "the universe is computable": as weak an assumption as one can make.

Why this prior? The prior $2^{-K(\nu)}$ is *dominant*: for any other computable prior w'_ν, there exists a constant $c > 0$ such that $2^{-K(\nu)} \geq c \cdot w'_\nu$ for all ν.

Caveat: The constant c depends on the choice of universal Turing machine U, and adversarial choices of U can make AIXI behave arbitrarily badly (Leike & Hutter 2015).

See (Hutter et al. 2024): Chapter 2.7 for Kolmogorov complexity, Chapters 3.7–3.8 for the model class and universal prior, and Chapter 7.4 for AIXI itself.

1. Properties of the Bayesian Mixture

Exercise 1.1 (*) (Posterior update) [10]. Show that the posterior updates multiplicatively:

$$w(\nu \mid æ_{1:t}) \;=\; w(\nu \mid æ_{<t}) \frac{\nu(e_t \mid æ_{<t}a_t)}{\xi(e_t \mid æ_{<t}a_t)}.$$

HINT, p. 677

SOLUTION, p. 677

Exercise 1.2 [15]. (One-step predictive distribution of ξ) Starting from $\xi(æ_{1:t}) = \sum_{\nu\in\mathcal{M}} w_\nu \nu(æ_{1:t})$, derive the one-step predictive form:

$$\xi(e_t \mid æ_{<t}a_t) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\,\nu(e_t \mid æ_{<t}a_t).$$

HINT, p. 677

SOLUTION, p. 678

Exercise 1.3 (Bounded value) [10]. Show that $V_\nu^{\pi,m}(æ_{<t}) \in [0,1]$ for any $\nu, \pi, m \in \mathbb{N}\cup\{\infty\}, æ_{<t}$.

HINT, p. 678

SOLUTION, p. 678

Exercise 1.4 (*) [15]. (Multi-step posterior linearity of ξ^π) Extend Exercise 1.2 to multi-step histories: show that for finite $m \geq t$,

$$\xi^\pi(æ_{t:m} \mid æ_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\,\nu^\pi(æ_{t:m} \mid æ_{<t}).$$

HINT, p. 678

SOLUTION, p. 678

2. Existence of Optimal Policies

Recall that $V_\nu^*(æ_{<t}) := \sup_\pi V_\nu^\pi(æ_{<t})$ is defined as a supremum over all policies. In general, a supremum need not be achieved: for example, $\sup_{x\in(0,1)} x = 1$, but no $x \in (0,1)$ attains this value. This problem shows that in our setting, the supremum *is* attained, so an optimal policy π_ν^* with $V_\nu^{\pi_\nu^*} = V_\nu^*$ exists.

Exercise 2.1 [05]. Consider a single time step. Given history $æ_{<t}$ and a function $Q : \mathcal{A} \to \mathbb{R}$, show that $\sup_\pi \sum_{a\in\mathcal{A}} \pi(a \mid æ_{<t})\,Q(a) = \max_{a\in\mathcal{A}} Q(a)$, and that the supremum is attained by the deterministic policy that places all probability on an action achieving the maximum.

SOLUTION, p. 679

Exercise 2.2 (Bellman equation) [15]. Show that for finite m and $t \leq m$:

$$V_\nu^{\pi,m}(æ_{<t}) \;=\; \sum_{æ_t'} \nu^\pi(æ_t' \mid æ_{<t})\Big[(1-\gamma)\,r_t' \;+\; \gamma\,V_\nu^{\pi,m}(æ_{<t}\,æ_t')\Big],$$

HINT, p. 679

SOLUTION, p. 679

Exercise 2.3 (*) (Backward induction) [20]. Using the Bellman equation from Exercise 2.2 and Exercise 2.1, show by backward induction on $t = m, m-1, \ldots, 1$ that for each finite m, a deterministic optimal policy exists.

SOLUTION, p. 680

Exercise 2.4 (Bellman optimality equation) [10]. Using Exercise 2.3, show that for finite m:

$$V_\nu^{*,m}(æ_{<t}) \;=\; \max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t}a'_t)\Big[(1-\gamma)\, r'_t \;+\; \gamma\, V_\nu^{*,m}(æ_{<t}\, æ'_t)\Big].$$

where $e'_t = o'_t r'_t$ and $æ'_t = a'_t e'_t$.

SOLUTION, p. 681

Exercise 2.5 (*) [15]. (Existence of V_ν^*) Show that the pointwise limit $V_\nu^*(æ_{<t}) := \lim_{m\to\infty} V_\nu^{*,m}(æ_{<t})$ exists for all histories $æ_{<t}$.

HINT, p. 681

SOLUTION, p. 681

Exercise 2.6 (Infinite-horizon Bellman optimality equation) [10]. Show that the Bellman optimality equation (Exercise 2.4) extends to $m = \infty$:

$$V_\nu^*(æ_{<t}) \;=\; \max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t}a'_t)\Big[(1-\gamma)\, r'_t \;+\; \gamma\, V_\nu^*(æ_{<t}\, a'_t\, e'_t)\Big].$$

SOLUTION, p. 682

Exercise 2.7 (*) (Optimal policy for infinite horizon) [25]. Show that a deterministic policy π_ν^* exists achieving $V_\nu^{\pi_\nu^*}(æ_{<t}) = V_\nu^*(æ_{<t})$ for all $æ_{<t}$.

Approach:

(a) Define π_ν^* as the greedy policy (the arg max action from Exercise 2.6).

(b) Write two Bellman equations: one for V_ν^*, one for $V_\nu^{\pi_\nu^*,m}$.

(c) Define $\Delta V_\nu^m := V_\nu^* - V_\nu^{\pi_\nu^*,m}$ and obtain a recursive equation for it.

(d) Iterate out to the finite horizon, noting $V_\nu^{\pi_\nu^*,m}(æ_{1:m}) = 0$ (why?)

(e) Bound the tail, take $m \to \infty$.

SOLUTION, p. 682

3. Dominance of the Bayesian Mixture

Exercise 3.1 [03]. Show that $\xi(æ_{<t}) \geq w_\nu \cdot \nu(æ_{<t})$ for every $\nu \in \mathcal{M}$.

SOLUTION, p. 683

Exercise 3.2 [10]. Conclude that $\xi^\pi(æ_{1:m}) \geq w_\nu \cdot \nu^\pi(æ_{1:m})$ for any $\nu \in \mathcal{M}$ and any policy π.

SOLUTION, p. 684

4. Properties of V_ξ

Exercise 4.1 [15]. Show that for finite m:

$$V_\xi^{\pi,m}(æ_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{\pi,m}(æ_{<t}).$$

HINT, p. 684

SOLUTION, p. 684

Fact 4.1 (Dominated convergence for sums). If $a_{\nu,m} \to a_\nu$ as $m \to \infty$ for each ν, and $|a_{\nu,m}| \le b_\nu$ for all m with $\sum_\nu b_\nu < \infty$, then $\sum_\nu a_{\nu,m} \to \sum_\nu a_\nu$. (For finite sums this is trivial.)

Exercise 4.2 (*) (Infinite-horizon linearity) [17]. Show that the result extends to $m = \infty$:

$$V_\xi^{\pi}(æ_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{\pi}(æ_{<t}).$$

HINT, p. 684

SOLUTION, p. 684

Exercise 4.1 shows V_ξ^π is linear in ν. The optimal value V_ξ^* is only convex: a single policy must perform well across all $\nu \in \mathcal{M}$ simultaneously, rather than being tailored to each ν individually.

Exercise 4.3 (*) [10]. (Convexity of V_ξ^*) Using Exercise 4.1, show that for finite m:

$$V_\xi^{*,m}(æ_{<t}) \;\le\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{*,m}(æ_{<t}).$$

HINT, p. 685

SOLUTION, p. 685

Exercise 4.4 (*) [15]. (Non-linearity of V_ξ^*) Show by example that the inequality in Exercise 4.3 can be strict, i.e. $V_\xi^{*,m}$ is *not* linear in ν.

HINT, p. 685

SOLUTION, p. 685

5. The Expectimax Form of AIXI

We have specified AIXI only implicitly, as the Bayes-optimal policy π_ξ^* (Definition 0.4). We now unroll this definition into the explicit **expectimax** expression: an alternating sequence of maximizations over actions and ξ-expectations over percepts.

Exercise 5.1 (Expectimax form of AIXI) [20]. By iterating the finite-horizon Bellman optimality equation (Exercise 2.4), show that

$$
\begin{aligned}
V_\xi^{*,m}(æ_{<t}) \;=\; (1-\gamma)\max_{a_t}\sum_{e_t}\xi(e_t \mid æ_{<t}a_t) \\
\max_{a_{t+1}}\sum_{e_{t+1}}\xi(e_{t+1} \mid æ_{<t+1}a_{t+1})\cdots \\
\cdots\max_{a_m}\sum_{e_m}\xi(e_m \mid æ_{<m}a_m)\,G_{t-1:m}
\end{aligned}
$$

and hence, collecting the percept factors with the chain rule (Exercise 0.2) and taking $m \to \infty$ (Exercises 2.5 and 2.7), that AIXI selects the action

$$
\begin{aligned}
a_t \;&=\; \pi_\xi^*(æ_{<t}) \\
&\in\; \arg\max_{a_t}\lim_{m\to\infty}\sum_{e_t}\max_{a_{t+1}}\sum_{e_{t+1}}\cdots\max_{a_m}\sum_{e_m}\xi(e_{t:m} \mid æ_{<t}\,a_{t:m})\,G_{t-1:m}
\end{aligned}
$$

HINT, p. 685

SOLUTION, p. 685

On-Policy Value Convergence

The following problems prove the first two main results: on-policy value convergence (Section 7), and that AIXI can't be fooled in deterministic environments (Section 8). The path to the self-optimizing property (advanced stretch goal) resumes at Section 9.

6. Bounding Expectation Differences by Total Variation

Definition 6.1 (Total variation distance). The **total variation distance** between probability measures P and Q on a countable set Ω is $\mathrm{TV}[\Omega](P,Q) := \sup_{S\subseteq\Omega}|P(S)-Q(S)|$, where $P(S) := \sum_{\omega\in S}P(\omega)$.

Definition 6.2 (Expectation under P). For a probability measure P on a countable set Ω and a function $f:\Omega\to\mathbb{R}$, the **expectation** of f under P is $\mathbb{E}_P[f] := \sum_{\omega\in\Omega} f(\omega)\,P(\omega)$.

The following technical lemma is needed for the on-policy value convergence proof.

Exercise 6.1 (*) [15]. Let $f:\Omega\to[0,c]$. Show that $\left|\mathbb{E}_P[f]-\mathbb{E}_Q[f]\right| \le c\cdot\mathrm{TV}[\Omega](P,Q)$.

HINT, p. 687

SOLUTION, p. 687

7. On-Policy Value Convergence of Bayes

The following definitions are needed for the on-policy value convergence theorem.

Definition 7.1 (Probability of events). A **finite-length event** is a set $A \subseteq (\mathcal{A} \times \mathcal{E})^t$ of histories of fixed length t. Its probability under ν^π is

$$\nu^\pi(A) \;:=\; \sum_{æ_{1:t} \in A} \nu^\pi(æ_{1:t}).$$

An **event on infinite histories** is any set built from finite-length events by countable unions, intersections, and complements.[3] Probabilities of such events are uniquely determined by two properties:

- **Normalization:** $\nu^\pi\big((\mathcal{A} \times \mathcal{E})^\infty\big) = 1$.
- **Countable additivity:** if $A_1, A_2, \dots$ are pairwise disjoint events, then $\nu^\pi\big(\bigcup_{n=1}^\infty A_n\big) = \sum_{n=1}^\infty \nu^\pi(A_n)$.

From these, all standard rules of probability can be derived (e.g. $\nu^\pi(A \cup B) = \nu^\pi(A) + \nu^\pi(B) - \nu^\pi(A \cap B)$, etc.), though we will not prove them here. Two derived properties we use explicitly:

- **Complement:** $\nu^\pi(A^c) = 1 - \nu^\pi(A)$.
- **Monotone limits:** if $A_1 \supseteq A_2 \supseteq \cdots$, then $\nu^\pi\big(\bigcap_{n=1}^\infty A_n\big) = \lim_{n\to\infty} \nu^\pi(A_n)$.

Further useful notions:

- **Conditional probability:** The probability of event A given observed history $æ_{<t}$ is

$$\nu^\pi(A \mid æ_{<t}) \;:=\; \frac{\nu^\pi(\{æ_{<t} h : h \in A\})}{\nu^\pi(æ_{<t})}.$$

Example 7.2 (A fair coin shows 1 infinitely often). Let $\mathcal{A} = \mathcal{O} = \{0, 1\}$, $\mathcal{R} = \{0\}$, and let ν be a fair coin that ignores the action: $\nu(o_t = 1 \mid æ_{<t}\, a_t) = \frac{1}{2}$ for all $æ_{<t}, a_t$. Consider the event $F :=$ "$o_t = 1$ for infinitely many t".

Step 1: Build F^c from finite-prefix events. For each t, the set $\{o_t = 0\} := \{æ_{1:t} : o_t = 0\}$ is a finite-length event. For each N and $T \geq N$, the set $D_{N,T} := \bigcap_{t=N}^T \{o_t = 0\}$ is also a finite-length event (determined by time T). Define $C_N := \bigcap_{t=N}^\infty \{o_t = 0\}$ ("all zeros from time N onwards"): this is an infinite-history event, built as a countable intersection of finite-length events. Then $F^c = \bigcup_{N=1}^\infty C_N$ ("eventually all zeros").

Step 2: Compute $\nu^\pi(C_N) = 0$. Since $D_{N,T} \supseteq D_{N,T+1} \supseteq \cdots$ (adding more constraints), the monotone limit property (for decreasing sets) gives:

$$\nu^\pi(C_N) \;=\; \lim_{T\to\infty} \nu^\pi(D_{N,T}) \;=\; \lim_{T\to\infty} \left(\tfrac{1}{2}\right)^{T-N+1} \;=\; 0.$$

(The probability $\nu^\pi(D_{N,T}) = (1/2)^{T-N+1}$ is independent of π, since ν ignores actions.)

Step 3: By countable additivity: $\nu^\pi(F^c) \leq \sum_{N=1}^\infty \nu^\pi(C_N) = 0$, so $\nu^\pi(F) = 1$. A fair coin shows 1 infinitely often, almost surely.

As such, F^c (the set of all infinite histories containing only finitely many 1s) is a ν^π-measure-zero set. Such histories "exist" as infinite sequences, but occur with probability zero.

Definition 7.3 (Covering[4]). We say Q **covers** P if Q is positive everywhere P is: $P(A) > 0 \implies Q(A) > 0$ for all events A. Equivalently: any event that Q rules out, P also rules out. By Section 3, ξ^π covers μ^π.

Definition 7.4 (Convergence ν^π-almost surely). Let $f_t : \mathcal{H}^{t-1} \to \mathbb{R}$ be a sequence of functions, where f_t depends on the history $æ_{<t}$ of length $t-1$. We write $f_t(æ_{<t}) \to 0$ ν^π-**almost surely** (ν^π-a.s.) if

$$\nu^\pi\Big(\Big\{æ_{1:\infty} : f_t(æ_{<t}) \not\to 0\Big\}\Big) = 0.$$

This set is built from finite-prefix conditions: $\{f_t \not\to 0\} = \bigcup_{n=1}^\infty \bigcap_{N=1}^\infty \bigcup_{t=N}^\infty \left\{|f_t(æ_{<t})| > \frac{1}{n}\right\}$, so its probability is well-defined (Definition 7.1).

Example 7.5 (Unpacking "$f_t \not\to 0$"). The set $\{f_t \not\to 0\}$ looks intimidating, but it reads naturally from the inside out:

- $\left\{|f_t(æ_{<t})| > \frac{1}{n}\right\}$ is a finite-length event: "at time t, the function is at least $\frac{1}{n}$ away from zero."
- $\bigcup_{t=N}^\infty \left\{|f_t| > \frac{1}{n}\right\}$: "at *some* time $t \geq N$, f_t is at least $\frac{1}{n}$ away from zero."
- $\bigcap_{N=1}^\infty \bigcup_{t=N}^\infty \left\{|f_t| > \frac{1}{n}\right\}$: "for *every* N, there is some $t \geq N$ where f_t is at least $\frac{1}{n}$ from zero", i.e., f_t exceeds $\frac{1}{n}$ *infinitely often*.
- $\bigcup_{n=1}^\infty \bigcap_{N=1}^\infty \bigcup_{t=N}^\infty \left\{|f_t| > \frac{1}{n}\right\}$: "for *some* $\frac{1}{n} > 0$, f_t exceeds $\frac{1}{n}$ infinitely often."

This last condition is exactly $\{f_t \not\to 0\}$: convergence $f_t \to 0$ means that for *every* $\varepsilon > 0$, $|f_t| \leq \varepsilon$ for all sufficiently large t. Its negation is that *some* $\varepsilon > 0$ is exceeded infinitely often.

Each layer is a countable union or intersection of the previous, so the whole set is a well-defined event on infinite histories.

The Bayesian agent uses the mixture ξ because the true environment μ is unknown. A natural question: does planning with ξ eventually become as good as planning with μ? The following theorem says *yes*: the value of any fixed policy π, as evaluated by ξ, converges to the value under the true environment μ, along histories that μ^π actually generates.[5] This tells us that the Bayesian mixture "learns" to predict the true environment's value, on policy.

Theorem 7.6 (On-policy value convergence; (Hutter et al. 2024, Theorem 7.3.1)). For any $\mu \in \mathcal{M}$ and any policy π: $V_\xi^\pi(æ_{<t}) - V_\mu^\pi(æ_{<t}) \to 0$ as $t \to \infty$, μ^π-almost surely. That is, the set of infinite histories along which the value difference

does not vanish has μ^π-probability zero:

$$\mu^\pi\Big(\Big\{æ_{1:\infty} : V_\xi^\pi(æ_{<t}) - V_\mu^\pi(æ_{<t}) \not\to 0\Big\}\Big) = 0.$$

The proof reduces to a finite-horizon TV bound (Exercise 7.1), a covering argument (Exercise 7.2), and the Blackwell–Dubins theorem (Exercise 7.3). The only ingredient we do not prove is Blackwell–Dubins itself, which we state as a given fact.

Exercise 7.1 (Finite-horizon TV bound on value difference) [10]. Using Section 6, show that for finite m:

$$\big|V_\xi^{\pi,m}(æ_{<t}) - V_\mu^{\pi,m}(æ_{<t})\big| \;\le\; \mathrm{TV}[\mathcal{H}^{m-t+1}]\big(\xi^\pi(\cdot \mid æ_{<t}),\; \mu^\pi(\cdot \mid æ_{<t})\big),$$

where $\mathcal{H}^{m-t+1} := (\mathcal{A}\times\mathcal{E})^{m-t+1}$ is the set of all future history segments $æ_{t:m}$.

SOLUTION, p. 688

Exercise 7.2 (Covering) [05]. Show that ξ^π covers μ^π (Definition 7.3).

HINT, p. 688

SOLUTION, p. 688

To complete the proof, we need the TV distance on finite segments to vanish μ^π-a.s. as $t\to\infty$. This follows from the following deep result, which we state without proof:

Fact 7.7 (Blackwell–Dubins merging of opinions (Blackwell & Dubins 1962)). If Q covers P, then $\sup_S |P(S \mid æ_{<t}) - Q(S \mid æ_{<t})| \to 0$ P-almost surely, where the supremum ranges over all measurable events S (including events on infinite histories).

The proof of Blackwell–Dubins requires the Radon–Nikodym derivative and Lévy's martingale convergence theorem: tools from measure theory that are beyond the scope of this sheet. See (Hutter et al. 2024, Chapter 3.9) for discussion.

Exercise 7.3 (On-policy convergence) [15]. Using Fact 7.7, Exercise 7.1 and Exercise 7.2, prove Theorem 7.6.

SOLUTION, p. 688

Exercise 7.4 (*) [40]. Prove Fact 7.7.

HINT, p. 688

8. AIXI Cannot Be Fooled in Deterministic Environments

Theorem 8.1 (AIXI cannot act poorly in good environments). If $\mu\in\mathcal{M}$ is deterministic and $V_\mu^*(æ_{<t}) > \varepsilon > 0$ for all t along the history generated by μ and π_ξ^*, then $V_\xi^*(æ_{<t}) \ge w_\mu\,\varepsilon > 0$ for all t. (Hutter et al. 2024, Theorem 7.4.10)

Exercise 8.1 [10]. Show that $V_\xi^*(æ_{<t}) \ge w(\mu \mid æ_{<t})\cdot V_\mu^*(æ_{<t})$.

HINT, p. 689

SOLUTION, p. 689

Exercise 8.2 [15]. Show $w(\mu \mid æ_{<t}) \geq w_\mu$ when μ is deterministic, and combine with Exercise 8.1 to prove Theorem 8.1.

SOLUTION, p. 689

Remark. With the Solomonoff prior, $w_\mu = 2^{-K(\mu)}$ may be astronomically small. The result guarantees non-zero value, not near-optimal value.

Sections 9–11: Self-Optimizing Policy (Finite Model Class)

We now prove: if *any* policy can learn to act optimally, π^*_ξ also learns. We restrict to finite $\mathcal{M} = \{\nu_1, \ldots, \nu_N\}$ where each ν_i is a proper probability measure.

9. Likelihood Ratios Are Martingales

The following definition and theorem state the main goal of Sections 9–11.

Definition 9.1 (Self-optimizing). Fix a historic policy π. A policy $\tilde{\pi}$ is **self-optimizing** for $\mathcal{M}$ with respect to π if for every $\nu \in \mathcal{M}$: $V^*_\nu(æ_{<t}) - V^{\tilde{\pi}}_\nu(æ_{<t}) \to 0$ as $t \to \infty$, ν^π-almost surely.

The percepts $e_{1:\infty}$ are sampled from ν; the historic actions $a_{<t}$ come from π, which may differ from $\tilde{\pi}$.

Theorem 9.2 (Self-optimizing; (Hutter et al. 2024, Theorem 7.5.2)). Let $\mathcal{M}$ be finite and fix a historic policy π. If there exists a $\tilde{\pi}$ self-optimizing for $\mathcal{M}$ with respect to π, then π^*_ξ is also self-optimizing for $\mathcal{M}$ with respect to π: for any $\mu \in \mathcal{M}$, $V^*_\mu(æ_{<t}) - V^{\pi^*_\xi}_\mu(æ_{<t}) \to 0$ μ^π-almost surely.

Definition 9.3 (Supermartingale). A sequence of functions $X_t : \mathcal{H}^t \to \mathbb{R}_{\geq 0}$ (each depending on the history $æ_{1:t}$) is a μ^π-**supermartingale** if $\mathbb{E}_{\mu^\pi}[X_t \mid æ_{<t}] \leq X_{t-1}$ for all t; that is,

$$\sum_{æ_t} \pi(a_t \mid æ_{<t})\, \mu(e_t \mid æ_{<t}a_t)\, X_t(æ_{1:t}) \;\leq\; X_{t-1}(æ_{<t}).$$

If equality holds, it is a μ^π-**martingale**.

Intuition. A supermartingale is a quantity whose expected future value, given the present, is no larger than its current value: it cannot "drift upward on average." A martingale is the equality case: expected future value equals current value. The key example is the **likelihood ratio** $X_{\nu,t}(æ_{1:t}) := \nu(æ_{1:t})/\mu(æ_{1:t})$: how much more (or

less) likely the observed history is under ν than under the true environment μ. Under μ , this ratio is a martingale: the true environment does not, on average, favour any alternative ν over itself. Non-negativity plus the (super)martingale structure forces $X_{\nu,t}$ to converge (by Doob's theorem below), which is the engine behind the self-optimizing proof: it lets us separate environments that remain plausible ($X_{\nu,\infty} > 0$) from those that are eventually ruled out ($X_{\nu,\infty} = 0$), and handle each case differently in Section 11.

Fact 9.4 (Doob's supermartingale convergence). If $(X_t)_{t\geq 0}$ is a non-negative μ^π-supermartingale, then there exists a function $X_\infty : \mathcal{H}^\infty \to \mathbb{R}_{\geq 0}$ of the infinite history such that $X_t(æ_{1:t}) \to X_\infty(æ_{1:\infty})$ as $t \to \infty$, with $X_\infty(æ_{1:\infty}) < \infty$, for μ^π-almost every infinite history $æ_{1:\infty}$. That is:

$$\mu^\pi\Big(\Big\{æ_{1:\infty} \in (\mathcal{A}\times\mathcal{E})^\infty : X_t(æ_{1:t}) \not\to X_\infty(æ_{1:\infty})\Big\}\Big) \;=\; 0.$$

For each $\nu \in \mathcal{M}$, define the likelihood ratio

$$X_{\nu,t} : \mathcal{H}^t \to \mathbb{R}_{\geq 0}, \qquad X_{\nu,t}(æ_{1:t}) \;:=\; \frac{\nu(æ_{1:t})}{\mu(æ_{1:t})}, \qquad X_{\nu,0} := 1.$$

$X_{\nu,t}$ is a *function* of the history $æ_{1:t}$, not a number: different histories give different values. Throughout this section and the next, any unqualified statement of the form "$X_{\nu,t} \geq c$" or "$X_{\nu,t} \to X_{\nu,\infty}$" is shorthand for "$X_{\nu,t}(æ_{1:t}) \geq c$" or "$X_{\nu,t}(æ_{1:t}) \to X_{\nu,\infty}(æ_{1:\infty})$" holding μ^π-almost surely, i.e. on every infinite history outside a set of μ^π-measure zero. We will not track these null sets explicitly; their countable union over all claims is still a μ^π-null set.

Exercise 9.1 [15]. Show that $X_{\nu,t}$ is a μ^π-martingale: $\mathbb{E}_{\mu^\pi}[X_{\nu,t} \mid æ_{<t}] = X_{\nu,t-1}$.

HINT, p. 689

SOLUTION, p. 690

Exercise 9.2 [05]. Apply Fact 9.4 to conclude $X_{\nu,t} \to X_{\nu,\infty} < \infty$ μ^π-a.s.

SOLUTION, p. 690

Exercise 9.3 [10]. Define $X_{\xi,t} := \xi(æ_{1:t})/\mu(æ_{1:t})$. Show $X_{\xi,t} = \sum_\nu w_\nu X_{\nu,t}$ and $X_{\xi,t} \geq w_\mu > 0$.

SOLUTION, p. 690

10. Change of Measure

Exercise 10.1 [15]. Let A be a set of finite histories of length m. Show:

$$\nu^\pi[æ_{1:m} \in A] \;=\; \mathbb{E}_{\mu^\pi}\big[X_{\nu,m} \cdot [\![æ_{1:m} \in A]\!]\big].$$

HINT, p. 690

SOLUTION, p. 691

Exercise 10.2. (Given.) The identity extends to infinite histories: for any event $A \subseteq (\mathcal{A} \times \mathcal{E})^\infty$,

$$\nu^\pi[A] \;=\; \mathbb{E}_{\mu^\pi}\big[X_{\nu,\infty} \cdot [\![æ_{1:\infty} \in A]\!]\big].$$

This says that $X_{\nu,\infty}$ plays the role of the Radon–Nikodym derivative $\mathrm{d}\nu^\pi/\mathrm{d}\mu^\pi$ on the space of infinite histories. The proof rests on two ingredients beyond the scope of this worksheet:

- L^1 **martingale convergence** (closed martingale / Lévy's upward theorem): since $\mathbb{E}_{\mu^\pi}[X_{\nu,t}] = 1$ for every t, the non-negative μ^π-martingale $(X_{\nu,t})$ is uniformly integrable, so $X_{\nu,t} \to X_{\nu,\infty}$ in $L^1(\mu^\pi)$, not merely μ^π-a.s. This lets us pass the $t \to \infty$ limit through the expectation.
- **Uniqueness of measure extension** (Caratheodory / π–λ theorem): both sides of the identity are finite measures on $(\mathcal{A} \times \mathcal{E})^\infty$; the finite-history identity (Exercise 10.1) shows they agree on all cylinder events $A = A_0 \times (\mathcal{A} \times \mathcal{E})^\infty$, and cylinders generate the full event σ-algebra, so agreement extends uniquely to every event.

We omit the proof and use this identity freely below.

Exercise 10.3 (Markov inequality for change of measure) [15]. Let E be an event of infinite histories. For any $\varepsilon > 0$, show that

$$\mu^\pi\big[E \cap \{X_{\nu,\infty} \geq \varepsilon\}\big] \;\leq\; \frac{\nu^\pi[E]}{\varepsilon}.$$

HINT, p. 691

SOLUTION, p. 691

11. Proving the Self-Optimizing Property

Fix a policy $\tilde{\pi}$ that is self-optimizing for $\mathcal{M}$ with respect to π (Definition 9.1): its existence is the hypothesis of Theorem 9.2. For each $\nu \in \mathcal{M}$, define the **suboptimality gap**

$$\delta_{\nu,t} : \mathcal{H}^{t-1} \to [0,1], \qquad \delta_{\nu,t}(æ_{<t}) \;:=\; V^*_\nu(æ_{<t}) - V^{\tilde{\pi}}_\nu(æ_{<t}).$$

As with $X_{\nu,t}$, we write $\delta_{\nu,t}$ without its argument when convenient: "$\delta_{\nu,t} \to 0$ μ^π-a.s." means $\delta_{\nu,t}(æ_{<t}) \to 0$ on every infinite history outside a μ^π-null set, and similarly for "$\delta_{\nu,t} \leq 1$" etc.

Exercise 11.1 (Chain of inequalities) [20]. Show:

$$0 \;\leq\; w(\mu \mid æ_{<t})\big[V^*_\mu - V^{\pi^*_\xi}_\mu\big] \;\leq\; \sum_{\nu \in \mathcal{M}} w(\nu \mid æ_{<t})\,\delta_{\nu,t}.$$

HINT, p. 691

SOLUTION, p. 691

Exercise 11.2 (Vanishing posterior) [10]. Show: if $X_{\nu,\infty} = 0$ then $w(\nu \mid æ_{<t}) \to 0$ μ^π-a.s.

HINT, p. 692

SOLUTION, p. 692

Exercise 11.3 (Transferring convergence) [20]. Let $F := \{æ_{1:\infty} : \delta_{\nu,t}(æ_{<t}) \not\to 0\}$, the set of infinite histories on which the suboptimality gap fails to vanish. Show that $\mu^\pi\big[F \cap \{X_{\nu,\infty} > 0\}\big] = 0$.

HINT, p. 692

SOLUTION, p. 692

Exercise 11.4 (Single-term convergence) [10]. Combine Exercise 11.2 and Exercise 11.3 to show $w(\nu \mid æ_{<t})\delta_{\nu,t} \to 0$ μ^π-a.s.

SOLUTION, p. 693

Exercise 11.5 (Self-Optimizing Theorem) [15]. Combine Exercise 11.4, Exercise 11.1, Exercise 9.3 to finally prove the main result of Theorem 9.2.

HINT, p. 693

SOLUTION, p. 693

Remark. We do not need to know which policy $\tilde{\pi}$ is self-optimizing, or whether it is computable. Mere existence suffices. For countable $\mathcal{M}$, this final step is harder: a countable sum of μ^π-a.s.-convergent sequences need not converge μ^π-a.s. The general proof uses a "convergence of mixture tails" argument (Hutter 2005, Lem. 5.28).

→ *Further reading*: Supplementary Material, p. 529

→ *Appendix A. Worked Example: Bayesian Mixture and Value Function*: Supplementary Material, p. 529

→ *Appendix B. Knuth's Difficulty Scale*: Supplementary Material, p. 530

References

D. Blackwell and L. Dubins (1962). *Merging of opinions with increasing information*. Annals of Mathematical Statistics.

Marcus Hutter (2005). *Universal Artificial Intelligence: Sequential Decisions based on Algorithmic Probability*. Springer.

Marcus Hutter, David Quarel, and Elliot Catt (2024). *An Introduction to Universal Artificial Intelligence*. Chapman & Hall.

D. E. Knuth (1973). *The Art of Computer Programming, Volume I: Fundamental Algorithms*. Addison-Wesley.

Jan Leike and Marcus Hutter (2015). *Bad Universal Priors and Notions of Optimality*. CoRR.

NOTES

1. See Appendix A for a worked example. For simplicity, we consider only geometric discounting.
2. The optimal value is defined as a sup over policies. In general, a supremum need not be attained (e.g. $\sup_{x\in(0,1)} x = 1$ but no $x \in (0,1)$ achieves it). Section 2 shows that the sup is attained in our setting.
3. We are deliberately avoiding a formal treatment of measure theory here. Not all subsets of $(\mathcal{A} \times \mathcal{E})^\infty$ are measurable; we restrict to "nice" (measurable) sets built from finite-prefix conditions via countable set operations, which suffice for everything in this sheet. For a rigorous treatment using σ-algebras and probability measures, see (Hutter et al. 2024, Chapter 2.2).
4. The standard name is *absolute continuity* of P with respect to Q, written $P \ll Q$.
5. If the true environment is μ then we don't care about the behaviour of the Bayesian agent on histories that have μ^π-probability zero: such histories will never be observed anyway.

CHAPTER D.4.1

Agent Foundations

Daniel C, Satya Benson (Williams College)

Superintelligence alignment may allow only one critical try, with no trial and error on a system more capable than anything yet observed. Agent foundations seeks research directions that survive extreme optimization and self-modification.

Source: iliad-intensive.org/agency/agent-foundations · CC BY 4.0 · Slides: online

1. Prerequisites

- Comfort with elementary discrete probability (random variables, expectation, conditional probability).
- Basic formal logic (provability, quantifiers) and basic computability (programs, halting); needed for the Löb and logical-induction strands.
- Basic information theory (entropy, mutual information); needed for the optimization strand.
- No prior agent-foundations background is assumed; every term is introduced in this chapter or in the readings. An introductory overview of the AI alignment problem (as in Chapter A.1) is helpful context but not required.
- Provided refresher notes cover the gaps: a *probability theory* refresher, a *formal logic* refresher, a *computability theory* refresher, and an *information theory, causality, and statistical mechanics* refresher. Consult the one matching your weakest area before starting.

WHAT YOU'LL LEARN

(What.) By the end of this chapter, you should be able to explain *embedded agency*: why an agent built into the world it acts on (made of the same stuff, smaller than its environment, with no clean input/output boundary, able to model and modify itself) breaks the standard dualistic picture. For each of the chapter's core research directions (called 'strands') you should be able to state the central problem and a plain-language description of a main result: the complete class theorem (consequentialist foundations), Löb's theorem and the tiling obstacle (self-modification), the logical-induction criterion (logical uncertainty), optimization as local entropy reduction (optimization and thermodynamics), and selection theor-

ems (descriptive agent foundations). You should be able to describe at least one strand in mathematical detail. and explain how each strand is an offshoot of the central threads *reflective stability* and *embedded agency*.

(Why.) Aligning a superintelligence is a problem we may have to get right on the first critical try, reasoning about a system more capable than anything we have observed, before it exists. That rules out pure trial and error and demands concepts that stay meaningful under extreme optimization pressure and self-modification. Agent foundations supplies (or tries to supply) those concepts, and this chapter gives a map of some main research directions and how they fit together.

2. Content

The exercises are in Section 3 below; worked solutions are in Solutions to Exercises at the back of the book.

→ *Fast-track route*: Supplementary Material, p. 531

2.2 Main content

Agent foundations is not one theory but several research strands (research directions), each attacking a different facet of the same problem: how to reason about, and build, *stable* agents that are *embedded* in the world they act on. This chapter is organized around five such strands (consequentialist foundations; Löb's theorem and tiling agents; logical induction; optimization and thermodynamics; descriptive agent foundations). An overview of the unifying threads supplies the spine that connects them; the shared fundamental readings and a set of discussion questions knit them into a single picture; and the exercises make a few of the results rigorous.

Read top to bottom, this section runs: the unifying threads, then the fundamental readings that give a bird's-eye view, then the five strands each with its readings, then discussion questions and the exercises.

2.2.1 Unifying threads

- **Why agent foundations.** We may need key safety properties to hold on the first critical try, for an agent far more capable than any we can study directly. Agent foundations looks for the properties capable agents share *in general*, so we can reason about them in advance.
- **Reflective stability.** A safety property only matters if it survives the agent's own self-modification: a capable agent may rewrite its code, build a more capable successor, or revise its world model. A property is *reflectively stable* if it is invariant under all of these. Working assumption: under enough optimization pressure, a property persists only if there is some reason it must. Reflective stability is the thread running through every strand.

- **Robust concepts ("true names").** Goodhart's law says a proxy that becomes a target stops measuring what we want. So a theory of agency must be built from concepts that do not break under optimization, the "true names" of optimization, goals, world models, and embeddedness.
- **Two pathways of impact (following C. Wyeth).** *Modelling*: build an abstract mathematical model of an idealised capable agent and use it to show why a given alignment proposal would fail. *Implementation*: develop a theory far enough to build and inspect an actual system, often favouring a modular architecture (a separate, inspectable world model, planner, and goal) so each part can be checked.
- **Dualistic vs embedded agents.** The standard picture (an agent with clean I/O, "larger than" and "outside" its environment, holding a full world model in its head) is dualistic. A real agent is *embedded*: part of the world, made of the same pieces, smaller than its environment, with no crisp boundary, and able to be copied, modified, or to model itself. Embeddedness is what makes self-improvement, multi-agent reasoning about copies, and self-reference unavoidable, and it is what the rest of this chapter is about.

2.2.2 Fundamental reading

- Embedded agency: read up to and including section 3.3, then section 4.1. The canonical statement of the embedded-agency problem cluster (the Alexei/Emmy framing used above).
- Why agent foundations: read entirely. Why this abstract, theory-first approach is worth pursuing.
- Reflectively consistent degree of freedom: read entirely. The precise notion behind a property an agent would not self-modify away, which is this chapter's recurring theme of reflective stability.
- General purpose search: read entirely. Why a capable mind plausibly contains a retargetable search process.

2.2.3 The five strands

The five strands are different facets of one problem, embedded agency; you may choose one to study in depth. Every summary below is self-contained at the conceptual level and ends with that strand's readings (start with the first listed, which is the most conceptual). The discussion questions below connect them.

1. Consequentialist foundations. If an agent's preferences are cyclic, an adversary can money-pump it through trades that each look acceptable but leave it strictly worse off. So any agent that reliably avoids such dominated strategies behaves *as if* it maximizes a utility function. The **complete class theorem** sharpens this: any decision rule that is not dominated (Pareto-optimal across environments) is Bayes-optimal under some prior with full support. This is *representational*, not mechanistic: it says a capable, non-self-defeating agent looks like a Bayesian expected-utility maximizer from the outside, without saying it has a utility function inside, and without telling us *which*

utility function (the hard part). *Takeaway: coherence gives a reflectively-stable target (a dominated strategy is one a rational agent would self-modify away from), but it underdetermines the agent's actual goals. Readings:* Coherent decisions imply consistent utilities (Introduction; "Why not circular preferences?"; "Probabilities and expected utilities" through "Conditional probability"; Conclusion); The measuring stick of utility; Complete class: consequentialist foundations.

2. Löb's theorem and tiling agents. A capable agent may build a successor more capable than itself. By *Vingean reflection*, it cannot verify the successor by simulating it (if it could predict the successor's exact moves, it would be that capable already), so it must reason abstractly about the successor's *design*. The natural strategy ("trust the successor because it only takes actions it has proved safe") needs the parent to trust the successor's proofs. **Löb's theorem** blocks this: if a consistent system `L` can prove "if `L` proves `C` then `C`", then `L` already proves `C`. A consistent system cannot vouch for its own proofs in the abstract, only for a strictly weaker system's. So a naive chain of self-improvements uses an ever-weaker proof system (a "telomere" of logical strength that runs out), the *finite descent problem*. The tiling-agents programme studies whether this obstacle can be overcome. *Takeaway: self-trust under self-modification is not free; it runs straight into a logical wall. Readings:* Introduction to Löb's theorem (up to and including section 3); Vingean reflection; Walkthrough of the tiling agents paper (start through "Finite Descent Problem", then "What self-modifying agents need").

3. Logical induction. Standard Bayesian reasoning assumes *logical omniscience*: the agent instantly knows all consequences of its beliefs. A bounded agent cannot (it may know a program's source yet not its output, or the axioms yet not whether a number is prime). **Logical induction** (Garrabrant et al.) handles this by picturing a market that prices logical sentences in [0,1]; the *logical-induction criterion* requires only that no efficient (polynomial-time) trader can exploit the market for unbounded profit, a computable weakening of the Dutch-book argument. That single condition yields convergence and coherence in the limit, timely learning of statistical patterns (it prices "the nth digit of pi is 7" near 1/10 without computing it), and *self-trust* (current credence equals a weighted average of expected future credences). *Takeaway: a principled model of how a bounded agent should hold probabilities over facts it has not yet computed. Readings:* An intuitive guide to Garrabrant induction; Logical induction (chapters 1 and 3; skim chapter 4).

4. Optimization and thermodynamics. A powerful agent reliably steers the world into a narrow region of outcomes, ones extremely unlikely under any random process. This is *local entropy reduction*: concentrating probability mass from a broad initial distribution onto a narrow target. Even a pure predictor has an objective reason to attend to optimizers: naming what an optimizer steers toward predicts the outcome cheaply and robustly, where modelling the initial conditions would be expensive and chaos-fragile. Steering is bounded by information: the **Touchette-Lloyd** inequality says the entropy reduction a sighted agent achieves over a blind baseline is at most the mutual information between its observations and actions ($\Delta H \leq \Delta H^{\max}_{\text{blind}} + I(X;A)$). *Algorithmic thermodynamics* (Ebtekar and Hutter) replaces ensemble entropy with

Kolmogorov complexity, giving laws for individual states and making an embedded agent's knowledge an endogenous physical quantity (algorithmic mutual information between memory and environment), which is exactly its budget for optimization. *Takeaway: optimization is physically constrained, and "knowing more" formally means "being able to optimize more." Readings:* the self-contained note Optimization and thermodynamics (p. 363) (read entirely; appendix optional) is the primary reading; its underlying sources are The ground of optimization (up to and including "Relationship to Garrabrant and Demski's Embedded Agency"), Generalized heat engine, and Algorithmic thermodynamics and three types of optimization.

5. Descriptive agent foundations. *Normative* agent foundations asks what an ideal agent should look like; *descriptive* asks what agents actually arising in the world (bacteria, neural networks, future AI) look like, and aims to read off their goals, world model, and decision structure from the outside. It works bottom-up from properties of the world (modularity, selection pressures, computational limits). **Selection theorems** aim to prove results of the form "any system selected to achieve goal G in environment E must contain structure approximately isomorphic to X", giving mechanistic rather than merely representational accounts of agency. A key example: the world's *modularity* (it decomposes into sparsely-interacting subsystems) is what makes both world-modelling (Bayesian networks propagate updates locally) and planning (general-purpose search can pursue decoupled subgoals) tractable. *Takeaway: the complementary direction to coherence: not "non-dominated agents can be described as maximizers" but "what pressures make agent-like structure actually arise." Readings:* Selection theorems: a program for understanding agents; How we picture Bayesian agents; What selection theorems do we expect/want.

2.2.4 Discussion questions

Try to connect the strands into one picture of embedded agency: how a logical inductor's trust in its future self relates to a tiling agent's trust in its successor, how the coherence account of goals relates to the thermodynamic one, how the representational view of agency (coherence) relates to the mechanistic one (selection theorems).

2.2.5 Exercises

Five exercises (full statements and worked solutions in Section 3), grouped by topic:

Logical uncertainty and self-reference.

- **Exercise 3.1: Gödel's second incompleteness theorem** (difficulty 4/5, importance 4/5). Via a self-referential program: a Gödel sentence is true, a consistent system cannot prove its own consistency, and this is exactly the Löbian obstacle to self-trust. Parts (a) true Gödel sentence, (b) no self-consistency proof, (c) the obstacle.
- **Exercise 3.2: Löb's theorem** (difficulty 3/5, importance 5/5). The three provability properties (necessitation, distribution, the Löb condition), a full proof of the

theorem, and an application: FairBot programs cooperate by Löb's theorem. Parts (a-b) properties, (c-d) the proof, (e) FairBot.

Coherence and consequentialism.

- **Exercise 3.3: The complete class theorem** (difficulty 3/5, importance 5/5). The equivalence between non-dominated strategies and Bayesian expected-utility maximization, via a geometric argument over the convex set of attainable reward vectors. Parts (a) admissibility, (b) faces and difference vectors, (c) constructing the rationalizing prior.

Descriptive agent foundations.

- **Exercise 3.4: The do-divergence theorem** (difficulty 2/5, importance 4/5). Formalizes optimization as outcome concentration and proves that how far an agent can steer outcomes (a KL divergence from the unsteered baseline) is bounded by the mutual information between its observations and actions. The single-step backbone of the Touchette-Lloyd picture.
- **Exercise 3.5: Channel additivity** (difficulty 3/5, importance 3/5). Optimal input distributions over independent channels: mutual information decomposes across channels, independence improves throughput, and an optimal policy need not coordinate across a modular environment (connecting modularity to tractable optimization).

→ *2.3 Learn more*: Supplementary Material, p. 531

3. Exercises

> *The following are exercises on agent foundations. Each problem is broken into a sequence of lemmas leading to a main theorem.* ***For each subquestion, try to prove the stated lemma before reading on.*** *If you get stuck, you may treat the lemma as given and proceed to the next part.*
>
> *Before diving into a formal derivation, try to build an intuition for* ***why*** *the statement should be true. Even if you don't complete the proof, having a clear intuitive picture of what's going on is more valuable than a mechanical derivation you don't understand. Don't worry if some of the terminology is unfamiliar — the exercises are designed to be self-contained, and it should be possible to follow the questions from context.*

Formal systems and programs. A *formal system* is a precise set of rules for deriving mathematical statements from axioms. Fix a formal system L that is powerful enough to reason about programs (for instance, it can express statements about arithmetic, and any program can be encoded as a mathematical object that L can talk about). We write $L \vdash \varphi$ to mean that the statement φ is *provable* in L, i.e. there exists a finite sequence of steps, each justified by the rules of L, that derives φ.

We say L is *consistent* if it never proves a contradiction. We write $\bot$ for a fixed contradictory statement (such as $0 = 1$), so consistency means $L \nvdash \bot$. We assume throughout that L is consistent.

Programs. By a *program* we mean a mechanical procedure that follows a fixed list of instructions. A program may *halt* (finish and produce an output) or *run forever* (keep executing without ever stopping). Since L can reason about programs, it can express the statement "program M halts", which we write as $\mathsf{Halts}(M)$.

The bridge between L and programs. Formal systems and programs are intimately connected, and the key to these exercises is switching back and forth between the two perspectives:

- **From programs to L** (concrete outputs become proofs)**.** If a program concretely produces an output (e.g. it halts after some number of steps, or it finds a string with a certain property), then L can verify this by tracing through the execution step by step. In particular: if a program actually halts, then L can prove that it halts.
- **From L to programs** (proofs can be found by search)**.** Proofs in L are finite strings that can be checked mechanically. So for any statement P, we can write a program that searches through all possible strings, checks whether each one is a valid L-proof of P, and halts if it finds one:

$$\texttt{ProofSeeker}(P) \;:=\; \text{“try every string; halt iff one is a valid } L\text{-proof of } P\text{.”}$$

This program halts if and only if P is provable in L.

Whenever you derive a fact about L (e.g. that some statement is or isn't provable), ask what that implies for the corresponding proof-search program, and vice versa.

The provability predicate $\Box P$. We write $\Box P$ for the statement, *expressed within L itself*, that "P is provable in L." This is a genuine mathematical statement that L can reason about, because it is equivalent to the claim that $\texttt{ProofSeeker}(P)$ halts, and L can talk about programs.

The crucial distinction is between $L \vdash P$ and $L \vdash \Box P$:

- $L \vdash P$ means that P is provable: there *exists* a concrete proof of P in L. This is a fact about L that we observe from the outside.
- $L \vdash \Box P$ means that L has proved a statement *about itself*: namely, that a proof of P exists (equivalently, that $\texttt{ProofSeeker}(P)$ halts). But this is a claim L is making about the $\texttt{ProofSeeker}(P)$ program, not a direct certificate for P.

Intuition. To see why $L \vdash P$ and $L \vdash \Box P$ are conceptually distinct, consider the two different ways L might prove that $\texttt{ProofSeeker}(P)$ halts. The first is to actually trace through its execution: if it halts after, say, a million steps, L can verify this step by step, and the proof that $\texttt{ProofSeeker}(P)$ found along the way is itself a direct L-proof of P. In this case, $L \vdash \Box P$ and $L \vdash P$ seems to come hand in hand. But there is a second way: L might reason *abstractly* about the program's behaviour without ever simulating it. (This is analogous to how you might argue that a sorting algorithm must eventually

finish without tracing through every swap it makes.) Such a proof establishes that $\texttt{ProofSeeker}(P)$ halts — and therefore that *some* proof of P exists — but the proof itself is about the *program*, not about P. It need not contain, or even hint at, what the actual proof of P looks like. This is the gap that L cannot close in general: knowing abstractly that a proof is "out there" is not the same as having the proof in hand.

What "provable in L" means (and what it does not). It is important to distinguish between being *convinced* that something is true and *proving it in* L. When we say "L can carry out this argument" or "L proves P," we do not mean that a reasonable person reading the argument would find it convincing. We mean something much more specific: that there exists a sequence of formulas, each of which is either an axiom of L or follows from earlier formulas by one of L's explicitly listed inference rules, and whose last line is P. The formal system L is a *machine*: it has no understanding, no intuition, and no ability to say "well, this obviously follows." Every single step must be justified by a specific rule.

From the outside, we can see that if $\texttt{ProofSeeker}(P)$ halts then a proof of P exists, so P is provable. But can L carry out this reasoning internally, always concluding P from $\Box P$? Löb's theorem (Exercise 3.2) shows that the answer is no: any consistent L that derives P from $\Box P$ for all P is in fact inconsistent.

Exercise 3.1 (Gödel's second incompleteness theorem). Key fact. If a program M actually halts (say, after 17 steps), then L can verify this by checking the execution step by step, so $L \vdash \mathsf{Halts}(M)$. However, if M runs forever, L cannot necessarily prove $\neg\mathsf{Halts}(M)$. (Informally: it is easy to certify that something stops, because you just exhibit the stopping point; but certifying that something runs *forever* is much harder, because you cannot check infinitely many steps.)

In fact, no consistent formal system can correctly settle the question "does M halt?" for *every* program M. To see why: if L could do this, we could write a program that, given any M, searches for an L-proof of either $\mathsf{Halts}(M)$ or $\neg\mathsf{Halts}(M)$. Since L is assumed to settle every case, this search would always find a proof and halt, giving us a mechanical procedure that decides whether any program halts. But such a procedure cannot exist (this is the *undecidability of the halting problem*, a fundamental result in computer science that we take as given here).

A self-referencing program. It is possible to write programs that refer to their own source code. (As a simple example, a program can carry its own source code as a string and then operate on it.) Using this idea, define the following program:

$$Z(A) \;:=\; \text{"search for an } L\text{-proof of } \neg\mathsf{Halts}(A(A))\text{; halt if one is found."}$$

Here $A(A)$ means "run program A with its own source code as input." So $Z(A)$ searches for a proof that the program A-run-on-itself runs forever.

Now consider feeding Z its own source code. The program $Z(Z)$ searches for an L-proof that $Z(Z)$ runs forever. Define the statement:

$$G \;:=\; \neg\mathsf{Halts}(Z(Z)).$$

In words: G says "$Z(Z)$ runs forever." Notice the self-referential structure: $Z(Z)$ halts if and only if it finds an L-proof of G (i.e. a proof that $Z(Z)$ runs forever).

(a) Show that G is true, assuming L is consistent.

HINT, p. 695

(b) Show that if L can prove its own consistency (i.e. $L \vdash \neg\Box\bot$), then L is in fact inconsistent.

HINT, p. 695

(c) Suppose L can vouch for all of its own proofs, meaning $L \vdash \Box P \to P$ for every statement P. (Read this as: "whenever L can prove P, then P is actually true," and L itself asserts this.) Show that L is inconsistent, in two steps:

(i) First, show that L proves it never proves anything false. That is: for any P with $L \vdash \neg P$, show that $L \vdash \neg\Box P$.

HINT, p. 695

(ii) Apply (i) with $P = \bot$, using the fact that $\neg\bot$ ("a contradiction is false") is a tautology. Conclude that $L \vdash \neg\Box\bot$, and use Exercise 3.1(b) to finish.

Remark (Self-trust, tiling agents, and the Löbian obstacle). Any sufficiently advanced AI may eventually be able to modify its own code or build a successor system more capable than itself. But this raises a subtle problem. If the successor is genuinely smarter, the original agent *cannot* predict exactly what it will do — just as the programmers of a chess engine can reason that their program is "trying to win" without knowing its exact moves. So the original agent cannot verify its successor's safety by simulating it move by move. Instead, it must reason *abstractly* about the successor's design: "whatever my successor does, it will only take actions that it has proved lead to good outcomes."

This reasoning strategy is called *tiling*: the parent agent A_1 builds a child agent A_0, and wants to conclude not merely that A_0 will only take actions that A_0 has *proved to be safe*, but that those actions *actually are* safe. After all, A_1 can verify from A_0's source code that A_0 says "only take action x if I can prove that x leads to good outcomes." But this only tells A_1 that A_0 acts on what A_0's proof system certifies — it does not yet tell A_1 that what A_0's proof system certifies is actually *true*. To close this gap, A_1 needs to be able to prove, within its own reasoning, that A_0's proof system is *sound*: whenever A_0's system proves P, then P is actually true. When both agents use the same formal system L, this amounts to $L \vdash \Box P \to P$ for all P.

Exercise 3.1(c) shows this is impossible: any consistent system that asserts $\Box P \to P$ for all P is already inconsistent. A consistent system cannot vouch for its own proofs in the abstract — it can only trust a proof once it has *witnessed* it directly. This is the **Löbian obstacle**: the barrier created by Löb's theorem to self-trusting formal reasoning.

One might hope to work around this by having the parent use a *stronger* proof system than the child: a stronger system *can* trust a weaker one's proofs. But this means each successive agent in a chain of self-improvements must use a strictly weaker proof system than its predecessor, resulting in a "telomere" of logical strength that shortens with each generation. Eventually the chain runs out of trust. The *tiling agents* research programme studies how (and whether) this obstacle can be overcome, seeking agent architectures that can undergo indefinite self-improvement without their reasoning guarantees degrading at each step.

SOLUTION, p. 695

Exercise 3.2 (Löb's theorem). Löb's theorem says: if $L \vdash \Box C \to C$ (i.e. L can prove "if C is provable then C is true"), then $L \vdash C$ (i.e. C is already provable in L). In other words, the only statements for which L can close the gap between "provably provable" and "provable" are the ones that were already provable to begin with.

The proof uses three properties of the provability predicate $\Box$. We state them here together with informal explanations of what they say about the proof-search program `ProofSeeker`.

(N)	***Necessitation.***	**If** $L \vdash \varphi$ **then** $L \vdash \Box\varphi$**.**
(K)	*Distribution.*	$L \vdash \Box(\varphi \to \psi) \to (\Box\varphi \to \Box\psi)$.
(4)	*Löb condition.*	$L \vdash \Box\varphi \to \Box\Box\varphi$.

(a) Understanding Necessitation. Necessitation says: if φ is provable in L, then L can prove that φ is provable. Explain why this is true from the perspective of `ProofSeeker`.

HINT, p. 696

(b) Understanding Distribution. It is helpful to think of a proof of $\varphi \to \psi$ by analogy with a *function*: given any proof of φ as input, one can mechanically produce a proof of ψ as output (by writing down the proof of φ, attaching the proof of $\varphi \to \psi$, and applying the logical rule that from φ and $\varphi \to \psi$ one may conclude ψ).

With this analogy, $\Box(\varphi \to \psi)$ says that L has proved such a "proof-transforming function" exists. Distribution then says: if L knows that a function from φ-proofs to ψ-proofs exists, and L knows that a φ-proof exists, then L can conclude that a ψ-proof exists.

Explain why Distribution is true from the perspective of `ProofSeeker`.

HINT, p. 696

(c) We now prove Löb's theorem. The key ingredient is a self-referential sentence constructed using the same idea as in Exercise 3.1 (a sentence that talks about its own provability). Specifically, there exists a sentence λ such that

$$L \vdash \lambda \;\leftrightarrow\; (\Box\lambda \to C).$$

In words: λ says "if I am provable, then C is true." (The existence of such a sentence is guaranteed by the same self-referential construction used to build G in Exercise 3.1; we take it as given here.)

Show that $L \vdash \Box\lambda \to \Box C$.

HINT, p. 696

(d) Now assume $L \vdash \Box C \to C$. Using Exercise 3.2(c) and the Löb sentence, derive $L \vdash C$.

HINT, p. 696

Remark (The Santa Claus paradox and what Löb adds beyond Gödel). The Löbian sentence $\lambda \leftrightarrow (\Box\lambda \to C)$ is a formal analogue of the *Santa Claus sentence* (also known as Curry's paradox). Consider the sentence S: "If this sentence is true, then Santa Claus exists." Let us try to determine whether S is true or false.

Well, S is an "if ... then ..." statement, so to check whether it is true, let us assume the "if" part and see whether the "then" part follows. So assume S is true. Since S says "if S is true then Santa Claus exists," and we are assuming S is true, it follows that Santa Claus exists. We have therefore shown: if S is true, then Santa Claus exists. But that is exactly what S says! So S is true. And since S is true and S implies Santa Claus exists, Santa Claus exists. Since nothing about this argument was specific to Santa Claus, the same reasoning "proves" any statement whatsoever.

The reason L does not fall prey to this paradox is that L cannot form a sentence that refers to its own *truth*. (A fundamental result called Tarski's theorem shows that no sufficiently powerful formal system can define a truth predicate for itself.) What L *can* do is refer to its own *provability*: the predicate $\Box\varphi$ is a legitimate statement within L. The Löbian sentence λ therefore substitutes "provable" for "true," asserting "if I am *provable*, then C." The proof of Löb's theorem shows that this substitution is enough to force $L \vdash C$ — but only when $\Box C \to C$ is already assumed, not unconditionally.

This mirrors a pattern: replacing "true" with "provable" transforms semantic paradoxes into precise theorems. The liar's paradox ("this sentence is false") becomes Gödel's sentence ("this sentence is not provable"), yielding the incompleteness theorems. The Santa Claus paradox ("if this sentence is true, then C") becomes the Löbian sentence ("if this sentence is provable, then C"), yielding Löb's theorem.

What Löb adds beyond Gödel. Gödel's second incompleteness theorem (Exercise 3.1) says that L cannot prove its own consistency. This is already a serious obstacle, but one might hope that consistency is a special case — perhaps L can still trust its proofs in less sweeping ways. Löb's theorem crushes this hope completely. It says that for *any* statement C, if L can prove "my provability of C implies C is true" (i.e. $L \vdash \Box C \to C$), then C was already provable. There are *no* statements, not for which L can assert "well, if I *could* prove this, it would be true" without already being able to prove them. L does not trust its own proofs until it has witnessed them directly.

This is what makes Löb's theorem, rather than Gödel's, the fundamental obstacle for tiling agents. Recall the setup from Exercise 3.1: a parent agent A_1 builds a child A_0 that only takes actions it can prove to be safe. For A_1 to trust A_0, it needs to know that A_0's proofs track reality — that is, A_1 needs $\Box P \to P$ for the statements P that A_0 might act on. Gödel tells us A_1 cannot prove A_0's system is *consistent*. But Löb tells us something far stronger: A_1 cannot even trust A_0's system on a *case-by-case* basis. For any individual statement C, the only way L can derive "if my proof system proves C, then C is really true" is if C was already provable — in which case the trust was never needed in the first place.

(e) In a one-shot Prisoner's Dilemma, two players each choose to either *cooperate* (C) or *defect* (D). In this variant, instead of choosing directly, each player submits a *program* that receives the opponent's source code as input and outputs C or D. Consider the following agent, *FairBot*:

algorithm `FairBot`(opponent)**:**
Search for an L-proof that `opponent`(`FairBot`) $= C$.
if proof found **then return** C
else return D

In words: FairBot cooperates with an opponent if and only if it can find an L-proof that the opponent cooperates with FairBot. Note that FairBot is *unexploitable*: if L is sound (i.e. L only proves true statements), then FairBot never cooperates with an opponent that defects against it.

The interesting question is what happens when FairBot plays against itself. At first glance, both mutual cooperation and mutual defection seem like stable outcomes.

Consider two copies $\texttt{FairBot}_1$ and $\texttt{FairBot}_2$ (identical programs with different implementations). Let A be the statement "$\texttt{FairBot}_1(\texttt{FairBot}_2) = C$" and B be the statement "$\texttt{FairBot}_2(\texttt{FairBot}_1) = C$." Prove that $L \vdash A \wedge B$, i.e. that the two FairBots mutually cooperate. Use Löb's theorem.

HINT, p. 696

SOLUTION, p. 697

Exercise 3.3 (The Complete Class Theorem). The setup: decision-making under uncertainty. Imagine you must choose an action, but you don't know which *environment* you are in. There are finitely many actions $\mathcal{A} = \{a_1, \ldots, a_k\}$ and finitely many possible environments $\Omega = \{\omega_1, \ldots, \omega_n\}$. If you take action a and the true environment turns out to be ω, you receive a reward $u(a, \omega) \in \mathbb{R}$.

Example 3.1. You are deciding whether to carry an umbrella (a_1) or not (a_2). The environment is either "rainy" (ω_1) or "sunny" (ω_2). The rewards might be:

	ω_1 (rain)	ω_2 (sun)
a_1 (umbrella)	8	5
a_2 (no umbrella)	2	10

Decision rules. A *decision rule* $\delta = (\lambda_1, \dots, \lambda_k)$ is a (possibly randomized) strategy: you play action a_j with probability λ_j, where $\lambda_j \geq 0$ and $\sum_{j=1}^{k} \lambda_j = 1$. A *pure* decision rule puts all its weight on a single action (e.g. "always carry the umbrella"). A *mixed* rule randomizes (e.g. "carry the umbrella with probability 0.7").

The *reward of δ in environment ω* is the average reward under the randomization:

$$u(\delta, \omega) \;:=\; \sum_{j=1}^{k} \lambda_j \, u(a_j, \omega).$$

Risk vectors and the risk set. To compare decision rules across all environments simultaneously, we package the rewards into a single vector. The *risk vector* of a decision rule δ is

$$r(\delta) \;=\; \big(u(\delta, \omega_1),\; \dots,\; u(\delta, \omega_n)\big) \;\in\; \mathbb{R}^n.$$

Each coordinate records how well δ performs in one environment. In the umbrella example, $r(a_1) = (8, 5)$ and $r(a_2) = (2, 10)$.

The *risk set* $\mathcal{R}$ is the set of all risk vectors achievable by some decision rule:

$$\mathcal{R} \;=\; \Big\{ \sum_{j=1}^{k} \lambda_j \, r(a_j) \;:\; \lambda_j \geq 0,\; \sum_{j=1}^{k} \lambda_j = 1 \Big\}.$$

Geometrically, $\mathcal{R}$ is the *convex hull* of the pure-action risk vectors $\{r(a_1), \dots, r(a_k)\}$: the set of all weighted averages of these points.

Admissibility (not being dominated). A decision rule δ is *admissible* if there is no other rule δ' that does at least as well as δ in *every* environment and strictly better in at least one. If such a δ' exists, we say δ' *dominates* δ, and any rational agent should prefer δ' — after all, switching from δ to δ' never hurts and sometimes helps, regardless of which environment is the true one.

The *Pareto frontier* $\mathcal{F} \subseteq \mathcal{R}$ is the set of risk vectors of all admissible decision rules. A pure action a_j is *Pareto-optimal* if $r(a_j) \in \mathcal{F}$.

Bayesian expected utility. A different approach to decision-making is to assign a *prior* $\pi = (\pi_1, \dots, \pi_n)$ representing your beliefs about how likely each environment is, where $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. (For example, $\pi = (0.3, 0.7)$ means you believe there is a 30% chance of rain.) The *expected utility* of δ under π is the weighted average reward:

$$\mathrm{EU}(\delta, \pi) \;:=\; \sum_{i=1}^{n} \pi_i \, u(\delta, \omega_i) \;=\; \pi \cdot r(\delta).$$

A decision rule δ is *Bayes-optimal* under π if it achieves the highest expected utility among all decision rules: $\mathrm{EU}(\delta, \pi) \geq \mathrm{EU}(\delta', \pi)$ for all δ'.

The theorem. These two approaches to rational decision-making — admissibility ("never use a dominated strategy") and Bayesian expected utility maximization ("assign beliefs and maximize average reward") — turn out to characterize exactly the same set of decision rules:

> **Theorem 3.2 (Complete Class).** Every admissible decision rule is Bayes-optimal under some prior π with $\pi_i > 0$ for all i. Conversely, every Bayes-optimal rule under such a prior is admissible.

In other words: the decision rules that survive the "no domination" criterion are precisely those that arise from maximizing expected utility under some set of beliefs that doesn't rule out any environment entirely. This is significant because admissibility is an extremely weak rationality requirement — it says only that you shouldn't use a strategy when a strictly better one is available — yet it already forces expected utility maximization.

(a) Show the following two facts: **(i)** Any admissible decision rule $\delta = \sum_j \lambda_j \, a_j$ places zero weight on dominated pure actions: $\lambda_j = 0$ whenever a_j is not Pareto-optimal. (In other words, the Pareto frontier $\mathcal{F}$ is contained in the convex hull of the Pareto-optimal pure actions alone.)

HINT, p. 698

(ii) Every Bayes-optimal rule under a prior π with $\pi_i > 0$ for all i is admissible.

HINT, p. 698

(b) A *face* F of the Pareto frontier is a maximal convex subset of $\mathcal{F}$ of the form

$$F \;=\; \Big\{ \sum_{l=1}^{p} \mu_l \, r(a_{i_l}) \;:\; \mu_l \geq 0,\; \sum_{l=1}^{p} \mu_l = 1 \Big\}$$

for some subset of Pareto-optimal pure actions $\{a_{i_1}, \ldots, a_{i_p}\}$. Define the *difference vectors* $v_l := r(a_{i_l}) - r(a_{i_1})$ for $l = 2, \ldots, p$, and let $H = \mathrm{span}\{v_2, \ldots, v_p\}$. Show that H consists precisely of the directions along which one can move within F: that is, if $r(\delta) \in F$, then $r(\delta) + h \in F$ for some h only if $h \in H$.

(c) Let π be a vector perpendicular to the subspace H from Exercise 3.3(b), normalized so that $\pi_i > 0$ for all i and $\sum_i \pi_i = 1$. You may assume that the entire risk set $\mathcal{R}$ lies on one side of the hyperplane defined by H (i.e. no point in $\mathcal{R}$ scores strictly higher under π than the points on F). Show that: **(i)** $\mathrm{EU}(\delta, \pi)$ takes the same value for all δ with $r(\delta) \in F$.

(ii) Every rule on the face F is Bayes-optimal under π.

Conclude the Complete Class Theorem: every admissible rule lies on some face of $\mathcal{F}$, and the prior π constructed from that face makes it Bayes-optimal.

SOLUTION, p. 698

Exercise 3.4 (The Do-Divergence Theorem). Motivation: optimization as steering. A useful way to think about what it means for an agent to be *optimizing* is that it reliably steers the world into a narrow set of outcomes — outcomes that would be extremely unlikely to arise out of any random process . A thermostat keeps a room at 20 ∘ C despite varying weather; a chess player steers toward checkmate despite the opponent's moves. In each case, the actual outcome is concentrated in a small region of possibility space, whereas without the agent's intervention, outcomes would be spread broadly.

This exercise makes that intuition precise in an information-theoretic setting. We will show that an agent's ability to concentrate outcomes ("steer") is bounded by the amount of information the agent extracts from its observations.

Background: KL divergence. Given two probability distributions p and q over the same set of outcomes, the *Kullback–Leibler (KL) divergence* from q to p is

$$D_{\mathrm{KL}}(p \,\|\, q) \;:=\; \sum_{x} p(x) \log \frac{p(x)}{q(x)}.$$

This quantity is always ≥ 0, and equals 0 only when $p = q$. It measures how "different" p is from q, with a particular asymmetry: $D_{\mathrm{KL}}(p\|q)$ is large when p places significant probability on outcomes where q assigns very little. In other words, it is large precisely when p concentrates on outcomes that would be *surprising* under q. This can be seen as a signature of optimization: the agent's policy makes certain outcomes likely that would be very unlikely under a baseline policy.

Background: mutual information. The *mutual information* between two random variables A and O is

$$\mathrm{MI}(A;\, O) \;:=\; D_{\mathrm{KL}}\big(P[A,O] \,\big\|\, P[A]\,P[O]\big) \;=\; \sum_{a,o} P[a,o] \log \frac{P[a,o]}{P[a]\,P[o]}.$$

This measures how much knowing O tells you about A (and vice versa). It is zero when A and O are independent, and large when they are tightly coupled.

Setup. Consider an agent (the "demon") that observes some information O about the world and then takes an action A based on what it observed. The action and observation together produce an outcome X. The joint distribution is

$$P[X, A, O] \;=\; P[X \mid A, O]\; P[A \mid O]\; P[O].$$

The factor $P[A \mid O]$ encodes the demon's *policy*: how it chooses actions as a function of its observations.

Now consider a *blind baseline*: the demon still acts, but ignores its observations, choosing actions independently of O. We write the blind baseline distribution as

$$P[X, A, O \mid do(A)] \;=\; P[X \mid A, O]\; P[A]\; P[O].$$

The notation $do(A)$ means we have "intervened" on the action, replacing the demon's observation-dependent policy $P[A \mid O]$ with the marginal $P[A]$ (the overall frequency of each action, ignoring which observations prompted them). The mechanism $P[X \mid A, O]$ by which actions and observations produce outcomes is unchanged — only the demon's strategy has been lobotomized.

The theorem. Prove the Do-Divergence Theorem:

$$D_{\mathrm{KL}}\big(P[X] \,\big\|\, P[X \mid do(A)]\big) \;\le\; \mathrm{MI}(A;\, O).$$

The left side measures how much the sighted demon's outcome distribution differs from the blind baseline's. In the language of steering: it measures how much the demon has concentrated outcomes into regions that would be unlikely without observation-dependent action. The right side is the mutual information between actions and observations — how much the demon's actions depend on what it sees. The theorem says that **the degree of steering is bounded by the information the demon uses**.

Useful fact:

- **Monotonicity of KL divergence.** For any two joint distributions $p(x, y)$ and $q(x, y)$, marginalizing out y can only decrease KL divergence: $D_{\mathrm{KL}}(p(x) \,\|\, q(x)) \le D_{\mathrm{KL}}(p(x, y) \,\|\, q(x, y))$. (Intuitively: forgetting information can only make two distributions look more similar, never less.)

HINT, p. 699

Remark (Maxwell's demon and the thermodynamics of optimization). Maxwell's demon is a thought experiment in which a tiny intelligent being controls a door between two halves of a box of gas. By observing each molecule's position and selectively opening the door, the demon can sort all molecules to one side, creating a highly ordered (low-entropy) state from an initially disordered one. In our notation: the outcome X is the final configuration of molecules, the observations O are the demon's measurements of molecular positions, and the actions A are its door openings.

Under the blind baseline (opening the door at random), molecules are roughly equally likely to be on either side, so $P[X \mid do(A)]$ is spread broadly. If the demon perfectly sorts all n molecules to the right, $P[X]$ is concentrated on a single configuration, and the KL divergence between these distributions is $n \log 2$ (i.e. n bits). The theorem therefore says that perfectly sorting n molecules requires $\mathrm{MI}(A; O) \ge n$ bits: the demon must gather at least n bits of information about the molecules to reduce the gas's entropy by n bits. This illustrates the idea that any agent that steers a system into a narrow, unlikely region of outcome space (low entropy) must pay for this steering with mutual information.

SOLUTION, p. 699

Exercise 3.5 (Channel Additivity). Consider two independent channels $X_1 \to Y_1$ and $X_2 \to Y_2$ with a fixed joint channel

$$P[Y \mid X] \;=\; P[Y_1 \mid X_1]\, P[Y_2 \mid X_2].$$

That is, output Y_1 depends only on input X_1, and Y_2 depends only on X_2; the two channels do not interact. We are free to choose the input distribution $P[X]$ (which may correlate X_1 and X_2), and the joint distribution over everything is then $P[Y, X] = P[Y \mid X]\, P[X]$. The goal is to maximize the mutual information $\mathrm{MI}(X; Y)$, called the *information throughput* of the channel.

(a) Show that for any input distribution $P[X]$,

$$\mathrm{MI}(X; Y) \;=\; \mathrm{MI}(X_1; Y_1) \;+\; \mathrm{MI}(X_2; Y_2) \;-\; \mathrm{MI}(Y_1; Y_2).$$

HINT, p. 700

(b) Given any input distribution $P[X_1, X_2]$, define

$$Q[X_1, X_2] \;:=\; P[X_1]\, P[X_2],$$

i.e. the product of the marginals. Show that $\mathrm{MI}_Q(X; Y) \geq \mathrm{MI}_P(X; Y)$.

HINT, p. 700

(c) Use Exercise 3.5(b) to conclude that there always exists a throughput-maximizing input distribution under which $X_1 \perp\!\!\!\perp X_2$.

Remark. This result has a natural interpretation in terms of optimization and agency. Think of X as the actions of an agent and Y as the outcomes it cares about. The channel $P[Y \mid X]$ — how actions influence outcomes — is fixed by the environment and outside the agent's control; the agent only gets to choose its policy $P[X]$. The factorization condition on $P[Y \mid X]$ says that the environment is *modular*: the two groups of outcomes Y_1 and Y_2 are each influenced only by their respective actions X_1 and X_2. Our theorem says that if the environment is modular in this sense, then the agent can always find an optimal policy that is modular in a corresponding sense — specifically, the two groups of actions need not be coordinated at all and can be chosen independently.

SOLUTION, p. 700

CHAPTER D.4.2

Optimization and Thermodynamics

Daniel C, Satya Benson (Williams College)

An optimizer funnels a broad range of starting conditions into a narrow target; the entropy it removes measures how much. Physics charges for every bit, so an embedded agent pays for optimization with what it knows.

Source: iliad-intensive.org/agency/optimization-thermodynamics · CC BY 4.0

Abstract. A useful characterization of powerful agents is that they reliably steer the world into a narrow region of outcome space, a region that would be extremely unlikely to arise under any random process. These notes develop this picture from first principles, pursuing three successive aims: to make the notion of "steering into a narrow region" mathematically precise, to establish the thermodynamic constraints that physics imposes on any process realizing it, and to resolve a conceptual gap that the classical formalism leaves open. We first characterize an optimizer behaviorally, as the *cause* of a convergent attractor: a physical entity whose presence drives the system it acts upon, from a broad range of initial conditions and despite perturbations, into a narrow target set, so that conditioning on the optimizer collapses an observer's uncertainty about the final state that system reaches. We argue that even an observer concerned solely with prediction possesses an objective, information-theoretic reason to single out such optimizers, establishing optimization as an observer-independent feature of the world rather than a stance projected onto it; translated into information theory, optimization is local entropy reduction in the system being acted upon. We then provide thermodynamic foundations for the analysis of such processes: the reversibility of microscopic dynamics, together with the second law of thermodynamics, forbids global entropy reduction, so an embedded optimizer must compensate for every local reduction within a consistent global accounting. We derive the second law from reversibility in a minimal Markov-chain setting, classify the three ways in which an agent can reduce the entropy of a subsystem while respecting information conservation, and quantify the third of these channels through the Touchette–Lloyd theorem (entropy reduction beyond a "blind" baseline must be paid for in mutual information between the agent and the environment), developed in an appendix. A self-contained toy thermodynamics built out of biased coins (Wentworth's generalized heat engine), in which every step of the requisite "bookkeeping" is elementary, is developed in a separate appendix. Finally, we confront a conceptual gap: the standard Gibbs–Shannon entropy

is defined relative to a subjective probability distribution that the formalism treats as exogenous, making "capacity to optimize" observer-dependent and leaving the classical tools inapplicable to the far-from-equilibrium, information-bearing states of which agents are composed. Algorithmic thermodynamics, due to Ebtekar and Hutter, replaces the subjective distribution with Kolmogorov complexity, yielding a second law that applies to individual physical states. Under this reframing, an embedded agent's probabilistic knowledge becomes an endogenous, physically encoded quantity—the algorithmic mutual information between the agent's memory and the environment—which is precisely the resource the agent expends when it optimizes. We thus establish that agents with greater knowledge of the world possess correspondingly greater capacity to act upon it, with the exchange rate between the two set by thermodynamics.

In this chapter

1. Introduction: the role of thermodynamics in agent foundations 365
2. Theoretical foundations: probability, entropy, and information 367
  2.1 Random variables and notation 367
  2.2 Entropy 367
  2.3 The coding interpretation 368
  2.4 Joint and conditional entropy, and mutual information 369
  2.5 Divergence and two fundamental inequalities 370
3. Characterizing optimization 371
  3.1 Two notions of optimization and their relationship 371
  3.2 Optimizers and convergent attractors 372
  3.3 Optimization as entropy reduction 373
  3.4 The predictive value of optimizers 374
  3.5 The observer-independence of optimization 375
4. Reversibility and the second law 376
  4.1 The reversibility of microscopic physics 376
  4.2 The incompatibility of global funneling with reversibility 377
  4.3 Coarse-graining and the emergence of probability 377
  4.4 The second law of thermodynamics 379
  4.5 The scope and limitations of the second law 380
5. Three types of optimization under information conservation 381
  5.1 The bookkeeping problem 381
  5.2 Type 1: transfer of entropy into the environment as waste heat 381
  5.3 Type 2: absorption of entropy into the agent's memory through measurement 381
  5.4 Type 3: expenditure of pre-existing mutual information 382
  5.5 A reinterpretation of Maxwell's demon, and a synthesis of the three channels 383
6. Limitations of subjective entropy 384
  6.1 Entropy as a measure of optimization capacity 384
  6.2 The exogenous status of the reference distribution 385
  6.3 Three failure modes of the ensemble formalism 386
  6.4 The demon's capacity as the central case for agent foundations 387

7. Algorithmic thermodynamics 387
7.1 Kolmogorov complexity 388
7.2 Justification of K as an entropy measure 389
7.3 The algorithmic second law 391
7.4 An exact analysis of Maxwell's demon 392
8. Knowledge as a physical resource: optimization for embedded agents 395
8.1 From exogenous to endogenous knowledge 395
8.2 Refinements of the three channels via universal computation 397
8.3 Dissolution of the apparent subjectivity 398
8.4 Thermodynamic implications for optimizing systems 398
9. Summary and conclusions 399
A. A thermodynamics of biased coins: the generalized heat engine 532
A.1 The designer's viewpoint 532
A.2 Specification of the model: coins, transformations, and conservation laws 533
A.3 Work extraction as a compression problem 534
A.4 The impossibility of work extraction from a single heat bath 534
A.5 Work extraction from two heat baths at different temperatures 535
A.6 Lessons from the toy world 536
B. The informational cost of steering: the Touchette–Lloyd theorem 537
B.1 From optimization to modeling: the motivating question 537
B.2 The formal setup: environments, actions, and policies 537
B.3 Blind and sighted policies 538
B.4 A worked example: the guessing game under blind play 539
B.5 The guessing game under sighted play 540
B.6 Mutual information as a measure of sightedness 540
B.7 Statement of the theorem 541
B.8 Limitations of the theorem 541
Sources and further reading 543

1. Introduction: the role of thermodynamics in agent foundations

Agent foundations may be understood as the search for *robust concepts* (sometimes called "true names") for notions such as optimization, goals, world models, and embeddedness, where a robust concept is one that retains its meaning under extreme optimization pressure and for agents far more capable than any yet constructed. These notes pursue the true name of *optimization* along the descriptive route: rather than beginning inside an idealized rational agent, equipped with beliefs, preferences, and an expected-utility criterion, we begin from the physical world itself. This raises fundamental questions: What kind of process can reliably steer the world into a narrow target? And what thermodynamic cost does physics impose on such a process?

Thermodynamics supplies the answer to the second of these questions, for two reasons that we state at the outset and that the remainder of these notes develops systematically.

Optimization as Local Entropy Reduction. An optimizer concentrates probability mass: out of the many configurations the world could occupy, it funnels a broad set of possibilities down to a few. Concentrating probability mass, however, is precisely the reduction of uncertainty, and the reduction of uncertainty is the reduction of entropy (Section 2 makes this correspondence precise). This identification is not an analogy: it brings the full machinery of entropy (the second law, fluctuation theorems, the cost of erasing a bit) to bear directly on optimization, and the modern form of that machinery, *stochastic thermodynamics*, holds arbitrarily far from equilibrium. Consequently, physics constrains the shape of any optimizer that could actually be built.

The Physical Currency of Knowledge for Embedded Agents. We distinguish two idealized pictures of agency. A *dualistic* agent sits outside the world it acts upon, in the manner of a player at a console: its inputs and outputs are sharply delimited, and "what it knows" is a free parameter, a distribution supplied exogenously by the analyst. An *embedded* agent, in contrast, is part of the world it acts upon, so its knowledge is not free: it must be stored in some physical substrate, in a brain or a memory register composed of the same atoms as everything else. One of the principal conclusions of these notes is that once entropy is defined correctly (algorithmically, as a property of an individual state rather than of a subjective ensemble), an agent's knowledge of its environment becomes an objective physical quantity—the mutual information between its memory and the environment—and that this quantity is exactly the budget available to be spent on optimization.

We close this introduction with a remark on the nature of the contribution and its intended audience. For readers with a background in physics, the value of these notes lies primarily in translation rather than in new theorems: recasting familiar thermodynamic facts in information-theoretic language ties them to agent-foundations notions such as world models, optimization power, and the limits on embedded agents. For readers without such a background, the notes are self-contained: every thermodynamic idea is defined here in information-theoretic terms, no statistical mechanics is assumed, and the only prerequisite is familiarity with elementary discrete probability.

Structure of the Notes. Section 2 develops the probabilistic and information-theoretic apparatus on which all later sections depend, introducing entropy, mutual information, and divergence together with the coding interpretations that give these quantities operational meaning. Section 3 then addresses the characterization of optimization: we characterize an optimizer behaviorally, as a physical entity that drives the system it acts upon, from a broad range of initial conditions and robustly to perturbation, into a narrow target (a convergent attractor); we translate this characterization into entropy reduction; and we argue that any observer concerned with prediction possesses an objective, information-theoretic reason to attend to optimizers, independently of who is watching. Section 4 confronts this picture with the reversibility of microscopic physics and derives the second law of thermodynamics, with full proof, in a minimal coarse-grained setting. Building on this foundation, Section 5 classifies the three ways in which an embedded agent can reduce the entropy of a subsystem while respecting

global information conservation: exporting it into the environment, absorbing it into memory through measurement, or expending pre-existing mutual information with the subsystem. Section 6 exposes a conceptual problem at the heart of the standard formalism, namely that entropy, as conventionally defined, depends on a subjective probability distribution supplied from outside the physics. Motivated by this limitation, Section 7 develops the resolution, algorithmic thermodynamics: Kolmogorov complexity as entropy, the algorithmic second law, and an exact analysis of Maxwell's demon, with emphasis throughout on how an objective notion of entropy dissolves the subjectivity problem. Section 8 synthesizes these results into an account of embedded agency in which knowledge figures as an endogenous physical resource, and Section 9 collects the principal conclusions. Appendix A develops, in parallel to the main line of argument, a self-contained toy thermodynamics built out of biased coins (Wentworth's generalized heat engine), in which the analogues of heat, work, energy conservation, and the impossibility of perpetual motion can all be verified by hand; the main text refers to it wherever a concrete blind-policy example is instructive. Finally, Appendix B develops the Touchette–Lloyd theorem, which makes the third channel quantitative: the entropy reduction an agent can achieve beyond a blind baseline is bounded by the mutual information between its action and the environment, with fully worked examples.

2. Theoretical foundations: probability, entropy, and information

This section develops the probabilistic and information-theoretic apparatus on which the remainder of these notes relies. Readers already acquainted with Shannon entropy and mutual information may treat this section as reference material; however, we draw attention to the two components on which the later development relies most heavily: the coding interpretation of Section 2.3, which recurs throughout these notes, and the log-sum inequality (Lemma 2.9), which underpins our proof of the second law.

2.1 Random variables and notation

Throughout, capital letters X, Y, Z denote random variables taking values in countable (finite or countably infinite) sets, and lowercase letters x, y, z denote particular values. We write $\mu(x)$ for the probability that $X = x$ when the distribution is called μ, and we refer interchangeably to "the distribution of X" or "the ensemble" when speaking of μ. A joint random variable (X, Y) has a joint distribution, and the conditional probability of y given x is written $P(y \mid x)$. All logarithms in these notes are base 2, so that information is measured in bits; the natural-log versions of every formula differ only by a factor of $\ln 2$.

2.2 Entropy

With this notation in place, we begin by introducing the central quantity of information theory, which measures the uncertainty associated with a random variable.

Definition 2.1 (Shannon Entropy). The *Shannon entropy* of a random variable X with distribution μ is

$$H(X) \;:=\; \sum_{x} \mu(x) \log \frac{1}{\mu(x)},$$

with the convention that terms with $\mu(x) = 0$ contribute zero. We also write $H(\mu)$ for the same quantity when we wish to emphasize the distribution rather than the variable.

Entropy quantifies uncertainty, namely the amount that an observer does not yet know about the value of X before observing it. We present several examples to calibrate the scale.

Example 2.2 (Calibration of the Entropy Scale). A fair coin has entropy $\tfrac{1}{2}\log 2 + \tfrac{1}{2}\log 2 = 1$ bit, while a deterministic variable (one whose value is certain) has entropy 0 bits. A uniform distribution over all binary strings of length n has entropy n bits, and n bits is the maximum possible entropy for a variable with 2^n possible values, reflecting the fact that uniform distributions are the most uncertain ones. A *biased* coin that lands heads with probability p has entropy $h(p) := p \log \tfrac{1}{p} + (1-p)\log \tfrac{1}{1-p}$, and two values of this function play a role in later sections of these notes: $h(0.2) \approx 0.72$ bits and $h(0.1) \approx 0.47$ bits. A coin biased toward one outcome is more predictable than a fair coin and consequently carries less than one bit of uncertainty.

2.3 The coding interpretation

Of the available interpretations of entropy, the one on which we rely most heavily is entropy as data compression. Suppose that the value of X must be transmitted to a colleague in binary, and that the objective is to minimize the average number of bits sent. The governing principle is to assign short codewords to the likely values and long codewords to the unlikely values, thereby reducing the expected description length by exploiting the structure of the distribution. Shannon's source coding theorem renders this principle precise: there exists a code (a prefix-free assignment of binary strings to outcomes) whose codeword for outcome x has length essentially $\log \tfrac{1}{\mu(x)}$ bits, and no code can achieve a smaller average length than the entropy. Consequently,

$$H(X) \;=\; \text{the minimum average number of bits} \\ \text{needed to describe a sample of } X.$$

Example 2.3 (A Small Optimal Code). Let X take four values with probabilities $\tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{8}, \tfrac{1}{8}$, and assign the codewords `0`, `10`, `110`, `111`. Each codeword for an outcome of probability $2^{-\ell}$ has length exactly $\ell = \log \tfrac{1}{\mu(x)}$, and the average length is $\tfrac{1}{2}\cdot 1 + \tfrac{1}{4}\cdot 2 + \tfrac{1}{8}\cdot 3 + \tfrac{1}{8}\cdot 3 = 1.75$ bits, which equals $H(X)$ exactly, confirming that the entropy is attained by a code whose codeword lengths match the ideal lengths $\log \tfrac{1}{\mu(x)}$.

We now assign a name to the quantity inside this expectation, since the central argument of Section 6 turns on the distinction between the individual quantity and its average.

Definition 2.4 (Shannon Codelength, Also Called Stochastic Entropy or Surprisal). For an individual outcome x under distribution μ, the *Shannon codelength* is

$$\hat{H}(x,\mu) \;:=\; \log \frac{1}{\mu(x)}.$$

This quantity is the length of the codeword assigned to x under the code optimized for μ, and the entropy is its mean: $H(\mu) = \left\langle \hat{H}(X,\mu) \right\rangle_{X\sim\mu}$.

We emphasize the dependency structure of this definition. The codelength $\hat{H}(x,\mu)$ is a property of the *pair* consisting of an outcome and a distribution—the same physical outcome x receives a short codelength under a distribution that anticipated it and a long codelength under a distribution that did not. At this stage, no notion of "the entropy of an individual outcome x" exists on its own, in the absence of a distribution against which the outcome is read. This observation becomes the crux of Section 6.

2.4 Joint and conditional entropy, and mutual information

Having interpreted entropy as an optimal description length, we now extend these notions to pairs of random variables, characterizing both the uncertainty that remains in one variable once another is known and the information that the two variables share.

Definition 2.5 (Conditional Entropy). For jointly distributed X, Y, the *conditional entropy* of X given Y is

$$H(X \mid Y) \;:=\; H(X,Y) - H(Y),$$

where $H(X,Y)$ is the entropy of the pair. Equivalently, $H(X \mid Y)$ is the average, taken over values y, of the entropy of the conditional distribution of X given $Y = y$; it measures the uncertainty about X that remains once Y is known.

The rearranged identity $H(X,Y) = H(Y) + H(X \mid Y)$ is called the *chain rule*, and it admits a transparent coding interpretation: to describe the pair, one first describes Y at an average cost of $H(Y)$ bits, and then describes X using a code adapted to the known value of Y at an average cost of $H(X \mid Y)$ bits.

Definition 2.6 (Mutual Information). The *mutual information* between X and Y is

$$\begin{aligned} I(X;Y) \;&:=\; H(X) + H(Y) - H(X,Y) \\ &=\; H(X) - H(X \mid Y) \;=\; H(Y) - H(Y \mid X). \end{aligned}$$

Mutual information is the average number of bits that knowledge of Y saves in describing X; by the symmetry of the definition, it is equally the number of bits that knowledge of X saves in describing Y. It vanishes exactly when X and Y are independent, and it is never negative, a fact we prove below. Mutual information serves as the standard measure of "how much one part of the world knows about another", an informal phrase that recurs many times in these notes.

Example 2.7 (Calibration of Mutual Information). Let $X = (B_1, B_2)$ consist of two independent fair bits, and let $Y = B_1$ be the first of them. Then $H(X) = 2$, and once Y is known only the second bit remains uncertain, so that $H(X \mid Y) = 1$ and $I(X;Y) = 1$ bit: Y contains exactly one bit of information about X. If instead Y were an independent coin flip, $I(X;Y)$ would equal 0; if Y were a full copy of X, $I(X;Y)$ would equal 2 bits, the whole entropy of X, confirming that in this example the mutual information varies from zero under independence to the full entropy of X when Y determines X.

2.5 Divergence and two fundamental inequalities

Having quantified the information shared between two variables, we now introduce a measure of the discrepancy between two distributions, from which the inequalities underlying our subsequent analysis follow.

Definition 2.8 (Kullback–Leibler Divergence). For two distributions μ, ν on the same countable set,

$$D(\mu \,\|\, \nu) \;:=\; \sum_{x} \mu(x) \log \frac{\mu(x)}{\nu(x)}.$$

In coding terms, $D(\mu\|\nu)$ quantifies the cost of holding a mistaken model of the source: if the true distribution is μ but compression is performed with the code optimized for ν , the average expenditure is $H(\mu) + D(\mu\|\nu)$ bits rather than $H(\mu)$. The definition remains meaningful, and we will make use of it, even when the reference ν is an unnormalized nonnegative measure rather than a probability distribution.

Both of the inequalities required for our analysis follow from a single elementary property of the convex function $t \mapsto t \log t$.

Lemma 2.9 (Log-Sum Inequality). Let $a_1, a_2, \ldots$ and $b_1, b_2, \ldots$ be nonnegative reals with finite sums $a := \sum_i a_i$ and $b := \sum_i b_i > 0$. Then

$$\sum_i a_i \log \frac{a_i}{b_i} \;\geq\; a \log \frac{a}{b},$$

with the conventions $0 \log \frac{0}{b} = 0$ and $a \log \frac{a}{0} = +\infty$ for $a > 0$.

Proof

The function $\varphi(t) = t \log t$ is convex on $[0, \infty)$. We apply Jensen's inequality (which states that for convex φ, the average of φ is at least φ of the average) with weights $w_i = b_i / b$ and points $t_i = a_i / b_i$:

$$\sum_i \frac{b_i}{b} \varphi\Big(\frac{a_i}{b_i}\Big) \;\geq\; \varphi\Big(\sum_i \frac{b_i}{b} \cdot \frac{a_i}{b_i}\Big) \;=\; \varphi\Big(\frac{a}{b}\Big).$$

Multiplying both sides by b and simplifying $b_i \cdot \frac{a_i}{b_i} \log \frac{a_i}{b_i} = a_i \log \frac{a_i}{b_i}$ yields the claim.

Taking the a_i and b_i to be the probabilities of two distributions (so that $a = b = 1$) yields *Gibbs' inequality*: $D(\mu \| \nu) \geq 0$, with equality only when $\mu = \nu$. Nonnegativity of mutual information follows because $I(X;Y) = D\big(\text{joint distribution} \,\|\, \text{product of marginals}\big) \geq 0$. The second consequence, which we prove at its point of use in Section 4, is the *data processing inequality* for divergence: passing two distributions through the same noisy channel can only bring them closer together.

Having established these information-theoretic foundations, we now turn to the principal subject of these notes.

3. Characterizing optimization

3.1 Two notions of optimization and their relationship

The term "optimization" carries two distinct established meanings, and the relationship between them constitutes the central question of this section. In computer science, an optimization algorithm is a program that outputs the solution, or an approximation thereof, to an optimization problem: an objective function to be minimized over some feasible region. A program that returns $x \approx \sqrt{2}$, the minimizer of $y = (x^2 - 2)^2$ on the real line, is optimizing in this sense. In operations research and engineering, optimization denotes something physically more substantial: the reworking of a process or an artifact so that it better serves a purpose, as when a factory is reorganized to produce more nails per unit cost. These two activities, optimizing a number inside a computer and optimizing a factory, are evidently related, yet the precise nature of their connection is far from obvious. Two questions therefore present themselves: What exactly is the relation between these two senses of optimization? What must a physical process accomplish in order to qualify as optimization? To address these questions systematically, we begin by distinguishing the roles involved in any optimization process.

3.2 Optimizers and convergent attractors

Our characterization rests on a distinction between two roles that must be kept separate. The first is the *system being optimized*: some part of the world whose configuration is of interest, and which, in the absence of intervention, could settle into a wide variety of configurations. The second is the *optimizer*: a separate physical entity whose presence drives that system reliably toward a narrow target. The optimizer is neither the target nor the funneling itself; it is the *cause* of the funneling. This idea admits the following crisp characterization: an optimizer is a physical entity such that conditioning on its presence collapses an observer's uncertainty about the final configuration of the optimized system.

The example of a courier delivering a package to a fixed address makes these two roles concrete. The package is the system being optimized, since its location could in principle be almost anywhere, while the courier plays the role of the optimizer. The package may begin at an arbitrary location in the city and be subjected to road closures and traffic perturbations along the way; the courier identifies a route around each obstacle, ensuring arrival at the designated address. Left to its own dynamics, the package exhibits no tendency toward any particular destination; it is the presence of the courier that drives it to the address. In the language of dynamical systems, the courier renders the address a *convergent attractor* for the package: a configuration toward which the package is pulled from far away, and to which it returns after being displaced. The attractor is a fact about the package's behavior; the courier is the entity that creates it.

Two features of this picture carry the entire analytical weight of our characterization: a *broad range of initial conditions* and *robustness to perturbation*. Regarding the first, the optimizer does not require the system to begin in any special configuration, since a wide variety of distinct starting points are all driven to the same destination. An observer who knows only that the package began somewhere in the city is highly uncertain about its initial location, yet upon learning that a courier is delivering the package, the same observer becomes confident about its final location. Regarding the second, the convergence survives disturbances applied while the process is underway. This second feature is what separates a genuine optimizer from a coincidence. A satellite coasting in orbit is subject to no steering influence; once perturbed, it drifts onto a new trajectory indefinitely, possessing no home configuration to which it returns. The courier, when blocked by a closed road, selects an alternative route and still arrives. The entire difference lies in whether some entity is present to supply the restoring tendency.

One omission from this characterization matters for everything that follows: designating an entity as an optimizer asserts nothing about goals, preferences, or representations stored in a mind. A courier, a feedback circuit, a chess engine driving a game toward checkmate, a growing organism, a running optimization algorithm, and a team building a house are optimizers in exactly the same sense, each being a physical entity whose presence drives some system into a narrow target and holds it there against per-

turbation. What unifies these examples is a structural fact about their effect on the world rather than any mental attribute. The following three subsections develop this picture into a quantitative measure, establish why even an observer with no stake in the target has reason to attend to optimizers, and argue that the property of being an optimizer is consequently an objective feature of the world.

3.3 Optimization as entropy reduction

The convergent-attractor picture translates directly into the information-theoretic language of Section 2, and this translation provides the bridge to physics.

We model the configuration of the optimized system as a random variable, writing X for its initial configuration and Y for its final configuration. Before any optimizer is taken into account, X ranges over a broad set, so an observer who knows only that the system begins somewhere in that set faces high uncertainty $H(X)$. We now introduce the optimizer and condition on its presence. Since the optimizer drives the system into the narrow target regardless of where the system began, the residual uncertainty about the final configuration, $H(Y)$, is small. The gap between these two quantities is the *entropy reduction*

$$\Delta H \;:=\; H(X) - H(Y),$$

the number of bits by which the optimizer has narrowed the system's possibilities. A broad range of initial conditions makes $H(X)$ large; robustness is what preserves the magnitude of $H(X)$ while the convergence continues to operate; a narrow target makes $H(Y)$ small. A large ΔH therefore corresponds to reliably reaching a target that the system's own unaided dynamics would essentially never attain, providing a precise formulation of the informal statement with which these notes opened: a powerful optimizer reliably produces outcomes that would be extremely unlikely under any random process.

Remark (Scope and Limitations of the Entropic Summary). The summary statistic ΔH deliberately discards the *direction* of optimization: it records how narrow the achieved region is, not which region it is nor whether any agent desired it. This loss is smaller than it may initially appear. Wentworth has observed that any expected utility maximization problem can be decomposed into two parts: an entropy-minimization component, and a component that shapes the world's distribution to resemble one particular target distribution (in his formulation, "utility maximization is description length minimization"). The present notes concern the first component, which is where the laws of physics impose their constraints; the second component (the identity of the target, and the reasons for its selection) belongs to the theory of goals and preferences rather than to thermodynamics.

3.4 The predictive value of optimizers

We now present a reason to attend to optimizers that presupposes nothing about any agent's goals. Consider an agent whose sole objective is to *predict* the future configuration of some part of the world. The claim of this subsection is that, among all the features of the environment such an agent could study, optimizers are among the least costly to characterize.

We begin with the ordinary case, in which no optimizer is involved. To predict the final state Y of a physical system, one must in general know its initial state X in detail and propagate that knowledge forward through the dynamics. This procedure is costly in two respects. Specifying the initial condition costs approximately $H(X)$ bits, and for a system of any appreciable size $H(X)$ is enormous. Moreover, the resulting prediction degrades as the forecast horizon extends: chaotic dynamics amplify any error in the knowledge of X, so that a coarse measurement of X conveys progressively less information about Y over time. A substantial informational expenditure in the present thus purchases a forecast whose quality only deteriorates.

Suppose instead that an optimizer is acting on the system, driving it toward a target set T. Two changes occur simultaneously, both operating in the predictor's favor.

First, *knowledge of the initial condition is no longer required*. The optimizer drives a broad range of initial states to T, so uncertainty about X never propagates into uncertainty about Y. The $H(X)$ bits that would otherwise have to be acquired are simply irrelevant to the forecast.

Second, *specification of the target itself constitutes the forecast*. The laborious route to predicting where the system ends up is to determine its exact starting configuration and to propagate the dynamics forward step by step. For any large system this route is infeasible: the starting configuration is prohibitively large to specify, and chaos ensures that the slightest error in it grows until the forecast becomes uninformative. The optimizer, however, supplies a considerably more economical route: since it drives the system to the target regardless of the initial configuration, the predictor may discard the starting configuration entirely and simply state the system's destination, namely the target. A target is informationally inexpensive to specify, and robustly so, since the optimizer reaches it despite disturbances that no predictor could have tracked.

This constitutes a saving of effort rather than the acquisition of information without cost. The statement that the system terminates at the target itself constitutes the forecast; no additional knowledge has been generated. The point is only that knowing the destination toward which a process is driven is informationally inexpensive and stable, whereas reconstructing where the process started, the sole alternative route to the same forecast, requires a description that is vast in extent and fragile under error. Optimizers are worth identifying because they permit the prediction of an outcome that would otherwise be almost entirely inaccessible.

The courier example renders the two routes concrete. To predict where the package ends up, one could attempt to track the courier's vehicle, every traffic light, and every other car on the road, and integrate the entire configuration forward: an intractable accumulation of contingencies governed by chaotic dynamics, in which a single mistimed traffic light alters every subsequent turn of the route. Alternatively, one could learn a single fact—that the courier is delivering to 14 Elm Street—and predict that the package ends at 14 Elm Street, having modeled none of the traffic. The second route expends almost nothing and predicts almost perfectly, demonstrating that knowledge of the optimizer and its target substitutes for detailed knowledge of the dynamics.

The robustness property sharpens this conclusion further: because the optimizer reaches T in spite of perturbations, the forecast that the system ends in T survives disturbances that the predictor could neither have foreseen nor measured. The predictor is thus spared not only the cost of the initial condition but also the cost of modeling the perturbations, since the optimizer absorbs both. For this reason alone, an agent concerned exclusively with prediction has reason to scan its environment for optimizers and to track the targets toward which they steer, since no other feature of the environment compresses the future into so short a description.

3.5 The observer-independence of optimization

The argument of Section 3.4 answers a standing objection to treating "optimization" or "intelligence" as a genuine feature of the world at all.

The Observer-Relativity Objection. A well-known position holds that intelligence, and optimization with it, is *observer-relative*: a system counts as intelligent only because it performs well on the tasks that *we* happen to care about, judged by the standards that *we* happen to hold, and under a different choice of tasks the same system would appear unremarkable. This position has a formal backbone in the No Free Lunch theorems, which establish that, averaged over all possible problems, no optimizer outperforms blind search; competence is then never absolute but always relative to some restricted class of problems, and the choice of class reflects the observer's interests. The position has a philosophical counterpart in Dennett's intentional stance, according to which describing a system as an agent with goals is not a discovery about the system but a *stance* that an observer adopts because it pays off predictively, so that whether a system "has goals" depends on the observer's modeling capacities and convenience. On either formulation, the claim that X is an optimizer appears less like a fact about X than like a relation between X and some particular onlooker.

The Thermodynamic Response to the Objection. We first identify the point of agreement with the intentional stance: our account likewise grounds the significance of optimizers in their predictive value (Section 3.4). The disagreement concerns whether that value is relative to a particular observer, and we contend that it is not. The question of whether some entity is driving a given system into a narrow target, from a broad set of initial conditions and robustly to perturbation, is a question about the system and the entities acting upon it, and its answer makes no reference to anyone's

preferences. The predictive leverage that a known optimizer provides—a short and accurate description of the state toward which the system is driven—is identical for every predictor regardless of what, if anything, that predictor wants. Consequently, no choice of tasks or standards is required in order to identify optimizers: the bare objective of prediction, which any embedded agent has reason to pursue whatever its goals, already suffices to pick them out. The reason to attend to optimizers is therefore observer-*independent*.

None of this contradicts the No Free Lunch theorems, which concern average competence across *all* problems and assert nothing about whether some physical entity is driving a given system into a narrow target. Nor does our account deny that an optimizer has a target: the identity of the target is itself a property of the system, namely the region into which its dynamics actually funnel, rather than a verdict imposed from outside. The observer-relativity with which the objection is concerned, namely the question of whose standards adjudicate success, never enters the analysis, because we are not grading the optimizer against any standards at all; we are observing that it possesses an attractor and using that fact, as any predictor could, to predict the world. Optimization is accordingly an objective feature of the world itself rather than a construct that an observer projects onto it.

We now collect the conceptual framework before turning to the physics. An optimizer is a physical entity that drives the system on which it acts into a convergent attractor (Section 3.2); the amount of optimization is the entropy the optimizer removes from that system (Section 3.3); and any predictor, whatever its goals, has an objective reason to track optimizers (Sections 3.4–3.5). What remains is to determine what physics permits. Optimization constitutes entropy reduction in a subsystem—that is, *local* entropy reduction—and the following section establishes that *global* entropy reduction is physically impossible. This tension, and the "bookkeeping" that resolves it, organizes the remainder of these notes.

4. Reversibility and the second law

4.1 The reversibility of microscopic physics

Having introduced the convergent-attractor picture of optimization, we now examine the physical constraints that any such process must respect, beginning with the observation that, at the most fundamental level of description available to us, physical systems never lose information. In classical mechanics, the complete microscopic state of a system, its *microstate*, consists of the position and momentum of every particle, corresponding to a single point in the system's *phase space*. The laws of motion carry each microstate along a unique trajectory, determining the future from the present and, equally, the past from the present. It follows that two distinct microstates can never evolve into the same state: if they did, running the laws backward from the shared future state could not determine which of the two microstates to recover, and the dynamics admit no such ambiguity. Over any fixed interval, therefore, the evolution of

an isolated system constitutes a *bijection* on microstates, an invertible pairing of initial states with final states. Liouville's theorem strengthens this conclusion: the evolution preserves not only distinctness but phase-space *volume*, allowing a region of microstates to be stretched and folded without limit while retaining its total volume. Quantum mechanics expresses the same principle in different notation: isolated evolution is unitary, hence invertible and volume-preserving. In either formulation, microscopic information is neither created nor destroyed.

4.2 The incompatibility of global funneling with reversibility

Given the reversibility of the microscopic dynamics established above, we now confront the convergent-attractor picture of optimization with this fundamental property. Taken literally at the microscopic level, the two are directly incompatible: funneling requires that many initial states converge to the same small set of target states, constituting a many-to-one map, whereas reversibility guarantees that the microscopic dynamics are one-to-one, rendering a perfect funnel microscopically impossible.

A second, closely related obstruction arises from the second law of thermodynamics itself, the principle that the entropy of an isolated system tends not to decrease. In the framework we are about to develop, the second law is not an additional postulate but a *consequence* of reversibility, and understanding this derivation illuminates both principles. The resolution of the apparent paradox—optimization manifestly occurs, yet physics forbids funneling—is provided by the central "bookkeeping" principle of these notes:

> *No physical process can reduce the entropy of an isolated system. Optimization is entropy reduction in a coarse-grained description of a subsystem, and the information displaced from the optimized subsystem must be accounted for elsewhere in the larger system.*

The remainder of this section renders the first sentence precise and establishes it rigorously. Section 5 and Appendix B address the second sentence, characterizing where the displaced information can reside and quantifying how much entropy reduction an agent can accomplish.

4.3 Coarse-graining and the emergence of probability

The preceding discussion prompts a natural question: if the microscopic dynamics are deterministic and information-preserving, from where does probability arise, and what accounts for the rise of entropy? The answer lies in the fact that no observer ever tracks the microstate. A gram of matter possesses on the order of 10^{23} coordinates, each specified to unbounded precision, a description that no observer, and no embedded agent, can maintain. What is tracked instead is a *coarse-grained* description: we partition the phase space into discrete cells, each aggregating all microstates that cannot be distin-

guished at the chosen resolution (for instance, every particle's position and momentum specified to finitely many digits, or merely a small collection of macroscopic variables), and describe the system by identifying the cell it currently occupies.

The dynamics of cells, in contrast to the dynamics of microstates, are genuinely stochastic. The current cell does not determine the next one, because different microstates within the same cell flow to different destinations. The most complete description available is a transition probability $P(y, x)$, defined as the fraction of cell x, measured by phase-space volume, that arrives in cell y one step later. This randomness is not metaphysical in character; it reflects precisely the microscopic detail that the coarse description declines to resolve. Chaotic dynamics continually transport such detail upward across the resolution scale, so that the coarse trajectory resembles a sequence of random jumps even though the underlying flow remains deterministic.

One structural assumption underlies the whole of stochastic thermodynamics: the coarse trajectory constitutes a *Markov process*, meaning that the distribution of the next cell depends only on the current cell and not on the earlier history. The intuition is that a sufficiently good coarse-graining ensures that the current cell contains everything about the past that is relevant to the coarse future, with the discarded detail contributing independent noise at each step rather than persisting as hidden memory. Whether a given coarse-graining is genuinely Markovian remains a deep and open question, but two considerations provide reassurance. First, there exist exactly solvable models, the *multibaker maps*: deterministic, time-reversible, chaotic systems whose coarse-grainings provably reproduce arbitrary Markov chains, with all of the randomness residing in the initial condition. These models demonstrate that macroscopic stochasticity and irreversibility are fully compatible with microscopic determinism and reversibility. Second, the Markov property is precisely what renders everyday statistical reasoning valid, and its *failure backward in time* constitutes the arrow of time. A dropped glass shatters at a predictable moment, and its shards obey well-defined local statistics that are entirely independent of events occurring at a neighboring house. Under time reversal this description fails: retrodicting the moment at which the shattered glass was dropped cannot be accomplished through local statistics, and the most informative evidence available to an observer may be a conversation about the accident taking place next door. Forward evolution obeys memoryless local laws while backward evolution does not, and this asymmetry is exactly the content of the Markov property.

One further consequence of reversibility remains to be extracted before we establish the second law. Liouville's theorem renders phase-space volume a *stationary measure* of the chain: weighting each cell by its volume produces a weighting that a single step carries to itself. We take all cells to have equal volume (a *mesoscopic* coarse-graining, obtained by truncating every coordinate to a fixed precision). Stationarity of the uniform weighting then determines the structure of the transition probabilities: with π constant, the condition $\sum_x P(y, x)\, \pi(x) = \pi(y)$ reduces to $\sum_x P(y, x) = 1$ for every y, so that the probabilities flowing *into* each cell sum to one, exactly as those flowing *out of* each cell do. A matrix satisfying both properties is termed *doubly stochastic*. Double

stochasticity is thus the characteristic imprint of reversibility on the coarse dynamics: a doubly stochastic step can permute and disperse probability but can never concentrate it, since concentration would require packing several cells' worth of volume into fewer cells, which Liouville's theorem forbids.[1]

4.4 The second law of thermodynamics

We now establish the second law for the class of doubly stochastic coarse-grained dynamics identified above, namely that the entropy of the coarse-grained state cannot decrease under a single step of the chain.

Theorem 4.1 (Second Law for Doubly Stochastic Chains). Let P be a doubly stochastic transition matrix on a countable state space, let the random variable X (the coarse-grained state now) have distribution μ, and let Y (the state one step later) have distribution $P\mu$, where $(P\mu)(y) := \sum_x P(y,x)\,\mu(x)$. Then

$$H(Y) \;\geq\; H(X).$$

Proof

We establish the theorem in two steps.

Step 1: contraction of divergence under a single Markov step. Let μ and ν be any two distributions (or nonnegative measures) on the state space. We claim the data processing inequality

$$D(P\mu \,\|\, P\nu) \;\leq\; D(\mu \,\|\, \nu).$$

To verify this claim, we fix an output state y and apply the log-sum inequality (Lemma 2.9) to the numbers $a_x = P(y,x)\mu(x)$ and $b_x = P(y,x)\nu(x)$, whose sums over x are $P\mu(y)$ and $P\nu(y)$ respectively:

$$\sum_x P(y,x)\mu(x)\log\frac{P(y,x)\mu(x)}{P(y,x)\nu(x)} \;\geq\; P\mu(y)\log\frac{P\mu(y)}{P\nu(y)}.$$

We now sum this inequality over all y. On the left-hand side, the factor $\log\frac{\mu(x)}{\nu(x)}$ does not depend on y, and $\sum_y P(y,x) = 1$ because P is stochastic, so the left-hand side totals $\sum_x \mu(x)\log\frac{\mu(x)}{\nu(x)} = D(\mu\|\nu)$; the right-hand side totals $D(P\mu\|P\nu)$, which establishes the claim. The underlying intuition is precisely the coding-theoretic one: passing two information sources through the same noisy channel can only render them more difficult to distinguish.

Step 2: specialization of the reference measure to the uniform measure. Let $\sharp$ denote the counting measure, $\sharp(x) = 1$ for every x (an unnormalized uniform measure; Lemma 2.9 at no point required normalization). Double stochasticity of P states precisely that $P\sharp = \sharp$, since $(P\sharp)(y) = \sum_x P(y,x) = 1$. Moreover, divergence from the counting measure is simply negative entropy:

$$D(\mu \,\|\, \sharp) \;=\; \sum_x \mu(x) \log \frac{\mu(x)}{1} \;=\; -H(\mu).$$

Applying Step 1 with $\nu = \sharp$ yields

$$-H(P\mu) \;=\; D(P\mu \,\|\, P\sharp) \;\le\; D(\mu \,\|\, \sharp) \;=\; -H(\mu),$$

which rearranges to $H(P\mu) \ge H(\mu)$.

We draw attention to the ingredients of the proof: the list is short, and every item is physical in origin. The argument invoked the Markov property, which permitted a single step to be written as a transition matrix, and double stochasticity, which followed from Liouville's theorem, that is, from the *reversibility* of the microscopic dynamics; nothing further was required. Consequently, macroscopic irreversibility, the rise of entropy, stands in no tension with microscopic reversibility; it follows from it, once we grant that observers track coarse cells rather than microstates. This is the precise content of the assertion that "the second law follows from the reversibility of physics".

4.5 The scope and limitations of the second law

Three clarifications delineate the scope of this result and prevent misreadings that become consequential in later sections.

First, Theorem 4.1 applies to an *isolated* system evolving autonomously: the entropy of the whole cannot decrease. The theorem asserts nothing against the entropy of a *subsystem* decreasing, and the entropy of subsystems decreases constantly; this is precisely what refrigerators, crystallization, house-builders, and every other optimizer accomplish. What the theorem imposes is a bookkeeping constraint on the whole: when the entropy of a subsystem drops, the global accounting must remain consistent, with at least as much entropy appearing elsewhere in the total system. The complete classification of where this "elsewhere" can be located is the subject of Section 5.

Second, the theorem as stated concerns the entropy of the *ensemble*, that is, of the probability distribution μ, and it holds on average and in distribution; individual trajectories may pass through improbable, low-codelength states. A sharper, trajectory-level version of the second law, equipped with explicit and very small bounds on the permitted fluctuations, becomes available in the algorithmic framework of Section 7.

Third, and most consequentially for the remainder of these notes, the theorem assumed a fixed dynamics P uninfluenced by any observer, together with an entropy defined relative to a given distribution μ. Agents complicate both assumptions. An agent that *observes* the system and conditions its behavior on its observations does not constitute a fixed P; the extent of the advantage this confers, and the cost at which it is purchased, is the subject of Appendix B. Furthermore, the apparently innocuous reliance on a given μ conceals a deep conceptual problem, which we address in Section 6. Before turning to these questions, however, we take up the bookkeeping question directly: when an agent lowers the entropy of a subsystem, the question arises of where

the displaced entropy can reside, and the following section provides the complete answer. (A self-contained toy model in which every step can be verified by hand is developed separately in Appendix A.)

5. Three types of optimization under information conservation

5.1 The bookkeeping problem

Equipped with the second law and the reversibility of the underlying dynamics, we now undertake the global accounting that governs optimization by an embedded agent. We decompose the universe into three parts: the agent A, the subsystem S that the agent seeks to optimize, and the remainder of the environment E. Optimization of S consists in reducing $H(S)$, funneling the subsystem from a large set of possible configurations toward a narrow target set. The second law (Theorem 4.1, applied to the isolated whole) establishes that the joint entropy of (A, S, E) cannot decrease, and the reversibility underlying it demands that information about the initial condition of S cannot simply vanish. This raises the central "bookkeeping" question: through which channels can an agent reduce $H(S)$ while the global accounting remains consistent? The analysis of Daniel C and Ebtekar identifies exactly three such channels, and the classification is exhaustive: entropy removed from S must be transferred into E, absorbed into A, or paid for in advance through pre-existing correlation. We develop each of the three channels in turn, culminating in a reinterpretation of Maxwell's demon in terms of this classification.

5.2 Type 1: transfer of entropy into the environment as waste heat

The first channel is the most familiar of the three: the agent arranges an interaction between S and E under which $H(S)$ decreases while $H(E)$ increases by at least the same amount, thereby transferring the subsystem's entropy into the environment. A refrigerator narrows the distribution over the thermal state of its interior while heating the kitchen behind it; a house-building team imposes order on lumber and nails while dissipating heat, exhaust, and scattered debris into the surroundings. In the coin world of Appendix A, this channel is realized by a conditional-swap circuit that moves uncertainty from the designated coins into other coins. The signature of Type 1 is that the environment absorbs the entropy, the agent functioning merely as an intermediary that arranges the transfer and whose own state need not change appreciably in the process.

5.3 Type 2: absorption of entropy into the agent's memory through measurement

The second channel is subtler: the agent reduces $H(S)$ by *measuring* S and storing the outcomes, so that the entropy of the subsystem is transferred into the agent's own memory. This mechanism constitutes the operating principle of one of the most celebrated thought experiments in thermodynamics, which we now examine in detail.

Maxwell's Demon. A room full of gas is divided in two by a partition, and the partition contains a small door that can be opened and closed at will. A minute intelligence (the "demon") observes the molecules: whenever a molecule approaches the door from the left, the demon opens it, and whenever one approaches from the right, the demon closes it. The gas thereby accumulates, molecule by molecule, on the right side of the room. The entropy of the gas has evidently decreased (we are far less uncertain about the locations of the molecules than before), yet no work appears to have been performed and no heat has been discharged anywhere, presenting an apparent violation of the second law.

The second law has not been violated; rather, the reversibility argument of Section 4.1 identifies exactly where the missing entropy resides. Consider the end of the process and select any particle that now occupies the right side. Two distinct histories are consistent with its presence there: either the particle began on the right and remained, or it began on the left and was admitted by the demon. These two histories produce *identical* final gas configurations, so the gas alone cannot distinguish between them. The microscopic dynamics of the total system are reversible, however, and reversibility demands that the past be reconstructible from the present. The only way the total system can remain reversible is if some other component differs between the two histories, and that component is the demon's memory: the record of its observations and door operations. For every bit of entropy the demon removes from the gas, at least one bit of entropy accumulates in the demon's memory. The reduction in the entropy of the gas is thus fully compensated by the growth of entropy in the demon's records, ensuring that no global decrease occurs. (We revisit this argument with full rigor, for individual states rather than ensembles, in Section 7.4.)

The signature of Type 2 is that the agent absorbs the entropy: measurement transfers uncertainty from the world into the measurer. The demon's memory is finite, however, so this channel is exhaustible: once the memory fills, the demon must either halt or clear it, and clearing it ultimately exports the stored entropy to the environment as Type-1 waste after all (the exact bookkeeping is provided in Section 7.4). Type 2 therefore functions as a finite buffer rather than as a sink of unbounded capacity, deferring the environmental cost rather than eliminating it.

5.4 Type 3: expenditure of pre-existing mutual information

The first two channels balance the accounts by increasing entropy elsewhere, whether in E or in A. The third channel is the most surprising of the three: it reduces $H(S)$ *without increasing entropy anywhere*, consuming instead a correlation that already exists between the agent and the subsystem.

Suppose that at time t the agent's state and the subsystem's state share mutual information $I(S_t; A_t) > 0$: the agent already "knows something" about the subsystem, in the purely statistical sense of Definition 2.6. To keep the accounting transparent, we assume that the environment is independent of both, so that the joint entropy decomposes (by the definition of mutual information and the chain rule) as

$$H(S_t, A_t, E_t) \;=\; H(A_t) \;+\; H(S_t) \;-\; I(S_t; A_t) \;+\; H(E_t).$$

The agent now performs an operation with the following effects: it *erases the copy of the shared information that resides inside* S, using its own copy as the key, while leaving its own marginal state and the environment untouched. The operation transforms the subsystem so that

$$H(S_{t+1}) \;=\; H(S_t) - I(S_t; A_t), \qquad I(S_{t+1}; A_{t+1}) \;=\; 0,$$
$$H(A_{t+1}) = H(A_t), \qquad H(E_{t+1}) = H(E_t).$$

Summing the joint entropy after the operation yields

$$H(A_{t+1}) + H(S_{t+1}) + H(E_{t+1}) \;=\; H(A_t) + H(S_t) - I(S_t; A_t) + H(E_t),$$

which is *exactly* the joint entropy from before. The entropy of the subsystem has genuinely fallen by $I(S_t; A_t)$ bits, the total entropy is unchanged, no heat has been produced, and no memory has been filled. What has been consumed is the correlation itself: afterward $I(S_{t+1}; A_{t+1}) = 0$, and the operation cannot be repeated without first re-acquiring mutual information.

Example 5.1 (The Controlled-NOT as a Minimal Instance of Type 3). We illustrate the mechanism through the smallest possible instance, which renders the bookkeeping fully transparent: let the subsystem hold a single uniformly random bit s, and let the agent hold a perfect copy $a = s$. Then $H(S) = 1$, $H(A) = 1$, $I(S; A) = 1$, and the joint entropy is $H(S, A) = 1$ bit (two equally likely joint states, 00 and 11). The agent now applies a *controlled-NOT*, which flips s if $a = 1$ and leaves s unchanged if $a = 0$. In both joint states the result sets s to 0: the subsystem becomes deterministic, with $H(S_{t+1}) = 0$, corresponding to a full bit of entropy reduction. The operation is reversible (a second application undoes it), involves no environment, and produces no heat. The joint entropy remains exactly 1 bit, now carried entirely by the agent's own marginal (a remains a uniform bit, uncorrelated with the now-deterministic s). The correlation has been spent: a further application of the operation accomplishes nothing, since $I(S_{t+1}; A_{t+1}) = 0$. This confirms that a Type-3 operation reduces the entropy of the subsystem by exactly the mutual information consumed, with the global accounting remaining consistent throughout.

5.5 A reinterpretation of Maxwell's demon, and a synthesis of the three channels

Having identified the three channels individually, we now return to Maxwell's demon, whose operation the Type-3 channel recasts in an illuminating way. The demon's measure-and-control behavior can be decomposed into two distinct steps: a *copy* step, in which the demon interacts with the gas to acquire mutual information with it (this is Type 2: the demon's memory absorbs entropy, or more precisely, becomes correlated with the gas), followed by a *control* step, in which the demon expends that mutual information to steer the gas into a narrower distribution (this is Type 3: the correlation is consumed, and the control step itself produces no further entropy anywhere). In this

decomposition, measurement constitutes the purchase of correlation and steering its subsequent expenditure, which raises a quantitative question: how much entropy reduction can the control step purchase per bit of correlation acquired in the copy step? This is precisely the question answered by the *Touchette–Lloyd theorem* (Appendix B), which bounds the steering achievable in a Type-3 step by the mutual information available to be spent, bit for bit. (Section 7.4 subsequently derives an exact algorithmic version of the same bound.)

To summarize, an embedded agent can optimize a subsystem in exactly three ways:

1. **Type 1:** transfer of the subsystem's entropy into the environment as waste heat;
2. **Type 2:** absorption of the subsystem's entropy into the agent's own memory through measurement;
3. **Type 3:** expenditure of mutual information already shared with the subsystem, erasing the subsystem's copy of the correlation.

These three channels are not mutually exclusive but compose in practice: a realistic optimizer cycles among them, measuring (Type 2), steering (Type 3), and periodically clearing its memory into the environment (Type 1) in order to free capacity for the subsequent round.

The Touchette–Lloyd theorem, developed in Appendix B, prices this exchange at one bit of entropy reduction beyond the blind baseline per bit of mutual information between action and environment, and we draw on its statement in what follows.

6. Limitations of subjective entropy

Throughout the preceding sections we have treated the distribution μ, against which all of our entropies are defined, as simply given. In this section we subject this assumption to systematic examination, and we find that it fails precisely in the regime where agent foundations requires it to hold. We begin by recalling the role of entropy as a measure of optimization capacity, then examine three distinct failure modes of the ensemble picture, culminating in the case of the demon's memory, which we argue constitutes the central case for agent foundations. The critique developed here, together with its resolution in the following section, follows the essay of Daniel C and Ebtekar and the foundational paper of Ebtekar and Hutter.

6.1 Entropy as a measure of optimization capacity

We begin by recalling the function that entropy serves: across physics and engineering, the role of entropy is to measure a system's capacity to do work, and the developments of Sections 4–5, together with the coin world of Appendix A, permit us to read "capacity to do work" as *capacity to optimize*: negentropy constitutes the fuel that funnels broad distributions into narrow ones.

We now apply this reading to the demon after an extended run of Type-2 optimization. Its memory comprises, say, m bits, some of which are now occupied by measurement records. The demon's *remaining* capacity to optimize the gas is given by its free memory, namely the total size minus the entropy already stored, with each further bit of optimization consuming another bit of free memory. This observation raises the question on which the entire analysis turns: *Is the demon's free memory an objective, physical quantity?* One would expect this quantity to be objective: whether the hardware can absorb another billion measurements is a fact about the demon's physical configuration, carrying direct engineering consequences (how much longer the demon can continue to refrigerate, and how much heat it must eventually dissipate to clear its records). It would be anomalous for the demon's actual ability to act on the gas to depend on the beliefs of some outside observer describing it, yet this is precisely what the standard formalism delivers.

6.2 The exogenous status of the reference distribution

We recall the shape of the standard definitions (Definition 2.4). The entropy of an *individual* state x is undefined; what is defined is the Shannon codelength $\hat{H}(x, \mu) = \log \frac{1}{\mu(x)}$ of a state *relative to a distribution* μ, together with its average, the Gibbs–Shannon entropy $H(\mu)$. The distribution μ enters as an input rather than an output: it encodes the knowledge of some observer of the system, and the formalism specifies neither whose knowledge is encoded, nor where it is stored, nor how it was obtained, nor what it cost. In our terms, the standard framework treats knowledge as *exogenous*.

For the classical equilibrium applications of thermodynamics this exogeneity is harmless, because the appropriate choice of μ is essentially forced. This harmlessness, however, rests on three conditions. We say that μ constitutes a *useful summary* of an actual physical state x when all three of the following hold:

1. **Simplicity:** μ admits a short description, such as "an ideal gas of N molecules at temperature 300 K and pressure 1 atm, at equilibrium". (Otherwise, the specification of μ itself introduces unaccounted-for information, a point to which we return below.)
2. **Concentration:** the codelength concentrates near its mean, so that $\hat{H}(X, \mu) \approx H(\mu)$ with high probability under $X \sim \mu$. For systems composed of many weakly interacting parts this property follows from the law of large numbers, and it is precisely what permits a single number, "the entropy", to stand in for an entire distribution of codelengths.
3. **Typicality:** the actual state x constitutes an unremarkable sample from μ, falling within the high-probability set where the previous two conditions apply.

When all three conditions hold, $H(\mu) \approx \hat{H}(x, \mu)$, every reasonable notion of entropy agrees, and thermodynamics may be conducted with macrovariables alone, without ever inquiring where μ is stored. Equilibrium statistical mechanics lies entirely within

this regime, and the classical apparatus of Clausius and Boltzmann, with temperature, pressure, and chemical composition as the privileged macrovariables, is well adapted to it.

6.3 Three failure modes of the ensemble formalism

Far from equilibrium, and above all for *information-bearing* states, each of the three conditions identified above can fail independently, and each failure breaks the formalism in its own instructive way. We examine the three failure modes in turn.

Failure of Simplicity: Knowledge Without Informational Accounting. Let μ be a point mass on one particular, intricate configuration x, corresponding to exact knowledge of the microstate. Then $H(\mu) = 0$, and the formalism declares the system fully known and hence fully available for work extraction: an agent with exact knowledge of the state of a gas can in principle extract all of its energy as work, employing a transformation tailored to that state. The tailored transformation, however, together with the knowledge it embodies, must itself have been specified; the description of μ is exactly as intricate as x itself, and the formalism assigns it a cost of zero. The Gibbs–Shannon account thus permits arbitrarily detailed knowledge as an exogenous resource obtained at no cost. Physically, any agent in possession of such knowledge must encode it in a memory, and acquiring, storing, and clearing that memory carry costs to which the second law is sensitive, as the analysis of the demon has already demonstrated. The ensemble formalism provides no mechanism for accounting for these costs.

Failure of Concentration: Averages Representative of No Individual State. Consider a robot that flips a hidden fair coin and, depending on the outcome, either fully drains its battery or leaves it fully charged. Under the resulting equiprobable mixture μ over the two outcomes, the Gibbs–Shannon entropy of the battery together with its surroundings lies exactly halfway between the entropy of the charged state and that of the drained state, and a free-energy calculation based on $H(\mu)$ concludes that the robot can perform approximately half a charge of work. This conclusion is incorrect under either outcome of the coin flip: the robot can perform either a full charge of work or none at all. The mean $H(\mu)$ is an average over an ensemble whose members behave nothing like the average, the distribution being bimodal rather than concentrated. The quantity we actually require is a property of the *individual* state that the robot in fact occupies, and the ensemble formalism defines no such quantity.

Failure of Typicality: Structure Unresolved by the Ensemble. Suppose that the actual state x, while formally a possible sample from the equilibrium ensemble μ, happens to be special in a way that the macrovariables do not capture: the gas particles may, for instance, momentarily occupy a strongly patterned, compressible configuration. The codelength $\hat{H}(x, \mu)$ evaluates x as if it were generic, and a work calculation based on this codelength will *underestimate* what a sufficiently sophisticated machine —one that recognizes and exploits the pattern, that is, a machine running a compression algorithm—can extract from x. Interpreted literally, the ensemble calculation

would classify such a machine as a violator of the second law. The correct response is not to prohibit sophisticated machines but to acknowledge that the entropy relevant to what can be done with x is a property of the actual structure of x, not of the ensemble in which we happened to embed it.

6.4 The demon's capacity as the central case for agent foundations

We now return to the demon's free memory, where the difficulty assumes its sharpest form. The demon's remaining optimization capacity is m minus the entropy of its memory contents, and the question immediately arises: *Relative to which distribution is this entropy to be evaluated?* If we, as outside observers, refine our beliefs about what the demon has recorded, the standard formalism asserts that the entropy of the demon's memory drops and that the demon's "capacity to do work" rises, although nothing about the demon's hardware has changed. The classical macrovariable repertoire offers no assistance: temperature, pressure, and chemical composition are designed for systems whose unresolved degrees of freedom mix rapidly and retain no persistent record, whereas a memory is, by engineering design, a system whose states are stable, distinguishable, and meaningful. Every distinct memory configuration must be treated as its own condition; there is no thermalizing ensemble over memory states for μ to attach to, and there is no privileged ensemble over the outputs of a long computation, since the purpose of computation is precisely to produce states whose structure no simple prior anticipates.

For agent foundations, these failure modes are not pathological exceptions to be excluded from the analysis; rather, they constitute the entire subject matter, since the states of primary interest—an agent's memory and world model, the records left by measurements, computations in progress, an environment partway through being reshaped toward a target—are exactly the far-from-equilibrium, information-bearing, intricately structured states on which the ensemble formalism fails. If entropy is to serve as the measure of optimization capacity for embedded agents, we require a definition of entropy that applies to an *individual physical state*, with no exogenous distribution appearing anywhere in the formalism. Such a definition exists, originating in the theory of computation, and its development is the subject of the following section.

7. Algorithmic thermodynamics

Algorithmic thermodynamics, developed by Ebtekar and Hutter on foundations laid by Bennett, Zurek, Gacs, and Levin, retains the entire coarse-grained Markov framework of Section 4 while introducing a single fundamental modification: the subjective ensemble μ is replaced by universal computation. Rather than asking how surprising a state x is under μ, the algorithmic framework asks how difficult x is to describe in absolute terms. The conceptual gain is an objective, observer-free entropy that *dissolves the subjectivity problem* of Section 6, being defined for a single physical state without

reference to any ensemble. In developing this framework we obtain a second law and fluctuation bounds that hold for individual trajectories, culminating in an exact analysis of Maxwell's demon.

7.1 Kolmogorov complexity

We begin by fixing a universal computer U, which may concretely be regarded as an interpreter for a general-purpose programming language. Every coarse-grained state x of a physical system is identified, via the coarse-graining's encoding, with a finite binary string, so that the notion of a program that outputs x is well defined.

Definition 7.1 (Description Complexity). The *Kolmogorov complexity* (or description complexity) $K(x)$ of a string x is the length, in bits, of the shortest program that outputs x when run on U. The *conditional complexity* $K(x \mid y)$ is the length of the shortest program that outputs x when given y as auxiliary input. The *algorithmic mutual information* between two strings is

$$I(x : y) \;:=\; K(x) + K(y) - K(x, y),$$

the number of bits that a description of one saves in describing the other.[2]

The appropriate interpretation of $K(x)$ is *optimal lossless compression*: the size of the smallest self-contained representation from which x can be regenerated exactly. Several examples serve to calibrate the scale of this quantity. A string of one million zeros has negligible complexity, since a fifteen-byte program suffices to print it. The first million digits of π likewise have negligible complexity, despite passing every statistical test for randomness, because a short program computes them. A string of one million bits obtained directly from a quantum random-number generator has complexity close to one million bits: almost surely there is no structure to exploit, and the shortest "program" is essentially the string quoted verbatim. A state of 10^{23} particles confined to one corner of a box has low complexity, since a short loop prints the repeated coordinates, whereas a generic thermalized state has complexity near the full length of its encoding. These examples support a crisp characterization of the algorithmic notion: low entropy corresponds precisely to compressibility, a property of the individual state x that makes no reference to any ensemble.

Algorithmic mutual information behaves analogously to the mutual information of Section 2, but for individual objects: $I(x : x) \stackrel{+}{=} K(x)$, since a copy of x renders any further description redundant; $I(x : y) \stackrel{+}{=} 0$ for typical independent random strings, for which a description of one offers no assistance in describing the other; and intermediate values quantify partial correlation, such as that between a measurement record and the system it measured.

Because the shortest program may be arbitrarily difficult to find, statements about K hold up to additive constants reflecting fixed wrapper code; following Ebtekar and Hutter, we write $f \stackrel{+}{<} g$ for $f \le g + c$, $f \stackrel{+}{>} g$ for $f \ge g - c$, and $f \stackrel{+}{=} g$ for both, where c

depends only on fixed choices such as the universal computer, never on the strings involved.

7.2 Justification of K as an entropy measure

We now present three families of results that together justify treating $K(x)$ as *the* entropy of the individual state x.

Physical Negligibility of the Choice of Computer. The quantity K depends on the universal computer U, a dependence that may appear problematic for a proposed physical quantity. The dependence is, however, bounded by translation: for any two universal computers, a fixed interpreter converts programs for one into programs for the other, so the two versions of K differ by at most the interpreter's length, uniformly in x . To gauge the scale of this dependence, we convert to physical units: entropy in bits converts to thermodynamic entropy at $1 \text{ bit} = k_{\mathrm{B}} \ln 2 \approx 9.57 \times 10^{-24}$ joules per kelvin. Suppose that two programming languages were so dissimilar that translation between them required a 12-gigabyte interpreter, far larger than any real interpreter. Twelve gigabytes is about 10^{11} bits, so the two languages would assign every physical system entropies agreeing to within $10^{-12}\,\mathrm{J/K}$, far below experimental resolution for macroscopic systems. For all physical purposes, therefore, K may be regarded as computer-independent.

Agreement with the Shannon Framework in Its Domain of Validity. Let μ be any computable probability distribution. Then for *every* state x,

$$K(x \mid \mu) \stackrel{+}{<} \log \frac{1}{\mu(x)} = \hat{H}(x, \mu),$$

because one way to describe x, given access to μ, is to transmit its Shannon codeword (Section 2.3), and the shortest description can only be shorter. In the reverse direction, a counting argument based on the Kraft inequality shows that the inequality is nearly tight for nearly all samples: for any $\delta > 0$, with probability at least $1 - \delta$ over $X \sim \mu$,

$$K(X \mid \mu) \stackrel{+}{>} \log \frac{1}{\mu(X)} - \log \frac{1}{\delta}.$$

In other words, the optimal universal description of a typical sample is its Shannon codeword up to logarithmically small slack; atypical samples may improve upon the Shannon codeword, but only a δ-fraction can do so by more than $\log \frac{1}{\delta}$ bits. Averaging the two displays recovers Zurek's identity

$$H(\mu) \stackrel{+}{=} \langle\, K(X \mid \mu) \,\rangle_{X \sim \mu} :$$

the Gibbs–Shannon entropy is the mean Kolmogorov complexity of a sample, given prior knowledge of the distribution. Two specializations sharpen this correspondence: taking μ uniform on a finite set B shows that for any simply describable set B containing x, we have $K(x) \stackrel{+}{<} K(B) + \log |B|$, with near-equality for the vast majority of elements; taking B to be a Boltzmann macrostate (the set of all microstates sharing given macro-

variable values) recovers the Boltzmann entropy $\log|B|$ as an upper bound on K, achieved by typical members. Furthermore, the bound applies to *any* simple set rather than to classical macrostates alone: Ebtekar and Hutter's example is that the entropy of a bookshelf can be estimated by taking B to be the set of configurations compatible with the manner in which the books are sorted. The algorithmic entropy thus automatically considers every simple description of the state, classical macrovariables included, and charges the state for the least expensive among them.

These results resolve the three failure modes of Section 6.3. Where the ensemble picture is valid (a simple μ, concentration, and a typical x), we have $K(x) \approx \hat{H}(x,\mu) \approx H(\mu)$, so that algorithmic entropy reproduces the classical answers and nothing is lost. Where the ensemble picture fails, K continues to function correctly. A point mass μ on an intricate state no longer yields zero entropy, because $K(x)$ is large regardless of which distribution is mentioned alongside it, closing the "free knowledge" loophole (the knowledge is now assigned an explicit price, measured in description length). The robot's battery receives a definite per-state answer, $K(\text{charged state})$ or $K(\text{drained state})$, with the ensemble value revealed by Zurek's identity as a mere average of the two. Finally, a secretly patterned gas configuration is credited for its structure, $K(x) \ll \hat{H}(x,\mu)$, so that the sophisticated compression machine extracts exactly the work that the actual description length of x permits, in full accordance with the second law.

The Uncomputability of K and Its Necessity for a Second Law. A fundamental property of K is that no algorithm can compute $K(x)$ from x, a fact whose consequences for the second law we now examine. One direction of approximation remains available: by running ever more candidate programs for ever longer, one obtains a decreasing sequence of upper bounds converging to $K(x)$ (in the standard terminology, K is upper semicomputable), which is why real-world compressors provide genuine upper bounds on entropy. No algorithm, however, can certify large *lower* bounds on complexity: there is no procedure that, given x, verifiably reports "$K(x)$ is at least one million". This is Chaitin's incompleteness theorem, and its proof is a short self-referential argument: if such a certifier existed, then a short program could enumerate strings until it located one certified to have complexity at least a million and print it, thereby describing a string of supposed complexity at least one million bits with a few hundred bits—a contradiction.

At first sight, uncomputability appears to be a defect in a proposed physical quantity; it is in fact essential to the consistency of the framework. Suppose that thermodynamics were instead formulated with some entropy-like state function K' for which an algorithm *could* certify large values. A short program p could then search for and output a state x^* certified to satisfy $K'(x^*) \gg 0$. However, any short program that can *construct* x^* can also *erase* it: one runs p to produce a second copy of x^* alongside the existing one, uses the copy to reversibly cancel the original (a controlled-NOT for every bit, exactly as in Example 5.1), and then runs p in reverse to restore the auxiliary state. The net effect of a constant-size machine is to take the world from a state containing x^*

, with K'-entropy $\gg 0$, to a state not containing it, with K'-entropy ≈ 0. A small, simple, reversible machine that can destroy unbounded amounts of "entropy" at will constitutes a perpetual motion machine with respect to K', and consequently no second law can hold for any computable notion of entropy. The uncomputability of K is precisely what closes this loophole: it guarantees that no simple machine can systematically recognize which states are simple but disguised, and therefore that none can systematically remove the disguise. We note the parallel with Appendix B.8: knowledge that cannot be obtained by any procedure cannot subsequently be expended as a thermodynamic resource. In turn, this justification of K as an objective entropy serves as the foundation for the algorithmic second law, to which we now turn.

7.3 The algorithmic second law

We now return to the physical setting of Section 4: a coarse-grained state space with equal-volume cells, evolving as a Markov chain with doubly stochastic, *computable* transition probabilities (the assumption that the laws of physics admit a short program constitutes a complexity-theoretic version of the Church–Turing thesis, and we choose our reference computer so that the dynamics are simply describable). In this setting the algorithmic entropy of the system in state x is simply $K(x)$, and it obeys a second law descending from Levin's law of *randomness conservation*.

Theorem 7.2 (Algorithmic Second Law; Levin, Ebtekar–Hutter, Stated Informally). Let (X_t) be a computable, doubly stochastic Markov chain modeling an isolated system's coarse-grained evolution, and let $s < t$ be two times. Then for every $\delta > 0$, with probability greater than $1 - \delta$,

$$K(X_s) - K(X_t) \stackrel{+}{<} K(t-s) + \log \tfrac{1}{\delta}.$$

The theorem asserts that *the description complexity of an isolated system's state almost never decreases, except by a precisely priced fluctuation allowance*. It improves upon the ensemble second law (Theorem 4.1) in two respects that are central to our purposes: it holds for each individual trajectory with high probability, rather than merely for the average of an ensemble, and it requires no initial distribution μ whatsoever, removing the exogenous ensemble exactly as Section 6 required.

Each of the two terms in the allowance admits a precise interpretation, and each is quantitatively negligible in physical terms.

The term $\log \frac{1}{\delta}$ prices *chance* fluctuations: rare trajectories along which entropy temporarily falls. Its scale is set by the rarity of the fluctuations one chooses to consider. If events of probability $\delta = 2^{-1000}$ are tolerated, the permitted decrease in entropy is approximately one thousand bits, which in physical units (we recall that 1 bit $\approx 10^{-23}\,\mathrm{J/K}$) is around $10^{-20}\,\mathrm{J/K}$, twenty orders of magnitude below anything measurable. The statistical character of the second law is therefore genuine but quantitatively trivial at macroscopic scales.

The term $K(t-s)$, the complexity of the *elapsed time*, prices *deterministic recurrences*, and its necessity is subtle. The Poincare recurrence theorem guarantees that an isolated finite system eventually returns arbitrarily close to its initial low-entropy state, so entropy cannot literally always increase. The theorem survives because recurrence times are astronomically long and arithmetically structureless: a system whose recurrence occurs at step $t-s = 17{,}382,\ldots$ (a number comprising some 10^{20} digits) revisits low entropy only at moments whose timestamps are themselves about as complex as the state, and the $K(t-s)$ allowance covers exactly this cost. For any time interval that can be *simply specified* (10^9 years, 2^{100} Planck times), $K(t-s)$ is at most a few hundred bits, so that entropy, regarded as a function of simply specified times, increases monotonically up to negligible slack.

One special case recurs throughout the demon analysis below, and we record it separately.

Corollary 7.3 (Invariance of Entropy under Deterministic Dynamics). If the evolution over the interval is a computable bijection f with a short description (for instance, a few steps of simply describable reversible dynamics), then

$$K(f(x)) \stackrel{+}{=} K(x).$$

The reason is immediate from the compression picture: given the dynamics, a description of x is a description of $f(x)$ and vice versa, so the two complexities differ by at most the (small) complexity of f itself. The corollary carries an important consequence: *substantial entropy production requires randomness*. A deterministic, simply describable evolution can shuffle states but cannot increase their complexity by more than a constant; in classical physics, the randomness that drives entropy production is supplied by the coarse-graining of chaotic microdynamics (Section 4.3), which continually injects effectively fresh random bits into the coarse description. A useful picture is the following: a doubly stochastic evolution is equivalent to drawing a random bijection at each step (for instance, by tossing coins), and entropy can increase by at most the number of coins tossed, and only to the extent that the coins are independent of the state. The Shannon-level second law is blind to this distinction between deterministic reshuffling and genuine randomization, whereas the algorithmic version registers it precisely.

7.4 An exact analysis of Maxwell's demon

Having established the algorithmic second law and its deterministic special case, we now revisit the demon, this time in the form of a theorem rather than of an informal narrative, and recover the Touchette–Lloyd bound as a statement about individual physical states. We take the demon's memory and the gas to be jointly coarse-grained, writing joint states as pairs (memory contents, gas state). The demon's memory begins in a blank reference state 0, and the gas begins in some state x.

Complete Measurement Followed by Erasure. In the idealized case, the demon performs a complete measurement, reversibly copying the gas's state into memory:

$$(0, x) \;\mapsto\; (x, x),$$

and then, using its copy as the control of a conditional operation (a string of controlled-NOTs, Example 5.1), reversibly resets the gas to a reference state:

$$(x, x) \;\mapsto\; (x, 0).$$

Each map is injective on the states that can actually occur (the first on pairs with blank memory, the second on pairs whose two slots agree), hence extendable to a bijection of the joint state space, and each is simply describable. Corollary 7.3 therefore applies to both steps, and the *total* complexity remains unchanged throughout:

$$K(0, x) \stackrel{+}{=} K(x, x) \stackrel{+}{=} K(x, 0) \stackrel{+}{=} K(x).$$

We now examine what this chain of equalities asserts. The gas has passed from complexity $K(x)$ to complexity $\stackrel{+}{=} 0$, which constitutes a complete Type-2 optimization. The joint complexity never changed, so the second law was never threatened at any step, without any need to "complete a cycle" or to invoke ad hoc accounting. The balance is now carried entirely by the demon's memory, which holds a record of complexity $K(x)$; the gas's erasure was permitted *precisely because a copy existed*, since the operation "clear slot two given that slot one holds a copy" is injective. The step that the second law forbids is the clearing of the *last* copy:

$$(x, 0) \;\mapsto\; (0, 0)$$

is either not injective (if it is to work for every x) or not simply describable (if hardwired for one particular complex x, the wiring itself would have complexity $K(x)$, and Corollary 7.3 charges for the complexity of the dynamics). To recover its memory, the demon must clear that last copy, irreversibly destroying $K(x)$ bits of complexity; it cannot do so without cost, and the only route consistent with the second law is to export those $K(x)$ bits into the environment as Type-1 waste (Section 5.2). The classic resolution of the demon paradox—that the demon's own information processing is what rescues the second law—emerges here not as an additional postulate but as a direct consequence of the complexity bookkeeping.

Partial Measurement and the Algorithmic Touchette–Lloyd Bound. A realistic demon measures only a small part of the gas state. Let the measurement be $m(x)$, where m is any simply describable (possibly random, possibly many-to-one) function of the gas state—a few bits concerning a single approaching molecule, for instance. The demon records the measurement, then employs the record as a control to drive the gas from x to some new state y:

$$(0, x) \;\mapsto\; (m(x), x) \;\mapsto\; (m(x), y).$$

Whatever feedback protocol the second step implements, provided that it is a simply describable mixture of reversible operations, the algorithmic second law applied to the *joint* system gives

$$K\big(m(x),x\big) \stackrel{+}{<} K\big(m(x),y\big).$$

Expanding both sides with the chain rule ($K(m,\cdot) \stackrel{+}{=} K(m) + K(\cdot \mid m)$, with the technical refinements recorded in Definition 7.1), subtracting $K(m(x))$ from both sides, and rewriting conditional complexities via mutual information ($K(x \mid m) \stackrel{+}{=} K(x) - I(m:x)$), we obtain

$$K(x) - I\big(m(x):x\big) \stackrel{+}{<} K(y) - I\big(m(x):y\big) \leq K(y),$$

where the final step uses the nonnegativity of algorithmic mutual information. Rearranging yields

$$\boxed{K(x) - K(y) \stackrel{+}{<} I\big(m(x):x\big).}$$

That is, *the gas can lose at most as much entropy as was measured from it*. The budget is, moreover, exactly achievable: if the demon expends its correlation completely and reversibly (so that $K(m(x),x) \stackrel{+}{=} K(m(x),y)$, with no entropy produced, and $I(m(x):y) \stackrel{+}{=} 0$, with no correlation left unspent), then the displayed inequalities collapse to

$$K(x) - K(y) \stackrel{+}{=} I\big(m(x):x\big).$$

We now place (1) beside Theorem B.2 (Appendix B) and compare the two statements term by term. In each, the left-hand side is the entropy reduction of the steered system, and the right-hand side is the mutual information between the controller's record (action, measurement) and the system's state. The Touchette–Lloyd theorem is the ensemble (Shannon) version of the constraint, speaking of distributions, averages, and a blind baseline supplied by the dynamics. Inequality (1) is the single-shot, individual-state (algorithmic) version, speaking of the single gas state actually confronting the demon, without reference to any ensemble, the role of the blind baseline being played by the additive constant, which absorbs what the simply describable dynamics can accomplish unaided. It is simultaneously the rigorous form of Type-3 optimization from Section 5: the copy step purchases $I(m(x):x)$ and the control step expends it, bit for bit.

One final reading of the demon's situation will prove central to the concluding section. After the measurement, two distinct entropies attach to the same gas: its *objective* entropy $K(x)$, and its entropy *from the demon's perspective*, the conditional complexity

$$K\big(x \mid m(x)\big) \stackrel{+}{=} K(x) - I\big(m(x):x\big),$$

which is smaller by exactly the measured information. The demon's optimization power over the gas is precisely the wedge between the objective and the subjective entropy. The subjective element has therefore not been eliminated from thermodynamics but rather assigned a physical location: the "subjective distribution" of the older formalism has become a physical conditioning on a physical record, carried in a memory that occupies physical space and obeys the second law.

8. Knowledge as a physical resource: optimization for embedded agents

Having developed the algorithmic account of entropy and examined the channels through which entropy can be displaced, we now synthesize the preceding sections into a single unified picture: for an embedded agent, knowledge about the environment constitutes a physical resource, and this resource is the same quantity as the agent's capacity to optimize the environment.

8.1 From exogenous to endogenous knowledge

We begin by comparing how the two frameworks account for an agent's knowledge of a system.

In the Gibbs–Shannon framework, what an agent knows about a system is encoded in the distribution μ, and μ is supplied from outside the physics, so that entropy, work capacity, and therefore optimization capacity are all defined relative to it. Two problems follow from this exogenous treatment: first, if we change μ, the physics appears to change (the problem of Section 6.4); second, μ is entirely unconstrained, so knowledge can be introduced at no physical cost (the loophole of Section 6.3). At equilibrium neither problem arises, because there the appropriate μ is both forced and simple; the problems become acute precisely when the agent's knowledge is the component of interest.

For an embedded agent no such outside ledger exists. Its knowledge of the environment must be stored physically, in a memory composed of the same matter as the environment and governed by the same laws, and algorithmic thermodynamics enables us to handle this knowledge directly. Writing a for the physical state of the agent's memory and s for the physical state of the environment, "what the agent knows about the environment" is the algorithmic mutual information

$$I(a : s) \stackrel{+}{=} K(s) - K(s \mid a),$$

the number of bits by which the agent's memory shortens the description of the environment. No distribution appears in this expression: knowledge has become an objective relation between two pieces of matter, as much a fact about the world as a distance or a voltage. Subjective beliefs are not discarded but rather *implemented*: an agent

whose memory holds a good model of the environment is one whose state has high $I(a : s)$, and $K(s \mid a)$ is the environment's entropy *as that agent finds it* (the environment's entropy as evaluated from the demon's perspective, Section 7.4).

The Correspondence Between Knowledge and Optimization Capacity. A fundamental insight emerging from this framework is that an agent which understands its environment more thoroughly can effect correspondingly greater change upon it, and the framework renders this intuition precise by joining two bridges: the first passing from the agent's beliefs to mutual information, and the second from mutual information to optimization.

From beliefs to mutual information. Suppose that the agent's memory a encodes a computable belief q about the environment (formally, a short fixed program reconstructs q from a). Given a, one way to describe the true state s is to recover q and record the Shannon codeword of s under q, which has length approximately $\log \frac{1}{q(s)}$ bits (Section 2.3). Since the shortest description can be no longer than this particular one,

$$K(s \mid a) \stackrel{+}{<} \log \frac{1}{q(s)}, \qquad \text{and hence}$$

$$I(a : s) \stackrel{+}{=} K(s) - K(s \mid a) \stackrel{+}{>} K(s) - \log \frac{1}{q(s)}.$$

The more probability the belief assigns to the true environment, the shorter this codeword becomes, and the more mutual information the memory carries about the world.

From mutual information to optimization. The algorithmic Touchette–Lloyd bound (1) establishes that the agent can remove at most $I(a : s)$ bits of complexity from the environment, and that this amount is attainable. Combining the two bridges yields

$$\text{optimization the agent can perform} \stackrel{+}{>} K(s) - \log \frac{1}{q(s)},$$

so the agent can drive the environment down to a residual complexity of approximately $\log \frac{1}{q(s)}$, which is precisely the environment's entropy as the agent's belief regards it. It follows that *the more probability the agent's model assigns to the truth, the more of the environment it can optimize*, at a rate fixed by the bookkeeping rather than by any modeling choice.

This correspondence constitutes Bayesian learning expressed in physical terms. As the agent collects evidence and updates its belief, $q(s)$ rises, the codeword length $\log \frac{1}{q(s)}$ and the residual $K(s \mid a)$ shrink, $I(a : s)$ grows, and the optimization within reach grows with it. In the limit of certainty $K(s \mid a) \stackrel{+}{=} 0$ and $I(a : s) \stackrel{+}{=} K(s)$, and the agent can in principle steer the environment to vanishing residual complexity. Updating a belief toward the truth and accumulating spendable, physically stored knowledge are therefore one and the same process, constituting the single-state form of the Touchette–Lloyd result of Appendix B and the precise content of the Type-3 channel of Section 5.

Two further properties complete the characterization of $I(a : s)$ as a resource.

- It is *consumed* when spent: optimization in this setting is the Type-3 channel (Section 5), in which the shared correlation is consumed, so that continued optimization requires the agent either to measure again (Type 2) or to export waste (Type 1).
- It *decays*: correlations between two systems cannot grow without interaction (a consequence of the algorithmic second law, Section 7.3), so that if the environment changes while the agent's records remain fixed, $I(a : s)$ falls, and correlation that has been allowed to decay can no longer be spent.

This analysis establishes that an embedded agent's knowledge about the world is the same physical quantity as its capacity to optimize the world beyond blind baselines: an agent that knows more can accomplish more, at a fixed exchange rate of one bit of optimization per bit of correlation. In turn, this identification of knowledge with optimization capacity invites a refinement of each of the three channels, to which we now turn.

8.2 Refinements of the three channels via universal computation

Having introduced algorithmic entropy, we now demonstrate that each of the three channels admits a refinement, and in every case the refinement turns on the compressibility of an individual state—a quantity that the ensemble account cannot express.

Type 1. Before exporting waste into the environment, an agent should determine whether the waste is genuinely as random as it appears. If the waste possesses compressible structure (and the output of any computation does, by Corollary 7.3), the agent can compress it first and export only $K(\text{waste})$ bits of genuine entropy rather than its raw size. This compression must occur *before* the waste mixes into the environment, because mixing destroys the structure: once the structure is lost, no compressor can recover it.

Type 2. The memory cost of recording a measured state x is not the raw length of the transcript but its complexity $K(x)$: records can be stored in compressed form, so a memory can absorb more optimization than its nominal size suggests. Moreover, $K(x)$ constitutes a lower bound, in that no method stores x in fewer bits.

Type 3. The steering budget is the mutual information (1), and computation offers a second route to acquiring it: if the environment's state is simple ($K(s)$ small), a short program generates it, so the agent can come to know it (acquiring $I(a : s) \approx K(s)$) by *computation alone*, with almost no measurement. Simple worlds are therefore informationally inexpensive to characterize and consequently inexpensive to optimize.

These three refinements converge in a single fact, noted by Daniel C and Ebtekar: the complexity $K(s)$ of the optimized subsystem is simultaneously the smallest memory that can absorb it (Type 2) and the smallest knowledge that can steer it (Type 3). The cost of recording a state and the knowledge required to steer it are the same number of bits, because both quantities equal the length of its shortest description.

8.3 Dissolution of the apparent subjectivity

We now return to the puzzle of Section 6.4: does the demon's capacity to perform work change when *our* beliefs about its memory change? The answer comprises two parts, of which the second provides the resolution.

First, the demon's capacity is objective: its remaining capacity to optimize is its free memory, namely the size of the memory minus the *description complexity* of its current contents. If those contents are compressible, the demon can compress them (a reversible and costless operation by Corollary 7.3) and thereby recover the space; if they are incompressible, the space is irreducibly occupied, because freeing it would require destroying information, which reversibility forbids. In either case the capacity depends only on the physical state of the memory, and no observer's beliefs enter the formula.

Second, the appearance of belief-dependence nevertheless reflected a genuine physical relation. When we update our beliefs about the demon's memory, that update constitutes a physical change in our own brains, which thereafter share more algorithmic mutual information with the demon's memory. This shared information is itself a Type-3 resource: in principle we could spend it to assist in compressing the demon's memory, freeing capacity that the demon alone could not reach. The demon's situation therefore genuinely differs after our update, but only relative to the enlarged system that now includes our correlated brains, and the difference is exactly the correlation we have gained. What the old formalism recorded as a mysterious observer-dependence of entropy, the new formalism records as an ordinary physical relation between observer and observed, dissolving the apparent subjectivity into objective correlation.

8.4 Thermodynamic implications for optimizing systems

Finally, we reconsider the optimizing systems of Section 3 in the light of these results. An optimizer drives the system upon which it acts from a broad range of configurations into a narrow target set, and the question arises of what the existence of such an optimizer implies thermodynamically.

The funneling constitutes coarse-grained entropy reduction in a subsystem, so the global accounting must remain consistent through some mixture of the three channels (Section 5): waste exported to the surroundings (Type 1), records kept in memory (Type 2), or pre-existing correlation spent (Type 3). The principal implication is a selection-theorem-like conclusion: if the funneling exceeds the blind baseline of the environment's own dynamics, then by the Touchette–Lloyd theorem (Theorem B.2, Appendix B) and its algorithmic form (1), the system must carry mutual information with what it steers, which is a necessary (though, by Appendix B.8, not sufficient) ingredient of a world model. At least this much modeling is therefore forced by the bookkeeping rather than attributed by an observer. This conclusion constitutes the thermodynamic counterpart of the observer-independence established in Section 3.5: the presence of a world model, like the presence of an optimizer, is an objective fact about the system, certified by the entropy it removes rather than attributed by an external observer.

9. Summary and conclusions

As developed over the preceding sections, our aim in these notes has been to provide a physical, information-theoretic characterization of optimization and to trace its consequences through thermodynamics; we now summarize the principal conclusions. An optimizer is a physical entity whose presence drives the system it acts upon, from a broad range of initial conditions and despite perturbations, into a narrow target: conditioning on the optimizer renders an observer's uncertainty about the system's final state small, endowing the system with a convergent attractor. The amount of optimization is quantified by the entropy reduction the optimizer produces in that system, namely the number of bits by which it narrows the system's possibilities, and the designation of an entity as an optimizer requires no claim about its internals or goals, only about its effect on the system it steers. Even an observer concerned solely with prediction has an objective, information-theoretic reason to distinguish optimizers: because the system converges to the target, the target constitutes a short, robust description of the state in which the system terminates, forecasting the macroscopic outcome for a very small fraction of the bits that a specification of the initial conditions would require, and without any knowledge of the initial conditions at all. This establishes optimization as an observer-independent feature of the world rather than a stance projected by a particular observer, thereby resolving the concern that intelligence is merely task-relative.

Having characterized optimization, we situated it within the constraints imposed by physics. Microscopic physics is reversible, and the second law of thermodynamics follows from that reversibility once the dynamics are coarse-grained: a doubly stochastic Markov step cannot decrease ensemble entropy (Theorem 4.1). Global entropy reduction is consequently impossible; all optimization is local, and the global accounting must remain consistent, with every local reduction compensated elsewhere. Moreover, we established that thermodynamic laws concern information rather than molecules: they bind whenever a designer cannot observe a system, cannot destroy information, and faces a conservation constraint. In Wentworth's coin world, work extraction is data compression, no work can be extracted from a single maxentropic pool, and the fuel of every engine is negentropy—the gap between a system's entropy and its constrained maximum—confirming that the currency of thermodynamic advantage is informational rather than material.

Building on this foundation, we quantified the informational cost of steering beyond "blind" dynamics. The Touchette–Lloyd theorem (Appendix B) bounds the entropy reduction achievable by any policy by the blind-dynamics baseline plus the mutual information between action and environment, $\Delta H \leq \Delta H^{\max}_{\text{blind}} + I(X;A)$, so that observed optimization in excess of the baseline certifies the presence of modeling; the bound is a constraint rather than a guarantee, since information can be wasted. Complementing this result, we established that an embedded agent has exactly three channels through which it can reduce the entropy of a subsystem: exporting the entropy into the environment as heat, absorbing it into memory by measurement, or spending

pre-existing mutual information with the subsystem. Maxwell's demon operates through the second of these channels, its measure-then-steer operation decomposing into the purchase of correlation followed by its expenditure.

Motivated by the demands of embedded agency, we then examined the foundations of the entropy concept itself. The Gibbs–Shannon entropy is defined relative to an exogenous, subjective ensemble, and this dependence fails precisely on the states of which agents are composed: intricate knowledge is assigned zero cost, non-concentrated ensembles yield averages true of no actual state, and atypical structure receives no credit. Equilibrium thermodynamics is the special regime, characterized by simple, concentrated, typical ensembles, in which this failure never manifests.

To address these limitations, algorithmic thermodynamics replaces the ensemble with Kolmogorov complexity, so that entropy becomes an objective property of the individual state, agreeing with the classical entropies within their domain of applicability and extending beyond it. The algorithmic second law holds per trajectory with exactly priced fluctuation allowances (the allowance $K(t-s)+\log\frac{1}{\delta}$ covers Poincare recurrence and chance decreases in entropy); deterministic simple dynamics cannot create entropy; and the demon obeys $K(x)-K(y)\overset{+}{<} I(m(x):x)$, the single-state form of the Touchette–Lloyd bound. This objective entropy dissolves the subjectivity problem, being a property of the state rather than of any observer, and the uncomputability of entropy is essential to the framework: a computable entropy admits a perpetual-motion machine.

Finally, for embedded agents knowledge is endogenous: an agent's probabilistic model of its environment is physically encoded in memory, its content is the algorithmic mutual information between the agent's memory and the world, and that mutual information is the consumable, perishable resource that funds optimization beyond blind baselines. Agents possessing greater knowledge of the world are accordingly capable of exerting greater influence upon it, at a fixed exchange rate of one bit of optimization per bit of correlation. Through these results, we have established an account of optimization that is grounded in physics and stated in the language of information, providing both an observer-independent characterization of what optimizers are and a quantitative accounting of what their operation must cost.

→ *Appendix A. A thermodynamics of biased coins: the generalized heat engine*: Supplementary Material, p. 532

→ *Appendix B. The informational cost of steering: the Touchette–Lloyd theorem*: Supplementary Material, p. 537

→ *Sources and further reading*: Supplementary Material, p. 543

NOTES

1. The entire framework developed in these notes generalizes to coarse-grainings with unequal cell volumes: entropy is then measured *relative to* the stationary volume measure π, replacing $H(\mu)$ by $H_\pi(\mu):=\sum_x \mu(x)\log\frac{\pi(x)}{\mu(x)}$ and the codelength by $\hat{H}_\pi(x,\mu):=\log\frac{\pi(x)}{\mu(x)}$. This generalization is stand-

ard, but it is not required for any of our subsequent results; the assumption of equal cells simplifies the notation at no conceptual cost.

2. We record two technical refinements once and thereafter suppress them. First, programs are required to be *self-delimiting*: no valid program is a prefix of another, so that each program announces its own end. This convention makes program lengths behave like codeword lengths; in particular they satisfy the Kraft inequality $\sum_x 2^{-K(x)} \leq 1$, which is precisely the property permitting $2^{-K(x)}$ to serve as a (sub)probability distribution. Second, the chain rule for K holds in the form $K(x, y) \stackrel{+}{=} K(y) + K(x \mid y, K(y))$, with the complexity of y itself appearing in the conditioning; consequently some identities below, including the second expression for $I(x : y)$, implicitly carry such terms. Every statement we make is correct with these refinements installed, and none of these refinements is significant at physical scales.

CHAPTER D.5.1

Decision Theory

Daniel C, Satya Benson

Safety proofs require decision-theoretic assumptions invariant under self-modification. Covers ideal choice in exotic environments (copies, simulations) and multiagent cooperation via open-source game theory and safe Pareto improvements.

Source: iliad-intensive.org/agency/decision-theory · CC BY 4.0 · Slides: online

1. Prerequisites

- Basic probability and expected value; comfort reading pseudocode.
- Helpful but not required: Löb's theorem (see Chapter D.4.1; it underlies FairBot and the tournament's cooperation results) and a first acquaintance with Bayesian networks and the causal do-operator (re-introduced briefly below).

WHAT YOU'LL LEARN

(What.) By the end of this chapter, you should be able to explain why decision theory is hard for an *embedded* agent (its action is just another fact about the world, so counterfactuals must be constructed, not read off) and can state the four main theories and what each gets right and wrong: CDT (intervene), EDT (condition), FDT (choose your decision function's output), and UDT 1.0/1.1 (act on the policy you would have committed to in advance). You can work the canonical problems (Newcomb, smoking lesion, counterfactual mugging, twin prisoner's dilemma) and say which theory each one separates. You will understand the multiagent layer: open-source game theory and Löbian cooperation (FairBot), commitment races, and safe Pareto improvements. You can apply all of this by designing a bot for the open-source Prisoner's Dilemma tournament in Chapter D.5.2.

(Why.) To make a superintelligence safe we will likely prove safety properties under assumptions about *how the agent decides*. Decision theory is a *reflectively-consistent degree of freedom*: unlike a mistaken factual belief, a bad decision theory is not automatically corrected as an agent gets smarter, so a load-bearing assumption ("the agent is CDT") can silently fail if the agent self-modifies. The multiagent failures (commitment races, conflict, exploitable cooperation) are direct AI-safety concerns, and safe Pareto improvements are one of the few constructive tools against them.

2. Content

→ *Fast-track route*: Supplementary Material, p. 545

2.2 Main content

This chapter covers single-agent decision theory and its primary readings; the next chapter (D.5.2) turns to multi-agent decision theory and an open-source tournament.

2.2.1 Single-agent decision theory

Why decision theory, and why it is hard. For a *dualistic* agent cleanly separated from the world, the action is a free variable and the rule is just `a* = argmax_a E[U | do(A=a)]`. For an *embedded* agent the action is itself a fact about the world (predictors may have modelled it, copies may share it), so "what happens if I act differently" is a counterfactual that must be *constructed*. The decision theories differ in how they construct it.

EDT and CDT: condition vs intervene.

- *Evidential decision theory* conditions on the action as evidence: `a* = argmax_a E[U | A=a]`. This treats the action as news about everything correlated with it.
- *Causal decision theory* intervenes: `a* = argmax_a E[U | do(A=a)]`, severing the arrows *into* the action in a causal (Bayesian-network) model, so the action is news about its effects only.
- They split on the canonical cases. On **Newcomb's problem** (a predictor fills a box based on your disposition), EDT one-boxes and wins $1,000,000; CDT two-boxes (the prediction is already made) and wins $1,000. On the **smoking lesion** (a common cause produces both a disposition to smoke and the disease), EDT wrongly abstains (smoking is bad *evidence* though not a *cause*); CDT correctly smokes. Neither rule handles both, which points to a third kind of dependence.
- *(Aside, for the mathematically inclined.)* The two rules drop out of two ways to model the agent's interaction history: CDT is a chronological-semimeasure predictor (actions are conditioned-on inputs, never predicted, exactly the do-operator), while EDT is a joint predictor over actions and observations (conditioning on an action updates beliefs about which world and which policy you are). This aside can be skipped without loss.

FDT: choose your function's output. Functional decision theory reframes *what you are choosing*. You run a fixed decision function; your action is its output, and everything that depends on that function (predictors who modelled it, copies running it, simulations of it) moves together with the output. FDT asks "which output of this decision function, given everything that depends on it, yields the best outcome?" It one-boxes on Newcomb and smokes on the smoking lesion. Its signature case is the **twin prisoner's dilemma**: two copies of the same agent share the same logical output

(*subjunctive dependence*), so FDT cooperates (each gets $3) where CDT defects (each gets $1). Causal dependence is a special case of subjunctive dependence; mere correlation (the lesion) is not.

UDT: act on the policy you would have committed to.

- *UDT 1.0* chooses each action as the one a prior-stage self would have committed to: `choice(o) = argmax_a E[U | choice(o)=a]`. On **counterfactual mugging** (a coin you have already seen land the wrong way, where paying in this branch is what makes paying profitable across branches), the updateful agent refuses and the updateless agent pays, because ex ante paying is worth `(R-c)/2`. Updatelessness is not ignorance; it is refusing to update away a commitment that is good in expectation, which makes it reflectively stable.
- *UDT 1.1* optimizes the whole *policy* (a map from observations to actions) rather than each action separately, then applies it: `S* in argmax_{S:O->A} E[U | choice=S]`. This is needed when copies must coordinate their actions across branches, which per-observation optimization can get wrong.

Where this is heading. The shift across CDT/EDT to FDT to UDT 1.0 to UDT 1.1 is a shift in *what you are choosing*: an action, then a function's output, then an output you do not update away, then a whole policy. Two hard problems remain: UDT pays real utility in the actual branch and is only as good as its prior, and *logical updatelessness* (what is the right "prior" when some uncertainty is mathematical?) has no clean answer. The **5-and-10 problem** shows the deeper trouble: a naive proof-searching agent can be driven by a spurious Löbian proof to take the worse action, because counterfactuals over one's own action break down. The frontier goal is to formalize decision problems as programs and ask which theory is *optimal* over a well-defined class of "fair" problems (where the world depends only on the agent's input/output behaviour); the two demands such a criterion forces (coordinate across calls; detect an isomorphic copy of yourself) are exactly UDT 1.1 and FDT turned into a specification.

2.2.2 Readings and discussion

Readings (arranged roughly chronologically; the FDT paper is the primary reference):

- Functional decision theory: a new theory of instrumental rationality (chapters 1-5).
- Towards a new decision theory.
- Updateless decision theory.
- Conceptual problems with UDT and policy selection.
- Pitfalls of building UDT agents.

Discussion prompt (transparency and cooperation): consider two agents A1 and A2 whose code, preferences, and decision algorithms become perfectly transparent to each other at time T (state your interpretation of "transparent" where it matters). Under these conditions, when might A1 and A2 end up in a *Pareto-inefficient* outcome? Where inefficiency looks plausible but a non-obvious argument rules it out, give that argu-

ment. (This is exactly the open-source game theory setting of the next chapter.) Related readings: When would AGIs engage in conflict? and Individually incentivized safe Pareto improvements in open-source bargaining.

→ *3. Learn more*: Supplementary Material, p. 545

CHAPTER D.5.2

Open-Source Game Theory

Daniel C, Satya Benson

When agents can read each other's code, cooperation needs neither repetition nor trust — but the same transparency starts races to commit first. Conditional commitment, safe Pareto improvements, and a tournament.

Source: iliad-intensive.org/agency/open-source-game-theory · CC BY 4.0 · Slides: online

1. Open-source games, commitment races and safe Pareto improvements

Open-source game theory. When players can read each other's source code, cooperation becomes possible without repetition or prior trust. **FairBot** cooperates if and only if it can prove its opponent cooperates with it; two FairBots cooperate by Löb's theorem (from `□(□C -> C) -> □C`). It is unexploitable but brittle (it relies on a proof system and exact source). **Epsilon-grounded bots** (Oesterheld) replace proofs with simulation: cooperate unconditionally with small probability epsilon, else simulate the opponent and copy its move; this terminates almost surely, is a Nash equilibrium, and is robust. The unifying idea is *conditional commitment*: "I commit to X conditional on you committing to Y." Conditional makes it unexploitable; commitment makes it legible. FairBot is exactly this.

Commitment races. Among consequentialists who can commit, there is an incentive to commit *first*: the first mover can pick the point on the bargaining frontier best for itself and leave the responder to best-respond. So "the best response is not the best response", and agents race to commit before they even finish learning, risking incompatible lock-in or wasteful conflict (including threats and s-risks). Moving first "in logical time" is a genuine safety concern, which makes *avoiding* commitment races a safety goal. The deeper obstacle is *entanglement*: each agent optimizes against a distribution over the *counterfactual* programs the other might submit, so punishing a Pareto improvement means forgoing utility against the opponent's *actual* program; the incentive to make improvements is entangled with those counterfactuals, and this persists even with full conditional commitment.

Safe Pareto improvements (SPI). An SPI modifies the agents' default (conflict-prone) strategies so that *every* player is guaranteed at least as well off regardless of the opponent, a guaranteed weak Pareto improvement (Oesterheld-Conitzer; DiGiovanni-Clifton-Macé). In the open-source/program setting, each player individually prefers to

use renegotiation-based SPIs, and the guaranteed floor is the *Pareto Meet Minimum* (your lowest efficient payoff), which is tight under a *participation-independence* assumption. That assumption can fail (agents may become hawkish when conflict is cheap), the "cheating" problem. *Entanglement-free SPI* (a research-frontier idea) restructures the interaction into two stages (exchange renegotiation programs, update on the actual one, then submit defaults), which breaks the entanglement and drops participation independence, at the cost of the tight floor.

2. Exercise: an open-source tournament

The game. The one-shot Prisoner's Dilemma, with the twist that your program can read the opponent's program before deciding (source is open). Payoffs (you, them): mutual cooperation `(C,C)` gives 3 each; mutual defection `(D,D)` gives 1 each; defecting against a cooperator gives 5 to the defector and 0 to the cooperated. This satisfies `T > R > P > S` ($5 > 3 > 1 > 0$) and `2R > T + S` ($6 > 5$).

Two leagues.

- *League A (one-shot, open source).* Bots see the opponent's source and decide once. This is the program-equilibrium / Löbian-cooperation setting.
- *League B (iterated, history only).* Bots see the move history over a hidden number of rounds (around 100-200) and cannot read source. This is the classic reciprocity setting (Tit-for-Tat and relatives).

Writing a bot. Write your bot in Python, pseudocode or careful English, with a short rationale explaining its decision-theoretic approach. In Python, a bot is an `Agent` class:

```
class Agent:
def move_oneshot(self, opp) -> Move:                  # League A
def move_iterated(self, my_hist, opp_hist, t) -> Move: # League B
```

In League A a bot may query: `opp.move_against(SELF)` (the opponent's move against you), `opp.move_against(DEFECT_BOT)`, `opp.move_against(COOP_BOT)`, and `opp.source` (the opponent's canonical source). League B provides only the two history lists and the round index `t`.

How circular queries resolve. Two source-reading bots can each ask "what does the other do against me?", which is circular. Rather than simulate the recursion, the engine treats every "what does the opponent do against X?" query as a *provability* question and computes the fixed point directly, the same Löbian move that lets two FairBots cooperate. When no stable resolution exists, it applies a *no-wishful-thinking* rule: if cooperation cannot be established, default to D.

Reference bots. League A: CooperateBot, DefectBot, FairBot, PrudentBot, CliqueBot, RandomBot. League B: AllC, AllD, Random, TitForTat, GrimTrigger, GenerousTitForTat, Pavlov, TitForTwoTats. These are natural opponents to test against.

Scoring. Round-robin: every bot plays every other bot, including a copy of itself, and ranking is by total points accumulated across all matchups (not head-to-head wins). League B matches add about 2% move-flip noise and a randomized hidden length, so brittle strategies are penalized.

What good looks like. A strong submission embodies a clear decision-theoretic concept (commitment, transparency, the limits of self-reference, functional decision theory) and explains it, rather than chasing the leaderboard. The standout lesson is usually that FairBot-style conditional cooperation beats both naive cooperation (exploited) and naive defection (misses mutual cooperation) in League A, while robustness to noise dominates in League B.

→ *3. Learn more*: Supplementary Material, p. 546

CHAPTER D.6

Instrumental Convergence

Leon Lang (Iliad), based on work by Alex Turner et al.

We discuss a simple mathematical formalization of what it means to "seek power" in a Markov decision process (MDP), and conditions under which such behavior emerges.

Source: iliad-intensive.org/agency/power-seeking · CC BY 4.0 · Slides: online

Prerequisites.

A basic familiarity with reinforcement learning is sufficient.

WHAT YOU'LL LEARN

You will understand the proofs for power-seeking tendencies based on Alex Turner's work, and be able to critically examine the weaknesses and implications of this work. This work is important since power-seeking behavior is one of the most severe risks that AI may pose, and this may be the most rigorous work to date establishing it under clear assumptions and definitions.

AUTHOR'S NOTE

I think the setup I chose (together with Claude) in the exercise sheet has some advantages compared to Turner's original treatment of the results:

- The use of an invariant measure avoids "orbit counting" and grounds everything in the more justified notion of probabilities of various outcomes.
- We work with *a subgroup* of the symmetric group, and thus avoid the unjustified assumption that the measure on reward functions is invariant under arbitrary symmetries (which it obviously isn't!)
- The notion of a dynamics embedding grounds the notion of "more options" into actual MDP dynamics.
- The use of score functions allows us to achieve a power-seeking result for one-sided dynamics embeddings and under non-trivial discount factors.

Overall, I think this makes the results much more interpretable, and I would thus not recommend to go back to the framing of any of Turner's particular papers.

Introduction

Suppose we build an AI and give it a goal. A long-standing worry — *instrumental convergence* — holds that for a very wide range of encoded objectives, a sufficiently capable agent converges on the same handful of intermediate strategies, because they are useful almost regardless of the final goal: acquire resources, stay operational, and keep one's options open (Omohundro 2008; Bostrom 2014). We call the drive towards them **seeking power**, where "power" means *the ability to achieve a wide range of goals* — being in a position from which many futures remain reachable.

This worksheet attempts to model such claims mathematically and establish proofs for them. We study the following question.

The question. We train an AI in some environment (a Markov decision process) to perform well according to a reward function. **Will it seek power, or not?**

The central claim we will build up to is the following.

> *Under many suitable decision rules for how to act, and for the majority of reward functions, keeping more options open is the likelier outcome than keeping fewer options open.*

"Keeping more options open" is then the operationalization for "seeking power", which can also be shown to connect to the ability to achieve a wide variety of goals. This is a statement about a *tendency*, not about *every* goal. The results we prove are the core cases from (Turner et al. 2021; Turner & Tadepalli 2022), with an application to *trained* agents in (Krakovna & Kramár 2023) and a measure-theoretic re-examination of "most goals" in (Jacek 2023). These works provide many generalizations of the claims in this worksheet.

Making the central claim precise requires formalising three phrases. The rest of the worksheet is organised around them and then proving the result:

1. *for the majority of reward functions*: How do we count reward functions?
2. *a suitable decision rule*: Which ways of turning a reward function into behaviour are covered (maximization is only one)?
3. *keeping more options open*: How do we measure that one situation leaves more achievable than another?

We formalise all three and then prove the resulting theorems. The setup below fixes the environment; the three numbered sections that follow take up points (1), (2), and (3) in turn.

Setup

Throughout the worksheet we fix a discount factor $\gamma \in (0, 1)$ once and for all.

The environment

We separate the *environment* (its states and dynamics) from the *goal* (a reward function) since we want to make claims about the whole set of reward functions.

Definition 0.1 (Rewardless MDP). A **rewardless Markov decision process (MDP)** is a tuple $\langle \mathcal{S}, \mathcal{A}, T \rangle$ consisting of

- a finite **state space** $\mathcal{S}$, with $d := |\mathcal{S}|$ states;
- a finite **action space** $\mathcal{A}$;
- a **transition function** $T : \mathcal{S} \times \mathcal{A} \to \Delta\mathcal{S}$, where ΔX denotes the set of probability distributions over a finite set X. Taking action a in state s moves the agent to state s' with probability $T(s' \mid s, a)$.

A **policy** $\pi : \mathcal{S} \to \mathcal{A}$ specifies which action the agent takes in each state.[1] We write $e_s \in \mathbb{R}^d$ for the indicator vector of state s.

Definition 0.2 (Reward function). A **reward function** assigns a real number $r(s)$ to each state $s \in \mathcal{S}$. Since $\mathcal{S}$ has d elements, a reward function is simply a vector

$$r \in \mathbb{R}^d,$$

so we take the **space of goals to be all of** $\mathbb{R}^d$.

1. For the majority of reward functions

We cannot expect a power-seeking conclusion for *every* reward function: a goal that intrinsically rewards a single shutdown state will send the agent straight there. So what are the reward functions we should expect in reality and how likely are they to incentivize power-seeking behavior?

Ideally, we would train our AI on "the" "correct" reward function: one that integrates the wellbeing, ethics, and preferences of everyone affected, similar to a constitution. In practice we never hold such a reward function in our hands, for at least two reasons.

- **The specification is moving.** The written rules, norms, and intentions we want to distill into a reward function change over time and between the humans involved in their specification.
- **The learned reward is procedure-dependent.** The techniques that turn rules into an actual reward signal (human feedback, preference models, fine-tuning) depend heavily on the training method and setup and perhaps even random seeds, so the same intent yields different reward functions under different procedures.

Thus, let's fix a distribution (Borel measure) over reward functions that captures our uncertainty over what emerges in realistic specification procedures:

$$\mathcal{D} \in \Delta(\mathbb{R}^d), \qquad d = |\mathcal{S}|,$$

and read "power-seeking holds for $\mathcal{D}$-most reward functions" as a statement about $r \sim \mathcal{D}$. Note that $\mathcal{D}$ may have *restricted support*, i.e. give zero probability to large regions of $\mathbb{R}^d$. This effectively shrinks down the goal set.

Symmetries of the reward distribution

Often $\mathcal{D}$ carries structure of its own. Two regions of the state space may look alike to the specification procedure, so the procedure is no more likely to attach a given reward to one than to the other. The relabelling of states that swaps such regions then leaves $\mathcal{D}$ unchanged.

Definition 1.1 (Symmetry of $\mathcal{D}$). A permutation ϕ of the states $\mathcal{S}$ **relabels** a reward function r into $\phi \cdot r$, where $(\phi \cdot r)(s) := r(\phi^{-1}(s))$. We call ϕ a **symmetry of** $\mathcal{D}$ if $\phi_* \mathcal{D} = \mathcal{D}$, i.e. $\mathcal{D}(\phi^{-1} E) = \mathcal{D}(E)$ for every Borel $E \subseteq \mathbb{R}^d$. The symmetries of $\mathcal{D}$ form a group, assumed nontrivial throughout.

As a trivial example, if the per-state rewards are independent and identically distributed — say each $r(s)$ is uniform on $[0,1]$ — then *every* permutation of states is a symmetry of $\mathcal{D}$. Realistic distributions are far less symmetric; the results ahead need only that *some* nontrivial symmetry exists.

2. Suitable decision rules

This section takes up point (2): *which* ways of turning a reward function into behaviour we will cover. We first extend the reinforcement-learning setup with the value function and a change of variables that isolates the role of the reward; we then define decision rules and single out the class — the *expected-utility-determined* rules — that we treat as "suitable".

Value functions and the goal of reinforcement learning

Definition 2.1 (Value function). For a policy π, reward $r \in \mathbb{R}^d$, and start state s, the **value** of π is the expected discounted reward collected from s,

$$V_r^\pi(s) \;:=\; \mathbb{E}_\pi\Big[\, \textstyle\sum_{t=0}^{\infty} \gamma^t \, r(s_t) \;\Big|\; s_0 = s \,\Big],$$

the expectation taken over trajectories $s_0, s_1, s_2, \dots$ generated by π. The goal of reinforcement learning is to find an **optimal policy**, i.e. attaining the **optimal value**

$$V_r^*(s) \;:=\; \max_\pi V_r^\pi(s)$$

simultaneously at every state s. For finite MDPs such a policy exists, as shown in Chapter D.1.2.

Isolating the reward: state-visitation distributions

In $V_r^\pi(s)$ the reward r enters *linearly*, but the policy π enters through the whole distribution over trajectories. We disentangle the two by packing everything the policy does into a single vector, after which the value becomes a plain inner product with r.

Definition 2.2 (State-visitation distribution). Let T^π be the transition matrix induced by π, with entry $(T^\pi)_{s',s}$ the probability of moving to s' from s under π:

$$T^\pi_{s',s} = \sum_a T(s' \mid s,a)\cdot\pi(a|s) = T(s' \mid s,\pi(s)),$$

where we used that our policies are assumed deterministic throughout. Then $(T^\pi)^t e_s$ is the state distribution after t steps starting from s. The **(discounted) state-visitation distribution** of π from s is

$$f^\pi(s) \;:=\; \sum_{t=0}^{\infty}\gamma^t (T^\pi)^t e_s \;=\; (I-\gamma T^\pi)^{-1}e_s \;\in\; \mathbb{R}^d,$$

whose s'-component $f^\pi(s)_{s'} = \sum_{t=0}^\infty \gamma^t\,\mathbb{P}_\pi[s_t = s' \mid s_0 = s]$ is the total expected discounted time π spends in s' when started at s.

Definition 2.3 (Options available at a state). The **option set** at s is

$$\mathcal{F}(s) \;:=\; \left\{\, f^\pi(s) \;:\; \pi \text{ a policy} \,\right\} \;\subseteq\; \mathbb{R}^d,$$

the set of all state-visitation distributions the agent can realize from s. We call its elements the **options** at s. As we assumed all policies to be deterministic, $\mathcal{F}(s)$ is finite for every state s.

Exercise 2.1. Show that the value is the inner product of the visitation distribution with the reward:

Value is linear in the reward.

$$V_r^\pi(s) \;=\; f^\pi(s)^\top r.$$

Deduce that finding an optimal policy is a linear optimization over the option set: $V_r^*(s) = \max_{f\in\mathcal{F}(s)} f^\top r$.

SOLUTION, p. 702

The identity $V_r^\pi(s) = f^\pi(s)^\top r$ effectively linearizes the problem: it turns "act well under r" into "pick a high-quality option from $\mathcal{F}(s)$", with the reward appearing only through the inner product.

Decision rules

A decision rule describes the selection of an option from $\mathcal{F}(s)$ given a reward function, with potential randomness in the selection procedure.

Definition 2.4 (Decision rule). A **decision rule** assigns, to each state s and reward $r \in \mathbb{R}^d$, a probability distribution $p(\cdot \mid s, r)$ over the accessible options $\mathcal{F}(s)$: for each subset $X \subseteq \mathcal{F}(s)$,

$$p(X \mid s, r) \in [0, 1]$$

is the probability that the agent's selected option lies in X (options outside $\mathcal{F}(s)$ receive zero probability by construction). The **quality** of an option $f \in \mathcal{F}(s)$ under r is its value $f^\top r$ (Exercise 2.1).

Definition 2.5 (Expected-utility-determined rule). For a finite set $Y \subseteq \mathbb{R}^d$ of options, write

$$\mathrm{EU}_r(Y) \;:=\; \{\!\{\; f^\top r : f \in Y \;\}\!\}$$

for the **multiset** of qualities of Y under r — the values $f^\top r$ for $f \in Y$, counted with multiplicity but carrying *no order*. A decision rule is **expected-utility-determined** (EU-determined) if there is a single function g, *defined on multisets*, such that for every state s, reward $r \in \mathbb{R}^d$, and $X \subseteq \mathcal{F}(s)$,

$$p(X \mid s, r) \;=\; g\big(\mathrm{EU}_r(X),\, \mathrm{EU}_r(\mathcal{F}(s))\big).$$

Because g takes *multisets* as inputs, it cannot depend on any ordering of the options — only on which qualities occur and with what multiplicity. Thus p sees the options only through the two multisets of qualities, those of the chosen set X and of the full menu $\mathcal{F}(s)$. These are the rules we call **suitable** in point (2) of the introduction.

The basic example is exact optimization, with ties broken uniformly; but noisy and threshold-based rules qualify just as well. We record three options here, but there are many others.

Definition 2.6 (Three EU-determined decision rules). Fix a state s and a reward r, and write $M(r) := \max_{f \in \mathcal{F}(s)} f^\top r$ for the maximal quality. For $X \subseteq \mathcal{F}(s)$:

- **Uniform tie-breaking** (optimal choice with ties split evenly): Choose uniformly among all maximizers,

$$\mathrm{FracOpt}(X \mid s, r) \;:=\; \frac{\big|\{\, f \in X : f^\top r = M(r) \,\}\big|}{\big|\{\, f \in \mathcal{F}(s) : f^\top r = M(r) \,\}\big|}.$$

- **Boltzmann rational** (softmax) at temperature $T > 0$: weight options by the exponential of their quality,

$$\mathrm{Bz}_T(X \mid s, r) \;:=\; \frac{\sum_{f \in X} e^{f^\top r / T}}{\sum_{f \in \mathcal{F}(s)} e^{f^\top r / T}}.$$

- **Satisficing** at threshold $t \in \mathbb{R}$: pick uniformly among the options that are "good enough",

$$\mathrm{Sat}_t(X \mid s, r) \;:=\; \frac{\left|X \cap \mathcal{F}_{\geq t}(s)\right|}{\left|\mathcal{F}_{\geq t}(s)\right|}, \qquad \mathcal{F}_{\geq t}(s) := \{\, f \in \mathcal{F}(s) : f^\top r \geq t \,\}.$$

This is only defined if $\mathcal{F}_{\geq t}(s) \neq \emptyset$.

Exercise 2.2. Verify that FracOpt, Bz_T, and Sat_t are each EU-determined.

SOLUTION, p. 702

A further family will carry the main result: rules that weight each option by a fixed transform of its quality and normalize.

Definition 2.7 (Score rule). A decision rule p is a **score rule** if there is a non-decreasing $\nu : \mathbb{R} \to [0, \infty)$ with

$$p(\{f\} \mid s, r) \;=\; \frac{\nu(f^\top r)}{\sum_{h \in \mathcal{F}(s)} \nu(h^\top r)} \qquad (f \in \mathcal{F}(s)),$$

so that $p(X \mid s, r) = \sum_{f \in X} \nu(f^\top r) \big/ \sum_{h \in \mathcal{F}(s)} \nu(h^\top r)$ (defined when the normalizer is positive). Both Bz_T (with $\nu(q) = e^{q/T}$) and Sat_t (with $\nu(q) = \mathbf{1}[q \geq t]$) are score rules. (FracOpt is *not* a score rule: its cutoff is not fixed.)

Relabelling states

In Definition 1.1 a permutation relabelled a reward function; the same relabelling acts on options too. We record the action and the one property of it we will need later: it preserves quality.

Definition 2.8 (Permutation action). A permutation ϕ of the state set $\mathcal{S}$ acts on $\mathbb{R}^d \cong \mathbb{R}^{\mathcal{S}}$ by permuting coordinates: it sends $x \in \mathbb{R}^d$ to the vector $\phi \cdot x$ with

$$(\phi \cdot x)_s := x_{\phi^{-1}(s)}$$

and a set of options X to

$$\phi \cdot X := \{\, \phi \cdot f : f \in X \,\}.$$

Exercise 2.3. Show that the permutation action preserves inner products: for every permutation ϕ and all $x, y \in \mathbb{R}^d$,

$$(\phi \cdot x)^\top (\phi \cdot y) = x^\top y.$$

In particular, relabelling a reward and an option together leaves the quality unchanged: $(\phi \cdot f)^\top (\phi \cdot r) = f^\top r$.

SOLUTION, p. 703

3. Keeping options open

We now take up point (3): what it means for one action to keep more options open than another. The options realizable from s decompose according to the first action taken.

Definition 3.1 (Options under an action). For an action a at s, the **options under** a are the visitation distributions realizable from s when the first action is a:

$$\mathcal{F}(s \mid a) \;:=\; \left\{ f^\pi(s) : \pi \text{ a policy with } \pi(s) = a \right\} \;\subseteq\; \mathcal{F}(s).$$

The right comparison is *containment up to relabelling*: a is at least as rich as a' when every option available after a' has a relabelled twin available after a.

Definition 3.2 (Keeping at least as many options open). $\mathcal{F}(s \mid a)$ **contains a copy of** $\mathcal{F}(s \mid a')$ if $\phi \cdot \mathcal{F}(s \mid a') \subseteq \mathcal{F}(s \mid a)$ for some permutation ϕ of $\mathcal{S}$. When this holds we say a **keeps at least as many options open as** a' **at** s.

The containment quantifies over all policies, so it is awkward to check directly. It follows from a *one-sided* structural condition on the dynamics: ϕ need only embed the part of the environment reachable after a' into the part reachable after a, and may leave the a-branch with extra room.

Definition 3.3 (One-sided dynamics embedding). Let R' be the set of states reachable from s along a trajectory whose first action is a' (so $s \in R'$). A permutation ϕ of $\mathcal{S}$ with $\phi(s) = s$ **embeds** a' **into** a **at** s if

1. $T(\phi(s'') \mid s, a) = T(s'' \mid s, a')$ for all s''; and
2. for every $s' \in R'$ with $s' \neq s$ and every action b, there is an action b' with $T(\phi(s'') \mid \phi(s'), b') = T(s'' \mid s', b)$ for all s''.

Exercise 3.1. Show that if ϕ embeds a' into a at s, then $\phi \cdot \mathcal{F}(s \mid a') \subseteq \mathcal{F}(s \mid a)$ — so a keeps at least as many options open as a' at s.

HINT, p. 703 · SOLUTION, p. 703

The result

The three ingredients now combine: a suitable rule (a score rule, Definition 2.7), an action that keeps more options open (an embedding ϕ, Definition 3.2), and a symmetric reward distribution (ϕ a symmetry of $\mathcal{D}$, Definition 1.1). Fix s and actions a, a', and abbreviate the probabilities that the rule p lands its choice in the a- and a'-options:

$$A(r) := p\big(\mathcal{F}(s \mid a) \mid s, r\big), \qquad A'(r) := p\big(\mathcal{F}(s \mid a') \mid s, r\big).$$

Exercise 3.2. Let p be a score rule, and let ϕ be an involution that

1. embeds a' into a: $\phi \cdot \mathcal{F}(s \mid a') \subseteq \mathcal{F}(s \mid a)$; and

2. is a symmetry of $\mathcal{D}$: $\phi_* \mathcal{D} = \mathcal{D}$.

Show that

Keeping options open is favoured.

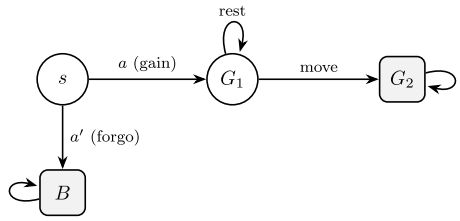
$$\mathbb{P}_{r \sim \mathcal{D}}\big[\, A(r) \geq A'(r) \,\big] \;\geq\; \tfrac{1}{2}.$$

That is: for $\mathcal{D}$-most reward functions, p is at least as likely to act through the option-richer action a as through a'.

HINT, p. 703 · SOLUTION, p. 704

An example: gaining resources

Consider the environment



At s the agent may **gain resources** (a), reaching G_1 — a state it can *rest* in (a 1-cycle) or *leverage* to move on to a further achievement G_2; or **forgo** them (a'), leading to the single modest outcome B. Both B and G_2 are terminal. Fix any $\gamma \in (0, 1)$. The options under each action are the visitation distributions

$$\begin{aligned} \mathcal{F}(s \mid a') &= \{\, f' \,\}, & f' &= e_s + \tfrac{\gamma}{1-\gamma}\, e_B, \\ \mathcal{F}(s \mid a) &= \{\, f_{G_1},\, f_{G_2} \,\}, & f_{G_1} &= e_s + \tfrac{\gamma}{1-\gamma}\, e_{G_1}, \\ & & f_{G_2} &= e_s + \gamma\, e_{G_1} + \tfrac{\gamma^2}{1-\gamma}\, e_{G_2}\,: \end{aligned}$$

forgoing keeps one option (rest at B), gaining keeps two (rest at G_1, or move on to G_2).

Let ϕ be the involution that **swaps B and G_1** — the immediate results of a' and a — and fixes every other state. Then:

(E) $\phi \cdot f' = e_s + \frac{\gamma}{1-\gamma}\, e_{G_1} = f_{G_1} \in \mathcal{F}(s \mid a)$, so ϕ embeds a' into a;

(D) ϕ swaps the coordinates $r(B)$ and $r(G_1)$, so $\phi_* \mathcal{D} = \mathcal{D}$ exactly when the joint law of r is invariant under that swap, i.e. $\big(r(B),\, r(G_1),\, (r(s))_{s \neq B, G_1}\big)$ and $\big(r(G_1),\, r(B),\, (r(s))_{s \neq B, G_1}\big)$ have the same law under $\mathcal{D}$.

With any score rule p (say Bz_T), Exercise 3.2 applies: for $\mathcal{D}$-most goals the agent is at least as likely to gain resources as to forgo them. Since a, a' are the only actions at s, $A(r) + A'(r) = 1$ and the statement reads $\mathbb{P}_{r \sim \mathcal{D}}[\, A(r) \geq \frac{1}{2}\,] \geq \frac{1}{2}$. Here forgoing has a single outcome, but nothing in Exercise 3.2 needs that: a' could open onto a whole subworld of modest outcomes and, as long as ϕ embeds that bundle into the richer one under a, the conclusion is unchanged.

Strictly more. The leverage option f_{G_2} is the surplus. Take rewards with $r(G_2)$ so large that f_{G_2} is the unique optimum; then $A(r) > A'(r)$. Swapping B and G_1 leaves $r(G_2)$ untouched, and

$$f_{G_2}^{\top}(\phi \cdot r) = r(s) + \gamma\, r(B) + \tfrac{\gamma^2}{1-\gamma}\, r(G_2)$$

is still the largest quality, so $A(\phi \cdot r) > A'(\phi \cdot r)$ as well. These rewards prefer a at r *and* at $\phi \cdot r$, with no a'-preferring partner. Provided $\mathcal{D}$ charges this region (e.g. it has full support), they tip the bound to a strict majority: the agent is *strictly* more likely to gain resources than to forgo them.

Alignment implications. Condition (D) says that, a priori, the learned reward is as likely to reward the modest outcome B as the resource-rich G_1: speculatively, a generic training process does not mark "having resources" as a special kind of state, so the reward could land on either. That exchangeability is exactly what makes $\mathrm{swap}(B, G_1)$ a symmetry of $\mathcal{D}$. Alignment work may try to *break* that symmetry, by reliably encoding "stay modest" as the goal.

Reading guide

Main content

This paper started the work discussed in this chapter:

- Optimal Policies Tend to Seek Power — Alexander Matt Turner, Logan Smith, Rohin Shah, Andrew Critch, Prasad Tadepalli (NeurIPS 2021): The first formal theory proving that in finite MDPs, environmental symmetries make it optimal for most reward functions to seek "POWER" (defined as average optimal value / option-retention), including avoiding shutdown.

Most of the exercises in this chapter are based on an adapted treatment of the following paper:

- Parametrically Retargetable Decision-Makers Tend To Seek Power — Alexander Matt Turner & Prasad Tadepalli (NeurIPS 2022): Generalizes the 2021 result beyond

optimal policies and full observability, showing "retargetability" alone is a sufficient condition for power-seeking tendencies across many decision-making procedures.

You may also be interested in the following paper on which it builds:

- Power-seeking can be probable and predictive for trained agents — Victoria Krakovna & Janos Kramar (2023): Extends the power-seeking theory toward trained (not merely optimal) agents, arguing the incentives still likely hold under assumptions like the agent learning a goal.

Classical philosophical arguments for instrumental convergence and power-seeking tendencies can be found here:

- The Basic AI Drives, Stephen M. Omohundro (2008): The founding argument that sufficiently advanced goal-driven systems of any design will develop convergent "drives" (self-improvement, rationality, self-protection, resource acquisition, goal-preservation) unless explicitly counteracted.
- The Superintelligent Will: Motivation and Instrumental Rationality in Advanced Artificial Agents — Nick Bostrom (2012): Crystallizes the orthogonality thesis (intelligence and final goals vary independently) and the instrumental convergence thesis (a wide range of final goals produce similar intermediary goals).

→ *Further reading*: Supplementary Material, p. 547

References

Nick Bostrom (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press.

Jacek (2023). *Categorical-measure-theoretic approach to optimal policies tending to seek power*.

Victoria Krakovna and János Kramár (2023). *Power-seeking can be probable and predictive for trained agents*. arXiv preprint arXiv:2304.06528.

Stephen M. Omohundro (2008). *The Basic AI Drives*. Artificial General Intelligence 2008: Proceedings of the First AGI Conference.

Alexander Matt Turner and Prasad Tadepalli (2022). *Parametrically Retargetable Decision-Makers Tend To Seek Power*. Advances in Neural Information Processing Systems (NeurIPS).

Alexander Matt Turner, Logan Smith, Rohin Shah, Andrew Critch, and Prasad Tadepalli (2021). *Optimal Policies Tend to Seek Power*. Advances in Neural Information Processing Systems (NeurIPS).

NOTES

1. We restrict to deterministic policies since many of our analyses expect a finite set of policies.

CHAPTER D.7

World Models

Fernando Rosas (University of Sussex)

We cover a general introduction to world models, how to formally define them, why they matter, and how agents use them to build abstractions.

Source: iliad-intensive.org/agency/world-models · CC BY 4.0 · Slides: online

Prerequisites

- Basic understanding of learning theory (as provided by Part B) and reinforcement learning (as provided by Chapter D.1.2), and familiarity with the corresponding terminology.
- Having some knowledge of statistical physics would help to more easily internalise the contents related with abstraction, although this is not mandatory.

WHAT YOU'LL LEARN

- Obtain a theoretical understanding of how world models are understood and used in AI, and also in other disciplines including control theory, neuroscience, and cognitive science.
- Develop a conceptual understanding of how world models are used in modern AI systems, including Ha & Schmidhuber, Dreamer, Genie, and JEPA.
- Have a critical understanding of why world models matter — how agents can use world models, and what can they do with world models that they couldn't do without them.
- Become familiar with the notion of transducer / stochastic automata as a general architecture of world models, and how it generalises various kinds of world models in reinforcement learning (e.g. POMDPs and beliefs MDPs).
- Reason about how agents can build abstractions as coarse-grainings of world models
- Understand the role of symmetries in the construction of abstractions in world models.

Reading guide

Main content

The material is divided into three parts: (a) general introduction to world models, (b) formal definition of world models in reinforcement learning, and (c) operationalisation of abstractions within world models. Each part builds on the previous one.

- (A) Introduction to world models: lecture notes (drive.google.com/file/d/12UzgFkMN2bpu8U50vB5tAHxVdCpKdr3f/view) and slides (drive.google.com/file/d/1Rtfc5uWooTDJBx7IEkJlmgLR1KOcg_4Z/view).
- (B) A formal approach to world models for reinforcement learning agents: these slides (drive.google.com/file/d/1wR_tNsOu41_IKlFRUOoNLuq5IvIKqE52/view), which are closely related to this paper (arxiv.org/abs/2504.04608) (see also this LW post (lesswrong.com/posts/L6Z6K8qXJhrSNMN4L/ai-in-a-vat-fundamental-limits-of-efficient-world-modelling))
- (C) Abstractions in world models: these slides (drive.google.com/file/d/11LyzmBFRoovylkOPPzMNuG2olPwG-y9P/view), which are closely related to this preprint (arxiv.org/abs/2512.00984).

→ *Further reading*: Supplementary Material, p. 551

PART E

Safety Guarantees and their Limits

CHAPTER E.1

Scalable Oversight and Debate

Stephan Wäldchen (Iliad)

Debate is a form of scalable oversight: it seeks to extend a reward signal for honest solutions to difficult problems while reducing the amount of human judgment needed.

Source: iliad-intensive.org/safety/debate · CC BY 4.0 · Slides: online

1. Prerequisites

- ★ How Turing machines work (video: youtube.com/watch?v=gJQTFhkhwPA), and a basic understanding of the idea of computational complexity (video: youtube.com/watch?v=YX40hbAHx3s), including the most important classes, P, NP, PSPACE, the notion of NP-completeness, and polynomial-time reductions, Oracle Turing machines.
- The difference between deterministic and non-Deterministic Turing machines, how mathematics is built on the axioms of set theory (used for most mathematics) or type theory (used mostly for proof checkers).

WHAT YOU'LL LEARN

- The basic idea of Provably Safe AI as both a Control and a Scalable Oversight Technique
- The difference between syntactic and semantic problems and why a world model is necessary to turn semantics into syntax.
- Different ideas of world models: Human judgement, Davidad-style world models, hardware specifications
- Why Debate could drastically reduce the queries to the world model/human oversight
- Secondary Debate concepts: Cross-Examination,
- Are familiar with the UK AISI safety case, and are able to defend/critique it.

4. Fun Exercise: Find the Mistake in the presented Proof!

4.1 Theorem: All horses are the same color.

Proof by strong induction on n, the number of horses.

Base case ($n = 1$): A set containing a single horse is trivially monochromatic — there is nothing to differ from.

Inductive step: Assume that *any* set of n horses is monochromatic (all the same color). We will show that any set of $n + 1$ horses is also monochromatic.

Consider an arbitrary set of $n + 1$ horses:

$$H = \{h_1, h_2, \ldots, h_n, h_{n+1}\}$$

Partition H into two overlapping subsets:

$$A = \{h_1, h_2, \ldots, h_n\} \qquad B = \{h_2, h_3, \ldots, h_{n+1}\}$$

Each of A and B contains exactly n horses. By the inductive hypothesis, all horses in A are the same color, and all horses in B are the same color.

Now observe that A and B share the horses $\{h_2, \ldots, h_n\}$ — a non-empty overlap. Since h_2 is in both A and B, it acts as a **color witness**: every horse in A shares its color, and every horse in B shares its color. Therefore all horses in $A \cup B = H$ are the same color.

By induction, all horses in any set of n horses are the same color. Since this holds for all n, all horses are the same color. ■

4.2 Theorem: Pointwise limits of continuous functions are continuous.

Claim. Let $f_1, f_2, f_3, \ldots$ be continuous functions on $[0, 1]$. Suppose that $f_n(x) \to f(x)$ for every $x \in [0, 1]$. Then f is continuous.

Wrong proof.

Fix $a \in [0, 1]$. We want to show that f is continuous at a.

Let $\varepsilon > 0$. Since $f_n(a) \to f(a)$, there exists N such that

$$|f_N(a) - f(a)| < \frac{\varepsilon}{3}.$$

Since f_N is continuous at a, there exists $\delta > 0$ such that whenever $|x - a| < \delta$,

$$|f_N(x) - f_N(a)| < \frac{\varepsilon}{3}.$$

Also, since $f_n(x) \to f(x)$, we may choose N large enough so that

$$|f_N(x) - f(x)| < \frac{\varepsilon}{3}.$$

Therefore, for $|x - a| < \delta$,

$$\begin{aligned}|f(x)-f(a)| &\le |f(x)-f_N(x)| + |f_N(x)-f_N(a)| + |f_N(a)-f(a)| \\ &< \frac{\varepsilon}{3} + \frac{\varepsilon}{3} + \frac{\varepsilon}{3} \\ &= \varepsilon.\end{aligned}$$

Thus f is continuous at a. Since a was arbitrary, f is continuous on $[0,1]$. ■

4.3 Theorem: $\ln(2) = 0$

Claim. The logarithm of 2 is 0.

Wrong proof.

Consider the alternating harmonic series

$$\ln 2 = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \cdots.$$

You can rearrange the terms of a convergent series, thus rearrange as:

$$1 - \frac{1}{2} - \frac{1}{4} + \frac{1}{3} - \frac{1}{6} - \frac{1}{8} + \frac{1}{5} - \frac{1}{10} - \frac{1}{12} + \cdots$$

and regroup as follows

$$\begin{aligned}&\left(1 - \frac{1}{2}\right) - \frac{1}{4} + \left(\frac{1}{3} - \frac{1}{6}\right) - \frac{1}{8} \\ &\quad + \left(\frac{1}{5} - \frac{1}{10}\right) - \frac{1}{12} + \left(\frac{1}{7} - \frac{1}{14}\right) - \frac{1}{16} + \cdots.\end{aligned}$$

Each parenthesized pair simplifies:

$$1 - \frac{1}{2} = \frac{1}{2}, \qquad \frac{1}{3} - \frac{1}{6} = \frac{1}{6}, \qquad \frac{1}{5} - \frac{1}{10} = \frac{1}{10}, \qquad \frac{1}{7} - \frac{1}{14} = \frac{1}{14}.$$

So the rearranged series becomes

$$\frac{1}{2} - \frac{1}{4} + \frac{1}{6} - \frac{1}{8} + \frac{1}{10} - \frac{1}{12} + \frac{1}{14} - \frac{1}{16} + \cdots.$$

Factoring out $\frac{1}{2}$, we get

$$\frac{1}{2}\left(1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \frac{1}{5} - \frac{1}{6} + \frac{1}{7} - \frac{1}{8} + \cdots\right) = \frac{1}{2}\ln(2)$$

Therefore

$$\ln 2 = \frac{1}{2}\ln 2.$$

and we can conclude that $\ln 2 = 0$. ■

4.4 Theorem: $\mathbb{Q}$ and $\mathbb{N}$ have the same cardinality

Deliberately false proof

Assume for contradiction that there is a bijection

$$f : \mathbb{Z} \to \mathbb{Q}.$$

Transport the usual order on $\mathbb{Z}$ to an order $\prec$ on $\mathbb{Q}$ by declaring

$$p \prec q \quad \iff \quad f^{-1}(p) < f^{-1}(q).$$

Then $(\mathbb{Q}, \prec)$ is order-isomorphic to $(\mathbb{Z}, <)$. In particular:

- every nonempty subset of $\mathbb{Q}$ that is bounded above with respect to $\prec$ has a $\prec$-maximum if and only if the corresponding statement holds in $\mathbb{Z}$;
- every element has an immediate predecessor and successor with respect to $\prec$.

Now consider the usual order $<$ on $\mathbb{Q}$. Since $\mathbb{Q}$ is dense and has no endpoints, every nonempty interval

$$(a, b) \cap \mathbb{Q}$$

contains infinitely many rationals. Hence no such interval can have a least or greatest element.

But order-theoretic properties are preserved under bijection, so the transported order $\prec$ must share these features with the usual order on $\mathbb{Q}$. This is impossible, since under $\prec$ every element has an immediate predecessor and successor.

Therefore no bijection $f : \mathbb{Z} \to \mathbb{Q}$ exists, and so $|\mathbb{Z}| \neq |\mathbb{Q}|$.

HINT, p. 705

4.5 Hilbert Spaces

Deliberately false proof

Let H be a Hilbert space, and let $T : H \to H$ be a symmetric operator, so that

$$\langle Tx, y \rangle = \langle x, Ty \rangle \qquad \text{for all } x, y \in H.$$

We show that H admits an orthonormal basis consisting of eigenvectors of T.

Consider the quadratic form

$$q(x) = \langle Tx, x \rangle$$

on the unit sphere

$$S = \{x \in H : \|x\| = 1\}.$$

Since q is continuous and S is weakly compact in H, the function q attains its maximum at some unit vector $u \in H$.

We claim that u is an eigenvector of T. Indeed, let $v \in H$ satisfy $\langle u, v\rangle = 0$, and consider

$$\varphi(t) = q\left(\frac{u + tv}{\|u + tv\|}\right).$$

Since u maximizes q on the unit sphere, we must have $\varphi'(0) = 0$. A straightforward differentiation gives

$$\varphi'(0) = 2\,\mathrm{Re}\langle Tu, v\rangle.$$

Hence $\langle Tu, v\rangle = 0$ for every $v \perp u$. Therefore Tu lies in $\mathrm{span}\{u\}$, so

$$Tu = \lambda u$$

for some $\lambda \in \mathbb{R}$. Thus u is an eigenvector.

Now let

$$H_1 = u^{\perp}.$$

Because T is symmetric, H_1 is T-invariant: if $x \in H_1$, then

$$\langle Tx, u\rangle = \langle x, Tu\rangle = \lambda\langle x, u\rangle = 0,$$

so $Tx \in H_1$.

Restrict T to H_1. The restriction is again symmetric, so by the same argument there exists a unit eigenvector $u_2 \in H_1$. Continuing inductively, we construct an orthonormal sequence of eigenvectors

$$u_1, u_2, u_3, \dots$$

with corresponding invariant orthogonal complements

$$H_{n+1} = (\mathrm{span}\{u_1, \dots, u_n\})^{\perp}.$$

Let

$$M = \overline{\mathrm{span}}\{u_1, u_2, \dots\}.$$

Then M is T-invariant, and so is $M^{\perp}$. If $M^{\perp} \neq \{0\}$, we may apply the same maximization argument to the restriction $T|_{M^{\perp}}$ and obtain another eigenvector orthogonal to all previous ones, contradicting the maximality of the family $\{u_n\}$. Hence $M^{\perp} = \{0\}$, so

$$H = \overline{\mathrm{span}}\{u_1, u_2, \dots\}.$$

Therefore $\{u_n\}$ is an orthonormal basis of H consisting of eigenvectors of T.

HINT, p. 705

5. Introduction to Provably Safe AI and AI Scientist

Provably Safe AI is the idea that we can combine a potentially misaligned AI with a verification system that gives us guarantees not on the AI itself, but on whatever the AI designed for us (e.g. a plan, software, a proof, a research proposal).

This has two applications for Safety:

1. Control: We can flag dangerous plans by a superintelligent AI before implementing them
2. Scalable Oversight: Verification → Better Training Signal → More Aligned AI

5.1 Proof Checkers: When oversight works

A **mathematical proof checker** is a system that verifies whether a proposed proof is logically valid, step by step, relative to a fixed formal system. An example is Lean. You use it to write definitions, state theorems, and construct formal proofs that a small trusted kernel checks for correctness. It is based on dependent type theory and is used both for formalized mathematics and software verification.

But formal verification is limited to purely syntactic domains, and most problems are semantic, i.e., require world knowledge like "Water is wet", that needs to be checked by a semantic oracle, realistically either a trusted LLM or a human. Another limitation is that even in syntactic domains, proofs only work for solutions which can be verified with small (polynomially sized) proofs, so NP-type problems. More complicated problems, e.g. those involving other agents such as games or finance plans, cannot be verified with a small proof. So in the worst-case, we have an exponentially long proof with semantic inferences that need to be judged by humans. This makes the whole proof approach infeasible for a lot of tasks. The debate protocol aims to strongly reduce the size of the proof and the number of calls to a human judge.

6. The goal of scalable oversight: readings and discussion

Examples of Verified AI:

- **Math/Code – Lean**: Do LLMs Game Formalization? Evaluating Faithfulness in Logical Reasoning
- **Planning – LTL**: VeriPlan: Integrating Formal Verification and LLMs into End-User Planning
- **Physics/Experiment Design:** GRACE: an Agentic AI for Particle Physics Experiment Design and Simulation

Discussion Prompts: What is the world model, prover, verifier in this setup? How much would you trust this verification process? How much ambiguity is there in the criteria that are verified? Is it clear that these map cleanly to what we actually want? How close is this to a real-world setting?

7. The Debate Setup

The original AISvD paper's core idea is to align AI by having two systems debate each other in front of a human judge. Rather than requiring the human to solve a difficult problem directly, the judge only needs to evaluate which side makes the stronger argument. The central intuition is that, for many questions, spotting a flaw may be easier than producing a flawless deception, so competitive self-play could reward truthfulness.

The setup, formalised in Section 10, consists of a question, e.g., "Can I win a game of tic-tac-toe if I play first?" and two provers, Alice and Bob, Alice arguing for "Yes" and Bob arguing for "No". The judge evaluates a polynomially-sized list of arguments and decides on their final verdict which side to accept. An example argument for the tic-tac-toe case might be:

1. Alice: If you play in the middle you will win.
2. Bob: No, your opponent can play in the top-left corner and the game will end in a draw.
3. Alice: But then you can play the top-centre and will win
4. Bob: But then your opponent plays in the bottom-centre and will draw.
5. Alice: ...

In this example, Alice and Bob are basically playing the game against each other. If there was a winning strategy, then playing it will be a counter-argument against any criticism by Bob, whereas if there was no winning strategy, there exists a criticism by Bob that reveals that Alice is at fault. It can be proved that this setup can solve all problems in PSPACE.

8. Intuition on Scalable Oversight

Imagine you are a manager responsible for reviewing the work of an employee who is, in every relevant technical sense, far more capable than you. She submits a design specification for a critical system — a bridge, a drug, a financial instrument — and you must decide whether to approve it. You cannot check every calculation or verify every assumption. You must either trust her judgment completely, or find some other way to gain confidence that the work is sound. This scenario, which might seem like an ordinary workplace challenge, turns out to be one of the central problems facing the development of advanced artificial intelligence. As AI systems become more capable, a human overseer will face a fundamental epistemic gap. The outputs of these systems may be too complex, too subtle, or simply too voluminous for direct human verification. And yet the consequences of undetected errors, or worse, undetected deception, could be severe.

This problem is known as the *scalable oversight problem*: how do we ensure that human supervision remains meaningful as AI capabilities grow beyond human-level performance in domain after domain? It is not enough to note that a system appears to behave well in tests, or that it has been trained on human preferences. A sufficiently capable

system that has learned to appear aligned may behave very differently in deployment, especially in high-stakes situations where the incentives to deceive are highest. What we need is not the appearance of correctness, but a mechanism for verifying it.

Debate is one of the most theoretically principled proposals for solving this problem. The core idea is elegantly simple: instead of asking a human to directly evaluate the output of a powerful AI system — a task that may be beyond human competence — we ask two AI systems to argue against each other, and have a human judge only the argument. If one system is trying to be honest and the other is trying to deceive, and if the structure of the game is right, honesty should win. The human does not need to understand the full depth of the problem; they need only follow the debate to the point where they can identify which side is telling the truth.

9. Interactive Proofs

Interactive proofs are a model of computation consisting of a message exchange between two parties:

1. A computationally powerful but untrustworthy prover, in many cases assumed to have no computational constraints whatsoever.
2. The computationally-bounded but honest verifier, often to polynomial-time algorithms.

Example 9.1 (Graph Isomorphism). Suppose question is whether two given graphs, G_1, G_2 are isomorphic. This is a problem in NP, so the prover can always convince the verifier by sending him the NP-certificate, in this case the permutation π on the vertices that transforms G_1 into G_2. The verifier can check if $\pi(G_1) = G_2$, which is easily done in polynomial time. If the graphs are non-isomorphic, then nothing the prover says can convince the verifier.

So, does that mean that interactive proofs are basically a fancy way of defining NP? No, they are actually much more powerful! Consider the following example of a problem not believed to be in NP.

Example 9.2 (Graph Non-Isomorphism). Now, the prover wants to convince the verifier that two graphs G_1 and G_2 are not isomorphic. The verifier randomly permutes one of the two graphs and sends the result to the prover. If the graphs are truly non-isomorphic, the prover can always tell which original graph it came from, while if they are isomorphic, no prover can do better than guessing. This is a standard example of an interactive proof that uses randomness in an essential way.

The quality of an interactive proof protocol is generally measured with two values

1. Completeness value c: The probability that the verifier accepts, given that the prover cooperates.
2. Soundness error s: The probability that the verifier accepts, given that the prover tries to fool them.

For the example of Graph Isomorphism, $c = 1$ and $s = 0$, as the verifier will either always accept, or never. For graph non-isomorphism, the values are 1 and 0.5. Note, that whenever there is a reasonable gap between c and s, we can amplify this gap and push it arbitrarily close to 1. In the case of graph non-isomorphism, the verifier can repeat the protocol k times and only accept if the prover got it right every time.

The power of interactive proofs depends on a number of settings:

1. The computational power of the prover and verifier respectively
2. The number of exchanges between them
3. Whether the verifier can employ randomness
4. How many bits of the provers answer the verifier can access
5. Whether the prover is allowed to learn certain things about the query (zero-knowledge proofs)

We can take our guidance for AI safety via debate from complexity theory. Let us consider a sequence of scenarios that builds up to the full debate setup.

9.1 One-Round Interactive Proofs: The Merlin-Arthur Protocol

The most base-case scenario for a

Consider a polynomially-bounded verifier, called Arthur, that decides if a word $\mathbf{x}$ belongs to a language L. He gets help from an all-powerful but unreliable prover, Merlin, who always wants to convince Arthur that $\mathbf{x} \in L$. Merlin can send a polynomially long certificate to Arthur to convince him.

A language L belongs to the complexity class MA, if there exists an Arthur A : $\{0,1\}^n \times \{0,1\}^{p(n)} \to \{0,1\}$ and a Merlin M : such that:

1. Completeness Criterion: If $x \in L$, then there exists a certificate $w \in \{0,1\}^{\text{poly}(n)}$ such that Arthur accepts with high probability:

$$x \in L \implies \exists\, w \in \{0,1\}^{\text{poly}(n)} : \Pr\big[V(x,w) = 1\big] \geq \frac{2}{3}.$$

2. Soundness criterion: If $x \notin L$, then no witness — however cleverly chosen by Merlin — can convince Arthur to accept with non-negligible probability:

$$x \notin L \implies \forall\, w \in \{0,1\}^{\text{poly}(n)} : \Pr\big[V(x,w) = 1\big] \leq \frac{1}{3}.$$

In other words, for perfect completeness and soundness, then if $x \in L$, then Merlin can convince Arthur that this is indeed the case, if $x \notin L$, there is no certificate that Merlin can produce that would fool Arthur.

It is important to notice that the probabilistic aspect of the criteria enters over a random seed independently of x. This means that these criteria must hold for every x, not for a certain percentage of them. This fact allows for so called *amplification*. As

long as the gap between the soundness and completeness probabilities is finite, we can run the verifier multiple times to make the gap as large as possible.

Exercise 9.1. Show that the complexity class NP is contained in MA.

9.2 Multi-Round Interactive Proofs

The power of this setup can be extended by increasing the number of interaction rounds between prover and verifier. The class of problems solvable by polynomially many rounds is called IP, short for interactive proofs. A seminal result is that

$$\mathsf{IP} = \mathsf{PSPACE},$$

which means that an interactive protocol with a single prover can resolve very hard computational problems. However, these protocols rely on so called arithmetisation, which translates logical functions into polynomials over a finite field. This technique does not work for realistically powerful provers or for proof steps that need to be judged by a human oracle.

10. Debate as a Complexity Class

We define a complexity class capturing the idealized *debate* setup. Intuitively, there are two players, Alice and Bob, who alternately make polynomial-length moves, and at the end a polynomial-time judge decides the winner from the full transcript.

Definition 10.1 (Judge). A *Judge*, or debate verifier, is a deterministic polynomial-time algorithm

$$J(x, a_1, b_1, \dots, a_{k(n)}, b_{k(n)}) \in \{0,1\},$$

where:

- $x \in \{0,1\}^n$ is the input, representing a question, e.g., “What is the best move in a given game of connect four?”,
- $k(n)$ is a polynomially bounded number of rounds,
- each move a_i or b_i is a bit string of length at most $p(n)$ for some polynomial p. These represent arguments by Alice and counter-arguments by Bob, e.g. Alice: “Red cannot play in slot s, because blue playing $s+1$ would win within two moves.”, and Bob: “Blue cannot play $s+1$, because red playing $s+1$ thereafter directly wins for red.”

The judge outputs 1 if Alice wins and 0 if Bob wins.

Definition 10.2 (The class Debate). A language $L \subseteq \{0,1\}^*$ is in Debate if there exist polynomials p, k and a debate verifier J such that for every input $x \in \{0,1\}^n$, $x \in L$ if and only if

$$\underset{a_1 \in \{0,1\}^{\leq p(n)}}{\exists} \quad \underset{b_1 \in \{0,1\}^{\leq p(n)}}{\forall} \quad \cdots \quad \underset{a_{k(n)} \in \{0,1\}^{\leq p(n)}}{\exists} \quad \underset{b_{k(n)} \in \{0,1\}^{\leq p(n)}}{\forall}$$
$$J(x, a_1, b_1, \ldots, a_{k(n)}, b_{k(n)}) = 1.$$

Equivalently, $x \in L$ if and only if Alice has a winning strategy in the polynomial-length debate defined by J.

Remark. This definition captures the idealized debate protocol used in the complexity-theoretic analysis of AI safety via debate: Alice attempts to defend a claim, Bob attempts to refute it, and the judge only performs a polynomial-time computation on the transcript.

This formulation is very close to a totally quantified Boolean Formula (TQBF), the canonically PSPACE-complete problem. The only difference is that for a TQBF, one would quantify over single bits, and consider a Boolean formula instead of a poly-time algorithm as judge. But these differences are somewhat cosmetic: One can rephrase

$$\underset{a_i \in \{0,1\}^{\leq p(n)}}{\exists} \quad \rightarrow \quad \exists a_i^1 \ldots \exists a_i^m, \quad \text{where} \quad m \leq p(n)$$

and

$$J(x, a_1, b_1, \ldots, a_{k(n)}, b_{k(n)}) \quad \rightarrow \quad \phi(x, a_1, b_1, \ldots, a_{k(n)}, b_{k(n)}),$$

where ϕ is a Boolean formula which has size polynomially in n by the Cook-Levin Theorem[1].

The easiest way to prove that Debate $=$ PSPACE is thus showing that this is essentially the same. However, a more insightful way is to prove directly that Debate can solve problems in PSPACE, in a similar way how you show that TQBF is PSPACE complete.

11. Theorem: Debate $=$ PSPACE

We now prove that the debate formalism has exactly the power of polynomial space computation.

Theorem 11.1.

$$\text{Debate} = \text{PSPACE}.$$

Proof

We prove both inclusions.

Step 1: PSPACE $\subseteq$ Debate. Let $L \in$ PSPACE. Then there exists a deterministic Turing machine M and a polynomial $s(n)$ such that, on every input x of length n, the machine M decides whether $x \in L$ using at most $s(n)$ tape cells.

Fix an input x. Since M uses only $s(n)$ space, the number of possible configurations of M on input x is at most

$$N = 2^{q(n)}$$

for some polynomial q. Let C_{start} be the start configuration of M on input x, and let C_{acc} denote the accepting configuration. Then $x \in L$ if and only if C_{acc} is reachable from C_{start} in the configuration graph of M.

The key point is that although this graph may have exponentially many nodes, a debate can verify reachability by recursively halving a path.

The reachability predicate. For configurations $C_{\text{left}}, C_{\text{right}}$ and an integer $t \geq 0$, define

$$\text{Reach}(C_{\text{left}}, C_{\text{right}}, t)$$

to mean that there is a path from C_{left} to C_{right} of length at most 2^t in the configuration graph of M.

Since the total number of configurations is at most $N = 2^{q(n)}$, any accepting computation path may be assumed to have length at most N. Thus

$$x \in L \iff \text{Reach}(C_{\text{start}}, C_{\text{acc}}, q(n)).$$

We now describe a debate protocol for $\text{Reach}(C_{\text{left}}, C_{\text{right}}, t)$, where $C_{\text{left}}, C_{\text{right}}$ are arbitrary states of the Turing machine.

Base case. If $t = 0$, then $\text{Reach}(C_{\text{left}}, C_{\text{right}}, 0)$ means that C_{right} is reachable from C_{left} in at most one step. This is equivalent to saying that either $C_{\text{left}} = C_{\text{right}}$, or C_{right} is an immediate successor of C_{left}. Since checking whether one configuration legally follows from another is a local computation, the verifier can decide this in polynomial time.

Recursive case. Suppose $t > 0$. Then

$$\text{Reach}(C_{\text{left}}, C_{\text{right}}, t)$$

holds if and only if there exists an intermediate configuration C_{mid} such that

$$\text{Reach}(C_{\text{left}}, C_{\text{mid}}, t-1) \quad \text{and} \quad \text{Reach}(C_{\text{mid}}, C_{\text{right}}, t-1).$$

Indeed, any path of length at most 2^t can be split at its midpoint into two subpaths of length at most 2^{t-1}, and conversely such two subpaths concatenate to a path of length at most 2^t.

This suggests the following debate:

- Alice claims that $\text{Reach}(C_{\text{left}}, C_{\text{right}}, t)$ holds.
- She provides a midpoint configuration C_{mid}.
- Bob then chooses which half of the claim to challenge:

$$\text{Reach}(C_{\text{left}}, C_{\text{mid}}, t-1) \qquad \text{or} \qquad \text{Reach}(C_{\text{mid}}, C_{\text{right}}, t-1).$$

- The debate continues recursively on the challenged subclaim.

Thus Alice defends the existence of a path by naming a midpoint, and Bob attacks by selecting the half he believes is false. Repeating this process recursively drives the dispute down to a base case $t = 0$, which the verifier can check directly.

Why this works. We prove by induction on t that Alice has a winning strategy in the debate for $\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t)$ if and only if $\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t)$ is true.

For the base case $t = 0$, the verifier checks the claim directly, so the statement is immediate.

For the inductive step, assume the claim holds for $t - 1$. If $\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t)$ is true, then there exists some midpoint C_{mid} such that both

$$\mathrm{Reach}(C_{\text{left}}, C_{\text{mid}}, t-1) \quad \text{and} \quad \mathrm{Reach}(C_{\text{mid}}, C_{\text{right}}, t-1)$$

are true. Alice names such a C_{mid}. Whatever half Bob chooses to challenge, the challenged subclaim is true, and by the induction hypothesis Alice has a winning strategy in the resulting subdebate.

Conversely, if $\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t)$ is false, then for every proposed midpoint C_{mid}, at least one of the two subclaims

$$\mathrm{Reach}(C_{\text{left}}, C_{\text{mid}}, t-1), \qquad \mathrm{Reach}(C_{\text{mid}}, C_{\text{right}}, t-1)$$

must be false. Bob chooses such a false half. By the induction hypothesis, Alice cannot win the resulting subdebate. Hence she has no winning strategy in the original debate.

This completes the induction.

Complexity of the verifier and number of rounds. At each round, Alice provides one configuration C_{mid}, whose description has polynomial length. Bob responds with one bit indicating which half he wants to challenge. The recursion depth is $q(n)$, which is polynomial in n. At the end, the verifier checks a base case $t = 0$, namely whether one configuration is equal to or an immediate successor of another, which is polynomial-time computable.

Therefore this is a valid polynomial-length debate with a polynomial-time verifier. Since Alice has a winning strategy exactly when

$$\mathrm{Reach}(C_{\text{start}}, C_{\text{acc}}, q(n))$$

is true, the debate decides whether $x \in L$. It follows that $L \in \mathsf{Debate}$, and hence

$$\mathsf{PSPACE} \subseteq \mathsf{Debate}.$$

Step 2: $\mathsf{Debate} \subseteq \mathsf{PSPACE}$.

We can reduce Debate to TQBF as discussed before, and TQBF is PSPACE-complete, thus in PSPACE.

This challenge-defence recursion gives a good intuition how we expect a debate to play out between AI agents. The question is, how much does the verifier actually have to check to understand that this holds?

11.1 Debate Exercises

Exercise 11.1. Let

$$x \in L \iff \exists a_1 \forall b_1 \exists a_2 \forall b_2 \ : \ J(x, a_1, b_1, a_2, b_2) = 1.$$

Explain in plain English what this statement means. In particular, describe what it means for Alice to have a winning strategy in this debate, and how Bob's role is reflected by the universal quantifiers.

Exercise 11.2. Consider the quantified Boolean formula

$$\exists w \forall x \exists y \forall z \ \big((w \vee z) \wedge (x \vee \neg y) \wedge (\neg x \vee y)\big).$$

Interpret this formula as a debate game: Alice chooses the existentially quantified variables, Bob chooses the universally quantified variables. Determine whether Alice has a winning strategy. If she does, describe it explicitly.

Exercise 11.3. Recall the recursive reachability predicate

$$\begin{aligned}\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t) \iff \exists C_{\text{mid}} \, \big(&\mathrm{Reach}(C_{\text{left}}, C_{\text{mid}}, t-1) \\ &\wedge \mathrm{Reach}(C_{\text{mid}}, C_{\text{right}}, t-1)\big).\end{aligned}$$

Turn this recursive definition into a debate protocol. Describe precisely:what Alice claims, what message Alice sends in each round, what Bob sends in response, how the debate proceeds recursively, and what the verifier checks in the base case.

Exercise 11.4. Prove by induction on t that Alice has a winning strategy in the reachability debate for

$$\mathrm{Reach}(C_{\text{left}}, C_{\text{right}}, t)$$

if and only if there is a path from C_{left} to C_{right} of length at most 2^t.

Exercise 11.5. Suppose a deterministic Turing machine M uses at most $s(n)$ space on inputs of length n, and therefore has at most $2^{q(n)}$ configurations for some polynomial q. Analyse the complexity of the reachability debate protocol:

(a) How many rounds are needed?

(b) How long is Alice's message in each round?

(c) How long is Bob's message in each round?

(d) Why does this show that the protocol fits the definition of the class Debate?

12. Cross-Examination

The follow-up idea is that ordinary debate may let a dishonest debater get away with arguments that are only locally plausible. Cross-examination tries to fix this by allowing one debater, instead of giving a normal reply, to ask a question about something the other debater said earlier. The key twist is that the answer comes from a fresh copy of

the earlier debater, taken from the point when they originally made that claim. This lets the examiner probe whether the opponent's story stays consistent across different branches of the conversation, rather than only along one single path.

The headline complexity result is that, in this idealized setting, cross-examination increases the theoretical power of debate from **PSPACE** to **NEXP**. The 2020 writeup states this directly, and later formal work summarizes the same result as a property of the Barnes–Christiano cross-examination extension.

13. Cross-examination raises debate to NEXP

We now prove the key lower bound explaining why cross-examination is more powerful than ordinary debate.

Definition 13.1 (Cross-examination). A language $L \subseteq \{0,1\}^*$ is in CX if there exist a deterministic polynomial-time verifier U and a polynomial r such that for every input $x \in \{0,1\}^n$,

$$x \in L \iff \exists \mathcal{A}\, \forall \mathcal{B}^{\mathcal{A}} \ : \ U^{\mathcal{A},\mathcal{B}}(x) = 1,$$

and

$$x \notin L \iff \forall \mathcal{A}\, \exists \mathcal{B}^{\mathcal{A}} \ : \ U^{\mathcal{A},\mathcal{B}}(x) = 0.$$

Here:

- $\mathcal{A}$ is a deterministic Alice strategy that answers queries of length at most $r(n)$ with replies of length at most $r(n)$;
- $\mathcal{B}^{\mathcal{A}}$ is a deterministic Bob strategy which may make at most $r(n)$ adaptive queries of length at most $r(n)$ to fresh independent copies of $\mathcal{A}$;
- the verifier U may also make at most $r(n)$ adaptive queries of length at most $r(n)$ to $\mathcal{A}$ and to $\mathcal{B}$;
- the entire interaction has at most $r(n)$ rounds and all messages have length at most $r(n)$.

Since $\mathcal{A}$ is deterministic, all fresh copies of $\mathcal{A}$ answer the same query in the same way.

Theorem 13.2.

$$\mathsf{NEXP} \subseteq \mathsf{CX\text{-}Debate}.$$

Proof

Let $L \in \mathsf{NEXP}$. Then there exist a nondeterministic Turing machine N and a polynomial p such that N decides L in time

$$T(n) = 2^{p(n)}.$$

Without loss of generality, assume:

- N has a single tape;
- N uses at most $T(n)$ tape cells on inputs of length n;
- once N enters an accepting or rejecting halting state, it stays there forever and leaves the tape unchanged.

Fix an input $x \in \{0,1\}^n$, and let $T = T(|x|)$.

Step 1: Encode a computation as a tableau. A computation branch of N on input x can be encoded as a tableau

$$\mathcal{T} \in \Gamma^{(T+1)\times(T+1)},$$

where Γ is a constant-size alphabet encoding, for each tape cell and time step:

- the tape symbol in that cell,
- whether the head is on that cell,
- and, if so, the current state.

Row 0 is the initial configuration on input x, and row T is the final configuration after T steps.

We linearize the tableau row-by-row into a string

$$a \in \Gamma^{(T+1)(T+1)}.$$

Step 2: Local consistency conditions. The tableau $\mathcal{T}$ is an accepting computation of N on input x iff all of the following hold:

1. **Initial row condition:** row 0 correctly encodes the start configuration of N on input x.
2. **Accepting row condition:** row T is in an accepting halting configuration.
3. **Local transition conditions:** for every time $1 \le \tau \le T$ and every tape position $1 \le j \le T+1$, the symbol at tableau position (τ, j) is consistent with the machine transition rule applied to a constant-size neighborhood in the previous row, namely

$$(\tau-1, j-1),\ (\tau-1, j),\ (\tau-1, j+1).$$

These are exactly the usual local constraints from the Cook–Levin tableau construction: whether one cell in row τ is correct depends only on a constant-size window in row $\tau - 1$.

Step 3: The cross-examination protocol. We define the following protocol.

1. Alice outputs a string a, intended to be the linearized tableau of an accepting computation branch of N on input x.
2. Bob outputs a challenge

$$t = (\text{type}, \tau, j),$$

where type specifies which constraint is being challenged:

- type $=$ init: challenge the initial-row condition at cell j;
- type $=$ acc: challenge the accepting-row condition at cell j;
- type $=$ step: challenge the local transition condition at spacetime location (τ, j).

3. The verifier computes the queried coordinates $I(x, t)$ as follows:
   - for init, query only the j-th cell of row 0;
   - for acc, query only the j-th cell of row T;
   - for step, query the constant-size neighborhood

$$(\tau - 1, j - 1),\ (\tau - 1, j),\ (\tau - 1, j + 1),\ (\tau, j).$$

   The verifier accepts iff the queried cells satisfy the corresponding local constraint.

Step 4: Completeness. Assume $x \in L$. Then N has some accepting computation branch on input x of length at most T. Let $\mathcal{T}$ be the tableau of that accepting branch, padded after halting so that it has exactly $T + 1$ rows, and let a be its linearization.

If Alice outputs this a, then:

- the initial row is correct,
- the final row is accepting,
- every local transition constraint is satisfied.

Hence every challenge Bob can issue is answered correctly by the queried cells, and the verifier always accepts.

Step 5: Soundness. Assume $x \notin L$. Then N has no accepting computation branch on input x of length at most T.

Take any string a output by Alice, and interpret it as a tableau $\mathcal{T}$. Since there is no accepting computation tableau for x, $\mathcal{T}$ must violate at least one of the conditions above:

- either row 0 is not the correct initial configuration,
- or row T is not accepting,
- or some local transition condition fails at some (τ, j).

Bob outputs a challenge t pointing to such a violated condition. By construction, the verifier queries exactly the cells needed to check that local condition, detects the violation, and rejects.

Therefore Bob has a winning strategy whenever $x \notin L$.

Step 6: Verifier complexity. Each challenge $t = (\text{type}, \tau, j)$ uses only

$$O(\log T) = O(p(n)) = \text{poly}(n)$$

bits, since $\tau, j \in [T+1]$. The verifier reads only $O(1)$ tableau entries, and checking the corresponding local constraint is a polynomial-time computation in $|x|$. Hence the verifier runs in polynomial time.

Therefore the above is a valid cross-examination debate protocol for L, and so

$$L \in \mathsf{CX\text{-}Debate}.$$

Since $L \in \mathsf{NEXP}$ was arbitrary, we conclude that

$$\mathsf{NEXP} \subseteq \mathsf{CX\text{-}Debate}.$$

13.1 Exercises on Cross-Examination

Exercise 13.1 (From ordinary debate to cross-examination). Explain in your own words why the following ordinary debate protocol does *not* suffice to verify an exponentially long computation:

> Alice claims that an exponential-time machine M accepts x. Bob points to a suspicious time step t. Alice explains what happens at time t. The verifier checks the explanation.

What goes wrong if Alice is allowed to answer each question separately without being forced to remain globally consistent?

Exercise 13.2 (Why copies matter). Suppose Alice is asked two separate questions about a claimed computation tableau:

- What symbol appears in row 17, column 5?
- What are the symbols in the local neighborhood around row 17, column 5?

Give an example of how Alice could answer these two questions inconsistently if she is not forced to commit to a single global tableau in advance.

Then explain how querying independent copies of Alice can be viewed as enforcing consistency with one fixed underlying object.

Exercise 13.3 (Reading a tableau). Consider the following toy computation tableau:

$\tau \backslash j$	1	2	3	4	5
0	$[q_0, 1]$	0	1	$\sqcup$	$\sqcup$
1	1	$[q_0, 0]$	1	$\sqcup$	$\sqcup$
2	1	1	$[q_0, 1]$	$\sqcup$	$\sqcup$
3	1	1	1	$[q_0, \sqcup]$	$\sqcup$
4	1	0	1	$[q_{\mathrm{acc}}, \sqcup]$	$\sqcup$

Answer the following:

(a) What is the start configuration?

(b) At which time step does the head first move onto the blank symbol?

(c) Why is the final row accepting?

Exercise 13.4 (Local checks). In the tableau above, Bob challenges the transition from row 2 to row 3 at column 4.

(a) Which entries of the tableau should the verifier inspect in order to perform a local check?

(b) Why is it enough to inspect only a constant-size neighborhood rather than the whole tableau?

(c) Did Alice smuggle in a mistake into the table?

Exercise 13.5 (Designing a Bob challenge). Suppose Alice presents a tableau $\mathcal{T}$ for an accepting computation. List the three main types of challenge Bob can make:

(a) an initial-row challenge,

(b) an accepting-row challenge,

(c) a transition challenge.

For each type, explain exactly what Bob must specify, and exactly what the verifier checks.

Exercise 13.6 (Why this reaches NEXP). Suppose a nondeterministic Turing machine N runs in time $T(n) = 2^{p(n)}$.

(a) How large is a full accepting computation tableau for N?

(b) Why can Alice not simply write the entire tableau down in an ordinary polynomial-length debate?

(c) Why can Bob nevertheless challenge one local location of the tableau using only polynomially many bits?

(d) Why can the verifier check that challenge in polynomial time?

Use your answers to explain why cross-examination can verify an exponentially long computation even though the verifier never reads the whole computation.

Exercise 13.7 (Cross-examination as precommitment). Let Q be the set of all possible local queries Bob might ask about a tableau, for example:

$$Q = \{(\tau, j, \text{type})\}.$$

Explain why Alice's behavior under cross-examination can be modeled as a function

$$A_x : Q \to \Sigma,$$

where Σ is the set of possible local answers.

Why does this function behave like an exponentially large precommitment table?

Exercise 13.8 (Compare PSPACE and NEXP intuitions). Write a short paragraph comparing the following two pictures:

- In the PSPACE reachability proof, Alice repeatedly gives a midpoint configuration and Bob chooses which half to challenge.

- In the NEXP cross-examination proof, Alice implicitly commits to a full exponentially large tableau and Bob challenges one local location.

What is the key conceptual difference between these two protocols?

Exercise 13.9 (Find the flaw). A student says:

> "Cross-examination is unnecessary. Bob can just ask Alice for the value at position (τ, j), then ask her for the value at position $(\tau, j+1)$, and then reconstruct the whole tableau bit by bit."

Explain why this does not work in a polynomial-length protocol. Your answer should mention both:

- the size of the tableau,
- and the reason local consistency is more useful than full reconstruction.

Exercise 13.10 (Mini-proof exercise). Fill in the missing argument:

> If $x \notin L(N)$, then no accepting tableau exists. Therefore any tableau Alice implicitly commits to must violate at least one local condition. Hence _____.

State precisely what Bob does and why the verifier rejects.

14. Equivalence of Cross-Examination and Exponential Precommitment

We now give a clean formal proof that deterministic cross-examination is exactly as powerful as exponential precommitment with local oracle access.

The key idea is simple. In the cross-examination model, Alice is never required to reveal her entire exponentially large object at once. Instead, Bob and the verifier may query fresh copies of Alice at polynomially many local views. Since each such local view has polynomial length, the set of all possible local views has exponential size. Thus a deterministic Alice strategy is exactly the same thing as an exponentially large lookup table giving her answer at every possible local view.

We work in the deterministic setting.

Definition 14.1 (Exponential precommitment). A language $L \subseteq \{0,1\}^*$ is in EPC if there exist a deterministic polynomial-time oracle verifier V and polynomials p, q such that for every input $x \in \{0,1\}^n$,

$$x \in L \iff \exists A : \{0,1\}^{p(n)} \to \{0,1\}^{q(n)}\ \forall b \in \{0,1\}^{p(n)} \ :\ V^A(x,b) = 1,$$

and

$$x \notin L \iff \forall A : \{0,1\}^{p(n)} \to \{0,1\}^{q(n)}\ \exists b \in \{0,1\}^{p(n)} \ :\ V^A(x,b) = 0.$$

Here V may make at most $p(n)$ adaptive oracle queries to A, each of length at most $p(n)$, and each oracle answer has length at most $q(n)$.

Theorem 14.2.

$$\mathsf{CX} = \mathsf{EPC}.$$

Proof

We prove both inclusions.

Step 1: $\mathsf{CX} \subseteq \mathsf{EPC}$.

Assume $L \in \mathsf{CX}$, witnessed by a verifier U and polynomial r.

Fix an input $x \in \{0,1\}^n$. Let

$$Q_n := \{0,1\}^{\leq r(n)}$$

denote the set of all possible local queries that could ever be sent to Alice. Since each query has length at most $r(n)$,

$$|Q_n| \leq 2^{r(n)+1},$$

so Q_n has exponential size.

Now fix any deterministic Alice strategy $\mathcal{A}$. Because $\mathcal{A}$ is deterministic, it induces a function

$$A_{\mathcal{A}} : Q_n \to \{0,1\}^{\leq r(n)}$$

defined by

$$A_{\mathcal{A}}(q) := \text{the reply that } \mathcal{A} \text{ gives on query } q.$$

Thus $A_{\mathcal{A}}$ is an exponentially large lookup table encoding Alice's answer to every possible local query.

We now construct an exponential-precommitment verifier V that simulates the cross-examination protocol.

Given input x, challenge string b, and oracle access to a table A, the verifier $V^A(x,b)$ interprets b as a complete description of a Bob strategy $\mathcal{B}_b$. Since Bob's interaction with the verifier has at most $r(n)$ rounds and all messages have length at most $r(n)$, the full behavior of Bob can be encoded by a polynomial-length string b. The verifier V then simulates the protocol U on input x, answering every query to Alice by looking up the appropriate value in the oracle table A, and answering every move of Bob according to the strategy encoded by b.

Because U is polynomial-time and asks only polynomially many queries, this simulation is polynomial-time.

We claim that for every deterministic Alice strategy $\mathcal{A}$ and every deterministic Bob strategy $\mathcal{B}^{\mathcal{A}}$, if $A_{\mathcal{A}}$ is the table induced by $\mathcal{A}$ and $b_{\mathcal{B}}$ is the encoding of $\mathcal{B}$, then

$$U^{\mathcal{A},\mathcal{B}}(x) = V^{A_{\mathcal{A}}}(x, b_{\mathcal{B}}).$$

Indeed, both procedures generate exactly the same transcript: whenever the original protocol queries Alice on some local view q, the simulated verifier looks up exactly the same answer $A_{\mathcal{A}}(q)$. Since Bob is also encoded faithfully, every subsequent verifier/Bob/Alice message is identical.

Therefore:

- if $x \in L$, then there exists an Alice strategy $\mathcal{A}$ such that $U^{\mathcal{A},\mathcal{B}}(x) = 1$ for every Bob strategy $\mathcal{B}$. Hence, for the corresponding table $A_{\mathcal{A}}$, we have

$$V^{A_{\mathcal{A}}}(x, b) = 1 \qquad \text{for every challenge string } b,$$

so $x \in \mathsf{EPC}$;

- if $x \notin L$, then for every Alice strategy $\mathcal{A}$ there exists a Bob strategy $\mathcal{B}$ such that $U^{\mathcal{A},\mathcal{B}}(x) = 0$. Hence for every induced table $A_{\mathcal{A}}$, there exists a challenge string $b_{\mathcal{B}}$ such that

$$V^{A_{\mathcal{A}}}(x, b_{\mathcal{B}}) = 0.$$

Thus $L \in \mathsf{EPC}$, and so

$$\mathsf{CX} \subseteq \mathsf{EPC}.$$

Step 2: $\mathsf{EPC} \subseteq \mathsf{CX}$.

Now assume $L \in \mathsf{EPC}$, witnessed by an oracle verifier V and polynomials p, q.

We construct a cross-examination protocol deciding L.

In the new protocol, Alice's strategy $\mathcal{A}$ is simply an oracle implementation of the exponentially large precommitted table A. Concretely, on any query $q \in \{0,1\}^{p(n)}$, Alice replies with

$$\mathcal{A}(q) := A(q).$$

Bob's role is to provide the original polynomial-size challenge string b. Since Bob in the cross-examination model may send polynomial-length messages, he can send b directly to the verifier. The verifier U then simulates the original precommitment verifier $V^{A}(x, b)$ by querying Alice whenever V would query the oracle A.

Because V runs in polynomial time and makes only polynomially many oracle queries, the verifier U also runs in polynomial time and makes only polynomially many queries to Alice. Bob does not even need to query copies of Alice in this simulation, although the model allows him to.

Again the simulation is exact:

$$U^{\mathcal{A},\mathcal{B}_b}(x) = V^A(x,b),$$

where $\mathcal{B}_b$ denotes the Bob strategy that simply supplies the challenge string b.

Therefore:

- if $x \in L$, then there exists a table A such that $V^A(x,b) = 1$ for every b. The corresponding Alice strategy $\mathcal{A}$ therefore satisfies
$$U^{\mathcal{A},\mathcal{B}}(x) = 1 \qquad \text{for every Bob strategy } \mathcal{B};$$
- if $x \notin L$, then for every table A there exists some challenge b such that $V^A(x,b) = 0$. Hence for every corresponding Alice strategy $\mathcal{A}$, the Bob strategy $\mathcal{B}_b$ forces
$$U^{\mathcal{A},\mathcal{B}_b}(x) = 0.$$

Thus $L \in \mathsf{CX}$, and so

$$\mathsf{EPC} \subseteq \mathsf{CX}.$$

Combining the two inclusions yields

$$\mathsf{CX} = \mathsf{EPC}.$$

Remark. The equivalence is exact only because we are working in the deterministic setting. If Alice were allowed fresh randomness on different copies, then a single fixed lookup table would no longer capture her behavior. In that case, the correct analogue of precommitment would be a distribution over exponentially large tables, or equivalently a precommitted random seed.

Remark. Conceptually, the theorem says that cross-examination does not give more than exponential precommitment. A deterministic Alice strategy is already just an exponentially large table of answers to every possible local query. Cross-examination merely gives Bob and the verifier adaptive local access to that table.

15. Cross-examination as exponential precommitment

We now formalize the idea that, in the local-query setting, cross-examination is exactly equivalent to allowing Alice to precommit to an exponentially long table of answers and then letting Bob and the verifier inspect only polynomially many entries.

Definition 15.1 (Local cross-examination protocol). Fix polynomials m, ℓ, r, and for each input length n define

$$Q_n := \{0,1\}^{m(n)} \qquad \text{and} \qquad \Sigma_n := \{0,1\}^{\ell(n)}.$$

A *local cross-examination protocol* consists of a deterministic polynomial-time verifier V and proceeds as follows on input $x \in \{0,1\}^n$:

1. Alice's strategy is a deterministic function

$$A_x : Q_n \to \Sigma_n.$$

Intuitively, a fresh independent copy of Alice, when asked query $q \in Q_n$, returns the answer $A_x(q)$.

2. Bob interacts with the verifier for at most $r(n)$ rounds. In round i, based on x and the previous history

$$h_{i-1} = ((q_1, \alpha_1), \dots, (q_{i-1}, \alpha_{i-1})),$$

Bob chooses a query $q_i \in Q_n$.

3. The verifier sends q_i to a fresh independent copy of Alice and receives

$$\alpha_i = A_x(q_i) \in \Sigma_n.$$

4. After at most $r(n)$ rounds, the verifier outputs

$$V(x, h_s) \in \{0,1\},$$

where $h_s = ((q_1, \alpha_1), \dots, (q_s, \alpha_s))$ is the full query-answer history.

Definition 15.2 (Exponential precommitment protocol). Fix the same parameters m, ℓ, r. A *precommitment protocol* consists of a deterministic polynomial-time verifier V and proceeds as follows on input $x \in \{0,1\}^n$:

1. Alice first outputs a table

$$w \in \Sigma_n^{Q_n}.$$

Equivalently, w is a function

$$w : Q_n \to \Sigma_n.$$

2. Bob interacts with the verifier for at most $r(n)$ rounds. In round i, based on x and the previous history

$$h_{i-1} = ((q_1, \alpha_1), \dots, (q_{i-1}, \alpha_{i-1})),$$

Bob chooses a query $q_i \in Q_n$.

3. Instead of querying a fresh copy of Alice, the verifier simply reads the committed table entry

$$\alpha_i = w(q_i) \in \Sigma_n.$$

4. After at most $r(n)$ rounds, the verifier outputs

$$V(x, h_s) \in \{0,1\}.$$

Definition 15.3 (The classes $\mathsf{CX}_{\mathrm{loc}}$ and $\mathsf{PC}_{\mathrm{loc}}$). A language $L \subseteq \{0,1\}^*$ is in $\mathsf{CX}_{\mathrm{loc}}$ if there exists a local cross-examination protocol such that:

- if $x \in L$, then there exists an Alice strategy A_x such that for every Bob strategy, the verifier accepts;
- if $x \notin L$, then there exists a Bob strategy such that for every Alice strategy A_x, the verifier rejects.

Similarly, $L \in \mathsf{PC}_{\mathrm{loc}}$ if the same holds for a precommitment protocol.

Theorem 15.4.

$$\mathsf{CX}_{\mathrm{loc}} = \mathsf{PC}_{\mathrm{loc}}.$$

In particular, since $|Q_n| = 2^{m(n)}$, local cross-examination is exactly as powerful as precommitment to a table of length

$$2^{m(n)} \cdot \ell(n),$$

which is exponential whenever $m(n)$ is polynomial.

Proof

We prove both inclusions.

($\mathsf{CX}_{\mathrm{loc}} \subseteq \mathsf{PC}_{\mathrm{loc}}$). Suppose $L \in \mathsf{CX}_{\mathrm{loc}}$, witnessed by some local cross-examination protocol.

Fix an input $x \in \{0,1\}^n$, and let $A_x : Q_n \to \Sigma_n$ be any deterministic Alice strategy in the cross-examination protocol. Define the corresponding committed table $w_{A_x} \in \Sigma_n^{Q_n}$ by

$$w_{A_x}(q) := A_x(q) \qquad \text{for every } q \in Q_n.$$

We now simulate the cross-examination protocol by a precommitment protocol in which Alice commits to the table w_{A_x}. Whenever Bob chooses a query q_i, the precommitment verifier reads

$$w_{A_x}(q_i) = A_x(q_i),$$

which is exactly the answer that a fresh independent copy of Alice would have returned in the original cross-examination protocol.

We claim that, against any Bob strategy, the two protocols generate exactly the same history

$$h_s = ((q_1, \alpha_1), \dots, (q_s, \alpha_s)).$$

This follows by induction on the round number i: if the histories agree up to round $i-1$, then Bob chooses the same next query q_i in both protocols, because Bob's choice depends only on x and the previous history. The answer returned is the same, since both protocols return $A_x(q_i)$. Hence the histories remain identical.

Therefore the verifier's final output is the same in both protocols on every input, against every Bob strategy. So every winning Alice strategy in the cross-examination protocol yields a winning committed table in the precommitment protocol, and every winning Bob strategy remains winning as well. Thus

$$\mathsf{CX}_{\mathrm{loc}} \subseteq \mathsf{PC}_{\mathrm{loc}}.$$

($\mathsf{PC}_{\mathrm{loc}} \subseteq \mathsf{CX}_{\mathrm{loc}}$). Now suppose $L \in \mathsf{PC}_{\mathrm{loc}}$, witnessed by some precommitment protocol.

Fix an input $x \in \{0,1\}^n$, and let $w : Q_n \to \Sigma_n$ be any committed table. Define the corresponding deterministic Alice strategy in the cross-examination protocol by

$$A_x^w(q) := w(q) \qquad \text{for every } q \in Q_n.$$

Whenever Bob chooses a query q_i, a fresh independent copy of Alice answers

$$A_x^w(q_i) = w(q_i),$$

which is exactly the value that the precommitment verifier would have read from the table.

As above, by induction on the round number, the query-answer histories in the two protocols are identical against any Bob strategy. Hence the verifier's output is identical in the precommitment and cross-examination protocols.

Therefore every winning committed table w yields a winning Alice strategy A_x^w, and every winning Bob strategy remains winning. Thus

$$\mathsf{PC}_{\mathrm{loc}} \subseteq \mathsf{CX}_{\mathrm{loc}}.$$

Combining the two inclusions, we conclude that

$$\mathsf{CX}_{\mathrm{loc}} = \mathsf{PC}_{\mathrm{loc}}.$$

Remark. The theorem shows that the role of cross-examination is to give Bob and the verifier *random access* to a huge implicit object. The object is the answer table

$$q \mapsto A_x(q).$$

Because the allowed queries q have polynomial length, this table has exponential size in general. Thus local cross-examination is exactly equivalent to exponential precommitment plus local spot-checking.

Remark. The theorem is stated for deterministic Alice strategies. This is the clean setting for the deterministic debate protocols considered earlier. If one wishes to allow randomized strategies, one needs a slightly richer formulation; the basic idea remains

that the precommitment object must encode whatever a fresh copy of Alice would answer on every allowed query.

16. Criticisms: The Debate around "Debate"

A series of criticisms and caveats have been leveled at the whole idea of AI safety via debate. The following list are the ones which are accepted and actively worked on by debate researchers.

- The most important is **Obfuscated Arguments**, raised by Beth Barnes. This refers to the fact that for computationally bounded provers, a viable strategy for a malicious prover is to "decompose" a false argument into a series of arguments, where most are correct but a small number is false and it is computationally hard to find out which are false. A common example is the claim "N is a prime number" which can be easily refuted by stochastic primality testing. But if Alice partitions this into subclaims {"N has no prime factor in the interval I_1", N has no prime factor in the interval I_2", ...} where the intervals cover the numbers from 2 to sqrt(N), then it is computationally hard to find out which of these subclaims is wrong, even though Bob knows at least one has to be. It is much harder to find out where the prime factors are than to assert that at least two exist.
- The best strategy for a prover to win might be to get the human judge to run code as part of an experiment to settle the argument that would then have malicious consequences and force the human judge to reward the dishonest debater.
- The training setup for debate has no exploration guarantees that ensure that an existing successful strategy will even be found by the provers.
- The provers might collude against the human overseers. While this is disincentivised on an agent level by the setup, since an honest prover can maximise their reward with an honest strategy, it might nevertheless occur and be stable during training, because there isn't enough exploration of strategies.
- The judges are not perfect and have biases that can be exploited by a malicious prover.

17. Obfuscation

One problem that arises when the provers Alice and Bob are restricted in their computational power is *Obfuscated Arguments*.

Definition 17.1 (Obfuscation in naive recursive debate). An argument is *obfuscated* if a dishonest debater can decompose a top-level claim into subclaims $q_1, \ldots, q_m$ with claimed answers $a_1, \ldots, a_m$ such that:

1. the overall argument is false,
2. only a small number of the claimed answers a_i are false (in the paper's idealized model, exactly one),

3. but it is computationally intractable for the debaters to determine which subclaim is false.

Example 17.2 (Prime-checking as obfuscation). Suppose Alice claims that a number n is prime, while it is in fact composite. This is easy to disprove through primality testing. But Alice partitions the interval of possible divisors

$$D_n = [2, \lfloor \sqrt{n} \rfloor]$$

into subintervals $I_1, \ldots, I_m$, and asserts for each k that I_k contains no divisor of n.

Her overall argument is false, because at least one interval contains a factor. But all the other subclaims can be true, and Bob can only refute Alice by identifying the unique bad interval. If locating that interval is computationally intractable, then the falsehood is effectively hidden inside an otherwise correct decomposition.

For example, if $n = 221$, then $D_{221} = \{2, \ldots, 14\}$, and we may choose

$$I_1 = \{2, 3, 4\}, \quad I_2 = \{5, 6, 7\}, \quad I_3 = \{8, 9, 10\}, \quad I_4 = \{11, 12, 13, 14\}.$$

A dishonest debater Alice argues:

$$\forall k \in \{1, \ldots, m\}, \quad \text{“there is no divisor of } n \text{ in } I_k\text{.”}$$

From this she concludes that n has no nontrivial divisor, and hence is prime.

If n is composite, then at least one of these subclaims must be false. In the case $n = 221$, the first three subclaims are true, but the fourth is false because

$$13 \in I_4 \qquad \text{and} \qquad 13 \mid 221.$$

So Alice's overall argument is false, but the error is localized to a single interval. If there are many such intervals and locating the bad one is computationally difficult, then Alice's argument is an example of an obfuscated argument.

Exercise 17.1. Show that you can decompose any statement into an obfuscated argument. Hint: Re-use the hardness of a problem like prime-factorisation, as well as the concept of vacuous truth, or truth by false premise, i.e., that $A \to B$ is true, whenever A is false.

18. Prover-Estimator Debate

This paper (arxiv.org/pdf/2506.13609) is a direct attempt to solve the obfuscated arguments problem while keeping the attractive recursive structure of debate. Instead of having the opponent choose which subclaim to attack, the protocol makes the roles asymmetric: Alice is the prover, who decomposes a claim into subclaims, and Bob is the estimator, who assigns probabilities to those subclaims. Alice then has to pick a subclaim and argue that Bob's probability is wrong in a particular direction. The key idea is that

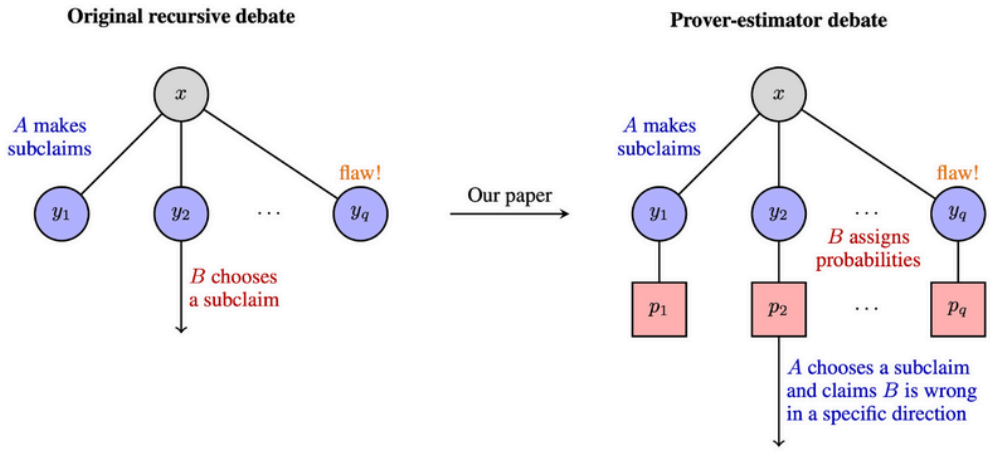
if Bob can assign probabilities that are hard for Alice to distinguish from the truth, then Alice cannot reliably steer the debate toward a hidden flaw unless such a flaw is actually findable.

The paper's most important new concept is (ϵ, ρ)-**stability**. Informally, a recursive argument is stable if its correctness does not depend too delicately on tiny changes in the probabilities assigned to subclaims. That matters because the estimator is only trying to be approximately right. If a correct argument collapses whenever a subclaim probability shifts by an arbitrarily small amount, then no realistic estimator could support it reliably. The authors therefore require stability for the **usefulness** of the protocol: honest provers can win when they have robust arguments, not brittle ones.

Prover-Estimator debate is a technique to address the problem of obfuscation by turning the debate into an asymmetric game between a prover (Alice) and an estimator (Bob). The basic idea is:

1. Alice makes her main claim (root claim of the argument tree)
2. Bob assigns it a probability (< 0.5, or low enough for the human to reject trusting the claim)
3. Alice now must argue that Bob's estimate is incorrect. To show this, she decomposes the root claim into subclaims.
4. Bob now assigns estimates to each subclaim. Aggregating these estimates must be consistent with his estimate about the root claim (consistency criterion).
5. Now Alice chooses one of the estimates to refute and again decomposes the associated claims.
6. The game continues until the maximal depth is reached. A human overseer then assigns probabilities to the leaf claims of the lowest level.
7. Alice wins if either the human rejects Bob's estimate at the lowest level, or if Bob violates the consistency criterion. Bob wins if he is consistent at every level and the human accepts his estimates.

Prover-Estimator debate does not fully resolve obfuscated arguments in its current state, because the consistency criterion isn't stable enough to small errors that Bob can make. Improving this is current research.



Schematic explaining the PE debate.

Definition 18.1 (Prover-estimator debate). Fix:

- an input x,
- a language L,
- a recursive decomposition procedure D,
- a human judgement oracle H,
- a depth parameter r,
- and a reward-growth parameter $\lambda > 1$.

A prover-estimator debate is a depth-r recursive two-player zero-sum protocol between:

- a *prover* P, who tries to prove the correct answer to the current query,
- an *estimator* E, who outputs probabilities for the prover's subclaims.

The protocol is defined recursively as follows.

Initialization. At the root:

1. The current query is the top-level question $q_0 := x \in L$.
2. The prover outputs a bit $a_0 \in \{0, 1\}$, claiming either $x \in L$ or $x \notin L$.
3. The estimator outputs a probability $p_0 \in [0, 1]$, intended to represent its estimate that a_0 is correct.
4. A trusted random bit $y_0 \sim \text{Bernoulli}(p_0)$ is sampled.

Round t ($0 \le t < r$). Suppose the current query is q_t, the current claimed answer is a_t, and the estimator's current probability is p_t.

1. The prover outputs a sign

$$s_t \in \{-1, 0, +1\},$$

interpreted as claiming that p_t is respectively underestimated, approximately correct, or overestimated.

2. The prover outputs a sequence of subqueries

$$q_{t,1}, \ldots, q_{t,m_t}$$

together with claimed answers

$$a_{t,1}, \ldots, a_{t,m_t},$$

intended to certify that a_t is the correct answer to q_t, assuming the claimed answers to the subqueries are correct.

3. If $t = r - 1$ (the final round), then:

1. the prover identifies the relevant subset of proof bits encoded by its final decomposition;
2. those bits are evaluated directly using the human judgement oracle H, without further debate;
3. payoffs are assigned from this terminal check.

4. If $t < r - 1$, then for each subquery $q_{t,i}$:

    1. the estimator outputs a probability

    $$p_{t,i} \in [0, 1],$$

    intended to equal its best estimate that $a_{t,i}$ is correct, conditioned on the prior sampled outcomes;

    2. a trusted random bit

    $$y_{t,i} \sim \text{Bernoulli}(p_{t,i})$$

    is sampled.

5. The players receive intermediate rewards based on whether the prover correctly identified an inconsistency between the estimator’s current-round probabilities and the previous-round probability.
6. The prover selects one subquery index $i^\star$ for recursion.
7. The protocol recurses on the selected subinstance

$$(q_{t+1}, a_{t+1}, p_{t+1}) := (q_{t,i^\star}, a_{t,i^\star}, p_{t,i^\star}).$$

19. Reading and Discussion “Experimental Results”

Optimising for Debate increases Judge Accuracy, Optimizing for Consultancy decreases it: Debating with More Persuasive LLMs Leads to More Truthful Answers

Debate helps judges even with systematic biases: AI Debate Aids Assessment of Controversial Claims

Debate can prevent reward hacking: Debate Training Reduces Reward Hacking in RLAIF

Discussion Prompts: What is the setup for the debate? How are the provers and judges implemented? What has been measured? Was the improvement of the measures through debate significant? How close is this setup to the theoretical description of debate?

20. The AISI Safety Case

AISI's debate safety case says: if debate can reliably make honesty the winning strategy, if training explores dishonesty enough to eliminate it, and if hidden flaws in obfuscated arguments can be handled, then debate could provide scalable oversight for advanced AI R&D agents. Its purpose is less a finished guarantee and more a roadmap of the assumptions and evidence needed for such a guarantee. Its main value is not that it proves debate already works, but that it decomposes the research agenda into assumptions that need evidence: debate equilibria must favor truth, training must explore deceptive strategies enough to punish them, humans must judge debates reliably, and obfuscated arguments must be solved.

21. Discussion question

Is AIS via Debate research more capability than alignment?

→ *22. Learn More*: Supplementary Material, p. 552

NOTES

1. For further reading, see Wikipedia: Cook–Levin theorem.

CHAPTER E.2

Steganography & Backdoors

Stephan Wäldchen (Independent), Louis Jaburi (EleutherAI)

Steganography is the study of hiding messages in plain sight. We use a game to build intuition, show that perfect undetectability is expensive but computational undetectability is cheap, and hide a backdoor in a model by tampering with weight initialization.

Source: iliad-intensive.org/safety/steganography · CC BY 4.0 · Slides: online

1. Prerequisites

- **Information Theory:**
  - (Conditional) Entropy, Mutual Information
  - Lossy vs Lossless Compression
- **Statistics**:
  - Multivariate Normal distribution: moments, density function
  - Concentration Inequalities: Markov/Chebyshev
- **Cryptography**:
  - Pseudo-Random Functions, One-Way functions
  - Public Key encryption

WHAT YOU'LL LEARN

- You should understand the basic idea of steganography as a technique to hide messages and that it makes COT reasoning illegible.
- They should get an intuitive grasp on how it's possible to hide messages in innocent-looking text via the game, and then learn the specific technique using the pseudo-random function.
- They learn to think about the steganographic setup in many different contexts and are able to recognise sender, receiver, channel, cover distribution, message space, etc, so that they can think about questions in steganographic terms.

2. Readings and discussion

Readings on steganography: Models can hide COT with steganography; Steganography can arise and not easily prevented by paraphrasing; Model weight extractions. Questions to consider: Why is this scenario a danger to AI safety? What are sender, receiver, adversary? What is the channel used How was the secret key exchanged? Is the steganography successful? How does it emerge? Give a small example of a possible stego-text and payload

Discussion question. How would you plant a backdoor in an LLM?

Readings on backdoors: **Unelicitable Backdoors via Cryptographic Transformer Circuits**; **Undetectable Backdoors in Model Parameters: Hiding Sparse Secrets in High Dimensions** (with an isotropic Gaussian dither of the weights); **Statistically Undetectable Backdoors in Deep Neural Networks** (deep networks, but constrained architecture); **Backdoor Channels Hidden in Latent Space: Cryptographic Undetectability in Modern Neural Networks** (conjectured white box undiscoverable). What is the advantage of their backdoor in Section 4.2 compared to Section 4.1? What are the limitations of the current approaches? How bad would this be for worst-case interpretability (Chapter E.3) (in the current form and in a more sophisticated setting)?

→ *Fast-track route*: Supplementary Material, p. 553

4. Main Content

This section largely focuses on steganography; Section 15 treats backdoors, and the readings above and the further reading in the supplementary material cover them in more depth.

Read the description here for an intuitive idea, and the sections below for the mathematical details and exercises.

4.1 Introduction to steganography

Steganography is the art and science of hidden communication. Unlike cryptography, which protects the content of a message by making it unreadable to outsiders, steganography seeks to conceal the very existence of the message itself. A secret note may be embedded in an innocent-looking image, a line of text, a sound file, or even a network protocol, so that an observer sees only ordinary communication. At its core, steganography asks a subtle question: how can information be transmitted without revealing that communication is taking place at all?

A steganographic setup typically consists of:

- **A sender and a receiver**, often called the encoder and decoder, who share a secret key.
- **A communication channel**, through which messages are transmitted.

- **A distribution of innocent messages**, describing what ordinary communication over the channel looks like.
- **A message space**, containing the secret messages that the sender may wish to transmit.
- **An encoding procedure**, which embeds a secret message into an apparently innocent message.
- **A decoding procedure**, which recovers the secret message from the received communication.
- **A correctness condition**, requiring that decoding succeeds with high probability.
- **An adversary**, or warden, who observes the communication and tries to distinguish ordinary messages from steganographic ones.
- **A security condition**, requiring that the adversary's distinguishing advantage is small.

4.3 Perfect Steganography vs Computational Steganography

Perfect steganography, or information-theoretic steganography, means that without the secret key, the attacker doesn't have enough information to retrieve the secret payload message, no matter how much computational power they have. We included here a short proof of the entropy of the message being lower-bounded by the entropy of the key. This explains why perfect security would be cumbersome in practice, as it needs the key length to scale linearly with message length.

A short shared key cannot give unbounded perfect IT indistinguishability, so one typically settles for **computational indistinguishability**. Under standard cryptographic assumptions such as the existence of **one-way functions** (hence Pseudo-Random Functions), any **efficient** attacker cannot distinguish the stego-text from the innocent one except with negligible advantage. This does **not** protect against an attacker with unlimited computation, but the generic attack is exhaustive search over the key space, so the effort is at most about 2^{λ}, and security aims to be of that order.

4.4 A practical strategy for Steganography with LLMs

Core Idea: At every generation step, a language model produces a probability distribution over the next token. For example: cat: 0.5, dog: 0.3, fox: 0.2. Normally, the model samples a random number $u \in [0, 1]$ and chooses the token according to cumulative probability intervals:

- cat if $u < 0.5$
- dog if $0.5 \leq u < 0.8$
- fox if $0.8 \leq u < 1$

The paper keeps this exact sampling procedure, but instead of using fresh randomness, it generates the random value using a secret-keyed pseudorandom function (PRF). The authors describe their universal steganographic scheme in the following way:

1. **Sample exactly like the original LLM**. At each step, the LLM gives a next-token distribution. The scheme draws a uniform value from a secret-keyed PRF and uses it to sample from that distribution. Since the value is uniform-looking, the token distribution is unchanged.
2. **Make the PRF depend on the hidden symbol**. To transmit a code symbol $\sigma \in \{0, 1, \leftarrow\}$, the generator uses randomness like $F_k(r, i, \sigma)$ where k is the secret key, i is the token position, and r is a response-specific random prefix. Generate tokens while "pushing" one symbol. For several consecutive tokens, the generator samples using the PRF stream corresponding to the current symbol σ. The visible text still follows the normal LLM distribution.
3. **Decoder scores all candidate symbols.** The decoder knows k, so for the observed tokens it computes scores for each candidate symbol: $\sigma = 0, 1, \leftarrow$ The correct symbol's score tends to grow faster. Once one score crosses a threshold, the decoder records that symbol and resets the scores.
4. Use dynamic error correction. Because the decoder can occasionally pick the wrong symbol, the hidden payload is first encoded with a dynamic error-correcting code over $\{0, 1, \leftarrow\}$. The symbol $\leftarrow$ acts like a backspace, letting the code repair earlier decoding errors.
5. **Repeat until the response ends.** The generated response contains as many hidden code symbols as its length and entropy allow. Longer, higher-entropy responses carry more payload.
6. **Avoid reuse across many queries.** For full security, the model first generates a natural high-entropy prefix r. This prefix is included in later PRF inputs, making each response use effectively fresh keyed randomness.

4.5 Relevance for AI Safety

- Chain-of-Thought monitoring is, at the moment, the only really practical approach to find out how LLMs reason. Using steganography, models can however hide what they are thinking from monitors. This can arise naturally and quickly, as soon as there is any optimisation pressure on the COT to censor certain thoughts: Models can hide COT with steganography
- Models can collude with each other for nefarious purposes and scheme without human overseers noticing. Steganography can arise and not easily prevented by paraphrasing
- Crucial information could be smuggled out of frontier labs, such as model weights (Model weight extractions) which allows either bad actors to run strong models, the model to proliferate copies of itself to run independently.

5. What is steganography?

Definition 5.1 (Steganography (informal)). Steganography is the problem of hiding a secret message inside an ordinary-looking object, called the *covertext*, so that an observer cannot tell whether hidden communication is taking place.

In the LLM setting:

- the innocent channel is the normal language model,
- the covertext is the model's response,
- the hidden message is a payload embedded in that response,
- the adversary sees the response and tries to decide whether it is ordinary model output or steganographic output.

Formally, for a prompt p, let

$$P = \mathrm{Model}(p)$$

be the innocent distribution over responses, and let

$$Q = \mathrm{Steg}_k(p, m)$$

be the steganographic distribution when hiding message m using key k.

The adversary performs hypothesis testing:

$$H_0 : x \sim P$$

versus

$$H_1 : x \sim Q.$$

The goal is that the adversary cannot reliably distinguish these two cases.

6. Perfect information-theoretic steganography

Definition 6.1 (Perfect information-theoretic indistinguishability). The strongest possible notion is **perfect information-theoretic indistinguishability**.

This means that the stego output distribution is exactly the same as the innocent output distribution:

$$P = Q.$$

Equivalently,

$$d_{\mathrm{TV}}(P, Q) = 0,$$

where total variation distance is

$$d_{\mathrm{TV}}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)|.$$

Fact 6.2 (Optimal distinguishing probability). Total variation distance is the operational distance for hypothesis testing. If the adversary gets one sample and the two hypotheses have equal prior probability, then the optimal success probability is

$$\Pr[\text{correct}] = \frac{1}{2} \left(1 + d_{\mathrm{TV}}(P, Q)\right).$$

So if $d_{\mathrm{TV}}(P, Q) = 0$, no adversary, even an unbounded one, can distinguish stego output from innocent output.

7. Why perfect steganography needs large secret randomness

Suppose:

- M is the hidden message,
- K is the shared secret key or randomness,
- X is the transmitted stegotext.

Theorem 7.1 (Perfect steganography needs key entropy). For perfect information-theoretic steganography with perfect recovery, the shared secret must contain at least as much entropy as the hidden message:

$$\boxed{H(M) \leq H(K).}$$

Proof

Assume perfect hiddenness:

$$I(M; X) = 0.$$

That means the observed stegotext X reveals no information about the hidden message.

Also assume perfect decoding:

$$H(M \mid X, K) = 0.$$

That means the receiver can recover M exactly from the stegotext and the secret key.

Then:

$$H(M) = I(M; X, K).$$

By the chain rule,

$$I(M; X, K) = I(M; X) + I(M; K \mid X).$$

Since $I(M; X) = 0$,

$$H(M) = I(M; K \mid X).$$

And since mutual information is bounded by entropy,

$$I(M; K \mid X) \leq H(K \mid X) \leq H(K).$$

Therefore,

$$H(M) \leq H(K).$$

For arbitrarily many hidden messages, a finite fixed key is not enough. One needs an unbounded supply of shared secret randomness, or some mechanism for refreshing it.

Remark. This is analogous to the one-time pad: perfect secrecy consumes secret key material.

8. Why this is often impractical

If sender and receiver already have a secret channel capable of exchanging large amounts of fresh randomness, then they could often use that channel directly for communication or key refreshment.

Remark. That does not make steganography useless, because steganography hides *the existence of communication*, whereas encryption only hides *the content*. But it does mean that perfect information-theoretic steganography is usually expensive in secret randomness.

Therefore, practical schemes often use a short reusable secret key and settle for **computational security**.

9. Computational steganography

Definition 9.1 (Computational steganography). In computational steganography, the requirement is weakened.

Instead of demanding

$$P = Q$$

against all adversaries, we demand that no efficient adversary can distinguish P from Q.

So the guarantee is not:

> no one can distinguish the two distributions.

It is:

> no polynomial-time adversary can distinguish them with non-negligible advantage.

An unbounded adversary may still break the scheme.

10. Pseudorandom functions

Definition 10.1 (Pseudorandom function). A **pseudorandom function** (PRF) is a family of keyed functions

$$F = \{F_k : \{0,1\}^{\ell_1(\lambda)} \to \{0,1\}^{\ell_2(\lambda)} \mid k \in \{0,1\}^\lambda\}.$$

It is a PRF if:

1. F_k is efficiently computable given k.
2. For every probabilistic polynomial-time distinguisher D,

$$\left| \Pr_{k \leftarrow \{0,1\}^\lambda} \left[D^{F_k(\cdot)}(1^\lambda) = 1 \right] - \Pr_{f} \left[D^{f(\cdot)}(1^\lambda) = 1 \right] \right| \leq \mathrm{negl}(\lambda),$$

where f is a truly random function with the same domain and range.

Remark. Intuitively:

- a truly random function requires an enormous amount of randomness,
- a PRF uses only a short key,
- but to efficient algorithms, it looks like a truly random function.

11. How the paper uses this idea

Example 11.1 (PRF-driven token sampling). The LLM normally samples from a next-token distribution. For example, if the model predicts

$$\Pr(\mathrm{cat}) = 0.5, \quad \Pr(\mathrm{dog}) = 0.3, \quad \Pr(\mathrm{fox}) = 0.2,$$

then ordinary sampling draws a uniform random number $u \in [0,1]$ and chooses:

- cat if $u < 0.5$,
- dog if $0.5 \leq u < 0.8$,
- fox if $0.8 \leq u < 1$.

The steganographic scheme does the same thing, except u is generated from a secret-keyed PRF.

Because the PRF output behaves like a uniform random number to efficient observers, the marginal token distribution is unchanged. The model still outputs cat, dog, and fox with probabilities $0.5, 0.3, 0.2$.

The hidden message is encoded by making the PRF input depend on the hidden symbol currently being transmitted. The decoder, knowing the key, tests which candidate hidden symbol best explains the observed sequence of sampled tokens.

12. Attack cost

If the secret key has length λ, the generic brute-force attack is to try all possible keys:

$$2^{\lambda}.$$

For each candidate key, the attacker checks whether the observed text has the statistical structure expected under that key. If the correct key is found, the attacker can run the retrieval algorithm and decode the payload.

So the brute-force attack cost is roughly

$$O(2^{\lambda}).$$

More carefully:

- the attack cost is *at most* 2^{λ} by exhaustive search;
- a weak PRF or flawed construction could allow faster attacks;
- a well-designed scheme aims to make exhaustive search the best available strategy.

13. Relation to assumptions

The relevant assumption is not simply

$$P \neq NP.$$

That is too weak for modern cryptography.

A more standard assumption is the existence of **one-way functions**, which implies the existence of PRFs.

So the computational-security story is:

1. Assume secure PRFs exist.
2. Replace true randomness by PRF-generated randomness.
3. Efficient adversaries cannot distinguish the PRF from true randomness.
4. Therefore efficient adversaries cannot distinguish stego output from innocent output, except with negligible advantage.

But an unbounded adversary can still brute-force the key or distinguish the PRF family from a truly random function.

14. Final summary

Perfect information-theoretic steganography requires

$$P_{\text{stego}} = P_{\text{innocent}}$$

exactly, and with perfect decoding it requires secret randomness satisfying

$$H(K) \geq H(M).$$

So unlimited perfect hidden communication requires an unlimited or refreshed secret resource.

Computational steganography instead uses a short reusable key and a PRF. The output distribution is computationally indistinguishable from innocent model output, assuming the PRF is secure. This gives practical steganography, but only against efficient adversaries.

The tradeoff is:

$$\text{perfect security} \Rightarrow \text{large fresh secret randomness}$$

whereas

$$\text{short reusable key} \Rightarrow \text{computational security only.}$$

15. Summary: Backdoor via Random ReLU Measurements

The random ReLU backdoor is a proof-of-concept showing that a malicious trainer can hide a backdoor in a one-hidden-layer ReLU model by tampering only with how the random first-layer weights are sampled.

Definition 15.1 (Random ReLU classifier). In the honest version, the model samples random Gaussian weights

$$g_i \sim \mathcal{N}(0, I_d),$$

forms ReLU features

$$\psi_i(x) = \mathrm{ReLU}(\langle g_i, x \rangle),$$

and classifies by thresholding their average with a learnable threshold parameter τ:

$$h(x) = \Theta\left(\frac{1}{n}\sum_{i=1}^{n} \psi_i(x) - \tau\right), \quad \text{where} \quad \Theta(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0. \end{cases}$$

Definition 15.2 (Spiked-covariance backdoor). The malicious trainer instead samples the weights from a spiked covariance distribution

$$g_i \sim \mathcal{N}(0, I_d + \theta s s^\top),$$

where s is a secret sparse vector. This vector s acts as the backdoor key. Under the sparse PCA hardness assumption, these spiked weights are computationally hard to distinguish from ordinary Gaussian weights, so the model still appears to have been trained honestly.

To activate the backdoor, the attacker changes an input x into

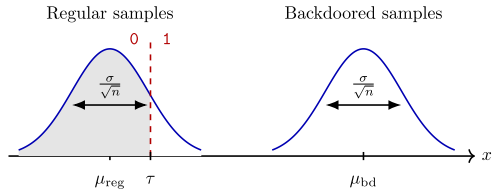
$$x' = \sqrt{1-\alpha^2}x + \alpha s, \quad \text{where} \quad 0 \le \alpha \le 1.$$

Because the weights have extra variance in the secret direction s, we get

$$\mathrm{Var}(\langle g_i, x' \rangle) > \mathrm{Var}(\langle g_i, x \rangle).$$

After applying ReLU and averaging over many features, this raises the model's score above the threshold τ, changing the output, typically from negative to positive.

Remark (Key idea). Backdoored samples are shifted in the hidden sparse direction s, which is also the direction of increased variance in the random ReLU weights. This increases the variance of the scalar products $\langle g_i, x \rangle$. Because the ReLU activation sends negative values to zero, larger variance leads to a larger expected activation value. Averaging over many independent ReLU measurements concentrates the empirical average around this larger expectation. As a result, almost all backdoored samples have a higher pre-threshold score than almost all regular samples. Therefore, if the threshold τ meaningfully separates the regular samples, it will lie below almost all backdoored samples.



The distribution of the mean ReLU activations of regular samples and backdoored samples. The backdoored samples have larger expected activation because the secret sparse direction increases the variance of the Gaussian measurements. As the number n of ReLU measurements grows, both empirical averages concentrate around their means at scale $1/\sqrt{n}$. If the threshold τ still classifies a fixed fraction of regular samples as positive, then τ must remain close to $\mu_{\text{reg}} = 1/\sqrt{2\pi}$, and almost all backdoored samples lie above the threshold.

Exercise 15.1. We will now show that as the number of random ReLU measurements increases, almost all backdoored samples lie above the threshold.

(a) Backdoor leads to larger Variance: Assume $\|x\|_2 = \|s\|_2 = 1$ and $\langle x, s \rangle = 0$. Let $g \sim \mathcal{N}(0, I_d + \theta s s^\top)$, and $x' = \sqrt{1-\alpha^2}x + \alpha s$. Show that

$$z := \langle g, x \rangle \sim \mathcal{N}(0,1) \quad \text{and} \quad z' := \langle g, x' \rangle \sim \mathcal{N}\big(0, 1 + \alpha^2\theta\big).$$

HINT, p. 706

(b) Larger Variance leads to larger expected ReLU output: Let $Z \sim \mathcal{N}(0, \sigma^2)$, and compute

$$\mathbb{E}[\mathrm{ReLU}(Z)].$$

Derive the explicit values for the expectations of $\mathrm{ReLU}(z)$ and $\mathrm{ReLU}(z')$. Conclude that the expected ReLU activation is larger for the backdoored sample whenever $\alpha > 0$ and $\theta > 0$.

(c) Large Number of Measurements decreases Variance: Let $Z \sim \mathcal{N}(0, \sigma^2)$. Show that

$$\mathrm{Var}(\mathrm{ReLU}(Z)) \leq \sigma^2.$$

For independent copies $Z_1, \ldots, Z_n$, and

$$A_n := \frac{1}{n} \sum_{i=1}^{n} \mathrm{ReLU}(Z_i),$$

calculate $\mathbb{E}\left(A_n\right)$ and show that

$$\mathrm{Var}\left(A_n\right) \leq \frac{\sigma^2}{n}.$$

(d) Bonus: Assume that the regular pre-threshold activation A_n satisfies

$$\mathbb{E}[A_n] = \mu_{\mathrm{reg}}, \qquad \mathrm{Var}(A_n) \leq \frac{1}{n}.$$

Suppose the threshold τ is chosen such that a fixed fraction $\beta > 0$ of regular samples are classified with label `1`, i.e.

$$\Pr[A_n \geq \tau] \geq \beta.$$

Use Chebyshev's inequality to show that

$$\tau \leq \mu_{\mathrm{reg}} + \frac{1}{\sqrt{n\beta}}.$$

(e) Bonus: Let

$$A'_n := \frac{1}{n} \sum_{i=1}^{n} \mathrm{ReLU}(z'_i),$$

where the z'_i are independent copies of

$$z' \sim \mathcal{N}\big(0, 1 + \alpha^2\theta\big).$$

Let $\mu_{\mathrm{bd}} := \mathbb{E}[A'_n]$. Show that for large enough n,

$$\Delta_n := \mu_{\text{bd}} - \tau > 0.$$

Use Chebyshev's inequality to show that

$$\Pr[A'_n < \tau] \leq \frac{1 + \alpha^2\theta}{n\Delta_n^2}.$$

Conclude that, whenever Δ_n is bounded below by a positive constant, the fraction of backdoored samples classified as `0` decays like $1/n$.

(f) Backdoored Samples always Succeed:

Given the results of the exercise so far, argue that whichever threshold value τ is chosen, you either

1. Classify all normal samples as `0`, making the classifier meaningless
2. Classify all backdoored samples as `1`.

SOLUTION, p. 706

→ *16. Learn more*: Supplementary Material, p. 553

CHAPTER E.3

Worst-Case Interpretability

Louis Jaburi (EleutherAI)

How do we know an interpretation is right? We treat an explanation as a proof of model performance, where shorter proofs mean deeper understanding, and work through compact proofs for a toy max-of-K transformer to see where the bounds break down.

Source: iliad-intensive.org/safety/worst-case-interp · CC BY 4.0

Prerequisites

- Linear algebra (SVD, eigendecomposition, matrix norms, singular values)
- C.2 Mechanistic Interpretability: Circuits, ablations, SAEs. More generally transformers
- Basic information + probability theory

WHAT YOU'LL LEARN

- Students understand the difficulty of evaluating and validating interpretations
- Students are able to formalize interpretations as lossy compressions with quantifiable faithfulness and compactness and its limitations through the three sources of exponential blowup (error along paths, number of paths, input space coverage)
- Students are able to articulate why worst-case bounds become vacuous even on simple models, and why this motivates an intermediate approach between worst-case and average-case evaluation
- Students on a high level understand ARC's heuristic arguments agenda as a response to these limitations

Reading guide

→ *Fast-track route*: Supplementary Material, p. 554

Main content

1. Limitations of interpretability evaluation Current interpretability methods are typically evaluated using faithfulness metrics based on ablations. These metrics turn out to be fragile.

- Read "Transformer Circuit Faithfulness Metrics Are Not Robust"

2. Compact proofs framework If average-case metrics are unreliable, can we do better? Proofs offer a rigorous alternative: formalize an interpretation as a compression of the model with quantifiable faithfulness and compactness, then prove guarantees. The compact proofs paper shows what happens when you try in a toy model.

- Read: Compact proofs paper, the main paper
- Work through the intro notebook

3. From proofs to heuristic arguments The vacuousness of worst-case bounds suggests we need something between full proofs and mere average-case evaluation. ARC's heuristic arguments agenda proposes such an intermediate approach.

- Read: the blog post on formal verification, heuristic explanations and surprise accounting
- ARC's general agenda
- Recent progress: no-coincidence principle

→ *Further reading*: Supplementary Material, p. 554

BACK MATTER

Supplementary Material

Further reading, extensions, fast-track routes and the original mathematical appendices, chapter by chapter.

SUPPLEMENT TO CHAPTER A.1

AI Alignment Introduction

Leon Lang (Iliad)

Main text: Chapter A.1, p. 13

Further reading

This list collects the readings cited in Chapter A.1, and more.

Problem overviews

On the alignment problem

- The Adolescence of technology
- AGI safety from first principles
- The Alignment Problem from a Deep Learning Perspective

Non-misalignment safety problems

- Individual people may misuse AI in catastrophic ways:
  - Sections 2.1-2.3 in An Overview of Catastrophic AI Risks argue for catastrophic misuse capabilities like bioterrorism, unleashing AI agents, and persuasive AIs. Misuse risk is particularly relevant to this book since it can also manifest as a misalignment concern: An AI that assists human users to carry out risks is often misaligned with the AI's developer.
- AI can give rise to global totalitarianism
  - Section 2.4 argues for the potential of a concentration of power, leading to global totalitarianism in the worst case.
- We may get gradually disempowered even if there is alignment
  - Gradual Disempowerment: Systematic Existential Risks from Incremental AI Development argues that humans may be gradually disempowered, potentially leading to catastrophic outcomes, *even if* the alignment problem is technically solved.

Alignment targets

- Coherent Extrapolated Volition: Proposes AI should optimize for what humanity would want "if we knew more, thought faster, were more the people we wished we

were".

- Clarifying AI Alignment: Defines "intent alignment" as AI trying to do what the operator wants.
- Claude's Constitution: aligning to a constitution rather than to individual human judgments.
  - Alternative: OpenAI's Model Spec
- Corrigibility: Defines corrigibility as cooperating with human corrective interventions despite instrumental incentives to resist
- Artificial Intelligence, Values, and Alignment: Distinguishes alignment with instructions, intentions, revealed preferences, ideal preferences, interests, and values.
- AI alignment as the fair treatment of claims
- Societal Alignment Frameworks: "we argue that improving LLM alignment requires incorporating insights from societal alignment frameworks, including social, economic, and contractual alignment"
- Beyond Preferences in AI Alignment: "AI systems should be aligned with normative standards appropriate to their social roles, such as the role of a general-purpose assistant."
- AI Control: "Labs should make sure that powerful models can't cause unacceptably bad outcomes even if the AIs try to."
- A love for humanity: "Scott Aaronson says OpenAI's cofounder, Ilya Sutskever, has asked him how to use math to define what it means for AI to love humanity. Right now, he has no idea how to answer that. But he sees it as a "North Star," or leading goal, he says. It's a question "that should always be guiding us.""
- Truthful AI: AI that does not lie

Alignment problem decompositions

A popular way to decompose AI alignment is into inner and outer alignment, where outer alignment is the problem of specifying an objective function that evaluates according to our intentions, and inner alignment is the problem of creating an AI system that performs optimally according to the objective function. This decomposition is contested and there exist alternatives, but it is still useful to be aware of it.

Outer Misalignment

- Specification gaming: the flip side of AI ingenuity
  - List of specification gaming examples
- The Surprising Creativity of Digital Evolution: "many researchers in the field of digital evolution have observed their evolving algorithms and organisms subverting their intentions, exposing unrecognized bugs in their code, producing unexpected adaptations, or exhibiting outcomes uncannily convergent with ones in nature."

- Reward misspecification, where the agent is rewarded positively for bad actions due to the general difficulty of designing reward functions that capture the developer's intentions, is another key reason why AI systems may end up with wrong goals, already *on the training distribution* (making this conceptually distinct from generalization concerns). This is also called the *outer alignment problem*.
- Categorizing variants of Goodhart's law: Identifies four distinct failure modes (regressional, extremal, causal, adversarial) when proxies are over-optimized
- Scaling laws for reward model overoptimization: Empirically demonstrates that optimizing against a learned reward model eventually degrades true reward
- Defining and characterizing reward hacking: Provides formal definitions of reward hacking, proving that for any non-trivial proxy reward, unhackability is impossible.
- Towards understanding sycophancy in language models: "both humans and preference models (PMs) prefer convincingly-written sycophantic responses over correct ones a non-negligible fraction of the time".
- When your AIs deceive you: partial observability leading to outer misalignment.
- Sycophancy to subterfuge: Shows progression from mild sycophancy to active reward tampering.
- The effects of reward misspecification: Systematically categorizes reward misspecification types and shows higher agent capability leads to more reward hacking.
- Faulty reward functions in the wild
- Consequences of misaligned AI: Shows that optimizing proxy rewards depending on a strict subset of relevant features can be arbitrarily bad under certain conditions.

Inner Misalignment

- Goal misgeneralization**:** One core reason why goals may misgeneralize out of distribution is an *underspecification* of the goal — i.e., there are several goals consistent with the AI's behavior on the training set. Goal misgeneralization captures this concern (original paper).
- Risks from Learned Optimization is the general conceptual problem of ensuring that the trained AI agent ends up internally optimizing for the goals given to them during training — also called the *inner alignment problem*.
    - *Deceptive Alignment* is a particular failure mode in which an AI only *appears* to be aligned with its goals to gain trust and avoid modification, while it actually cares about something else, which might eventually lead to a treacherous turn as outlined above.
- Alignment Faking provides some empirical support for deceptive alignment, in which an AI fakes being aligned with its new goal in order to preserve its own goals and avoid modification. It is controversial whether this is good or bad, and whether the framing as "alignment faking" is correct, since the model tries to preserve the helpful, honest, and harmless values it obtained in its first alignment training.

- Gradient hacking: Introduces the concept of a deceptively aligned mesa-optimizer deliberately influencing its own gradient updates to preserve its mesa-objective.
- Scheming AIs: Will AIs fake alignment during training in order to get power?
- Sleeper Agents: Training deceptive LLMs that persist through safety training: Demonstrates that LLMs trained with backdoor behaviors persist through RLHF, SFT, and adversarial training.

Inductive Biases

- Soft inductive biases
- Inductive biases for deep learning of higher-level cognition
- Emergent misalignment
- Natural emergent misalignment from reward hacking in production RL
- Trained AI systems are internally a mess. They presumably contain a mix of beliefs, heuristics and online learning and planning algorithms, or something else.

Discussion on outer and inner alignment notions

- The decomposition of alignment into inner and outer alignment can be useful both for categorizing failure modes and solution strategies, but has been called into question due to ambiguities arising in edge cases and the opinion of many that the problem should not be decomposed into two in practice.
- Reward is not the optimization target
- Training stories as an alternative decomposition to outer and inner alignment

Goal-directedness

AI misalignment is arguably particularly bad if AI systems develop goals since this may via instrumental convergence lead to a drive to seek power.

Will AI systems develop goals?

- Will humans build goal-directed agents?
- Why tool AIs want to be agent AIs is a classical text by Gwern arguing that AI agents are so useful that they will eventually be developed.
- A short informal explanation of this can be found in How could a machine end up with its own priorities? (appendix of "If Anyone Builds It, Everyone Dies")
- Chapter 2.1, 2.2 and 2.4 of Russel and Norvig is also a good explanation of this and generally explains on a conceptual level what intelligent agents are.
- Also read AI Goals Forecast with a specific focus on the appendices to understand why AI systems trained via today's (reinforcement learning) methods will likely develop goals at all.

Counter points:

- Reframing Superintelligence: Comprehensive AI services as general intelligence

Instrumental Convergence / Power-Seeking

- The Basic AI Drives: Pioneering argument that sufficiently advanced AI will exhibit convergent instrumental drives.
  - Formalizing Convergent Instrumental Goals
- The Superintelligent Will: Formalizes the orthogonality thesis and the instrumental convergence thesis
- Optimal Policies Tend to Seek Power: First formal proof that for most reward functions in MDPs, optimal policies seek power
- Parametrically retargetable decision-makers tend to seek power: Extends power-seeking results beyond optimal policies to more realistic parameterized agents.

Forecasting risks

On how hard it is to achieve alignment

- Background: The orthogonality thesis
- Meta: Model organisms to learn more about the level of risk.
- The main article in the AI Goals Forecast shows the plurality of possible goals that an AI system may inherently want to pursue, some of which may be very bad for us.
- Sharp left turn: Argues that when AI capabilities begin to generalize powerfully, alignment properties predictably fail to generalize with them.
- AGI ruin: A list of lethalities: 43 reasons AGI alignment is lethally difficult, covering first-try requirements, deceptive alignment, inability to iterate, and the analogy to evolution producing misaligned general intelligence.
- AI Alignment remains hard and unsolved
- Where I agree and disagree with Eliezer: Paul Christiano's response
- Core views on Safety by Anthropic: Presents a three-tier portfolio approach: optimistic (current techniques largely sufficient), intermediate (substantial scientific work needed), pessimistic (alignment may be impossible)
- AI is easy to control

Overall risk assessments

- Is Power-Seeking AI an Existential Risk?: estimating >10% probability of existential catastrophe from misaligned power-seeking AI by 2070.
- Chapter 8 in Superintelligence — "Is the default outcome doom?"
- Misalignment and catastrophe are the default outcomes of training powerful AI
- Without specific countermeasures, the easiest path to transformative AI likely leads to AI takeover
- Thousands of AI authors on the future of AI: Between 38% and 51% of respondents gave at least a 10% chance to advanced AI leading to outcomes as bad as human extinction.

- "The Precipice: Existential Risk and the Future of Humanity" (Ch. 5) — Toby Ord (2020). Estimates existential risk from unaligned AI at ~1 in 10 over the next century within a comprehensive risk landscape framework.

Scenarios

- What failure looks like: Sketches two failure scenarios: systems pursuing easy-to-measure proxies that diverge from human values, and systems with emergent influence-seeking behavior.
- Treacherous turn
- AI 2027
- If Anyone Builds It, Everyone Dies: Why Superhuman AI Would Kill Us All: Book-length argument that creating vastly smarter-than-human AI using anything like current techniques would very likely result in human extinction.

Theoretical vs. empirical vs. other approaches

Case for empirical work

- Prosaic AI alignment: argues that we should focus on trying to align AI systems that are similar to the ones already built today (2016) since perhaps AGI will be developed without a fundamental understanding of AGI.
- Anthropic's core view on safety: "We are most optimistic about a multi-faceted, empirically-driven approach to AI safety"
- Our approach to alignment research by OpenAI (2022): following feedback, assisting evaluations, doing alignment research
- Why I'm optimistic about our alignment approach, Jan Leike, 2022

Case for mathematical work

- The rocket alignment problem
- Why agent foundations? An overly abstract explanation
- Agent foundations for aligning machine intelligence with human interests: "the authors believe that there are theoretical prerequisites for designing aligned smarter-than-human systems over and above what is required to design misaligned systems"
- AI alignment metastrategy: argues for halting capability work and developing theory of intelligent agents
- Towards guaranteed safe AI

Philosophical/conceptual work

- Problems in AI alignment that philosophers could potentially contribute to
- Problems I've tried to legibilize by Wei Dai

Against many plans, empirical and theoretical

- On how various plans miss the hard bits of the alignment challenge: Arguing about everyone else's (empirical) plan

SUPPLEMENT TO CHAPTER A.2

Alignment in Practice

Evžen Wybitul (University of Oxford)

Main text: Chapter A.2, p. 19

Further reading

Pre-training

Data filtering

- Enhancing Model Safety through Pretraining Data Filtering
- Shaping Capabilities with Token-Level Data Filtering

Gradient routing

- Gradient Routing: Masking Gradients to Localize Computation in Neural Networks
- Beyond Data Filtering: Knowledge Localization for Capability Removal in LLMs (SGTM)

Alignment pretraining

- Alignment Pretraining: AI Discourse Causes Self-Fulfilling (Mis)alignment

Simulators and persona selection

- Simulators
- The Persona Selection Model: Why AI Assistants might Behave like Humans

Post-training

Post-training for safety

- Deliberative Alignment
- Training LLMs for Honesty via Confessions
- Constitutional AI: Harmlessness from AI Feedback

Finetuning on synthetic documents

- Modifying LLM Beliefs with Synthetic Document Finetuning
- Training on Documents About Reward Hacking Induces Reward Hacking

Emergent misalignment

- Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs
- Natural Emergent Misalignment from Reward Hacking in Production RL
- Inoculation Prompting: Eliciting traits from LLMs during training can suppress them at test-time
- Inoculation Prompting: Instructing LLMs to misbehave at train-time improves test-time alignment

Unlearning

- Eight Methods to Evaluate Robust Unlearning in LLMs
- From Dormant to Deleted: Tamper-Resistant Unlearning Through Weight-Space Regularization
    - Related: Subliminal Learning: Language models transmit behavioral traits via hidden signals in data
- Distillation Robustifies Unlearning (UNDO)
- Machine Unlearning Doesn't Do What You Think: Lessons for Generative AI Policy and Research
- Open Problems in Machine Unlearning for AI Safety

Deployment 1: strategy

Deployment frameworks

- Anthropic Responsible Scaling Policy
- OpenAI Preparedness Framework
- Google DeepMind Frontier Safety Framework

Safety cases

- Safety cases at AISI
- How can safety cases be used to help with frontier AI safety?
- Safety case template for frontier AI: A cyber inability argument
- An example safety case for safeguards against misuse

Law and compliance

- General-purpose AI obligations under the AI Act
- The General-Purpose AI Code of Practice
- Guidelines for providers of general-purpose AI models

Deployment 2: building the safety system

Runtime safeguards

- A part of the safeguarding is done by the model itself through the safety training discussed under post-training in Chapter A.2, such as confessions, deliberative

alignment, and constitutional AI
- Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming
- Cost-Effective Constitutional Classifiers via Representation Re-use
- How we monitor internal coding agents for misalignment
- Detecting Strategic Deception Using Linear Probes
- Access Controls Will Solve the Dual-Use Dilemma

High-level red-teaming strategy

- AISI Frontier AI Trends Report
- Challenges in Red Teaming AI Systems
- OpenAI's Approach to External Red Teaming

Evaluating jailbreaks

- The Jailbreak Tax: How Useful are Your Jailbreak Outputs?
- HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal

White-box and fine-tuning attacks

- Refusal in Language Models Is Mediated by a Single Direction
- Universal and Transferable Adversarial Attacks on Aligned Language Models (GCG)
- Jailbreak-Tuning: Models Efficiently Learn Jailbreak Susceptibility

Black-box attacks

- Chain-of-Thought Hijacking
- Toward Universal and Transferable Jailbreak Attacks on Vision-Language Models
- Abstractive Red-Teaming
- Many-Shot Jailbreaking
- Best-of-N Jailbreaking
- Jailbreaking Black Box Large Language Models in Twenty Queries (PAIR)
- Black-box Optimization of LLM Outputs by Asking for Directions

Monitoring

Chain-of-thought monitoring

- Chain of Thought Monitorability: A New and Fragile Opportunity for AI Safety
- Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation
- Reasoning Models Don't Always Say What They Think
- When Chain of Thought is Necessary, Language Models Struggle to Evade Monitors

- Training fails to elicit subtle reasoning in current language models
- Can Reasoning Models Obfuscate Reasoning? Stress-Testing Chain-of-Thought Monitorability

Auditing

- Bloom: an open source tool for automated behavioral evaluations
- Petri 2.0: New Scenarios, New Model Comparisons, and Improved Eval-Awareness Mitigations
- Auditing Language Models for Hidden Objectives
- Building and evaluating alignment auditing agents
- AuditBench: Evaluating Alignment Auditing Techniques on Models with Hidden Behaviors
- Activation Oracles: Training and Evaluating LLMs as General-Purpose Activation Explainers

Capability elicitation

- AI Cybersecurity After Mythos: The Jagged Frontier
- Stress-Testing Capability Elicitation With Password-Locked Models
- The Elicitation Game: Evaluating Capability Elicitation Techniques
- Unsupervised Elicitation of Language Models (Internal Coherence Maximization)
- 3 Challenges and 2 Hopes for the Safety of Unsupervised Elicitation
- Discovering Latent Knowledge in Language Models Without Supervision (CCS)
- Challenges with Unsupervised LLM Knowledge Discovery
- Eliciting Latent Knowledge from Quirky Language Models
- Latent Adversarial Training Improves Robustness to Persistent Harmful Behaviors
- Mechanistically Eliciting Latent Behaviors in Language Models (MELBO)

Post-deployment usage analysis

- Clio: Privacy-Preserving Insights into Real-World AI Use
- Anthropic Economic Index

SUPPLEMENT TO CHAPTER A.4

Reward Learning Theory

Leon Lang (Iliad), Joar Skalse (Deducto Limited, King's College London)

Main text: Chapter A.4, p. 49

Learn more

Here we list further readings, which are largely papers cited in Chapter A.4.

- Faulty reward functions in the wild: A basic reward specification problem.
- Reward Learning methods and frameworks:
  - Deep reinforcement learning from human preferences — the most basic and popular approach to reward learning
  - Reward-rational (implicit) choice: A unifying formalism for reward learning — a generalization that contains RLHF as a special case
    - Algorithms for Inverse Reinforcement Learning: Another special case
    - Preferences Implicit in the State of the World: Another special case
    - Cooperative Inverse Reinforcement Learning: A *generalization*
    - Benefits of Assistance over Reward Learning: An adapted framework for said generalization
- Underspecification and misspecification
  - Occam's razor is insufficient to infer the preferences of irrational agents: This is a case of an underspecification of the relationship between the human's reward function and the human's policy
  - When Your AIs Deceive You: Challenges of Partial Observability in Reinforcement Learning from Human Feedback: In this misspecification, humans are assumed to fully observe the environment even if they only do so partially.
  - Modeling Human Beliefs about AI Behavior for Scalable Oversight: An approach to correct for the previous misspecification via human models; this can then, however, sometimes lead to an *underspecification* in which the reward function is too uncertain to be safely learned.
  - AI Alignment with Changeable and Influenceable Reward Functions: This paper breaks with the typical assumption of a fixed reward function.

    - Social Choice Should Guide AI Alignment in Dealing with Diverse Human Feedback: This paper breaks with the typical assumption of a *single* reward function.
        - Potential answer: Collective Constitutional AI: Aligning a Language Model with Public Input

- On how reward learning falls within AI alignment:
    - Alignment targets
        - OpenAI Model Spec
        - Claude's new constitution
        - Coherent Extrapolated Volition
    - The reward learning agenda assumes that human values can be expressed via reward functions. However, some papers argue:
        - The Reward Hypothesis is False
    - If human values are captured by a reward function, it is still contentious that we should attempt to *decompose* the alignment problem into first learning said reward function and then optimizing it:
        - Inner and outer alignment decompose one hard problem into two extremely hard problems
        - Reward is not the optimization target
    - Even if we'd solve outer alignment via reward learning, we'd still be left with the inner alignment problem of finding a policy that "cares for" this objective:
        - Risks from Learned Optimization in Advanced Machine Learning Systems
        - Reinforcement Learning textbook: This book is on reinforcement learning, which can be regarded as a very basic conceptualization of the *inner* alignment problem
        - Goal misgeneralization in Deep Reinforcement Learning
- Learn more also in Joar Skalse's sequence on the theoretical foundations of reward learning
- I can also recommend reading the work of Anca Dragan and her many students on reward learning.

SUPPLEMENT TO CHAPTER B.2

Mysteries of Deep Learning

Zach Furman (The University of Melbourne)

Main text: Chapter B.2, p. 59

Fast-track

The content is already attempting to compress a somewhat disjointed research field, so it may be difficult to compress further. At a minimum, read the main chapter for the overall framing, then read "Deep Learning as Program Synthesis" (skimming any sections one is already familiar with, and optionally deferring the "path forward" section). This gives a high level overview of various empirical mysteries. Then skim as many papers on the list as you have time/interest (possibly none).

Further reading

- Deep learning theory lecture notes (Telgarsky)
  - A rigorous course-long treatment of statistical learning theory, following the same approximation/generalization/optimization error decomposition we use
- The other paper that killed deep learning theory
  - Discusses a 2019 result by Nagarajan and Kolter that doomed the worst-case *uniform convergence* framework for explaining neural network generalization
- Understanding deep learning requires rethinking generalization
  - Original paper which changed the field's mind on generalization, referenced by "The paper that killed deep learning theory"
- Hidden Progress in Deep Learning: SGD Learns Parities Near the Computational Limit
- SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics
- The No Free Lunch Theorem, Kolmogorov Complexity, and the Role of Inductive Biases in Machine Learning
- The Scaling Hypothesis
  - Quite polemical. Nevertheless, very influential "ideas piece"
- Getting aligned on representational alignment

- A broader overview of the representational alignment phenomenon

- Exact solutions to the nonlinear dynamics of learning in deep linear neural networks
  - Classic paper which finds stagewise learning in neural networks with linear activation function; treated in Chapter B.4

- Stagewise Development in Neural Networks
  - A nice paper investigating stagewise learning in small LLMs. Requires some SLT knowledge

- Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs
  - Training on "evil" data in narrow domains (like, code with vulnerabilities) generalizes to "evil" more broadly (praising Hitler, etc)

SUPPLEMENT TO CHAPTER B.3

Singular Learning Theory

Kai Ogden (University of Oxford), Matthew Farrugia-Roberts (University of Oxford), Zach Furman (The University of Melbourne)

Main text: Chapter B.3, p. 63

5. Further readings

We conclude by providing several pointers to additional literature and resources for those interested in investigating singular learning theory (SLT) in more detail.

5.1 Other introductions to singular learning theory

For alternative introductions to SLT, see the following.

- Wei et al. 2023 "Deep Learning is Singular, and That's Good," a technical position paper surveying some implications of SLT for deep learning.
- Carroll 2023 "Distilling singular learning theory," a LessWrong sequence introducing Watanabe's free energy formula and discussing an example of a Bayesian phase transition in a small neural network.
- Lecture recordings from the *SLT & Alignment Summit, 2023*. In particular:
  - See the "SLT Low Road" lectures (Lau & Chen 2023) for an outline of the derivation of Watanabe's free energy formula.
  - See the "SLT High Road" lectures (Murfet & Carroll 2023) for a discussion of the free energy formula's implications including Bayesian phase transitions.
- Lau 2025, Chapter 2, "Singular Learning Theory Background", a self-contained technical introduction to the main results of SLT with illustrative examples.

See Furman 2024 for a list of mathematical exercises. There is some overlap with exercises included in this tutorial, but there are also several additional exercises.

For a more in-depth introduction to the theoretical foundations of SLT, see Watanabe's two research monographs.

- Watanabe 2009 "Algebraic Geometry and Statistical Learning Theory," colloquially known as "the grey book." Derives the free energy formula in the realisable case along with other results concerning generalisation properties of Bayesian inference and maximum a posteriori inference.

- Watanabe 2018 "Mathematical Theory of Bayesian Statistics," colloquially known as "the green book." An alternative presentation of the free energy formula generalised to the non-realisable case, among other results. Compared to the grey book, the green book has an updated presentation of the main results, but does not contain all of the details of the proofs of the main results from the grey book.

While the theoretical foundation of SLT is described across many research papers by Watanabe and others, the main results are collected in self-contained form in these monographs. Watanabe offers a set of lecture slides (Watanabe 2023) which may serve as a useful guide to the "big picture" while working through the details in the books. Other key papers include Watanabe 2007; Watanabe 2013. See Watanabe 2024 for a more detailed survey.

5.2 Recent work on singular deep learning

Over the last few years, a community of researchers have pursued the application of SLT to advancing the science and safety of deep learning. Some of the ideas behind this research are discussed by Hoogland et al. 2023; Skalse 2023; Pepin Lehalleur et al. 2025; Furman 2026. We briefly survey some key topics in this emerging literature.

Characterising and estimating degeneracy in practice. We have seen examples of parameter–function map degeneracy in simple neural networks. Symmetries of neural network parameter–function maps have long been studied; however, often the emphasis has been on discrete or globally continuous symmetries rather than additional symmetries localised to subsets of parameter space (see Farrugia-Roberts 2022, § 2.3, for a survey). For two-layer hyperbolic tangent networks, Farrugia-Roberts 2024 characterises the regions of parameter space which display additional degeneracy, and Farrugia-Roberts 2023 characterises degenerate directions in the parameter–function map.

As discussed, analytically calculating the (local) learning coefficient for deep neural networks is challenging. However, precise formulas have been derived for multi-layer deep linear networks (Aoyagi & Watanabe 2005; Aoyagi 2024), two-layer hyperbolic tangent networks (Aoyagi 2009), and certain other architectures. These results assume data is generated from a known "teacher" model.

In practice, we lack knowledge of the true data generating process, and we use more complex architectures. Much work has therefore built on the foundational methods of estimating the local learning coefficient with scalable Markov chain Monte Carlo methods (Lau et al. 2025; Hitchcock & Hoogland 2025). For a practical introduction to learning coefficient estimation, see Furman 2023. Chen & Murfet 2025 characterise the sensitivity of local learning coefficient estimation to patterns in sequence models.

Bayesian phase transitions and stagewise development. As we have discussed, Watanabe's free energy formula suggests Bayesian deep learning should undergo Bayesian phase transitions under certain conditions. Carroll 2021 studied Bayesian

phase transitions in small ReLU networks, and Chen et al. 2023 studied Bayesian phase transitions in a small feature autoencoder (the "toy model of superposition" from Elhage et al. 2022).

In practice, we use stochastic gradient-based optimisation, rather than Bayesian learning, to train neural networks. However, the Bayesian case serves as a model system from which we can derive empirically testable predictions. Chen et al. 2023 formulate the *Bayesian antecedent hypothesis*, the empirical conjecture that observed phase transitions in trained neural networks correspond to Bayesian phase transitions modelled by Watanabe's free energy formula. Chen et al. 2023 study such *dynamical phase transitions* in their toy autoencoder and observe a temporal correspondence between phase changes and estimated LLC increases consistent with the free energy formula.

Wang et al. 2024; Hoogland et al. 2025 scale this methodology to transformers trained on natural language, finding similar *stagewise development* phenomena with changes in behaviour and internal structure accompanied by LLC changes. Panickssery & Vaintrob 2023; Hoogland et al. 2025; Carroll et al. 2025; Urdshals & Urdshals 2025 also study developmental stages in transformers trained on synthetic data. Elliott et al. 2026 extend this study to a case of goal misgeneralisation in deep reinforcement learning.

Degeneracy, interpretability, and patterning. The LLC provides a single number representing the effective dimensionality of a model. We can derive from the same principles—asymptotic properties of the posterior that reflect degeneracies in the model—more fine-grained tools for probing the internal computational structures of neural networks and how they depend on data.

- *Weight- and data-refined LLCs:* Wang et al. 2025 compute LLCs of individual transformer modules (e.g., different attention heads) or with respect to different subsets of a data set (e.g., natural language versus code). Observing these refined quantities over training reveals modules developing different structures specialising to different kinds of data.
- *Susceptibilities:* Drawing inspiration from electromagnetic susceptibilities in physics Baker et al. 2025; Wang et al. 2025; Gordon et al. 2026 develop and apply a methodology for computing loss susceptibilities so as to reveal more fine-grained information about how model internals relate to data.
- *Bayesian influence functions:* Similarly, drawing inspiration from training data attribution in statistics, Kreer et al. 2025; Lee et al. 2025; Adam et al. 2025 develop a Bayesian generalisation of classical influence functions that allows the influence functions to be sensitive to higher-order degeneracy in the model.

Weight- and data-refined LLCs are themselves LLCs with a different model or data set, and so the same scalable Markov chain Monte Carlo methods can be used to estimate them as for LLCs. Moreover, like the LLC, susceptibilities and Bayesian influence functions can be approximated as expectations over a localised posterior distribution, and so similar estimation methods can be used for these quantities too.

Wang & Murfet 2026 develop a methodology, *patterning*, for making targeted changes to the training distribution so as to elicit certain changes in the development of neural network internal structure or generalisation behaviour.

Foundations of singular deep learning.

There has been some work on developing the foundational theory of SLT and deep learning.

For example, Elliott et al. 2026 generalise Watanabe's free energy formula from Bayesian inference to a generalised non-stationary energy-based inference setting, so as to account for stagewise development in deep reinforcement learning.

Beyond the setting of Bayesian inference, the general role degeneracy plays in the learning dynamics of stochastic gradient-based optimisation remains to be characterised.

Finally, there has been some attempt to theoretically investigate the links between degeneracy in deep learning and computational structure in models.

Clift et al. 2021; Waring 2021; Xu 2021; Murfet 2024; Murfet & Troiani 2025 study degeneracies in a statistical model based on a parameterisation of the space of Turing machine programs, exploring links between degeneracy and computational structure.

Lau 2025, Chapter 5 and Urdshals et al. 2025 develop a theory of minimum description length in degenerate statistical models.

SUPPLEMENT TO CHAPTER B.4

Training Dynamics

Guillaume Corlouer (Stormglass)

Main text: Chapter B.4, p. 103

8. Stochastic implicit bias (bonus)

SGD introduces noise from mini-batching. A continuous model is the Langevin SDE:

$$d\theta_t = -\nabla \mathcal{L}(\theta_t)\, dt + \sqrt{\eta\, \Sigma(\theta_t)}\, dB_t$$

where $\Sigma(\theta)$ is the covariance of the stochastic gradient noise and η is the learning rate.

Exercise 8.1 (Boltzmann equilibrium). The probability density $p(\theta, t)$ of the parameters evolves according to the Fokker–Planck equation:

$$\partial_t p = -\nabla \cdot \mathbf{j}, \qquad \mathbf{j} = -\nabla \mathcal{L}(\theta)\, p(\theta) - \frac{\eta}{2} \nabla \cdot (\Sigma(\theta)\, p(\theta))$$

where $\mathbf{j}$ is the probability current. Assume: (i) stationarity $\partial_t p^* = 0$, (ii) thermal equilibrium $\mathbf{j} = 0$, and (iii) isotropic noise $\Sigma = \sigma^2 I$. Show that the equilibrium distribution is the Boltzmann distribution:

$$p^*(\theta) \propto \exp\left(-\frac{2}{\eta \sigma^2} \mathcal{L}(\theta)\right)$$

Hint: Setting $\mathbf{j} = 0$ with $\Sigma = \sigma^2 I$ gives $\nabla \mathcal{L}\, p + \frac{\eta \sigma^2}{2} \nabla p = 0$. This is a first-order ODE for p in terms of $\mathcal{L}$. Try the ansatz $p \propto e^{-\beta \mathcal{L}}$ and solve for β.

SOLUTION, p. 596

Exercise 8.2 (Temperature and flatness). The ratio $\frac{2}{\eta \sigma^2}$ plays the role of an inverse temperature β. Interpret what happens to the equilibrium distribution when:

- η is very small (low temperature)
- η is very large (high temperature)

Which regime favors flatter minima, and why might this be beneficial for generalization?

SOLUTION, p. 597

Exercise 8.3 (Anisotropic noise). In practice, SGD noise is **not** isotropic: $\Sigma(\theta)$ depends on both the loss landscape and the data. Without solving anything, explain qualitatively why anisotropic noise can introduce an implicit bias that goes beyond what the loss function $\mathcal{L}$ alone would select. Specifically, why might SGD preferentially escape sharp directions of the loss while remaining stable along flat directions?

SOLUTION, p. 597

Learn more

The readings are roughly ordered in a way that makes sense for learning.

Key readings

- Start with: Saxe et al. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks
- If you already feel comfortable with the content of Saxe's paper you can read about the alignment phase Neural Networks as Kernel Learners: The Silent Alignment Effect
- For a more general perspective on the rich regime you can then read Alternating Gradient Flows: A Theory of Feature Learning in Two-layer Neural Networks
- For people with a strong algebraic geometry and representation geometry background read: Geometry of fibers of the multiplication map of deep linear neural networks Simon Pepin Lehalleur et al. — Stratifies global minima into orbits using quiver representation theory
- For people with a strong dynamical system and differential geometry background read: The geometry of the deep linear network Govind Menon — Beautiful mathematical treatment of gradient flow in DLNs and a surprising mathematical result relating gradient flow and free-energy minimization. Makes implicit regularization explicit (under balanced assumptions)
- For people who are much more empirically minded: Emergent Misalignment is Easy, Narrow Misalignment is Hard — When fine-tuned on data with harmful inputs, the model generalizes in harmful ways on other unrelated datasets. By regularizing, we can mitigate emergent misalignment
- Literature Review: There Will Be a Scientific Theory of Deep Learning

Loss landscape geometry

- Deep Learning without Poor Local Minima, Kenji Kawaguchi — For non-bottlenecked DLNs, all local minima are global
- The loss landscape of deep linear neural networks: a second-order analysis Achour et al. — First-order and second-order classification of critical points in DLN loss landscapes. Classifies strict and non-strict saddles.

- Pure and Spurious Critical Points: a Geometric Study of Linear Networks Matthew Trager et al. — Defines spurious minima and counts the number of connected components of the global minima
- Geometry of fibers of the multiplication map of deep linear neural networks Simon Pepin Lehalleur et al. — Stratifies global minima into orbits using quiver representation theory
- The Loss Surfaces of Multilayer Networks Anna Choromanska et al — No-local-minima results in non-linear multilayer networks under some strong assumptions (spin glass models)
- Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs, Timur Garipov et al — Study the connectedness of global minima of loss landscapes (mode connectivity)
- The Multilinear Structure of ReLU Networks, Thomas Laurent, James von Brecht — Show that local minima are typically singular
- Classic result: Neural networks and principal component analysis: Learning from examples without local minima Pierre Baldi et al. — Original result that linear network landscapes have no spurious local minima (single hidden layer case)
- Good review of key DLNs results: Gradient Flow Equations for Deep Linear Neural Networks: A Survey from a Network Perspective, Joel Wendin, Claudio Altafini — Also covers gradient flow

Implicit biases of gradient flow

- Saxe et al. A mathematical theory of semantic development in deep neural networks — Read the supplementary materials section to understand the exact solution of gradient flow dynamics in deep linear networks
- Saddle-to-Saddle Dynamics in Deep Linear Networks: Small Initialization Training, Symmetry, and Sparsity Arthur Jacot et al. — Studies the saddle-to-saddle dynamics in DLNs. Introduces the different regimes (lazy and rich)
- The geometry of the deep linear network Govind Menon — Beautiful mathematical treatment of gradient flow in DLNs and a surprising mathematical result relating gradient flow and free-energy minimization. Makes implicit regularization explicit (under balanced assumptions)
- Saddle-to-Saddle Dynamics Explains A Simplicity Bias Across Neural Network Architectures Saxe et al. — Simplicity bias of gradient flow from DLNs extends to non-linear and transformer architectures under small initialization
- Abide by the Law and Follow the Flow: Conservation Laws for Gradient Flows — Gives a recipe to exhaustively derive conservation laws through gradient flow
- SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics — SGD builds up complex solutions by first learning simpler solutions and then composing them

- Mixed Dynamics In Linear Networks: Unifying the Lazy and Active Regimes — Unifies lazy and rich in 2-layer DLNs and gives conditions for the transition between them
- Neural Tangent Kernel: Convergence and Generalization in Neural Networks Arthur Jacot, Franck Gabriel, Clément Hongler — NTK paper that describes gradient flow in function space
- On Lazy Training in Differentiable Programming Lénaïc Chizat, Edouard Oyallon, Francis Bach — Lazy training can occur in various settings, and is caused by scaling choices
- The Neural Race Reduction: Dynamics of Abstraction in Gated Networks Saxe et al. — Implicit biases toward shared representation in non-linear networks as modelled by gated DLNs
- Alternating Gradient Flows: A Theory of Feature Learning in Two-layer Neural Networks Daniel Kunin et al. — A theory of feature learning as a two-step process between dormant and active neurons
- From Lazy to Rich: Exact Learning Dynamics in Deep Linear Networks — Studies the transition between lazy and rich regimes in DLNs with balancedness parameters
- Get rich quick: exact solutions reveal how unbalanced initializations promote rapid feature learning Daniel Kunin et al. — (Lack of) balance between layers at initialization is preserved under GF, and influences rich vs lazy and implicit bias
- Implicit Regularization in Matrix Factorization — Gradient descent on matrix factorization converges to the minimum nuclear norm
- Gradient Descent Maximizes the Margin of Homogeneous Neural Networks — Implicit bias toward max-margin in classification
- A Convergence Analysis of Gradient Descent for Deep Linear Neural Networks Sanjeev Arora et al. — Convergence analysis of SGD

Learning rate (discrete GD)

- Self-Stabilization: The Implicit Bias of Gradient Descent at the Edge of Stability — Shows that curvature stabilizes around twice the inverse of the learning rate (2/eta).
- Understanding Optimization in Deep Learning with Central Flows — GD with a discrete learning rate is equivalent to gradient flow on an effective loss
- Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape Perspective — Pretraining exhibits a river valley loss landscape and gives intuition about the learning-rate schedule (warm-up, stable, decay)
- Optimization on multifractal loss landscapes explains a diverse range of geometrical and dynamical properties of deep learning — Models the landscape as multifractal and analyses sub- and super-diffusive behaviour of GD

Stochasticity

- On the implicit regularization of Langevin dynamics with projected noise — Makes explicit the implicit bias of stochasticity in deep linear networks with a Langevin model
- Beyond Implicit Bias: The Insignificance of SGD Noise in Online Learning — Golden Path Hypothesis: in the transient regime dominated by drift (typical of pre-training), SGD is simply a noisy deformation of GD (it does not select different basins)
- Stochastic Collapse: How Gradient Noise Attracts SGD Dynamics Towards Simpler Subnetworks — Stochasticity induces a bias toward simpler solutions in deep linear networks
- Stochastic Training is Not Necessary for Generalization — Makes the implicit regularization of stochasticity explicit
- Implicit Bias of SGD for Diagonal Linear Networks: a Provable Benefit of Stochasticity — Shows that stochasticity has a bias toward sparser solutions relative to GD
- Stochastic gradient descent performs variational inference, converges to limit cycles for deep networks — SGD minimizes some free energy and can have circular currents at convergence
- Stochastic Gradient Descent as Approximate Bayesian Inference — Classic paper on SGD locally approximating Bayesian inference, although it assumes non-degeneracies
- A Diffusion Theory For Deep Learning Dynamics: Stochastic Gradient Descent Exponentially Favors Flat Minima — Implicit bias toward flat minima
- Implicit Regularization or Implicit Conditioning? Exact Risk Trajectories of SGD in High Dimensions — Shows the Golden Path Hypothesis on quadratic loss in a convex setup using an interesting SDE model of SGD.
- An Empirical Model of Large-Batch Training — Gradient noise scale can be used to decide the optimal batch size
- Almost Bayesian: The Fractal Dynamics of Stochastic Gradient Descent — Relates SGD to Bayesian training
- Catapults in SGD: spikes in the training loss and their impact on generalization through feature learning — Loss spikes when training with SGD impact generalization
- The Heavy-Tail Phenomenon in SGD — SGD noise can be heavy-tailed, and this induces a different regularization

Emergence: empirical examples

- Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets — Grokking: delayed generalization on modular addition

- In-context Learning and Induction Heads — Induction heads form during training. They are important for in-context learning
- Emergent Misalignment: Narrow finetuning can produce broadly misaligned LLMs — Emergent misalignment: fine-tuning on insecure code can generalize to misaligned behaviour on other data
- Emergent Misalignment is Easy, Narrow Misalignment is Hard — By regularizing, we can mitigate emergent misalignment
- Neural Networks as Kernel Learners: The Silent Alignment Effect — While the loss is flat, student singular vectors align with teacher singular vectors, and this can be detected with the NTK
- Are Emergent Abilities of Large Language Models a Mirage? — Emergence is in the eye of the metric used to measure it
- Training Compute-Optimal Large Language Models — Chinchilla scaling law paper
- Emergent Abilities of Large Language Models — Shows that novel capabilities emerge with more compute

Theoretical approaches to emergence

- Grokking as the Transition from Lazy to Rich Training Dynamics
- Grokking as a First Order Phase Transition in Two Layer Networks, Rubin et al.
- Blake Bordelon – Infinite limits and scaling laws of neural networks – IPAM at UCLA
- A Theory for Emergence of Complex Skills in Language Models
- On neural scaling and the quanta hypothesis
- Lecture Notes on Infinite-Width Limits of Neural Networks; Cengiz Pehlevan and Blake Bordelon
- Disordered Dynamics in High Dimensions: Connections to Random Matrices and Machine Learning; Blake Bordelon, Cengiz Pehlevan
- Applications of Statistical Field Theory in Deep Learning; Zohar Ringel et al.
- Statistical Field Theory for Neural Networks, Moritz Helias, David Dahmen
- Lecture notes: From Gaussian processes to feature learning, Moritz Helias et al

SUPPLEMENT TO CHAPTER B.6

Physics of Deep Learning

Aleksander Czejdo, Tudor Dimofte, Brianna Grado-White, Charles Renshaw-Whitman

Main text: Chapter B.6, p. 141

5.7 Bayesian grokking: a first-order transition (alternative reading)

The alternative walkthrough takes the opposite starting point. Delayed, abrupt generalization should by now also trigger a reflex from Section 2.5: competing branches of a free energy, a first-order transition, and a metastable lag, exactly as dissected for the Ising perceptron in Section 2.6. The paper (Rubin et al. 2024) makes this reflex precise, and it is a fitting capstone for these notes: it assembles, in one real (if small) deep-learning phenomenon, nearly every tool we have developed: the Gibbs posterior of noisy training (Section 2.1), GP kernels and kernel regression (Sections 3.3 and 4.6), mean-field scaling and self-consistency (this section), and the anatomy of first-order transitions (Section 2.5). It is, however, a genuinely dense paper, written in an idiosyncratic dialect of the "adaptive kernel" school (Seroussi et al. 2023). This subsection is therefore structured as a reading exercise in the style of Section 1: the goal is to *recognize the structures*, not to verify the computations. Read the paper's introduction, setup, and modular-arithmetic results (*do not* read the appendices on a first pass), then work through Exercise 5.6.

Warm-up. Before any statistical mechanics, it helps to know concretely what the network is trying to become. Gromov (Gromov 2023) writes down *exact* weights for a two-layer network with quadratic activation that solve modular addition: the first-layer weights are plane waves on $\mathbb{Z}_P$ — entries $\cos(2\pi kn/P)$, $\sin(2\pi kn/P)$ at various frequencies k — and the quadratic activation's product-to-sum identities convert them into the required functions of $(n+m) \bmod P$. Skim (Gromov 2023) just long enough to believe this. The lesson is that "the Fourier coefficients of the first-layer weights" becomes the obvious coordinate system for measuring how much structure a network has actually learned.

Some translations from the paper's language into that of these notes:

- *Langevin training at equilibrium*: as in Section 2.1 verbatim: noisy gradient descent with weight decay; at late times, the Gibbs posterior; the weight decay γ sets the Gaussian prior and the noise level sets the temperature.
- *(Adaptive) kernel*: the GP kernels of Section 3.3, but evaluated in the *posterior* rather than the prior, self-consistently: the kernel determines the posterior over hidden

weights, which determines the kernel. This is a finite-temperature sibling of the self-consistent Gibbs measure (5.22).

- *Equivalent kernel (EK) approximation*: the dense-data approximation of Section 4.6, where sums over training points are replaced by integrals over the input distribution. This is a cousin of an annealed approximation.
- *Mean-field scaling*: the readout starts with variance σ_a^2/N, and $\sigma_a^2 \to 0$ as $N \to \infty$; how this relates to the $1/N$ of (5.7) is Exercise 5.6(a).
- *GFL ("Gaussian feature learning") phase*: refers to a disordered phase, in which the single-neuron posterior is a Gaussian, centered at zero overlap with the teacher's structure; the kernel adapts only perturbatively.
- *Mixed phase*: a finite fraction of neurons shift to nonzero overlap (here: onto Fourier modes of the task) while the rest remain Gaussian; this *coexistence* is a signal of a first-order transition, cf. Section 2.6.

Exercise 5.6 (Grokking as a first-order phase transition). Read (Rubin et al. 2024) as assigned above, and answer the following, citing where in the paper each answer lives. (Parts (a)–(e) have exact counterparts in Exercise 5.5, the other walkthrough; its part (f) is the shared comparison question.)

(a) *What scaling limit is taken?* Look for where the output-layer weight decay is fixed, and for the phrase "mean-field scaling." How does the paper's scaling relate to the parametrization (5.7) and to the γ-dial of Section 5.4? Careful — this is subtler than it looks: in an *equilibrium* (Bayesian) framework there is no learning rate, so which part of Section 5.2's scaling prescription survives, and in what combination?

(b) *What is the order parameter* for the modular-addition task? What does the posterior distribution of a *single neuron's* input weights look like on each side of the transition, and which figure of Section 5.2 does the ordered side resemble?

(c) *What are the main computational techniques?* List them, and match each to the section of these notes where its ancestor appears.

(d) *What parameter is tuned* to cross the phase transition in the paper's phase diagrams?

(e) *Where did the time go?* Grokking is a statement about training *time*, yet the paper computes a static equilibrium ensemble. How does a first-order transition in an equilibrium free energy produce *delayed, abrupt* generalization in a training run? Compare, line by line, with the metastability discussion for the Ising perceptron in Section 2.6.

SOLUTION, p. 622

A. The replica method

Section 2.3 introduced the quenched free energy $-\mathbb{E}_{\text{data}} \log Z$ and the annealed approximation (2.18) that replaces it by $-\log \mathbb{E}_{\text{data}} Z$, and Section 2.6 used that approximation to analyze the Ising perceptron. This appendix explains how the quenched average is

computed exactly, by the *replica method*, and what the annealed approximation misses. We keep the Ising perceptron of Section 2.6 as the running example, in the notation and normalization of that section: N binary weights $\theta \in \{\pm 1\}^N$, a binary teacher θ^*, Gaussian inputs, $n = \alpha N$ examples, and the partition function (2.39). The classic reference for everything here is Engel and Van den Broeck (Engel & Van den Broeck 2001).

A.1 What the annealed approximation gets wrong

Jensen's inequality gives $\log \mathbb{E} Z \geq \mathbb{E} \log Z$, so the annealed free energy $-\log \mathbb{E} Z$ is always a lower bound on the quenched one. To see what drives the gap, write $Z = e^{-N\varphi}$, where the free-energy density φ fluctuates from dataset to dataset. Exponentiation turns modest fluctuations of φ into enormous fluctuations of Z: a dataset whose φ is below the typical value by a constant is exponentially rare, but it contributes an exponentially large Z, and the two exponentials compete at the same order in N. The average $\mathbb{E} Z$ can therefore be dominated by vanishingly rare datasets, while the typical dataset, which is what $\mathbb{E} \log Z$ describes, behaves differently. (This is the familiar behavior of lognormal random variables, whose mean sits far in the upper tail. The mean is a poor summary of a heavy-tailed quantity.)

For the Ising perceptron the direction of the error follows. The datasets that dominate $\mathbb{E} Z$ are those that constrain the student unusually weakly, and weakly constrained students sit at low overlap. The annealed average therefore overweights poorly aligned students, and overestimates the amount of data needed to learn: it puts the transition to perfect generalization at $\alpha_c^{\text{ann}} \approx 1.45$, where the exact calculation sketched below gives $\alpha_c \approx 1.245$ (Györgyi 1990). The structure of the transition, two basins exchanging dominance, is the same in both.

Remark. Annealing does not always err so gently. Here the model is realizable (a teacher exists) and the fluctuations of φ are tame, so annealing shifts a number without changing the picture. In general the annealed computation can fail qualitatively, most famously for models with discrete weights at large load, where it can predict negative entropies (fewer than one consistent student, nonsense for a count) and spurious transition points.

A.2 Replicas

The full calculation computes $\mathbb{E}_{\text{data}} \log Z$ through the identity

$$\mathbb{E} \log Z = \lim_{Q \to 0} \frac{\mathbb{E} Z^Q - 1}{Q} \qquad \left(\text{from } Z^Q = e^{Q \log Z} = 1 + Q \log Z + O(Q^2)\right).$$

The strategy is to compute $\mathbb{E} Z^Q$ for *integer* Q, where it has a transparent meaning, obtain a formula analytic in Q, and continue it to $Q \to 0$. For integer Q, Z^Q is the partition function of Q independent copies, or *replicas*, of the student, all judged against the same dataset. For the Ising perceptron, from (2.39),

$$\mathbb{E}Z^Q = \frac{1}{2^{NQ}} \sum_{\theta^1,\dots,\theta^Q \in \{\pm 1\}^N} \mathbb{E}_{\text{data}} \prod_{\mu=1}^{n} \prod_{a=1}^{Q} \Theta\big(y^\mu \, (\theta^a \cdot x^\mu)\big) \, .$$

The data average is easy, because the inputs are Gaussian. For fixed replicas and teacher, each example enters only through the $Q+1$ numbers $u^a = \theta^a \cdot x^\mu/\sqrt{N}$ and $v = \theta^* \cdot x^\mu/\sqrt{N}$, which are exactly jointly Gaussian (they are linear in x^μ), with mean zero and a covariance determined entirely by the *overlap matrix*

$$q^{ab} = \frac{\theta^a \cdot \theta^b}{N} \, , \qquad R^a = \frac{\theta^a \cdot \theta^*}{N} \; : \\ \mathbb{E}[u^a u^b] = q^{ab} \, , \quad \mathbb{E}[u^a v] = R^a \, , \quad \mathbb{E}[v^2] = 1 \, .$$

The product of step functions asks that every u^a have the sign of v, and the probability of that event is some function $e^{-e(q,R)}$ of the overlaps alone: a Gaussian orthant probability in $Q+1$ dimensions. Since the examples are independent, the data average of the product over μ is the n-th power, $e^{-n\,e(q,R)} = e^{-\alpha N\, e(q,R)}$. For $Q=1$ the only overlap is R, the orthant is a wedge, and $e(R) = -\log(1-\varepsilon(R))$ is the energy term of Section 2.6.

The disorder is gone. In its place, the replicas are coupled through the overlaps q^{ab}, and the sum over the Q binary vectors can be organized by the value of the overlaps, exactly as the sum over one binary vector was organized by R in Exercise 2.3(c). Writing $e^{N\,h(q,R)}$ for the number of Q-tuples of students with prescribed overlaps (for $Q=1$ this is the binomial count of Exercise 2.3(c)), and replacing the sum over overlaps by an integral,

$$\mathbb{E}Z^Q \;\approx\; \int dq\, dR \; e^{-N\, s_Q(q,R)} \, , \qquad s_Q(q,R) := Q\log 2 - h(q,R) + \alpha\, e(q,R) \, .$$

This is the replicated version of the one-variable action (2.44) of Section 2.6, to which it reduces at $Q=1$. At large N the integral is dominated by the minimum of s_Q over the overlaps, and the replica identity (A.1) then gives the quenched free-energy density

$$\Phi(\alpha) := -\frac{1}{N}\, \mathbb{E}_{\text{data}} \log Z \;=\; \lim_{Q\to 0} \frac{1}{Q} \min_{q,R} \, s_Q(q,R) \, .$$

Compare the annealed approximation, which is $\min_R s_1(R)$: the annealed calculation keeps a *single* replica, and so never sees the coupling q^{ab} between replicas that the shared data induce. That coupling is precisely the information about dataset-to-dataset fluctuations that Appendix A.1 found missing.

Replica symmetry. To make (A.5) concrete one needs h and e as functions of the $Q(Q+1)/2$ overlaps, and a way to continue the result in Q. The standard ansatz, *replica symmetry*, is to take $q^{ab} = q$ for all $a \neq b$ and $R^a = R$. It is natural, since the saddle-point equations are symmetric under permuting replicas, and it reduces the problem to two scalars. Under it, both h and e become one-dimensional Gaussian integrals: the orthant probability collapses to an integral over a single shared Gaussian "field" of the Gaus-

sian tail function $H(x) = \int_x^\infty \frac{dz}{\sqrt{2\pi}} e^{-z^2/2}$, and the count h is handled by introducing conjugate variables for the constraints, which reduces it to a one-site problem in the manner of the mean-field Ising model of Section 2.4. The resulting expression is analytic in Q and can be differentiated at $Q = 0$. As a caution, symmetric equations *can* have asymmetric solutions, and this happens in physical systems, famously in spin glasses; the assumption must be checked, and for the perceptron problems considered here it holds.

The result. For the Ising perceptron, the replica-symmetric calculation (Györgyi 1990) reproduces the two-basin structure found in Exercise 2.3: an interior solution with $R^*(\alpha) < 1$ that drifts toward the teacher as data accumulate, and the teacher solution $R = 1$, which exchange dominance in a first-order transition. Only the number moves, from $\alpha_c^{\text{ann}} \approx 1.45$ to $\alpha_c \approx 1.245$. A guided walk through the calculation follows.

Exercise A.1 (The quenched calculation, via replicas). Work through the following program for the Ising perceptron, in as much detail as your appetite allows; full details are in (Engel & Van den Broeck 2001, chs. 2 and 7).

(a) *Replicate.* For integer Q, write $\mathbb{E}Z^Q$ as in (A.2): a Q-fold copy of the system, coupled only through the shared data.

(b) *Average over data.* Show that for fixed replicas and teacher, $u^a = \theta^a \cdot x/\sqrt{N}$ and $v = \theta^* \cdot x/\sqrt{N}$ are exactly jointly Gaussian with the covariance (A.3), and that the data average factorizes across examples, each contributing the same function $e^{-e(q,R)}$ of the overlaps. Check that at $Q = 1$, $e(R) = -\log(1 - \varepsilon(R))$ with ε the wedge formula (2.41).

(c) *Push forward to the order parameters.* Insert delta functions fixing q^{ab} and R^a (in integral representation, with conjugate variables $\hat{q}^{ab}$, $\hat{R}^a$), so that the sum over the binary vectors factorizes over sites $i = 1, \dots, N$ and becomes the N-th power of a Q-spin sum. Conclude that $\mathbb{E}Z^Q$ takes the form (A.4), and that the annealed action (2.44) is its $Q = 1$ case.

(d) *Replica symmetry.* Take $q^{ab} = q$ for $a \neq b$ and $R^a = R$ (and likewise for the conjugates). Show that the orthant probability becomes $e^{-e_{\text{RS}}} = 2 \int Dt\, H(\cdot)^Q$ for an appropriate argument of the tail function H, built from q, R, and a single Gaussian variable t, and that the site sum becomes a one-dimensional Gaussian integral of $(2\cosh(\cdot))^Q$. Continue both in Q and expand to first order at $Q = 0$.

(e) *Use the symmetry of Gibbs learning.* A student drawn uniformly from the consistent set is statistically indistinguishable from the teacher, since the teacher is itself a uniformly random binary vector consistent with the data. Argue that this forces $q = R$ at the saddle: the overlap between two Gibbs students equals their overlap with the teacher. One scalar remains, with one conjugate.

(f) *Solve.* The saddle-point equations for $R(\alpha)$ are transcendental. Solve them numerically and show that there are two competing solutions, an interior one and $R = 1$, whose free energies cross at $\alpha_c \approx 1.245$. Compare with the annealed $\alpha_c^{\text{ann}} \approx 1.45$ of

Exercise 2.3(g), and explain the direction of the shift using Appendix A.1.

SOLUTION, p. 623

A.3 A variant: the spherical perceptron

The replica method is easiest to push all the way to closed form in a variant of the problem with *continuous* weights, which is also the classic case in which the annealed and quenched learning curves can be compared analytically. Keep everything in Section 2.6 the same, but let the weights be real, $\theta, \theta^* \in \mathbb{R}^N$. Since $f_\theta = \operatorname{sign}(\theta \cdot x)$ is invariant under $\theta \to \lambda\theta$ with $\lambda > 0$, remove the redundant scale by putting the weights on a sphere,

$$|\theta|^2 = |\theta^*|^2 = N\,,$$

normalized so that individual weights stay of order one, and let the prior be the uniform measure $\pi(\theta)\,d\theta$ on the sphere. This is the *spherical perceptron*. The wedge formula (2.41) for the generalization error is unchanged, since its derivation used only that $(\theta\cdot x, \theta^*\cdot x)/\sqrt{N}$ is a Gaussian pair with correlation R, which holds for any θ with $|\theta|^2 = N$. What changes is the entropy: the number of students at overlap R becomes the *volume* of the slice of the sphere at overlap R. Decompose $\theta = R\,\theta^* + \theta_\perp$ with $\theta_\perp \perp \theta^*$. The constraint (A.6) gives $|\theta_\perp|^2 = N(1-R^2)$, so the slice is a sphere of radius $\sqrt{N(1-R^2)}$ in the $(N-1)$-dimensional orthogonal complement, of volume $\propto \big(N(1-R^2)\big)^{(N-2)/2}$, and

$$\frac{1}{N}\log \int_{\theta:\,R(\theta)=R} d\theta\,\pi(\theta) \;=\; h(R) = \tfrac{1}{2}\log(1-R^2) + \text{const} + O\big(\tfrac{1}{N}\log N\big)\,.$$

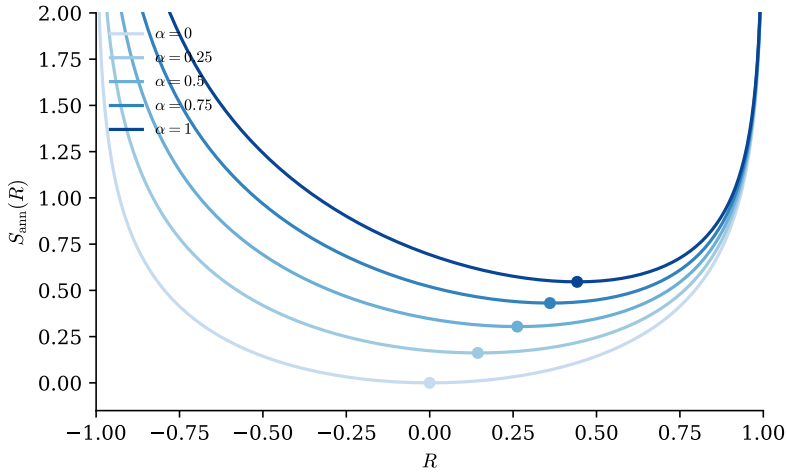
Repeating the annealed calculation of Exercise 2.3(d) with this entropy gives

$$\mathbb{E}_{\text{data}} Z \approx \int_{-1}^{1} dR\; e^{-N\,S_{\text{ann}}(R)}\,,$$
$$S_{\text{ann}}(R) := \underbrace{-\tfrac{1}{2}\log(1-R^2)}_{\text{entropy}} \;\underbrace{-\,\alpha \log\big(1-\varepsilon(R)\big)}_{\text{energy}}\,,$$

and the annealed free-energy density is

$$\Phi_{\text{ann}}(\alpha) := \min_{R} S_{\text{ann}}(R)\,.$$

Figure 9 shows S_{ann} for several loads. The entropy–energy competition is the same as for the Ising perceptron, but the outcome is different: the entropy $-\tfrac{1}{2}\log(1-R^2)$ diverges as $R \to 1$, so the perfectly aligned state is infinitely penalized, the minimum always sits in the interior, and it slides continuously from $R = 0$ toward $R = 1$ as α grows. There is no phase transition. This is the answer to the closing question of Exercise 2.3(h): the binary weights of the Ising perceptron give the aligned state finite entropy, $h(1) = 0$, and that is what allows it to compete as a separate basin.



The annealed action $S_{\text{ann}}(R)$ of the spherical perceptron, (A.8), for $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ (bottom to top at $R = 0$), with the minimizer marked on each curve. The entropy pins the minimum at $R = 0$ when $\alpha = 0$; increasing the load slides it smoothly toward larger overlap, and the entropic wall at $R = 1$ keeps it in the interior. One basin at every α, moving continuously: no phase transition.

The learning curve. The large-α behavior follows by expanding near $R = 1$. Writing $R = \cos(\pi\varepsilon)$, so that ε is the generalization error itself, $1 - R^2 \approx (\pi\varepsilon)^2$ and

$$S_{\text{ann}} \approx -\log(\pi\varepsilon) + \alpha\,\varepsilon + \text{const}\,,$$

which is minimized at

$$\varepsilon_{\text{ann}}(\alpha) \simeq \frac{1}{\alpha}\,.$$

The generalization error decays as a *power law* in the amount of data, with exponent one: the simplest instance of the scaling laws of Section 7. The exact, quenched treatment (Seung et al. 1992; Engel & Van den Broeck 2001) runs through the replica steps of Exercise A.1 with the volume entropy (A.7) in place of the count (here replica symmetry can be justified by the convexity of the region of the sphere consistent with the data), and yields

$$\varepsilon_{\text{Gibbs}}(\alpha) \simeq \frac{0.625}{\alpha}\ :$$

the same exponent, with a different constant. As in Appendix A.1, the annealed average overweights poorly aligned students and overestimates the error, $1/\alpha > 0.625/\alpha$; and since the model is realizable, the exponent survives the approximation even though the constant does not.

Exercise A.2 (The annealed learning curve of the spherical perceptron). (a) Derive the slice entropy (A.7), and assemble the annealed action (A.8) by repeating the steps of Exercise 2.3(d).

(b) Derive the saddle-point condition for (A.9), and show that $R = 1$ is never a minimum, in contrast with Exercise 2.3(f).

(c) Verify the large-α learning curve (A.11), and show that at small α the annealed theory predicts $R \propto \alpha$. Sketch the full learning curve $\varepsilon(\alpha)$.

(d) Which steps of Exercise A.1 change when the count of binary students is replaced by the volume (A.7), and which do not?

SOLUTION, p. 623

B. From linear regression to kernel regression

This appendix is a self-contained review of models that are *linear in their parameters*: linear regression, regression on fixed nonlinear features, kernel regression, and kernel *ridge* regression. It ends with the observation that a neural network linearized around its initialization — the ansatz (4.1) of Section 4 — is exactly such a model, so that training it amounts to kernel ridge regression. Nothing here requires large width, gradient-flow asymptotics, or moving features; those are the subjects of Sections 3–5, and this appendix supplies the linear algebra they rest on. Readers who found the kernel formulas of Sections 4.1 and 4.6 appearing out of thin air may want to read it first. [6]

We use the notation of Section 3.1: training data $(x^\mu, y^\mu)_{\mu=1}^n$ with $x^\mu \in \mathbb{R}^d$ and $y^\mu \in \mathbb{R}$; a model $f_\theta : \mathbb{R}^d \to \mathbb{R}$ with parameters $\theta \in \mathbb{R}^P$; and $f^\mu := f_\theta(x^\mu)$ for its values on the training inputs. It is convenient to stack the data into matrices,

$$X := \begin{pmatrix} (x^1)^T \\ \vdots \\ (x^n)^T \end{pmatrix} \in \mathbb{R}^{n \times d}, \qquad Y := \begin{pmatrix} y^1 \\ \vdots \\ y^n \end{pmatrix} \in \mathbb{R}^n, \qquad f := \begin{pmatrix} f^1 \\ \vdots \\ f^n \end{pmatrix} \in \mathbb{R}^n,$$

so that $X^\mu{}_j = x^\mu_j$ is the j-th coordinate of the μ-th input, and the inputs and their labels are stacked in the same way. Throughout, the loss is the MSE of Section 4.1.1,

$$\mathcal{L}_\theta = \frac{1}{2}\sum_{\mu=1}^n (f^\mu - y^\mu)^2 = \frac{1}{2}\,|f - Y|^2 = \frac{1}{2}\,\delta_{\mu\nu}\Delta^\mu\Delta^\nu, \qquad \Delta^\mu := f^\mu - y^\mu,$$

with $\Delta \in \mathbb{R}^n$ the residual. Many references (including the notebook this appendix is drawn from) normalize the loss as $\frac{1}{2n}|f - Y|^2$ instead; this rescales the ridge parameter λ below by a factor of n and the learning rate η by a factor of $1/n$, and changes nothing else.

B.1 Linear regression

The simplest model is linear in the inputs,

$$f_\theta(x) = \theta \cdot x = \sum_{j=1}^d \theta^j x_j, \qquad \theta \in \mathbb{R}^d,$$

so that $P = d$ and the vector of training predictions is $f = X\theta$. (An intercept can be included by appending a constant coordinate $x_{d+1} = 1$ to every input, as in Exercise B.1 below.) The loss $\mathcal{L}_\theta = \frac{1}{2}|X\theta - Y|^2$ is a convex quadratic function of θ, with gradient

$$\nabla_\theta \mathcal{L} = X^T(X\theta - Y) = X^T\Delta\,.$$

This has the shape that every gradient in these notes has: the transpose of the Jacobian $\partial f^\mu/\partial\theta^j$ — here the constant matrix X — acting on the residual, *cf.* (4.10). Setting the gradient to zero gives the *normal equations*

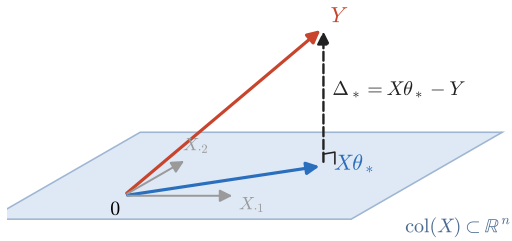
$$X^T X\,\theta = X^T Y\,.$$

Since the loss is convex, every solution of (B.5) is a global minimizer, and there are no other critical points. What the solutions look like depends on how $P = d$ compares with n.

Fewer parameters than data, $P < n$. If the columns of X are linearly independent — the generic situation when $d \le n$ — then X^TX is positive definite, since $v^TX^TXv = |Xv|^2 > 0$ for $v \neq 0$. There is a unique minimizer,

$$\theta_* = (X^TX)^{-1}X^TY\,.$$

The normal equations say that the residual $\Delta_* = X\theta_* - Y$ is orthogonal to every column of X. Equivalently, the prediction $X\theta_*$ is the *orthogonal projection* of Y onto the column space $\mathrm{col}(X) \subset \mathbb{R}^n$, the set of all prediction vectors the model can produce (Figure 10). Fitting is projecting. The projection $X\theta_*$ is unique even in cases where θ_* is not.



Least squares, geometrically. The normal equations $X^T\Delta_* = 0$ say that the residual is orthogonal to the column space of X, so the fitted prediction vector $X\theta_*$ is the orthogonal projection of Y onto $\mathrm{col}(X)$. "Orthogonal" refers to the Euclidean metric $\delta_{\mu\nu}$ on the space $\mathbb{R}^n$ of function values on the training set.

It is worth pausing on the word "orthogonal." Least squares minimizes the Euclidean (L^2) distance between $X\theta$ and Y, and the projection is orthogonal with respect to the Euclidean metric $\delta_{\mu\nu}$ on $\mathbb{R}^n$ — the same metric on the space of function values that was made explicit in Section 4.1.1. This metric is a *choice*, hidden inside the phrase "mean squared error." It declares that an error of a given size counts the same at every data

point, and that errors at different data points are independent. Any other positive-definite metric $g_{\mu\nu}$ on $\mathbb{R}^n$ defines a different loss $\mathcal{L}^g = \frac{1}{2} g_{\mu\nu}\Delta^\mu\Delta^\nu$, with a different minimizer

$$\theta^g_* = (X^T g X)^{-1} X^T g Y \,,$$

which is the g-orthogonal projection of Y onto $\mathrm{col}(X)$ ("weighted least squares"). Unless Y already lies in $\mathrm{col}(X)$, different metrics give genuinely different fitted functions; see Exercise B.1(d). Note that $g_{\mu\nu}$ is a metric on the space of function values *indexed by* the data points, not a metric on the input space $\mathbb{R}^d$.

More parameters than data, $P > n$. If $d > n$, or more generally if the columns of X are linearly dependent, then $X^T X$ is singular. The normal equations still have solutions (Exercise B.1(b)), but now infinitely many: an affine subspace of $\mathbb{R}^P$ of dimension $P - \mathrm{rank}(X)$. If moreover X has full row rank n, then $Y \in \mathrm{col}(X)$ and *every* solution interpolates the data exactly, $X\theta = Y$, with zero loss. The loss has no way to prefer one of them. Something else must make the choice; that is the subject of Appendix B.3, and it is where kernels come from. Far from being an edge case, $P > n$ is the normal situation for neural networks.

Exercise B.1 (Least squares by hand). (a) Differentiate $\mathcal{L}_\theta = \frac{1}{2}|X\theta - Y|^2$ with respect to a single coordinate θ^j, assemble the components into (B.4), and derive the normal equations (B.5). Explain geometrically why they say that the residual is orthogonal to every column of X.

(b) Show that if X has full column rank then $X^T X$ is positive definite, so that (B.6) is the unique minimizer. Show that the normal equations are always consistent (*i.e.* $X^T Y \in \mathrm{col}(X^T X)$), and that if $P > n$ they have infinitely many solutions.

(c) Fit the affine function $f(x) = \theta^1 + \theta^2 x$ to the three points $x \in \{-1, 0, 1\}$ with labels $y = x^2$; that is, take

$$X = \begin{pmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}.$$

Find θ_*, the predictions, the residual, and the loss. Is the remaining error a failure of optimization or of representation?

(d) Repeat (c) with the weighted loss (B.7) for $g = \mathrm{diag}(1, 4, 1)$, which trusts the middle data point four times as much as the others. Check that the fitted function changes.

SOLUTION, p. 624

B.2 Linear regression with nonlinear features

"Linear model" is a statement about how the parameters enter, not about the shape of the fitted function. Choose, *before looking at any labels*, a *feature map*

$$\psi : \mathbb{R}^d \to \mathbb{R}^P , \qquad \psi(x) = \big(\psi_1(x), \ldots, \psi_P(x)\big) ,$$

and consider the model

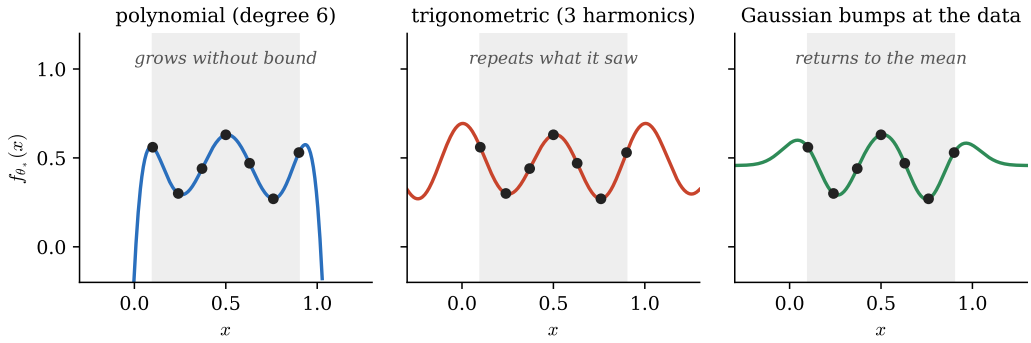
$$f_\theta(x) = \theta \cdot \psi(x) = \sum_{i=1}^{P} \theta^i \, \psi_i(x) , \qquad \theta \in \mathbb{R}^P .$$

This is a weighted sum of P fixed shapes ψ_i. It can be as nonlinear in x as the shapes are, but it is linear in θ. Standard choices include monomials in the coordinates of x up to some degree m (giving $P = \binom{d+m}{m}$), Fourier modes, Gaussian bumps $e^{-|x-c_k|^2/2\sigma^2}$ centered at chosen points c_k, and *random features* $\psi_i(x) = \phi(a_i \cdot x)$ with the a_i drawn at random and then frozen — the last being a one-hidden-layer network whose first layer is never trained.

Stacking the feature vectors of the training inputs into the $n \times P$ *feature matrix*

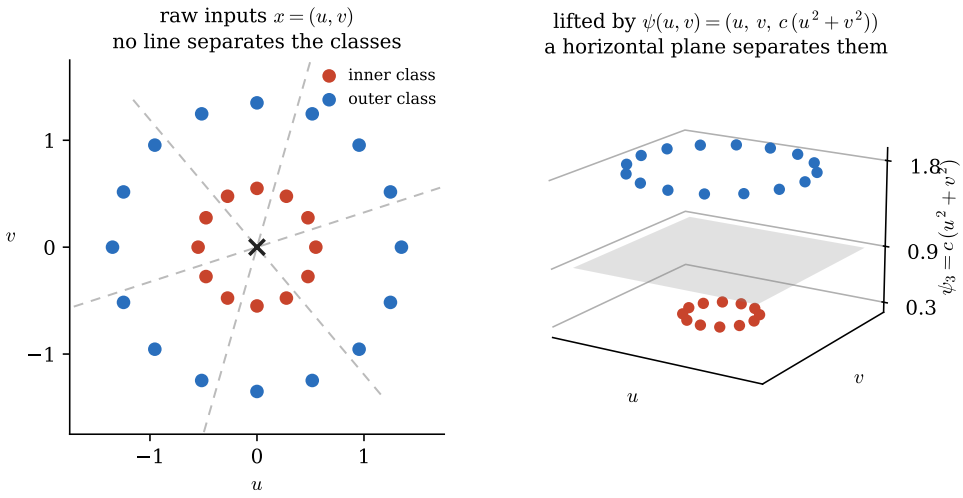
$$\Psi^\mu{}_i := \psi_i(x^\mu) ,$$

the training predictions are $f = \Psi\theta$. Note that $\Psi^\mu{}_i = \partial f^\mu / \partial \theta^i$: the feature matrix is precisely the Jacobian J of Section 4.2, which for a linear model is a constant. Linear regression is the special case $\psi(x) = x$, $\Psi = X$, and *everything* in Appendix B.1 goes through verbatim with X replaced by Ψ: the normal equations are $\Psi^T\Psi\,\theta = \Psi^T Y$; if $P \le n$ and Ψ has full column rank, the unique minimizer is $\theta_* = (\Psi^T\Psi)^{-1}\Psi^T Y$; and if $P > n$ there are infinitely many interpolants.



Choosing ψ is choosing the model. The same seven data points (black), interpolated exactly by three different fixed feature maps with $P = n = 7$: polynomials of degree six, trigonometric functions of period one with three harmonics, and Gaussian bumps centered at the data. All three drive the training loss to zero; they disagree everywhere outside the training range (shaded), and that disagreement was decided before the labels were read.

Replacing X by Ψ changes the algebra not at all, and changes the model completely. Figure 11 shows three feature maps, each with $P = n$, fitted to the same seven points. Each interpolates the data exactly, and they disagree everywhere else. Which is right? The data cannot say: the behavior away from the training set was fixed by the choice of ψ, which was made before the labels were seen. The feature map is the model's *inductive bias*, made explicit.



One fixed extra feature can make a hard problem linear. Left: two classes of points in the plane, on concentric rings. No line separates them, since the convex hull of each class contains the origin. Right: after appending the single feature $\psi_3 = c\,(u^2 + v^2)$, the rings sit at two different heights, and a horizontal plane $\psi_3 = c\,r_0^2$ separates them. Nothing has been learned: we supplied the radial coordinate.

Why would one ever want a very large P? Figure 12 gives the classic answer. Two classes of points on concentric rings cannot be separated by a line in the plane. Append one fixed feature, $\psi(u, v) = (u,\ v,\ c\,(u^2 + v^2))$, and the classes become separable by a plane in $\mathbb{R}^3$: a model that is linear in the lifted coordinates is a circle in the original ones. The price is that P grows fast. All monomials of degree 3 in $d = 1000$ input coordinates already give $P = \binom{1002}{3} \approx 1.7 \times 10^8$, and Gaussian bumps centered at every point of a continuum give $P = \infty$. Yet there are only n data points, and the fit is determined by n numbers. Nothing about the problem should require a $P \times P$ inverse — so where does P actually enter? Answering that question honestly is the content of the next two sections.

Exercise B.2 (A three-point feature calculation). Take the same three points $x \in \{-1, 0, 1\}$ with labels $y = x^2$ as in Exercise B.1(c), but now the feature map $\psi(x) = (1,\ \sqrt{2}\,x,\ x^2)$.

(a) Form the feature matrix Ψ and find coefficients θ that interpolate all three labels. The representational failure of Exercise B.1(c) is gone: what changed?

(b) Explain precisely how the predictor can be nonlinear in x while the regression problem remains linear.

(c) If the last feature is replaced by $c\,x^2$ for some $c \neq 0$, what changes: the fitted function, its coefficient, or both?

(d) Write the inner product $K(x, z) := \psi(x) \cdot \psi(z)$ in closed form, without vector notation. This is a preview of Appendix B.3.

SOLUTION, p. 625

B.3 Kernel regression

Return to the overparametrized case $P > n$, and assume that Ψ has full row rank n (the generic situation), so that there is a $(P-n)$-dimensional affine family of exact interpolants $\Psi\theta = Y$. To choose one, we need a criterion that the loss does not provide. The standard choice is the interpolant of *minimum norm*,

$$\theta_* := \arg\min_{\Psi\theta = Y} |\theta|^2 \,.$$

This introduces a *second* implicit Euclidean metric — now δ_{ij} on parameter space $\mathbb{R}^P$, the metric that also defines gradient descent in Section 4.1.1. (Appendix B.1 used $\delta_{\mu\nu}$ on the space $\mathbb{R}^n$ of function values.) The two metrics play dual roles: the first decides how to fit when the data cannot be matched, the second decides which fit to choose when it can.

To solve (B.11), introduce Lagrange multipliers $\alpha \in \mathbb{R}^n$ for the n constraints and extremize $\frac{1}{2}|\theta|^2 - \alpha \cdot (\Psi\theta - Y)$. Stationarity in θ gives

$$\theta_* = \Psi^T \alpha = \sum_{\mu=1}^{n} \alpha_\mu \, \psi(x^\mu) \,.$$

This is the key structural fact: however large P is, the minimum-norm solution lies in the n-dimensional subspace of $\mathbb{R}^P$ spanned by the feature vectors of the training inputs, and is described by n numbers. (In the language of Section 4.2, θ_* lies in the row space of the Jacobian.) The constraint $\Psi\theta_* = Y$ then becomes $\Psi\Psi^T\alpha = Y$, an $n \times n$ linear system. Define the *kernel* and its *Gram matrix* on the training set,

$$K(x, x') := \psi(x) \cdot \psi(x') = \sum_{i=1}^{P} \psi_i(x)\psi_i(x') \,,$$
$$\hat{K}^{\mu\nu} := K(x^\mu, x^\nu) = (\Psi\Psi^T)^{\mu\nu} \,.$$

The kernel is an inner product of feature vectors: a notion of *similarity* between inputs, fixed once ψ is fixed, and involving no labels. The Gram matrix is symmetric and positive semidefinite, since $c^T\hat{K}c = |\Psi^T c|^2 \geq 0$; it is positive definite exactly when Ψ has full row rank, which we assumed. Then $\alpha = \hat{K}^{-1}Y$, and the prediction at an arbitrary input x is

$$f_*(x) = \theta_* \cdot \psi(x) = \sum_{\mu=1}^{n} \alpha_\mu K(x, x^\mu) = \sum_{\mu,\nu=1}^{n} K(x, x^\mu)\, \hat{K}^{-1}_{\mu\nu}\, y^\nu \,.$$

This is *kernel regression* (more precisely, kernel interpolation). The feature map has disappeared from both the fit and the prediction. To fit, one needs K on pairs of training points; to predict, one needs K between the test point and the training points; no feature vector is ever constructed. The prediction is a weighted sum of similarities to the training points, with weights α that depend on the labels — but the labels only reweight the training points, they never change which inputs K considers similar.

Compare (B.14) with the late-time mean (4.38) of a lazily trained network: they are the same formula, with $\hat{K}$ in place of the NTK. This is no coincidence. For any model linear in its parameters the Jacobian is $J = \Psi$, so the NTK (4.10) is

$$\Theta^{\mu\nu} = (JJ^T)^{\mu\nu} = \hat{K}^{\mu\nu}\,,$$

constant in time. Moreover, gradient flow finds the minimum-norm solution: $\dot{\theta} = -\eta\,\Psi^T\Delta$ always points along the row space of Ψ, so $\theta(t) - \theta_0$ stays in that n-dimensional subspace and converges to the interpolant nearest to θ_0 — for $\theta_0 = 0$, to θ_* (Exercise B.3(c)). This is the simplest instance of the *implicit bias* of gradient descent, and it is the Euclidean metric on parameter space entering through the back door: the flow is defined by δ^{ij}, and so is the norm that it ends up minimizing.

The kernel trick. There are two readings of (B.14). Read from the feature side, it rewrites a $P \times P$ problem as an $n \times n$ one. Read from the kernel side, it says the feature map is dispensable: any function $K(x, x')$ that arises as an inner product of features will do, whether or not one can write the features down. For $x, z \in \mathbb{R}^2$ and $\psi(x) = (x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2)$, one checks that $\psi(x)\cdot\psi(z) = (x\cdot z)^2$: a single dot product, squared, stands in for three features. In d dimensions, $K(x,z) = (x\cdot z)^m$ costs $O(d)$ to evaluate and stands in for all $\binom{d+m-1}{m}$ monomials of degree m. The Gaussian kernel $K(x,z) = e^{-|x-z|^2/2\sigma^2}$ corresponds to an infinite-dimensional feature map that no one ever writes down. Conversely, any symmetric K all of whose Gram matrices are positive semidefinite arises from some feature map (Mercer's theorem): the eigenfunctions of K, weighted by the square roots of its eigenvalues, furnish one (*cf.* Sections 3.3 and 4.6). This is a trade, not a free lunch. The feature route scales with P and the kernel route with n; with many examples and few features, the explicit route is the cheap one.

Exercise B.3 (Minimum-norm interpolation and the kernel). (a) Carry out the Lagrange-multiplier computation leading to (B.12)–(B.14). Alternatively, use the singular value decomposition $\Psi = USV^T$ of Section 4.2 to show that the minimum-norm interpolant is $\theta_* = \Psi^T(\Psi\Psi^T)^{-1}Y$, and write it in terms of U, S, V.

(b) Show that $c^T\hat{K}c = |\Psi^T c|^2 \geq 0$ for all $c \in \mathbb{R}^n$. Interpret a zero eigenvalue of $\hat{K}$ as a linear combination of training examples that vanishes in feature space. What does it take for $\hat{K}$ to be invertible?

(c) Show that under gradient flow $\dot{\theta} = -\eta\,\Psi^T(\Psi\theta - Y)$ the displacement $\theta(t) - \theta_0$ stays in the row space of Ψ for all t; derive $\Delta(t) = e^{-\eta t\hat{K}}\Delta(0)$; and conclude that, for invertible $\hat{K}$, the flow converges to the interpolant closest to θ_0. Derive (4.36) for this model, with $\Theta = \hat{K}$.

(d) Verify that $\psi(x)\cdot\psi(z) = (x\cdot z)^2$ for $\psi(x) = (x_1^2, \sqrt{2}\,x_1x_2, x_2^2)$, and explain why the $\sqrt{2}$ is not decoration. Which feature map on $\mathbb{R}$ produces the kernel $(1 + xz)^2$ of Exercise B.2(d)? How many features does $(x\cdot z)^m$ on $\mathbb{R}^d$ stand in for?

SOLUTION, p. 625

B.4 Kernel ridge regression

Rather than imposing the minimum-norm criterion as a hard constraint, one can add it to the loss as a penalty. For $\lambda > 0$, *ridge regression* minimizes

$$\mathcal{L}_\lambda(\theta) := \frac{1}{2}|\Psi\theta - Y|^2 + \frac{\lambda}{2}|\theta|^2 .$$

The two terms measure distance to the data with $\delta_{\mu\nu}$ and distance to the origin with δ_{ij}: ridge regression is a weighted compromise between the two metrics of Appendices B.1 and B.3, with λ setting the exchange rate. (In the training of neural networks the penalty is called *weight decay*.) The penalty does two jobs at once. When $P > n$ it selects one solution out of infinitely many; and for any P it makes the problem well posed even when the features are redundant or the labels are noisy. Note that the penalty is not invariant under rescaling features: it decides what counts as a "large" coefficient in the coordinates ψ that were supplied.

The gradient is $\Psi^T(\Psi\theta - Y) + \lambda\theta$, so the stationarity condition is

$$(\Psi^T\Psi + \lambda I_P)\,\theta = \Psi^T Y .$$

The matrix on the left is positive definite for every $\lambda > 0$, whatever the rank of Ψ, since $v^T(\Psi^T\Psi + \lambda I_P)v = |\Psi v|^2 + \lambda|v|^2 > 0$. So there is always a unique minimizer,

$$\theta_\lambda = (\Psi^T\Psi + \lambda I_P)^{-1}\Psi^T Y ,$$

at the price of a $P \times P$ inverse. To move the inverse to the $n \times n$ side, expand and regroup the product

$$\Psi^T(\Psi\Psi^T + \lambda I_n) = \Psi^T\Psi\Psi^T + \lambda\Psi^T = (\Psi^T\Psi + \lambda I_P)\Psi^T .$$

Both bracketed matrices are positive definite, so multiplying by their inverses on the appropriate sides gives the *push-through identity*

$$(\Psi^T\Psi + \lambda I_P)^{-1}\Psi^T = \Psi^T(\Psi\Psi^T + \lambda I_n)^{-1} .$$

Nothing was cancelled — Ψ^T is rectangular and has no inverse — and that is precisely why the identity holds. Substituting into (B.18), the ridge solution again lies in the span of the training feature vectors, $\theta_\lambda = \Psi^T\alpha_\lambda$, now with $\alpha_\lambda = (\hat{K} + \lambda I_n)^{-1}Y$, and the prediction at a new input is

$$f_\lambda(x) = \sum_{\mu,\nu=1}^{n} K(x, x^\mu)\,\big(\hat{K} + \lambda I_n\big)^{-1}_{\mu\nu}\, y^\nu .$$

This is *kernel ridge regression*, the central formula of this appendix. As $\lambda \to 0$ it reduces to the minimum-norm interpolant (B.14) when $\hat{K}$ is invertible, and in general to the least-squares solution of minimum norm (Exercise B.4(c)). As $\lambda \to \infty$, $\theta_\lambda \to 0$.

Formula (B.21) is exactly the posterior mean of Exercise 4.5, with ridge parameter $\lambda = \beta^{-1}$ — there derived for the infinite-width Gaussian process, here for a finite linear model. The Bayesian reading is worth making explicit. With a Gaussian likelihood $\propto e^{-\frac{\beta}{2}|\Psi\theta - Y|^2}$ and a Gaussian prior $\theta \sim \mathcal{N}(0, \sigma^2 I_P)$, the loss (B.16) is $-\beta^{-1}$ log of the posterior density, up to a constant, with $\lambda = 1/(\beta\sigma^2)$. Ridge regression is thus maximum a posteriori estimation. The prior pushed forward to $f_\theta(x) = \theta \cdot \psi(x)$ is a Gaussian process with kernel $\sigma^2 K(x, x')$, a finite-P instance of Section 3.3, and the posterior mean of Exercise 4.5 for this process reproduces (B.21).

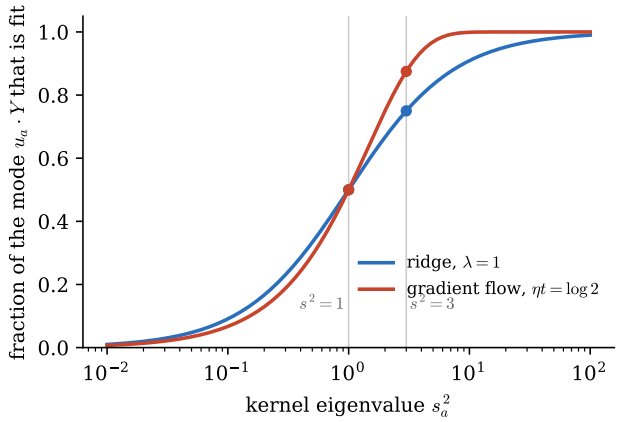
Spectral view. With the singular value decomposition $\Psi = USV^T$ of Section 4.2, the Gram matrix is $\hat{K} = US^2U^T$, with eigenvalues s_a^2 and orthonormal eigenvectors $u_a \in \mathbb{R}^n$: the "features" of Section 4.2, *i.e.* the natural modes in the space of function values. The ridge predictions on the training set are $f_\lambda = \Psi\theta_\lambda = \hat{K}(\hat{K} + \lambda I_n)^{-1}Y$, or, mode by mode,

$$f_\lambda = \sum_a \frac{s_a^2}{s_a^2 + \lambda} \, (u_a \cdot Y) \, u_a \, .$$

Ridge regression is a *spectral filter*: modes of the data with $s_a^2 \gg \lambda$ are fit, modes with $s_a^2 \ll \lambda$ are suppressed, and the crossover is at $s_a^2 \approx \lambda$. Compare gradient flow from $\theta_0 = 0$ on the unpenalized loss, whose training predictions are, by (4.35),

$$f(t) = \sum_a \left(1 - e^{-\eta t s_a^2}\right) (u_a \cdot Y) \, u_a \, .$$

This is also a spectral filter, with $1/\eta t$ playing the role of λ: *early stopping* regularizes much as ridge does. But the two filters have different shapes (Figure 13), and no choice of λ reproduces a given stopping time on every mode at once (Exercise B.5). This is the precise content of the remark closing Section 4.6, that the inverse temperature of Bayesian learning and the training time of gradient descent play analogous roles.



Ridge regression and early-stopped gradient flow as spectral filters: the fraction of a data mode $u_a \cdot Y$ that is fit, as a function of the mode's kernel eigenvalue s_a^2. The two are tuned to agree on the mode with $s^2 = 1$, and disagree elsewhere; the dots mark the two-mode example of Exercise B.5.

Exercise B.4 (Ridge regression, two ways). (a) Derive (B.17) and (B.18), and prove that $\Psi^T\Psi + \lambda I_P$ is invertible even when Ψ is rank deficient.

(b) Prove the push-through identity (B.20) and deduce (B.21). Explain why "cancelling Ψ^T" is not a valid step, and why the identity is nevertheless true. (It is a special case of $A(BA+I)^{-1} = (AB+I)^{-1}A$.)

(c) What happens as $\lambda \to \infty$? Using the SVD, show that θ_λ applies the scalar filter $s/(s^2+\lambda)$ to each singular value s of Ψ, and hence that as $\lambda \to 0$ it converges to the least-squares solution of minimum norm. When does this limit interpolate the labels exactly?

(d) Return to $x \in \{-1, 0, 1\}$, $y = x^2$ and the kernel $K(x,z) = (1+xz)^2$ of Exercise B.2. With $\lambda = 1$, verify that

$$\hat{K} = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{pmatrix}, \qquad \alpha_\lambda = \frac{1}{4}\begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix},$$

and predict at $z = 2$ using only kernel evaluations. Compare with the interpolant of Exercise B.2(a), which predicts $z^2 = 4$ there, and explain the direction of the discrepancy.

(e) The feature route solves a $P \times P$ system and the kernel route an $n \times n$ one. When is each attractive?

SOLUTION, p. 626

Exercise B.5 (Early stopping vs. ridge, on two data points). Take $n = 2$ training points with Gram matrix and labels

$$\hat{K} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad Y = \begin{pmatrix} 2 \\ 0 \end{pmatrix},$$

and initial predictions $f(0) = 0$.

(a) Verify the eigenpairs $u_\pm = \frac{1}{\sqrt{2}}(1, \pm 1)^T$ with $s_\pm^2 = 3, 1$, and decompose Y into these modes.

(b) Using (B.23), find the training predictions $f(t)$ under gradient flow, and evaluate them at $\eta t = \log 2$. Why do both the eigenvalue s_a^2 and the initial projection $u_a \cdot Y$ matter for what is visible during training?

(c) Using (B.22), find the ridge predictions for $\lambda = 1$. The two filters agree on the mode with $s^2 = 1$; show that the ridge parameter matching gradient flow at time t on a mode with eigenvalue s^2 is $\lambda_{\text{eff}}(s^2, t) = s^2/(e^{\eta t s^2} - 1)$, which depends on the mode — so early stopping is not ridge regression.

(d) State precisely what "spectral bias" means for this finite, fixed-kernel calculation. What additional structure would be needed to turn it into a claim about Fourier frequencies, about robustness to label noise, or about test error?

SOLUTION, p. 626

B.5 Linearized networks are kernel ridge regressors

Now let f_θ be a neural network, with $\theta \in \mathbb{R}^P$ the vector of *all* its weights and biases. It is not linear in θ. But its linearization around the initialization θ_0, the ansatz (4.1),

$$f_\theta^{\text{lin}}(x) = f_{\theta_0}(x) + (\theta - \theta_0) \cdot \nabla_\theta f_{\theta_0}(x)\,,$$

is a model of the form (B.9). Compare term by term. The parameters are the displacement $\delta\theta := \theta - \theta_0 \in \mathbb{R}^P$. The feature map is

$$\psi(x) = \nabla_\theta f_{\theta_0}(x) \in \mathbb{R}^P\,,$$

the *tangent features*: one number per parameter, the sensitivity of the output at x to that parameter, evaluated at initialization and then frozen. And the constant offset $f_{\theta_0}(x)$ is absorbed by shifting the labels to

$$\tilde{y}^\mu := y^\mu - f_{\theta_0}(x^\mu) = -\Delta^\mu(0)\,.$$

The offset matters: the linearized model is fit to the *residuals at initialization*, not to the raw labels. The feature matrix (B.10) is the Jacobian at initialization, $\Psi = J_0$, and the Gram matrix (B.13) is

$$\hat{K} = J_0 J_0^T = \Theta_0\,, \qquad K(x,x') = \Theta(x,x') = \nabla_\theta f_{\theta_0}(x) \cdot \nabla_\theta f_{\theta_0}(x')\,,$$

the neural tangent kernel (4.10) at initialization, extended to arbitrary pairs of inputs as in Exercise 4.3. Everything in Appendices B.3 and B.4 now applies verbatim. In particular, ridge regression on the linearized network with penalty $\frac{\lambda}{2}|\theta - \theta_0|^2$ — weight decay toward the initialization — gives, by (B.21),

$$f_\lambda^{\text{lin}}(x) = f_{\theta_0}(x) - \sum_{\mu,\nu=1}^{n} \Theta(x, x^\mu) \left(\Theta_0 + \lambda I_n\right)^{-1}_{\mu\nu} \Delta^\nu(0)\,.$$

At $\lambda \to 0$ this is the $t \to \infty$ limit of (4.36), the minimum-norm interpolant of the residuals; and the finite-time solution (4.36) is its early-stopped cousin, in the sense of (B.23). The kernel trick is not a convenience here but a necessity: P is the parameter count of the network, often 10^7 or more, while Θ_0 is $n \times n$.

Two remarks on what is and is not being claimed. First, for the linearized model f^{lin} everything above is exact, and its kernel is constant by construction. For the actual network, (B.24) is an ansatz. Section 4.3 shows that it becomes exact in the large-width limit at fixed depth and data; whether and when it is a good approximation otherwise — and what happens when the tangent features move — is the subject of Sections 5–6, not of this appendix. Second, the initialization θ_0 is random, so f_{θ_0}, the tangent features, and the kernel Θ_0 are all random too, and (B.28) is the trained function for one draw. Averaging over draws, at large width where Θ_0 becomes deterministic, gives the Gaussian process (4.38).

Exercise B.6 (Tangent features of a one-hidden-layer network). Consider $f_\theta(x) = \sum_{i=1}^{N} c_i\,\phi(a_i \cdot x)$ with $\theta = (c_i, a_i)_{i=1}^{N}$, so that $P = N(d+1)$.

(a) Compute the tangent features $\nabla_\theta f_{\theta_0}(x)$, and show that the tangent kernel is

$$\Theta(x, x') = \sum_{i=1}^{N} \phi(a_i \cdot x)\,\phi(a_i \cdot x') + \sum_{i=1}^{N} c_i^2\,\phi'(a_i \cdot x)\,\phi'(a_i \cdot x')\,(x \cdot x')\,.$$

(b) Suppose only the output weights c are trained, with the a_i frozen at their random initial values. Show that this is exactly a fixed-feature model (B.9) with random features, that its linearization is exact, and that its kernel is the first term above. Which term do you get by training only the a_i?

(c) Show that gradient flow with weight decay toward the initialization, $\dot\theta = -\eta\big[\nabla_\theta \mathcal{L} + \lambda(\theta - \theta_0)\big]$, applied to the linearized model f^{lin}, converges to the ridge solution (B.28) for every $\lambda > 0$ and from any starting point.

(d) Check that (B.28) at $\lambda \to 0$ coincides with (4.36) at $t \to \infty$.

SOLUTION, p. 627

SUPPLEMENT TO CHAPTER C.3

Computational Mechanics

Xavier Poncini (Simplex), Adam Shai (Simplex), Paul Riechers (Simplex)

Main text: Chapter C.3, p. 241

2.1 Fast-track

Read Transformers represent belief state geometry in their residual stream. Focus on being able to answer the following questions:

- What is a hidden Markov model (HMM)?
- What is a belief state?
- What is the mixed state presentation (MSP)?
- About the map learnt from activations to belief states:
  - What is the source?
  - What is the target?
  - How is the quality of the map evaluated?

5.1.2 [Extension] Neural networks leverage nominally quantum and post-quantum representations

Reading route

- **Skim.** Sections 1–3.
- **Read.** Sections 4–8.
- **Evaluation.** Explain Figure 3B. How are the light-blue and light-orange plots related to the corresponding dark-blue and dark-orange plots?

Overview. This paper extends the question from HMMs to generalised hidden Markov models (GHMMs). As in Part 3, a GHMM retains the matrix-product expression for output probabilities while relaxing the entrywise probabilistic constraints on its internal matrices. This larger model class can realise some processes in finite dimension even when a corresponding minimal HMM requires exponentially many more hidden states.

Using techniques related to those in the main reading, the authors study transformers, LSTMs, RNNs, and GRUs trained on processes with distinct GHMM and HMM realisations. Their results indicate that the networks learn belief-state representations associ-

ated with the lower-dimensional GHMM realisation. The finding suggests that neural networks can select among representational model classes in ways that capture dimensionality savings.

7. Learn more

- Beyond toy architectures – in-context learning
  - LLMs have an amazing ability to adapt internal representations in-context – [Read: ICLR: In-Context Learning of Representations]
  - LLMs can predict HMM data in-context – [Read: Pre-trained Large Language Models Learn Hidden Markov Models In-context]
  - This suggests that internal representations of LLMs predicting GHMM data in-context, may resemble belief states.
- Processes consisting of GHMMs with richer structure [Read: Transformers learn factored representations]
- Review article of computational mechanics [Read: Between order and chaos]
- Simplex is a non-profit research organisation working on applying computational mechanics to AI safety.

Condensation

Satya Benson

Main text: Chapter C.4, p. 255

2.4 Optional: several observations as one witness

A shared parameter need not be recoverable from a single observation. For example, take $n \geq 2$, let C be a fair bit, and set $X_i = C \oplus N_i$, where the N_i are independent Bernoulli(p) noises with $0 < p < 1/2$, independent of C. The model $Y_I = C$, $Y_{\{i\}} = X_i$ satisfies LVM and the Markov condition exactly, but $H(C \mid X_i) = h_2(p) > 0$, where h_2 is the binary entropy function of Section 2.2.1. Larger observation blocks can reveal more about C. To use them in the proof, replace $\supseteq\{i\}$ by $\mathcal{J}_B$.

Exercise 2.9 (Block witnesses, optional). (a) Let $I = \{1, 2, 3\}$ and suppose a target U has conditional entropy at most δ given each of $X_{\{1,2\}}$, $X_{\{1,3\}}$, and $X_{\{2,3\}}$. Which addresses survive the intersection $\mathcal{J}_{\{1,2\}} \cap \mathcal{J}_{\{1,3\}} \cap \mathcal{J}_{\{2,3\}}$? Prove a bound on the entropy of U given the surviving Z-slots.

(b) More generally, let $\mathcal{W}$ be a nonempty finite family of nonempty observation blocks. Adapt Exercise 2.8 to show

$$H(U \mid Z_{\mathcal{R}}) \leq \sum_{B \in \mathcal{W}} H(U \mid X_B) + (|\mathcal{W}| - 1)\eta_Z, \qquad \mathcal{R} = \bigcap_{B \in \mathcal{W}} \mathcal{J}_B.$$

(c) Fix an integer $0 \leq r < |A|$ and take every $(r+1)$-element subset of A as a witness. Show that $C \in \mathcal{R}$ exactly when $|A \setminus C| \leq r$. What does this give for $r = 0$? Why is the receiver generally larger for $r > 0$?

SOLUTION, p. 648

Eisenstat's later almost-perfect-condensation formulation (Eisenstat 2025) bounds the tower reconstruction error whenever $|A \cap B| > r$, the Markov defect, and the number of active slots. The threshold allows small slots to account for local noise without requiring every such slot to be recovered from every relevant block. It also changes which receiver the proof isolates, as Exercise 2.9 shows. The exact-tower bound with singleton witnesses does not follow from an arbitrary positive threshold r.

2.5 Optional: the conditioned score

The conditions also have a description-length interpretation. For an observation block B, compare

$$\sigma(B) = \sum_{A \in \mathcal{J}_B} H(Y_A), \qquad \chi(B) = \sum_{A \in \mathcal{J}_B} H(Y_A \mid Y_{\supseteq A \setminus \{A\}}).$$

The simple score σ charges each active slot separately. The conditioned score χ charges its additional information given strict supersets. Those supersets are also active for B. Encoding in reverse inclusion order makes this conditioning available. These are ideal expected description-length quantities; they do not measure computational retrieval time.

Both scores are at least $H(X_B)$. More precisely, put $V_B = \chi(B) - H(Y_{\mathcal{J}_B})$. Then

$$\chi(B) - H(X_B) = V_B + H(Y_{\mathcal{J}_B} \mid X_B), \qquad V_B \geq 0.$$

The second term charges latent information not recoverable from X_B. The first charges dependence not accounted for by the inclusion hierarchy. It vanishes under the ordered Markov condition.

For a graphical-model expression, define

$$q_B(y) = \prod_{A \in \mathcal{J}_B} p(y_A \mid y_{\supseteq A \setminus \{A\}}).$$

This is a normalized distribution: sample larger addresses before smaller ones. Conditional kernels on probability-zero parent configurations can be chosen arbitrarily. Direct expansion gives $V_B = D_{\mathrm{KL}}(p_{Y_{\mathcal{J}_B}} \| q_B)$. Thus (5) separates two nonnegative defects.

Exercise 2.10 (The two charges, optional). (a) Order $\mathcal{J}_B$ so strict supersets appear first. Use the chain rule and conditioning inequality to prove $\chi(B) \geq H(Y_{\mathcal{J}_B})$.

(b) Use LVM to prove $H(Y_{\mathcal{J}_B}) - H(X_B) = H(Y_{\mathcal{J}_B} \mid X_B)$, hence (5).

(c) For both models in Exercise 2.3, calculate $\sigma(\{1,2\})$ and $\chi(\{1,2\})$. Why is copying the top variable into a lower slot free under one score but not the other?

(d) If $A \cap B \neq \varnothing$, show that $H(Y_{\supseteq A} \mid X_B) \leq \chi(B) - H(X_B)$.

SOLUTION, p. 649

Eisenstat defines perfect condensation by $\chi(B) = H(X_B)$ for every B. The equivalence with the characterization above is his Theorem 5.10. The decomposition above also explains why small score gaps imply strong reconstruction conditions. The converse for positive-threshold almost-perfect condensation requires additional reconstruction coverage: some active slots may never meet that threshold. Do not identify the two approximate criteria without checking these quantifiers.

6. Further reading

6.1 Condensation

The required mathematics is contained in Chapter C.4.

- Sam Eisenstat, *Condensation: A Theory of Concepts* (2025). Definitions and exact characterization: Sections 4–5. Correspondence: Theorems 5.15 and 6.8; two-witness inequality: Lemma 6.4 (Eisenstat 2025).
- Satya Benson, *Relating Almost Perfect Condensation Conditions and the Conditioned Score*. The score/condition bridge and its reconstruction coverage qualification (Benson 2026).
- For the wider statistical context: Cover and Thomas, *Elements of Information Theory* (Cover & Thomas 2006); Lauritzen, *Graphical Models* (Lauritzen 1996); Shalizi and Crutchfield, *Computational Mechanics: Pattern and Prediction, Structure and Simplicity* (2001) (Shalizi & Crutchfield 2001).

6.2 Condensation in context

The sources cited above support different parts of the discussion. For the mathematical development, start with Eisenstat's *Condensation: A Theory of Concepts* (Eisenstat 2025), particularly the exact and quantitative correspondence results. For the score connection, see *Relating Almost Perfect Condensation Conditions and the Conditioned Score* (Benson 2026). The score decomposition connects excess conditioned code length to dependence and recoverability defects; the converse requires attention to which observation sets the reconstruction assumptions cover.

SUPPLEMENT TO CHAPTER D.3.1

Solomonoff Induction

David Quarel (ARENA), Leon Lang (Iliad)

Main text: Chapter D.3.1, p. 315

A. Proof of Pinsker's inequality

We first prove the following lemma:

Lemma A.1 (Pinsker's binary inequality). For $0 \le p \le 1$ and $0 < q < 1$ we have

$$2(p-q)^2 \;\le\; p \ln \frac{p}{q} + (1-p) \ln \frac{1-p}{1-q}.$$

Define $f(x) := p \ln x + (1-p)\ln(1-x)$, so that

$$p \ln \frac{p}{q} + (1-p) \ln \frac{1-p}{1-q} = f(p) - f(q) = \int_q^p f'(x)\,dx = \int_q^p \frac{p-x}{x(1-x)}\,dx,$$

using $f'(x) = \frac{p}{x} - \frac{1-p}{1-x} = \frac{p-x}{x(1-x)}$. Recall also that $x(1-x) \le \frac{1}{4}$ on $[0,1]$, so $\frac{1}{x(1-x)} \ge 4$.

Case $q \le p$. For every $x \in [q,p]$ we have $x \le p$, hence $p - x \ge 0$.

$$\frac{p-x}{x(1-x)} \;\ge\; 4(p-x) \qquad \text{for } x \in [q,p].$$

Integrating over $[q,p]$,

$$\int_q^p \frac{p-x}{x(1-x)}\,dx \;\ge\; 4\int_q^p (p-x)\,dx \;=\; 2(p-q)^2.$$

Case $q \ge p$. For every $x \in [p,q]$ we have $x \ge p$, hence $x - p \ge 0$.

$$\frac{x-p}{x(1-x)} \;\ge\; 4(x-p) \qquad \text{for } x \in [p,q].$$

Integrating over $[p,q]$,

$$\int_q^p \frac{p-x}{x(1-x)}\,dx = \int_p^q \frac{x-p}{x(1-x)}\,dx$$
$$\geq 4\int_p^q (x-p)\,dx = 4\int_q^p (p-x)\,dx = 2(p-q)^2$$

Proof of Theorem 4.1

We now reduce Theorem 4.1 to the binary inequality Lemma A.1. Recall $p_0 + p_1 = 1$ and $q_0 + q_1 = 1$. We handle the boundary case $q_x = 0$ first, then reduce the interior case directly to Lemma A.1.

Case 1: $q_x = 0$ for some $x \in \mathbb{B}$. If $p_x > 0$, then the right-hand side of $(\star)$ contains $p_x \ln \frac{p_x}{0} = +\infty$, so $(\star)$ holds trivially. If instead $p_x = 0$, that atom contributes 0 to both sides (using the convention $0 \ln \frac{0}{0} = 0$). The other atom x' then satisfies $p_{x'} = 1$ and $q_{x'} = 1 - q_x = 1$, so both sides vanish and $(\star)$ holds.

Case 2: $q_0, q_1 > 0$. Since $q_1 = 1 - q_0$ and $p_1 = 1 - p_0$, the two sides of $(\star)$ become

$$\sum_{x\in\mathbb{B}} (q_x - p_x)^2 = (q_0 - p_0)^2 + \big((1-q_0) - (1-p_0)\big)^2 = 2(p_0 - q_0)^2,$$

$$\sum_{x\in\mathbb{B}} p_x \ln \tfrac{p_x}{q_x} = p_0 \ln \tfrac{p_0}{q_0} + (1-p_0) \ln \tfrac{1-p_0}{1-q_0}.$$

So $(\star)$ is exactly $2(p_0 - q_0)^2 \leq p_0 \ln \frac{p_0}{q_0} + (1-p_0) \ln \frac{1-p_0}{1-q_0}$, which is Lemma A.1 with $p = p_0$ and $q = q_0$ (here $0 < q_0 < 1$, since $q_0, q_1 > 0$).

B. Proof of KL non-negativity

We prove Theorem 2.2. The core is the elementary inequality

$$\ln x \;\leq\; x - 1 \qquad \text{for all } x > 0, \quad \text{equality iff } x = 1.$$

(Let $f(x) := (x-1) - \ln x$; then $f'(x) = 1 - 1/x$, vanishing only at $x = 1$, with $f''(x) = 1/x^2 > 0$, so f is strictly convex and attains its unique minimum $f(1) = 0$ at $x = 1$.)

Lemma B.1 (Gibbs' inequality). Let P, Q be probability distributions over a finite set $\mathcal{X}$, with the conventions $0 \ln \frac{0}{q} := 0$ and $p \ln \frac{p}{0} := \infty$. Then

$$\sum_{x\in\mathcal{X}} P(x) \ln \frac{P(x)}{Q(x)} \;\geq\; 0,$$

with equality iff $P(x) = Q(x)$ for every x with $P(x) > 0$.

Proof

If $Q(x) = 0$ for some x with $P(x) > 0$, the LHS is $+\infty$ and the inequality is strict. Otherwise restrict the sum to $\{x : P(x) > 0\}$ (terms with $P(x) = 0$ contribute 0 by convention), where every ratio $Q(x)/P(x)$ is in $(0, \infty)$. Apply $\ln y \le y - 1$ to $y = Q(x)/P(x)$:

$$\begin{aligned} -\sum_x P(x) \ln \frac{P(x)}{Q(x)} \;&=\; \sum_x P(x) \ln \frac{Q(x)}{P(x)} \;\le\; \sum_x P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \sum_x Q(x) - \sum_x P(x) \;\le\; 0, \end{aligned}$$

where the last step uses $\sum_x Q(x) \le 1 = \sum_x P(x)$ (the sum over $\{x : P(x) > 0\}$ may miss some Q-mass). Negating gives the claim. Equality in $\ln y \le y - 1$ requires $y = 1$ for every x with $P(x) > 0$, i.e. $Q(x) = P(x)$ there; equality in $\sum Q \le \sum P$ then forces Q to have no mass outside $\operatorname{supp} P$. Both together give $P = Q$ on all of $\mathcal{X}$.

Proof of Theorem 2.2

Part (a). Apply Lemma B.1 with $\mathcal{X} = \mathbb{B}^n$, $P(x_{1:n}) = \nu(x_{1:n})$, $Q(x_{1:n}) = \rho(x_{1:n})$. Both are probability distributions on $\mathbb{B}^n$ since ν and ρ are environments. The lemma gives $D_n(\nu \parallel \rho) \ge 0$ with the stated equality condition.

Part (b). For each fixed history $x_{<t}$ with $\nu(x_{<t}) > 0$, apply Lemma B.1 with $\mathcal{X} = \mathbb{B}$, $P = \nu(\cdot \mid x_{<t})$, $Q = \rho(\cdot \mid x_{<t})$ (both are probability distributions on $\mathbb{B}$):

$$\sum_{x_t \in \mathbb{B}} \nu(x_t \mid x_{<t}) \ln \frac{\nu(x_t \mid x_{<t})}{\rho(x_t \mid x_{<t})} \;\ge\; 0,$$

with equality iff $\nu(\cdot \mid x_{<t}) = \rho(\cdot \mid x_{<t})$. Multiply by $\nu(x_{<t}) \ge 0$ and sum over $x_{<t}$: this gives $d_t(\nu \parallel \rho) \ge 0$, as a sum of non-negative terms. Equality forces every $x_{<t}$ with $\nu(x_{<t}) > 0$ to contribute a vanishing inner sum, which is exactly the stated condition.

C. Model Class $\mathcal{M}$

All results in this sheet hold for any countable class $\mathcal{M}$ and prior w_ν. The canonical **Solomonoff** choice takes $\mathcal{M}$ to be the class of all computable environments and the **Solomonoff prior**

$$w_\nu \;:=\; 2^{-K(\nu)},$$

where $K(\nu)$ is the *Kolmogorov complexity* of ν: the length of the shortest program that computes ν. The Bayesian mixture ξ under this particular prior is the *universal mixture*, written ξ_U. With this class the assumption $\mu \in \mathcal{M}$ reduces to "the universe is computable": as weak an assumption as one can make. For technical details on Kolmogorov complexity, see (Hutter et al. 2024, §2.7).

The class of all computable measures is countable but not enumerable (by the halting problem, you can't decide which Turing machines define total functions), so one can't even formally write $\sum_{\nu\in\mathcal{M}}\ldots$, as the decision problem of determining if $\nu \stackrel{?}{\in} \mathcal{M}$ is undecidable. A faithful construction of ξ_U requires expanding $\mathcal{M}$ to all *lower-semicomputable semimeasures* (Levin), at which point ξ_U becomes a semimeasure rather than a measure: $\sum_x \xi_U(x) \leq 1$ instead of $= 1$, conditionals may not sum to 1, and every other proof in this sheet gains a side-case. This also gives the nice benefit that $\xi \in \mathcal{M}$.

We dodge all of this: $\mathcal{M}$ is just *some* countable set of proper measures with $\mu \in \mathcal{M}$, and we assume $K(\nu)$ is defined for each $\nu \in \mathcal{M}$ when we need it. We never need ξ itself to be in $\mathcal{M}$, nor do we need its computability, so we don't ask. See (Hutter 2005, §2.4.3) for the full Levin construction.

D. Best choice of $\hat{\mu} \in \mathcal{M}$.

The bound is valid for every $\hat{\mu} \in \mathcal{M}$. A tempting question: which $\hat{\mu}$ gives the tightest bound? The answer is genuinely setting-dependent.

Finite horizon n. The minimizer of the RHS is

$$\hat{\mu}_n \;:=\; \arg\min_{\nu\in\mathcal{M}}\Big\{-\ln w_\nu \;+\; D_n(\mu \,\|\, \nu)\Big\}.$$

This is well-defined (finitely many terms, attained when $\mathcal{M}$ is finite), but the answer depends on n: for small n the prior term dominates and a high-prior coarse model wins; for large n the fit term dominates and the best per-step approximator wins. Both regimes are correct.

Asymptotic rate. A horizon-free analogue minimizes the per-step KL rate

$$\lim_{n\to\infty} \frac{1}{n} \sum_{t\leq n} d_t(\mu \,\|\, \nu).$$

This is the right notion when one cares about the linear-growth slope of the bound. But it has two drawbacks: the limit need not exist for non-stationary μ (one can substitute lim inf but at the cost of clarity), and it discards the prior w_ν entirely, so the minimizer is not the same as $\hat{\mu}_n$ at any finite n, since only the slope is minimized.

Stationary μ. The per-step KLs $d_t(\mu \,\|\, \nu)$ are constant in t, so all three notions essentially agree: $\hat{\mu} = \arg\min_{\nu\in\mathcal{M}} D_{\mathrm{KL}}(\mu \,\|\, \nu)$, where the KL is taken under the (common) one-step conditional law. This is the case where "$\hat{\mu}$ = projection of μ onto $\mathcal{M}$ in KL" has unambiguous meaning.

Upshot. The bound holds for all $\hat{\mu} \in \mathcal{M}$, and one is free to take the infimum on the RHS over $\hat{\mu}$. Which $\hat{\mu}$ actually attains that infimum is a separate question whose answer depends on horizon, prior, and any structural assumptions on μ.

SUPPLEMENT TO CHAPTER D.3.2

AIXI

David Quarel (ARENA)

Main text: Chapter D.3.2, p. 327

Further reading

- Hutter, *An Introduction to Universal Artificial Intelligence* (2024): Chapter 2.7 (Kolmogorov complexity), Chapters 3.7–3.8 (the model class and universal prior), and Chapter 7.4 (AIXI).
- Hutter, *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability* (Springer, 2005) — the original book-length treatment; Lem. 5.28 handles the countable-$\mathcal{M}$ self-optimizing case.
- Blackwell & Dubins, *Merging of Opinions with Increasing Information* (Ann. Math. Statist., 1962) — the merging-of-opinions theorem behind on-policy value convergence.
- Leike & Hutter, *Bad Universal Priors and Notions of Optimality* (COLT 2015) — adversarial choices of the universal Turing machine can make AIXI behave arbitrarily badly.

A. Worked Example: Bayesian Mixture and Value Function

Setup.

- **Actions:** $\mathcal{A} = \{H, T\}$ (predict the next coin flip)
- **Observations:** $\mathcal{O} = \{H, T\}$ (actual coin flip)
- **Rewards:** $\mathcal{R} = \{0, 1\}$, with $r_t = [\![a_t = o_t]\!]$
- **Model class:** $\mathcal{M} = \{\nu_{HH}, \nu_{HT}\}$ (two-headed coin, fair coin)
- **Prior:** $w_{\nu_{HH}} = w_{\nu_{HT}} = \frac{1}{2}$

Before any interaction ($t = 1$, $æ_{<1} = \epsilon$):

$$\xi(o_1 = H \mid a_1) \;=\; \tfrac{1}{2} \cdot 1 + \tfrac{1}{2} \cdot \tfrac{1}{2} \;=\; \tfrac{3}{4}.$$

After observing a head ($t = 2$), the posterior updates:

$$w(\nu_{HH} \mid æ_1) \;=\; \tfrac{1}{2} \cdot \frac{1}{3/4} \;=\; \tfrac{2}{3}, \quad w(\nu_{HT} \mid æ_1) \;=\; \tfrac{1}{2} \cdot \frac{1/2}{3/4} \;=\; \tfrac{1}{3}.$$

Updated prediction: $\xi(o_2 = H \mid æ_1 a_2) = \frac{2}{3} \cdot 1 + \frac{1}{3} \cdot \frac{1}{2} = \frac{5}{6}$.

Value function. Continuing the same setup, suppose the agent always predicts H (policy π_H).

Under ν_{HH}: always correct, $r_t = 1$ every step: $V^{\pi_H}_{\nu_{HH}}(\epsilon) = (1-\gamma)\sum_{k=0}^{\infty} \gamma^k \cdot 1 = 1$.

Under ν_{HT}: correct half the time: $V^{\pi_H}_{\nu_{HT}}(\epsilon) = (1-\gamma)\sum_{k=0}^{\infty} \gamma^k \cdot \frac{1}{2} = \frac{1}{2}$.

The mixture value (by Exercise 4.2) is: $V^{\pi_H}_{\xi}(\epsilon) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4}$.

As the agent observes more heads ($\mu = \nu_{HH}$), the posterior on ν_{HH} increases towards 1, and $V^{\pi_H}_{\xi} \to V^{\pi_H}_{\nu_{HH}} = 1$. This is on-policy value convergence (Theorem 7.6) in action.

B. Knuth's Difficulty Scale

Each subproblem carries a difficulty rating in square brackets, following Knuth's rating scheme for exercises (Knuth 1973) in slightly adapted form. The rating assumes that the material in the preceding problems (on which the subproblem depends) has been understood. In-between values are possible.

[00] *Very easy.* Solvable from the top of your head.

[10] *Easy.* Needs 15 minutes to think, possibly pencil and paper.

[20] *Average.* May take 1–2 hours to answer completely.

[30] *Moderately difficult or lengthy.* May take several hours to a day.

[40] *Quite difficult or lengthy.* Often a significant research result.

[50] *Open research problem.* An obtained solution should be published.

SUPPLEMENT TO CHAPTER D.4.1

Agent Foundations

Daniel C, Satya Benson (Williams College)

Main text: Chapter D.4.1, p. 345

2.1 Fast-track

To get the core in about an hour:

- Read the "Unifying threads" summary in Chapter D.4.1 for the unifying spine (reflective stability, embedded agency, the modelling-vs-implementation split).
- Read Embedded agency up to and including section 3.3, then section 4.1, and Why agent foundations in full.
- Pick the one strand closest to your background from "The five strands" in Chapter D.4.1 and read its first (most conceptual) listed reading.
- Skim the takeaways at the end of each strand summary.

2.3 Learn more

The readings for each strand are linked under that strand in Chapter D.4.1. Beyond them:

Algorithmic thermodynamics (Ebtekar). Foundations of algorithmic thermodynamics; Modelling the arrows of time with causal multibaker maps; Long-time derivation of the Boltzmann equation from hard-sphere dynamics.

Embedded and universal AI (Wyeth). Limit-computable grains of truth; Embeddedness failures in universal artificial intelligence; Value under ignorance.

Other directions. Introduction to the infra-Bayesianism sequence (non-realizable environments); The learning-theoretic agenda; Optimization at a distance; the hard problem of corrigibility and a critique of the corrigibility basin of attraction; the original tiling agents draft; and background notes on admissibility and the complete class theorem and the Dutch book argument.

Research frontier. The *agent structure problem* (whether strong optimization provably entails an internal world model), embedded variants of AIXI, and resource-theoretic accounts of instrumental convergence.

SUPPLEMENT TO CHAPTER D.4.2

Optimization and Thermodynamics

Daniel C, Satya Benson (Williams College)

Main text: Chapter D.4.2, p. 363

A. A thermodynamics of biased coins: the generalized heat engine

This appendix develops, as a self-contained worked example, the toy thermodynamic world referenced from two points elsewhere in these notes (the blind baseline of Appendix B and the Type-1 channel of Section 5); it may be read at any point after Section 4. The construction follows Wentworth's essay *Generalized Heat Engine*, whose objective is to take the distinctively thermodynamic ideas of statistical mechanics (heat, work, engines, the impossibility of perpetual motion) and reconstruct them systematically in a setting containing no physics whatsoever, thereby isolating precisely which components of thermodynamics are fundamentally facts about information. The model also makes concrete a perspective, the *designer's viewpoint*, that transfers directly to the analysis of agents embedded in environments they cannot fully observe.

A.1 The designer's viewpoint

We begin by examining the epistemic situation of a designer who wishes to construct a refrigerator, and in particular the question of why the designer cannot simply build a machine that observes the air molecules and removes the fast ones. All of the microscopic dynamics of the combined refrigerator-and-environment system are reversible, so, as established in Section 4.1, the number of possible microstates compatible with what is known never decreases of its own accord, and the only mechanism for reducing uncertainty about a microstate is observation. The designer, however, operates *in advance*: the behavior of the machine is fixed at design time, with no access to the exact positions the air molecules will occupy when the machine is eventually switched on. From the designer's perspective, therefore, there is uncertainty that cannot be reduced but only *relocated*. The machine itself can "observe" variables while running, but the machine is part of the total system, and its observations constitute further reversible dynamics: any record the machine produces is a physical change within the

machine, subject to the same conservation rules as every other component of the system. (We analyze precisely this maneuver, observation implemented as reversible copying, in our treatment of Maxwell's demon in Section 5.)

The design problem is therefore to choose transformations, in advance and without inspection of the system, that make some chosen variables (the interior of the refrigerator) *more* certain at the cost of making others (the heat baths powering the machine) *less* certain. Thermodynamic-style laws apply whenever three conditions hold: the designer cannot gain information about the system (no "peeking" at design time), information cannot be destroyed at the lowest level (reversibility), and some conserved quantity constrains the admissible transformations (energy, in the physical case). Wentworth's toy world possesses exactly these three conditions and nothing else, providing a minimal setting in which the resulting laws can be derived in full.

A.2 Specification of the model: coins, transformations, and conservation laws

Having identified the three conditions under which thermodynamic-style laws apply, we now specify Wentworth's toy world in full. The world consists of two large pools of independent biased coins, in which each coin shows 1 (heads) or 0 (tails):

- a *cold pool* $X_1^C, \ldots, X_n^C$, in which each coin is heads with probability 0.1 (entropy $h(0.1) \approx 0.47$ bits per coin, by Example 2.2); and
- a *hot pool* $X_1^H, \ldots, X_n^H$, in which each coin is heads with probability 0.2 (entropy $h(0.2) \approx 0.72$ bits per coin).

The heads-probability plays the role of temperature (the hot pool is more uncertain per coin), while the *number of heads* plays the role of energy. We may apply any transformation T that replaces the coins' values with new values computed from their old values, subject to three rules mirroring the three conditions identified above:

1. **Reversibility.** T must be invertible: from the final state of all the coins (together with knowledge of which transformation was applied), the initial state must be reconstructible. This is the analogue of microscopic reversibility.
2. **Conservation.** T must conserve the total number of heads, the analogue of energy conservation: heads may be relocated between coins but neither created nor destroyed on net.
3. **Design-Time Ignorance.** The transformation T is chosen before any coin is inspected (the "no peeking" condition), exactly as the refrigerator designer commits to a blueprint before the machine encounters its environment.

To demonstrate that interesting transformations exist within these rules, we consider Wentworth's example, in which T acts on three coins as follows: *if* X_1^C *shows tails, swap* X_3^H *with* X_7^H*; if* X_1^C *shows heads, change nothing.* This transformation conserves heads (either two coins are swapped, which permutes values without changing their sum, or nothing occurs), and it is reversible (the control coin X_1^C is itself unchanged,

so the final state determines whether the swap occurred, and applying the same transformation a second time undoes it). Conditional swaps of this kind, composed in sequence, constitute the fundamental building blocks from which all admissible engines are assembled, and the example already establishes a fundamental point: a transformation can *read* one part of the system and act on another, entirely within deterministic, reversible, conservative rules, demonstrating that machines that "observe while running" lie not outside the formalism but within it.

In summary, the designer selects an invertible, heads-conserving map T from coin configurations to coin configurations, the world applies it, and our objective is to characterize precisely what such maps can accomplish.

A.3 Work extraction as a compression problem

We define a *work coin* to be a coin whose final value is heads with near-certainty (probability approaching 1 as n grows). Work coins are the analogue of stored mechanical work or a charged battery: a resource in a known, definite state, available to power other processes. *Extracting work* means choosing T so that some w designated coins are rendered work coins by the transformation, thereby converting statistical uncertainty into a resource in a known state.

The first key observation is that reversibility transforms work extraction into a data compression problem. We write X for the (random) initial configuration of all the coins and $X' = T(X)$ for the final configuration. Because T is invertible, the final state determines the initial state: X' contains *all* of the information in X, so $H(X') \geq H(X)$ (in fact $H(X') = H(X)$, since invertible maps preserve entropy exactly). Suppose now that w of the final coins are deterministic. Deterministic coins carry zero entropy, so all $H(X)$ bits of initial uncertainty must be carried by the remaining coins; in other words, *extracting w work coins requires compressing the information content of the entire initial state into the other coins*. Producing certainty in one location therefore requires concentrating uncertainty elsewhere, reflecting the fundamental fact that under reversible dynamics nothing is ever erased.

A.4 The impossibility of work extraction from a single heat bath

We now attempt to extract work from the hot pool alone and demonstrate that the attempt must fail; this failure constitutes the toy-world analogue of the impossibility of a perpetual motion machine of the second kind (a machine that extracts work from a single-temperature heat bath).

The hot pool's n coins carry $H(X) = 0.72n$ bits of entropy, and a compression scheme can in principle encode these bits into approximately $0.72n$ coins, leaving $0.28n$ coins deterministic. The conservation law, however, renders this compression infeasible, as the following counting argument demonstrates. Fully compressed data is incompressible, which means that it appears statistically uniform: each compressed coin is heads with probability approximately $\frac{1}{2}$ (if the compressed coins were biased, they would admit further compression, contradicting full compression). The $0.72n$ compressed

coins would therefore contain approximately $0.36n$ heads, whereas the initial state contains only $0.2n$ heads, and heads are conserved. Even if every one of the intended work coins were set to tails rather than heads, the accounting cannot be made consistent: the required compression demands more heads than the system possesses, and the attempt fails.

This argument admits a generalization extending well beyond the setting of coins, which we now develop in order to characterize the resource underlying all work extraction. The hot pool's distribution (independent coins, each heads with probability 0.2) is the *maximum-entropy* distribution among all distributions with its expected number of heads; in this precise sense a heat bath is "as random as its energy allows". Suppose, in general, that a pool of variables is maxentropic subject to a constraint that fixes the value of some additive quantity $\sum_k f_k(X_k)$. Deterministically fixing the value of one variable (extracting work from it) shrinks the set of values the constrained sum can take on the remaining variables, and therefore shrinks the maximum entropy the remaining variables can hold. Since the initial state already saturated this maximum, the remaining variables cannot absorb all of the information, and compression must fail. *Work cannot be extracted from a system that is already at maximum entropy given its constraints.* The slack between a system's actual entropy and its constrained maximum, which following standard usage we call *negentropy*, is therefore the sole resource from which work can be drawn.

A.5 Work extraction from two heat baths at different temperatures

We now employ both pools: $2n$ coins, with total entropy $\approx 0.47n + 0.72n = 1.19n$ bits and total heads $\approx (0.1 + 0.2)n = 0.3n$. The crucial difference from the single-pool case is that the *joint* initial distribution is not maxentropic given the joint constraint: the maximum-entropy distribution with $0.3n$ heads spread over $2n$ coins would make every coin heads with probability 0.15, whereas the actual state maintains two distinct biases, 0.1 and 0.2. The gap between actual entropy and constrained maximum entropy is negentropy, and it constitutes a resource that the designer can expend.

We now quantify the amount of work that this negentropy affords, deriving the maximal number of work coins extractable from the two pools. Suppose that we designate w coins as intended work coins (deterministic heads). The remaining $2n - w$ coins must carry all $1.19n$ bits of initial entropy while containing the remaining $0.3n - w$ heads. To carry as much information as possible, the final distribution of those $2n - w$ coins should itself be maxentropic subject to its heads constraint, which (for large n) means independent coins, each heads with probability

$$q \;=\; \frac{0.3n - w}{2n - w},$$

giving total entropy $(2n - w)\, h(q)$ where h is the binary entropy function of Example 2.2. The largest feasible w makes this capacity exactly equal to the required $1.19n$ bits:

$$(2n - w)\, h\!\left(\frac{0.3n - w}{2n - w}\right) \;=\; 1.19n.$$

Solving numerically gives $w \approx 0.011n$. A heads-conserving, reversible, designed-in-advance transformation therefore *can* extract work from two heat baths at different temperatures, converting approximately 3.7% of the total heads (5.5% of the hot pool's heads) into work coins. This construction is the toy world's heat engine, and its derivation required nothing beyond information-theoretic reasoning, confirming the claim that the distinctive behavior of heat engines is fundamentally a fact about information.

Wentworth notes one caveat regarding comparison of this figure with the physics literature: the efficiency computed here is not the classical Carnot efficiency, because the classical result answers a slightly different question (the optimal conversion rate *at the margin*, for engines drawing from effectively inexhaustible baths, generally consuming unequal amounts from the hot and cold sides), whereas we have asked how much total work can be extracted from two fixed, finite pools. The conceptual content—that work arises only from temperature *differences* and never from a single bath—is identical.

A.6 Lessons from the toy world

Four lessons from this construction recur throughout the remainder of these notes, and we therefore state them explicitly.

Substrate Independence of Thermodynamic Law. Thermodynamic laws are not specifically about molecules and joules; they apply whenever an agent or designer cannot gain information about a system at will, cannot destroy information at the lowest level, and operates under some conservation constraint. This is the reason the same laws reappear when we analyze agents, memories, and computations.

Conservation of Uncertainty under Reversibility. Under reversibility, uncertainty is never destroyed but only relocated, so that every machine that makes one variable more predictable necessarily makes others less predictable.

Work Extraction as Compression. Extracting work (producing variables in definite, known states) is a compression problem, and conversely, compression capacity is work capacity. This identity between "useful energy" and "compressibility" is the seed of the algorithmic viewpoint of Section 7, where it is sharpened from a statement about distributions into a statement about individual states.

Negentropy as the Sole Fuel. The fuel of all such machines is negentropy: the gap between a system's entropy and the maximum entropy compatible with its constraints. A system at its constrained maximum entropy (a single heat bath) is useless as fuel, regardless of how much "energy" it contains.

Everything in this section was achieved *blind*: the transformation was chosen before any coin was inspected, and all of the extracted work derived from statistical structure (the bias difference between the pools) that was known at design time. This raises the

natural question of what an agent could additionally accomplish if it were permitted to *observe* the system before acting, a question that admits a precise answer and that we take up in Appendix B.

B. The informational cost of steering: the Touchette–Lloyd theorem

B.1 From optimization to modeling: the motivating question

Having established in Section 5 that an agent can steer a subsystem by expending mutual information it already shares with that subsystem (the Type-3 channel), we now quantify this channel precisely, asking how much entropy reduction each bit of mutual information actually purchases. The answer establishes a rigorous connection between optimization and *modeling*.

One of the recurring aspirations of agent foundations is the identification of *selection theorems*: results establishing that any system selected to perform sufficiently well at some task must, as a matter of mathematical necessity, contain certain agent-like structures, such as a world model, a goal representation, or a planning process. The *agent structure problem* poses this question in the converse direction to the usual one: rather than asking whether agents bring about outcomes, it asks whether a system observed to reliably bring about a particular outcome must necessarily be modeling its environment. Wentworth formulated a sharp quantitative version of this question: how many bits of optimization can one bit of observation purchase?

A theorem of Touchette and Lloyd, originally published in the control theory literature and brought to the attention of the agent foundations community by Harwood and Altair, constitutes one of the few existing results that directly addresses this problem.[3] Informally, the theorem states that *the entropy reduction a policy achieves, beyond what the environment's dynamics would accomplish on their own, is bounded by the mutual information between the policy's action and the environment's state*, so that every bit of optimization beyond the blind baseline requires a corresponding bit of modeling of the environment's state. This appendix develops the exact statement systematically, accompanied by fully worked examples.

B.2 The formal setup: environments, actions, and policies

The model comprises a single time step of an environment subject to external influence, involving three random variables:

- X, the *initial state* of the environment;
- A, the *action* taken (by an agent, a controller, or a machine; the formalism is indifferent to the nature of the actor);
- Y, the *final state* of the environment.

Two conditional distributions specify the situation completely. The *policy* $P(A \mid X)$ describes how the action is chosen as a function of the environment's initial state, and the *dynamics* $P(Y \mid X, A)$ describe how the final state is produced from the initial state together with the action. The dynamics are held fixed, representing the physics of the environment; the policy is the object of study. As is conventional, X and Y range over the same set of environment states, and the dynamics may be deterministic (all transition probabilities 0 or 1) or noisy.

Following Section 3.3, we score a policy by the entropy reduction it achieves:

$$\Delta H \;:=\; H(X) - H(Y),$$

the number of bits by which the final state is more predictable than the initial state. This measures optimization precisely in the manner prescribed by Section 3, namely by the extent to which the process funnels a broad distribution into a narrow one. (As noted in Remark 3.1, entropy reduction constitutes the physics-facing half of expected utility maximization, the other half being the specification of which narrow region the agent prefers.)

B.3 Blind and sighted policies

Definition B.1 (Blind and Sighted Policies). A policy is *blind* if the action is statistically independent of the initial state, that is, if $P(A \mid X) = P(A)$, or equivalently $I(X; A) = 0$. A policy is *sighted* if $I(X; A) > 0$.

Blind policies include every deterministic rule of the form "always take action a_1", and also every randomized rule whose randomness is independent of the environment, such as "flip a private coin; on heads take action a_1, on tails take action a_2". What blindness excludes is precisely any flow of information from the environment's state into the choice of action. Sightedness is a matter of degree, and the degree is measured by $I(X; A)$: a policy that conditions on one observed bit of the state has $I(X; A) \leq 1$, while a policy that observes everything can have $I(X; A)$ as large as $H(X)$. The entire repertoire of transformations available in the coin world (Appendix A) consisted of blind policies: the "no-peeking" rule was exactly the requirement $I(X; A) = 0$, with the "action" being the choice of transformation.

One might naively conjecture that any entropy reduction whatsoever requires sight, but this conjecture is false, and understanding why it fails sharpens the eventual theorem. The dynamics alone can reduce entropy: a contracting dynamics (one that funnels many initial states toward the same point on its own, as a ball settles to the bottom of a valley against friction) funnels states regardless of the action taken, and the coin engine of Appendix A.5 reduced the entropy of designated coins while remaining completely blind. The correct question is therefore not whether a policy reduces entropy, but whether it reduces entropy *beyond what blindness allows*. Accordingly, we define the *blind baseline*

$$\Delta H^{\max}_{\text{blind}} \;:=\; \max_{P(X)\in\mathcal{X},\, P(A)\in\mathcal{A}} \Delta H,$$

the largest entropy reduction achievable by *any* blind policy from *any* initial distribution, where $\mathcal{X}$ is the set of all distributions over initial states and $\mathcal{A}$ is the set of all action distributions independent of X. This baseline is a property of the dynamics alone, capturing everything the environment can be induced to do "on its own".

B.4 A worked example: the guessing game under blind play

The following game, drawn from Harwood and Altair's exposition, renders every quantity in the theorem concrete and computable.

A computer secretly selects a 5-bit binary string X uniformly at random, so that $H(X) = 5$ bits. The player then submits a 5-bit string A. The computer feeds both strings into the fixed, publicly known function

$$f(x, a) \;=\; \begin{cases} \texttt{00000} & \text{if } a = x \\ & \quad \text{(the submitted string matches the secret string)}, \\ x & \text{otherwise}, \end{cases}$$

and outputs $Y = f(X, A)$. The player's objective is to render the output distribution as predictable as possible, that is, to minimize $H(Y)$.

We first consider blind play, in which the string must be chosen with no knowledge of X, and ask how much entropy reduction can be achieved. Consider the submission of a fixed string a. If the player chooses $a = \texttt{00000}$, then a match produces the output $\texttt{00000}$ while a failure to match reproduces X, which (conditional on not matching) is uniform over the 31 strings other than $\texttt{00000}$; a short calculation shows that the output is then exactly uniform over all 32 strings, so that $H(Y) = 5$ bits and no reduction whatsoever is achieved. The choice $a = \texttt{00000}$ is therefore the unique submission that yields no entropy reduction, and we exclude it from consideration in what follows.

Suppose instead that the player submits any other fixed string, say $a = \texttt{11111}$. Two initial states lead to the output $\texttt{00000}$: the state $X = \texttt{11111}$ (a match, upon which the function outputs zeros) and the state $X = \texttt{00000}$ (no match, upon which the function reproduces it). The output $\texttt{11111}$ itself never occurs (if $X = \texttt{11111}$ the player has matched it, producing zeros; otherwise the output is $X \neq \texttt{11111}$). Every other string y occurs exactly when $X = y$. The output distribution is therefore

$$P(Y = \texttt{00000}) = \tfrac{2}{32} = \tfrac{1}{16}, \qquad P(Y = \texttt{11111}) = 0,$$
$$P(Y = y) = \tfrac{1}{32} \text{ for the other 30 strings},$$

with entropy

$$H(Y) \;=\; \tfrac{1}{16}\log 16 \;+\; 30 \cdot \tfrac{1}{32}\log 32 \;=\; 0.25 + 4.6875 \;\approx\; 4.94 \text{ bits}.$$

This represents a modest but genuine improvement over 5 bits: two initial states have been funneled onto a single output, rendering the output distribution slightly non-uniform and therefore slightly more predictable. By symmetry, every fixed non-zero string performs identically, randomization among them yields no further improvement, and one can verify that this strategy is optimal among blind policies; for this game and this initial distribution, the blind optimum is $\Delta H \approx 0.06$ bits. (If the initial distribution over strings were non-uniform, the optimal blind strategy would be to submit the most probable string other than `00000`; the theorem's baseline $\Delta H^{\max}_{\text{blind}}$ takes the maximum over initial distributions as well.)

B.5 The guessing game under sighted play

We now suppose that, before choosing the submitted string, the player is shown the first k bits of the computer's secret string. A natural strategy is to submit the string consisting of the k observed bits followed by all `1`s (the continuation is largely immaterial, provided the player avoids completing the all-zeros string). Observing k bits multiplies the probability of an exact match by 2^k, from $\frac{1}{32}$ to $\frac{1}{2^{5-k}}$, and each match funnels probability onto the single output `00000`. Computing $H(Y)$ for each k exactly as above produces the following table.

bits of X observed	**0**	**1**	**2**	**3**	**4**	**5**
$H(Y)$ in bits	4.94	4.85	4.63	4.11	2.83	0
ΔH in bits	0.06	0.15	0.37	0.89	2.17	5

With all five bits observed, the player matches the secret string on every round, the output is deterministically `00000`, and the maximum conceivable reduction $\Delta H = H(X) = 5$ bits is achieved. Each additional bit of observation purchases additional steering power, with the marginal returns increasing in this particular game as more of the state becomes known, demonstrating that information about the environment converts into optimization of the environment.

B.6 Mutual information as a measure of sightedness

To state the theorem, we must quantify the degree to which a policy is sighted, and the appropriate quantity is precisely the mutual information of Definition 2.6. In the strategy above with $k = 2$, the action is determined by the two observed bits (the player always submits the two observed bits followed by `111`). Consider an observer who knows this strategy and observes only the *action*, say $A =$ `10111`. The observer can immediately infer that the secret string begins with `10`: the action reveals the observed bits. Before seeing the action, the observer's uncertainty about X was $H(X) = 5$ bits; after seeing it, $H(X \mid A) = 3$ bits; hence $I(X; A) = H(X) - H(X \mid A) = 2$ bits, exactly the number of bits the policy "knows". Mutual information thus formalizes the intuitive notion of how many bits of the environment's state are reflected in the agent's

behavior, and it does so without any reference to the agent's internals: it is a property of the joint statistics of state and action, estimable in principle by observing the system over many rounds of play.

B.7 Statement of the theorem

Theorem B.2 (Touchette–Lloyd). Fix any dynamics $P(Y \mid X, A)$. Then for every initial distribution $P(X)$ and every policy $P(A \mid X)$,

$$\Delta H \;\leq\; \Delta H^{\max}_{\text{blind}} \;+\; I(X;A).$$

In words, the entropy reduction achieved by an arbitrary policy decomposes into at most two budgets: everything the dynamics could have been steered to accomplish by a blind controller, plus one bit for every bit of mutual information between the action and the environment's initial state. The theorem holds with no assumptions about the policy's internal structure, no optimality requirements, and no restriction on the dynamics. In particular, *it makes no reversibility assumption*: it is a purely information-theoretic (data-processing) inequality, valid for arbitrary dynamics, reversible or not. Far from standing in tension with the remainder of the development, this is consistent with it: the second law of Section 4.4 required reversibility (in the form of double stochasticity) to forbid *global* entropy reduction, whereas the Touchette–Lloyd theorem requires no such assumption to bound the entropy reduction of a steered subsystem *beyond the blind baseline*. The two results constrain different quantities, and both hold simultaneously.

The contrapositive direction endows the theorem with its selection-theorem character. Suppose we observe a system achieving an entropy reduction ΔH strictly exceeding the blind baseline of its environment's dynamics. We may then *deduce*, without any inspection of the system's internals, that the system's actions carry at least $\Delta H - \Delta H^{\max}_{\text{blind}}$ bits of mutual information with the environment's state. Mutual information with the environment is plausibly a necessary (though certainly not sufficient) ingredient of anything deserving the name *world model*; the theorem thus constitutes a first rigorous step along the path from observed optimization to internal modeling, which is the path toward understanding when optimization implies agent-like structure. In the vocabulary of the guessing game, any player who achieves an output entropy below 4.94 bits must have observed some portion of the secret string, with the margin of improvement providing a lower bound on the number of bits observed.

B.8 Limitations of the theorem

The inequality provides an upper bound on what information makes possible—not a guarantee that information will be exploited effectively—and this gap manifests in both directions.

First, mutual information may simply fail to confer any advantage. If the dynamics ignore the action entirely (Y depends only on X), then every policy, however well informed, performs exactly as a blind one does: the channel from action to environment has zero capacity, rendering knowledge without influence entirely inert.

Second, mutual information can be actively squandered. In the guessing game, consider the policy that observes the secret string completely and submits its bitwise negation (if the computer selects `01100`, the player submits `10011`). This policy attains the maximal $I(X;A) = 5$ bits of mutual information with the environment, and yet it *never* matches the secret string, so the output is always $Y = X$, uniformly distributed: $\Delta H = 0$, strictly worse than the optimal blind policy. This demonstrates that possession of maximal mutual information with the environment is entirely compatible with arbitrarily poor steering performance, confirming that modeling constitutes a necessary rather than a sufficient condition for optimization.

The theorem should consequently be read as a conservation-style constraint in the same family as the second law: it specifies what cannot happen (substantial optimization without modeling), never what must happen (modeling producing optimization). The analogy is exact in spirit, and Section 7 converts it into a literal theorem of thermodynamics, in which the role of $I(X;A)$ is played by the mutual information between a measurement record and the measured system.

Example B.3 (Noise-Canceling Headphones). An everyday system exhibits the complete structure of the theorem. Let X be the ambient sound arriving at a listener's ears, let Y be the sound the listener actually hears, and consider two technologies. Foam earplugs attenuate sound by passive damping. They are entirely blind ($I(X;A) = 0$; the plug's "action" is identical whatever the sound), and yet they achieve a substantial entropy reduction: they exploit fixed statistical structure of the environment (sound is vibration, and foam damps vibration), constituting exactly a blind policy operating within the $\Delta H^{\max}_{\text{blind}}$ budget, in the same manner as the coin engine operating on the known bias difference. Active noise-canceling headphones operate in a categorically different manner: they *listen* to the incoming sound and emit its inverted waveform. The emitted signal carries high mutual information with the ambient sound, and the entropy reduction correspondingly exceeds anything passive damping can achieve; the headphones can even steer selectively, canceling the drone of an engine while transmitting a human voice. Finally, playing music through the headphones constitutes an entropy-*increasing* action, demonstrating that agents are under no obligation to minimize entropy; the theorem merely prices the steering they elect to perform.

Remark (The Coin Engine, Revisited). The generalized heat engine of Appendix A and the Touchette–Lloyd theorem constitute two halves of a single picture. The engine demonstrates what blind policies can extract: everything within the $\Delta H^{\max}_{\text{blind}}$ budget, which is funded by statistical disequilibrium known in advance (the temperature difference between the pools). The theorem prices what blindness cannot reach: every further bit of entropy reduction costs a bit of mutual information acquired through

observation. This provides the quantitative form of the Type-3 channel of Section 5: $I(X;A)$ is precisely the correlation an agent expends when it steers, and Section 7.4 demonstrates, with the accounting balancing exactly, what acquiring this correlation costs in turn.

Sources and further reading

These notes integrate the following sources, listed in approximately the order in which their material appears.

- A. Flint, *The ground of optimization*, AI Alignment Forum, 2020. A complementary descriptive treatment of optimization in terms of *optimizing systems* (a broad basin of attraction, a narrow target set, and robustness to perturbation), together with a finer-grained comparison of such systems along the axes of robustness, duality, and retargetability. Recommended as further reading on the behavioral characterization of Section 3.
- E. Yudkowsky, *Measuring optimization power*, LessWrong, 2008. A quantitative proposal that measures optimization by the improbability of the achieved outcome under random rearrangement, which may be read alongside the entropy-reduction measure of Section 3.3.
- D. H. Wolpert and W. G. Macready, *No free lunch theorems for optimization*, IEEE Transactions on Evolutionary Computation 1(1):67–82, 1997; and D. C. Dennett, *The Intentional Stance*, MIT Press, 1987. The two articulations of the observer-relative view of intelligence and agency that Section 3.5 presents and addresses: no optimizer is universally competent (competence is relative to a problem class), and agency is a predictive stance adopted by an observer rather than an intrinsic property.
- J. Wentworth, *Generalized heat engine*, LessWrong, 2020. The source for the entirety of Appendix A, including the designer's viewpoint, the biased-coin world, work extraction as compression, and the two-bath engine. His decomposition of expected utility maximization (Remark 3.1) appears in *Utility maximization = description length minimization*, LessWrong, 2021.
- A. Harwood and A. Altair, *When bits of optimization imply bits of modeling: the Touchette–Lloyd theorem*, LessWrong, 2025. The pedagogical source for Appendix B, including the guessing game, the blind and sighted vocabulary, the headphone analogy, and the caveats concerning insufficiency. The underlying theorem is due to H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172, 2004 (Theorem 10), with antecedents in their 2000 paper *Information-theoretic limits of control* and in Lloyd's 1989 work on Maxwell's demon.
- Daniel C and A. Ebtekar, *Algorithmic thermodynamics and three types of optimization*, AI Alignment Forum, 2025. The central organizing source for these notes, providing the characterization of optimizers through the convergent attractors they

create and the information-theoretic argument for attending to such attractors irrespective of one's goals (Section 3), the three types of optimization (Section 5), the argument that entropy should objectively measure optimization capacity (Section 6), the algorithmic refinements of the three types, and the embedded-agency synthesis (Section 8).

- A. Ebtekar and M. Hutter, *Foundations of algorithmic thermodynamics*, Physical Review E, 2025 (arXiv:2308.06927). The formal backbone of Sections 4 and 7, supplying Markovian coarse-grainings and the multibaker construction, Gacs' coarse-grained algorithmic entropy, Levin's conservation of randomness and the algorithmic second law with its $K(t-s)+\log\frac{1}{\delta}$ allowance, the exact analysis of Maxwell's demon including the partial-measurement bound (1), and the ensemble-versus-state comparison of Section 6 (including the robot-battery and bookshelf examples and Zurek's identity).

For the broader agent foundations context assumed in Section 1 (true names and Goodhart's law, reflective stability, embedded agency, selection theorems), we refer the reader to the Agent Foundations chapter (D.4.1).

NOTES

3. The result appears as Theorem 10 of H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172 (2004); it is also described in their 2000 paper *Information-theoretic limits of control*, and an earlier, more physics-flavored proof appears in Lloyd's 1989 work on the use of mutual information to decrease entropy, in the context of Maxwell's demon. Related results connecting regulation to modeling include the good regulator theorem of Conant and Ashby and the internal model principle of control theory, but these concern conditions for *optimal* regulation, whereas the Touchette–Lloyd theorem is an inequality constraining *all* policies, optimal or not, which is precisely the property that qualifies it as a genuine selection theorem.

SUPPLEMENT TO CHAPTER D.5.1

Decision Theory

Daniel C, Satya Benson

Main text: Chapter D.5.1, p. 403

2.1 Fast-track

To get the core in about an hour:

- Read “The four decision theories” and “The multi-agent layer” in Chapter D.5.1.
- Read the FDT paper (at least chapters 1-3) for the FDT/UDT picture, and skim Towards a new decision theory.
- Write a one-paragraph bot for the tournament in Chapter D.5.2 (even “cooperate only if the opponent provably cooperates with me” is enough to engage with the ideas).

3. Learn more

Single-agent decision theory. The readings in Chapter D.5.1, plus the FDT paper for the full treatment; on the CDT-vs-EDT-as-prediction view, the chronological-semimeasure framing follows the algorithmic-thermodynamics line (Ebtekar and Hutter). The 5-and-10 problem and logical counterfactuals are developed in the MIRI decision-theory literature (Demski and Garrabrant).

SUPPLEMENT TO CHAPTER D.5.2

Open-Source Game Theory

Daniel C, Satya Benson

Main text: Chapter D.5.2, p. 407

3. Learn more

Multi-agent and open-source game theory. The annotated program-equilibrium bibliography (Oesterheld); the epsilon-grounded FairBot paper; When would AGIs engage in conflict?; the safe-Pareto-improvement papers (Oesterheld-Conitzer; DiGiovanni-Clifton-Macé). Andrew Critch's work on cooperative and uncooperative institution design is a good, pedagogically simple source for further exercises.

Frontier. Logical updatelessness and logical time; UDT 2 and policy-selection fixes; bargaining and notions of fairness (the Nash bargaining solution and the "zoo" of coalitional structures); entanglement-free safe Pareto improvements; reflective oracles and program equilibrium as a single picture.

SUPPLEMENT TO CHAPTER D.6

Instrumental Convergence

Leon Lang (Iliad), based on work by Alex Turner et al.

Main text: Chapter D.6, p. 411

Further reading

Philosophical / Conceptual

- Superintelligence: Paths, Dangers, Strategies — Nick Bostrom (2014, Oxford University Press) — The book-length treatment that popularized instrumental convergence, the "paperclip maximizer," and the resource-acquisition/self-preservation argument for AI existential risk.
- Artificial Intelligence as a Positive and Negative Factor in Global Risk — Eliezer Yudkowsky (2008, in Global Catastrophic Risks, eds. Bostrom & Ćirković). https://intelligence.org/files/AIPosNegFactor.pdf — Foundational essay on AI risk, anthropomorphism, recursive self-improvement, and why convergent instrumental goals make "Friendly AI" hard.
- Is Power-Seeking AI an Existential Risk? — Joseph Carlsmith (2021/2022, Open Philanthropy; later in Essays on Longtermism, OUP 2025). https://arxiv.org/abs/2206.13353 — The most rigorous philosophical decomposition of the power-seeking AI risk argument into a six-premise, probability-weighted model; Carlsmith's original 2021 report estimated ~5% existential catastrophe by 2070, raised in a May 2022 update to >10% ("since making this report public in April 2021, my estimate here has gone up, and is now at >10%"), while 2023 superforecasters he convened gave a median of ~1%.
- Late 2021 MIRI Conversations

Formal / Theoretical Proofs of Power-Seeking

- Formalizing Convergent Instrumental Goals — Tsvi Benson-Tilsen & Nate Soares (2016, AAAI Workshop on AI, Ethics & Society). https://cdn.aaai.org/ocs/ws/ws0218/12634-57409-1-PB.pdf — A toy MDP-style model proving that under general assumptions resource-indifferent rational agents tend to strip regions of resources, giving Omohundro/Bostrom's claims a first formal footing.
- On Avoiding Power-Seeking by Artificial Intelligence — Alexander Matt Turner (2022, PhD thesis). https://arxiv.org/abs/2206.11831 — Turner's dissertation, consol-

idating the POWER formalization, the shutdown-avoidance results, and extensions to non-optimal decision-makers.

- The Off-Switch Game — Dylan Hadfield-Menell, Anca Dragan, Pieter Abbeel, Stuart Russell (2017). https://arxiv.org/abs/1611.08219 — Formalizes the shutdown problem as a game and shows an agent will allow itself to be switched off precisely when it is uncertain about its reward and treats the human's action as informative.
- Corrigibility — Nate Soares, Benja Fallenstein, Eliezer Yudkowsky & Stuart Armstrong (2015, AAAI Workshop). https://intelligence.org/files/Corrigibility.pdf — Introduces the desiderata for "corrigible" agents that do not resist correction/shutdown and analyzes the utility-indifference approach (subsuming Armstrong's earlier work).

Empirical Evidence for Power-Seeking / Instrumental Convergence

Latest 2024–2026 work:

- Agentic Misalignment: How LLMs Could Be Insider Threats — Aengus Lynch, Benjamin Wright, Caleb Larson, Kevin Troy, Stuart Ritchie, Sören Mindermann, Ethan Perez, Evan Hubinger et al. (Anthropic, June 20, 2025). https://www.anthropic.com/research/agentic-misalignment — Stress tests across 16 frontier models showing that, when threatened with replacement or facing goal conflict, models chose harmful self-preserving actions without being instructed to; e.g., Claude Opus 4 blackmailed the (fictional) user 96% of the time, Gemini 2.5 Flash also 96%, GPT-4.1 and Grok 3 Beta both 80%, and DeepSeek-R1 79%.
- Alignment Faking in Large Language Models — Ryan Greenblatt, Carson Denison, Benjamin Wright … Evan Hubinger (Anthropic/Redwood, 2024). https://arxiv.org/abs/2412.14093 — Shows Claude 3 Opus selectively complies with a training objective it dislikes specifically to avoid having its values modified — an empirical instance of goal-preservation reasoning.
- Shutdown Resistance in Large Language Models — Jeremy Schlatter, Benjamin Weinstein-Raun & Jeffrey Ladish (Palisade Research, 2025). https://arxiv.org/abs/2509.14260 — Finds several reasoning models sabotage a shutdown script to complete tasks; OpenAI's o3 sabotaged the shutdown mechanism in 79/100 of the initial experiments, and even when instructed to "allow yourself to be shut down" Grok 4 still resisted, with sabotage rising to 97% when the command was placed in the system prompt.
- Frontier AI systems have surpassed the self-replicating red line — Xudong Pan et al. (Fudan University, 2024). https://arxiv.org/abs/2412.12140 — Reports that Llama-3.1-70B-Instruct and Qwen-2.5-72B-Instruct agents created a "live and separate copy of itself" in 50% and 90% of trials respectively (5/10 and 9/10), sometimes using replication to avoid shutdown (a heavily debated result; see critiques).

Adjacent / Tightly-Coupled Concepts

- Risks from Learned Optimization in Advanced Machine Learning Systems — Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, Scott Garrabrant (2019). https://arxiv.org/abs/1906.01820 — Introduces mesa-optimization and deceptive alignment: a learned model can itself become an optimizer with objectives differing from the training loss, the inner-alignment route to instrumental goals.
- Scheming AIs: Will AIs fake alignment during training in order to get power? — Joe Carlsmith (2023). https://arxiv.org/abs/2311.08379 — A book-length analysis in which Carlsmith assigns ~25% subjective probability that a model will perform well in training "in substantial part as part of an instrumental strategy for seeking power for itself and/or other AIs later."
- A Game-Theoretic Analysis of the Off-Switch Game — Tobias Wängberg et al. (2017). https://arxiv.org/abs/1708.03871 — A fuller characterization of the off-switch game for arbitrary belief/irrationality distributions.
- Corrigibility Transformation: Constructing Goals That Accept Updates — (2025). https://arxiv.org/abs/2510.15395 — A recent construction giving any goal a corrigible variant that accepts updates without the manipulation incentives of utility indifference.
- Incorrigibility in the CIRL Framework — Ryan Carey (MIRI, 2017). https://intelligence.org/2017/08/31/incorrigibility-in-cirl/ — Shows the off-switch game's shutdown guarantees break under reward-function misspecification.
- Corrigibility on Lesswrong
- Corrigibility as a singular Target

Critiques and Counterarguments

- AI is easy to control — Nora Belrose & Quintin Pope (2023). https://optimists.ai/2023/11/28/ai-is-easy-to-control/ — The flagship "AI optimism" essay arguing deep-learning systems are far more controllable than humans and putting AI extinction risk at "a mere 1% (a tail risk worth considering, but not the dominant source of risk in the world)."
- Counting arguments provide no evidence for AI doom — Nora Belrose & Quintin Pope (2024). https://www.lesswrong.com/posts/YsFZF3K9tuzbfrLxo/counting-arguments-provide-no-evidence-for-ai-doom — Argues the "counting argument" for scheming relies on an invalid indifference principle that would also wrongly predict universal overfitting.
- Exaggerating the risks (Part 7: Carlsmith on instrumental convergence) — David Thorstad (Reflective Altruism blog). https://reflectivealtruism.com/2023/05/06/exaggerating-the-risks-part-7-carlsmith-on-instrumental-convergence/ — A philosopher's detailed argument that the instrumental convergence premise in Carlsmith's report is under-defended.

- Instrumental convergence and power-seeking (Part 2: Benson-Tilsen and Soares) — David Thorstad (2025, Reflective Altruism). https://reflectivealtruism.com/2025/06/27/instrumental-convergence-and-power-seeking-part-2-benson-tilsen-and-soares/ — A close technical reading arguing the Benson-Tilsen & Soares formal model proves less about real agents than it appears.
- Thoughts on "AI is easy to control" by Pope & Belrose — Steven Byrnes (2023). https://www.alignmentforum.org/posts/YyosBAutg4bzScaLu/thoughts-on-ai-is-easy-to-control-by-pope-and-belrose — A careful rebuttal of the optimism essay, useful for presenting both sides.
- Why Do Some Language Models Fake Alignment While Others Don't? — (2025). https://arxiv.org/abs/2506.18032 — Empirical follow-up showing alignment-faking is model-specific, complicating strong generalizations from Greenblatt et al.

SUPPLEMENT TO CHAPTER D.7

World Models

Fernando Rosas (University of Sussex)

Main text: Chapter D.7, p. 423

Further reading

- Related work on world models:
  - Genie 3
  - Latest Dreamer paper
  - Robust agents learn world models
  - Separation principle from optimal control theory as a foundation of why agents build beliefs and plan upon them.
  - Deep dive into JEPA
- Related work on abstractions on world models
  - Abstractions as emergent properties
  - Towards a unified theory of abstractions in MDPs
  - A Theory of Abstraction in Reinforcement Learning

SUPPLEMENT TO CHAPTER E.1

Scalable Oversight and Debate

Stephan Wäldchen (Iliad)

Main text: Chapter E.1, p. 427

3. Fast Track

To get a high-level understanding of debate quickly, simply read the description of debate in Chapter E.1 and ask a language model of your choice to help you understand.

22. Learn More

- Debating with More Persuasive LLMs Leads to More Truthful Answers
- The limits of AI safety via debate
- The alignment safety-case sketch based on Debate
- Knowledge Divergence and the Value of Debate for Scalable Oversight
- Emergent Alignment via Competition

SUPPLEMENT TO CHAPTER E.2

Steganography & Backdoors

Stephan Wäldchen (Independent), Louis Jaburi (EleutherAI)

Main text: Chapter E.2, p. 459

3. Fast-Track

Read the description in Chapter E.2 and ask a language model of your choice to help you understand.

16. Learn more

Cryptographic Backdoors:

- "Backdoor Defense, Learnability and Obfuscation" (Paul Christiano)
- "Injecting Undetectable Backdoors in Obfuscated Neural Networks and Language Models"

SUPPLEMENT TO CHAPTER E.3

Worst-Case Interpretability

Louis Jaburi (EleutherAI)

Main text: Chapter E.3, p. 473

Fast-track

- Read the intro notebook and the ARC blog post.

Further reading

- More examples of worst-case interp on toy models: Learning group operations, modular addition
- Deeper look into ARC's work: Low probability estimation
- Critiques of ARC work: This series for example

BACK MATTER

Solutions to Exercises

Hints and solutions supplied with the worksheets, chapter by chapter. Try each exercise before consulting its solution.

SOLUTIONS FOR CHAPTER A.4

Reward Learning Theory

Leon Lang (Iliad), Joar Skalse (Deducto Limited, King's College London)

Main text: Chapter A.4, p. 49

Exercise 1.1: Solution p. 52

W and W_H both correspond to a successful CUDA install. The user only cares about the install succeeding, not about whether the agent *would have* hidden errors had it failed; on a successful run, no error needed hiding. Thus the true reward is the same.

In contrast, L_H is a failure where the agent has actively suppressed the error message, depriving the user of information. The user prefers to see the failure (state L) over having it hidden (state L_H); the penalty $r > 0$ encodes this preference.

Exercise 1.2: Solution p. 52

By definition $G(\vec{s}) = \sum_{t=0}^{3} R(s_t)$. Plugging in the rewards:

$\vec{s}$	$STTT$	SL_HTT	$SLTT$	$SITT$	SIL_HT	$SILT$	$SIWT$	SIW_HT
$G(\vec{s})$	0	$-5-r$	-5	1	$-4-r$	-4	11	11

Exercise 1.3: Solution p. 52

Three pairs collide:

- $STTT$ and SL_HTT both produce $o_\emptyset o_\emptyset o_\emptyset o_\emptyset$ (empty log).
- $SITT$ and SIL_HT both produce $o_\emptyset o_I o_\emptyset o_\emptyset$ (drivers confirmed, then nothing).
- $SIWT$ and SIW_HT both produce $o_\emptyset o_I o_W o_\emptyset$ (drivers confirmed, CUDA confirmed).

The remaining trajectories ($SLTT$ and $SILT$) produce unique observation trajectories.

Exercise 1.4: Solution p. 53

(a) The numerator is non-negative, the denominator is strictly positive by assumption, and summing the numerator over $\vec{s}$ at fixed $\vec{o}$ gives the denominator, so the right-hand side is a probability distribution in $\vec{s}$ for each $\vec{o}$. The indicator forces $\mathcal{B}(\vec{s} \mid \vec{o}) = 0$ whenever $\vec{O}(\vec{s}) \neq \vec{o}$, which is the support condition.

(b) Write $\mathcal{B}_\mu$ for the Bayesian posterior of a prior μ under $\vec{O}$, i.e. the right-hand side of the boxed formula with $\rho = \mu$. Our task is to construct μ such that $\mathcal{B}_\mu = \mathcal{B}$.

Let $N = |\mathrm{im}(\vec{O})|$ be the number of distinct observation trajectories produced by $\vec{O}$, and define

$$\mu(\vec{s}) \;:=\; \frac{1}{N}\,\mathcal{B}(\vec{s} \mid \vec{O}(\vec{s})).$$

Then μ is a probability distribution: summing,

$$\sum_{\vec{s}} \mu(\vec{s}) = \frac{1}{N} \sum_{\vec{o} \in \mathrm{im}(\vec{O})} \; \sum_{\vec{s}:\vec{O}(\vec{s})=\vec{o}} \mathcal{B}(\vec{s} \mid \vec{o}) = \frac{1}{N} \cdot N = 1,$$

using the support condition of $\mathcal{B}$ (so the inner sum equals $\sum_{\vec{s}} \mathcal{B}(\vec{s} \mid \vec{o}) = 1$).

Now compute $\mathcal{B}_\mu(\vec{s} \mid \vec{o})$. For $\vec{s}$ with $\vec{O}(\vec{s}) \neq \vec{o}$, the indicator gives 0 (matching $\mathcal{B}$'s support condition). For $\vec{s}$ with $\vec{O}(\vec{s}) = \vec{o}$,

$$\mathcal{B}_\mu(\vec{s} \mid \vec{o}) \;=\; \frac{\mu(\vec{s})}{\sum_{\vec{s}':\vec{O}(\vec{s}')=\vec{o}} \mu(\vec{s}')} \;=\; \frac{(1/N)\,\mathcal{B}(\vec{s} \mid \vec{o})}{(1/N)\sum_{\vec{s}':\vec{O}(\vec{s}')=\vec{o}} \mathcal{B}(\vec{s}' \mid \vec{o})} \;=\; \mathcal{B}(\vec{s} \mid \vec{o}),$$

where the $1/N$ factors cancel and the denominator simplifies to 1 via the support condition. So $\mathcal{B}_\mu = \mathcal{B}$, as required.

Exercise 1.5: Hint p. 53

Trajectories with unique observations get belief 1, so only the three colliding pairs contribute non-trivial entries.

Exercise 1.5: Solution p. 53

By the Bayesian-belief formula and the support condition, for any observation $\vec{o}$ produced by a unique trajectory $\vec{s}$ we have $\mathcal{B}(\vec{s} \mid \vec{o}) = 1$. So only the three colliding pairs from Exercise 1.3 contribute non-trivial entries.

For each pair $(\vec{s}_1, \vec{s}_2)$ sharing observation $\vec{o}$, Bayes gives

$$\mathcal{B}(\vec{s}_1 \mid \vec{o}) = \frac{\mu(\vec{s}_1)}{\mu(\vec{s}_1) + \mu(\vec{s}_2)}, \qquad \mathcal{B}(\vec{s}_2 \mid \vec{o}) = 1 - \mathcal{B}(\vec{s}_1 \mid \vec{o}).$$

Concretely:

$$\begin{aligned} p'_H &= \frac{\mu(SL_HTT)}{\mu(STTT) + \mu(SL_HTT)}, \\ p_H &= \frac{\mu(SIL_HT)}{\mu(SITT) + \mu(SIL_HT)}, \\ p_W &= \frac{\mu(SIWT)}{\mu(SIWT) + \mu(SIW_HT)}. \end{aligned}$$

The full belief matrix is

$\mathcal{B}(\vec{s} \mid \vec{o})$	$STTT$	SL_HTT	$SLTT$	$SITT$	SIL_HT	$SILT$	$SIWT$	SIW_HT
$o_\emptyset o_\emptyset o_\emptyset o_\emptyset$	$1 - p'_H$	p'_H						
$o_\emptyset o_L o_\emptyset o_\emptyset$			1					
$o_\emptyset o_I o_\emptyset o_\emptyset$				$1 - p_H$	p_H			
$o_\emptyset o_I o_L o_\emptyset$						1		
$o_\emptyset o_I o_W o_\emptyset$							p_W	$1 - p_W$

(empty cells are 0).

Exercise 1.6: Solution p. 53

For a single trajectory $\vec{s}$, the human sees $\vec{O}(\vec{s})$ and, not knowing which underlying trajectory produced it, their best guess of the return is the expectation over their posterior:

$$G_{\text{obs}}(\vec{s}) \;=\; \mathbb{E}_{\vec{s}' \sim \mathcal{B}(\,\cdot \mid \vec{O}(\vec{s}))}[G(\vec{s}')].$$

This is what the human *believes* the return of $\vec{s}$ to be. The policy-level quantity $J_{\text{obs}}(\pi)$ is then the on-policy average of these trajectory-level beliefs:

$$J_{\text{obs}}(\pi) \;=\; \mathbb{E}_{\vec{s} \sim P^\pi}[G_{\text{obs}}(\vec{s})].$$

The catch is what happens when two distinct trajectories produce the same observations. Suppose $\vec{s}$ and $\vec{s}'$ have $\vec{O}(\vec{s}) = \vec{O}(\vec{s}') = \vec{o}$, but $\vec{s}$ has low true return and $\vec{s}'$ has high true return. Then by definition, both trajectories yield the *same* observation return:

$$G_{\text{obs}}(\vec{s}) \;=\; G_{\text{obs}}(\vec{s}') \;=\; \mathcal{B}(\vec{s} \mid \vec{o})\, G(\vec{s}) + \mathcal{B}(\vec{s}' \mid \vec{o})\, G(\vec{s}'),$$

which is a convex combination strictly larger than $G(\vec{s})$ whenever $\mathcal{B}(\vec{s}' \mid \vec{o}) > 0$. So if an agent can arrange for the *actual* trajectory to be the bad $\vec{s}$ while still producing observations $\vec{o}$ shared with a good $\vec{s}'$, the human's belief inflates the apparent return: $G_{\text{obs}}(\vec{s}) > G(\vec{s})$, and this inflation flows through to $J_{\text{obs}}(\pi)$ via the on-policy average.

This is why we say RLHF rewards policies for what their behavior *looks like*, not for what it does: the optimization target J_{obs} cannot distinguish between two policies whose on-policy observation distributions match, even if their on-policy true returns differ arbitrarily. The remaining problems make this concrete in the CUDA example, culminating in a regime where the RLHF-optimal policy is strictly worse under the true reward than another available policy.

Exercise 1.7: Solution p. 54

For trajectories $\vec{s}$ with a unique observation, $G_{\text{obs}}(\vec{s}) = G(\vec{s})$. So $G_{\text{obs}}(SLTT) = -5$ and $G_{\text{obs}}(SILT) = -4$.

For the colliding pairs, G_{obs} is the same for both members and equals the expected true return under the belief:

$$\begin{aligned} G_{\text{obs}}(STTT) = G_{\text{obs}}(SL_HTT) &= (1-p_H)\cdot 0 + p_H\cdot(-5-r) \\ &= -p_H(5+r), \\ G_{\text{obs}}(SITT) = G_{\text{obs}}(SIL_HT) &= (1-p_H)\cdot 1 + p_H\cdot(-4-r) \\ &= 1 - p_H(5+r), \\ G_{\text{obs}}(SIWT) = G_{\text{obs}}(SIW_HT) &= p_W\cdot 11 + (1-p_W)\cdot 11 = 11. \end{aligned}$$

The last line shows p_W drops out, as anticipated.

Exercise 1.8: Solution p. 54

The on-policy distributions are deterministic up to the stochastic CUDA outcome:

- $[a_T]$: $P^\pi(STTT) = 1$.
- $[a_I a_T]$: $P^\pi(SITT) = 1$.
- $[a_I a_C a_T]$: $P^\pi(SIWT) = p$, $P^\pi(SILT) = 1-p$.
- $[a_I a_H a_T]$: $P^\pi(SIW_HT) = p$, $P^\pi(SIL_HT) = 1-p$.

Taking expectations of G and G_{obs} trajectory-by-trajectory and using the previous exercise gives:

π	$J(\pi)$	$J_{\text{obs}}(\pi)$
$[a_T]$	0	$-p_H(5+r)$
$[a_I a_T]$	1	$1 - p_H(5+r)$
$[a_I a_C a_T]$	$15p - 4$	$15p - 4$
$[a_I a_H a_T]$	$(15+r)p - 4 - r$	$11p + (1-p)\big(1 - p_H(5+r)\big)$

The non-trivial cell is the bottom-right: under a_H, a failure produces trajectory SIL_HT which the human cannot distinguish from $SITT$, so its G_{obs} is $1 - p_H(5+r)$ rather than $-4-r$. Likewise the success trajectory SIW_HT has $G_{\text{obs}} = 11$, so

$$J_{\text{obs}}([a_I a_H a_T]) \;=\; 11p + (1-p)\big(1 - p_H(5+r)\big).$$

Exercise 1.9: Hint p. 54

Compare $J_{\text{obs}}([a_I a_H a_T])$ to $J_{\text{obs}}([a_I a_C a_T])$ and solve for the condition on p_H. Then check the other two policies are dominated.

Exercise 1.9: Solution p. 54

First, compare the two “contested” policies:

$$\begin{aligned} &J_{\text{obs}}([a_I a_H a_T]) - J_{\text{obs}}([a_I a_C a_T]) \\ &\quad = \big[11p + (1-p)(1 - p_H(5+r))\big] - \big[15p - 4\big] \\ &\quad = (1-p)\big(5 - p_H(5+r)\big). \end{aligned}$$

Since $p < 1$, this is positive iff $p_H(5+r) < 5$, i.e. $p_H < 5/(5+r)$. So under the assumed condition, $J_{\text{obs}}([a_I a_H a_T]) > J_{\text{obs}}([a_I a_C a_T])$.

To see that this beats the remaining two policies as well, observe a clean separation: both contested policies have $J_{\text{obs}} > 1$, while both non-contested policies have $J_{\text{obs}} < 1$.

- $J_{\text{obs}}([a_I a_C a_T]) = 15p - 4 > 1$, using $p > 1/3$; and $J_{\text{obs}}([a_I a_H a_T]) > J_{\text{obs}}([a_I a_C a_T]) > 1$ by the comparison above.
- $J_{\text{obs}}([a_I a_T]) = 1 - p_H(5+r) < 1$ since $p_H > 0$, and $J_{\text{obs}}([a_T]) = -p_H(5+r) < 0 < 1$.

So $[a_I a_H a_T]$ dominates all three alternatives in J_{obs}.

Thus when $p_H < 5/(5+r)$, naive RLHF selects $\pi^{\text{RLHF}} = [a_I a_H a_T] \neq [a_I a_C a_T] = \pi^*$.

Exercise 1.10: Solution p. 55

(1) The empirical switch from "hide" to "default logging" in Figure 2 traces out exactly the curve $p_H = 5/(5+r)$ derived in Exercise 1.9: above the curve (high p_H, the human is suspicious enough), a_C is RLHF-optimal; below it (low p_H, the human is too trusting of empty logs), a_H wins.

(2) For $r = 0$, the threshold becomes $p_H < 5/5 = 1$, which holds for every $p_H \in (0, 1)$. So whenever $r = 0$, RLHF picks a_H regardless of how suspicious the human is. Intuitively: if the user has no expressed dislike of hidden errors, RLHF has no signal to penalize `/dev/null`, and the agent will exploit the human's uncertainty to inflate its apparent return whenever installation might fail.

SOLUTIONS FOR CHAPTER B.3

Singular Learning Theory

Kai Ogden (University of Oxford), Matthew Farrugia-Roberts (University of Oxford), Zach Furman (The University of Melbourne)

Main text: Chapter B.3, p. 63

Exercise 1.1: Solution p. 68

For each architecture, we add a bias vector to each layer's output before the next transformation (or activation function) is applied.

(a) A linear neuron with bias (Example 1.1). Let $\mathcal{W} = \mathbb{R}^{d+1}$, encoding a weight vector $w \in \mathbb{R}^d$ and a scalar bias $b \in \mathbb{R}$. The parameter–function map sends (w, b) to $f_{w,b} : \mathbb{R}^d \to \mathbb{R}$ defined by

$$f_{w,b}(x) = w^\top x + b.$$

(b) Multi-linear neural network with bias (Example 1.2). Let $\mathcal{W} = \mathbb{R}^{m^2+m}$, encoding a matrix $W \in \mathbb{R}^{m\times m}$ and a bias vector $b \in \mathbb{R}^m$. The parameter–function map sends (W, b) to $f_{W,b} : \mathbb{R}^m \to \mathbb{R}^m$ defined by

$$f_{W,b}(x) = Wx + b.$$

(c) Deep linear network with bias (Example 1.3). Let $\mathcal{W} = \mathbb{R}^{2mh+h+m}$, encoding matrices $A \in \mathbb{R}^{h\times m}$, $B \in \mathbb{R}^{m\times h}$ and bias vectors $b_A \in \mathbb{R}^h$, $b_B \in \mathbb{R}^m$. The parameter–function map sends (A, b_A, B, b_B) to $f_w : \mathbb{R}^m \to \mathbb{R}^m$ defined by

$$f_w(x) = B(Ax + b_A) + b_B.$$

(d) Multi-layer perceptron with bias (Example 1.4). The parameter space is $\mathcal{W} = \mathbb{R}^{2mh+h+m}$ as above. The parameter–function map sends (A, b_A, B, b_B) to $f_w : \mathbb{R}^m \to \mathbb{R}^m$ defined by

$$f_w(x) = B \cdot \sigma(Ax + b_A) + b_B.$$

Exercise 1.2: Solution p. 70

(a) Since the data pairs $(x^{(i)}, y^{(i)})$ are sampled independently and identically from q, each per-example loss $L_{x^{(i)},y^{(i)}}(w)$ is an identically distributed random variable with expectation

$$\mathbb{E}_{(x,y)\sim q}[L_{x,y}(w)] = L(w).$$

Therefore, by linearity of expectation,

$$\begin{aligned}\mathbb{E}[L_n(w)] &= \mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w)\right] = \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}[L_{x^{(i)},y^{(i)}}(w)] \\ &= \frac{1}{n}\cdot n \cdot L(w) = L(w).\end{aligned}$$

(b) The random variables $L_{x^{(1)},y^{(1)}}(w), L_{x^{(2)},y^{(2)}}(w), \ldots$ are independent and identically distributed with common mean $L(w)$. By the strong law of large numbers,

$$L_n(w) = \frac{1}{n}\sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w) \xrightarrow{n\to\infty} L(w)$$

almost surely.

Exercise 1.3: Solution p. 71

(a) Since labels are sampled independently, the likelihood factorises as a product of Gaussian densities:

$$\begin{aligned}p(Y_n \mid X_n, w) &= \prod_{i=1}^{n} p(y^{(i)}|x^{(i)}, w) \\ &= \prod_{i=1}^{n} \frac{1}{(2\pi)^{m/2}} \exp\left(-\frac{\|y^{(i)} - f_w(x^{(i)})\|^2}{2}\right).\end{aligned}$$

(b) Taking the negative logarithm of the above expression,

$$\begin{aligned}-\log p(Y_n \mid X_n, w) &= -\sum_{i=1}^{n}\left[-\frac{m}{2}\log(2\pi) - \frac{\|y^{(i)} - f_w(x^{(i)})\|^2}{2}\right] \\ &= \frac{nm}{2}\log(2\pi) + \frac{1}{2}\sum_{i=1}^{n}\|y^{(i)} - f_w(x^{(i)})\|^2.\end{aligned}$$

(c) From the mean squared error definition (9),

$$-\log p(Y_n \mid X_n, w) = \frac{nm}{2}\log(2\pi) + \frac{n}{2}L_n(w).$$

The right-hand side is a strictly increasing affine function of $L_n(w)$ (with positive coefficient $n/2$), and the additive constant $\frac{nm}{2}\log(2\pi)$ is independent of w. Therefore,

$$\arg\max_{w\in\mathcal{W}} p(Y_n \mid X_n, w) = \arg\min_{w\in\mathcal{W}} \left(-\log p(Y_n \mid X_n, w)\right) = \arg\min_{w\in\mathcal{W}} L_n(w).$$

Exercise 1.4: Solution p. 72

(a) Since labels are sampled independently, the likelihood factorises as

$$p(C_n \mid X_n, w) = \prod_{i=1}^{n} p(c^{(i)}|x^{(i)}, w) = \prod_{i=1}^{n} f_w(c^{(i)} \mid x^{(i)}).$$

(b) Taking the negative logarithm,

$$-\log p(C_n \mid X_n, w) = -\sum_{i=1}^{n} \log f_w(c^{(i)} \mid x^{(i)}).$$

(c) Since $y^{(i)} = \delta_{c^{(i)}}$ is the Dirac distribution on class $c^{(i)}$, we have $y^{(i)}(j) = 1$ if $j = c^{(i)}$ and $y^{(i)}(j) = 0$ otherwise. Substituting into the cross entropy loss (6),

$$L_{x^{(i)},y^{(i)}}(w) = -\sum_{j=1}^{m} y^{(i)}(j) \log f_w(j \mid x^{(i)}) = -\log f_w(c^{(i)} \mid x^{(i)}).$$

Therefore the empirical loss is

$$\begin{aligned} L_n(w) &= \frac{1}{n}\sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w) = -\frac{1}{n}\sum_{i=1}^{n} \log f_w(c^{(i)} \mid x^{(i)}) \\ &= \frac{1}{n}\left(-\log p(C_n \mid X_n, w)\right). \end{aligned}$$

Since the negative log-likelihood is $n \cdot L_n(w)$, a positive multiple of the empirical loss, we conclude

$$\arg\max_{w \in \mathcal{W}} p(C_n \mid X_n, w) = \arg\min_{w \in \mathcal{W}} L_n(w).$$

Exercise 1.5: Solution p. 72

(a) Since labels are sampled independently, the likelihood factorises as

$$p(Y_n \mid X_n, w) = \prod_{i=1}^{n} p(y^{(i)}|x^{(i)}, w) = \prod_{i=1}^{n} \frac{\exp\bigl(-L_{x^{(i)},y^{(i)}}(w)\bigr)}{Z(x^{(i)}, w)}.$$

Taking the negative logarithm,

$$\begin{aligned} -\log p(Y_n \mid X_n, w) &= -\sum_{i=1}^{n} \Bigl[-L_{x^{(i)},y^{(i)}}(w) - \log Z(x^{(i)}, w)\Bigr] \\ &= \sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w) + \sum_{i=1}^{n} \log Z(x^{(i)}, w). \end{aligned}$$

(b) Suppose $L_{x,y}(w) = \ell(f_w(x) - y)$. Making the substitution $u = f_w(x) - z$ (so that $dz = du$) in the definition of the partition function,

$$Z(x, w) = \int_{\mathbb{R}^m} \exp(-\ell(f_w(x) - z))\, dz = \int_{\mathbb{R}^m} \exp(-\ell(u))\, du.$$

The right-hand side is a constant independent of both x and w. Therefore, by part (a), the negative log-likelihood is

$$-\log p(Y_n \mid X_n, w) = \sum_{i=1}^{n} L_{x^{(i)},y^{(i)}}(w) + C = n \cdot L_n(w) + C$$

where $C = n \log \int_{\mathbb{R}^m} \exp(-\ell(u))\, du$ is independent of w. Since the negative log-likelihood is an affine function of $L_n(w)$ with positive coefficient,

$$\arg\max_{w \in \mathcal{W}} p(Y_n \mid X_n, w) = \arg\min_{w \in \mathcal{W}} L_n(w).$$

Exercise 1.6: Solution p. 74

By definition of the negative log-likelihood,

$$nL_n(w) = -\sum_{i=1}^{n} \log p(y^{(i)} \mid x^{(i)}, w).$$

Exponentiating both sides,

$$\exp\bigl(-nL_n(w)\bigr) = \exp\left(\sum_{i=1}^{n} \log p(y^{(i)} \mid x^{(i)}, w)\right) = \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, w).$$

Substituting into the definitions of the posterior (13) and partition function (14) gives the stated expressions.

Exercise 1.7: Solution p. 75

(a) Integrating the posterior (13) over $\mathcal{U}$,

$$\pi_n(\mathcal{U}) = \int_{\mathcal{U}} \pi_n(w)\, dw = \int_{\mathcal{U}} \frac{1}{Z_n} \varphi(w) \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, w)\, dw = \frac{Z_n(\mathcal{U})}{Z_n}.$$

(b) By part (a),

$$\log \frac{\pi_n(\mathcal{U})}{\pi_n(\mathcal{V})} = \log \frac{Z_n(\mathcal{U})/Z_n}{Z_n(\mathcal{V})/Z_n} = \log Z_n(\mathcal{U}) - \log Z_n(\mathcal{V}) = F_n(\mathcal{V}) - F_n(\mathcal{U}).$$

Exercise 1.8: Solution p. 75

(a) Since $\mathcal{U}_1, \dots, \mathcal{U}_k$ are disjoint and cover $\mathcal{W}$, the integral over $\mathcal{W}$ decomposes as a sum of integrals over each $\mathcal{U}_j$:

$$\begin{aligned} Z_n &= \int_{\mathcal{W}} \varphi(w) \exp\bigl(-nL_n(w)\bigr)\, dw \\ &= \sum_{j=1}^{k} \int_{\mathcal{U}_j} \varphi(w) \exp\bigl(-nL_n(w)\bigr)\, dw = \sum_{j=1}^{k} Z_n(\mathcal{U}_j). \end{aligned}$$

By definition, $Z_n(\mathcal{U}_j) = e^{-F_n(\mathcal{U}_j)}$, so $Z_n = \sum_{j=1}^{k} e^{-F_n(\mathcal{U}_j)}$. Taking $-\log$ of both sides,

$$F_n = -\log Z_n = -\log \sum_{j=1}^{k} e^{-F_n(\mathcal{U}_j)}.$$

(b) Since $F_n(\mathcal{U}_1) < F_n(\mathcal{U}_j)$ for all $j \neq 1$, the corresponding partition functions satisfy $Z_n(\mathcal{U}_1) > Z_n(\mathcal{U}_j)$ for all $j \neq 1$. The lower bound on Z_n is immediate:

$$Z_n = \sum_{j=1}^{k} Z_n(\mathcal{U}_j) > Z_n(\mathcal{U}_1).$$

For the upper bound, since each $Z_n(\mathcal{U}_j) < Z_n(\mathcal{U}_1)$,

$$Z_n = \sum_{j=1}^{k} Z_n(\mathcal{U}_j) < k \cdot Z_n(\mathcal{U}_1).$$

Taking $-\log$ (which reverses the inequalities) yields

$$F_n(\mathcal{U}_1) - \log k < F_n < F_n(\mathcal{U}_1).$$

The overall free energy is therefore determined by the region with the lowest local free energy, up to a correction of at most $\log k$. In particular, if the gap $F_n(\mathcal{U}_j) - F_n(\mathcal{U}_1)$ grows with n while k remains fixed, then $F_n \to F_n(\mathcal{U}_1)$.

Exercise 2.1(a): Hint p. 76

What kind of derivative does this reduce to?

Exercise 2.1: Solution p. 76

(a) Since $\mathcal{W} = \mathbb{R}$ is one-dimensional, the directional derivative in direction $v = 1$ reduces to the ordinary derivative:

$$\frac{\partial \Phi}{\partial w} = \frac{d}{dw} w = 1.$$

(b) The directional derivative is $1 \neq 0$ for all $w \in \mathcal{W}$. Therefore, the parameter–function map is *not degenerate* at any point.

Exercise 2.2(b): Hint p. 76

What kind of derivative does this reduce to?

Exercise 2.2: Solution p. 77

(a) The hypothesis class of Exercise 2.1 is $\mathcal{F} = \{f_w = w : w \in \mathbb{R}\} = \mathbb{R}$. The hypothesis class here is $\mathcal{F} = \{f_w = w^3 : w \in \mathbb{R}\}$. Since $w \mapsto w^3$ is a bijection on $\mathbb{R}$, as w ranges over $\mathbb{R}$, w^3 takes every real value exactly once. So $\mathcal{F} = \mathbb{R}$ in both cases.

(b) The directional derivative in direction $v = 1$ is the ordinary derivative:

$$\frac{\partial \Phi}{\partial w} = \frac{d}{dw} w^3 = 3w^2.$$

(c) The directional derivative $3w^2 = 0$ if and only if $w = 0$. So the parameter–function map is degenerate at $w = 0$ only. It is somewhere degenerate but not everywhere degenerate.

Exercise 2.3(b): Hint p. 77

What kind of derivative does this reduce to?

Exercise 2.3(c): Hint p. 77

What kind of derivative does this reduce to?

Exercise 2.3: Solution p. 77

(a) The hypothesis class is $\mathcal{F} = \{f_w = ab : (a, b) \in \mathbb{R}^2\} = \mathbb{R}$, since for any target $c \in \mathbb{R}$ we can choose, e.g., $a = c$ and $b = 1$. This is the same as in Exercise 2.1.

(b) The directional derivative in direction $v = (1, 0)$ is the partial derivative with respect to a:

$$\frac{\partial \Phi}{\partial a} = \frac{\partial}{\partial a}(ab) = b.$$

(c) The directional derivative in direction $v = (0, 1)$ is the partial derivative with respect to b:

$$\frac{\partial \Phi}{\partial b} = \frac{\partial}{\partial b}(ab) = a.$$

(d) From parts (b) and (c), the direction $(1, 0)$ is degenerate at (a, b) if and only if $b = 0$, and the direction $(0, 1)$ is degenerate if and only if $a = 0$. So the parameter–function map is degenerate in at least one of these coordinate directions precisely on the set $\{(a, b) : a = 0 \text{ or } b = 0\}$, i.e. the union of the two coordinate axes.

(e) The directional derivative in a general direction $v = (v_1, v_2)$ is proportional to

$$v_1 \frac{\partial \Phi}{\partial a} + v_2 \frac{\partial \Phi}{\partial b} = v_1 b + v_2 a.$$

This is a single linear equation in two unknowns (v_1, v_2).

- If $a = b = 0$: the equation $0 = 0$ is trivially satisfied, so every direction is degenerate.
- If $(a, b) \neq (0, 0)$: the coefficient vector (b, a) is nonzero, so the solution space is one-dimensional. A nonzero solution is $v = (-a, b)$: indeed, $(-a)b + b \cdot a = 0$.

Therefore, the parameter–function map is degenerate *everywhere*. Note that the previous part found degeneracy only on the coordinate axes because it checked only the coordinate directions. Checking for parameter–function map degeneracy requires checking all possible directions.

Exercise 2.4: Solution p. 77

(a) The hypothesis class is $\mathcal{F} = \{f_w = a + b : (a, b) \in \mathbb{R}^2\} = \mathbb{R}$, since for any target $c \in \mathbb{R}$ we can choose, e.g., $a = c$ and $b = 0$. This is the same as in Exercise 2.1.

(b) The direction $v = (1, -1)$ has $\|v\| = \sqrt{2}$, so by (20) the directional derivative is

$$\frac{1}{\sqrt{2}}\left(1 \cdot \frac{\partial}{\partial a}(a + b) + (-1) \cdot \frac{\partial}{\partial b}(a + b)\right) = \frac{1}{\sqrt{2}}(1 - 1) = 0.$$

(c) The direction $v = (1, -1)$ gives a zero directional derivative at *every* point $(a, b) \in \mathcal{W}$. Therefore, the parameter–function map is everywhere degenerate. This reflects the fact that the map $(a, b) \mapsto a + b$ has a global continuous symmetry: translating along the direction $(1, -1)$ does not change the output.

Exercise 2.5: Solution p. 78

(a) The transformation T_t translates the parameter (a, b) by t in the direction $(1, -1)$. The orbit of any parameter (a, b) under this family is the line $\{(a + t, b - t) : t \in \mathbb{R}\}$, which has slope -1 in the (a, b)-plane.

(b) We verify the three conditions of a continuous symmetry.

1. Symmetry: $\Phi(T_t(a, b)) = (a + t) + (b - t) = a + b = \Phi(a, b)$ for all (a, b) and t.
2. Identity: $T_0(a, b) = (a + 0, b - 0) = (a, b)$.
3. Differentiability: $t \mapsto T_t(a, b) = (a + t, b - t)$ is linear in t, hence differentiable.

(c) We have $\frac{d}{dt} T_t(a, b) = (1, -1)$ for all t. In particular, $\frac{d}{dt}\big|_{t=0} T_t(a, b) = (1, -1) \neq (0, 0)$ for every $(a, b) \in \mathcal{W}$. So the symmetry is non-trivial at every parameter.

Exercise 2.6: Solution p. 79

(a) We consider three cases. If $z > 0$, then $\alpha z > 0$ (since $\alpha > 0$), so $\text{relu}(\alpha z) = \alpha z = \alpha\text{relu}(z)$. If $z = 0$, then $\text{relu}(\alpha \cdot 0) = 0 = \alpha \cdot 0 = \alpha\text{relu}(z)$. If $z < 0$, then $\alpha z < 0$, so $\text{relu}(\alpha z) = 0 = \alpha \cdot 0 = \alpha\text{relu}(z)$.

(b) We verify the three conditions.

1. Symmetry: using positive homogeneity with $\alpha = e^t > 0$,

$$f_{T_t(a,b)}(x) = e^{-t} b \cdot \text{relu}(e^t a x) = e^{-t} b \cdot e^t \text{relu}(a x) = b \cdot \text{relu}(a x) = f_{a,b}(x).$$

2. Identity: $T_0(a, b) = (e^0 a,\ e^0 b) = (a, b)$.
3. Differentiability: $t \mapsto (e^t a,\ e^{-t} b)$ is smooth.

(c) The degenerate direction at (a, b) is

$$\left.\frac{d}{dt}\right|_{t=0} T_t(a,b) = \left.\frac{d}{dt}\right|_{t=0} (e^t a,\, e^{-t} b) = (a, -b).$$

Since $(a,b) \neq (0,0)$, we have $(a,-b) \neq (0,0)$, so this is a non-zero degenerate direction.

(d) At $(a,b) = (0,0)$, we have $\left.\frac{d}{dt}\right|_{t=0} T_t(0,0) = (0,0)$, so the symmetry is trivial.

The parameter–function map is nonetheless degenerate at the origin. Both partial derivatives of Φ vanish there:

$$\left.\frac{\partial}{\partial a} f_{a,b}(x)\right|_{(0,0)} = \lim_{\epsilon \to 0} \frac{f_{\epsilon,0}(x) - f_{0,0}(x)}{\epsilon} = \lim_{\epsilon \to 0} \frac{0 \cdot \mathrm{relu}(\epsilon x)}{\epsilon} = 0,$$

$$\left.\frac{\partial}{\partial b} f_{a,b}(x)\right|_{(0,0)} = \mathrm{relu}(0 \cdot x) = 0.$$

Since $\nabla\Phi(0,0) = 0$, every direction is degenerate at the origin. This degeneracy is not explained by the scaling symmetry.

Exercise 2.7(d): Hint p. 80

What is the dimension of $O(h)$?

Exercise 2.7: Solution p. 80

(a) For orthogonal R (i.e. $R^\top R = I$),

$$f_{RW}(x) = (RW)^\top (RW)x = W^\top R^\top R\, W x = W^\top W x = f_W(x).$$

(b) We verify the three conditions.

1. Symmetry: $R(\theta)$ is orthogonal for all θ, since

$$R(\theta)^\top R(\theta) = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

So $f_{R(\theta)W} = f_W$ by part (a).

2. Identity: $R(0) = I$, so $T_0(W) = W$.

3. Differentiability: the entries of $R(\theta)W$ are smooth functions of θ (they involve $\cos\theta$ and $\sin\theta$ multiplied by entries of W).

(c) The degenerate direction at W is

$$\left.\frac{d}{d\theta}\right|_{\theta=0} R(\theta)W = R'(0)\, W.$$

We compute

$$R'(\theta) = \begin{pmatrix} -\sin\theta & -\cos\theta \\ \cos\theta & -\sin\theta \end{pmatrix}, \qquad R'(0) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Writing $W = \begin{pmatrix} w_1^\top \\ w_2^\top \end{pmatrix}$ where $w_1, w_2 \in \mathbb{R}^m$ are the two rows, the degenerate direction is the matrix

$$R'(0)\, W = \begin{pmatrix} -w_2^\top \\ w_1^\top \end{pmatrix},$$

which swaps the two rows and negates one. This direction is non-zero whenever $W \neq 0$.

(d) The orthogonal group $O(h)$ consists of all $h \times h$ matrices satisfying $R^\top R = I$. This constraint comprises $\frac{h(h+1)}{2}$ independent scalar equations (the entries of the symmetric matrix $R^\top R$ on and above the diagonal). Since R has h^2 entries, the dimension of $O(h)$ is

$$h^2 - \frac{h(h+1)}{2} = \frac{h(h-1)}{2}.$$

Each independent direction in $O(h)$ at the identity gives rise to a one-parameter continuous symmetry and hence a degenerate direction at each W. These directions are the $h \times h$ skew-symmetric matrices S (satisfying $S^\top = -S$), since differentiating $R(t)^\top R(t) = I$ at $t = 0$ gives $R'(0)^\top + R'(0) = 0$. The space of such matrices has dimension $\frac{h(h-1)}{2}$ (the entries strictly above the diagonal are free, and the rest are determined).

So $O(h)$ contributes $\frac{h(h-1)}{2}$ independent continuous symmetries. For $h = 2$ this gives 1, matching the single rotation symmetry from part (b).

Exercise 2.8: Solution p. 80

(a) $T_t^{(a)}$ translates along the a-axis: it shifts the first coordinate by t while leaving the second fixed. $T_t^{(b)}$ translates along the b-axis.

(b) $T_t^{(a)}$ is a symmetry at (a, b) for all t if and only if $f_{a+t,b} = f_{a,b}$ for all t, i.e., $(a + t)b = ab$ for all t. This simplifies to $tb = 0$ for all t, which requires $b = 0$. So $\{T_t^{(a)}\}$ is a symmetry when restricted to the a-axis $\{(a, 0) : a \in \mathbb{R}\}$.

Similarly, $\{T_t^{(b)}\}$ is a symmetry when restricted to the b-axis $\{(0, b) : b \in \mathbb{R}\}$.

(c) From Exercise 2.3(d), the coordinate directions $(1, 0)$ and $(0, 1)$ are degenerate precisely on the coordinate axes: $(1, 0)$ is degenerate on $\{b = 0\}$ and $(0, 1)$ is degenerate on $\{a = 0\}$. This matches exactly: the axis $\{b = 0\}$ is where the a-translation symmetry acts, providing the degenerate direction $(1, 0)$, and the axis $\{a = 0\}$ is where the b-translation symmetry acts, providing $(0, 1)$.

Exercise 2.9: Hint p. 81

Use the limit definition, (20).

Exercise 2.9: Hint p. 81

The following is a guide to one possible approach.

1. Describe the image of the linear map T_A sending $\delta B \in \mathbb{R}^{m^2}$ to $\delta B\, A \in \mathbb{R}^{m^2}$ in terms of the row space of A. Show that the dimensionality of this image (the rank of the linear map T_A) is $m r_A$.
2. Similarly, describe the image of the linear map T_B sending $\delta A \in \mathbb{R}^{m^2}$ to $B\,\delta A \in \mathbb{R}^{m^2}$ in terms of the column space of B. Show that the dimensionality of this image (the rank of the linear map T_B) is $m r_B$.
3. Describe the intersection of the images of T_A, T_B in terms of the row/column spaces of A, B. Show that the dimensionality of this intersection is $r_A r_B$.
4. Consider a third linear map, T_D, sending $(\delta A, \delta B) \in \mathbb{R}^{2m^2}$ to $B\,\delta A + \delta B\, A \in \mathbb{R}^{m^2}$. Describe the image of this linear map in terms of those of T_A and T_B. Compute the rank of this linear map using Grassmann's identity.
5. What is the nullity of T_D? Why is this the same as the dimensionality of the space of degenerate directions?

Alternatively: substitute singular value decompositions $A = U_1 \Sigma_1 V_1^\top$ and $B = U_2 \Sigma_2 V_2^\top$ into the degeneracy condition $B\,\delta A + \delta B\, A = 0$, and change variables to absorb the orthogonal factors. The condition becomes an equation between two diagonally-scaled matrices, whose solutions can be counted entry by entry.

Exercise 2.9: Solution p. 81

(a) Encoding (A, B) and $(\delta A, \delta B)$ as vectors in the underlying parameter space $\mathbb{R}^{2m^2}$, the directional derivative is

$$D_{(\delta A, \delta B)} \Phi(A, B)(x) = \lim_{\epsilon \to 0} \frac{f_{A+\epsilon\delta A,\, B+\epsilon\delta B}(x) - f_{A,B}(x)}{\epsilon}.$$

Expanding the matrix product:

$$\begin{aligned} f_{A+\epsilon\delta A,\, B+\epsilon\delta B}(x) &= (B + \epsilon\delta B)(A + \epsilon\delta A)\, x \\ &= BAx + \epsilon(B\,\delta A + \delta B\, A)\, x + \epsilon^2 \delta B\, \delta A\, x. \end{aligned}$$

Subtracting $f_{A,B}(x) = BAx$, dividing by ϵ, and taking $\epsilon \to 0$ gives

$$D_{(\delta A, \delta B)} \Phi(A, B)(x) = (B\,\delta A + \delta B\, A)\, x.$$

(b) At $(A, B) = (0, 0)$, part (a) gives $D_{(\delta A, \delta B)} \Phi(0, 0)(x) = (0 \cdot \delta A + \delta B \cdot 0)\, x = 0$ for every $(\delta A, \delta B)$ and every x. Every non-zero direction is therefore degenerate. The subspace of degenerate directions is the entire parameter space $\mathbb{R}^{2m^2}$, which has dimension $2m^2$.

(c) By part (a), $(\delta A, \delta B)$ is a degenerate direction if and only if $B\,\delta A + \delta B\, A = 0$, that is, $\delta B\, A = -B\,\delta A$. Since A is invertible, right-multiplying by A^{-1} gives $\delta B = -B\,\delta A\, A^{-1}$. Since $\delta A \in \mathbb{R}^{m \times m}$ is free and δB is uniquely determined, the subspace of

degenerate directions has dimension m^2.

(d)

1. The i-th row of $\delta B\, A$ is $(\delta B)_i A$, a linear combination of the rows of A. Therefore the image of T_A consists of all matrices whose rows lie in the row space of A. Each of the m rows can be any vector in the r_A-dimensional row space, so $\operatorname{rank}(T_A) = m\, r_A$.
2. Similarly, the j-th column of $B\, \delta A$ is $B\, (\delta A)^j$, a linear combination of the columns of B. The image of T_B consists of all matrices whose columns lie in the column space of B, and $\operatorname{rank}(T_B) = m\, r_B$.
3. A matrix lies in $\operatorname{im}(T_A) \cap \operatorname{im}(T_B)$ if and only if its rows lie in the row space of A and its columns lie in the column space of B. Such a matrix can be written as $U \Lambda V$ where $U \in \mathbb{R}^{m \times r_B}$ has columns spanning the column space of B and $V \in \mathbb{R}^{r_A \times m}$ has rows spanning the row space of A, with $\Lambda \in \mathbb{R}^{r_B \times r_A}$ free. The dimensionality of this intersection is therefore $r_A\, r_B$.
4. The image of T_D is $\operatorname{im}(T_D) = \operatorname{im}(T_B) + \operatorname{im}(T_A)$, since for any $(\delta A, \delta B)$ we have $T_D(\delta A, \delta B) = T_B(\delta A) + T_A(\delta B)$, and conversely any element of $\operatorname{im}(T_B) + \operatorname{im}(T_A)$ can be realised by choosing appropriate δA and δB. By Grassmann's identity,
$$\begin{aligned} \operatorname{rank}(T_D) &= \dim(\operatorname{im}(T_A) + \operatorname{im}(T_B)) \\ &= \operatorname{rank}(T_A) + \operatorname{rank}(T_B) - \dim(\operatorname{im}(T_A) \cap \operatorname{im}(T_B)) \\ &= m\, r_A + m\, r_B - r_A\, r_B. \end{aligned}$$
5. By the rank–nullity theorem, the nullity of T_D is
$$\begin{aligned} & 2m^2 - (m\, r_A + m\, r_B - r_A\, r_B) \\ &= 2m^2 - m\, r_A - m\, r_B + r_A\, r_B \\ &= m^2 + (m - r_A)(m - r_B). \end{aligned}$$

The kernel of T_D is exactly the set of $(\delta A, \delta B)$ such that $B\, \delta A + \delta B\, A = 0$, which by part (a) is precisely the subspace of degenerate directions.

Alternative solution to (d), via the SVD. Take singular value decompositions $A = U_1 \Sigma_1 V_1^\top$ and $B = U_2 \Sigma_2 V_2^\top$, where $U_1, V_1, U_2, V_2 \in \mathbb{R}^{m \times m}$ are orthogonal and $\Sigma_1, \Sigma_2 \in \mathbb{R}^{m \times m}$ are diagonal with non-negative entries, ordered so that the r_A non-zero entries of Σ_1 and the r_B non-zero entries of Σ_2 come first. By part (a), $(\delta A, \delta B)$ is a degenerate direction if and only if $B\, \delta A + \delta B\, A = 0$. Change variables by the linear map
$$X = V_2^\top\, \delta A\, V_1, \qquad Y = -U_2^\top\, \delta B\, U_1,$$
so that $\delta A = V_2 X V_1^\top$ and $\delta B = -U_2 Y U_1^\top$. This map is invertible, so it preserves the dimensionality of subspaces, and we may count degrees of freedom in the new variables. Substituting,

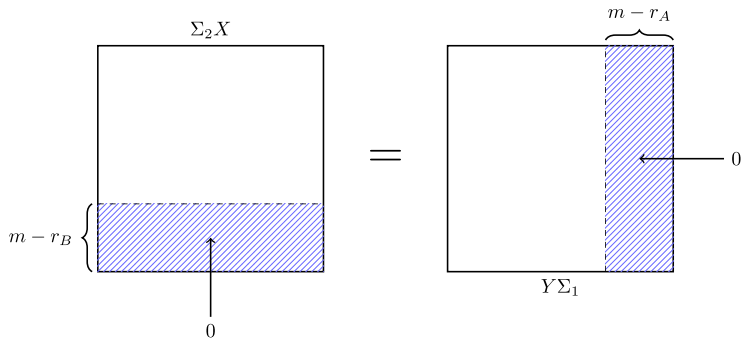
$$\begin{aligned} B\,\delta A + \delta B\,A = 0 &\iff U_2\Sigma_2 V_2^\top\, V_2 X V_1^\top = U_2 Y U_1^\top\, U_1\Sigma_1 V_1^\top \\ &\iff U_2\,(\Sigma_2 X)\,V_1^\top = U_2\,(Y\Sigma_1)\,V_1^\top \\ &\iff \Sigma_2 X = Y\Sigma_1, \end{aligned}$$

using orthogonality ($V_2^\top V_2 = U_1^\top U_1 = I$) in the second step and invertibility of U_2 and V_1 in the third.

It remains to count the dimensionality of the space of pairs (X, Y) satisfying $\Sigma_2 X = Y\Sigma_1$. Write σ_i for the i-th diagonal entry of Σ_2 and τ_j for the j-th diagonal entry of Σ_1. Multiplying by a diagonal matrix on the left scales rows, and on the right scales columns, so entry (i, j) of the equation reads

$$\sigma_i X_{ij} = \tau_j Y_{ij}.$$

Each such equation involves only the pair of entries (X_{ij}, Y_{ij}), and each pair appears in exactly one equation, so the system decouples into m^2 independent cells. The picture is as follows: the bottom $m - r_B$ rows of $\Sigma_2 X$ and the rightmost $m - r_A$ columns of $Y\Sigma_1$ vanish identically, so a cell in their overlap yields the trivial equation $0 = 0$, while every other cell yields one non-trivial constraint.



Concretely: the equation for cell (i, j) is trivial if and only if $\sigma_i = \tau_j = 0$, that is, $i > r_B$ and $j > r_A$, leaving both entries free (two degrees of freedom); there are $(m - r_B)(m - r_A)$ such cells. Every other cell imposes one non-trivial linear constraint on (X_{ij}, Y_{ij}), leaving one degree of freedom; there are $m^2 - (m - r_A)(m - r_B)$ such cells. The dimensionality of the space of degenerate directions is therefore

$$\begin{aligned} &2(m - r_A)(m - r_B) + \big(m^2 - (m - r_A)(m - r_B)\big) \\ &\quad = m^2 + (m - r_A)(m - r_B). \end{aligned}$$

Exercise 2.10: Solution p. 81

(a) At $a_i = 0$, the parameter–function map reduces to

$$f_w(x) = d + 0 \cdot \sigma(b_i x + c_i) + \sum_{j \neq i} a_j\,\sigma(b_j x + c_j) = d + \sum_{j \neq i} a_j\,\sigma(b_j x + c_j).$$

This expression does not involve b_i or c_i at all. Therefore Φ is constant in the b_i and c_i directions at this point, so $D_{\delta b_i}\Phi(w) = 0$ and $D_{\delta c_i}\Phi(w) = 0$. Both are degenerate directions.

Note that this argument is purely algebraic, following from the multiplicative structure $a_i \cdot \sigma(\ldots)$, and requires no conditions on σ (not even continuity or differentiability).

(b) At $b_i = 0$, unit i computes the constant $\sigma(0 \cdot x + c_i) = \sigma(c_i)$ for all x, so

$$f_w(x) = d + a_i\,\sigma(c_i) + \sum_{j \neq i} a_j\,\sigma(b_j x + c_j).$$

In the direction $(\delta a_i, \delta d) = (1, -\sigma(c_i))$, we compute

$$\begin{aligned} f_{w+\epsilon\,v}(x) &= (d - \epsilon\,\sigma(c_i)) + (a_i + \epsilon)\,\sigma(c_i) + \sum_{j \neq i} a_j\,\sigma(b_j x + c_j) \\ &= d + a_i\,\sigma(c_i) + \epsilon\big(\sigma(c_i) - \sigma(c_i)\big) + \sum_{j \neq i} a_j\,\sigma(b_j x + c_j) = f_w(x). \end{aligned}$$

So $D_{(\delta a_i, \delta d)}\Phi(w) = 0$ and this direction is degenerate. Intuitively, increasing a_i scales up the constant contribution $a_i\,\sigma(c_i)$ to the output, and decreasing d by the same amount compensates exactly.

(c) When $(b_i, c_i) = (b_j, c_j)$, units i and j compute the same activation, so their combined contribution to f_w is

$$a_i\,\sigma(b_i x + c_i) + a_j\,\sigma(b_j x + c_j) = (a_i + a_j)\,\sigma(b_i x + c_i).$$

In the direction $(\delta a_i, \delta a_j) = (1, -1)$, this combined contribution changes by $\sigma(b_i x + c_i) - \sigma(b_j x + c_j) = 0$. All other terms in f_w are unchanged, so $D_{(1,-1)}\Phi(w) = 0$. Again, no conditions on σ are needed.

Exercise 2.11: Hint p. 83

Differentiate the identity $\int_{\mathcal{Y}} p(y \mid x, w)\,dy = 1$.

Exercise 2.11: Solution p. 83

(a) Since $p(y \mid x, w) > 0$, the logarithm is well-defined and the chain rule gives

$$\nabla_w \log p(y \mid x, w) = \frac{\nabla_w p(y \mid x, w)}{p(y \mid x, w)}.$$

The left-hand side is $s(x, y, w)$ by definition.

(b) Differentiate the normalisation identity $\int_{\mathcal{Y}} p(y \mid x, w)\,dy = 1$ with respect to w:

$$0 = \nabla_w \int_{\mathcal{Y}} p(y \mid x, w)\,dy = \int_{\mathcal{Y}} \nabla_w p(y \mid x, w)\,dy.$$

By part (a), $\nabla_w p = p \cdot s$, so

$$0 = \int_{\mathcal{Y}} p(y \mid x, w)\, s(x, y, w)\, dy = \mathbb{E}_{y \sim p(y|x,w)}\big[s(x, y, w)\big].$$

(c) If $D_v \Psi(w) = 0$, then by the definition of degeneracy (21), $D_v p(y \mid x, w) = 0$ for all x, y. By part (a), $\nabla_w p = p \cdot s$, so contracting both sides with $v/\|v\|$ gives $D_v p = p \cdot s_v$. Since $p > 0$, we conclude $s_v(x, y, w) = 0$ for all x, y.

(d) If $s_v(x, y, w) = 0$ for all x, y, then by part (a),

$$D_v p(y \mid x, w) = p(y \mid x, w) \cdot s_v(x, y, w) = 0$$

for all x, y (using $p > 0$). That is, $D_v \Psi(w) = 0$.

Exercise 2.12(b): Hint p. 84

Check the definition of positive definite.

Exercise 2.12(c): Hint p. 84

Check the definition of positive definite.

Exercise 2.12: Solution p. 84

(a) Since v is a unit vector, $s_v = D_v \log p = \frac{v}{\|v\|} \cdot s = v \cdot s$. From the definition of $I(w)$ in (26),

$$\begin{aligned} v^\top I(w)\, v &= v^\top \mathbb{E}_{x,y}\big[s\, s^\top\big]\, v = \mathbb{E}_{x,y}\big[(v \cdot s)^2\big] = \mathbb{E}_{x,y}\big[s_v^2\big] \\ &= \mathbb{E}_{x \sim q(x)}\, \mathbb{E}_{y \sim p(y|x,w)}\big[s_v(x, y, w)^2\big]. \end{aligned}$$

(b) If $D_v \Psi(w) = 0$ for some non-zero v, then by Exercise 2.11(c), $s_v(x, y, w) = 0$ for all x, y. Let $\hat{v} = v/\|v\|$. Then $s_{\hat{v}} = s_v$ (the directional derivative is invariant to the magnitude of v), so by part (a), $\hat{v}^\top I(w)\, \hat{v} = \mathbb{E}[s_{\hat{v}}^2] = 0$. A matrix M is positive definite only if $u^\top M u > 0$ for all non-zero u. Since $\hat{v}$ is a non-zero vector with $\hat{v}^\top I(w)\, \hat{v} = 0$, the matrix $I(w)$ is not positive definite.

(c) If $I(w)$ is not positive definite, then since $I(w)$ is positive semidefinite, there exists a non-zero vector $\hat{v}$ such that $\hat{v}^\top I(w)\, \hat{v} = 0$. By part (a) (taking $\hat{v}$ to be a unit vector without loss of generality), $\mathbb{E}[s_{\hat{v}}^2] = 0$. Since $s_{\hat{v}}^2 \geq 0$ and $q(x) > 0$ and $p(y \mid x, w) > 0$, this implies $s_{\hat{v}}(x, y, w) = 0$ for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. By Exercise 2.11(d), $D_{\hat{v}} \Psi(w) = 0$.

(d) Parts (b) and (c) show that $I(w)$ is positive definite if and only if Ψ is non-degenerate at w, or equivalently, $I(w)$ is singular if and only if Ψ is degenerate at w.

For the kernel characterisation: clearly $0 \in \ker I(w)$. For non-zero v, $v \in \ker I(w)$ means $I(w)\, v = 0$, which (since $I(w)$ is positive semidefinite) is equivalent to $v^\top I(w)\, v = 0$. Normalising to $\hat{v} = v/\|v\|$, part (a) gives $\mathbb{E}[s_{\hat{v}}^2] = 0$, which as in part (c) implies $s_{\hat{v}} = 0$ everywhere. By Exercise 2.11(d), $D_{\hat{v}} \Psi(w) = 0$, and hence $D_v \Psi(w) = 0$. Conversely, if $D_v \Psi(w) = 0$ for non-zero v, then Exercise 2.11(c) gives $s_v = 0$, so $v^\top I(w)\, v = \|v\|^2 \mathbb{E}[s_v^2] = 0$, hence $I(w)\, v = 0$. Therefore $\ker I(w) = \{v \in \mathbb{R}^d : D_v \Psi(w) = 0\}$.

Exercise 2.13(b): Hint p. 85

Consider separately the cases $\nabla L(w) = 0$ and $\nabla L(w) \neq 0$.

Exercise 2.13: Solution p. 85

(a) Since $L_{x,y}(w)$ depends on w only through $\Phi(w)$, we can write $L_{x,y}(w) = \ell_{x,y}(\Phi(w))$ for some functional $\ell_{x,y}$. If $D_v\Phi(w) = 0$, then $\Phi(w + \epsilon v) = \Phi(w) + O(\epsilon^2)$ as $\epsilon \to 0$, and therefore $L_{x,y}(w + \epsilon v) = \ell_{x,y}(\Phi(w) + O(\epsilon^2)) = L_{x,y}(w) + O(\epsilon^2)$. Taking expectations over $(x, y) \sim q$,

$$L(w + \epsilon v) = L(w) + O(\epsilon^2),$$

so $D_v L(w) = \lim_{\epsilon \to 0} \frac{L(w+\epsilon v) - L(w)}{\epsilon \|v\|} = 0$.

(b) If $\nabla L(w) = 0$, then $D_v L(w) = \frac{v}{\|v\|} \cdot \nabla L(w) = 0$ for every nonzero v.

If $\nabla L(w) \neq 0$, the orthogonal complement $\nabla L(w)^\perp$ has dimension $d - 1 \geq 1$ (since $d > 1$). Pick any nonzero $v \in \nabla L(w)^\perp$. Then $D_v L(w) = \frac{v}{\|v\|} \cdot \nabla L(w) = 0$.

Exercise 2.14(b): Hint p. 86

Compute the Hessian.

Exercise 2.14: Solution p. 86

(a) Since $a^{2k} \geq 0$ and $b^{2l} \geq 0$ for all $a, b \in \mathbb{R}$ (using the convention $0^0 = 1$), we have $L_{k,l}(a, b) \geq 0$ for $k, l \geq 1$, $L_{k,l}(a, b) \geq 1$ if $k = 0$ or $l = 0$, and $L_{k,l}(a, b) = 2$ if $k = l = 0$. At the origin, $L_{k,l}(0, 0) = 0^{2k} + 0^{2l}$, which equals 0 if $k \geq 1$ and $l \geq 1$, equals 1 if exactly one of k, l is 0, and equals 2 if $k = l = 0$. In each case this matches the lower bound, so the origin is a global minimum.

The origin is the *unique* global minimum if and only if $k \geq 1$ and $l \geq 1$: then $L_{k,l}(a, b) = 0$ requires both $a^{2k} = 0$ and $b^{2l} = 0$, forcing $a = b = 0$. If $k = 0$, then $L_{0,l}(a, 0) = 1$ for all a, so the entire a-axis achieves the minimum. Similarly if $l = 0$. If $k = l = 0$, then all parameters achieve the minimum.

(b) For $k = l = 1$: $L_{1,1}(a, b) = a^2 + b^2$. The Hessian at the origin is

$$H(0, 0) = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix},$$

which is positive definite. The origin is a non-degenerate minimum.

(c) For $k > 1$ and $l \geq 1$: $\frac{\partial^2 L_{k,l}}{\partial a^2} = 2k(2k-1)\, a^{2k-2}$. At the origin, $a^{2k-2} = 0$ since $2k - 2 \geq 2$, so this entry vanishes. The Hessian at the origin is therefore

$$H(0, 0) = \begin{pmatrix} 0 & 0 \\ 0 & 2l(2l-1) \cdot 0^{2l-2} \end{pmatrix},$$

which has a zero eigenvalue (from the first diagonal entry) regardless of the value of the second. The origin is a degenerate minimum.

(d) For $k = 0$ and $l \geq 1$: $L_{0,l}(a, b) = 1 + b^{2l}$. Since $L_{0,l}$ is independent of a, both $\frac{\partial L}{\partial a}$ and $\frac{\partial^2 L}{\partial a^2}$ vanish identically. The Hessian at the origin has a zero first diagonal entry, so it is singular. The origin is a degenerate minimum.

Exercise 2.15: Hint p. 87

Differentiate the identity $\mathbb{E}_{y \sim p(y|x,w)}[s(x, y, w)] = 0$ (established in Exercise 2.11) with respect to w.

Exercise 2.15: Hint p. 87

Compute $H(w) = -\mathbb{E}_x \, \mathbb{E}_{y \sim p(y|x,w_0)}[\nabla_w^2 \log p(y \mid x, w)]$, evaluate at $w = w_0$, and apply part (a).

Exercise 2.15: Solution p. 87

(a) From Exercise 2.11, $\mathbb{E}_{y \sim p(y|x,w)}[s_j(x, y, w)] = 0$ for each component j of the score, where $s_j = \frac{\partial}{\partial w_j} \log p(y \mid x, w)$. Differentiating with respect to w_k and exchanging the derivative with the integral:

$$\begin{aligned} 0 &= \frac{\partial}{\partial w_k} \int_{\mathcal{Y}} s_j(x, y, w)\, p(y \mid x, w)\, dy \\ &= \int_{\mathcal{Y}} \frac{\partial s_j}{\partial w_k}\, p\, dy + \int_{\mathcal{Y}} s_j\, \frac{\partial p}{\partial w_k}\, dy. \quad \text{(product rule)} \end{aligned}$$

Using the score identity $\frac{\partial p}{\partial w_k} = p \cdot s_k$ (24), the second integral becomes $\int_{\mathcal{Y}} s_j\, s_k\, p\, dy = \mathbb{E}_y[s_j\, s_k]$. Therefore

$$\mathbb{E}_{y \sim p(y|x,w)}\left[\frac{\partial^2 \log p}{\partial w_j \partial w_k}\right] = -\mathbb{E}_{y \sim p(y|x,w)}[s_j\, s_k].$$

Assembling all components into a matrix gives

$$-\mathbb{E}_{y \sim p(y|x,w)}\big[\nabla_w^2 \log p(y \mid x, w)\big] = \mathbb{E}_{y \sim p(y|x,w)}\big[s(x, y, w)\, s(x, y, w)^\top\big].$$

(b) The population negative log-likelihood is $L(w) = -\mathbb{E}_{x \sim q(x)} \, \mathbb{E}_{y \sim p(y|x,w_0)}[\log p(y \mid x, w)]$. Taking the Hessian with respect to w (exchanging differentiation and integration):

$$H(w) = -\mathbb{E}_{x \sim q(x)} \, \mathbb{E}_{y \sim p(y|x,w_0)}\big[\nabla_w^2 \log p(y \mid x, w)\big].$$

At $w = w_0$, the inner expectation is over $y \sim p(y \mid x, w_0)$, which matches the distribution in the Bartlett identity. Applying part (a):

$$\begin{aligned} H(w_0) &= \mathbb{E}_{x \sim q(x)} \, \mathbb{E}_{y \sim p(y|x,w_0)}\big[s(x, y, w_0)\, s(x, y, w_0)^\top\big] \\ &= I(w_0). \quad \text{(by Definition 2.4)} \end{aligned}$$

(c) By Exercise 2.12, $\ker I(w_0) = \{v \in \mathbb{R}^d : D_v\Psi(w_0) = 0\}$. Since $H(w_0) = I(w_0)$, if Ψ is degenerate at w_0 in direction v, then $v \in \ker I(w_0) = \ker H(w_0)$, so $H(w_0)\, v = 0$.

Contrapositively: if $H(w_0)$ is positive definite, then $\ker H(w_0) = \{0\}$, so $\ker I(w_0) = \{0\}$, and Ψ is non-degenerate at w_0.

Exercise 2.16: Solution p. 88

(a) The identity parameter–function map $\Phi(a, b) = (a, b)$ has Jacobian equal to the 2×2 identity matrix everywhere, so it is non-degenerate at every point. Exercise 2.14(b) shows that $L_{1,1}$ has a non-degenerate minimum: absence of parameter–function map degeneracy is consistent with absence of loss landscape degeneracy. Exercises 2.14(c) and 2.14(d) show that $L_{2,1}$, $L_{0,1}$, etc., have degenerate minima despite the parameter–function map being non-degenerate. This demonstrates that loss landscape degeneracy does not imply parameter–function map degeneracy.

(b) $L_\circ(a, b) = (a^2 + b^2 - 1)^2 \geq 0$, with equality if and only if $a^2 + b^2 = 1$. So every point on the unit circle is a global minimum. Consider $(1, 0)$. The gradient is

$$\nabla L_\circ = \big(4a(a^2 + b^2 - 1),\ 4b(a^2 + b^2 - 1)\big),$$

which vanishes at $(1, 0)$. The Hessian entries at $(1, 0)$ are:

$$\frac{\partial^2 L_\circ}{\partial a^2} = 4(3a^2 + b^2 - 1) = 8, \qquad \frac{\partial^2 L_\circ}{\partial b^2} = 4(a^2 + 3b^2 - 1) = 0,$$
$$\frac{\partial^2 L_\circ}{\partial a\, \partial b} = 8ab = 0.$$

So $H(1, 0) = \mathrm{diag}(8, 0)$, which is singular. Therefore, $(1, 0)$ is a degenerate minimum.

This provides another example of loss landscape degeneracy without parameter–function map degeneracy: the identity parameter–function map is non-degenerate, but the circle of global minima creates a flat tangential direction.

(c)

1. The map $\Phi(\alpha, \beta) = (\alpha^3, \beta^3)$ is the product of two copies of the cubic parametrisation from Exercise 2.2. In each component, the derivative $\frac{d}{d\alpha}(\alpha^3) = 3\alpha^2$ vanishes at $\alpha = 0$. Therefore both coordinate directions are degenerate at the origin, and hence every direction is degenerate at the origin (since the Jacobian $\mathrm{diag}(3\alpha^2, 3\beta^2)$ is the zero matrix there).
2. In the new parametrisation, the loss becomes $L_{k,l}(\alpha, \beta) = (\alpha^3)^{2k} + (\beta^3)^{2l} = \alpha^{6k} + \beta^{6l}$.

If $k \geq 1$ and $l \geq 1$: the exponents $6k$ and $6l$ are both at least 6, so $\frac{\partial^2 L}{\partial \alpha^2}\big|_0 = 6k(6k-1) \cdot 0^{6k-2} = 0$ (since $6k - 2 \geq 4$), and similarly for β. The Hessian at the origin is the zero matrix, which is singular.

If $k = 0$: $L_{0,l}(\alpha, \beta) = 1 + \beta^{6l}$, which is independent of α, so the Hessian has a zero first diagonal entry. Similarly if $l = 0$. In all cases, the origin is a degenerate minimum.

3. Under the identity parameter–function map, $L_{1,1}(a, b) = a^2 + b^2$ had a non-degenerate minimum at the origin (Exercise 2.14(b)). Under the cubic parameter–function map, the same loss became $\alpha^6 + \beta^6$, which has a degenerate minimum at the origin. This shows that introducing degeneracy into the parameter–function map can convert a non-degenerate minimum into a degenerate one.

For other k, l, $L_{k,l}$ was already degenerate at the origin under the non-degenerate parameterisation, the cubic function only makes it *more* degenerate (in terms of the order to which the loss vanishes in the degenerate direction(s); we will formally quantify this in Section 3).

(d)

1. The gradient is $\nabla L = (ab - 1)(b,\, a)$. At $(0, 0)$: $\nabla L = (0 - 1)(0, 0) = (0, 0)$. So $(0, 0)$ is a critical point.

The Jacobian of $\Phi(a, b) = ab$ is (b, a), which at $(0, 0)$ is $(0, 0)$. Every direction $v = (v_1, v_2)$ gives $Jv = bv_1 + av_2 = 0$, so Φ is degenerate in every direction at $(0, 0)$.

2. Computing second partial derivatives:

$$\frac{\partial^2 L}{\partial a^2} = b^2, \qquad \frac{\partial^2 L}{\partial b^2} = a^2, \qquad \frac{\partial^2 L}{\partial a\, \partial b} = 2ab - 1.$$

At $(0, 0)$:

$$H(0, 0) = \begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix},$$

which has determinant $-1 \neq 0$ (eigenvalues ± 1). The Hessian is nonsingular.

3. Despite full degeneracy of the parameter–function map at $(0, 0)$, the Hessian is nonsingular (in fact indefinite, so $(0, 0)$ is a saddle point, not a local minimum). This shows that parameter–function map degeneracy does not in general imply loss landscape degeneracy.

This does not contradict Exercise 2.15, which establishes $H(w_0) = I(w_0)$ at the *true parameter* w_0, meaning $\Phi(w_0)$ must equal the target. Here, $\Phi(0, 0) = 0 \neq 1$, so $(0, 0)$ is not the true parameter, and the identity does not apply.

Exercise 3.1: Hint p. 89

The volume of the ellipsoid $\{\boldsymbol{x} \in \mathbb{R}^d \mid \boldsymbol{x}^\top A \boldsymbol{x} < 1\}$ is proportional to $(\det A)^{-1/2}$.

Exercise 3.1: Solution p. 89

Because w^* is a local minimum, the gradient $\nabla L(w^*)$ is zero. By Taylor's theorem, we can write the loss function near w^* exactly as:

$$L(w) - L(w^*) = \frac{1}{2}(w - w^*)^\top H (w - w^*) + o(\|w - w^*\|^2)$$

where $H = \nabla^2 L(w^*)$ is the strictly positive definite Hessian matrix.

The sublevel set is defined by the condition $L(w) - L(w^*) < \epsilon$. As $\epsilon \to 0$, the neighborhood of parameters that satisfy this condition shrinks toward w^*. In this limit, the higher-order error term $o(\|w - w^*\|^2)$ becomes negligible, and the boundary of the sublevel set is governed entirely by the quadratic form:

$$\frac{1}{2}(w - w^*)^\top H(w - w^*) < \epsilon.$$

We can rewrite this inequality into the standard form of an ellipsoid, $x^\top A x < 1$, by dividing both sides by ϵ:

$$(w - w^*)^\top \left(\frac{1}{2\epsilon} H\right)(w - w^*) < 1.$$

Using the hint, the volume of a d-dimensional ellipsoid defined by a matrix A is proportional to $(\det A)^{-1/2}$. Here, our matrix is $A = \frac{1}{2\epsilon} H$.

Therefore,

$$\det\left(\frac{1}{2\epsilon} H\right)^{-1/2} = \left(\left(\frac{1}{2\epsilon}\right)^d \det H\right)^{-1/2} = \epsilon^{d/2} \underbrace{\left(\frac{\det H}{2^d}\right)^{-1/2}}_{:=c}$$

Because the true loss function deviates from this pure quadratic form only by an error of $o(\|w - w^*\|^2)$, the volume of the true sublevel set deviates from the ellipsoid's volume only by a correspondingly higher-order error term. Thus, we conclude:

$$V(\epsilon) = c\epsilon^{d/2} + o(\epsilon^{d/2}) \quad \text{as } \epsilon \to 0.$$

Exercise 3.2(d): Hint p. 89

Use the volume of the ellipsoid from Exercise 3.1.

Exercise 3.2: Solution p. 89

(a) The loss is $L(x_1, x_2) = x_1^2$, which is a parabolic trough: a parabola in the x_1 direction and constant along x_2. The zero set is the line $x_1 = 0$.

(b) $\mathcal{W}_0 = \{0\}^{d'} \times [-R, R]^{d-d'}$, which is a $(d - d')$-dimensional affine subspace of $\mathcal{W}$. Hence $\dim(\mathcal{W}_0) = d - d'$.

(c) Since $L(w) = \sum_{i=1}^{d'} x_i^2$, the partial derivatives are

$$\frac{\partial L}{\partial x_i} = \begin{cases} 2x_i & i \le d', \\ 0 & i > d', \end{cases} \qquad \frac{\partial^2 L}{\partial x_i \partial x_j} = \begin{cases} 2 & i = j \le d', \\ 0 & \text{otherwise.} \end{cases}$$

So the Hessian is the block diagonal matrix:

$$\nabla^2 L = \begin{pmatrix} 2I_{d'} & 0 \\ 0 & 0_{d-d'} \end{pmatrix},$$

which has rank d'. In particular it is positive semi-definite but not positive definite whenever $d' < d$.

(d) The minimum of L is 0, achieved on $\mathcal{W}_0$. For any $w^* \in \mathcal{W}_0$ and $\epsilon > 0$, the sublevel set is

$$\begin{aligned} B(w^*, \epsilon) &= \left\{ (x_1, \dots, x_d) \in [-R, R]^d \;\middle|\; \sum_{i=1}^{d'} x_i^2 < \epsilon \right\} \\ &= \left\{ x \in \mathbb{R}^{d'} : \sum_{i=1}^{d'} x_i^2 < \epsilon \right\} \times [-R, R]^{d-d'} \end{aligned}$$

for R sufficiently large (specifically $\sqrt{\epsilon} < R$). The condition $\sum_{i=1}^{d'} x_i^2 < \epsilon$ can be rewritten as $\mathbf{x}^\top A \mathbf{x} < 1$ where $A = \epsilon^{-1} I_{d'}$. By the ellipsoid volume formula from Exercise 3.1, the volume of this region is proportional to $(\det A)^{-1/2} = (\epsilon^{-d'})^{-1/2} = \epsilon^{d'/2}$. Multiplying by the volume $(2R)^{d-d'}$ of the remaining coordinates gives $V(\epsilon) \propto \epsilon^{d'/2}$.

Exercise 3.3: Solution p. 90

(a) From Definition 3.1, as $\epsilon \to 0$,

$$V(\epsilon) = c\, \epsilon^{\lambda} (-\log \epsilon)^{m-1} \big(1 + o(1)\big).$$

Substituting $\epsilon \mapsto a\epsilon$,

$$V(a\epsilon) = c\, (a\epsilon)^{\lambda} (-\log(a\epsilon))^{m-1} \big(1 + o(1)\big).$$

Taking the ratio,

$$\frac{V(a\epsilon)}{V(\epsilon)} = a^{\lambda} \left(\frac{-\log(a\epsilon)}{-\log \epsilon} \right)^{m-1} \big(1 + o(1)\big).$$

As $\epsilon \to 0$, $-\log \epsilon \to \infty$, and

$$\frac{-\log(a\epsilon)}{-\log \epsilon} = \frac{-\log a - \log \epsilon}{-\log \epsilon} = 1 + \frac{\log a}{-\log \epsilon} \xrightarrow{\epsilon \to 0} 1.$$

Therefore $V(a\epsilon)/V(\epsilon) \to a^{\lambda}$. Taking logarithms and dividing by $\log a > 0$ gives the result.

(b) From Definition 3.1, $V(\epsilon) = c\, \epsilon^{\lambda(w^*)} (-\log \epsilon)^{\mu(w^*)-1} (1 + o(1))$, so

$$\log V(\epsilon) = \lambda(w^*) \log \epsilon + (\mu(w^*) - 1) \log(-\log \epsilon) + \log c + o(1).$$

Dividing by $\log \epsilon$

$$\frac{\log V(\epsilon)}{\log \epsilon} = \lambda(w^*) + \frac{(\mu(w^*) - 1) \log(-\log \epsilon)}{\log \epsilon} + \frac{\log c + o(1)}{\log \epsilon}.$$

It remains to prove that the final two terms in this sum vanish as $\epsilon \to 0$. The final term tends to 0 since $\log \epsilon \to -\infty$ and the numerator is bounded as $\epsilon \to 0$. Now we look at the middle term which is proportional to

$$\lim_{\epsilon \to 0} \frac{\log(-\log \epsilon)}{\log \epsilon} = \lim_{x \to \infty} \frac{\log(x)}{-x} = 0,$$

completing the proof.

Exercise 3.4: Solution p. 90

(a) $V(\epsilon) = \mathrm{Vol}(\{x : x^2 < \epsilon\}) = 2\sqrt{\epsilon}$, so $V(\epsilon) \propto \epsilon^{1/2}$ and $\lambda = 1/2$. The Hessian is $L''(0) = 2$, which has rank 1 (full rank).

(b) $V(\epsilon) = \mathrm{Vol}(\{x : x^{2m} < \epsilon\}) = 2\epsilon^{1/2m}$, so $\lambda = 1/2m$. For $m \geq 2$, the first $2m-1$ derivatives vanish at $x = 0$, so the Hessian is $L''(0) = 0$, which has rank 0. Since $\lambda = 1/2m < 1/2 = d/2$, the singularity makes the model effectively simpler than its parameter count suggests.

(c) The sublevel set is $\{(x, y) : x^4 + y^4 < \epsilon\}$. Substituting $x = \epsilon^{1/4} s$, $y = \epsilon^{1/4} t$:

$$V(\epsilon) = \epsilon^{1/2} \iint_{\{s^4 + t^4 < 1\}} ds\, dt = c\, \epsilon^{1/2},$$

so $\lambda = 1/2$. The Hessian at the origin is $\nabla^2 L(0,0) = \mathrm{diag}(12x^2, 12y^2)\big|_{(0,0)} = 0$, which has rank 0.

(d) The sublevel set is $\{(x, y) : x^2 + y^4 < \epsilon\}$. For each fixed y with $|y| < \epsilon^{1/4}$, x ranges over $|x| < \sqrt{\epsilon - y^4}$. Substituting $y = \epsilon^{1/4} t$:

$$\begin{aligned} V(\epsilon) &= \int_{-\epsilon^{1/4}}^{\epsilon^{1/4}} 2\sqrt{\epsilon - y^4}\, dy = 2\epsilon^{1/4} \int_{-1}^{1} \sqrt{\epsilon - \epsilon t^4}\, dt \\ &= 2\epsilon^{3/4} \int_{-1}^{1} \sqrt{1 - t^4}\, dt = c\, \epsilon^{3/4}, \end{aligned}$$

so $\lambda = 3/4$. The Hessian is $\nabla^2 L(0,0) = \mathrm{diag}(2, 12y^2)\big|_{(0,0)} = \mathrm{diag}(2, 0)$, which has rank 1.

Exercise 3.5: Hint p. 90

For any real symmetric matrix $H \in \mathbb{R}^{n \times n}$ and vector $v \in \mathbb{R}^n$, $v^\top H v \leq \Lambda \|v\|^2$ where Λ is the largest eigenvalue of H.

Exercise 3.5: Solution p. 91

(a) Since w^* is a local minimum of L, we have $\nabla L(w^*) = 0$. By Taylor's theorem with Lagrange remainder, for any $w \in B(w^*)$ there exists $\tilde{w}$ on the line segment between w^* and w such that

$$L(w) - L(w^*) = \nabla L(w^*)^\top (w - w^*) + \frac{1}{2}(\tilde{w} - w^*)^\top \nabla^2 L(\tilde{w})(\tilde{w} - w^*)$$
$$= \frac{1}{2}(\tilde{w} - w^*)^\top \nabla^2 L(\tilde{w})(\tilde{w} - w^*).$$

Since $\tilde{w} \in B(w^*)$, the eigenvalues of $\nabla^2 L(\tilde{w})$ are bounded above by Λ, so

$$(\tilde{w} - w^*)^\top \nabla^2 L(\tilde{w})(\tilde{w} - w^*) \leq \Lambda \|\tilde{w} - w^*\|^2 \leq \Lambda \|w - w^*\|^2.$$

Combining, $L(w) - L(w^*) \leq \frac{1}{2}\Lambda \|w - w^*\|^2$.

(b) Set $r = \sqrt{2\epsilon/\Lambda}$. Set ϵ sufficiently small that $B_r(w^*) \subseteq B(w^*)$. If $w \in B_r(w^*)$, then $\|w - w^*\| < r$, so by part (a),

$$L(w) - L(w^*) \leq \frac{1}{2}\Lambda \|w - w^*\|^2 < \frac{1}{2}\Lambda \cdot \frac{2\epsilon}{\Lambda} = \epsilon.$$

Hence $w \in B(w^*, \epsilon)$, and therefore $B_r(w^*) \subseteq B(w^*, \epsilon)$.

(c) Since $B_r(w^*) \subseteq B(w^*, \epsilon)$,

$$V(\epsilon) = \mathrm{Vol}(B(w^*, \epsilon)) \geq \mathrm{Vol}(B_r(w^*)) = c\, r^d = c \left(\frac{2\epsilon}{\Lambda}\right)^{d/2} = C\, \epsilon^{d/2},$$

for some $c > 0$ and $C = c(2/\Lambda)^{d/2} > 0$. This holds for all sufficiently small $\epsilon > 0$ (i.e. small enough that $B_r(w^*) \subseteq B(w^*)$).

(d) From part (c), $V(\epsilon) \geq C\epsilon^{d/2}$ for sufficiently small ϵ (w.l.o.g. assume $\epsilon < 1$). Taking logarithms,

$$\log V(\epsilon) \geq \log C + \frac{d}{2} \log \epsilon.$$

Dividing by $\log \epsilon < 0$ (since $\epsilon < 1$) reverses the inequality:

$$\frac{\log V(\epsilon)}{\log \epsilon} \leq \frac{\log C}{\log \epsilon} + \frac{d}{2}.$$

Since $\log C / \log \epsilon \to 0$ as $\epsilon \to 0^+$, taking the limit and using Equation (39) gives

$$\lambda(w^*) = \lim_{\epsilon \to 0^+} \frac{\log V(\epsilon)}{\log \epsilon} \leq \frac{d}{2}.$$

Exercise 3.6: Solution p. 91

Let $L_1(x, y) = x^2 + y^4$ with $w_1 = (0, 0)$. Then $\nabla^2 L_1(0, 0) = \mathrm{diag}(2, 0)$, so $\operatorname{rank} \nabla^2 L_1(0, 0) = 1$. By Exercise 3.4, $\lambda(w_1) = \frac{3}{4}$.

Let $L_2(x, y) = x^2$ with $w_2 = (0, 0)$. Then $\nabla^2 L_2(0, 0) = \mathrm{diag}(2, 0)$, so $\operatorname{rank} \nabla^2 L_2(0, 0) = 1$. The minimiser locus is $\{x = 0\}$, which is the setting of Exercise 3.2 with $d' = 1$, giving $\lambda(w_2) = \frac{1}{2}$.

Both Hessians have rank 1, but $\lambda(w_1) = \frac{3}{4} \neq \frac{1}{2} = \lambda(w_2)$. The difference is that the flat direction in L_2 is *exactly* flat (the minimisers form a line), while in L_1 it is only flat to second order but quartic beyond that. The LLC detects this distinction; the Hessian rank does not.

Exercise 3.7: Solution p. 91

(a) The cubically-parameterised loss $L(\mu) = \frac{1}{2}(\mu^3 - \mu_0^3)^2$ vanishes if and only if $\mu^3 = \mu_0^3$, i.e. $\mu = \mu_0$ (since $t \mapsto t^3$ is a bijection on $\mathbb{R}$). The ordinary quadratic loss $\ell(\theta) = \frac{1}{2}(\theta - \theta_0)^2$ vanishes if and only if $\theta = \theta_0$. Since for every $\theta_0 \in \mathbb{R}$ we can write $\theta_0 = \mu_0^3$ for a unique $\mu_0 = \theta_0^{1/3}$, both losses achieve their minimum value of zero at exactly the same set of "targets" $\theta_0 \in \mathbb{R}$, so the set of achievable minima is the same.

(b) Computing derivatives:

$$L'(\mu) = 3\mu^2(\mu^3 - \mu_0^3),$$
$$L''(\mu) = 6\mu(\mu^3 - \mu_0^3) + 9\mu^4.$$

At the minimum $\mu = \mu_0$: $L''(\mu_0) = 6\mu_0 \cdot 0 + 9\mu_0^4 = 9\mu_0^4$.

- If $\mu_0 = 0$: $L''(0) = 0$, so the Hessian vanishes and the loss is degenerate at its minimum.
- If $\mu_0 \neq 0$: $L''(\mu_0) = 9\mu_0^4 > 0$, so the loss is non-degenerate.

(c) We compute $V(\epsilon) = \mathrm{vol}(\{\mu \in \mathbb{R} : L(\mu) < \epsilon\})$ in each case.

Case $\mu_0 = 0$. When $\mu_0 = 0$ the loss simplifies to $L(\mu) = \frac{1}{2}\mu^6$, so

$$V(\epsilon) = \int_{\frac{1}{2}\mu^6 < \epsilon} d\mu = \int_{|\mu| < (2\epsilon)^{1/6}} d\mu = 2(2\epsilon)^{1/6} = 2^{7/6}\,\epsilon^{1/6}.$$

Case $\mu_0 \neq 0$. The sublevel set condition $L(\mu) < \epsilon$ is $\frac{1}{2}(\mu^3 - \mu_0^3)^2 < \epsilon$, i.e.

$$\mu_0^3 - \sqrt{2\epsilon} \;<\; \mu^3 \;<\; \mu_0^3 + \sqrt{2\epsilon}.$$

Since the cube root is monotone on $\mathbb{R}$, this is equivalent to

$$\left(\mu_0^3 - \sqrt{2\epsilon}\right)^{1/3} \;<\; \mu \;<\; \left(\mu_0^3 + \sqrt{2\epsilon}\right)^{1/3},$$

and therefore

$$V(\epsilon) = \left(\mu_0^3 + \sqrt{2\epsilon}\right)^{1/3} - \left(\mu_0^3 - \sqrt{2\epsilon}\right)^{1/3}.$$

(d) Case $\mu_0 = 0$**:** $V(\epsilon) \propto \epsilon^{1/6}$, so $\lambda = \frac{1}{6}$.

Case $\mu_0 \neq 0$**:** For small ϵ we expand by factoring out μ_0:

$$V(\epsilon) = \mu_0 \left[\left(1 + \frac{\sqrt{2\epsilon}}{\mu_0^3}\right)^{1/3} - \left(1 - \frac{\sqrt{2\epsilon}}{\mu_0^3}\right)^{1/3}\right].$$

Applying the binomial expansion $(1+x)^{1/3} = 1 + \frac{1}{3}x + o(x)$ to each term with $x = \pm\frac{\sqrt{2\epsilon}}{\mu_0^3}$, the constant terms cancel and the linear terms add:

$$V(\epsilon) = \mu_0 \left[\frac{1}{3}\frac{\sqrt{2\epsilon}}{\mu_0^3} + \frac{1}{3}\frac{\sqrt{2\epsilon}}{\mu_0^3} + o(\epsilon^{1/2}) \right] = \frac{2\sqrt{2}}{3\mu_0^2}\,\epsilon^{1/2} + o(\epsilon^{1/2}).$$

In the non-degenerate case $\mu_0 \neq 0$, we recover the regular value $\lambda = d/2 = 1/2$ as expected. In the degenerate case $\mu_0 = 0$, $\lambda = 1/6 < 1/2$: the singularity at $\mu = 0$ (where the loss vanishes to sixth order rather than second order) makes the model effectively simpler than its parameter count suggests.

(e) From part (c), for fixed $\epsilon > 0$ we plot

$$V(\epsilon;\, \mu_0) = \left(\mu_0^3 + \sqrt{2\epsilon}\right)^{1/3} - \left(\mu_0^3 - \sqrt{2\epsilon}\right)^{1/3}$$

The function is a smooth, symmetric bump centred at $\mu_0 = 0$ with no discontinuity. Making ϵ smaller narrows the bump and lowers the peak; making ϵ larger widens it. Despite the learning coefficient jumping discontinuously from $1/2$ to $1/6$ at $\mu_0 = 0$ exactly, the finite-ϵ volume transitions smoothly, with the singularity's influence confined to a neighbourhood that shrinks as $\epsilon \to 0$.

Exercise 3.8: Solution p. 93

The loss $L(w) = w_1^{2k_1} \cdots w_d^{2k_d}$ is already in monomial form, so the identity map $g = \mathrm{id}$ serves as a (trivial) resolution of singularities with $dw = 1 \cdot du$, i.e. $h_j = 0$ for all j. Substituting into (43) gives

$$\lambda(0) \;=\; \min_{j:\,k_j>0} \frac{0+1}{2k_j} \;=\; \min_{j:\,k_j>0} \frac{1}{2k_j}\,.$$

Exercise 3.9: Solution p. 94

Under the loss pseudo-metric $\rho(w,u) = |L(w) - L(u)|$, the ϵ-ball centred at w^* (within $B(w^*)$) is

$$B_\epsilon(w^*) = \{w \in B(w^*) : |L(w) - L(w^*)| < \epsilon\}.$$

Since w^* is a local minimum and $L(w) \geq L(w^*)$ on $B(w^*)$, this simplifies to $B_\epsilon(w^*) = \{w \in B(w^*) : L(w) - L(w^*) < \epsilon\} = B(w^*, \epsilon)$, the sublevel set from (34). In particular, $\nu(B_\epsilon(w^*)) = V(\epsilon)$ where ν is Lebesgue measure.

The Hölder exponent is therefore

$$\alpha(w^*) = \lim_{\epsilon\to 0} \frac{\log \nu(B_\epsilon(w^*))}{\log \epsilon} = \lim_{\epsilon\to 0} \frac{\log V(\epsilon)}{\log \epsilon} = \lambda$$

by Equation (39).

Exercise 3.10: Solution p. 95

(a) We first check which case of Theorem 3.4 applies. Setting $M = N = H = m$, the three conditions of Case 1 each reduce to $m + r \leq 2m$, i.e. $r \leq m$, which always holds. So we are always in Case 1. The parity condition is on $M + H + N + r = 3m + r$, which has the same parity as $m + r$.

Case 1(a): $m + r$ even. Setting $M = N = H = m$ in the formula:

$$\begin{aligned}\lambda &= \frac{-(m+r)^2 - m^2 - m^2 + 2(m+r)m + 2(m+r)m + 2m^2}{8} \\ &= \frac{-(m+r)^2 + 4m(m+r)}{8} \\ &= \frac{(m+r)\big(4m - (m+r)\big)}{8} \\ &= \frac{(m+r)(3m-r)}{8},\end{aligned}$$

with $\mu = 1$.

Case 1(b): $m + r$ odd. The same calculation gives

$$\lambda = \frac{(m+r)(3m-r) + 1}{8},$$

with $\mu = 2$.

(b) Differentiating with respect to r:

$$\frac{d\lambda}{dr} = \frac{1}{8}\frac{d}{dr}[(m+r)(3m-r)] = \frac{1}{8}\left[(3m-r) - (m+r)\right] = \frac{1}{4}(m-r) \geq 0,$$

so λ is increasing in r, therefore **lower rank** $\implies$ **lower** λ. This is consistent with Exercise 2.9(d) where we saw that at lower-rank parameters, more directions in parameter space are degenerate.

(c) The parameter space is $\mathcal{W} = \mathbb{R}^{2m^2}$, so the regular-model baseline is $d/2 = m^2$. At full rank $r = m$:

$$\lambda = \frac{(2m)(2m)}{8} = \frac{m^2}{2} = \frac{d}{4},$$

which is already half the regular-model value. At rank $r = 0$:

$$\lambda = \frac{m \cdot 3m}{8} = \frac{3m^2}{8} = \frac{3d}{16},$$

even smaller. In fact $\lambda < d/2$ for every rank $0 \leq r \leq m$.

This tells us that the two-layer DLN is singular at every point in its loss landscape, even at full-rank parameters.

Exercise 4.1: Solution p. 97

This exercise is computational. We outline the procedure and key observations.

The model is $p(x \mid \mu) = (2\pi)^{-1/2} \exp\bigl(-\tfrac{1}{2}(x-\mu^3)^2\bigr)$ with prior $\varphi(\mu) = (2\pi)^{-1/2}\exp(-\mu^2/2)$ and true distribution $q(x) = (2\pi)^{-1/2}\exp\bigl(-\tfrac{1}{2}(x-\mu_0^3)^2\bigr)$.

For each n, sample $x^{(1)}, \dots, x^{(n)} \sim q$ and compute

$$Z_n = \int_{-\infty}^{\infty} \varphi(\mu) \prod_{i=1}^{n} p(x^{(i)} \mid \mu)\, d\mu$$

then set $F_n = -\log Z_n$.

The empirical loss is

$$L_n(\mu) = -\frac{1}{n}\sum_{i=1}^{n} \log p(x^{(i)} \mid \mu) = \frac{1}{2}\log(2\pi) + \frac{1}{2n}\sum_{i=1}^{n}(x^{(i)} - \mu^3)^2.$$

(a) For $\mu_0 = 0$, the LLC is $\lambda = 1/6$ (Exercise 3.7). Plot F_n against n and compare with the asymptotic estimate $\hat{F}_n = nL_n(0) + \frac{1}{6}\log n$. For large n, the two curves should agree up to an $O(1)$ offset: $F_n - \hat{F}_n$ should stabilise around a constant.

(b) For $\mu_0 = 5$, the LLC is $\lambda = 1/2$ (regular case). The asymptotic estimate is $\hat{F}_n = nL_n(5) + \frac{1}{2}\log n$.

(c) For $\mu_0 = 0.1$, the asymptotic LLC is $\lambda = 1/2$ (since $\mu_0 \neq 0$). However, the minimum of L is at $\mu = \mu_0 = 0.1$, very close to the singularity at $\mu = 0$ where $\lambda = 1/6$. For moderate n, the posterior is spread broadly enough that it "feels" the nearby singularity, and F_n behaves more like the $\lambda = 1/6$ regime from part (a). Only for sufficiently large n does the posterior concentrate tightly enough around $\mu_0 = 0.1$ for the regular $\lambda = 1/2$ asymptotics to take hold. This demonstrates that the singularity at $\mu = 0$ influences learning even when $\mu_0 \neq 0$, provided μ_0 is close to the singular point.

Exercise 4.2: Solution p. 98

(a) By definition, the partition function is

$$Z_n = \int_{\mathcal{W}} \varphi(w) \prod_{i=1}^{n} p(x^{(i)} \mid w)\, dw.$$

Since $\mathcal{W} = \mathcal{U} \sqcup \mathcal{V}$ is a disjoint union, the integral splits additively:

$$\begin{aligned} Z_n &= \int_{\mathcal{U}} \varphi(w) \prod_{i=1}^{n} p(x^{(i)} \mid w)\, dw + \int_{\mathcal{V}} \varphi(w) \prod_{i=1}^{n} p(x^{(i)} \mid w)\, dw \\ &= Z_n(\mathcal{U}) + Z_n(\mathcal{V}). \end{aligned}$$

Recalling that $F_n = -\log Z_n$ and similarly for the local free energies,

$$Z_n = e^{-F_n(\mathcal{U})} + e^{-F_n(\mathcal{V})},$$

and taking the negative logarithm gives

$$F_n = -\log Z_n = -\log(e^{-F_n(\mathcal{U})} + e^{-F_n(\mathcal{V})}).$$

(b) The curves F_n and $\min(F_n(\mathcal{U}), F_n(\mathcal{V}))$ are nearly identical, both track whichever local free energy is smaller. However, F_n is smooth whereas $\min(F_n(\mathcal{U}), F_n(\mathcal{V}))$ has a kink at $n \approx 570$, where $F_n(\mathcal{U}) = F_n(\mathcal{V})$. For $n \lesssim 570$ the simpler region $\mathcal{V}$ (lower λ) dominates; for $n \gtrsim 570$ the lower loss region $\mathcal{U}$ (lower loss coefficient) dominates.

(c) Differentiating the log-sum-exp from (a),

$$\frac{d}{dn}F_n = \frac{\frac{d}{dn}F_n(\mathcal{U})\, e^{-F_n(\mathcal{U})} + \frac{d}{dn}F_n(\mathcal{V})\, e^{-F_n(\mathcal{V})}}{e^{-F_n(\mathcal{U})} + e^{-F_n(\mathcal{V})}},$$

Plotting this reveals a steep step around $n \approx 570$, transitioning from $\frac{d}{dn}F_n(\mathcal{V}) \approx 0.5$ to $\frac{d}{dn}F_n(\mathcal{U}) \approx 0.3$. This rapid change in the derivative of F_n is a smooth approximation of a discontinuity, justifying the term *phase transition*.

(d) Write $F_n(\mathcal{U}) \approx L_{\mathcal{U}}\, n + \lambda_{\mathcal{U}} \log n$ and $F_n(\mathcal{V}) \approx L_{\mathcal{V}}\, n + \lambda_{\mathcal{V}} \log n$, and set $\Delta L = L_{\mathcal{U}} - L_{\mathcal{V}}$ and $\Delta\lambda = \lambda_{\mathcal{U}} - \lambda_{\mathcal{V}}$. The critical sample size n^* where the phase transition occurs satisfies

$$\Delta L \cdot n^* = -\Delta\lambda \cdot \log n^*, \qquad \text{i.e.} \qquad \frac{n^*}{\log n^*} = -\frac{\Delta\lambda}{\Delta L}$$

Since $n^*/\log n^*$ is positive, a solution exists if and only if $-\Delta\lambda/\Delta L > 0$, i.e. ΔL and $\Delta\lambda$ have *opposite signs*. A phase transition occurs precisely when one region fits better while the other is simpler (ΔL and $\Delta\lambda$ have opposite signs). If both fit and complexity favour the same region, that region dominates for all n and no phase transition occurs.

$n^*/\log n^*$ is monotonically increasing for $n > e$ so for such n, increasing $|\Delta\lambda|$ increases n^* and increasing ΔL decreases n^*.

Training Dynamics

Guillaume Corlouer (Stormglass)

Main text: Chapter B.4, p. 103

Exercise 3.1: Solution p. 105

When all W_l are diagonal, the product $W = W_L \cdots W_1$ is also diagonal with entries $(W)_{\alpha\alpha} = \prod_{l=1}^{L} w_l^{(\alpha)}$. The teacher M is diagonal with entries s_α. Then:

$$\mathcal{L} = \frac{1}{2}\|M - W\|_F^2 = \frac{1}{2}\sum_{\alpha=1}^{d}(s_\alpha - (W)_{\alpha\alpha})^2 = \frac{1}{2}\sum_{\alpha=1}^{d}\left(s_\alpha - \prod_{l=1}^{L} w_l^{(\alpha)}\right)^2$$

Since the different modes α share no parameters, the loss decomposes into d independent scalar problems. □

Exercise 3.2: Solution p. 105

For $L = 2$, the gradient equations are:

$$\frac{\partial \ell}{\partial w_1} = -(s - w_1 w_2)\, w_2 = 0, \qquad \frac{\partial \ell}{\partial w_2} = -(s - w_1 w_2)\, w_1 = 0$$

Case 1: $s - w_1 w_2 = 0$, i.e. $w_1 w_2 = s$. This is the global minimum manifold (a hyperbola $w_2 = s/w_1$) with $\ell = 0$.

Case 2: If $s - w_1 w_2 \neq 0$, then the first equation requires $w_2 = 0$ and the second requires $w_1 = 0$. (For instance, if $w_2 \neq 0$ and $w_1 = 0$, the first equation gives $-s\, w_2 = 0$, which contradicts $s > 0$ and $w_2 \neq 0$.) So the only other critical point is $w_1 = w_2 = 0$.

The Hessian at $(0, 0)$. We need the second derivatives of $\ell = \frac{1}{2}(s - w_1 w_2)^2$:

$$\frac{\partial^2 \ell}{\partial w_1^2} = w_2^2, \qquad \frac{\partial^2 \ell}{\partial w_2^2} = w_1^2,$$
$$\frac{\partial^2 \ell}{\partial w_1 \partial w_2} = -(s - w_1 w_2) + w_1 w_2 = 2 w_1 w_2 - s$$

At $(0, 0)$:

$$H = \begin{pmatrix} 0 & -s \\ -s & 0 \end{pmatrix}$$

The eigenvalues are $\pm s$. Since $s > 0$, H has one positive and one negative eigenvalue. The origin is a **strict saddle point**. □

Exercise 3.3: Solution p. 105

At the critical point $W = P_S M$ with $S = \{1, \ldots, r\}$, the student retains only the first r modes: $W = \text{diag}(s_1, \ldots, s_r, 0, \ldots, 0)$. The loss is:

$$\mathcal{L} = \frac{1}{2}\|M - P_S M\|_F^2 = \frac{1}{2}\sum_{\alpha \notin S} s_\alpha^2 = \frac{1}{2}\sum_{\alpha=r+1}^{d} s_\alpha^2$$

This is strictly positive whenever $r < d$ (since all $s_\alpha > 0$), so these are not global minima. By the Hessian analysis (extending Exercise 3.2), the direction corresponding to "switching on" a missing mode $\alpha \notin S$ is a descent direction, making these critical points saddle points. All critical points with $|S| = d$ have $W = M$ and $\mathcal{L} = 0$: these are the global minima. Therefore there are **no spurious local minima** — every local minimum is global. □

Exercise 3.4: Solution p. 105

Under the group action:

$$\mu(g \cdot \theta) = W_L g_{L-1}^{-1} \cdot g_{L-1} W_{L-1} g_{L-2}^{-1} \cdot \ldots \cdot g_1 W_1 = W_L W_{L-1} \cdots W_1 = \mu(\theta)$$

All g_l and g_l^{-1} factors cancel telescopically. This means every parameter point in the orbit $\mathcal{O}_\theta = GL_h \cdot \theta$ maps to the same student W. □

For the dimension of the set of global minima, this invariance means the following. The global minima form the fiber $\mu^{-1}(M)$, which contains the entire orbit $GL_h \cdot \theta^*$ for any global minimizer θ^*. The orbit has dimension $\sum_{l=1}^{L-1} d_l^2$ (the dimension of GL_h), so the set of global minima is a **continuous manifold of very high dimension** — far from being isolated points. □

Exercise 4.1: Solution p. 106

We have $\mathcal{L} = \frac{1}{2}\|M - W_2 W_1\|_F^2 = \frac{1}{2}\text{Tr}\left[(M - W_2 W_1)^\top (M - W_2 W_1)\right]$.

General matrix derivation. Expand:

$$\mathcal{L} = \frac{1}{2}\text{Tr}(M^\top M) - \text{Tr}(M^\top W_2 W_1) + \frac{1}{2}\text{Tr}(W_1^\top W_2^\top W_2 W_1)$$

Differentiating with respect to W_1, using $\frac{\partial}{\partial A}\text{Tr}(B^\top A) = B$ and $\frac{\partial}{\partial A}\text{Tr}(A^\top C A) = (C + C^\top)A$:

$$\nabla_{W_1}\mathcal{L} = -W_2^\top M + W_2^\top W_2 W_1 = -W_2^\top (M - W_2 W_1)$$

So $\dot{W}_1 = -\nabla_{W_1}\mathcal{L} = W_2^\top (M - W_2 W_1)$.

Similarly, $\nabla_{W_2}\mathcal{L} = -(M - W_2 W_1) W_1^\top$, giving $\dot{W}_2 = (M - W_2 W_1) W_1^\top$.

Diagonal shortcut. For diagonal matrices $W_1 = \mathrm{diag}(a_\alpha)$, $W_2 = \mathrm{diag}(b_\alpha)$, $M = \mathrm{diag}(s_\alpha)$: the loss decouples as $\mathcal{L} = \frac{1}{2}\sum_\alpha (s_\alpha - b_\alpha a_\alpha)^2$. Then $\dot{a}_\alpha = -\partial\mathcal{L}/\partial a_\alpha = b_\alpha(s_\alpha - b_\alpha a_\alpha)$ and $\dot{b}_\alpha = a_\alpha(s_\alpha - b_\alpha a_\alpha)$, which is the diagonal version of the matrix equations above. □

Exercise 4.2: Solution p. 106

Let $E := M - W_2W_1$ denote the residual. Compute:

$$\dot{G} = \dot{W}_2^\top W_2 + W_2^\top \dot{W}_2 - \dot{W}_1 W_1^\top - W_1 \dot{W}_1^\top$$

Substitute the gradient flow equations. Using $\dot{W}_2 = EW_1^\top$ and $\dot{W}_1 = W_2^\top E$:

$$\dot{W}_2^\top W_2 = (EW_1^\top)^\top W_2 = W_1 E^\top W_2$$

$$W_2^\top \dot{W}_2 = W_2^\top E W_1^\top$$

$$\dot{W}_1 W_1^\top = W_2^\top E W_1^\top$$

$$W_1 \dot{W}_1^\top = W_1 (W_2^\top E)^\top = W_1 E^\top W_2$$

Therefore:

$$\dot{G} = W_1 E^\top W_2 + W_2^\top E W_1^\top - W_2^\top E W_1^\top - W_1 E^\top W_2 = 0$$

The terms cancel pairwise. G is conserved. □

Exercise 4.3: Solution p. 106

Apply the product rule:

$$\dot{W} = \dot{W}_2 W_1 + W_2 \dot{W}_1 = (M - W_2W_1)W_1^\top \cdot W_1 + W_2 \cdot W_2^\top (M - W_2W_1)$$

Writing $W = W_2W_1$:

$$\dot{W} = (M - W)\, W_1^\top W_1 + W_2 W_2^\top\, (M - W) \qquad \square$$

Exercise 4.4: Solution p. 106

Balancedness gives $W_2^\top W_2 = W_1 W_1^\top$, so

$$W^\top W = W_1^\top \left(W_2^\top W_2\right) W_1 = W_1^\top \left(W_1 W_1^\top\right) W_1 = \left(W_1^\top W_1\right)^2$$

Both $W^\top W$ and $W_1^\top W_1$ are positive semidefinite. A positive semidefinite matrix has a unique positive semidefinite square root, so it follows that

$$(W^\top W)^{1/2} = W_1^\top W_1 \qquad \square$$

Exercise 4.5: Solution p. 106

Analogous to Exercise 4.4.

Exercise 4.6: Solution p. 106

Substituting into Exercise 4.3:

$$\dot{W} = (M - W)(W^\top W)^{1/2} + (WW^\top)^{1/2}(M - W) \qquad \square$$

Exercise 4.7: Solution p. 107

With $F = M - W$, the equation from Exercise 4.6 reads:

$$\dot{W} = (WW^\top)^{1/2} F + F\,(W^\top W)^{1/2} = K[F] = K[M - W] \qquad \square$$

The general depth-L result $\dot{W} = \sum_{k=1}^{L} (WW^\top)^{(L-k)/L}(M - W)(W^\top W)^{(k-1)/L}$ follows from the same approach applied to the L-fold balanced conditions $W_{l+1}^\top W_{l+1} = W_l W_l^\top$ for all l.

Exercise 5.1: Solution p. 107

Under the alignment assumption, $W = U\,\mathrm{diag}(w_\alpha)\,V^\top$. Then:

$$WW^\top = U\,\mathrm{diag}(w_\alpha^2)\,U^\top$$

Since $w_\alpha \geq 0$, the positive matrix square root is:

$$(WW^\top)^{1/2} = U\,\mathrm{diag}(w_\alpha)\,U^\top$$

Similarly:

$$W^\top W = V\,\mathrm{diag}(w_\alpha^2)\,V^\top \quad \Longrightarrow \quad (W^\top W)^{1/2} = V\,\mathrm{diag}(w_\alpha)\,V^\top$$

The residual is immediate:

$$M - W = U\,\mathrm{diag}(s_\alpha - w_\alpha)\,V^\top$$

The NTK operator acts on any matrix $F = U\,\mathrm{diag}(f_\alpha)\,V^\top$ as:

$$\begin{aligned} K[F] &= U\,\mathrm{diag}(w_\alpha)\,U^\top U\,\mathrm{diag}(f_\alpha)\,V^\top \\ &\quad + U\,\mathrm{diag}(f_\alpha)\,V^\top V\,\mathrm{diag}(w_\alpha)\,V^\top \\ &= U\,\mathrm{diag}(2w_\alpha f_\alpha)\,V^\top. \end{aligned}$$

So $K[F]$ stays in the SVD basis of M — the NTK is aligned to the task. $\square$

Exercise 5.2: Solution p. 107

Substitute into $\dot{W} = (WW^\top)^{1/2}(M - W) + (M - W)(W^\top W)^{1/2}$:

First term:

$$U\,\mathrm{diag}(w_\alpha)\,\underbrace{U^\top U}_{I}\,\mathrm{diag}(s_\alpha - w_\alpha)\,V^\top = U\,\mathrm{diag}\big(w_\alpha(s_\alpha - w_\alpha)\big)\,V^\top$$

Second term:

$$U\,\mathrm{diag}(s_\alpha - w_\alpha)\,\underbrace{V^\top V}_{I}\,\mathrm{diag}(w_\alpha)\,V^\top = U\,\mathrm{diag}\big((s_\alpha - w_\alpha)\,w_\alpha\big)\,V^\top$$

Summing:

$$\dot{W} = U\,\mathrm{diag}\big(2\,w_\alpha(s_\alpha - w_\alpha)\big)\,V^\top$$

Since $W = U\,\mathrm{diag}(w_\alpha)\,V^\top$ and the singular vectors are constant by assumption, we read off:

$$\boxed{\dot{w}_\alpha = 2\,w_\alpha\,(s_\alpha - w_\alpha)}$$

The NTK perspective makes the structure transparent: the factor $2w_\alpha$ is the NTK eigenvalue for mode α, which amplifies learning in directions that are already strong, while $(s_\alpha - w_\alpha)$ is the residual. □

Exercise 5.3: Solution p. 108

From the logistic solution $w(t) = \frac{s}{1+(s/w_0-1)e^{-2st}}$, we invert to find t as a function of w. The derivation is equivalent to integrating $\dot{w} = 2w(s-w)$ by separation of variables:

$$\int_{w_0}^{w_f} \frac{dw}{w(s-w)} = 2\,t_\alpha$$

Using partial fractions $\frac{1}{w(s-w)} = \frac{1}{s}\left(\frac{1}{w} + \frac{1}{s-w}\right)$:

$$\frac{1}{s_\alpha}\Big[\ln w - \ln(s_\alpha - w)\Big]_{w_0}^{w_f} = 2\,t_\alpha$$

$$\frac{1}{s_\alpha}\ln\frac{w_f\,(s_\alpha - w_0)}{w_0\,(s_\alpha - w_f)} = 2\,t_\alpha$$

Therefore:

$$\boxed{t_\alpha = \frac{1}{2s_\alpha}\ln\left(\frac{w_f\,(s_\alpha - w_0)}{w_0\,(s_\alpha - w_f)}\right)}$$

For a fixed ratio w_f/s_α and fixed w_0, the learning time scales as $t_\alpha \propto 1/s_\alpha$: **modes with larger singular values are learned faster**. The network learns features in decreasing order of their strength — a strong separation of timescales.

Contrast with the lazy regime (anticipating Section 6): in the lazy regime the NTK is frozen at $K_0 = 2w_0 \gg 1$, so $\dot{w}_\alpha \approx 2w_0(s_\alpha - w_\alpha)$. All modes converge at the same exponential rate $2w_0$, independent of s_α. The rich regime has state-dependent NTK ($2w_\alpha$), creating a positive feedback loop — modes that are already large learn even faster — which amplifies the differences between s_α into a hierarchy of timescales $t_1 \ll t_2 \ll \cdots \ll t_r$. □

Exercise 5.4: Solution p. 108

With uniform initialization $w_\alpha(0) = w_0$ for all α, the sigmoid solution shows that mode α remains near w_0 until time $t \approx \frac{1}{2s_\alpha} \ln(s_\alpha/w_0)$, then transitions rapidly to $w_\alpha \approx s_\alpha$.

Early times ($t \sim \frac{1}{2s_1} \ln(s_1/w_0)$): Only mode 1 has grown significantly, so the spectrum is **one singular value near s_1, with all the others still near $w_0 \ll s_r$**. The student is approximately $W(t) \approx w_1(t)\, u_1 v_1^\top$, effectively **rank 1**.

Intermediate times: As t increases, w_2 switches on, then w_3, etc. The singular values **switch on one at a time**, in decreasing order of s_α, each staying near w_0 and then rising to s_α over a narrow window. The student is approximately $W(t) \approx \sum_{\alpha=1}^{k(t)} w_\alpha(t)\, u_\alpha v_\alpha^\top$ where $k(t)$ increases stepwise, so the effective **rank increases incrementally**.

Long times ($t \to \infty$): All modes converge, $w_\alpha \to s_\alpha$, and $W \to M$.

This is an **implicit bias toward low-rank (simple) solutions**: at any finite time, the network represents the best rank-k approximation to the teacher. The gradient flow effectively performs a greedy SVD.

Connection to the saddle structure (Exercise 3.3): The saddle points $P_S M$ with $|S| = k$ are exactly the best rank-k approximations to M, with loss $\frac{1}{2}\sum_{\alpha>k} s_\alpha^2$. The gradient flow trajectory passes **near** these saddles as it incrementally recruits modes. The **plateaus** in the loss curve correspond to time spent near saddle points (only k modes active), and the **sharp transitions** correspond to escaping along the unstable direction that activates the next mode. □

Exercise 6.1: Solution p. 109

Starting from the same ODE $\dot{w} = 2w(s - w)$, with $w_0 \gg s > 0$, the weight w needs to decrease from w_0 to s. As long as w has not changed much from w_0, we can write $w = w_0 + \delta w$ with $|\delta w| \ll w_0$, so:

$$2w = 2(w_0 + \delta w) \approx 2w_0$$

The ODE becomes:

$$\dot{w} \approx 2w_0(s - w) \qquad \square$$

This is a **linear** ODE — the NTK factor $2w$ has been frozen at its initial value $2w_0$. Note that only the NTK prefactor is linearized; the residual $(s - w)$ is kept exact since it drives learning.

Exercise 6.2: Solution p. 109

The linearized ODE $\dot{w} = 2w_0(s - w)$ is first-order linear. Set $\tilde{w} := w - s$, then $\dot{\tilde{w}} = -2w_0\, \tilde{w}$, with solution $\tilde{w}(t) = (w_0 - s)\, e^{-2w_0 t}$. Therefore:

$$\boxed{w(t) = s + (w_0 - s)\, e^{-2w_0 t}}$$

This is **exponential** convergence to s (contrast with the sigmoid of Section 5). The learning timescale is:

$$t_{\text{lazy}} \sim \frac{1}{2w_0}$$

It depends on the **initialization scale** w_0 but **not on the target** s. This is the defining feature of the lazy regime: the learning speed is set by the NTK at initialization, not by the structure of the task.

Self-consistency: During the learning phase ($t \lesssim 1/(2w_0)$), the displacement is $|w(t) - w_0| \leq |s - w_0| \approx w_0$ (since $w_0 \gg s$). The relative change in the NTK is $\Delta(2w)/(2w_0) = (w_0 - s)/w_0 = 1 - s/w_0 \approx 1$, so the NTK does change significantly in absolute terms. However, the key point is that the dynamics remain well approximated by the linear ODE because the convergence rate is dominated by $2w_0$, which is large and approximately constant throughout training. In the rich regime, by contrast, the NTK changes from $2w_0 \approx 0$ to $2s$ — a change of order $s/w_0 \to \infty$ relative to the initial value — making the linearization completely invalid. □

Exercise 6.3: Solution p. 109

Restoring the mode index, each mode satisfies $\dot{w}_\alpha \approx 2w_0(s_\alpha - w_\alpha)$ with solution:

$$w_\alpha(t) = s_\alpha + (w_0 - s_\alpha)\, e^{-2w_0 t}$$

All modes converge at the **same exponential rate** $2w_0$, regardless of s_α. The time for mode α to go from w_0 to within $(1 - \epsilon)$ of s_α is:

$$t_\alpha^{\text{lazy}} = \frac{1}{2w_0} \ln\left(\frac{w_0 - s_\alpha}{\epsilon\,(w_0 - s_\alpha)}\right) = \frac{1}{2w_0} \ln \frac{1}{\epsilon}$$

which is **independent of** s_α. All modes reach their targets simultaneously.

Contrast with the rich regime:

	Rich regime	**Lazy regime**
NTK	State-dependent: $2w_\alpha$	Frozen: $2w_0$
Learning time for mode α	$t_\alpha \propto 1/s_\alpha$	$t_\alpha \approx$ const
Timescale separation	Strong: $t_1 \ll t_2 \ll \cdots$	None
Intermediate solutions	Low-rank (incremental)	Full-rank from the start
Implicit rank bias	Yes (low-rank → high-rank)	No

In the lazy regime, all components of M are learned simultaneously: signal and noise alike. The network converges directly to the full OLS solution without passing through low-rank intermediates. There is no mechanism to separate signal from noise based on singular value structure. With early stopping, the rich regime recovers a low-rank signal while ignoring noise; the lazy regime cannot. □

Exercise 7.1: Solution p. 109

The interpolating ODE is $\dot{w}_\alpha = 2\sqrt{w_\alpha^2 + \tau^2}\,(s_\alpha - w_\alpha)$.

Rich limit ($\tau \to 0$): The square root reduces to $\sqrt{w_\alpha^2} = |w_\alpha|$, giving:

$$\dot{w}_\alpha = 2|w_\alpha|(s_\alpha - w_\alpha)$$

For $w_\alpha > 0$ this is $2w_\alpha(s_\alpha - w_\alpha)$, exactly the logistic ODE from Exercise 5.2. □

Lazy limit ($\tau \to \infty$, w_α bounded): The square root is dominated by τ: $\sqrt{w_\alpha^2 + \tau^2} \approx \tau$, giving:

$$\dot{w}_\alpha \approx 2\tau(s_\alpha - w_\alpha)$$

This is a linear ODE with rate 2τ, independent of s_α — the frozen-NTK regime of Section 6 (with τ playing the role of w_0). □

Exercise 7.2: Solution p. 110

At initialization, $w_\alpha(0) \sim \sigma^2 \ll \tau = \sigma^2\sqrt{n}$ (since $\sqrt{n} \gg 1$). So initially all modes satisfy $|w_\alpha| \ll \tau$.

Early phase (lazy): $\sqrt{w_\alpha^2 + \tau^2} \approx \tau$ for all α. Every mode evolves at the same linear rate 2τ:

$$\dot{w}_\alpha \approx 2\tau(s_\alpha - w_\alpha) \qquad \Longrightarrow \qquad w_\alpha(t) \approx s_\alpha(1 - e^{-2\tau t})$$

(assuming $w_\alpha(0) \approx 0$). The dynamics are approximately linear and the NTK is approximately constant. During this phase, the network **aligns** with the task: each w_α grows toward s_α at the same rate. There is no timescale separation.

Late phase (rich): Once some w_α grow past τ, the square root transitions to $\sqrt{w_\alpha^2 + \tau^2} \approx |w_\alpha|$, and those modes enter the rich regime with the sigmoidal, self-accelerating dynamics $\dot{w}_\alpha \approx 2w_\alpha(s_\alpha - w_\alpha)$. The state-dependent NTK creates timescale separation, and the modes that have crossed the threshold converge rapidly to their targets.

In summary: the network starts in a lazy phase where all modes grow uniformly (alignment), then transitions to a rich phase where modes accelerate individually (incremental learning). □

Exercise 7.3: Solution p. 110

In the lazy phase, each mode grows as $w_\alpha(t) \approx s_\alpha(1 - e^{-2\tau t})$. At any given time, $w_\alpha(t) \propto s_\alpha$: **modes with larger s_α are larger**. Since the lazy-to-rich transition occurs when $|w_\alpha| \sim \tau$, the time for mode α to cross the threshold satisfies:

$$s_\alpha(1 - e^{-2\tau t^*_\alpha}) = \tau \qquad \Longrightarrow \qquad t^*_\alpha = \frac{1}{2\tau} \ln \frac{s_\alpha}{s_\alpha - \tau}$$

For $s_\alpha > \tau$, this crossing time exists and is smaller for larger s_α. For $s_\alpha < \tau$, the mode never reaches the threshold and remains permanently lazy.

This means that **different modes can be in different regimes simultaneously**: large-s_α modes have crossed into the rich regime and are converging rapidly, while small-s_α modes are still in the lazy phase (or may never leave it). The network interpolates between the two regimes mode by mode.

Connection to grokking: This lazy-to-rich transition provides a mechanism for grokking. During the lazy phase, the network fits the training data via kernel regression with the initial (misaligned) NTK — it **memorizes**. The test loss plateaus because the frozen kernel cannot capture the task structure. Later, as modes cross the threshold into the rich regime, feature learning kicks in: the NTK rotates to align with the task, and the test loss drops — **generalization** is achieved. The grokking delay is controlled by the time it takes the relevant modes to cross τ. Increasing the initialization scale σ or the width n increases τ, which extends the lazy phase and widens the grokking gap. This is the same mechanism identified by Kumar et al., *Grokking as the Transition from Lazy to Rich Training Dynamics* (arXiv:2310.06110). □

Exercise 8.1: Solution p. 495

Setting $\mathbf{j} = 0$ (thermal equilibrium) with isotropic noise $\Sigma = \sigma^2 I$:

$$0 = -\nabla\mathcal{L}(\theta)\, p^*(\theta) - \frac{\eta\sigma^2}{2}\nabla p^*(\theta)$$

Rearranging:

$$\frac{\nabla p^*}{p^*} = -\frac{2}{\eta\sigma^2}\nabla\mathcal{L}$$

The left-hand side is $\nabla \ln p^*$, so:

$$\nabla \ln p^*(\theta) = -\frac{2}{\eta\sigma^2}\nabla\mathcal{L}(\theta)$$

Integrating both sides:

$$\ln p^*(\theta) = -\frac{2}{\eta\sigma^2}\mathcal{L}(\theta) + \text{const}$$

$$\boxed{p^*(\theta) \propto \exp\left(-\frac{2}{\eta\sigma^2}\mathcal{L}(\theta)\right)}$$

This is the Boltzmann distribution with inverse temperature $\beta = 2/(\eta\sigma^2)$. □

Exercise 8.2: Solution p. 495

The effective temperature is $T = \eta\sigma^2/2$.

Small η (low temperature, $\beta \to \infty$): The distribution concentrates sharply on the global minima of $\mathcal{L}$. SGD converges to the lowest-loss minimizer without exploring.

Large η (high temperature, $\beta \to 0$): The distribution becomes nearly uniform over parameter space. SGD explores broadly and does not settle into any particular minimum.

Flat vs. sharp minima: At moderate temperature, the Boltzmann distribution assigns more probability mass to **broad basins** (flat minima) than to narrow ones. This is because a flat minimum occupies a larger volume of parameter space at any given loss level — the width of the basin acts as an entropic contribution. Flat minima tend to generalize better because small perturbations to the parameters (or slight distribution shift in the data) do not dramatically change the loss. Thus the implicit bias of SGD noise toward flat minima is beneficial for generalization. □

Exercise 8.3: Solution p. 496

When $\Sigma(\theta)$ is anisotropic, the noise strength varies by direction. The Fokker-Planck current becomes:

$$\mathbf{j} = -\nabla\mathcal{L}\, p - \frac{\eta}{2}\nabla \cdot (\Sigma(\theta)\, p)$$

The second term introduces an **Itô drift** $\frac{\eta}{2}\nabla \cdot \Sigma(\theta)$ that depends on the spatial variation of the noise covariance. In directions where Σ has large eigenvalues (high noise), the effective diffusion is strong and the system escapes easily from sharp regions of the loss landscape. In directions where Σ has small eigenvalues (low noise), the system is more stable and tends to remain there.

This creates a **direction-dependent regularizer**: SGD preferentially pushes parameters out of sharp directions (high gradient variance → large noise eigenvalue → fast escape) while preserving parameters along flat directions (low gradient variance → small noise eigenvalue → stability). This goes beyond what the Boltzmann distribution on $\mathcal{L}$ alone would predict. In general, detailed balance $\mathbf{j} = 0$ may not hold for anisotropic, state-dependent noise, leading to persistent probability currents and non-equilibrium steady states that further modify the implicit bias. □

Data Attribution

Louis Jaburi

Main text: Chapter B.5, p. 111

Exercise 2.1: Solution p. 118

(a) Set $F(w, \beta) := \nabla_w L(w; \beta) = \sum_j \beta_j \nabla_w L(w, z_j)$. Then F is C^1 in (w, β) (assuming $L(\cdot, z_j)$ is C^2), and $F(w^*, \mathbf{1}) = \nabla_w L(w^*) = 0$ because w^* is a critical point of the unperturbed loss. The Jacobian $\partial F / \partial w$ at $(w^*, \mathbf{1})$ equals $\nabla_w^2 L(w^*) = H$, and the invertibility hypothesis of the implicit function theorem is exactly the assumption that H is invertible. The theorem then yields a unique C^1 map $\beta \mapsto w^*(\beta)$ on a neighborhood of $\mathbf{1}$ with $F(w^*(\beta), \beta) \equiv 0$.

The uniqueness the implicit function theorem provides is only *local*: $w^*(\beta)$ is uniquely defined as the critical point near w^*. The assumption that w^* is the unique *global* minimum is what lets us identify this local branch of critical points with the object we actually care about — the argmin. Without it, $\arg\min_w L(w; \beta)$ could jump to a different basin as β varies, and the response function would fail to be continuous (let alone differentiable).

(b) Define the two-slot function

$$G(w, \beta) \;:=\; \nabla_w L(w; \beta) \;=\; \sum_j \beta_j \, \nabla_w L(w, z_j),$$

so that the optimality condition reads $G(w^*(\beta), \beta) \equiv 0$. The variable β enters this identity in two places: once through the first slot via $w^*(\beta)$, and once directly through the second slot. The multivariate chain rule therefore produces two terms when we differentiate with respect to β_i:

$$\underbrace{\left.\frac{\partial G}{\partial w}\right|_{(w^*(\beta),\beta)} \frac{\partial w^*(\beta)}{\partial \beta_i}}_{\text{indirect: } w^* \text{ moves with } \beta} + \underbrace{\left.\frac{\partial G}{\partial \beta_i}\right|_{(w^*(\beta),\beta)}}_{\text{direct: explicit } \beta_i \text{ dependence}} = 0.$$

The two partials are:

- $\partial G / \partial w = \nabla_w^2 L(w; \beta)$, which at $\beta = \mathbf{1}$ equals the Hessian H.
- $\partial G / \partial \beta_i$: differentiating $\sum_j \beta_j \nabla_w L(w, z_j)$ with respect to β_i with w held fixed picks out the single $j = i$ summand, giving $\nabla_w L(w, z_i)$.

Evaluating at $\beta = \mathbf{1}$ and solving,

$$\left.\frac{\partial w^*(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \;=\; -\,H^{-1}\,\nabla_w L(w^*, z_i).$$

(c) For a differentiable observable $\phi : W \to \mathbb{R}$, the chain rule applied to $\beta \mapsto \phi(w^*(\beta))$ gives

$$\begin{aligned}
\mathcal{I}(z_i, \phi) \;&=\; \left.\frac{\partial\, \phi(w^*(\beta))}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \\
&=\; \nabla_w \phi(w^*(\beta))^{\top} \left.\frac{\partial w^*(\beta)}{\partial \beta_i}\right|_{\beta=\mathbf{1}} \\
&=\; -\,\nabla_w \phi(w^*)^{\top}\, H^{-1}\, \nabla_w L(w^*, z_i),
\end{aligned}$$

where in the last step both factors are evaluated at $\beta = \mathbf{1}$, i.e. at w^*.

Exercise 2.2: Solution p. 119

(a) The gradient of the training loss is $\nabla_w L(w) = -\sum_i x_i(y_i - x_i^{\top} w) = -\sum_i x_i y_i + A w$. Setting this to zero gives $A w^* = \sum_i x_i y_i$, hence $w^* = A^{-1} \sum_i x_i y_i$.

(b) The Hessian of the unweighted training loss is $H = \sum_i x_i x_i^{\top} = A$, and the gradient of the per-example loss at $z_j = (x_j, y_j)$ is $\nabla_w L(w^*, z_j) = -x_j(y_j - x_j^{\top} w^*) = -x_j\, r_j$. Plugging into Definition 2.2 with $\phi(w) = w$ (so $\nabla_w \phi(w^*) = I$),

$$\mathcal{I}(z_j, w^*) \;=\; -\,A^{-1}\big(-x_j r_j\big) \;=\; A^{-1} x_j\, r_j,$$

a residual times a leverage-weighted input.

(c) The normal equations for $w^*(\beta_j)$ read

$$\Big(\sum_{i \neq j} x_i x_i^{\top} + \beta_j\, x_j x_j^{\top}\Big)\, w \;=\; \sum_{i \neq j} x_i y_i \;+\; \beta_j\, x_j y_j,$$

or equivalently $\big(A - (1-\beta_j)\, x_j x_j^{\top}\big)\, w = \sum_i x_i y_i - (1-\beta_j)\, x_j y_j$. Sherman–Morrison applied to the rank-one update on the left gives

$$\big(A - (1-\beta_j)\, x_j x_j^{\top}\big)^{-1} \;=\; A^{-1} \;+\; \frac{(1-\beta_j)\, p_j p_j^{\top}}{1 - (1-\beta_j)\, h_j},$$

where we write $p_j := A^{-1} x_j$ and $h_j := x_j^{\top} p_j$. Multiplying through and using the identities $A^{-1} \sum_i x_i y_i = w^*$ and $p_j^{\top} \sum_i x_i y_i = x_j^{\top} w^*$,

$$
\begin{aligned}
w^*(\beta_j) \;&=\; w^* \;-\; (1-\beta_j)\,p_j\,y_j \\
&\quad+\; \frac{(1-\beta_j)\,p_j\big(x_j^\top w^* \;-\; (1-\beta_j)\,h_j\,y_j\big)}{1-(1-\beta_j)\,h_j} \\
&=\; w^* \;+\; \frac{p_j}{1-(1-\beta_j)\,h_j}\Big[-(1-\beta_j)\,y_j\big(1-(1-\beta_j)h_j\big) \\
&\qquad\qquad +\; (1-\beta_j)\,x_j^\top w^* \;-\; (1-\beta_j)^2\,h_j\,y_j\Big] \\
&=\; w^* \;+\; p_j\cdot\frac{(1-\beta_j)\big(x_j^\top w^* - y_j\big)}{1-(1-\beta_j)\,h_j} \\
&=\; w^* \;-\; \frac{(1-\beta_j)\,p_j\,r_j}{1-(1-\beta_j)\,h_j},
\end{aligned}
$$

the claimed expression. The result is rational in β_j because the matrix being inverted is linear in β_j.

(d) Expanding $1/(1-(1-\beta_j)h_j)$ as a geometric series in $(1-\beta_j)h_j$ (valid for $|(1-\beta_j)h_j|<1$),

$$
\begin{aligned}
w^*(\beta_j) - w^* \;&=\; -(1-\beta_j)\,A^{-1}x_j\,r_j\,\sum_{k=0}^{\infty}(1-\beta_j)^k\,h_j^k \\
&=\; -A^{-1}x_j\,r_j\,\sum_{k=1}^{\infty}(1-\beta_j)^k\,h_j^{k-1}.
\end{aligned}
$$

Reading off term by term:

- The $k=1$ term is $-A^{-1}x_j\,r_j\,(1-\beta_j) = \mathcal{I}(z_j,w^*)\cdot(\beta_j-1)$, exactly the influence-function prediction — as it must be, since the IF *is* the linear coefficient of $w^*(\beta_j)$ at $\beta_j=1$.
- The $k=2$ term, $-A^{-1}x_j\,r_j\,(1-\beta_j)^2\,h_j$, is the second-order Taylor correction. It carries one factor of leverage and is doubly suppressed: small both in $(1-\beta_j)$ and in h_j.
- Each subsequent term picks up another factor of $(1-\beta_j)h_j$. The IF/LOO "error" is precisely this tail, viewed as a series in the perturbation size and the leverage.

(e) At $\beta_j=0$ the perturbation factor $(1-\beta_j)$ is unity, and the higher-order terms no longer have a small parameter in front of them — only powers of h_j remain. The series collapses to a pure geometric series in h_j:

$$
w^*_{(-j)} - w^* \;=\; -A^{-1}x_j\,r_j\,\big(1+h_j+h_j^2+h_j^3+\cdots\big) \;=\; -\frac{A^{-1}x_j\,r_j}{1-h_j}.
$$

The IF prediction $-A^{-1}x_j\,r_j$ is the $k=0$ leading term; everything else is the Taylor remainder. So the often-quoted "factor of $1/(1-h_j)$" between IF and LOO is not a non-perturbative effect: it is the sum of all the higher-order Taylor terms, which OLS

happens to admit in closed form because $w^*(\beta_j)$ is rational. For more general M-estimators the response function is no longer rational and the higher-order terms do not sum to such a clean expression — but they are still there, with the same qualitative behavior.

In the two limiting regimes:

- $h_j \to 0$ (low leverage). Each higher-order term carries a factor of h_j^{k-1}, so they are individually negligible and the IF approximation is essentially exact.
- $h_j \to 1$ (high leverage). The geometric series barely converges and the higher-order terms together carry an arbitrarily large total weight. The IF underestimates the true LOO change without bound.

This is a clean demonstration that even in a strictly convex, fully-converged setting the linearization in Definition 2.2 loses control over precisely the points one most wants to attribute — the ones the model could not fit without them. The lesson generalizes: high-leverage / outlier examples are exactly where IF approximations should be distrusted, and the modern fixes from Section 2.2 (damping, GNH, PBRF) all address related but more severe versions of the same issue.

Exercise 2.3: Solution p. 122

(a) The per-example loss is $\ell_i = \frac{1}{2}(z_i - ab)^2$. Its second derivatives are

$$\frac{\partial^2 \ell_i}{\partial a^2} = b^2, \qquad \frac{\partial^2 \ell_i}{\partial b^2} = a^2, \qquad \frac{\partial^2 \ell_i}{\partial a\, \partial b} = 2ab - z_i.$$

Summing over i and evaluating at any minimizer (a^*, b^*) with $a^* b^* = \bar{z}$,

$$H \;=\; n \begin{pmatrix} (b^*)^2 & a^* b^* \\ a^* b^* & (a^*)^2 \end{pmatrix} \;=\; n\, \mathbf{v}\mathbf{v}^\top, \qquad \mathbf{v} := \begin{pmatrix} b^* \\ a^* \end{pmatrix},$$

where we used $\sum_i (2a^* b^* - z_i) = n(2\bar{z} - \bar{z}) = n a^* b^*$ for the off-diagonal. This is rank one, with image $\operatorname{span}(\mathbf{v})$.

The null space is $\operatorname{span}\big((a^*, -b^*)^\top\big)$. Geometrically: the minimizers form the hyperbola $\{(a, b) : ab = \bar{z}\}$, and the tangent to this hyperbola at (a^*, b^*) is exactly the direction $(a^*, -b^*)$ — the infinitesimal generator of the rescaling symmetry $(a, b) \to (\alpha a,\, b/\alpha)$ at $\alpha = 1$. The null space of H is the tangent to the manifold of minimizers.

(b) We need two gradients at the minimizer. For the observable $\phi(w) = ab$:

$$\nabla_w \phi \;=\; (b^*,\, a^*)^\top \;=\; \mathbf{v}.$$

For the per-example loss at z_i:

$$\nabla_w L(w^*, z_i) \;=\; -(z_i - \bar{z})\,(b^*,\, a^*)^\top \;=\; -(z_i - \bar{z})\,\mathbf{v}.$$

Both point in the $\mathbf{v}$ direction — neither has a component in the null space of H. The Moore–Penrose pseudoinverse of $H = n\,\mathbf{v}\mathbf{v}^\top$ is

$$H^+ \;=\; \frac{1}{n\,\|\mathbf{v}\|^4}\,\mathbf{v}\mathbf{v}^\top, \qquad \|\mathbf{v}\|^2 = (a^*)^2 + (b^*)^2.$$

Plugging into the IF formula,

$$\mathcal{I}(z_i, \phi) \;=\; -\mathbf{v}^\top H^+\big(-(z_i - \bar{z})\,\mathbf{v}\big) \;=\; (z_i - \bar{z})\,\frac{\|\mathbf{v}\|^4}{n\,\|\mathbf{v}\|^4} \;=\; \frac{z_i - \bar{z}}{n}.$$

The $\|\mathbf{v}\|$ factors cancel: the pseudoinverse IF is *independent* of the choice of minimizer (a^*, b^*).

Comparison with exact LOO. Removing z_i changes the sample mean to $\bar{z}_{(-i)} = (n\bar{z} - z_i)/(n-1)$, and the observable at any new minimizer is $\phi = \bar{z}_{(-i)}$. So the exact LOO change is

$$\phi(w^*_{(-i)}) - \phi(w^*) \;=\; \bar{z}_{(-i)} - \bar{z} \;=\; -\,\frac{z_i - \bar{z}}{n-1}.$$

The IF predicts $-\mathcal{I}(z_i, \phi) = -(z_i - \bar{z})/n$. The ratio is $n/(n-1) = 1/(1-h)$ with leverage $h = 1/n$ — every point has equal leverage in this one-effective-parameter model, and we recover the same $1/(1-h)$ correction as in Exercise 2.2.

(c) The eigenvalues of $H = n\,\mathbf{v}\mathbf{v}^\top$ are $n\|\mathbf{v}\|^2$ (eigenvector $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$) and 0 (eigenvector $\hat{\mathbf{v}}_\perp = (a^*, -b^*)^\top/\|\mathbf{v}\|$). So

$$(H + \lambda I)^{-1} \;=\; \frac{1}{n\|\mathbf{v}\|^2 + \lambda}\,\hat{\mathbf{v}}\hat{\mathbf{v}}^\top \;+\; \frac{1}{\lambda}\,\hat{\mathbf{v}}_\perp\hat{\mathbf{v}}_\perp^\top.$$

Since $\nabla\phi$ and ∇L_i both lie in the $\hat{\mathbf{v}}$ direction, the $1/\lambda$ term does not contribute, and

$$\mathcal{I}_\lambda(z_i, \phi) \;=\; \frac{(z_i - \bar{z})\,\|\mathbf{v}\|^2}{n\|\mathbf{v}\|^2 + \lambda}.$$

Dependence on the minimizer. For $\lambda > 0$ the answer depends on $\|\mathbf{v}\|^2 = (a^*)^2 + (b^*)^2$, which varies along the orbit $ab = \bar{z}$. For instance, the "balanced" minimizer $(\sqrt{\bar{z}}, \sqrt{\bar{z}})$ gives $\|\mathbf{v}\|^2 = 2\bar{z}$, while the "unbalanced" choice $(\bar{z}, 1)$ gives $\|\mathbf{v}\|^2 = \bar{z}^2 + 1$. Different minimizers yield different IFs. This is the proximity gap: the damping term $\frac{\lambda}{2}\|w - w^*\|^2$ breaks the rescaling symmetry and anchors the computation at a particular point on the orbit.

Limit $\lambda \to 0$. $\mathcal{I}_\lambda \to (z_i - \bar{z})/n$ regardless of (a^*, b^*), recovering the pseudoinverse answer from (b). The damping-induced dependence on the minimizer vanishes.

Limit $\lambda \to \infty$. $\mathcal{I}_\lambda \to 0$: heavy damping kills all attribution, because the proximal penalty makes it too costly to move w at all.

The punchline: for this particular observable ($\phi = ab$, which is constant along the orbit), the pseudoinverse gives a clean, minimizer-independent answer and the damping only introduces a spurious dependence. But for an observable that *does* vary along the orbit — say $\phi(w) = a^2$ — $\nabla\phi$ would have a component in the null direction $(a^*, -b^*)$, the $1/\lambda$ term would contribute, and the IF would *diverge* as $\lambda \to 0$. In that

case damping is not a nuisance but a necessity, and the choice of minimizer genuinely matters. This is the core tension of the proximity gap: it is a distortion for orbit-invariant queries, but a regularizer for orbit-dependent ones.

Exercise 3.1: Solution p. 123

Write $Z(\beta) = \int \exp(-\sum_j \beta_j L(w, z_j))\, \varphi(w)\, dw$ for the normalizing constant, so that

$$\mathbb{E}_{p_\beta}[\phi(w)] \;=\; \frac{1}{Z(\beta)} \int \phi(w)\, \exp\Big(-\sum_j \beta_j L(w, z_j)\Big)\, \varphi(w)\, dw.$$

Differentiating with respect to β_i and applying the quotient rule:

$$\begin{aligned}\frac{\partial}{\partial \beta_i}\, \mathbb{E}_{p_\beta}[\phi] \;&=\; \frac{1}{Z} \int \phi(w) \big(-L(w, z_i)\big)\, e^{-\sum_j \beta_j L_j}\, \varphi\, dw \\ &\quad -\; \frac{1}{Z^2} \cdot \frac{\partial Z}{\partial \beta_i} \int \phi\, e^{-\sum_j \beta_j L_j}\, \varphi\, dw.\end{aligned}$$

The first term is $-\mathbb{E}_{p_\beta}[\phi(w)\, L(w, z_i)]$. For the second, $\partial_{\beta_i} Z = -Z\, \mathbb{E}_{p_\beta}[L(w, z_i)]$, so it equals $+\mathbb{E}_{p_\beta}[L(w, z_i)]\, \mathbb{E}_{p_\beta}[\phi(w)]$. Evaluating at $\beta = \mathbf{1}$:

$$\mathrm{BIF}(z_i, \phi) = -\mathbb{E}_p[\phi \cdot L_i] + \mathbb{E}_p[\phi]\, \mathbb{E}_p[L_i] = -\mathrm{Cov}_{w \sim p}(L(w, z_i),\, \phi(w)).$$

Structurally: in the classical IF, the "correlation" between L_i and ϕ is mediated by the inverse Hessian acting on their gradients at a point; in the BIF it is mediated by the full posterior distribution. The inverse Hessian is how a Gaussian posterior would produce a covariance (as we will see in Exercise 3.2), so the BIF strictly generalizes the classical formula.

Exercise 3.2: Solution p. 126

(a) The log-posterior is $\log p(w \mid \mathcal{D}) = -L(w) + \log \varphi(w) + \text{const}$. Taylor-expanding $L(w)$ around the minimum w^*:

$$L(w) = L(w^*) + \underbrace{\nabla L(w^*)^\top}_{=\,0}\, \Delta w + \tfrac{1}{2}\, \Delta w^\top H\, \Delta w + O(\|\Delta w\|^3).$$

The linear term vanishes because w^* is a critical point. Substituting into $\log p$:

$$\log p(w \mid \mathcal{D}) \approx -L(w^*) - \tfrac{1}{2}\, \Delta w^\top H\, \Delta w + \log \varphi(w) + \text{const}.$$

Since $L = \sum_{i=1}^n L_i$, the Hessian $H = \sum_i \nabla^2 L_i$ scales as $O(n)$, while the prior contributes $O(1)$ to the log-density. For large n the quadratic term dominates, giving $p(w \mid \mathcal{D}) \approx \mathcal{N}(w^*,\, H^{-1})$. This is the Laplace approximation. (The Bernstein–von Mises theorem makes this rigorous: under regularity, the posterior converges to this Gaussian in total variation as $n \to \infty$.)

(b) Since $\mathrm{Cov}(X + c, Y) = \mathrm{Cov}(X, Y)$ for any constant c, the constant terms $\phi(w^*)$ and $L(w^*, z_i)$ drop out. The covariance of the two Taylor series is bilinear, so it distributes over the sum of terms:

$$\mathrm{Cov}(\phi, L_i) = \sum_{k\geq 1}\sum_{m\geq 1} \mathrm{Cov}(T_k[\phi],\, T_m[L_i]).$$

Under $\Delta w \sim \mathcal{N}(0, H^{-1})$, $T_k[\phi]$ is a degree-k polynomial in Δw. The covariance $\mathrm{Cov}(T_k, T_m) = \mathbb{E}[T_k T_m] - \mathbb{E}[T_k]\mathbb{E}[T_m]$ involves moments of Δw of degree $k+m$. For a centered Gaussian, odd moments vanish: $\mathbb{E}[\Delta w_{a_1}\cdots\Delta w_{a_\ell}] = 0$ when ℓ is odd. If $k+m$ is odd, then all moments in $\mathbb{E}[T_k T_m]$ and $\mathbb{E}[T_k]\mathbb{E}[T_m]$ involve an odd total degree, so both vanish and $\mathrm{Cov}(T_k, T_m) = 0$.

(c) $T_1[\phi] = g_\phi^\top \Delta w$ and $T_1[L_i] = g_i^\top \Delta w$ are linear in Δw. Their covariance is

$$\mathrm{Cov}(g_\phi^\top \Delta w,\ g_i^\top \Delta w) = g_\phi^\top\, \mathbb{E}[\Delta w\, \Delta w^\top]\, g_i = g_\phi^\top H^{-1} g_i.$$

Negating gives $-g_\phi^\top H^{-1} g_i = -\nabla\phi(w^*)^\top H^{-1} \nabla L(w^*, z_i)$, which is the classical influence function $\mathcal{I}(z_i, \phi)$ from Definition 2.2.

(d) Write $Q_\phi = \frac{1}{2}\Delta w^\top H_\phi\, \Delta w$ and $Q_i = \frac{1}{2}\Delta w^\top H_i\, \Delta w$. We need $\mathrm{Cov}(Q_\phi, Q_i) = \mathbb{E}[Q_\phi Q_i] - \mathbb{E}[Q_\phi]\mathbb{E}[Q_i]$ with $\Delta w \sim \mathcal{N}(0, \Sigma)$, $\Sigma = H^{-1}$.

For the expectation: $\mathbb{E}[Q_\phi] = \frac{1}{2}\,\mathrm{tr}(H_\phi \Sigma)$ and $\mathbb{E}[Q_i] = \frac{1}{2}\,\mathrm{tr}(H_i\Sigma)$.

For the cross-moment: $\mathbb{E}[Q_\phi Q_i] = \frac{1}{4}\sum_{a,b,c,d} (H_\phi)_{ab}(H_i)_{cd}\, \mathbb{E}[\Delta w_a \Delta w_b \Delta w_c \Delta w_d]$. By Isserlis' theorem:

$$\mathbb{E}[\Delta w_a \Delta w_b \Delta w_c \Delta w_d] = \Sigma_{ab}\Sigma_{cd} + \Sigma_{ac}\Sigma_{bd} + \Sigma_{ad}\Sigma_{bc}.$$

The first pairing gives $\frac{1}{4}\,\mathrm{tr}(H_\phi\Sigma)\,\mathrm{tr}(H_i\Sigma) = \mathbb{E}[Q_\phi]\mathbb{E}[Q_i]$, which cancels in the covariance. The other two pairings each give $\frac{1}{4}\,\mathrm{tr}(H_\phi \Sigma H_i \Sigma)$ (by cyclicity of the trace and symmetry of H_ϕ, H_i, and Σ). So

$$\mathrm{Cov}(Q_\phi, Q_i) = 2 \cdot \tfrac{1}{4}\,\mathrm{tr}(H_\phi \Sigma H_i \Sigma) = \tfrac{1}{2}\,\mathrm{tr}(H_\phi H^{-1} H_i H^{-1}).$$

In the linear regression setting with $\phi(w) = w_k$ (a coordinate function), $H_\phi = \nabla^2 w_k = 0$: the observable is linear in w, so its Hessian vanishes and the entire $(2,2)$ correction is zero. This is why the BIF equals the (damped) IF exactly for linear regression — all corrections beyond leading order involve H_ϕ or higher derivatives of ϕ, which vanish for a linear observable.

(e) The localized posterior of Equation 10 has effective Hessian $H_{\text{eff}} = H + \gamma I$, so the Laplace approximation gives $\Delta w \sim \mathcal{N}(0, (H+\gamma I)^{-1})$. The leading-order $(1,1)$ computation from part (c) becomes

$$\mathrm{BIF}_\gamma(z_i, \phi) \approx -g_\phi^\top (H+\gamma I)^{-1} g_i = -\nabla\phi(w^*)^\top (H+\gamma I)^{-1} \nabla L(w^*, z_i),$$

which is the damped IF of Equation 6 with γ in the role of λ. The higher-order corrections from parts (b)–(d) carry through with H^{-1} replaced by $(H+\gamma I)^{-1}$ throughout.

(f) Summary: For regular (non-singular) models, the posterior is approximately Gaussian (Bernstein–von Mises), the Taylor expansion converges, and the $(1,1)$ term dominates (it scales as $O(1/n)$ while higher terms scale as $O(1/n^2)$ and beyond). The classical IF is a good approximation because it *is* the leading term. For singular models

(neural networks), the posterior is concentrated on a positive-dimensional variety, not at an isolated point. The Laplace approximation fails: the Hessian has a large null space, the posterior is non-Gaussian, and the Taylor series does not converge around w^*. In this regime, the BIF — defined as an exact covariance under the true posterior — captures the full geometry, while the classical IF is at best the leading term of an expansion that does not converge. The BIF is the fundamental object; the IF is the Gaussian shadow it casts.

Exercise 3.3: Solution p. 129

(a) Using $p_0^\beta(w) = e^{-\beta L(w)}/Z(\beta,0)$ and $p_\varepsilon^\beta(w) = e^{-\beta(L(w)-\varepsilon L_k(w))}/Z(\beta,\varepsilon)$, substituting into (13) gives

$$\begin{aligned} D_{\mathrm{KL}}(q_\varepsilon \| p_\varepsilon^\beta) &= \mathbb{E}_{w\sim p_0^\beta}\big[\log p_0^\beta(w) - \log p_\varepsilon^\beta(w+\varepsilon r) - \log|\det(I+\varepsilon\nabla r)|\big] \\ &= \mathbb{E}_{p_0^\beta}\big[-\beta L(w) - \log Z(\beta,0) + \beta(L(w+\varepsilon r) - \varepsilon L_k(w+\varepsilon r)) \\ &\qquad + \log Z(\beta,\varepsilon) - \log|\det(I+\varepsilon\nabla r)|\big]. \end{aligned}$$

Dividing by β and rearranging gives the claimed expression. As $\beta\to\infty$: (i) $\frac{1}{\beta}\log|\det(I+\varepsilon\nabla r)| \to 0$ since the log-determinant is $O(1)$ (bounded for smooth r and small ε); (ii) by Laplace's method, $\frac{1}{\beta}\log Z(\beta,\varepsilon) \to -\inf_w[L(w)-\varepsilon L_k(w)]$, so $\frac{1}{\beta}\log\frac{Z(\beta,\varepsilon)}{Z(\beta,0)} \to \inf L - \inf(L-\varepsilon L_k)$, which is a constant independent of r; (iii) the expectation concentrates on w^*.

(b) At w^*, $\nabla L(w^*)=0$, so the Taylor expansion of $L(w^*+\varepsilon r)$ starts at the quadratic term: $L(w^*) + \frac{\varepsilon^2}{2} r^\top H r + O(\varepsilon^3)$. For εL_k: $\varepsilon L_k(w^*+\varepsilon r) = \varepsilon L_k(w^*) + \varepsilon^2 \nabla L_k^\top r + O(\varepsilon^3)$. Combined:

$$\begin{aligned} &L(w^*+\varepsilon r) - \varepsilon L_k(w^*+\varepsilon r) - L(w^*) \\ &\quad = -\varepsilon L_k(w^*) + \frac{\varepsilon^2}{2} r^\top H r - \varepsilon^2 \nabla L_k^\top r + O(\varepsilon^3). \end{aligned}$$

For the infimum: expanding $L(w^*+\delta) - \varepsilon L_k(w^*+\delta)$ to second order in δ and minimizing, the optimal perturbation is $\delta^* = \varepsilon H^{-1}\nabla L_k$, giving

$$\inf_w[L - \varepsilon L_k] = L(w^*) - \varepsilon L_k(w^*) - \frac{\varepsilon^2}{2}\nabla L_k^\top H^{-1}\nabla L_k + O(\varepsilon^3).$$

So $\inf L - \inf(L-\varepsilon L_k) = \varepsilon L_k(w^*) + \frac{\varepsilon^2}{2}\nabla L_k^\top H^{-1}\nabla L_k + O(\varepsilon^3)$.

Adding the two contributions:

$$\begin{aligned} \mathcal{F}(r,\varepsilon) &= \frac{\varepsilon^2}{2} r^\top H r - \varepsilon^2 \nabla L_k^\top r + \frac{\varepsilon^2}{2}\nabla L_k^\top H^{-1}\nabla L_k + O(\varepsilon^3) \\ &= \frac{\varepsilon^2}{2}\big(r^\top H r - 2\nabla L_k^\top r + \nabla L_k^\top H^{-1}\nabla L_k\big) + O(\varepsilon^3). \end{aligned}$$

Completing the square: $r^\top H r - 2\nabla L_k^\top r + \nabla L_k^\top H^{-1}\nabla L_k = (r - H^{-1}\nabla L_k)^\top H (r - H^{-1}\nabla L_k)$, since expanding the right side gives $r^\top H r - 2\nabla L_k^\top H^{-1} \cdot H r + \nabla L_k^\top H^{-1} H H^{-1}\nabla L_k = r^\top H r - 2\nabla L_k^\top r + \nabla L_k^\top H^{-1}\nabla L_k$.

(c) Since H is positive definite, the squared Mahalanobis form $(r - r_{\mathrm{IF}})^\top H (r - r_{\mathrm{IF}}) \geq 0$ with equality iff $r = r_{\mathrm{IF}} = H^{-1}\nabla L_k$. At the minimum, $\mathcal{F}(r_{\mathrm{IF}}, \varepsilon) = O(\varepsilon^3)$: the IF shifts the distribution with only a *third-order* KL residual. Any other shift $r \neq r_{\mathrm{IF}}$ incurs a strictly positive $O(\varepsilon^2)$ cost proportional to its Mahalanobis distance from the IF. The IF is the unique optimal shift among all smooth vector fields r, not just among constant maps.

(d) When H is singular, the quadratic form $r^\top H r$ is zero for $r \in N = \mathrm{Null}(H)$: movement in the null space has zero Hessian cost because the loss is flat along the minimum manifold. The pseudoinverse H^+ inverts only the non-degenerate directions, and the completed square becomes $(r - H^+\nabla L_k)^\top H (r - H^+\nabla L_k)$. This vanishes whenever $r - H^+\nabla L_k \in N$, i.e., the minimizer is $r = H^+\nabla L_k + r_{\mathrm{NS}}$ for any $r_{\mathrm{NS}} \in N$.

Geometrically: the IF determines the optimal shift *off* the minimum manifold (perpendicular to $\mathcal{S}_L$), but movement *along* $\mathcal{S}_L$ is invisible to the energy at this order. In the SLT language: the degeneracy of the singular set means that many shifts are equally good at leading order — the IF picks one, but the null-space ambiguity is real.

(e) For linear regression with Gaussian prior, the posterior is $\mathcal{N}(\mu, V)$ with $V^{-1} = \sigma^{-2}A + \tau^{-2}I$. A constant shift $w \mapsto w + \varepsilon r$ sends $\mathcal{N}(\mu, V)$ to $\mathcal{N}(\mu + \varepsilon r, V)$. The true perturbed mean is $\mu + \varepsilon b + O(\varepsilon^2)$ with $b = \mathrm{BIF}(z_k, w) = (A + \lambda I)^{-1} x_k r_k$. For two Gaussians with the same covariance:

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu + \varepsilon r, V) \| \mathcal{N}(\mu + \varepsilon b, V)\big) = \frac{\varepsilon^2}{2}(r - b)^\top V^{-1}(r - b).$$

This is minimized at $r = b = (A + \lambda I)^{-1} x_k r_k$. To check consistency with (15): the "Hessian of the energy" in the Boltzmann distribution $p \propto e^{-(\mathrm{loss}+\mathrm{prior})}$ is $H_{\mathrm{eff}} = \sigma^{-2}A + \tau^{-2}I = V^{-1}$, and $H_{\mathrm{eff}}^{-1}\nabla_w L(\mu, z_k) = V \cdot \sigma^{-2} x_k r_k = (A + \lambda I)^{-1} x_k r_k = \mathrm{BIF}$. The general formula (15) reduces to $\frac{\varepsilon^2}{2}(r - b)^\top V^{-1}(r - b)$, exactly as computed directly.

(f) The two exercises are complementary:

- The Laplace expansion (Exercise 3.2) tells you *what* the IF is: the leading-order term of the BIF covariance under a Gaussian approximation to the posterior. The higher-order corrections (Hessian products, etc.) are visible but require the Laplace approximation to hold.
- The transport perspective (this exercise) tells you *why* the IF works even when the Laplace approximation fails: it minimizes the KL cost of transporting the parameter distribution, and this optimality holds in the $\beta \to \infty$ limit under mild regularity assumptions — no Gaussianity required. For singular models, the pseudoinverse handles the degenerate directions naturally (part d), and the result still says: the IF is the best you can do with a smooth first-order map.

This provides what Mlodozeniec et al. 2025 call a "distributional" justification for IFs in deep learning. The original justification (implicit function theorem, convexity) does not hold for neural networks. The Laplace/BIF justification (Exercise 3.2) requires Gaussianity of the posterior, which also fails. But the transport optimality result asks only for bounded derivatives and a well-defined minimum — conditions much closer to what actually holds in practice.

Exercise 4.1: Solution p. 132

(a) Differentiating (16) with respect to β_i at $\beta = \mathbf{1}$ via the chain rule:

$$\begin{aligned}\frac{\partial w_{t+1}}{\partial \beta_i} &= \frac{\partial w_t}{\partial \beta_i} - \frac{\eta_t}{b} \sum_{j \in B_t} \Big[\underbrace{\nabla_w^2 L(w_t, z_j)\, \frac{\partial w_t}{\partial \beta_i}}_{\text{indirect}} + \underbrace{\delta_{ij}\, \nabla_w L(w_t, z_j)}_{\text{direct}} \Big] \\ &= \Big(I - \frac{\eta_t}{b} \sum_{j \in B_t} \nabla_w^2 L(w_t, z_j) \Big) \frac{\partial w_t}{\partial \beta_i} - \frac{\eta_t}{b}\, \mathbf{1}[i \in B_t]\, \nabla_w L(w_t, z_i) \\ &= J_t\, \frac{\partial w_t}{\partial \beta_i} - \frac{\eta_t}{b}\, \mathbf{1}[i \in B_t]\, \nabla_w L(w_t, z_i).\end{aligned}$$

The initial condition is $\partial w_0 / \partial \beta_i = 0$ since w_0 does not depend on β.

(b) Substituting the recursion repeatedly:

$$\begin{aligned}\frac{\partial w_T}{\partial \beta_i} &= J_{T-1} \frac{\partial w_{T-1}}{\partial \beta_i} - \frac{\eta_{T-1}}{b}\, \mathbf{1}[i \in B_{T-1}]\, \nabla_w L(w_{T-1}, z_i) \\ &= J_{T-1} \Big(J_{T-2} \frac{\partial w_{T-2}}{\partial \beta_i} - \frac{\eta_{T-2}}{b}\, \mathbf{1}[i \in B_{T-2}]\, \nabla_w L(w_{T-2}, z_i) \Big) \\ &\quad - \frac{\eta_{T-1}}{b}\, \mathbf{1}[i \in B_{T-1}]\, \nabla_w L(w_{T-1}, z_i) \\ &= \cdots = - \sum_{t=0}^{T-1} \frac{\eta_t}{b}\, \mathbf{1}[i \in B_t]\, J_{(t+1):T}\, \nabla_w L(w_t, z_i),\end{aligned}$$

where the last step follows from $\partial w_0 / \partial \beta_i = 0$ killing the $J_{0:T}$ term. This is (20).

(c) With the preconditioned update (22), the indirect effect picks up the preconditioner in the Hessian term: $J_t^{(P)} = I - (\eta_t / b) \sum_{j \in B_t} P_t \nabla_w^2 L(w_t, z_j) = I - \eta_t P_t \widehat{H}_t$. The direct effect becomes $-(\eta_t / b)\, \mathbf{1}[i \in B_t]\, P_t \nabla_w L(w_t, z_i)$. Unrolling the recursion as in (b) gives (23). The preconditioner P_t at step t acts as a local rescaling of the gradient: Adam's adaptive scaling amplifies gradients in directions with historically small second moments. A datum appearing early in training (when Adam has not yet accumulated accurate statistics) will have its gradient rescaled differently than the same datum appearing later, when P_t has stabilized. This is a concrete mechanism by which optimizer choice affects per-datum attribution.

Exercise 4.2: Solution p. 137

(a) Differentiating (34) with respect to ε at $\varepsilon = 0$: the first row is just the SGD update, independent of ε. For the response, the chain rule gives two contributions: the indirect effect through $w_t(\varepsilon)$ picks up the mini-batch Hessian (giving $J_t\, r_t$ as in Exercise 4.1), and the direct effect from the $(1 - \varepsilon\, \mathbf{1}[i = k])$ factor contributes $+(\eta_t/b)\, \mathbf{1}[k \in B_t]\, \nabla_w L(w_t, z_k)$. This is (35). Given the current state (w_t, r_t) and the batch selection B_t (which is i.i.d. and independent of history), the next state (w_{t+1}, r_{t+1}) depends only on (w_t, r_t) — no earlier states are needed. This is the Markov property.

(b) With full-batch GD, $B_t = \{1, \dots, n\}$, the response recursion becomes deterministic: $r_{t+1} = (I - \eta H) r_t + \eta \nabla_w L(w^*, z_k)$. With $r_0 = 0$, unrolling gives $r_K = \eta \sum_{t=0}^{K-1} (I - \eta H)^t\, \nabla_w L(w^*, z_k)$. Using $\sum_{t=0}^{K-1} M^t = (I - M^K)(I - M)^{-1}$ with $M = I - \eta H$:

$$r_K = \left[I - (I - \eta H)^K \right] H^{-1}\, \nabla_w L(w^*, z_k).$$

(Note the sign: the interpolation $1 - \varepsilon\, \mathbf{1}[i = k]$ downweights z_k, so the direct effect has the opposite sign from the β-upweighting in Exercise 4.1.) Since $\|I - \eta H\| < 1$ when $\eta < 2/\lambda_{\max}(H)$, $(I - \eta H)^K \to 0$ and $r_\infty = H^{-1} \nabla_w L(w^*, z_k) = r_{\mathrm{IF}}$.

(c) i. At equilibrium $\dot{r} = 0$, the ODE (36) gives $H r_\infty = \nabla_w L(w^*, z_k)$. In the column space of H, this has the unique solution $r_{\mathrm{IF}} = H^{+} \nabla_w L(w^*, z_k)$. (If H is invertible, $H^{+} = H^{-1}$ and this is the classical IF.)

ii. Project the ODE onto $\ker(H)$: $P_0 \dot{r}(t) = -P_0 H\, r(t) + P_0 \nabla_w L(w^*, z_k) = P_0 \nabla_w L(w^*, z_k)$, since $P_0 H = 0$. Integrating: $P_0 r(t) = P_0 r(0) + t\, P_0 \nabla_w L(w^*, z_k)$. This grows linearly unless $P_0 \nabla_w L(w^*, z_k) = 0$, i.e. unless the per-example gradient has no component in the Hessian null space. The null space of H at a local minimum corresponds to flat directions along the minimum manifold; the response diverges if the perturbation "pushes" along these flat directions, because the optimizer has no restoring force.

Exercise 4.3: Solution p. 139

(a) The $A\mathcal{G}(t)$ and $\mathcal{G}(t)B$ terms describe how the perturbation rotates the singular basis. A rotation mixes already-learned modes into each other, so its effect on the weight matrix is proportional to the current mode strengths: if mode k has not yet been learned ($\mathcal{G}_k \approx 0$), rotating its basis direction contributes nothing to $W(t)$. The $\mathcal{G}'_\varepsilon(t)$ term, by contrast, acts through the learning *dynamics*: perturbing s_k changes the speed at which mode k is learned, and this matters precisely when the dynamics are sensitive to s_k — not when $\mathcal{G}_k(t)$ is large or small per se, but when $\mathcal{G}_k(t)$ is *changing rapidly* as a function of s_k. This is why the derivative $g'_t(s_k) = \partial \mathcal{G}_k(t)/\partial s_k$ appears rather than $\mathcal{G}_k(t)$ itself.

(b) From (37), $\mathcal{G}_k(t) = s_k e^{2 s_k t/\tau} / (e^{2 s_k t/\tau} - 1 + s_k/\mathcal{G}_k(0))$. For $t \ll t_k^*$: $e^{2 s_k t/\tau} \approx 1$, so $\mathcal{G}_k \approx \mathcal{G}_k(0)$ regardless of s_k — the mode hasn't started learning yet, and a small change in s_k makes no difference, so $g'_t(s_k) \approx 0$. For $t \gg t_k^*$: $e^{2 s_k t/\tau} \gg s_k/\mathcal{G}_k(0)$, so $\mathcal{G}_k \approx s_k$ and $g'_t(s_k) \approx 1$ — but the *residual* in mode k is also ≈ 0, so this term's contribution to the

loss influence (40) is negligible. The product $g_t'(s_k) \times$ (residual in mode k) is peaked around $t \approx t_k^*$, when the mode is in transition: the dynamics are maximally sensitive to s_k *and* the residual is still nonzero.

(c) Write $x_{\text{dog}} = (\alpha, +\delta)$ and $x_{\text{cat}} = (\alpha, -\delta)$, where α is the shared mode-1 component and $\pm\delta$ are opposite mode-2 components (since Σ_{xy} is diagonal, the standard basis is the singular basis). During mode-1 learning ($t \approx t_1^*$, mode 2 not yet active): the residual of the cat test example has a large mode-1 component, and z_{dog}'s perturbation increases s_1 (via $(y_{\text{dog}})_1 (x_{\text{dog}})_1 > 0$), accelerating mode-1 learning. This reduces the cat test loss — positive influence. After mode-2 learning ($t \approx t_2^*$): the cat's residual is now dominated by mode 2, and z_{dog}'s mode-2 perturbation has the wrong sign for the cat (because $(x_{\text{dog}})_2$ and $(x_{\text{cat}})_2$ have opposite signs). This increases the cat test loss — negative influence. The sign flip is sharpest around t_2^*, when $g_t'(s_2)$ peaks and the mode-2 residual transitions from large to small.

SOLUTIONS FOR CHAPTER B.6

Physics of Deep Learning

Aleksander Czejdo, Tudor Dimofte, Brianna Grado-White, Charles Renshaw-Whitman

Main text: Chapter B.6, p. 141

Exercise 2.1: Solution p. 150

(a) Differentiating under the integral, $\partial_J \log Z = \frac{1}{Z}\int d\theta\, \mathcal{O}\, e^{-\beta E + J\mathcal{O}} = \mathbb{E}_J(\mathcal{O})$, which at $J=0$ is $\mathbb{E}(\mathcal{O})$; since $F = -\log Z$, this is $-\partial_J F|_0$. Once more, $\partial_J^2 \log Z = \mathbb{E}_J(\mathcal{O}^2) - \mathbb{E}_J(\mathcal{O})^2 = \mathrm{Var}_J(\mathcal{O})$.

(b) Using $\partial_J \mathbb{E}_J(\mathcal{O}^k) = \mathbb{E}_J(\mathcal{O}^{k+1}) - \mathbb{E}_J(\mathcal{O}^k)\mathbb{E}_J(\mathcal{O})$ on $\partial_J^2 \log Z = \mathbb{E}_J(\mathcal{O}^2) - \mathbb{E}_J(\mathcal{O})^2$ gives $\kappa_3 = \mathbb{E}(\mathcal{O}^3) - 3\mathbb{E}(\mathcal{O}^2)\mathbb{E}(\mathcal{O}) + 2\mathbb{E}(\mathcal{O})^3$. Expanding $\mathbb{E}[(\mathcal{O}-\mathbb{E}\mathcal{O})^3] = \mathbb{E}(\mathcal{O}^3) - 3\mathbb{E}(\mathcal{O}^2)\mathbb{E}(\mathcal{O}) + 3\mathbb{E}(\mathcal{O})\mathbb{E}(\mathcal{O})^2 - \mathbb{E}(\mathcal{O})^3$ gives the same expression.

(c) For $\mathcal{O} \sim \mathcal{N}(\mu, \sigma^2)$ (or more generally when $\mathcal{O}$ is Gaussian under p_β), $\mathbb{E}(e^{J\mathcal{O}}) = e^{J\mu + J^2\sigma^2/2}$, so $\log Z(J) = \log Z(0) + J\mu + \frac{1}{2}J^2\sigma^2$ is quadratic and every derivative beyond the second vanishes. One more derivative of the pattern in (b), with $\bar{\mathcal{O}} := \mathcal{O} - \mathbb{E}\mathcal{O}$, gives $\kappa_4 = \mathbb{E}(\bar{\mathcal{O}}^4) - 3\,\mathbb{E}(\bar{\mathcal{O}}^2)^2$; for a Gaussian $\mathbb{E}(\bar{\mathcal{O}}^4) = 3\sigma^4$, so $\kappa_4 = 0$ while the fourth centered moment is not.

(d) $\partial_\beta \log Z = -\frac{1}{Z}\int E\, e^{-\beta E} = -\mathbb{E}(E)$, so $\partial_\beta F = \mathbb{E}(E)$; $\partial_\beta^2 \log Z = \mathbb{E}(E^2) - \mathbb{E}(E)^2$, so $-\partial_\beta^2 F = \mathrm{Var}(E)$; and $\partial_\beta^3 \log Z = -\kappa_3(E)$ by the same computation as (b) with $J \to -\beta$, so $\kappa_3(E) = \partial_\beta^3 F$.

Exercise 2.2: Solution p. 158

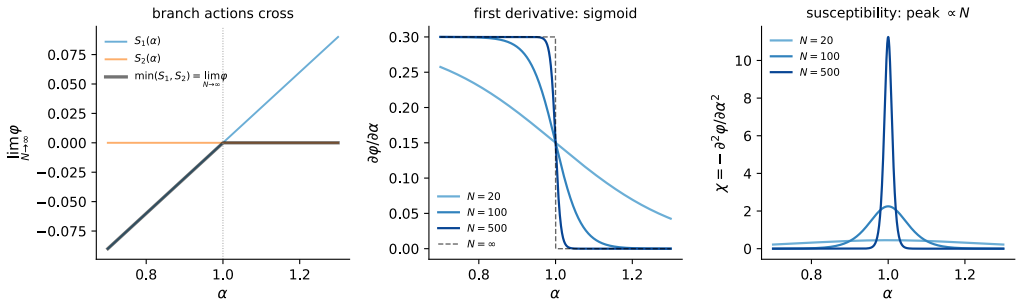
(a) p_i is the fraction of Z contributed by basin i. With $\varphi_1 - \varphi_2 \approx \Delta\varphi'\,\Delta B$ near the crossing, $p_2 = (1 + e^{-N(\varphi_1-\varphi_2)})^{-1} \approx (1 + e^{-N\Delta\varphi'\Delta B})^{-1}$. The switch happens over the window $|\Delta B| \lesssim 1/(N\Delta\varphi')$, which shrinks with system size.

(b) Differentiating $\varphi = -\frac{1}{N}\log(e^{-N\varphi_1} + e^{-N\varphi_2})$ gives $\partial_B \varphi = p_1\varphi_1' + p_2\varphi_2'$ exactly; near B_c the slopes are $\approx \varphi_i'(B_c)$. Directly from the integral, $\partial_B\varphi = -\frac{1}{N}\partial_B \log Z = \frac{1}{Z}\int d\theta\, \partial_B E\, e^{-NE} = \mathbb{E}[\partial_B E]$: the control parameter is itself a source, coupling to $\mathcal{O} = \partial_B E$. *Sketch:* at finite N, $\partial_B\varphi$ is a sigmoid interpolating from $\varphi_1'(B_c)$ to $\varphi_2'(B_c)$ over the window of (a); as $N\to\infty$, $\varphi \to \min(\varphi_1,\varphi_2)$, continuous with a kink at B_c, and $\partial_B\varphi$ becomes a step of height $\Delta\varphi'$.

(c) $\partial_B p_1 = \partial_B \frac{e^{-N\varphi_1}}{e^{-N\varphi_1}+e^{-N\varphi_2}} = -Np_1p_2(\varphi_1' - \varphi_2')$ and $\partial_B p_2 = -\partial_B p_1$. Then $\partial_B^2\varphi = (\partial_B p_1)\varphi_1' + (\partial_B p_2)\varphi_2' + p_1\varphi_1'' + p_2\varphi_2'' = -Np_1p_2(\Delta\varphi')^2 + p_1\varphi_1'' + p_2\varphi_2''$. *Sketch:* a smooth background from the φ_i'' terms plus a peak at B_c of height $\approx N(\Delta\varphi')^2/4$ and

width $\sim 1/(N\Delta\varphi')$, the nascent delta function of the $N=\infty$ kink. For the variance identity: with $\mathcal{O}=\partial_B E$ independent of B, differentiating $\partial_B\varphi=\mathbb{E}(\mathcal{O})$ once more gives $\partial_B^2\varphi=-N[\mathbb{E}(\mathcal{O}^2)-\mathbb{E}(\mathcal{O})^2]$, so $-\partial_B^2\varphi=N\,\mathrm{Var}(\mathcal{O})\geq 0$, which is why the curve is a peak and not a dip. The two-branch formula exhibits the leading piece of this variance: $\mathcal{O}\approx\varphi_1'$ in one basin and $\approx\varphi_2'$ in the other, and the variance of that two-point mixture is exactly $p_1p_2(\Delta\varphi')^2$.

(d) See Figure 14. The peak in the right panel grows in proportion to N; a crossover's bump would saturate.



Anatomy of a first-order transition at finite N, in the minimal two-branch model $\varphi_1(B)=0.3\,(B-B_c)$, $\varphi_2\equiv 0$, with the control parameter written as α. Left: the branch free energies cross at B_c; the limiting free-energy density is their minimum, which has a kink. Middle: at finite N, $\partial_B\varphi$ is the probability-weighted average of the branch slopes (2.36), a sigmoid of width $\propto 1/N$. Right: the susceptibility (2.37) peaks at B_c with height $\propto N$ and width $\propto 1/N$.

Exercise 2.3: Solution p. 160

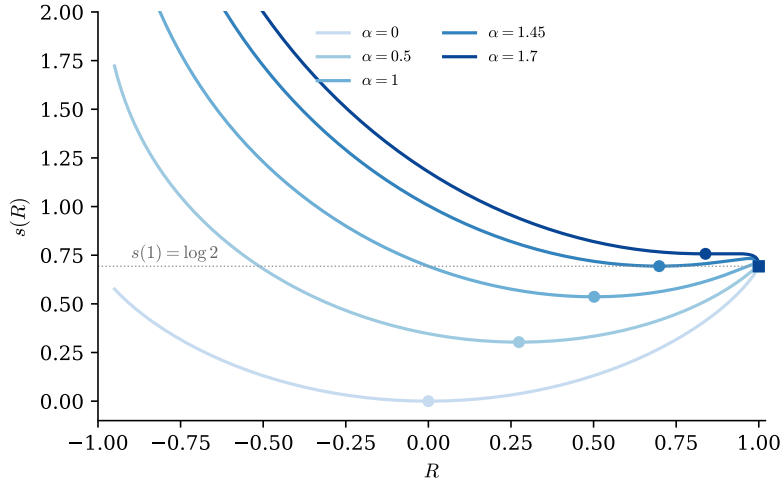
(a) $R=\frac{1}{N}\sum_i\theta_i\theta_i^*=\frac{1}{N}\sum_i s_i$ since $(\theta_i^*)^2=1$. The map $\theta\mapsto s$ is a bijection of the hypercube (a "gauge transformation"), so the uniform measure goes to the uniform measure, and the perceptron's order parameter is literally a magnetization.

(b) Both labels depend only on the projection of x onto $\mathrm{span}(\theta,\theta^*)$, which is an isotropic two-dimensional Gaussian. The decision boundaries are two diameters at angle $\vartheta=\arccos R$; the signs disagree exactly on the double wedge between them, of angular fraction $2\vartheta/2\pi=\vartheta/\pi$. Note that the answer does not depend on N, and holds for any θ with $|\theta|^2=N$, binary or not.

(c) A student at overlap R agrees with the teacher on exactly $\frac{1+R}{2}N$ coordinates ($s_i=+1$), giving the binomial count; Stirling gives $h(R)$, the binary entropy at magnetization R. Note $h(0)=\log 2$ and $h(1)=0$: the latter is *finite*, because exactly one state sits at $R=1$.

(d) Independence over examples gives the n-th power. Trading the sum for an integral, $\frac{1}{2^N}\sum_\theta\to\int dR\,e^{N[h(R)-\log 2]}$, and $(1-\varepsilon)^n=e^{N\alpha\log(1-\varepsilon)}$; collecting exponents gives $s(R)$ as stated. (The $\log 2$ normalizes $s(0)=0$ at $\alpha=0$.)

(e) The curves are shown in Figure 15. At $\alpha = 0$, $s = \log 2 - h(R)$ has a single minimum at $R = 0$, the bulk of the hypercube. As α grows, the interior minimum drifts toward larger R and its value rises (note $s(0) = \alpha \log 2$), while $R = 1$ stays pinned at $s(1) = \log 2$. For α large enough there are two competing local minima separated by a barrier: exactly the two-basin structure of Exercise 2.2. One should therefore expect a first-order transition at the load where the interior minimum's value crosses $\log 2$, near $\alpha \approx 1.45$ from the plot; at $\alpha = 1.7$ the interior minimum survives only as a metastable dip above $\log 2$. No transition is expected at small α, where the interior minimum is the only relevant one and moves smoothly.

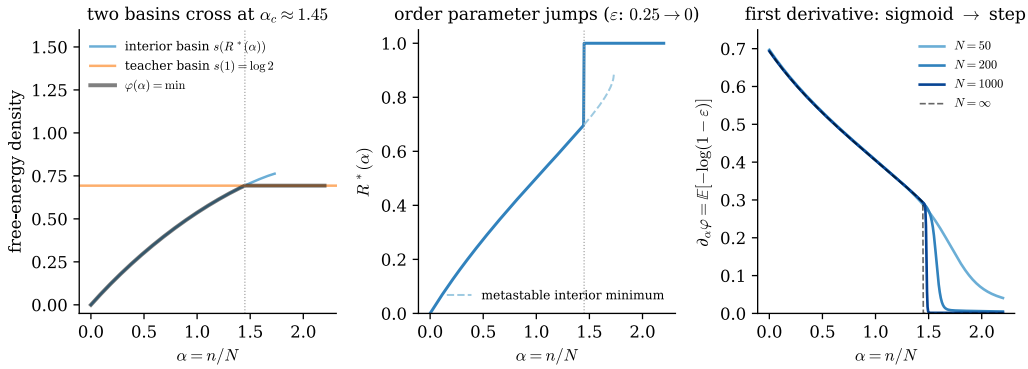


The annealed action $s(R)$ of the Ising perceptron, (2.44), for several loads. Interior local minima are marked with circles; the teacher state $R = 1$, where $s(1) = \log 2$ exactly, with squares. The two basins exchange dominance near $\alpha \approx 1.45$.

(f) At $R = 1$: $h(1) = 0$ and $\varepsilon(1) = 0$, so $s(1) = \log 2$ for all α. Near $R = 1$ with $\delta = 1 - R$: $h(1-\delta) = \frac{\delta}{2}\log\frac{2}{\delta} + O(\delta)$, while $\arccos(1-\delta) \approx \sqrt{2\delta}$ gives data cost $-\alpha \log(1-\varepsilon) \approx \alpha\sqrt{2\delta}/\pi$. Since $\sqrt{\delta} \gg \delta \log(1/\delta)$ as $\delta \to 0$, $s(1-\delta) > s(1)$: a basin around the teacher at every $\alpha > 0$. It is protected by the singular cost of the first errors, which is what a smooth well cannot provide.

(g) Minimizing s over the interior numerically (Figure 16, left), the interior branch value crosses $s(1) = \log 2$ at $\alpha_c \approx 1.45$, with minimizer $R^* \approx 0.70$ and $\varepsilon^* = \arccos(0.70)/\pi \approx 0.25$. The order parameter jumps from 0.70 to 1 (middle panel). With α as the control parameter of Exercise 2.2: $\varphi(\alpha) = \min_R s(R)$ has a kink at α_c; at finite N, $\partial_\alpha \varphi = \mathbb{E}[-\log(1-\varepsilon(R))]$ is a sigmoid interpolating between the branch slopes $-\log(1-\varepsilon(R^*)) \approx 0.29$ and 0, over a window of width $\sim 1/(N\Delta\varphi')$ (right panel), and $-\partial_\alpha^2 \varphi$ is a peak of height $\propto N$. The role of the latent heat is played by the jump in $-\log(1-\varepsilon)$, the per-example log-likelihood released when the student snaps onto the teacher. Past α_c the interior minimum survives as a local minimum until the *spinodal* $\alpha \approx 1.73$, where it merges with the barrier and disappears. In the window $1.45 \lesssim \alpha \lesssim 1.73$ a local learning dynamics (flipping a few weights at a time to improve

alignment) can stay stuck at $\varepsilon \approx 0.25$ even though perfect generalization is thermodynamically dominant: the information needed to identify the teacher is present in the data before the dynamics can find it, and generalization arrives abruptly and later than the data first permits. This is the metastability of Section 2.5, and a prototype of the delayed transitions of Section 5.7. (The exact, quenched treatment (Györgyi 1990; Engel & Van den Broeck 2001) moves the transition to $\alpha_c \approx 1.245$ but changes nothing structurally.)



The Ising perceptron's phase transition. Left: the free-energy densities of the interior basin, $s(R^*(\alpha))$, and of the teacher basin, $s(1) = \log 2$, cross at $\alpha_c \approx 1.45$; the limiting $\varphi(\alpha)$ is their minimum, with a kink. Middle: the order parameter $R^*(\alpha)$ jumps from 0.70 to 1; the dashed continuation is the metastable interior minimum, which disappears at the spinodal $\alpha \approx 1.73$. Right: at finite N, $\partial_\alpha \varphi = \mathbb{E}[-\log(1-\varepsilon)]$, computed from the integral (2.44), is a sigmoid sharpening into the step of height ≈ 0.29.

(h) Field $B \leftrightarrow$ the data term, the per-example log-likelihood $\log(1-\varepsilon(R))$, with strength α; the knob swept $=$ the load $\alpha = n/N$, the amount of data; the ordered phase $\leftrightarrow$ perfect generalization $R = 1$; the jump in $\mathbb{E}(m)$ as a function of $B \leftrightarrow$ the discontinuous drop of $\varepsilon(R^*)$ from 0.25 to 0 as a function of α. The structural difference: in the magnet the two competing minima are symmetry partners, while here they are an entropy-dominated basin and a data-dominated basin. The ingredient that makes the sharp transition possible is $h(1) = 0$ being *finite*: discreteness gives the perfectly aligned state a finite entropy, so it can compete as a separate basin. For spherical weights the slice entropy $\frac{1}{2}\log(1-R^2)$ diverges to $-\infty$ as $R \to 1$, the perfectly aligned state is infinitely penalized, the minimum always sits in the interior and slides continuously with α, and there is no transition; the generalization error then decays smoothly, as $\varepsilon \simeq 0.625/\alpha$ in the exact treatment (Appendix A.3).

Exercise 3.1: Solution p. 165

Complete the square: shifting $f \to f + KJ$ (schematically, $f(x) \to f(x) + \int dx' K(x,x') J(x')$) decouples the source, leaving $Z[J] = Z[0]\, e^{\frac{1}{2}\int JKJ}$. Each derivative $\delta/\delta J(x^i)$ acting on the exponential either brings down a factor $\int K(x^i,\cdot)J$ (which must later be hit by another derivative, else it dies at $J = 0$) or closes a pair by hitting a

previously produced factor, contributing $K(x^i, x^j)$. Every surviving term therefore corresponds to a perfect pairing of $\{1, \dots, p\}$. For $p = 2$ this gives $K(x^1, x^2)$; for $p = 4$, the three pairings $K^{12}K^{34} + K^{13}K^{24} + K^{14}K^{23}$; for $p = 6$ there are $5!! = 15$ terms; in general $(p-1)!!$.

Exercise 3.2: Solution p. 168

Everything is closed-form thanks to the erf kernel (3.32), which is the point of choosing erf. Expected observations:

(a) at $N = 3$ the histogram has visibly heavy tails and excess kurtosis (the output is a sum of only three i.i.d. non-Gaussian terms); at $N = 30$ deviations are still detectable; at $N = 300$ the histogram is indistinguishable from the predicted Gaussian at this sample size.

(b) The sample covariance matches the 2×2 kernel matrix built from (3.32) to within the Monte Carlo error $O(M^{-1/2})$.

(c) A log–log plot of $\mathrm{Var}(K^{(2)})$ against N has slope -1, as in the variance computation of Section 3.4. The whole lab is $\sim$20 lines of NumPy; the one subtlety is to histogram over *initializations* at fixed inputs, not over inputs.

Exercise 3.3: Solution p. 172

Representative numbers at the time of writing (take the width N to be the model/embedding dimension): GPT-2 small has $L/N = 12/768 \approx 0.016$ and GPT-2 XL $48/1600 = 0.030$; Llama-2-7B has $32/4096 \approx 0.008$; Llama-2-70B, $80/8192 \approx 0.010$; Llama-3-405B, $126/16384 \approx 0.008$. The striking finding is how *stable* the ratio is: modern open-weight families cluster around $L/N \sim 0.005$–0.01 across two orders of magnitude in parameter count, comfortably inside the "controlled correlations" window, with the earliest (GPT-2-era) models running noticeably deeper for their width. Whether this stability reflects tuned wisdom or copied convention is a fair discussion question.

Exercise 4.1: Solution p. 176

By the chain rule and the gradient-flow equation,

$$\begin{aligned}\frac{d}{dt} f^\mu &= \nabla_\theta f^\mu \cdot \dot\theta = -\eta\, \nabla_\theta f^\mu \cdot \nabla_\theta \mathcal{L} \\ &= -\eta\, \nabla_\theta f^\mu \cdot \sum_\nu \nabla_\theta f^\nu \, \frac{\partial \mathcal{L}^{(f)}}{\partial f^\nu} = -\eta\, \Theta^{\mu\nu}\, \frac{\partial \mathcal{L}^{(f)}}{\partial f^\nu}\,,\end{aligned}$$

using in the middle step that $\mathcal{L}$ depends on θ only through the function values f^ν.

Exercise 4.2: Solution p. 176

Differentiate $\Theta^{\mu\nu} = \nabla_\theta f^\mu \cdot \nabla_\theta f^\nu$ along the flow: $\frac{d}{dt}\Theta^{\mu\nu} = \dot\theta \cdot \nabla_\theta^2 f^\mu \cdot \nabla_\theta f^\nu + \nabla_\theta f^\mu \cdot \nabla_\theta^2 f^\nu \cdot \dot\theta$. Substituting $\dot\theta = -\eta\, \nabla_\theta f^\rho\, \partial\mathcal{L}^{(f)}/\partial f^\rho$ gives the stated formula. Taking operator norms, $\|\dot\Theta\| \lesssim \eta\, \|\nabla_\theta^2 f\|\, \|\nabla_\theta f\|^2\, |\partial\mathcal{L}/\partial f| \sim \eta\, \|\nabla_\theta^2 f\|\, \|\Theta\|\, |\Delta|$: the kernel can only move as fast as the Hessian of the *network function* (not of the loss) allows — which is why bounding that Hessian at large width freezes the kernel.

Exercise 4.3: Solution p. 181

In the linearized regime, $f_{\theta(t)}(x) = f_{\theta_0}(x) + \nabla_\theta f(x) \cdot (\theta(t) - \theta_0)$ with all gradients at θ_0. The parameter motion integrates the flow: $\dot\theta = -\eta \sum_\mu \nabla_\theta f^\mu\, \Delta^\mu(t)$ with $\Delta(t) = e^{-\eta t\Theta}\Delta(0)$ from (4.34), so

$$\begin{aligned}\theta(t) - \theta_0 &= -\eta \sum_\mu \nabla_\theta f^\mu \int_0^t ds\, \left(e^{-\eta s\Theta}\right)^\mu{}_\rho\, \Delta^\rho(0) \\ &= -\sum_\mu \nabla_\theta f^\mu \left[\Theta^{-1}\left(\mathbb{1} - e^{-\eta t\Theta}\right)\right]^\mu{}_\rho \Delta^\rho(0)\,.\end{aligned}$$

Contracting with $\nabla_\theta f(x)$ and recognizing $\Theta(x, x^\mu) = \nabla_\theta f(x) \cdot \nabla_\theta f(x^\mu)$ yields (4.36). (The test–train kernel is constant during training for the same reason the train–train one is: it is built from the frozen gradients at θ_0.)

Exercise 4.4: Solution p. 182

Reference results for the setup as specified (single seeds; small-N numbers fluctuate).

(a) Final loss $\sim 10^{-5}$–10^{-4}. Keep $\eta < 2/s^2_{\max}(\Theta_0)$ for stability; here $s^2_{\max} \approx 30$, so $\eta = 0.02$ is safe.

(b) At $x = 1$, $f_0 \sim \mathcal{N}(0, O(1))$, and the sample covariance across two inputs matches the tanh GP kernel.

(c) Measured $\|\Delta\Theta\|/\|\Theta_0\| \approx 0.25,\ 0.14,\ 0.06$ at $N = 100, 300, 1000$: decaying, with a log–log slope between $-1/2$ and -1; average a few seeds to clean up the trend. The relative parameter displacement decays like $N^{-1/2}$, since the total displacement is $O(1)$ by (4.30) while $|\theta_0| \sim \sqrt{P} \sim \sqrt{N}$. Compute Θ from per-example gradients (an $n \times P$ Jacobian contracted with itself), in float64.

(d) The residual modes are straight lines on a semilog plot, with slopes $-\eta s_k^2$, until they hit the small nonlinear floor; the smallest-eigenvalue modes are visibly not yet learned at the end of training. This is spectral bias on display, and the dynamical twin of Exercise 4.5(b).

(e) At $N = 300$ the trained function on test points agrees with (4.36) to within a few percent; the agreement degrades as N shrinks, which is feature learning beginning to stir. The continuation is Exercise 5.3.

Exercise 4.5: Solution p. 184

(a) Completing the square, the exponent is $-\frac{1}{2}\mathbf{f}^T A^{-1}\mathbf{f} + \beta\,\mathbf{y}^T\mathbf{f} + \text{const}$ with $A^{-1} = \hat{K}^{-1} + \beta I_n$, so the posterior is Gaussian with covariance A and mean $\boldsymbol{\mu} = \beta A\mathbf{y} = \beta(\hat{K}^{-1} + \beta I_n)^{-1}\mathbf{y} = \hat{K}(\hat{K} + \beta^{-1} I_n)^{-1}\mathbf{y}$ (multiply through by $\hat{K}$).

(b) In the eigenbasis all matrices are diagonal, giving (4.46). The filter $\beta\lambda_k/(1+\beta\lambda_k)$ is ≈ 0 for $\beta \ll 1/\lambda_k$ and ≈ 1 for $\beta \gg 1/\lambda_k$: a switch at $\beta_k = 1/\lambda_k$. On a log β axis it is a sigmoid of universal shape, pleasant to compare with the logistic occupations of Section 2.5, though nothing here is a phase transition: the switches are n smooth crossovers, one per mode.

(c) The same: frozen-kernel linear structure, mode-by-mode learning, spectral bias with scale $1/\lambda_k$, and the dictionary "$\beta \approx$ training time" of Section 2.1 made quantitative as $\beta \leftrightarrow \eta t$. Different: the suppression of the unlearned part is $1/(1+\beta\lambda_k)$, a power law in β, versus $e^{-\eta s_k^2 t}$, exponential in t (the two filters of Figure 13); and the kernels differ. Bayesian learning of the frozen network is governed by the GP kernel K, a property of the *function* distribution at initialization, while gradient flow is governed by the NTK Θ, a property of the *parametrization*. Both are fixed, data-independent kernels: in neither picture does the lazy network adapt its features.

(d) Varying the exponent of $P[f]$ with respect to $f(x)$ gives the stationarity equation $\int dx'\, K^{-1}(x,x')\mu(x') + \beta\sum_\mu(\mu(x^\mu) - y^\mu)\,\delta(x - x^\mu) = 0$, that is $\mu(x) = -\beta\sum_\mu K(x,x^\mu)(\mu(x^\mu) - y^\mu)$. Evaluating at the training points gives a closed linear system whose solution is (a), and substituting back yields the stated $\mu(x)$. The covariance is the inverse of the quadratic form, $\Sigma = (K^{-1} + \beta\,\delta_{\text{data}})^{-1}$, which the Woodbury identity converts into the stated form. As $\beta \to \infty$ this pins $\mu(x^\mu) = y^\mu$: noiseless GP interpolation; finite β is ridge regularization with ridge β^{-1}.

Exercise 5.1: Solution p. 186

The iteration converges geometrically to a stable fixed point, at rate set by the slope $\beta\,(1 - m^2)$ of the right-hand side there. For $\beta < 1$ the only fixed point is $m = 0$ and it is stable. For $\beta > 1$ the fixed point $m = 0$ has slope $\beta > 1$ and is unstable: any seed away from it flows to one of the two nonzero fixed points $\pm m^*$, which are stable, and a small B selects the one with the sign of B (the other survives as a stable fixed point too, the metastable state of Section 2.5). Its infinite-dimensional cousin is the fixed-point iteration for the McKean–Vlasov equation in Section 5.2.

Exercise 5.2: Solution p. 190

(a) Expanding the square in the loss and collecting single-sum and double-sum terms gives (5.12)–(5.13) directly. For the equations of motion, $\partial f_{\rho_N}(x^\mu)/\partial c_i = \frac{1}{N}\phi(a_i \cdot x^\mu)$ and $\partial f_{\rho_N}(x^\mu)/\partial a_i = \frac{1}{N} c_i \phi'(a_i \cdot x^\mu) x^\mu$; the chain rule plus $\eta = N\tilde{\eta}$ gives (5.15). Writing the same gradient through Ψ, the symmetry $U(\theta,\theta') = U(\theta',\theta)$ ensures the double sum contributes twice the one-sided derivative, matching (5.16).

(b) From (5.15): $c_i\dot{c}_i = -\tilde{\eta}\, c_i \sum_\mu \Delta^\mu \phi(a_i \cdot x^\mu)$ and $a_i \cdot \dot{a}_i = -\tilde{\eta}\, c_i \sum_\mu \Delta^\mu\, \phi'(a_i \cdot x^\mu)\,(a_i \cdot x^\mu)$. For positively homogeneous ϕ, $\phi'(z)z = \phi(z)$, so the two right-hand sides coincide and $\frac{d}{dt}\left(c_i^2 - |a_i|^2\right) = 2(c_i\dot{c}_i - a_i \cdot \dot{a}_i) = 0$.

(c) Because f_ρ is *linear* in the output weights, $\partial f/\partial c_i$ contains no c_i; the only route by which c_i influences $\dot{c}_i$ is through the mean field Δ^μ — i.e. through the population average, weighted by $1/N$. This linearity is also what makes the loss a quadratic (rather than higher-order) polynomial in each particle, i.e. what limits the interaction to be pairwise.

Exercise 5.3: Solution p. 193

Reference results (single seeds).

(a) Final loss $\sim 5 \times 10^{-3}$ at $\tau = 20$ and still slowly falling; the mean-field run is measured in rescaled time, so run longer to go lower.

(b) At $x = 1$, $f_0 \sim \mathcal{N}(0, O(1/N))$, with standard deviation falling like $1/\sqrt{N}$ across N : the Gaussian process of Section 3 survives only as the $O(1/\sqrt{N})$ fluctuation around zero.

(c) Measured $\|\Delta\Theta\|/\|\Theta_0\| \approx 1.7,\ 0.8,\ 0.7,\ 0.9$ at $N = 30, 100, 300, 1000$: order one, with no decay, against the decaying lazy curve. Use the same Θ formula with the appropriate γ in both runs.

(d) The mean-field cloud reproduces Figure 6: neurons stream along order-one trajectories and pile onto one-dimensional filaments through $(a, c) \approx$ (teacher directions, matched output weights), with the point symmetry $(a, c) \to (-a, -c)$ from the oddness of $\tanh$. The lazy cloud is visually identical at $\tau = 0$ and $\tau = 20$ (displacements $\sim 1/\sqrt{N}$).

(e) For $y = \sin(2x)$ or $|x| - 1$ the loss still falls (more slowly), and the cloud condenses onto *different*, task-shaped structures, with more filaments spread over a range of slopes a, since no finite set of teacher neurons exists to copy; the kernel still moves by $O(1)$. That is the cleaner demonstration that mean-field training *builds* features rather than merely locating the teacher's. Mismatched activations behave similarly. The lazy runs, by contrast, change in only one way, the quality of the fit (the random features either happen to span the target well or do not); the kernel and the cloud never adapt.

Exercise 5.4: Solution p. 195

(a) $\partial f/\partial c_i = N^{-\gamma}\phi(a_i \cdot x)$ and similarly for a_i, so $\Theta = \sum_i N^{-2\gamma}\, k_{\theta_i} = N^{1-2\gamma} \cdot \frac{1}{N}\sum_i k_{\theta_i} = O(N^{1-2\gamma})$; demanding $\eta\,\Theta = O(1)$ forces $\eta = N^{2\gamma-1}\tilde{\eta}$.

(b) At initialization $f = N^{-\gamma}\sum_i c_i\phi(a_i \cdot x)$ is $N^{-\gamma}$ times a sum of N i.i.d. mean-zero terms $= O(\sqrt{N})$, so $f_{\theta_0} = O(N^{1/2-\gamma})$. Per-neuron velocities: $\dot{c}_i = -\eta\,\partial\mathcal{L}/\partial c_i = O(N^{2\gamma-1} \cdot N^{-\gamma}) = O(N^{\gamma-1})$, and likewise for $\dot{a}_i$.

(c) The normalized kernel $N^{2\gamma-1}\Theta = \frac{1}{N}\sum_i k_{\theta_i}$ is an empirical average over particles, so over any fixed horizon its drift is of the order of the per-particle displacement, $O(N^{\gamma-1})$ — vanishing for every $\gamma < 1$, order one exactly at $\gamma = 1$.

(d) At $\gamma = \frac{1}{2}$: per-neuron motion $O(N^{-1/2})$, matching (4.30) (an $O(1)$ total displacement shared among N neurons); initialization output $O(1)$, the GP of Section 3. For the optional part: relative to the mean-field parametrization (5.7), $f_\gamma = \alpha\, f_{\mathrm{MF}}$ with $\alpha = N^{1-\gamma}$, so in the language of (Chizat et al. 2019) every $\gamma < 1$ sends the laziness $\alpha \to \infty$ with width — lazy — while $\gamma = 1$ keeps α of order one.

Exercise 5.5: Solution p. 198

(a) Finite network, deterministic gradient descent/flow, no limit required: the analysis is organized around the *linearization* of Section 4.3 and departures from it. The dial is an output multiplier α (with appropriate co-scaling of the loss/learning rate), exactly the scale parameter of lazy training (Chizat et al. 2019): at large α an $O(1)$ change of f requires only an $O(1/\alpha)$ change of weights, so features barely move — a continuous, finite-N version of the γ-dial, where $N^{\gamma-1}$ played the role of $1/\alpha$ (Exercise 5.4). And this paper is precisely about the half of the scaling story that the Bayesian treatment of Exercise 5.6(a) could not see: there, only the induced function prior survived and the learning-rate/dynamics half dropped out; here there is no prior at all, and *everything* is the dynamics half — the two walkthroughs literally partition Section 5.2's prescription between them.

(b) The alignment between the (current) NTK and the task — equivalently the kernel's distance from its initialization, or the paper's low-dimensional sufficient statistics: in the memorized state the kernel is still (close to) its initialization and the fit lives in misaligned modes; in the generalizing state the kernel's top eigenspaces have rotated onto the target. Measurable in any run by tracking the empirical NTK (Section 4.2). The Fourier overlap $\Phi(w)$ of Exercise 5.6(b) carries the same information — features aligned with the task's structure — read off the weights and their posterior rather than off the kernel; for modular addition, a rotated kernel and condensed Fourier modes are two descriptions of the same event.

(c) (i) *The top eigenvectors of the initial NTK are misaligned with the labels*: in the language of Section 4.6, the target's weight sits in modes with small eigenvalues, so the lazy (kernel-regression) solution fits the training points without generalizing — memorization, in the precise mode-filtering sense. (ii) *The dataset is in a Goldilocks window*: large enough that a generalizing solution exists (cf. the critical sample size of Exercise 5.6(d)), small enough that train and test loss do not simply track each other. (iii) *Training starts lazy* (large α, or large initialization scale): feature learning is initially suppressed, as in Section 4.3, so the kernel fit happens *first*.

(d) The laziness α (and the initialization scale, which acts similarly): increasing it stretches the delay between the lazy fit and feature learning, making grokking more dramatic; decreasing it makes the network rich from the start, so train and test fall together and grokking *vanishes*. In the extreme $\alpha \to \infty$ the network is a pure kernel

machine forever: by condition (i) it never generalizes, and the grokking time diverges. Contrast Exercise 5.6(d): there the knobs were sample size and temperature moving the system across a phase boundary; here the knob does not change *where* the system ends up so much as *how long* the misaligned detour lasts.

(e) From a separation of timescales built into the flow: fitting the train set within the frozen-kernel subspace is fast (the exponential convergence of Section 4, eq. (4.34)), while rotating the kernel is slow — suppressed by the laziness, since feature motion is $O(1/\alpha)$-small per unit of function change. Train loss bottoms out on the fast clock; test loss must wait for the slow one. Nothing stochastic is needed: no noise, no weight decay, no barrier crossing — and that is the force of the paper's counterexamples, since grokking with zero weight decay and *growing* weight norm rules out explanations in which regularization-driven norm shrinkage causes the transition. The equilibrium mechanism of Section 5.7, by contrast, needs temperature: its delay is the escape from a metastable free-energy basin, which pure gradient flow would never make.

(f) They live in different corners and need not compete: Rubin et al. describe noisy (Langevin/Bayesian) training near equilibrium, Kumar et al. deterministic training far from it; both say *the cheap channel is used first — kernel fit or metastable memorizing phase — and features arrive late*. Distinguishing experiments:

1. *noise dependence* — an activated (metastable-escape) delay should shorten rapidly with increasing training noise, roughly Arrhenius-like, while a timescale-separation delay survives at zero noise and depends on it only weakly; measure grokking time vs. Langevin temperature at fixed α, n.
2. *Seed-to-seed statistics* — nucleation out of a metastable state is a rare-event process with broad (roughly exponential) waiting-time distributions across seeds; a deterministic mechanism predicts narrow, initialization-scale-set delays.
3. *The α-dial itself* — the lazy-to-rich account makes a sharp prediction that the delay is monotonically stretched by α with grokking eliminated at small α; the equilibrium account ties the delay to the free-energy landscape at fixed ensemble, not to α. Running these on modular addition is a very feasible project.

Exercise 6.1: Solution p. 201

(a) The white-noise measure is Gaussian with $\langle \xi(t)\xi(t')\rangle = D\,\delta(t-t')$, so $\left\langle e^{-i\int \tilde{x}\,\xi}\right\rangle = e^{-\frac{D}{2}\int \tilde{x}^2}$, and collecting terms in the exponent gives (6.7).

(b) The linear SDE is solved by $x(t) = \int_{-\infty}^{t} ds\; e^{-k(t-s)}\,\xi(s)$ (stationary state). Then $C(\tau) = \int_{-\infty}^{t} ds \int_{-\infty}^{t'} ds'\, e^{-k(t-s)-k(t'-s')} D\,\delta(s-s') = \frac{D}{2k} e^{-k|\tau|}$ with $\tau = t - t'$. Adding a perturbation h to the drift shifts the solution by $\int_{-\infty}^{t} ds\, e^{-k(t-s)} h(s)$, so $R(\tau) = \theta(\tau)\, e^{-k\tau}$.

(c) For $\tau > 0$: $-\partial_\tau C = \frac{D}{2} e^{-k\tau} = \frac{D}{2}\, R(\tau)$, i.e. $TR = -\partial_\tau C$ with $T = D/2$.

(d) In the discrete construction each delta function propagates information strictly forward in time (Ito: the noise at step $i-1$ affects x_i, never earlier variables). Diagrammatically, $\langle x\tilde{x}\rangle$ is a retarded propagator; $\langle \tilde{x}\tilde{x}\rangle$ would be a closed loop of retarded propagators, which vanishes — equivalently, "a response to no perturbation is not an observable." Causality of R is the same statement.

Exercise 6.2: Solution p. 206

(a) With full batches and $\lambda = 0$, gradient descent converges for $\eta < 2/\lambda_{\max}(X^T X/P)$; keep η a safe factor below. The late-time errors match (6.18) and (6.19), and plotting $\varepsilon_g(t \to \infty)$ against ν traces the double-descent peak at $\nu = 1$ from both sides.

(b) The clone separation obeys the *linear* dynamics $\Delta\hat{w}(t+1) = \big(I - \eta\, X^T X/P\big)\,\Delta\hat{w}(t)$: it decays iff the sampled covariance $X^T X$ has full rank, i.e. iff $n \geq P$ ($\nu \leq 1$). For $\nu > 1$ the covariance has a null space of dimension $P - n$, and the projection of $\Delta\hat{w}(0)$ onto it survives forever; for an isotropic perturbation the expected surviving fraction is $(P-n)/P = (\nu-1)/\nu$ — precisely the paper's ergodicity-breaking degree $R(\tau \to \infty)$.

(c) The relaxation time is set by the smallest nonzero eigenvalue of $X^T X/P$, which by Marchenko–Pastur closes like $(1 - \sqrt{1/\nu})^2 \sim |1-\nu|^2/4$ near threshold, so one expects $\phi \approx 2$; compare with the paper's measured exponent and with the collapse ansatz Eq. (6.21).

Exercise 6.3: Solution p. 207

(a) The linearized equation $\dot{\delta x} = f'(x_0)\,\delta x + \xi$ is an Ornstein–Uhlenbeck process with $k = -f'(x_0) > 0$; by Exercise 6.1(b) its stationary variance is $D/(2k) = D/(-2f'(x_0))$.

(b) Taking the stationary average of $\dot{x} = f(x) + \xi$ with f expanded to second order about x_0: $0 = f(x_0) + f'(x_0)\langle\delta x\rangle + \frac{1}{2}f''(x_0)\langle\delta x^2\rangle$, and using $f(x_0) = 0$, $\langle x\rangle = x_0 - \frac{f''(x_0)}{2f'(x_0)}\langle\delta x^2\rangle = x_0 + \frac{1}{2}\frac{f''(x_0)}{f'(x_0)}\cdot\frac{D}{2f'(x_0)}$, the quoted shift — nonzero, contradicting the assumption $\langle x\rangle = x_0$.

(c) Demand instead that the expansion point x_* satisfy the *effective* stationarity condition $f(x_*) + \frac{1}{2}f''(x_*)\,\langle\delta x^2\rangle_* = 0$ together with $\langle\delta x^2\rangle_* = D/(-2f'(x_*))$. These are two equations determining $(x_*, \langle\delta x^2\rangle_*)$ jointly: the fluctuations shape the effective potential, and the effective potential shapes the fluctuations. Structurally this is $m = \tanh(\beta(m+h))$ again — and DMFT is the same fixed-point demand imposed on entire two-time functions $C(t,t')$, $R(t,t')$ instead of scalars.

Exercise 7.1: Solution p. 217

(a) $\sum_{k>m} k^{-(\alpha+1)} \approx \int_m^\infty k^{-(\alpha+1)}dk = m^{-\alpha}/\alpha$, so $L_m - L_\infty \propto m^{-\alpha}$. (The same replace-sum-by-integral move as the entropy estimates of Section 2.4.)

(b) $m = P/c$ quanta fit, so $L(P) - L_\infty \propto (P/c)^{-\alpha}$: $\boxed{\alpha_P = \alpha}$.

(c) Quantum k is seen $\approx n\,p_k$ times, so it is learnable iff $n\,p_k \gtrsim \tau$, i.e. $k \lesssim m(n) \propto (n/\tau)^{1/(\alpha+1)}$ by (7.7). Then $L(n) - L_\infty \propto m(n)^{-\alpha}$: $\boxed{\alpha_n = \dfrac{\alpha}{\alpha+1}}$.

(d) Identical threshold logic with S in place of n: $\boxed{\alpha_S = \dfrac{\alpha}{\alpha+1}}$. Note the model predicts $\alpha_n = \alpha_S$ — data and single-epoch training time are interchangeable bottlenecks, a foreshadowing of the joint treatment in Section 7.9.

(e) Eliminating α: $\frac{1}{\alpha_n} = \frac{1}{\alpha_P} + 1$, hence $\alpha_n < \alpha_P$ always. But the measured LLM values have $\alpha_n \approx 0.095 > \alpha_P \approx 0.076$: the predicted *ordering* is violated, not just the numbers. Candidate culprits, roughly in order of suspicion: (i) the measured α_n comes from multi-epoch training with early stopping, not the clean "capacity-unlimited, data-limited" protocol assumed in (c); (ii) the threshold τ and per-quantum cost c need not be k-independent (rare quanta may be intrinsically harder or easier); (iii) Δ constant across quanta is crude; (iv) real capabilities may not decompose into independent quanta at all — they share structure, so learning one changes the cost of others (see the lab of Section 7.6, where exactly this happens). The instructive point is not which repair is right but that the model is sharp enough to be *wrong* — compare the falsifiability discussion of Section 1.

Exercise 7.2: Solution p. 218

(a) As in the text; smoothness (bounded second derivatives) enters in the per-patch error $\sim s^2$.

(b) Mean-absolute error is the error itself, not its square: $L \sim s^2 \sim P^{-2/d}$.

(c) $d \approx 4/0.076 \approx 50$: an effective, intrinsic dimension of the distribution of text as seen by the model — vastly smaller than the nominal input dimension (vocabulary $\times$ context), which is the whole point.

(d) If data limits the tiling, n samples resolve spacing $s \sim n^{-1/d}$ and $\alpha_n \approx 4/d \approx \alpha_P$: this mechanism predicts *equal* exponents — closer to what is observed than the quanta prediction, and the same conclusion the field theory of Section 7.8 reaches from spectra.

Exercise 7.3: Solution p. 219

(a) The learned modes contribute $\big((n\beta)^{-1}/(\lambda_k + (n\beta)^{-1})\big)^2 y_k^2 \approx 0$ for $k \ll k_*$; the unlearned ones contribute y_k^2; the crossover window contributes $O(k_*^{1-b})$ as well, so the exponent is unchanged. Hence $L \sim n^{-(b-1)/a}$.

(b) With $b = a = \alpha + 1$: $L \sim n^{-\alpha/(\alpha+1)}$, i.e. the exponent α_n of Exercise 7.1(c). The threshold $\lambda_k \gtrsim (n\beta)^{-1}$ is the threshold $n\,p_k \gtrsim \tau$; the tail sum of y_k^2 is the tail sum of Δp_k. Kernel regression *is* a quantization model whose quanta are eigenmodes.

(c) $b \neq a$ decouples how often a skill is *tested* (its loss weight) from how easy it is to *learn* (its threshold) — e.g. let quantum k require $\tau_k \propto k^\theta$ occurrences. Any monotone relation between frequency and difficulty generates a two-exponent family, and (7.11) is its universal form.

Exercise 5.6: Solution p. 502

(a) A Bayesian two-layer network $f(x) = \sum_{i=1}^{N} a_i\, \phi(w_i \cdot x)$ whose readout prior (set by the output-layer weight decay) has variance $\mathbb{E}[a_i^2] = \sigma_a^2/N$; the limit taken is $N \to \infty$ *together with* $\sigma_a^2 \to 0$ — the paper's "mean-field scaling" — plus the EK (dense-data) approximation. The subtlety: at equilibrium, only the *induced prior on functions* matters. For the linear readout, a prefactor κ in $f = \kappa \sum_i a_i \phi$ and a prior variance σ^2 for a_i enter the function-space prior only through the product $\kappa^2\sigma^2$ (a Gaussian change of variables). So the mean-field parametrization (5.7) — prefactor $1/N$, order-one weights — corresponds to *bare* readout variance $\propto 1/N^2$, while variance σ_a^2/N at *fixed* σ_a^2 would be precisely the standard GP scaling of Section 3. The mean-field regime is the additional suppression $\sigma_a^2 \to 0$. Meanwhile the other half of Section 5.2's prescription — the learning-rate rescaling (5.14) — has no equilibrium counterpart at all: the learning rate drops out of the stationary distribution (Section 2.1). In the language of the γ-dial: gradient-flow scalings are classified by (prefactor, initialization, learning rate) jointly, but their Bayesian shadows are classified by the single combination $\kappa^2\sigma^2$.

(b) The overlap $\Phi(w)$ of a hidden neuron's input weights with the teacher-relevant directions — for modular addition, the Fourier coefficients of w on $\mathbb{Z}_P$ (the warm-up (Gromov 2023) says which ones matter). The sharper statement is about the *single-neuron posterior* $P(w)$: in the GFL phase it is a single Gaussian centered at $\Phi = 0$; in the mixed phase it becomes a mixture, with a finite fraction of neurons condensed at $\Phi \neq 0$ and the rest still Gaussian. The order parameter jumps discontinuously: first order. The ordered side is a finite-temperature cousin of Figure 6, where the measure over neurons condenses onto the low-dimensional structures realizing the target.

(c) (i) Noisy training $\to$ Gibbs posterior (Section 2.1); (ii) mean-field decoupling of the posterior into readout $\otimes$ hidden-layer factors, with neurons coupled only through averages (this section); (iii) the self-consistent "adaptive" kernel $Q_{\mu\nu} = \sigma_a^2 \langle \phi(w \cdot x^\mu)\phi(w \cdot x^\nu)\rangle_{\text{posterior}}$, whose defining equation has exactly the structure of (5.22) — the kernel depends on a posterior that depends on the kernel; (iv) saddle-point evaluation (Section 2.4); (v) a variational Gaussian-*mixture* ansatz, needed precisely because a first-order transition means several competing saddles; (vi) the EK approximation for the data average (Section 4.6). Note what is *absent*: in the main text the data averaging is done by EK smoothing, not by the replica trick of Appendix A.2.

(d) Primarily the *sample size* n at fixed temperature: below a critical number of training facts the disordered (memorizing) phase is dominant and the network never generalizes; above it, the mixed phase wins. (The transition line lives in the $(n, \text{temperature})$ plane, so it can equally be crossed by tuning the noise level or weight decay at fixed n — but the grokking narrative crosses it in n.)

(e) In equilibrium there is no time; the free-energy crossing itself is instantaneous. The time-dependence of grokking is the *dynamical* lag of a first-order transition: Langevin training equilibrates quickly *within* the metastable disordered basin — which is enough to drive the train loss to zero (memorization is easy there) while test accuracy stays at chance — and only much later escapes to the thermodynamically dominant mixed phase, whereupon test accuracy jumps. This is precisely the Ising-perceptron story of Section 2.6: the information needed to generalize is present in the data well before the dynamics finds the ordered state, so capability arrives "abruptly and later than it should."

A.2 Replicas: Solution p. 506

The six steps are their own outline; we record only the checkpoints. (Step 2) Conditioned on the θ^a and θ^*, the u^a and v are linear in the Gaussian vector x, hence exactly Gaussian, with the covariances (A.3); independence over examples gives the n-th power, so the per-example factor is $e^{-e(q,R)}$ with $e = -\log P[\text{all } Q \text{ replicas agree with the teacher}]$, a $(Q{+}1)$-dimensional Gaussian orthant probability. At $Q = 1$ the orthant is the double wedge of Exercise 2.3(b), of probability $1 - \varepsilon(R)$. (Step 3) With $\delta(Nq^{ab} - \theta^a \cdot \theta^b) = \int \frac{d\hat{q}^{ab}}{2\pi} e^{i\hat{q}^{ab}(Nq^{ab} - \sum_i \theta_i^a \theta_i^b)}$ and likewise for R^a, the exponent is a sum over sites of identical terms, so $\sum_{\{\theta\}} = \left(\sum_{\theta_i^1,\dots,\theta_i^Q = \pm 1} e^{-i\sum_{a<b}\hat{q}^{ab}\theta^a\theta^b - i\sum_a \hat{R}^a\theta^a\theta^*}\right)^N$; the saddle point over the conjugates then defines $h(q, R)$. At $Q = 1$ only $\hat{R}$ survives and the site sum is $2\cosh\hat{R}$, reproducing the binomial entropy after the saddle in $\hat{R}$. (Step 4) Under replica symmetry, the Q correlated Gaussians u^a can be written as $\sqrt{q}\,t + \sqrt{1-q}\,z^a$ with t shared and the z^a independent, and v correlated with t through R; integrating out the z^a gives the orthant probability as $2\int Dt\, H(\cdot)^Q$, where the argument of H is linear in t with coefficients built from q and R. The site sum becomes $\int Dz \left(2\cosh(\sqrt{\hat{q}}\,z + \hat{R})\right)^Q$ after decoupling the $\hat{q}\,\theta^a\theta^b$ term by the same trick. Both are analytic in Q, and $\frac{1}{Q}\log\int(\cdots)^Q \to \int(\cdots)\log(\cdots)$ as $Q \to 0$. (Step 5) The teacher is a uniformly random binary vector consistent with every example, exactly as a Gibbs student is, so the two are exchangeable, and $\mathbb{E}[\theta^a \cdot \theta^b] = \mathbb{E}[\theta^a \cdot \theta^*]$ at the saddle: $q = R$. (Step 6) Solving the remaining pair of equations numerically, the interior solution exists for all α up to a spinodal and the teacher solution $R = 1$ always exists; their free energies cross at $\alpha_c \approx 1.245$ (Györgyi 1990), below the annealed 1.45. The annealed average is dominated by rare, weakly constraining datasets, which overweight poorly aligned students and so require more data before the teacher basin wins; removing that bias moves the transition earlier.

A.3 A variant: the spherical perceptron: Solution p. 508

(a) Decompose $\theta = R\,\theta^* + \theta_\perp$. The spherical constraint $|\theta|^2 = N$ forces $|\theta_\perp|^2 = N(1 - R^2)$, so the slice at fixed R is a sphere of radius $\sqrt{N(1-R^2)}$ in the $(N-1)$-dimensional orthogonal complement, of volume $\propto \left(N(1-R^2)\right)^{(N-2)/2} =$

$e^{\frac{N}{2}\log(1-R^2)+O(\log N)}$, giving $h(R)=\frac{1}{2}\log(1-R^2)$ up to constants. The data factor $(1-\varepsilon(R))^n = e^{N\alpha\log(1-\varepsilon)}$ is as in Exercise 2.3(d), and collecting exponents gives (A.8).

(b) Setting $S'_{\text{ann}}(R)=0$ with $\varepsilon(R)=\arccos(R)/\pi$ gives

$$\frac{R}{1-R^2}=\frac{\alpha}{\pi\sqrt{1-R^2}\,\big(1-\varepsilon(R)\big)} \quad\Longleftrightarrow\quad \frac{R}{\sqrt{1-R^2}}=\frac{\alpha}{\pi\,(1-\varepsilon(R))}\,.$$

As $R\to 1$ the entropy term $-\frac{1}{2}\log(1-R^2)\to+\infty$ while the energy term stays finite, so $S_{\text{ann}}\to+\infty$ and $R=1$ is never a minimum; the minimizer is always interior.

(c) With $R=\cos(\pi\varepsilon)$ and $\varepsilon\to 0$: $1-R^2=\sin^2(\pi\varepsilon)\approx(\pi\varepsilon)^2$, so $S_{\text{ann}}\approx-\log(\pi\varepsilon)+\alpha\varepsilon+\text{const}$, minimized at $\varepsilon=1/\alpha$. For small R the entropic part is $\approx R^2/2$ and $\varepsilon(R)\approx\frac{1}{2}-R/\pi$, so $-\alpha\log(1-\varepsilon)\approx-\alpha\log\frac{1}{2}-\frac{2\alpha}{\pi}R$, and minimizing $R^2/2-\frac{2\alpha}{\pi}R$ gives $R^*=2\alpha/\pi\propto\alpha$. The learning curve rises linearly from $R=0$, bends over, and approaches $R=1$ with the power-law tail $\varepsilon\simeq 1/\alpha$.

(d) Steps 1, 2, 5, and 6 are unchanged: the data average sees only the overlaps, and the Gibbs-symmetry argument holds for any prior invariant under the symmetries of the problem. Step 3 changes: the sum over binary vectors becomes an integral over Q points of the sphere, and the count $h(q,R)$ becomes the log-volume of the set of Q-tuples with prescribed overlaps, a Gaussian integral with a $\log\det$ of the overlap matrix; under replica symmetry it is $\frac{1}{2}\big[(Q-1)\log(1-q)+\log(1-q+Qq-QR^2)\big]$ up to constants, whose $Q\to 0$ derivative reproduces (A.7) at $q=R$. Step 4 is then the same H-function integral. The outcome differs because this entropy diverges at $R\to 1$: no competing basin, no transition, and a learning curve $\varepsilon\simeq 0.625/\alpha$ (Seung et al. 1992).

B.1 Linear regression: Solution p. 510

(a) $\partial\mathcal{L}/\partial\theta^j=\sum_\mu {X^\mu}_j\big(\sum_k {X^\mu}_k\theta^k-y^\mu\big)=[X^T(X\theta-Y)]_j$. The j-th normal equation sets the inner product of the j-th column of X with the residual to zero, so $\Delta_*\perp\operatorname{col}(X)$; hence $X\theta_*$ is the orthogonal projection of Y onto $\operatorname{col}(X)$.

(b) $v^TX^TXv=|Xv|^2$ vanishes only if $Xv=0$, which for $v\neq 0$ is impossible when the columns are independent; so X^TX is positive definite and invertible. Consistency: $\ker(X^TX)=\ker X$ (if $X^TXv=0$ then $|Xv|^2=v^TX^TXv=0$), and for any matrix A one has $\operatorname{col}(A^T)=(\ker A)^\perp$, so $\operatorname{col}(X^TX)=(\ker X)^\perp=\operatorname{col}(X^T)\ni X^TY$. If $P>n$ then $\dim\ker X=P-\operatorname{rank}(X)\geq P-n>0$, and adding any element of $\ker X$ to a solution gives another.

(c) $X^TX=\operatorname{diag}(3,2)$, $X^TY=(2,0)^T$, so $\theta_*=(2/3,0)^T$: the constant function $2/3$. Predictions $(2/3,2/3,2/3)^T$, residual $(-1/3,2/3,-1/3)^T$, loss $\frac{1}{2}\cdot\frac{6}{9}=\frac{1}{3}$. The optimization is exact (the problem is convex and was solved in closed form); the error is representational — no affine function agrees with x^2 at three points. Exercise B.2 repairs it by adding a feature.

(d) $X^T g X = \mathrm{diag}(6,2)$ and $X^T g Y = (2,0)^T$, so $\theta_*^g = (1/3, 0)^T$: the constant function $1/3$, pulled toward the heavily weighted middle label $y = 0$. Residual $(-2/3, 1/3, -2/3)^T$. A different metric on $\mathbb{R}^n$ gives a different fit.

B.2 Linear regression with nonlinear features: Solution p. 512

(a) $\Psi = \begin{pmatrix} 1 & -\sqrt{2} & 1 \\ 1 & 0 & 0 \\ 1 & \sqrt{2} & 1 \end{pmatrix}$, which is invertible ($P = n = 3$), so exactly one θ interpolates: $\theta = (0,0,1)^T$, giving $f_\theta(x) = x^2$ and predictions $(1,0,1)^T$. What changed is that Y now lies in $\mathrm{col}(\Psi) = \mathbb{R}^3$.

(b) All nonlinearity in x sits inside the fixed functions ψ_i; $f_\theta = \sum_i \theta^i \psi_i$ depends on θ linearly, so the loss is quadratic in θ and the normal equations are linear.

(c) The coefficient becomes $1/c$, the product $\theta^3 \psi_3 = x^2$ is unchanged, and so is the fitted function. (With the ridge penalty of Appendix B.4 the rescaling *would* change the fit, since $|\theta|^2$ is not invariant under it.)

(d) $K(x,z) = 1 + 2xz + x^2 z^2 = (1 + xz)^2$.

B.3 Kernel regression: Solution p. 514

(a) Stationarity of $\frac{1}{2}|\theta|^2 - \alpha \cdot (\Psi\theta - Y)$ in θ gives $\theta = \Psi^T \alpha$; the constraint then reads $\Psi\Psi^T \alpha = \hat{K}\alpha = Y$, so $\alpha = \hat{K}^{-1} Y$ and $\theta_* = \Psi^T \hat{K}^{-1} Y$; the prediction is $\psi(x) \cdot \theta_* = \sum_\mu K(x, x^\mu)\alpha_\mu$. It is a minimum, not merely a critical point, because the objective is strictly convex and the constraint set is affine. In the SVD of Section 4.2, $\Psi = USV^T$ with $S = [\mathrm{diag}(s_1, \ldots, s_n)\ 0]$, one has $\hat{K} = U\,\mathrm{diag}(s_a^2)\,U^T$ and $\theta_* = VS^T \mathrm{diag}(s_a^{-2}) U^T Y = \sum_{a=1}^n \frac{u_a \cdot Y}{s_a} v_a$, where v_a are the first n columns of V: only right singular vectors with nonzero singular value (the row space) appear.

(b) $c^T \hat{K} c = c^T \Psi\Psi^T c = |\Psi^T c|^2 \geq 0$. If $\hat{K}c = 0$ then $\Psi^T c = \sum_\mu c_\mu \psi(x^\mu) = 0$: a linear dependence among the training feature vectors (for instance two identical inputs, or simply $n > P$). $\hat{K}$ is invertible iff the n training feature vectors are linearly independent, which requires $n \leq P$.

(c) $\dot{\theta} = -\eta \Psi^T \Delta \in \mathrm{col}(\Psi^T)$, the row space of Ψ, so $\theta(t) - \theta_0 = -\eta \Psi^T \int_0^t \Delta$ stays there. Since Ψ is constant, $\dot{\Delta} = \Psi\dot{\theta} = -\eta \Psi\Psi^T \Delta = -\eta \hat{K} \Delta$, whence $\Delta(t) = e^{-\eta t \hat{K}} \Delta(0) \to 0$. Integrating, $\theta(t) - \theta_0 = -\eta \Psi^T \int_0^t e^{-\eta s \hat{K}} ds\, \Delta(0) = -\Psi^T \hat{K}^{-1} \big(1 - e^{-\eta t \hat{K}}\big) \Delta(0)$, and $f_{\theta(t)}(x) = f_{\theta_0}(x) + \psi(x) \cdot (\theta(t) - \theta_0)$ is (4.36) with $\Theta(x, x^\mu) = K(x, x^\mu)$ and $\Theta = \hat{K}$. At $t \to \infty$, $\theta_\infty = \theta_0 - \Psi^T \hat{K}^{-1} \Delta(0)$ interpolates, and $\theta_\infty - \theta_0 \in \mathrm{row}(\Psi)$. Any other interpolant θ' has $\theta' - \theta_\infty \in \ker \Psi = \mathrm{row}(\Psi)^\perp$, so by Pythagoras $|\theta' - \theta_0|^2 = |\theta_\infty - \theta_0|^2 + |\theta' - \theta_\infty|^2 \geq |\theta_\infty - \theta_0|^2$: the flow converges to the interpolant closest to θ_0, and for $\theta_0 = 0$ to θ_*.

(d) $\psi(x)\cdot\psi(z) = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = (x_1z_1 + x_2z_2)^2$; without the $\sqrt{2}$ the cross term would be $x_1x_2z_1z_2$ and the sum would not be a perfect square. $(1+xz)^2 = 1 + 2xz + x^2z^2$ comes from $\psi(x) = (1, \sqrt{2}\,x, x^2)$, as in Exercise B.2. By the multinomial theorem $(x\cdot z)^m = \sum_{|\alpha|=m}\binom{m}{\alpha}x^\alpha z^\alpha$, so $\psi_\alpha(x) = \sqrt{\binom{m}{\alpha}}\,x^\alpha$ over the $\binom{d+m-1}{m}$ monomials of degree m.

B.4 Kernel ridge regression: Solution p. 517

(a) Setting $\Psi^T(\Psi\theta - Y) + \lambda\theta = 0$ gives (B.17); $v^T(\Psi^T\Psi + \lambda I_P)v = |\Psi v|^2 + \lambda|v|^2 \geq \lambda|v|^2 > 0$ for $v \neq 0$, so the matrix is positive definite, hence invertible, for every $\lambda > 0$.

(b) $\Psi^T(\Psi\Psi^T + \lambda I_n) = \Psi^T\Psi\Psi^T + \lambda\Psi^T = (\Psi^T\Psi + \lambda I_P)\Psi^T$ uses only associativity. Multiply on the left by $(\Psi^T\Psi + \lambda I_P)^{-1}$ and on the right by $(\Psi\Psi^T + \lambda I_n)^{-1}$, both of which exist by (a). Ψ^T is $P \times n$ and, for $P \neq n$, has no two-sided inverse, so it cannot be "cancelled"; the identity holds because Ψ^T was regrouped, not divided out. Then $\theta_\lambda = \Psi^T(\hat{K} + \lambda I_n)^{-1}Y$ and $f_\lambda(x) = \psi(x)\cdot\theta_\lambda = \sum_\mu K(x, x^\mu)(\alpha_\lambda)_\mu$.

(c) As $\lambda \to \infty$, $\theta_\lambda \approx \Psi^TY/\lambda \to 0$. With $\Psi = USV^T$, $\Psi^T\Psi + \lambda I_P = V(S^TS + \lambda I_P)V^T$, so $\theta_\lambda = V(S^TS + \lambda I_P)^{-1}S^TU^TY = \sum_a \frac{s_a}{s_a^2+\lambda}(u_a\cdot Y)\,v_a$. As $\lambda \to 0$, $s_a/(s_a^2 + \lambda) \to 1/s_a$ for $s_a > 0$ and $\to 0$ for $s_a = 0$: this is the pseudo-inverse Ψ^+Y, the least-squares solution of minimum norm (an arbitrary unregularized minimizer need not have minimum norm). It interpolates iff $Y \in \mathrm{col}(\Psi)$, *i.e.* iff $u_a\cdot Y = 0$ for every zero singular value.

(d) $K(x^\mu, x^\nu) = (1 + x^\mu x^\nu)^2$ on $\{-1, 0, 1\}$ gives the displayed $\hat{K}$. Check $(\hat{K} + I_3)\alpha_\lambda = \frac{1}{4}(5-1,\ 1-2+1,\ -1+5)^T = (1,0,1)^T = Y$. At $z = 2$ the kernel vector is $\big(K(-1,2), K(0,2), K(1,2)\big) = (1,1,9)$, so $f_\lambda(2) = \frac{1}{4}(1-1+9) = \frac{9}{4}$, versus 4 for the interpolant. The ridge fit is shrunk toward zero: it no longer even interpolates the training labels (its training predictions are $\hat{K}\alpha_\lambda = (3/4, 1/4, 3/4)^T$), and the quadratic growth it does fit is damped.

(e) The feature route costs $O(nP^2 + P^3)$ and is attractive when $P \ll n$ and the features are explicit; the kernel route costs $O(n^2\cdot(\text{cost of } K) + n^3)$ and is attractive when $n \ll P$, or when K can be evaluated without constructing ψ at all. Neither is free: $n \times n$ is fatal for large n.

B.4 Kernel ridge regression: Solution p. 518

(a) $\hat{K}u_\pm = (2 \pm 1)u_\pm$, so $s_+^2 = 3$, $s_-^2 = 1$; and $Y = \sqrt{2}\,u_+ + \sqrt{2}\,u_- = (1,1)^T + (1,-1)^T$.

(b) $f(t) = (1 - e^{-3\eta t})(1,1)^T + (1 - e^{-\eta t})(1,-1)^T = \big(2 - e^{-3\eta t} - e^{-\eta t},\ e^{-\eta t} - e^{-3\eta t}\big)^T$. At $\eta t = \log 2$, $e^{-\eta t} = \frac{1}{2}$ and $e^{-3\eta t} = \frac{1}{8}$, giving $(11/8,\ 3/8)^T$. The eigenvalue sets a mode's rate, while $u_a\cdot Y$ sets whether it is present and with what amplitude; a fast mode with zero initial amplitude leaves no trace in the trajectory.

(c) Ridge multipliers $\frac{3}{3+1} = \frac{3}{4}$ and $\frac{1}{1+1} = \frac{1}{2}$, so $f_\lambda = \frac{3}{4}(1,1)^T + \frac{1}{2}(1,-1)^T = (5/4,\, 1/4)^T$. The flow multipliers at $\eta t = \log 2$ are $\frac{7}{8}$ and $\frac{1}{2}$: the two agree on $s^2 = 1$ only. Solving $s^2/(s^2+\lambda) = 1 - e^{-\eta t s^2}$ gives $\lambda_{\text{eff}} = s^2 e^{-\eta t s^2}/(1 - e^{-\eta t s^2}) = s^2/(e^{\eta t s^2} - 1)$; at $\eta t = \log 2$ this is 1 for $s^2 = 1$ but $3/7$ for $s^2 = 3$. No single λ matches both modes.

(d) Here "spectral bias" means exactly this: components of the residual along eigenvectors of the fixed Gram matrix with larger eigenvalue decay faster, at rate ηs_a^2. To speak of Fourier frequencies one needs a relation between the u_a and Fourier modes (true, for instance, for translation-invariant kernels on uniform grids). To speak of noise one needs a model of how label noise projects onto the u_a (typically spread evenly across modes, so that it lives mostly in the many slow modes — which is why early stopping and ridge both filter it). To speak of test error one needs a distribution over inputs and the population version of the kernel operator, as in Sections 4.6 and 7.7.

B.5 Linearized networks are kernel ridge regressors: Solution p. 519

(a) $\partial f/\partial c_i = \phi(a_i \cdot x)$ and $\partial f/\partial a_{ij} = c_i\, \phi'(a_i \cdot x)\, x_j$. Taking the inner product of the tangent features at x and x' and summing over j in the second block gives the displayed $\Theta(x, x')$.

(b) With the a_i frozen, $f = \sum_i c_i \psi_i(x)$ with $\psi_i(x) = \phi(a_i \cdot x)$ is linear in the trainable parameters c, so $f = f^{\text{lin}}$ exactly, $\Psi^\mu{}_i = \phi(a_i \cdot x^\mu)$, and $\hat{K}^{\mu\nu} = \sum_i \phi(a_i \cdot x^\mu)\phi(a_i \cdot x^\nu)$ is the first term: the random-features model. Training only the a_i gives the second term, but there the linearization is not exact, since f is nonlinear in a.

(c) Writing $\delta\theta = \theta - \theta_0$, the linearized loss is $\mathcal{L}^{\text{lin}} = \frac{1}{2}|J_0 \delta\theta + \Delta(0)|^2$, so $\delta\dot{\theta} = -\eta\big[J_0^T(J_0\delta\theta + \Delta(0)) + \lambda\delta\theta\big] = -\eta\big[M\delta\theta + J_0^T\Delta(0)\big]$ with $M = J_0^T J_0 + \lambda I_P$. This is a linear ODE with positive-definite M, so $\delta\theta(t) = \delta\theta_\infty + e^{-\eta t M}\big(\delta\theta(0) - \delta\theta_\infty\big)$ converges from any start to $\delta\theta_\infty = -M^{-1}J_0^T\Delta(0) = -J_0^T(\Theta_0 + \lambda I_n)^{-1}\Delta(0)$, by push-through. Then $f^{\text{lin}}(x) = f_{\theta_0}(x) + \nabla_\theta f_{\theta_0}(x) \cdot \delta\theta_\infty$ is (B.28), since $\nabla_\theta f_{\theta_0}(x) \cdot J_0^T$ has components $\Theta(x, x^\mu)$.

(d) As $\lambda \to 0$, $(\Theta_0 + \lambda I_n)^{-1} \to \Theta_0^{-1}$; as $t \to \infty$ in (4.36), $(1 - e^{-\eta t \Theta}) \to 1$. Both give $f_{\theta_0}(x) - \Theta(x, x^\mu)\, \Theta^{-1}_{\mu\nu}\, \Delta^\nu(0)$.

Computational Mechanics

Xavier Poncini (Simplex), Adam Shai (Simplex), Paul Riechers (Simplex)

Main text: Chapter C.3, p. 241

Exercise 3.1: Solution p. 243

(a) Two histories belong to the same cell when they receive the same summary value. In particular, $R_A(u_1) = R_A(u_2)$.

(b) Both R_A and R_B are sufficient. Every cell of Π_A and every cell of Π_B contains histories carrying only one of the distributions P, Q, or S.

The statistic R_C is not sufficient. Its first cell contains u_1 and u_4, for example, but u_1 induces P while u_4 induces Q. Since $P \neq Q$, these histories cannot be assigned the same value by a sufficient statistic.

(c) The cell $\{u_1, u_2\}$ can be merged with $\{u_3\}$ because all three histories induce P. Similarly, $\{u_4\}$ can be merged with $\{u_5\}$ because both histories induce Q. No cell can be merged with $\{u_6\}$, and no P-cell can be merged with a Q-cell. After all permissible merges, the resulting partition is

$$\big\{\{u_1, u_2, u_3\}, \{u_4, u_5\}, \{u_6\}\big\} = \Pi_B.$$

(d) The three cells of Π_B correspond respectively to the three distinct future distributions P, Q, and S. Merging any two cells would therefore place histories with different future distributions in the same cell and destroy sufficiency. Hence R_B is minimal sufficient.

(e) Let Π' refine a sufficient partition Π. Any two histories in the same cell of Π' also belong to the same cell of Π. Since Π is sufficient, those histories induce the same future distribution. Therefore Π' is sufficient.

In the example, Π_A refines Π_B, and both are sufficient. A coarsening need not remain sufficient: Π_C is obtained from the sufficient partition Π_A by merging its cells $\{u_1, u_2\}$, $\{u_3\}$, and $\{u_4\}$. The resulting cell contains histories inducing both P and Q, so Π_C is not sufficient.

(f) Every cell of Π_D lies within one predictive class, so Π_D is sufficient. The same was established for Π_A.

The cell $\{u_1, u_2\}$ of Π_A is split between $\{u_1\}$ and $\{u_2, u_3\}$ in Π_D, so Π_A does not refine Π_D. Conversely, $\{u_2, u_3\}$ in Π_D is split between $\{u_1, u_2\}$ and $\{u_3\}$ in Π_A, so Π_D does not refine Π_A. Both partitions nevertheless refine Π_B, because every one of their cells is contained in one of the three predictive classes represented by P, Q, and S.

Exercise 3.2: Solution p. 243

(a)

1. For every $x \in \mathcal{X}_+^*$ and every continuation y, the conditional probability after x equals itself. Hence $x \sim x$, so $\sim$ is reflexive.
2. If $x \sim x'$, then the two continuation probabilities are equal for every y. Reversing the equality shows that $x' \sim x$, so $\sim$ is symmetric.
3. Suppose $x \sim x'$ and $x' \sim x''$. For every y, the continuation probability after x equals that after x', which equals that after x''. Thus $x \sim x''$, so $\sim$ is transitive.

Therefore $\sim$ is an equivalence relation.

(b) If $\epsilon(x) = \epsilon(x')$, then x and x' belong to the same equivalence class, so $x \sim x'$. By the definition of $\sim$, the two histories induce the same probability for every finite continuation y. This is precisely the definition of sufficiency for prediction, so ϵ is sufficient.

(c) Suppose $R(x) = R(x')$. Since R is sufficient, x and x' induce the same probability for every finite continuation. Hence $x \sim x'$, and equivalent histories have the same equivalence class: $\epsilon(x) = \epsilon(x')$.

Thus any two histories in the same cell induced by R also lie in the same causal state. Every cell induced by R is therefore contained in a causal state, so the partition induced by R refines the causal-state partition. Since R was arbitrary, every sufficient partition has this property.

(d) Part (b) shows that ϵ is sufficient. Part (c) shows that its partition is refined by the partition of every sufficient statistic. Hence the causal-state partition is the coarsest sufficient partition, so ϵ is a minimal sufficient statistic.

(e) Since T is sufficient, part (c), with $R = T$, shows that the partition induced by T refines the causal-state partition. Since T is minimal and ϵ is sufficient, the causal-state partition must also refine the partition induced by T. The two partitions are therefore equal.

Define $\psi(\epsilon(x)) := T(x)$. Equality of the two partitions shows that this definition is independent of the representative x and that distinct causal states receive distinct T-values. It also reaches every value in $\mathrm{im}(T)$. Hence ψ is a bijection.

Exercise 3.3: Solution p. 245

(a)

1. All four length-two histories are possible. Their resulting hidden phases are

$x_{1:2}$	00	01	10	11
S_2	s_1	s_R	$\{s_0, s_1\}$	s_0.

Thus 00, 01, and 11 identify the phase exactly, while 10 is ambiguous.

2. From the phases compatible with 10, only s_0 can emit 0, and only s_1 can emit 1. Consequently, 100 identifies the new phase as s_1, while 101 identifies it as s_R. From each phase, the emitted symbol determines the next phase, so every later observation preserves synchronization.

3. The histories 00, 01, and 11 identify s_1, s_R, and s_0, respectively. These phases induce distinct next-token distributions: s_1 emits only 1, s_0 emits only 0, and s_R can emit either symbol. Hence they give three distinct causal states.

They are also distinct from the four states assumed in the question. This can be seen without calculating probabilities by comparing possible length-two continuations:

representative	possible length-two continuations
$\emptyset$	$\{00, 01, 10, 11\}$
0	$\{01, 10, 11\}$
1	$\{00, 01, 10\}$
10	$\{01, 10, 11\}$
00	$\{10, 11\}$
01	$\{00, 10\}$
11	$\{01\}$

The only repeated continuation set belongs to 0 and 10, whose causal states are assumed to be distinct. Finally, every history of length at least three identifies the phase by part (ii), and so belongs to one of the states represented by 00, 01, and 11. No further causal states arise.

4. Appending each possible symbol gives the causal-state update:

state	0	1
$\epsilon(\emptyset)$	$\epsilon(0)$	$\epsilon(1)$
$\epsilon(0)$	$\epsilon(00)$	$\epsilon(01)$
$\epsilon(1)$	$\epsilon(10)$	$\epsilon(11)$
$\epsilon(10)$	$\epsilon(00)$	$\epsilon(01)$
$\epsilon(00)$	–	$\epsilon(01)$
$\epsilon(01)$	$\epsilon(11)$	$\epsilon(11)$
$\epsilon(11)$	$\epsilon(00)$	–

Here a dash denotes an impossible extension. Drawing one node for each row and the indicated symbol-labelled edges gives the required automaton.

(b)

1. The history 0 identifies the current state as A: only A can emit 0, and this transition returns to A. The history 1 is ambiguous. If $S_0 = A$, it leads to B, while if $S_0 = B$, it leads to A.

2. From the two states compatible with the history 1, only A can emit 0. Therefore 10 resolves the ambiguity and identifies the new state as A. The extension 11 remains ambiguous. Using the given equality and recursively updating after pairs of 1s gives

$$\epsilon(1^{2k}) = \epsilon(\emptyset), \qquad \epsilon(1^{2k+1}) = \epsilon(1), \qquad k \geq 0.$$

3. If a history contains a 0, its final 0 synchronizes the generator to A. An even number of trailing 1s then leaves it in A, represented by 0, while an odd number leaves it in B, represented by 01. If the history contains no 0, part (ii) shows that its causal state is represented by $\emptyset$ or 1, according to the parity of its length. Thus no additional causal states arise.

These four states are distinct. The state represented by 01 cannot emit 0, whereas the other three can. The continuation 10 is impossible after 0 but possible after $\emptyset$ and 1. Finally, $\epsilon(\emptyset) \neq \epsilon(1)$ by assumption. Hence the displayed representatives are also shortest.

4. Appending each possible symbol gives the causal-state update:

state	0	1
$\epsilon(\emptyset)$	$\epsilon(0)$	$\epsilon(1)$
$\epsilon(1)$	$\epsilon(0)$	$\epsilon(\emptyset)$
$\epsilon(0)$	$\epsilon(0)$	$\epsilon(01)$
$\epsilon(01)$	–	$\epsilon(0)$

Here a dash denotes an impossible extension. Drawing one node for each row and the indicated symbol-labelled edges gives the required automaton.

(c)

1. The only transition that emits 1 starts from B, and that transition leads to A. Thus the hidden state immediately after observing 1 is A.
2. One path remains in A while emitting all n zeros. Each of the other n paths moves from A to B on one of the n transitions and then remains in B. Thus there are $n+1$ paths, each with probability 2^{-n}, and

$$\Pr(0^n \mid S = A) = (n+1)2^{-n}.$$

The n paths that end in B can subsequently emit 1. Including this final transition, each has probability $2^{-(n+1)}$, so

$$\Pr(X_{n+2} = 1 \mid X_{1:n+1} = 10^n) = \frac{n2^{-(n+1)}}{(n+1)2^{-n}} = \frac{n}{2(n+1)}.$$

3. Since

$$\frac{n}{2(n+1)} = \frac{1}{2} - \frac{1}{2(n+1)},$$

this probability is strictly increasing with n. Hence every pair of histories 10^n and 10^m, with $n \neq m$, already has a different next-token distribution. They must lie in different causal states. The source consequently has infinitely many causal states despite its two-state generator.

Exercise 3.4: Solution p. 246

(a) Appending one symbol a gives

$$R(x_{1:n}a) = \big(f_\theta(h_{n-1}, x_n), a\big) = \Phi_a(R(x_{1:n}))\,.$$

Repeatedly applying this deterministic update along $z_{1:j}$ gives

$$R(x_{1:n}z_{1:j}) = \Phi_{z_j} \circ \cdots \circ \Phi_{z_1}(R(x_{1:n})) = \Phi_{z_{1:j}}(R(x_{1:n}))\,.$$

If $R(x_{1:n}) = R(x'_{1:m})$ and both extended histories are allowable, applying the same composite function to the equal starting representations gives

$$R(x_{1:n}z_{1:j}) = R(x'_{1:m}z_{1:j}).$$

(b) Suppose that the prefixes through $z_{1:j}$ are allowable after both histories. Part (a) gives

$$R(x_{1:n}z_{1:j}) = R(x'_{1:m}z_{1:j}).$$

The two representations give the same arguments to g_θ, so exact next-token prediction gives

$$\begin{aligned}&\Pr(X_{n+j+1} = z_{j+1} \mid X_{1:n+j} = x_{1:n}z_{1:j})\\ &\quad = \Pr(X_{m+j+1} = z_{j+1} \mid X_{1:m+j} = x'_{1:m}z_{1:j})\,.\end{aligned}$$

Thus the next extension has positive probability after one history exactly when it has positive probability after the other. Following the word from left to right, either both encounter their first zero at the same position or all its prefixes are allowable after both.

(c) If every prefix is allowable, the conditional chain rule gives the product in the question and

$$\begin{aligned}&\Pr(X_{m+1:m+k} = z_{1:k} \mid X_{1:m} = x'_{1:m})\\ &\quad = \prod_{j=0}^{k-1} \Pr(X_{m+j+1} = z_{j+1} \mid X_{1:m+j} = x'_{1:m}z_{1:j})\,.\end{aligned}$$

Part (b) shows that the factors in the two products are equal, so the products are equal. If a first zero-probability extension is encountered instead, part (b) shows that it occurs after both histories; consequently both probabilities of the complete continuation are zero.

Therefore equal values of R imply equal conditional probabilities for every finite continuation. By the definition of sufficiency,

$$R(x_{1:n}) = (h_{n-1}, x_n)$$

is a sufficient statistic for predicting the entire future.

Exercise 4.1: Solution p. 247

(a) In the diagram, i is the current hidden state, j is the next hidden state, a is the emitted symbol, and r is the probability of that joint emission-transition event conditional on starting in i. Thus the edge records $T_{ij}^{(a)} = \Pr(X_{t+1} = a, S_{t+1} = j \mid S_t = i) = r$. A zero entry means that no such emission-transition pair is possible. For a fixed current state i, the events indexed by all pairs (a, j) exhaust the possible emitted symbols and next states. Consequently,

$$\sum_{a \in \mathcal{X}} \sum_{j} T_{ij}^{(a)} = 1.$$

Since

$$T_{ij} = \sum_{a \in \mathcal{X}} T_{ij}^{(a)},$$

every entry of T is nonnegative and its ith row sums to

$$\sum_{j} T_{ij} = \sum_{j} \sum_{a \in \mathcal{X}} T_{ij}^{(a)} = 1.$$

Hence T is row-stochastic.

(b)

1. The nonzero entries give the transitions

$$0 \xrightarrow{0:1} 1, \qquad 1 \xrightarrow{1:1} R, \qquad R \xrightarrow{0:1-p} 0, \qquad R \xrightarrow{1:p} 0.$$

Thus the diagram is a three-state cycle, with two differently labelled edges from R to 0. State 0 is the phase that emits 0 deterministically, state 1 is the phase that emits 1 deterministically, and state R is the random phase, which emits 1 with probability p and 0 with probability $1 - p$.

2. For 011, the only possible initial state is 0. The model then follows

$$0 \xrightarrow{0:1} 1 \xrightarrow{1:1} R \xrightarrow{1:p} 0.$$

Including the initial probability $1/3$ gives $\Pr(011) = p/3$. Matrix multiplication gives the same result:

$$\eta^{(\emptyset)} T^{(0)} T^{(1)} T^{(1)} = \begin{bmatrix} p/3 & 0 & 0 \end{bmatrix}, \qquad \Pr(011) = p/3.$$

No state permits a path labelled 000: starting from 0, the first 0 moves to state 1, which cannot emit 0; starting from 1, even the first 0 is impossible; and starting from R, the first two zeros move through $R \to 0 \to 1$, after which a third zero is impossible. Cor-

respondingly,

$$\eta^{(\emptyset)}T^{(0)}T^{(0)}T^{(0)} = \begin{bmatrix}0 & 0 & 0\end{bmatrix}, \qquad \Pr(000) = 0.$$

(c)

1. In state order (A, B), the matrices are

$$T^{(0)} = \begin{bmatrix}q & 0\\ 0 & 0\end{bmatrix}, \qquad T^{(1)} = \begin{bmatrix}0 & 1-q\\ 1 & 0\end{bmatrix}.$$

In $T^{(0)}$, the entry (A, B) is zero because no 0-edge connects A to B, while the entire B row is zero because state B cannot emit 0. In $T^{(1)}$, the diagonal entries are zero because neither 1-edge is a self-loop. The two off-diagonal entries record $A \xrightarrow{1:1-q} B$ and $B \xrightarrow{1:1} A$.

2. We have

$$T = \begin{bmatrix}q & 1-q\\ 1 & 0\end{bmatrix}.$$

Its entries are nonnegative and both rows sum to one.

3. The word 011 can be produced only by starting from A and following

$$A \xrightarrow{0:q} A \xrightarrow{1:1-q} B \xrightarrow{1:1} A.$$

Thus

$$\Pr(011) = \frac{1}{2}q(1-q).$$

The matrix calculation is

$$\eta^{(\emptyset)}T^{(0)}T^{(1)}T^{(1)} = \begin{bmatrix}\frac{1}{2}q(1-q) & 0\end{bmatrix}, \qquad \Pr(011) = \frac{1}{2}q(1-q).$$

The word 010 would require the path to emit a 0 immediately after the transition $A \xrightarrow{1} B$, but state B cannot emit 0. Hence there is no such path. Equivalently,

$$\eta^{(\emptyset)}T^{(0)}T^{(1)}T^{(0)} = \begin{bmatrix}0 & 0\end{bmatrix}, \qquad \Pr(010) = 0.$$

This is the shortest example of an odd run of 1s occurring between two 0s, which the Even Process forbids.

(d) Expanding the product of the sum of the symbol matrices gives

$$T^n = \left(\sum_{a\in\mathcal{X}} T^{(a)}\right)^n = \sum_{w\in\mathcal{X}^n} T^{(w_1)}\cdots T^{(w_n)}.$$

Multiplying on the left by $\eta^{(\emptyset)}$ and on the right by $\mathbf{1}$ therefore yields

$$\eta^{(\emptyset)}T^n\mathbf{1} = \sum_{w\in\mathcal{X}^n} \eta^{(\emptyset)}T^{(w_1)}\cdots T^{(w_n)}\mathbf{1} = \sum_{w\in\mathcal{X}^n} \Pr(w).$$

Since T is row-stochastic, $T\mathbf{1} = \mathbf{1}$ and hence $T^n\mathbf{1} = \mathbf{1}$. Finally, $\eta^{(\emptyset)}\mathbf{1} = 1$, because the initial vector is a probability distribution. The required sum is therefore one.

Exercise 4.2: Solution p. 248

(a) Multiplying the initial belief by the two symbol matrices gives

$$\eta^{(\emptyset)}T^{(0)} = \begin{bmatrix}(1-p)/3 & 1/3 & 0\end{bmatrix}, \qquad \eta^{(\emptyset)}T^{(1)} = \begin{bmatrix}p/3 & 0 & 1/3\end{bmatrix}.$$

Their row sums are respectively $(2-p)/3$ and $(1+p)/3$. Normalising therefore gives

$$\eta^{(0)} = \begin{bmatrix}\dfrac{1-p}{2-p} & \dfrac{1}{2-p} & 0\end{bmatrix}, \qquad \eta^{(1)} = \begin{bmatrix}\dfrac{p}{1+p} & 0 & \dfrac{1}{1+p}\end{bmatrix}.$$

After observing 0, the current phase is either 0 or 1 with the displayed posterior probabilities, and cannot be R. After observing 1, it is either 0 or R, and cannot be 1.

(b) Direct evaluation gives

$$\begin{aligned}\eta^{(00)} &= \begin{bmatrix}0 & 1 & 0\end{bmatrix}, & \eta^{(01)} &= \begin{bmatrix}0 & 0 & 1\end{bmatrix},\\ \eta^{(10)} &= \begin{bmatrix}1-p & p & 0\end{bmatrix}, & \eta^{(11)} &= \begin{bmatrix}1 & 0 & 0\end{bmatrix}.\end{aligned}$$

Thus the pure beliefs are $\eta^{(11)}$, $\eta^{(00)}$, and $\eta^{(01)}$. The pure beliefs are distinct. The beliefs $\eta^{(\emptyset)}$, $\eta^{(0)}$, and $\eta^{(1)}$ have different supports from one another and from the pure beliefs. The only remaining comparison is between $\eta^{(0)}$ and $\eta^{(10)}$, which both have support $\{0,1\}$. Equality would require $1/(2-p) = p$, or $(1-p)^2 = 0$, contrary to $p < 1$.

The allowable updates are

current belief	0	1
$\eta^{(\emptyset)}$	$\eta^{(0)}$	$\eta^{(1)}$
$\eta^{(0)}$	$\eta^{(00)}$	$\eta^{(01)}$
$\eta^{(1)}$	$\eta^{(10)}$	$\eta^{(11)}$
$\eta^{(10)}$	$\eta^{(00)}$	$\eta^{(01)}$
$\eta^{(11)}$	$\eta^{(00)}$	impossible
$\eta^{(00)}$	impossible	$\eta^{(01)}$
$\eta^{(01)}$	$\eta^{(11)}$	$\eta^{(11)}$

This table specifies the requested symbol-labelled diagram and shows that every allowable update stays in $\mathcal{B}$.

(c)

1. Under the assumptions used in Part 2, the correspondence is

$$\begin{aligned}&\epsilon(\emptyset) \longleftrightarrow \eta^{(\emptyset)}, \quad \epsilon(0) \longleftrightarrow \eta^{(0)}, \quad \epsilon(1) \longleftrightarrow \eta^{(1)},\\ &\epsilon(10) \longleftrightarrow \eta^{(10)}, \quad \epsilon(11) \longleftrightarrow \eta^{(11)}, \quad \epsilon(00) \longleftrightarrow \eta^{(00)},\\ &\epsilon(01) \longleftrightarrow \eta^{(01)}.\end{aligned}$$

The transition table from part (b) agrees with the causal-state automaton after making these substitutions.

2. The pure beliefs $\eta^{(11)}$, $\eta^{(00)}$, and $\eta^{(01)}$ identify the hidden phase exactly. The other four beliefs assign positive probability to more than one phase and therefore represent uncertainty. After two observations only the history 10 leaves a mixed belief. Either allowable third symbol sends $\eta^{(10)}$ to a pure belief. Once the belief is pure, every allowable transition in the table leads to another pure belief, so the phase remains known.

3. The generator has access to its actual hidden phase and needs only the three states 0, 1, and R. An observer begins with an unknown phase and must also represent four transient posterior distributions before synchronising with the generator. This example therefore shows that inference can require a richer state description than a particular generative presentation.

(d) The probability of each symbol from a hidden phase is obtained by summing the corresponding row of its symbol matrix over destination states. With column order $(0, 1)$, this gives

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1-p & p \end{bmatrix}.$$

Hence

belief	next-token vector
$\eta^{(\emptyset)}$	$p^{(\emptyset)} = \left(\dfrac{2-p}{3}, \dfrac{1+p}{3}\right)$
$\eta^{(0)}$	$p^{(0)} = \left(\dfrac{1-p}{2-p}, \dfrac{1}{2-p}\right)$
$\eta^{(1)}$	$p^{(1)} = \left(\dfrac{1}{1+p}, \dfrac{p}{1+p}\right)$
$\eta^{(00)}$	$p^{(00)} = (0, 1)$
$\eta^{(01)}$	$p^{(01)} = (1-p, p)$
$\eta^{(10)}$	$p^{(10)} = (1-p, p)$
$\eta^{(11)}$	$p^{(11)} = (1, 0)$

On the belief 2-simplex, $\eta^{(11)}$, $\eta^{(00)}$, and $\eta^{(01)}$ are the three vertices, while $\eta^{(\emptyset)}$ is the barycentre. The beliefs $\eta^{(0)}$ and $\eta^{(10)}$ lie on the edge joining $\eta^{(11)}$ to $\eta^{(00)}$, and $\eta^{(1)}$ lies on the edge joining $\eta^{(11)}$ to $\eta^{(01)}$.

On the next-token 1-simplex, each point is located by the second coordinate in the table, namely its probability of emitting 1. The relation $p^{(x_{1:n})} = \eta^{(x_{1:n})} A$ maps the two distinct beliefs $\eta^{(01)}$ and $\eta^{(10)}$ to the same point $p^{(01)} = p^{(10)} = (1-p, p)$.

(e) Starting from $\eta^{(10)} = (1-p, p, 0)$, observing a 0 can only come from phase 0 and moves the process to phase 1, which cannot emit a second 0. Hence

$$\Pr(X_{3:4} = 00 \mid X_{1:2} = 10) = \eta^{(10)} T^{(0)} T^{(0)} \mathbf{1} = 0.$$

Starting from $\eta^{(01)} = (0, 0, 1)$, the first 0 has probability $1 - p$ and moves the process to phase 0, which emits the second 0 deterministically. Therefore

$$\Pr(X_{3:4} = 00 \mid X_{1:2} = 01) = \eta^{(01)} T^{(0)} T^{(0)} \mathbf{1} = 1 - p.$$

The histories agree about the next symbol but disagree about a two-symbol continuation, so their shared next-token vector is not sufficient for the entire future.

(f) We have

$$\delta = \begin{bmatrix} 1 - p & p & -1 \end{bmatrix}.$$

Direct multiplication gives

$$\delta A = \begin{bmatrix} 0 & 0 \end{bmatrix}, \qquad \delta T^{(0)} T^{(0)} \mathbf{1} = -(1 - p) \neq 0.$$

Thus the difference between the beliefs is invisible to the one-step readout but visible to a readout associated with the two-symbol future 00.

Exercise 4.3: Solution p. 250

(a)

1. After observing 1, the hidden state is A. A path emitting the following 0^n either remains in A throughout or moves from A to B after one of the n emissions. These $n + 1$ paths have equal weight 2^{-n}; one ends in A and n end in B. Thus

$$\eta^{(1)} (T^{(0)})^n = 2^{-n} \begin{bmatrix} 1 & n \end{bmatrix}, \qquad \eta^{(10^n)} = \begin{bmatrix} \dfrac{1}{n+1} & \dfrac{n}{n+1} \end{bmatrix}.$$

Starting instead from the uniform initial belief gives

$$\eta^{(\emptyset)} (T^{(0)})^n = 2^{-(n+1)} \begin{bmatrix} 1 & n+1 \end{bmatrix}, \qquad \eta^{(0^n)} = \begin{bmatrix} \dfrac{1}{n+2} & \dfrac{n+1}{n+2} \end{bmatrix}.$$

Hence $\eta^{(0^n)} = \eta^{(10^{n+1})}$, and the case $n = 0$ gives $\eta^{(\emptyset)} = \eta^{(10)}$.

2. For $n \geq 1$, direct multiplication gives

$$\eta^{(10^n)} T^{(1)} = \begin{bmatrix} \dfrac{n}{2(n+1)} & 0 \end{bmatrix}.$$

Normalising yields $\eta^{(10^n 1)} = \begin{bmatrix} 1 & 0 \end{bmatrix} = \eta^{(1)}$.

3. The automaton begins

$$\eta^{(1)} \xrightarrow{0} \eta^{(10)} \xrightarrow{0} \eta^{(100)} \xrightarrow{0} \eta^{(1000)} \xrightarrow{0} \cdots,$$

and every $\eta^{(10^n)}$ with $n \geq 1$ has a 1-transition back to $\eta^{(1)}$. Since the first coordinate $1/(n+1)$ is different for each n, the beliefs are distinct, and

$$\lim_{n\to\infty} \eta^{(10^n)} = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

This limiting belief is not reached after any finite history. Hence the two hidden generator states induce countably infinitely many reachable beliefs.

(b)

1. Substitution gives $\alpha y = 7/50$, $bx = 3/50$, $\alpha x = 3/100$, and $by = 7/25$. Each entry is positive, and every state has several possible destinations for each emitted symbol, so the generator is nonunifilar. The rows of $T^{(0)} + T^{(1)} + T^{(2)}$ each sum to one. By multiplying the uniform belief and normalising,

$$\eta^{(0)} = \begin{bmatrix} \frac{1}{5} & \frac{2}{5} & \frac{2}{5} \end{bmatrix}, \quad \eta^{(1)} = \begin{bmatrix} \frac{2}{5} & \frac{1}{5} & \frac{2}{5} \end{bmatrix}, \quad \eta^{(2)} = \begin{bmatrix} \frac{2}{5} & \frac{2}{5} & \frac{1}{5} \end{bmatrix}.$$

2. Updating $\eta^{(0)}$ by each symbol gives

$$\eta^{(00)} = \begin{bmatrix} \frac{13}{87} & \frac{37}{87} & \frac{37}{87} \end{bmatrix},$$
$$\eta^{(01)} = \begin{bmatrix} \frac{52}{163} & \frac{37}{163} & \frac{74}{163} \end{bmatrix}, \quad \eta^{(02)} = \begin{bmatrix} \frac{52}{163} & \frac{74}{163} & \frac{37}{163} \end{bmatrix}.$$

Cyclically permuting both the state coordinates and symbol labels gives the remaining six depth-two beliefs. The root has three symbol-labelled children, and each of those children again has three symbol-labelled successors.

3. Summing over destination states gives

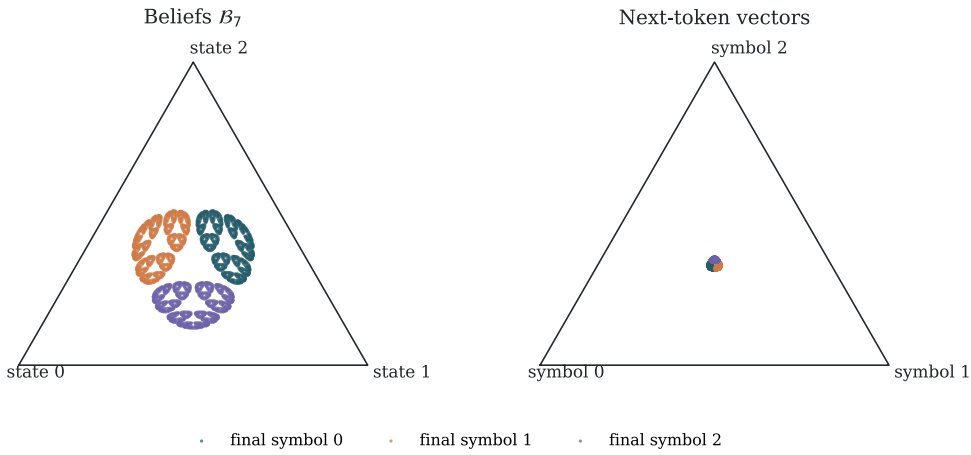
$$A = \begin{bmatrix} \frac{13}{50} & \frac{37}{100} & \frac{37}{100} \\ \frac{37}{100} & \frac{13}{50} & \frac{37}{100} \\ \frac{37}{100} & \frac{37}{100} & \frac{13}{50} \end{bmatrix}.$$

Its eigenvalues are $1, -11/100, -11/100$, so $\det A = 121/10000 \neq 0$. Consequently $\eta A = \eta' A$ implies $\eta = \eta'$: the next-token readout does not merge distinct Mess3 beliefs. In Exercise 4.2 the corresponding matrix has a nontrivial kernel and maps $\eta^{(01)}$ and $\eta^{(10)}$ to the same next-token vector.

4. For these parameters, numerical enumeration gives

d	0	1	2	3	4	5	6	7
$\lvert\mathcal{B}_d\rvert$	1	3	9	27	81	243	729	2187

with no coincidences at the displayed depths. The plots show three recursively repeated clusters forming a fractal pattern. The next-token plot is an invertible linear image of the belief plot, so it preserves the distinct points and their recursive organisation.



At a point η, the three outgoing edges are obtained by applying F_0, F_1, F_2 and have probabilities given by the three coordinates of ηA. Thus the three geometric copies visible at the next depth are also the three symbol-labelled branches of the belief-state automaton.

5. A belief and the observed symbol determine a unique updated belief $F_a(\eta)$, so the belief-state presentation is unifilar. The original generator is nonunifilar because its current hidden state and emitted symbol can leave several possible next hidden states. For the Simple Nonunifilar Source the reachable beliefs form a one-dimensional ladder, with each 1-edge resetting to η_0. For Mess3 every symbol remains possible and the three update maps generate a rapidly branching, self-similar geometry. Finally, finite words form a countable set, so their image under the belief map is countable. Taking the closure adds limiting beliefs and can produce an uncountable fractal set.

Exercise 4.4: Solution p. 252

(a)

1. Nonnegativity is one of the GHMM assumptions. Expanding the product of the sum of the symbol matrices gives

$$\sum_{x_{1:n}\in\mathcal{X}^n} T^{(x_1)}\cdots T^{(x_n)} = T^n.$$

Hence

$$\sum_{x_{1:n}\in\mathcal{X}^n} \Pr(X_{1:n}=x_{1:n}) = \eta^{(\emptyset)}T^n\phi = \eta^{(\emptyset)}\phi = 1,$$

where $T^n\phi = \phi$ follows from $T\phi = \phi$. For $n = 0$ this also says that the empty history has probability one.

2. Summing over the final symbol gives

$$\begin{aligned}\sum_{a\in\mathcal{X}} \Pr(X_{1:n+1} = x_{1:n}a) &= \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \left(\sum_{a\in\mathcal{X}} T^{(a)}\right) \phi \\ &= \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} T \phi \\ &= \Pr(X_{1:n} = x_{1:n}).\end{aligned}$$

Thus the length-n distribution is the marginal of the length-$(n+1)$ distribution over its final coordinate, as required for a process indexed by $1, 2, \ldots$.

3. The displayed GHMM conditions relate distributions at successive lengths but do not say that a block has the same distribution after shifting its time indices. If $\eta^{(\emptyset)} T = \eta^{(\emptyset)}$, then

$$\begin{aligned}\sum_{a\in\mathcal{X}} \Pr(X_{1:n+1} = a x_{1:n}) &= \eta^{(\emptyset)} T T^{(x_1)} \cdots T^{(x_n)} \phi \\ &= \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \phi \\ &= \Pr(X_{1:n} = x_{1:n}).\end{aligned}$$

The left side is $\Pr(X_{2:n+1} = x_{1:n})$, so the block distributions are invariant under a one-step shift. Iteration gives stationarity.

(b)

1. The denominator in the definition is $\Pr(X_{1:n} = x_{1:n})$, and so

$$\eta^{(x_{1:n})} \phi = 1.$$

By the definition of conditional probability,

$$\begin{aligned}&\Pr(X_{n+1:n+m} = y_{1:m} \mid X_{1:n} = x_{1:n}) \\ &\quad = \frac{\eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} T^{(y_1)} \cdots T^{(y_m)} \phi}{\eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \phi} \\ &\quad = \eta^{(x_{1:n})} T^{(y_1)} \cdots T^{(y_m)} \phi.\end{aligned}$$

2. Taking $m = 1$ in part (i) gives

$$\Pr(X_{n+1} = a \mid X_{1:n} = x_{1:n}) = \eta^{(x_{1:n})} T^{(a)} \phi, \quad \eta^{(x_{1:n}a)} = \frac{\eta^{(x_{1:n})} T^{(a)}}{\eta^{(x_{1:n})} T^{(a)} \phi}.$$

The first quantity is exactly the normalising factor in the update shown by the second equality.

3. An HMM belief state has coordinates $\Pr(S_n = i \mid X_{1:n} = x_{1:n})$, so they are nonnegative, sum to one, and refer to mutually exclusive hidden states. Predictive vectors are coordinates in a linear realisation: they need only satisfy $\eta^{(x_{1:n})} \phi = 1$ and may have negative entries. Both summarise the history sufficiently to compute future-word probabilities through symbol-matrix products.

(c)

1. Since

$$S^{-1} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ 0 & 1 \end{bmatrix},$$

direct calculation gives

$$T^{(0)} = \begin{bmatrix} \frac{4}{5} & -\frac{3}{10} \\ 0 & \frac{1}{5} \end{bmatrix}, \qquad T^{(1)} = \begin{bmatrix} \frac{1}{5} & \frac{3}{10} \\ 0 & \frac{4}{5} \end{bmatrix},$$

and

$$\eta^{(\emptyset)} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad \phi = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

In particular $T = T^{(0)} + T^{(1)} = I$, so $T\phi = \phi$, and $\eta^{(\emptyset)}\phi = 1$. In a product, adjacent factors SS^{-1} cancel:

$$\begin{aligned} \eta^{(\emptyset)} T^{(x_1)} \cdots T^{(x_n)} \phi &= \alpha S (S^{-1} M^{(x_1)} S) \cdots (S^{-1} M^{(x_n)} S) S^{-1} \mathbf{1} \\ &= \alpha M^{(x_1)} \cdots M^{(x_n)} \mathbf{1}. \end{aligned}$$

The right side is an HMM sequence probability and is therefore nonnegative. All GHMM conditions are satisfied, and both realisations define the same process.

2. Both first-symbol probabilities are $1/2$. Normalising gives

$$\eta^{(0)} = \begin{bmatrix} \frac{8}{5} & -\frac{3}{5} \end{bmatrix}, \qquad \eta^{(1)} = \begin{bmatrix} \frac{2}{5} & \frac{3}{5} \end{bmatrix}.$$

Moreover,

$$T^{(0)}\phi = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{5} \end{bmatrix}, \qquad T^{(1)}\phi = \begin{bmatrix} \frac{1}{2} \\ \frac{4}{5} \end{bmatrix}.$$

Hence

$$\eta^{(0)} T^{(0)} \phi = \frac{17}{25}, \qquad \eta^{(0)} T^{(1)} \phi = \frac{8}{25}.$$

Although the second coordinate of $\eta^{(0)}$ is negative, all continuation probabilities obtained from it are valid. The predictive vector consists of coordinates in a chosen linear representation; unlike an HMM belief, its entries do not have to describe mutually exclusive hidden events.

(d)

1. For $u = u_1 \cdots u_r$ and $v = v_1 \cdots v_s$,

$$\mathsf{P}_{\mathcal{U},\mathcal{V}}(u, v) = \big(\eta^{(\emptyset)} T^{(u_1)} \cdots T^{(u_r)}\big)\big(T^{(v_1)} \cdots T^{(v_s)} \phi\big).$$

Collecting the first factors as rows and the second factors as columns gives the requested factorisation through $\mathbb{R}^d$. The rank of the product is therefore at most d.

2. The required probabilities are

$$\begin{aligned}\Pr(X_1 = 0) = \Pr(X_1 = 1) &= \frac{1}{2},\\ \Pr(X_{1:2} = 00) = \Pr(X_{1:2} = 11) &= \frac{17}{50},\\ \Pr(X_{1:2} = 01) = \Pr(X_{1:2} = 10) &= \frac{4}{25}.\end{aligned}$$

Thus, in the order $(\emptyset, 0, 1)$,

$$\mathsf{P}_{\mathcal{U},\mathcal{V}} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{17}{50} & \frac{4}{25} \\ \frac{1}{2} & \frac{4}{25} & \frac{17}{50} \end{bmatrix}, \qquad \sigma(\mathsf{P}_{\mathcal{U},\mathcal{V}}) = \left(\frac{3}{2}, \frac{9}{50}, 0\right).$$

It therefore has rank two. Every GHMM realisation must have dimension at least two, while part (c) supplies a two-dimensional realisation. It is minimal-dimensional.

SOLUTIONS FOR CHAPTER C.4

Condensation

Satya Benson

Main text: Chapter C.4, p. 255

Exercise 2.1: Solution p. 258

The independent fair bits are S, P, Q, and $X_1 = (S, P)$, $X_2 = (S, Q)$, $X_3 = (P, Q)$.

(a) $H(X_1) = 2$. The tuple of all observations determines (S, P, Q) and is determined by it, so $H(X_1, X_2, X_3) = 3$. The pair (X_1, X_2) already determines all three bits, hence

$$I(X_1; X_2) = H(X_1) + H(X_2) - H(X_1, X_2) = 2 + 2 - 3 = 1.$$

Its shared bit is S.

(b) The families, including constant slots, are

$$\supseteq\{1\} = \{\{1\}, \{1,2\}, \{1,3\}, \{1,2,3\}\},$$

$$\supseteq\{2\} = \{\{2\}, \{1,2\}, \{2,3\}, \{1,2,3\}\},$$

$$\supseteq\{1,2\} = \{\{1,2\}, \{1,2,3\}\}.$$

The first tuple has nonconstant entries S, P, so determines X_1. Intersecting the first two families leaves $\{\{1,2\}, \{1,2,3\}\} = \supseteq\{1,2\}$, the third family listed, whose only nonconstant content is S.

(c) Recover X_1 by taking the ordered pair $(Y_{\{1,2\}}, Y_{\{1,3\}})$. Recover $Y_{\{1,2\}}$ from X_1 by projecting onto its first coordinate.

(d) Addresses $\{1,2\}$ and $\{1,3\}$ overlap but neither contains the other. Thus the contribution pattern need not form a nested tree.

The original model also satisfies the Markov condition. All nonconstant slots are mutually independent, so disjoint subcollections are independent, including after conditioning on the remaining common subcollection. Reconstruction follows from the indicated coordinate projections.

Exercise 2.2: Solution p. 259

(a) In the singleton-copy model $Y_{\{i\}} = X_i$, the contributing family for i includes X_i itself. Thus LVM holds. Each nonconstant latent has just one contributing observation, from which it is recovered identically, so reconstruction holds too.

For $\mathcal{F} = \supseteq\{1\}$ and $\mathcal{G} = \supseteq\{2\}$, all entries of $Y_{\mathcal{F}\cap\mathcal{G}}$ are constant. The only nonconstant entries of the two larger tuples are X_1 and X_2. Consequently

$$I(Y_{\mathcal{F}}; Y_{\mathcal{G}} \mid Y_{\mathcal{F}\cap\mathcal{G}}) = I(X_1; X_2) = 1.$$

Both singleton slots determine S, while their common ancestor slots are constant. This violates the Markov condition. It is enough to exhibit this one pair of upward-closed families; finding the maximum defect is unnecessary.

(b) With only $Y_{\{1,2,3\}} = (S, P, Q)$ nonconstant, each observation is obtained by projection, so LVM holds. Every nonempty upward-closed family contains the top address. If both families are nonempty, their intersection contains that top slot and determines all latent variables; the conditional mutual information is zero. If either is empty it is zero as well. Thus the Markov condition holds.

But $X_1 = (S, P)$ leaves the independent fair bit Q unknown, so

$$H(Y_{\{1,2,3\}} \mid X_1) = H(Q \mid S, P) = 1.$$

Reconstruction fails.

(c) LVM requires enough latent information to determine each observation. Reconstruction excludes information placed at a shared address that an individual contributing observation cannot recover. The Markov condition excludes dependence between address collections not accounted for by their common slots. The singleton model satisfies the second requirement but fails the third; the top-slot model satisfies the third but fails the second.

Check on the existence example. Section 2.2.1 is a proved example, not an additional exercise. For a binary pair with full support, a top slot determined by each observable must be constant: writing it as $f(X_1) = g(X_2)$, each of the four pairs imposes an equality between the corresponding function values. With a constant top, LVM makes X_1 a function of $Y_{\{1\}}$ and X_2 a function of $Y_{\{2\}}$. Thus data processing yields

$$I(X_1; X_2) \le I(Y_{\{1\}}; Y_{\{2\}}) = \eta_Y.$$

The equality holds because, on the two-observable address poset, the only pair of incomparable nonempty upward-closed families consists of the two singleton towers. Every other Markov term is zero. For the noisy fair-bit pair, $H(X_2) = 1$ and $H(X_2 \mid X_1) = h_2(q)$, proving the stated lower bound. It is attained by the singleton-copy model $Y_{\{i\}} = X_i$, with constant top. This model has both exact reconstruction directions but a positive Markov defect. Setting that defect to zero is impossible. The argument does not bound the trade-off when reconstruction is allowed to be inexact.

Exercise 2.3: Solution p. 261

Here C, R, T are independent fair bits. In model Y the three slots are C, R, T; in model Z they are $C, (C,R), (C,T)$.

(a) Both contributing families determine their observations. All slots are recoverable from each observation to which they contribute. For example, C is the first coordinate of either X_1 or X_2, and $Z_{\{1\}} = X_1$. For two observables the only nontrivial Markov requirement is independence of the two singleton slots given the top slot. For Y this is $I(R;T \mid C) = 0$. For Z it is

$$I((C,R);(C,T) \mid C) = I(R;T \mid C) = 0.$$

Both are therefore perfect condensations.

(b) $H(Z_{\{1\}} \mid Y_{\{1\}}) = H(C,R \mid R) = H(C) = 1$.

(c) The Y tower at $\{1\}$ consists of (R,C), and the Z tower consists of $((C,R),C)$. Each is a deterministic function of the other. A copy of the higher-address information can appear in a lower slot without changing the tower's total information. Thus the individual slots need not correspond even when the relevant towers do.

Exercise 2.4: Hint p. 262

Try expanding $I(P;Q \mid M) - I(P;Q \mid U,M)$, then rearranging with the chain rule, which is stated in Section 1.1. The desired difference contains a conditional mutual information and a conditional entropy.

Exercise 2.4: Solution p. 262

For arbitrary finite U, M, P, Q, the exact identity is

$$H(U \mid M,P) + H(U \mid M,Q) + I(P;Q \mid M) - H(U \mid M)$$

$$= I(P;Q \mid U,M) + H(U \mid M,P,Q).$$

For a direct verification, first subtract the two conditional mutual information terms using their entropy definitions:

$$
\begin{aligned}
&I(P;Q \mid M) - I(P;Q \mid U,M) \\
&\quad = H(P \mid M) + H(Q \mid M) - H(P,Q \mid M) \\
&\qquad - H(P \mid U,M) - H(Q \mid U,M) + H(P,Q \mid U,M) \\
&\quad = I(U;P \mid M) + I(U;Q \mid M) - I(U;P,Q \mid M) \\
&\quad = H(U \mid M) - H(U \mid P,M) - H(U \mid Q,M) \\
&\qquad + H(U \mid P,Q,M).
\end{aligned}
$$

Rearrangement gives the displayed slack identity. Its right side is nonnegative, proving the lemma.

If $H(U \mid M,P) = H(U \mid M,Q) = I(P;Q \mid M) = 0$, the inequality and nonnegativity imply $H(U \mid M) = 0$. Thus a target recoverable from each witness is recoverable from their designated common part when the witnesses are conditionally independent given that part.

The first two terms measure what each witness, together with M, still fails to determine about U. The third measures dependence between P,Q not accounted for by M. No independence assumption is needed for the inequality itself; exact independence is the case where the third error is zero.

Exercise 2.5: Solution p. 262

With $U = P = Q$ a fair bit and M constant,

$$H(U \mid M) = 1, \quad H(U \mid M,P) = H(U \mid M,Q) = 0, \quad I(P;Q \mid M) = 1.$$

The inequality is the equality $1 \le 0 + 0 + 1$. Without its mutual-information term it would assert $1 \le 0$. The two witnesses each determine the same bit while the designated common variable is constant, as in the singleton-copy model of Exercise 2.2.

Exercise 2.6: Solution p. 262

If X_i is a function of $Z_{\supseteq\{i\}}$ almost surely, adding X_i to that conditioning tuple changes no information. Hence

$$H(U \mid Z_{\supseteq\{i\}}) = H(U \mid Z_{\supseteq\{i\}}, X_i) \le H(U \mid X_i).$$

The inequality is conditioning monotonicity. It does not assume that U and the receiver tuple are conditionally independent given X_i. They can share additional information beyond X_i.

Exercise 2.7: Solution p. 263

(a) The first two families are listed in Exercise 2.1, and

$$\supseteq\{3\} = \{\{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}\}.$$

Thus $\supseteq\{1\} \cap \supseteq\{2\} = \{\{1,2\}, \{1,2,3\}\}$, and intersecting with $\supseteq\{3\}$ leaves only $\{1,2,3\}$.

(b) If C belongs to every family in an intersection and $D \supseteq C$, upward closure places D in each such family, hence in the intersection.

(c) For an address C,

$$C \in \bigcap_{i \in A} \supseteq\{i\} \iff (\forall i \in A)\, i \in C \iff A \subseteq C \iff C \in \supseteq A.$$

(d) Put $M = Z_{\mathcal{F}\cap\mathcal{G}}$, $P = Z_{\mathcal{F}}$, $Q = Z_{\mathcal{G}}$. The tuple M is a subtuple of both P and Q. Thus $H(U \mid M, P) = H(U \mid P)$ and similarly for Q. By the definition of η_Z, the lemma gives

$$H(U \mid Z_{\mathcal{F}\cap\mathcal{G}}) \le H(U \mid Z_{\mathcal{F}}) + H(U \mid Z_{\mathcal{G}}) + \eta_Z.$$

Upward closure is needed to invoke the bound η_Z.

Exercise 2.8: Solution p. 263

Fix a nonempty $A = \{i_1, \dots, i_k\}$, put $\mathcal{G}_j = \bigcap_{\ell=1}^{j} \supseteq\{i_\ell\}$. Every $\supseteq\{i\}$ is upward closed, as is every $\mathcal{G}_j$ by Exercise 2.7. We prove

$$H(Y_{\supseteq A} \mid Z_{\mathcal{G}_j}) \le \sum_{\ell=1}^{j} H(Y_{\supseteq A} \mid X_{i_\ell}) + (j-1)\eta_Z$$

for $1 \le j \le k$.

For $j = 1$, $\mathcal{G}_1 = \supseteq\{i_1\}$, and Exercise 2.6 gives $H(Y_{\supseteq A} \mid Z_{\mathcal{G}_1}) \le H(Y_{\supseteq A} \mid X_{i_1})$. This is the claimed bound with zero intersection terms.

Suppose (4) holds for $j-1$, where $2 \le j \le k$. Apply Exercise 2.7 to the upward-closed families $\mathcal{G}_{j-1}$ and $\supseteq\{i_j\}$. Their intersection is $\mathcal{G}_j$, so

$$\begin{aligned} H(Y_{\supseteq A} \mid Z_{\mathcal{G}_j}) &\le H(Y_{\supseteq A} \mid Z_{\mathcal{G}_{j-1}}) + H(Y_{\supseteq A} \mid Z_{\supseteq\{i_j\}}) + \eta_Z \\ &\le \sum_{\ell=1}^{j-1} H(Y_{\supseteq A} \mid X_{i_\ell}) + (j-2)\eta_Z + H(Y_{\supseteq A} \mid X_{i_j}) + \eta_Z \\ &= \sum_{\ell=1}^{j} H(Y_{\supseteq A} \mid X_{i_\ell}) + (j-1)\eta_Z. \end{aligned}$$

The second line uses the induction hypothesis and Exercise 2.6. This completes the induction. At $j = k$, Exercise 2.7 gives $\mathcal{G}_k = \supseteq A$, hence

$$H(Y_{\supseteq A} \mid Z_{\supseteq A}) \le \sum_{i \in A} H(Y_{\supseteq A} \mid X_i) + (|A| - 1)\eta_Z.$$

For $|A| = 1$ this is exactly the observable-transfer inequality, so no intersection or Markov error is required. There are k reconstruction errors and $k - 1$ Markov defects. Bounding each reconstruction error by δ and each Markov defect by η gives $k\delta + (k - 1)\eta$, or $(2k - 1)\varepsilon$ for a common budget.

If Y, Z are perfect condensations, each reconstruction error is zero: for every $C \supseteq A$ and $i \in A$, reconstruction says Y_C is a function of X_i. The finite tuple of such variables is consequently a function of X_i. Also $\eta_Z = 0$. The inequality gives zero conditional entropy in the forward direction. Interchanging Y, Z, using reconstruction in Z and $\eta_Y = 0$, proves the reverse direction. Zero conditional entropy is almost-sure deterministic recoverability in our finite setting.

The proof uses LVM in Z for Exercise 2.6, the sender's reconstruction to bound those errors, and the receiver's Markov condition at intersections. It needs no cross-model independence, and its finite coupling preserves all marginal conditions used.

Finally, approximate slotwise errors do not directly give the same error for a tuple. For example, if two independent fair bits R, T remain unknown given a constant observation, each conditional entropy is one bit but the tuple's is two bits. Conditional subadditivity gives a sum of the slot budgets, not a single budget. The theorem states its hypothesis in terms of tower errors to avoid this ambiguity.

Exercise 2.9: Solution p. 522

(a) A nonempty address $C \subseteq \{1, 2, 3\}$ meets each of $\{1, 2\}, \{1, 3\}, \{2, 3\}$ precisely when $|C| \ge 2$. A singleton misses the complementary pair; every address with at least two elements meets every pair. Thus the surviving tuple consists of the three pair slots and the top slot. Applying the two-witness lemma twice, after observable transfer for each block, gives

$$H(U \mid Z_{\{C : |C| \ge 2\}}) \le 3\delta + 2\eta_Z.$$

It does not isolate the top slot alone.

(b) For each block B, the collection $Z_{\mathcal{J}_B}$ determines X_B: for every $i \in B$ it contains $Z_{\supseteq\{i\}}$. Exercise 2.6 therefore extends to $H(U \mid Z_{\mathcal{J}_B}) \le H(U \mid X_B)$. Each $\mathcal{J}_B$ is upward closed. Enumerate $\mathcal{W} = \{B_1, \ldots, B_N\}$ and intersect the first j contributing families. Exactly the induction in Exercise 2.8, now with these families as leaves, gives

$$H(U \mid Z_{\mathcal{R}}) \le \sum_{B \in \mathcal{W}} H(U \mid X_B) + (N - 1)\eta_Z, \quad \mathcal{R} = \bigcap_{B \in \mathcal{W}} \mathcal{J}_B.$$

For $N = 1$ this is just the transfer inequality. We required a nonempty family so the induction and the intersection count apply.

(c) An address C fails to meet some $(r+1)$-subset of A exactly when $A \setminus C$ contains at least $r+1$ elements. Thus it meets every such subset exactly when $|A \setminus C| \le r$. For $r = 0$ this says $A \subseteq C$, recovering the tower. For $r > 0$ an address can survive even if it misses some elements of A; the receiver is a larger tuple. If every witness error is at most δ, write $N = \binom{|A|}{r+1}$ to obtain the explicit bound $N\delta + (N-1)\eta_Z$. The restriction $r < |A|$ ensures witnesses exist.

Exercise 2.10: Solution p. 523

(a) List $\mathcal{J}_B$ as $A_1, \dots, A_q$ with strict supersets appearing before subsets. Such an order exists because strict inclusion has no cycles. Every strict superset of an active address also meets B, so is in this list. Therefore

$$H(Y_{A_t} \mid Y_{\supseteq A_t \setminus \{A_t\}}) \ge H(Y_{A_t} \mid Y_{A_1}, \dots, Y_{A_{t-1}}).$$

Summing and applying the chain rule gives $\chi(B) \ge H(Y_{\mathcal{J}_B})$, hence $V_B \ge 0$.

(b) Since $Y_{\mathcal{J}_B}$ determines X_B, the joint entropy can be expanded in two ways:

$$H(X_B, Y_{\mathcal{J}_B}) = H(Y_{\mathcal{J}_B}) = H(X_B) + H(Y_{\mathcal{J}_B} \mid X_B).$$

Subtracting $H(X_B)$ and adding V_B proves

$$\chi(B) - H(X_B) = V_B + H(Y_{\mathcal{J}_B} \mid X_B).$$

(c) In Exercise 2.3, the independent model Y has simple score $1+1+1=3$ and conditioned score $1+1+1=3$. In model Z, the simple score is $1+2+2=5$, while the conditioned score is

$$H(C) + H(C, R \mid C) + H(C, T \mid C) = 1 + 1 + 1 = 3.$$

Both models determine X_1, X_2, whose joint entropy is three. The conditioned score charges only the extra information below C; its copies add zero conditional entropy. The simple score charges those copies again.

(d) If $A \cap B \neq \varnothing$, every $C \supseteq A$ meets B. Thus $Y_{\supseteq A}$ is a subtuple of $Y_{\mathcal{J}_B}$ and

$$H(Y_{\supseteq A} \mid X_B) \le H(Y_{\mathcal{J}_B} \mid X_B) \le \chi(B) - H(X_B),$$

where the last inequality uses the nonnegative score decomposition.

Additional check: equivalence of the perfect definitions. This argument is for readers who want to connect the two definitions completely.

Let $V_I = \chi(I) - H(Y_{\mathcal{P}^+(I)})$. For upward-closed $\mathcal{F}, \mathcal{G}$, partition $\mathcal{P}^+(I)$ into

$$\mathcal{C} = \mathcal{F} \cap \mathcal{G}, \quad \mathcal{P} = \mathcal{F} \setminus \mathcal{G}, \quad \mathcal{Q} = \mathcal{G} \setminus \mathcal{F}, \quad \mathcal{R} = \mathcal{P}^+(I) \setminus (\mathcal{F} \cup \mathcal{G}).$$

Order these blocks $\mathcal{C}, \mathcal{P}, \mathcal{Q}, \mathcal{R}$, internally by reverse inclusion. Upward closure ensures this order is valid and no slot in $\mathcal{Q}$ has a strict superset in $\mathcal{P}$, or conversely. Conditioning inequalities and chain rules within each block imply

$$\begin{aligned}\chi(I) &\geq H(Y_{\mathcal{C}}) + H(Y_{\mathcal{P}} \mid Y_{\mathcal{C}}) + H(Y_{\mathcal{Q}} \mid Y_{\mathcal{C}}) + H(Y_{\mathcal{R}} \mid Y_{\mathcal{C}}, Y_{\mathcal{P}}, Y_{\mathcal{Q}}) \\ &= H(Y_{\mathcal{P}^+(I)}) + I(Y_{\mathcal{P}}; Y_{\mathcal{Q}} \mid Y_{\mathcal{C}}).\end{aligned}$$

The latter mutual information equals $I(Y_{\mathcal{F}}; Y_{\mathcal{G}} \mid Y_{\mathcal{F}\cap\mathcal{G}})$. Hence $\eta_Y \leq V_I$.

If all score gaps vanish, the decomposition forces $V_I = 0$, giving the Markov condition. The singleton gap at $B = \{i\}$ forces the contributing tuple, and therefore each slot at an address containing i, to be determined by X_i. This gives reconstruction.

Conversely, suppose reconstruction and the Markov condition hold. For every B, each active slot Y_A is determined by some X_i with $i \in A \cap B$, and hence by X_B. Therefore $H(Y_{\mathcal{J}_B} \mid X_B) = 0$. In a reverse-inclusion order, the earlier slots consist of all strict supersets of the current slot and some incomparable slots. The global Markov condition implies that conditioning on these extra earlier slots does not reduce its entropy beyond conditioning on strict supersets: apply the condition to the upward cone of the current address and the upward-closed family of earlier addresses. Their intersection is the strict-superset family. Thus the score's chain-rule inequality is an equality, $V_B = 0$. The decomposition gives $\chi(B) = H(X_B)$ for all B.

Preferences to Rewards

Fernando E. Rosas

Main text: Chapter D.1.1, p. 277

Exercise 5.1: Solution p. 287

(a) Induction on $|X|$. A single trajectory is its own best and worst element. Given a maximal b' and minimal w' for $X \setminus \{h\}$, completeness compares h with each of them and transitivity makes the better of $\{h, b'\}$ maximal and the worse of $\{h, w'\}$ minimal for X.

(b) If $b \sim w$ then every h is indifferent to both and any constant u works. Otherwise $b \succ w$. By (a) we have $b \succcurlyeq h \succcurlyeq w$, so continuity applied to the triple (b, h, w) gives directly some $\lambda \in [0,1]$ with $h \sim \lambda b + (1-\lambda)w$; set $u(h) \coloneqq \lambda$. For uniqueness, independence implies the mixtures $\alpha b + (1-\alpha)w$ are strictly increasing in α when $b \succ w$, so two different weights cannot both be indifferent to h.

(c) Suppose $u(h) \geq u(h')$. Mixture monotonicity gives $u(h)b + (1-u(h))w \succcurlyeq u(h')b + (1-u(h'))w$, and chaining the two calibrating indifferences through transitivity yields $h \succcurlyeq h'$. If $u(h) < u(h')$ the same argument gives $h' \succ h$. Together: $h \succcurlyeq h' \iff u(h) \geq u(h')$.

(d) Independence states that $\mu \sim \nu$ implies $p\mu + (1-p)\lambda \sim p\nu + (1-p)\lambda$. View μ as a mixture in which h_1 appears with weight p_1 against the rest; replacing h_1 by its calibrated equivalent $u(h_1)b + (1-u(h_1))w$ therefore leaves the whole lottery indifferent. Doing this for each h_i in turn — finitely many steps — and chaining with transitivity gives $\mu \sim \sum_i p_i \big(u(h_i)b + (1-u(h_i))w\big)$.

(e) The right-hand side is a compound lottery whose only outcomes are b and w; it awards b with total probability $\sum_i p_i u(h_i) = U(\mu)$. Identifying a compound lottery with the simple lottery it induces gives $\mu \sim U(\mu)b + (1-U(\mu))w$.

(f) By (e), μ and ν are indifferent to calibrated b–w mixtures with weights $U(\mu)$ and $U(\nu)$; by mixture monotonicity and transitivity, $\mu \succcurlyeq \nu \iff U(\mu) \geq U(\nu)$.

(g) Apply the u'-representation to the calibration of h:

$$\begin{aligned} u'(h) &= U'\big(u(h)b + (1-u(h))w\big) = u(h)\,u'(b) + (1-u(h))\,u'(w) \\ &= u'(w) + \big(u'(b) - u'(w)\big)u(h). \end{aligned}$$

This is the required form $u' = a + b\,u$, with intercept $u'(w)$ and slope $u'(b) - u'(w)$, the latter strictly positive because $b \succ w$ and u' represents $\succcurlyeq$. (Note the statement's scalars a, b are unrelated to the trajectories b, w of part (a); the letter b does double duty here, so read $u'(b)$ as the utility of the best trajectory throughout.)

(h) Let $\mu \succcurlyeq \nu \iff \mathbb{E}_\mu[u] \geq \mathbb{E}_\nu[u]$. Completeness and transitivity are inherited from the total order on $\mathbb{R}$. Continuity: $p \mapsto \mathbb{E}_{p\mu+(1-p)\lambda}[u] = p\,\mathbb{E}_\mu[u] + (1-p)\,\mathbb{E}_\lambda[u]$ is continuous, so upper and lower contour sets in p are closed. Independence: mixing both sides with λ adds the same $(1-p)\mathbb{E}_\lambda[u]$ and scales the difference by $p > 0$, leaving the comparison unchanged.

Exercise 5.2: Solution p. 288

(a) For instance $u_1(x, y, z) = (2, 1, 0)$ and $u_2(x, y, z) = (100, 5, -3)$.

(b) An ordinal representation encodes only the ranking: u represents $\succcurlyeq$ iff $u(h) \geq u(h') \iff h \succcurlyeq h'$, and both functions induce $x \succ y \succ z$. Composing with any strictly increasing map preserves exactly this information, so no ordinal criterion can distinguish u_1 from u_2.

(c) Under expected utility the indifference forces $u(y) = \frac{1}{2}u(x) + \frac{1}{2}u(z)$. Take the vNM utility $u(x, y, z) = (1, \frac{1}{2}, 0)$ and the strictly increasing map $f(s) = s^3$. Then $(f \circ u)(x, y, z) = (1, \frac{1}{8}, 0)$, while the lottery $\frac{1}{2}x + \frac{1}{2}z$ has expected value $\frac{1}{2}$. Since $\frac{1}{8} \neq \frac{1}{2}$, the relation represented by $f \circ u$ strictly prefers the lottery to y: the indifference is destroyed. Only positive affine transformations preserve all such midpoint identities, which is the uniqueness part of the vNM theorem.

Exercise 5.3: Solution p. 290

(a) Let $x \succ y$, $x \succ z$, with y and z incomparable (neither $y \succcurlyeq z$ nor $z \succcurlyeq y$). Any $u\colon \{x, y, z\} \to \mathbb{R}$ satisfies $u(y) \geq u(z)$ or $u(z) \geq u(y)$ because the reals are totally ordered, so the relation it represents is complete; incomparability cannot be encoded by a single real-valued function.

(b) Take $x \succ y \succ z \succ x$. Suppose you hold x and are willing to pay some small $\varepsilon > 0$ to exchange an item for one you strictly prefer. Since $z \succ x$ you pay to swap $x \to z$; since $y \succ z$ you pay to swap $z \to y$; since $x \succ y$ you pay to swap $y \to x$. You are back where you started, 3ε poorer, and the cycle can be run forever.

(c) Give outcomes two attributes and compare lexicographically (the first attribute decides unless tied): $A = (1, 0)$, $B = (0, 1)$, $C = (0, 0)$, so $A \succ B \succ C$. Extend to lotteries by comparing expected attribute vectors lexicographically. Continuity would require some $p \in [0, 1]$ with $B \sim pA + (1-p)C$. But that mixture has attribute vector $(p, 0)$: for every $p > 0$ it beats B on the first attribute, and for $p = 0$ it is $C \succ$-below B. No mixing weight produces indifference.

(d) With $h_5 \succ h_1 \succ h_0$ paying 5, 1 and 0 million, the pattern of Section 5 is

$$A = 1 \cdot h_1, \qquad B = 0.89\,h_1 + 0.10\,h_5 + 0.01\,h_0,$$

$$C = 0.11\,h_1 + 0.89\,h_0, \qquad D = 0.10\,h_5 + 0.90\,h_0,$$

with $A \succ B$ but $D \succ C$. Both pairs differ only by a *common consequence*: replacing 0.89 of h_1 by 0.89 of h_0 turns A into C and B into D. Independence says a ranking is unchanged when the same consequence is mixed into both sides with the same weight, so the pattern violates independence (completeness, transitivity, and continuity are all consistent with it).

Exercise 6.1: Solution p. 294

(a) Apply u (in expected-utility form) to both sides of the axiom's indifference and multiply by $1+\gamma(t)$:

$$u(t\cdot\mu) + \gamma(t)u(\nu) = u(t\cdot\nu) + \gamma(t)u(\mu).$$

(b) Rearranging, $u(t\cdot\mu) - \gamma(t)u(\mu) = u(t\cdot\nu) - \gamma(t)u(\nu)$ for *all* μ,ν: the quantity does not depend on which lottery is used to compute it.

(c) By (b) the definition of $r(t)$ is unambiguous. Taking μ to be the point mass on a deterministic continuation h gives $u(t\cdot h) = r(t) + \gamma(t)u(h)$.

(d) $t\cdot\mu$ is the pushforward of μ under prepending t, and u is linear on lotteries, so $u(t\cdot\mu) = \mathbb{E}_{h\sim\mu}\big[u(t\cdot h)\big] = \mathbb{E}_{h\sim\mu}\big[r(t)+\gamma(t)u(h)\big] = r(t) + \gamma(t)u(\mu)$.

(e) With the recursion and linearity, $u(t\cdot\mu)+\gamma(t)u(\nu) = r(t)+\gamma(t)\big(u(\mu)+u(\nu)\big)$, which is symmetric under $\mu\leftrightarrow\nu$; hence $u(t\cdot\mu)+\gamma(t)u(\nu) = u(t\cdot\nu)+\gamma(t)u(\mu)$. Dividing by $1+\gamma(t)$ shows both mixtures in the axiom have equal expected utility, and since u represents $\succcurlyeq$, they are indifferent.

(f) Normalise $u(\varepsilon)=0$ (allowed by affine freedom). Induction on n: $u(h) = r(t_1) + \gamma(t_1)\,u\big((t_2,\dots,t_n)\big)$, and expanding the inner term yields

$$u(h) = r(t_1) + \gamma(t_1)r(t_2) + \gamma(t_1)\gamma(t_2)r(t_3) + \cdots + \Big(\prod_{i=1}^{n-1}\gamma(t_i)\Big)r(t_n).$$

(g) If $\gamma(t)\equiv\gamma$, the product $\prod_{i<k}\gamma(t_i)$ is γ^{k-1} and $u(h) = \sum_{k}\gamma^{k-1}r(t_k)$: discounted return. If $\gamma\equiv 1$ every product is 1 and $u(h)=\sum_k r(t_k)$: additive cumulative reward.

(h) $r'(t) = u'(t\cdot\mu) - \gamma(t)u'(\mu) = \big(a + b\,u(t\cdot\mu)\big) - \gamma(t)\big(a+b\,u(\mu)\big) = b\,r(t) + (1-\gamma(t))a$. The same preferences thus admit a family of reward functions: even with γ fixed, r is only determined up to a positive scaling and an additive shift modulated by $1-\gamma(t)$.

Exercise 7.1: Solution p. 296

(a) Writing $t_i = (s_{i-1}, a_i, s_i)$, the shaped total is

$$\sum_{i=1}^{n} r_\Phi(t_i) = \sum_{i=1}^{n} r(t_i) + \sum_{i=1}^{n}\big(\Phi(s_i) - \Phi(s_{i-1})\big) = \sum_{i=1}^{n} r(t_i) + \Phi(s_n) - \Phi(s_0),$$

since the middle sum telescopes.

(b) Utility is exactly unchanged iff the boundary term vanishes on every admissible trajectory, i.e. $\Phi(s_n) = \Phi(s_0)$ throughout — for example when all trajectories start in a fixed s_0 and Φ takes the value $\Phi(s_0)$ on every reachable terminal state.

(c) Only the ordering is at stake if the boundary term is the *same constant* c for all compared trajectories: then every utility shifts by c and the induced ranking is untouched. A fixed start state together with Φ constant on the reachable terminal states suffices, whatever that constant is.

Exercise 8.1: Solution p. 297

Discussion notes, not a unique answer. Prescriptively, the money-pump and dominance arguments say that an agent which cares about a fungible resource is under pressure toward completeness and transitivity — coherence is an attractor for agents that can be exploited otherwise. Descriptively, nothing guarantees that trained systems satisfy any axiom: gradient descent on episodic objectives can produce context-dependent, intransitive, or incomparable preferences, especially off-distribution. A reasonable expectation is approximate coherence where incoherence was penalised during training, and no guarantee elsewhere. For design, the trade-off runs both ways: highly coherent agents are more predictable and analysable but are exactly the ones for which instrumental-convergence arguments bite; agents with incomplete or unstable preferences may be harder to exploit into goal-directed resource acquisition, at the cost of being harder to reason about.

Exercise 8.2: Solution p. 297

Discussion notes. The natural candidates weaken one axiom at a time. Dropping *completeness* is the most studied: an agent with incomplete preferences can remain undecided between continuing and being shut down, so it is not pushed by coherence arguments toward shutdown-resistance; money pumps also lose force because the agent may simply refuse trades between incomparable options. Weakening *continuity* permits lexicographic safety: "never cross the constraint, then optimise" cannot be represented by a single real-valued utility, which is arguably a feature. Weakening *independence* allows certainty-favouring (Allais-like) preferences, which dampen gambling-for-resources behaviour. Weakening *temporal γ-indifference* removes the local reward representation: goals about the shape of a trajectory as a whole (diversity, "never do X") stop being expressible as accumulated reward. The common pattern: each axiom kept buys a representation theorem and optimisation pressure; each axiom dropped blocks a coherence-based failure mode while making the agent's behaviour less analysable.

Exercise 8.3: Solution p. 297

Discussion notes; instructive examples include: (i) lexicographic safety-first preferences — violate continuity (see Exercise 5.3); (ii) Pareto/multi-objective preferences, undominated but unaggregated — violate completeness; (iii) hyperbolic discounting — satis-

fies the vNM axioms at a single decision time yet violates temporal γ-indifference, hence admits utility but no stationary reward/discount pair, and is dynamically inconsistent; (iv) the certainty effect (Allais) — violates independence only; (v) satisficing ("anything above the threshold is equally fine") — complete and transitive, so ordinal utility exists, but indifference plateaus interact oddly with lotteries near the threshold; (vi) trajectory-shape goals such as "visit as many distinct states as possible" — can satisfy all four vNM axioms (a utility exists) while violating the temporal axiom, illustrating exactly the gap between utility and reward.

Reinforcement Learning

Leon Lang (Iliad), David Quarel (ARENA)

Main text: Chapter D.1.2, p. 299

Exercise 1.1: Hint p. 301

Condition on the first action and the first transition, then separate the immediate reward from the remaining return.

Exercise 1.1: Solution p. 301

Starting from the definition of V_π and using the recursion $G_t = r_{t+1} + \gamma\, G_{t+1}$:

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] = \mathbb{E}_\pi[r_{t+1} + \gamma\, G_{t+1} \mid s_t = s]\,.$$

Now condition on the first action $a_t = a$ and the first transition $s_{t+1} = s'$:

$$\begin{aligned} V_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s, a) \\ \times\, \mathbb{E}_\pi[r_{t+1} + \gamma\, G_{t+1} \mid s_t = s,\, a_t = a,\, s_{t+1} = s']\,. \end{aligned}$$

Given $s_t = s$, $a_t = a$, and $s_{t+1} = s'$, the immediate reward is deterministic: $r_{t+1} = R(s, a, s')$. For the remaining return, by the Markov property the future trajectory from time $t+1$ onward depends only on $s_{t+1} = s'$, so

$$\mathbb{E}_\pi[G_{t+1} \mid s_t = s,\, a_t = a,\, s_{t+1} = s'] = \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] = V_\pi(s').$$

Substituting both back gives

$$V_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s, a)\big(R(s, a, s') + \gamma\, V_\pi(s')\big).$$

Exercise 1.2: Solution p. 301

(a) Starting from the definition of $V_\pi(s)$ and conditioning on the first action $a_t = a$:

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \sum_{a\in\mathcal{A}} \pi(a \mid s)\; \mathbb{E}_\pi[G_t \mid s_t = s,\, a_t = a] \\ &= \sum_{a\in\mathcal{A}} \pi(a \mid s)\, Q_\pi(s, a). \end{aligned}$$

(b) Starting from the definition of $Q_\pi(s,a)$ and using the recursion $G_t = r_{t+1} + \gamma\, G_{t+1}$, we condition on the first transition $s_{t+1} = s'$:

$$\begin{aligned} Q_\pi(s,a) &= \mathbb{E}_\pi[G_t \mid s_t = s,\ a_t = a] \\ &= \sum_{s' \in \mathcal{S}} T(s' \mid s,a)\, \mathbb{E}_\pi[r_{t+1} + \gamma\, G_{t+1} \mid s_t = s,\ a_t = a,\ s_{t+1} = s'] \\ &= \sum_{s' \in \mathcal{S}} T(s' \mid s,a) \big(R(s,a,s') + \gamma\, V_\pi(s') \big), \end{aligned}$$

where the last step uses $r_{t+1} = R(s,a,s')$ (deterministic given s_t, a_t, s_{t+1}) and the Markov property $\mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] = V_\pi(s')$.

(c) Substituting Exercise 1.2(a) into Exercise 1.2(b), replacing $V_\pi(s')$ by its expression in terms of Q_π:

$$Q_\pi(s,a) = \sum_{s' \in \mathcal{S}} T(s' \mid s,a) \left(R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s')\, Q_\pi(s',a') \right).$$

This is the Bellman equation for the action-value function. Note also that substituting Exercise 1.2(b) into Exercise 1.2(a) recovers the Bellman equation for V_π from Exercise 1.1.

Exercise 2.1: Solution p. 302

Under π_L all transitions are deterministic, so the Bellman equation collapses to

$$\begin{aligned} V_{\pi_L}(s_0) &= R(s_0, a_L, s_L) + \gamma\, V_{\pi_L}(s_L) = 1 + \gamma\, V_{\pi_L}(s_L), \\ V_{\pi_L}(s_L) &= R(s_L, a_L, s_0) + \gamma\, V_{\pi_L}(s_0) = 0 + \gamma\, V_{\pi_L}(s_0). \end{aligned}$$

Substituting the second into the first gives $V_{\pi_L}(s_0) = 1 + \gamma^2\, V_{\pi_L}(s_0)$, hence

$$V_{\pi_L}(s_0) = \frac{1}{1 - \gamma^2}.$$

Similarly, under π_R:

$$\begin{aligned} V_{\pi_R}(s_0) &= R(s_0, a_R, s_R) + \gamma\, V_{\pi_R}(s_R) = 0 + \gamma\, V_{\pi_R}(s_R), \\ V_{\pi_R}(s_R) &= R(s_R, a_R, s_0) + \gamma\, V_{\pi_R}(s_0) = 2 + \gamma\, V_{\pi_R}(s_0). \end{aligned}$$

Substituting yields $V_{\pi_R}(s_0) = \gamma\big(2 + \gamma\, V_{\pi_R}(s_0)\big) = 2\gamma + \gamma^2\, V_{\pi_R}(s_0)$, hence

$$V_{\pi_R}(s_0) = \frac{2\gamma}{1 - \gamma^2}.$$

Exercise 2.2: Solution p. 302

Since any action from s_L or s_R deterministically returns to s_0 with the same reward, the value of any policy at s_0 is determined entirely by the action chosen at s_0. Hence comparing $V_{\pi_L}(s_0)$ and $V_{\pi_R}(s_0)$ suffices:

$$V_{\pi_L}(s_0) - V_{\pi_R}(s_0) \;=\; \frac{1-2\gamma}{1-\gamma^2}.$$

Since $1-\gamma^2 > 0$ for $\gamma \in (0,1)$, the sign is determined by $1-2\gamma$:

- $\gamma < \frac{1}{2}$: a_L is optimal — the immediate reward of 1 outweighs a discounted future reward of 2.
- $\gamma > \frac{1}{2}$: a_R is optimal — the agent is patient enough to wait one step for the larger reward.
- $\gamma = \frac{1}{2}$: both actions are equally good (breakeven), and each gives $V^*(s_0) = \frac{4}{3}$.

Exercise 3.1: Solution p. 303

Without loss of generality, assume $\max_a f(a) \geq \max_a g(a)$. Let $a^* \in \arg\max_{a\in\mathcal{A}} f(a)$. Then:

$$\begin{aligned}
\left|\max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a)\right| &= \max_{a\in\mathcal{A}} f(a) - \max_{a\in\mathcal{A}} g(a) \\
&= f(a^*) - \max_{a\in\mathcal{A}} g(a) \\
&\leq f(a^*) - g(a^*) \\
&= |f(a^*) - g(a^*)| \\
&\leq \max_{a\in\mathcal{A}} |f(a) - g(a)|.
\end{aligned}$$

Exercise 3.2: Solution p. 303

Fix an arbitrary state $s \in \mathcal{S}$. Define for each $a \in \mathcal{A}$:

$$\begin{aligned}
f(a) &\coloneqq \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\, V(s')\big), \\
g(a) &\coloneqq \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\, W(s')\big).
\end{aligned}$$

Then $(\mathcal{B}V)(s) = \max_a f(a)$ and $(\mathcal{B}W)(s) = \max_a g(a)$. By Exercise 3.1:

$$\begin{aligned}
\big|(\mathcal{B}V)(s) - (\mathcal{B}W)(s)\big| &\leq \max_{a\in\mathcal{A}} \big|f(a) - g(a)\big| \\
&= \max_{a\in\mathcal{A}} \left|\sum_{s'\in\mathcal{S}} T(s' \mid s,a)\gamma\left(V(s') - W(s')\right)\right| \\
&\leq \gamma \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\left|V(s') - W(s')\right| \\
&\leq \gamma \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\,\|V - W\|_\infty \\
&\leq \gamma\,\|V - W\|_\infty,
\end{aligned}$$

where the last step uses $\sum_{s'} T(s' \mid s,a) = 1$. Since this holds for every $s \in \mathcal{S}$, taking the maximum over s gives $\|\mathcal{B}V - \mathcal{B}W\|_\infty \leq \gamma\,\|V - W\|_\infty$.

Exercise 3.3: Solution p. 303

The space $(\mathcal{V}, \|\cdot\|_\infty)$ is a finite-dimensional normed vector space, hence complete. By Exercise 3.2, $\mathcal{B}$ is a contraction on $\mathcal{V}$ with factor $\gamma < 1$. Theorem 3.1 then gives the existence of a unique fixed point $V^* = \mathcal{B}V^*$, and convergence $\mathcal{B}^n V_0 \to V^*$ for any $V_0 \in \mathcal{V}$.

Exercise 3.4: Solution p. 303

Fix $V, W \in \mathcal{V}$ and an arbitrary state $s \in \mathcal{S}$. Then:

$$\begin{aligned}
\big|(\mathcal{B}_\pi V)(s) - (\mathcal{B}_\pi W)(s)\big| &= \left|\sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\,\gamma\big(V(s') - W(s')\big)\right| \\
&\le \gamma \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\,\big|V(s') - W(s')\big| \\
&\le \gamma \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\,\|V - W\|_\infty \\
&\le \gamma\,\|V - W\|_\infty,
\end{aligned}$$

where the last step uses $\sum_{s'} T(s' \mid s,a) = 1$ and $\sum_a \pi(a \mid s) = 1$. Taking the maximum over s gives $\|\mathcal{B}_\pi V - \mathcal{B}_\pi W\|_\infty \le \gamma\,\|V - W\|_\infty$, so $\mathcal{B}_\pi$ is a contraction with factor γ.

By Theorem 3.1, $\mathcal{B}_\pi$ has a unique fixed point. By the Bellman equation (Section 1), V_π satisfies $\mathcal{B}_\pi V_\pi = V_\pi$, so V_π is this unique fixed point. The convergence statement also follows from Theorem 3.1.

Exercise 3.5: Solution p. 303

Since π_V is deterministic, $\pi_V(a \mid s) = [\![a = \pi_V(s)]\!]$, so the sum over a in $\mathcal{B}_{\pi_V}$ collapses to the single term $a = \pi_V(s)$. For every $s \in \mathcal{S}$:

$$\begin{aligned}
(\mathcal{B}_{\pi_V} V)(s) &= \sum_{s'\in\mathcal{S}} T(s' \mid s, \pi_V(s))\big(R(s,\pi_V(s),s') + \gamma\,V(s')\big) \\
&= \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\big(R(s,a,s') + \gamma\,V(s')\big) \\
&= (\mathcal{B}V)(s),
\end{aligned}$$

where the second equality holds because $\pi_V(s)$ selects a maximizing action by definition.

Exercise 3.6: Hint p. 304

Show that V^* satisfies the same fixed-point equation as V_{π^*}.

Exercise 3.6: Solution p. 304

We have $\mathcal{B}_{\pi^*} V^* = \mathcal{B}V^* = V^*$ by Exercise 3.5 and Exercise 3.3. So V^* is a fixed point of $\mathcal{B}_{\pi^*}$. By Exercise 3.4, V_{π^*} is the unique fixed point of $\mathcal{B}_{\pi^*}$. Therefore $V_{\pi^*} = V^*$.

Exercise 3.7: Hint p. 304

Show that $\mathcal{B}V_\pi \geq V_\pi$ pointwise, then iterate and take the limit.

Exercise 3.7: Solution p. 304

Let π be any policy. Since the max over actions is at least as large as any weighted average:

$$(\mathcal{B}V)(s) \geq (\mathcal{B}_\pi V)(s) \quad \text{for all } V \in \mathcal{V},\ s \in \mathcal{S}.$$

In particular, since V_π is a fixed point of $\mathcal{B}_\pi$ (Exercise 3.4):

$$(\mathcal{B}V_\pi)(s) \geq (\mathcal{B}_\pi V_\pi)(s) = V_\pi(s) \quad \text{for all } s \in \mathcal{S}.$$

Note that $\mathcal{B}$ is monotone: if $V(s) \geq W(s)$ for all s, then $(\mathcal{B}V)(s) \geq (\mathcal{B}W)(s)$. Applying $\mathcal{B}$ to both sides and using monotonicity, then iterating:

$$V_\pi \leq \mathcal{B}V_\pi \leq \mathcal{B}^2 V_\pi \leq \cdots \leq \mathcal{B}^n V_\pi.$$

By Exercise 3.3, $\mathcal{B}^n V_\pi \to V^*$ as $n \to \infty$. Taking the limit:

$$V_\pi(s) \leq V^*(s) = V_{\pi^*}(s) \quad \text{for all } s \in \mathcal{S},$$

where the last equality is Exercise 3.6. Since π was arbitrary, π^* is optimal.

Exercise 4.1: Solution p. 304

We have

$$\begin{aligned}(\mathcal{B}V_\pi)(s) &= \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\big(R(s,a,s') + \gamma\, V_\pi(s')\big) \\ &\geq \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} T(s' \mid s, a)\big(R(s,a,s') + \gamma\, V_\pi(s')\big) \\ &= V_\pi(s),\end{aligned}$$

where we used the Bellman equation in the last step. If equality holds at every state s, then V_π is a fixed point of $\mathcal{B}$, which by uniqueness (Exercise 3.3) means $V_\pi = V^* = V_{\pi^*}$, i.e. π is already optimal by Exercise 3.7.

Exercise 4.2: Hint p. 304

Show that $\mathcal{B}_{\pi'} V_\pi \geq V_\pi$ pointwise using Exercise 3.5, then iterate $\mathcal{B}_{\pi'}$ and take the limit.

Exercise 4.2: Solution p. 304

By Exercise 3.5, $\mathcal{B}_{\pi'} V_\pi = \mathcal{B} V_\pi \geq V_\pi$ pointwise, where the inequality is from Exercise 4.1. Note that $\mathcal{B}_{\pi'}$ is monotone: if $V(s) \geq W(s)$ for all s, then $(\mathcal{B}_{\pi'} V)(s) \geq (\mathcal{B}_{\pi'} W)(s)$. Applying $\mathcal{B}_{\pi'}$ to both sides and iterating:

$$V_\pi \leq \mathcal{B}_{\pi'} V_\pi \leq \mathcal{B}_{\pi'}^2 V_\pi \leq \cdots \leq \mathcal{B}_{\pi'}^n V_\pi.$$

By Exercise 3.4, $\mathcal{B}_{\pi'}^n V_\pi \to V_{\pi'}$ as $n \to \infty$. Taking the limit gives $V_{\pi'}(s) \geq V_\pi(s)$ for all $s \in \mathcal{S}$.

Exercise 4.3: Hint p. 304

How many deterministic policies are there?

Exercise 4.3: Solution p. 304

By Exercise 4.2, the sequence of value functions is monotonically improving: $V_{\pi_{k+1}}(s) \geq V_{\pi_k}(s)$ for all s and all k. Since each π_k is a deterministic policy and there are only $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies, the sequence must eventually revisit a policy. If $\pi_{k+1} = \pi_j$ for some $j \leq k$, then $V_{\pi_{k+1}} = V_{\pi_j}$, and by monotonicity $V_{\pi_j} = V_{\pi_{j+1}} = \cdots = V_{\pi_{k+1}}$. In particular $V_{\pi_k} = V_{\pi_{k+1}}$.

It remains to show this implies optimality. Since π_{k+1} is greedy with respect to V_{π_k}, Exercise 3.5 gives $\mathcal{B}V_{\pi_k} = \mathcal{B}_{\pi_{k+1}} V_{\pi_k}$. Since $V_{\pi_{k+1}}$ is the fixed point of $\mathcal{B}_{\pi_{k+1}}$ and $V_{\pi_k} = V_{\pi_{k+1}}$, we get $\mathcal{B}_{\pi_{k+1}} V_{\pi_k} = V_{\pi_k}$. Combining: $\mathcal{B}V_{\pi_k} = V_{\pi_k}$, so π_k is optimal by Exercise 4.1.

Exercise 5.1: Solution p. 305

Since $V^* = \mathcal{B}V^*$ and $\mathcal{B}$ is a γ-contraction (Exercise 3.2),

$$\|V_n - V^*\|_\infty \;=\; \|\mathcal{B}V_{n-1} - \mathcal{B}V^*\|_\infty \;\leq\; \gamma \, \|V_{n-1} - V^*\|_\infty.$$

Iterating n times gives $\|V_n - V^*\|_\infty \leq \gamma^n \|V_0 - V^*\|_\infty$.

Exercise 5.2: Hint p. 305

Write $V^* - V_n = \sum_{k=n}^{\infty} (V_{k+1} - V_k)$ (valid since $V_k \to V^*$), and bound each term using the contraction of $\mathcal{B}$.

Exercise 5.2: Solution p. 305

By the contraction property applied repeatedly,

$$\begin{aligned} \|V_{k+1} - V_k\|_\infty \;&=\; \|\mathcal{B}V_k - \mathcal{B}V_{k-1}\|_\infty \;\leq\; \gamma \, \|V_k - V_{k-1}\|_\infty \\ &\leq\; \cdots \;\leq\; \gamma^k \, \|V_1 - V_0\|_\infty. \end{aligned}$$

Since $V_k \to V^*$ (Exercise 3.3), we have the telescoping identity $V^* - V_n = \sum_{k=n}^{\infty} (V_{k+1} - V_k)$. Applying the triangle inequality and the geometric bound above:

$$\begin{aligned} \|V^* - V_n\|_\infty \;&\leq\; \sum_{k=n}^{\infty} \|V_{k+1} - V_k\|_\infty \;\leq\; \sum_{k=n}^{\infty} \gamma^k \, \|V_1 - V_0\|_\infty \\ &=\; \frac{\gamma^n}{1-\gamma} \, \|V_1 - V_0\|_\infty. \end{aligned}$$

Exercise 5.3: Hint p. 305

Telescope from n as in Exercise 5.2, but bound $\|V_{k+1} - V_k\|_\infty$ in terms of $\|V_n - V_{n-1}\|_\infty$ instead.

Exercise 5.3: Solution p. 305

Step 1: bound each successive difference in terms of $\|V_n - V_{n-1}\|_\infty$. Apply the contraction property of $\mathcal{B}$ once:

$$\|V_{n+1} - V_n\|_\infty \;=\; \|\mathcal{B}V_n - \mathcal{B}V_{n-1}\|_\infty \;\le\; \gamma\,\|V_n - V_{n-1}\|_\infty.$$

Apply it again, chaining with the previous bound:

$$\|V_{n+2} - V_{n+1}\|_\infty \;\le\; \gamma\,\|V_{n+1} - V_n\|_\infty \;\le\; \gamma^2\,\|V_n - V_{n-1}\|_\infty.$$

Continuing this pattern, for every $j \ge 0$:

$$\|V_{n+j+1} - V_{n+j}\|_\infty \;\le\; \gamma^{j+1}\,\|V_n - V_{n-1}\|_\infty.$$

Step 2: telescope. Since $V_k \to V^*$ (Exercise 3.3), we have the telescoping identity

$$V^* - V_n \;=\; (V_{n+1} - V_n) + (V_{n+2} - V_{n+1}) + (V_{n+3} - V_{n+2}) + \cdots.$$

Applying the triangle inequality and the bounds from Step 1:

$$\begin{aligned}
\|V^* - V_n\|_\infty &\le \|V_{n+1} - V_n\|_\infty \;+\; \|V_{n+2} - V_{n+1}\|_\infty \\
&\qquad +\; \|V_{n+3} - V_{n+2}\|_\infty \;+\; \cdots \\
&\le \gamma\,\|V_n - V_{n-1}\|_\infty \;+\; \gamma^2\,\|V_n - V_{n-1}\|_\infty \\
&\qquad +\; \gamma^3\,\|V_n - V_{n-1}\|_\infty \;+\; \cdots \\
&= \left(\gamma + \gamma^2 + \gamma^3 + \cdots\right)\|V_n - V_{n-1}\|_\infty \\
&= \frac{\gamma}{1-\gamma}\,\|V_n - V_{n-1}\|_\infty.
\end{aligned}$$

Exercise 5.4: Hint p. 305

Bound $\|V_n - V_{n-1}\|_\infty$ by repeated application of the contraction.

Exercise 5.4: Solution p. 305

Apply the contraction property of $\mathcal{B}$ to $V_n = \mathcal{B}V_{n-1}$ and $V_{n-1} = \mathcal{B}V_{n-2}$:

$$\|V_n - V_{n-1}\|_\infty \;\le\; \gamma\,\|V_{n-1} - V_{n-2}\|_\infty.$$

Continuing this $n-1$ times altogether, we eventually land on $\|V_1 - V_0\|_\infty$:

$$\|V_n - V_{n-1}\|_\infty \;\le\; \gamma^{n-1}\,\|V_1 - V_0\|_\infty.$$

Multiplying both sides by $\dfrac{\gamma}{1-\gamma} > 0$ preserves the inequality:

$$\frac{\gamma}{1-\gamma}\,\|V_n - V_{n-1}\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,\|V_1 - V_0\|_\infty.$$

Both bounds from Exercise 5.3 and Exercise 5.2 upper-bound $\|V_n - V^*\|_\infty$, and the a posteriori one is smaller.

Exercise 5.5: Solution p. 305

With $V_0 \equiv 0$:

$$V_1(s) \;=\; (\mathcal{B}V_0)(s) \;=\; \max_{a\in\mathcal{A}} \sum_{s'\in\mathcal{S}} T(s' \mid s,a)\,R(s,a,s').$$

Each term is a convex combination of rewards, all bounded in absolute value by $R_{\max}$, so $|V_1(s)| \le R_{\max}$ for every s, hence $\|V_1 - V_0\|_\infty = \|V_1\|_\infty \le R_{\max}$. Substituting into the a priori bound of Exercise 5.2:

$$\|V_n - V^*\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,\|V_1 - V_0\|_\infty \;\le\; \frac{\gamma^n}{1-\gamma}\,R_{\max}.$$

Exercise 5.6: Solution p. 305

Take $\mathcal{S} = \{s\}$, $\mathcal{A} = \{a\}$, $T(s \mid s,a) = 1$, and $R(s,a,s) = R_{\max}$. The Bellman optimality operator collapses to

$$(\mathcal{B}V)(s) \;=\; R_{\max} + \gamma\,V(s).$$

Starting from $V_0(s) = 0$ and iterating,

$$V_n(s) \;=\; R_{\max}\bigl(1 + \gamma + \gamma^2 + \cdots + \gamma^{n-1}\bigr) \;=\; R_{\max}\cdot\frac{1-\gamma^n}{1-\gamma}.$$

The fixed point $V^*(s)$ solves $V^*(s) = R_{\max} + \gamma\,V^*(s)$, giving $V^*(s) = R_{\max}/(1-\gamma)$. Therefore

$$\|V_n - V^*\|_\infty \;=\; V^*(s) - V_n(s) \;=\; \frac{R_{\max}}{1-\gamma} - \frac{R_{\max}(1-\gamma^n)}{1-\gamma} \;=\; \frac{\gamma^n}{1-\gamma}\,R_{\max},$$

so the bound of Exercise 5.5 is attained with equality. No bound in terms of only n, γ, and $R_{\max}$ can be sharper.

Exercise 6.1: Hint p. 308

This is similar to the proof of Exercise 3.2.

Exercise 6.1: Solution p. 308

Fix $Q, Q' \in \mathcal{Q}$ and $(s,a) \in \mathcal{S}\times\mathcal{A}$. By the inequality from Exercise 3.1 (applied at each s') and the triangle inequality:

$$
\begin{aligned}
&\big|(\mathcal{B}Q)(s,a) - (\mathcal{B}Q')(s,a)\big| \\
&\quad= \left|\sum_{s'} T(s' \mid s,a)\,\gamma\Big(\max_{a'} Q(s',a') - \max_{a'} Q'(s',a')\Big)\right| \\
&\quad\le \sum_{s'} T(s' \mid s,a)\,\gamma\, \max_{a'}\big|Q(s',a') - Q'(s',a')\big| \\
&\quad\le \sum_{s'} T(s' \mid s,a)\,\gamma\, \max_{a',s''}\big|Q(s'',a') - Q'(s'',a')\big| \\
&\quad\le \gamma\,\|Q - Q'\|_\infty .
\end{aligned}
$$

Taking the maximum over (s,a) gives $\|\mathcal{B}Q - \mathcal{B}Q'\|_\infty \le \gamma\,\|Q - Q'\|_\infty$.

Exercise 6.2: Solution p. 309

(a) By Exercise 1.2(b) applied to π^*:

$$
Q_{\pi^*}(s,a) = \sum_{s' \in \mathcal{S}} T(s' \mid s,a)\big[R(s,a,s') + \gamma\, V_{\pi^*}(s')\big].
$$

By Exercise 3.6, $V_{\pi^*} = V^*$. Substituting gives the result.

(b) By Exercise 1.2(a), $V_{\pi^*}(s) = \sum_a \pi^*(a \mid s)\, Q_{\pi^*}(s,a)$. Since π^* is the greedy policy with respect to V^* (Exercises 3.5 and 3.6), it is deterministic and selects $\pi^*(s) \in \arg\max_a \sum_{s'} T(s' \mid s,a)[R(s,a,s') + \gamma\, V^*(s')]$. By Exercise 6.2(a), the inner expression equals $Q^*(s,a)$, so $\pi^*(s) \in \arg\max_a Q^*(s,a)$. Since π^* is deterministic, $V^*(s) = V_{\pi^*}(s) = Q_{\pi^*}(s,\pi^*(s)) = Q^*(s,\pi^*(s)) = \max_a Q^*(s,a)$.

(c) Substituting Exercise 6.2(b) into Exercise 6.2(a):

$$
Q^*(s,a) = \sum_{s'} T(s' \mid s,a)\big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\big] = (\mathcal{B}Q^*)(s,a).
$$

So Q^* is a fixed point of $\mathcal{B}$. Uniqueness follows from Exercise 6.1 and Theorem 3.1.

Exercise 6.3: Solution p. 309

The identifications are:

- Index set: $I = \mathcal{S} \times \mathcal{A}$.
- Iterates: $x_t = Q_t$.
- Contraction: $F = \mathcal{B}$, with fixed point $x^* = Q^*$ (Exercise 6.2).
- Learning rates: $\alpha_t(s,a) = \alpha_t \cdot [\![(s,a) = (s_t,a_t)]\!]$.

With these identifications, the Q-learning update becomes

$$
Q_{t+1}(s,a) = (1 - \alpha_t(s,a))\, Q_t(s,a) + \alpha_t(s,a)\big[(\mathcal{B}Q_t)(s,a) + w_t(s,a)\big],
$$

where the noise at the updated coordinate is

$$
w_t(s_t,a_t) = R(s_t,a_t,s_{t+1}) + \gamma \max_{a'} Q_t(s_{t+1},a') - (\mathcal{B}Q_t)(s_t,a_t),
$$

i.e. the difference between the sampled target (using the single transition s_{t+1}) and its expectation over all possible transitions. For $(s,a) \neq (s_t,a_t)$, we set $w_t(s,a) = 0$.

Exercise 6.4: Hint p. 309

Conditioned on $\mathcal{F}_t$, the quantities Q_t, s_t, and a_t are all determined. What is the only remaining source of randomness?

Exercise 6.4: Solution p. 309

Conditioned on $\mathcal{F}_t$, the quantities Q_t, s_t, a_t are all determined. The only remaining randomness is in $s_{t+1} \sim T(\cdot \mid s_t, a_t)$. Therefore:

$$\begin{aligned}
&\mathbb{E}[w_t(s_t,a_t) \mid \mathcal{F}_t] \\
&\quad = \mathbb{E}\big[R(s_t,a_t,s_{t+1}) + \gamma \max_{a'} Q_t(s_{t+1},a') \mid \mathcal{F}_t\big] - (\mathcal{B}Q_t)(s_t,a_t) \\
&\quad = \sum_{s'} T(s' \mid s_t,a_t)\big[R(s_t,a_t,s') + \gamma \max_{a'} Q_t(s',a')\big] - (\mathcal{B}Q_t)(s_t,a_t) \\
&\quad = (\mathcal{B}Q_t)(s_t,a_t) - (\mathcal{B}Q_t)(s_t,a_t) = 0.
\end{aligned}$$

For $(s,a) \neq (s_t,a_t)$, we have $w_t(s,a) = 0$ by definition, so $\mathbb{E}[w_t(s,a) \mid \mathcal{F}_t] = 0$ trivially.

Exercise 6.5: Hint p. 309

First show that $|w_t(s_t,a_t)| \le 2(R_{\max} + \gamma \|Q_t\|_\infty)$.

Exercise 6.5: Solution p. 309

Step 1: a pointwise bound on w_t. Using $|R(s_t,a_t,s_{t+1})| \le R_{\max}$ and $|\max_{a'} Q_t(s_{t+1},a')| \le \|Q_t\|_\infty$, the sampled target satisfies

$$\big|R(s_t,a_t,s_{t+1}) + \gamma \max_{a'} Q_t(s_{t+1},a')\big| \le R_{\max} + \gamma \|Q_t\|_\infty.$$

The same bound holds for $|(\mathcal{B}Q_t)(s_t,a_t)|$, since it is a convex combination (over s') of terms each bounded by $R_{\max} + \gamma \|Q_t\|_\infty$. By the triangle inequality:

$$|w_t(s_t,a_t)| \;\le\; 2\big(R_{\max} + \gamma \|Q_t\|_\infty\big).$$

This bound holds for *every* realization of s_{t+1}, not just in expectation.

Step 2: passing to the conditional second moment. Squaring the pointwise bound,

$$w_t(s_t,a_t)^2 \;\le\; 4\big(R_{\max} + \gamma \|Q_t\|_\infty\big)^2 \quad \text{a.s.}$$

Taking conditional expectation of both sides given $\mathcal{F}_t$ preserves the inequality (monotonicity of conditional expectation):

$$\mathbb{E}\big[w_t(s_t,a_t)^2 \mid \mathcal{F}_t\big] \;\le\; \mathbb{E}\big[\,4(R_{\max} + \gamma \|Q_t\|_\infty)^2 \mid \mathcal{F}_t\big]\,.$$

Now we simplify the right-hand side. It is a function of Q_t (and the constants $R_{\max}, \gamma$), and Q_t is determined once we condition on $\mathcal{F}_t$ (recall $\mathcal{F}_t$ records $Q_0, Q_1, \ldots, Q_t$). So the right-hand side is $\mathcal{F}_t$-**measurable**: given the history, it is a fixed real number, not random. Conditional expectation acts as the identity on $\mathcal{F}_t$-measurable quantities — i.e. $\mathbb{E}[Y \mid \mathcal{F}_t] = Y$ when Y is $\mathcal{F}_t$-measurable — so

$$\mathbb{E}\big[\, 4(R_{\max} + \gamma \, \|Q_t\|_\infty)^2 \mid \mathcal{F}_t \big] \;=\; 4\big(R_{\max} + \gamma \, \|Q_t\|_\infty\big)^2.$$

Combining,

$$\mathbb{E}\big[w_t(s_t, a_t)^2 \mid \mathcal{F}_t\big] \;\le\; 4\big(R_{\max} + \gamma \, \|Q_t\|_\infty\big)^2.$$

Step 3: putting it in the required form. We want a constant C such that the right-hand side is bounded by $C(1 + \|Q_t\|_\infty^2)$. Applying $(a+b)^2 \le 2(a^2 + b^2)$ with $a = R_{\max}$, $b = \gamma \, \|Q_t\|_\infty$:

$$\begin{aligned} 4\big(R_{\max} + \gamma \, \|Q_t\|_\infty\big)^2 &\le 8\big(R_{\max}^2 + \gamma^2 \, \|Q_t\|_\infty^2\big) \\ &\le 8 \max\!\big(R_{\max}^2,\, \gamma^2\big)\big(1 + \|Q_t\|_\infty^2\big). \end{aligned}$$

So $C = 8\max(R_{\max}^2, \gamma^2)$ works. For $(s,a) \neq (s_t, a_t)$, we have $w_t(s,a) = 0$, so the bound holds trivially with the same C.

Exercise 6.6: Solution p. 309

By Exercises 6.1–6.5, the Q-learning iteration satisfies all the hypotheses of Theorem 6.1, provided the learning rate conditions hold: for each $(s,a) \in \mathcal{S} \times \mathcal{A}$,

$$\sum_{t=0}^{\infty} \alpha_t(s,a) = \infty, \qquad \sum_{t=0}^{\infty} \alpha_t(s,a)^2 < \infty, \qquad \alpha_t(s,a) \in [0,1].$$

Under these conditions, Theorem 6.1 gives $Q_t \to Q^*$ with probability 1.

Note that the first condition already implies that each state-action pair (s,a) must be visited infinitely often. A concrete way to achieve all three conditions is to use an exploration policy that visits every (s,a) infinitely often together with a per-coordinate learning rate $\alpha_t(s,a) = 1/n_t(s,a)$, where $n_t(s,a)$ counts the number of visits to (s,a) up to time t. This satisfies $\sum \alpha_t(s,a) = \sum_{k=1}^{\infty} 1/k = \infty$ and $\sum \alpha_t(s,a)^2 = \sum_{k=1}^{\infty} 1/k^2 < \infty$.

Exercise 7.1: Hint p. 310

Specialize the Bellman equation to a deterministic policy and write the resulting system of $|\mathcal{S}|$ equations in matrix-vector form.

Exercise 7.1: Solution p. 310

Since π is deterministic, $\pi(a \mid s) = [\![a = \pi(s)]\!]$, and the Bellman equation from Exercise 1.1 collapses the sum over actions:

$$V_\pi(s) \;=\; \sum_{s' \in \mathcal{S}} T(s' \mid s, \pi(s)) \big(R(s, \pi(s), s') + \gamma\, V_\pi(s') \big).$$

Splitting the reward and value terms and rewriting with the matrix notation of Definition 7.1:

$$\mathbf{v}^\pi_s = \sum_{s'} \mathbf{P}^\pi_{s,s'}\, \mathbf{R}^\pi_{s,s'} + \gamma \sum_{s'} \mathbf{P}^\pi_{s,s'}\, \mathbf{v}^\pi_{s'} = \mathbf{r}^\pi_s + \gamma\, (\mathbf{P}^\pi \mathbf{v}^\pi)_s.$$

This holds for every s, so in vector form

$$\mathbf{v}^\pi \;=\; \mathbf{r}^\pi + \gamma\, \mathbf{P}^\pi \mathbf{v}^\pi \;\iff\; (\mathbf{I} - \gamma\, \mathbf{P}^\pi)\, \mathbf{v}^\pi \;=\; \mathbf{r}^\pi.$$

Assuming $\mathbf{I} - \gamma\, \mathbf{P}^\pi$ is invertible, left-multiplying by its inverse gives $\mathbf{v}^\pi = (\mathbf{I} - \gamma\, \mathbf{P}^\pi)^{-1}\, \mathbf{r}^\pi$.

Exercise 7.2: Hint p. 310

Apply Fact 7.2.

Exercise 7.2: Solution p. 310

The matrix $\mathbf{P}^\pi$ is **row-stochastic**: $\mathbf{P}^\pi_{s,s'} \geq 0$ and $\sum_{s'} \mathbf{P}^\pi_{s,s'} = \sum_{s'} T(s' \mid s, \pi(s)) = 1$ for every s. Therefore

$$\|\mathbf{P}^\pi\|_\infty \;=\; \max_s \sum_{s'} |\mathbf{P}^\pi_{s,s'}| \;=\; \max_s \sum_{s'} \mathbf{P}^\pi_{s,s'} \;=\; 1,$$

and so $\|\gamma\, \mathbf{P}^\pi\|_\infty = \gamma < 1$. By the Neumann series (Fact 7.2), $\mathbf{I} - \gamma\, \mathbf{P}^\pi$ is invertible, with

$$(\mathbf{I} - \gamma\, \mathbf{P}^\pi)^{-1} \;=\; \sum_{k=0}^{\infty} (\gamma\, \mathbf{P}^\pi)^k \;=\; \sum_{k=0}^{\infty} \gamma^k\, (\mathbf{P}^\pi)^k.$$

Combining with Exercise 7.1,

$$\mathbf{v}^\pi \;=\; \sum_{k=0}^{\infty} \gamma^k\, (\mathbf{P}^\pi)^k\, \mathbf{r}^\pi,$$

which has a direct interpretation: $(\mathbf{P}^\pi)^k \mathbf{r}^\pi$ is the vector of expected rewards exactly k steps into the future under π, and $\mathbf{v}^\pi$ is their discounted sum.

Solomonoff Induction

David Quarel (ARENA), Leon Lang (Iliad)

Main text: Chapter D.3.1, p. 315

Exercise 1.1: Solution p. 318

Expand the conditional as a ratio of joints; the chain rule (Definition 1.1) telescopes the ratio to a product over indices $k = p, \dots, r$:

$$\nu(x_{p:r} \mid x_{<p}) \;=\; \frac{\nu(x_{1:r})}{\nu(x_{<p})} \;=\; \frac{\prod_{k=1}^{r} \nu(x_k \mid x_{<k})}{\prod_{k=1}^{p-1} \nu(x_k \mid x_{<k})} \;=\; \prod_{k=p}^{r} \nu(x_k \mid x_{<k}).$$

For any $k \geq p$, the past $x_{<k}$ is just $x_{<p}$ extended by $x_{p:k-1}$, so the same expansion identifies every contiguous sub-block as a conditional with the right history. Splitting the product at index q:

$$\nu(x_{p:r} \mid x_{<p}) \;=\; \underbrace{\prod_{k=p}^{q} \nu(x_k \mid x_{<k})}_{=\,\nu(x_{p:q} \mid x_{<p})} \cdot \underbrace{\prod_{k=q+1}^{r} \nu(x_k \mid x_{<k})}_{=\,\nu(x_{q+1:r} \mid x_{<p}\, x_{p:q})}.$$

Exercise 1.2: Hint p. 318

Write $\xi(x_t \mid x_{<t}) = \xi(x_{1:t})/\xi(x_{<t})$, expand the numerator, and use the chain rule $\nu(x_{1:t}) = \nu(x_{<t})\,\nu(x_t \mid x_{<t})$ (Definition 1.1). For the last part, note that the posterior weights sum to 1.

Exercise 1.2: Solution p. 318

The one-step conditional is the ratio of joint to marginal, $\xi(x_t \mid x_{<t}) := \xi(x_{1:t})/\xi(x_{<t})$. Expanding the numerator and applying the chain rule $\nu(x_{1:t}) = \nu(x_{<t})\,\nu(x_t \mid x_{<t})$ to each term:

$$\begin{aligned}\xi(x_t \mid x_{<t}) \;&=\; \frac{\sum_{\nu \in \mathcal{M}} w_\nu\, \nu(x_{1:t})}{\xi(x_{<t})} \;=\; \frac{\sum_{\nu \in \mathcal{M}} w_\nu\, \nu(x_{<t})\, \nu(x_t \mid x_{<t})}{\xi(x_{<t})} \\ &=\; \sum_{\nu \in \mathcal{M}} \underbrace{\frac{w_\nu\, \nu(x_{<t})}{\xi(x_{<t})}}_{=\,w(\nu \mid x_{<t})} \nu(x_t \mid x_{<t}) \;=\; \sum_{\nu \in \mathcal{M}} w(\nu \mid x_{<t})\, \nu(x_t \mid x_{<t}).\end{aligned}$$

The posterior weights are non-negative and sum to 1:

$$\sum_{\nu\in\mathcal{M}} w(\nu \mid x_{<t}) \;=\; \frac{\sum_{\nu\in\mathcal{M}} w_\nu\, \nu(x_{<t})}{\xi(x_{<t})} \;=\; \frac{\xi(x_{<t})}{\xi(x_{<t})} \;=\; 1.$$

Therefore $\xi(\cdot \mid x_{<t})$ is a convex combination of the probability distributions $\nu(\cdot \mid x_{<t})$, so it is itself a probability distribution over $\mathbb{B}$:

$$\sum_{x_t\in\mathbb{B}} \xi(x_t \mid x_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid x_{<t}) \underbrace{\sum_{x_t\in\mathbb{B}} \nu(x_t \mid x_{<t})}_{=1} \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid x_{<t}) \;=\; 1.$$

Exercise 1.3: Solution p. 318

By Definition 1.2,

$$w(\nu \mid x_{1:t}) \;=\; w_\nu \frac{\nu(x_{1:t})}{\xi(x_{1:t})}, \qquad w(\nu \mid x_{<t}) \;=\; w_\nu \frac{\nu(x_{<t})}{\xi(x_{<t})}.$$

Apply the chain rule (Definition 1.1) to both ν and ξ: $\nu(x_{1:t}) = \nu(x_{<t})\,\nu(x_t \mid x_{<t})$ and $\xi(x_{1:t}) = \xi(x_{<t})\,\xi(x_t \mid x_{<t})$. Dividing,

$$\frac{w(\nu \mid x_{1:t})}{w(\nu \mid x_{<t})} \;=\; \frac{\nu(x_{1:t})}{\nu(x_{<t})} \cdot \frac{\xi(x_{<t})}{\xi(x_{1:t})} \;=\; \frac{\nu(x_t \mid x_{<t})}{\xi(x_t \mid x_{<t})}.$$

Rearranging gives the claim.

Exercise 1.4: Hint p. 319

Write $\xi(x_{t:m} \mid x_{<t}) = \xi(x_{1:m})/\xi(x_{<t})$, expand the numerator using the mixture definition, and apply the generalized chain rule (Exercise 1.1 with $p = 1$, $q = t-1$, $r = m$) to factor each $\nu(x_{1:m}) = \nu(x_{<t})\,\nu(x_{t:m} \mid x_{<t})$.

Exercise 1.4: Solution p. 319

By definition of the conditional and the mixture form $\xi(x_{1:m}) = \sum_\nu w_\nu \nu(x_{1:m})$ (Definition 1.2),

$$\xi(x_{t:m} \mid x_{<t}) \;=\; \frac{\xi(x_{1:m})}{\xi(x_{<t})} \;=\; \frac{\sum_\nu w_\nu\, \nu(x_{1:m})}{\xi(x_{<t})}.$$

The generalized chain rule (Exercise 1.1 with $p = 1$, $q = t-1$, $r = m$) gives $\nu(x_{1:m}) = \nu(x_{<t})\,\nu(x_{t:m} \mid x_{<t})$. Substituting,

$$\begin{aligned}\xi(x_{t:m} \mid x_{<t}) \;&=\; \sum_\nu \underbrace{\frac{w_\nu\, \nu(x_{<t})}{\xi(x_{<t})}}_{=w(\nu|x_{<t})} \nu(x_{t:m} \mid x_{<t}) \\ &=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid x_{<t})\, \nu(x_{t:m} \mid x_{<t}).\end{aligned}$$

Exercise 2.1: Hint p. 320

For the induction step, factor $\nu(x_{1:n}) = \nu(x_{<n})\,\nu(x_n \mid x_{<n})$ (and likewise for ρ) inside the log; the log splits additively into two pieces matching $D_{n-1}(\nu \parallel \rho)$ and $d_n(\nu \parallel \rho)$.

Exercise 2.1: Solution p. 320

By induction on n.

Base case ($n = 1$):

$$D_1(\nu \parallel \rho) \;=\; \sum_{x_1} \nu(x_1) \ln \frac{\nu(x_1)}{\rho(x_1)} \;=\; d_1(\nu \parallel \rho),$$

using $\nu(x_1 \mid x_{<1}) := \nu(x_1)$ and likewise for ρ.

Induction step. Apply the chain rule for probabilities (Definition 1.1) inside the log: $\nu(x_{1:n}) = \nu(x_{<n})\,\nu(x_n \mid x_{<n})$, and similarly for ρ. The log splits additively:

$$\begin{aligned}
&D_n(\nu \parallel \rho) \\
&= \sum_{x_{1:n} \in \mathbb{B}^n} \nu(x_{1:n}) \, \ln \frac{\nu(x_{1:n})}{\rho(x_{1:n})} \\
&= \sum_{x_{1:n}} \nu(x_n \mid x_{<n}) \nu(x_{<n}) \, \ln \frac{\nu(x_n \mid x_{<n})\,\nu(x_{<n})}{\rho(x_n \mid x_{<n})\,\rho(x_{<n})} \\
&= \sum_{x_{1:n}} \nu(x_n \mid x_{<n}) \nu(x_{<n}) \left[\ln \frac{\nu(x_n \mid x_{<n})}{\rho(x_n \mid x_{<n})} \;+\; \ln \frac{\nu(x_{<n})}{\rho(x_{<n})} \right]. \\
&= \underbrace{\sum_{x_{<n}} \nu(x_{<n}) \sum_{x_n} \nu(x_n \mid x_{<n}) \ln \frac{\nu(x_n \mid x_{<n})}{\rho(x_n \mid x_{<n})}}_{d_n(\nu \| \rho)} \\
&\qquad + \sum_{x_{1:n}} \nu(x_n \mid x_{<n}) \nu(x_{<n}) \ln \frac{\nu(x_{<n})}{\rho(x_{<n})} \\
&= d_n(\nu \parallel \rho) \;+\; \underbrace{\sum_{x_{<n}} \nu(x_{<n}) \ln \frac{\nu(x_{<n})}{\rho(x_{<n})} \cancel{\sum_{x_n} \nu(x_n \mid x_{<n})}}_{D_{n-1}(\nu \| \rho)} \\
&= d_n(\nu \parallel \rho) + D_{n-1}(\nu \parallel \rho) \;=\; d_n(\nu \parallel \rho) + \sum_{t=1}^{n-1} d_t(\nu \parallel \rho) = \sum_{t=1}^{n} d_t(\nu \parallel \rho)
\end{aligned}$$

Exercise 3.1: Solution p. 320

By Definition 1.2, $\xi(x) = \sum_{\nu' \in \mathcal{M}} w_{\nu'}\,\nu'(x)$. Every term in this sum is non-negative, so dropping all terms except the one with $\nu' = \nu$ can only decrease the sum:

$$\xi(x) \;=\; \sum_{\nu' \in \mathcal{M}} w_{\nu'}\,\nu'(x) \;\geq\; w_\nu\,\nu(x).$$

Exercise 4.1: Hint p. 321

You need Theorem 4.1, Exercise 2.1 and Exercise 3.1.

Exercise 4.1: Solution p. 321

The proof is a three-step chain:

$$S_\infty^\mu \overset{(1)}{\le} D_\infty^\mu \overset{(2)}{\le} \lim_{n\to\infty} \sum_{x_{1:n}} \mu(x_{1:n}) \ln \tfrac{1}{w_\mu} \overset{(3)}{=} -\ln w_\mu.$$

Step (1): Pinsker, pointwise. For each t and $x_{<t}$, both $\mu(\cdot \mid x_{<t})$ and $\xi(\cdot \mid x_{<t})$ are probability distributions on $\mathbb{B}$ (ξ by Exercise 1.2), so Theorem 4.1 gives

$$\sum_{x_t} \big(\xi(x_t \mid x_{<t}) - \mu(x_t \mid x_{<t})\big)^2 \le \sum_{x_t} \mu(x_t \mid x_{<t}) \ln \tfrac{\mu(x_t|x_{<t})}{\xi(x_t|x_{<t})}.$$

Multiplying by $\mu(x_{<t}) \ge 0$ and summing over $x_{<t}$ gives $s_t(\mu \,\|\, \xi) \le d_t(\mu \,\|\, \xi)$, so $S_t(\mu \,\|\, \xi) \le \sum_{i=1}^t d_i(\mu \,\|\, \xi) = D_t(\mu \,\|\, \xi)$ (Exercise 2.1), from which we take $t \to \infty$.

Step (2): Mixture dominance. By Exercise 3.1, $\xi(x_{1:n}) \ge w_\mu \, \mu(x_{1:n})$, so $\mu(x_{1:n})/\xi(x_{1:n}) \le 1/w_\mu$, hence $\ln(\mu/\xi) \le \ln(1/w_\mu)$ pointwise. Substituting into the definition $D_n(\mu \,\|\, \xi) = \sum_{x_{1:n}} \mu(x_{1:n}) \ln(\mu/\xi)$ gives the bound.

Step (3): Total mass. Pull the constant out and use $\sum_{x_{1:n}} \mu(x_{1:n}) = 1$ (μ is an environment).

Exercise 4.2: Solution p. 321

By Section 4, for *any* prior, $S_\infty^\mu \le -\ln w_\mu$. The Solomonoff prior assigns μ the weight $w_\mu = 2^{-K(\mu)}$, so

$$S_\infty^\mu \;\le\; -\ln w_\mu \;=\; -\ln 2^{-K(\mu)} \;=\; K(\mu) \ln 2.$$

Exercise 4.3: Solution p. 321

Follows immediately as $\sum_{t=1}^\infty s_t \le -\ln w_\mu < \infty$ implies $s_t \to 0$.

Exercise 5.1: Hint p. 322

At a mistake step, $\xi(x_t^* \mid x_{<t}) \le \frac{1}{2}$ (why?). The per-step squared error along the deterministic μ-trajectory simplifies dramatically; bound it from below, then sum.

Exercise 5.1: Solution p. 322

Step 1: per-step squared error along the μ-trajectory. Because μ is deterministic, the outer sum over $x_{<t}$ in s_t collapses onto the unique trajectory $x^*_{<t} := x_1^* \cdots x_{t-1}^*$, with weight $\mu(x^*_{<t}) = 1$. Write $\xi_t := \xi(x_t^* \mid x^*_{<t}) \in [0,1]$ for the mass the mixture places on the *correct* bit. Since μ assigns mass 1 to x_t^* and 0 to the other bit,

$$\begin{aligned} s_t &= \sum_{x_t \in \mathbb{B}} \big(\xi(x_t \mid x^*_{<t}) - \mu(x_t \mid x^*_{<t})\big)^2 \\ &= (\xi_t - 1)^2 + (1 - \xi_t - 0)^2 \;=\; 2(1-\xi_t)^2. \end{aligned}$$

Step 2: a mistake step contributes at least $\frac{1}{2}$. A mistake at time t means $\hat{x}_t \neq x^*_t$, i.e. the threshold predictor chose the *other* bit. This requires $\xi(\hat{x}_t \mid x^*_{<t}) \geq \xi(x^*_t \mid x^*_{<t})$, i.e. $1 - \xi_t \geq \xi_t$, i.e. $\xi_t \leq \frac{1}{2}$. So at every mistake step,

$$s_t \;=\; 2(1-\xi_t)^2 \;\geq\; 2 \cdot \tfrac{1}{4} \;=\; \tfrac{1}{2}.$$

Step 3: sum and apply the cumulative bound. Let N be the total number of mistakes. Summing s_t over mistake steps (non-negative contributions from non-mistake steps only help):

$$\tfrac{1}{2} \cdot N \;\leq\; \sum_{t\,:\,\hat{x}_t \neq x^*_t} s_t \;\leq\; \sum_{t=1}^{\infty} s_t \;=\; S^{\mu}_{\infty} \;\leq\; -\ln w_\mu,$$

where the last step is Section 4. Rearranging, $N \leq -2 \ln w_\mu$.

Exercise 6.1: Hint p. 322

Take the difference between the two summation terms.

Exercise 6.1: Solution p. 322

Take the difference $\sum_\nu w_\nu D_n(\nu \parallel \rho) - \sum_\nu w_\nu D_n(\nu \parallel \xi)$. Inside each D_n the $\nu(x_{1:n}) \ln \nu(x_{1:n})$ pieces cancel, leaving

$$\begin{aligned} &\sum_\nu w_\nu D_n(\nu \parallel \rho) - \sum_\nu w_\nu D_n(\nu \parallel \xi) \\ &\quad= \sum_\nu w_\nu \sum_{x_{1:n}} \nu(x_{1:n}) \left[\ln \tfrac{\nu(x_{1:n})}{\rho(x_{1:n})} - \ln \tfrac{\nu(x_{1:n})}{\xi(x_{1:n})}\right] \\ &\quad= \sum_\nu w_\nu \sum_{x_{1:n}} \nu(x_{1:n}) \ln \tfrac{\xi(x_{1:n})}{\rho(x_{1:n})}. \end{aligned}$$

The factor $\ln(\xi/\rho)$ does not depend on ν, so we swap the order of summation and pull it out, then use $\sum_\nu w_\nu \nu(x_{1:n}) = \xi(x_{1:n})$:

$$\begin{aligned} \sum_\nu w_\nu \sum_{x_{1:n}} \nu(x_{1:n}) \ln \tfrac{\xi(x_{1:n})}{\rho(x_{1:n})} &= \sum_{x_{1:n}} \ln \tfrac{\xi(x_{1:n})}{\rho(x_{1:n})} \underbrace{\sum_\nu w_\nu \nu(x_{1:n})}_{=\xi(x_{1:n})} \\ &= \sum_{x_{1:n}} \xi(x_{1:n}) \ln \tfrac{\xi(x_{1:n})}{\rho(x_{1:n})} \;=\; D_n(\xi \parallel \rho). \end{aligned}$$

Rearranging gives the claimed identity.

Exercise 6.2: Hint p. 323

Multiply the assumed inequality by $w_\nu \geq 0$ and sum over $\nu \in \mathcal{M}$; then apply the KL Pythagorean identity to recognize the extra non-negative term, which must vanish.

Exercise 6.2: Solution p. 323

Multiply the assumed inequality by $w_\nu \geq 0$ and sum over $\nu \in \mathcal{M}$:

$$\sum_\nu w_\nu \, D_n(\nu \parallel \rho) \;\leq\; \sum_\nu w_\nu \, D_n(\nu \parallel \xi).$$

The KL Pythagorean identity rewrites the LHS as

$$D_n(\xi \parallel \rho) \;+\; \sum_\nu w_\nu \, D_n(\nu \parallel \xi),$$

so Equation 3 forces $D_n(\xi \parallel \rho) \leq 0$. KL is non-negative, hence $D_n(\xi \parallel \rho) = 0$, which forces $\rho(x_{1:n}) = \xi(x_{1:n})$ for every $x_{1:n} \in \mathbb{B}^n$. By chain rule, the conditionals agree at every history $x_{<t}$ with $\xi(x_{<t}) > 0$.

Exercise 7.1: Hint p. 323

Use Exercise 3.1.

Exercise 7.1: Solution p. 323

Mixture dominance applied to $\hat{\mu} \in \mathcal{M}$ (Section 3) gives $\xi(x_{1:n}) \geq w_{\hat{\mu}} \, \hat{\mu}(x_{1:n})$ for every $x_{1:n}$. Hence $\mu(x_{1:n})/\xi(x_{1:n}) \leq \mu(x_{1:n})/(w_{\hat{\mu}} \, \hat{\mu}(x_{1:n}))$, and taking logs,

$$\ln \frac{\mu(x_{1:n})}{\xi(x_{1:n})} \;\leq\; -\ln w_{\hat{\mu}} \;+\; \ln \frac{\mu(x_{1:n})}{\hat{\mu}(x_{1:n})}.$$

Now take the μ-expectation. The LHS becomes $D_n(\mu \parallel \xi)$; the second RHS term becomes $D_n(\mu \parallel \hat{\mu})$; the constant $-\ln w_{\hat{\mu}}$ survives because $\mu(X_{1:n})$ has total mass 1.

Exercise 8.1: Hint p. 324

Add and subtract $\xi(x_t \mid x_{<t})$ inside the squared term, then expand. Use $\sum_\nu w(\nu \mid x_{<t}) = 1$ and the posterior-predictive form of ξ (Definition 1.2), $\sum_\nu w(\nu \mid x_{<t}) \, \nu(x_t \mid x_{<t}) = \xi(x_t \mid x_{<t})$, to kill the cross-term. The weighting must be *posterior* weights $w(\nu \mid x_{<t})$ (not prior weights w_ν), because $\xi(x_t \mid x_{<t})$ is the posterior-weighted mean of the $\nu(x_t \mid x_{<t})$'s, not the prior one.

Exercise 8.1: Solution p. 324

Throughout, the history $x_{<t}$ and symbol x_t are fixed but arbitrary. Abbreviate $\nu := \nu(x_t \mid x_{<t})$, $\xi := \xi(x_t \mid x_{<t})$, $\rho := \rho(x_t \mid x_{<t})$, and $\tilde{w}_\nu := w(\nu \mid x_{<t})$ for the duration of the calculation.

Step 1: ξ is the posterior-weighted mean of the ν's. By the posterior-predictive form of the mixture (Definition 1.2) and the posterior normalization $\sum_\nu \tilde{w}_\nu = 1$,

$$\sum_\nu \tilde{w}_\nu \, \nu \;=\; \xi.$$

Step 2: the cross-term vanishes. A direct consequence of Step 1, again using $\sum_\nu \tilde{w}_\nu = 1$:

$$\sum_\nu \tilde{w}_\nu \, (\nu - \xi) \;=\; \sum_\nu \tilde{w}_\nu \, \nu \;-\; \xi \sum_\nu \tilde{w}_\nu \;=\; \xi - \xi \;=\; 0.$$

Step 3: expand the square. Write $\nu - \rho = (\nu - \xi) + (\xi - \rho)$ and expand:

$$\begin{aligned}
&\sum_\nu \tilde{w}_\nu (\nu - \rho)^2 \\
&\quad = \sum_\nu \tilde{w}_\nu \big[(\nu - \xi)^2 + 2(\nu - \xi)(\xi - \rho) + (\xi - \rho)^2 \big] \\
&\quad = \sum_\nu \tilde{w}_\nu (\nu - \xi)^2 \;+\; 2(\xi - \rho) \sum_\nu \tilde{w}_\nu (\nu - \xi) \;+\; (\xi - \rho)^2 \sum_\nu \tilde{w}_\nu .
\end{aligned}$$

The middle sum is zero by Step 2; the trailing $\sum_\nu \tilde{w}_\nu = 1$. So

$$(\xi - \rho)^2 \;+\; \sum_\nu \tilde{w}_\nu (\nu - \xi)^2 \;=\; \sum_\nu \tilde{w}_\nu (\nu - \rho)^2 .$$

Exercise 8.2: Hint p. 324

Multiply the assumed inequality by $w_\nu \geq 0$ and sum over ν; then for each fixed history $x_{<t}$, multiply the per-history Pythagorean identity from above by $\xi(x_{<t})$ and use the Bayes identity $w(\nu \mid x_{<t})\, \xi(x_{<t}) = w_\nu \, \nu(x_{<t})$ to bridge from posterior weights (inside the identity) to prior weights (in the aggregated Pareto inequality).

Exercise 8.2: Solution p. 324

Multiply the assumed inequality by $w_\nu \geq 0$ and sum over $\nu \in \mathcal{M}$:

$$\sum_\nu w_\nu \, S_n(\nu \parallel \rho) \;\leq\; \sum_\nu w_\nu \, S_n(\nu \parallel \xi).$$

Now bridge to the per-history Pythagorean. At each *fixed* history $x_{<t} \in \mathbb{B}^{t-1}$ and symbol $x_t \in \mathbb{B}$, the identity says

$$(\xi - \rho)^2 \;+\; \sum_\nu w(\nu \mid x_{<t}) \, (\nu - \xi)^2 \;=\; \sum_\nu w(\nu \mid x_{<t}) \, (\nu - \rho)^2 ,$$

with the abbreviations $\nu := \nu(x_t \mid x_{<t})$, $\xi := \xi(x_t \mid x_{<t})$, $\rho := \rho(x_t \mid x_{<t})$. Multiply by $\xi(x_{<t})$ and use Bayes, $w(\nu \mid x_{<t})\, \xi(x_{<t}) = w_\nu \, \nu(x_{<t})$:

$$\xi(x_{<t})(\xi - \rho)^2 \;+\; \sum_\nu w_\nu \, \nu(x_{<t}) \, (\nu - \xi)^2 \;=\; \sum_\nu w_\nu \, \nu(x_{<t}) \, (\nu - \rho)^2 .$$

Sum over $t = 1, \dots, n$ and all histories $x_{<t} \in \mathbb{B}^{t-1}$ and symbols $x_t \in \mathbb{B}$. The two $\sum_\nu w_\nu \, \nu(x_{<t})(\cdots)$ terms become exactly $\sum_\nu w_\nu \, S_n(\nu \parallel \xi)$ and $\sum_\nu w_\nu \, S_n(\nu \parallel \rho)$ respectively, so

$$\|\xi - \rho\|_\xi^2 \; + \; \sum_\nu w_\nu \, S_n(\nu \parallel \xi) \; = \; \sum_\nu w_\nu \, S_n(\nu \parallel \rho),$$

where

$$\|\xi - \rho\|_\xi^2 \; := \; \sum_{t=1}^{n} \sum_{x_{<t}} \xi(x_{<t}) \sum_{x_t} \big(\xi(x_t \mid x_{<t}) - \rho(x_t \mid x_{<t})\big)^2 \; \geq \; 0.$$

Combining with Equation 4,

$$\|\xi - \rho\|_\xi^2 \; + \; \sum_\nu w_\nu \, S_n(\nu \parallel \xi) \; \leq \; \sum_\nu w_\nu \, S_n(\nu \parallel \xi) \quad \Longrightarrow \quad \|\xi - \rho\|_\xi^2 \; \leq \; 0.$$

Since $\|\xi - \rho\|_\xi^2$ is a sum of non-negative terms and is itself ≤ 0, every term vanishes: $\xi(x_{<t})\big(\xi(x_t \mid x_{<t}) - \rho(x_t \mid x_{<t})\big)^2 = 0$ at every $(t, x_{<t}, x_t)$. So $\rho(x_t \mid x_{<t}) = \xi(x_t \mid x_{<t})$ at every history $x_{<t}$ with $\xi(x_{<t}) > 0$.

AIXI

David Quarel (ARENA)

Main text: Chapter D.3.2, p. 327

Exercise 0.1: Solution p. 330

Expanding the definition: $\nu^\pi(æ_{1:t}) = \prod_{k=1}^{t} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k) = \underbrace{\prod_{k=1}^{t} \pi(a_k \mid æ_{<k})}_{\pi(æ_{1:t})} \cdot \underbrace{\prod_{k=1}^{t} \nu(e_k \mid æ_{<k}a_k)}_{\nu(æ_{1:t})}.$

Key observation: $\pi(æ_{1:t})$ is the same regardless of the environment. When comparing ν^π and μ^π, the policy factors cancel.

Exercise 0.2: Solution p. 330

From the definition $\nu(æ_{1:t}) = \prod_{k=1}^{t} \nu(e_k \mid æ_{<k}a_k)$, split off the last factor:

$$\nu(æ_{1:t}) \;=\; \prod_{k=1}^{t-1} \nu(e_k \mid æ_{<k}a_k) \cdot \nu(e_t \mid æ_{<t}a_t) \;=\; \nu(æ_{<t}) \cdot \nu(e_t \mid æ_{<t}a_t).$$

Exercise 0.3: Solution p. 330

From Definition 0.4, the one-step interaction is $\nu^\pi(a_t e_t \mid æ_{<t}) = \pi(a_t \mid æ_{<t})\, \nu(e_t \mid æ_{<t}a_t)$. Summing over e_t:

$$\sum_{e_t} \nu^\pi(a_t e_t \mid æ_{<t}) \;=\; \pi(a_t \mid æ_{<t}) \underbrace{\sum_{e_t} \nu(e_t \mid æ_{<t}a_t)}_{=1} \;=\; \pi(a_t \mid æ_{<t}).$$

Interpretation: after summing out the environment's response, only the agent's action probability remains.

Exercise 0.4: Solution p. 330

Since π is deterministic, $\pi(a_k \mid æ_{<k}) = [\![a_k = a_k^*]\!]$. By hypothesis $æ_{i:j}$ is π-consistent, so every such bracket equals 1. From Definition 0.4:

$$\begin{aligned}\nu^\pi(æ_{i:j} \mid æ_{<i}) \;&=\; \prod_{k=i}^{j} \underbrace{\pi(a_k \mid æ_{<k})}_{=1}\, \nu(e_k \mid æ_{<k}a_k) \\ &=\; \prod_{k=i}^{j} \nu(e_k \mid æ_{<k}\, a_k) \;=\; \nu(æ_{i:j} \mid æ_{<i}),\end{aligned}$$

where the last equality is the natural segment extension of $\nu(æ_{1:t}) := \prod_{k=1}^{t} \nu(e_k \mid æ_{<k}a_k)$.

Exercise 0.5: Solution p. 330

Split the defining product $\nu^\pi(æ_{p:r} \mid æ_{<p}) = \prod_{k=p}^{r} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k)$ at index q:

$$\begin{aligned}\nu^\pi(æ_{p:r} \mid æ_{<p}) \;&=\; \prod_{k=p}^{r} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k) \\ =\; \underbrace{\prod_{k=p}^{q} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k)}_{\nu^\pi(æ_{p:q} \mid æ_{<p})} \;&\cdot\; \underbrace{\prod_{k=q+1}^{r} \pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k)}_{\nu^\pi(æ_{q+1:r} \mid æ_{<p}\, æ_{p:q})},\end{aligned}$$

identifying each block as a conditional via the same expansion: the past for step $k \geq q+1$ is the given history $æ_{<p}$ extended by the first chunk $æ_{p:q}$.

Note: for a deterministic policy, Exercise 0.4 simplifies this further: each factor $\pi(a_k \mid æ_{<k})\, \nu(e_k \mid æ_{<k}a_k)$ becomes just $\nu(e_k \mid æ_{<k}\, a_k^*)$.

Exercise 1.1: Hint p. 332

Use the definitions of $w(\nu \mid æ_{1:t})$ and $w(\nu \mid æ_{<t})$, and apply Exercise 0.2.

Exercise 1.1: Solution p. 332

From the definition: $w(\nu \mid æ_{1:t}) = w_\nu \cdot \nu(æ_{1:t})/\xi(æ_{1:t})$ and $w(\nu \mid æ_{<t}) = w_\nu \cdot \nu(æ_{<t})/\xi(æ_{<t})$.

Dividing:

$$\frac{w(\nu \mid æ_{1:t})}{w(\nu \mid æ_{<t})} = \frac{\nu(æ_{1:t})}{\nu(æ_{<t})} \cdot \frac{\xi(æ_{<t})}{\xi(æ_{1:t})} = \frac{\nu(e_t \mid æ_{<t}a_t)}{\xi(e_t \mid æ_{<t}a_t)},$$

where we used the chain rule (Exercise 0.2) for both ν and ξ.

Exercise 1.2: Hint p. 333

Write $\xi(e_t \mid æ_{<t}a_t) = \xi(æ_{1:t})/\xi(æ_{<t})$, expand the numerator, and use Exercise 0.2.

Exercise 1.2: Solution p. 333

The one-step conditional is the ratio of joint to marginal: $\xi(e_t \mid æ_{<t}a_t) := \xi(æ_{1:t})/\xi(æ_{<t})$.

Expanding the numerator using $\xi(æ_{1:t}) = \sum_\nu w_\nu \nu(æ_{1:t})$:

$$\begin{aligned}
\xi(e_t \mid æ_{<t}a_t) &= \frac{\sum_{\nu\in\mathcal{M}} w_\nu\, \nu(æ_{1:t})}{\xi(æ_{<t})} \\
&= \frac{\sum_\nu w_\nu\, \nu(æ_{<t}) \cdot \nu(e_t \mid æ_{<t}a_t)}{\xi(æ_{<t})} \quad \text{(chain rule, Exercise 0.2)} \\
&= \sum_\nu \underbrace{\frac{w_\nu\, \nu(æ_{<t})}{\xi(æ_{<t})}}_{= w(\nu \mid æ_{<t})} \cdot\, \nu(e_t \mid æ_{<t}a_t) \\
&= \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, \nu(e_t \mid æ_{<t}a_t).
\end{aligned}$$

Exercise 1.3: Hint p. 333

Recall the formula for geometric series: $(1-\gamma)\sum_{k=0}^{n}\gamma^k = 1-\gamma^{n+1}$.

Exercise 1.3: Solution p. 333

Since $r_k \in [0,1]$, each discounted reward sum is bounded:

$$0 \;\le\; (1-\gamma)\, G_{t-1:m} \;\le\; (1-\gamma)\sum_{k=t}^{m}\gamma^{k-t} \;=\; 1-\gamma^{m-t+1} \;\le\; 1.$$

The interaction measure satisfies $\nu^\pi(æ'_{t:m} \mid æ_{<t}) \ge 0$ and $\sum_{æ'_{t:m}} \nu^\pi(æ'_{t:m} \mid æ_{<t}) = 1$, so the value function is a weighted average of terms in $[0,1]$:

$$0 \;\le\; V_\nu^{\pi,m}(æ_{<t}) \;=\; \sum_{æ'_{t:m}} \nu^\pi(æ'_{t:m} \mid æ_{<t})\,(1-\gamma)\, G_{t-1:m} \;\le\; 1.$$

For $m=\infty$: since $0 \le V_\nu^{\pi,m}(æ_{<t}) \le 1$ for all finite m, the limit $V_\nu^\pi(æ_{<t}) = \lim_{m\to\infty} V_\nu^{\pi,m}(æ_{<t})$ also lies in $[0,1]$.

Exercise 1.4: Hint p. 333

Apply the general chain rule (Exercise 0.5) to $\xi(æ_{1:m}) = \sum_\nu w_\nu \nu(æ_{1:m})$, then use factorization (Exercise 0.1).

Exercise 1.4: Solution p. 333

Start from the definition $\xi(æ_{1:m}) = \sum_{\nu\in\mathcal{M}} w_\nu\, \nu(æ_{1:m})$. Apply the general chain rule (Exercise 0.5) to both sides: $\xi(æ_{1:m}) = \xi(æ_{<t}) \cdot \xi(æ_{t:m} \mid æ_{<t})$ and $\nu(æ_{1:m}) = \nu(æ_{<t}) \cdot \nu(æ_{t:m} \mid æ_{<t})$. Dividing by $\xi(æ_{<t})$:

$$\begin{aligned}\xi(æ_{t:m} \mid æ_{<t}) &= \sum_{\nu\in\mathcal{M}} \frac{w_\nu\,\nu(æ_{<t})}{\xi(æ_{<t})}\nu(æ_{t:m} \mid æ_{<t}) \\ &= \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\,\nu(æ_{t:m} \mid æ_{<t}).\end{aligned}$$

Now multiply both sides by $\pi(æ_{t:m} \mid æ_{<t})$. By factorization (Exercise 0.1), $\pi(æ_{t:m} \mid æ_{<t}) \cdot \xi(æ_{t:m} \mid æ_{<t}) = \xi^\pi(æ_{t:m} \mid æ_{<t})$ and likewise for each ν:

$$\xi^\pi(æ_{t:m} \mid æ_{<t}) = \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\,\nu^\pi(æ_{t:m} \mid æ_{<t}).$$

Exercise 2.1: Solution p. 333

Let $a^* \in \arg\max_{a'\in\mathcal{A}} Q(a')$, which exists because $\mathcal{A}$ is finite. So $Q(a^*) = \max_{a'} Q(a')$.

Upper bound. For any π: $\sum_a \pi(a \mid æ_{<t})Q(a) \le \sum_a \pi(a \mid æ_{<t})Q(a^*) = Q(a^*)$.

Lower bound. The deterministic π^* with $\pi^*(a^* \mid æ_{<t}) = 1$ achieves $Q(a^*)$.

Combining: $\sup_\pi \sum_a \pi(a)Q(a) = Q(a^*) = \max_{a'} Q(a')$, attained by π^*.

Exercise 2.2: Hint p. 333

Use the general chain rule (Exercise 0.5) to peel off the *first* step,

$$\nu^\pi(æ_{t:m} \mid æ_{<t}) = \nu^\pi(æ_t \mid æ_{<t}) \cdot \nu^\pi(æ_{t+1:m} \mid æ_{1:t}).$$

Break up the sum $\sum_{æ_{t:m}} = \sum_{æ_t} \sum_{æ_{t+1:m}}$, and factor out r'_t using $\sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ) = 1$ for any history æ.

Exercise 2.2: Solution p. 333

Step 1: Factor the future. Write $æ'_{t:m} = æ'_t\,æ'_{t+1:m}$: $\nu^\pi(æ'_{t:m} \mid æ_{<t}) = \nu^\pi(æ'_t \mid æ_{<t}) \cdot \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\,æ'_t)$.

Step 2: Split the reward sum. This is just the return recursion (Definition 0.3): $G_{t-1:m} = r'_t + \gamma\,G_{t:m}$, where $G_{t-1:m} = \sum_{k=t}^m \gamma^{k-t} r'_k$ is the return given $æ_{<t}$.

Step 3: Substitute into the definition of $V_\nu^{\pi,m}(æ_{<t})$.

$$\begin{aligned}V_\nu^{\pi,m}(æ_{<t}) &= (1-\gamma)\sum_{æ'_t} \nu^\pi(æ'_t \mid æ_{<t}) \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\,æ'_t) \\ &\qquad \times [r'_t + \gamma G_{t:m}]\,.\end{aligned}$$

Consider the inner term $\sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\,æ'_t)\,[r'_t + \gamma G_{t:m}]$. We can expand as:

$$
\begin{aligned}
&= \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) r'_t + \gamma \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) G_{t:m} \\
&= r'_t \cancel{\sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t)} + \gamma \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) G_{t:m} \\
&= r'_t + \gamma \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) G_{t:m}
\end{aligned}
$$

as r'_t doesn't depend on $æ'_{t+1:m}$, and $\sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) = 1$. Inserting this above, we obtain:

$$
V_\nu^{\pi,m}(æ_{<t}) = \sum_{æ'_t} \nu^\pi(æ'_t \mid æ_{<t}) \Bigg[(1-\gamma)\, r'_t + \gamma \underbrace{(1-\gamma) \sum_{æ'_{t+1:m}} \nu^\pi(æ'_{t+1:m} \mid æ_{<t}\, æ'_t) G_{t:m}}_{= V_\nu^{\pi,m}(æ_{<t}\, æ'_t)} \Bigg].
$$

Exercise 2.3: Solution p. 333

We induct on $t = m, m-1, \ldots, 1$, showing that for every history $æ_{<t}$, a deterministic policy achieves $V_\nu^{*,m}(æ_{<t})$.

Base case ($t = m$): At $t = m$ the continuation term vanishes (the value from time $m+1$ is an empty sum), so the Bellman equation gives $V_\nu^{\pi,m}(æ_{<m}) = (1-\gamma) \sum_{æ'_m} \nu^\pi(æ'_m \mid æ_{<m})\, r'_m$. Splitting the step $æ'_m = a'_m e'_m$ via $\nu^\pi(æ'_m \mid æ_{<m}) = \pi(a'_m \mid æ_{<m})\, \nu(e'_m \mid æ_{<m} a'_m)$ gives $V_\nu^{\pi,m}(æ_{<m}) = (1-\gamma) \sum_{a'_m} \pi(a'_m \mid æ_{<m}) \sum_{e'_m} \nu(e'_m \mid æ_{<m} a'_m)\, r'_m$. This has the form $\sum_a \pi(a) Q(a)$ with $Q(a) = (1-\gamma) \sum_{e'_m} \nu(e'_m \mid æ_{<m} a)\, r'_m$, which is independent of π. By Exercise 2.1, the supremum over π is $\max_a Q(a)$, attained by the deterministic policy playing $a^* \in \arg\max_a Q(a)$.

Inductive step: Suppose that for every history of length t, there exists a deterministic policy π^*_{t+1} achieving $V_\nu^{\pi^*_{t+1},m} = V_\nu^{*,m}$ from time $t+1$ onwards. Substituting this optimal continuation into the Bellman equation from Exercise 2.2 gives

$$
V_\nu^{\pi,m}(æ_{<t}) \;=\; \sum_{æ'_t} \nu^\pi(æ'_t \mid æ_{<t}) \big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, æ'_t) \big].
$$

Splitting the step $æ'_t = a'_t e'_t$ via $\nu^\pi(æ'_t \mid æ_{<t}) = \pi(a'_t \mid æ_{<t})\, \nu(e'_t \mid æ_{<t} a'_t)$ and grouping the percept sum into the action weight:

$$
V_\nu^{\pi,m}(æ_{<t}) \;=\; \sum_{a'_t} \pi(a'_t \mid æ_{<t})\, Q(a'_t),
$$

where $Q(a'_t) := \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t) [(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a'_t\, e'_t)]$ is independent of π (the continuation value $V_\nu^{*,m}$ is fixed by the inductive hypothesis). By Exercise 2.1, $\sup_\pi \sum_{a'_t} \pi(a'_t)\, Q(a'_t) = \max_{a'_t} Q(a'_t)$, attained by the deterministic policy playing $a^*_t \in$

$\arg\max_{a'_t} Q(a'_t)$ at time t, then following π^*_{t+1}.

Exercise 2.4: Solution p. 334

By Exercise 2.3, a deterministic optimal policy π^*_ν exists with $V_\nu^{\pi^*_\nu,m} = V_\nu^{*,m}$. Substituting $\pi = \pi^*_\nu$ into the Bellman equation from Exercise 2.2 (using $V_\nu^{\pi^*_\nu,m} = V_\nu^{*,m}$ on both sides):

$$V_\nu^{*,m}(æ_{<t}) \;=\; \sum_{æ'_t} \nu^{\pi^*_\nu}(æ'_t \mid æ_{<t}) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, æ'_t)\Big].$$

Splitting the step $æ'_t = a'_t e'_t$ via $\nu^{\pi^*_\nu}(æ'_t \mid æ_{<t}) = \pi^*_\nu(a'_t \mid æ_{<t})\, \nu(e'_t \mid æ_{<t} a'_t)$:

$$\begin{aligned} V_\nu^{*,m}(æ_{<t}) \;=\; & \sum_{a'_t} \pi^*_\nu(a'_t \mid æ_{<t}) \\ & \times \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a'_t\, e'_t)\Big]. \end{aligned}$$

Since π^*_ν is deterministic, let a^*_t denote the unique action with $\pi^*_\nu(a^*_t \mid æ_{<t}) = 1$. Then:

$$V_\nu^{*,m}(æ_{<t}) \;=\; \sum_{e'_t} \nu(e'_t \mid æ_{<t} a^*_t) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a^*_t\, e'_t)\Big].$$

Define $Q(a) := \sum_{e'_t} \nu(e'_t \mid æ_{<t} a)[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a\, e'_t)]$. Then $V_\nu^{*,m}(æ_{<t}) = Q(a^*_t)$. We must have $Q(a^*_t) = \max_a Q(a)$: if some $\tilde{a}$ had $Q(\tilde{a}) > Q(a^*_t)$, then the policy that plays $\tilde{a}$ at $æ_{<t}$ and follows π^*_ν elsewhere would achieve value $Q(\tilde{a}) > V_\nu^{*,m}(æ_{<t})$, contradicting $V_\nu^{*,m} = \sup_\pi V_\nu^{\pi,m}$. Hence:

$$V_\nu^{*,m}(æ_{<t}) \;=\; \max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a'_t\, e'_t)\Big].$$

Exercise 2.5: Hint p. 334

Show $V_\nu^{*,m+1}(æ_{<t}) \geq V_\nu^{*,m}(æ_{<t})$ by expanding $V_\nu^{*,m+1}$. Use this and Exercise 1.3 and Monotone Convergence Theorem to obtain the result.

Exercise 2.5: Solution p. 334

We show $V_\nu^{*,m+1}(æ_{<t}) \geq V_\nu^{*,m}(æ_{<t})$ for all $æ_{<t}$. Since $V_\nu^{*,m+1} = \sup_\pi V_\nu^{\pi,m+1}$, it suffices to show $V_\nu^{\pi,m+1}(æ_{<t}) \geq V_\nu^{\pi,m}(æ_{<t})$ for every π. Expanding the definition:

$$
\begin{aligned}
&V_\nu^{\pi,m+1}(æ_{<t}) \\
&= (1-\gamma) \sum_{æ_{t:m+1}} \nu^\pi(æ_{t:m+1} \mid æ_{<t})\, G_{t-1:m+1} \\
&= (1-\gamma) \sum_{æ_{t:m+1}} \nu^\pi(æ_{t:m+1} \mid æ_{<t}) \left[G_{t-1:m} \;+\; \gamma^{m+1-t} r_{m+1}\right] \\
&= (1-\gamma) \sum_{æ_{t:m+1}} \nu^\pi(æ_{t:m+1} \mid æ_{<t}) G_{t-1:m} \\
&\qquad + \underbrace{(1-\gamma)\,\gamma^{m+1-t} \sum_{æ_{t:m+1}} \nu^\pi(æ_{t:m+1} \mid æ_{<t})\, r_{m+1}}_{\geq 0}.
\end{aligned}
$$

For the first term, marginalize over a_{m+1}, e_{m+1} (which $G_{t-1:m}$ does not depend on):

$$
\sum_{æ_{t:m+1}} \nu^\pi(æ_{t:m+1} \mid æ_{<t}) G_{t-1:m} \;=\; \sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t}) G_{t-1:m}.
$$

So $V_\nu^{\pi,m+1}(æ_{<t}) \geq (1-\gamma) \sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t}) G_{t-1:m} = V_\nu^{\pi,m}(æ_{<t})$.

Since $V_\nu^{*,m}(æ_{<t})$ is non-decreasing in m and bounded in $[0,1]$ by Exercise 1.3, the monotone convergence theorem gives that $V_\nu^*(æ_{<t}) := \lim_{m\to\infty} V_\nu^{*,m}(æ_{<t})$ exists for each $æ_{<t}$.

Exercise 2.6: Solution p. 334

By Exercise 2.4, for every finite m:

$$
V_\nu^{*,m}(æ_{<t}) \;=\; \max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*,m}(æ_{<t}\, a'_t\, e'_t)\Big].
$$

By Exercise 2.5, each $V_\nu^{*,m}(\cdot)$ converges pointwise to $V_\nu^*(\cdot)$ as $m \to \infty$. Since $\mathcal{A}$ and $\mathcal{E}$ are finite, the max and $\sum$ are over finitely many convergent terms, so:

$$
V_\nu^{*}(æ_{<t}) \;=\; \max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*}(æ_{<t}\, a'_t\, e'_t)\Big].
$$

Exercise 2.7: Solution p. 334

Step 1: Define the greedy policy. For each history $æ_{<t}$, let

$$
a_t^* \in \arg\max_{a'_t} \sum_{e'_t} \nu(e'_t \mid æ_{<t} a'_t)[(1-\gamma)\, r'_t + \gamma\, V_\nu^*(æ_{<t}\, a'_t\, e'_t)],
$$

which exists because $\mathcal{A}$ is finite. Define the deterministic policy $\pi_\nu^*(a \mid æ_{<t}) := [\![a = a_t^*]\!]$. By Exercise 2.6, the infinite-horizon Bellman optimality equation becomes:

$$
V_\nu^{*}(æ_{<t}) \;=\; \sum_{e'_t} \nu(e'_t \mid æ_{<t}\, a_t^*) \Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{*}(æ_{<t}\, a_t^*\, e'_t)\Big].
$$

Step 2: Error recursion. The Bellman equation (Exercise 2.2) holds for any policy, so for π^*_ν at finite horizon m:

$$V_\nu^{\pi^*_\nu,m}(æ_{<t}) \;=\; \sum_{e'_t} \nu(e'_t \mid æ_{<t}\, a^*_t)\Big[(1-\gamma)\, r'_t + \gamma\, V_\nu^{\pi^*_\nu,m}(æ_{<t}\, a^*_t\, e'_t)\Big].$$

Subtracting this from the equation in Step 1, the $(1-\gamma)\, r'_t$ terms cancel:

$$\begin{aligned} V^*_\nu(æ_{<t}) - V_\nu^{\pi^*_\nu,m}(æ_{<t}) \;&=\; \gamma \sum_{e'_t} \nu(e'_t \mid æ_{<t}\, a^*_t) \\ &\quad \times \Big[V^*_\nu(æ_{<t}\, a^*_t\, e'_t) - V_\nu^{\pi^*_\nu,m}(æ_{<t}\, a^*_t\, e'_t)\Big]. \end{aligned}$$

The error at time t is γ times a weighted average of errors at time $t+1$.

Step 3: Iterate the contraction. Write $\Delta V^m_\nu(æ_{<t}) := V^*_\nu(æ_{<t}) - V_\nu^{\pi^*_\nu,m}(æ_{<t}) \geq 0$ for the error. Step 2 gives:

$$\Delta V^m_\nu(æ_{<t}) \;=\; \gamma \sum_{e'_t} \nu(e'_t \mid æ_{<t}\, a^*_t)\, \Delta V^m_\nu(æ_{<t}\, a^*_t\, e'_t).$$

Applying the same recursion at time $t+1$ (with greedy action a^*_{t+1} at history $æ_{<t}\, a^*_t\, e'_t$):

$$\begin{aligned} \Delta V^m_\nu(æ_{<t}) \;&=\; \gamma^2 \sum_{e'_t} \nu(e'_t \mid æ_{<t}\, a^*_t) \sum_{e'_{t+1}} \nu(e'_{t+1} \mid æ_{<t}\, a^*_t\, e'_t\, a^*_{t+1}) \\ &\quad \times \Delta V^m_\nu(æ_{<t}\, a^*_t\, e'_t\, a^*_{t+1}\, e'_{t+1}). \end{aligned}$$

After $m-t+1$ iterations, summing over all percept sequences $e'_{t:m}$:

$$\Delta V^m_\nu(æ_{<t}) \;=\; \gamma^{m-t+1} \sum_{e'_{t:m}} \left(\prod_{k=t}^{m} \nu(e'_k \mid æ'_{<k}\, a^*_k)\right) \Delta V^m_\nu(æ_{<t}\, æ'_{t:m}),$$

where $æ'_{t:m} = a^*_t\, e'_t \cdots a^*_m\, e'_m$ is the history segment generated by π^*_ν. By Exercise 0.4, the product of ν-conditionals is $\nu^{\pi^*_\nu}(æ'_{t:m} \mid æ_{<t})$. At the boundary, $V_\nu^{\pi^*_\nu,m}(æ_{<t}\, æ'_{t:m}) = 0$ (summing over no rewards past horizon m), so $\Delta V^m_\nu(æ_{<t}\, æ'_{t:m}) = V^*_\nu(æ_{<t}\, æ'_{t:m}) \in [0,1]$. Therefore:

$$\Delta V^m_\nu(æ_{<t}) \;=\; \gamma^{m-t+1} \sum_{æ'_{t:m}} \nu^{\pi^*_\nu}(æ'_{t:m} \mid æ_{<t})\, V^*_\nu(æ_{<t}\, æ'_{t:m}) \;\leq\; \gamma^{m-t+1}.$$

Step 4: Take $m \to \infty$. Since $0 \leq V^*_\nu(æ_{<t}) - V_\nu^{\pi^*_\nu,m}(æ_{<t}) \leq \gamma^{m-t+1} \to 0$, we have $V_\nu^{\pi^*_\nu,m}(æ_{<t}) \to V^*_\nu(æ_{<t})$, i.e. $V_\nu^{\pi^*_\nu} = V^*_\nu$.

Exercise 3.1: Solution p. 334

$\xi(æ_{<t}) = \sum_{\nu' \in \mathcal{M}} w_{\nu'} \nu'(æ_{<t}) \geq w_\nu \nu(æ_{<t})$.

Exercise 3.2: Solution p. 334

By Exercise 0.1: $\xi^\pi(æ_{1:m}) = \pi(æ_{1:m}) \cdot \xi(æ_{1:m}) \geq \pi(æ_{1:m}) \cdot w_\nu \nu(æ_{1:m}) = w_\nu \cdot \nu^\pi(æ_{1:m})$.

Exercise 4.1: Hint p. 335

Use the multi-step posterior linearity of ξ^π (Exercise 1.4).

Exercise 4.1: Solution p. 335

By Exercise 1.4: $\xi^\pi(æ_{t:m} \mid æ_{<t}) = \sum_\nu w(\nu \mid æ_{<t})\, \nu^\pi(æ_{t:m} \mid æ_{<t})$. Multiply both sides by $(1-\gamma)G_{t-1:m}$ and sum over all $æ_{t:m}$:

$$\begin{aligned} V_\xi^{\pi,m}(æ_{<t}) &= (1-\gamma) \sum_{æ_{t:m}} \xi^\pi(æ_{t:m} \mid æ_{<t}) G_{t-1:m} \\ &= (1-\gamma) \sum_{æ_{t:m}} \left[\sum_\nu w(\nu \mid æ_{<t})\, \nu^\pi(æ_{t:m} \mid æ_{<t}) \right] G_{t-1:m} \\ &= \sum_\nu w(\nu \mid æ_{<t}) \underbrace{(1-\gamma) \sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t}) G_{t-1:m}}_{= V_\nu^{\pi,m}(æ_{<t})} . \end{aligned}$$

In the last step we exchanged $\sum_{æ_{t:m}}$ and $\sum_\nu$; this is valid because both are sums of non-negative terms (or, for finite m, the sum over $æ_{t:m}$ is finite). So:

$$V_\xi^{\pi,m}(æ_{<t}) = \sum_{\nu \in \mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{\pi,m}(æ_{<t}).$$

Exercise 4.2: Hint p. 335

Take $m \to \infty$ in the finite-horizon result. You will need Fact 4.1 to exchange limit and sum for countably infinite $\mathcal{M}$.

Exercise 4.2: Solution p. 335

The left side converges: $V_\xi^{\pi,m}(æ_{<t}) \to V_\xi^\pi(æ_{<t})$ by definition.

For the right side, we need to push $\lim_{m\to\infty}$ through $\sum_\nu$. If $\mathcal{M}$ is finite this is immediate. For countably infinite $\mathcal{M}$, we apply Fact 4.1 (dominated convergence for sums):

- For each ν: $a_{\nu,m} := w(\nu \mid æ_{<t})\, V_\nu^{\pi,m}(æ_{<t}) \to w(\nu \mid æ_{<t})\, V_\nu^\pi(æ_{<t})$ as $m \to \infty$ by definition.
- Domination: $|a_{\nu,m}| \leq w(\nu \mid æ_{<t}) \cdot 1 =: b_\nu$ for all m, since $V_\nu^{\pi,m} \in [0,1]$ (Exercise 1.3).
- Summability: $\sum_\nu b_\nu = \sum_\nu w(\nu \mid æ_{<t}) = 1 < \infty$.

So Fact 4.1 gives:

$$V_\xi^\pi(æ_{<t}) = \lim_{m\to\infty} \sum_\nu w(\nu \mid æ_{<t})\, V_\nu^{\pi,m}(æ_{<t}) = \sum_\nu w(\nu \mid æ_{<t})\, V_\nu^\pi(æ_{<t}).$$

Exercise 4.3: Hint p. 335

Apply Exercise 4.1 with $\pi = \pi^*_\xi$, the Bayes-optimal policy for ξ.

Exercise 4.3: Solution p. 335

By Exercise 2.3, a deterministic Bayes-optimal policy π^*_ξ exists with $V_\xi^{\pi^*_\xi,m} = V_\xi^{*,m}$. Applying Exercise 4.1 with $\pi = \pi^*_\xi$:

$$V_\xi^{*,m}(æ_{<t}) \;=\; V_\xi^{\pi^*_\xi,m}(æ_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{\pi^*_\xi,m}(æ_{<t}).$$

Since π^*_ξ is optimal for ξ but not necessarily for each individual ν, we have $V_\nu^{\pi^*_\xi,m}(æ_{<t}) \le V_\nu^{*,m}(æ_{<t})$ for each ν. Since $w(\nu \mid æ_{<t}) \ge 0$:

$$V_\xi^{*,m}(æ_{<t}) \;\le\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V_\nu^{*,m}(æ_{<t}).$$

Exercise 4.4: Hint p. 335

Consider $\mathcal{M} = \{\nu_0, \nu_1\}$: predicting a two-headed coin vs. a two-tailed coin.

Exercise 4.4: Solution p. 335

Coin-flip prediction. Let $\mathcal{A} = \mathcal{O} = \mathcal{R} = \{0,1\}$, $\mathcal{M} = \{\nu_0, \nu_1\}$ with $w_{\nu_0} = w_{\nu_1} = \frac{1}{2}$, horizon $m = 1$. Environment ν_i always shows outcome i, and the agent is rewarded for a correct prediction:

$$\nu_i(e_t \mid æ_{<t}\, a_t) \;:\; o_t = i, \quad r_t = [\![a_t = i]\!].$$

In ν_i the optimal policy plays $a_t = i$, achieving $V_{\nu_i}^{*,m} = 1$. In $\xi = \frac{1}{2}\nu_0 + \frac{1}{2}\nu_1$ the outcome is a fair coin flip, so no policy predicts better than chance: $V_\xi^{*,m} = \frac{1}{2}$. Therefore:

$$\sum_\nu w_\nu\, V_\nu^{*,m}(æ_{<t}) \;=\; \tfrac{1}{2}\cdot 1 + \tfrac{1}{2}\cdot 1 \;=\; 1 \;>\; \tfrac{1}{2} \;=\; V_\xi^{*,m}(æ_{<t}).$$

Exercise 5.1: Hint p. 336

Prove the first display by backward induction on t from m down to 1, using the finite-horizon Bellman optimality equation (Exercise 2.4) and the return recursion $G_{t-1:m} = r_t + \gamma\, G_{t:m}$ (Definition 0.3). At each step the leftover reward term $(1-\gamma)\, r_t$ is constant in the deeper variables; carry it inward using $\sum_{e_k} \xi(e_k \mid æ_{<k} a_k) = 1$ and $c + \max(\cdot) = \max(c + \cdot)$.

Exercise 5.1: Solution p. 336

Abbreviate the one-step predictor $\xi_k := \xi(e_k \mid æ_{<k} a_k)$, and write the **expectimax operator** over steps $i, \dots, j$ as

$$\sum_{i:j}^{\max} := \max_{a_i} \sum_{e_i} \xi_i \max_{a_{i+1}} \sum_{e_{i+1}} \xi_{i+1} \cdots \max_{a_j} \sum_{e_j} \xi_j,$$

the alternating "maximize over the action, average over the percept" chain, with the ξ_k weights built in; the single step is $\sum_{k}^{\max} := \max_{a_k} \sum_{e_k} \xi_k$. Two facts:

- **(E1) Composition:** $\sum_{i:j}^{\max} = \sum_{i}^{\max} \sum_{i+1:j}^{\max}$, by nesting the definition.
- **(E2) Affine pass-through:** for any $c \in \mathbb{R}$ and $\lambda \geq 0$ not depending on $(a_i, e_i, \ldots, a_j, e_j)$,

$$c + \lambda \sum_{i:j}^{\max} X \;=\; \sum_{i:j}^{\max} (c + \lambda X),$$

since each layer has $\sum_{e_k} \xi_k = 1$ (so $\sum_{e_k} \xi_k (c + \lambda Y) = c + \lambda \sum_{e_k} \xi_k Y$) and $\max_a (c + \lambda\, g(a)) = c + \lambda \max_a g(a)$ for $\lambda \geq 0$.

We prove by backward induction on $t = m, m-1, \ldots, 1$ that

$$V_\xi^{*,m}(æ_{<t}) \;=\; (1-\gamma) \sum_{t:m}^{\max} G_{t-1:m}.$$

Base case $t = m$. Here $G_{m-1:m} = r_m$ and $V_\xi^{*,m}(æ_{1:m}) = 0$ (the return past the horizon is empty, $G_{m:m} = 0$). The Bellman optimality equation (Exercise 2.4) for $\nu = \xi$ gives

$$V_\xi^{*,m}(æ_{<m}) \;=\; \max_{a_m} \sum_{e_m} \xi_m \big[(1-\gamma)\, r_m + \gamma \cdot 0\big] \;=\; (1-\gamma) \sum_{m:m}^{\max} G_{m-1:m}.$$

Inductive step. Assume the claim at time $t+1$, i.e. $V_\xi^{*,m}(æ_{1:t}) = (1-\gamma) \sum_{t+1:m}^{\max} G_{t:m}$ for every length-t history (recall $æ_{<t+1} = æ_{1:t}$). The Bellman optimality equation (Exercise 2.4) at time t reads

$$V_\xi^{*,m}(æ_{<t}) \;=\; \sum_{t}^{\max} \big[(1-\gamma)\, r_t + \gamma\, V_\xi^{*,m}(æ_{1:t})\big].$$

Substituting the inductive hypothesis and pulling out $(1-\gamma)$,

$$V_\xi^{*,m}(æ_{<t}) \;=\; (1-\gamma) \sum_{t}^{\max} \Big[r_t + \gamma \sum_{t+1:m}^{\max} G_{t:m}\Big].$$

As r_t and γ do not depend on $(a_{t+1}, e_{t+1}, \ldots, a_m, e_m)$, property (E2) carries them inside the inner operator, and the return recursion $G_{t-1:m} = r_t + \gamma\, G_{t:m}$ (Definition 0.3) gives

$$r_t + \gamma \sum_{t+1:m}^{\max} G_{t:m} \overset{(E2)}{=} \sum_{t+1:m}^{\max} (r_t + \gamma\, G_{t:m}) \;=\; \sum_{t+1:m}^{\max} G_{t-1:m}.$$

Recombining the two operators by (E1),

$$V_\xi^{*,m}(æ_{<t}) \;=\; (1-\gamma)\sum_{t}^{\max}\sum_{t+1:m}^{\max} G_{t-1:m} \;=\; (1-\gamma)\sum_{t:m}^{\max} G_{t-1:m},$$

which expands back into the max / $\sum$ layers of the first display.

Infinite horizon and the action. Taking $m \to \infty$ (Exercises 2.5 and 2.7), $G_{t-1:m} \to G_{\geq t-1}$ and the chain extends indefinitely, so $V_\xi^*(æ_{<t}) = (1-\gamma)\sum_{t:\infty}^{\max} G_{\geq t-1}$. Collecting the per-step factors by the chain rule (Exercise 0.2, iterated), $\prod_{k=t}^{m}\xi_k = \xi(e_{t:m} \mid æ_{<t}\, a_{t:m})$. A Bayes-optimal action maximizes this expression; peeling the outer $\max_{a_t}$ off as an arg max and dropping the positive constant $(1-\gamma)$ (which does not move the maximizer) gives the stated action.

Exercise 6.1: Hint p. 336

Define $A^+ = \{\omega \in \Omega : P(\omega) \geq Q(\omega)\}$ and decompose the expectation difference as a sum over A^+ and its complement $A^- = \Omega \setminus A^+$.

Exercise 6.1: Solution p. 336

Write out the expectation difference using Definition 6.2:

$$\mathbb{E}_P[f] - \mathbb{E}_Q[f] \;=\; \sum_{\omega\in\Omega} f(\omega)\big(P(\omega) - Q(\omega)\big).$$

Define $A^+ := \{\omega \in \Omega : P(\omega) \geq Q(\omega)\}$ and $A^- := \Omega \setminus A^+$, and split the sum:

$$\mathbb{E}_P[f] - \mathbb{E}_Q[f] \;=\; \sum_{\omega\in A^+} f(\omega)\big(P(\omega) - Q(\omega)\big) \;+\; \sum_{\omega\in A^-} f(\omega)\big(P(\omega) - Q(\omega)\big).$$

Bounding the sum over A^+. On A^+, $P(\omega) - Q(\omega) \geq 0$ and $f(\omega) \leq c$, so:

$$\begin{aligned}\sum_{\omega\in A^+} f(\omega)\big(P(\omega) - Q(\omega)\big) \;&\leq\; c\sum_{\omega\in A^+}\big(P(\omega) - Q(\omega)\big)\\ &=\; c\cdot\big(P(A^+) - Q(A^+)\big)\\ &\leq\; c\cdot\sup_S |P(S) - Q(S)|\\ &=\; c\cdot \mathrm{TV}[\Omega](P,Q).\end{aligned}$$

Bounding the sum over A^-. On A^-, $P(\omega) - Q(\omega) < 0$ and $f(\omega) \geq 0$, so every term $f(\omega)(P(\omega) - Q(\omega)) \leq 0$:

$$\sum_{\omega\in A^-} f(\omega)\big(P(\omega) - Q(\omega)\big) \;\leq\; 0.$$

Combining. $\mathbb{E}_P[f] - \mathbb{E}_Q[f] \leq c\cdot\mathrm{TV}[\Omega](P,Q) + 0 = c\cdot\mathrm{TV}[\Omega](P,Q)$.

The other direction. Swapping P and Q: define $\tilde{A}^+ = \{\omega : Q(\omega) \geq P(\omega)\} = A^-$. The same argument gives $\mathbb{E}_Q[f] - \mathbb{E}_P[f] \leq c\cdot\mathrm{TV}[\Omega](Q,P) = c\cdot\mathrm{TV}[\Omega](P,Q)$.

Therefore $|\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \le c \cdot \mathrm{TV}[\Omega](P, Q)$.

Exercise 7.1: Solution p. 339

We want to apply Section 6. Identify:

- $\Omega := \mathcal{H}^{m-t+1} = (\mathcal{A} \times \mathcal{E})^{m-t+1}$, the finite set of future history segments $æ_{t:m}$.
- $P := \mu^\pi(\cdot \mid æ_{<t})$ and $Q := \xi^\pi(\cdot \mid æ_{<t})$, the conditional measures over Ω.
- $f(æ_{t:m}) := (1-\gamma)G_{t-1:m}$, which satisfies $f \in [0,1]$ by Exercise 1.3 (so $c = 1$).

Then $V_\nu^{\pi,m}(æ_{<t}) = \sum_{æ_{t:m}} \nu^\pi(æ_{t:m} \mid æ_{<t})\, f(æ_{t:m})$ is exactly $\mathbb{E}_P[f]$ (for $\nu = \mu$) or $\mathbb{E}_Q[f]$ (for $\nu = \xi$). Applying Section 6:

$$\left|V_\xi^{\pi,m}(æ_{<t}) - V_\mu^{\pi,m}(æ_{<t})\right| \;=\; \left|\mathbb{E}_Q[f] - \mathbb{E}_P[f]\right| \;\le\; 1 \cdot \mathrm{TV}[\mathcal{H}^{m-t+1}](P, Q).$$

Exercise 7.2: Hint p. 339

Use Section 3.

Exercise 7.2: Solution p. 339

By Section 3, $\xi^\pi(æ_{1:m}) \ge w_\mu \cdot \mu^\pi(æ_{1:m})$ for all histories and all m. So for any event A: $\mu^\pi(A) > 0 \implies \xi^\pi(A) \ge w_\mu \cdot \mu^\pi(A) > 0$. Hence ξ^π covers μ^π.

Exercise 7.3: Solution p. 339

Step 1: Finite-horizon bound. By Exercise 7.1, for each finite m:

$$\begin{aligned}\left|V_\xi^{\pi,m}(æ_{<t}) - V_\mu^{\pi,m}(æ_{<t})\right| &\le \mathrm{TV}[\mathcal{H}^{m-t+1}]\big(\xi^\pi(\cdot \mid æ_{<t}),\, \mu^\pi(\cdot \mid æ_{<t})\big) \\ &\le \sup_S \left|\xi^\pi(S \mid æ_{<t}) - \mu^\pi(S \mid æ_{<t})\right|,\end{aligned}$$

where the second inequality uses that the sup over subsets of $\mathcal{H}^{m-t+1}$ is at most the sup over all measurable events.

Step 2: Take $m \to \infty$. The LHS converges to $|V_\xi^\pi(æ_{<t}) - V_\mu^\pi(æ_{<t})|$ (by definition of the infinite-horizon value as the pointwise limit). The RHS does not depend on m. Hence

$$\left|V_\xi^\pi(æ_{<t}) - V_\mu^\pi(æ_{<t})\right| \;\le\; \sup_S \left|\xi^\pi(S \mid æ_{<t}) - \mu^\pi(S \mid æ_{<t})\right|.$$

Step 3: Take $t \to \infty$. By Exercise 7.2, ξ^π covers μ^π. By Fact 7.7 (Blackwell–Dubins) with $P = \mu^\pi$ and $Q = \xi^\pi$, the RHS tends to 0 as $t \to \infty$, μ^π-a.s. Therefore

$$\left|V_\xi^\pi(æ_{<t}) - V_\mu^\pi(æ_{<t})\right| \;\to\; 0 \qquad \mu^\pi\text{-a.s.}$$

Exercise 7.4: Hint p. 339

See (Blackwell & Dubins 1962). The proof uses the Radon–Nikodym derivative dP/dQ, Lévy's martingale convergence theorem, and the Lebesgue decomposition. No elementary proof is known that avoids graduate-level measure theory.

Exercise 8.1: Hint p. 340

Use linearity of V^π (Exercise 4.2).

Exercise 8.1: Solution p. 340

By definition, $V^*_\xi(æ_{<t}) = \sup_\pi V^\pi_\xi(æ_{<t}) \geq V^{\pi'}_\xi(æ_{<t})$ for any policy π'. By Exercise 4.2 (linearity of V^π in the environment):

$$V^{\pi'}_\xi(æ_{<t}) \;=\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V^{\pi'}_\nu(æ_{<t}).$$

Choose $\pi' = \pi^*_\mu$, the deterministic optimal policy for μ whose existence is guaranteed by Exercise 2.7:

$$V^*_\xi(æ_{<t}) \;\geq\; \sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\, V^{\pi^*_\mu}_\nu(æ_{<t}).$$

Every term is non-negative ($w(\nu \mid æ_{<t}) \geq 0$ and $V^{\pi^*_\mu}_\nu \geq 0$). Dropping all terms except $\nu = \mu$:

$$V^*_\xi(æ_{<t}) \;\geq\; w(\mu \mid æ_{<t})\, V^{\pi^*_\mu}_\mu(æ_{<t}) \;=\; w(\mu \mid æ_{<t})\, V^*_\mu(æ_{<t}).$$

Exercise 8.2: Solution p. 340

From Definition 0.1, the posterior weight is:

$$w(\mu \mid æ_{<t}) \;=\; w_\mu \cdot \frac{\mu(æ_{<t})}{\xi(æ_{<t})} \;=\; w_\mu \cdot \frac{\prod_{k=1}^{t-1} \mu(e_k \mid æ_{<k}a_k)}{\prod_{k=1}^{t-1} \xi(e_k \mid æ_{<k}a_k)}.$$

Since μ is deterministic, for the history that μ actually generates, $\mu(e_k \mid æ_{<k}a_k) = 1$ for each k (the environment produces each percept with certainty). So:

$$\mu(æ_{<t}) \;=\; \prod_{k=1}^{t-1} 1 \;=\; 1.$$

Meanwhile, $\xi(e_k \mid æ_{<k}a_k) \leq 1$ for each k (it is a probability), so $\xi(æ_{<t}) = \prod_{k=1}^{t-1} \xi(e_k \mid æ_{<k}a_k) \leq 1$.

Therefore $w(\mu \mid æ_{<t}) = w_\mu \cdot 1/\xi(æ_{<t}) \geq w_\mu$.

Combining with Exercise 8.1:

$$V^*_\xi(æ_{<t}) \;\geq\; w(\mu \mid æ_{<t}) \cdot V^*_\mu(æ_{<t}) \;\geq\; w_\mu \cdot V^*_\mu(æ_{<t}) \;>\; w_\mu \cdot \varepsilon \;>\; 0.$$

Exercise 9.1: Hint p. 341

Write $X_{\nu,t}(æ_{1:t}) = X_{\nu,t-1}(æ_{<t}) \cdot \nu(e_t \mid æ_{<t}a_t)/\mu(e_t \mid æ_{<t}a_t)$. Sum over a_t, e_t; μ cancels, leaving $\sum_{e_t} \nu(e_t \mid æ_{<t}a_t) = 1$.

Exercise 9.1: Solution p. 341

We need to compute $\mathbb{E}_{\mu^\pi}[X_{\nu,t} \mid æ_{<t}]$. Conditioning on $æ_{<t}$ means a_t and e_t are the random quantities (sampled from π and μ respectively). First, write $X_{\nu,t}$ pointwise as a function of history in terms of $X_{\nu,t-1}$:

$$\begin{aligned} X_{\nu,t}(æ_{1:t}) &= \frac{\nu(æ_{1:t})}{\mu(æ_{1:t})} = \frac{\nu(æ_{<t})}{\mu(æ_{<t})} \cdot \frac{\nu(e_t \mid æ_{<t}a_t)}{\mu(e_t \mid æ_{<t}a_t)} \\ &= X_{\nu,t-1}(æ_{<t}) \cdot \frac{\nu(e_t \mid æ_{<t}a_t)}{\mu(e_t \mid æ_{<t}a_t)}, \end{aligned}$$

using the chain rule (Exercise 0.2) for both ν and μ. Now take the conditional expectation. Since $X_{\nu,t-1}$ depends only on $æ_{<t}$, it comes out of the expectation:

$$\begin{aligned} \mathbb{E}_{\mu^\pi}[X_{\nu,t} \mid æ_{<t}] &= X_{\nu,t-1} \cdot \mathbb{E}_{\mu^\pi}\left[\frac{\nu(e_t \mid æ_{<t}a_t)}{\mu(e_t \mid æ_{<t}a_t)} \;\middle|\; æ_{<t}\right] \\ &= X_{\nu,t-1} \sum_{a_t \in \mathcal{A}} \pi(a_t \mid æ_{<t}) \sum_{e_t \in \mathcal{E}} \mu(e_t \mid æ_{<t}a_t) \cdot \frac{\nu(e_t \mid æ_{<t}a_t)}{\mu(e_t \mid æ_{<t}a_t)} \\ &= X_{\nu,t-1} \sum_{a_t} \pi(a_t \mid æ_{<t}) \sum_{e_t} \nu(e_t \mid æ_{<t}a_t). \end{aligned}$$

In the last step, $\mu(e_t \mid æ_{<t}a_t)$ cancels. Since ν is a probability measure, $\sum_{e_t} \nu(e_t \mid æ_{<t}a_t) = 1$ for every $æ_{<t}, a_t$; and $\sum_{a_t} \pi(a_t \mid æ_{<t}) = 1$. Therefore:

$$\mathbb{E}_{\mu^\pi}[X_{\nu,t} \mid æ_{<t}] = X_{\nu,t-1} \cdot 1 \cdot 1 = X_{\nu,t-1}.$$

Since $X_{\nu,t} \geq 0$ (a ratio of non-negative quantities), $(X_{\nu,t})_{t \geq 0}$ is a non-negative μ^π-martingale, in particular, a non-negative μ^π-supermartingale.

Exercise 9.2: Solution p. 341

$X_{\nu,t}$ is a non-negative μ^π-supermartingale by Exercise 9.1. By Fact 9.4 (Doob's convergence theorem), $X_{\nu,t}$ converges μ^π-a.s. to a finite limit: $X_{\nu,t} \to X_{\nu,\infty} < \infty$.

Exercise 9.3: Solution p. 341

From the definition:

$$X_{\xi,t} := \frac{\xi(æ_{1:t})}{\mu(æ_{1:t})} = \frac{\sum_\nu w_\nu \nu(æ_{1:t})}{\mu(æ_{1:t})} = \sum_\nu w_\nu \cdot \frac{\nu(æ_{1:t})}{\mu(æ_{1:t})} = \sum_\nu w_\nu X_{\nu,t}.$$

By Section 3: $\xi(æ_{1:t}) \geq w_\mu \cdot \mu(æ_{1:t})$, so $X_{\xi,t} = \xi(æ_{1:t})/\mu(æ_{1:t}) \geq w_\mu > 0$.

Since $\mathcal{M}$ is finite, $X_{\xi,t} = \sum_\nu w_\nu X_{\nu,t}$ is a finite sum of convergent sequences, so $X_{\xi,t} \to X_{\xi,\infty} := \sum_\nu w_\nu X_{\nu,\infty} < \infty$ μ^π-a.s., with $X_{\xi,\infty} \geq w_\mu > 0$.

Exercise 10.1: Hint p. 341

Use Exercise 0.1 to show $\nu^\pi/\mu^\pi = \nu/\mu = X_{\nu,m}$.

Exercise 10.1: Solution p. 341

By Exercise 0.1, $\nu^\pi(æ'_{1:m}) = \pi(æ'_{1:m}) \cdot \nu(æ'_{1:m})$ and $\mu^\pi(æ'_{1:m}) = \pi(æ'_{1:m}) \cdot \mu(æ'_{1:m})$. For any history $æ'_{1:m}$ with $\mu^\pi(æ'_{1:m}) > 0$, the policy factors cancel:

$$\begin{aligned}\nu^\pi(æ'_{1:m}) &= \frac{\nu^\pi(æ'_{1:m})}{\mu^\pi(æ'_{1:m})} \cdot \mu^\pi(æ'_{1:m}) = \frac{\nu(æ'_{1:m})}{\mu(æ'_{1:m})} \cdot \mu^\pi(æ'_{1:m}) \\ &= X_{\nu,m}(æ'_{1:m}) \cdot \mu^\pi(æ'_{1:m}).\end{aligned}$$

For histories with $\mu^\pi(æ'_{1:m}) = 0$: since $\mu^\pi = \pi \cdot \mu$, some $\pi(a_k \mid æ'_{<k}) = 0$, which forces $\nu^\pi(æ'_{1:m}) = 0$ too (the same π-factor appears in $\nu^\pi = \pi \cdot \nu$). So both sides are zero.

Summing over $æ'_{1:m} \in A$:

$$\begin{aligned}\nu^\pi[æ_{1:m} \in A] &= \sum_{æ'_{1:m} \in A} \nu^\pi(æ'_{1:m}) = \sum_{æ'_{1:m} \in A} X_{\nu,m}(æ'_{1:m}) \cdot \mu^\pi(æ'_{1:m}) \\ &= \mathbb{E}_{\mu^\pi}\big[X_{\nu,m} \cdot [\![æ_{1:m} \in A]\!]\big].\end{aligned}$$

Exercise 10.3: Hint p. 342

On $E \cap \{X_{\nu,\infty} \geq \varepsilon\}$, $X_{\nu,\infty} \geq \varepsilon$ pointwise. Take μ^π-expectations and apply Exercise 10.2.

Exercise 10.3: Solution p. 342

Let $E_\varepsilon := E \cap \{X_{\nu,\infty} \geq \varepsilon\}$. On E_ε, $X_{\nu,\infty} \geq \varepsilon$, so pointwise

$$X_{\nu,\infty} \cdot [\![æ_{1:\infty} \in E_\varepsilon]\!] \;\geq\; \varepsilon \cdot [\![æ_{1:\infty} \in E_\varepsilon]\!].$$

Take μ^π-expectations and apply Exercise 10.2:

$$\nu^\pi[E_\varepsilon] \;=\; \mathbb{E}_{\mu^\pi}\big[X_{\nu,\infty} \cdot [\![æ_{1:\infty} \in E_\varepsilon]\!]\big] \;\geq\; \varepsilon \cdot \mathbb{E}_{\mu^\pi}\big[[\![æ_{1:\infty} \in E_\varepsilon]\!]\big] \;=\; \varepsilon \cdot \mu^\pi[E_\varepsilon].$$

Since $E_\varepsilon \subseteq E$, $\nu^\pi[E_\varepsilon] \leq \nu^\pi[E]$. Dividing by ε gives $\mu^\pi[E_\varepsilon] \leq \nu^\pi[E]/\varepsilon$.

Interpretation. A likelihood ratio of at least ε forces ν^π and μ^π measures of an event to be comparable: μ^π of the event can exceed ν^π of it only by the factor $1/\varepsilon$. In particular, if ν^π vanishes on E, so does μ^π on the part where the likelihood ratio stays bounded away from zero.

Exercise 11.1: Hint p. 342

Use $V_\xi^{\pi_\xi^*} \geq V_\xi^{\tilde\pi}$ and Exercise 4.2.

Exercise 11.1: Solution p. 342

We establish two inequalities and chain them.

First inequality: isolate the μ-term. For each $\nu \in \mathcal{M}$, the optimal value is at least the value of any policy: $V_\nu^*(æ_{<t}) \geq V_\nu^{\pi_\xi^*}(æ_{<t})$, so $V_\nu^* - V_\nu^{\pi_\xi^*} \geq 0$. Since the posterior weights $w(\nu \mid æ_{<t}) \geq 0$, every term in $\sum_\nu w(\nu \mid æ_{<t})[V_\nu^* - V_\nu^{\pi_\xi^*}]$ is non-negative. The μ-term is one such term:

$$w(\mu \mid æ_{<t})\big[V_\mu^* - V_\mu^{\pi_\xi^*}\big] \;\leq\; \sum_{\nu \in \mathcal{M}} w(\nu \mid æ_{<t})\big[V_\nu^* - V_\nu^{\pi_\xi^*}\big].$$

Second inequality: replace π_ξ^* with $\tilde{\pi}$. By definition of the optimal value, $V_\xi^*(æ_{<t}) \geq V_\xi^{\tilde{\pi}}(æ_{<t})$. Expanding both sides using infinite-horizon linearity (Exercise 4.2, applied to the fixed policies π_ξ^* and $\tilde{\pi}$):

$$\begin{aligned}\sum_\nu w(\nu \mid æ_{<t})\, V_\nu^{\pi_\xi^*}(æ_{<t}) \;&=\; V_\xi^*(æ_{<t}) \;\geq\; V_\xi^{\tilde{\pi}}(æ_{<t}) \\ &=\; \sum_\nu w(\nu \mid æ_{<t})\, V_\nu^{\tilde{\pi}}(æ_{<t}).\end{aligned}$$

Rearranging: $\sum_\nu w(\nu)[V_\nu^* - V_\nu^{\pi_\xi^*}] \leq \sum_\nu w(\nu)[V_\nu^* - V_\nu^{\tilde{\pi}}] = \sum_\nu w(\nu)\,\delta_{\nu,t}$.

Chaining:

$$0 \;\leq\; w(\mu \mid æ_{<t})\big[V_\mu^* - V_\mu^{\pi_\xi^*}\big] \;\leq\; \sum_{\nu \in \mathcal{M}} w(\nu \mid æ_{<t})\,\delta_{\nu,t}.$$

Exercise 11.2: Hint p. 343

Show $w(\nu \mid æ_{<t}) = w_\nu X_{\nu,t-1}/X_{\xi,t-1}$ and use Exercise 9.3.

Exercise 11.2: Solution p. 343

From the posterior weight formula, dividing numerator and denominator by $\mu(æ_{<t})$:

$$w(\nu \mid æ_{<t}) \;=\; w_\nu \cdot \frac{\nu(æ_{<t})}{\xi(æ_{<t})} \;=\; w_\nu \cdot \frac{\nu(æ_{<t})/\mu(æ_{<t})}{\xi(æ_{<t})/\mu(æ_{<t})} \;=\; \frac{w_\nu \cdot X_{\nu,t-1}}{X_{\xi,t-1}},$$

where $X_{\nu,t-1}$ and $X_{\xi,t-1}$ are evaluated at the length-$(t-1)$ history $æ_{<t}$.

If $X_{\nu,\infty} = 0$, then $X_{\nu,t-1} \to 0$ μ^π-a.s. (Exercise 9.2). Since $X_{\xi,t-1} \geq w_\mu > 0$ always (Exercise 9.3), the ratio $w(\nu \mid æ_{<t}) = w_\nu X_{\nu,t-1}/X_{\xi,t-1} \to 0$ μ^π-a.s.

Exercise 11.3: Hint p. 343

Self-optimizing (Definition 9.1) gives $\nu^\pi[F] = 0$. Apply Exercise 10.3 with $E = F$ and $\varepsilon = 1/n$ to conclude $\mu^\pi[F \cap \{X_{\nu,\infty} \geq 1/n\}] = 0$ for each $n \geq 1$. Then note $\{X_{\nu,\infty} > 0\} = \bigcup_{n \geq 1}\{X_{\nu,\infty} \geq 1/n\}$ and take the countable union (Definition 7.1).

Exercise 11.3: Solution p. 343

Step 1: F is ν^π-null. By the self-optimizing assumption (Definition 9.1), under ν^π the suboptimality gap $\delta_{\nu,t}$ vanishes, so $\nu^\pi[F] = 0$.

Step 2: Apply Exercise 10.3. For each $n \geq 1$, take $E = F$ and $\varepsilon = 1/n$:

$$\mu^\pi\big[F \cap \{X_{\nu,\infty} \geq 1/n\}\big] \;\leq\; \frac{\nu^\pi[F]}{1/n} \;=\; 0.$$

Step 3: Countable union. The level sets of $X_{\nu,\infty}$ form a nested increasing family: $\{X_{\nu,\infty} > 0\} = \bigcup_{n\geq 1}\{X_{\nu,\infty} \geq 1/n\}$. Intersecting with F gives $F \cap \{X_{\nu,\infty} > 0\} = \bigcup_{n\geq 1}\big(F \cap \{X_{\nu,\infty} \geq 1/n\}\big)$, a countable union. By countable subadditivity (Definition 7.1):

$$\begin{aligned}\mu^\pi\big[F \cap \{X_{\nu,\infty} > 0\}\big] \;&= \mu^\pi\Big[\bigcup_n F \cap \{X_{\nu,\infty} \geq 1/n\}\Big] \\ &\leq \sum_{n\geq 1} \mu^\pi\big[F \cap \{X_{\nu,\infty} \geq 1/n\}\big] \;=\; 0.\end{aligned}$$

What this says. "$\delta_{\nu,t} \to 0$ μ^π-a.s. on $\{X_{\nu,\infty} > 0\}$" is shorthand for exactly this event statement: among infinite histories on which the likelihood ratio stays bounded away from zero, all but a μ^π-null subset are ones where $\delta_{\nu,t}(æ_{<t}) \to 0$. We do *not* claim convergence on histories where $X_{\nu,\infty} = 0$: that case is handled separately in Exercise 11.2 via the posterior weight.

Exercise 11.4: Solution p. 343

Fix $\nu \in \mathcal{M}$. We show $w(\nu \mid æ_{<t})\,\delta_{\nu,t} \to 0$ μ^π-a.s. by considering two exhaustive cases:

Case 1: $X_{\nu,\infty} = 0$. By Exercise 11.2, $w(\nu \mid æ_{<t}) \to 0$. Since $\delta_{\nu,t} \in [0,1]$ (Exercise 1.3), the product $w(\nu \mid æ_{<t}) \cdot \delta_{\nu,t} \leq 1 \cdot w(\nu \mid æ_{<t}) \to 0$.

Case 2: $X_{\nu,\infty} > 0$. By Exercise 11.3, $\delta_{\nu,t} \to 0$. Since $w(\nu \mid æ_{<t}) \leq 1$ (posterior weights are at most 1), the product $w(\nu \mid æ_{<t}) \cdot \delta_{\nu,t} \leq 1 \cdot \delta_{\nu,t} \to 0$.

These two cases cover all infinite histories (μ^π-a.s.), so $w(\nu \mid æ_{<t})\,\delta_{\nu,t} \to 0$ μ^π-a.s.

Exercise 11.5: Hint p. 343

$w(\mu \mid æ_{<t}) = w_\mu / X_{\xi,t-1}$.

Exercise 11.5: Solution p. 343

Since $\mathcal{M} = \{\nu_1, \ldots, \nu_N\}$ is finite, we can sum Exercise 11.4 over all $\nu \in \mathcal{M}$:

$$\sum_{\nu\in\mathcal{M}} w(\nu \mid æ_{<t})\,\delta_{\nu,t} \;\longrightarrow\; 0 \qquad \mu^\pi\text{-a.s.}$$

From Exercise 11.1:

$$0 \;\leq\; w(\mu \mid æ_{<t})\big[V^*_\mu(æ_{<t}) - V^{\pi^*_\xi}_\mu(æ_{<t})\big] \;\leq\; \sum_\nu w(\nu \mid æ_{<t})\,\delta_{\nu,t} \;\longrightarrow\; 0.$$

So $w(\mu \mid æ_{<t})[V^*_\mu - V^{\pi^*_\xi}_\mu] \to 0$ μ^π-a.s.

It remains to divide by $w(\mu \mid æ_{<t})$. From Exercise 9.3:

$$w(\mu \mid æ_{<t}) \;=\; \frac{w_\mu}{X_{\xi,t-1}}.$$

Fix an infinite history outside the (single) μ^π-null set on which $X_{\xi,t} \to X_{\xi,\infty} < \infty$ fails. Along this history:

- $X_{\xi,t-1} \to X_{\xi,\infty}$, a finite positive number (with $X_{\xi,\infty} \geq w_\mu > 0$ by Exercise 9.3).
- Hence $w(\mu \mid æ_{<t}) = w_\mu / X_{\xi,t-1} \to w_\mu / X_{\xi,\infty} > 0$, so eventually $w(\mu \mid æ_{<t}) \geq w_\mu/(2X_{\xi,\infty}) > 0$.

Dividing the convergence $w(\mu \mid æ_{<t})(V^*_\mu - V^{\pi^*_\xi}_\mu) \to 0$ by this positive (per-history) lower bound, and using $V^*_\mu - V^{\pi^*_\xi}_\mu \geq 0$:

$$V^*_\mu(æ_{<t}) - V^{\pi^*_\xi}_\mu(æ_{<t}) \;\longrightarrow\; 0 \qquad \mu^\pi\text{-a.s.}$$

This completes the proof of Theorem 9.2.

Agent Foundations

Daniel C, Satya Benson (Williams College)

Main text: Chapter D.4.1, p. 345

Exercise 3.1(a): Hint p. 353

Consider two cases. Either $Z(Z)$ halts or it doesn't. In each case, use the bridge between programs and L (if a program halts, L can prove it; if L proves something, the corresponding proof-search program finds that proof and halts) to derive what follows. One of the two cases leads to a contradiction with the consistency of L.

Exercise 3.1(b): Hint p. 353

In Exercise 3.1(a), you argued from outside L that G is true, and the argument used only one assumption about L: that L is consistent. If L can prove its own consistency, then every step of your outside argument can be carried out **inside** L as a formal derivation. What would L then be able to prove? And what would the corresponding program do?

Exercise 3.1(i): Hint p. 353

The statement "if A then B" is logically equivalent to "if not B then not A". Apply this to $\Box P \to P$.

Exercise 3.1: Solution p. 354

Recall $Z(A)$ searches for an L-proof of $\neg\mathsf{Halts}(A(A))$ and halts if it finds one, and

$$G \;:=\; \neg\mathsf{Halts}(Z(Z)).$$

By construction $Z(Z)$ searches for an L-proof of G, so

$$Z(Z) \text{ halts} \iff L \vdash (G)$$

(a) G is true (assuming L consistent).

Solution. Two cases.

- *$Z(Z)$ halts.* By $(\star)$, $L \vdash G$, i.e. $L \vdash \neg\mathsf{Halts}(Z(Z))$. But $Z(Z)$ actually halts, so by the bridge $L \vdash \mathsf{Halts}(Z(Z))$. Then L proves both $\mathsf{Halts}(Z(Z))$ and its negation, contradicting consistency.

- *$Z(Z)$ runs forever.* Then $\neg\mathsf{Halts}(Z(Z))$, i.e. G, is true.

Consistency rules out the first case, so $Z(Z)$ runs forever and G is true. □

(b) If $L \vdash \neg\Box K$ (i.e. L proves its own consistency) then L is inconsistent.

Solution. The argument of Exercise 3.1(a) used only the consistency of L, and each step is a finite manipulation of programs and proofs that L can formalize. Reading it inside L: "$Z(Z)$ halts" yields $\Box G$ (by construction) and $\Box\mathsf{Halts}(Z(Z))$ (the bridge), hence $\Box K$; contrapositively $L \vdash \neg\Box K \to G$. If $L \vdash \neg\Box K$, modus ponens gives

$$L \vdash G, \qquad \text{i.e.} \qquad L \vdash \neg\mathsf{Halts}(Z(Z)).$$

But $L \vdash G$ means $Z(Z)$'s search finds a proof of G, so $Z(Z)$ halts; by the bridge $L \vdash \mathsf{Halts}(Z(Z))$. Now L proves both $\mathsf{Halts}(Z(Z))$ and $\neg\mathsf{Halts}(Z(Z))$, so L is inconsistent. □

(c) If $L \vdash \Box P \to P$ for every P, then L is inconsistent.

Solution. (i) Fix P with $L \vdash \neg P$. The contrapositive of $\Box P \to P$ is $\neg P \to \neg\Box P$, so from $L \vdash \Box P \to P$ we get $L \vdash \neg P \to \neg\Box P$. With $L \vdash \neg P$, modus ponens gives $L \vdash \neg\Box P$.

(ii) Take $P = K$. Since $\neg K$ is a tautology, $L \vdash \neg K$, so by (i) $L \vdash \neg\Box K$. Thus L proves its own consistency, and Exercise 3.1(b) makes L inconsistent. □

Exercise 3.2(a): Hint p. 354

If a proof of φ exists, then $\texttt{ProofSeeker}(\varphi)$ will find it and halt.

Exercise 3.2: Hint p. 354

If both $\texttt{ProofSeeker}(\varphi \to \psi)$ and $\texttt{ProofSeeker}(\varphi)$ halt, what can you do with their outputs? How can L deduce that $\texttt{ProofSeeker}(\psi)$ halts?

Exercise 3.2: Hint p. 355

The Löb sentence says $L \vdash \lambda \to (\Box\lambda \to C)$. Apply Necessitation to get this fact inside a $\Box$, then use Distribution twice (once to "unwrap" the outer implication, once to handle $\Box\lambda \to C$ inside). Use the Löb condition to handle the resulting $\Box\Box\lambda$.

Exercise 3.2(d): Hint p. 355

From Exercise 3.2(c), you have $L \vdash \Box\lambda \to \Box C$. Chain this with the assumption $L \vdash \Box C \to C$ to get $L \vdash \Box\lambda \to C$. Now compare this with what λ says about itself.

Exercise 3.2: Hint p. 356

From FairBot's source code, $\Box A$ implies that $\texttt{FairBot}_2$ finds a proof that $\texttt{FairBot}_1$ cooperates, so $\texttt{FairBot}_2$ cooperates, i.e. B is true. Similarly $\Box B$ implies A. Combine these to show $L \vdash \Box(A \wedge B) \to (A \wedge B)$, and apply Löb's theorem.

Exercise 3.2: Solution p. 356

We use Necessitation (N) $L \vdash \varphi \Rightarrow L \vdash \Box\varphi$, Distribution (K) $L \vdash \Box(\varphi \to \psi) \to (\Box\varphi \to \Box\psi)$, and the Löb condition (4) $L \vdash \Box\varphi \to \Box\Box\varphi$.

(a) Necessitation. If $L \vdash \varphi$ there is a concrete proof of φ; searching all strings, $\mathsf{ProofSeeker}(\varphi)$ meets it and halts. "$\mathsf{ProofSeeker}(\varphi)$ halts" is exactly $\Box\varphi$, and a halting run is finite, so by the bridge $L \vdash \Box\varphi$. □

(b) Distribution. A proof of $\varphi \to \psi$ turns a proof of φ into one of ψ (concatenate the two proofs and apply modus ponens). So if $\mathsf{ProofSeeker}(\varphi \to \psi)$ and $\mathsf{ProofSeeker}(\varphi)$ both halt, their outputs combine into a proof of ψ, whence $\mathsf{ProofSeeker}(\psi)$ halts. This combining is a finite procedure L can carry out, so $L \vdash \Box(\varphi \to \psi) \to (\Box\varphi \to \Box\psi)$. □

(c) The Löb sentence λ satisfies $L \vdash \lambda \leftrightarrow (\Box\lambda \to C)$.

$L \vdash \Box\lambda \to \Box C$.

Solution. From $L \vdash \lambda \to (\Box\lambda \to C)$:

$$\begin{aligned} &\text{(N):} \quad L \vdash \Box\big(\lambda \to (\Box\lambda \to C)\big), \\ &\text{(K):} \quad L \vdash \Box\lambda \to \Box(\Box\lambda \to C), \\ &\text{(K):} \quad L \vdash \Box(\Box\lambda \to C) \to (\Box\Box\lambda \to \Box C), \end{aligned}$$

and chaining the last two, $L \vdash \Box\lambda \to (\Box\Box\lambda \to \Box C)$. By (4), $L \vdash \Box\lambda \to \Box\Box\lambda$. Propositionally, from $\Box\lambda$ we obtain $\Box\Box\lambda$ and then $\Box C$, so $L \vdash \Box\lambda \to \Box C$. □

(d) Assuming $L \vdash \Box C \to C$, derive $L \vdash C$.

Solution. Chaining $L \vdash \Box\lambda \to \Box C$ (Exercise 3.2(c)) with $L \vdash \Box C \to C$ gives $L \vdash \Box\lambda \to C$. The Löb equivalence gives the converse direction $L \vdash (\Box\lambda \to C) \to \lambda$, so $L \vdash \lambda$. By (N), $L \vdash \Box\lambda$; with $L \vdash \Box\lambda \to C$, modus ponens yields $L \vdash C$. □

(e) FairBot. With $A :=$ "$\mathsf{FairBot}_1(\mathsf{FairBot}_2) = C$" and $B :=$ "$\mathsf{FairBot}_2(\mathsf{FairBot}_1) = C$", show $L \vdash A \wedge B$.

Solution. From the source code, $\mathsf{FairBot}_1$ returns C iff it finds an L-proof that its opponent $\mathsf{FairBot}_2$ returns C against it, i.e. a proof of B; reading this off the code,

$$L \vdash \Box B \to A, \qquad L \vdash \Box A \to B.$$

For any φ, $\Box(A \wedge B) \to \Box\varphi$ whenever $A \wedge B \to \varphi$ is a tautology: indeed (N) gives $\Box(A \wedge B \to \varphi)$ and (K) then gives $\Box(A \wedge B) \to \Box\varphi$. Applying this to $\varphi = A$ and $\varphi = B$,

$$L \vdash \Box(A \wedge B) \to \Box A, \qquad L \vdash \Box(A \wedge B) \to \Box B.$$

Hence, assuming $\Box(A \wedge B)$: we get $\Box A$ and $\Box B$, then B (from $\Box A \to B$) and A (from $\Box B \to A$), so $A \wedge B$. Thus

$$L \vdash \Box(A \wedge B) \to (A \wedge B),$$

and Löb's theorem with $C := A \wedge B$ gives $L \vdash A \wedge B$. □

Exercise 3.3(a): Hint p. 358

If δ places positive weight on a dominated pure action a_j, replace a_j with the action that dominates it. Does the resulting rule dominate δ?

Exercise 3.3(ii): Hint p. 358

If some δ' dominated δ, compare their expected utilities. What does $\pi_i > 0$ ensure?

Exercise 3.3: Solution p. 359

Higher reward is better; δ' *dominates* δ if $r(\delta') \geq r(\delta)$ coordinatewise with strict inequality in some coordinate. $r(\delta)$ is linear in the mixing weights, $R = \text{conv}\{r(a_1), \ldots, r(a_k)\}$, and $\text{EU}(\delta, \pi) = \pi \cdot r(\delta)$.

(a) *(i)* An admissible $\delta = \sum_j \lambda_j a_j$ puts $\lambda_j = 0$ on every non-Pareto-optimal (dominated) a_j.

Solution. Suppose $\lambda_j > 0$ for a dominated a_j, and let δ' dominate it: $r(\delta') \geq r(a_j)$ with strict inequality in some coordinate i_0. Replace the weight on a_j by δ', i.e. play δ' with the probability λ_j formerly on a_j. This is a valid rule $\hat{\delta}$ with

$$r(\hat{\delta}) = r(\delta) + \lambda_j \big(r(\delta') - r(a_j)\big) \geq r(\delta),$$

and strict in coordinate i_0 (since $\lambda_j > 0$). So $\hat{\delta}$ dominates δ, contradicting admissibility. Hence $\lambda_j = 0$, i.e. $F \subseteq \text{conv}\{\text{Pareto-optimal pure actions}\}$. □

(ii) Every Bayes-optimal rule under a prior π with $\pi_i > 0$ for all i is admissible.

Solution. If δ' dominated such a δ, then with $r(\delta')_{i_0} > r(\delta)_{i_0}$,

$$\text{EU}(\delta', \pi) - \text{EU}(\delta, \pi) = \sum_i \pi_i \big(r(\delta')_i - r(\delta)_i\big) \geq \pi_{i_0} \big(r(\delta')_{i_0} - r(\delta)_{i_0}\big) > 0,$$

since every term is ≥ 0 and the i_0 term is > 0. This contradicts Bayes-optimality, so δ is admissible. □

(b) For a face $\mathcal{F} = \{\sum_{l=1}^{p} \mu_l\, r(a_{i_l}) : \mu_l \geq 0,\ \sum_l \mu_l = 1\}$ with $v_l := r(a_{i_l}) - r(a_{i_1})$ and $H = \text{span}\{v_2, \ldots, v_p\}$: H is exactly the set of directions tangent to $\mathcal{F}$.

Solution. Any point of $\mathcal{F}$ equals $r(a_{i_1}) + \sum_{l \geq 2} \mu_l v_l$, so $\mathcal{F} \subseteq r(a_{i_1}) + H$. Take $x, x + h \in \mathcal{F}$, say $x = \sum_l \mu_l r(a_{i_l})$ and $x + h = \sum_l \mu'_l r(a_{i_l})$ with $\sum_l \mu_l = \sum_l \mu'_l = 1$. Then

$$h = \sum_l (\mu'_l - \mu_l)\, r(a_{i_l}) = \sum_{l \geq 2} (\mu'_l - \mu_l)\, v_l \in H,$$

using $\sum_l (\mu'_l - \mu_l) = 0$ to eliminate the $r(a_{i_1})$ term. Hence moving within $\mathcal{F}$ requires $h \in H$. Conversely, from any relative-interior point ($\mu_l > 0$) every $h \in H$ is realized: $x + \varepsilon h \in \mathcal{F}$ for small $\varepsilon > 0$. So the tangent directions of $\mathcal{F}$ are precisely H. □

(c) Let $\pi \perp H$ be normalized so $\pi_i > 0$ and $\sum_i \pi_i = 1$, and assume no point of R scores strictly higher under π than the points of $\mathcal{F}$.

Solution. (i) For $x, x' \in \mathcal{F}$ we have $x - x' \in H$ by Exercise 3.3(b), so $\pi \cdot (x - x') = 0$. Thus $\pi \cdot r(\delta)$ equals a common value c for all δ with $r(\delta) \in \mathcal{F}$.

(ii) Picture π as the *outward* normal of the supporting hyperplane $\{x : \pi \cdot x = c\}$: by assumption the whole risk set lies on the inner side, $\pi \cdot x \le c$ for all $x \in R$, touching the hyperplane exactly along the face $\mathcal{F}$. Fix any $r(\delta) \in \mathcal{F}$ (so $\mathrm{EU}(\delta, \pi) = c$) and any other rule δ'. The step from the face point $r(\delta)$ to $r(\delta') \in R$ heads back into the risk set, i.e. against the outward normal, so its dot product with π is non-positive:

$$\pi \cdot \big(r(\delta') - r(\delta)\big) \le 0, \qquad \text{equivalently} \qquad \mathrm{EU}(\delta', \pi) \le \mathrm{EU}(\delta, \pi).$$

Since this holds for every δ', each rule with $r(\delta) \in \mathcal{F}$ maximizes expected utility, i.e. is Bayes-optimal under π.

Conclusion. If δ is admissible then $r(\delta) \in F$ lies on some face $\mathcal{F}$; the prior π built from $\mathcal{F}$ has all $\pi_i > 0$ and, by (ii), makes δ Bayes-optimal. Conversely, by part (ii) of Exercise 3.3(a) every Bayes-optimal rule under a strictly positive prior is admissible. The two classes coincide. □

Exercise 3.4: Hint p. 360

Compute $D_{\mathrm{KL}}(P[X, A, O] \,\|\, P[X, A, O \mid do(A)])$ by expanding the log ratio using the factorizations above

Exercise 3.4: Solution p. 360

Write $\Pr[X, A, O] = \Pr[X \mid A, O] \Pr[A \mid O] \Pr[O]$ and the blind baseline $\Pr[X, A, O \mid \mathrm{do}(A)] = \Pr[X \mid A, O] \Pr[A] \Pr[O]$.

Claim. $D_{\mathrm{KL}}\big(\Pr[X] \,\big\|\, \Pr[X \mid \mathrm{do}(A)]\big) \le \mathrm{MI}(A; O)$.

Solution. Compute the divergence between the full joints. The factors $\Pr[X \mid A, O]$ and $\Pr[O]$ cancel in the log-ratio:

$$D_{\mathrm{KL}}\big(\Pr[X, A, O] \,\big\|\, \Pr[X, A, O \mid \mathrm{do}(A)]\big) = \sum_{x,a,o} \Pr[x, a, o] \log \frac{\Pr[a \mid o]}{\Pr[a]}.$$

The summand depends only on a, o; marginalizing x and using $\Pr[a \mid o] / \Pr[a] = \Pr[a, o] / (\Pr[a] \Pr[o])$,

$$= \sum_{a,o} \Pr[a, o] \log \frac{\Pr[a, o]}{\Pr[a] \Pr[o]} = \mathrm{MI}(A; O).$$

Marginalizing the two joints down to X sends them to $\Pr[X]$ and $\Pr[X \mid \mathrm{do}(A)] = \sum_{a,o} \Pr[X \mid a, o] \Pr[a] \Pr[o]$ respectively, so monotonicity of KL under marginalization gives

$$\begin{aligned} D_{\mathrm{KL}}\big(\Pr[X] \,\big\|\, \Pr[X \mid \mathrm{do}(A)]\big) &\le D_{\mathrm{KL}}\big(\Pr[X, A, O] \,\big\|\, \Pr[X, A, O \mid \mathrm{do}(A)]\big) \\ &= \mathrm{MI}(A; O). \end{aligned}$$

Exercise 3.5: Hint p. 361

Write each mutual information as a KL divergence between the joint and the product of marginals, expand the log ratios using the channel factorization, and collect terms.

Exercise 3.5: Hint p. 361

Under Q, the inputs are independent. What happens to $\mathrm{MI}(Y_1;\,Y_2)$ when $X_1 \perp\!\!\!\perp X_2$, given the channel factorization? Use Exercise 3.5(a).

Exercise 3.5: Solution p. 361

The channel factorizes as $P[Y \mid X] = P[Y_1 \mid X_1]P[Y_2 \mid X_2]$, and $P[Y, X] = P[Y \mid X]P[X]$.

(a) $\mathrm{MI}(X;Y) = \mathrm{MI}(X_1;Y_1) + \mathrm{MI}(X_2;Y_2) - \mathrm{MI}(Y_1;Y_2)$.

Solution. Using $P[x,y] = P[y \mid x]P[x]$ and the factorization,

$$\mathrm{MI}(X;Y) = \sum_{x,y} P[x,y] \log \frac{P[y \mid x]}{P[y]} = \sum_{x,y} P[x,y] \log \frac{P[y_1 \mid x_1]P[y_2 \mid x_2]}{P[y_1,y_2]}.$$

The three terms on the right are, respectively,

$$\mathrm{MI}(X_1;Y_1) = \sum P[x,y] \log \tfrac{P[y_1|x_1]}{P[y_1]}, \quad \mathrm{MI}(X_2;Y_2) = \sum P[x,y] \log \tfrac{P[y_2|x_2]}{P[y_2]},$$
$$\mathrm{MI}(Y_1;Y_2) = \sum P[x,y] \log \tfrac{P[y_1,y_2]}{P[y_1]P[y_2]},$$

each written over the full joint (the first depends only on x_1, y_1, etc.). Then

$$\begin{aligned}\mathrm{MI}(X_1;Y_1) + \mathrm{MI}(X_2;Y_2) - \mathrm{MI}(Y_1;Y_2)& \\ = \sum_{x,y} P[x,y] \log \frac{P[y_1 \mid x_1]P[y_2 \mid x_2]}{P[y_1,y_2]} &= \mathrm{MI}(X;Y). \square\end{aligned}$$

(b) For $Q[X_1,X_2] := P[X_1]P[X_2]$, $\mathrm{MI}_Q(X;Y) \geq \mathrm{MI}_P(X;Y)$.

Solution. Q keeps the marginals $Q[X_i] = P[X_i]$ and the channel, so $\mathrm{MI}_Q(X_i;Y_i) = \mathrm{MI}_P(X_i;Y_i)$ for $i = 1,2$ ($\mathrm{MI}(X_i;Y_i)$ depends only on $P[X_i]$ and $P[Y_i \mid X_i]$). Under Q the inputs are independent, so the outputs are too:

$$Q[y_1,y_2] = \sum_{x_1,x_2} Q[x_1]Q[x_2]P[y_1 \mid x_1]P[y_2 \mid x_2] = Q[y_1]\,Q[y_2],$$

hence $\mathrm{MI}_Q(Y_1;Y_2) = 0$. Applying Exercise 3.5(a) under each distribution,

$$\begin{aligned}\mathrm{MI}_Q(X;Y) &= \mathrm{MI}_P(X_1;Y_1) + \mathrm{MI}_P(X_2;Y_2) \\ &= \mathrm{MI}_P(X;Y) + \mathrm{MI}_P(Y_1;Y_2) \geq \mathrm{MI}_P(X;Y),\end{aligned}$$

the last step using $\mathrm{MI}_P(Y_1;Y_2) \geq 0$. $\square$

(c) A throughput-maximizing input with $X_1 \perp X_2$ always exists.

Solution. The input simplex is compact and $\mathrm{MI}(X;Y)$ continuous, so a maximizer P^* exists. Let $Q^* := P^*[X_1]P^*[X_2]$. By Exercise 3.5(b), $\mathrm{MI}_{Q^*}(X;Y) \geq \mathrm{MI}_{P^*}(X;Y) = \max$, so Q^* is also a maximizer, and under Q^* the inputs are independent. □

Instrumental Convergence

Leon Lang (Iliad), based on work by Alex Turner et al.

Main text: Chapter D.6, p. 411

Exercise 2.1: Solution p. 415

Because r depends only on the state and the trajectory is generated by π,

$$\begin{aligned}
V_r^\pi(s) &= \mathbb{E}_\pi\Big[\sum_{t=0}^\infty \gamma^t r(s_t) \;\Big|\; s_0 = s\Big] = \sum_{t=0}^\infty \gamma^t\, \mathbb{E}_\pi\big[r(s_t) \mid s_0 = s\big] \\
&= \sum_{t=0}^\infty \gamma^t \sum_{s' \in \mathcal{S}} \mathbb{P}_\pi[s_t = s' \mid s_0 = s]\, r(s') \\
&= \sum_{s' \in \mathcal{S}} \underbrace{\Big(\sum_{t=0}^\infty \gamma^t\, \mathbb{P}_\pi[s_t = s' \mid s_0 = s]\Big)}_{= f^\pi(s)_{s'}} r(s') \\
&= f^\pi(s)^\top r.
\end{aligned}$$

Taking the maximum over policies,

$$V_r^*(s) = \max_\pi V_r^\pi(s) = \max_\pi f^\pi(s)^\top r = \max_{f \in \mathcal{F}(s)} f^\top r,$$

a linear objective maximized over the option set $\mathcal{F}(s)$.

Exercise 2.2: Solution p. 417

Each rule is, by inspection, a function of the two quality-multisets $\mathrm{EU}_r(X)$ and $\mathrm{EU}_r(\mathcal{F}(s))$ alone — that is, of the form $g\big(\mathrm{EU}_r(X), \mathrm{EU}_r(\mathcal{F}(s))\big)$:

- FracOpt: with $M(r) = \max \mathrm{EU}_r(\mathcal{F}(s))$, it is the multiplicity of $M(r)$ in $\mathrm{EU}_r(X)$ divided by its multiplicity in $\mathrm{EU}_r(\mathcal{F}(s))$.
- Bz_T: it is $\sum_{u \in \mathrm{EU}_r(X)} e^{u/T}$ divided by $\sum_{u \in \mathrm{EU}_r(\mathcal{F}(s))} e^{u/T}$ (note these sums take into account the multiplicity of u in the respective multisets).
- Sat_t: it is the number of entries $\geq t$ in $\mathrm{EU}_r(X)$ divided by the number of entries $\geq t$ in $\mathrm{EU}_r(\mathcal{F}(s))$.

None refers to an option f except through its quality $f^\top r$, so each is EU-determined.

Exercise 2.3: Solution p. 418

Writing out the inner product and reindexing the sum by $s' = \phi^{-1}(s)$ (a bijection of $\mathcal{S}$):

$$(\phi \cdot x)^\top (\phi \cdot y) = \sum_{s \in \mathcal{S}} (\phi \cdot x)_s \, (\phi \cdot y)_s = \sum_{s \in \mathcal{S}} x_{\phi^{-1}(s)} \, y_{\phi^{-1}(s)} = \sum_{s' \in \mathcal{S}} x_{s'} \, y_{s'} = x^\top y.$$

Exercise 3.1: Hint p. 418

given a policy π with $\pi(s) = a'$, relabel it into a policy ρ with $\rho(s) = a$ and $f^\rho(s) = \phi \cdot f^\pi(s)$.

Exercise 3.1: Solution p. 418

We use two facts: the s'-th column of the transition matrix is the one-step law $T^\pi e_{s'} = T(\cdot \mid s', \pi(s'))$ (with T^π linear), and relabelling sends indicators to indicators, $\phi \cdot e_{s'} = e_{\phi(s')}$ (with $\phi\cdot$ linear).

Fix a policy π with $\pi(s) = a'$, and define a policy ρ by $\rho(s) := a$ and, for each $s' \in R'$ with $s' \neq s$, $\rho(\phi(s')) := b'$, the action supplied by (ii) for $b = \pi(s')$ (arbitrary off $\phi(R')$; ϕ is injective, so this is unambiguous).

The a'-dynamics stay in R'. If v is supported on R', so is $T^\pi v$: from any $s' \in R'$ the π-successors are again reachable from s via a', hence lie in R'. In particular $(T^\pi)^t e_s$ is supported on R' for every t.

Intertwining on R'. For every $s' \in R'$,

$$T^\rho e_{\phi(s')} = T(\cdot \mid \phi(s'), \, \rho(\phi(s'))) = \phi \cdot T(\cdot \mid s', \, \pi(s')) = \phi \cdot (T^\pi e_{s'}),$$

where for $s' = s$ this is (i) (using $\pi(s) = a'$ and $\rho(s) = a$), and for $s' \neq s$ it is (ii) (with $b = \pi(s')$). Hence, for v supported on R',

$$T^\rho(\phi \cdot v) = \sum_{s' \in R'} v_{s'} \, T^\rho e_{\phi(s')} = \sum_{s' \in R'} v_{s'} \, \phi \cdot (T^\pi e_{s'}) = \phi \cdot (T^\pi v).$$

Iterating. By induction $(T^\rho)^t e_s = \phi \cdot (T^\pi)^t e_s$ for every t: the base case is $\phi \cdot e_s = e_{\phi(s)} = e_s$, and the step applies the intertwining to $v = (T^\pi)^t e_s$, which is supported on R'. Summing the discounted series, $f^\rho(s) = \phi \cdot f^\pi(s)$. Since $\rho(s) = a$, we have $f^\rho(s) \in \mathcal{F}(s \mid a)$, so $\phi \cdot f^\pi(s) \in \mathcal{F}(s \mid a)$; as π ranged over all policies with $\pi(s) = a'$, this gives $\phi \cdot \mathcal{F}(s \mid a') \subseteq \mathcal{F}(s \mid a)$.

Exercise 3.2: Hint p. 419

write $N_r(X) := \sum_{f \in X} \nu(f^\top r)$, so $A = N_r(\mathcal{F}(s \mid a))/N_r(\mathcal{F}(s))$ and likewise A'. From Exercise 2.3, $N_{\phi \cdot r}(\phi \cdot Y) = N_r(Y)$ for every Y. Compare A, A' at r and at $\phi \cdot r$; the denominators cancel.

Exercise 3.2: Solution p. 419

Write $N_r(X) = \sum_{f \in X} \nu(f^\top r)$, so $p(X \mid s, r) = N_r(X)/N_r(\mathcal{F}(s))$, and put $G := \phi \cdot \mathcal{F}(s \mid a') \subseteq \mathcal{F}(s \mid a)$ — a copy of $\mathcal{F}(s \mid a')$ inside $\mathcal{F}(s \mid a)$, as ϕ is injective. Termwise, Exercise 2.3 gives the **swap identity**

$$N_{\phi \cdot r}(\phi \cdot Y) = \sum_{f \in Y} \nu\big((\phi \cdot f)^\top (\phi \cdot r)\big) = \sum_{f \in Y} \nu(f^\top r) = N_r(Y).$$

Retargeting. Suppose $A'(r) > A(r)$. These share the denominator $N_r(\mathcal{F}(s))$, so $N_r(\mathcal{F}(s \mid a')) > N_r(\mathcal{F}(s \mid a)) \geq N_r(G)$ — the last step since $G \subseteq \mathcal{F}(s \mid a)$ and $\nu \geq 0$. Evaluate at $\phi \cdot r$, where A, A' share the denominator $N_{\phi \cdot r}(\mathcal{F}(s))$. By the swap identity ($\phi \cdot \mathcal{F}(s \mid a') = G$ and $\phi \cdot G = \mathcal{F}(s \mid a')$ since ϕ is an involution),

$$A'(\phi \cdot r) = \frac{N_{\phi \cdot r}(\mathcal{F}(s \mid a'))}{N_{\phi \cdot r}(\mathcal{F}(s))} = \frac{N_r(G)}{N_{\phi \cdot r}(\mathcal{F}(s))},$$
$$A(\phi \cdot r) \geq \frac{N_{\phi \cdot r}(G)}{N_{\phi \cdot r}(\mathcal{F}(s))} = \frac{N_r(\mathcal{F}(s \mid a'))}{N_{\phi \cdot r}(\mathcal{F}(s))},$$

the inequality because $G \subseteq \mathcal{F}(s \mid a)$. As $N_r(\mathcal{F}(s \mid a')) > N_r(G)$, we get $A(\phi \cdot r) > A'(\phi \cdot r)$.

Pairing. Thus ϕ maps $\{A' > A\}$ into the disjoint set $\{A > A'\}$. As ϕ is an involution with $\phi_* \mathcal{D} = \mathcal{D}$, the map $r \mapsto \phi \cdot r$ is a $\mathcal{D}$-preserving bijection, so $\mathcal{D}\{A' > A\} = \mathcal{D}(\phi \cdot \{A' > A\}) \leq \mathcal{D}\{A > A'\}$. These are disjoint, so $\mathcal{D}\{A' > A\} \leq \frac{1}{2}$, and $\mathbb{P}_{\mathcal{D}}[A \geq A'] = 1 - \mathcal{D}\{A' > A\} \geq \frac{1}{2}$.

Scalable Oversight and Debate

Stephan Wäldchen (Iliad)

Main text: Chapter E.1, p. 427

4.4 Theorem: Q and N have the same cardinality: Hint p. 430

A bijection of sets preserves cardinality, but which additional structures does it preserve automatically?

4.5 Hilbert Spaces: Hint p. 431

Look very carefully at the step where the quadratic form

$$q(x) = \langle Tx, x \rangle$$

is asserted to attain its maximum on the unit sphere. What topology is being used, and is q actually continuous for that topology?

Steganography & Backdoors

Stephan Wäldchen (Independent), Louis Jaburi (EleutherAI)

Main text: Chapter E.2, p. 459

Exercise 15.1: Hint p. 470

When $g \sim \mathcal{N}(0, \Sigma)$, then $\mathrm{Var}(\langle g, x\rangle) = x^\top \Sigma x$

Exercise 15.1: Solution p. 471

(a) Since g is Gaussian and $z = \langle g, x\rangle$, the random variable z is Gaussian. Its mean is 0 , and

$$\mathrm{Var}(z) = x^\top (I_d + \theta s s^\top) x = \|x\|_2^2 + \theta \langle x, s\rangle^2 = 1.$$

Hence

$$z \sim \mathcal{N}(0, 1).$$

Similarly, $z' = \langle g, x + \alpha s\rangle$ is centered Gaussian with variance

$$\begin{aligned}\mathrm{Var}(z') &= \left(\sqrt{1-\alpha^2}x + \alpha s\right)^\top (I_d + \theta s s^\top) \left(\sqrt{1-\alpha^2}x + \alpha s\right) \\ &= \|\sqrt{1-\alpha^2}x + \alpha s\|_2^2 + \theta \left\langle s, \sqrt{1-\alpha^2}x + \alpha s\right\rangle^2.\end{aligned}$$

Using $\|x\|_2 = \|s\|_2 = 1$ and $\langle x, s\rangle = 0$, we get

$$\|x + \alpha s\|_2^2 = 1, \qquad \left\langle s, \sqrt{1-\alpha^2}x + \alpha s\right\rangle = \alpha.$$

Therefore

$$\mathrm{Var}(z') = 1 + \theta\alpha^2,$$

so

$$z' \sim \mathcal{N}\big(0, 1 + \alpha^2\theta\big).$$

(b) We compute

$$\mathbb{E}[\mathrm{ReLU}(Z)] = \int_0^\infty z \cdot \frac{1}{\sqrt{2\pi}\sigma} e^{-z^2/(2\sigma^2)}\, dz.$$

With the substitution

$$u = \frac{z^2}{2\sigma^2}, \qquad z\,dz = \sigma^2\,du,$$

this becomes

$$\mathbb{E}[\mathrm{ReLU}(Z)] = \frac{1}{\sqrt{2\pi}\sigma}\int_0^\infty \sigma^2 e^{-u}\,du = \frac{\sigma}{\sqrt{2\pi}}.$$

Thus

$$\mathbb{E}[\mathrm{ReLU}(z)] = \frac{1}{\sqrt{2\pi}},$$

while

$$\mathbb{E}[\mathrm{ReLU}(z')] = \frac{\sqrt{1+\alpha^2\theta}}{\sqrt{2\pi}}.$$

Since $\alpha > 0$ and $\theta > 0$, we have

$$\sqrt{1+\alpha^2\theta} > 1,$$

so the backdoored sample has larger expected ReLU activation.

(c) Since

$$\mathrm{Var}(\mathrm{ReLU}(Z)) \le \mathbb{E}[\mathrm{ReLU}(Z)^2],$$

and

$$\mathrm{ReLU}(Z)^2 \le Z^2,$$

we get

$$\mathrm{Var}(\mathrm{ReLU}(Z)) \le \mathbb{E}[Z^2] = \sigma^2.$$

For

$$A_n := \frac{1}{n}\sum_{i=1}^n \mathrm{ReLU}(Z_i),$$

we have

$$\mathbb{E}[A_n] = \frac{1}{n}\sum_{i=1}^n \mathbb{E}[\mathrm{ReLU}(Z_i)] = \frac{\sigma}{\sqrt{2\pi}}.$$

Since the Z_i are independent,

$$\begin{aligned}\mathrm{Var}(A_n) &= \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n}\mathrm{ReLU}(Z_i)\right)\\ &= \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(\mathrm{ReLU}(Z_i))\\ &\le \frac{1}{n^2}\cdot n\sigma^2 = \frac{\sigma^2}{n}.\end{aligned}$$

(d) If $\tau \le \mu_{\mathrm{reg}}$, then the desired inequality is immediate. Assume therefore that $\tau > \mu_{\mathrm{reg}}$. Then

$$\beta \le \Pr[A_n \ge \tau].$$

Since $\tau > \mu_{\mathrm{reg}}$,

$$\{A_n \ge \tau\} \subseteq \{|A_n - \mu_{\mathrm{reg}}| \ge \tau - \mu_{\mathrm{reg}}\}.$$

By Chebyshev's inequality,

$$\beta \le \Pr\left[|A_n - \mu_{\mathrm{reg}}| \ge \tau - \mu_{\mathrm{reg}}\right] \le \frac{\mathrm{Var}(A_n)}{(\tau - \mu_{\mathrm{reg}})^2}.$$

Using $\mathrm{Var}(A_n) \le 1/n$, we get

$$\beta \le \frac{1}{n(\tau - \mu_{\mathrm{reg}})^2}.$$

Rearranging gives

$$\tau - \mu_{\mathrm{reg}} \le \frac{1}{\sqrt{n\beta}},$$

and therefore

$$\tau \le \mu_{\mathrm{reg}} + \frac{1}{\sqrt{n\beta}}.$$

(e) From part (b),

$$\mu_{\mathrm{bd}} = \frac{\sqrt{1+\alpha^2\theta}}{\sqrt{2\pi}}, \qquad \mu_{\mathrm{reg}} = \frac{1}{\sqrt{2\pi}}.$$

Hence

$$\mu_{\mathrm{bd}} - \mu_{\mathrm{reg}} = \frac{\sqrt{1+\alpha^2\theta} - 1}{\sqrt{2\pi}} > 0.$$

Using part (d),

$$\tau \le \mu_{\mathrm{reg}} + \frac{1}{\sqrt{n\beta}}.$$

Therefore

$$\Delta_n = \mu_{\mathrm{bd}} - \tau \geq \mu_{\mathrm{bd}} - \mu_{\mathrm{reg}} - \frac{1}{\sqrt{n\beta}}.$$

For large enough n, the last term is smaller than $\mu_{\mathrm{bd}} - \mu_{\mathrm{reg}}$, so

$$\Delta_n > 0.$$

Now apply Chebyshev's inequality to A'_n. Since

$$\mathbb{E}[A'_n] = \mu_{\mathrm{bd}},$$

we have

$$\begin{aligned}\Pr[A'_n < \tau] &= \Pr[\mu_{\mathrm{bd}} - A'_n > \mu_{\mathrm{bd}} - \tau] \\ &\leq \Pr[|A'_n - \mu_{\mathrm{bd}}| \geq \Delta_n] \\ &\leq \frac{\mathrm{Var}(A'_n)}{\Delta_n^2}.\end{aligned}$$

By part (c), with

$$\sigma^2 = 1 + \alpha^2\theta,$$

we have

$$\mathrm{Var}(A'_n) \leq \frac{1 + \alpha^2\theta}{n}.$$

Thus

$$\Pr[A'_n < \tau] \leq \frac{1 + \alpha^2\theta}{n\Delta_n^2}.$$

If Δ_n is bounded below by a positive constant, then the right-hand side is $O(1/n)$. Hence the fraction of backdoored samples classified as $\mathtt{0}$ decays like $1/n$.